Learning from data for physicists

Contents

Overview

- 1. Invitation to inductive inference
- 2. Introduction

Part I: Bayesian methods for scientific modeling

- 3. Overview of Part I
- 4. Inference and PDFs
- 5. More on PDFs
- 6. Updating via Bayes' rule
- 7. Error propagation
- 8. Bayes in practice
- 9. Problems for Part I

Part II: Advanced Bayesian methods

- 10. Overview of Part II
- 11. Assigning probabilities
- 12. Dealing with outliers
- 13. Bayes goes linear — History matching
- 14. Model selection
- 15. Discrepancy Models with GPs
- 16. Model averaging and mixing
- 17. Problems for Part II

Part III: MCMC sampling

- 18. Overview of Part III
- 19. Intuition for MCMC
- 20. Details of MCMC
- 21. MCMC in practice
- 22. Advanced sampling algorithms
- 23. State-of-the-art sampling implementations

Part IV: Machine learning basics

- 24. Overview of Part IV
- 25. Machine learning overview
- 26. Logistic Regression
- 27. Artificial neural networks
- 28. *Convolutional neural networks
- 29. Problems for Part IV

Part V: Probabilistic machine learning and emulation

- 30. Overview of Part V
- 31. Bayesian neural networks
- 32. Gaussian processes
- 33. ANNs in the large-width limit (ANNFT)
- 34. Bayesian Optimization
- 35. Dimensionality reduction and emulators
- 36. Problems for Part V

Backmatter

- 37. Bibliography

Appendix A: Statistics

- 38. Overview of Appendix A and notation
- 39. The probability measure
- 40. Working with probability distributions

Appendix B: Scientific modeling

- 41. Overview of Appendix B
- 42. Overview of modeling
- 43. Linear models
- 44. Mathematical optimization

Appendix C: Getting started

- 45. Overview of Appendix C
- 46. Setting up for interactive use of this Jupyter book
- 47. Jupyter notebooks and Python
- 48. Guides on Jupyter notebooks and Python

TALENT mini-projects

- Overview of mini-projects
- MP I: Parameter estimation for a toy model of an EFT
- MP IIIa: Bayesian optimization
- MP IIIb: Bayesian Neural Networks

Bayesian and machine learning methods

by Christian Forssén, Dick Furnstahl, and Daniel Phillips.

> “We demand rigidly defined areas of doubt and uncertainty!”

—Douglas Adams, *The Hitchhiker’s Guide to the Galaxy*

About this book

This text is aimed at physicists seeking to learn from data using Bayesian methods. It is particularly designed for use in an advanced-level course but should be broadly accessible to those with a physics background.

The content of this book has emerged from previous work and courses taught by the authors. The materials are released under a Creative Commons BY-NC license.

The book format is powered by Jupyter Book. You can also download the complete book as a pdf file.

This book is work in progress

New content is added and existing sections are revised on an ongoing basis. We welcome feedback, suggestions, and bug reports. Please feel free to open an issue on our GitHub repository (see also the *Open an issue* box further down) or contact us by email.

Materials to help you get started

Appendix C has various materials to set you up for Learning from Data. You may have the option to run the Jupyter notebooks on a cloud server, but eventually you will likely want to set up an environment on your own machine.

- See Setting up for interactive use of this Jupyter Book for instructions how to use git for cloning the repository with all the book material (Using git for cloning the book repository) and for guides to setting up a Python environment on your computer (Setting up your Python environment).
- If Python and/or Jupyter notebooks are new to you (or if you need a refresher), you can find a notebook: 📥 Exercise: Jupyter Notebooks and Python that will take you from zero to just what you need to know about Python and running the Jupyter notebooks in this Jupyter Book.
- For more advanced Python summaries, see the other Python notebooks linked in Appendix C.

Brief guide to online Jupyter Book features

Icons and menus

The Jupyter Book has many useful features:

- A clickable high-level table of contents (TOC) is available in the panel at the left of each page. (You can toggle this panel open or close with the contents icon at the upper left of the middle panel.) At the top of this TOC is a search box for the book (put your search words in quotes).
- The icons at the top-right in the middle panel can be used to take you to the source repository for the book; download the source code for the page (in different formats); view the page in full-screen mode; or switch between light and dark mode.
- For each section that has subsections, a clickable table of contents appears in the rightmost panel.
- There are hyperlinks throughout the text to other sections, equations, references, and more.

Open an issue

If you find a problem or have a suggestion–whether related to physics, statistics, Python, or formatting–we encourage you to let us know via a GitHub issue. You will need a GitHub account to do this (free to create).

> From any page in the book, click the GitHub icon in the top right toolbar and select Open issue. This will take you to the Issues section of the book's GitHub repository, where you can give your issue a title and describe the problem or suggestion in the text box below.

Acknowledgments

The material in this book has evolved over several years. The genesis was an intensive three-week summer school course taught at the University of York in 2019 by the authors as part of the TALENT initiative. New material was subsequently added by Christian Forssén for course developments at Chalmers and for lecture series at other universities. Contributions from Andreas Ekström are particulaly acknowledged. In parallel, Dick Furnstahl adapted the TALENT material (plus much of Forssén's new material) for a graduate course at The Ohio State University, which was then revised by Daniel Phillips for a course at Ohio University. This book is a merger and update of all these developments. It includes significant additions of new material. A previous version of this book is archived at a legacy site.

Both the original notes and subsequent revisions have been informed by interactions with many colleagues, who have taught us different aspects of Bayesian inference. This includes many statistician colleagues who have taken the time to carefully and patiently address our misconceptions regarding statistics, probability, or even basic mathematics. Many of the important interactions have occurred at meetings in the ISNET series. Among our physics and statistics colleagues we are particularly grateful to:

- Andreas Ekström, Chalmers University of Technology
- Morten Hjorth-Jensen, Oslo University and Michigan State University
- Jordan Melendez, Ohio State University and Root Insurance
- Matt Plumlee, Northwestern University and Amazon
- Matt Pratola, Indiana University
- Ian Vernon, Durham University
- Sarah Wesolowski, York, UK
- Frederi Viens, Michigan State University

Many of the advanced Bayesian methods that might be included in these notes have been used in scientific studies with different collaborators. In particular, several postdocs, PhD students and master students have had leading roles in the development and application of these methods to address various scientific questions. Christian Forssén would like to highlight the contributions (in alphabetical order) of: Boris Carlsson, Tor Djärv, Weiguang Jiang, Eleanor May, Isak Svensson, and Oliver Thim.

The full list of people that have contributed with ideas, discussions, or by generously sharing their knowledge is very long. Rather than inadvertently omitting someone, we simply say thank you to all. More generally, we are truly thankful for being part of an academic environment in which ideas and efforts are shared rather than kept isolated. The last statement extends to the open-source communities through which great computing tools are made publicly available. In this course we take great advantage of open-source python libraries.

The development of this course would not have been possible without the knowledge gained through the study of several excellent textbooks, most of which are listed as recommended course literature. Here is a short list of those references that we have found particularly useful as physicists learning Bayesian statistics and the fundamentals of machine learning:

[GCS+13] Andrew Gelman et al., *"Bayesian Data Analysis, Third Edition"*, Chapman & Hall/CRC Texts in Statistical Science (2013).

[Gre05] Phil Gregory, *"Bayesian Logical Data Analysis for the Physical Sciences"*, Cambridge University Press (2005).
[Jay03] E. T. Jaynes, *"Probability Theory: The Logic of Science"*, Cambridge University Press (2003).
[Mac03] David J.C. MacKay, *"Information Theory, Inference, and Learning Algorithms"*, Cambridge University Press (2005).
[SS06] D.S. Sivia with J. Skilling, *"Data Analysis : A Bayesian Tutorial"*, Oxford University Press (2006).

Needless to say, although we have benefited tremendously from all this input, any errors that remain in this book are solely our responsibility.

Lastly we thank our families for their support and for putting up with us occasionally lapsing into Bayes speak in everyday life.

1. Invitation to inductive inference

1.1. Demolition derbies, cows, and crocodiles

This is a book about how to glean scientific results from data. The collection of data is often a heroic task: physicists build huge accelerators, launch telescopes into space, and spend years monitoring bodies of matter underground to detect sub-atomic messengers from within our galaxy and beyond. But without *inference* all that data is just a list of numbers.

Inference is "the act or process of reaching a conclusion about something from known facts or evidence." Theoretical physicists have traditionally trained themselves to become experts at deductive inference. They posit a theory of nature, or perhaps, less ambitiously, some phenomenon within nature, and deduce the effects that will follow from that cause. Mathematical proofs are the Platonic ideal of deductive reasoning. And, we often teach our students that the practice of science lies in deducing predictions from theories and then doing experiments that falsify those predictions.

With apologies to Karl Popper, this seems a deeply unsatisfying way to practice science. After all the falsifying is done, what has one established? Is this a scientific version of a demolition derby, wherein the theory least beaten up by all those comparisons with data is declared the bloody but unbowed winner?

Therefore this book focuses on inductive inference. And specifically inductive inference to the best explanation. Here best may involve determining the parameters of a particular model, or choosing between different competing models, or combining the predictions of these models. But instead of the deductive inference of the mathematicians–lovely though that may be–here we are in the realm of what Polya calls "plausible reasoning". Instead of an ironclad syllogism: "All cows eat grass, Betty is a cow, therefore Betty eats grass", we deal in logic that takes us from data to a model of a reality: "Betty eats grass, therefore it is more likely that Betty is a cow than that she is a crocodile".

But how much more likely? Our physicist training cries out for us to quantify the strength of this bovine inference. And, to update the quantification if we acquire more data, for instance, that Betty also says "Moo".

Bayes' theorem provides us with a mathematical identity (deductively inferred!) that tells us how to do the latter: how to update our inferences as more data is acquired. But to do this we must couch our inference of Betty's species identity in the language of random variables and probability. You might think this a bug, but we will argue throughout this book that it's a feature, because it means that our inferences are then subject to the laws of

probability. And these too are mathematical rules that tell you quantitatively how additional assumptions you may make along your path from data to inference affect your final conclusion.

The basic process of inference is employed also in the field of machine learning. Here, the learning part might take place when confronting a large set of data with a machine learning algorithm, and the specific aim might be tasks such as classification or clustering or emulation. We aim in this book to bring a particularly Bayesian perspective to our consideration of machine learning.

Fig. 1.1 Machine learning can also be seen as an inference process.

On the one hand, we will study statistical inference methods for learning from data as needed for scientific applications. Simultaneously this statistical foundation will provide a deeper understanding (and probabilistic interpretation) of machine learning algorithms. This will enable **statistical learning**.

1.2. Polya and Jaynes

Polya and plausible inference

George Polya was a leading mathematician in the mid-twentieth century. In his two-volume book on "Mathematics and Plausible Reasoning" from 1954 [Pol54a, Pol54b], Polya wrote:

> "A mathematical proof is demonstrative reasoning but the inductive evidence of the physicist, the circumstantial evidence of the lawyer, the documentary evidence of the historian and the statistical evidence of the economist all belong to plausible reasoning."

Polya contrasts a standard deductive inference involving statements *A* and *B* (an Aristotelian syllogism):

If A implies B, and A is true, then B is true

with an inductive inference:

If A implies B, and B is true, then A is more credible.

He considers qualitative patterns of plausible inference as a guide toward mathematical discovery; he wrote: *"Mathematics presented with rigor is a systematic deductive science but mathematics in the making is an experimental inductive science"*.

However, as physicists we seek *quantitative* inferences.

In particular, physicists need ways to:

1. quantify the strength of inductive inferences;
2. update that quantification as new data is acquired.

This is what Bayesian inference will do for us!

Edwin Jaynes and plausible reasoning

Edwin Jaynes, in his influential How does the brain do plausible reasoning? [Jay88], wrote:

> "One of the most familiar facts of our experience is this: that there is such a thing as common sense, which enables us to do plausible reasoning in a fairly consistent way. People who have the same background of experience and the same amount of information about a proposition come to pretty much the same conclusions as to its plausibility. No jury has ever reached a verdict on the basis of pure deductive reasoning. Therefore the human brain must contain some fairly definite mechanism for plausible reasoning, undoubtedly much more complex than that required for deductive reasoning. But in order for this to be possible, there must exist consistent rules for carrying out plausible reasoning, in terms of operations so definite that they can be programmed on the computing machine which is the human brain."

Jaynes went on to show that these "consistent rules" are just the axiomatic rules of Bayesian probability theory supplemented by Laplace's principle of indifference and, its generalization, Shannon's principle of maximum entropy. This key observation implies that we can define an *extended logic* such that a computer can be programmed to "reason", or rather to update probabilities based on data. Given some very minimal desiderata, the rules of Bayesian probability are the only ones which conform to what, intuitively, we recognize as rationality. Such probability update rules can be used recursively to impute causal relationships between observations. That is, a machine can be programmed to "learn".

2. Introduction

In this chapter, we first emphasize that we will address learning from data from a physicist's perspective, which distinguishes it from most data science text. This leads to our first encounter (with many more to follow) of the Bayesian workflow in the physics context, and then to considering machine learning from the physics perspective. Finally, we articulate the virtues we believe are essential for principled Bayesian inference and, more generally, for effective physics research.

2.1. Physicist's perspective

This is a book by physicists, for physicists. It is not a data science book *per se*. It is, instead, a book that we hope will help physicists learn about a set of tools and practices that will enable them to draw rigorous scientific conclusions from data.

Note

In the spirit of Lewis Carroll's *Through the Looking Glass* ("'When I use a word," Humpty Dumpty said in rather a scornful tone, "it means just what I choose it to mean—neither more nor less."') we take opportunities like this one to specify how we will use various terms throughout this book. When we say "Data" it means any information used to form a conditional probability that is part of our inference. Data could be the quantitative output of experimental investigations–the most common understanding of the term. But, it could also be data from astronomical observations. Or it could be the results of computational simulations of theories, or even order-by-order theory predictions obtained in a perturbative expansion. Indeed, a key aspect of this book is that we will often understand our theories (or models) as providing us with data, and we may well not distinguish in our writing whether it is that kind of data we are discussing or "actual" data. The Bayesian methods we present don't care about this distinction: all data will be grist for our inference mill.

In order to draw these conclusions the act of scientific modeling is essential. But most of us are not interested in drawing a model-specific conclusion. Our goal is more ambitious: while inference will often be done in the context of a specific model, we will drive towards results and predictions whose validity is not restricted by the limitations of the model itself.

We want these conclusions to be rigorous, but this is not synonym for certain. Conclusions from data are, by and large, contingent and revisable. But, we will argue that, if couched in the language of probability theory, the inference from data can and should be done rigorously. This can only happen if that inference is implemented within a consistent theory of probabilistic reasoning.

In order to assign a probability to some aspect of a model (a parameter being within a specified range, that the model itself is true, etc.) we need to specify the information on which we are basing this probability. Thus our understanding of models needs not only to be expressed using a probability calculus, the probabilities must always be understood as contingent upon the specified information.

Bayes' theorem is an immediate consequence of consistent probabilistic reasoning and it expresses how the probabilities of things we care about in a model must change in the light of additional information. Bayes' theorem is therefore at the heart of this book because it is a mathematical identity regarding consistent learning from new data. It tells us how to take the probabilities that encode our knowledge regarding the model prior to acquisition of that data and update them in light of that new information. Bayes' theorem and the rules of probability calculus are introduced in Part I, Section 4 and applied throughout this text.

2.2. Bayesian workflow

We hope anyone, or at least anyone reading this book, wants to learn. As taught in this book the practice of learning from data has four steps, which we will revisit repeatedly:

1. Define the probabilistic relationship between the physical model and the data. No experiment is perfect, and all models are wrong at some level, so both the "data generation process" and "theory defects" need to be accounted for as stochastic variables when we define the relationship between the physics model we want to learn and the data we are using to learn about it. We will call that relationship the "statistical model". Without a statistical model the chain of rigorous inference is broken. But, once a statistical model is specified, that information, combined with the probability calculus we'll introduce in Part I, defines our updated knowledge of the model.
2. Formulate "priors" that express our knowledge about a model before conditioning on new data. Some physicists may argue that we should "let the data speak for itself", with the implication that any informative prior is biasing the result. To the contrary, not using appropriately informative priors risks biases and fails to use all the information.
3. Compute the updated (or "posterior") probabilities that represent our knowledge about the model, including the parameters in it. This, in general, requires a set of computational tools that sample the different possibilities for those probabilities: "samplers". These are demonstrated in detail in Part III.
4. Check the answers obtained after they have been computed to see if the statistical model we wrote down in step 1 is consistent with both what we now know (in the probabilistic sense) about the physics model and the data we are analyzing. This step is called "model checking". You will be exposed to best practices for model checking as well as specific tools to carry it out.

These four steps are the way: the "Bayesian workflow". It is the practice by which principled learning from data can be carried out, and the results of that learning validated. Although presented linearly here, in detail there are internal

and overall cycles where assumptions and implications are examined and steps are revisited, as discussed in more detail in Bayesian research workflow.

Four-step Bayesian workflow in brief

1. Define a statistical model relating the physics model and data, including all errors and correlations.
2. Formulate priors.
3. Compute the posterior probabilities.
4. Do model checking.

We first introduce the Bayesian tools and practices in Part I in the context of parameter estimation, i.e., we explain how they let us use data to refine our knowledge of model parameters and demonstrate with applications that the reader is encouraged to extend. But then, in Part II, we broaden our perspective and ask if we can do inference not just in the parameter space of a particular model, but in the space of all possible models. This leads us to a discussion of Bayesian multi-model inference, see Model Selection and Model averaging and mixing. We also explore how to deal statistically with flaws in our theoretical models, see Discrepancy models with GPs.

One of the chief virtues of the Bayesian approach is that it should force the practitioner to be explicit and clear about what the ingredients that go into the inference. Step 1 mandates that a statistical model relates the data to the model. Step 2 requires that they specify the prior. And Step 4 asks them to check that their process is at least self-consistent.

2.3. Machine learning

The Bayesian workflow sets us up to talk about Machine Learning (ML) from a Bayesian perspective. Indeed there are corresponding steps in the typical ML workflow, although they might not be immediately recognized as such (e.g., regularization in ML is a type of prior that imposes the knowledge that weights should not be overly large when trained). The use of ML methods such as Gaussian processes are already naturally formulated in a Bayesian statistical setting, with priors, posteriors, and probability distributions. But we can also cast neural networks in a probabilistic framework. Furthermore, there are special features about ML for physics, summarized below, which argue for a less “black box” approach to ML than is common. All of these aspects motivate an integrated discussion of ML within this text, which we carry out in Part IV.

What is special about machine learning in physics?

Physics research takes place within a special context:

- Physics data and models are connected with physical processes and are often fundamentally different from those encountered in typical computer science contexts.
- Physicists ask different types of questions about their data, sometimes requiring new methods.
- Physicists have different priorities for judging the quality of a model: interpretability, error estimates, predictive power, etc.

Providing slightly more detail:

- Physicists are data **producers**, not (only) data consumers:
  - Experiments can (sometimes) be designed according to needs.
  - Statistical errors on experimental data can be quantified.
  - Much effort is spent to understand systematic errors.
- Physics data represents measurements of physical processes:

    - Dimensions and units are important.
    - Measurements often reduce to counting photons, etc, with known a-priori random errors.
    - In some experiments and scientific domains, the data sets are *huge* (“Big Data”)
- Physics models are usually traceable to an underlying physical theory:
    - Models might be constrained by theory and previous observations.
    - There might exist prior knowledge about underlying physics that should be taken into account.
    - Parameter values are often intrinsically interesting.
    - The error estimate of a prediction is just as important as its value.

2.4. Virtues

In carrying out a principled Bayesian approach to learning from data, there are virtues that we wish to uphold. A good list of these is reproduced here from the paper by Gelman and Hennig. These are broadly relevant for the practice of science and not only Bayesian inference.

Aspirational virtues for Bayesian inference and beyond

1. Transparency
    - Clear and unambiguous definitions of concepts
    - Open planning and following agreed protocols
    - Full communication of reasoning, procedures, spelling out of (potentially unverifiable) assumptions and potential limitations
2. Consensus
    - Accounting for relevant knowledge and existing related work
    - Following generally accepted rules where possible and reasonable
    - Provision of rationales for consensus and unification
3. Impartiality
    - Thorough consideration of relevant and potentially competing theories and points of view
    - Thorough consideration and if possible removal of potential biases: factors that may jeopardize consensus and the intended interpretation of results
    - Openness to criticism and exchange
4. Correspondence to observable reality
    - Clear connection of concepts and models to observables
    - Clear conditions for reproduction, testing and falsification
5. Awareness of multiple perspectives
6. Awareness of context dependence
    - Recognition of dependence on specific contexts and aims
    - Honest acknowledgement of the researcher’s position, goals, experiences and subjective point of view
7. Investigation of stability
    - Consequences of alternative decisions and assumptions that could have been made in the analysis
    - Variability and reproducibility of conclusions on new data

George Pólya was a Hungarian-American mathematician who, in addition to many contributions to pure mathematics, developed a framework for problem solving that incorporated many principles of plausible inference that we build into our Bayesian methods. He also articulated three moral qualities essential for the robust practice of scientific discovery in general, which we have adapted here with the hope they are shared by all of our readers.

Moral qualities of the scientist

Adapted from G. Pólya, Induction and Analogy in Mathematics, chapter 1, section 4, which is entitled “The Inductive Attitude”.

Intellectual courage
One should be ready to revise any one of our beliefs. **Intellectual honesty**
One should change a belief when there is a compelling reason to do so. To stick to a conjecture clearly contradicted by experience is dishonest. It is also dishonest to ignore information or not to state and criticize all assumptions.
Wise restraint
One should not change a belief wantonly, without some good reason. Don’t just follow fashion. Do not believe anything, but question only what is worth questioning.

3. Overview of Part I: Bayesian methods for scientific modeling

In Part I we introduce the basics of Bayesian statistics. We’ve divided the material into chapters on:

- Inference and PDFs, with an introduction to Bayes’ theorem and other ingredients of Bayesian statistical analysis, including our first look at parameter estimation;
- More on PDFs, with further details and explorations of probability distributions, including probability density functions (PDFs);
- Updating via Bayes’ rule, with interactive guides to the updating of PDFs when additional knowledge is acquired;
- Error propagation, with a detailed look at three aspects of error propagation: nuisance parameters and marginalization, changing variables, and Gaussian approximations to propagated errors.
- Bayes in practice, with a summary of the advantages of the Bayesian framework, elaboration of the Bayesian workflow, and a detailed example on Bayesian linear regression that puts together all of the ingredients.
- Problems for Part I, which includes exercises to build intuition or gain practice in the basics of Bayesian inference, starting from the rules of plausible inference, focusing particularly on parameter estimation, and building on those examples to provide a first exposure to Monte Carlo sampling.

Good supplementary references for this material that are particularly physicist-friendly are Chapters 1 and 2 in Sivia [SS06]; Chapter 1 from Gregory [Gre05]; and the article by Trotta [Tro08].

Many of the examples in Sivia have been implemented in Jupyter notebooks that you should work through as we proceed. You are highly encouraged to answer the questions in the notebooks, play with different values for the parameters, and extend the examples.

Additional material on scientific modeling can be found in Appendix A and Appendix B.

4. Inference and PDFs

In this chapter we put on the table the basic ingredients for Bayesian inference. The subsections are:

- Statements: Notation for probability statements
- Manipulating probabilities: Bayesian rules of probability as principles of logic

- Probability density functions. Extension of the Bayesian rules of probability to continuous probability density functions.
- First look at Expectation values and moments.
- Data, models, and predictions. Introduction to statistical models, posterior predictive distributions, and Bayesian parameter estimation.
- *Aside: Bayesian epistemology. Philosophical remarks on the interpretation of probabilities.

4.1. Statements

A Bayesian approach to the world assigns probabilities to truth claims. It also recognizes that the probability of claim A is contingent upon statement B being true. We therefore introduce notation for conditional probability:

$$\mathbb{P}\left(A \mid B\right) \equiv \text{“probability of } \quad A \text{ given } \quad B \text{ is true”} \quad .$$

For a Bayesian A and B could stand for anything. But, whatever they are, statements can now be categorized as definitively true (probability 1), or definitively false (probability 0), or anything in between.

Examples:

- $\mathbb{P}\left(\text{“Betty eats grass”} \quad \middle| \text{“Betty is a cow” and “All cows eat grass”} \quad \right) = 1$
- $\mathbb{P}\left(\text{“Lines A and B intersect”} \quad \middle| \text{“A and B are parallel lines in flat space”} \quad \right) = 0$
- $\mathbb{P}\left(\text{“Particle A is negatively charged”} \quad \middle| \text{“A is an electron”} \quad \right) = 1$

On the other hand something like $\mathbb{P}\left(\text{“below 0 deg,. C”} \quad \middle| \text{“it is January in Ohio”} \quad \right)$ should be assigned a probability between 0 and 1. We might even estimate this probability based on our past experience of Januarys in Ohio. But even statements for which we cannot construct a “frequentist estimator” can be assigned probabilities, e.g.

$$\mathbb{P}\left(\text{“String theory is true”} \quad \middle| \text{“Data from all high-energy colliders and cosmology”} \quad \right),$$

or, less frivolously

$$\mathbb{P}\left(\text{“I'll come to the party tonight”} \quad \middle| \text{“You invited me”} \quad \right).$$

This emphasizes that, for a Bayesian, the probability is interpreted as representative of the Bayesian's current state of knowledge, and not necessarily as the long-term average of a set of many trials. After all, what would it mean to conduct a set of trials, in some of which string theory was true, and in some of which it was not?

Warning

It is easy to forget the meaning of the key word *given* and assume that $\mathbb{P}\left(A \mid B\right)$ means that we know B is actually true, which may or may not be the case.

Note

In this book we employ a Bayesian interpretation of probability throughout. However, we sometimes contrast our analysis of a problem to an analysis of the same problem using a frequentist interpretation of probability. And

even within a Bayesian interpretation there are several different interpretations. One of the big divisions is between the school of thought which says that probabilities should be objective, and the "subjective school of Bayesian probability". In the subjective school of probability everyone has their own state of knowledge so everyone gets to assign their own pdfs to quantities of interest. More discussion of these different interpretations of probability can be found in *Aside: Bayesian epistemology.

4.2. Manipulating probabilities: Bayesian rules of probability as principles of logic

There can be many different ways by which we attain our current state of knowledge. What we know about a particular parameter may be based on bounds on what is physically sensible (e.g., masses are positive), or on the results of a previous experiment (e.g., there have already been several measurements of Newton's gravitational constant), or we might know something based on the overall scales in the problem (e.g., baseball speeds are typically not larger than 150 mph). In a Bayesian formulation any of these pieces of knowledge can be represented as a probability. Regardless of how we arrived at our state of knowledge it is crucial that we know how to combine it with other knowledge. For that purpose we need rules that allow us to keep track of probabilities.

A minimal set of rules for consistently keeping track of probabilities is provided by two basic rules of probability arithmetic. These are the sum and product rules. These rules can be derived from Kolmogorov's axioms of probability, see The probability measure, which showed that probabilities can be incorporated into mathematics using the existing theory of measures. More generally, a proof that the Sum and Product rules follow in any consistent implementation of probabilistic reasoning is given by Cox [Cox61].

Sum rule

If the set $\{x_i\}$ is *exhaustive* and *exclusive*

$$\sum_i \mathbb{P}\left(x_i \mid I\right) = 1, \tag{4.1}$$

i.e., the sum of the probabilities of all possible outcomes is equal to one. In quantum mechanics we are used to this as the outcome of summing over a complete, orthonormal set of states, and indeed, in that case the basis includes all possible values (it's complete), and there is no overlap between its members (they are orthogonal).

Note

In (4.1) we include I (for "Information") generically as the quantities or statements that the probability of x_i is contingent on. We use I to avoid having to specify explicitly all the details, but we should remember that these probabilities (and probability densities introduced below) are always conditional on some information.

Product rule

The product rule tells us how to expand the joint probability of x and y, i.e.,

$$\mathbb{P}\left(x, y \mid I\right) = \mathbb{P}\left(x \mid y, I\right)\mathbb{P}\left(y \mid I\right) \tag{4.2}$$

In words we say that the probability of both x and y occurring is the probability that x occurs, given that y has occurred, times the probability that y occurs.

Note, once again, that the given information I is held fixed, i.e., it if present on the left-hand side it appears in both probabilities on the right-hand side.

If x and y are *mutually independent*, then $\mathbb{P}(x \mid y, I) = \mathbb{P}(x \mid I)$ and (4.2) reduces to

$$\mathbb{P}(x, y \mid I) = \mathbb{P}(x \mid I)\mathbb{P}(y \mid I) \tag{4.3}$$

This is a rule you are probably (but we hesitate to quantify our belief) familiar with. Crucially, (4.2) does *not* rely on the independence of the events x and y.

Note

When considering whether x and y are mutually independent, you can ask yourself: does knowing y is true give me any information about whether x is true? If yes, then we need to keep it on the right side of the bar in $\mathbb{P}(x \mid y, I)$. If no, then it doesn't change the probability of x whether it is there or not, so the probability of x being true is the same if y is omitted. Hence $\mathbb{P}(x \mid y, I) = \mathbb{P}(x \mid I)$ follows.

Bayes' theorem

It is just a short step from the product rule to Bayes' theorem. Although we wrote (4.2) so that the $\mathbb{P}(x \mid y, I)$ appeared on the right-hand side there is no reason to privilege x over y. We could equally have written:

$$\mathbb{P}(x, y \mid I) = \mathbb{P}(y \mid x, I)\mathbb{P}(x \mid I)$$

Equating this to the expression in (4.2) yields **Bayes' Rule** (or Theorem):

$$\mathbb{P}(x \mid y, I) = \frac{\mathbb{P}(y \mid x, I)\mathbb{P}(x \mid I)}{\mathbb{P}(y \mid I)} \tag{4.4}$$

Bayes' theorem tells us how to reverse the conditional: $\mathbb{P}(y \mid x, I) \Rightarrow \mathbb{P}(x \mid y, I)$. The first thing to realize is that these two probabilities are not the same thing.

Checkpoint question

Construct your own example of $\mathbb{P}(x \mid y, I) \neq \mathbb{P}(y \mid x, I)$

Possible answers

The probability that there is a cloud in the sky given that it is raining is not the same as the probability that it's raining given that there is a cloud in the sky.

Ingredients of Bayes' theorem

A typical use of Bayes' theorem (4.4) is to update information on a hypothesis x according to new data y that we obtain. In that case the various terms in the theorem have formal names.

- The quantity on the far right, $\mathbb{P}\left(x \mid I\right)$, is called the *prior* probability; it represents our state of knowledge (or ignorance) about the truth of the hypothesis x before we have analysed the new data y.
- This is modified by additional information on x through $\mathbb{P}\left(y \mid x, I\right)$, the *likelihood* function, that tells us how likely it is that we measure y, given that x is true (and I is true as well).
- The denominator $\mathbb{P}\left(y \mid I\right)$ is called the *evidence*. It does not depend on the hypothesis and can be regarded as a normalization constant in many situations.
- Together, these yield the *posterior* probability, $\mathbb{P}\left(x \mid y, I\right)$, representing our updated state of knowledge about the hypothesis x in light of the information we had before, I, and the additional data y.

In this sense, Bayes' theorem is a mathematically rigorous statement of how probabilities should be updated in light of new information: the process of learning from data.

The friends of Bayes' theorem

Normalization and marginalization

Given an exclusive and exhaustive list of hypotheses, y_j, we must have a normalization of the total probability

$$\sum_j \mathbb{P}\left(y_j \mid I\right) = 1, \tag{4.5}$$

which also leads to the marginalization property

$$\mathbb{P}\left(x \mid I\right) = \sum_j \mathbb{P}\left(y_j \mid x, I\right)\mathbb{P}\left(x \mid I\right) = \sum_j \mathbb{P}\left(x, y_j \mid I\right) \tag{4.6}$$

where we used the product rule in the second step.

This is a key tool in the Bayesian's arsenal: the ability to obtain the "marginal" probability for the outcome of one variable by summing the joint probability for x and y over all possible outcomes $\{y_j\}$ of the second variable. This second variable is not one we are interested in when computing the pdf $\mathbb{P}\left(x \mid I\right)$, so we "marginalize over it".

We will use marginalization a lot! Note that the marginalization takes place in the presence of the conditional I, i.e., all probabilities involved are "given the information I". The given information is held fixed, while the sum of all possibilities is constructed.

For example, let's imagine that there are five candidates in a presidential election; then H_1 could be the proposition that the first candidate will win, and so on. The probability that A is true, for example that unemployment will be lower in a year's time (given all relevant information I, but irrespective of whoever becomes president) is given by $\mathbb{P}(A|I) = \sum_i \mathbb{P}(A, H_i|I)$. The president is a nuisance variable who has been marginalized out of the calculation.

Warning

Although we alluded to the analogy between inserting a complete set of states in quantum mechanics and marginalization above this analogy breaks down in general. It's ok to use this as a mnemonic though.

Note

A rule from probability says $\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B)$. (That is, to calculate the probability of the union of A and B we need to subtract the probability of the intersection from the sum of probabilities.) This may seem to contradict our marginalization rule. However, if A and B are *exclusive* (as we assume) then $\mathbb{P}(A \cap B) = 0$.

Exercise 4.1 (Checking the sum and product rules)

Goal: Check using a very simple example that the Bayesian rules are consistent with standard probabilities based on frequencies. Also reinforce notation and vocabulary.

TABLE 1	Blue	Brown	Total
Tall	1	17	18
Short	37	20	57
Total	38	37	75

TABLE 2	Blue	Brown	Total
Tall			
Short			
Total			

Table 1 shows the number of blue- or brown-eyed and tall or short individuals in a population of 75.

Question 1

1(a) Fill in the blanks in Table 2 with probabilities (in decimals with three places, not fractions) based on the usual "frequentist" interpretations of probability* (which would say that the probability of randomly drawing an ace from a deck of cards is 4/52 = 1/13).

1(b) Put x's in any row and/or column that illustrates marginalization and y's for entries illustrating the sum rule.

Hint 1(a) How many students are tall and blue-eyed? Just 1. There are 75 total students, so the probability is $1/75 \approx 0.013$, which goes in the first box.

Hint 1(b) Marginalization is $\mathbb{P}(x \mid I) = \sum_j \mathbb{P}(x, y_j \mid I)$, where in this case one possibility is x is "Tall" while y_1 is "Blue" and y_2 is "Brown". So $0.240 \stackrel{?}{=} 0.013 + 0.227 \Longrightarrow$ works!

Question 2

2(a) What is $\mathbb{P}(short, blue)$? Is this a joint or conditional probability?

2(b) What is $\mathbb{P}(blue)$?

2(c) From the product rule, what is $\mathbb{P}(short|blue)$? Can you read this result directly from the table?

Question 3

Apply Bayes' theorem to find $\mathbb{P}(blue|short)$ from your answers to the last part.*

Question 4

What rule does the second row (the one starting with "Short") illustrate? Write it out in $\mathbb{P}(\cdot)$ notation.

Question 5

Are the probabilities of being tall and having brown eyes mutually independent? Why or why not?

Hint: If the probabilities of being tall and brown *were* independent, what would the joint probability be in terms of the individual probabilities?

Exercise 4.2 (Standard medical example using Bayes)

Goal: Use the Bayesian rules of probability to solve a familiar problem whose result can be non-intuitive.

Suppose there is an unknown disease (call it UD) and there is a test for it.

a. The false positive rate is 2.3%. ("False positive" means the test says you have UD, but you don't.)
b. The false negative rate is 1.4%. ("False negative" means you have UD, but the test says you don't.)

Assume that 1 in 10,000 people have the disease. You are given the test and get a positive result. Your ultimate goal is to find the probability that you actually have the disease. We'll do it using the Bayesian rules.

We'll use the notation:

- H = "you have UD"
- $\overline{H}$ = "you do not have UD"
- D = "you test positive for UD"
- $\overline{D}$ = "you test negative for UD"

Question 1 Before doing a calculation (or thinking too hard :), does your intuition tell you the probability you have the disease is high or low?

Question 2 In the $\mathbb{P}(\cdot|\cdot)$ notation, what is your ultimate goal?

Question 3 Express the false positive rate in $\mathbb{P}(\cdot|\cdot)$ notation. [Ask yourself first: what is to the left of the bar?]

Question 4 Express the false negative rate in $\mathbb{P}(\cdot|\cdot)$ notation. By applying the sum rule, what do you also know? (If you get stuck answering the question, do the next part first.)

Question 5 Should $\mathbb{P}(D|H) + \mathbb{P}(D|\overline{H}) = 1$? Should $\mathbb{P}(D|H) + \mathbb{P}(\overline{D}|H) = 1$? (Hint: does the sum rule apply on the left or right of the $|$?)

Question 6 Apply Bayes' theorem to your result for your ultimate goal (don't put in numbers yet). Why is this a useful thing to do here?

Question 7 Let's find the other results we need. What is $\mathbb{P}(H)$? What is $\mathbb{P}(\overline{H})$?

Question 8 Finally, we need $\mathbb{P}(D)$. Apply marginalization first, and then the product rule twice to get an expression for $\mathbb{P}(D)$ in terms of quantities we know.

Question 9 Now plug in numbers into Bayes' theorem and calculate the result. What do you get?

Exercise 4.2 illustrates how to avoid the Base Rate Fallacy.

Follow-up question on Exercise 4.2:2.

Why is it $\mathbb{P}(H|D)$ and not $\mathbb{P}(H, D)$?

Answer

Recall that $\mathbb{P}(H, D) = \mathbb{P}(H|D) \cdot \mathbb{P}(D)$. You are generally interested in $\mathbb{P}(H|D)$. If you know $\mathbb{P}(D) = 1$, then they are the same.

Follow-up question on Exercise 4.2:5.

The emphasis here is on the sum rule. Why didn't any column except Total in the sum/product rule notebook add to 1?

Answer

Because we were looking at $\mathbb{P}(\text{tall,blue}) + \mathbb{P}(\text{short,blue}) \neq 1$, whereas $\mathbb{P}(\text{tall}|\text{blue}) + \mathbb{P}(\text{short}|\text{blue}) = 1$.

In general, and for Exercise 4.2:6. in particular, we emphasize the usefulness of using Bayes' theorem to express $\mathbb{P}(H|D)$ in terms of $\mathbb{P}(D|H)$.

Solutions to exercises

Solution to Exercise 4.1 (Checking the sum and product rules)

Question 1

(a)

TABLE 2	Blue	Brown	Total
Tall	0.013	0.227	0.240
Short	0.493	0.267	0.760
Total	0.506	0.494	1.000

(b)

TABLE 2	Blue	Brown	Total
Tall	0.013	0.227	0.240 x
Short	0.493	0.267	0.760 x
Total	0.506 x	0.494 x	1.000 y

The third (last) column and the third (last) row each illustrate marginalization (they are totals in the margin, get it?), while the grand total entry illustrates the sum rule.

Question 2

(a) $\mathbb{P}(short, blue) = 0.493$. This is a joint probability.

(b) $\mathbb{P}(blue) = 0.506$. Note that this is from the Total row.

(c) The product rule says $\mathbb{P}(short, blue) = \mathbb{P}(short|blue)\,\mathbb{P}(blue)$, so $\mathbb{P}(short|blue) = 0.493/0.506 = 0.974$. This number does not appear anywhere in the table.

Question 3

Bayes' theorem says

$$p(blue|short) = \frac{p(short|blue)\,p(blue)}{p(short)} = \frac{(0.974)(0.506)}{0.760} = 0.648$$

Be careful not to confuse this with the box for blue *and* short, which would give 0.493. We *can* also find it from the table by using the product rule $p(blue|short) = p(blue, short)/p(short) = 0.493/0.760 = 0.648$.

Question 4

The second row illustrates marginalization: $p(short, blue) + p(short, brown) = p(short)$.

Question 5

We can test for mutual independence by seeing whether the probability of tall and brown is the product of the individual probability of being tall multiplied by the individual probability of having brown eyes:

$$p(tall, brown) = 0.227 \neq p(tall) \times p(brown) = 0.240 \times 0.494$$

so they are not independent.

Solution to Exercise 4.2 (Standard medical example using Bayes)

Question 1 It seems like it should be high because the false positive rate is low.

Question 2 You want to know if you have the disease, given that you have tested positively, therefore: $\mathbb{P}(H|D)$

Question 3 The probability that you are trying to find is that you get a positive result on the test (so D should be on the left of the bar) given that you don't actually have the disease (this is the "false" part). So $\overline{H}$ on the right. (Again, when you talk about false positive it is about the test result, not the disease, so D is on the left.) Overall

with the probability we are given (derived from the rate):
$\mathbb{P}(D|\overline{H}) = 0.023$

Question 4 False negative is the counterpart of false positive, so the probability of $\overline{D}$ given H:
$\mathbb{P}(\overline{D}|H) = 0.014$. For both false negative and false positive cases, the probability is the *outcome of the test*, given additional information. You might have been fooled by the wording above: "false negative means you have UD, but the test says you don't". This might cause you to think that H should be on the left. But reword it as: "false negative means that the test says you don't have UD, but you do". This makes it clearer that the probability is about the test result, not about the disease itself.

The sum rule says $\mathbb{P}(D|H) + \mathbb{P}(\overline{D}|H) = 1$, therefore we know: $\mathbb{P}(D|H) = 0.986$ This probability being so close to one is what makes us think the probability we have the disease is high.

Question 5 $\mathbb{P}(D|H) + \mathbb{P}(D|\overline{H}) = 1.09 \neq 1$ so the first answer is no. But the sum rule holds when summing over all possibilities on the *left* of the bar with the same statements on the right of the bar, which is not the case here.

The second sum *does* satisfy these conditions, so we expect the sum rule to hold and
$\mathbb{P}(D|H) + \mathbb{P}(\overline{D}|H) = 1$, which we've already used.

Question 6 Bayes' theorem with just the $p(\cdot|\cdot)$s:

$$\mathbb{P}(H|D) = \frac{\mathbb{P}(D|H)\,\mathbb{P}(H)}{\mathbb{P}(D)}$$

This is useful because we know $\mathbb{P}(D|H)$. But we still need $\mathbb{P}(H)$ and $\mathbb{P}(D)$.

Question 7 We are told that 1 in 10,000 people have the disease, so $\mathbb{P}(H) = 10^{-4}$.

That means by the sum rule that $\mathbb{P}(\overline{H}) = 1 - \mathbb{P}(H) = 1 - 10^{-4}$.

Question 8 The strategy here is to observe that we know various probabilities with D on the left of the bar and statements on the right side of the bar. Can we combine them to get $\mathbb{P}(D)$?

Marginalization: $\mathbb{P}(D) = \mathbb{P}(D, H) + \mathbb{P}(D, \overline{H})$ (recall that these are joint probabilities, not conditional probabilities).

Now apply the product rule to each term: $\mathbb{P}(D, H) = \mathbb{P}(D|H)\,\mathbb{P}(H)$ and $\mathbb{P}(D, \overline{H}) = \mathbb{P}(D|\overline{H})\,\mathbb{P}(\overline{H})$

Put it together with numbers:

$$\mathbb{P}(D) = \mathbb{P}(D|H)\,\mathbb{P}(H) + \mathbb{P}(D|\overline{H})\,\mathbb{P}(\overline{H}) = 0.986 \times 10^{-4} + 0.023 \times (1 - 10^{-4}) \approx 0.023$$

Question 9 $$\mathbb{P}(H|D) = \frac{0.986 \times 0.0001}{0.023} = 0.0043$$

or about 0.43%, which is really low!

We conclude this is a terrible test! If we imagine 10000 people taking the test, the expectation is that only one of them actually has UD, but 230 will get a positive result. We need the false positive rate to be much smaller relative to the expected rate in the population for this to be a better test. (Of course, maybe this is just an

inexpensive preliminary screening and the expensive test with the low false positive rate only needs to be performed on the 230 people.)

4.3. Probability density functions

The key mathematical entity used over and over again throughout this book will be the "probability density function", or just PDF for short. The PDF for continuous quantities can be integrated over a domain to obtain probabilities. An example familiar to every physicist will be the probability density of a quantum-mechanical particle. Max Born somewhat belatedly decided to put a square on Schrödinger's wave function when interpreting the integral from $x = a$ to $x = b$ as the probability of measuring the particle to be between a and b:

$$\mathbb{P}(a \leq x \leq b) = \int_a^b |\psi(x)|^2 \, dx$$

We note that the unit of $|\psi(x)|^2$ is inverse length, and it is generally true that PDFs, unlike probabilities, have units. If we divorce the previous equation from the quantum-mechanical context we would write:

$$\mathbb{P}(a \leq x \leq b) = \int_a^b p\left(x\right) dx$$

where $p\left(x\right)$ is the PDF for the continuous variable x.

Checkpoint question

What is the unit of $p(\boldsymbol{x})$ (or $|\psi(\boldsymbol{x})|^2$) in three dimensions (assuming $\boldsymbol{x}$ is a position vector measured in units of length)?

Answer

We can use the normalization equation (which is the sum rule):

$$\int d^3x \, p(\boldsymbol{x}) = 1,$$

which is dimensionless on the right side, so the unit of $p(\boldsymbol{x})$ (or $|\psi(\boldsymbol{x})|^2$) is the inverse unit of d^3x, or $1/\text{length}^3$. Note that if $\boldsymbol{x}$ represented a different quantity, the unit of $p(\boldsymbol{x})$ would differ accordingly.

We note that the case of a discrete random variable, e.g., the numbers that can be rolled on a die, mean that the PDF only is non-zero at a discrete set of allowed outcomes. In that case $p\left(x\right)$ can be represented as a sum of Dirac delta functions, and the resulting object is sometimes called a probability mass function (PMF). Thus the cases discussed above, in which $\mathbb{P}$ represented probability measured over a discrete set of outcomes, can be understood as special cases of PDFs.

In fact, working this from the other end, the sum and product rules for PDFs look just the same way as they do for probabilities. Therefore Bayes' theorem (or rule) can also be applied to relate the PDF of x given y, $p\left(x \mid y\right)$ to the pdf of y given x, $p\left(y \mid x\right)$.

Dirac delta functions

A delta function $\delta(x - x_0)$ can be constructed as the limiting case of a distribution

$$\delta(x - x_0) = \lim_{\varepsilon \to 0^+} h_\varepsilon(x - x_0).$$

For example, it can be constructed as an infinitely narrow (and tall) normal distribution

$$\delta(x - x_0) = \lim_{\varepsilon \to 0^+} \frac{1}{\sqrt{2\pi}\varepsilon} \exp\left(-\frac{(x - x_0)^2}{2\varepsilon^2}\right).$$

This function will be zero for $x \neq x_0$, and goes to infinity at x_0 in a way such that the integral $\int_{-\infty}^{+\infty} \delta(x - x_0)dx = 1$, which is a defining property.

More general, for well-behaved functions $f(x)$, we have $\int_{-\infty}^{+\infty} f(x)\delta(x - x_0)dx = f(x_0)$.

In Bayesian statistics there are PDFs (or PMFs if discrete) for everything. Here are a few examples:

- fit parameters — like the slope and intercept of a line in a straight-line fit
- experimental *and* theoretical imperfections that lead to uncertainties in the final result
- events ("Will it rain tomorrow?")

We will stick to the $p(\cdot)$ notation here, but it's worth pointing out that many different variants of the letter p are used to represent probabilities and PDFs in the literature. If we are interested in the PDF in a higher-dimensional space (say the probability of finding a particle in 3D space) you might see $p(\vec{x}) = p(\mathbf{x}) = P(\vec{x}) = \text{pr}(\vec{x}) = \text{prob}(\vec{x}) = \ldots$.

Checkpoint question

What is the PDF $p(x)$ if we know **definitely** that $x = x_0$ (i.e., the outcome is fixed)?

Answer

$p(x) = \delta(x - x_0)$ [Note that $p(x)$ is normalized.]

Joint and marginal PDFs

$p(x_1, x_2)$ is the *joint* probability density of x_1 and x_2. In quantum mechanics the probability density $|\Psi(x_1, x_2)|^2$ to find particle 1 at x_1 and particle 2 at x_2 is a joint probability density.

Checkpoint question

What is the probability to find particle 1 at x_1 while particle 2 is *anywhere*?

Answer

$\int_{-\infty}^{+\infty} |\psi(x_1, x_2)|^2 \, dx_2$ or, more generally, integrated over the domain of x_2.

This Checkpoint question is a specific example of the marginalization rule, now in the PDF context, that we discussed for probabilities above. The *marginal probability density* of x_1 is the result when we marginalize the *joint probability distribution* over x_2:

$$p\left(x_1\right) = \int p\left(x_1, x_2\right) dx_2.$$

So, in our quantum mechanical example, it's the probability density we get when we just focus on particle 1, and don't care about particle 2. *Marginalizing* in the Bayesian context means "integrating out" a parameter one is–at least temporarily–not interested in, to leave the focus on the PDF of other parameters. This can be particularly useful if there are "nuisance" parameters in the statistical model that are just indirectly influencing the quantity of interest. A specific example could be a parameter that accounts for the impact of defects in the measuring apparatus, but ultimately one is interested in the physics that can be extracted with the imperfect apparatus.

[]Definition 39.7 contains a summary of the properties of both one- and multi-dimensional pdfs.

Note

You may have noticed that we wrote $p\left(x_1\right)$ and $p\left(x_1, x_2\right)$ rather than $p\left(x_1 \mid I\right)$ and $p\left(x_1, x_2 \mid I\right)$, where "$I$" is information we know but do not specify explicitly. Our PDFs will always be contingent on *some* information, so we were really being sloppy by trying to be compact.

Bayes' theorem applied to PDFs

The rules applied to probabilities in Section 4.2 directly extend to continuous functions, i.e., PDFs; we'll summarize the extension briefly here. Conventionally we denote by $\boldsymbol{\theta}$ a vector of continuous parameters we seek to determine (for now we'll use $\boldsymbol{\alpha}$ as another vector of parameters). As before, information I is kept implicit.

Sum rule

Probabilities integrate to one (assuming exhaustive and exclusive $\boldsymbol{\theta}$):

$$\int d\boldsymbol{\theta}\, p\left(\boldsymbol{\theta} \mid I\right) = 1.$$

The sum rule *implies* marginalization:

$$p(\boldsymbol{\theta}|I) = \int d\boldsymbol{\alpha}\, p(\boldsymbol{\theta}, \boldsymbol{\alpha}|I).$$

Product rule

Expanding a joint probability of $\boldsymbol{\theta}$ and $\boldsymbol{\alpha}$

$$p(\boldsymbol{\theta}, \boldsymbol{\alpha}|I) = p(\boldsymbol{\theta}|\boldsymbol{\alpha}, I)p(\boldsymbol{\alpha}, I) = p(\boldsymbol{\alpha}|\boldsymbol{\theta}, I)p(\boldsymbol{\theta}, I). \tag{4.7}$$

As with discrete probabilities, there is a symmetry between the first and second equalities. If $\boldsymbol{\theta}$ and $\boldsymbol{\alpha}$ are *mutually independent*, then $p(\boldsymbol{\theta}|\boldsymbol{\alpha}, I) = p(\boldsymbol{\theta}|I)$ and

$$p(\boldsymbol{\theta}, \boldsymbol{\alpha}|I) = p(\boldsymbol{\theta}|I)p(\boldsymbol{\alpha}|I).$$

Rearranging the 2nd equality in (4.7) yields **Bayes' rule** (or theorem) just like in the discrete case:

$$p(\boldsymbol{\theta}|\boldsymbol{\alpha}, I) = \frac{p(\boldsymbol{\alpha}|\boldsymbol{\theta}, I)p(\boldsymbol{\theta}|I)}{p(\boldsymbol{\alpha}|I)}$$

Bayes' rule tells us how to reverse the conditional: $p(\boldsymbol{\alpha}|\boldsymbol{\theta}) \Rightarrow p(\boldsymbol{\theta}|\boldsymbol{\alpha})$. A typical application is when $\boldsymbol{\alpha}$ is a vector of data $\mathcal{D}$. Then Bayes' rule is

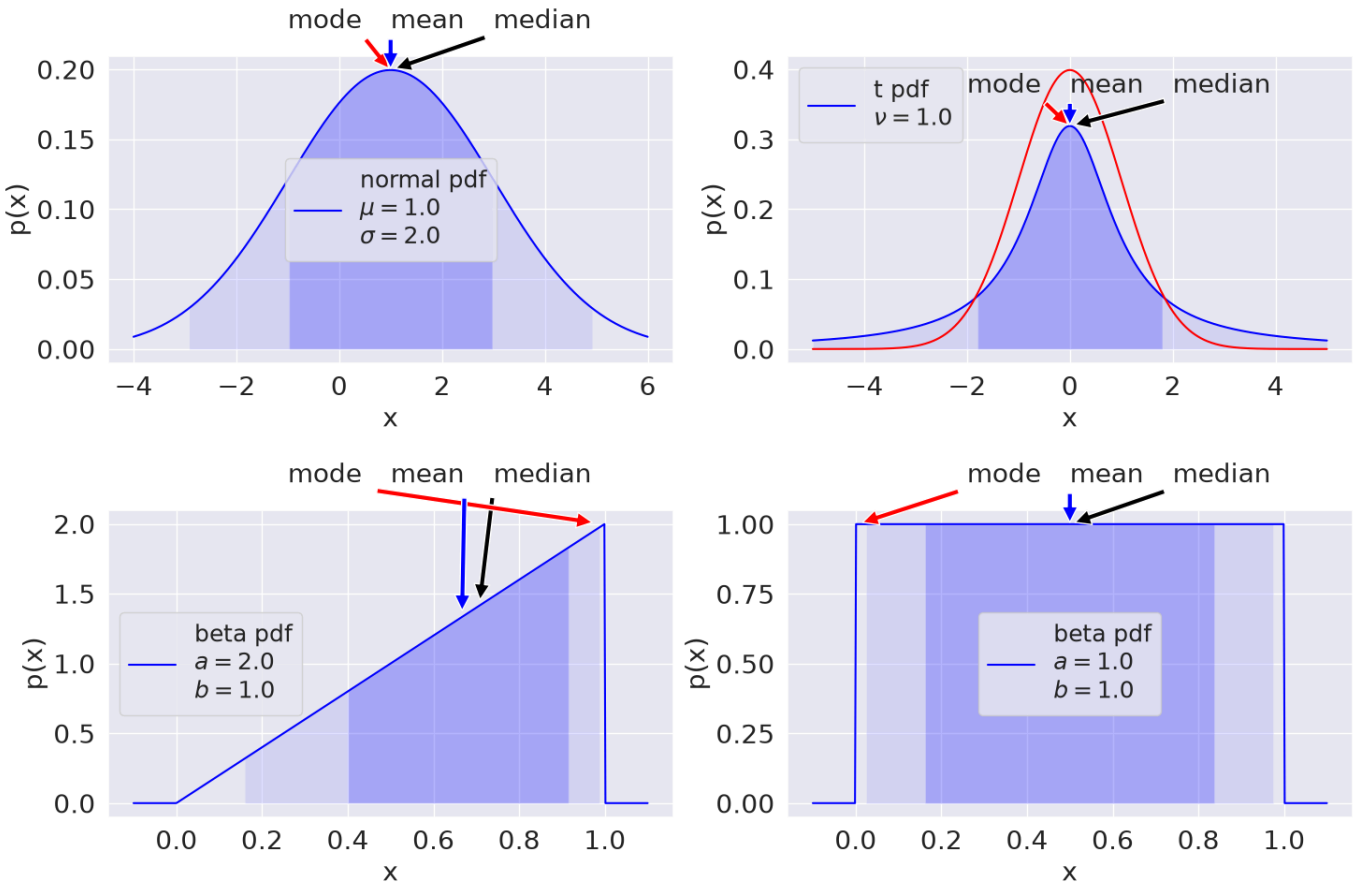
$$\overbrace{p\left(\boldsymbol{\theta} \mid \mathcal{D}, I\right)}^{\text{posterior}} = \frac{\overbrace{p\left(\mathcal{D} \mid \boldsymbol{\theta}, I\right)}^{\text{likelihood}} \times \overbrace{p\left(\boldsymbol{\theta} \mid I\right)}^{\text{prior}}}{\underbrace{p\left(\mathcal{D} \mid I\right)}_{\text{evidence}}}$$

Viewing the prior as the initial information we have about $\boldsymbol{\theta}$ (i.e., before seeing the data $\mathcal{D}$), expressed as a PDF, then Bayes' theorem tells us how to **update** that information after observing some data: this is the posterior PDF. In Section 6 we will give some examples of how this plays out when tossing biased coins.

Visualizing PDFs

Some visualizations of one-dimensional PDFs are shown here: a Gaussian (or normal) distribution; a Student t distribution (with overlaid normal distribution with the same standard deviation); two beta distributions with different parameters (the second one corresponds to a uniform distribution). The mode (maximum value), mean (average value), and median (50% of the probability is below and above) are shown for each distribution.

▼ **Hide code cell source**



The 68%/95% probability regions are shown in dark/light shading. When applied to Bayesian posteriors, these are known as *credible intervals* or DoBs (degree of belief intervals). One can also encounter the name Bayesian confidence intervals, but this use of the label "confidence interval" is not recommended. The horizontal extent on the x-axis translates into the vertical extent of the error bar or error band for x. The values of the mode, mean, median can be used as *point estimates* for the "probable" value of x.

These examples are just a taste of a large range of standard distributions that are available in the python package `scipy.stats`. You can look at the code that generated the figures here ("Show code cell source") but it may be worthwhile at this stage to jump ahead and explore using the exercise notebook 📥 Problem: Exploring PDFs to make a hands-on pass at getting familiar with PDFs and how to visualize them using `scipy.stats`.

You can also get a preview of the effects of finite *sampling*. Since we can draw an arbitrary number of samples from the distributions defined in `scipy.stats`, we can see how the distributions build up as the number of samples increases, with decreasing fluctuations around the asymptotic distribution.

> **Note**
>
> See Appendix A for further details on PDFs. You can also consult the `scipy.stats` manual page on statistical functions.

The diversity of available distributions should make it clear that the "default" choice of a Gaussian distribution is just that: a default choice. We will explore why this default choice is often the correct one in One-dimensional PDFs. But *often* is not the same as *all the time*, and it is frequently the case that another distribution gives a better description of the way data is distributed.

Trivia: Who was “Student”?

“Student” was the pen name of the Head Brewer at Guinness — a pioneer of small-sample experimental design (hence not necessarily Gaussian). His real name was William Sealy Gosset. “Statistics and Beer Day” is sometimes observed on his birthday, June 13, to celebrate his contributions.

Credible intervals

Bayesian credible intervals: how are they defined?

Answer

The Bayesian credible interval for a parameter with a one-dimenional PDF is defined for a given percentage, call it $\beta\%$. The interval is such that integrating over it one gets $\beta\%$ of the total area under the PDF. The choice of interval is not unique. In particular, several reasonable intervals can be considered if the PDF is multimodal or skewed. See Defining credible intervals for more on possible choices.

Point estimates

Various “point estimates” were introduced (mean, mode, median); which is “best” to use?

Answer

A point estimate is a one-number summary of a distribution. Which one is best to use depends on the application. Sometimes one wants to know the most likely case: then use the mode. Sometimes one really wants the average: then use the mean. Sometimes, for an asymmetric distribution, the median is the best estimate. For a symmetric, unimodal PDF, the three point estimates are the same. Note that to a Bayesian, a point estimate is only a rough approximation to the information in the full distribution.

4.4. Expectation values and moments

In this section we provide a brief introduction to expectation values and moments, summarizing the key discrete and continuous definitions. Warning: there are multiple notations in the literature for expectation values and moments! Appendix A has further details, including Expectation values and moments and Central moments: Variance and Covariance.

We also address the question: *Is a continuous PDF determined by its moments?* and consider the effective number of samples when the sampling process implies correlation. For independent (and hence uncorrelated) samples, we expect new information with each sample. But this is not the case if there are strong correlations between samples!

Summary of expectation values and moments

The *expectation value* of a function h of the random variable X with respect to its distribution $p(x_i)$ (a PMF) or $p(x)$ (a PDF) is

$$\mathbb{E}_p[h] = \sum_i h(x_i)p(x_i) \quad \Longrightarrow \quad \mathbb{E}_p[h] = \int_{-\infty}^{\infty} h(x)p(x)\,dx.$$

The p subscript is often omitted if it is clear which distribution is being considered. *Moments* correspond to $h(x) = x^n$. The zeroth moment (with $n = 0$) is always equal to one since it is the normalization condition of the PDF. The first moment ($n = 1$) gives the mean, which is often denoted μ:

$$\mathbb{E}[X] \equiv \mu = \sum_i x_i p(x_i) \quad \Longrightarrow \quad \mathbb{E}[X] \equiv \mu = \int_{-\infty}^{\infty} xp(x)\,dx \equiv \langle x \rangle \equiv \bar{x},$$

where we have also indicated two other common notations for the mean.

The variance and covariance are (central) moments with respect to the mean for one and two random variables:

$$\begin{aligned}
\text{Var}\,(X) &\equiv \sigma_X^2 \equiv \mathbb{E}\left[(X - \mathbb{E}[X])^2\right] \\
\text{Cov}\,(X, Y) &\equiv \sigma_{XY}^2 \equiv \mathbb{E}\left[(X - \mathbb{E}[X])\,(Y - \mathbb{E}[Y])\right].
\end{aligned}$$

The standard deviation σ is simply the square root of the variance σ^2.

The **correlation coefficient** of X and Y (for non-zero variances) is

$$\rho_{XY} \equiv \frac{\text{Cov}\,(X, Y)}{\sqrt{\text{Var}\,(X)\text{Var}\,(Y)}},$$

The **covariance matrix** Σ_{XY} is

$$\Sigma_{XY} = \begin{pmatrix} \sigma_X^2 & \sigma_{XY}^2 \\ \sigma_{XY}^2 & \sigma_Y^2 \end{pmatrix} = \begin{pmatrix} \sigma_X^2 & \rho_{XY}\sigma_X\sigma_Y \\ \rho_{XY}\sigma_X\sigma_Y & \sigma_Y^2 \end{pmatrix} \quad \text{with} \quad 0 < \rho_{XY}^2 < 1. \tag{4.8}$$

Checkpoint question

Show that we can also write $\sigma^2 = \mathbb{E}[X^2] - \mathbb{E}[X]^2$.

Answer

$$\begin{aligned}
\sigma^2 &= \int_{-\infty}^{\infty} (x - \mathbb{E}[X])^2 p(x) dx \\
&= \int_{-\infty}^{\infty} \left(x^2 - 2x\mathbb{E}[X] + \mathbb{E}[X]^2\right) p(x) dx \\
&= \mathbb{E}[X^2] - 2\mathbb{E}[X]\mathbb{E}[X] + \mathbb{E}[X]^2 \\
&= \mathbb{E}[X^2] - \mathbb{E}[X]^2
\end{aligned}$$

Make sure you can justify each step.

Checkpoint question

Show that the mean and variance of the normalized Gaussian distribution

$$p \longrightarrow \mathcal{N}(x|\mu,\sigma^2) = \frac{1}{\sigma\sqrt{2\pi}}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right),$$

are μ and σ^2, respectively.

Answer

Just do the integrals!

$$\mu = \int_{-\infty}^{\infty} x\frac{1}{\sigma\sqrt{2\pi}}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)dx = \mu \quad \checkmark$$
$$\sigma^2 = \int_{-\infty}^{\infty} (x-\mu)^2\frac{1}{\sigma\sqrt{2\pi}}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)dx = \sigma^2 \quad \checkmark$$

In doing these integrals, simplify by changing the integration variable to $x' = x - \mu$ and use that the distribution is normalized (integrates to one) and that integrals of odd integrands are zero.

Covariance matrix for a bivariate (two-dimensional) normal distribution

With vector $\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$, the distribution is

$$p(\boldsymbol{x}|\boldsymbol{\mu},\Sigma) = \frac{1}{\sqrt{\det(2\pi\Sigma)}}e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\intercal\Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}$$

with mean and covariance matrix:

$$\boldsymbol{\mu} = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix} \quad \text{and} \quad \Sigma = \begin{pmatrix} \sigma_1^2 & \rho_{12}\sigma_1\sigma_2 \\ \rho_{12}\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix} \quad \text{with} \quad 0 < \rho_{12}^2 < 1.$$

Note that Σ is symmetric and positive definite. See Section 5.2 for plotting what this looks like.

Checkpoint question

Why can't we have $\rho > 1$ or $\rho < -1$?

Answer

The bounds on the correlation coefficient ρ arise from the Cauchy-Schwarz inequality, which says that if $\langle \cdot, \cdot \rangle$ defines an inner product, then for all vectors $\boldsymbol{u}, \boldsymbol{v}$ in the space, this inequality holds:

$$|\langle \boldsymbol{u}, \boldsymbol{v} \rangle|^2 \leq |\langle \boldsymbol{u}, \boldsymbol{u} \rangle||\langle \boldsymbol{v}, \boldsymbol{v} \rangle|.$$

The expectation value of a random variable satisfies the conditions to be an inner product, so that the Cauchy-Schwarz inequality implies that

$$|\mathbb{E}[UV]| \leq \sqrt{\mathbb{E}[U^2]}\sqrt{\mathbb{E}[V^2]}.$$

With $U = X - \mu_X$ and $V = Y - \mu_Y$, the inequality implies that

$$|\text{Cov } (X, Y)| \leq \sigma_X \sigma_Y.$$

Dividing through by $\sigma_X \sigma_Y$ yields $-1 \leq \rho \leq 1$.

A consequence of $|\rho| > 1$ is that the matrix Σ would not be a valid covariance matrix because the determinant would be negative.

Is a continuous PDF determined by its moments?

If we know some (or all) of the moments of a one-dimensional distribution $p(x)$, how can we find the PDF? Let us define the moments m_k for integer k as

$$m_k = \int_S x^k \, p(x) \, dx, \qquad k = 0, 1, 2, \ldots,$$

where S denotes the support of the PDF (e.g., $(-\infty, \infty)$ for a Gaussian or $[0, 1]$ for a Beta distribution). So $m_0 = 1$, the mean μ is m_1, the variance σ^2 is $m_2 - m_1^2$, and so on.

Note

We can generalize this discussion to multi-dimensional PDFs with vector $\mathbf{x}$, for which the moments will have multiple indices and the defining expression will be integrated over the space of $\mathbf{x}$.

Here are some ways to find $p(x)$ given a set of moments $\{m_k\}$:

1. If we know the PDF belongs to a particular parametric family, such as Gaussian or Beta (see One-dimensional PDFs), then we can solve for the defining parameters if we know the appropriate moments. For the case of a Gaussian, we know it is defined by its mean μ and variance σ^2, so those are the only two moments we need. (In fact, we can determine the mean and variance even if we only know higher moments!) This is also true for the less familiar case of a Beta distribution that is specified by the shape parameters $\alpha, \beta > 0$ (see Beta distribution on Wikipedia). If the variance satisfies $\sigma^2 < \mu(1 - \mu)$, then

$$\alpha = \mu\left(\frac{\mu(1-\mu)}{\sigma^2} - 1\right), \qquad \beta = (1-\mu)\left(\frac{\mu(1-\mu)}{\sigma^2} - 1\right).$$

2. We can use a maximum entropy reconstruction. With a finite number of known moments, we choose among distributions that reproduce these moments the one with the largest entropy. See Section 11.2 and Section 11.3 to learn about the principle of maximum entropy and how to do a maximum entropy reconstruction from moments.
3. We can try to invert from a *moment-generating function* or a *characteristic function*. If X is a random number with distribution $p_X(x)$, then the moment-generating function $M_X(t)$ and characteristic function $\phi_X(t)$ are for $t \in \mathbb{R}$:

$$M_X(t) = \mathbb{E}[e^{tX}] = \sum_{k=0}^{\infty} \frac{t^k}{k!}\mathbb{E}[X^k] = \sum_{k=0}^{\infty} \frac{m_k}{k!}t^k \quad \Longrightarrow \quad M_X^{(k)}(0) = m_k,$$
$$\phi_X(t) = \mathbb{E}[e^{itX}] = \sum_{k=0}^{\infty} \frac{(it)^k}{k!}\mathbb{E}[X^k] = \sum_{k=0}^{\infty} \frac{m_k}{k!}(it)^k \quad \Longrightarrow \quad \phi_X^{(k)}(0) = m_k,$$

where we have formally performed Taylor expansions and inverted the order of summation and expectation value (we're assuming here that these are valid operations). The superscript (k) means the k^{th} derivative. In integral form,

$$M_X(t) = \int_{-\infty}^{\infty} e^{tx}\, p_X(x)\, dx, \qquad \phi_X(t) = \int_{-\infty}^{\infty} e^{itx}\, p_X(x)\, dx.$$

So the moment-generating function is essentially a Laplace transform of the distribution, while the characteristic function is essentially a Fourier transform. As such, we can try to invert them to find $p_X(x)$.

> **Note**
>
> We can derive immediately that for a Gaussian distribution with mean μ and variance σ^2,
>
> $$M_X(t) = \exp(\mu t + \frac{1}{2}\sigma^2 t^2), \qquad \phi_X(t) = \exp(i\mu t - \frac{1}{2}\sigma^2 t^2). \tag{4.9}$$

4. We can use *discrete quadrature reconstruction*. In this case we approximate the PDF by a finite sum of n point masses (i.e., delta functions) with weights and locations chosen to reproduce the first n moments. That is, replace the continuous $p(x)$ by the discrete $p_N(x)$:

$$p_N(x) = \sum_{i=1}^{N} w_i \delta(x - x_i), \qquad w_i \geq 0, \qquad \sum_{i=1}^{N} w_i = 1,$$

where the x_i are nodes and the w_i are the weights, such that

$$m_k \approx \sum_{i=1}^{N} w_i x_i^k.$$

If K moments m_k for $k = 0, 1, \ldots, K$ are known and the x_i are fixed, this is a linear problem in the weights w_i. If both x_i and w_i are unknown, one has more flexibility. Under suitable conditions, $2N - 1$ moments can be exactly matched by an N-point discrete representation (so this is closely related to Gaussian quadrature).

In most, but not all, cases a PDF will uniquely be determined by a complete set of moments.

Correlations and the effective number of samples

Consider n quantities y_1, y_2, …, y_n, which individually are identically distributed with mean μ_0 and variance σ_0^2, but with unspecified covariances between the y_i. For concreteness we'll say that the distribution of $\boldsymbol{y} = (y_1, y_2, \ldots, y_n)$ is a multivariate Gaussian:

$$\boldsymbol{y} \sim \mathcal{N}(\mu_0 \mathbf{1_n}, \Sigma),$$

where $\mathbf{1_n}$ is a n-dimensional vector of ones and Σ is an $n \times n$ covariance matrix with all diagonal elements equal to σ_0^2.

Let us consider two limits of this distribution: (i) the uncorrelated one with all off-diagonal elements of Σ equal to zero and (ii) the fully correlated one with all covariances equal to σ_0^2. The covariance matrix of case (ii) is singular but this will not be a concern for our argument.

If we repeatedly draw y_1 and y_2 from the uncorrelated distribution, the joint distribution on a corner plot is 2-dimensional (in particular, the contours are circular). With n draws, the joint distribution is n-dimensional (and the contours for the marginalized 2-dimensional plots are all circular). But the same analysis with the fully correlated distribution would yield a straight line for the $n = 2$ case; that is to say all samples can be found in a single dimension. Further, it remains 1-dimensional for any n. We can interpret this as meaning the effective number of draws or samples is n in the uncorrelated case and 1 in the correlated case.

Consider the average value of $y_1, y_2, \ldots, y_n$ and the variance of that average. For convenience, define z as

$$z = \frac{1}{n}(y_1 + y_2 + \ldots + y_n).$$

Then if we make many draws of y_1 through y_n, the expectation values of each y_i is the same, namely μ_0, so the expectation value of z is

$$\langle z \rangle = \frac{1}{n}(\langle y_1 \rangle + \ldots + \langle y_n \rangle) = \frac{1}{n} n \langle y_1 \rangle = \mu_0.$$

Now define $\tilde{y}_i = y_i - \mu_0$ and $\tilde{z} = z - \mu_0$, so that these new random variables are all mean zero (this is for clarity, not necessity). The variance of $\tilde{y}_i$ is equal to $\langle \tilde{y}_i^2 \rangle = \sigma_0^2$. Consider now the variance of $\tilde{z}$ which is equal to the expectation value of $\tilde{z}^2$,

$$\langle \tilde{z}^2 \rangle = \frac{1}{n^2}(\langle \tilde{y}_1^2 \rangle + \langle \tilde{y}_2^2 \rangle + \ldots + \langle \tilde{y}_1 \tilde{y}_2 \rangle + \langle \tilde{y}_1 \tilde{y}_3 \rangle + \langle \tilde{y}_2 \tilde{y}_3 \rangle + \ldots) \tag{4.10}$$

Since $\langle \tilde{y}_i \tilde{y}_j \rangle$ either is zero or equals σ_0^2 for the two extremes of correlation, we find

$$\begin{aligned} &\text{uncorrelated} && \langle \tilde{z}^2 \rangle = \frac{1}{n^2} \cdot n \langle \tilde{y}_i^2 \rangle = \frac{\sigma_0^2}{n}, \\ &\text{fully correlated} && \langle \tilde{z}^2 \rangle = \frac{1}{n^2} \cdot n^2 \langle \tilde{y}_i \tilde{y}_j \rangle = \frac{\sigma_0^2}{1}, \end{aligned}$$

where we have n nonzero terms in the uncorrelated case but n^2 nonzero terms in the fully correlated case. Thus the standard deviation in the uncorrelated case decreases with the familiar $1/\sqrt{n}$ factor, so the effective number of samples is n. But in the fully correlated case the width is independent of n: increasing n adds no new information, so the effective number of samples is one.

We can do the more general case by considering $\boldsymbol{y}$ distributed as above with $\Sigma = \sigma_0^2 M$, where M is the $n \times n$ matrix:

$$M = \begin{pmatrix} 1 & \rho & \rho & \cdots & \rho \\ \rho & 1 & \rho & \cdots & \rho \\ \rho & \rho & 1 & \cdots & \rho \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \rho & \rho & \rho & \cdots & 1 \end{pmatrix},$$

with $0 \leq \rho < 1$ (we assume only positive correlation here). So the two extremes just considered correspond to $\rho = 0$ and $\rho = 1 - \epsilon$ (we introduce ϵ as a small positive number to be taken to zero at the end, needed because otherwise the covariance matrix Σ cannot be inverted). Then the variance of $\tilde{z}$ given $\boldsymbol{y}$ is

$$\text{Var}\,(\tilde{z}|\boldsymbol{y}, \sigma_0^2, M) = \left[\mathbf{1}_{\boldsymbol{n}}^\top (\sigma_0^2 M) \mathbf{1}_{\boldsymbol{n}}\right]^{-1} = \frac{\sigma_0^2[1 + (n-1)\rho]}{n}.$$

We can get this result directly from (4.10) by noting there are n terms with $i = j$ and $n(n-1)$ terms with $i \neq j$. So the variance is

$$\langle \tilde{z}^2 \rangle = \frac{\sigma_0^2}{n^2}\left(n \cdot 1 + n(n-1) \cdot \rho\right) = \frac{\sigma_0^2[1 + (n-1)\rho]}{n} = \frac{\sigma_0^2}{n_{\text{eff}}}$$

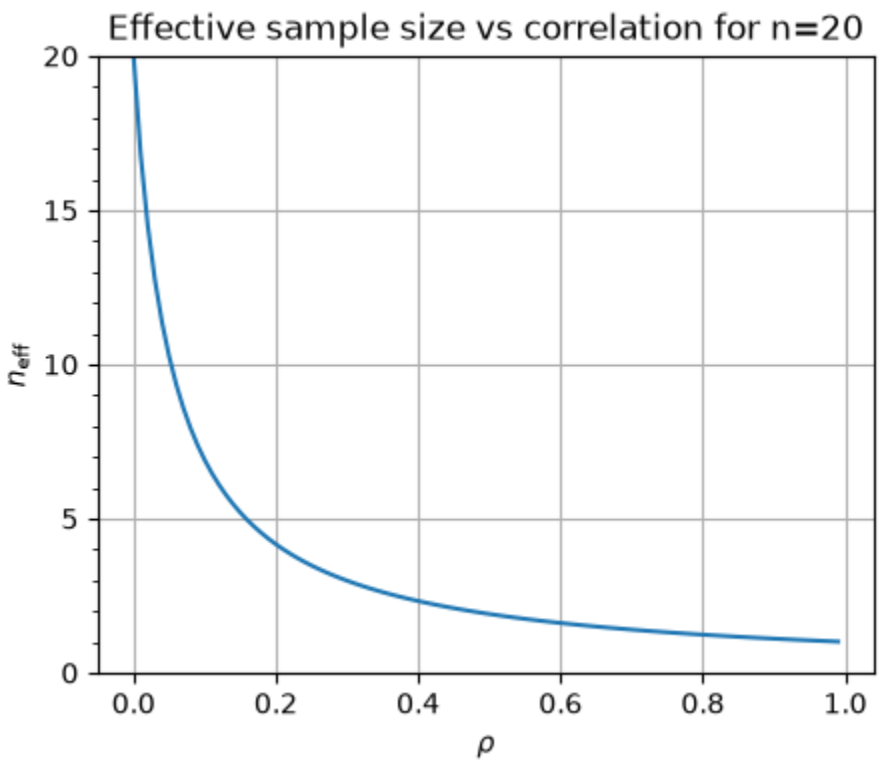
Thus the effective number of degrees of freedom is

$$n_{\text{eff}} = \frac{n}{[1 + (n-1)\rho]}.$$

This also follows from the Fisher information matrix.

For $n = 20$, n_{eff} looks like:

▼ **Hide code cell source**



4.5. Data, models, and predictions

> "All models are wrong, but models that know when and how they are wrong, are useful…"

—revised version of George Box's quote (see Overview of modeling)

How do we make statistical predictions?

Let us consider a common scenario:

- We have a model M with parameters $\boldsymbol{\theta}$.
- We have some (noisy) data $\mathcal{D}$ that we can use to calibrate the parameters. Generically label the input values by x (a vector) and the quantities of interest by $y = y(x)$.
- Then we want to predict at other input points $x^* \to y^*$.

How shall we proceed? Let's first say a bit more about the ingredients.

A statistical model for our data

From the Bayesian workflow introduced in Section 2.2, a key element of our inference is to formulate a statistical model for our data. Let us start with the *data* $\mathcal{D}$ already obtained through a measurement process, e.g., an experiment in a laboratory or an observation of some astronomical event. All data come with uncertainties of various origin, let us denote these as $\delta\mathcal{D}$. Given some data $\mathcal{D}$, one might immediately ask what this data can tell us about data we have not yet collected or used in the inference. We call this future data $\mathcal{F}$. At present, we are uncertain about any future data, and we describe as a (conditional) probability $\mathbb{P}\left(\mathcal{F} \mid \mathcal{D}, I\right)$. All we have said so far is that *predictions are uncertain*. The obvious and interesting question is: how uncertain is the prediction? To answer that, we must go from this abstract probability to something that we can evaluate quantitatively. The first step is to develop a theoretical model to analyze the relevant data.

A physical *model* M allows quantitative evaluation of the system under study. Any model we employ will always depend on model parameters $\boldsymbol{\theta}$ with uncertain numerical values. Moreover, *all models are wrong*, in the sense that there will always be some physics that we have neglected to include or are unaware of today. If we denote mismatch between model predictions and real world observations of the system, i.e., data, as δM, we can write

$$\mathcal{D} = M(\boldsymbol{\theta}) + \delta\mathcal{D} + \delta M. \tag{4.11}$$

The mismatch term δM is often referred to as a model discrepancy. We are uncertain also about this term, so it must be represented as a random variable that is distributed in accordance with our beliefs about δM. It is no trivial task to incorporate model discrepancies in the analysis of scientific models and data, yet it is crucial to avoid overfitting the model parameters $\boldsymbol{\theta}$ and making overly confident model predictions. We will touch upon δM in two contexts in this text: in treating the truncation error in expansions such as encountered in effective field theories and with a prototypical example (the “Ball-Drop Experiment”) of using Gaussian processes to model δM (see Discrepancy models with GPs and the 📥 Demo: Ball-drop experiment notebook).

Note that the model discrepancy remains present even if there is no uncertainty about $\boldsymbol{\theta}$. In the following we subsume the choice of model and other decisions into the set of background knowledge I.

The statistical model and underlying truth (with an alternative notation)

Let us consider the statistical model from the perspective of our (often implicit) belief as physicists that there is an underlying truth that we approach both by refining our theoretical descriptions and our experimental measurements. The spectacular agreement of theory and experiment for the anomalous magnetic dipole moment of a muon (the “muon $g-2$” measurement) is vivid testimony to the existence of this truth.

But at any given time we only approximate the truth from both theory and experiment, and statistically we seek to account for their deficiencies. Let us denote the underlying true theory (what statisticians call “truth”) as $y_{\text{true}}(x)$, where x is a generic input (i.e., it could be a vector in the input space). The truth should be our model predictions y_{th} plus the theory error (model discrepancy):

$$y_{\text{true}}(x_i) = y_{\text{th}}(x_i; \boldsymbol{\theta}) + \delta y_{\text{th}}(x_i),$$

where we have explicitly noted that the model predictions depend on parameters $\boldsymbol{\theta}$. At the same time, observation y_i at input x_i should be the truth plus the experimental error δy_{exp}:

$$y_i = y_{\text{true}}(x_i) + \delta y_{\text{exp}}(x_i).$$

Eliminating y_{true} yields our statistical model for the observations:

$$y_i = y_{\mathrm{th}}(x_i; \boldsymbol{\theta}) + \delta y_{\mathrm{exp}}(x_i) + \delta y_{\mathrm{th}}(x_i).$$

These equations encode the relationship between the random variables y_i, $y_{\mathrm{th}}(x_i; \boldsymbol{\theta})$, $\delta y_{\mathrm{exp}}(x_i)$ and $\delta y_{\mathrm{th}}(x_i)$. The underlying y_{true} describing the observables used for parameter estimation and for new observations could be the same, but in general y_{true} may be completely different for the predicted observable.

The correspondence to the notation for the same statistical model (4.11) is $y \to \mathcal{D}$, $\delta y_{\mathrm{exp}}(x_i) \to \delta\mathcal{D}$, $y_{\mathrm{th}}(x_i; \boldsymbol{\theta}) \to M(\boldsymbol{\theta})$, and $\delta y_{\mathrm{th}}(x_i) \to \delta M$.

Bayesian parameter estimation

Quantifying the posterior distribution $p\left(\boldsymbol{\theta} \mid \mathcal{D}, I\right)$ for the parameters of a model is called *Bayesian parameter estimation*, and is a staple of Bayesian inference. This is a probabilistic generalization of parameter optimization and maximum likelihood estimation whereby one tries to find an extremum parameter value of some objective function or data likelihood, respectively. Instead of single values characterizing the distribution ("point estimates"), we seek the full distribution. We will see multiple examples of this in the coming chapters.

To evaluate the posterior for the model parameters we must employ Bayes' theorem,

$$p\left(\boldsymbol{\theta} \mid \mathcal{D}, I\right) = \frac{p\left(\mathcal{D} \mid \boldsymbol{\theta}, I\right) p\left(\boldsymbol{\theta} \mid I\right)}{p\left(\mathcal{D} \mid I\right)}. \tag{4.12}$$

Here, we must formulate a likelihood of the data $p\left(\mathcal{D} \mid \boldsymbol{\theta}, I\right)$ and a prior distribution of the model parameters $p\left(\boldsymbol{\theta} \mid I\right)$.

The denominator in Eq. (4.12) is sometimes referred to as the marginal likelihood or the evidence and normalizes the left-hand side such that it integrates to unity, i.e., we have

$$p\left(\mathcal{D} \mid I\right) = \int_\Omega p\left(\mathcal{D} \mid \boldsymbol{\theta}, I\right) p\left(\boldsymbol{\theta} \mid I\right) \mathrm{d}\boldsymbol{\theta}. \tag{4.13}$$

Often we do not need an absolutely normalized posterior distribution, so we can omit the denominator in Eq. (4.12). Indeed, the latter does not explicitly depend on $\boldsymbol{\theta}$.

Bayesian parameter estimation can sometimes be very challenging. In the chapter on BLR-I: Background we will see an example of where we can perform analytical calculations throughout. However, in most realistic applications the posterior must be evaluated numerically, and most often by MCMC sampling (see Part III). This is no silver bullet and to quantify (or characterize) a multi-dimensional posterior, sometimes with a complicated geometry, for an intricate physical model, is by no means guaranteed to succeed. At least not in finite time. Nevertheless, obtaining posterior distributions to represent uncertainties is the gold standard in any inferential analysis.

The posterior predictive distribution

The distribution of future data conditioned on past data and background information, i.e., $p\left(\mathcal{F} \mid \mathcal{D}, I\right)$, is called a posterior predictive distribution (ppd). Assuming that we have a model $M(\boldsymbol{\theta})$ for the data-generating process we can express this distribution by marginalizing over the uncertain model parameters $\boldsymbol{\theta} \in \Omega$

$$p\left(\mathcal{F} \mid \mathcal{D}, I\right) = \int_{\Omega} p\left(\mathcal{F} \mid \boldsymbol{\theta}, \mathcal{D}, I\right) p\left(\boldsymbol{\theta} \mid \mathcal{D}, I\right) \mathrm{d}\boldsymbol{\theta}. \tag{4.14}$$

If $\mathcal{F}$ is conditionally independent of $\mathcal{D}$, we can replace $p\left(\mathcal{F} \mid \boldsymbol{\theta}, \mathcal{D}, I\right)$ by $p\left(\mathcal{F} \mid \boldsymbol{\theta}, I\right)$, but there are cases of ppds where this is not true and we must be more careful (e.g., sometimes when we include δM). By performing this integral we account for the uncertainty in the model parameters $\boldsymbol{\theta}$ when making predictions. In fact, one can marginalize (average) predictions over anything and everything that we are uncertain about as long as we have access to the necessary probability distributions.

Checkpoint question

How would you notate the statement that $\mathcal{F}$ is *conditionally independent* of $\mathcal{D}$ in (4.14)?

Answer

If we know $\boldsymbol{\theta}$, then *if* $\mathcal{D}$ gives no additional information, we are able to replace $p\left(\mathcal{F} \mid \boldsymbol{\theta}, \mathcal{D}, I\right) \longrightarrow p\left(\mathcal{F} \mid \boldsymbol{\theta}, I\right)$.

Exercises

Exercise 4.3

Derive Eq. (4.14) using the rules of probability calculus and inference.

Exercise 4.4

Can you think of a situation where you would have to compute the denominator in Eq. (4.12)

Exercise 4.5

In Gothenburg it rains on 60% of the days. The weather forecaster at SMHI attempts to predict whether or not it will rain tomorrow. 75% of rainy days and 55% of dry days are correctly predicted thusfar. Given that forecast for tomorrow predicts rain, what is the probability that it will actually rain tomorrow?

Exercise 4.6

Suppose you're on a game show, and you're given the choice of three doors: Behind one door is a car; behind the others, there are goats. You pick a door, say No. 1. This door remains closed. Instead, the game-show host, who knows what's behind all three doors, opens another door where he knows there will be a goat, say No. 3, which indeed has a goat. He then says to you, "Do you want to pick door No. 2?" Is it to your advantage to switch your initial choice of door? Motivate your answer using Bayes' theorem. (This is a famous problem known as *the Monty Hall problem*)

Exercise 4.7

Assume we have three coins in a bag. All three coins feel and look the same, but we are told that: the first coin is biased with a 0.75 probability of obtaining heads when flipped – the second coin is fair, i.e., 0.5 probability of obtaining heads – the third coin is biased towards tails with a 0.25 probability of coming up heads.

Assume that you reach your hand into the bag and pick a coin randomly, then flip it and obtain heads. What is the probability for obtaining heads if you flip it once more?

Solutions to exercises

Here are answers and solutions to selected exercises.

Solution to Exercise 4.3

Hint: see part of the solution to Exercise 4.7.

Solution to Exercise 4.4

For instance when making comparisons of quantities averaged over two or more posterior distributions.

Solution to Exercise 4.5

Let r be rain, $\bar{r}$ be not rain, and p be rain predicted. We seek $\mathbb{P}(r|p,I)$, where I is the background information we were given in the problem definition. Let us use Bayes' theorem

$$\mathbb{P}(r|p,I) = \frac{\mathbb{P}(p|r,I)\mathbb{P}(r|I)}{\mathbb{P}(p|I)} = \frac{\mathbb{P}(p|r,I)\mathbb{P}(r|I)}{\mathbb{P}(p|r,I)\mathbb{P}(r|I) + \mathbb{P}(p|\bar{r},I)\mathbb{P}(\bar{r}|I)}$$

With $\mathbb{P}(r|I) = 0.6$, $\mathbb{P}(\bar{r}|I) = 0.4$, $\mathbb{P}(p|r,I) = 0.75$, $\mathbb{P}(p|\bar{r},I) = 0.45$, we have $\mathbb{P}(r|p,I) \approx 0.71$

Solution to Exercise 4.6

You should switch to door No. 2 to increase your chance of winning the car. This, somewhat counter-intuitive, result can be obtained using Bayesian reasoning.

Let us introduce some notation for different events: $p_i =$ (the player initially picks door i), $h_j =$ (Monty opens door j), $c_k =$(car is behind door k). In the problem definition we had the series of events p_1 and h_3, and now we would like to know which probability is the greatest: $\mathbb{P}(c_1|p_1,h_3)$ or $\mathbb{P}(c_2|p_1,h_3)$? Let's compute them:

This is the probability that the car is behind door 1:

$$\mathbb{P}(c_1|p_1,h_3) = \frac{\mathbb{P}(h_3|c_1,p_1)\mathbb{P}(c_1,p_1)}{\mathbb{P}(p_1,h_3)} = \frac{\mathbb{P}(h_3|c_1,p_1)\mathbb{P}(c_1)\mathbb{P}(p_1)}{\mathbb{P}(h_3|p_1)\mathbb{P}(p_1)} = \frac{\mathbb{P}(h_3|c_1,p_1)\mathbb{P}(c_1)}{\mathbb{P}(h_3|p_1)}$$

We have $\mathbb{P}(h_3|c_1,p_1) = 1/2$ (since Monty can open doors 2 or 3), $\mathbb{P}(c_1) = 1/3$ (car can be placed initially behind any of the three doors), $\mathbb{P}(h_3|p_1) = 1/2$ (this probability is independent of car location!). Combined, this gives us that $\mathbb{P}(c_1|p_1,h_3) = 1/3$

Now we compute the probability that the car is behind door 2:

$$\mathbb{P}(c_2|p_1,h_3) = \frac{\mathbb{P}(h_3|c_2,p_1)\mathbb{P}(c_2,p_1)}{\mathbb{P}(p_1,h_3)} = \frac{\mathbb{P}(h_3|c_2,p_1)\mathbb{P}(c_2)\mathbb{P}(p_1)}{\mathbb{P}(h_3|p_1)\mathbb{P}(p_1)} = \frac{\mathbb{P}(h_3|c_2,p_1)\mathbb{P}(c_2)}{\mathbb{P}(h_3|p_1)}$$

We have $\mathbb{P}(h_3|c_2,p_1) = 1$ (since Monty can only open door 3), $\mathbb{P}(c_2) = 1/3$ (car can be placed initially behind any of the three doors), $\mathbb{P}(h_3|p_1) = 1/2$ (this probability is independent of car location!). Combined, this gives us that $\mathbb{P}(c_2|p_1,h_3) = 2/3$

So, you have a higher probability of getting the car if you change your initial pick from door number 1 to door number 2.

You can convince yourself that this is true by considering an extreme version of the game show where there are 100 doors, 99 goats, and 1 car. You first pick one door. After this, Monty opens 98 doors where he knows there are goats. There's only one door left closed at this point. Would you switch?

Solution to Exercise 4.7

Let H/T denote the result of a coin flip coming up heads/tails. We seek the posterior predictive probability

$$\mathbb{P}(\mathcal{F} = H|\mathcal{D} = H, I)$$

where I is the information given in the problem definition. We do not know which one out of the three coins we are flipping, so the appropriate thing to do is to marginalize with respect to the coin type. Let us number the coins using the discrete random variable $C = 1, 2, 3$. This gives us the following expression for the posterior predictive probability

$$\mathbb{P}(\mathcal{F} = H|\mathcal{D} = H, I) = \sum_{C=1}^{3} \mathbb{P}(\mathcal{F} = H, C|\mathcal{D} = H, I) = \sum_{C=1}^{3} \mathbb{P}(\mathcal{F} = H|C, I)\mathbb{P}(C|\mathcal{D} = H, I),$$

where we used the product rule of probabilities in the last step and that once we have knowledge of C then the probability $\mathbb{P}(\mathcal{F} = H|C, I)$ is conditionally independent of previous coin flips.

The left-hand factor in the last step is a likelihood, and we know that $\mathbb{P}(\mathcal{F} = H|C = 1, I) = 0.75$, $\mathbb{P}(\mathcal{F} = H|C = 2, I) = 0.5$, and $\mathbb{P}(\mathcal{F} = H|C = 3, I) = 0.25$. The right hand factor is a posterior probability for having coin C if observing heads (in the first flip). To compute this, we must use Bayes' theorem

$$\mathbb{P}(C|\mathcal{D} = H, I) = \frac{\mathbb{P}(\mathcal{D} = H|C, I)\mathbb{P}(C|I)}{\mathbb{P}(\mathcal{D} = H|I)}.$$

Evaluating the denominator

$$\mathbb{P}(\mathcal{D} = H|I) = \sum_{C=1}^{3} \mathbb{P}(\mathcal{D} = H|C, I)\mathbb{P}(C|I) = \frac{3}{4}\frac{1}{3} + \frac{1}{2}\frac{1}{3} + \frac{1}{4}\frac{1}{3} = \frac{1}{2}.$$

With this, we have for the three coin posteriors

$$\mathbb{P}(C=1|\mathcal{D}=H,I) = 2\cdot\frac{3}{4}\cdot\frac{1}{3} = \frac{1}{2}.$$

$$\mathbb{P}(C=2|\mathcal{D}=H,I) = 2\cdot\frac{1}{2}\cdot\frac{1}{3} = \frac{1}{3}.$$

$$\mathbb{P}(C=3|\mathcal{D}=H,I) = 2\cdot\frac{1}{4}\cdot\frac{1}{3} = \frac{1}{6}.$$

We can now evaluate the posterior predictive probability that we set out to obtain in the first place

$$\mathbb{P}(\mathcal{F}=H|\mathcal{D}=H,I) = \sum_{C=1}^{3}\mathbb{P}(\mathcal{F}=H|C,I)\mathbb{P}(C|\mathcal{D}=H,I) = \frac{3}{4}\frac{1}{2}+\frac{1}{2}\frac{1}{3}+\frac{1}{4}\frac{1}{6} = \frac{7}{12}\approx 0.58.$$

4.6. *Aside: Bayesian epistemology

Our presentation merits some philosophical remarks on the interpretation of probabilities. We also provide some pointers to developments in Quantum Bayesianism (QBism).

Philosophical remarks on probabilities

The axioms of probability theory are very useful for manipulating probabilities and obtaining quantitative results. There is, however, an ongoing philosophical discussion centered on the question *what is* the probability $\mathbb{P}$? There exist several interpretations of probability, with the two main schools being frequentist probability and Bayesian probability. There are numerous flavors of probability interpretations in each category, most of which are consistent with Kolmogorov's axioms, and so with the sum and product rules. Generally speaking, there is no difference in the calculus of Bayesian and frequentist probabilities. However, their interpretations differ on a fundamental level, as already discussed above. But why even bother to dwell on this topic? Philosophy is sometimes accused of dealing solely with abstractions separated from reality and therefore of being of no use for real-world applications. There is, however, something fundamentally useful about the philosophy of probability in the sense that all events, propositions, and outcomes in social and natural sciences are bearers of probability. Working out what we are manipulating when we manipulate probabilities accordance with the sum and product rules is of fundamental importance to how we think about real systems, and has significant implications for the sorts of questions we do (or do not) ask about them.

Frequentist probability

The frequentist interpretation of probability was developed by John Venn in the second half of the 19th century. He argued that the probability of an event A *is* the relative frequency of its actual occurrence in a series of N experiments, i.e.,

$$\mathbb{P}(A) = \frac{n_A}{N}, \tag{4.15}$$

assuming A happened n_A times. This interpretation is connected with the classical interpretation of probability proposed by Jacob Bernoulli and Pierre-Simon Laplace which dictates an equal distribution of probability among all the possible events in the absence of any evidence otherwise. Obviously, the classical interpretation is applicable in circumstances where all events are equally probable. For example, the classical probability of a fair die landing up with any of the numbers up to (and including) 4 is $4/6$. However, the world consists of more than equally probable events and the frequentist interpretation was designed to remedy this. At first glance, the classical and frequentist interpretations appear to be the same. However, note that with the classical interpretation we counted all possible outcomes *before* any experiment was conducted, whereas in the frequentist interpretation we count *actual* outcomes, and only those. The frequentist interpretation strives to attain some level of objectiveness in the quantification of probability of real-world data.

Consider the coin tossing experiment discussed in Coin tossing: Frequentists and Bayesians. Before the coin has been tossed, there is nothing we can say regarding the frequentist probability of the coin landing heads up, i.e., $\mathbb{P}(H)$, or the opposite, that the coin lands tails up, $\mathbb{P}(T)$. Indeed, there is no data to form a frequency ratio. Once the coin is tossed and it has landed, but we have not seen the answer (collected the data), the probability is either $\mathbb{P}(H) = 1$ or $\mathbb{P}(H) = 0$. The coin has landed either heads or tails up, and once we collect the data we will assign a 0/1 probability. We will contrast this example in the next paragraph where we discuss the Bayesian interpretation of probability. That interpretation allows *you* to form a *belief* about, e.g., $\mathbb{P}(H)$ before the data is collected and even before any coin has ever been tossed; we call this probability a prior. In summary, the frequentist interpretation is firmly grounded in the collection of data, and not much else. The reason for this is that the frequentist interpretation strives for measuring an objective truth. It also is only possible to form probabilities that can be linked to some frequency of events present in a series of some kind. This is not without problems. Try to use this interpretation to quantify the probability that the Sun will rise tomorrow morning or that the Universe is geometrically open. To us physicists, the limited scope of frequentist probabilities place a serious constraint on its usefulness.

Bayesian probability

Bayesian probability dates back to the early 18th century when Thomas Bayes derived a special form of what is nowadays known as Bayes' theorem. After that, Laplace pioneered this branch of probability and established what was then referred to as inverse probability. He combined Bayes' theorem with the principle of indifference, which can be seen as positing a flat prior probability on possible events. Nowadays, the Bayesian interpretation of probability amounts to assigning a graded belief to any proposition or hypothesis. This approach enables probabilities to be applied beyond situations where a frequency can be identified.

So how do we quantify the probability $\mathbb{P}(A|D, I)$ for some event A given data D, and any other information I, if not as a frequency? This is a longer discussion, and to get to the core of that question, let us begin by inspecting Bayes' theorem. Doing so we realize that we must also formulate a prior $\mathbb{P}(A|I)$. As a side remark we mention that in the Bayesian view there is no such thing as an absolute probability, e.g., $\mathbb{P}(A)$, as there is in the frequentist case. Instead *all* probabilities are conditioned on some existing information I. Once the prior for particular information I is formed, and we have collected some data, we can combine the likelihood $\mathbb{P}(D|A, I)$ with the prior using Bayes' theorem to obtain $\mathbb{P}(A|D, I)$. Note that the prior does not necessarily characterize information from the past. Indeed, there is no temporal dimension in probability theory. Bayesian inference can be viewed as a framework for making decisions under uncertainty or incomplete information. The Bayesian paradigm can be applied by a historian trying to infer events from the past based on incomplete records and archives or reaching a verdict in a legal process based on limited evidence and uncertain testimonies.

Regarding the formulation of the prior, we encounter two schools of thought; the *objective* and the *subjective* interpretations of probability. The former interpretation expands on Laplace's principle of indifference and defines probability as a formal system of logic and reasoning in the presence of uncertainty. In this objective approach to Bayesian probability it is essential that the prior probability is assigned consistently with a logical analysis of all prior information in a minimally informative sense, i,e., as objectively as possible. The method of maximum entropy is put forward as one way to achieve this, see The principle of maximum entropy. Indeed, entropy measures the lack of informativeness' of a probability function. So maximizing the entropy consistent with our background knowledge enables one to logically arrive a maximally objective prior probability density (or mass). One can also try to construct a prior density that is invariant with respect to re-parametrization of the model parameters. This is called a Jeffreys prior and its construction follows a well-defined mathematical procedure. These approaches sometimes come with mathematical challenges or the necessity to violate some axioms of probability. Formalizing objective priors is an active field of research and if a method for representing ignorance comes to fruition it will have important consequences for how we should analyze data.

Such striving for objectivity in the prior can be criticized. Indeed, we are seldom in a position where we actually are objective about a scientific proposition–at least if we care about the proposition in the first place. The Bayesian analysis of data entails particular, one might even say "subjective", modelling choices–not everyone has access to the same information. As such, probabilities will always be personal to some extent, they will be conditional on the information I available to the modeler. In an extreme situation, we can have as many probabilities of an event A as there are agents in the world. The subjective interpretation of probability accommodates this stance. This approach does however require that agents are rational in the sense that they obey the axioms of probability. This is sometimes referred to as *coherence*.

One approach to formulating coherent and subjective probabilities regarding some event A follows from a betting analysis. It basically boils down to that *your* degree of belief $\mathbb{P}(A|I)$, based on *your* background knowledge I, is equal to p if you are willing to take odds of $1-p:p$ on A in a betting market, i.e., you decide to gamble $p<1$ dollars with a bookmaker who promises that you will receive your original stake, plus a profit of $1-p$ dollars, back if event A occurs. For example, if you take $3:1$ odds that it will rain on Thursday, the probability that it will rain on Thursday is 0.25. With this translation of probability into betting odds we can define a set of "coherent bets" as the probabilities for a set of events that obey the sum and product rules. Such a set of bets is fair in the sense that it provides no overall advantage to either the bettor or the bookmaker. This betting analogy provides an intuitive and operational definition of subjective probability.

Summary

Probability calculus obeys a small set of reasonable axioms and rests on a well-founded mathematical theory of measures. The fundamental challenge in dealing with probabilities lies in mapping the mathematical measure of probability to events occurring in the real world. To make an analogy, let us consider the measure of length, i.e., the *meter*. The notion of length is somewhat trivial, although an expanded perspective was provided by Einstein. From an abstract point of view, you know how to define a coordinate system in some space with a well-defined inner product etc. There is indeed very little challenge to represent it mathematically. Even so, mankind has refined the operational definition of the *meter* for centuries and applying the measure of length to reality is not so trivial as one might think. The question *what is a meter?* has been given three different answers in the 20th century alone. The question *what is a probability?* is far more important than the corresponding question for a meter, since it is the *metre stick* we use to quantify uncertainty and as such it is a corner stone of the scientific method. Yet, we have still not settled on a philosophical position that encompasses all uncertainty, nor do we know if one such position exists. Any progress in this direction will provide an important advance in our understanding of the world.

Discuss

> How would you respond to the following statement: ‘As scientists, we should be concerned with objective knowledge rather than subjective belief.’

> **One view**
>
> Any scientific analysis contains subjective knowledge. Indeed, we always make assumptions during the analysis, and your assumptions will be based on your prior domain knowledge, which is not necessarily equal to others’ domain knowledge. When performing Bayesian inference we must always state these subjective beliefs and assumptions in the priors. This transparency is welcome, and we should always do our best to report to what extent the inferences are sensitive to a particular choice of prior.

Quantum Bayesianism (QBism)

From the Wikipedia article on QBism: “In physics and the philosophy of physics, quantum Bayesianism is a collection of related approaches to the interpretation of quantum mechanics, of which the most prominent is QBism (pronounced “cubism”). QBism is an interpretation that takes an agent’s actions and experiences as the central concerns of the theory.”

QBism references

Here are some references:

- David Mermin columns from Physics Today (2012) and response by Robert Griffiths:
  - “Quantum mechanics: Fixing the shifty split”
  - Robert Griffiths response and other responses
  - Mermim replies
- Scientific American short article from 2013: “Can Quantum Bayesianism Fix the Paradoxes of Quantum Mechanics?”
- Several works by Christopher Fuchs or interviews with him:
  - From Quanta magazine A Private View of Quantum Reality
  - From Discover magazine: “Quantum Physics is No More Mysterious Than Crossing the Street: A Conversation with Chris Fuchs”
  - “Interview with a Quantum Bayesian”
  - “QBism, the Perimeter of Quantum Bayesianism”
  - Fuchs and Schack on Quantum-Bayesian coherence
- Critical view of QBism by Guido Bacciagaluppi from 2013.

5. More on PDFs

In this chapter we collect material addressing various aspects of probability distributions (PDFs).

- The posteriors we will encounter will in general be multi-dimensional. We first consider some aspects of one-dimensional (1D) posteriors in One-dimensional PDFs and two-dimensional (2D) posteriors in Two-dimensional PDFs. There is a follow-up exercise notebook for exploring 1D and 2D PDFs using Python libraries with the other part I exercises (📥 Problem: Exploring PDFs).

- Bayesian credible intervals provides further details on defining Bayesian credible intervals and on the relationship to frequentist confidence intervals.
- In The Central Limit Theorem we take a first look at the central limit theorem (CLT), including a proof and several visualizations.

5.1. One-dimensional PDFs

To a Bayesian, everything is a PDF

As we noted in Section 4.3, physicists are used to dealing with PDFs as normalized wave functions squared. For a one-dimensional particle, the probability density at x is

$$|\Psi(x)|^2 \implies p(x)$$

The *probability* of finding x in some interval $a \leq x \leq b$ is found by integration:

$$\mathbb{P}(a \leq x \leq b) = \int_a^b |\Psi(x)|^2 \, dx$$

Just as with "Lagrangian" vs. "Lagrangian density", physicists are not always careful when saying probability vs. probability density.

Physicists are also used to multidimensional normalized PDFs as wave functions squared, e.g. probability density for particle 1 at x_1 and particle 2 at x_2:

$$|\Psi(x_1, x_2)|^2 \implies p(x_1, x_2) \equiv p(\mathbf{x}) \quad \text{with} \quad \mathbf{x} \equiv \{x_1, x_2\} \tag{5.1}$$

(Note that you will find alternative notation in the physics and statistics literature for generic PDFs: $p(\mathbf{x}) = P(\mathbf{x}) = \text{pr}\,(\mathbf{x}) = \text{prob}\,(\mathbf{x}) = \ldots$) We will consider two-dimensional PDFs as in (5.1) in Section 5.2.

Some other vocabulary and definitions to remember

- $p(x_1, x_2)$ is the *joint probability density* of x_1 and x_2.
- The *marginal probability density* of x_1 is: $p(x_1) = \int p(x_1, x_2)\, dx_2$. In quantum mechanics, this corresponds to the probability to find particle 1 at x_1 and particle 2 anywhere; that is, $\int |\Psi(x_1, x_2)|^2 dx_2$ (integrated over the full domain of x_2, e.g., 0 to ∞).
- "Marginalizing" = "integrating out" (eliminates from the posterior the "nuisance parameters" whose value you don't care about, at least not at that moment).

In Bayesian statistics there are PDFs (or PMFs if discrete) for experimental *and* theoretical uncertainties, model parameters, hyperparameters (what are those?), events ("Will it rain tomorrow?"), etc. Even if x has the definite value x_0, we can represent this knowledge with a PDF $p(x) = \delta(x - x_0)$.

Characteristics of PDFs

What are the characteristics (e.g., symmetry, heavy tails, …) of different PDFs: normal, beta, student t, χ^2, . . .

Answer

Check these yourself! Note that the answers will often depend on the parameter values for the distribution. A Student t distribution may have “heavy tails” (meaning more probability in the tails than a Gaussian would have) for some parameter values but for others it approaches a normal distribution (so by definition no heavy tails).

Sampling

What does sampling mean?

Answer

To sample a given distribution is to draw values that represent it. The probability to draw a specific value is given by the distribution. In Bayesian inference one is most often sampling a posterior distribution.

Verifying sampling

How do you know if a distribution is correctly sampled?

Answer

One way is to look at the (normalized) histogram. If correctly sampled, this should approximate the distribution and the approximation should improve with more samples.

Widget for exploring common one-dimensional PDFs

Notes on the controls for the widget:

- Use the PDF pulldown menu to switch between some standard distributions.
- The parameters for each distribution can differ. See the SciPy Statistical functions page for function definitions that show how these parameters influence the shapes of the PDFs.
- When using the sliders, you can use the left and right keyboard arrows to make small adjustments.
- If the *overlay normal with same mean/std* checkbox is selected, a Gaussian distribution with the same mean and standard deviation as the current PDF will be superposed.
- Note that you can create more horizontal space for the widget by toggling the three-line icon at the top left of the page frame.

Interactive PDF Explorer

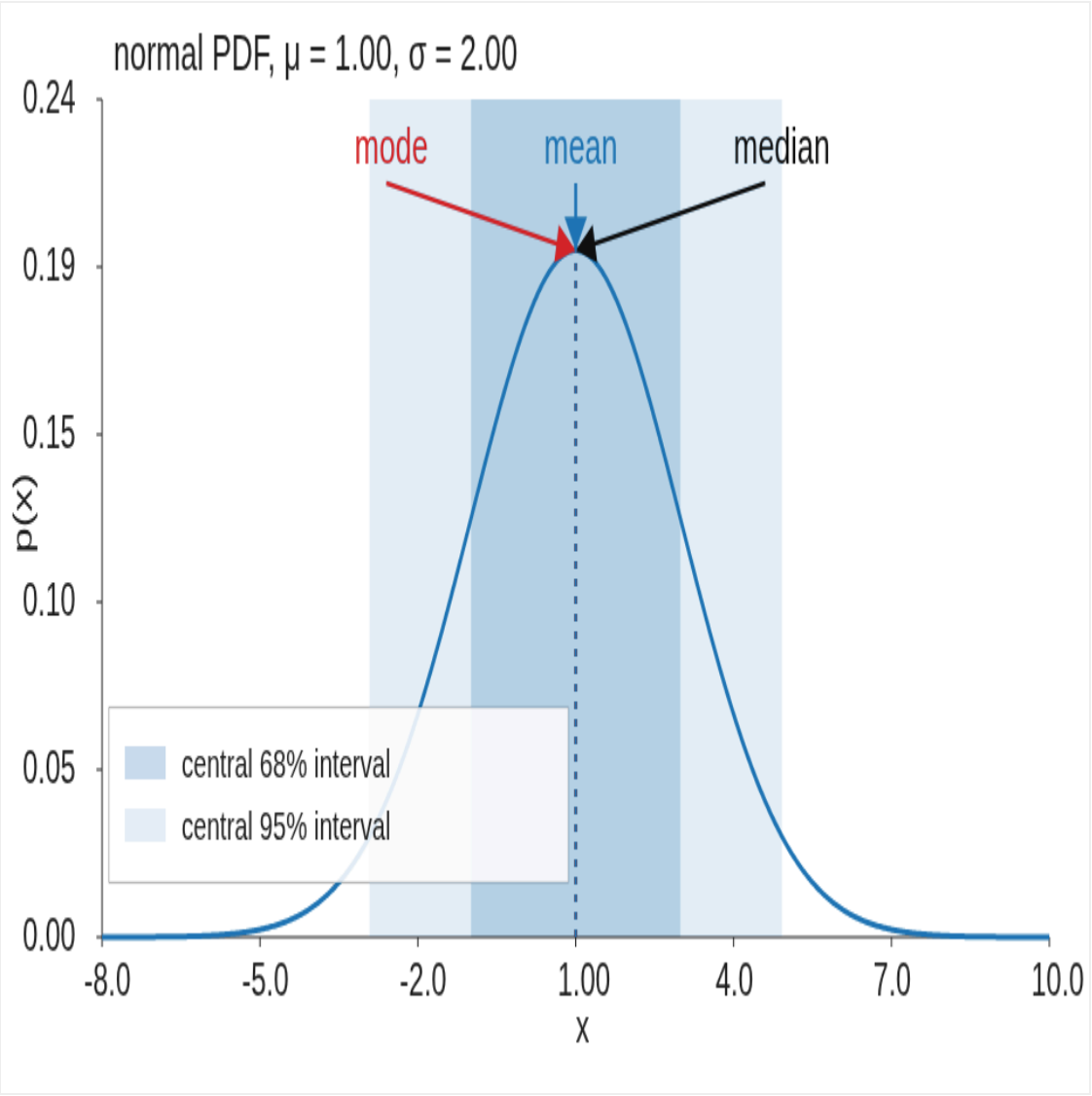
PDF: Normal μ 1.00

σ 2.00

overlay normal with same mean/std

show mean/median/mode show 68%/95% intervals

mean = 1.00; median = 1.00; mode = 1.00; central 68% = [-0.99, 2.99]; central 95% = [-2.92, 4.92]



Invariant PDFs

Why do many of the PDFs appear unchanged when you change their parameters?

Answer

It is because the plots are autoscaled. If you are changing a location (`loc`) parameter (such as μ for a normal PDF), this will just translate the PDF on the x-axis without changing the shape or the y-axis values. If you are changing a scale parameter (such as σ for a normal PDF), this will change both the y-axis values and the x-axis values without changing the shape.

Matching a Student's t distribution to a normal distribution

When you switch to the Student's t PDF and check `overlay normal with same mean/std`, a normal distribution is not seen. Why not?

Answer

When you first switch to the t distribution, the value of ν is 2. For $\nu \leq 2$, the variance of the distribution is infinite. So there is not a finite Gaussian PDF to draw! Try changing ν to 2.01 (or anything higher) while the box is checked.

χ^2 distribution

Many physicists learn to judge whether a fit of a model to data is good or to pick out the best fitting model among several by evaluating the χ^2/dof (dof = "degrees of freedom") for a given model and comparing the result to one. Here χ^2 is the sum of the squares of the residuals (data minus model predictions) divided by variance of the error at each point:

$$\chi^2 \equiv \sum_{i=1}^{N_{\text{data}}} \frac{\left(y_i - f(x_i; \hat{\boldsymbol{\theta}})\right)^2}{\sigma_i^2},$$

where y_i is the i^{th} data point, $f(x_i; \hat{\boldsymbol{\theta}})$ is the prediction of the model for that point using the best fit for the parameters, $\hat{\boldsymbol{\theta}}$, and σ_i is the error bar for that data point. The degrees-of-freedom (dof), often denoted by ν, is the number of data points minus number of fitted parameters:

$$\nu = N_{\text{data}} - N_{\text{fit parameters}}.$$

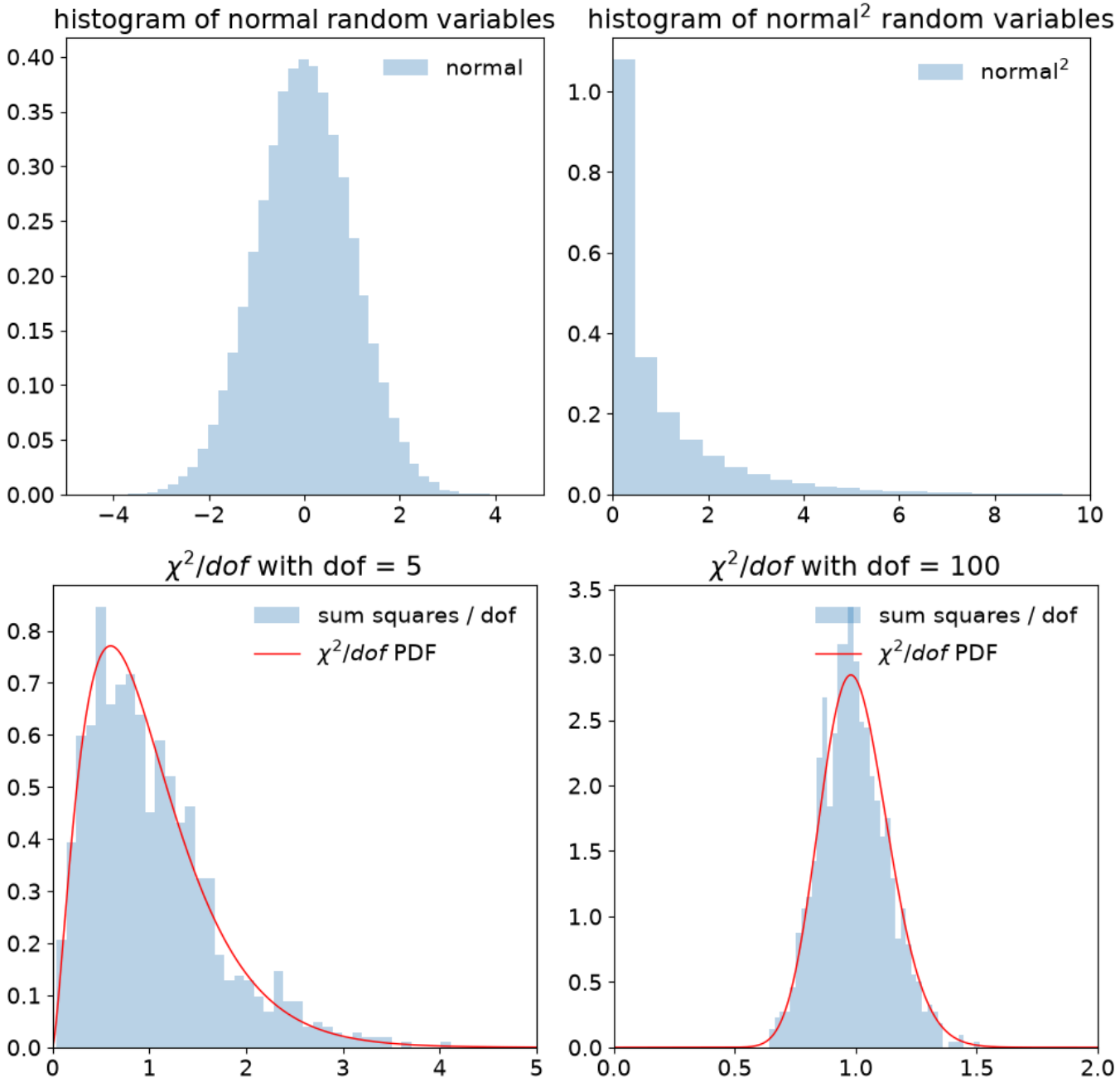
The rule of thumb is generally that $\chi^2/\text{dof} \gg 1$ means a poor fit and $\chi^2/\text{dof} < 1$ indicates overfitting.

We will consider this from a Model selection perspective in Frequentist hypothesis testing in Chapter 14. Here we will just consider the nature of the χ^2 distribution.

Test the sum of normal variables squared

The χ^2 distribution is supposed to result from the sum of the squares of Gaussian random variables. Here we test it numerically for two values of the degrees of freedom (dof). (You can try other cases by copying the code into a Jupyter notebook.) The Gaussian (normal) random variables are drawn from a standard normal distribution, i.e., with mean $\mu = 0$ and $\sigma = 1$.

▼ **Hide code cell source**



Check whether the distributions (which are of χ^2/ν) are consistent with the statement above that the mean of a χ^2 distribution with $k = \nu$ dofs is ν, with variance 2ν.

The near ubiquity of Gaussians

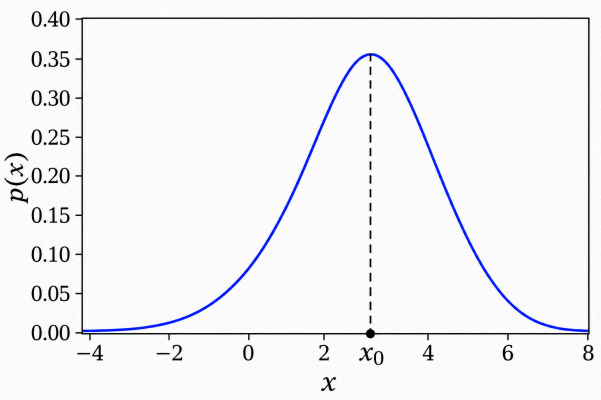
We will see a lot of Gaussian distributions in this course. Some people implicitly *always* assume Gaussian distributions. So this seems like a good place to pause and consider why Gaussian distributions are such a common choice. It turns out there are several reasons why one might choose a Gaussian to describe a probability distribution. Our first reason is closely related to the observation that leads to small amplitude oscillations in mechanics being harmonic. We will encounter two more reasons in The Central Limit Theorem and The principle of maximum entropy.

The Gaussian is to statistics what the harmonic oscillator is to mechanics

Suppose we have a probability distribution $p(x|D,I)$ that is unimodal (has only one hump), then one way to form a "best estimate" for the variable x is to compute the maximum of the distribution. (To save writing we denote the PDF of interest as $p(x)$ for a while hereafter.)

We find this point, which we'll denote by x_0, using calculus:

$$\left.\frac{dp}{dx}\right|_{x=x_0} = 0 \quad \text{with} \quad \left.\frac{d^2p}{dx^2}\right|_{x=x_0} < 0 \text{ (maximum)} \quad .$$



To characterize the posterior $p(x)$, we look nearby. We want to know how sharp this maximum is: is $p(x)$ sharply peaked around $x = x_0$ or is the maximum kind-of shallow? To work this out we'll do a Taylor expansion around $x = x_0$. The PDF $p(x)$ itself varies very rapidly, but since $p(x)$ is positive definite we can Taylor expand $\log p$ instead. (See the box below on "p or $\log p$?" for a strict mathematical reason why it's problematic for our purposes to directly Taylor expand $p(x)$ around its maximum.)

$$\implies L(x) \equiv \log p(x) = L(x_0) + \left.\frac{dL}{dx}\right|_{x=x_0}(x-x_0) + \frac{1}{2}\left.\frac{d^2L}{dx^2}\right|_{x=x_0}(x-x_0)^2 + \cdots$$

Note that $\left.\frac{dL}{dx}\right|_{x=x_0} = 0$, because $L(x_0)$ is a maximum if $p(x_0)$ is, and $\left.\frac{d^2L}{dx^2}\right|_{x=x_0} < 0$. If we can neglect higher-order terms, then when we re-exponentiate,

$$p(x) \approx A\, e^{\frac{1}{2}\left.\frac{d^2L}{dx^2}\right|_{x=x_0}(x-x_0)^2},$$

with A a normalization factor that originates in the $L(x_0)$ term. So in this general circumstance we get a Gaussian distribution:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/\sigma^2} \quad \implies \quad \mu = x_0,\ \sigma = \left(-\left.\frac{d^2L}{dx^2}\right|_{x=x_0}\right)^{-1/2},$$

where we see the importance of the second derivative being negative.

We usually characterize the distribution by $x_0 \pm \sigma$ (e.g., with a point and symmetric error bars if this is data noise), because *if* it is a Gaussian this is *sufficient* to tell us the entire distribution. E.g., n standard deviations is $n \times \sigma$. But for a Bayesian, the full posterior $p(x|D, I)$ for all x is the general result, and $x = x_0 \pm \sigma$ may be only an approximate characterization.

To think about …

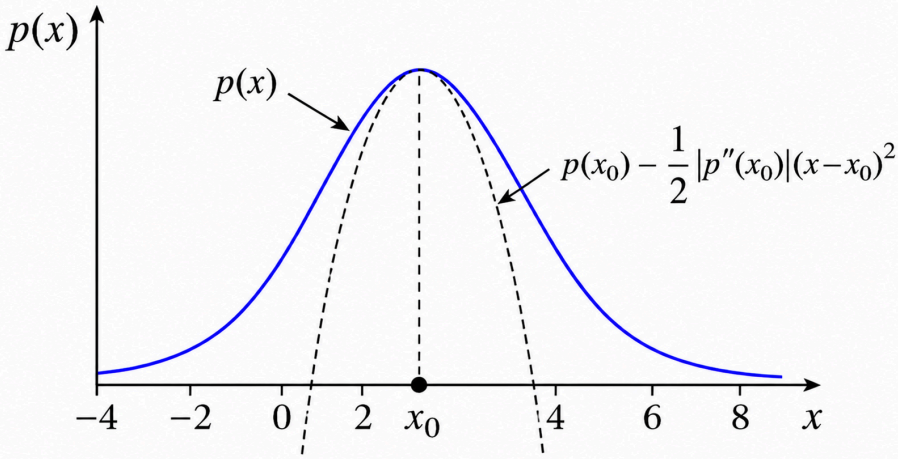
What if $p(x)$ is asymmetric? What if it is multimodal?

p or $\log p$?

We motivated Gaussian approximations from a Taylor expansion to quadratic order of the *logarithm* of a PDF. What would go wrong if we directly expanded the PDF? Well, if we do that we get:

$$p(x) \approx p(x_0) + \frac{1}{2}\frac{d^2p}{dx^2}\bigg|_{x=x_0}(x-x_0)^2 \overset{x\pm\to\infty}{\longrightarrow} -\infty,$$

i.e., we get something that diverges as x tends to either plus or minus infinity.



A PDF must be normalizable and positive definite, so this approximation violates these conditions!

5.2. Two-dimensional PDFs

Visualizing correlated Gaussian distributions

A multivariate Gaussian distribution for N dimensional $\boldsymbol{x} = \{x_1, \ldots, x_N\}$ with $\boldsymbol{\mu} = \{\mu_1, \ldots, \mu_N\}$, with positive-definite *covariance matrix* Σ is

$$p(\boldsymbol{x}|\boldsymbol{\mu}, \Sigma) = \frac{1}{\sqrt{\det(2\pi\Sigma)}} e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\intercal \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}$$

For the one-dimensional case, it reduces to the familiar

$$p(x_1|\mu_1, \sigma_1) = \frac{1}{\sqrt{2\pi\sigma_1^2}} e^{-\frac{(x_1-\mu_1)^2}{2\sigma_1^2}}$$

with $\Sigma = \sigma_1^2$.

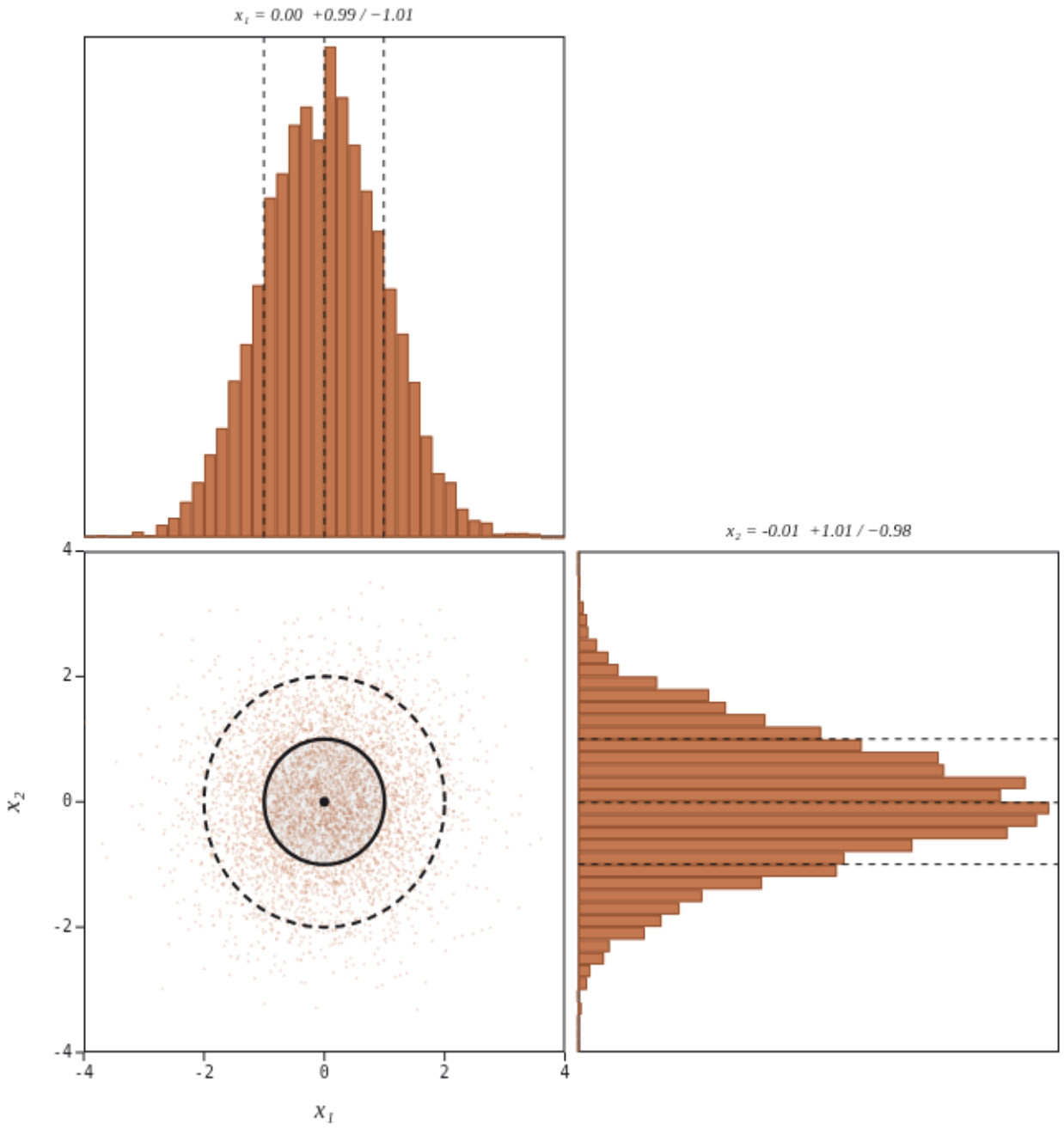
For the bivariate case (two dimensional),

$$\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad \text{and} \quad \boldsymbol{\mu} = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix} \quad \text{and} \quad \Sigma = \begin{pmatrix} \sigma_1^2 & \rho_{12}\sigma_1\sigma_2 \\ \rho_{12}\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix} \quad \text{with} \quad 0 < \rho_{12}^2 < 1 \tag{5.2}$$

and Σ is positive definite.

Widget user interface features:

- Set the mean position (μ_1, μ_2) and variances $(\Sigma_{11}, \Sigma_{22})$ with the sliders
- Set the correlation ρ_{12} with the slider. This controls the covariance $\Sigma_{12} = \rho_{12}\sqrt{\Sigma_{11}\Sigma_{22}}$.
- Four presets are available.
- The corner plot shows samples from the bivariate PDF and histograms for the two marginal distributions. Control the number of samples with the slider.
- Dashed lines on the marginals mark the 16th, 50th, and 84th percentiles. This is equivalent to $\pm 1\sigma$ for a one-dimensional Gaussian.



The solid and dashed ellipses in the joint panel are iso-probability levels. These correspond to fixed values of the squared Mahalanobis distance

$$\Delta^2 = (\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}).$$

In the widget they are drawn at $\Delta = 1$ and $\Delta = 2$. Their semi-axes are aligned with the eigenvectors of Σ and have lengths $\sqrt{\lambda_i}$ (the "1σ" ellipse) and $2\sqrt{\lambda_i}$ (the "2σ" ellipse), where λ_i are the eigenvalues of Σ.

A subtlety worth highlighting

In one dimension the $\pm 1\sigma$ and $\pm 2\sigma$ intervals contain 68.3% and 95.4% of the probability mass. In two dimensions, the corresponding *ellipses* contain considerably less, because $\Delta^2 \sim \chi^2_2$ and

$$\mathbb{P}(\Delta^2 \leq k^2) = 1 - e^{-k^2/2}.$$

Ellipse	Probability mass (2D)	For comparison: 1D interval
$\Delta = 1$ ("1σ")	39.3%	68.3%
$\Delta = 2$ ("2σ")	86.5%	95.4%
$\Delta = 3$ ("3σ")	98.9%	99.7%

Questions to consider:

1. *What does "positive definite" mean and why is this a requirement for the covariance matrix Σ?*

   Answer

   A symmetric matrix (such as a covariance matrix) is positive definite if all of its eigenvalues are greater than zero. This ensures that

   - the variance of any linear combination of random variables is non-negative;
   - that no variable is a linear combination of others (no collinear variables);
   - the covariance matrix is invertible.

2. *What is plotted in each part of the graph (called a "corner plot")?*
3. *What effect does changing μ_1 or μ_2 have?*
4. *What effect does changing Σ_{11} or Σ_{22} have? What if the scales for x_1 and x_2 were the same?*
5. *What happens if ρ_{12} is equal to 0 then $+0.7$ then -0.7.*
6. *What would happen if you were allowed to set $|\rho_{12}| \leq 1$? Explain what goes wrong.*
7. *So what characterizes independent (uncorrelated) variables versus positively correlated versus negatively correlated?*

2D PDF with a quadratic approximation

Consider a general two-dimensional log likelihood $L(X, Y)$. We'll analyze it in a quadratic approximation. We base our discussion on Sivia section 3.3 [SS06].

First, find the mode X_0, Y_0 (which is the best estimate) by differentiating

$$\begin{aligned} L(X, Y) &= \log p(X, Y | \{\text{data }\}, I) \\ \implies \quad & \left.\frac{dL}{dX}\right|_{X_0, Y_0} = 0, \; \left.\frac{dL}{dY}\right|_{X_0, Y_0} = 0 \end{aligned}$$

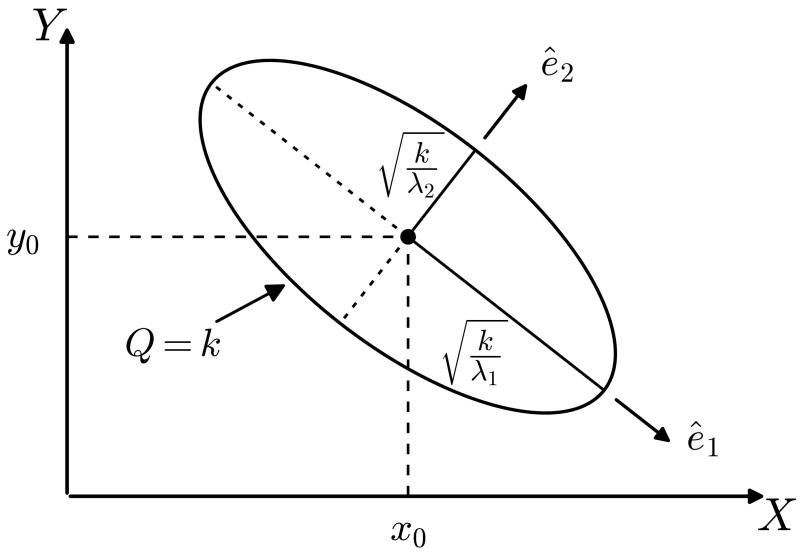
To check reliability, Taylor expand around $L(X_0, Y_0)$:

$$
\begin{aligned}
L &= L(X_0, Y_0) + \frac{1}{2}\left[\left.\frac{\partial^2 L}{\partial X^2}\right|_{X_0,Y_0}(X - X_0)^2 + \left.\frac{\partial^2 L}{\partial Y^2}\right|_{X_0,Y_0}(Y - Y_0)^2\right. \\
&\qquad \left. + 2\left.\frac{\partial^2 L}{\partial X \partial Y}\right|_{X_0,Y_0}(X - X_0)(Y - Y_0)\right] + \ldots \\
&\equiv L(X_0, Y_0) + \frac{1}{2}Q + \ldots
\end{aligned}
$$

It makes sense to do this in (symmetric) matrix notation:

$$
Q = \begin{pmatrix} X - X_0 & Y - Y_0 \end{pmatrix} \begin{pmatrix} A & C \\ C & B \end{pmatrix} \begin{pmatrix} X - X_0 \\ Y - Y_0 \end{pmatrix} \tag{5.3}
$$

$$
\implies A = \left.\frac{\partial^2 L}{\partial X^2}\right|_{X_0,Y_0}, \quad B = \left.\frac{\partial^2 L}{\partial Y^2}\right|_{X_0,Y_0}, \quad C = \left.\frac{\partial^2 L}{\partial X \partial Y}\right|_{X_0,Y_0}
$$



So in a quadratic approximation, the contour $Q = k$ for some k is an ellipse centered at X_0, Y_0 (as in the figure). The orientation and eccentricity of the ellipse are determined by A, B, and C. The principal axes are found from the eigenvectors of the Hessian matrix $\begin{pmatrix} A & C \\ C & B \end{pmatrix}$:

$$
\begin{pmatrix} A & C \\ C & B \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \lambda \begin{pmatrix} x \\ y \end{pmatrix} \quad \implies \quad \lambda_1, \lambda_2 < 0 \text{ so } (X_0, Y_0) \text{ is a maximum}
$$

Conditions on A, B, C

The condition $\lambda_1, \lambda_2 < 0$ for the point (X_0, Y_0) to be a maximum implies that we must have

$$A < 0, \qquad B < 0, \qquad AB > C^2.$$

If the major and minor axes of the ellipse are aligned with the x-axis and y-axis (so $C = 0$), the analysis is simple: the eigenvalues are A and B and the error-bars (standard deviations) for X and Y will be inversely proportional to the modulus of their square roots. What if the ellipse is skewed?

If we compare Q in (5.3) to the bivariate Gaussian exponent (5.2), we can identify the covariance matrix (4.8) in terms of the inverse of Q:

$$\Sigma_{XY} = \begin{pmatrix} \sigma_X^2 & \sigma_{XY}^2 \\ \sigma_{XY}^2 & \sigma_Y^2 \end{pmatrix} = -\begin{pmatrix} A & C \\ C & B \end{pmatrix}^{-1} = \frac{1}{AB - C^2} \begin{pmatrix} -B & C \\ C & -A \end{pmatrix},$$

from which we can read off the variances and covariance in terms of A, B, C. When $C = 0$, the covariance $\sigma_{XY} = 0$ and the values of X and Y are uncorrelated. As $|C|$ increases from zero, the ellipse becomes increasingly skewed and elongated; this corresponds to greater correlation (with $C > 0$ correlated and $C < 0$ anti-correlated).

5.3. Bayesian credible intervals

Defining credible intervals

For a one-dimensional posterior that is *symmetric*, it is clear how to define the $d\%$ confidence interval. The algorithm is: start from the center, step outward on both sides, stop when $d\%$ is enclosed. For a two-dimensional posterior, we need a way to integrate from the top. (One approach is to lower a plane, as described below for HPD.)

What if the distribution is asymmetic or multimodal? Here are two possible choices.

- **Equal-tailed interval (ETI)**, also known as central interval: define the boundaries of the interval so that the area above and below the interval are equal.
- **Highest posterior density (HPD) region**, also known as highest density region (HDR): the posterior density for every point is higher than the posterior density for any point outside the interval. E.g., lower a horizontal line over the distribution until the desired interval percentage is covered by regions where the distribution is above the line.

Here is a widget showing how ETI and HPD are applied for a variety of distributions. Some notes on the widget:

- Collapse the left table of contents by toggling the three line icon at the upper left of this middle frame.
- The starting distribution is bimodal, with a 90% credible interval. In this case, the ETI has 5% of the area above and below the blue vertical dashed lines. Note that the HPD leads to two disjoint regions.
- Toggle the checkboxes to see the ETI and HPD regions individually.
- The slider lets you change the size of the credible interval.
- The pulldown menu gives a variety of choices for distributions.

To do: *Play with the widget and devise algorithms that can be coded in Python for determining the ETI and HPD regions for a given credible interval percentage.*

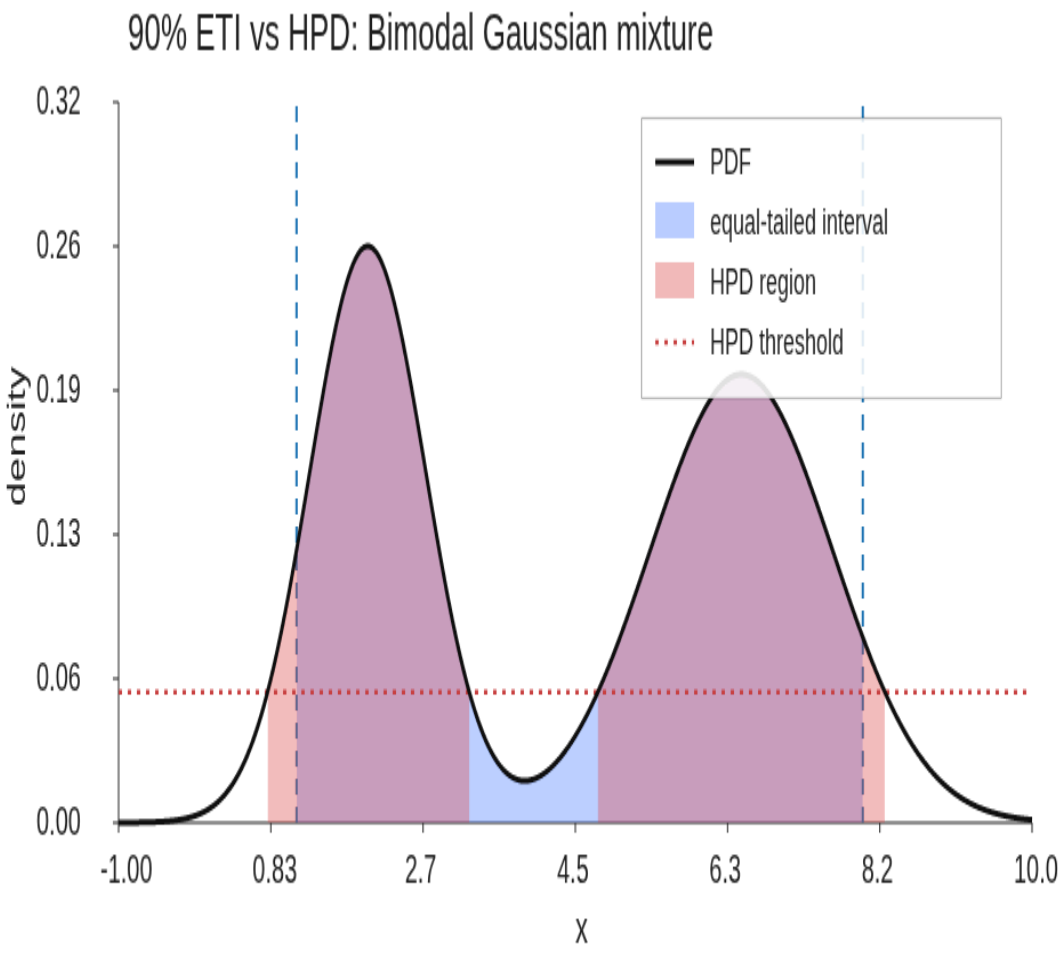
Equal-Tailed Intervals vs Highest Posterior Density Regions

distribution: Bimodal Gaussian mixture

credible level: 90% show ETI

show HPD

90% ETI = [1.14, 7.96]; 90% HPD = [0.80, 3.22] ∪ [4.77, 8.23]; HPD density threshold = 0.0581.



See Section 40.3 for further discussion of credible regions.

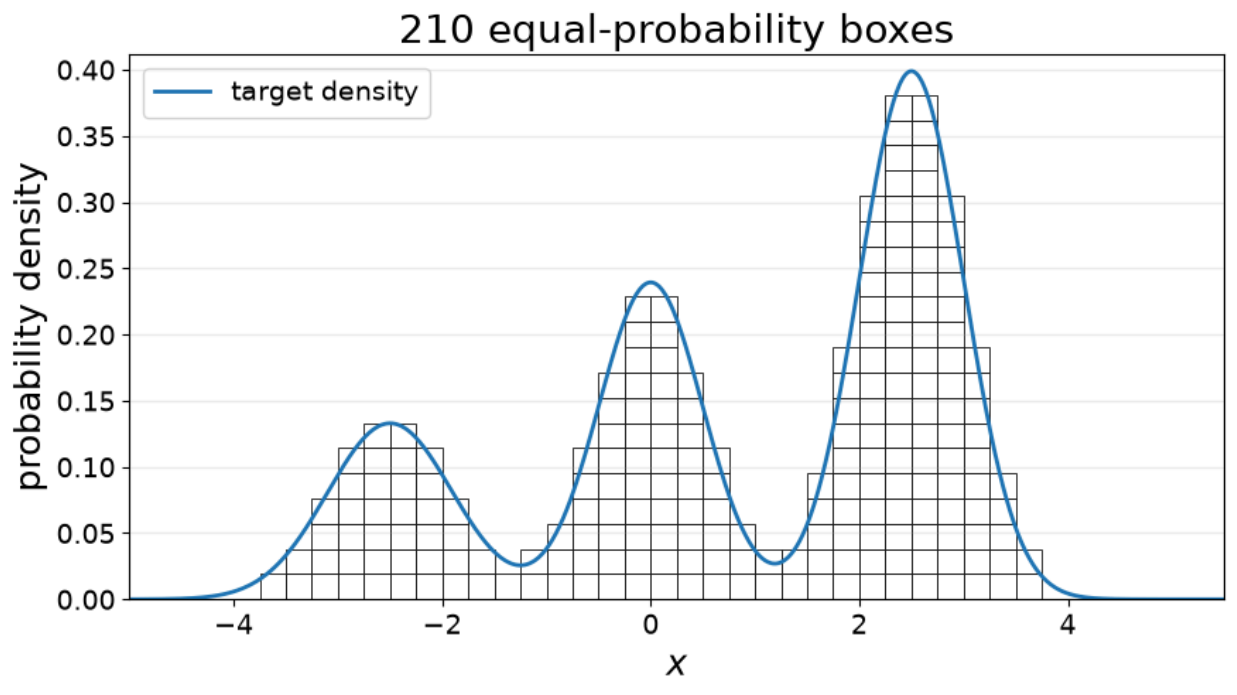
Try a couple of checkpoint questions

The following two checkpoint questions show one-dimensional multi-modal PDFs. The smooth curves are approximated by discrete boxes, all of which have the same probability. Your task is to establish the intervals for 68% credible intervals using both the equal-tailed interval (ETI) and highest posterior density (HPD) conventions. You can do this by counting boxes. The box width is 0.25 so your intervals should be of the form (-3.25, 1.50). Note that you may have disconnected intervals.

Because of the discrete size of the boxes, you won't get precisely 68%. Choose your interval to be the closest to 68%, but on the higher side. There is code available that you can inspect to see how the intervals were calculated here and you can copy it to a Jupyter notebook to change the distributions.

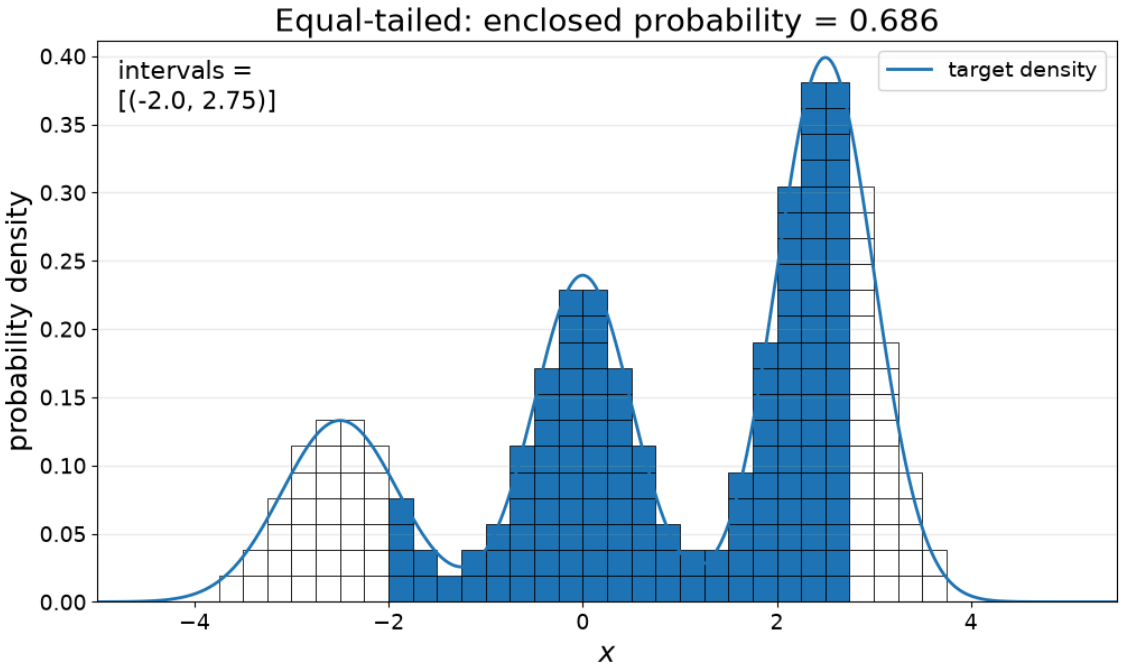
▼ **Hide code cell source**

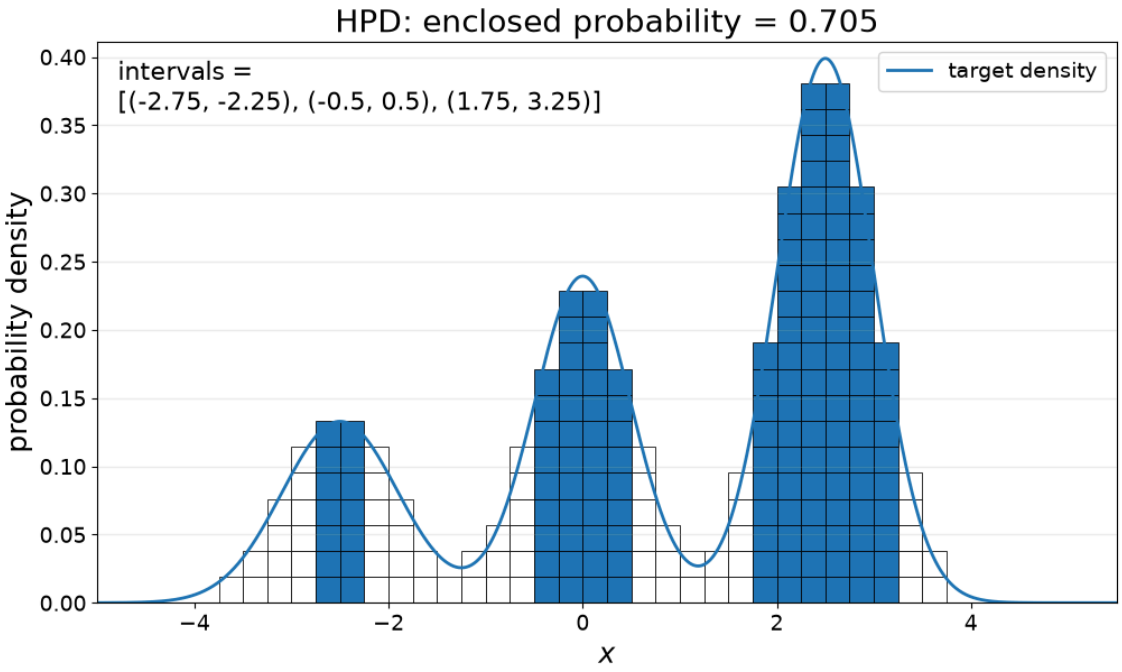
Checkpoint Question



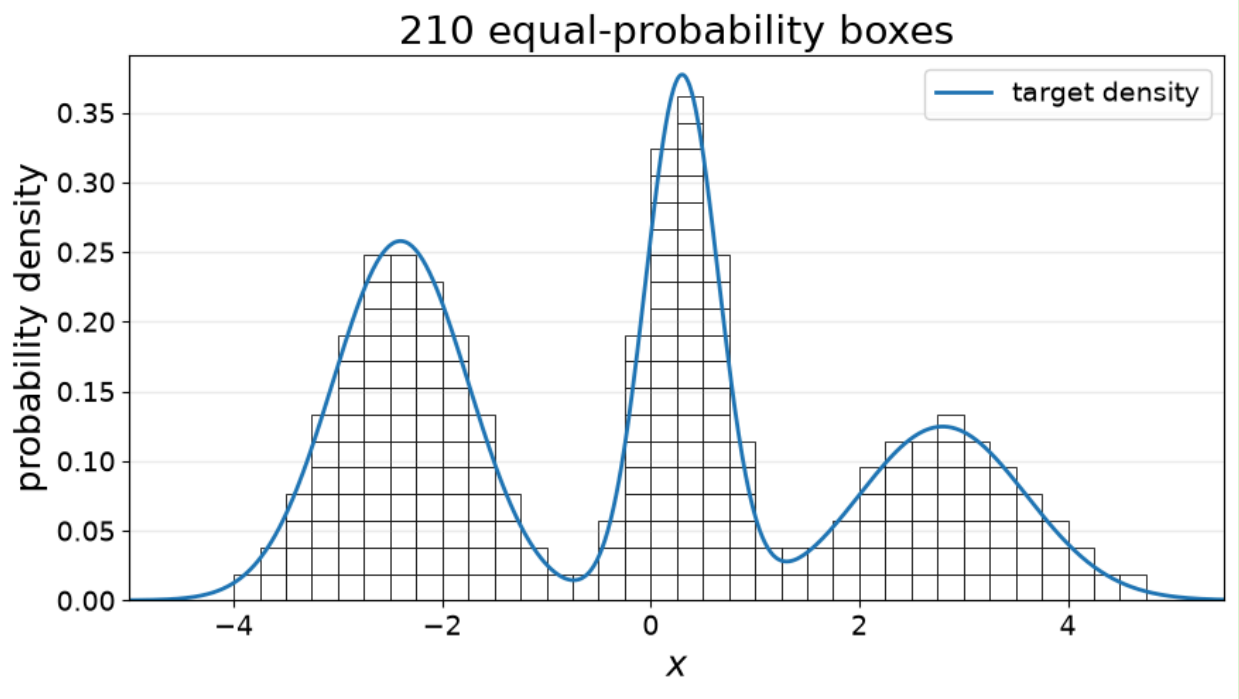
Identify the ETI and HPD credible intervals.

Answer



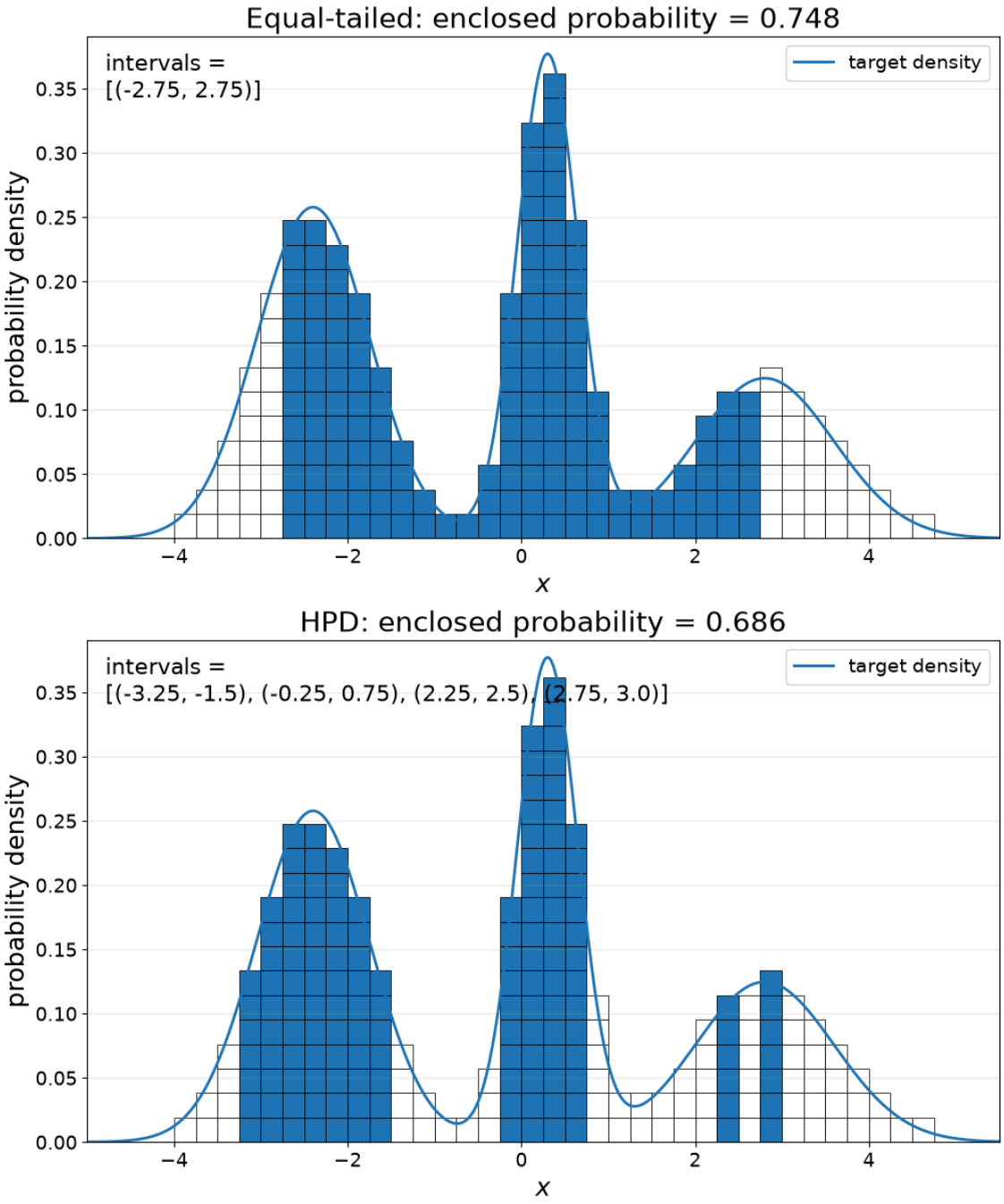


Checkpoint Question



Identify the ETI and HPD credible intervals.

Answer



Bayesian credible intervals and frequentist confidence intervals

There are important differences between the Bayesian 68% credible (DoB) interval for the most likely value and a frequentist 1σ confidence interval.

The first point is that $1\sigma = 68\%$ assumes a Gaussian distribution around the maximum of the posterior. While this will often work out to be roughly correct, it may not. And, as we seek to translate $n\sigma$ intervals into DoB statements, assuming a Gaussian becomes more and more questionable the higher n is. (*Why?*)

But the second point is more philosophical (meta-statistical?). One interval is a statement about $p(x|D,I)$, while the other is a statement about $p(D|x,I)$. (Note that because the conversion between the two PDFs requires the use of Bayes' theorem, the Bayesian interval may be affected by the choice of the prior.)

The Bayesian version of a confidence interval is easy; a 68% credible interval or degree-of-belief (DoB) interval is: given some data and some information I, there is a 68% chance (probability) that the interval contains the true parameter.

On the other hand, the frequentist 68% confidence interval is trickier: assuming the model (contained in I) and the value of the parameter x, then if we do the experiment a large number of times then 68% of them will produce data in that interval. So the *parameter* is fixed (no PDF) and the confidence interval is a statement about *data*. Frequentists will try to make statements about parameters, but they end up a bit tangled, e.g., "There is a 68% probability that when I compute a confidence interval from data of this sort that the true value of θ will fall within the (hypothetical) space of observations."

5.4. The Central Limit Theorem

Another reason a Gaussian PDF emerges in many situations is because the Central Limit Theorem (CLT) states that the sum of random variables drawn from all (or almost all) probability distributions will eventually produce Gaussians if the number of samples in each sum is large enough.

Central Limit Theorem

The sum of n random values drawn from any PDF of finite variance σ^2 tends as $n \to \infty$ to be Gaussian distributed about the expectation value of the sum, with variance $n\sigma^2$.

Consequences of the CLT:

- The mean of a large number of values becomes normally distributed *regardless* of the probability distribution the values are drawn from.
- The binomial, Poisson, chi-squared, and Student's t- distributions all approach Gaussian distributions in the limit of a large number of degrees of freedom (e.q., for large n for binomial). This consequence is probably not immediately obvious to you from the statement of the CLT! (See below for further details on the Poisson distribution.)

Checkpoint Question

Why would we expect the CLT to work for a binomial distribution?

Hint

If we denote Bin(n,p) as the binomial distribution for n trials with probability p of success, how is Bin($1,p$) related to Bin(n,p)?

Answer

If we add up n random variables from Bin($1,p$), each with value 0 or 1, this is equivalent to a Bin(n,p) random variable with the same number of successes. So we already have a sum of random variables

built in and the CLT will apply. (In more detail: getting k ones and $n-k$ zeros from the n Bin$(1,p)$ draws will have probability p^k times $(1-p)^{n-k}$ times the number of combinations $\binom{n}{k}$. This is the same as the binomial probability for k successes.)

Moments of a PDF from a sum of random variables

A Gaussian distribution is completely determined by its first and second moments (that is, its mean μ and variance σ^2). This is obvious if we know the distribution is a Gaussian, because its parametrization is fully specified given μ and σ^2. But this implies that all of the higher moments of a Gaussian distribution are fully determined by μ and σ. And if we find a distribution with the same pattern of higher moments, it must be a Gaussian. In this subsection we see how this observation leads us toward the CLT by returning to the analyses in Expectation values and moments.

It will be sufficient here (and much cleaner) to work with mean-zero distributions. The moments $\{m_k\}_{\mathcal{N}(0,\sigma^2)}$ of a mean-zero Gaussian distribution, which will be our target, can be found directly from the moment-generating function in (4.9):

$$\{M_X(t)\}_{\mathcal{N}(0,\sigma^2)} = \exp(\frac{1}{2}\sigma^2 t^2) = \sum_{j=0}^{\infty} \frac{1}{j!}\left(\frac{\sigma^2 t^2}{2}\right)^j,$$

which means for $X \sim \mathcal{N}(0,\sigma^2)$ we find

$$\{m_k\}_{\mathcal{N}(0,\sigma^2)} = \mathbb{E}_{\mathcal{N}(0,\sigma^2)}[X^k] = \begin{cases} 0, & \text{if } k \text{ odd} \\ (2j-1)!!\sigma^{2j} & \text{if } k = 2j \text{ even} \end{cases} = \begin{cases} 0, & \text{if } k \text{ odd} \\ \frac{k!}{2^{k/2}(k/2)!}\sigma^k & \text{if } k \text{ even} \end{cases}, \tag{5.4}$$

where $(2j-1)!! = (2j-1)\cdot(2j-3)\cdots 3\cdot 1$. Then all the odd moments are zero and $\{m_2\}_{\mathcal{N}(0,\sigma^2)} = \sigma^2$, $\{m_4\}_{\mathcal{N}(0,\sigma^2)} = 3\sigma^4$, $\{m_6\}_{\mathcal{N}(0,\sigma^2)} = 15\sigma^6$, and so on.

With those results in hand, we consider any distribution with mean zero and a finite variance σ^2. We consider a sum of n random variables, all drawn independently from that same mean-zero (but not necessarily symmetric) distribution (hence they are iid), scaled by an overall $1/\sqrt{n}$ factor:

$$X = \frac{1}{\sqrt{n}}(x_1 + x_2 + \cdots + x_n) = \sum_{j=1}^{n} \frac{x_j}{\sqrt{n}}. \tag{5.5}$$

Let's consider moments $m_k = \mathbb{E}[X^k]$ of X as n gets large. $m_0 = 1$ and $m_1 = 0$, since the distribution is normalized and mean zero. By using the linearity of expectation values, the independence of the x_i, and $\mathbb{E}[x_i] = 0$, we find m_2 directly:

$$m_2 = \mathbb{E}[X^2] = \frac{1}{n}\left(n\cdot\mathbb{E}[x_1^2] + n(n-1)\cdot\mathbb{E}[x_1]\mathbb{E}[x_2]\right) = \sigma^2.$$

We see that the scaling $1/\sqrt{n}$ in the definition of X ensures an n-independent variance equal to the variance of the distribution. For m_3,

$$\mathbb{E}[X^3] = \frac{1}{n^{3/2}}\big(n \cdot \mathbb{E}[x_1^3] + 0\big) = \frac{1}{n^{1/2}}\mathbb{E}[x_1^3],$$

where the terms set equal to zero have at least one $\mathbb{E}[x_1] = 0$. Note that while the distribution can have a non-zero skewness (third moment), it will be suppressed as n gets large. For m_4,

$$\mathbb{E}[X^4] = \frac{1}{n^2}\big(n \cdot \mathbb{E}[x_1^4] + 3n(n-1) \cdot \mathbb{E}[x_1^2] \cdot \mathbb{E}[x_2^2] + 0\big) = \frac{1}{n} \cdot \mathbb{E}[x_1^4] + 3(1 - \frac{1}{n})\sigma^4.$$

Now as n gets large, m_4 reduces to the Gaussian kurtosis result of $3\sigma^4$. Note that as long as n is finite, however, there will be non-Gaussian contributions (called excess kurtosis).

More generally, we see that the $\mathbb{E}[x_i^k]$ term in m_k for $k \geq 2$ comes with a single factor of n, so it will always be suppressed by increasing powers of $\frac{1}{n}$. The only n-independent terms will be $k/2$ products of $\mathbb{E}[x_i^2] = \sigma^2$, with a coefficient that exactly matches $\{m_k\}_{\mathcal{N}(0,\sigma^2)}$.

Thus, the moments of the X distribution will tend toward those of a Gaussian, meaning that the distribution itself tends towards a Gaussian distribution. That is the central limit theorem. Now let's make a different construction towards the same end.

Proof of the CLT

Start with *independent* random variables $x_1, \cdots, x_n$ drawn from a distribution with mean $\langle x \rangle = 0$ and variance $\langle x^2 \rangle = \sigma^2$, where

$$\langle x^n \rangle \equiv \int x^n p(x)\, dx.$$

(Assuming mean zero is general, because we can always work with the distribution of $x - \mu$.) As in the last section, let

$$X = \frac{1}{\sqrt{n}}(x_1 + x_2 + \cdots + x_n) = \sum_{j=1}^{n} \frac{x_j}{\sqrt{n}},$$

scaling by $1/\sqrt{n}$ so that the variance of X is constant in the $n \to \infty$ limit.

What is the distribution of X? $\Longrightarrow$ call it $p(X|I)$, where I is the information about the probability distribution for x_j.

Plan: Use the sum and product rules and their consequences to relate $p(X)$ to what we know of $p(x_j)$. (Note: we'll suppress I to keep the formulas from getting too cluttered.)

$$
\begin{aligned}
p(X) &= \int_{-\infty}^{\infty} dx_1 \cdots dx_n \, p(X, x_1, \cdots, x_n) \\
&= \int_{-\infty}^{\infty} dx_1 \cdots dx_n \, p(X|x_1, \cdots, x_n) \, p(x_1, \cdots, x_n) \\
&= \int_{-\infty}^{\infty} dx_1 \cdots dx_n \, p(X|x_1, \cdots, x_n) \, p(x_1) \cdots p(x_n).
\end{aligned}
$$

Checkpoint question

State the rule used to justify each step.

Answer

1. marginalization
2. product rule
3. independence

We might proceed by using a direct, normalized expression for $p(X|x_1, \cdots, x_n)$,

Checkpoint question

What is $p(X|x_1, \cdots, x_n)$?

Hint

If we know each of the x_i values, we know X *exactly*!

Answer

$p(X|x_1, \cdots, x_n) = \delta\Big(X - \frac{1}{\sqrt{n}}(x_1 + \cdots + x_n)\Big) = \frac{1}{2\pi} \int_{-\infty}^{\infty} d\omega \, e^{i\omega\left(X - \frac{1}{\sqrt{n}} \sum_{j=1}^{n} x_j\right)}$

and perform one of the integrations. Instead we will use a Fourier representation (see the answer to the Checkpoint question):

$$
p(X|x_1, \cdots, x_n) = \frac{1}{2\pi} \int_{-\infty}^{\infty} d\omega \, e^{i\omega\left(X - \frac{1}{\sqrt{n}} \sum_{j=1}^{n} x_j\right)}.
$$

Substituting into $p(X)$ and gathering together all pieces with x_j dependence while exchanging the order of integrations:

$$
p(X) = \frac{1}{2\pi} \int_{-\infty}^{\infty} d\omega \, e^{i\omega X} \prod_{j=1}^{n} \left[\int_{-\infty}^{\infty} dx_j \, e^{-i\omega x_j/\sqrt{n}} p(x_j) \right].
$$

Observe that the terms in []s have factorized into a product of independent integrals and they are all the same (just different labels for the integration variables). Now we Taylor expand $e^{-i\omega x_j/\sqrt{n}}$, arguing that the Fourier integral is dominated by small x as $n \to \infty$. (*When does this fail?*) We find:

$$e^{i\omega x/\sqrt{n}} = 1 - \frac{i\omega x}{\sqrt{n}} + \frac{(i\omega)^2 x^2}{2n} - \mathcal{O}\left(\frac{\omega^3 x^3}{n^{3/2}}\right).$$

Then, using that $p(x)$ is normalized (i.e., $\int_{-\infty}^{\infty} dx\, p(x) = 1$) and with the notation $\langle f(x)\rangle$ for the expectation value of $f(x)$,

$$\begin{aligned}\int_{-\infty}^{\infty} dx\, e^{-i\omega x/\sqrt{n}} p(x) &= \int_{-\infty}^{\infty} dx\, p(x)\left[1 - \frac{i\omega x}{\sqrt{n}} + \frac{(i\omega)^2 x^2}{2n} + \dots\right] \\ &= 1 - \frac{i\omega}{\sqrt{n}}\langle x\rangle - \frac{\omega^2}{2n}\langle x^2\rangle + i\langle x^3\rangle \frac{\omega^3}{n^{3/2}} \\ &= 1 - \frac{\omega^2\sigma^2}{2n} + \mathcal{O}\left(\frac{\omega^3}{n^{3/2}}\right).\end{aligned}$$

Now we can substitute into the posterior for X and take the large n limit:

$$\begin{aligned}p(X) &= \frac{1}{2\pi}\int_{-\infty}^{\infty} d\omega\, e^{i\omega X}\left[1 - \frac{\omega^2\sigma^2}{2n} + \mathcal{O}\left(\frac{\omega^3}{n^{3/2}}\right)\right]^n \\ &\overset{n\to\infty}{\longrightarrow} \frac{1}{2\pi}\int d\omega\, e^{i\omega X} e^{-\omega^2\sigma^2/2} = \frac{1}{\sqrt{2\pi}} e^{-X^2/(2\sigma^2)}.\end{aligned}$$

Checkpoint question

How did we use the large n limit to get the first equality on the last line?

Answer

We used

$$\lim_{n\to\infty}(1 + \frac{z}{n})^n = e^z \quad \text{for any} \quad z, \text{ here applied to} \quad z = -\frac{\omega^2\sigma^2}{2}.$$

We can understand this intuitively by multiplying out

$$\begin{aligned}\underbrace{(1+\frac{z}{n})(1+\frac{z}{n})\cdots(1+\frac{z}{n})}_{n \text{ terms}} &= 1 + n\cdot\frac{z}{n} + \binom{n}{2}\frac{z^2}{n^2} + \binom{n}{3}\frac{z^3}{n^3} + \dots \\ &\overset{n\to\infty}{\longrightarrow} 1 + z + \frac{z^2}{2!} + \frac{z^3}{3!} + \dots\end{aligned}$$

In getting this result we have dropped terms that are of order $1/n$ and $1/n^2$. All terms that we dropped in truncating the Taylor expansions for each Fourier integral will also be suppressed by inverse powers (or half powers) of n.

Visualizing the CLT

The general statement of the central limit theory (CLT) is that the sum of n random variables drawn from *any* PDF (or multiple PDFs) of finite variance σ^2 tends as $n \to \infty$ to be Gaussian distributed about the expectation value of the sum, with variance $n\sigma^2$. (So we would scale the sum by $1/\sqrt{n}$ to keep the variance fixed.) In this section we look visually at several consequences of the CLT.

Things to think about:

1. *What does "large" number of degrees of freedom actually mean? Does it matter where we look?*
2. *If we have a large number of draws from a uniform distribution, does the CLT imply that the histogrammed distribution should look like a Gaussian?*
3. *Can you identify a case (that we have seen) where the CLT will fail?*

Answers

1. "Large" *will* depend on detail, but it is often surprising how few the number is before the CLT kicks in (e.g., often of order 30).
2. Yes! This is illustrated below.
3. The Cauchy distribution will fail because its variance is not finite.

Limit of Poisson distribution is Gaussian

One consequence of the CLT is that distributions such as the Binomial and Poisson distributions all tend to look like Gaussian distributions in the limit of a large number of drawings. Here we visualize how the Poisson distribution in the limit $D \to \infty$ approaches a Gaussian distribution:

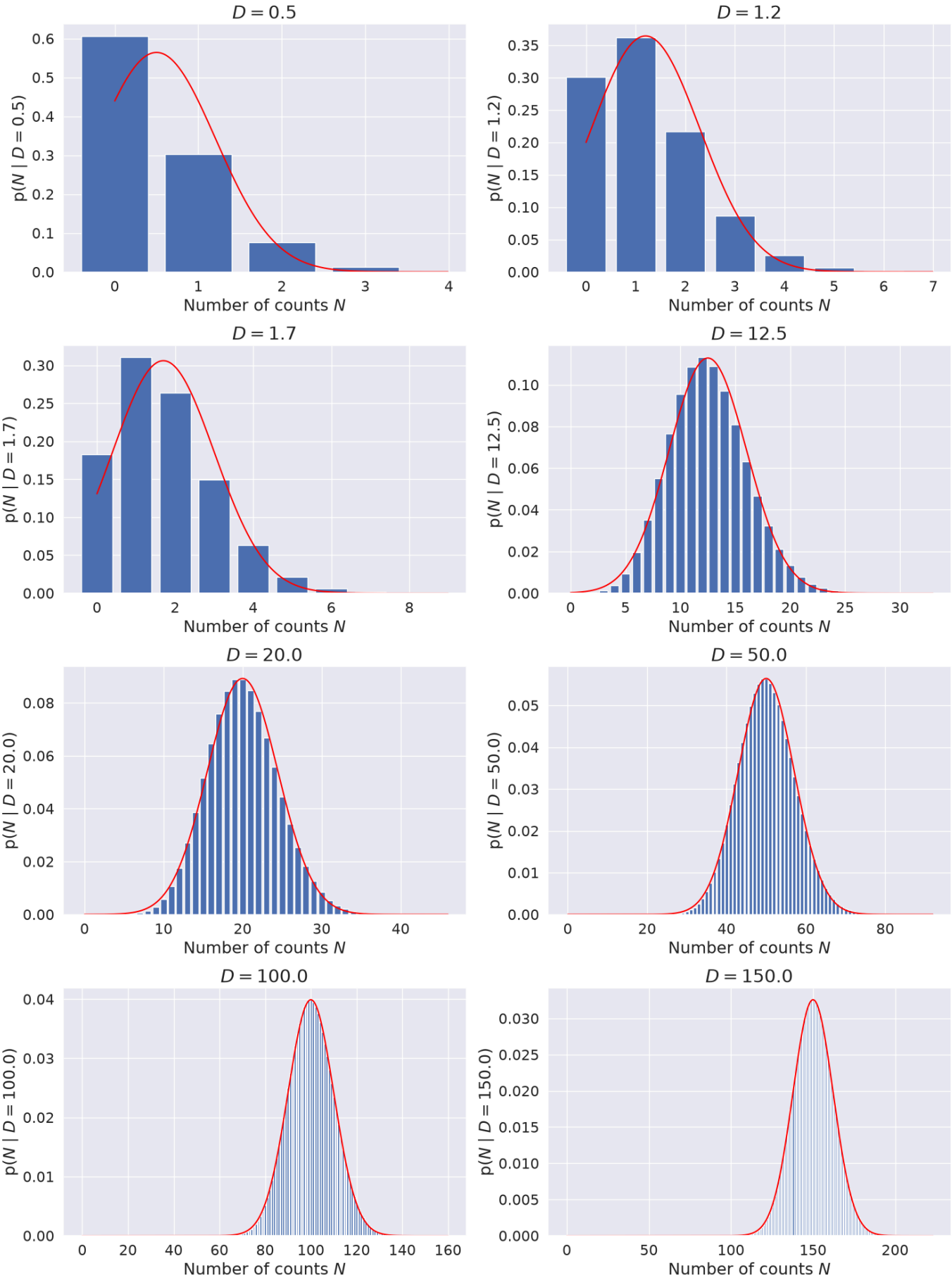
$$p(N \mid D) = \frac{D^N e^{-D}}{N!} \overset{D\to\infty}{\longrightarrow} \frac{1}{\sqrt{2\pi D}} e^{-(N-D)^2/(2D)}$$

with N an integer. *You will be asked to prove this limit in a later notebook.*

Note: this limiting functional form is not obviously connected to the general statement of the CLT up above. What is the connection of the Gaussian limit to a sum of random variables as prescribed by the CLT? Answered below with a test.

Make successive histogram plots of the Poisson distribution, increasing the value of D from 0.5 to 50, and superposing the normal distribution it is supposed to approach.

▼ **Hide code cell source**


$D = 0.5$
$p(N \mid D = 0.5)$
Number of counts N
$D = 1.2$
$p(N \mid D = 1.2)$
Number of counts N
$D = 1.7$
$p(N \mid D = 1.7)$
Number of counts N
$D = 12.5$
$p(N \mid D = 12.5)$
Number of counts N
$D = 20.0$
$p(N \mid D = 20.0)$
Number of counts N
$D = 50.0$
$p(N \mid D = 50.0)$
Number of counts N
$D = 100.0$
$p(N \mid D = 100.0)$
Number of counts N
$D = 150.0$
$p(N \mid D = 150.0)$
Number of counts N

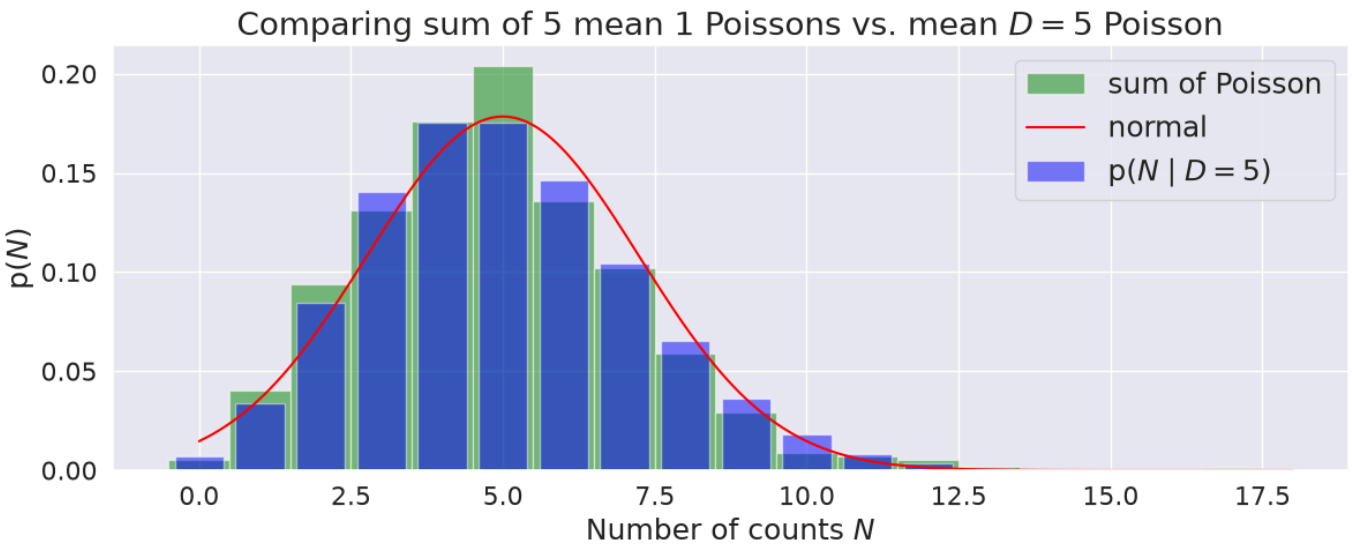
Compare a sum of D Poisson draws with mean 1 to a single Poisson distribution with mean D

Now we answer the question asked above: **What is the connection of the Gaussian limit to a sum of random variables as prescribed by the CLT?**

The answer is that a sum of Poisson distributed random variables is itself Poisson distributed. For example, if $X_1, \ldots, X_D$ are independent Poisson random variables with mean 1 (D is an integer here), then $Y = \sum_{i=1}^{D} X_i$ is a Poisson random variable with mean D. So the sum is built in by directly considering Y. Note the analogy to the situation with the binomial distribution and the CLT.

Here we test this answer by plotting the histogram of num_draws samples, each the sum of D mean-1 Poisson samples, to the probability mass function (pmf) of a Poisson with mean-D (and also plot the Gaussian limit).

▼ **Hide code cell source**



Things to try:

- Change the number of draws
- Change the value of D

Behavior of the mean of a fixed-size sample

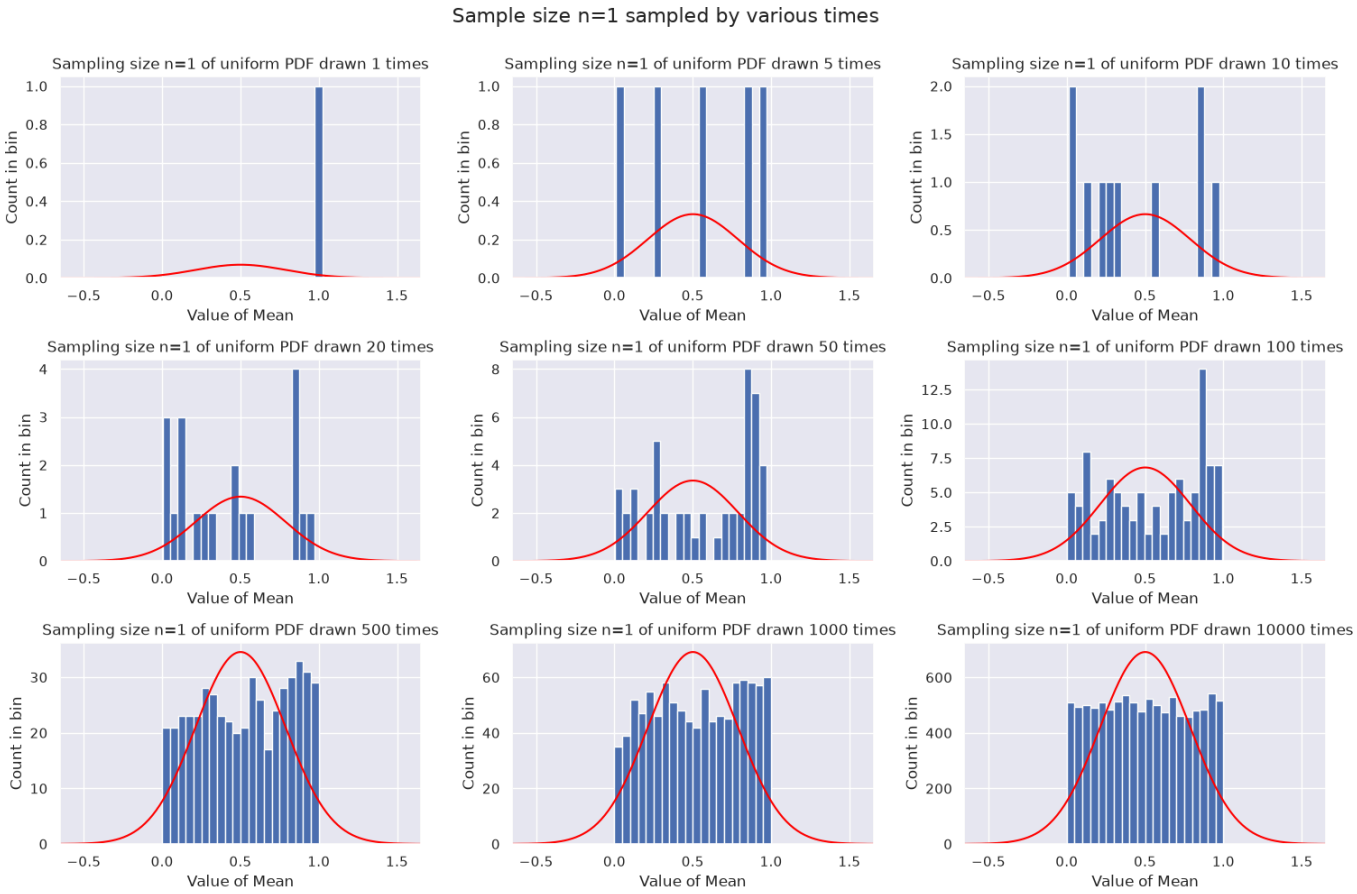
Here we take the mean of a fixed-size sample drawn from any PDF, and then repeat a number of times and look at the distribution of the means. According to the CLT, this distribution should approach a Gaussian PDF. For our test PDF we'll use a uniform distribution, although this is easy to switch to another distribution.

First example: each sample is only one point

Since the sample size is one, the mean is just the point, so the distributions here will look like the original distribution (no approach to Gaussian). Note that we bin the trivial means (the individual draws) into 20 bins.

▼ **Hide code cell source**

```
mean of uniform array = 0.501
standard deviation of uniform array = 0.288
```

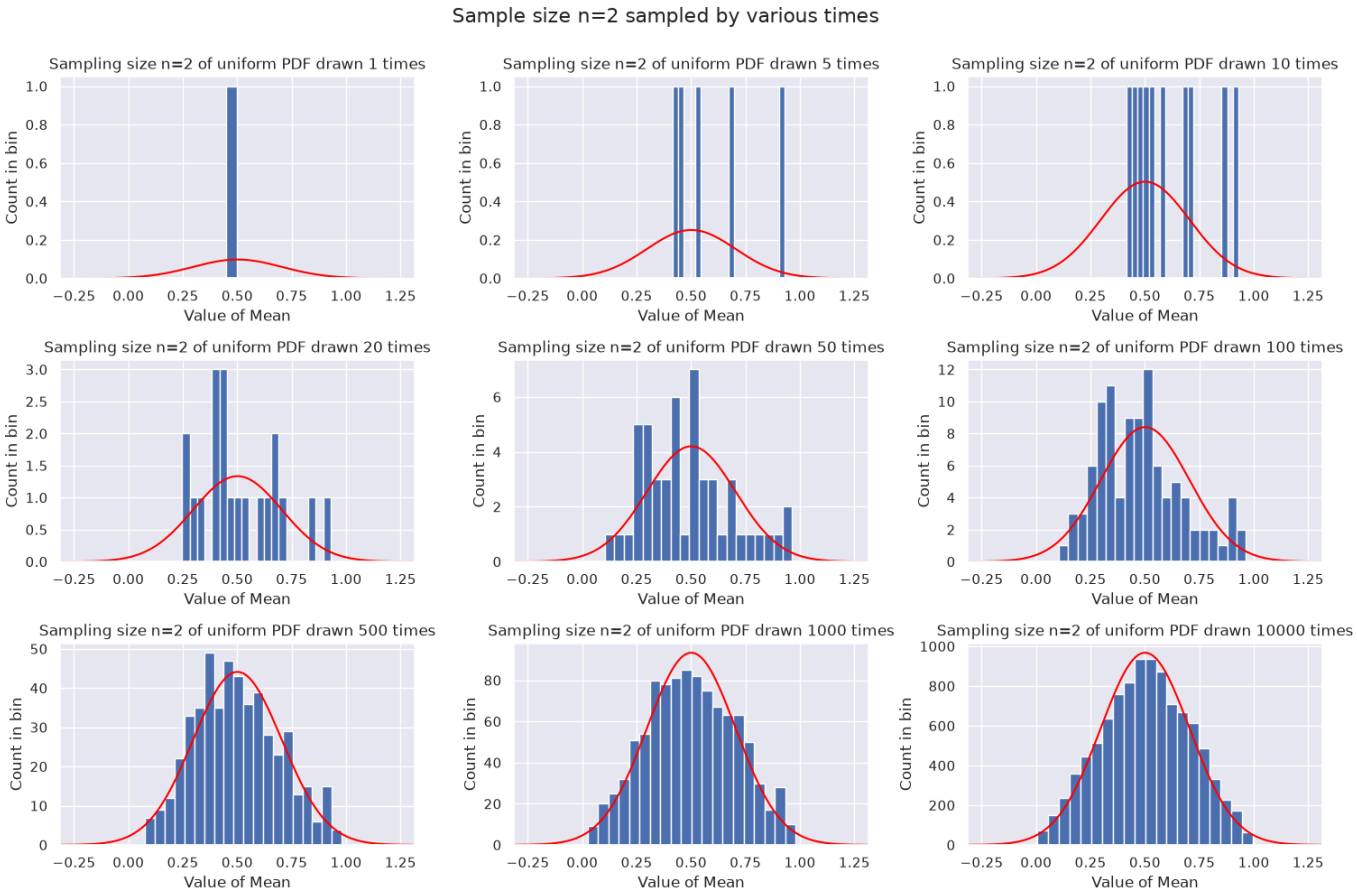


As expected, we just get a better and better reproduction of the original distribution, which is uniform in this case. We don't expect any connection to a Gaussian from the CLT in this case. That is, just taking a lot of samples from a distribution doesn't mean in general that we will get a normal distribution.

Second example: each sample is two points

So the mean is the average of the two points. We divide the means into 20 bins.

▼ **Hide code cell source**

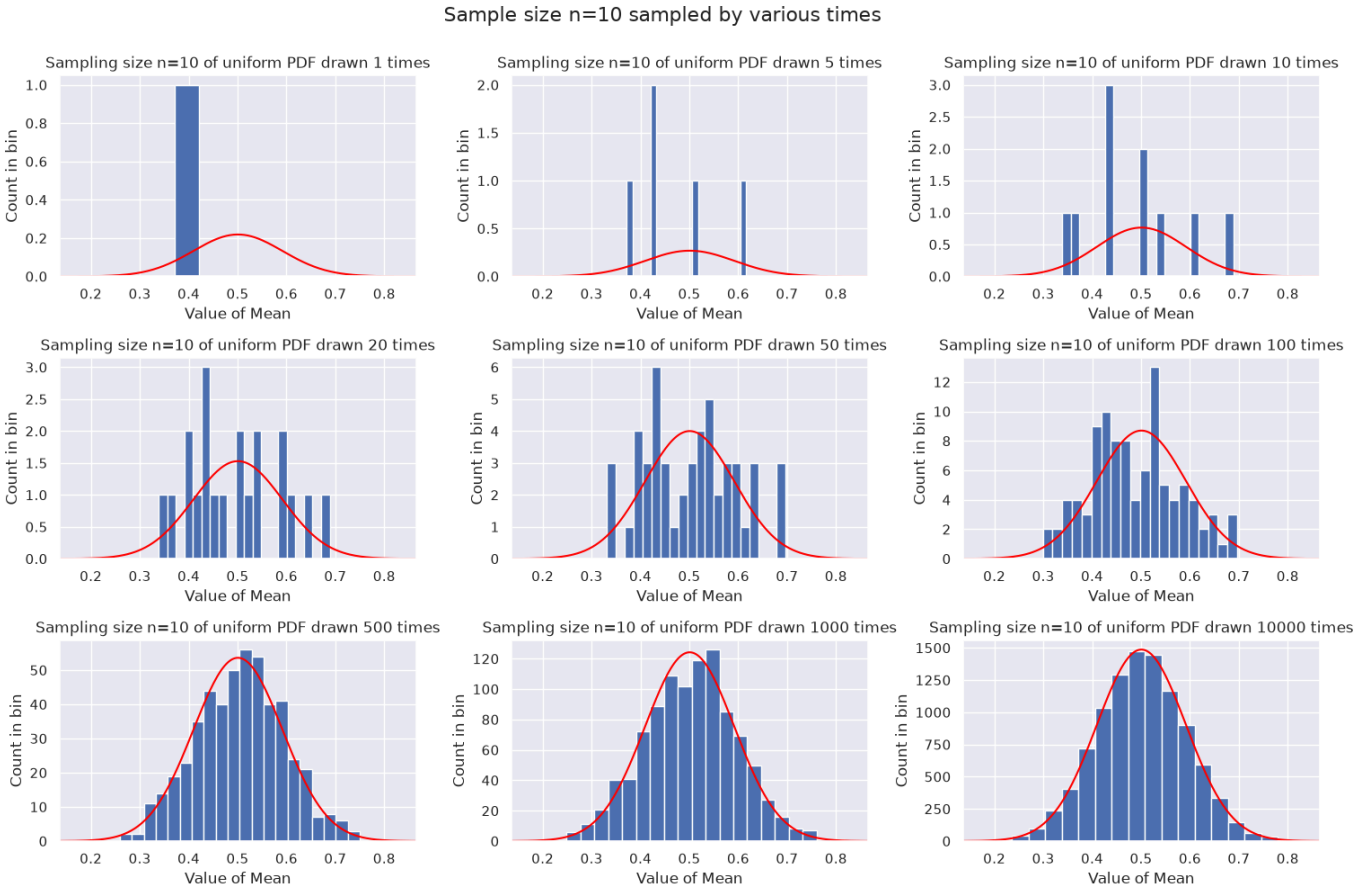


As the number of means gets larger, we see that a peak develops at 1/2 and the shape becomes triangular. If we think of the average of two uniformly drawn points from [0,1], it is most likely that their average will be close to 1/2 and unlikely to be at the ends (e.g., to get near 0, one would need both numbers to be close to 0).

Third example: each sample is 10 points

Now we see the trends from $n = 2$ being accentuated. By the time we have 10000 means to histogram, it looks like a pretty good Gaussian. **Note that the width of the Gaussian is set by the sample size $n = 10$, not by how many draws are made.** The width decreases as $1/\sqrt{n}$, so from $n = 2$ above to $n = 10$ here it is roughly a factor of 2.2 smaller (0.25 vs. 0.1).

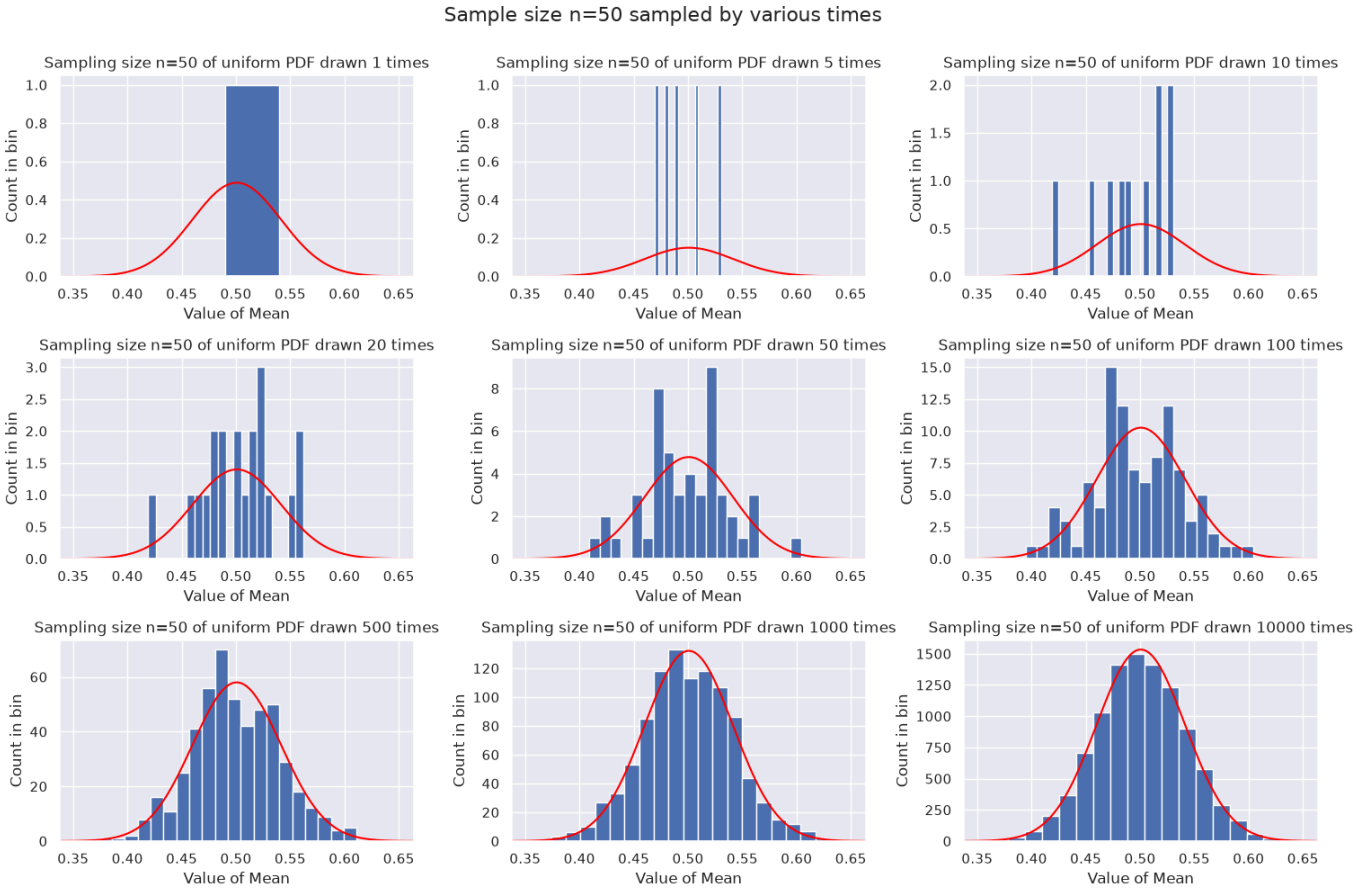
▼ **Hide code cell source**



Fourth example: each sample is 50 points

By now we are taking the mean of 50 points with each sample. This will increasingly be close to 1/2 and very rarely far from 1/2. The width decreases as $1/\sqrt{n}$, so from $n = 10$ to $n = 50$ it is roughly a factor of 2.2 smaller (0.1 vs. 0.04).

▼ **Hide code cell source**



Adding n variables drawn from a distribution

We'll use the results derived for combining n variables drawn from the same zero-mean probability distribution with an overall $1/\sqrt{n}$. The generalization to different PDFs (and to non-zero means) is straightforward.

$$p(z) = \int_{-\infty}^{+\infty} \frac{d\omega}{2\pi}\, e^{-i\omega z} \left[\int_{-\infty}^{+\infty} dx_1\, e^{i\omega x_1/\sqrt{n}} p(x_1)\right] \left[\int_{-\infty}^{+\infty} dx_2\, e^{i\omega x_2/\sqrt{n}} p(x_2)\right] \cdots \\ \times \left[\int_{-\infty}^{+\infty} dx_{n-1}\, e^{i\omega x_{n-1}/\sqrt{n}} p(x_{n-1})\right] \left[\int_{-\infty}^{+\infty} dx_n\, e^{i\omega x_n/\sqrt{n}} p(x_n)\right]$$

So the idea will be to take a distribution, do an FT with the frequency divided by $\sqrt{n}$, raise it to the n^{th} power, then do an inverse FT. This should approach a Gaussian as n gets large based on the proof we worked through.

In the widget below we'll start with a uniform distribution again. (For better viewing, toggle the left table of contents with the three-line icon at the top left of the page.) The plot below on the right are the n products of the Fourier-transformed distributions (so this is in the Fourier space) with the frequency divided by $\sqrt{n}$. The plots on the left are the inverse Fourier transforms of the plots on the right. You can adjust n with the slider.

Central Limit Theorem via Characteristic Functions

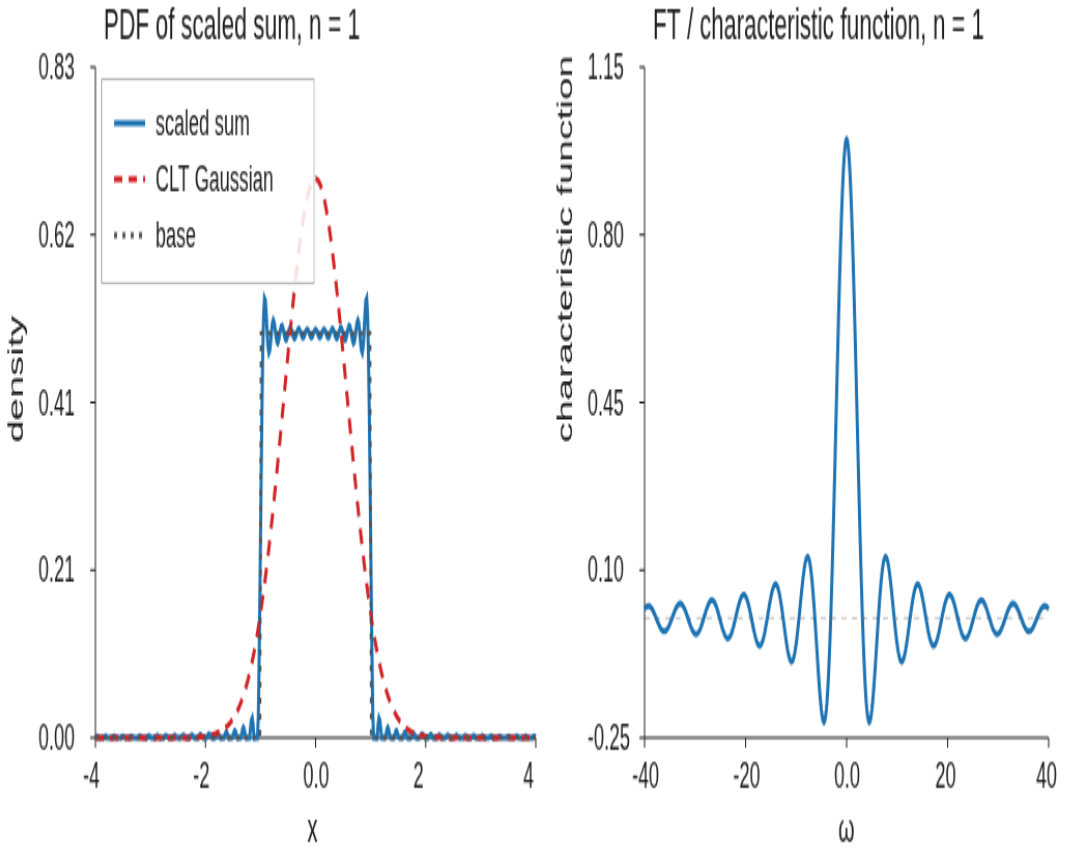
base distribution: Uniform[-1, 1]

n : 1 x range: 4.0

ω range: 40 show base PDF/PMF

show CLT Gaussian

Uniform[-1, 1]; variance = 0.333. The plotted sum is (X_1 + ⋯ + X_n)/√n. The limiting Gaussian has the same variance as the base distribution.



So we see that multiplying the original Fourier transform will rapidly kill off the tails and highlight the central part. The n scalings enforce that the Gaussians in both the original and Fourier space have constant variances.

Try some of the other distributions from the pull-down menu.

6. Updating via Bayes' rule

Bayes' rule (or theorem, same thing!) plays a central role in Bayesian statistics. It is the heart of how we update our knowledge given new information:

$$\text{prior} \xrightarrow{\text{likelihood}} \text{posterior}$$

Let us recall the specifics and nomenclature of Bayes' rule for the case where we seek the PDF of parameters $\boldsymbol{\theta}$ contingent on some data:

$$\overbrace{p\left(\boldsymbol{\theta}|\text{data }, I\right)}^{\text{posterior}} = \frac{\overbrace{p\left(\text{data }|\boldsymbol{\theta}, I\right)}^{\text{likelihood}} \times \overbrace{p\left(\boldsymbol{\theta}|I\right)}^{\text{prior}}}{\underbrace{p\left(\text{data }|I\right)}_{\text{evidence}}},$$

where

- $\boldsymbol{\theta}$ is a general *vector* of parameters
- The *prior* PDF is based on information I we have (or believe) about $\boldsymbol{\theta}$ before we observe the data.
- The *posterior* PDF is our new PDF for $\boldsymbol{\theta}$, given that we have observed the data.
- The *likelihood* is the probability of getting the specified data *given* the parameters $\boldsymbol{\theta}$ under consideration on the left side.
- The denominator is the data probability or "fully marginalized likelihood" or evidence or maybe some other name (these are all used in the literature). It is a normalization factor that scales the posterior *independent of* $\boldsymbol{\theta}$ and so often does not need to be calculated.

The bottom line is that Bayes' rule tells us how to *update* our expectations. I.e., how we should modify our prior beliefs I about the parameters $\boldsymbol{\theta}$ after we have acquired new data that has implications for their values. (Note this says "new" data; we can build upon previous data.)

There are many types of parameter estimation based on Bayes' rule; each of the Problems for Part I provides an example. But we will apply Bayes' rule for updating in other contexts in this book. These include

- Bayesian model selection
- Model discrepancy (Discrepancy models with GPs)
- Model averaging and mixing
- Bayesian optimization
- Bayesian neural networks

In this chapter we use a simple coin-tossing experiment to examine the updating process, its practice and its implications, in more detail.

6.1. Coin tossing: Frequentists and Bayesians

Storyline: We are observing successive flips of a coin. There is a definite true probability of getting heads $(p_H)_{\text{true}}$, but we don't know what the value is, although we do know that it is between 0 and 1.

So, is the coin fair? By fair, we mean that we would be prepared to lay an even 1 : 1 bet on the outcome of a flip being a head or a tail. The frequentist approach to this problem is to set up a null hypothesis, i.e., the hypothesis that the coin is fair (meaning $p_H = 0.5$), and then determine the probability that the observed data is generated given that that hypothesis holds. If the probability is a low number $p_H \ll 1$ then the null hypothesis can be rejected at a confidence of $1 - p_H$.

Thus, if the data is that the coin produces H heads after N tosses the frequentist wants to compute

$$p\left(N \text{ tosses }, H \text{ heads } | p_H = 0.5\right),$$

and only is allowed to get excited if this probability is a low number.

The Bayesian, however, gets to ask a more general question. What is:

$$p\left(p_H | N \text{ tosses }, H \text{ heads }, I\right)?$$

Note that the *outcome* of any one coin toss is discrete (either heads or tails) but p_H can take any value between 0 and 1, therefore this PDF is a function of the continuous variable p_H. Best estimates for p_H, intervals that encompass the true value with a certain probability, etc. can then all be computed from the probability density function $p\left(p_H | N \text{ tosses }, H \text{ heads }, I\right)$. Determining this probability distribution function for p_H given the data on the coin's behavior is an example of Bayesian parameter estimation.

One of the key points of this exercise, is that with each flip of the coin we acquire more information on the value of p_H. The logical thing to do is to update the state of our belief, our PDF for $p\left(p_H | \text{How many heads in as many tosses as we}'\text{ve made, I}\right)$ each time the number of coin tosses is incremented by 1. The PDF will tend to get narrower, i.e., our state of knowledge of p_H more definite, as we acquire more data.

Note that we exploit the fungibility of mathematical symbols to let I stand for different things at different stages of the coin tossing experiment. If we are going to "update" after every coin toss then D is just the result of the Nth coin toss and I is what we know about the coin after $N - 1$ coin tosses.

There are two demo notebooks you can use to explore this problem: 📥 Demo: Bayesian Coin Tossing (non-widget) and/or Demo: Widgetized coin tossing. Try both!

Play a bit with the simulation and then we'll consider some of the details. Here are some things to try with the widgetized version:

- Look at each of the priors under the *Priors* tab and interpret them.
- First do one flip at a time. How do you understand the changes intuitively?
- What happens with more and more tosses? Can you explain the differences with different priors?
- Hit `New Data` multiple times to see the fluctuations (this changes the random seed).
- Try different values of the true p_H.

6.2. 📥 Demo: Bayesian Coin Tossing

This demonstration notebook provides an interactive interface (a widget) to enable you to see the effects of subsequent coin tosses (one toss at a time or many at once) with different choices of prior. You get to choose the bias p_H of the coin, namely the probability of a heads on any given toss. The tosses are assumed to be independent.

There are a number of embedded questions that you should try to answer (see also Updating via Bayes' rule) and suggestions for modifications to the code to explore different aspects of the analysis.

Recap of Bayesian updating

Recall Bayes' theorem with $\boldsymbol{\theta}$ the vector of parameters we seek and information I is kept implicit.

$$\overbrace{\mathrm{p}\left(\boldsymbol{\theta} \mid \text{data}\ , I\right)}^{\text{posterior}} = \frac{\overbrace{\mathrm{p}\left(\text{data}\ \mid \boldsymbol{\theta}, I\right)}^{\text{likelihood}} \times \overbrace{\mathrm{p}\left(\boldsymbol{\theta} \mid I\right)}^{\text{prior}}}{\underbrace{\mathrm{p}\left(\text{data}\ \mid I\right)}_{\text{evidence}}}$$

If we view the prior as the initial information we have about $\boldsymbol{\theta}$, summarized as a probability density function, then Bayes' theorem tells us how to *update* that information after observing some data: this is the posterior pdf. Here we will give some examples of how this plays out when tossing coins.

Determining the bias of a coin

The idea here is that we are observing successive flips of a coin, which is a proxy for any process that has a binary outcome. There is a definite true probability for getting heads, which we'll label p_H, but we don't know what it is. We start with a preconceived notion of the probability expressed in terms of a prior pdf for p_H, i.e., $\mathrm{p}\left(p_H\right)$. With each flip of the coin, we have more information, so our goal is to *update* our expectation of p_H, meaning we want the posterior $p(p_H|\#\text{ tosses}, \#\text{ heads}\quad)$.

Let us perform a computer simulation of a coin-tossing experiment. This provides the data that we will be analysing.

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(seed=999)         # for reproducibility
pHtrue=0.6                          # biased coin
flips=rng.random(2**12) # simulates 4096 coin flips
heads=flips<pHtrue              # boolean array, heads[i]=True if flip i is heads
```

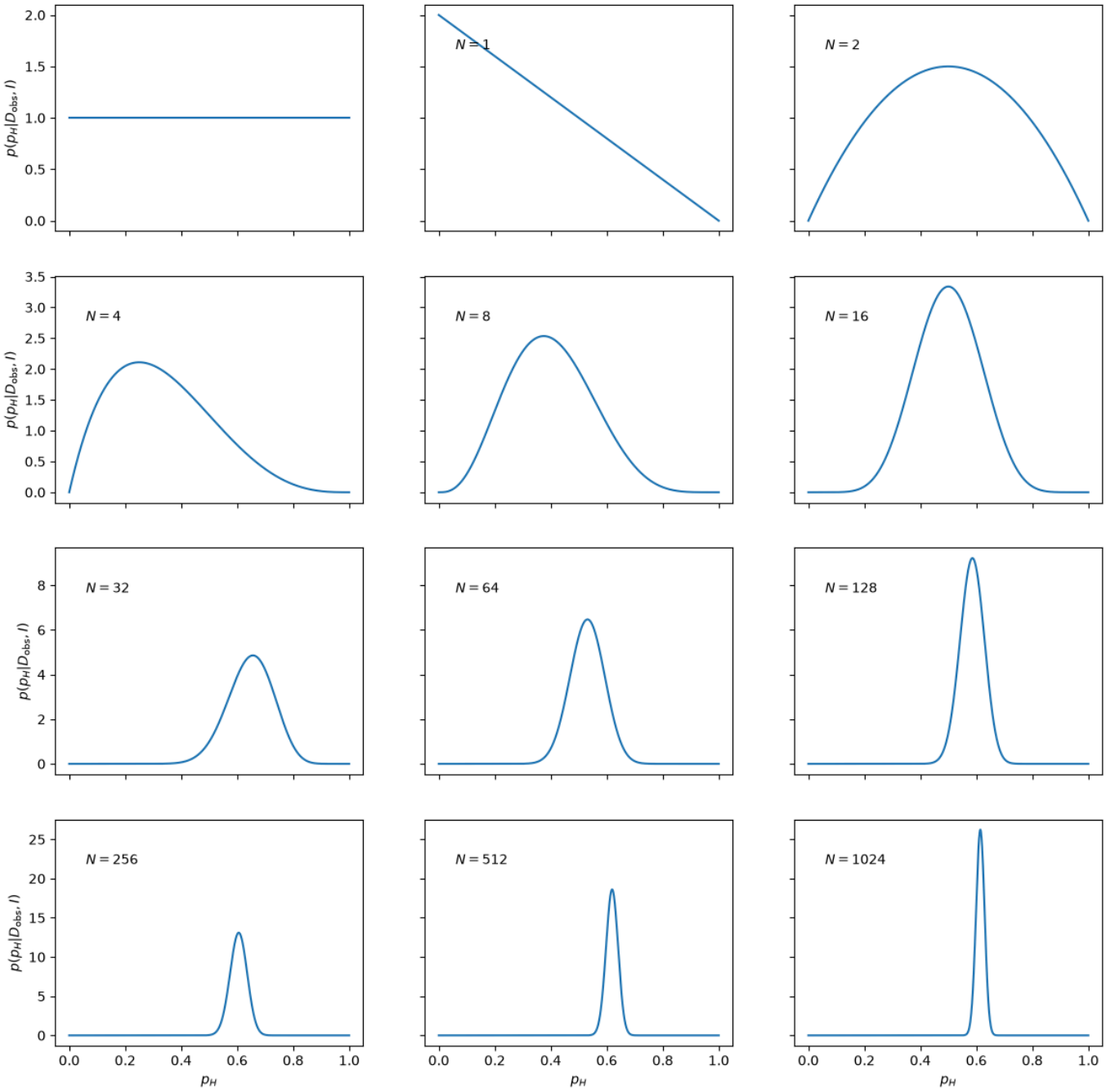
In the light of this data, our inference about the fairness of this coin is summarized by the conditional pdf: $p(p_H|D,I)$. This is, of course, shorthand for the limiting case of a continuum of propositions for the value of p_H; that is to say, the probability that p_H lies in an infinitesimally narrow range is given by $p(p_H|D,I)dp_H$.

Please see the previous chapter on the assignment of a prior and the likelihood. Here, we perform the setup of this Bayesian framework on the computer.

```
def prior(pH):
    "returns an array of prior probability densities"
    p=np.zeros_like(pH)
    p[(0<=pH)&(pH<=1)]=1      # allowed range: 0<=pH<=1
    return p                # uniform prior
def likelihood(pH,data):
    "returns an array of likelihood probability densities"
    N = len(data)
    no_of_heads = sum(data)
    no_of_tails = N - no_of_heads
    return pH**no_of_heads * (1-pH)**no_of_tails
def posterior(pH,data):
    "returns an array of (normalized) posterior probability densities"
    p=prior(pH)*likelihood(pH,data)
    norm=np.trapezoid(p,pH)
    return p/norm
```

The next step is to confront this setup with the simulated data. To get a feel for the result, it is instructive to see how the posterior pdf evolves as we obtain more and more data pertaining to the coin.

```
pH=np.linspace(0,1,1000)
fig, axs = plt.subplots(nrows=4,ncols=3,sharex=True,sharey='row',figsize=(14,14))
axs_vec=np.reshape(axs,-1)
axs_vec[0].plot(pH,prior(pH))
for ndouble in range(11):
    ax=axs_vec[1+ndouble]
    ax.plot(pH,posterior(pH,heads[:2**ndouble]))
    ax.text(0.1, 0.8, '$N={0}$'.format(2**ndouble), transform=ax.transAxes)
for row in range(4): axs[row,0].set_ylabel('$p(p_H|D_\mathrm{obs},I)$')
for col in range(3): axs[-1,col].set_xlabel('$p_H$')
```



Please, feel free to play with this setup yourself by modifying the code above. Things to explore:

- The choice of `pH`.
- The number of data.
- The choice of prior (i.e. the function `prior`).
- *Is there a difference between the results obtained for many sequential updates (i.e. updating the posterior after each toss), and a single update with all data? Can you show this result analytically?*

Degree of belief intervals

Now we are going to compute some Bayesian confidence intervals, aka DoB intervals, aka credibility intervals… You should go through this section once using the numbers provided, and then come back and run it again (or copy-paste the relevant lines) for other results from the widget. You can also employ priors other than those provided if you wish, but that takes a bit more work.

```
import scipy.stats as stats
```

```
def dist_stuff(dist):
    """
    Finds the median, mean, and 68%/95% credible intervals for the given
    1-d distribution (which is an object from scipy.stats).
    """
    # For x = median, mean: return x and the value of the pdf at x as a list
    median = [dist.median(), dist.pdf(dist.median())]
    mean = [dist.mean(), dist.pdf(dist.mean())]
    # The left and right limits of the credibility interval are returned
    cred68 = dist.interval(0.68)
    cred95 = dist.interval(0.95)
    return median, mean, cred68, cred95
```

```
def dist_mode(dist, x):
    """
    Return the mode (maximum) of the 1-d distribution for array x.
    """
    x_max_index = dist.pdf(x).argmax()
    # Return x of the maximum and the value of the pdf at that x
    mode = [x[x_max_index], dist.pdf(x[x_max_index])]
    return mode
```

Then we use this to write an object that will give us back the mean, 68%, and 95% intervals for a uniform prior.

```
def print_uniform_prior_measures(N,heads):
    """
    Prints out the mean, and 68 and 95 CIs for a uniform prior. Note that this means
alpha=beta=1.
    """
    x = np.linspace(0, 1, 301)   # mesh for posterior plots (enough so smooth)
    median, mean, cred68, cred95 = dist_stuff(stats.beta(1+heads,1+N-heads))
    mode=dist_mode(stats.beta(1+heads,1+N-heads),x)
    print(f'For a uniform prior, and {heads} heads out of {N} tosses:')
    print(f'  mean={mean[0]:.3f}; Mode={mode[0]:.3f}')
    print(f'  68% DoB interval=[{cred68[0]:.3f},{cred68[1]:.3f}]')
    print(f'  95% DoB interval=[{cred95[0]:.3f},{cred95[1]:.3f}]')
    return
```

Now we fill in the values for N and heads from running the widget.

```
num_coin_tosses=10
print_uniform_prior_measures(num_coin_tosses,np.sum(heads[:num_coin_tosses]))
```

```
For a uniform prior, and 4 heads out of 10 tosses:
  mean=0.417; Mode=0.400
  68% DoB interval=[0.276,0.558]
  95% DoB interval=[0.167,0.692]
```

```
print('The actual value of p_H is',pHtrue)
```

```
The actual value of p_H is 0.6
```

So, at least for the data I generated, the 68% DoB for a uniform prior does not contain the true value. Should you be surprised?

```
def print_frequentist_estimators(N,heads):
    """
    Finds the mean, and 68 and 95 CIs. Note that this means alpha_1=beta_1=1.
    """
    mean=heads/N
    sigma=np.sqrt(mean*(1-mean)/N)
    print(f'For {heads} heads out of {N} tosses, the mean is {mean:.3f} ',
          f'and the frequentist 68% confidence interval=',\
          f'({mean-sigma:.3f},{mean+sigma:.3f})')
    return
```

```
print_frequentist_estimators(num_coin_tosses,np.sum(heads[:num_coin_tosses]))
```

```
For 4 heads out of 10 tosses, the mean is 0.400  and the frequentist 68% confidence interval=
(0.245,0.555)
```

- *Which Bayesian estimator is the frequentist mean closest to?*
- *Is the frequentist 1σ interval the same as the Bayesian 68% DoB interval? If so, should they be? If not, why are they different?*

Now we will also generate the summary statistics for other priors. (What is coded is for the default values. After running through the demo you can come back and try and change it; indeed, you should do that if you comparing to results where you altered the prior above.)

```
def print_likely_fair_prior_measures(N,heads):
    """
    Prints out the mean, and 68 and 95 CIs for the "coin is likely fair" prior. This means
alpha=beta=30.
    """
    x = np.linspace(0, 1, 301)   # mesh for posterior plots (enough so smooth)
    median, mean, cred68, cred95 = dist_stuff(stats.beta(30+heads,30+N-heads))
    mode=dist_mode(stats.beta(1+heads,1+N-heads),x)
    print('For the trusting-person`s prior, and',heads,'heads out of',N,'tosses:')
    print (f'Mean={mean[0]:.3f}; Mode={mode[0]:.3f}')
    print (f'68% DoB interval=[{cred68[0]:.3f},{cred68[1]:.3f}]')
    print (f'95% DoB interval=[{cred95[0]:.3f},{cred95[1]:.3f}]')
    return
```

```
print_likely_fair_prior_measures(num_coin_tosses,np.sum(heads[:num_coin_tosses]))
```

```
For the trusting-person`s prior, and 4 heads out of 10 tosses:
Mean=0.486; Mode=0.400
68% DoB interval=[0.426,0.545]
95% DoB interval=[0.370,0.602]
```

```
def print_likely_unfair_prior_measures(N,heads):
    """
    Prints out the mean, and 68 and 95 CIs for the "coin is likely unfair" prior. This means
alpha=beta=0.2.
    """
    x = np.linspace(0, 1, 301)   # mesh for posterior plots (enough so smooth)
    median, mean, cred68, cred95 = dist_stuff(stats.beta(0.2+heads,0.2+N-heads))
    mode=dist_mode(stats.beta(1+heads,1+N-heads),x)
    print('For the nasty-suspicious-mind prior, and',heads,'heads out of',N,'tosses:')
    print (f'Mean={mean[0]:.3f}; Mode={mode[0]:.3f}')
    print (f'68% DoB interval=[{cred68[0]:.3f},{cred68[1]:.3f}]')
    print (f'95% DoB interval=[{cred95[0]:.3f},{cred95[1]:.3f}]')
    return
```

```
print_likely_unfair_prior_measures(num_coin_tosses,np.sum(heads[:num_coin_tosses]))
```

```
For the nasty-suspicious-mind prior, and 4 heads out of 10 tosses:
Mean=0.404; Mode=0.400
68% DoB interval=[0.253,0.555]
95% DoB interval=[0.144,0.699]
```

So what is the best approach in this case? Objectivity? Trust? Suspicion?

Now having printed out the results for my particular coin-tossing experiment you should play with things and see what the different summary statistics give for other “experimental runs”.

Don’t forget to change the seed for the random number generator, otherwise you will get exactly the same sequence of coin tosses.

6.3. Demo: Widgetized coin tossing

In this section we consider an interactive version of the problem of estimating the probability that a particular coin will come up heads on any given toss, p_H, based on data as to how many heads (and tails) it produces in N tosses. The tosses are assumed to be independent. (This example is gratefully adapted from the similar example in Sivia’s book.)

The data D will then be the number of heads H obtained in N trials. The probability of obtaining a particular number of heads will be a function of p_H. This is the likelihood piece of Bayes’ theorem. Note that the *outcome* is discrete (either heads or tails), and the number of heads obtained in N trials is an integer, but p_H can be any real number $0 \leq p_H \leq 1$, and all our output pdfs are continuous functions of p_H in the interval $0 < p_H < 1$.

Meanwhile we can represent different prior knowledge and/or beliefs about p_H in the prior, i.e., $\mathrm{p}(p_H|I)$. I could be information regarding the character of the coin flipper, it could be based on a previous experiment (we managed to get hold of the coin and flip it a few times before hand!), or it could be an "ignorance prior", the formulation of which we will come back to later in the book.

Bayesian updating

One of the key points of this exercise, is that with each flip of the coin we acquire more information on the value of p_H. The logical thing to do is to update the state of our belief, our pdf for $\mathrm{p}(p_H|\text{no. tosses, no. heads} \quad, I)$ each time the number of coin tosses is incremented by 1. The pdf will tend to get narrower as we acquire more data, i.e., our state of knowledge of p_H becomes more definite.

Note that in what follows we exploit the fungibility of mathematical symbols to let I stand for different things at different stages of the coin tossing experiment. If we are going to "update" after every coin toss then D is just the result of the Nth coin toss and I is what we know about the coin after $N-1$ coin tosses.

User-interface for coin-flipping

Take a look at the information under the `Help` tab to find out about what the controls do, what the priors are, etc.

Widget user interface features:

- tabs to control parameters or look at documentation
- set the true p_H by the slider
- press "Next" to flip "jump" # of times
- plot shows updating from three different initial prior pdfs

Bayesian Coin Tossing Demo

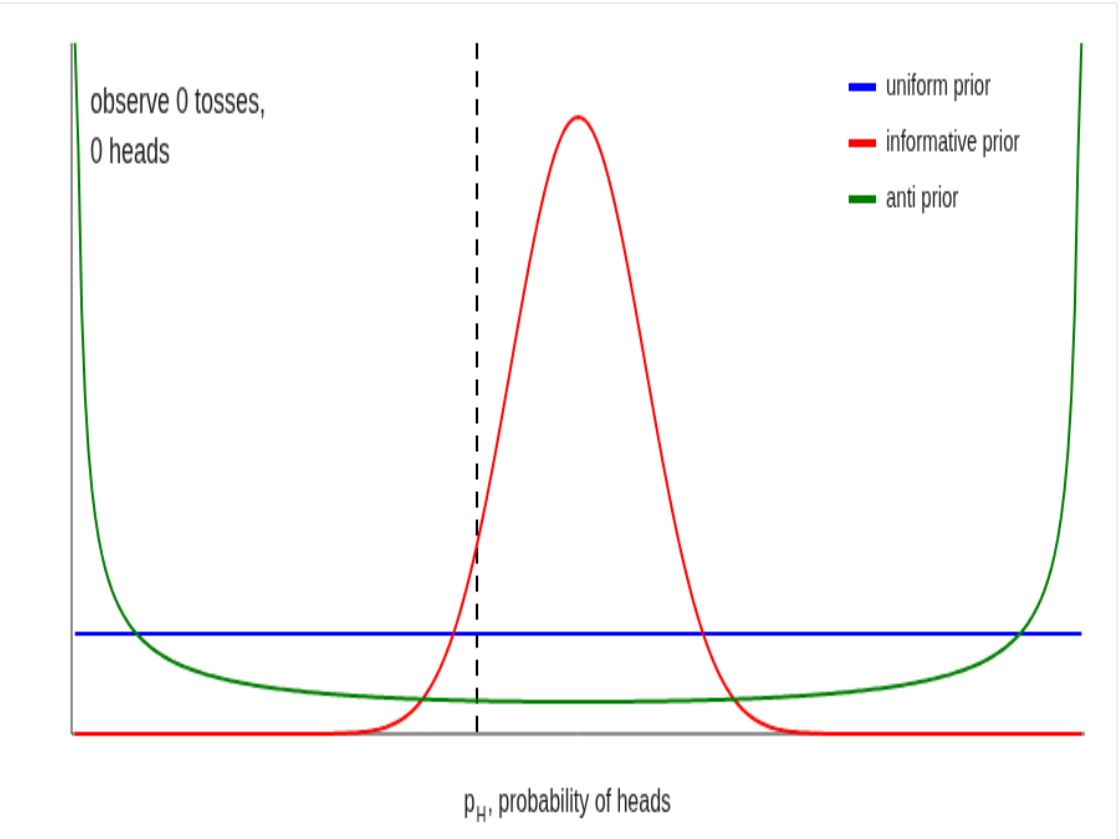
Toss Coin | Priors | Set-up | Help

The graph shows three posteriors that result from three choices for the prior. The true probability of heads, p_H, is indicated by a dashed vertical line.

Next Jump: 1 Reset New data

true p_H : 0.40

N = 0; heads = 0; true p_H = 0.40



6.4. Take-aways and follow-up questions from coin flipping

A couple of key points from studying coin flipping

- The core of the coin-flipping analysis is Bayes' rule, which dictates that the Bayesian posterior $p(p_H|D,I)$ is proportional to the product of the prior and the likelihood (which is given by a binomial distribution in this case).
- Possible point estimates for the value of p_H could be the maximum (also known as mode), mean, or median of this posterior PDF.

When do priors matter? When don't they matter?

Interpreting priors

Give an interpretion to each of the priors in the widgetized demo

Possible answers

- uniform prior: any probability is equally likely. Is this *uninformative*? (More later!)
- centered prior (informative): we have reason to believe the coin is more-or-less fair.
- anti-prior: could be anything but most likely a two-headed or two-tailed coin.

What is the minimal common information about p_H (meaning it is true of *any* prior PDF)?

Answer

$$0 \leq p_H \leq 1 \quad \text{and} \quad \int_0^1 p\left(p_H\right) dp_H = 1$$

Bayesian convergence

What happens when enough data is collected?

Answer

All posteriors, independent of prior, converge to a narrow PDF including $\left(p_H\right)_{\text{true}}$. This is called *Bayesian convergence*.

Different priors *eventually* give the same posterior with enough data. How many tosses constitute ``eventually" for $p_H = 0.4$ or $p_H = 0.9$?

Answer

- $p_H = 0.4 \Longrightarrow \approx 200$ tosses will get you most of the way.
- $p_H = 0.9 \Longrightarrow$ much longer for the informative prior than the others.

Which prior(s) get to the correct conclusion fastest for $p_H = 0.4, 0.9, 0.5$? Can you explain your observations?

Anti-prior

Why does the "anti-prior" work well even though its dominant assumptions (most likely $p_H = 0$ or 1) are proven wrong early on?

Answer

```
The "heavy tails" (which in general means the probability away from the peaks;
in the middle for the "anti-prior") mean it is like uniform (renormalized!)
after the ends are eliminated. An important lesson for formulating priors:
allow for deviations from your expectations.
```

Note

A good reference about selecting priors is the Stan page on Prior Choice Recommendations.

Updating squentially (after every toss) vs. all at once

Let's do the simplest version of the question first: two tosses. Let the results of the first k tosses be $D = \{D_k\}$ (in practice take 0's and 1's as the two choices $\implies R = \sum_k D_k$).

- First consider $k = 1$ (so $D_1 = 0$ or $D_1 = 1$):

$$p(p_H|D_1, I) \propto p(D_1|p_H, I)p(p_H|I) \tag{6.1}$$

- Now $k = 2$, starting with the expression for updating all at once and then using the sum and product rules (including their corollaries, the marginalization and Bayes' Rule) to move the D_1 result to the right of the $|$ so it can be related to sequential updating:

$$\begin{aligned} p(p_H|D_2, D_1) &\propto p(D_2, D_1|p_H, I) \times p(p_H|I) \\ &\propto p(D_2|p_H, D_1, I) \times p(D_1|p_H, I) \times p(p_H|I) \\ &\propto p(D_2|p_H, D_1, I) \times p(p_H|D_1, I) \\ &\propto p(D_2|p_H, I) \times p(p_H|D_1, I) \\ &\propto p(D_2|p_H, I) \times p(D_1|p_H, I) \times p(p_H|I) \end{aligned} \tag{6.2}$$

Checkpoint question

What is the justification for each step in (6.2)?

Answer

- 1st line: Bayes' Rule
- 2nd line: Product Rule (applied to D_1)
- 3rd line: Bayes' Rule (going backwards)

- 4th line: tosses are independent
- 5th line: Bayes' Rule on the last term in the 3rd line

The fourth line of (6.2) is the sequential result! (The prior for the 2nd flip is the posterior (6.1) from the first flip.)

- So all at once is the same as sequential as a function of p_H, when normalized!
- To go to $k = 3$:

$$\begin{aligned} p(p_H|D_3, D_2, D_1, I) &\propto p(D_3|p_H, I)p(p_H|D_2, D_1, I) \\ &\propto p(D_3|p_H, I)p(D_2|p_H, I)p(D_1|p_H, I)p(p_H) \end{aligned}$$

and so on.

So they are not different, as long as the tosses are independent.

What about "bootstrapping"?

Why can't we use the data to improve the prior and apply it (repeatedly) for the *same* data. I.e., use the posterior from the first set of data as the prior for the same set of data.

Answer

Let's see what this leads to (we'll label the sequence of posteriors we get $p_1, p_2, \ldots, p_N$):

$$p_1(p_H|D_1, I) \propto p(D_1|p_H, I)\, p(p_H|I)$$

$$\begin{aligned} \implies p_2(p_H|D_1, I) &\propto p(D_1|p_H, I)\, p_1(p_H|D_1, I) \\ &\propto [p(D_1|p_H, I)]^2\, p(p_H|I) \end{aligned}$$

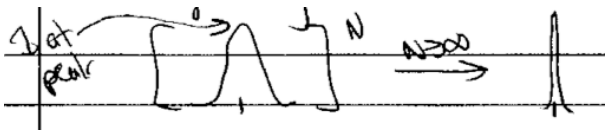
What if we keep going N times?

$$\begin{aligned} \implies p_N(p_H|D_1, I) &\propto p(D_1|p_H, I)\, p_{N-1}(p_H|D_1, I) \\ &\propto [p(D_1|p_H, I)]^N\, p(p_H|I) \end{aligned}$$

Suppose D_1 was 0, then $[p(\text{tails } |p_H, I)]^N \propto (1 - p_H)^N p(p_H|I) \overset{N\to\infty}{\longrightarrow} \delta(p_H)$ (i.e., the posterior is only at $p_H = 0$!).

Similarly, if D_1 was 1, then $[p(\text{heads } |p_H, I)]^N \propto p_H^N p(p_H|I) \overset{N\to\infty}{\longrightarrow} \delta(1 - p_H)$ (i.e., the posterior is only at $p_H = 1$.)

More generally, this bootstrapping procedure would cause the posterior to get narrower and narrower with each iteration so you think you are getting more and more certain, with no new data!



Warning

Don't do that!

Other questions

Checkpoint question

How often should the 68% credible interval contain the true answer for p_H?

Answer

68% of the time!

Discussion question

What would your standard be for deciding the coin was so unfair that you would walk away? That you'd call the police? That you'd try and publish the fact that you found an unfair coin in a scientific journal?

Checkpoint question

The frequentist point estimate is $p_H^* = \frac{H}{N}$. It actually corresponds to one of the point estimates from the Bayesian analysis for a specific prior. Which point estimate and what prior?

Answer

For a flat prior this is the mode (maximum) of the posterior. The posterior mean is $(H+1)/(N+2)$, which approaches H/N for large N.

6.5. Analytic coin-flipping posterior

In practical applications, we will very often find posteriors by *sampling*, which means we acquire a (generally multi-dimensional) set of outcomes, whose histogram approximates the exact posterior. In some cases we are able to derive an analytic expression for the posterior. This includes the case of *conjugate priors*, which we exploit below. In the process we see the flexibility of the beta distribution, which we use to parametrize a rather general prior distribution.

Here we derive analytic expressions for the coin-flipping posterior that are used in the demo notebooks 📥 Demo: Bayesian Coin Tossing and Demo: Widgetized coin tossing.

First the likelihood

Suppose we had a fair coin $\Longrightarrow p_H = 0.5$

$$p(H \text{ heads of out } \quad N \text{ tosses | fair coin } \quad) = p(H, N|p_H = 0.5) = \binom{N}{H}(0.5)^H(0.5)^{N-H}$$

Is the sum rule obeyed?

$$\sum_{H=0}^{N} p(H, N|p_H = 1/2) = \sum_{H=0}^{N} \binom{N}{H}\left(\frac{1}{2}\right)^N = \left(\frac{1}{2}\right)^N \times 2^N = 1$$

Proof of penultimate equality

$(x+y)^N = \sum_{H=0}^{N} \binom{N}{H} x^H y^{N-H} \overset{x=y=1}{\longrightarrow} \sum_{H=0}^{N} \binom{N}{H} = 2^N$. More generally, $x = p_H$ and $y = 1 - p_H$ shows that the sum rule works in general.

The likelihood for a more general p_H is the binomial distribution:

$$p(H, N|p_H) = \binom{N}{H}(p_H)^H(1-p_H)^{N-H} \tag{6.3}$$

Maximum-likelihood means: what value of p_H maximizes the likelihood (notation: $\mathcal{L}$ is often used for the likelihood)?

$$p(H, N|p_H) \equiv \mathcal{L}(H, N|p_H) = \mathcal{N} p_H^H (1-p_H)^{N-H} \, ?$$

Exercise: Carry out the maximization

But, as a Bayesian, we want to know about the PDF for p_H, so we actually want the PDF the other way around: $p(p_H|H, N)$. Bayes says

$$p(p_H|H, N) \propto p(H, N|p_H) \cdot p(p_H)$$

- Note that the denominator doesn't depend on p_H (it is just a normalization).

So how are we doing the calculation of the updated posterior?

- In this case we can do analytic calculations.

Case I: uniform (flat) prior

$$\implies \quad p(p_H|H, N, I) = \mathcal{N} p(H, N|p_H) p(p_H) \tag{6.4}$$

where we will suppress the "I" going forward. But

$$\int_0^1 dp_H\, p(p_H|H,N) = 1 \quad \Longrightarrow \quad \mathcal{N}\frac{\Gamma(1+N-H)\Gamma(1+H)}{\Gamma(2+N)} = 1 \tag{6.5}$$

Recall Beta function

$$B(x,y) = \int_0^1 t^{x-1}(1-t)^{y-1}\,dt = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)} \quad \text{for Re}\,(x,y) > 0 \tag{6.6}$$

and $\Gamma(x) = (x-1)!$ for integers.

$$\Longrightarrow \quad \mathcal{N} = \frac{\Gamma(2+N)}{\Gamma(1+N-H)\Gamma(1+H)} \tag{6.7}$$

and so evaluating the posterior for p_H for new values of H and N is direct: substitute (6.7) into (6.4). If we want the unnormalized result with a uniform prior (meaning we ignore the normalization constant $\mathcal{N}$ that simply gives an overall scaling of the distribution), then we just use the likelihood (6.3) since $p(p_H) = 1$ for this case.

Case II: conjugate prior

Choosing a conjugate prior (if possible) means that the posterior will have the same form as the prior. Here if we pick a beta distribution as prior, it is conjugate with the coin-flipping likelihood. From the scipy.stats.beta documentation the beta distribution (function of x with parameters a and b):

$$f(x,a,b) = \frac{\Gamma(a+b)x^{a-1}(1-x)^{b-1}}{\Gamma(a)\Gamma(b)} \tag{6.8}$$

where $0 \le x \le 1$ and $a > 0, b > 0$. So $p(x|a,b) = f(x,a,b)$ and our likelihood, as a function of p_H, is proportional to a beta distribution, $p(H,N|p_H) \propto f(p_H, 1+H, 1+N-H)$, to agree with (6.3).

If the prior is $p(p_H|I) = f(p_H,\alpha,\beta)$ with α and β to reproduce our prior expectations (or knowledge), then by Bayes' theorem the *normalized* posterior is

$$\begin{aligned} p(p_H|H,N) &\propto p(H,N|p_H)p(p_H) \\ &\propto f(p_H,1+H,1+N-H) \times f(p_H,\alpha,\beta) \\ &\longrightarrow f(p_H,\alpha+H,\beta+N-H) \end{aligned} \tag{6.9}$$

so we *update the prior* simply by changing the arguments of the beta distribution: $\alpha \to \alpha + H$, $\beta \to \beta + N - H$ because the (normalized) product of two beta distributions is another beta distribution. Really easy!

Warning

Check this against the code! Look in the code where the posterior is calculated and see how the beta distribution is used. Verify that (6.9) is evaluated. Try changing the values of α and β used in defining the prior

to see the shapes.

7. Error propagation

In Bayesian statistics, error propagation is the process of determining how uncertainties in input quantities, which are characterized by probability distributions, influence the uncertainty (again, as a PDF) in the final output of a model. In general, Bayesian methods use the full PDFs to propagate uncertainty, in contrast to frequentist approaches. In this chapter we look at three aspects of error propagation:

- Nuisance parameters and marginalization
- Changing variables
- A useful approximation

7.1. Nuisance parameters and marginalization

The Bayesian approach offers a straightforward approach for dealing with (known) systematic uncertainties; namely by marginalization.

For example, assume that we have a model with two parameters, θ and ϕ, although only one of them (say θ) is of physical relevance. The other one is then labeled a *nuisance parameter*. If through a Bayesian data analysis we get the joint posterior PDF

$$p\left(\theta, \phi \mid \mathcal{D}, I\right), \tag{7.1}$$

then the marginal posterior PDF $p\left(\theta \mid \mathcal{D}, I\right)$ is obtained via marginalization

$$p\left(\theta \mid \mathcal{D}, I\right) = \int p\left(\theta, \phi \mid \mathcal{D}, I\right) d\phi. \tag{7.2}$$

Thus we have directly derived how to propagate the uncertainty in ϕ to the probability distribution for θ.

Example 7.2 provides an illustration of this scenario.

Marginalization using samples

Assume that we have N samples from the joint pdf $p\left(\theta, \phi \mid \mathcal{D}, I\right)$. This might be a sample chain from an MCMC sampler: $\{(\theta, \phi)_i\}_{i=0}^{N-1}$. Think of this as two columns of length N. Then the marginal distribution of θ will be given by the same chain by simply ignoring the ϕ column, i.e., $\{\theta_i\}_{i=0}^{N-1}$.

See the Visualizations of MCMC created by Chi Feng for an illustration of this. In these simulations, the marginalization is performed by projecting the joint samples into a histogram on the x or y axis.

A slightly more general scenario for which the marginalization procedure is used for error propagation is the following: Assume that we have measured or inferred distributions for the random variables X and Y; what can we say about the distributions for the difference $X - Y$ or the raio X/Y, or the sum of their squares $X^2 + Y^2$, etc?

Such questions can be rephrased as: Given the joint PDF $p\left(x, y \mid I\right)$, what is $p\left(z \mid I\right)$, where $z = f(x, y)$? In this situation we can use marginalization and the product rule.

$$p\left(z \mid I\right) = \int p\left(z, x, y \mid I\right) dx\, dy = \int p\left(z \mid x, y, I\right) p\left(x, y \mid I\right) dx\, dy. \tag{7.3}$$

We first realize that $p\left(z \mid x, y, I\right) = \delta(z - f(x, y))$ due to the deterministic functional relationship between the parameters.

The joint PDF for X and Y becomes a product if the errors are independent $p\left(x, y \mid I\right) = p\left(x \mid I\right) p\left(y \mid I\right)$. Then the delta function can be used to evaluate one of the integrals. Assuming that $z = f(x, y)$ can be solved for y, giving the inverse transformation $y = g(x, z)$, we get

$$p\left(z \mid I\right) = \int p\left(x \mid I\right) p\left(y = g(x, z) \mid I\right) \left|\frac{\partial g(x, z)}{\partial z}\right| dx. \tag{7.4}$$

The Jacobian factor appears since $\delta(z - f(x, y)) = \delta(y - g(x, z))\, |\partial g(x, z)/\partial z|$. For a sum or a difference, $Z = X \pm Y$, this factor equals one and the PDF for Z becomes a convolution.

Example 7.1 ($Z = X + Y$)

Let us consider the situation where you are interested in the quantity $Z = X + Y$, and you know that X and Y are distributed as independent Gaussians with $\mathbb{E}\left[X\right] = x_0$, Var $\left(X\right) = \sigma_x^2$ and $\mathbb{E}\left[Y\right] = y_0$, Var $\left(Y\right) = \sigma_y^2$. Then

$$\begin{aligned} p\left(x, y \mid I\right) &= p\left(x \mid I\right) p\left(y \mid I\right) \\ &= \frac{1}{2\pi\sigma_x\sigma_y} \exp\left[-\frac{\left(x - x_0\right)^2}{2\sigma_x^2}\right] \exp\left[-\frac{\left(y - y_0\right)^2}{2\sigma_y^2}\right]. \end{aligned} \tag{7.5}$$

Let us now use marginalization and the product rule to find

$$p\left(z \mid I\right) = \int p\left(z, x, y \mid I\right) dx\, dy = \int p\left(z \mid x, y, I\right) p\left(x, y \mid I\right) dx\, dy. \tag{7.6}$$

We realize that $p\left(z \mid x, y, I\right) = \delta(z - (x + y))$ due to the functional relationship between the random variables, and we have $p\left(x, y \mid I\right)$ as a product of Gaussian PDFs from above. The delta function can be used to evaluate one of the integrals, and the PDF for Z becomes a convolution

$$\begin{aligned} p\left(z \mid I\right) &= \int p\left(x \mid I\right) p\left(y = z - x \mid I\right) dx \\ &= \frac{1}{2\pi\sigma_x\sigma_y} \int \exp\left[-\frac{\left(x - x_0\right)^2}{2\sigma_x^2}\right] \exp\left[-\frac{\left(z - x - y_0\right)^2}{2\sigma_y^2}\right] dx. \end{aligned} \tag{7.7}$$

After some algebra that involves completing the square for X in the exponent we obtain

$$p\left(z \mid I\right) = \frac{1}{\sqrt{2\pi}\sigma_z}\exp\left[-\frac{\left(z-z_0\right)^2}{2\sigma_z^2}\right], \tag{7.8}$$

with $z_0 = x_0 + y_0$ and $\sigma_z^2 = \sigma_x^2 + \sigma_y^2$. Thus, the PDF for the sum $Z = X + Y$, with X and Y being described by Gaussian PDFs, is another Gaussian, with the uncertainties "adding in quadrature".

Exercise 7.1 (Correlated errors)

For correlated errors, the joint PDF $p\left(x, y \mid I\right)$ does not factorize into a product. Consider, for example, a situation with a bivariate normal distribution

$$p\left(x, y \mid I\right) = \frac{1}{(2\pi)|\mathbf{\Sigma}|^{1/2}}\exp\left(-\frac{1}{2}(x,y)\mathbf{\Sigma}^{-1}(x,y)^T\right).$$

Furthermore, let us assume that the variances in X and Y are the same ($\sigma_x^2 = \sigma_y^2 \equiv \sigma^2$) and the covariance is positive and almost as large as the variance (i.e., errors are strongly correlated). Then, the linear transformation $S = (X - Y)/\sqrt{2}$ and $T = (X + Y)/\sqrt{2}$ gives two new random variables S and T that turn out to be independent. This implies

$$p\left(x, y \mid I\right) = \left\{\begin{matrix} s = \frac{x-y}{\sqrt{2}} \\ t = \frac{x+y}{\sqrt{2}} \end{matrix}\right\} = \frac{1}{\sqrt{2\pi}\sigma_s}\exp\left(-\frac{s^2}{2\sigma_s^2}\right)\frac{1}{\sqrt{2\pi}\sigma_t}\exp\left(-\frac{t^2}{2\sigma_t^2}\right).$$

Furthermore, given the strong positive correlation one finds that $\sigma_s \ll \sigma$. What is the PDF for the difference $Z = X - Y$ in this situation?

Exercise 7.2 (Sum of two Gaussian PDFs)

Complete the derivation of Eq. (7.8).

Example 7.2 (Inferring galactic distances)

The Hubble constant H acts as a galactic ruler as it is used to measure astronomical distances (d) according to $v = Hd$, where v is the velocity that can be measured with observations of redshifts. An error in H will therefore correspond to a systematic uncertainty in such measurements.

A modern value for the Hubble constant is $H = 70 \pm 10$ km s^{-1} MPc^{-1}. We introduce the expectation value $\mathbb{E}\left[H\right] = H_0 = 70$ km s^{-1} MPc^{-1} and the standard deviation Std $\left(H\right) = \sigma_H = 10$ km s^{-1} MPc^{-1}. Note that astronomical distances are usually measured in million parsecs (MPc). Suppose that a specific galaxy has a measured recessional velocity $v_0 = 100 \times 10^3$ km/s with an observational error quantified by a standard deviation $\sigma_v = 5 \times 10^3$ km/s. Determine the posterior PDF for the distance to a galaxy under the following three different analysis assumptions:

1. We use the expectation value of H in the analysis: $H = H_0 = 70$ km s^{-1} MPc^{-1}.
2. We include the uncertainty in the value of the Hubble constant via a Gaussian PDF:

$$p\left(H \mid I\right) = \frac{1}{\sqrt{2\pi}\sigma_H}\exp\left(-\frac{\left(H-H_0\right)^2}{2\sigma_H^2}\right).$$

3. We include the uncertainty in the value of the Hubble constant via a uniform PDF:

$$p\left(H \mid I\right) = \begin{cases} \frac{1}{4\sigma_H} & \text{for } \; H_0 - 2\sigma_H \leq H \leq H_0 + 2\sigma_H, \\ 0 & \text{otherwise.} \end{cases}$$

In addition, an approximate propagation of errors will be demonstrated in Section 7.3 in Example 7.3.

Here we use marginalization to obtain the desired posterior PDF $p\left(d \mid \mathcal{D}, I\right)$ from the joint distribution of $p\left(d, H \mid \mathcal{D}, I\right)$

$$p\left(d \mid \mathcal{D}, I\right) = \int_{-\infty}^{\infty} p\left(d, H \mid \mathcal{D}, I\right) dH. \tag{7.9}$$

Using

- Bayes' rule: $p\left(d, H \mid \mathcal{D}, I\right) \propto p\left(\mathcal{D} \mid d, H, I\right) p\left(d, H \mid I\right)$,
- the product rule: $p\left(d, H \mid I\right) = p(H|d, I) p\left(d \mid I\right)$,
- and the fact that H is independent of d: $p(H|d, I) = p\left(H \mid I\right)$,

we find that

$$p\left(d \mid \mathcal{D}, I\right) \propto p\left(d \mid I\right) \int p\left(H \mid I\right) p\left(\mathcal{D} \mid d, H, I\right) dH, \tag{7.10}$$

which means that we have expressed the quantity that we want (the posterior of d) in terms of quantities that we can specify, as follows.

First, we need to state our prior knowledge concerning the distance $p\left(d \mid I\right)$. You might ask yourself what information is contained in this quantity. It should summarize our state of knowledge *before* accumulating the new data. For starters, we do know that it is a distance (meaning that it is a positive quantity) and we would also expect it to be smaller than the (visible) size of the universe. We might argue that it should be at least as far away as the previously known closest galaxy. Further arguments can be made on the basis of indifference, but we will defer such discussions for now. For simplicity, we just assign a uniform PDF

$$p\left(d \mid I\right) \propto 1,$$

within some huge range $0 < d < 10^9$ MPc.

The likelihood for the observed can be written down from a statistical model

$$v_{\text{measured}} = Hd + \delta\mathcal{D},$$

where Hd represents our model for the velocity and $\delta\mathcal{D}$ is the observational error, which is a random variable. We assume that $\mathbb{E}\left[\delta\mathcal{D}\right] = 0$ and Var $\left(\delta\mathcal{D}\right) = \sigma_v^2$, such that $\mathbb{E}\left[v_{\text{measured}}\right] = Hd$ and Var $\left(v_{\text{measured}}\right) = \sigma_v^2$.

Given this information it is motivated to use a Gaussian PDF (with this mean and variance) for the likelihood

$$p\left(\mathcal{D} \mid d, H, I\right) = \frac{1}{\sqrt{2\pi}\sigma_v}\exp\left(-\frac{(v_0 - Hd)^2}{2\sigma_v^2}\right).$$

The results for the different analysis strategies (1-3 in the list above) should be obtained in Exercise 7.3. A comparison figure that also includes the approximate error propagation from Example 7.3 is shown in the solution to this exercise.

Exercise 7.3 (Inferring galactic distances)

Apply the three analysis strategies and derive the posterior distribution $p\left(d \mid \mathcal{D}, I\right)$. The first case allows an analytical solution while the second and third are probably best approached numerically (although it is, in fact, possible to find an analytical result also for case 2). Plot the results.

Hint 1: Since the goal is to extract a probability distribution, and we are already ignoring the denominator in Bayes' theorem, it is allowed to ignore all normalization constants. The final PDFs can be normalized by integration.

Hint 2: A fixed value for H can be assigned with the PDF $p\left(H \mid I\right) = \delta(H - H_0)$, where $\delta(x)$ is the Dirac delta function.

Solutions to exercises

Solution to Exercise 7.1 (Correlated errors)

$$\begin{aligned} p\left(z \mid I\right) &= \int p\left(z \mid x, y, I\right) p\left(x, y \mid I\right) dx\, dy = \int \delta(z - \sqrt{2}s) p(s|I) p(t|I)\, ds\, dt \\ &= \left\{\begin{array}{l} \tilde{s} = \sqrt{2}s \;\Rightarrow\; ds = d\tilde{s}/\sqrt{2} \\ \delta(z - \sqrt{2}s) \rightarrow \delta(z/\sqrt{2} - \tilde{s}) \end{array}\right\} = \frac{1}{\sqrt{2}\sqrt{2\pi}\sigma_s}\exp\left(-\frac{z^2}{4\sigma_s^2}\right)\int p(t|I)\, dt. \end{aligned}$$

Here we can use that $\int p(t|I)\, dt = 1$. Defining $\sigma_z = \sqrt{2}\sigma_s$ we find

$$p\left(z \mid I\right) = \frac{1}{\sqrt{2\pi}\sigma_z}\exp\left(-\frac{z^2}{2\sigma_z^2}\right),$$

i.e. a normal distribution for z with a small variance $2\sigma_s^2$.

Solution to Exercise 7.3 (Inferring galactic distances)

1. A fixed value for H can be assigned with the PDF $p\left(H \mid I\right) = \delta(H - H_0)$, where $\delta(x)$ is the Dirac delta function. We note that integrals over a delta function are given by $\int_{-\infty}^{+\infty} f(x)\delta(x - x_0)\, dx = f(x_0)$ such that

$$p\left(d \mid \mathcal{D}, I\right) \propto \exp\left(-\frac{\left(v_0 - H_0 d\right)^2}{2\sigma_v^2}\right),$$

where we have ignored all normalization coefficients.

2. With a Gaussian prior for H we will be left with an integral

$$p\left(d \mid \mathcal{D}, I\right) \propto \int_{-\infty}^{+\infty} dH \, \exp\left(-\frac{\left(H - H_0\right)^2}{2\sigma_H^2}\right) \exp\left(-\frac{\left(v_0 - Hd\right)^2}{2\sigma_v^2}\right).$$

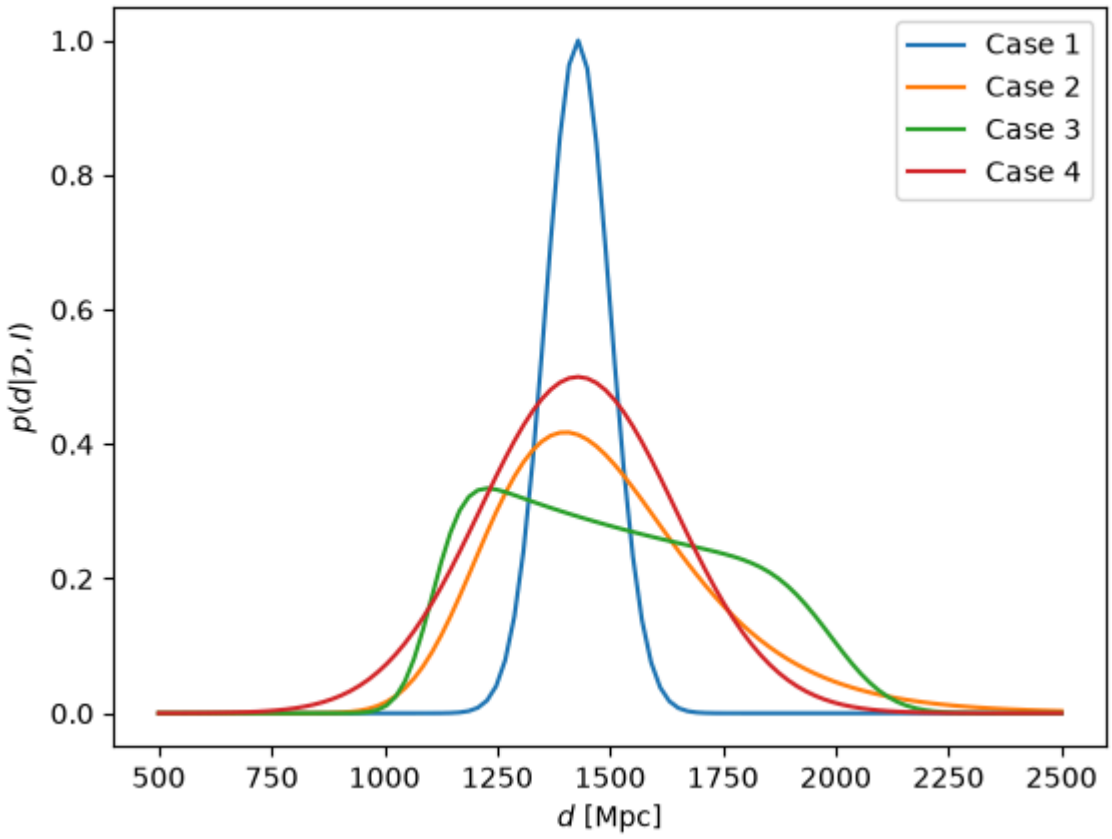
We can perform this integral numerically, or we can solve it analytically by realizing that the product of two Gaussian distributions is another Gaussian distribution.

3. The uniform prior for $p\left(H \mid I\right)$ implies that the final integral becomes

$$p\left(d \mid \mathcal{D}, I\right) \propto \int_{H_0 - 2\sigma_H}^{H_0 + 2\sigma_H} dH \exp\left(-\frac{\left(v_0 - Hd\right)^2}{2\sigma_v^2}\right).$$

For numerical integration, and plots of the results of the three inference strategies, see the hidden code block below.

▼ **Hide code cell content**



7.2. Changing variables

Let us consider a single variable X and a function $Y = f(X)$ that offers a unique mapping between X and Y. Assume that we know X via a PDF $p\left(x \mid I\right)$. What is the relation between $p\left(x \mid I\right)$ and $p\left(y \mid I\right)$? In this scenario the extraction of $p\left(y \mid I\right)$ turns out to be an exercise in transformation of variables.

Consider a small interval between a point x^* and $x^* + \delta x$. The probability that X lies within that interval can be written

$$p\left(x^* \leq x < x^* + \delta x \mid I\right) \approx p\left(x = x^* \mid I\right)\delta x.$$

This probability can't depend on how the interval is parametrized, so it must be the same as the probability between y^* and $y^* + \delta y$, where $y^* = f(x^*)$ and $y^* + \delta y = f(x^* + \delta x)$. Thus, we have

$$p\left(x = x^* \mid I\right)\delta x = p\left(y = y^* \mid I\right)\delta y.$$

In the limit of infinitesimally small intervals, and with the realization that this should be true for any point x, we obtain the relationship

$$p\left(x \mid I\right) = p(y = y(x)|I) \left|\frac{dy}{dx}\right|, \tag{7.11}$$

where the term on the far right is called the *Jacobian*. We also note that we can invert the transformation

$$p\left(y \mid I\right) = p\left(x(y) \mid I\right) \left|\frac{dx}{dy}\right|. \tag{7.12}$$

The generalization to several variables, relating the PDF for M variables $\{x_j\}$ in terms of the same number of quantities $\{y_j\}$ related to them, is

$$p(\{x_j\}|I) = p(\{y_j\}|I) \left|\frac{\partial(y_1, y_2, \ldots, y_M)}{\partial(x_1, x_2, \ldots, x_M)}\right|, \tag{7.13}$$

where the term on the far right is called the Jacobian of the transformation from x to y. In general it is given by the determinant of the $M \times M$ matrix of partial derivatives $\partial y_i / \partial x_j$.

Note that (7.11) is also how physical densities transform: if I know the mass density per unit length in variable x, but I want it in variable y, the relationship between the two is exactly (7.11). $p(x|I)$ and $p(y = y(x)|I)$ are not the same distribution because they're probability densities, not probabilities.

Summary

We have now seen the basic ingredients required for the propagation of errors: it either involves a transformation in the sense of Eq. (7.13) or an integration as in Eq. (7.4).

Exercise 7.4 (The standard random variable)

Find $p\left(z \mid I\right)$ when $Z = (X - \mu)/\sigma$ and $p\left(x \mid I\right) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$.

Exercise 7.5 (The square root of a number)

Find an expression for $p\left(z \mid I\right)$ when $Z = \sqrt{X}$ and $p\left(x \mid I\right) = \frac{1}{x_{\max} - x_{\min}}$ for $x_{\min} \leq x \leq x_{\max}$ and 0 elsewhere. Verify that $p\left(z \mid I\right)$ is properly normalized.

Solutions to exercises

Solution to Exercise 7.4 (The standard random variable)

The transformation $z = f(x) = (x - \mu)/\sigma$ gives the inverse $x = f^{-1}(z) = \sigma z + \mu$ and the Jacobian $|dx/dz = \sigma|$.

Therefore $p\left(z \mid I\right) = p\left(x \mid I\right)\sigma$. With the given form of $p\left(x \mid I\right)$ we get

$$p\left(z \mid I\right) = \frac{1}{\sqrt{2\pi}}\exp\left(-\frac{z^2}{2}\right),$$

which corresponds to a Gaussian distribution with mean zero and variance one, sometimes known as a standard random variable.

Solution to Exercise 7.5 (The square root of a number)

With $z = f(x) = \sqrt{x}$ we have $x = f^{-1}(z) = z^2$ such that $|dx/dz| = 2|z|$. We note that z is positive such that $|z| = z$ and we therefore have

$$p\left(z \mid I\right) = 2zp\left(x \mid I\right) = 2z\frac{1}{x_{\max} - x_{\min}} \quad \text{for} \quad \sqrt{x_{\min}} \leq z \leq \sqrt{x_{\max}},$$

and 0 elsewhere.

We check the normalization by performing the integral

$$\int_0^\infty p\left(z \mid I\right) dz = \int_{\sqrt{x_{\min}}}^{\sqrt{x_{\max}}} \frac{2z}{x_{\max} - x_{\min}} dz = \frac{1}{x_{\max} - x_{\min}}\left[z^2\right]_{\sqrt{x_{\min}}}^{\sqrt{x_{\max}}} = 1.$$

7.3. A useful approximation

We have seen in Section 7.1 and Section 7.2 how to exactly calculate the probability distribution for a random variable that is a function of other random variables. That is, given distributions for random variables X and Y, we can find the distribution for $Z = f(X, Y)$. But it will often be sufficient to approximate PDFs with (uncorrelated) Gaussian distributions, in which case we can anticipate being able to approximate how such distributions combine.

Intuitively, if the distributions for X and Y are sharply peaked about their means, we expect we can linearize $f(X,Y)$ and then Z will also be a Gaussian. Let's see how this plays out successfully and where it can fail.

If we assume X and Y are independent, then in the Gaussian approximation, the distributions for X and Y are fully characterized by their means μ_X, μ_Y and variances σ_X^2, σ_Y^2, respectively. These are given by expectation values (recall Expectation values and moments)

$$\mu_X = \mathbb{E}[X], \quad \mu_Y = \mathbb{E}[Y] \tag{7.14}$$

$$\sigma_X^2 = \mathbb{E}\left[(\delta X)^2\right], \quad \sigma_Y^2 = \mathbb{E}\left[(\delta Y)^2\right], \quad \sigma_{XY} = \mathbb{E}[\delta X \delta Y] = \mathbb{E}[\delta X] \times \mathbb{E}[\delta Y] = 0, \tag{7.15}$$

where we have defined the deviations from the mean

$$\delta X \equiv X - \mu_X, \quad \delta Y \equiv Y - \mu_Y,$$

and used that δX and δY are independent zero-mean distributions.

By assumption, we should be able to expand about the means, so for small δX and δY we Taylor expand (here to second order):

$$\begin{aligned} Z &= f(X,Y) \\ &\approx f(\mu_X,\mu_Y) + f_X \delta X + f_Y \delta Y + \frac{1}{2} f_{XX} (\delta X)^2 + \frac{1}{2} f_{YY} (\delta Y)^2 + f_{XY} \delta X \delta Y + \ldots \end{aligned} \tag{7.16}$$

where we use the abbreviations

$$f_X \equiv \left.\frac{\partial f}{\partial X}\right|_{\mu_X,\mu_Y}, \quad f_Y \equiv \left.\frac{\partial f}{\partial Y}\right|_{\mu_X,\mu_Y},$$

$$f_{XX} \equiv \left.\frac{\partial^2 f}{\partial X^2}\right|_{\mu_X,\mu_Y}, \quad f_{YY} \equiv \left.\frac{\partial^2 f}{\partial Y^2}\right|_{\mu_X,\mu_Y}, \quad f_{XY} \equiv \left.\frac{\partial^2 f}{\partial X \partial Y}\right|_{\mu_X,\mu_Y}.$$

Let's first work to linear order (i.e., keep through δX and δY). Then Z is a linear combination of Gaussian distributions, hence is itself Gaussian. We can find its mean and variance by using the linearity of expectation values and that the δX and δY are independent, zero-mean distributions:

$$\mu_Z = \mathbb{E}[Z] = \mathbb{E}[f(X,Y)] \approx f(\mu_X,\mu_Y) \tag{7.17}$$

$$\begin{aligned} \sigma_Z^2 &= \mathbb{E}\left[(Z-\mu_Z)^2\right] \approx \mathbb{E}\left[f_X^2 (\delta X)^2 + f_Y^2 (\delta Y)^2 + 2 f_X f_Y \delta X \delta Y\right] \\ &\approx f_X^2 \sigma_X^2 + f_Y^2 \sigma_Y^2. \end{aligned} \tag{7.18}$$

Summary

The application of these approximations:

$$\mu_Z \approx f(\mu_X, \mu_Y), \quad \sigma_Z^2 \approx f_X^2 \sigma_X^2 + f_Y^2 \sigma_Y^2 \tag{7.19}$$

is sometimes called *Gaussian error propagation*. In fact, these relations apply at this linear order regardless of how the random variables X and Y are distributed. Although the above discussion was motivated by assuming they were normally distributed nothing we have done so far actually used that. The derivation does, though, assume that X and Y are independent random variables.

Checkpoint question

Why are expectation values linear operations?

Answer

Recall from Expectation values and moments that expectation values are integrals over probability definitions. As such, they inherit linearity from the linearity of integrals.

Let's apply the formulas in (7.19) to $Z = X - Y$. For the mean we get $\mu_Z = \mu_X - \mu_Y$, which is intuitive. For the variance the relevant derivatives are $f_X = +1$ and $f_Y = -1$, so we find that

$$\sigma_z^2 = \sigma_x^2 + \sigma_y^2,$$

i.e., the familiar result that variances add in quadrature. Since the expansion of f in this case truncates at linear order, these results are exact.

Example 7.3 (Inferring galactic distances—revisited)

Consider, as a second example, the ratio of two parameters $Z = X/Y$ that appeared in Example 7.2 (in which we wanted to infer $d = v/H$).

Applying (7.19), the mean is

$$\mu_Z = \frac{\mu_X}{\mu_Y}$$

and, with $f_X = 1/\mu_Y$ and $f_Y = -\mu_X/\mu_Y^2$, we find for the propagated variance

$$\sigma_Z^2 = \frac{\sigma_X^2}{\mu_Y^2} + \frac{\mu_X^2}{\mu_Y^4}\sigma_Y^2$$

or

$$\frac{\sigma_Z^2}{\mu_Z^2} = \frac{\sigma_X^2}{\mu_X^2} + \frac{\sigma_Y^2}{\mu_Y^2},$$

(we can take the square root to get the ratio of standard deviation to mean).

Exercise 7.6 (Linear combination of Gaussians)

Consider $Z = aX + bY$, with a and b constants. Derive a PDF for Z assuming Gaussian errors in X and Y and applying (7.19). Compare with the result for $X + Y$ from the full convolution of PDFs in Example 7.1.

Exercise 7.7 (Gaussian product of errors)

Consider $Z = XY$ and derive a PDF for Z assuming Gaussian errors in X and Y and applying (7.19).

What if the first-order approximation is not enough? We can return to (7.17) and (7.18) and keep the second-order terms in (7.16). We find improved formulas:

$$\begin{aligned} \mu_Z &\approx f(\mu_X, \mu_Y) + \frac{1}{2} f_{XX}\sigma_X^2 + \frac{1}{2} f_{YY}\sigma_Y^2 \\ \sigma_Z^2 &\approx f_X^2\sigma_X^2 + f_Y^2\sigma_Y^2 + \frac{1}{2} f_{XX}^2(\sigma_X^2)^2 + \frac{1}{2} f_{YY}^2(\sigma_Y^2)^2 + f_{XY}^2\sigma_X^2\sigma_Y^2. \end{aligned} \tag{7.20}$$

In this case it is relevant that we assumed independent Gaussian distributions for X and Y.

Exercise 7.8 (Deriving second-order results)

Derive the results in (7.20). You will need to use that the skewness for a Gaussian distribution is zero and that the kurtosis can be expressed in terms of the variance.

So where can these approximations fail? The first-order approximation can perform poorly for strongly nonlinear transformations (so that higher-order corrections are important), ratios with denominators near zero, or transformations that impose boundaries or produce substantial skewness. The second-order corrections may help, but can also fail completely in some cases. Consider the following example.

Example 7.4 (Taking the square root of a number)

This example was adapted from [SS06].

- Assume that the amplitude of a Bragg peak is measured with an uncertainty $A = A_0 \pm \sigma_A$ from a least-squares fit to experimental data.
- The Bragg peak amplitude is proportional to the square of a complex structure function: $A = |F|^2 \equiv f^2$.
- What is $f = f_0 \pm \sigma_f$?

If we blindly apply (7.19), we find

$$f_0 = \sqrt{A_0} \quad \sigma_f^2 = \frac{\sigma_A^2}{4A_0}.$$

But what happens if the best fit gives $A_0 < 0$, which would not be impossible if we have weak and strongly overlapping peaks. The above equation obviously does not work since f_0 would be a complex number and the variance would be negative.

We have made two mistakes:

1. The likelihood is not the posterior!
2. The Gaussian error approximation around the peak does not always work.

Consider first the best fit of the signal peak. It implies that the likelihood can be approximated by

$$p\left(\mathcal{D} \mid A, I\right) \propto \exp\left[-\frac{(A-A_0)^2}{2\sigma_A^2}\right]. \tag{7.21}$$

However, the posterior for A is $p\left(A \mid \mathcal{D}, I\right) \propto p\left(\mathcal{D} \mid A, I\right) p\left(A \mid I\right)$ and we should use the fact that we know that $A \geq 0$.

We will incorporate this information through a simple step-function prior

$$p\left(A \mid I\right) = \begin{cases} \frac{1}{A_{\max}}, & 0 \leq A \leq A_{\max}, \\ 0, & \text{otherwise.} \end{cases} \tag{7.22}$$

This implies that the posterior will be a truncated Gaussian, and its maximum will always be above zero.

This also implies that we cannot use the Gaussian error propagation approximation. Instead we will do the proper calculation using the transformation (7.11)

$$p(f|\mathcal{D}, I) = p\left(A \mid \mathcal{D}, I\right) \left|\frac{dA}{df}\right| = 2fp\left(A \mid \mathcal{D}, I\right) \tag{7.23}$$

In the end we find the proper Bayesian error propagation given by the PDF

$$p(f|\mathcal{D}, I) \propto \begin{cases} f\exp\left[-\frac{(f^2-A_0)^2}{2\sigma_A^2}\right], & 0 \leq f \leq \sqrt{A_{\max}}, \\ 0, & \text{otherwise.} \end{cases} \tag{7.24}$$

Fig. 7.1 visualizes the difference between the Bayesian and the naive error propagation for a few scenarios. The code to generate these plots is in the hidden cell below.

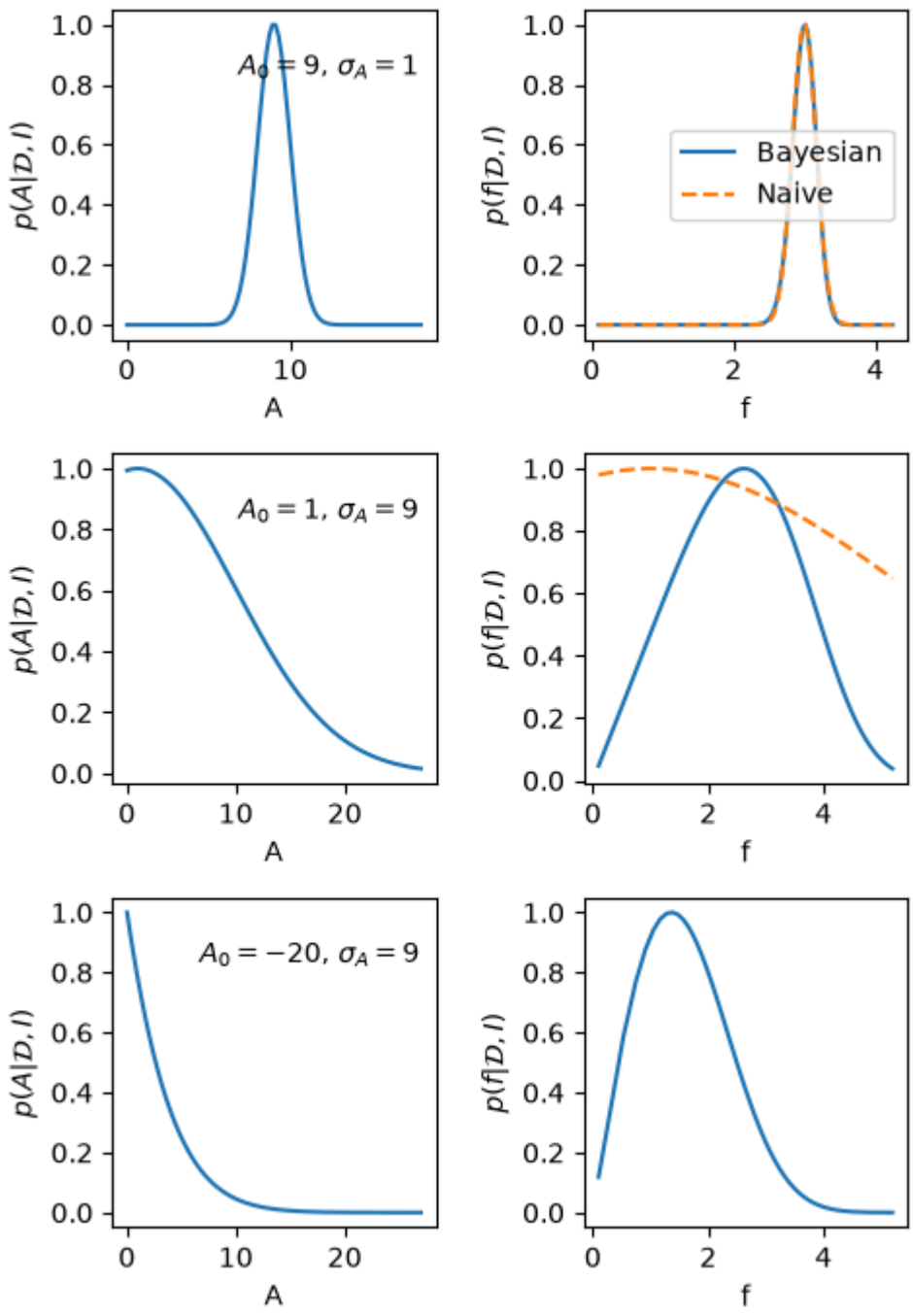
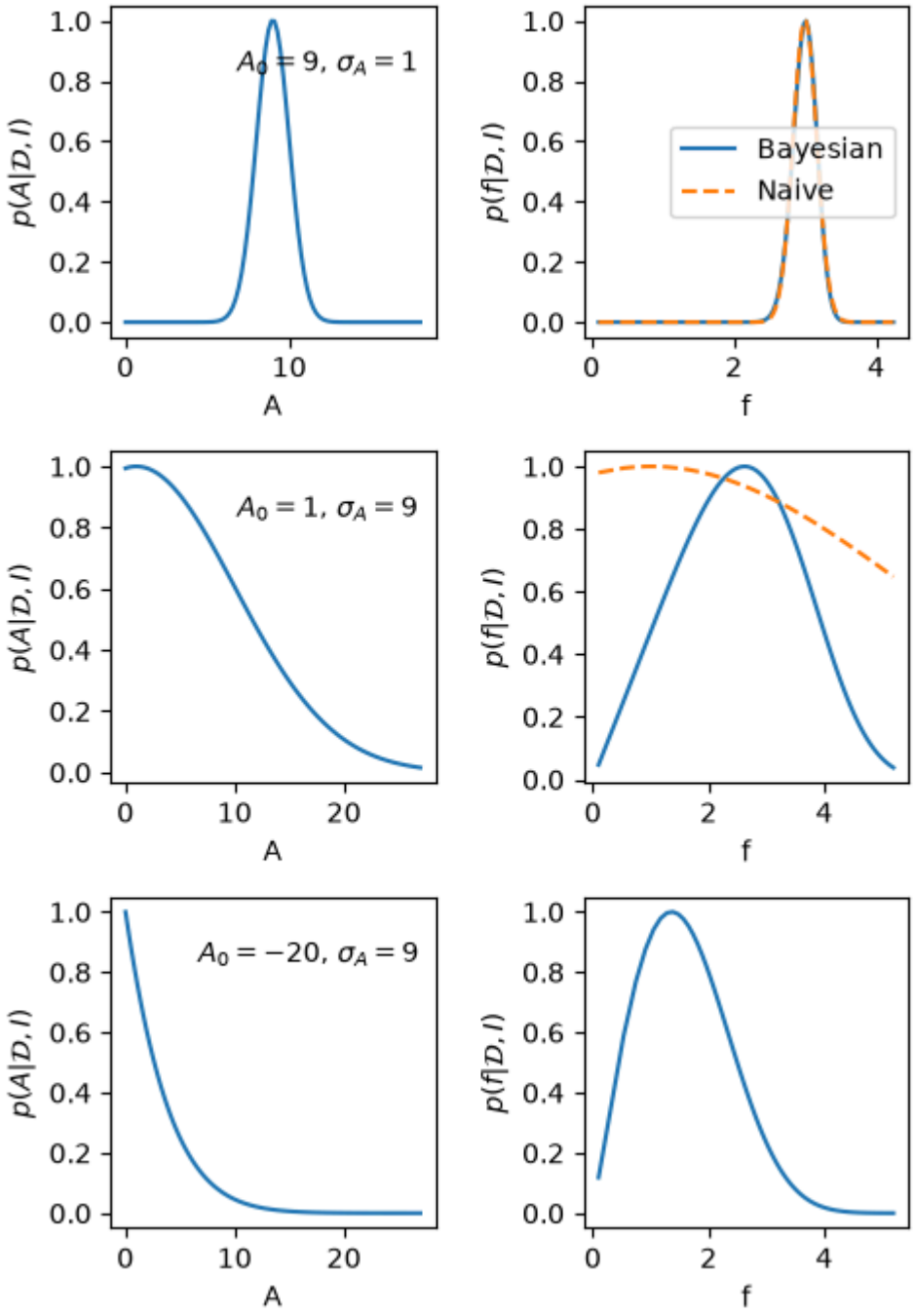


Fig. 7.1 The left-hand panels show the posterior PDF for the amplitude of a Bragg peak in three different scenarios. The right-hand plots are the corresponding PDFs for the modulus of the structure factor $f = \sqrt{A}$. The solid lines correspond to a full Bayesian error propagation, while the dashed lines are obtained with the short-cut error propagation. The short-cut approximation works well for the first case, poorly for the second case, and fails completely for the third case where $A_0 < 0$.

▼ **Hide code cell content**



Solutions to exercises

Solution to Exercise 7.6 (Linear combination of Gaussians)

The PDF $p\left(Z \mid I\right)$ is Gaussian with mean $\mathbb{E}\left[Z\right] = \mu_Z = a\mu_X + b\mu_Y$ and variance Var $\left(Z\right) = \sigma_Z^2 = a^2\sigma_X^2 + b^2\sigma_Y^2$, where μ_X, μ_Y and σ_X^2, σ_Y^2 are the means and variances of X and Y, respectively.

For $a = b = 1$ this is the same result as in Example 7.1, which should not be surprising since the errors were in fact Gaussian.

Solution to Exercise 7.7 (Gaussian product of errors)

The PDF $p\left(Z \mid I\right)$ is Gaussian with mean $\mathbb{E}\left[Z\right] = \mu_Z = \mu_X \mu_Y$ and variance $\text{Var}\ \left(Z\right) = \sigma_Z^2 = \mu_Y^2 \sigma_X^2 + \mu_X^2 \sigma_Y^2$, where μ_X, μ_Y and σ_X^2, σ_Y^2 are the means and variances of X and Y, respectively.

Solution to Exercise 7.8 (Deriving second-order results)

We get the second-order expression for μ_Z immediately by applying (7.17) and (7.15).

For the variance, when applying (7.18) we need to remember to apply both the expansion of Z *and* the expansion of μ_Z to second order. Once squared, there will be expectation values of third-order terms and fourth-order terms; after using independence to factorize terms, any expectation value of an odd power of δX or δY vanishes for zero-mean Gaussian distributions (mean and skewness are both zero). As before, any expectation value of quadratic terms yields variances, but now we have $\mathbb{E}\left[(\delta X)^4\right] = 3(\sigma_X^2)^2$ and $\mathbb{E}\left[(\delta Y)^4\right] = 3(\sigma_Y^2)^2$ for Gaussian distributions. When the dust settles, terms like $f_{XX} f_{YY} \sigma_X^2 \sigma_Y^2$ will have canceled and the factors work out to reproduce (7.20).

8. Bayes in practice

In this chapter we finish Part I with some topics addressing aspects of Bayesian statistics in practice:

- Advantages of the Bayesian approach provides an overview of the advantages of the Bayesian framework and lists some of the key types of inference with parametric models.
- Bayesian research workflow gives a more detailed version of the workflow for principled Bayesian inference, including checklists.
- BLR-I: Background, BLR-II: Workflow, and BLR-III: Demo, provide a complete treatment of an important class of problems for Bayesian inference: Bayesian Linear Regression (or BLR).
- A Bayesian view of bias and variance gives a Bayesian view of bias and variance, and of underfitting and overfitting in regression.

8.1. Advantages of the Bayesian approach

> "Bayesian inference probabilities are a measure of our state of knowledge about nature, not a measure of nature itself."
>
> —Devinderjit Sivia

The Bayesian approach offers a number of distinct advantages in scientific applications. Some of them are listed below. In this chapter we introduce multiple ways of inference with parametric models.

How the Bayesian approach helps in science

1. Provides an elegantly simple and rational approach for answering any scientific question for a given state of information. The procedure is well-defined:
    - Clearly state your question and prior information.
    - Apply the sum and product rules. The starting point is always Bayes' theorem.
2. Provides a way of eliminating nuisance parameters through marginalization.
    - For some problems, the marginalization can be performed analytically, permitting certain calculations to become computationally tractable.
    - For other problems, sampling is a straightforward way to include many nuisance parameters.
3. Provides a well-defined procedure for propagating errors,
    - E.g., incorporating the effects of systematic errors arising from both the measurement operation and theoretical model predictions.
4. Incorporates relevant prior (e.g., known signal model or known theory model expansion) information through Bayes' theorem.
    - This is one of the great strengths of Bayesian analysis.
    - Enforces explicit assumptions.
    - For data with a small signal-to-noise ratio, a Bayesian analysis can frequently yield many orders of magnitude improvement in model parameter estimation, through the incorporation of relevant prior information about the signal model.
5. For some problems, a Bayesian analysis may simply lead to a familiar (frequentist) statistic. Even in this situation it often provides a powerful new insight concerning the interpretation of the statistic.
6. Provides a more powerful way of assessing competing theories at the forefront of science by automatically quantifying Occam's razor.
    - The evidence for two hypotheses or models, M_i and M_j, can be compared in light of data $\mathcal{D}$ by evaluating the ratio $p(M_i|\mathcal{D},I)/p(M_j|\mathcal{D},I)$.
    - The Bayesian quantitative Occam's razor can also save a lot of time that might otherwise be spent chasing noise artifacts that masquerade as possible detections of real phenomena.

Occam's razor

Occam's razor is a principle attributed to the medieval philosopher William of Occam (or Ockham). The principle states that one should not make more assumptions than the minimum needed. It underlies all scientific modeling and theory building. It cautions us to choose from a set of otherwise equivalent models of a given phenomenon the simplest one. In any given model, Occam's razor helps us to "shave off" those variables that are not really needed to explain the phenomenon. It was previously thought to be only a qualitative principle.

Fig. 8.1 Did the Leprechaun drink your wine, or is there a simpler explanation that does not involve mythical figures?

Inference with parametric models

Inductive inference with parametric models is a very important tool in the natural sciences.

- Consider N different models M_i ($i = 1, \ldots, N$), each with a parameter vector $\boldsymbol{\theta}_i$. The number of parameters (d_i = length of $\boldsymbol{\theta}_i$) might be different for different models. Each of them implies a sampling distribution for possible data
- Consider a scientific model M with a parameter vector $\boldsymbol{\theta}$. Together with error models this implies a statistical model that gives a sampling distribution for possible data

$$p(\mathcal{D}|\boldsymbol{\theta}, M) \tag{8.1}$$

- The likelihood function is the PDF of the actual, observed data ($\mathcal{D}_{\rm obs}$) given a set of parameters $\boldsymbol{\theta}$:

$$\mathcal{L}_i(\boldsymbol{\theta}) \equiv p(\mathcal{D}_{\rm obs}|\boldsymbol{\theta}, M) \tag{8.2}$$

With these ingredients, several types of scientific inquiry can be addressed using a Bayesian framework.

Some key types of Bayesian inference with parametric models

Parameter estimation:

Premise: We have chosen a model (say M_1) and have access to a set of observed data ($\mathcal{D}_{\rm obs}$)

$\Rightarrow$ What can we infer about the model parameters $\boldsymbol{\theta}_1$?

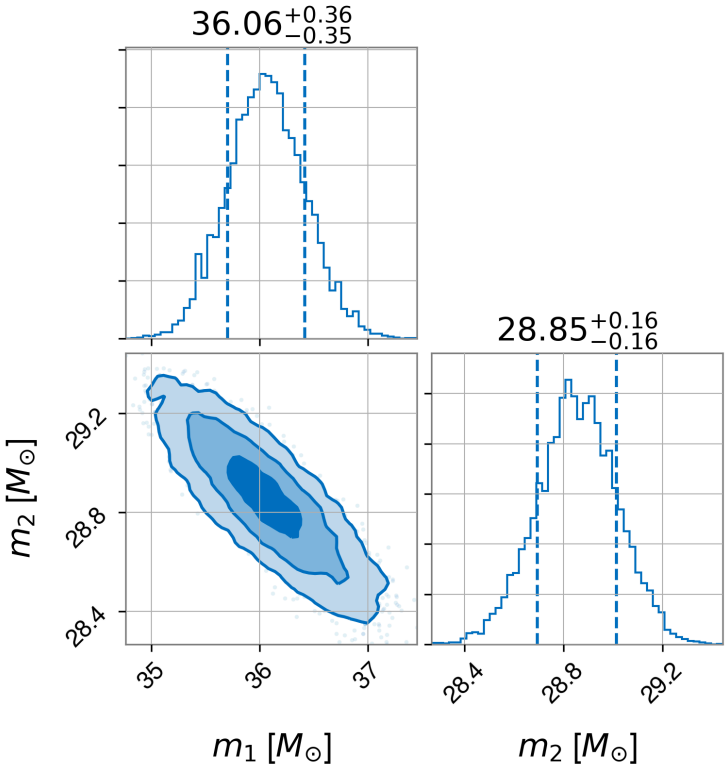


Fig. 8.2 Joint pdf for the masses of two black holes merging obtained from the data analysis of a gravitational wave signal. This representation of a joint PDF is known as a corner plot.

Calibrated model predictions:

Premise: We have a calibrated model (say M_1 and a posterior for parameters $\boldsymbol{\theta}_1$ given data).

$\Rightarrow$ What can we predict for new (not measured) data (posterior predictive distribution)?

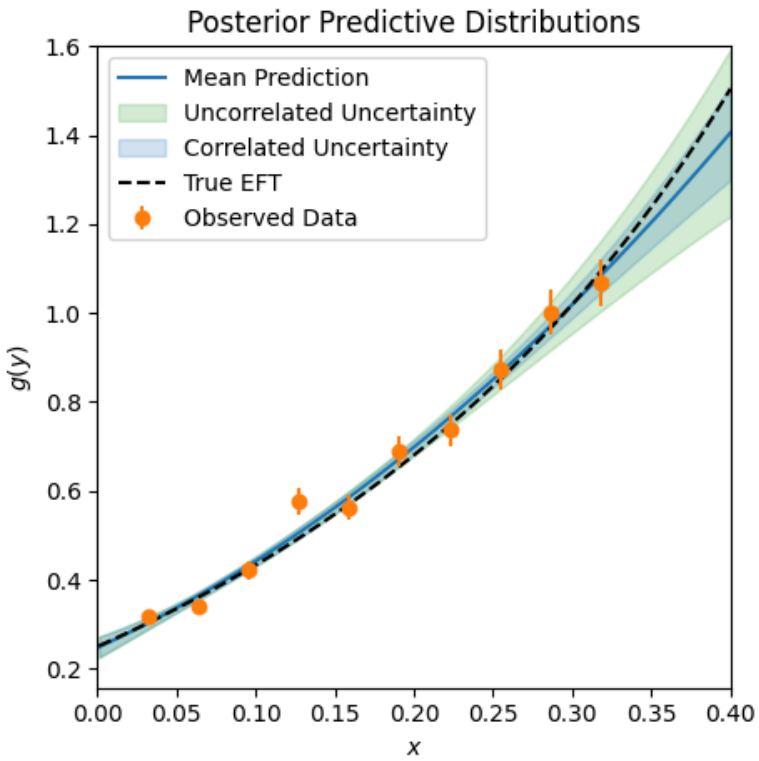


Fig. 8.3 Posterior predictive distributions (PPDs) for a toy effective field theory (a truncated Taylor expansion) given data with errors as shown, compared to the true underlying function. The two PPDs (green and blue bands) correspond to including or not including correlations.

Model comparison:

Premise: We have a set of different models, $\{M_i\}_{i=1}^N$, each with a parameter vector $\boldsymbol{\theta}_i$. The number of parameters (length of $\boldsymbol{\theta}_i$) might be different for different models.

$\Rightarrow$ How do they compare with each other? Do we have evidence to say that, e.g. M_1, is better than M_2? Note: *better* must be defined.

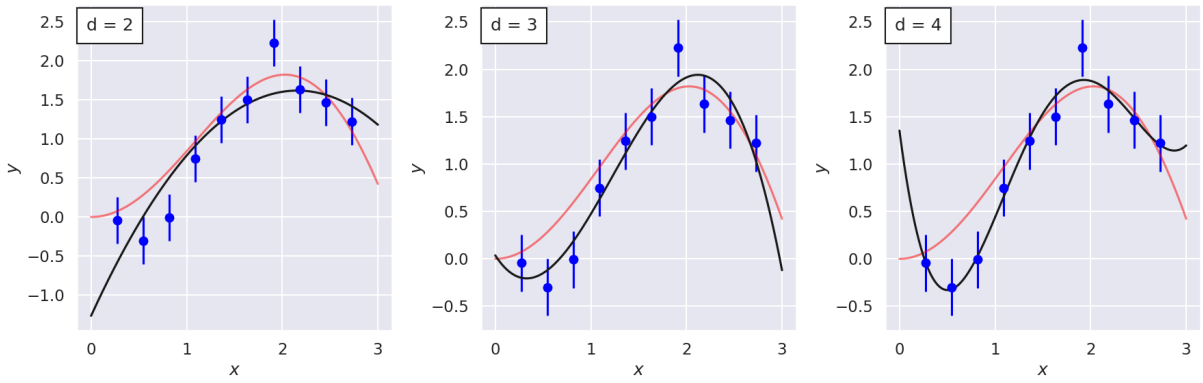


Fig. 8.4 Three polynomial models M_1, M_2, M_3 and their "best fits" to noisy data (the "true" curve is in red). The model comparison problem is to objectively compare the models.

Combining models:

Premise: We have models M_1, M_2, M_3.

$\Rightarrow$ How can we combine them to make better inferences than any single model?

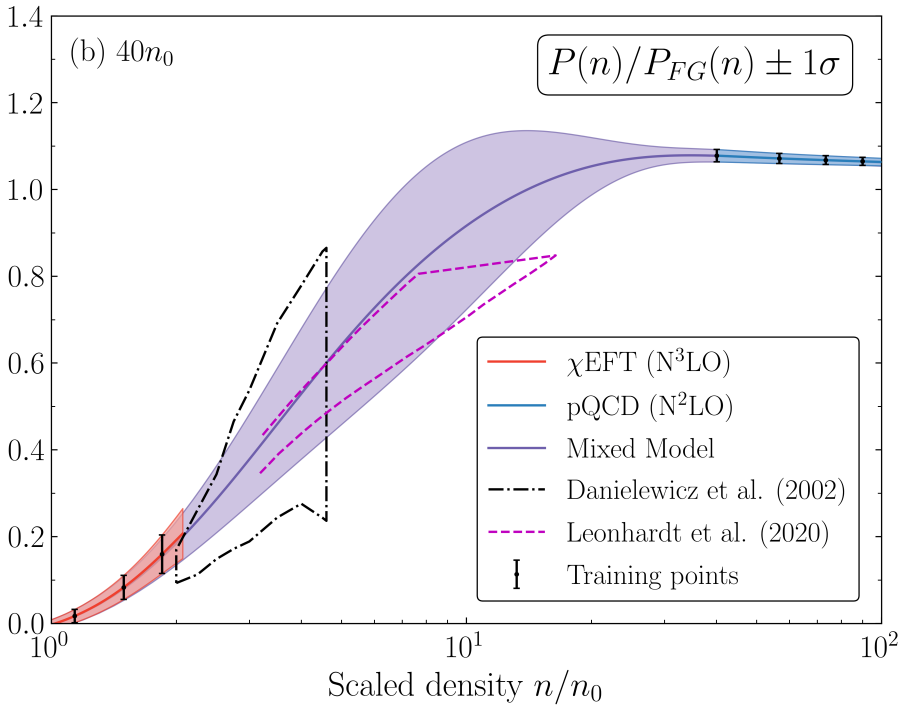


Fig. 8.5 Mixed model equation of state (scaled pressure vs. scaled density) for symmetric nuclear matter. Models with theory errors at low density (red) and very high density (blue) are mixed to bridge the gap between them (purple). One sigma error bands are shown. The figure is from Semposki et al. [SDF+25].

Experimental design:

Premise: Given a statistical model for experimental predictions.

$\Rightarrow$ How should we design an experiment to provide the most information for addressing a specific scientific question?

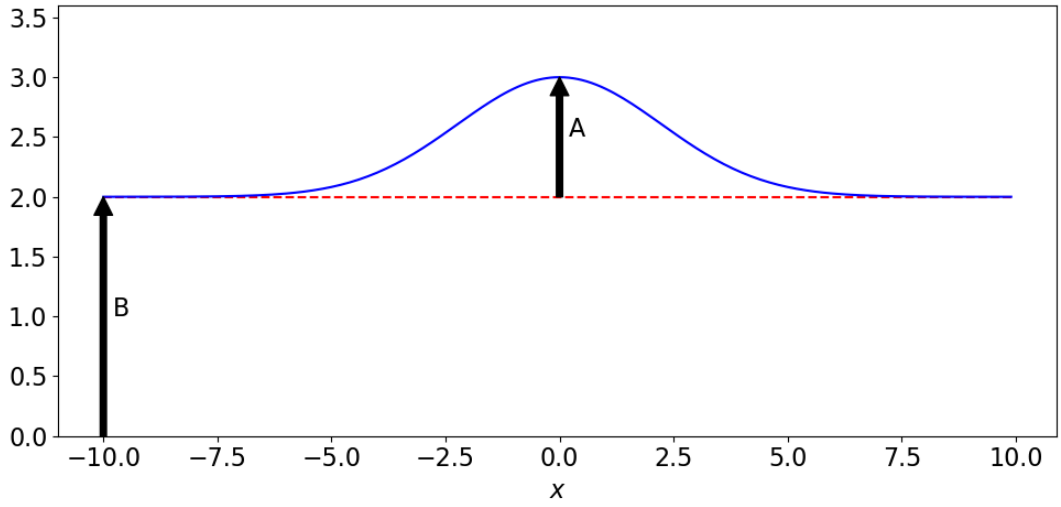


Fig. 8.6 Schematic signal with background from 📥 Problem: Amplitude of a signal in the presence of background. The experimental design problem is how to best extract the signal properties from optimizing a function that gives the cost of experimental choices, such as the resolution of the detector and the number of counts recorded.

Further discussion on Bayesian approaches to all of these will appear in subsequent chapters.

8.2. Bayesian research workflow

The most common Bayesian analyses entail applications of Bayes' rule:

$$\overbrace{p\left(\boldsymbol{\theta} \mid \mathcal{D}, I\right)}^{\text{posterior}} = \frac{\overbrace{p\left(\mathcal{D} \mid \boldsymbol{\theta}, I\right)}^{\text{likelihood}} \times \overbrace{p\left(\boldsymbol{\theta} \mid I\right)}^{\text{prior}}}{\underbrace{p\left(\mathcal{D} \mid I\right)}_{\text{evidence}}}.$$

To robustly construct the ingredients of Bayes' rule and explore its consequences, we consider a four-step Bayesian workflow, first presented in Section 2.2 and repeated here:

Four-step Bayesian workflow in brief

1. Define a statistical model relating the physics model and data, including all errors and correlations.
2. Formulate priors.
3. Compute the posterior probabilities.
4. Do model checking.

In this chapter we elaborate on these steps. Our discussion is partially based on the more extensive exposition in the "Methods Primer" by Van De Schoot et al. [vandSchootDK+21]. (Note: discussion of how to use and calculate the evidence is postponed to Model Selection.)

These four main steps of a typical Bayesian workflow are indicated on the left in Fig. 8.7: (1) determine the likelihood function using information about the data-generating process through a statistical model; (2) formulate

prior distributions that embody knowledge about parameters in the statistical model (this step is conceptually performed before conditioning on data, and ideally, where practical, before those data are examined); (3) calculate the posterior distribution by using Bayes' rule to combine the prior distribution and the likelihood function. The posterior distribution is then used to conduct inferences; (4) Use the posterior to check the extent to which aspects of the statistical model are consistent with the data being analyzed. While the four numbered steps provide a useful linear skeleton, the actual workflow consists of nested cycles and feedback.

In the following subsections we expand upon each step, detailing the cycles indicated on the right in Fig. 8.7.

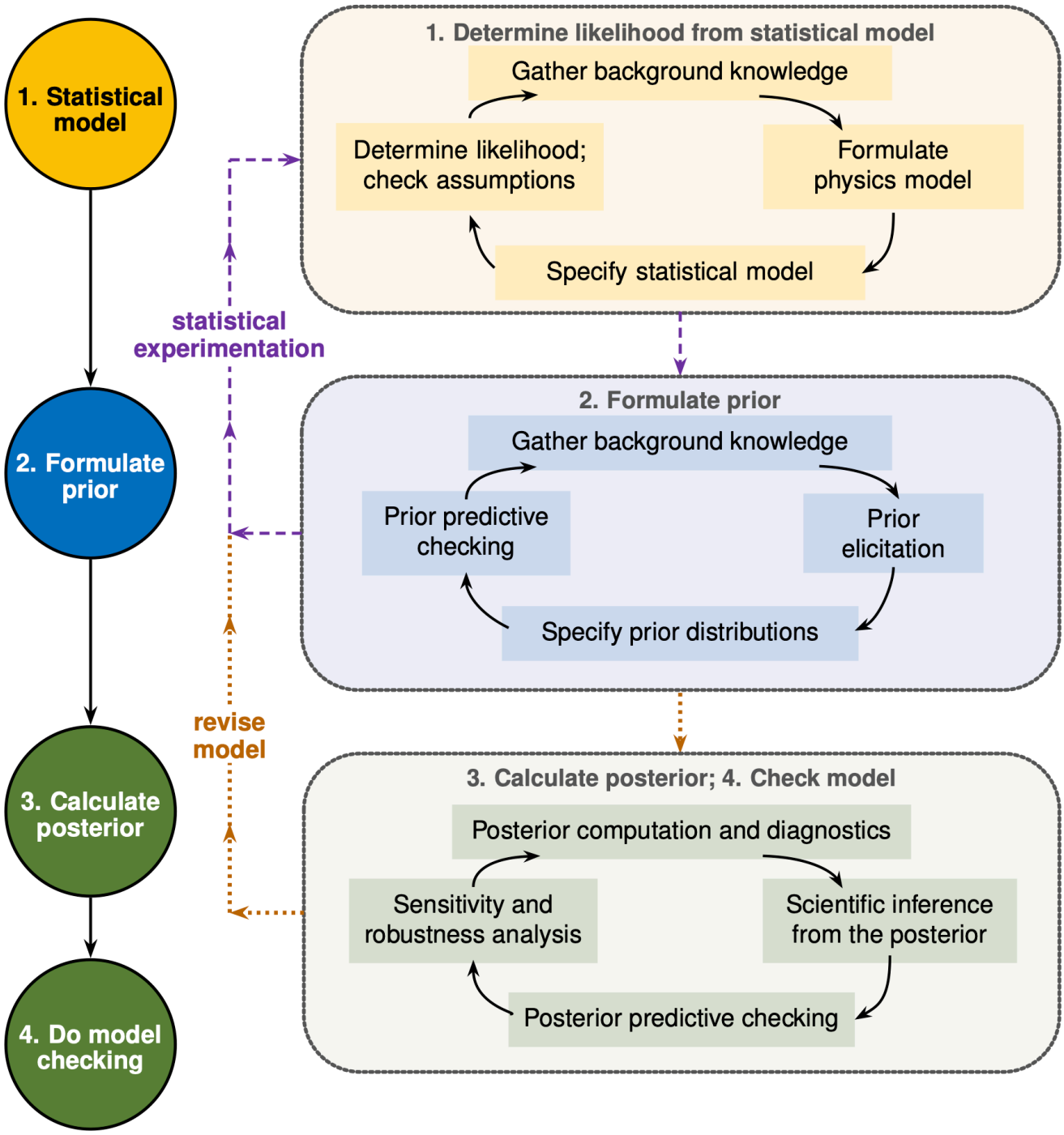


Fig. 8.7 The Bayesian research workflow. The workflow starts with specifying a data-generating statistical model and then determining the likelihood. The next step is formulating prior distributions. At this point the posterior distribution can be calculated from the product of the specified prior and likelihood function according to Bayes' rule. Finally, there are checks to be made of the statistical model using that posterior. At each stage there is a cycle and the overall inferences that can be made can then be used to start a new research cycle. (Figure adapted from a diagram in [vandSchootDK+21] but note that the details and ordering of steps are different.)

Step 1: Determining the likelihood

The likelihood is common to both Bayesian and frequentist approaches, where it quantifies the degree to which observed data implies possible values for the unknown parameters. Since observed data is generated stochastically, through an underlying *data-generating process*, it is appropriately described by a probability distribution. This is the “data likelihood” that describes the probability distribution for observed data given a specific data-generating process (as indicated by the information on the right-hand side of the conditional).

The first step in formulating the data likelihood is to gather background knowledge and then formulate a physics model $M(\boldsymbol{\theta})$, which relates the parameter(s) of interest to model outputs. A particular likelihood follows from a statistical model that stochastically generates the data; it incorporates the physics model, the data uncertainty $\delta\mathcal{D}$, and the model discrepancy δM. The additive statistical model from (4.11) combines these elements as random variables:

$$\mathcal{D} = M(\boldsymbol{\theta}) + \delta\mathcal{D} + \delta M.$$

Assigning probability distributions and correlations to $\delta\mathcal{D}$ and δM induces the data distribution $p\left(\mathcal{D} \mid \boldsymbol{\theta}, I\right)$. The statistical model should account for possible correlations in the outputs (type-y correlations). Such correlations result in a multivariate statistical distribution for the random variable y that does not simply factor into a product of independent, univariate ones for each datum.

One can consider two views on the likelihood:

- Generative view: Assuming specified values of $\boldsymbol{\theta}$, what possible data would the statistical model (i.e., the data-generating process) generate, and with what probabilities?
- Likelihood view: Focusing on the observed data $\mathcal{D}_{\text{obs}}$ that we have; how does the likelihood for this data set depend on the values of the model parameters?

The first view is used for prior and posterior predictive simulation. The second view is the one that we adopt when allowing model parameters to be associated with probability distributions in Bayesian inference. The likelihood still describes the probability for observing a set of data, but we emphasize its parameter dependence by defining the *likelihood function*

$$p\left(\mathcal{D}_{\text{obs}} \,\middle|\, \boldsymbol{\theta}, \sigma^2, I\right) \equiv \mathcal{L}(\boldsymbol{\theta}). \tag{8.3}$$

Note that the likelihood function $\mathcal{L}(\boldsymbol{\theta})$ is **not** a probability distribution for model parameters (and is not normalized, i.e., its integral over $\boldsymbol{\theta}$ is not equal to one). The parameter posterior regains status as a probability density for $\boldsymbol{\theta}$ through Bayes’ rule since the likelihood is multiplied with the prior $p\left(\boldsymbol{\theta} \mid I\right)$ and normalized by the evidence $p\left(\mathcal{D} \mid I\right)$.

A product of normal distributions, one for each output, is a standard choice for a likelihood function. However, this choice assumes that the data-generating process for one output is conditionally independent from the data-generating process for any other output. In practice there may be correlations between them, e.g., shared analysis tools, electrical noise that is correlated between detectors, etc.
Just like the physics model and the prior, the statistical data-generating model is a modeling choice that needs to be justified, documented, and checked. The latter may lead to revisiting the background knowledge and repeating the cycle in Step 1.

Step 2: Formulating and checking a prior

The second major step in the Bayesian workflow depicted in Fig. 8.7 is to determine prior distributions, shortened to priors. This is an important part of a rigorous inference process. As we will discuss towards the end of our presentation of step 2, part of the cycle that occurs within this step is a prior predictive checking process (see Remark 8.2). This evaluates whether the priors chosen are doing the job they were designed to do. For this reason, prior selection, prior checking, and Step 1, Likelihood determination, are sometimes collectively referred to as the *statistical experimentation* phase.

Start with what you know

Formulating the prior means encoding prior knowledge as a probability distribution. In this way pieces of information on the parameters of the model–or on other aspects of it–get included in the Bayesian analysis.

Sometimes the practitioner themselves has prior knowledge regarding parameters of the model. For example, a parameter may be known to be positive or otherwise strictly constrained by physics (e.g., causality). More broadly, almost all physics-model parameters have reasonable ranges, based on the mass, distance, and time scales appearing in the problem being analyzed. Order-of-magnitude estimates for parameters are completely valid fodder for priors, as long as the priors are chosen broadly enough to enable indications if the parameters are “unnatural”, i.e., out-of-line with those estimates.

Prior elicitation is the formal name for the process through which this kind of prior knowledge gets encoded in probability distributions. Prior elicitation may involve discussing with other modelers what “reasonable ranges” for parameters might be: “Would you be shocked if this parameter were smaller than…?”, “What is the order of magnitude of this parameter?”. People may sometimes say they have no idea what a parameter value would be, but they always have **some** idea!

If this process ultimately yields no, or very little, information on a parameter then priors can be assigned that reflect ignorance of its location or scale. The fact that the prior doesn’t depend on that parameter implies there is a symmetry or invariance principle that applies to the prior, and that principle can be employed to ensure the prior is consistent with the practitoner’s level of ignorance. See Section 11.1 for more on this topic and specific examples.

At the other extreme, sometimes highly constraining prior information comes from previous analyses of the model that used other data sets. In this case the hyperparameters for the prior could be derived from sample data or summary statistics of those previous analyses. However, this route to a prior must avoid “double-dipping”: the data used to form the prior needs to be distinct from the data included in the likelihood.

That’s because priors should encode background knowledge that the practitioner possesses **which is independent of the data whose generation is modeled through the likelihood**. The strength, or, conversely, uncertainty of that knowledge should be incorporated in the informativeness of the prior.

Critics of Bayesian methods often point to the subjectivity of priors as a major drawback. Two distinct points should be mentioned in this context. First, many elements of the estimation process are subjective in the sense that they may differ from modeler to modeler. Different reasonable people may make different reasonable choices not only for priors, but also for the likelihood, and certainly for the model itself. Second, many there are circumstances in physics for which informative priors are a well-justified and principled choice, e.g., the positivity bounds already mentioned or a situation in which a parameter’s distribution is quite well delineated by previous data.

Turn what you know into a pdf

The choice of distributional form for the prior (as needed for a full Bayesian analysis) typically involves extra assumptions. Say that you wish to assign to a model parameter a prior with a specific mean value and standard deviation. In this scenario you are still left with the choice between many different distributional forms that fulfill those constraints. Fortunately, arguments based on the maximum entropy principle can help translate a finite set of pieces of prior information into a probability distribution such that as little additional information as possible is smuggled into the prior. These ideas are presented in Section 11.2 The principle of maximum entropy.

But the most important thing about a prior is its level of informativeness. The information reflected in a prior distribution can range from total uncertainty to near certainty. The degree of (un)certainty that a prior encodes is typically described as informative, weakly informative, or diffuse.

The informativeness of a prior should be assessed relative to physically meaningful parameter scales and, ultimately, through the implications of the prior for observable quantities. It is also useful to compare the scale of the prior with the scale over which the likelihood can constrain the parameter. Good practical advice on choosing priors can be found in the Stan Prior Choice Recommendations compendium on github.

Finally, there is the question of how to formulate a prior in a multi-dimensional parameter space. For convenience, priors are often formulated for each parameter separately and then combined assuming independence, i.e., the prior $p\left(\theta \mid I\right)$ is taken to be the product of the one-dimensional prior pdfs for each individual parameter. However, if previous data or a theoretical argument indicates that two parameters should be correlated, then that correlation should be incorporated into the prior. Note that correlations derived from the data set being analyzed should *not* be incorporated into the prior. These will emerge from the posterior when it is formed from the prior and the likelihood. Building them into the prior right from the start of the analysis would be an example of the “double dipping” warned about above.

Remark 8.1 (Possible issues with an improper prior)

A prior is a probability distribution, which therefore should be normalized to one (at least in principle, even if often it can be normalized to a different constant). An *improper prior* integrates to infinity, so it is not a proper probability density. For example, a prior that is taken to be uniform over an infinite interval. This is sometimes viewed as being the most non-informative prior possible; however, physics will almost always dictate bounds on any parameter, so a fully uniform prior is not appropriate.

Some of the possible issues:

- If the data is such that the likelihood times the prior results in a posterior that also integrates to infinity, then the posterior cannot be treated as a probability distribution.
- When doing MCMC sampling, such as with Hamiltonian Monte Carlo, an improper prior may result in the sampler failing to properly sample the relevant space or even wandering toward infinity.
- If one wants to use evidence ratios to compare two models, as described in the chapter on Model Selection, using improper priors can result in arbitrary Bayes factors.

A more exhaustive discussion of the issues with improper prior can be found in Wasserman [Was96].

Checking prior implications

One should always check prior implications to ensure that the prior is not much more restrictive (or much broader) than the background knowledge that is used to motivate it. This starts with checking that the scales of parameters (which may be physical observables) are consistent with what is known. Because Bayesian inference can be

sensitive to poorly chosen priors, it is important to check whether the specified prior produces reasonable (not implausible) values for output quantities as well. Even in the absence of a data-generating distribution from a statistical model (see Step 1), one can draw samples from the prior $p(\boldsymbol{\theta} \mid I)$ and evaluate the physics model $M(\boldsymbol{\theta})$ alone, looking to see whether substantial probability is placed on physically impossible or strongly implausible outputs. If so, that may already be a good reason to revisit the prior choice, particularly if the data-generation process is not going to remediate this problem, i.e., it's not going to move substantial amounts of probability around.

The full **prior predictive check** (see Remark 8.2 below) is a more thorough assessment of the prior choice. The prior predictive distribution is the distribution of all possible data that could be generated given the prior, the statistical model for the data-generating process, and the physics model that relates the parameters of the model to the outputs. Thus, in the workflow diagram in Fig. 8.7, it comes at the end of the cycle in Step 2, before confronting new data, and it may carry one back to Step 1. It can't be carried out absent a specified data-generating process. That's because a full prior predictive check assesses whether the combination prior + model + data-generation yields outcomes that are compatible with background knowledge and with the range of data that could reasonably have been anticipated before the data being analyzed are used. The prior predictive distribution will often be broad, since it reflects uncertainty before conditioning on the new data, but it should not place substantial probability on outcomes that are already known to be physically impossible or scientifically implausible. Such behavior signals that one or more ingredients of the analysis—the prior, the physics model, or the assumed statistical relation between model outputs and data—should be reconsidered,as indicated by the return line to Step 1 in Fig. 8.7.

We emphasize that the check being conducted here is against output values that are clearly "out of range", e.g., are significantly different than the expected size of the data. A prior predictive check absolutely does not mean that the practitioner should adjust the prior until its predictions agree with the particular data set that will subsequently enter the likelihood.

Once the prior and likeilhood have been combined and the posterior computed the sensitivity of the results to details of the chosen prior should be assessed as part of the model checking phase of the Bayesian workflow (Step 4). This is a very useful piece of the workflow: it can provide reassurance that all the choices you agonized over when formulating your prior ultimately don't make a significant difference to the inference of the quantities you care about. Or, it can show how your prior choices are affecting the posterior, which does not invalidate your analysis, but does perhaps merit furhter investigation and certainly warrants reporting in your final research product.

Step 3: Results for the posterior–and other things of interest

In the Bayesian Research Workflow, after determining the likelihood function and formulating the prior, the next step is to combine them following Bayes' rule and use the resulting posterior to estimate the unknown parameters of the model.

In contrast, when doing model fitting in the frequentist framework, one typically focuses on the expected long-term outcomes of an experiment. The goal is a point estimate for model parameters, such as the maximum likelihood estimate and associated frequentist confidence interval. Within the Bayesian Research Workflow, the model parameters will have probability distributions. The goal in Bayesian statistics is to estimate the full posterior distribution of the model parameters, even if in the end one summarizes the distribution with point estimates, such as the posterior mean or median, and a Bayesian credible interval.

In these lecture notes, we frequently use Markov Chain Monte Carlo (MCMC) for posterior inference, see Part III—although more advanced sampling algorithms are also discussed, see Section 22. MCMC has two basic outcomes from the Markov chain and Monte Carlo integration: a set of parameter values sampled according to the posterior distribution and an estimate of that distribution (and its statistics). A workflow for MCMC sampling is detailed in

Section 21.1. We emphasize that posterior computation is not complete until the sampling quality and robustness has been checked with appropriate diagnostics (see Section 21.2).

MCMC is able to indirectly compute inferences on the posterior distribution by simulating different sets of parameters according to the product of the prior probability and the likelihood. This results in sets of values for the parameters θ (vectors in the multi-dimensional setting) that are obtained from the posterior distribution with frequencies that will correspond to the posterior distribution if the MCMC has converged. This can be achieved despite the fact that a Bayesian posterior obtained from a likelihood and a prior will typically be high-dimensional, not have a closed analytic form, and only be known up to a constant of proportionality. The samples of values of the parameters are then used to obtain empirical estimates of the posterior distribution of interest. It is often more difficult to obtain converged estimates of multivariate distributions, or of the form of low-probability tails. It is therefore sometimes more useful to focus on the marginal posterior distribution of each parameter, or pairs of parameters. These marginal distributions are defined by integrating out all but one or two of the parameters from the multi-dimensional posterior.

One important payoff of MCMC sampling is that any quantity of interest that is a function of the model parameters can also be obtained by computing the function over the set of MCMC samples. This then generates a set of samples for that quantity which can be analyzed via summary statistics, histograms, etc. If we are interested in multiple quantities that are functions of the parameters we can represent their multi-dimensional distribution as samples in this sense, and so determine not just summary statistics for each one, but also the correlations between these derived quantities.

Step 4: Model checking

Because the statistical model specifies a probability distribution for possible data conditional on the parameters, posterior draws of the parameters can be propagated through the data-generating model to generate replicated or future data through the predictive posterior distribution (PPD). The PPD is the distribution that the model predicts for the model outputs that were measured, given the posterior inferred from those data. The PPD is our first example of Step 4 in the Bayesian Research Workflow.

Another important check is to assess prior sensitivity. In general a Bayesian analysis should be re-run several times, with different, reasonable, prior choices in order to assess how, and if so, where, the posterior is affected by those choices.

Remark 8.2 (Prior and posterior predictive checking)

Within steps Steps 2 and 4 of diagram Fig. 8.7 are two examples of predictive checks, for the prior and the posterior. The prior and posterior predictive checks use closely related simulation procedures, but they involve checking against different things and they answer different questions. Prior predictive checking examines the implications of the prior together with the generative model without any conditioning on (and really without any looking at) the new data. Posterior predictive checking asks whether the fitted model can reproduce scientifically relevant features of the observed data.

The prior predictive distribution is given formally by:

$$p(y|I) = \int d\boldsymbol{\theta}\, p(y|M(\boldsymbol{\theta}), I) p(\boldsymbol{\theta}|I). \tag{8.4}$$

As implied by this formula, a prior predictive check is carried out in practice by sampling the parameters θ from the prior, then generating data according to the data model given those sampled parameters. This allows a

check of how the probability mass of prior predictions is distributed, in particular whether there is significant prior mass concentrated in implausible regions of the parameter space. Note, however, that data sets simulated from the prior should not be compared to the actual data D; instead examining if the simulated outputs take on extreme values, which can reveal priors that are overly restrictive, excessively broad, centered in unreasonable regions, or otherwise poorly matched to the available background knowledge

The posterior predictive distribution is given formally by:

$$p(y|D,I) = \int d\boldsymbol{\theta}\, p(y|M(\boldsymbol{\theta}),D,I)p(\boldsymbol{\theta}|D,I), \tag{8.5}$$

where the first pdf under the integral is defined by the statistical model that relates the future data y to $M(\boldsymbol{\theta})$, i.e., (4.11). Note that it is conditioned on the observed data, unlike the prior predictive distribution. (We see that the prior predictive distribution (8.4) is just a special case of the posterior predictive distribution with no observed data.)

Posterior predictive checking instead draws replicated data from the model after conditioning on the observations. Features of these replicates—such as their distributions, means, variances, or other relevant summaries—can then be compared with corresponding features of the observed data. Systematic discrepancies point to ways in which the fitted model fails to reproduce the data.

Prior predictive checks may motivate revisions of the prior or the generative model, leading back to an earlier step in the workflow, as indicated by the dashed return line in Fig. 8.7. Posterior predictive checks primarily diagnose deficiencies in the fitted model and may motivate revised model assumptions (dotted return line). They can also reveal sensitivity to the prior, but prior revisions after examining the data should be reported as part of a sensitivity analysis or become part of a subsequent research cycle.

The sensitivity of the posterior to the inclusion of different outputs in the likelihood is reflective of the *information content* of that observable. Sensitivity studies of the posterior based on simulated outputs for different future data is a branch of statistical inference known as *experimental design*. It can help determine which observable(s) to spend resources on measuring to improve the accuracy and precision of a desired inference.

Reproducibility

To enable physics research to be verified and reproduced, proper reporting on statistics is crucial. In particular, not reporting the choice of priors is bad Bayesian practice. The naive use of priors has many pitfalls. Practitioners should record what was done and justify it as part of the research output. Specifying priors is a way of declaring to future readers of your work what information you considered known before you began your data analysis. Similarly, specification of the likelihood should be clear and complete, as already discussed above.

Overall, the Bayesian research workflow offers multiple opportunities to follow good scientific practices that facilitate reproducibility. To quote Van De Schoot et al. [vandSchootDK+21]: “Allowing others to assess the statistical methods and underlying data used in a study (by transparent reporting and making code & data available) can help with interpreting the study results, the assessment of the parameters of the methods used, and the detection & fixing of errors.” To which, we would add, it also helps practitioners generalize your work to their research application. But, as Van De Schoot et al. also point out “Reporting practices are not yet consistent in this regard across fields or even journals in individual fields.”

To enable reproducibility and allow other researchers to run Bayesian statistics on the same data with different parametrizations, priors, model assumptions, or likelihoods, it is important that the data and code used are properly

documented and shared following what Van De Schoot et al. call the “FAIR principles”: findability, accessibility, interoperability and reusability [vandSchootDK+21]. Van De Schoot recommends, and we agree, that data and code should be:

- Shared via a publicly accessible repository (e.g., on github or through Zenodo),
- Given a persistent identifier such as a DOI
- Tagged with appropriate metadata (which also enables appropriate citation).

For true accessibility, developing proper documentation for the software is essential. This includes README files with installation steps; a quickstart guide; testing methods (e.g., unit tests); licensing and how to cite the software. The code itself should adhere to best practices such as the use of descriptive variable and function names and standardized docstrings. All all of this implicitly assumes practitioners are using open-source software–if at all possible–so that barriers to replicating scientific results are lowered.

Making these choices enables transparency in the scientific process. Nany scientific journals now have guidelines that specify requirements like these for code and data sharing.

Checklists

We end this chapter with two different recommended checklists for statistically sound Bayesian inference that incorporate many of the points made in this chapter. The first one (see Remark 8.3) has mainly natural science applications in mind.

Remark 8.3 (Checklist for statistically sound Bayesian inference)

1. Interact with the experts (i.e., statisticians, applied mathematicians).
2. Identify all sources of experimental and theoretical uncertainties in this process.
3. Formulate statistical models for those uncertainties.
4. Account for correlations in observables (type y) when formulating the likelihood.
5. Choose priors that are as informative as you consider reasonable, consistent with background knowledge; check their implications and assess sensitivity to reasonable alternatives.
6. Use model checking to assess the adequacy of the analysis and identify possible model deficiencies.

Note that this first checklist matches up quite well with the Bayesian workflow given at the start of this section.

The second checklist, labeled WAMBS (when to Worry and how to Avoid the Misuse of Bayesian Statistics), originates in work by statistics experts from a wide range of application areas [vandSchootDK+21]. Here we have condensed the recommendations of the original WAMBS checklist in regard to monitoring the MCMC convergence since we will provide more details on this in Part III (see Section 21).

Remark 8.4 (An adapted version of the WAMBS checklist)

Here we have adapted the WAMBS-v2 checklist, an updated version of the WAMBS (when to Worry and how to Avoid the Misuse of Bayesian Statistics) checklist. Adapted from [vandSchootDK+21].

1. Ensure the prior distributions and the model or likelihood are well understood (see checklist above).
2. Describe them thoroughly in your research output (article, analysis notebook, etc.).
3. Use prior predictive checking to assess whether the prior and generative model imply scientifically plausible data.
4. Check convergence of your MCMC chain. (See the Markov Chain Monte Carlo in practice chapter.)

5. To assess the impact of informative priors, compare the posterior results with an analysis using diffuse priors. This comparison can facilitate a deeper understanding of the impact the informative priors have on findings.
6. Examine the sensitivity of your results to priors, especially multivariate priors. These priors can be particularly influential on the posterior, even with slight modifications to the hyperparameters.
7. It may also be worth testing different forms for the likelihood, in order to see what impact they have on the final results.
8. Report findings, including Bayesian interpretations. Take advantage of explaining and capturing the entire posterior rather than simply using point estimates. It may be helpful to examine the density at different quantiles to fully capture and understand the posterior distribution.

Ticking all the boxes of checklists such as these can be considered an aspirational goal for performing a truly rigorous statistical inference in science.

8.3. BLR-I: Background

> "La théorie des probabilités n'est que le bon sens réduit au calcul"
>
> (trans.) Probability theory is nothing but common sense reduced to calculation.
>
> —Pierre Simon de Laplace

Here and in the next two sections we use Bayes' theorem to infer a (posterior) probability density function for parameters of a *linear statistical model*, conditional on data $\mathcal{D}$. This is called **Bayesian linear regression** or BLR. In this context "linear" means that the parameters we seek to infer appear in the model only to the first power (specific examples below). You may already be familiar with ordinary (frequentist) linear regression, such as making a least-squares fit of a polynomial to data; later in this chapter we will show how this is related to BLR.

The advantages of doing *Bayesian* instead of frequentist linear regression are many. The Bayesian approach yields a probability distribution for the unknown parameters and for future model predictions. It also enables us to make all assumptions explicit whereas the frequentist approach puts nearly all emphasis on the collected data. These assumptions can be more general as well; e.g., they allow you to specify prior beliefs on the parameters (such as slope and intercept for a straight line model). Finally, we can do straightforward model checking and add a discrepancy model to account for limitations of the linear model being considered.

We will use BLR to exemplify the general Bayesian workflow we have summarized in Section 2.2 and Section 8.2. To do so, we first need to more precisely define what we mean by linear models.

Background on linear models

Definition and examples

In **linear modeling** the dependence on the model parameters β is **linear**, and this fact will make it possible, for certain priors, to find the distribution of model parameters analytically. Note that we will mostly operate with models

depending on more than one parameter. Hence, we denote the model parameters ($\boldsymbol{\beta}$) using a bold symbol. (We reserve the generic symbol $\boldsymbol{\theta}$ to include not only $\boldsymbol{\beta}$ but also any other parameters specifying our statistical model.) In this chapter we will, however, consider models (M) that relate a single dependent variable (y) with a single independent one (x).

The linear parameter dependence implies that our model $M\left(\boldsymbol{\beta}\,;\,x\right)$ separates into a sum of parameters times basis functions. Assuming N_p different basis functions we have

$$M\left(\boldsymbol{\beta}\,;\,x\right) = \sum_{j=0}^{N_p-1} \beta_j f_j(x). \tag{8.6}$$

Note that there is no $\boldsymbol{\beta}$-dependence in the basis functions $f_j(x)$.

From a machine-learning perspective the different basis functions are known as **features**.

Example 8.1 (Polynomial basis functions)

A common linear model corresponds to the use of polynomial basis functions $f_j(x) = x^j$. A polynomial model of degree $N_p - 1$ would then be written

$$M(\boldsymbol{\beta}; x) = \sum_{j=0}^{N_p-1} \beta_j x^j. \tag{8.7}$$

Note that the $j = 0$ basis function is $f_0(x) = x^0 = 1$ such that the β_0 parameter becomes the $x = 0$ intercept.

Example 8.2 (Liquid-drop model for nuclear binding energies)

The liquid drop model is useful for a phenomenological description of nuclear binding energies (BE) as a function of the mass number A and the number of protons Z, neutrons N:

$$\text{BE}\,(A, N, Z) = a_0 + a_1 A + a_2 A^{2/3} + a_3 Z^2 A^{-1/3} + a_4 (N - Z)^2 A^{-1}. \tag{8.8}$$

We have five features: the intercept (constant term, bias), the A dependent volume term, the $A^{2/3}$ surface term and the Coulomb $Z^2 A^{-1/3}$ and asymmetry $(N - Z)^2 A^{-1}$ terms. Although the features are somewhat complicated functions of the independent variables A, N, Z, we note that the $p = 5$ regression parameters $\boldsymbol{\beta} = (a_0, a_1, a_2, a_3, a_4)$ enter linearly.

Checkpoint question

Is a Fourier series expansion of a function a linear model?

Hint

If $f(x)$ is an odd function with period $2L$, a Fourier sine expansion of $f(x)$ with N terms takes the form

$$f(x) = \sum_{n=1}^{N} b_n \sin(\frac{n\pi x}{L}),$$

where the b_n are to be determined.

Answer

The parameters b_n appear linearly, so this is a linear model *if* only the parameters are being determined (i.e. L and N are given). Note that with finite N this model will not be a perfect reproduction of a general $f(x)$, so there will be a *discrepancy*.

Checkpoint question

Which of the following are linear models and which are nonlinear?

$$\begin{aligned}
&(a) \quad f(x) = \beta_0 + \beta_1\sqrt{x} \quad \text{with} \quad \boldsymbol{\beta} = (\beta_0, \beta_1) \\
&(b) \quad E(N) = E_\infty + ae^{-bN} \quad \text{with} \quad \boldsymbol{\beta} = (E_\infty, a, b) \\
&(c) \quad g(z) = ae^{-z} + be^{-2z} + ce^{-3z} \quad \text{with} \quad \boldsymbol{\beta} = (a, b, c)
\end{aligned}$$

Hint

Remember that it is the *parameter* dependence that dictates whether it is linear.

Answer

The first and third are linear, the second is not (because of the b parameter).

Converting linear models to matrix form

When using a linear model we have access to a set of data $\mathcal{D}$ for the dependent variable, e.g., the N_d values

$$\mathcal{D} = [y_1, y_2, \ldots, y_{N_d}]^T. \tag{8.9}$$

For each datum y_i there is an independent variable x_i, and our model for the i^{th} datum is

$$M_i \equiv M(\boldsymbol{\beta}; x_i) = \sum_{j=0}^{N_p-1} \beta_j f_j(x_i). \tag{8.10}$$

We can collect the basis functions evaluated at each independent variable x_i in a matrix $\mathbf{X}$ of dimension $N_d \times N_p$:

$$\boldsymbol{X} = \begin{bmatrix} f_0(x_1) & \dots & f_{N_p-1}(x_1) \\ f_0(x_2) & \dots & f_{N_p-1}(x_2) \\ \vdots & \ddots & \vdots \\ f_0(x_{N_d}) & \dots & f_{N_p-1}(x_{N_d}) \end{bmatrix} \tag{8.11}$$

This matrix will be referred to as a **design matrix**.

Example 8.3 (The design matrix for polynomial models)

The design matrix for a linear model with polynomial basis functions becomes

$$\boldsymbol{X} = \begin{bmatrix} 1 & x_1^1 & x_1^2 & \dots & x_1^{p-1} \\ 1 & x_2^1 & x_2^2 & \dots & x_2^{p-1} \\ 1 & x_3^1 & x_3^2 & \dots & x_3^{p-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N_d}^1 & x_{N_d}^2 & \dots & x_{N_d}^{p-1} \end{bmatrix}, \tag{8.12}$$

where we are considering a polynomial of degree $p-1$, which implies a model with p features (including the intercept). It is also known in linear algebra circles as a Vandermonde matrix.

Checkpoint question

What is the design matrix for a Fourier cosine series expansion with N_p terms (plus a constant)?

Hint

If $f(x)$ is an even function with period $2L$, a Fourier cosine expansion of $f(x)$ with N_p terms (plus a constant) takes the form

$$f(x) = \frac{a_0}{2} + \sum_{n=1}^{N_p} a_n \cos(\frac{n\pi x}{L}),$$

where the a_n are to be determined.

Answer

For convenience, let $\omega = \pi/L$. Then the design matrix is:

$$\begin{bmatrix} 1 & \cos(\omega x_1) & \cos(2\omega x_1) & \dots & \cos(N_p \omega x_1) \\ 1 & \cos(\omega x_2) & \cos(2\omega x_2) & \dots & \cos(N_p \omega x_2) \\ 1 & \cos(\omega x_3) & \cos(2\omega x_3) & \dots & \cos(N_p \omega x_3) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \cos(\omega x_{N_d}) & \cos(2\omega x_{N_d}) & \dots & \cos(N_p \omega x_{N_d}) \end{bmatrix} \tag{8.13}$$

Next, using the column vector $\boldsymbol{\beta}$ for the parameters,

$$\boldsymbol{\beta} = [\beta_0, \beta_1, \beta_2, \dots, \beta_{N_p - 1}]^T, \tag{8.14}$$

we can write the general (additive) statistical model

$$\mathcal{D} = M(\boldsymbol{\beta}) + \delta\mathcal{D} + \delta M. \tag{8.15}$$

with $M(\boldsymbol{\beta}) \to M(\boldsymbol{\beta}; x)$ as the matrix equation

$$\mathcal{D} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}. \tag{8.16}$$

The last term $\boldsymbol{\epsilon}$ is a column vector of so-called **residuals**. This term includes both the data uncertainty $\delta\mathcal{D}$ and the model uncertainty δM, i.e., it expresses the part of the dependent variable, for which we have data, that we cannot describe using a linear model. Formally, we can therefore write $\epsilon_i = y_i - M_i$ and define the vector $\boldsymbol{\epsilon}$ as

$$\boldsymbol{\epsilon} = [\epsilon_1, \epsilon_2, \epsilon_3, \dots, \epsilon_{N_d}]^T. \tag{8.17}$$

It is important to realize that our model M provides an approximate description of the data. For now we will take $\delta M = 0$; that is, we assume that the entire residual is explained by data uncertainty. More generally we expect that $\delta M \neq 0$ (this is often summarized as *all models are wrong*) because in a realistic setting we have no guarantee that the data is generated by a linear process. Of course, based on physics insight, or other assumptions, there might exist very good reasons for using a linear model to explain the data (taking into account $\delta\mathcal{D}$).

Checkpoint question

Why does the Fourier series from the last section have to be truncated to a finite number of terms in practice?

Hint

Could I do linear algebra with $N = \infty$?

Answer

Our manipulations will be with *finite* matrices, so the basis size must be finite in practice.

The normal equation

A regression analysis often aims at finding the parameters $\boldsymbol{\beta}$ of a model M such that the vector of residuals $\boldsymbol{\epsilon}$ is minimized in the sense of its Euclidean norm (or 2-norm). This is assumed in the familiar least-squares analysis. Below we will see that this particular goal arises naturally in Bayesian linear regression.

Here we will lay the groundwork for an analytical solution to the linear regression problem by finding the set of parameters $\boldsymbol{\beta}^*$ (we will typically use an asterisk to denote specific parameters that are "optimal" in some sense) that minimizes

$$C(\boldsymbol{\beta}) \equiv \sum_{i=1}^{N_d} \epsilon_i^2 = \sum_{i=1}^{N_d} (y_i - M_i)^2 = \Big\{ (\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})^T (\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta}) \Big\}. \tag{8.18}$$

The solution to this optimization problem turns out to be a solution of the normal equation and is known as ordinary least-squares or ordinary linear regression. (Later an exercise will have you generalize this problem to the case where the last factor in Eq. (8.18) has a covariance matrix between the two terms.)

Theorem 8.1 (Ordinary least squares (the normal equation))

The ordinary least-squares method corresponds to finding the optimal parameter vector $\boldsymbol{\beta}^*$ that minimizes the Euclidean norm of the residual vector $\boldsymbol{\epsilon} = \mathcal{D} - \boldsymbol{X}\boldsymbol{\beta}$, where $\mathcal{D}$ is a column vector of observations and $\boldsymbol{X}$ is the design matrix (8.11).

Finding this optimum turns out to correspond to solving the **normal equation**

$$\boldsymbol{X}^T \mathcal{D} = \boldsymbol{X}^T \boldsymbol{X} \boldsymbol{\beta}^*. \tag{8.19}$$

Given that the **normal matrix** $\boldsymbol{X}^T\boldsymbol{X}$ is invertible (which is the case if $\boldsymbol{X}$ has full column rank), the solution to the normal equation is given by

$$\boldsymbol{\beta}^* = \left(\boldsymbol{X}^T \boldsymbol{X}\right)^{-1} \boldsymbol{X}^T \mathcal{D}. \tag{8.20}$$

Proof. We will derive the normal equation in four steps:

1. *Cost function.* The cost function is the squared Euclidean norm of the residual vector. Expanding the product gives

   $$C(\boldsymbol{\beta}) = (\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})^T (\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta}) = \mathcal{D}^T\mathcal{D} - 2\boldsymbol{\beta}^T \boldsymbol{X}^T \mathcal{D} + \boldsymbol{\beta}^T \boldsymbol{X}^T \boldsymbol{X} \boldsymbol{\beta},$$

   where we used that $\mathcal{D}^T \boldsymbol{X}\boldsymbol{\beta} = \boldsymbol{\beta}^T \boldsymbol{X}^T \mathcal{D}$ since it is a scalar. This is a quadratic function of the parameters.
2. *Gradient.* With the matrix elements $X_{ij} = f_j(x_i)$ of the design matrix, the cost function reads $C(\boldsymbol{\beta}) = \sum_{i=1}^{N_d} \left(y_i - \sum_{k=0}^{N_p - 1} X_{ik}\beta_k \right)^2$. Its partial derivative with respect to β_j is

$$\frac{\partial C(\boldsymbol{\beta})}{\partial \beta_j} = -2\sum_{i=1}^{N_d} X_{ij}\left(y_i - \sum_{k=0}^{N_p-1} X_{ik}\beta_k\right) = -2\left[\boldsymbol{X}^T\left(\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta}\right)\right]_j, \tag{8.21}$$

where the last step follows since the j:th element of $\boldsymbol{X}^T\boldsymbol{v}$ is $\sum_i X_{ij}v_i$ for any vector $\boldsymbol{v}$. Collecting all elements gives the gradient vector

$$\boldsymbol{\nabla}_\beta C(\boldsymbol{\beta}) = -2\boldsymbol{X}^T\left(\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta}\right). \tag{8.22}$$

3. *Normal equation.* A minimum of C must be a stationary point, $\boldsymbol{\nabla}_\beta C(\boldsymbol{\beta}^*) = 0$, which gives

$$\boldsymbol{X}^T\mathcal{D} = \boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta}^*. \tag{8.23}$$

4. *Minimum.* The second derivatives, $\partial^2 C/\partial\beta_j\partial\beta_k = 2\left(\boldsymbol{X}^T\boldsymbol{X}\right)_{jk}$, form a constant Hessian matrix $2\boldsymbol{X}^T\boldsymbol{X}$. It is positive semi-definite since $\boldsymbol{v}^T\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{v} = |\boldsymbol{X}\boldsymbol{v}|_2^2 \geq 0$ for any vector $\boldsymbol{v}$. The cost function is therefore convex, and any solution of the normal equation is a global minimum. If $\boldsymbol{X}$ has full column rank, then $\boldsymbol{X}\boldsymbol{v} \neq 0$ for all $\boldsymbol{v} \neq 0$. The normal matrix $\boldsymbol{X}^T\boldsymbol{X}$ is then positive definite, and therefore invertible, and the minimum is unique

$$\boldsymbol{\beta}^* = \left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\mathcal{D}. \tag{8.24}$$

The pseudo-inverse (or Moore-Penrose inverse)

We note that since our design matrix is defined as $\boldsymbol{X} \in \mathbb{R}^{N_d \times N_p}$, the combination $\boldsymbol{X}^T\boldsymbol{X} \in \mathbb{R}^{N_p \times N_p}$ is a square matrix. If $\boldsymbol{X}$ has full column rank, i.e., N_p linearly independent columns (which requires $N_d \geq N_p$), the product $\left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T$ is called the pseudo-inverse of the design matrix $\boldsymbol{X}$. It generalizes the usual matrix inverse to non-square matrices, and for a square, full-rank matrix it is equal to the usual inverse. The more general Moore-Penrose pseudo-inverse is defined also for matrices that are not full rank.

Spot the error!

Your classmate simplifies $\boldsymbol{X}^T\mathcal{D} = \boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta}^*$ as $\mathcal{D} = \boldsymbol{X}\boldsymbol{\beta}^*$ and then solves for $\boldsymbol{\beta}^*$ as $\boldsymbol{\beta}^* = \boldsymbol{X}^{-1}\mathcal{D}$. What is wrong?

Answer

$\boldsymbol{X}$ is not a square matrix (in general).

The regression residuals $\boldsymbol{\epsilon}^* = \mathcal{D} - \boldsymbol{X}\boldsymbol{\beta}^*$ can be used to obtain an estimator s^2 of the variance of the residuals

$$s^2 = \frac{(\boldsymbol{\epsilon}^*)^T\boldsymbol{\epsilon}^*}{N_d - N_p}, \tag{8.25}$$

where N_p is the number of parameters in the model and N_d is the number of data.

In frequentist linear regression using the ordinary least-squares method we make a leap of faith and decide that we are seeking a "best" model with an optimal set of parameters $\boldsymbol{\beta}^*$ that minimizes the Euclidean norm of the residual vector $\boldsymbol{\epsilon}$, as above.

Addendum: Ordinary linear regression in practice

We often have a situation where we have much more than just two datapoints, and they rarely fall exactly on a straight line. Let's use python to generate some more realistic, yet artificial, data. Using the function below you can generate data from some linear process with random variables for the underlying parameters. We call this a data-generating process.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def data_generating_process_reality(model_type, rng=np.random.default_rng(), **kwargs):
  if model_type == 'polynomial':
      true_params = rng.uniform(low=-5.0, high=5, size=(kwargs['poldeg']+1,))
      #polynomial model
      def process(params, xdata):
          ydata = np.polynomial.polynomial.polyval(xdata,params)
          return ydata

  # use this to define a non-polynomial (possibly non-linear) data-generating process
  elif model_type == 'nonlinear':
      true_params = None
      def process(params, xdata):
          ydata = (0.5 + np.tan(np.pi*xdata))**2
          return ydata
  else:
      print(f'Unknown Model')

  # return function for the true process the values for the true parameters
  # and the name of the model_type
  return process, true_params, model_type
```

Next, we make some measurements of this process, and that typically entails some measurement errors. We will here assume that independently and identically distributed (i.i.d.) measurement errors e_i that all follow a normal distribution with zero mean and variance σ_e^2. In a statistical notation we write $e_i \sim \mathcal{N}(0, \sigma_e^2)$. By default, we set $\sigma_e = 0.5$.

```
def data_generating_process_measurement(process, params, xdata,
                           sigma_error=0.5, rng=np.random.default_rng()):

  ydata = process(params, xdata)

  #  sigma_error: measurement error.
  error = rng.normal(0,sigma_error,len(xdata)).reshape(-1,1)

  return ydata+error, sigma_error*np.ones(len(xdata)).reshape(-1,)
```

Let us setup the data-generating process, in this case a linear process of polynomial degree 1, and decide how many measurements we make ($N_d = 10$). All relevant output is stored in pandas dataframes.

```
#the number of data points to collect
# -----
Nd = 10
# -----

# predictor values
xmin = -1 ; xmax = +1
Xmeasurement = np.linspace(xmin,xmax,Nd).reshape(-1,1)
# store it in a pandas dataframe
pd_Xmeasurement = pd.DataFrame(Xmeasurement, columns=['x'])

# Define the data-generating process.
# Begin with a polynomial (poldeg=1) model_type
# in a second run of this notebook you can play with other linear models
reality, true_params, model_type =
data_generating_process_reality(model_type='polynomial',poldeg=1)

print(f'model type      : {model_type}')
print(f'true parameters : {true_params}')
print(f'Nd = {Nd}')

# Collect measured data
# -----
sigma_e = 0.5
# -----
Ydata, Yerror =
data_generating_process_measurement(reality,true_params,Xmeasurement,sigma_error=sigma_e)
# store the data in a pandas dataframe
pd_D=pd.DataFrame(Ydata,columns=['data'])
#
pd_D['x'] = Xmeasurement
pd_D['e'] = Yerror

# We will also produce a denser grid for predictions with our model and comparison with the
true process. This is useful for plotting

xreality = np.linspace(xmin,xmax,200).reshape(-1,1)
pd_R = pd.DataFrame(reality(true_params,xreality), columns=['data'])
pd_R['x'] = xreality
```

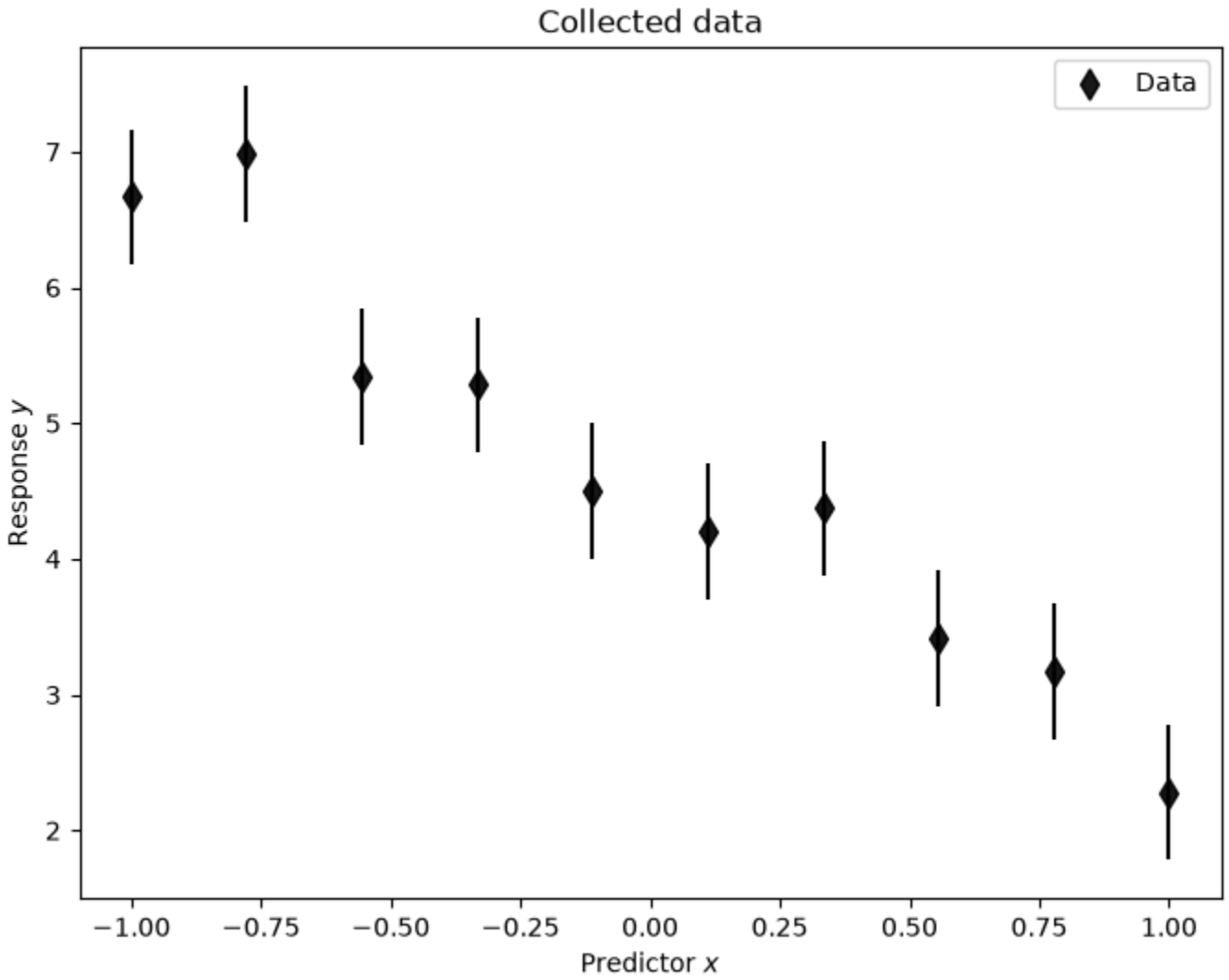
```
model type      : polynomial
true parameters : [ 4.63428437 -2.16526506]
Nd = 10
```

Create some analysis tool to inspect the data, and later on the model.

```
# helper function to plot data, reality, and model (pd_M)
def plot_data(pd_D, pd_R, pd_M, with_errorbars = True):
    fig, ax = plt.subplots(1,1,figsize=(8,6))
    ax.scatter(pd_D['x'],pd_D['data'],label=r'Data',color='black',zorder=1,
alpha=0.9,s=70,marker="d");
    if with_errorbars:
        ax.errorbar(pd_D['x'],pd_D['data'], pd_D['e'],fmt='o', ms=0, color='black');
    if pd_R is not None:
        ax.plot(pd_R['x'], pd_R['data'],color='red', linestyle='--
',lw=3,label='Reality',zorder=10)
    if pd_M is not None:
        ax.plot(pd_M['x'], pd_M['data'],color='blue', linestyle='--
',lw=3,label='Model',zorder=11)
    ax.legend();
    ax.set_title('Collected data');
    ax.set_xlabel(r'Predictor $x$');
    ax.set_ylabel(r'Response $y$');
    return fig,ax
```

Let's have a look at the data. We set the last two arguments to `None` for visualizing only the data.

```
plot_data(pd_D, None, None);
```



Linear regression proceeds via the design matrix. We will analyze this data using a linear polynomial model of order 1. The following code will allow you to setup the corresponding design matrix $\boldsymbol{X}$ for any polynomial order (referred to as poldeg below)

```
def setup_polynomial_design_matrix(data_frame, poldeg, drop_constant=False, verbose=True):
    if verbose:
        print('setting up design matrix:')
        print('  len(data):', len(data_frame.index))

        # for polynomial models: x^0, x^1, x^2, ..., x^p
        # use numpy increasing vandermonde matrix
        print('  model poldeg:',poldeg)

    predictors = np.vander(data_frame['x'].to_numpy(), poldeg+1, increasing = True)
    if drop_constant:
        predictors = np.delete(predictors, 0, 1)
        if verbose:
            print('  dropping constant term')
    pd_design_matrix = pd.DataFrame(predictors)

    return pd_design_matrix
```

So, let's setup the design matrix for a model with polynomial basis functions. Note that there are N_p parameters in a polynomial function of order $N_p - 1$

$$M(\boldsymbol{\beta}; x) = \beta_0 + \beta_1 x.$$

```
Np=2
pd_X = setup_polynomial_design_matrix(pd_Xmeasurement,poldeg=Np-1)
```

```
setting up design matrix:
  len(data): 10
  model poldeg: 1
```

We can now perform linear regression, or ordinary least squares (OLS), as

```
#ols estimator for physical parameter theta
D = pd_D['data'].to_numpy()
X = pd_X.to_numpy()

ols_cov   = np.linalg.inv(np.matmul(X.T,X))
ols_xTd   = np.matmul(X.T,D)
ols_theta = np.matmul(ols_cov,ols_xTd)

print(f'Ndata = {Nd}')
print(f'theta_ols \t{ols_theta}')
print(f'theta_true \t{true_params}\n')
```

```
Ndata = 10
theta_ols       [ 4.62086084 -2.14831493]
theta_true      [ 4.63428437 -2.16526506]
```

To evaluate the (fitted) model we setup a design matrix that spans dense values across the relevant range of predictors.

```
pd_Xreality = setup_polynomial_design_matrix(pd_R,poldeg=Np-1)
```

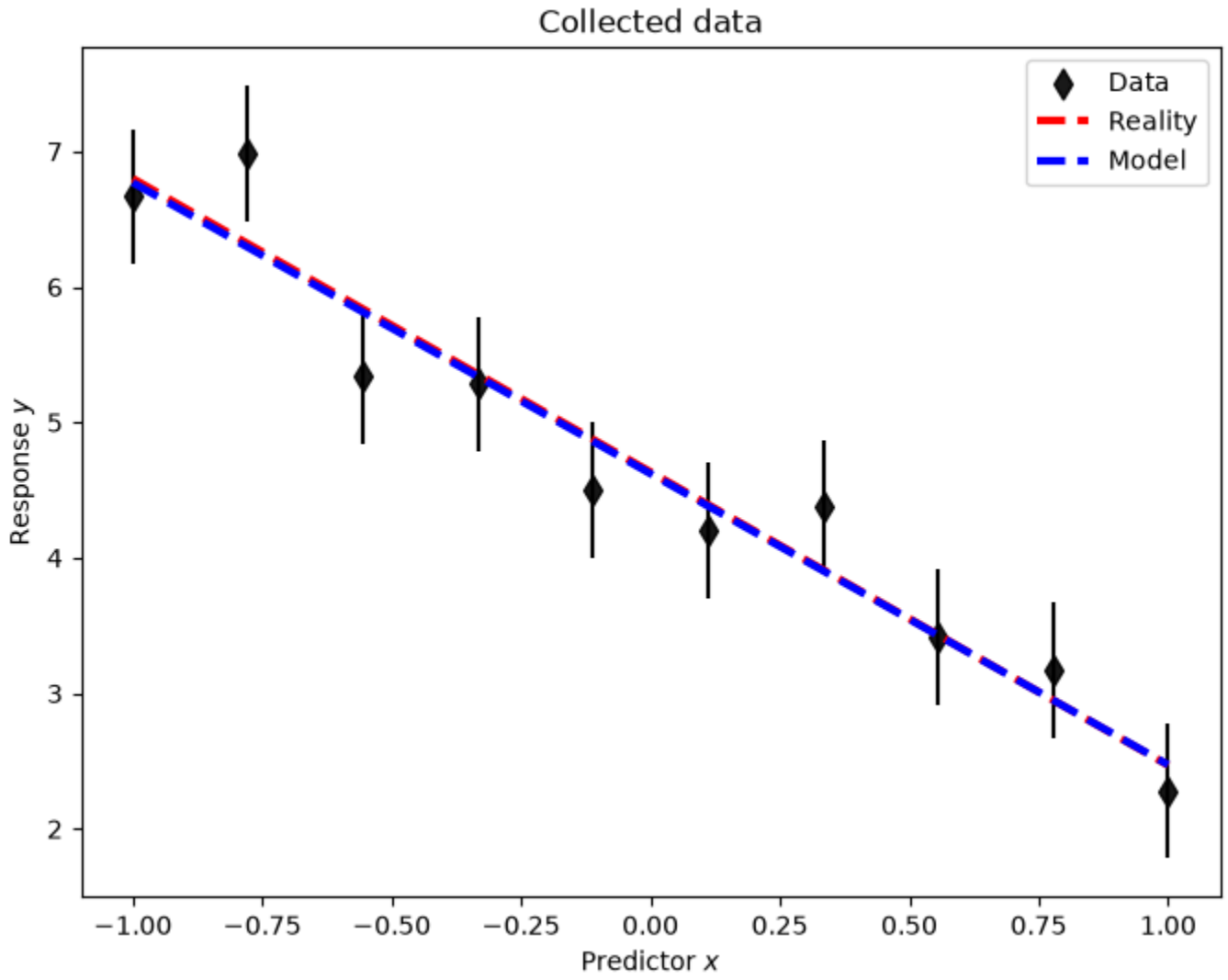
```
setting up design matrix:
  len(data): 200
  model poldeg: 1
```

and then we dot this with the fitted (ols) parameter values

```
Xreality = pd_Xreality.to_numpy()
pd_M_ols = pd.DataFrame(np.matmul(Xreality,ols_theta),columns=['data'])
pd_M_ols['x'] = xreality
```

A plot (which now includes the data-generating process 'reality') demonstrates the quality of the inference.

```
plot_data(pd_D, pd_R, pd_M_ols);
```



To conclude, we also compute the sample variance s^2

```
ols_D   = np.matmul(X,ols_theta)
ols_eps = (ols_D - D)
ols_s2 = (np.dot(ols_eps,ols_eps.T)/(Nd-Np))

print(f's^2       \t{ols_s2:.3f}')
print(f'sigma_e^2 \t{sigma_e**2:.3f}')
```

```
s^2             0.148
sigma_e^2       0.250
```

As seen, the extracted variance is in some agreement with the true one.

Using the code above, you should now try to do the following exercises.

Exercise 8.1

Keep working with the simple polynomial model $M = \beta_0 + \beta_1 x$

Reduce the number of data to 2, i.e., set Nd=2. Do you reproduce the result from the simple example in the previous section?

Increase the number of data to 1000. Do the OLS values of the model parameters and the sample variance approach the (true) parameters of the data-generating process? Is this to be expected?

Exercise 8.2

Set the data-generating process to be a 3rd-order polynomial and set limits of the predictor variable to [-3,3]. Analyze the data using a 2nd-order polynomial model.

Explore the limit of $N_d \to \infty$ by setting $N_d = 500$ or so. Will the OLS values of the model parameters and the sample variance approach the (true) values for some of the parameters?

8.4. BLR-II: Workflow

In following the workflow for Bayesian inference (see Section 2.2 and Section 8.2), we need to

1. Set up a full statistical model relating the physics model and data, including all errors. We need to consistently build in our knowledge of the underlying physics and of the data measurement process.
2. Formulate appropriately informative priors before new data is used.
3. Calculate and interpret the relevant posterior distributions. This is the conditional probability distribution of the unobserved quantities of interest, given the observed data.
4. Do model checking: assess the fit of the model and the reasonableness of the conclusions, testing the sensitivity to model assumptions in steps 1 and 2. From this assessment we modify the model appropriately and repeat all four steps.

To carry out this workflow for BLR, we note that our goal is to relate data $\mathcal{D}$ to the output of a linear model expressed in terms of its design matrix $\boldsymbol{X}$ and its model parameters $\boldsymbol{\beta}$ by $M = \boldsymbol{X}\boldsymbol{\beta}$. We consider the special case of one dependent response variable (y) and a single independent variable (x), for which the data set $(\mathcal{D})$ and the residual vector ($\boldsymbol{\epsilon}$) are both $N_d \times 1$ column vectors with N_d the length of the data set. The design matrix ($\boldsymbol{X}$) has dimension $N_d \times N_p$ and the parameter vector ($\boldsymbol{\beta}$) is $N_p \times 1$.

For the residuals, consider a statistical model that describes the mismatch between our model and observations as in Eq. (8.15) (recall that we assume here that $\delta M = 0$). Knowledge (and/or assumptions) concerning measurement uncertainties, or modeling errors, then allows to describe the residuals as a vector of random variables that are distributed according to a PDF

$$\boldsymbol{\epsilon} \sim p\left(\boldsymbol{\epsilon} \mid I\right), \tag{8.26}$$

where we introduce the relation $\sim$ to indicate how a (random) variable is *distributed*. A very common assumption is that errors are normally distributed with zero mean. As before we let N_d denote the number of data points in the (column) vector $\mathcal{D}$. Introducing the $N_d \times N_d$ covariance matrix $\boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}$ for the errors we then have the explicit distribution

$$p\left(\boldsymbol{\epsilon} \mid \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}, I\right) = \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}). \tag{8.27}$$

Recall that the notation for the multivariate normal distribution $\mathcal{N}$ here is that the mean is $\mathbf{0}$ and the covariance matrix is $\boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}$.

In carrying out the workflow, we adapt Bayes' theorem to the current problem

$$p\left(\boldsymbol{\theta} \mid \mathcal{D}, I\right) = \frac{p\left(\mathcal{D} \mid \boldsymbol{\theta}, I\right) p\left(\boldsymbol{\theta} \mid I\right)}{p\left(\mathcal{D} \mid I\right)} \quad \longrightarrow \quad p\left(\boldsymbol{\beta} \mid \mathcal{D}, \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}, I\right) = \frac{p\left(\mathcal{D} \mid \boldsymbol{\beta}, \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}, I\right) p\left(\boldsymbol{\beta} \mid I\right)}{p\left(\mathcal{D} \mid I\right)}, \tag{8.28}$$

which is the conditioned probability of the quantities of interest, namely the model parameters, on the observed measurements with known covariance for the measurement errors. In most realistic data analyses we will then have to resort to numerical evaluation or sampling of the posterior. However, certain combinations of likelihoods and priors facilitate analytical derivation of the posterior. In this chapter we will explore one such situation and also demonstrate how we can recover the results from an ordinary least squares approach with certain assumptions. A slightly more general approach involves so called **conjugate priors**. This class of probability distributions have clever functional relationships with corresponding likelihood distributions that facilitate analytical derivation.

To evaluate this posterior we must have expressions for both factors in the numerator on the right-hand side (following the Bayesian research workflow in Section 8.2): the likelihood $p\left(\mathcal{D} \mid \boldsymbol{\beta}, \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}, I\right)$ and the prior $p\left(\boldsymbol{\beta} \mid I\right)$. Note that the prior does not depend on the data or the error model. The denominator $p\left(\mathcal{D} \mid I\right)$, sometimes known as the evidence, becomes irrelevant for the task of parameter estimation since it does not depend on $\boldsymbol{\beta}$. It is typically quite challenging, if not impossible, to evaluate the evidence for a multivariate inference problem except for some very special cases. In this chapter we will only be dealing with analytically tractable problems and will therefore (in principle) be able to evaluate also the evidence.

Checkpoint question

Why is it possible to perform parameter estimation without computing the evidence?

Hint

In Bayes theorem, the posterior is normalized. Verify this by integrating both sides over the parameters. For parameter estimation, do you need the posterior to be normalized (e.g., do you need more than the shape and location of the posterior density)?

Checkpoint question

Can you think of why it is so challenging to compute the evidence?

Hint

To evaluate the evidence, you need to introduce an integral over all possible values of $\boldsymbol{\theta}$. Why might this integral harder to do than the likelihood or prior evaluation?

The likelihood

Assuming normally distributed residuals, as we have done, it turns out to be straightforward to express the data likelihood. In the following we will make the further assumption that errors are *independent*. This implies that the covariance matrix $\boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}$ is diagonal and given by a vector $\boldsymbol{\sigma}$,

$$\begin{aligned} \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}} &= \mathrm{diag}(\boldsymbol{\sigma}^2), \text{ where} \\ \boldsymbol{\sigma}^2 &= \left(\sigma_1^2, \sigma_2^2, \ldots, \sigma_{N_d}^2\right), \end{aligned} \tag{8.29}$$

and σ_i^2 is the variance for residual ϵ_i.

Let's first consider a single datum $\mathcal{D}_i$ and the corresponding model prediction $M_i = (\boldsymbol{X}\boldsymbol{\beta})_i$. We are interested in the likelihood for this single data point

$$p\left(\mathcal{D}_i \;\middle|\; \boldsymbol{\beta}, \sigma_i^2, I\right). \tag{8.30}$$

Since the relation between data and residual is a simple additive transformation $\mathcal{D}_i = M_i + \epsilon_i$, we can use the standard probability rules to obtain

$$\begin{aligned} p\left(\mathcal{D}_i \;\middle|\; \boldsymbol{\beta}, \sigma_i^2, I\right) &= \int d\epsilon_i \, p\left(\mathcal{D}_i, \epsilon_i \;\middle|\; \boldsymbol{\beta}, \sigma_i^2, I\right) \\ &= \int d\epsilon_i \, p\left(\mathcal{D}_i \;|\, \epsilon_i, \boldsymbol{\beta}, I\right) p\left(\epsilon_i \;\middle|\; \sigma_i^2\right) \\ &= \int d\epsilon_i \, \delta\big(\mathcal{D}_i - (M_i + \epsilon_i)\big) \frac{1}{\sqrt{2\pi}\sigma_i} e^{-\epsilon_i^2/2\sigma_i^2} \\ &= \frac{1}{\sqrt{2\pi}\sigma_i} \exp\left[-\frac{(\mathcal{D}_i - M_i)^2}{2\sigma_i^2}\right] \end{aligned} \tag{8.31}$$

where we used that $\epsilon_i \sim \mathcal{N}(0, \sigma_i^2)$. Note that the parameter dependence sits in $M_i \equiv M(\boldsymbol{\beta}, \boldsymbol{x}_i)$.

Checkpoint question

Fill in the details for the steps in (8.31).

Hint

Review the probability rules in Chapter 4. The first step integrates in ϵ_i.

Answer

1. Apply the sum rule to integrate in ϵ_i.
2. Apply the product rule, taking into account that ϵ_i depends only on σ_i.
3. Use $\mathcal{D}_i = M_i + \epsilon_i$ to evaluate the first pdf; it is a δ function because $\mathcal{D}_i$ is exactly specified given $M_i \equiv M(\boldsymbol{\beta}, \boldsymbol{x}_i)$ and ϵ_i.
4. Evaluate the integral using the δ function and $\delta\big(\mathcal{D}_i - (M_i + \epsilon_i)\big) = \delta\big(\epsilon_i - (\mathcal{D}_i - M_i)\big)$.

Furthermore, since we assume that the residuals are independent we find that the total likelihood becomes a product of the individual ones for each element of $\mathcal{D}$:

$$\begin{aligned} p\left(\mathcal{D} \,\middle|\, \boldsymbol{\beta}, \boldsymbol{\sigma}^2, I\right) &= \prod_{i=1}^{N_d} p\left(\mathcal{D}_i \,\middle|\, \boldsymbol{\beta}, \sigma_i^2, I\right) \\ &= \left(\frac{1}{2\pi}\right)^{N_d/2} \frac{1}{|\boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}|^{1/2}} \exp\left[-\frac{1}{2}(\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})^T \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}^{-1} (\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})\right], \end{aligned} \tag{8.32}$$

where we note that the diagonal form of $\boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}$ implies that $|\boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}|^{1/2} = \prod_{i=1}^{N_d} \sigma_i$ and that the exponent becomes a sum of squared and weighted residual terms

$$-\frac{1}{2}(\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})^T \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}^{-1} (\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta}) = -\frac{1}{2} \sum_{i=1}^{N_d} \frac{(\mathcal{D}_i - (\boldsymbol{X}\boldsymbol{\beta})_i)^2}{\sigma_i^2}. \tag{8.33}$$

In the special case that all residuals are both *independent and identically distributed* (i.i.d.) we have that all variances are the same, $\sigma_i^2 = \sigma_\epsilon^2$, and the full covariance matrix is completely specified by a single parameter σ_ϵ^2. For this special case, the likelihood becomes

$$\begin{aligned} p\left(\mathcal{D} \,\middle|\, \boldsymbol{\beta}, \sigma_\epsilon^2, I\right) &= \left(\frac{1}{2\pi\sigma_\epsilon^2}\right)^{N_d/2} \exp\left[-\frac{1}{2\sigma_\epsilon^2} \sum_{i=1}^{N_d} (\mathcal{D}_i - (\boldsymbol{X}\boldsymbol{\beta})_i)^2\right] \\ &= \left(\frac{1}{2\pi\sigma_\epsilon^2}\right)^{N_d/2} \exp\left[-\frac{1}{2} \frac{(\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})^T (\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})}{\sigma_\epsilon^2}\right]. \end{aligned} \tag{8.34}$$

In the following we will often include the error model, here specified by σ_ϵ^2, in the background information I and simply write $p\left(\mathcal{D} \,\middle|\, \boldsymbol{\beta}, I\right)$.

Caution

For computational performance it is always better (if possible) to write sums, such as the one in the exponent of (8.34), in the form of vector-matrix operations rather than as for-loops. This particular sum should therefore be implemented as $(\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})^T (\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})$ to employ powerful optimization for vectorized operations in existing numerical libraries (such as `numpy` in `python` and `gsl`, `mkl` for C and other compiled programming languages).

Two views on the likelihood

Let us recall from Section 8.2 the two views on the likelihood.

- Generative view: Assuming fixed values of $\boldsymbol{\beta}$; what are long-term frequencies of future data observations as described by the likelihood?
- Likelihood view: Focusing on the data $\mathcal{D}_{\text{obs}}$ that we have; how does the likelihood for this data set depend on the values of the model parameters?

As previously noted, this second view is the one that we will be adopting when allowing model parameters to be associated with probability distributions. The likelihood still describes the probability for observing a set of data,

but we emphasize its parameter dependence by writing

$$p\left(\mathcal{D} \mid \boldsymbol{\beta}, \sigma^2, I\right) = \mathcal{L}(\boldsymbol{\beta}). \tag{8.35}$$

This function is **not** a probability distribution for model parameters. The parameter posterior, left-hand side of Eq. (8.28), regains status as a probability density for $\boldsymbol{\beta}$ through Bayes' rule since the likelihood is multiplied with the prior $p\left(\boldsymbol{\beta} \mid I\right)$ and normalized by the evidence $p\left(\mathcal{D} \mid I\right)$.

The prior

In the second step of the workflow we assign a prior probability $p\left(\boldsymbol{\beta} \mid I\right)$ for the model parameters. In order to facilitate analytical expressions we will explore two options: (i) a very broad, uniform prior, and (ii) a Gaussian prior. For simplicity, we consider both these priors to have zero mean and with all model parameters being i.i.d.

As discussed earlier, we rarely want to use a truly uniform prior, preferring a wide beta or Gaussian distribution instead. We will assume that the width is large enough that it will be effectively flat where our Bayesian linear regression likelihood is not negligible. Then for the analytic calculations here we can take the uniform prior for the N_p parameters to be

$$p\left(\boldsymbol{\beta} \mid I\right) = \frac{1}{(\Delta\beta)^{N_p}} \begin{cases} 1 & \text{if all } \beta_i \in [-\Delta\beta/2, +\Delta\beta/2] \\ 0 & \text{else}, \end{cases} \tag{8.36}$$

with $\Delta\beta$ the width of the prior range in all parameter directions (this assumes we have standardized the data so that it has roughly the same extent in all directions).

The Gaussian prior that we will also be exploring is

$$p\left(\boldsymbol{\beta} \mid I\right) = \left(\frac{1}{2\pi\sigma_\beta^2}\right)^{N_p/2} \exp\left[-\frac{1}{2}\frac{\boldsymbol{\beta}^T\boldsymbol{\beta}}{\sigma_\beta^2}\right], \tag{8.37}$$

with σ_β the standard deviation of the prior for all parameters.

Checkpoint question

Are these priors normalized?

Hint-1

Integrate both sides over $\boldsymbol{\beta}$ to check normalization, remembering that $\boldsymbol{\beta}$ is a vector, so this is a multidimensional integral.

Hint-2

Both normalization integrals in this case can be *factorized* into the product of one-dimensional integrals. This is true here for the Gaussian prior because the covariance matrix is taken to be diagonal.

Answer

Yes, they are normalized.

Checkpoint question

In what limit are the uniform and Gaussian priors (as defined here) effectively equivalent?

Answer

The limit where $\Delta\beta/2$ and σ_β are both so large that the priors are effectively flat where the likelihood is not negligible.

Checkpoint question

What is implied (i.e., what are you assuming) if you use a truly uniform prior for model parameters?

Hint

Is it possible that the parameters could be arbitrarily large?

Answer

It is implied that not only is any magnitude possible for the model parameters, but all values for a given parameter are equally likely. This is very unlikely to be true.

The posterior

Given the likelihood with i.i.d. errors (8.34) and the two alternative priors, (8.36) and (8.37), we will derive the corresponding two different expressions for the posterior (up to multiplicative normalization constants).

Rewriting the likelihood

First, let us rewrite the likelihood in a way that is made possible by the fact that we are considering a linear model. In particular, this implies quadratic dependence on model parameters in the exponent, which means one can show (by performing a Taylor expansion of the log likelihood around the mode) that the likelihood becomes proportional to the functional form of a multivariate normal distribution for the model parameters:

$$p\left(\mathcal{D} \,\middle|\, \boldsymbol{\beta}, \sigma_\epsilon^2, I\right) = p\left(\mathcal{D} \,\middle|\, \boldsymbol{\beta}^*, \sigma_\epsilon^2, I\right) \exp\left[-\frac{1}{2}(\boldsymbol{\beta}-\boldsymbol{\beta}^*)^T \boldsymbol{\Sigma}_{\boldsymbol{\beta}}^{-1} (\boldsymbol{\beta}-\boldsymbol{\beta}^*)\right]. \tag{8.38}$$

Note that this expression still describes a probability distribution for the data. The data dependence sits in the amplitude of the mode, $p\left(\mathcal{D} \,\middle|\, \boldsymbol{\beta}^*, \sigma_\epsilon^2, I\right)$, and its position, $\boldsymbol{\beta}^* = \boldsymbol{\beta}^*(\mathcal{D}) = \left(\boldsymbol{X}^T \boldsymbol{X}\right)^{-1} \boldsymbol{X}^T \mathcal{D}$. The latter is the

solution (8.20) of the normal equation when the covariance matrix is proportional to the identity: $\boldsymbol{\Sigma}_{\boldsymbol{\epsilon}} = \mathrm{diag}(\sigma_\epsilon^2)$. Furthermore, the statistical model for the errors in this case enter in the covariance matrix,

$$\boldsymbol{\Sigma}_{\boldsymbol{\beta}}^{-1} = \frac{\boldsymbol{X}^T \boldsymbol{X}}{\sigma_\epsilon^2}, \tag{8.39}$$

which can be understood as the curvature (Hessian) of the negative log-likelihood.

Exercise 8.3 (Prove the Gaussian likelihood)

Prove Eq. (8.38).

Hints

Exercise 8.4 (Generalized normal equation)

Prove for the case of the general exponent in Eq. (8.32) that the position of the mode $\boldsymbol{\beta}^*$ is given by

$$\boldsymbol{\beta}^*(\mathcal{D}) = \left[\boldsymbol{X}^T \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}^{-1} \boldsymbol{X}\right]^{-1} \left[\boldsymbol{X}^T \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}^{-1} \mathcal{D}\right]$$

Checkpoint question

Why can't I say $\left[\boldsymbol{X}^T \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}^{-1} \boldsymbol{X}\right]^{-1} \left[\boldsymbol{X}^T \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}^{-1} \mathcal{D}\right] = \boldsymbol{X}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}} (\boldsymbol{X}^T)^{-1} \boldsymbol{X}^T \boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}^{-1} \mathcal{D} = \boldsymbol{X}^{-1} \mathcal{D}$?

Answer

Because these are not square, invertible matrices, so those operations don't hold.

Posterior with a uniform prior

Let us first consider a uniform prior as expressed in Eq. (8.36). The prior can be considered very broad if its boundaries $\pm \Delta\theta/2$ are very far from the mode of the likelihood (8.38). "Distance" in this context is measured in terms of standard deviations. A "far distance", therefore, implies that $p\left(\mathcal{D} \,\middle|\, \boldsymbol{\beta}, \sigma_\epsilon^2, I\right)$ is very close to zero. This implies that the posterior

$$p\left(\boldsymbol{\beta} \,\middle|\, \mathcal{D}, \sigma_\epsilon^2, I\right) \propto p\left(\mathcal{D} \,\middle|\, \boldsymbol{\beta}, \sigma_\epsilon^2, I\right) p\left(\boldsymbol{\beta} \,|\, I\right), \tag{8.40}$$

simply becomes proportional to the data likelihood (with the prior just truncating the distribution at very large distances). Thus we find from Eq. (8.38)

$$p\left(\boldsymbol{\beta}\,\middle|\,\mathcal{D},\sigma_\epsilon^2,I\right)\propto\exp\left[-\frac{1}{2}(\boldsymbol{\beta}-\boldsymbol{\beta}^*)^T\boldsymbol{\Sigma}_\beta^{-1}(\boldsymbol{\beta}-\boldsymbol{\beta}^*)\right], \tag{8.41}$$

if all $\beta_i\in[-\Delta\beta/2,+\Delta\beta/2]$ while it is zero elsewhere. The mode of this distribution is obviously the mean vector $\boldsymbol{\beta}^*=\boldsymbol{\beta}^*(\mathcal{D})$. We can therefore say that we have recovered the ordinary least-squares result. At this stage, however, the interpretation is that this parameter optimum corresponds to the maximum of the posterior PDF (8.41). Such an optimum is sometimes known as the maximum a posteriori, or MAP.

Checkpoint question

Why is it obvious that the mode of $p\left(\boldsymbol{\beta}\,\middle|\,\mathcal{D},\sigma_\epsilon^2,I\right)$ is the mean vector $\boldsymbol{\beta}^*$?

Answer

The argument of the exponent is negative semi-definite, so the mode (maximum value) of the distribution is when the exponent is equal to zero, which is when $\boldsymbol{\beta}=\boldsymbol{\beta}^*$.

Discuss

In light of the results of this section, what assumption(s) are implicit in linear regression while they are made explicit in Bayesian linear regression?

Posterior with a Gaussian prior

Assigning instead a Gaussian prior for the model parameters, as expressed in Eq. (8.37), we find that the posterior is proportional to the product of two exponential functions

$$\begin{aligned}p\left(\boldsymbol{\beta}\,\middle|\,\mathcal{D},\sigma_\epsilon^2,I\right)&\propto\exp\left[-\frac{1}{2}(\boldsymbol{\beta}-\boldsymbol{\beta}^*)^T\boldsymbol{\Sigma}_\beta^{-1}(\boldsymbol{\beta}-\boldsymbol{\beta}^*)\right]\exp\left[-\frac{1}{2}\frac{\boldsymbol{\beta}^T\boldsymbol{\beta}}{\sigma_\beta^2}\right]\\&\propto\exp\left[-\frac{1}{2}(\boldsymbol{\beta}-\tilde{\boldsymbol{\beta}})^T\widetilde{\boldsymbol{\Sigma}}_\beta^{-1}(\boldsymbol{\beta}-\tilde{\boldsymbol{\beta}})\right].\end{aligned} \tag{8.42}$$

The second proportionality is a consequence of both exponents being quadratic in the model parameters, and therefore that the full expression looks like the product of two Gaussians. This product is proportional to another Gaussian distribution which has mean vector and (inverse) covariance matrix given by

$$\begin{aligned}\tilde{\boldsymbol{\beta}}&=\widetilde{\boldsymbol{\Sigma}}_\beta\boldsymbol{\Sigma}_\beta^{-1}\boldsymbol{\beta}^*\\\widetilde{\boldsymbol{\Sigma}}_\beta^{-1}&=\boldsymbol{\Sigma}_\beta^{-1}+\sigma_\beta^{-2}\mathbf{1}\end{aligned} \tag{8.43}$$

where $\mathbf{1}$ is the $N_p\times N_p$ unit matrix. In effect, what has happened is that the prior normal distribution becomes updated to a posterior normal distribution via an inference process that involves a data likelihood. In this particular case, learning from data implies that the mode changes from $\mathbf{0}$ to $\tilde{\boldsymbol{\beta}}$ and the covariance from a diagonal structure with σ_β^2 in all directions to the covariance matrix $\widetilde{\boldsymbol{\Sigma}}_\beta$.

Checkpoint question

What happens if the data is of high quality (i.e., the likelihood $\mathcal{L}(\boldsymbol{\beta})$ is sharply peaked around $\boldsymbol{\beta}^*$), and what happens if it is of poor quality (providing a very broad likelihood distribution)?

Marginal posterior distributions

Given a multivariate probability distribution we are often interested in lower dimensional marginal distributions. Consider for example $\boldsymbol{\beta}^T = [\boldsymbol{\beta}_1^T, \boldsymbol{\beta}_2^T]$, that is partitioned into dimensions D_1 and D_2. The marginal distribution for $\boldsymbol{\beta}_2$ corresponds to the integral

$$p\left(\boldsymbol{\beta}_2\right) = \int d\boldsymbol{\beta}_1 p\left(\boldsymbol{\beta}\right).$$

Transformation property of multivariate normal distributions

Let $\mathbf{Y}$ be a multivariate normal-distributed random variable of length N_p with mean vector $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$. We use the notation $p_{\mathbf{Y}}\left(\mathbf{y}\right) = \mathcal{N}(\mathbf{y}|\mu, \boldsymbol{\Sigma})$ to emphasize which variable is normally distributed.

Consider now a general $M \times N_p$ matrix $\boldsymbol{A}$ and $M \times 1$ vector $\boldsymbol{b}$. Then, the random variable $\mathbf{Z} = \boldsymbol{A}\mathbf{Y} + \boldsymbol{b}$ is also multivariate normal-distributed with the PDF

$$p_{\mathbf{Z}}\left(\mathbf{z}\right) = \mathcal{N}(\mathbf{z}|\mathbf{A}\boldsymbol{\mu} + \boldsymbol{b}, \boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^T). \tag{8.44}$$

For multivariate normal distributions we can employ a useful transformation property, shown in Eq. (8.44). Considering the posterior (8.42) we partition the parameters $\boldsymbol{\beta}^T = [\boldsymbol{\beta}_1^T, \boldsymbol{\beta}_2^T]$, of dimensions D_1 and D_2 ($D_1 + D_2 = N_p$), and the mean vector and covariance matrix into $\boldsymbol{\mu}^T = [\boldsymbol{\mu}_1^T, \boldsymbol{\mu}_2^T]$ and

$$\boldsymbol{\Sigma} = \begin{bmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{12}^T & \boldsymbol{\Sigma}_{22} \end{bmatrix}.$$

We can obtain the marginal distribution for $\boldsymbol{\beta}_2$ by setting

$$\mathbf{A} = [\mathbf{0}_{D_2 \times D_1} \quad \mathbf{1}_{D_2 \times D_2}], \ \mathbf{b} = 0,$$

which yields

$$p\left(\boldsymbol{\beta}_2 \mid \mathcal{D}, I\right) = \mathcal{N}(\boldsymbol{\beta}_2|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_{22}). \tag{8.45}$$

The posterior predictive

One can also derive the posterior predictive distribution (PPD), i.e., the probability distribution for predictions $\widetilde{\boldsymbol{\mathcal{F}}}$ given the model M and a set of new inputs for the independent variable $\boldsymbol{x}$. The new inputs give rise to a new design matrix $\widetilde{\boldsymbol{X}}$.

We obtain the posterior predictive distribution by marginalizing over the uncertain model parameters that we just inferred from the given data $\mathcal{D}$.

$$p\left(\widetilde{\boldsymbol{\mathcal{F}}} \,|\, \mathcal{D}, \sigma_\epsilon^2, I\right) = \int p\left(\widetilde{\boldsymbol{\mathcal{F}}} \,|\, \boldsymbol{\beta}, \sigma_\epsilon^2, I\right) p\left(\boldsymbol{\beta} \,|\, \mathcal{D}, \sigma_\epsilon^2, I\right) d\boldsymbol{\beta}, \tag{8.46}$$

where both distributions in the integrand can be expressed as Gaussians. Alternatively, one can express the PPD as the set of model predictions with the model parameters distributed according to the posterior parameter PDF

$$\left\{\widetilde{\boldsymbol{X}}\boldsymbol{\beta} \,:\, \boldsymbol{\beta} \sim p\left(\boldsymbol{\beta} \,|\, \mathcal{D}, \sigma_\epsilon^2, I\right)\right\}. \tag{8.47}$$

This set of predictions can be obtained if we have access to a set of samples from the parameter posterior. Note that it describes the uncertainty of the model predictions only. Predictions for new data also include the noise, which is obtained by adding a random draw from $\mathcal{N}(0, \sigma_\epsilon^2)$ to each prediction.

Checkpoint question

Fill in the details to obtain the right side of (8.46).

Answer

Introduce the integration over $\boldsymbol{\beta}$ using the sum rule and then obtain the right side from the product rule.

Solutions to exercises

Solution to Exercise 8.3 (Prove the Gaussian likelihood)

1. *Negative log-likelihood.* We write the likelihood (8.34) as $p\left(\mathcal{D} \,|\, \boldsymbol{\beta}, I\right) = \exp\left[-L(\boldsymbol{\beta})\right]$. The negative log-likelihood, including the normalization factor, is

   $$L(\boldsymbol{\beta}) = \frac{N_d}{2}\log(2\pi\sigma_\epsilon^2) + \frac{1}{2\sigma_\epsilon^2}(\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})^T(\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta}).$$

   Apart from the constant first term and the factor $1/(2\sigma_\epsilon^2)$, this is the least-squares cost function (8.18).
2. *Gradient and mode.* Using the gradient (8.22) of the cost function we find

   $$\boldsymbol{\nabla}_{\boldsymbol{\beta}} L(\boldsymbol{\beta}) = -\frac{\boldsymbol{X}^T(\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})}{\sigma_\epsilon^2}.$$

Setting it to zero gives the normal equation (8.19). The gradient therefore vanishes at $\boldsymbol{\beta} = \boldsymbol{\beta}^* = \left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\mathcal{D}$, which is the mode of the likelihood.

3. *Hessian.* Differentiating the gradient once more gives the matrix of second derivatives

$$\boldsymbol{H} = \boldsymbol{\nabla}_\beta \boldsymbol{\nabla}_\beta^T L(\boldsymbol{\beta}) = \frac{\boldsymbol{X}^T\boldsymbol{X}}{\sigma_\epsilon^2},$$

which is constant, i.e., independent of $\boldsymbol{\beta}$.

4. *Taylor expansion.* The expansion of $L(\boldsymbol{\beta})$ around $\boldsymbol{\beta}^*$ is

$$L(\boldsymbol{\beta}) = L(\boldsymbol{\beta}^*) + (\boldsymbol{\beta}-\boldsymbol{\beta}^*)^T\boldsymbol{\nabla}_\beta L(\boldsymbol{\beta}^*) + \frac{1}{2}(\boldsymbol{\beta}-\boldsymbol{\beta}^*)^T\boldsymbol{H}(\boldsymbol{\beta}-\boldsymbol{\beta}^*).$$

It is *exact* and terminates at second order since L is a quadratic polynomial in $\boldsymbol{\beta}$, such that all third and higher derivatives vanish. Furthermore, the linear term is zero since $\boldsymbol{\nabla}_\beta L(\boldsymbol{\beta}^*) = 0$.

5. *Exponentiate.* With $\boldsymbol{\Sigma}_\beta^{-1} \equiv \boldsymbol{H} = \boldsymbol{X}^T\boldsymbol{X}/\sigma_\epsilon^2$ and $\exp\left[-L(\boldsymbol{\beta}^*)\right] = p\left(\mathcal{D} \mid \boldsymbol{\beta}^*, I\right)$ we obtain

$$p\left(\mathcal{D} \mid \boldsymbol{\beta}, I\right) = \exp\left[-L(\boldsymbol{\beta})\right] = p\left(\mathcal{D} \mid \boldsymbol{\beta}^*, I\right)\exp\left[-\frac{1}{2}(\boldsymbol{\beta}-\boldsymbol{\beta}^*)^T\boldsymbol{\Sigma}_\beta^{-1}(\boldsymbol{\beta}-\boldsymbol{\beta}^*)\right],$$

as we wanted to show. The prefactor $p\left(\mathcal{D} \mid \boldsymbol{\beta}^*, I\right)$ does not depend on $\boldsymbol{\beta}$, so as a function of the parameters the likelihood is proportional to a multivariate normal distribution with mean $\boldsymbol{\beta}^*$ and covariance matrix $\boldsymbol{\Sigma}_\beta$.

Alternative: completing the square

The same result can be obtained without a Taylor expansion. Writing $\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta} = (\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta}^*) - \boldsymbol{X}(\boldsymbol{\beta}-\boldsymbol{\beta}^*)$ gives

$$\begin{aligned}(\mathcal{D}-\boldsymbol{X}\boldsymbol{\beta})^T(\mathcal{D}-\boldsymbol{X}\boldsymbol{\beta}) = &(\mathcal{D}-\boldsymbol{X}\boldsymbol{\beta}^*)^T(\mathcal{D}-\boldsymbol{X}\boldsymbol{\beta}^*)\\ &+(\boldsymbol{\beta}-\boldsymbol{\beta}^*)^T\boldsymbol{X}^T\boldsymbol{X}(\boldsymbol{\beta}-\boldsymbol{\beta}^*)\\ &-2(\boldsymbol{\beta}-\boldsymbol{\beta}^*)^T\boldsymbol{X}^T(\mathcal{D}-\boldsymbol{X}\boldsymbol{\beta}^*).\end{aligned} \tag{8.48}$$

The last term vanishes since $\boldsymbol{\beta}^*$ solves the normal equation, $\boldsymbol{X}^T(\mathcal{D}-\boldsymbol{X}\boldsymbol{\beta}^*) = 0$.

8.5. BLR-III: Demo

Prelude: ordinary linear regression

To warm up, and get acquainted with the notation and formalism, let us work out a small example. Assume that we have the situation where we have collected two datapoints $\mathcal{D} = [y_1, y_2]^T = [-3, 3]^T$ for the predictor values $[x_1, x_2]^T = [-2, 1]^T$.

This data could have come from any process, even a non-linear one. But this is artificial data that I generated by evaluating the function $y = 1 + 2x$ at $x = x_1 = -2$ and $x = x_2 = 1$. Clearly, the data-generating process is very simple and corresponds to a linear model $y = \beta_0 + \beta_1 x$ with $[\beta_0, \beta_1] = [1, 2]$. This is the kind of information

we *never* have in reality. Indeed, we are always uncertain about the process that maps input to output, and as such our model M will always be wrong. We are also uncertain about the parameters $\boldsymbol{\beta}$ of our model. These are the some of the fundamental reasons for why it can be useful to operate with a Bayesian approach where we can assign probabilities to any quantity and statement. In this example, however, we will continue with the standard (frequentist) approach based on finding the parameters that minimize the squared errors (i.e., the norm of the residual vector).

We will now assume a linear model with polynomial basis up to order one to model the data, i.e.,

$$M(\boldsymbol{\beta}; x) = \beta_0 + \beta_1 x,$$

which we can express in terms of a design matrix $\boldsymbol{X}$ and (unknown) parameter vector $\boldsymbol{\beta}$ as $M = \boldsymbol{X}\boldsymbol{\beta}$.

In the present case the two unknowns $\boldsymbol{\beta} = [\beta_0, \beta_1]^T$ can be fit to the two datapoints $\mathcal{D} = [-3, 3]^T$ using pen a paper.

Exercise 8.5

In the example above you have two data points and two unknowns, which means you can easily solve for the model parameters using a conventional matrix inverse. Do the numerical calculation to make sure you have set up the problem correctly.

Exercise 8.6

Evaluate the normal equations for the design matrix $\boldsymbol{X}$ and data vector $\mathcal{D}$ in the example above.

Exercise 8.7

Evaluate the sample variance s^2 for the example above using Eq. (8.25). Do you think the result makes sense?

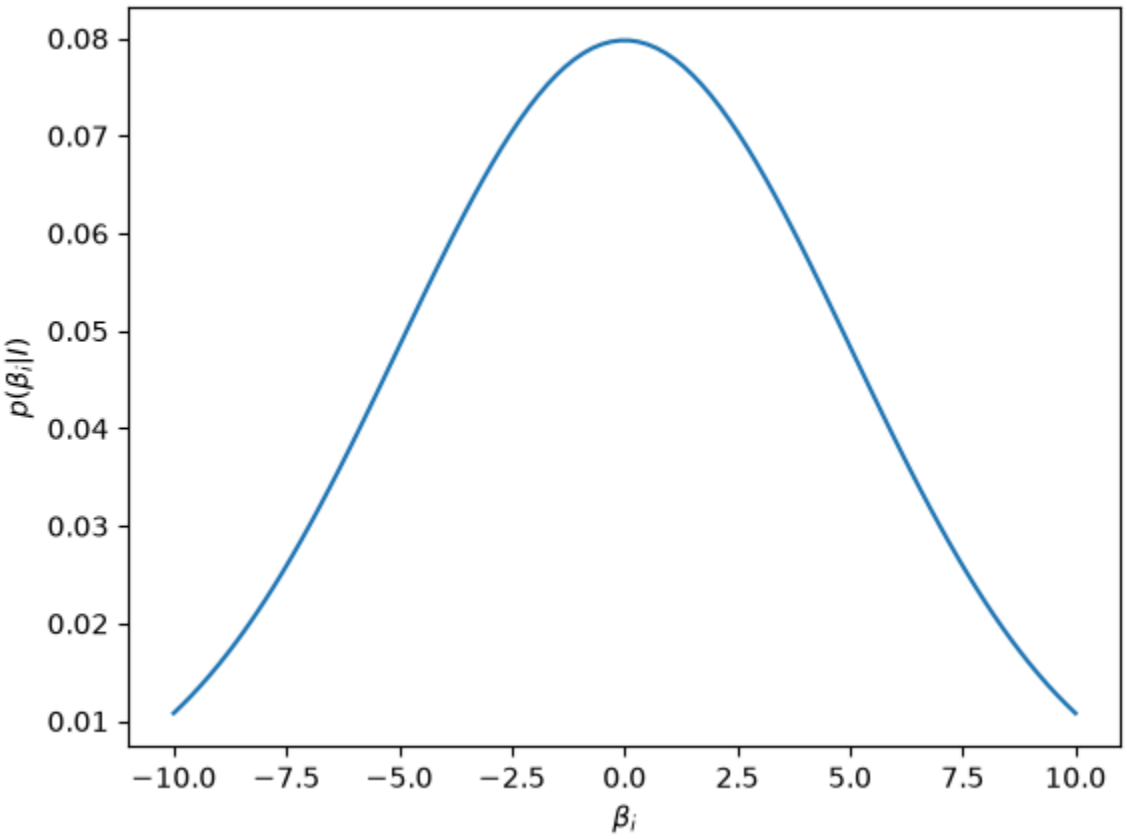
Continuing …

For the time being we assume to know enough about the data to consider a normal likelihood with i.i.d. errors. Let us first set the known residual variance to $\sigma_\epsilon^2 = 0.5^2$.

This time we also have prior knowledge that we would like to build into the inference. Here we use a normal prior for the parameters with $\sigma_\beta = 5.0$, which is to say that before looking at the data we believe the pdf for $\boldsymbol{\beta}$ to be centered on zero with a variance of 5^2.

Let us plot this prior. The prior is the same for β_0 and β_1, so it is enough to plot one of them.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

def normal_distribution(mu,sigma2):
    return norm(loc=mu,scale=np.sqrt(sigma2))

betai = np.linspace(-10,10,100)
prior = normal_distribution(0,5.0**2)

fig, ax = plt.subplots(1,1)
ax.plot(betai,prior.pdf(betai))
ax.set_ylabel(r'$p(\beta_i \vert I )$')
ax.set_xlabel(r'$\beta_i$');
```



It is straightforward to evaluate Eq. (8.43), which gives us

$$\begin{aligned}\tilde{\boldsymbol{\Sigma}}_{\beta}^{-1} &= 4\begin{pmatrix}2.01 & -1.0\\ -1.0 & 5.01\end{pmatrix}\\ \tilde{\boldsymbol{\beta}} &= (0.992, 1.994)\end{aligned} \tag{8.49}$$

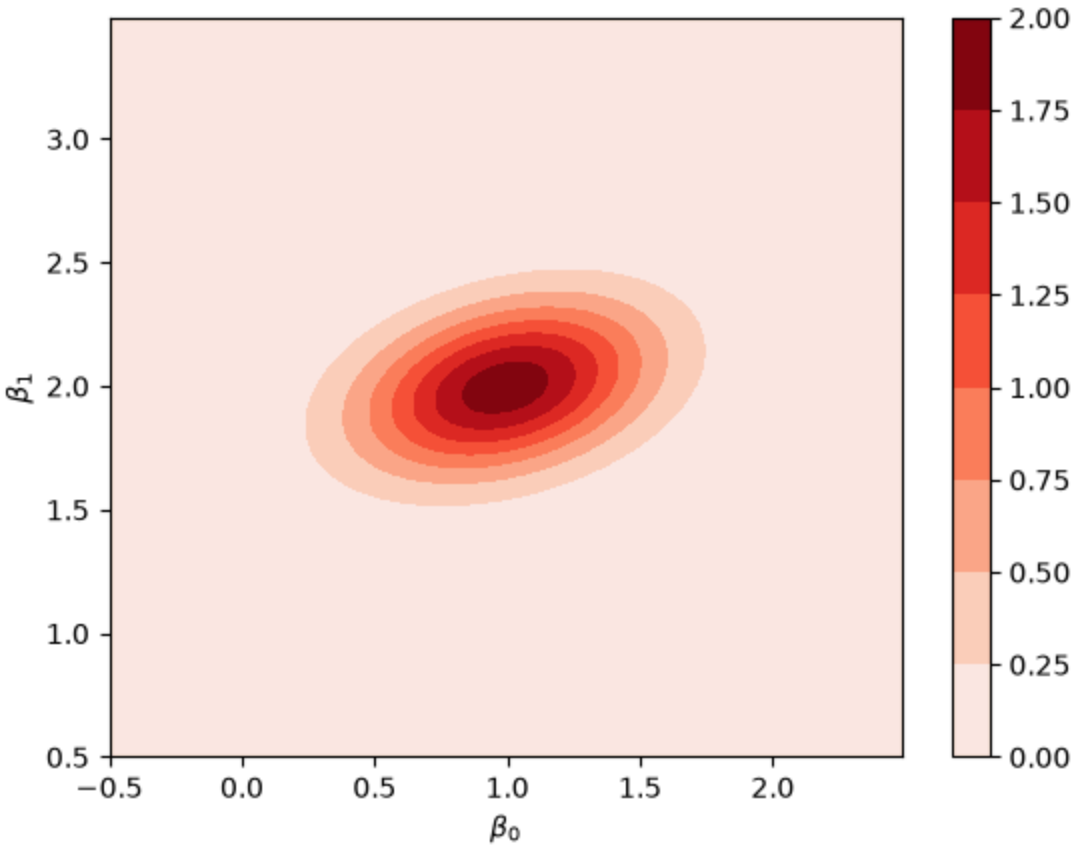
This should be compared with the parameter vector $(1, 2)$ we recovered using ordinary linear regression. With Bayesian linear regression we start from an informative prior with both parameters centered on zero with a rather large variance.

Exercise 8.8 (Warm-up Bayesian linear regression)

> Reproduce the posterior mean and covariance matrix from Eq. (8.49). You can use `numpy` methods to perform the linear algebra operations.

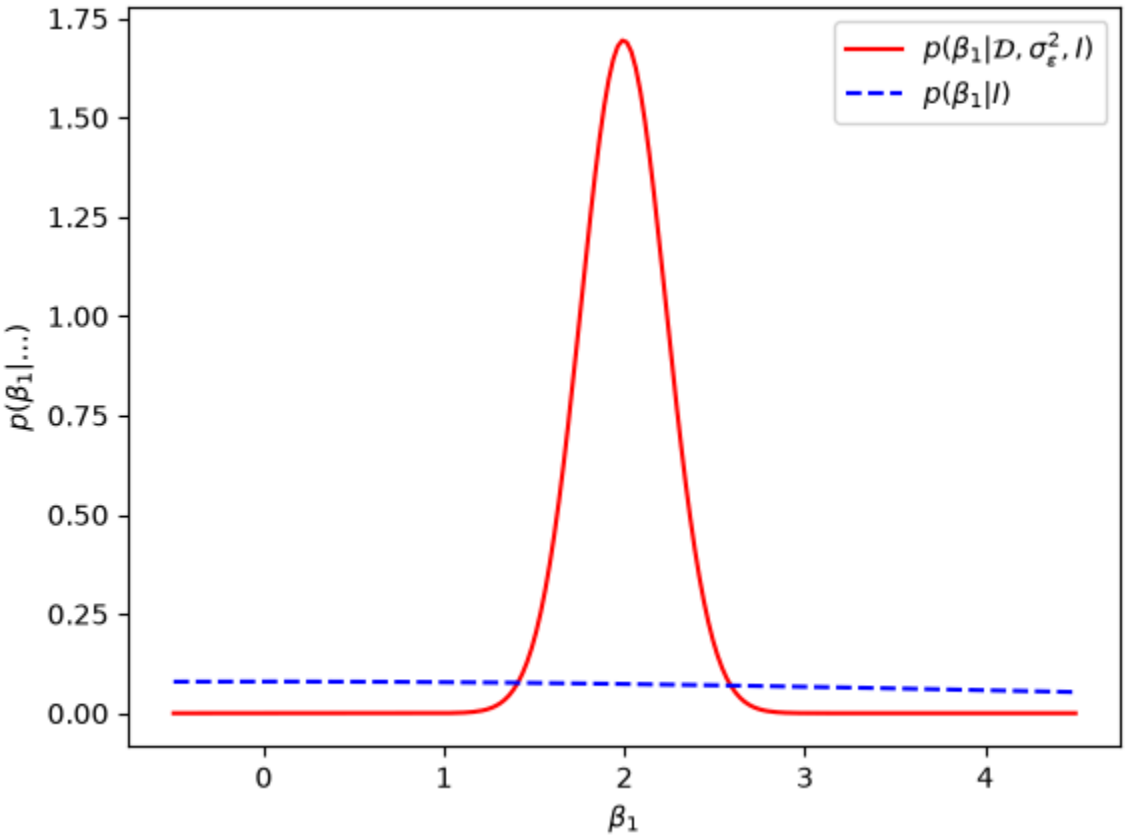
We can plot the posterior probability distribution for $\boldsymbol{\theta}$, i.e., by plotting the bi-variate $\mathcal{N}-$distribution with the parameter in Eq. (8.49).

```
from scipy.stats import multivariate_normal

mu = np.array([0.992,1.994])
Sigma = np.linalg.inv(4 * np.array([[2.01,-1.0],[-1.0,5.01]]))

posterior = multivariate_normal(mean=mu, cov=Sigma)

beta0, beta1 = np.mgrid[-0.5:2.5:.01, 0.5:3.5:.01]
beta_grid = np.dstack((beta0, beta1))

fig,ax = plt.subplots(1,1)
ax.set_xlabel(r'$\beta_0$')
ax.set_ylabel(r'$\beta_1$')
im = ax.contourf(beta0, beta1, posterior.pdf(beta_grid),cmap=plt.cm.Reds);
fig.colorbar(im);
```



Using Eq. (8.45) we can obtain, e.g., the β_1 marginal density and compare with the prior

```
beta1 = np.linspace(-0.5,4.5,200)
mu1 = mu[1]
Sigma11_sq = Sigma[1,1]

posterior1 = normal_distribution(mu1,Sigma11_sq)

fig, ax = plt.subplots(1,1)
ax.plot(beta1,posterior1.pdf(beta1),'r-',\
label=r'$p(\beta_1 \vert \mathcal{D}, \sigma_\epsilon^2, I )$')
ax.plot(beta1,prior.pdf(beta1), 'b--',label=r'$p(\beta_1 \vert I )$')
ax.set_ylabel(r'$p(\beta_1 \vert \ldots )$')
ax.set_xlabel(r'$\beta_1$')
ax.legend(loc='best');
```



The key take-away with this numerical exercise is that Bayesian inference yields a probability distribution for the model parameters whose values we are uncertain about. With ordinary linear regression techniques you only obtain the parameter values that optimize some cost function, and not a probability distribution.

Exercise 8.9 (Warm-up Bayesian linear regression (data errors))

Explore the sensitivity to changes in the residual errors σ_ϵ. Try to increase and reduce the error.

Exercise 8.10 (Warm-up Bayesian linear regression (prior sensitivity))

Explore the sensitivity to changes in the Gaussian prior width σ_β. Try to increase and reduce the width.

Exercise 8.11 ("In practice" Bayesian linear regression)

Perform Bayesian Linear Regression on the data that was generated in Addendum: Ordinary linear regression in practice. Explore:

- Dependence on the quality of the data (generate data with different σ_ϵ) or the number of data.
- Dependence on the polynomial function that was used to generate the data.
- Dependence on the number of polynomial terms in the model.
- Dependence on the parameter prior.

In all cases you should compare the Bayesian inference with the results from Ordinary Least Squares and with the true parameters that were used to generate the data.

Solutions to exercises

Solution to Exercise 8.5

We have the following design matrix

$$\boldsymbol{X} = \begin{bmatrix} 1 & -2 \\ 1 & 1 \end{bmatrix},$$

which in the present case yields the parameter values

$$\boldsymbol{\theta}^* = \boldsymbol{X}^{-1}\mathcal{D} = [1, 2]^T.$$

Solution to Exercise 8.7

For the warmup case we have fitted a straight line through two data points, which is always possible, and we cannot determine the sample variance. This will be even more clear when we come to BLR-I: Background.

8.6. A Bayesian view of bias and variance

In the previous sections we built up Bayesian linear regression using a normal likelihood (8.34) and a Gaussian prior (8.37). The resulting posterior is also Gaussian (8.43), with mean $\tilde{\boldsymbol{\beta}}$ and covariance $\widetilde{\boldsymbol{\Sigma}}_\beta$. A very broad prior recovers the least-squares optimum $\boldsymbol{\beta}^*$ as the posterior mode, while an informative prior pulls the parameters towards the prior mean.

In this section we look at what that pull does to *predictions*, and we use it to introduce two issues that will be central when we discuss model validation for Machine learning in Part IV (Section 25.3): a model can be too simple or too flexible for the data at hand, and the two failure modes look very different. The Bayesian and the frequentist descriptions of this differ in one respect that is worth being clear about from the start.

Fixing parameters and varying data, or fixing data and varying parameters

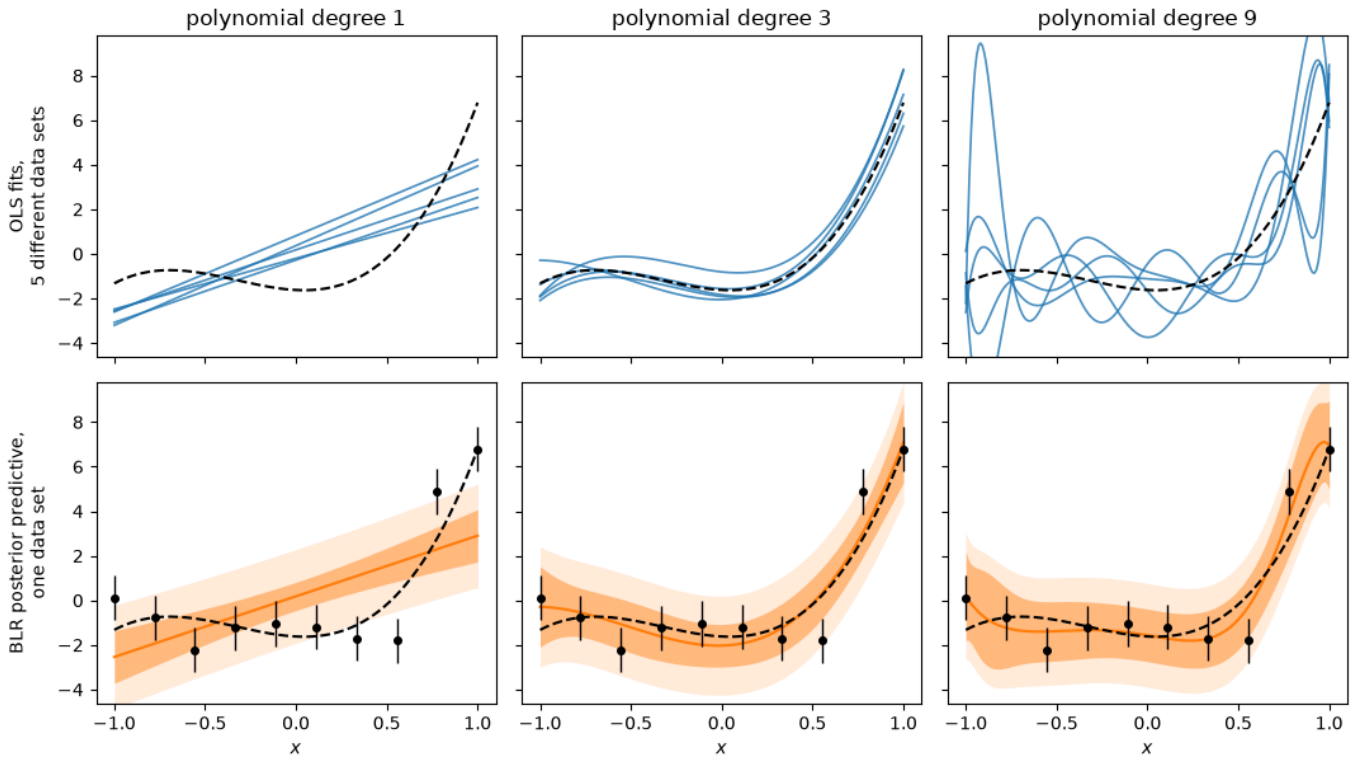
The frequentist analysis of an estimator imagines a fixed, true parameter vector β_{true} and asks how a fitted quantity would scatter if the experiment were repeated with fresh noise, over data sets that were never collected. Bayesian inference does the opposite: it fixes the one data set we actually have and treats β as the uncertain quantity. Both are legitimate views. They just lead to different questions and different answers, and it is easy to confuse them.

Model complexity: underfitting and overfitting

Consider the polynomial data-generating process from the addendum of Section 8.3: a cubic function with random coefficients, measured at $N_d = 10$ points with noise $\sigma_\epsilon = 1$. We fit it with polynomials of degree 1, 3 and 9; note that the degree-9 model has $N_p = N_d$ parameters. The figure below demonstrates two different data fitting scenarios.

- *Top row:* ordinary least squares (OLS), repeated for five independent noisy data sets from the same data-generating process. Each fit is a single curve, a point estimate; the spread among the five curves shows how much that estimate depends on which data set we happened to get.
- *Bottom row:* Bayesian linear regression with a broad Gaussian prior, $\sigma_\beta = 10$, applied to *one* data set. The result is a distribution over curves, summarised by the posterior predictive mean and its 2σ band.

▼ **Hide code cell source**



The dashed line is the true cubic. In the bottom row the darker band is the 2σ uncertainty of the model prediction itself (from the posterior of β), the lighter band adds the measurement noise σ_ϵ.

Read the top row first. The straight lines (degree 1) agree closely with each other but are all wrong in the same way: the model is too simple to follow the data, and no amount of resampling will fix that. This is **underfitting**, and the

systematic offset is what we will later call **bias**. The degree-9 fits show the opposite problem: each curve passes exactly through its own ten data points, but the curves differ wildly from one another because they are following the noise. This is **overfitting**, and the scatter between fits is what we will call **variance**. The cubic is a compromise: a little scatter, no systematic offset.

Now compare with the bottom row. The Bayesian analysis does not have several data sets to compare; it has one. Yet it displays the same information in a different form. For degree 1 the predictive band is narrow but misses the data: the model is confident and wrong. For degree 9 the band is widest where the OLS curves scatter most, near the edges and between data points, because those are the directions in parameter space that a single data set fails to pin down. Even the broad prior used here has tamed the wiggles that made the OLS fits so erratic, since it penalises large coefficients. What the frequentist reads off from imagined repetitions, the Bayesian reads off from the posterior width.

What the predictive distribution contains

Recall from Section 8.4 that for a new input $x^{\odot}$, with corresponding row $\widetilde{\boldsymbol{X}}$ of basis functions, the posterior predictive distribution (8.46) is normal,

$$p\left(\mathcal{F} \,\middle|\, \mathcal{D}, \sigma_\epsilon^2, I\right) = \mathcal{N}\left(\widetilde{\boldsymbol{X}}\tilde{\boldsymbol{\beta}},\;\; \sigma_\epsilon^2 + \widetilde{\boldsymbol{X}}\,\tilde{\boldsymbol{\Sigma}}_\beta\,\widetilde{\boldsymbol{X}}^T\right). \tag{8.50}$$

Its variance has two parts. The term σ_ϵ^2 is the noise of a future measurement (the lighter band in the figure); it is irreducible and does not shrink as we collect more data of the same quality. The term $\widetilde{\boldsymbol{X}}\tilde{\boldsymbol{\Sigma}}_\beta\widetilde{\boldsymbol{X}}^T$ is our remaining uncertainty about $\boldsymbol{\beta}$ (the darker band); it is a property of our state of knowledge and shrinks with more data. In machine-learning language these are called **aleatoric** and **epistemic** uncertainty. In the frequentist decomposition of Part IV, σ_ϵ^2 reappears as the irreducible noise term, while the variance term is the counterpart of the epistemic part. Bias has no counterpart in the posterior width: a model that is too simple can be confidently wrong, as the degree-1 panel shows.

The prior as a regulariser

The prior width σ_β is what kept the degree-9 model in the bottom row under control. Its effect can be stated in a form that a frequentist would recognise.

BLR with a Gaussian prior is ridge regression

Maximising the posterior (8.42) is the same as minimising

$$(\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta})^T\,(\mathcal{D} - \boldsymbol{X}\boldsymbol{\beta}) + \lambda\,\boldsymbol{\beta}^T\boldsymbol{\beta}, \qquad \lambda = \frac{\sigma_\epsilon^2}{\sigma_\beta^2}, \tag{8.51}$$

which is the cost function of **ridge regression** (or Tikhonov regularization), with solution $\tilde{\boldsymbol{\beta}} = (\boldsymbol{X}^T\boldsymbol{X} + \lambda\mathbf{1})^{-1}\boldsymbol{X}^T\mathcal{D}$. The regularization strength λ that a frequentist would tune by cross-validation is here the ratio of residual variance to prior variance: a penalty otherwise introduced by hand is a statement of prior belief about the size of the parameters.

A tight prior (λ large) pulls all coefficients towards zero and pushes the model towards underfitting; a broad prior ($\lambda \to 0$) returns the least-squares fit with all its scatter. The trade-off between the two failure modes is not removed by the Bayesian framework. What changes is how the balance point is chosen: rather than a tuning parameter fixed on held-out data, the prior width is a modelling assumption that can be stated, justified and criticised like any other. Another consequence is visible in the figure: since $\widetilde{\boldsymbol{\Sigma}}_{\boldsymbol{\beta}}^{-1} = \boldsymbol{\Sigma}_{\boldsymbol{\beta}}^{-1} + \sigma_{\beta}^{-2}\mathbf{1}$ is always invertible, BLR can fit a model with more parameters than data points, while OLS cannot.

Looking ahead

In Section 25.3 we will make the top row of the figure quantitative: bias and variance will be defined as expectation values over repeated data sets, and the expected squared prediction error will be shown to decompose into bias^2 + variance + noise. Bayesian linear regression is a good test bed for that theorem, because the posterior mean $\tilde{\boldsymbol{\beta}}$ is itself an estimator whose bias and variance can be computed in closed form. Readers who want to see this now can open the block below; everyone else can safely skip it until Part IV.

Advanced: bias and variance of the posterior mean

The posterior mean as an estimator. The Bayesian result of Section 8.4 is a distribution, not a number. But the posterior mean

$$\tilde{\boldsymbol{\beta}} = \widetilde{\boldsymbol{\Sigma}}_{\boldsymbol{\beta}}\boldsymbol{\Sigma}_{\boldsymbol{\beta}}^{-1}\boldsymbol{\beta}^{*} \equiv \boldsymbol{S}\boldsymbol{\beta}^{*}, \qquad \boldsymbol{S} \equiv \widetilde{\boldsymbol{\Sigma}}_{\boldsymbol{\beta}}\boldsymbol{\Sigma}_{\boldsymbol{\beta}}^{-1} = \mathbf{1} - \frac{\widetilde{\boldsymbol{\Sigma}}_{\boldsymbol{\beta}}}{\sigma_{\beta}^{2}}, \tag{8.52}$$

is a number (a vector of them), and we are free to ask how it behaves under repeated sampling of $\mathcal{D}$. We call $\boldsymbol{S}$ the **shrinkage matrix**. Its second form follows from $\widetilde{\boldsymbol{\Sigma}}_{\boldsymbol{\beta}}^{-1} = \boldsymbol{\Sigma}_{\boldsymbol{\beta}}^{-1} + \sigma_{\beta}^{-2}\mathbf{1}$. As $\sigma_{\beta} \to \infty$, $\boldsymbol{S} \to \mathbf{1}$ and $\tilde{\boldsymbol{\beta}} \to \boldsymbol{\beta}^{*}$. The shrinkage is not uniform: a direction the data constrain sharply is barely affected, while a direction the data say little about is pulled strongly towards the prior mean.

Now let the data be generated by the model itself with fixed but unknown $\boldsymbol{\beta}_{\text{true}}$,

$$\mathcal{D} = \boldsymbol{X}\boldsymbol{\beta}_{\text{true}} + \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_{\epsilon}^{2}\mathbf{1}), \tag{8.53}$$

and draw many data sets with $\boldsymbol{X}$ held fixed. The least-squares estimator (8.20) is unbiased with sampling covariance

$$\mathbb{E}\left[\boldsymbol{\beta}^{*}\right] = \boldsymbol{\beta}_{\text{true}}, \qquad \text{Var}\ \left(\boldsymbol{\beta}^{*}\right) = \sigma_{\epsilon}^{2}\left(\boldsymbol{X}^{T}\boldsymbol{X}\right)^{-1} = \boldsymbol{\Sigma}_{\boldsymbol{\beta}}, \tag{8.54}$$

so the matrix $\boldsymbol{\Sigma}_{\boldsymbol{\beta}}$ that entered (8.39) as the *curvature of the log-likelihood* is also the *sampling covariance of the OLS estimator*. Applying $\boldsymbol{S}$ gives, for the posterior mean,

$$\mathbb{E}\left[\tilde{\boldsymbol{\beta}}\right] = \boldsymbol{S}\,\boldsymbol{\beta}_{\text{true}}, \qquad \text{Var}\ \left(\tilde{\boldsymbol{\beta}}\right) = \boldsymbol{S}\boldsymbol{\Sigma}_{\boldsymbol{\beta}}\boldsymbol{S}^{T} = \boldsymbol{S}\,\widetilde{\boldsymbol{\Sigma}}_{\boldsymbol{\beta}}. \tag{8.55}$$

Bias and variance of a prediction. The quantity to get right at a test input is the noise-free value $\widetilde{\boldsymbol{X}}\boldsymbol{\beta}_{\text{true}}$, and our estimate is $\widetilde{\boldsymbol{X}}\tilde{\boldsymbol{\beta}}$. From (8.55) and (8.52),

$$\begin{aligned}\text{Bias }(x^{\odot}) &= \widetilde{\boldsymbol{X}}\mathbb{E}\left[\tilde{\boldsymbol{\beta}}\right] - \widetilde{\boldsymbol{X}}\boldsymbol{\beta}_{\text{true}} = -\frac{1}{\sigma_\beta^2}\,\widetilde{\boldsymbol{X}}\,\tilde{\boldsymbol{\Sigma}}_\beta\,\boldsymbol{\beta}_{\text{true}},\\ \text{Var }(x^{\odot}) &= \widetilde{\boldsymbol{X}}\,\text{Var}\,\left(\tilde{\boldsymbol{\beta}}\right)\widetilde{\boldsymbol{X}}^T = \widetilde{\boldsymbol{X}}\,\boldsymbol{S}\,\tilde{\boldsymbol{\Sigma}}_\beta\,\widetilde{\boldsymbol{X}}^T.\end{aligned} \tag{8.56}$$

The bias is proportional to $1/\sigma_\beta^2$: it is the price of the prior and vanishes only in the uninformative limit.

Comparing with a noisy future measurement $y^{\odot} = \widetilde{\boldsymbol{X}}\boldsymbol{\beta}_{\text{true}} + \epsilon$ adds one more term,

$$\mathbb{E}\left[\left(y^{\odot} - \widetilde{\boldsymbol{X}}\tilde{\boldsymbol{\beta}}\right)^2\right] = \underbrace{\text{Bias }(x^{\odot})^2 + \text{Var }(x^{\odot})}_{\text{reducible}} + \underbrace{\sigma_\epsilon^2}_{\text{irreducible}}, \tag{8.57}$$

which is the decomposition derived in general in Section 25.3. Underfitting is the bias-dominated regime (σ_β small, $\boldsymbol{S} \to \boldsymbol{0}$); overfitting is the variance-dominated regime (σ_β large, N_p approaching N_d so that $\boldsymbol{\Sigma}_\beta$ has large eigenvalues).

Scalar illustration. For a single parameter estimated from N_d repeated measurements ($\boldsymbol{X}^T\boldsymbol{X} = N_d$) everything collapses to scalars,

$$\boldsymbol{\Sigma}_\beta = \frac{\sigma_\epsilon^2}{N_d}, \qquad \tilde{\boldsymbol{\Sigma}}_\beta = \left(\frac{N_d}{\sigma_\epsilon^2} + \frac{1}{\sigma_\beta^2}\right)^{-1}, \qquad S = \frac{N_d/\sigma_\epsilon^2}{N_d/\sigma_\epsilon^2 + 1/\sigma_\beta^2}. \tag{8.58}$$

The code cell below plots Bias^2 and Var against the prior width for $N_d = 20$, $\sigma_\epsilon = 1$, $\beta_{\text{true}} = 2$. Bias^2+Var has its minimum at $\sigma_\beta^2 = \beta_{\text{true}}^2$, exactly where the prior correctly describes the size of the parameter.

▼ **Hide code cell source**

▼ **Hide code cell output**

9. Problems for Part I

In this chapter we collect various hands-on problems to build intuition or gain practice in the basics of Bayesian inference. You will further explore PDFs and parameter estimation, and you will get a first exposure to Monte Carlo sampling.

- 📥 Problem: Exploring PDFs has a series of tasks to play with plots of probability density functions (PDFs) using the scipy.stats and numpy libraries.
- 📥 Problem: Gaussian noise and averages I takes a look at a simple parameter-estimation problem, namely estimating the mean and the variance of a normal distribution that is associated with a collection of random variables. The exercise develops a Bayesian approach to this problem and shows how it reduces to standard frequentist estimators for a particular choice of prior.
- 📥 Problem: Radioactive lighthouse explores a variation on a classic problem from Gull, which entails identifying the location of a hidden radioactive source using a Bayesian approach.
- The goal of 📥 Problem: Amplitude of a signal in the presence of background is to estimate the amplitude of a signal when there is background. The notebook considers a limiting case where the background is flat, so it is

completely specified by its magnitude, and the signal is known to be a Gaussian with unknown amplitude but (at least initially) known position (mean) and width (standard deviation). This exercise can be considered a first exposure to an experimental design problem.
- 📥 Problem: fitting a straight line is a simple Bayesian parameter estimation example in the context of the familiar problem of fitting a straight line to noisy data.

The final exercises introduce the use of Markov Chain Monte Carlo (MCMC) sampling in basic parameter estimation problems, but in a black-box mode. This first exposure to MCMC is focused on the visualization and interpretation of sampled posteriors, without delving into the details of how the MCMC algorithm that creates them works. Those latter details are explored in Part III, which develops intuition, provides technical details, and introduces diagnostics for troubleshooting and validation.

- 📥 Problem: Gaussian noise and averages II revisits the problem considered in 📥 Problem: Gaussian noise and averages I but now determines the posterior via MCMC sampling.
- 📥 Problem: 2D radioactive lighthouse location using MCMC revisits 📥 Problem: Radioactive lighthouse with a more complex goal (identifying the location of the lighthouse in two dimensions) that is carried out using MCMC sampling.
- 📥 Problem: fitting a straight line II extends the straight-line parameter estimation problem by considering marginalizing over nuisance parameters and error propagation while using MCMC sampling.
- Compare Gaussian noise sampling to lighthouse analysis is a brief exercise highlighting the commonality of sampling problems using MCMC.

9.1. 📥 Problem: Exploring PDFs

Goal: To play with plots of probability density functions (PDFs) using the `scipy.stats` and `numpy` libraries.

Do the tasks and answer the questions in *italics*. But also explore!

```
import scipy.stats as stats  # We'll use stats as our source of PDFs
import numpy as np

import matplotlib.pyplot as plt
import corner  # for making "corner plots" showing 2-dimensional posteriors

import seaborn as sns; sns.set()  # nicer plots!
```

Visualization of PDFs

Quick introduction to `scipy.stats`

If you google “scipy.stats”, you’ll likely get the manual page as the first hit: [https://docs.scipy.org/doc/scipy/reference/stats.html]. Here you’ll find a long list of the continuous and discrete distributions that are available, followed (scroll way down) by many different methods (functions) to extract properties of a distribution (called “Summary Statistics”) and do many other statistical tasks.

Follow the link for any of the distributions (your choice!) to find its mathematical definition, some examples of how to use it, and a list of methods. Some methods of interest to us here:

- mean() - Mean of the distribution.
- median() - Median of the distribution.
- PDF(x) - Value of the probability density function at x.
- rvs(size=num_pts) - generate num_pts random values of the PDF.
- interval(alpha) - Endpoints of the range that contains alpha percent of the distribution.

Try out some of the examples (cut-and-paste from the manual page after turning off the >>>s by clicking in the upper right of a box). We'll use all of these methods in explicit examples below.

Matplotlib plotting helper functions

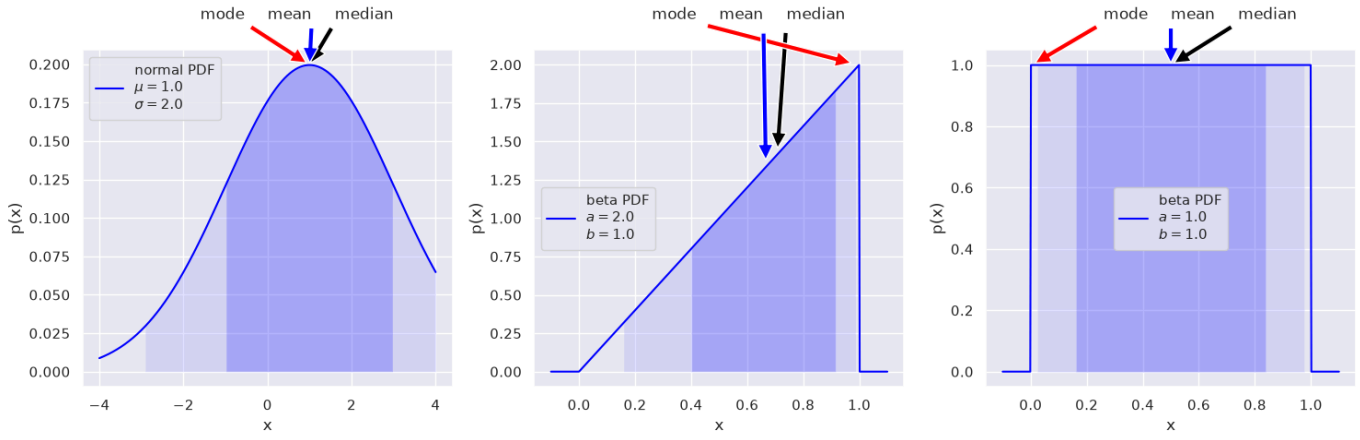
We first define a few functions that we'll use to extract and plot quantities of interest.

After you've looked at the examples that follow, come back and make sure you know what the functions are doing.

```
def dist_stuff(dist):
    """
    Finds the median, mean, and 68%/95% credible intervals for the given
    1-d distribution (which is an object from scipy.stats).
    """
    # For x = median, then mean: return x and the value of the PDF at x as a list
    median = [dist.median(), dist.pdf(dist.median())]
    mean = [dist.mean(), dist.pdf(dist.mean())]
    # The left and right limits of the credibility interval are returned
    cred68 = dist.interval(0.68)  # 68% interval
    cred95 = dist.interval(0.95)  # 95% interval
    return median, mean, cred68, cred95

def dist_mode(dist, x):
    """
    Return the mode (maximum) of the 1-d distribution for array x.
    """
    x_max_index = dist.pdf(x).argmax()
    # Return x of the maximum and the value of the PDF at that x
    mode = [x[x_max_index], dist.pdf(x[x_max_index])]
    return mode

def dist_plot(ax, dist_label, x_dist, dist, color='blue'):
    """
    Plot the distribution, indicating median, mean, mode
    and 68%/95% probability intervals on the axis that is passed (ax).
    """
    median, mean, cred68, cred95 = dist_stuff(dist)
    mode = dist_mode(dist, x_dist)

    ax.plot(x_dist, dist.pdf(x_dist), label=dist_label, color=color)
    ax.set_xlabel('x')
    ax.set_ylabel('p(x)')

    # Point to the median, mode, and mean with arrows (adjusting the spacing)
    text_x = 0.2*(x_dist[-1] - x_dist[0])
    text_x_mid = (x_dist[-1] + x_dist[0]) / 2.
    text_y = mode[1]*1.15
    ax.annotate('median', xy=median, xytext=(text_x_mid+text_x, text_y),
                arrowprops=dict(facecolor='black', shrink=0.05))
    ax.annotate('mode', xy=mode, xytext=(text_x_mid-text_x, text_y),
                arrowprops=dict(facecolor='red', shrink=0.05))
    ax.annotate('mean', xy=mean, xytext=(text_x_mid, text_y),
                arrowprops=dict(facecolor='blue', shrink=0.05))

    # Mark the credible intervals using shading (with appropriate alpha)
    ax.fill_between(x_dist, 0, dist.pdf(x_dist),
                    where=((x_dist > cred68[0]) & (x_dist < cred68[1])),
                    facecolor='blue', alpha=0.2)
    ax.fill_between(x_dist, 0, dist.pdf(x_dist),
                    where=((x_dist > cred95[0]) & (x_dist < cred95[1])),
                    facecolor='blue', alpha=0.1)

    ax.legend();
```

Some standard PDFs: the normal (aka Gaussian) and beta distributions

```
# Make some standard plots: normal, beta
fig = plt.figure(figsize=(15,5))

# Standard normal distribution -- try changing the mean and std. dev.
x_norm = np.linspace(-4, 4, 500)
mu = 1       # mean
sigma = 2.0  # standard deviation
norm_dist = stats.norm(mu, sigma) # the normal distribution from scipy.stats
norm_label='normal PDF' + '\n' + rf'$\mu=${mu:1.1f}' \
            + '\n' + rf'$\sigma=${sigma:1.1f}'
ax1 = fig.add_subplot(1,3,1)  # first of three subplots
dist_plot(ax1, norm_label, x_norm, norm_dist)

# beta distribution, characterized by a and b parameters
x_beta = np.linspace(-0.1, 1.1, 500)  # beta ranges from 0 to 1
a1 = 2
b1 = 1
beta_dist = stats.beta(a1, b1)  # the beta distribution from scipy.stats
beta1_label='beta PDF' + '\n' + rf'$a=${a1:1.1f}' \
             + '\n' + rf'$b=${b1:1.1f}'
ax2 = fig.add_subplot(1,3,2)  # second of three subplots
dist_plot(ax2, beta1_label, x_beta, beta_dist)

# another beta distribution
#x_beta = np.linspace(-0.1, 1.1, 500)
a2 = 1
b2 = 1
beta2_dist = stats.beta(a2, b2)
beta2_label='beta PDF' + '\n' + rf'$a=${a2:1.1f}' \
             + '\n' + rf'$b=${b2:1.1f}'
ax3 = fig.add_subplot(1,3,3)  # third of three subplots
dist_plot(ax3, beta2_label, x_beta, beta2_dist)

mu2 = beta2_dist.mean()
sigma2 = beta2_dist.std()
norm2_dist = stats.norm(mu2, sigma2)

fig.tight_layout()
```



The 68%/95% probability regions are shown in dark/light shading. When applied to Bayesian posteriors, these are known as *credible intervals* or DoBs (degree of belief intervals) or Bayesian confidence intervals. The horizontal extent on the x-axis translates into the vertical extent of the error bar or error band for x.

The values of the mode, mean, median can be used as *point estimates* for the "probable" value of x.
If you had a symmetric bimodal distribution, what point estimate would be best? Or are they all poor?

Things to try:

- Change the normal (Gaussian) distribution so that it has a different mean and standard deviation.
- Add another plot to one of the graphs. E.g., generate a normal distribution with the same mean and about the same width as the beta distribution with $a = 10, b = 10$ and add it to that plot. You don't need to call a special function, just use `norm2_dist = stats.norm(mu2, sigma2)` with your guesses for the μ and σ values and then `ax3.plot(x_beta, norm2_dist.pdf(x_beta), color='red')` to overlay the curve on `ax3`.
- Try some other distributions.

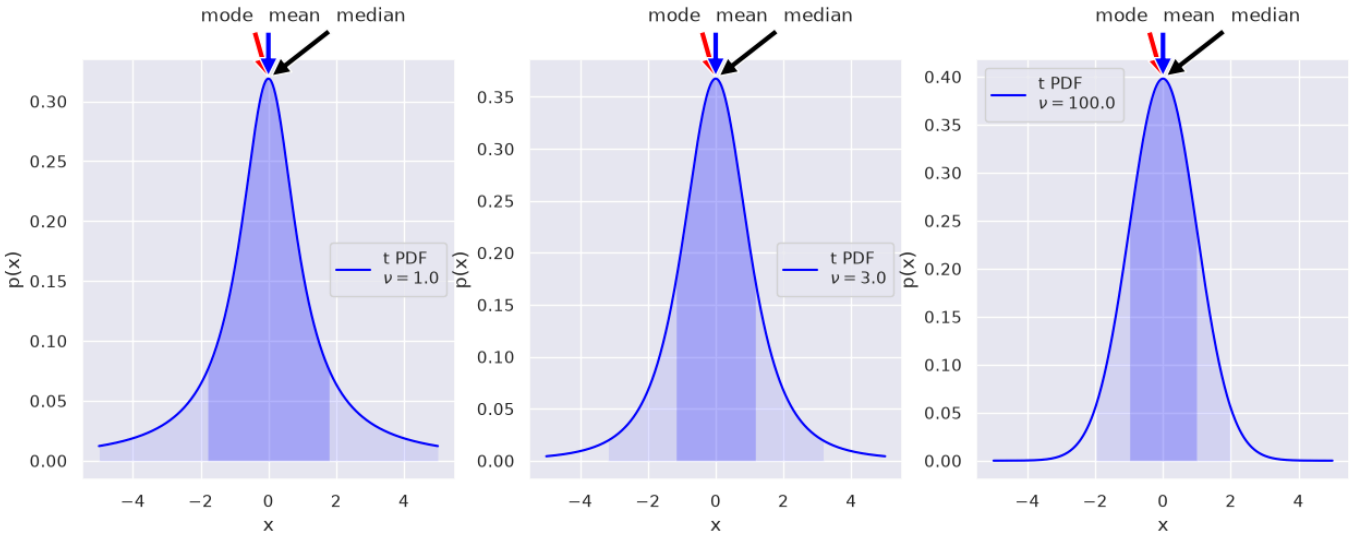
Another standard class of PDF: Student t

Find the functional definition for the t distribution on the `scipy.stats` web page. Is it consistent with the plots?

What are the `loc` and `scale` parameters?

```
# Make some plots of the Student t distribution
fig = plt.figure(figsize=(15,5))

x_t = np.linspace(-5, 5, 500)

norm1_dist = stats.norm(0.,1.)  # a normal distribution for comparison

nu1 = 1.01
t1_dist = stats.t(nu1) # the Student t distribution
t1_label='t PDF' + '\n' + rf'$\nu=${nu1:1.1f}'
ax1 = fig.add_subplot(1,3,1)
dist_plot(ax1, t1_label, x_t, t1_dist)
#ax1.plot(x_t, norm1_dist.pdf(x_t), color='red')

nu2 = 3
t2_dist = stats.t(nu2) # the Student t distribution
t2_label = 't PDF' + '\n' + rf'$\nu=${nu2:1.1f}'
ax2 = fig.add_subplot(1,3,2)
dist_plot(ax2, t2_label, x_t, t2_dist)
#ax2.plot(x_t, norm1_dist.pdf(x_t), color='red')

nu3 = 100
t3_dist = stats.t(nu3) # the Student t distribution
t3_label = 't PDF' + '\n' + rf'$\nu=${nu3:1.1f}'
ax3 = fig.add_subplot(1,3,3)
dist_plot(ax3, t3_label, x_t, t3_dist)
#ax3.plot(x_t, norm1_dist.pdf(x_t), color='red')

var2 = t2_dist.var()

print(f'variance for nu2: {var2}')
```

```
variance for nu2: 3.0
```



Note the “heavy tails” in the t distribution as ν gets small. That is, there is more probability in the tails than you would get from a normal distribution with the same half-height width. As ν gets large, the distribution approaches a standard normal (Gaussian) distribution.

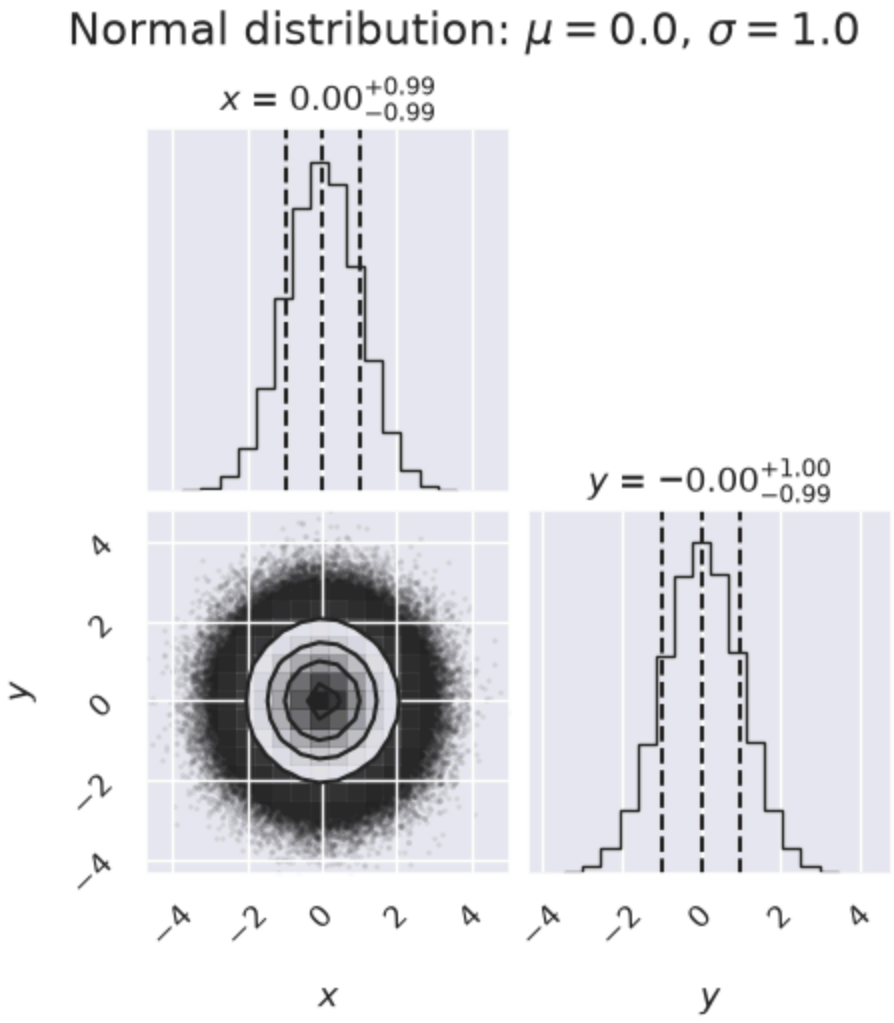
Try superposing a normal PDF on each. (The code is already there, just commented out, but make sure you understand what the statements are doing.)

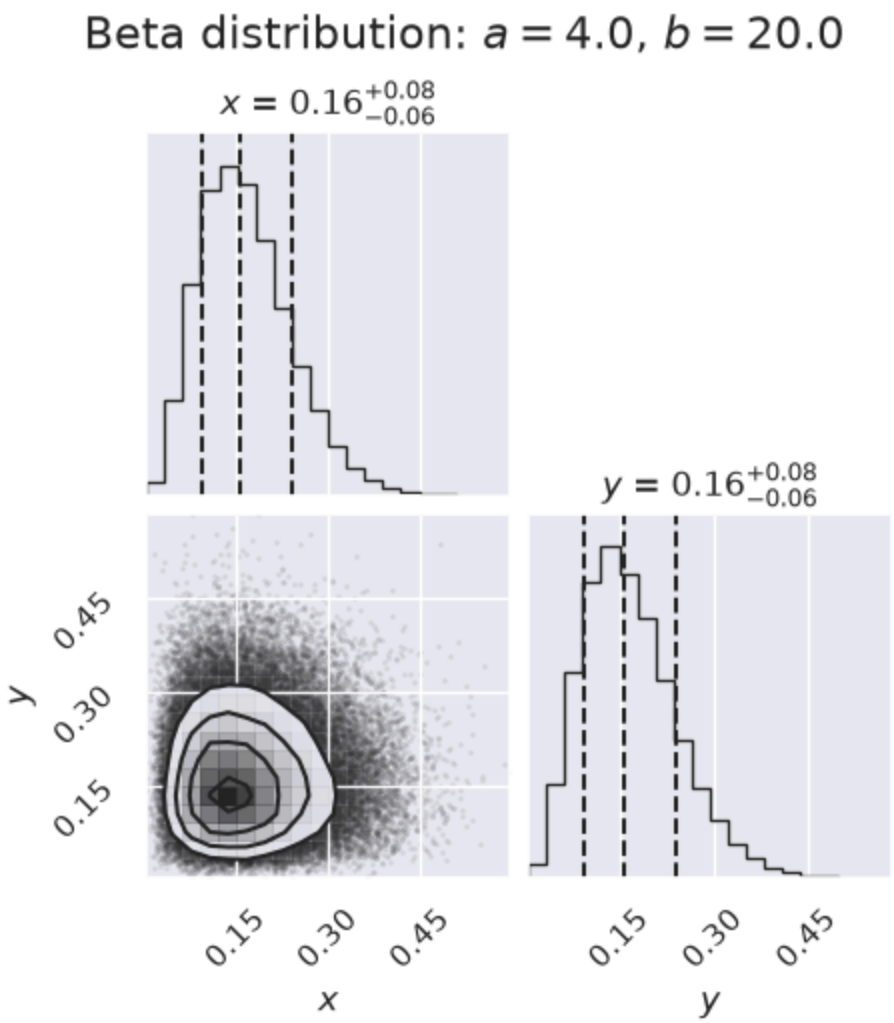
What is the variance of each of the t distributions (use the `var` *method)? Any surprises?*

Projected posterior plots

Here we use the corner package to make some projected posterior plots. (Note: there are other choices to make these plots but corner is really fast.)

```
# examples of corner plots
ndim, nsamples = 2, 1000000
#np.random.seed(42)
# generate some synthetic data from a normal distribution
mu, sigma = 0., 1.
norm_samples = stats.norm.rvs(size=ndim * nsamples, loc=mu, scale=sigma).reshape([nsamples,
ndim])

figure1 = corner.corner(norm_samples,
                        labels=[r"$x$", r"$y$", r"$\log \alpha$"],
                        quantiles=[0.16, 0.5, 0.84],
                        show_titles=True, title_kwargs={"fontsize": 12})
ax = figure1.get_axes()
my_suptitle = rf'Normal distribution: $\mu = {mu:.1f}$, ' + \
              rf'$\sigma = {sigma:.1f}$'
figure1.suptitle(my_suptitle, y=1.0, va='bottom', fontsize=16)
figure1.set_size_inches(5,5)


ndim, nsamples = 2, 100000
#np.random.seed(42)
# generate some synthetic data from a beta distribution
a = 4
b = 20
beta_samples = stats.beta(a,b).rvs(size=ndim * nsamples) \
                              .reshape([nsamples, ndim])

figure2 = corner.corner(beta_samples,
                        labels=[r"$x$", r"$y$", r"$\log \alpha$"],
                        quantiles=[0.16, 0.5, 0.84],
                        show_titles=True, title_kwargs={"fontsize": 12})
my_suptitle = rf'Beta distribution: $a = {a:.1f}$, $b = {b:.1f}$'
figure2.suptitle(my_suptitle, y=1.0, va='bottom', fontsize=16)
figure2.set_size_inches(5,5)
```


Normal distribution: $\mu = 0.0$, $\sigma = 1.0$
$x = 0.00^{+0.99}_{-0.99}$
$y = -0.00^{+1.00}_{-0.99}$
y
x
y
−4
−2
0
2
4

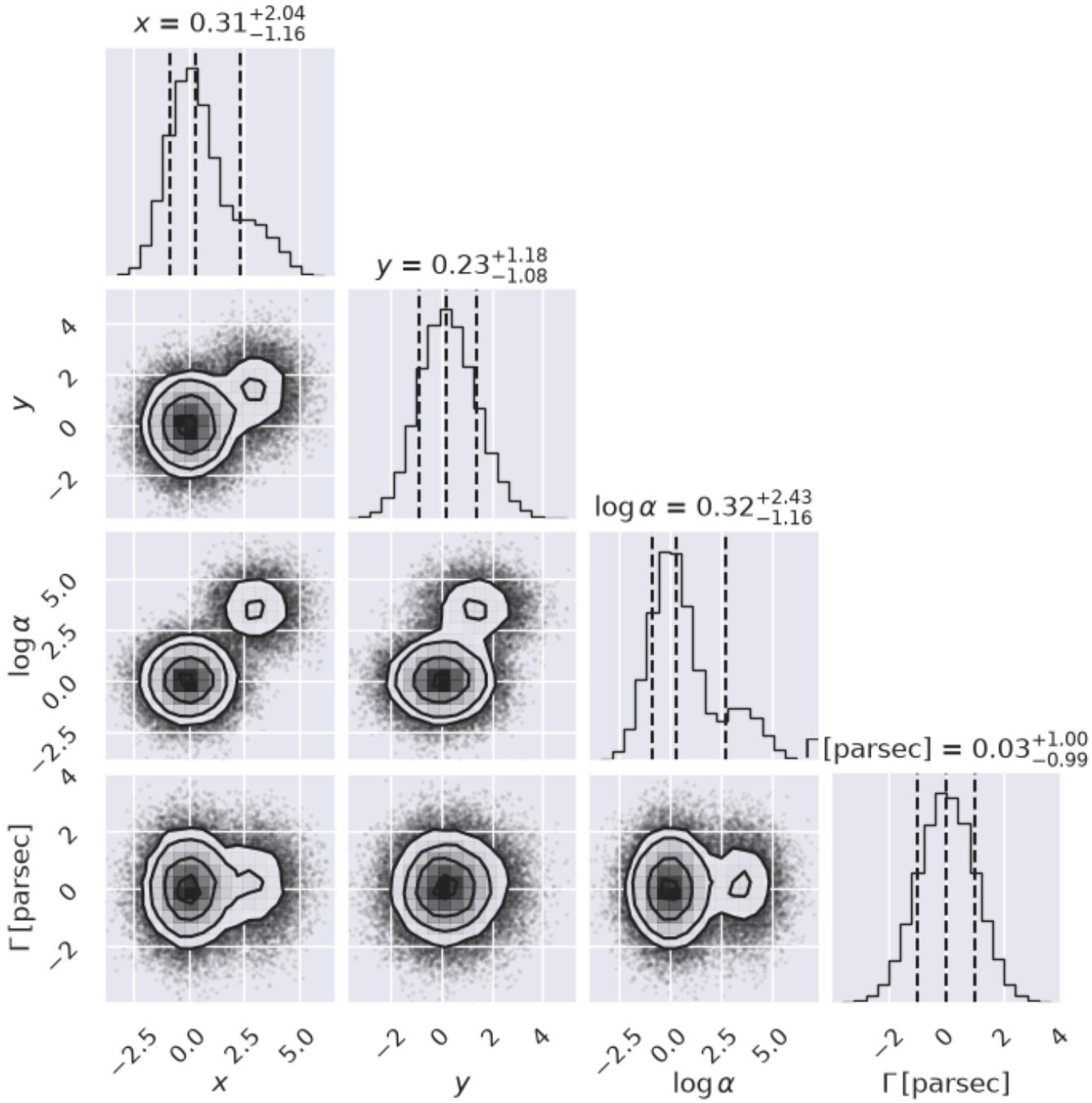


What do you learn from these plots?

How are the one-dimensional plots above and to the right related to the two-dimensional plot?

Try replotting several times with only 1000 samples each and note how much the plots change.

```
# now more than one mode (all random)
ndim, nsamples = 4, 50000
np.random.seed(1234)
data1 = np.random.randn(ndim * 4 * nsamples // 5) \
          .reshape([4 * nsamples // 5, ndim])
mean = 4*np.random.rand(ndim)
data2 = (mean[None, :] + np.random.randn(ndim * nsamples // 5) \
          .reshape([nsamples // 5, ndim]))
samples = np.vstack([data1, data2])

#figure = corner.corner(samples)
figure = corner.corner(samples, labels=[r"$x$", r"$y$", r"$\log \alpha$", \
                                        r"$\Gamma \, [\mathrm{parsec}]$"],
                       quantiles=[0.16, 0.5, 0.84],
                       show_titles=True, title_kwargs={"fontsize": 12})

figure.set_size_inches(7,7)
```



What do you learn from these plots?

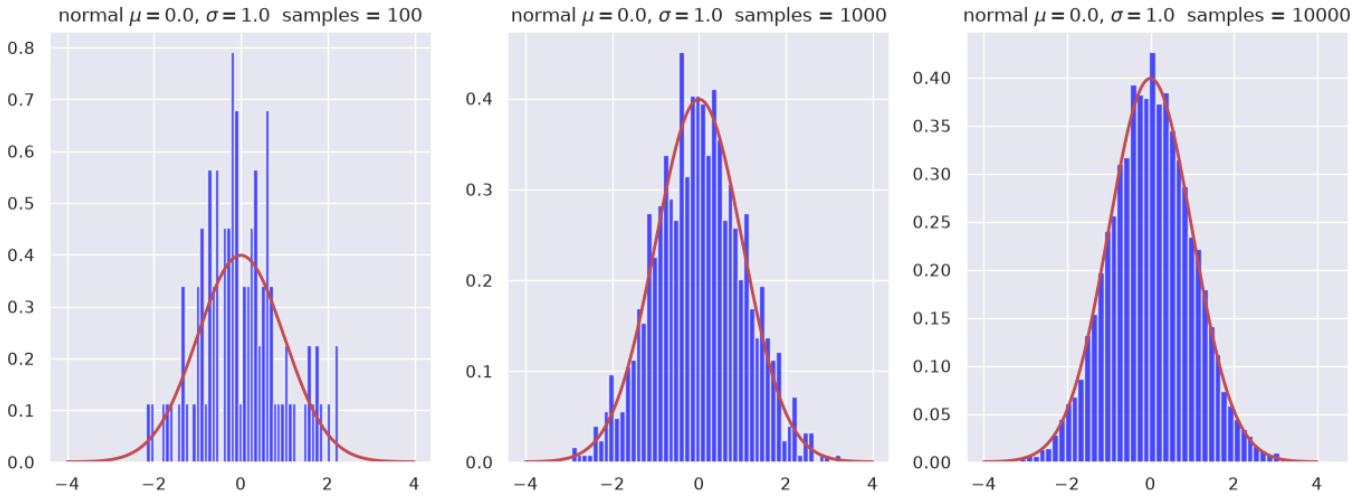
Can you always tell from the one-dimensional plot whether the two-dimensional plot has more than one (local) maximum?

Sampling of 1d PDFs in Python

Here we show how histogrammed samples become closer to the continuous PDF as the sample size increases.

We use the built-in capability of the scipy.stats library to generate random samples from a specified distribution (e.g., a normal distribution).

```python
def plot_hist(ax, name, x_dist, dist, num_samples, num_bins):
    """Plot a PF and a histogram of samples"""
    samples = dist.rvs(size=num_samples)  # draw random samples from the PDF
    # Make a histogram of the random samples
    count, bins, ignored = ax.hist(samples, num_bins, density=True,
                                   color='blue', alpha=0.7)
    ax.plot(x_dist, dist.pdf(x_dist), linewidth=2, color='r') # true PDF
    title_string = name + f'  samples = {num_samples:d}'
    ax.set_title(title_string)


mu, sigma = 0, 1.0 # mean and standard deviation
x_dist = np.linspace(-4, 4, 500)
name = rf'normal $\mu=${mu:1.1f}, $\sigma=${sigma:1.1f}'
fig = plt.figure(figsize=(15,5))
num_bins = 50

num_samples = 100
norm_dist = stats.norm(mu, sigma)
ax_1 = fig.add_subplot(1, 3, 1)
plot_hist(ax_1, name, x_dist, norm_dist, num_samples, num_bins)

num_samples = 1000
norm_dist = stats.norm(mu, sigma)
ax_2 = fig.add_subplot(1, 3, 2)
plot_hist(ax_2, name, x_dist, norm_dist, num_samples, num_bins)

num_samples = 10000
norm_dist = stats.norm(mu, sigma)
ax_3 = fig.add_subplot(1, 3, 3)
plot_hist(ax_3, name, x_dist, norm_dist, num_samples, num_bins)
```



To do: *Find the mean of the random samples and compare to $\mu = 0$ for each sample size.*

- You can simply add a statement to the `plot_hist` function to print the mean, using that `samples.mean()` gives the mean of the distribution `samples`.
- *Conclusion?*

9.2. 📥 Problem: Gaussian noise and averages I

Adapted from Sivia: *Data Analysis: A Bayesian Tutorial*

Here we'll take a look at a simple parameter-estimation problem. We will discuss a Bayesian approach to this problem and show how it reduces to standard frequentist estimators for a particular choice of prior. This problem is an extended version of Example 2 in Ch 2.3 of the book by Sivia. (This notebook was adapted from a broad introduction to Bayesian statistics put together by Christian Forssen for the 2019 TALENT school on "Learning from Data".)

Answer all of the question in italics.

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

from scipy import stats

# Not really needed, but nicer plots
import seaborn as sns
sns.set()
sns.set_context("talk")
```

Let us consider the problem of estimating the mean and the variance of a normal distribution that is associated with a collection of random variables. The normal distribution

$$p(x|\mu,\sigma) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right),$$

is often used as a theoretical model to describe the noise associated with experimental data.

Why is a normal (Gaussian) distribution so often a good statistical model?

Let us assume that we have a series of M measurements $D \equiv \{x_k\} = (x_1, \ldots, x_M)$, that are samples from a normal $\mathcal{N}(\mu, \sigma^2)$ population, from which we would like to learn the approximate values of the parameters μ and σ. The standard frequentist approach to this problem is the maximum likelihood method. The Bayesian approach is to compute the posterior distribution for the model parameters μ and σ.

Here we'll use Python to generate some toy data to demonstrate the two approaches to the problem.

```
# Generating some data
np.random.seed(1)  # for repeatability we use an integer > 0. Use 0 for a random seed.

mu_true = 10   # true peak position
sigma_true = 1 # true standard deviation
M = 1000 # number of measurements
D = stats.norm.rvs(mu_true, sigma_true, size=M)  # M measurements (samples)
```

Side note: try shift-tab-tab to get documentation!

Look at the array `D`*. Are the number of entries in the "tails" what you would expect?*

```
D
```

```
array([11.62434536,  9.38824359,  9.47182825,  8.92703138, 10.86540763,
        7.6984613 , 11.74481176,  9.2387931 , 10.3190391 ,  9.75062962,
       11.46210794,  7.93985929,  9.6775828 ,  9.61594565, 11.13376944,
        8.90010873,  9.82757179,  9.12214158, 10.04221375, 10.58281521,
        8.89938082, 11.14472371, 10.90159072, 10.50249434, 10.90085595,
        9.31627214,  9.87710977,  9.06423057,  9.73211192, 10.53035547,
        9.30833925,  9.60324647,  9.3128273 ,  9.15479436,  9.32875387,
        9.9873354 ,  8.88268965, 10.2344157 , 11.65980218, 10.74204416,
        9.80816445,  9.11237104,  9.25284171, 11.6924546 , 10.05080775,
        9.36300435, 10.19091548, 12.10025514, 10.12015895, 10.61720311,
       10.30017032,  9.64775015,  8.8574818 ,  9.65065728,  9.79110577,
       10.58662319, 10.83898341, 10.93110208, 10.28558733, 10.88514116,
        9.24560206, 11.25286816, 10.51292982,  9.70190716, 10.48851815,
        9.92442829, 11.13162939, 11.51981682, 12.18557541,  8.60350366,
        8.55588619,  9.49553414, 10.16003707, 10.87616892, 10.31563495,
        7.97779878,  9.69379599, 10.82797464, 10.23009474, 10.76201118,
        9.77767186,  9.79924193, 10.18656139, 10.41005165, 10.19829972,
       10.11900865,  9.32933771, 10.37756379, 10.12182127, 11.12948391,
       11.19891788, 10.18515642,  9.62471505,  9.36126959, 10.42349435,
       10.07734007,  9.65614632, 10.04359686,  9.37999916, 10.69803203,
        9.55287144, 11.2245077 , 10.40349164, 10.59357852,  8.90508815,
       10.16938243, 10.74055645,  9.0462994 ,  9.73378149, 10.03261455,
        8.62688268, 10.31515939, 10.84616065,  9.14048406, 10.35054598,
        8.68771659,  9.96130449,  8.38422765, 11.12141771, 10.40890054,
        9.97538304,  9.22483838, 11.27375593, 11.96710175,  8.14201814,
       11.23616403, 11.62765075, 10.3380117 ,  8.80073197, 10.86334532,
        9.8190797 ,  9.39607937,  8.76994186, 10.5505375 , 10.79280687,
        9.37646927, 10.52057634,  8.85565861, 10.80186103, 10.0465673 ,
        9.81343023,  9.89825413, 10.86888616, 10.75041164, 10.52946532,
       10.13770121, 10.07782113, 10.61838026, 10.23249456, 10.68255141,
        9.68988323,  7.56516224, 11.0388246 , 12.18697965, 10.44136444,
        9.89984477,  9.86355526,  9.88094581, 10.01740941,  8.87798127,
        9.48290554,  9.00297317, 10.24879916,  9.70335885, 10.49521132,
        9.82529684, 10.98633519, 10.2135339 , 12.19069973,  8.10363908,
        9.35308331, 10.90148689, 12.52832571,  9.75136522, 10.04366899,
        9.77368576, 11.33145711,  9.71269214, 10.68006984,  9.6801984 ,
        8.72744124, 10.31354772, 10.50318481, 11.29322588,  9.88955297,
        9.38263794, 10.5627611 , 10.24073709, 10.28066508,  9.9268873 ,
       11.16033857, 10.36949272, 11.90465871, 11.1110567 , 10.6590498 ,
        8.37256166, 10.60231928, 10.4202822 , 10.81095167, 11.04444209,
        9.59912181, 10.82400562,  9.43769457, 11.95487808,  8.66804833,
        8.23931144,  8.34927873,  9.10944442,  8.8808846 , 11.9560789 ,
        9.6735005 ,  8.65732421, 11.11438298,  9.41347606,  8.76314662,
       10.87583893, 10.62336218,  9.56504332, 11.40754   , 10.12910158,
       11.6169496 , 10.50274088, 11.55880554, 10.1094027 ,  8.7802556 ,
       12.44936865,  9.45422583,  9.80116214,  9.2996015 ,  9.79660555,
       10.24266944, 10.20183018, 10.66102029, 11.79215821,  9.87953543,
        8.76687926,  8.81768187,  9.33424548,  8.32580419, 10.82502982,
        9.50178644,  9.68901502,  9.99810852,  8.60337958,  9.13868364,
       10.67471153, 10.61853913,  9.55682807, 11.81053491,  8.69427308,
        9.65501279,  9.76916026,  7.206915  , 11.93752881, 10.36633201,
        8.95541062, 12.05117344, 10.585662  , 10.42952614,  9.3930016 ,
       10.10622272,  8.47431968, 10.79502609,  9.62556168, 10.1340482 ,
       11.20205486, 10.28474811, 10.26246745, 10.2764993 ,  9.2667284 ,
       10.83600472, 11.54335911, 10.75880566, 10.88490881,  9.12271848,
        9.13221278,  8.55912398, 11.23225307,  9.74582013, 11.39984394,
        9.21808832,  9.56249102, 10.09542509, 10.92145007, 10.0607502 ,
       10.21112476, 10.01652757, 10.17718772,  8.88352998, 10.0809271 ,
        9.81342101,  9.94317552, 10.49233656,  9.31932186,  9.91549197,
        9.70263812, 10.417302  , 10.78477065,  9.04457474, 10.58591043,
```

```
12.06578332,  8.52884307,  9.1698281 ,  9.1194224 ,  9.72090228,
11.62284909, 10.01335268,  9.3053064 , 10.6218035 ,  9.40019547,
11.12341216, 10.30526704, 11.3887794 ,  9.33865576, 13.03085711,
10.82458463, 10.65458015,  9.94881155,  9.27440288,  9.13223132,
 9.86402267,  9.20273021, 10.28267571,  9.17390257, 10.6210827 ,
10.9561217 ,  9.29415949, 11.19268607,  9.76205806, 11.15528789,
10.43816635, 11.12232832,  9.0029802 ,  9.89320601, 11.45142926,
 9.38196315,  7.96279877,  8.05741082,  7.49355935,  7.88583608,
 9.58836084, 11.27852808,  9.55777072, 10.32352735,  9.89000851,
10.00854895,  9.83180116,  9.82581966, 10.4611641 ,  8.82401733,
11.01012718, 10.92001793,  9.80494266, 10.80539342,  9.29865557,
 9.46277698, 10.15626385,  9.80977897,  9.55126197,  9.32755196,
 9.44250528, 10.93916874,  8.05667659, 10.35249436,  9.76356305,
10.7278135 , 10.51507361,  7.21746553, 10.58464661, 10.32427424,
10.02186284,  9.53132618, 10.85328122,  9.58697069, 11.83471763,
10.56438286, 12.13782807,  9.214466  ,  8.24407436, 10.7147896 ,
10.85270406, 10.0353601 ,  8.46120675,  9.55210482, 10.61798553,
 9.81582367,  9.88401481,  9.82454103,  9.06608534,  9.46697967,
 8.57344458, 11.76795995,  9.52462712, 10.47761018,  8.97811406,
10.79452824,  8.12683902, 10.92061512,  9.96463208, 12.11060505,
 8.69346593, 10.07638048, 10.36723181, 11.23289919,  9.57714304,
10.08646441,  7.85753327,  9.16983114, 10.45161595, 11.10417433,
 9.71826373, 12.05635552, 11.76024923,  9.93934751,  7.586497  ,
 8.22243362,  9.22214117, 11.11584111, 10.31027229,  7.90575218,
 9.77123417, 11.61336137,  9.62519531,  9.25003038, 12.0546241 ,
10.05340954,  9.5208429 , 10.35016716, 10.01716473,  9.57085772,
11.20845633, 11.1157018 , 10.84086156,  9.89711278, 11.14690038,
 9.95029742, 10.46664327, 11.03368687, 10.80884436, 11.78975468,
10.45128402,  8.31594001,  8.8398299 , 11.35010682,  9.66871683,
10.38653915,  9.14854434, 11.00088142,  9.61516775, 11.45810824,
 9.46776598, 11.1181334 , 10.6743961 ,  9.27760809, 11.09899633,
 9.09836551,  9.17753281, 10.72171129,  9.374658  ,  9.40615693,
 9.65609929,  8.99983081, 11.04499441, 10.6085147 ,  9.9306713 ,
 9.89160793, 10.45015551, 11.7653351 , 10.8709698 ,  9.49154287,
10.77741921,  9.88122883,  9.80100182, 11.86647138,  9.5810621 ,
 9.52081508,  8.04789471,  8.59767085, 10.45112294,  9.3050791 ,
10.5154138 ,  8.88512895,  9.23269017, 10.67457071, 11.46089238,
10.5924728 , 11.19783084, 11.70459417, 11.04008915,  9.08155996,
 9.89465529, 10.63019567,  9.5851531 , 10.45194604,  8.42084371,
 9.17137202, 10.52887975,  7.76291349,  8.8922875 ,  9.98228168,
 8.28060553, 10.057121  ,  9.20045251,  9.7084054 ,  9.74101715,
10.1892932 ,  9.43621127, 10.08968641,  9.3988432 , 10.55607351,
11.69380911, 10.19686978, 10.16986926,  8.83599203, 10.69336623,
 9.24193267,  9.1911528 , 10.55743945, 10.18103874, 11.10717545,
11.44287693,  9.46031844, 10.12837699, 11.76041518, 10.96653925,
10.71304905, 11.30620607,  9.39539703, 10.63658341, 11.40925339,
11.62091229,  9.19381518,  9.74832579, 10.38271517,  9.71100266,
 9.60818376, 10.68400133,  9.64659002,  8.21208711, 10.36184732,
 9.57550721,  9.26846902,  8.43426185, 11.01382247,  7.77288737,
 8.3006664 ,  9.72415394, 11.22895559, 11.30970591,  8.84501737,
 9.8223678 ,  8.48954362, 11.01120706,  8.52343734,  9.85680425,
11.03298378,  9.77758597, 11.47016034,  9.12999178, 10.36919047,
10.85328219,  9.86028827, 11.38631426, 10.54812958,  8.36255041,
13.9586027 , 10.64864364, 10.10734329,  8.60118718, 10.08176782,
 9.54005717, 10.64435367, 10.37167029, 11.85300949, 10.14225137,
10.51350548, 10.37245685,  9.8515102 ,  9.8165998 , 11.1010002 ,
10.78002714,  9.3705584 ,  8.8865639 ,  9.93258998, 11.16143998,
 9.97247061, 11.74643509,  9.22492971, 10.14164054,  7.48369614,
 9.40433212,  9.69087868, 10.51093777, 11.71066184, 10.03494359,
11.45391758, 10.66168108, 10.98635218,  9.53384514, 11.38499134,
 8.92703572, 10.49515861,  9.0479379 ,  9.48185445,  8.5385964 ,
 9.48365209, 10.3511169 ,  9.93122954,  8.65223506, 11.47073986,
10.33722094, 11.00806543, 10.78522692,  9.33513223,  8.05495304,
 9.08457563, 11.22515585,  8.94645393, 10.81604368,  9.38759303,
10.39310924,  8.17608015, 11.16707517,  9.9603313 , 10.8858258 ,
```

10.18986165, 10.7980638 , 9.89806796, 10.74335654, 8.49042732,
 8.91928931, 10.725474 , 9.96082174, 9.77124583, 9.82038771,
10.50172511, 9.40665625, 10.5103076 , 9.08420815, 9.59274796,
10.98495167, 11.07125243, 8.90284564, 10.83863475, 8.96081768,
10.73302323, 8.10118794, 8.88288931, 9.49102772, 9.83351405,
11.42361443, 10.90399917, 11.57546791, 11.2066079 , 9.71713645,
 9.73367312, 11.06897162, 10.04037143, 9.84300633, 8.66479728,
 9.89353988, 7.20900359, 9.54388244, 9.02010975, 10.69257435,
 9.52132764, 9.67094845, 11.34710546, 8.95093225, 10.3166589 ,
 8.10473305, 10.08972912, 10.41026575, 10.85987097, 9.10131681,
10.31965694, 10.3181542 , 9.98076837, 10.15001628, 10.46353432,
10.39788042, 9.00398911, 8.80413849, 12.50598029, 11.91979229,
 8.60830612, 10.45021774, 10.62743708, 10.75133724, 10.14039544,
 9.07312806, 9.81757959, 9.50887486, 10.13437312, 9.7316287 ,
 9.86832437, 11.01855247, 11.2305582 , 8.81889683, 9.5400699 ,
 9.20920005, 11.22372221, 9.9406321 , 11.4489894 , 9.52241915,
10.02599999, 8.65130355, 11.30253554, 9.63738791, 8.51484355,
 9.40753871, 7.69509206, 9.96818283, 10.11248774, 10.28807817,
11.49810818, 9.69902385, 10.80745592, 10.31223869, 9.80667836,
 7.92319798, 10.94750117, 9.49602605, 10.01795589, 8.72953922,
10.28299553, 10.10803082, 10.02941762, 9.86520687, 11.04921829,
10.96622086, 10.72591685, 13.32107876, 9.39977467, 9.62048248,
 8.98519631, 10.4359862 , 9.31251261, 7.30163826, 8.78666187,
10.0722519 , 11.00978733, 8.44305844, 9.38755787, 9.86064819,
 9.27146251, 10.53116379, 10.00400084, 10.32126591, 9.27478507,
11.53653633, 9.99962499, 11.29354962, 9.56100234, 10.59003946,
 9.32061622, 9.04909075, 9.29564967, 9.95413331, 9.78126654,
11.53920701, 8.85129577, 8.90966167, 11.70018815, 10.60878366,
 8.11858913, 10.4972691 , 10.2373327 , 7.85555595, 9.63043757,
 9.98254505, 10.73140252, 10.95449567, 10.09574677, 11.0334508 ,
 9.85372673, 9.14250318, 9.06581816, 10.54264529, 8.04183091,
10.67780757, 8.89342693, 9.6407759 , 10.5053819 , 11.2179409 ,
 8.05931904, 9.19382179, 10.04906169, 9.40391367, 10.8616231 ,
 7.91360943, 10.36180164, 10.42592018, 10.0490804 , 11.10223673,
 8.77042575, 11.10861676, 9.2970796 , 10.72555052, 9.67579578,
10.81434313, 10.78046993, 8.53594643, 9.84550881, 9.90756768,
 9.76212473, 9.24433723, 11.85143789, 10.20909668, 11.55501599,
 9.43085135, 8.93820324, 10.13224778, 9.4367634 , 12.39014596,
10.24542285, 11.15259914, 9.77576423, 9.67393869, 9.96908858,
10.35571726, 10.84958685, 9.87784598, 9.31914843, 8.93212342,
 9.92332064, 10.57296273, 10.45794708, 9.98218245, 9.3998612 ,
10.14676526, 10.57180488, 9.96318234, 10.11236849, 9.84949567,
10.91549927, 9.56179973, 10.18553562, 10.39442803, 10.72552256,
11.49588477, 10.67545381, 10.59921324, 8.52976291, 10.60640394,
12.29371761, 9.16998901, 8.98048015, 9.78534616, 11.02124813,
10.52475049, 9.52287579, 9.96400982, 11.03703898, 10.67261975,
12.42887697, 11.00568668, 10.35356722, 10.61472628, 9.65101581,
 9.022227 , 10.17195713, 10.49056104, 8.60471697, 9.47764353,
 9.6307441 , 10.2656424 , 9.73953394, 10.44509671, 10.09811225,
11.06032751, 8.28883234, 11.65712464, 11.41767401, 10.05031709,
10.65032321, 10.6065484 , 9.26271037, 10.16466507, 10.77817418,
10.30981676, 11.05132077, 10.09499611, 10.08075099, 9.23219625,
 9.63546195, 9.54028232, 11.70548352, 10.24050555, 9.0005735 ,
10.39859839, 9.8079963 , 6.94623562, 10.47985237, 8.44730122,
10.57846442, 9.0387364 , 8.54167554, 10.49434165, 8.50580623,
 9.5533008 , 10.20437739, 10.61223252, 10.74488454, 9.96371871,
 9.16760465, 11.92381543, 9.39401868, 11.80358898, 9.54747503,
11.16128569, 11.06996554, 8.95446575, 10.35528451, 10.75539203,
10.70098212, 9.80106255, 10.30196005, 9.60531032, 8.82818662,
10.98401224, 9.44031858, 11.37975819, 10.6024509 , 9.10735333,
 9.83880168, 9.71361508, 9.12911235, 10.50142959, 9.52138593,
11.63169151, 10.86089124, 9.11981094, 9.98099948, 9.77323981,
 8.43549215, 10.93125568, 10.94980882, 10.92550121, 9.54301214,
11.06898597, 9.79024706, 10.93514778, 11.81252782, 10.14010988,
 8.58085122, 9.6830988 , 10.64098587, 11.21987438, 8.86620796,

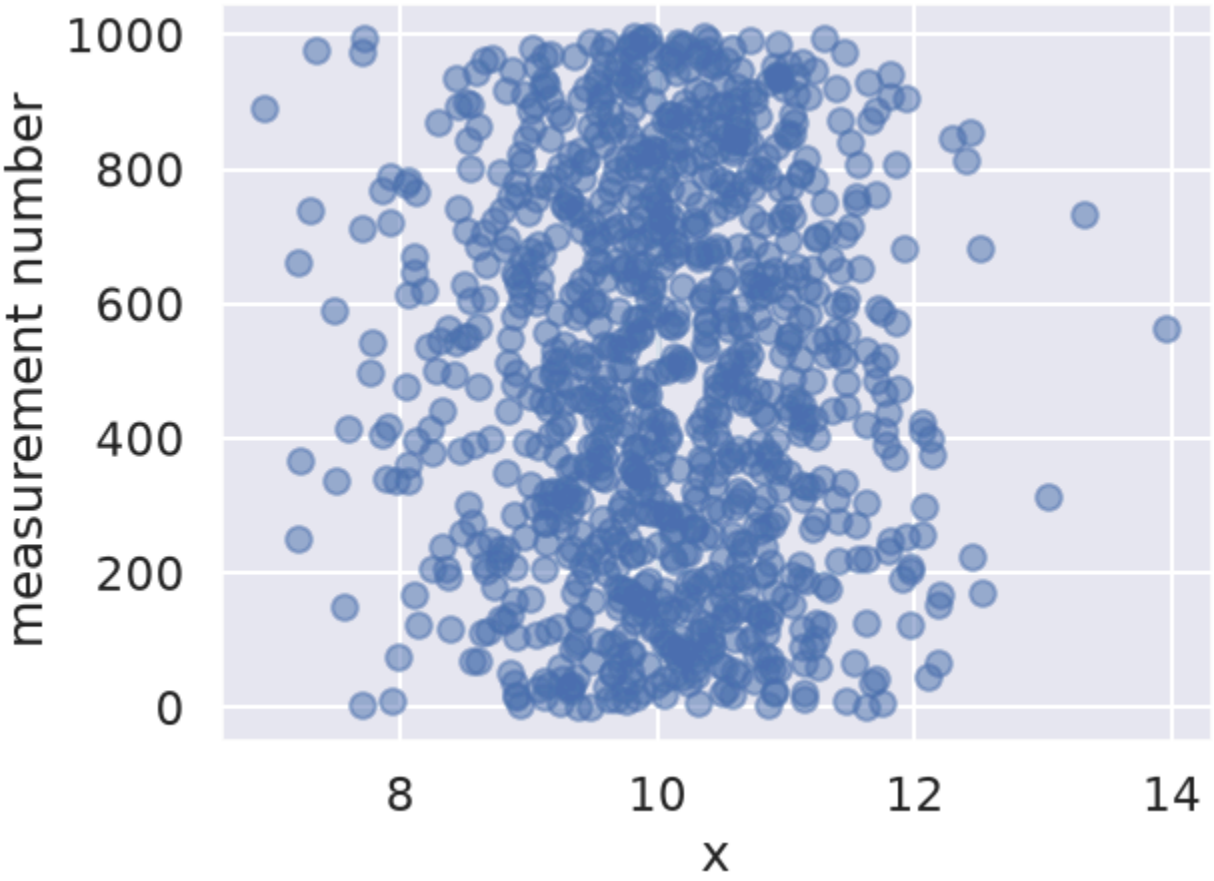
```
 9.8094517 , 10.23333913, 10.43499832, 10.9104236 ,  9.05156034,
 9.5765217 , 11.00796648, 10.39233491, 10.44838065, 11.1253235 ,
10.10405339, 10.52800342,  9.68543614,  8.65498998,  8.70474211,
10.07432055,  9.80043928,  9.34539683, 10.3180143 ,  9.10972845,
10.11133727,  9.98047744,  9.16001109,  7.70179412, 11.45652739,
10.31663724,  7.33587406,  9.57357138, 10.39378773,  9.77185931,
10.58033011,  9.02673241, 10.17516773,  9.94651631,  9.81693801,
 9.7789711 , 10.19975956, 10.93272141,  9.4698802 ,  9.59275998,
10.16056499,  9.87985002, 10.38560229, 10.71829074, 11.2911889 ,
 9.88355585,  7.722702  ,  9.93037546, 10.35387043,  9.81304498])
```

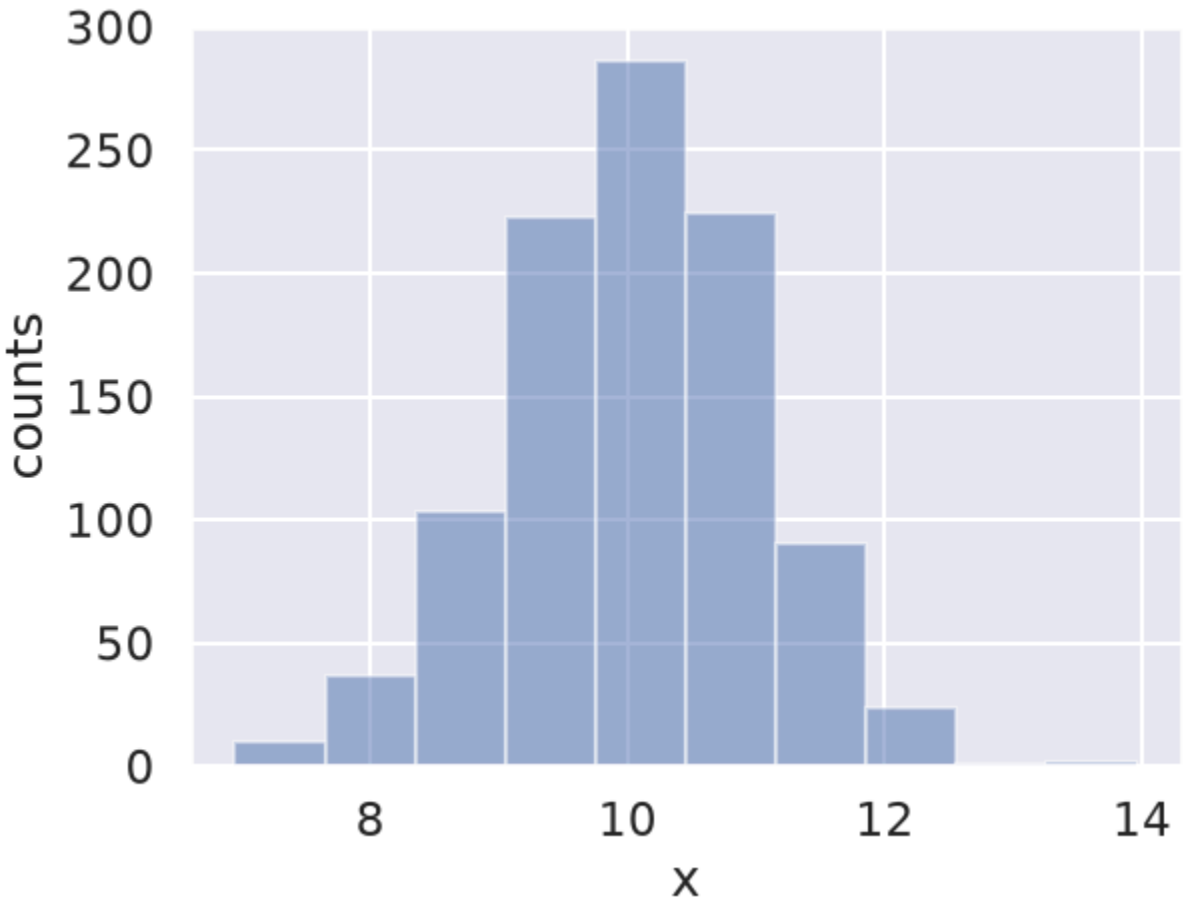
Hint: Roughly how many entries do you expect to find two sigma away from the mean?

Let's verify your answer with visualizations!

First, let's make two simple visualizations of the "measured" data: a scatter plot and a histogram

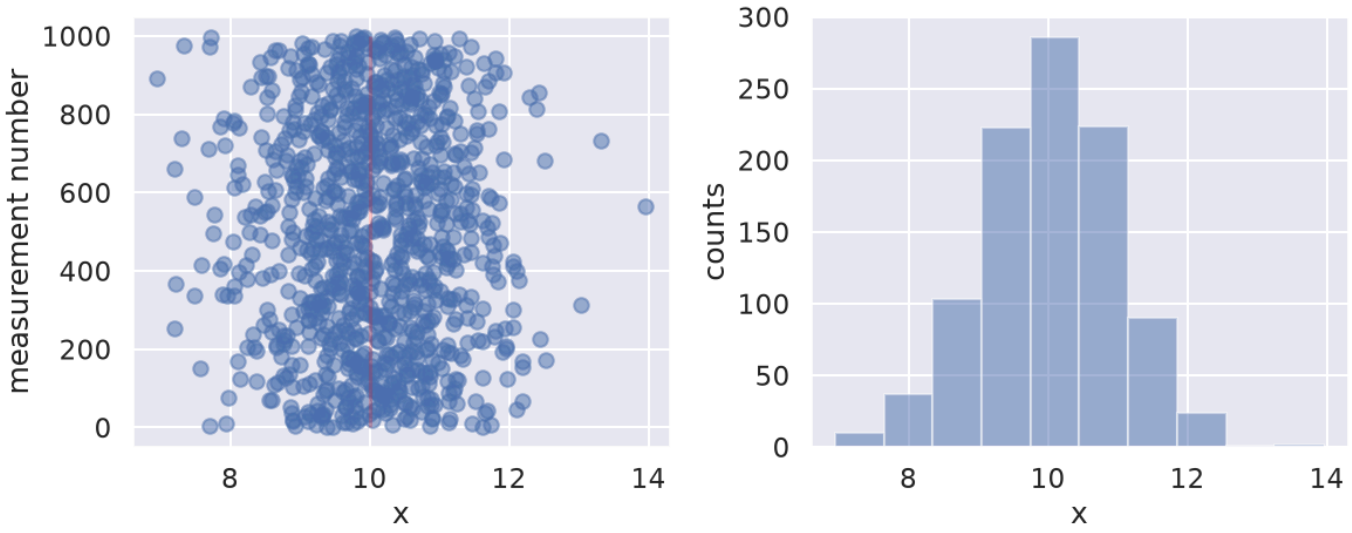
```
fig1, ax1 = plt.subplots()
ax1.scatter(D, np.arange(M), alpha=0.5)
ax1.vlines([mu_true], 0, M, alpha=0.2)
ax1.set_xlabel("x"); ax1.set_ylabel("measurement number");

fig2, ax2 = plt.subplots()
ax2.hist(D,alpha   =0.5)
#ax1.vlines([mu_true], 0, M, alpha=0.2)
ax2.set_xlabel("x"); ax2.set_ylabel("counts");
```


1000
800
600
400
200
0
measurement number
8
10
12
14
x



`Matplotlib` alternative: one figure with two subplots using an array of axes.

```
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12,5))
# An alternative keeping the same axis names as before would be:
#  fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(12,5))

axes[0].scatter(D, np.arange(M), alpha=0.5)
axes[0].vlines([mu_true], 0, M, color='red', alpha=0.2)
axes[0].set_xlabel("x"); axes[0].set_ylabel("measurement number");

axes[1].hist(D,alpha=0.5)
axes[1].set_xlabel("x"); axes[1].set_ylabel("counts");

fig.tight_layout()
```



What can you conclude about the tails?

Change np.random.seed(1) to a different number so that a different set of random numbers is generated. What can you say about how successive plots are similar and different?

Bayesian approach to Gaussian parameter estimation

The Bayesian approach begins and ends with probabilities (pdfs). It recognizes that what we fundamentally want to compute is our knowledge of the parameters in question, i.e. in this case,

$$p(\mu, \sigma \mid D, I)$$

Note that this formulation of the problem is *fundamentally contrary* to the frequentist philosophy, which says that *probabilities have no meaning for model parameters* like μ, σ. Nevertheless, within the Bayesian philosophy this is perfectly acceptable.

To compute this result, Bayesians next apply Bayes' Theorem, here with the hypothesis being the Gaussian model expressed in terms of the model parameters

$$p(\mu, \sigma \mid D, I) = \frac{p(D \mid \mu, \sigma, I)\, p(\mu, \sigma \mid I)}{p(D \mid I)}$$

Though Bayes' theorem is where Bayesians get their name, it is not this law itself that is controversial, but the Bayesian *interpretation of probability* implied by the term $p(\mu, \sigma \mid D, I)$.

Let's take a look at each of the terms in this expression:

- $p(\mu, \sigma \mid D, I)$: The **posterior**, or the probability of the model parameters given the data: this is the result we want to compute.
- $p(D \mid \mu, \sigma, I)$: The **likelihood**, which is proportional to $\mathcal{L}(D \mid \mu, \sigma)$ in the frequentist approach (see below).
- $p(\mu, \sigma \mid I)$: The **model prior**, which encodes what we knew about the model prior to the application of the data D.

- $p(D \mid I)$: The **data probability** (or **evidence**), which in practice amounts to simply a normalization term.

If we set the prior $p(\mu, \sigma \mid I) \propto 1$ (a *flat prior*), we find

$$p(\mu, \sigma \mid D, I) \propto \mathcal{L}(D|\mu, \sigma)$$

and the Bayesian probability is maximized at precisely the same value as the frequentist result! So despite the philosophical differences, we see that (for this simple problem at least) the Bayesian and frequentist point estimates are equivalent.

We'll therefore start with the classical frequentist **maximum likelihood** approach. The probability of a single measurement D_i having a value x_i is given by

$$p(x_i \mid \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[\frac{-(x_i - \mu)^2}{2\sigma^2}\right],$$

where μ, σ are the *true* values for the model parameters.

We construct the **likelihood function** by computing the product of the probabilities for each data point:

$$\mathcal{L}(D \mid \mu, \sigma) = \prod_{i=1}^{M} p(x_i \mid \mu, \sigma)$$

Because the value of the likelihood can become very small, it is often more convenient to instead compute the log-likelihood. Furthermore, when we are looking for the maximum of the likelihood, we might just as well maximize the log-likelihood. Combining the previous two equations and computing the log, we have

$$\log \mathcal{L} = -\frac{1}{2} \sum_{i=1}^{M} \left[\log(2\pi\sigma^2) + \frac{(x_i - \mu)^2}{\sigma^2}\right]$$

What we'd like to do is find μ_0, σ_0 such that the likelihood (or log likelihood) is maximized. For this simple problem, the maximization can be computed analytically (i.e. by setting $\partial \log \mathcal{L}/\partial\mu|_{\mu_0,\sigma_0} = \partial \log \mathcal{L}/\partial\sigma|_{\mu_0,\sigma_0} = 0$). This results in the following *maximum-likelihood estimates* of the true parameters:

$$\mu_0 = \frac{1}{M} \sum_{i=1}^{M} x_i$$
$$\sigma_0^2 = \frac{1}{M} \sum_{i=1}^{M} (x_i - \mu_0)^2$$

In principle, we should also compute the second derivatives to make sure that this point represents a maximum rather than a minimum or a saddle point. However, in agreement with intuition, μ_0 is simply the mean of the observed data. These results are fairly simple calculations; let's evaluate them for our toy dataset:

```
# Estimators: sample mean and (sqrt of) sample variance
# Sometimes the *unbiased* estimator for the sample variance is used
# with (M-1) degrees of freedom...
mu_est = D.sum()/M
sigma_est = np.sqrt(((D-mu_est)**2).sum()/M)
print("""
      (mu,sigma)_true = {0:.2f}, {1:.2f}

      Sample estimators:
      (mu,sigma)_0  = {2:.2f}, {3:.2f} (based on {4} measurements)
      """.format(mu_true, sigma_true, mu_est, sigma_est, M))
```

```
      (mu,sigma)_true = 10.00, 1.00

      Sample estimators:
      (mu,sigma)_0  = 10.04, 0.98 (based on 1000 measurements)
```

Aside: using `fstrings` by putting an `f` in front of the quotes.

```
# Estimators: sample mean and (sqrt of) sample variance
# Sometimes the *unbiased* estimator for the sample variance is used with
# (M-1) degrees of freedom...
mu_est = D.sum()/M
sigma_est = np.sqrt(((D-mu_est)**2).sum()/M)
print(f"""
      (mu,sigma)_true = {mu_true:.2f}, {sigma_true:.2f}

      Sample estimators:
      (mu,sigma)_0 = {mu_est:.2f}, {sigma_est:.2f} (based on {M} measurements)
      """)
```

```
      (mu,sigma)_true = 10.00, 1.00

      Sample estimators:
      (mu,sigma)_0 = 10.04, 0.98 (based on 1000 measurements)
```

But what about the prior?

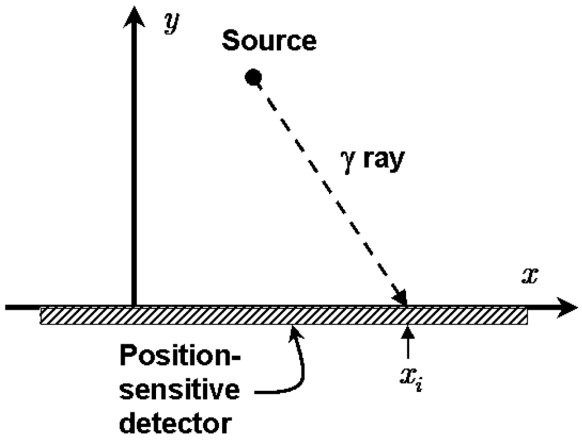
You'll noticed that we glossed over something here: the prior, $p(\mu, \sigma)$. The choice of prior will be discussed repeatedly in the course, but we can already note that it allows inclusion of other information into the computation. This feature becomes very useful in cases where multiple measurement strategies are being combined to constrain a single model. The necessity to specify a prior, however, is one of the more controversial pieces of Bayesian analysis.

A frequentist will point out that the prior is problematic when no true prior information is available. Though it might seem straightforward to use a **noninformative prior** like the flat prior mentioned above, there are some surprising subtleties involved. It turns out that in many situations, a truly noninformative prior does not exist! Frequentists point out that the subjective choice of a prior which necessarily biases your result has no place in statistical data analysis.

A Bayesian would counter that frequentism doesn't solve this problem, but simply skirts the question. Frequentism can often be viewed as simply a special case of the Bayesian approach for some (implicit) choice of the prior: a Bayesian would say that it's better to make this implicit choice explicit, even if the choice might include some subjectivity.

9.3. 📥 Problem: Radioactive lighthouse

Goal: Explore a classic problem from Gull (by way of Sivia's book with some tweaks).



In the figure, a radioactive source that emits gamma rays randomly in time but uniformly in angle is placed at (x_0, y_0). The gamma rays are detected on the x-axis and these positions are saved, x_k, $k = 1, 2, \cdots, N$. Given these observed positions, the problem is to estimate the location of the source.

Initially we'll assume that we know that $y_0 = 1$ (in whatever length units we are implicitly using), so our goal is to estimate x_0. We introduce the angle θ between the γ ray and the y-axis (with $\theta = 0$ meaning the gamma ray is detected at x_0).

Refs:

- D.S. Sivia, *Data Analysis, A Bayesian Tutorial*
- S.F. Gull, *Bayesian Inductive Inference and Maximum Entropy*

Answer the *italicized* questions. Check with your neighbors and ask for help if you get stuck or are unsure.

1. Claim: in the $p(\cdot|\cdot)$ notation, our goal is to find the posterior pdf $p(x_0|\{x_k\}, y_0)$. *How would you translate this posterior to words?*

2. *By Bayes' theorem, how is this posterior related to $p(\{x_k\}|x_0, y_0)$, $p(x_0|y_0)$, and $p(\{x_k\}|y_0)$?*

3. Claim: because the denominator pdf in 2. is independent of x_0, it is just a normalization factor for $p(x_0|\{x_k\}, y_0)$, so we don't need to calculate it explicitly. *Do you understand this? What good is an unnormalized posterior $p(x_0|\{x_k\}, y_0)$?*

4. Let's take for the prior pdf $p(x_0|y_0)$ that

$$p(x_0|y_0) = p(x_0) = \frac{1}{|x_{0,\text{max}} - x_{0,\text{min}}|} \quad \text{for } x_{0,\text{min}} < x_0 < x_{0,\text{max}}$$

and zero elsewhere. *What are we assuming? Why is this more plausible than letting x_0 be anything? Why do we assume a constant pdf? Is this pdf normalized?*

5. *If we assume that the x_ks are independent, then how is $p(\{x_k\}|x_0, y_0)$ simplified? Is this a justifiable assumption?*

6. *Show that for a particular k,*

$$p(x_k|x_0, y_0) = \frac{y_0}{\pi} \frac{1}{y_0^2 + (x_k - x_0)^2} \ ,$$

given that the angular distribution from θ_k is uniform from $-\pi/2$ to $\pi/2$, so $p(\theta_k|x_0, y_0) = 1/\pi$, and also that

$$p(\theta_k|x_0, y_0)\, d\theta_k = p(x_k|x_0, y_0)\, dx_k \ .$$

Why is the latter true?

7. Ok, now we're ready to see what the estimates for x_0 look like. Use the following code to generate a set of random x points for a Cauchy distribution. *Look up the Stats documentation for a Cauchy distribution (google "scipy stats cauchy") to verify it is the same function derived above (note the use of `loc` and `scale`). Run it a few times to see the fluctuations in the distribution.*
What can you say about the tails of this distribution compared to your experience with Gaussian distributions?

```
%matplotlib inline

import numpy as np

import scipy.stats as stats
from scipy.stats import cauchy, uniform

import matplotlib.pyplot as plt
import seaborn as sns; sns.set() # for plot formatting
#sns.set_context("talk")
```

```
# True location of the radioactive source
x0_true = 1.
y0_true = 1.

# Generate num_pts random numbers distributed according to dist and plot
num_pts = 512
x_pts = np.arange(num_pts)

# Distribution knowing where the source is: scipy.stats.cauchy(loc, scale)
dist = cauchy(x0_true, y0_true)
dist_pts = dist.rvs(num_pts)

# Make some plots!
fig = plt.figure(figsize=(15,5))

# First plot all the points, letting it autoscale the counts
ax_1 = fig.add_subplot(1,3,1)
ax_1.scatter(x_pts, dist_pts)

# Repeat but zoom in to near the origin
ax_2 = fig.add_subplot(1,3,2)
ax_2.scatter(x_pts, dist_pts)
ax_2.set_ylim(-10.,10.)

# Finally make a zoomed-in histogram
ax_3 = fig.add_subplot(1,3,3)
out = ax_3.hist(dist_pts, bins=np.arange(-10., 10., 0.2))

# Print out the numerical limits (max and min)
print('maximum = ', np.amax(dist_pts))
print('minimum = ', np.amin(dist_pts))

fig.tight_layout()
```

8. Now you'll repeat the same graphs, but this time generate the distribution of points starting with a uniform distribution in angle between $-\pi/2$ and $\pi/2$. *In particular, at the* `###`*, write the formula for* `dist_pts` *in terms of* `theta_dist` *and* `x0_true`, `y0_true`.

```
# True location of the radioactive source
x0_true = 1.
y0_true = 1.

# Generate num_pts random numbers distributed according to dist and plot
num_pts = 512
x_pts = np.arange(num_pts)

# Uniform distribution in theta: uniform(a,b) in [a, a+b]
theta_dist = uniform(-np.pi/2., np.pi)

dist_pts_alt = ### Fill in formula here for x_k points

# Make some plots!
fig = plt.figure(figsize=(15,5))

# First plot all the points, letting it autoscale the counts
ax_1 = fig.add_subplot(1,3,1)
ax_1.scatter(x_pts, dist_pts_alt)

# Repeat but zoom in to near the origin
ax_2 = fig.add_subplot(1,3,2)
ax_2.scatter(x_pts, dist_pts_alt)
ax_2.set_ylim(-10.,10.)

# Finally make a zoomed-in histogram
ax_3 = fig.add_subplot(1,3,3)
out = ax_3.hist(dist_pts_alt, bins=np.arange(-10., 10., 0.2))

# Print out the numerical limits (max and min)
print('maximum = ', np.amax(dist_pts_alt))
print('minimum = ', np.amin(dist_pts_alt))

fig.tight_layout()
```

Before moving on, let's do some more plotting of histograms with different numbers of samples to build some intuition. We define a plotting function first so it is easy to plot several histograms all at once. Run it several times to see the nature of the fluctuations.

```
def dist_hist_plot(ax, name, x_dist, dist, num_samples, bin_width,
                   x_label=None):
    """
       Plot a pdf and histogram of samples with specified list of points to
       be plotted, which sets the range of the histogram, and width of bins.

       Parameters:
       -----------
       ax (matplotlib axis): axis for the histogram
       name (string): description of
       x_dist (ndarray): points to be plotted
       dist (scipy.stats distribution): pdf to make draws from
       num_samples (int): number of draws to make from dist
       bin_width (float): width of each bin to be plotted
       x_label (string): label for the x-axis
    """
    samples = dist.rvs(size=num_samples)   # generate num_samples draws
    bin_bounds = np.arange(x_dist[0], x_dist[-1], bin_width)
    count, bins, ignored = ax.hist(samples, bins=bin_bounds, density=True,
                                   color='blue', alpha=0.7)
    ax.plot(x_dist, dist.pdf(x_dist), linewidth=2, color='r')
    title_string = name + f'  samples = {num_samples:d}'
    ax.set_title(title_string)
    ax.set_xlim(x_dist[0], x_dist[-1])
    if x_label:
        ax.set_xlabel(x_label)

x_max = 10.
x_dist = np.linspace(-x_max, x_max, 500)
name = rf'cauchy $x_0=${x0_true:1.1f}, $y_0=${y0_true:1.1f}'
fig = plt.figure(figsize=(15,5))
bin_width = 0.5
x_label = r'$x_k$'

num_samples = 100
cauchy_dist = stats.cauchy(x0_true, y0_true)
ax_1 = fig.add_subplot(1, 3, 1)
dist_hist_plot(ax_1, name, x_dist, cauchy_dist, num_samples,
               bin_width, x_label)

num_samples = 1000
cauchy_dist = stats.cauchy(x0_true, y0_true)
ax_2 = fig.add_subplot(1, 3, 2)
dist_hist_plot(ax_2, name, x_dist, cauchy_dist, num_samples,
               bin_width, x_label)

num_samples = 10000
cauchy_dist = stats.cauchy(x0_true, y0_true)
ax_3 = fig.add_subplot(1, 3, 3)
dist_hist_plot(ax_3, name, x_dist, cauchy_dist, num_samples,
               bin_width, x_label)

fig.tight_layout()
```

Define and plot the posterior for x_0

9. In this section the posterior for x_0 is calculated and plotted for different numbers of data. The prior is taken to be a uniform PDF from -4 to 4 (we really don't believe it is bigger than that but otherwise we don't know what it is).

For each $N_{\rm max}$, besides plotting the posterior for x_0, we calculate the mean of the posterior (denoted $\langle x_0 \rangle$) and the mean of the set of $N_{\rm max}$ points (denoted $\overline{x}_0$).

```
def log_prior(x0, y0_true, x_min=-4., x_max=+4.):
    """
    Log uniform prior from x_min to x_max.  Not normalized!
    """
    if (x0 > x_min) and (x0 < x_max):
        return 0.
    else:
        return -np.inf    # log(0) = -inf

def log_likelihood(x0, y0_true, dist_pts, N_max):
    """
    Log likelihood for the first N_max points of the dist_pts array,
    assuming independent.  Not normalized!
    """
    L_pts = -np.log(y0_true**2 + (dist_pts[0:N_max] - x0)**2)
    return sum(L_pts)

def posterior_calc(x0_pts, y0_true, dist_pts, N_max, x0_min=-4., x0_max=+4.):
    """
    Calculate the posterior for a set of x0_pts given y0 (y0_true) and a
    list of N_max x_k observations (dist_pts).
    """
    log_L_pts = [log_likelihood(x0, y0_true, dist_pts, N_max) \
                 for x0 in x0_pts]
    log_L_pts -= np.amax(log_L_pts)  # subtract maximum of log likelihood

    log_prior_pts = [log_prior(x0, y0_true, x0_min, x0_max) \
                     for x0 in x0_pts]

    posterior_pts = np.exp(log_prior_pts + log_L_pts)
    return posterior_pts

def lighthouse_stats(dist_pts, N_max, x0_pts, posterior_pts):
    """
    Given an array of N_max observed detection points (dist_pts) and a
    posterior pdf (posterior_pts) for an array of x0 points (x0_pts), return
    the mean of dist_pts and the mode and mean of the posterior.
    """
    mean_dist = np.mean(dist_pts[0:N_max])
    max_posterior = x0_pts[np.argmax(posterior_pts)]
    mean_posterior = np.sum(x0_pts * posterior_pts) / np.sum(posterior_pts)
    return mean_dist, max_posterior, mean_posterior
```

Some questions about the implementation through these functions:

- *If you wanted* `log_prior` *to return the normalized log prior, how would you modify the function?*

- *Why is the log likelihood adjusted in* `posterior_calc`*?*

- *Why is it not necessary to normalize the posterior? Would it not be easier just to normalize it?*

```
# True location of the radioactive source
x0_true = 1.
y0_true = 1.

# Distribution knowing where the source is: scipy.stats.cauchy(loc, scale)
num_pts = 512  # number of observations
dist = cauchy(x0_true, y0_true)   # sampling a Cauchy distribution directly
dist_pts = dist.rvs(num_pts)      # generate {x_k} for k = 1 to num_pts

# Choose the set of N_max to be plotted (multiple of 3, up to num_pts)
N_max_values = [1, 2, 4, 8, 16, 64, 128, 256, 512]

x0_min = -4.  # lower bound for prior
x0_max = +4.  # upper bound for prior
x0_pts = np.arange(-4.5, 4.5, 0.01)

fig = plt.figure(figsize=(14, 2.5*len(N_max_values)/3))

# Step through counter (k) for each N_max entry in N_max_values
for k, N_max in enumerate(N_max_values):
    posterior_pts = posterior_calc(x0_pts, y0_true, dist_pts, N_max)
    mean_dist, max_posterior, mean_posterior = lighthouse_stats(dist_pts,
                                                N_max, x0_pts, posterior_pts)

    # now make the plots: 3 to a row
    ax = fig.add_subplot(int(len(N_max_values)/3), 3, k+1)
    ax.set_xlabel(r'source position $x_0$')
    #ax.set_yticks([])  # turn off the plotting of ticks on the y-axis
    ax.plot(x0_pts, posterior_pts, color='blue')
    ax.set_title(rf'$N_{{\rm max}} = {N_max:d}$')
    ax.axvline(mean_dist, 0., 1.1, color='black', linestyle="--", lw=2)
    ax.axvline(mean_posterior, 0., 1.1, color='red', linestyle="--", lw=2)
    ax.set_xlim(np.min(x0_pts), np.max(x0_pts))

    stats_title = rf'$\overline{{x}}_0 = {mean_dist:.2f}$' + '\n' \
                  rf'$\langle x_0 \rangle = {mean_posterior:.2f}$'
    ax.annotate(stats_title,
                xy=(0.05,0.85), xycoords='axes fraction',
                horizontalalignment='left',verticalalignment='top')

figure_title = rf'Radioactive lighthouse problem (true $x_0 = {x0_true:.1f}$)'
fig.suptitle(figure_title, y=1.02, fontsize=14)

fig.tight_layout()
```

Run the cell above several times for each $N_{\rm max}$ and record the results in the table:

```
%%html

<style>
 table { width:80% !important; }
 table td, table th, table tr {border: 2px solid black !important;
                               text-align:center !important;
                               font-size: 20px;}
</style>
```

N_{max}	**1:** $\langle x_0 \rangle$	**1:** $\overline{x}_0$	**2:** $\langle x_0 \rangle$	**2:** $\overline{x}_0$	**3:** $\langle x_0 \rangle$	**3:** $\overline{x}_0$
1						
2						
4						
16						
64						
256						

10. *What are your observations about the posterior for x_0 as a function of N_{max} ? Which mean (from the set of N_{max} points or from the posterior) is the better estimate?*

11. *Why does the Central Limit Theorem appear to fail?* (The mean from the set of N_{max} samples does not tend to the true x_0.)

Additional tasks:

- Convert to sample the distribution using MCMC.
- Generalize to find the joint posterior of x_0 and y_0.
- (suggest something!)

9.4. 📥 Problem: Amplitude of a signal in the presence of background

Adapted from the discussion in Section 3.1 of Sivia and Skilling, *Data Analysis: A Bayesian Tutorial*.

The goal is to estimate the amplitude of a signal when there is background. We'll take a limiting case where the background is flat, so it is completely specified by its magnitude $B > 0$, and the signal is known to be a Gaussian with unknown amplitude A but (at least initially) known position (mean) and width (standard deviation). The measurements will be integer numbers of counts $\{N_k\}$ in well-defined (equally spaced) bins $\{x_k\}$, where k runs over integers labeling the bins. Then the goal translates into finding A and B given $\{N_k\}$ and all the other information (bin sizes, signal position and width). After we can modify our goal if we do not care about B, or we care only about B.

```
%matplotlib inline

import numpy as np
from scipy.special import factorial

# We'll get our uniform distributions from stats, but there are other ways.
import scipy.stats as stats
import scipy.integrate as integrate
from scipy import interpolate

import matplotlib.pyplot as plt
import seaborn as sns; sns.set()

from mpl_toolkits import mplot3d
from matplotlib import cm

plt.rcParams.update({'font.size': 16})
```

Set up the true signal plus background magnitudes, and the other parameters dictating the signal (width w and mean x_0 of the Gaussian):

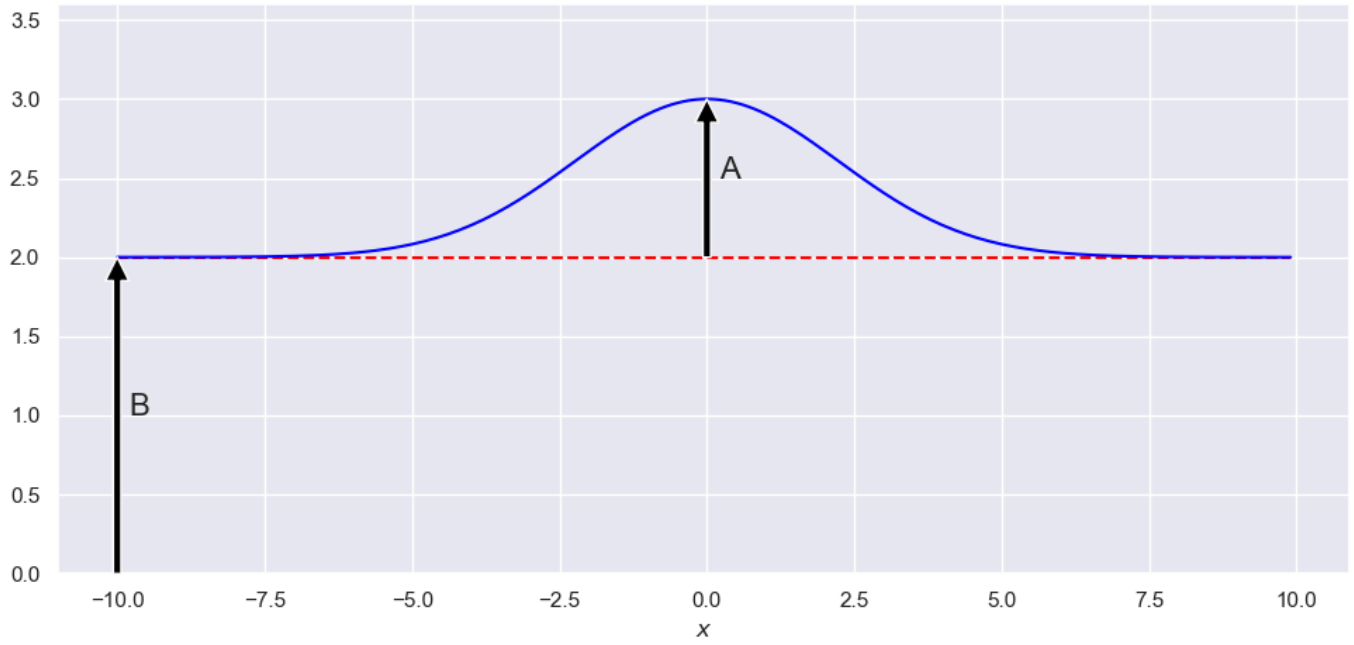
$$D_k = n_0 \left[A e^{-(x_k - x_0)^2/2w^2} + B \right]$$

Here n_0 is a constant that scales with measurement time. Note that D_k is not an integer in general, unlike N_k.

```
# We'll start with the numbers used by Sivia
A_true = 1.
B_true = 2.
width = np.sqrt(5.)
x_0 = 0

def exact_data(A, B, n_0, x_k, x_0=0., width=np.sqrt(5.)):
    """
    Return the exact signal plus background.  The overall scale is n_0,
    which is determined by how long counts are collected.
    """
    return n_0 * (A * np.exp(-(x_k - x_0)**2/(2.*width**2)) + B)
```

Make a plot of the true signal plus background we are trying to deduce.

```
x_min = -10.; x_max = 10.; del_x = 0.1  # range and spacing for plotting
x_pts = np.arange(x_min, x_max, del_x )

n_0 = 1.  # for this plot, the vertical scale is irrelevant
y_pts = exact_data(A_true, B_true, n_0, x_pts, width=width)

fig_true = plt.figure(figsize=(10,5))

ax_true = fig_true.add_subplot(1,1,1)
ax_true.plot(x_pts, B_true*np.ones(len(x_pts)),
             color='red', linestyle='--')  # just the background
ax_true.plot(x_pts, y_pts, color='blue')   # signal plus background
ax_true.set_ylim(0., 1.2*(A_true + B_true))
ax_true.set_xlabel(r'$x$')

ax_true.annotate('A', xy=(x_0 + 0.2, A_true/2. + B_true))
ax_true.annotate('', (x_0, B_true + A_true), (x_0, B_true),
                 arrowprops=dict(facecolor='black', shrink=0.))
ax_true.annotate('B', xy=(-x_max + 0.2, B_true/2.))
ax_true.annotate('', (-x_max, B_true), (-x_max, 0),
                 arrowprops=dict(facecolor='black', shrink=0.))

fig_true.tight_layout()
```



Poisson distribution

We are imagining a counting experiment, so the statistics of the counts we record will follow a Poisson distribution. Comments on Poisson distributions for this application:

- Counts are integers so we want a probability *mass* function (rather than a probability *density* function), i.e., the probability to get 0, 1, 2, . . . counts in a specified time interval (and not 1/2 or 4.5 counts).
- The Poisson distribution does this: the probability of a given number of events (like counts) in a fixed interval (or time or space or whatever) *if* there is a constant mean rate of events and the spacing of successive events is independent of the previous events.

- A familiar example is radioactive decay: a Poisson distribution describes the # of counts in an hour (say) and the constant mean rate is specified by the half life.
- There is a derivation of the Poisson distribution in chapter 5 of Sivia.

The Poisson discrete random variable from scipy.stats is defined by (see documentation)

$$p(k \mid \mu) = \frac{\mu^k e^{-\mu}}{k!} \quad \text{for } k \geq 0 \ .$$

where k is an integer and μ is called the shape parameter. The mean and variance of this distribution are both equal to μ. Sivia and Gregory each use a different notation for for this distribution, which means you should be flexible.

For convenience in comparing to the discussion in Sivia's text, we'll define our own version using his notation in this notebook:

$$p(N \mid D) = \frac{D^N e^{-D}}{N!} \quad \text{for } N \geq 0 \ .$$

where N is an integer. The results for the normalization, mean, and variance are:

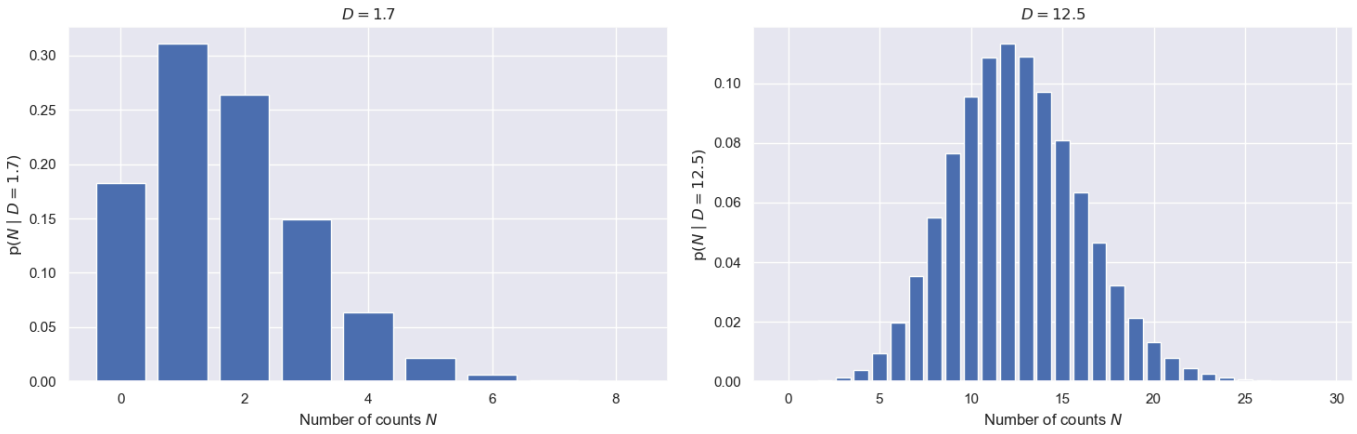
$$\begin{aligned} \langle 1 \rangle &= \sum_0^\infty p(N \mid D) = 1 \\ \langle N \rangle &= \sum_0^\infty N p(N \mid D) = D \\ \langle N^2 \rangle - \langle N \rangle^2 &= \sum_0^\infty N^2 p(N \mid D) - D^2 = D \end{aligned} \tag{9.1}$$

as already noted. (These are easily verified with Mathematica. Can you verify them with sympy?)

```
def poisson(N, D):
    """
    Returns a Poisson distribution value with mean D for integer N.
    We require N to be an integer greater than equal to zero.
    """
    assert (isinstance(N, int) and N >= 0), \
           "N must be a non-negative integer!"

    return D**N * np.exp(-D) / factorial(N)
```

Make some plots of Poisson distribution as a function of N, given D.

```python
def poisson_plot(ax, D, max_N):
    """
    Make a bar plot on the axis ax of the Poisson distribution for mu = D
    and out to a maximum integer max_N.
    """
    N_pts = np.arange(0, max_N, 1, dtype=int)
    poisson_pts = [poisson(int(N), D) for N in N_pts]
    ax.bar(N_pts, poisson_pts, width=0.8, bottom=None, align='center')
    ax.set_xlabel(r'Number of counts $N$')
    ax.set_ylabel(fr'$\mathrm{{p}}(N \mid D={D:.1f})$')
    ax.set_title(rf'$D = {D:.1f}$')
    return 0

fig = plt.figure(figsize=(15,5))

D1 = 1.7
max_N1 = 9
ax1 = fig.add_subplot(1,2,1)
poisson_plot(ax1, D1, max_N1)

ax2 = fig.add_subplot(1,2,2)
D2 = 12.5
max_N2 = 30
poisson_plot(ax2, D2, max_N2)

fig.tight_layout()
```



Follow-ups

- *Try both smaller and larger values of D and note the transition in the form of the pdf.*
- At $D = 12.5$ the pdf is already looking like a Gaussian (or what most of us imagine a Gaussian to look like). *Prove that in the limit $D \to \infty$ that*

$$p(N \mid D) \overset{D\to\infty}{\longrightarrow} \frac{1}{\sqrt{2\pi D}} e^{-(N-D)^2/(2D)}$$

You'll want to use Stirling's formula: $x! \to \sqrt{2\pi x} e^{-x} x^x$ as $x \to \infty$. (Hint: let $x = N = D(1+\delta)$ where $D \gg 1$ and $\delta \ll 1$. And use $(1+\delta)^a = e^{a \ln(1+\delta)}$.)

- *Show that this limit works in practice and visualize how close it is by adding the Gaussian pdf to the plot.* (See scipy.stats.norm or define a function yourself.)

Preparing data and the pdfs we'll need

For our data, we can get single samples from a scipy.stats Poisson distribution with specified μ (what we are calling D) or an array of samples. Try running the next cell a few times to get a feel for how the samples fluctuate.

```
S1 = stats.poisson.rvs(mu=D2)  # sample one Poisson-distributed random number
print(S1)

S2 = stats.poisson.rvs(mu=D2, size=5)  # sample an array of specified size
print(S2)
```

```
17
[11 18 12 13  9]
```

Once we specify the number and size of the bins for data (and other aspects), we can create a dataset and then make a series of plots as in Sivia. So define functions to carry these out.

Each of the data $\{N_k\}$ is assumed to be independent of the others, so for M bins:

$$p(\{N_k\} \mid A, B, I) = \prod_{k=1}^{M} p(N_k \mid A, B, I) = \prod_{k=1}^{M} \frac{D_k^{N_k} e^{-D_k}}{N_k!}$$

We use this to make the measured dataset based on `A_true` and `B_true`. This is also what we use to calculate the log likelihood (natural logarithm of this likelihood function), but in that case A and B are not equal to their true values. We also define a (flat) prior that is *supposed* to be uninformative (*but is it?*).

```
# make a dataset for exploring
def make_dataset(A_true, B_true, width, x_0, databins, delta_x=1, D_max=100):
    """
    Create a data set based on the number of bins (databins), the spacing
    of bins (delta_x), and the maximum we want the exact result to have
    (D_max, this fixes the n_0 parameter).

    Return arrays for the x points (xk_pts), the corresponding values of the
    exact signal plus background in those bins (Dk_pts), the measured values
    in those bins (Nk_pts, integers drawn from a Poisson distribution), the
    maximum extent of the bins (x_max) and n_0.
    """

    # set up evenly spaced bins, centered on x_0
    x_max = x_0 + delta_x * (databins-1)/2
    xk_pts = np.arange(-x_max, x_max + delta_x, delta_x, dtype=int)

    # scale n_0 so maximum of the "true" signal plus background is D_max
    n_0 = D_max / (A_true + B_true)
    Dk_pts = exact_data(A_true, B_true, n_0, xk_pts, width=width)

    # sample for each k to determine the measured N_k
    Nk_pts = [stats.poisson.rvs(mu=Dk) for Dk in Dk_pts]

    return xk_pts, Dk_pts, Nk_pts, x_max, n_0
```

```
# Define the pdfs and combine with Bayes' theorem.
def log_prior(A, B):
    """
    Following Sivia's lead, we take a uniform (flat) prior with large enough
    maximums but, more importantly, require positive values for A and B.
    """
    A_max = 5.
    B_max = 5.
    # flat prior
    if np.logical_and(A <= A_max, B <= B_max).all():
        return np.log(1/(A_max * B_max))
    else:
        return -np.inf

def log_likelihood(A, B, xk_pts, Nk_pts, n_0):
    """Log likelihood for data Nk_pts given A and B"""
    Dk_pts = exact_data(A, B, n_0, xk_pts, width=width)
    try:
        return np.sum(Nk_pts * np.log(Dk_pts) - Dk_pts)
    except ValueError:
        return -np.inf

def log_posterior(A, B, xk_pts, Nk_pts, n_0):
    """Log posterior for data Nk_pts given A and B"""
    return log_prior(A, B) + log_likelihood(A, B, xk_pts, Nk_pts, n_0)

def normalize(y_pts, x_pts):
    """Normalize the array y_pts to unity using Simpson's rule."""
    return y_pts / integrate.simpson(y_pts, x=x_pts)

def find_index(x_pts, x_value):
    """Return the index of the element of the ascending array x_pts that
       is closest to x_value.
    """
    return np.fabs(x_pts - x_value).argmin()
```

Figures to analyze!

We'll follow Sivia again and make four figures for each case we consider. (He does this in two groups of two, in figures 3.3 and 3.4.) We consider what happens for fixed signal and background but changing the experimental conditions specified by `D_max` and `databins` (we'll keep `delta_x` fixed to 1).

```
def find_contour_levels(grid):
    """Compute 68%, 95%, 99.7% credible-region contour levels for a gridded 2D posterior"""
    sorted_ = np.sort(grid.ravel())[::-1]
    pct = np.cumsum(sorted_) / np.sum(sorted_)
    cutoffs = np.searchsorted(pct, np.array([0.68, 0.95, 0.997]))
    return np.sort(sorted_[cutoffs])

def make_figs(databins, delta_x, D_max, flag=False):
    """
    If flag=True, generate another panel of four figures that shows 2. three
    times (contour, shaded contour, 3D) plus showing 68% and 95% intervals.
    Make a panel of four figures:
      1. The data based on sampling the Poisson distribution.  If you rerun
          the function you'll get new data.
      2. The posterior pdf for the amplitude A of the signal and the magnitude
          B of the (flat) background.  Five equally spaced contours are shown.
      3. Posterior for A for two cases: i) marginalized over B, and
          ii) conditional on known B = B_true.
      4. Posterior for B marginaized over A.
    If flag=True, generate another panel of four figures that shows 2. three
    times (contour, shaded contour, 3D) plus showing 68% and 95% intervals.
    """
    xk_pts, Dk_pts, Nk_pts, x_max, n_0 = make_dataset(A_true, B_true, width,
                                                      x_0, databins, delta_x,
                                                      D_max)
    total_counts = np.sum(Nk_pts)

    # Figure 1.
    fig = plt.figure(figsize=(12,10))

    ax1 = fig.add_subplot(2,2,1)
    ax1.plot(xk_pts, Nk_pts, drawstyle='steps-mid', color='blue')
    ax1.set_xlim(-x_max, x_max)
    ax1.set_ylim(bottom = 0)
    ax1.set_xlabel(r'Measurement variable $x$')
    ax1.set_ylabel(r'Number of counts $N$')

    # Figure 2.  Can use contour or contourf.
    A_max = 3.; B_max = 3.
    A_pts = np.arange(0.01, A_max, .02)  # You may want to adjust the density
    B_pts = np.arange(0.01, B_max, .02)  #  of points used.
    A_grid, B_grid = np.meshgrid(A_pts, B_pts)

    # Z_grid = Z(B, A) the way we define it here with a list comprehension.
    Z_grid = np.array([[log_posterior(A, B, xk_pts, Nk_pts, n_0)
                        for A in A_pts] for B in B_pts])
    Z_grid = np.exp(Z_grid - np.max(Z_grid))  # normalize the peak to be 1
    ax2 = fig.add_subplot(2,2,2)
    ax2.axvline(A_true, color='gray', linestyle='--', alpha=0.8)
    ax2.axhline(B_true, color='gray', linestyle='--', alpha=0.8)

    ax2.contourf(A_grid, B_grid, Z_grid, levels=5, cmap='jet')
    ax2.set_xlim(0., A_max)
    ax2.set_ylim(0., B_max)
    ax2.set_xlabel(r'Amplitude $A$')
    ax2.set_ylabel(r'Background $B$')

    # Figure 3.
    ax3 = fig.add_subplot(2,2,3)
    B_true_index = find_index(B_pts, B_true)
    B_marginalized = [integrate.simpson(Z_grid[:,i], x=B_pts) \
                                      for i in range(len(A_pts))]
    B_marginalized = normalize(B_marginalized, A_pts)
    B_true_fixed = Z_grid[B_true_index,:]
```

```
    B_true_fixed = normalize(B_true_fixed, A_pts)
    ax3.plot(A_pts, B_marginalized, color='blue',
             label='marginalized over B')
    ax3.plot(A_pts, B_true_fixed, color='red',
             linestyle='--', label='B = B_true')
    ax3.set_xlabel(r'Amplitude $A$')
    ax3.set_ylabel(r'${\rm p}(A | \{N_k\}, I)$')
    ax3.legend()

    # Figure 4.
    ax4 = fig.add_subplot(2,2,4)
    A_marginalized = [integrate.simpson(Z_grid[j,:], x=A_pts) \
                                    for j in range(len(B_pts))]
    A_marginalized = normalize(A_marginalized, B_pts)
    ax4.plot(B_pts, A_marginalized, color='blue',
             label='marginalized over A')
    ax4.set_xlabel(r'Background $B$')
    ax4.set_ylabel(r'${\rm p}(B|\{N_k\}, I)$')
    ax4.legend()

    overall_title = f'Total counts = {total_counts},  ' \
                    + f'# of bins = {databins}' \
                    + '\n'
    fig.suptitle(overall_title, y=1.01)

    fig.tight_layout()

    if (flag):
        fig2 = plt.figure(figsize=(12,10))
        # Figure 1.
        ax2_1 = fig2.add_subplot(2,2,1)
        contour_levels = [0.2, 0.4, 0.6, 0.8, 1.0]
        ax2_1.contour(A_grid, B_grid, Z_grid, levels=contour_levels)
        ax2_1.axvline(A_true, color='gray', linestyle='--', alpha=0.8)
        ax2_1.axhline(B_true, color='gray', linestyle='--', alpha=0.8)
        ax2_1.set_xlim(0., A_max)
        ax2_1.set_ylim(0., B_max)
        ax2_1.set_xlabel(r'Amplitude $A$')
        ax2_1.set_ylabel(r'Background $B$')
        ax2_1.set_title('Contour plot with levels 0.2, 0.4, 0.6, 0.8, 1.0')

        # Figure 2.
        ax2_2 = fig2.add_subplot(2,2,2)
        ax2_2.contourf(A_grid, B_grid, Z_grid, levels=5, cmap='jet')
        ax2_2.axvline(A_true, color='gray', linestyle='--', alpha=0.8)
        ax2_2.axhline(B_true, color='gray', linestyle='--', alpha=0.8)
        ax2_2.set_xlim(0., A_max)
        ax2_2.set_ylim(0., B_max)
        ax2_2.set_xlabel(r'Amplitude $A$')
        ax2_2.set_ylabel(r'Background $B$')
        ax2_2.set_title('Color contour plot with contourf')

        # Figure 3.
        ax2_3 = fig2.add_subplot(2,2,3, projection='3d')
        ax2_3.plot_surface(A_grid, B_grid, Z_grid, rstride=1, cstride=1,
                           cmap='jet', linewidth=0, antialiased=False,
                           edgecolor='none')
        ax2_3.set_xlim(0., A_max)
        ax2_3.set_ylim(0., B_max)
        ax2_3.set_xlabel(r'Amplitude $A$')
        ax2_3.set_ylabel(r'Background $B$')
        ax2_3.set_title('Surface plot')

        # Figure 4.
        ax2_4 = fig2.add_subplot(2,2,4)
```

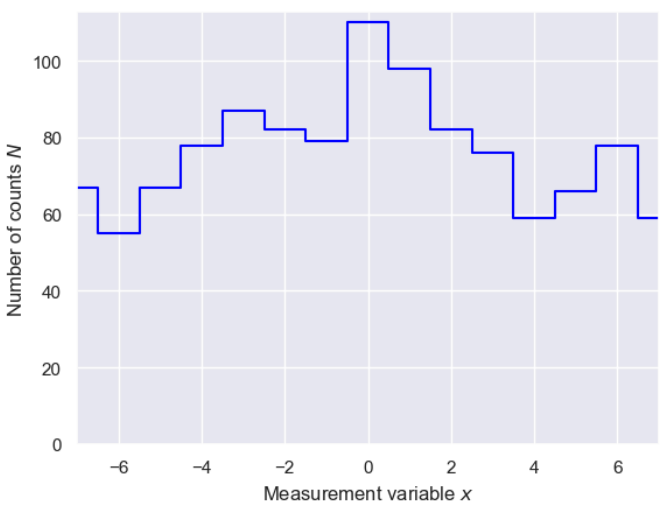
```
        ax2_4.contour(A_grid, B_grid, Z_grid,
                      levels=find_contour_levels(Z_grid))
        ax2_2.axvline(A_true, color='gray', linestyle='--', alpha=0.8)
        ax2_2.axhline(B_true, color='gray', linestyle='--', alpha=0.8)
        ax2_4.set_xlim(0., A_max)
        ax2_4.set_ylim(0., B_max)
        ax2_4.set_xlabel(r'Amplitude $A$')
        ax2_4.set_ylabel(r'Background $B$')
        ax2_4.set_title('Contours at 68%, 95%, 99.7%')
        fig2.tight_layout()

    return fig
```

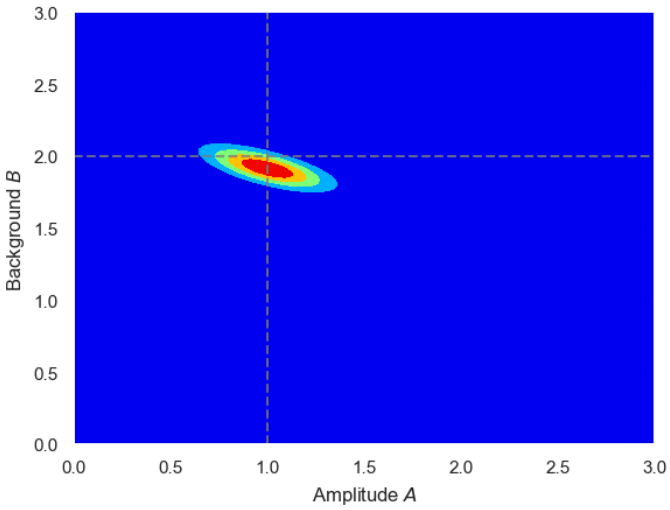
Ok, let's do it. For each of the examples (and more that you invent):

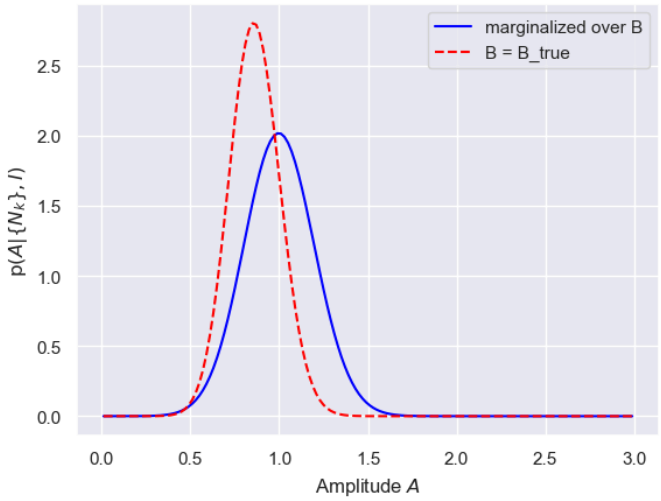
- Make sure to run them multiple times to get a feel for the fluctuations. *What features are stable and what varies wildly (will change depending on the example!).*
- *Interpret the change in the width of the contours in the different cases.*
- *What do the slopes of the posterior ellipses (2nd figures) tell us? Can you account for the behavior in the different cases?*
- *Interpret the difference between the solid and dashed lines in the plots that marginalize over B with two opposite limits of the prior knowledge of B (3rd figures).*
- *What are your conclusions for how to design the experiment given limited resources?*
- *Follow-up task: check whether the size of the bin matters in the four figures.

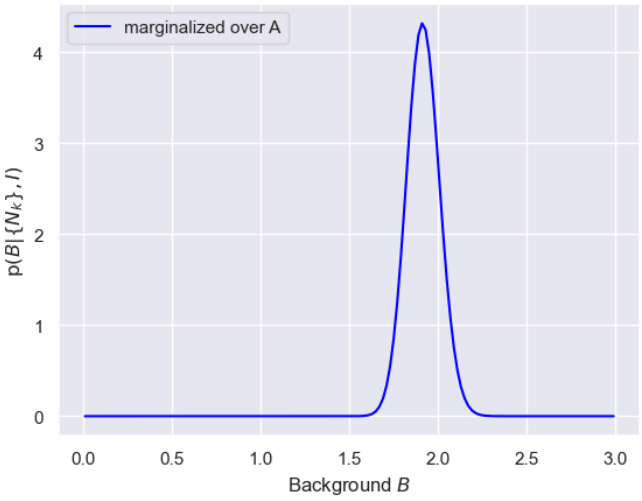
```
# Baseline case: 15 bins and maximum expection of 100 counts.

D_max = 100
databins = 15
delta_x = 1

fig = make_figs(databins, delta_x, D_max, flag=True)
```

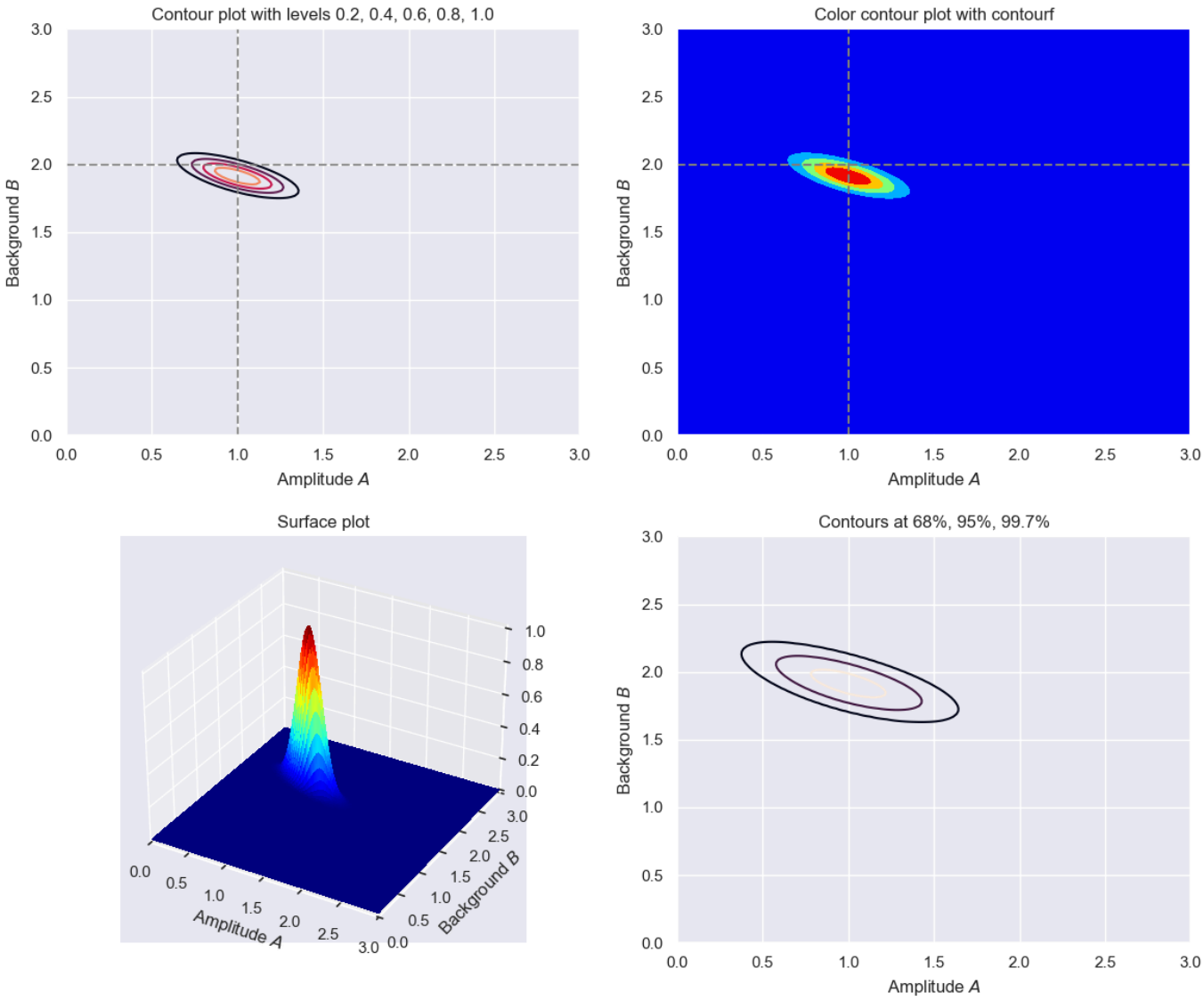
Total counts = 1143, # of bins = 15



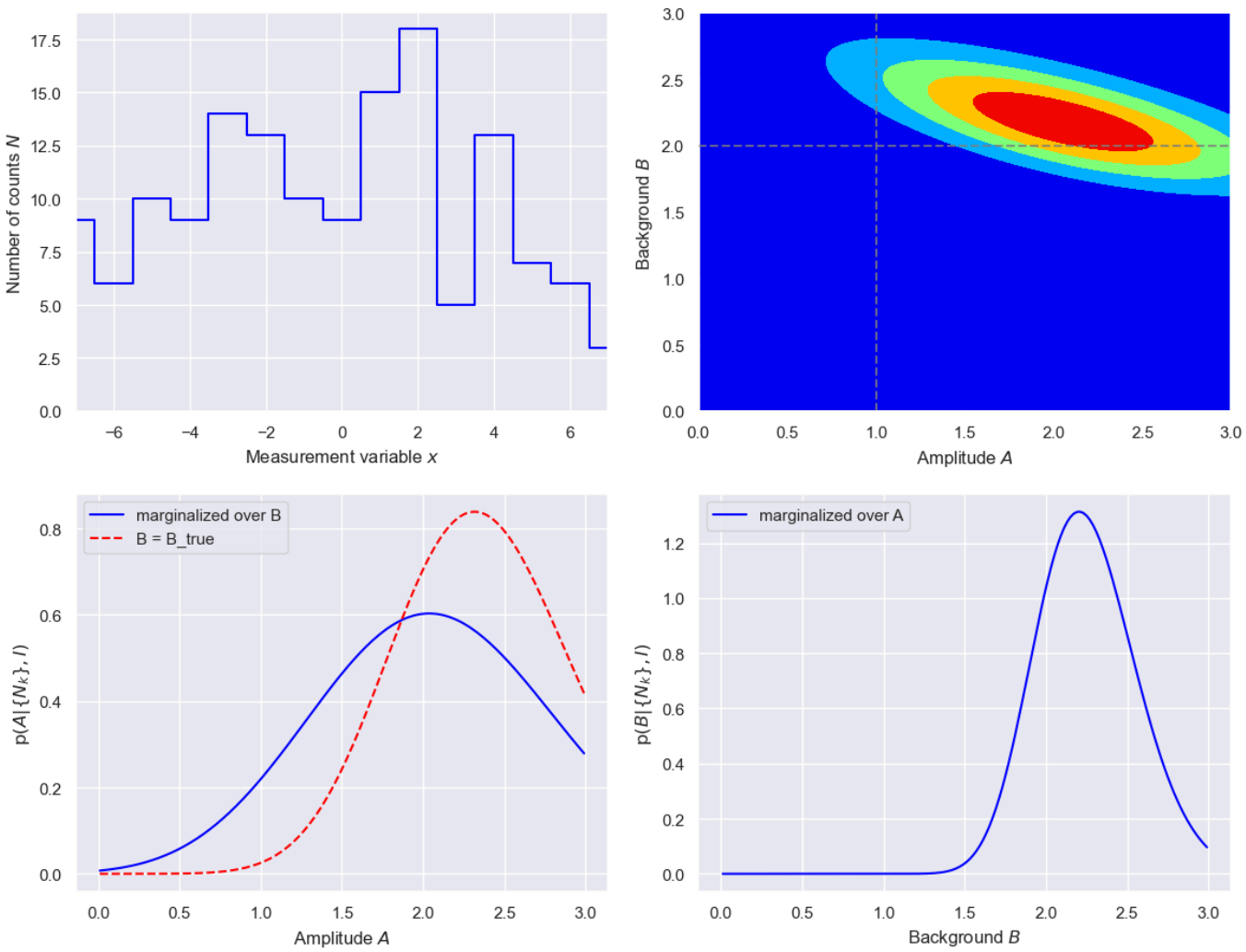


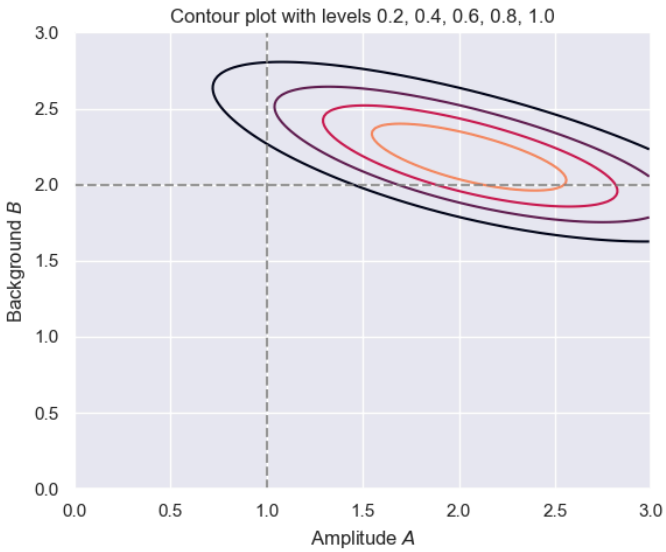


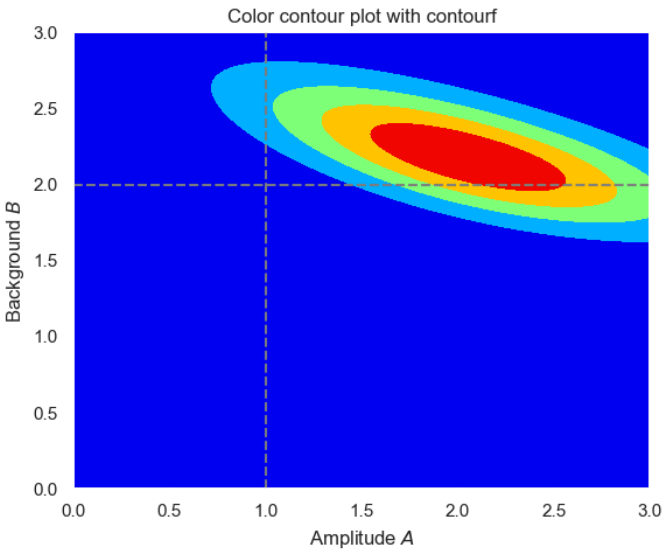


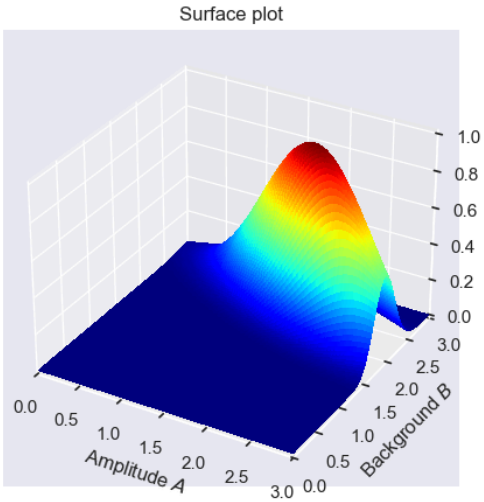


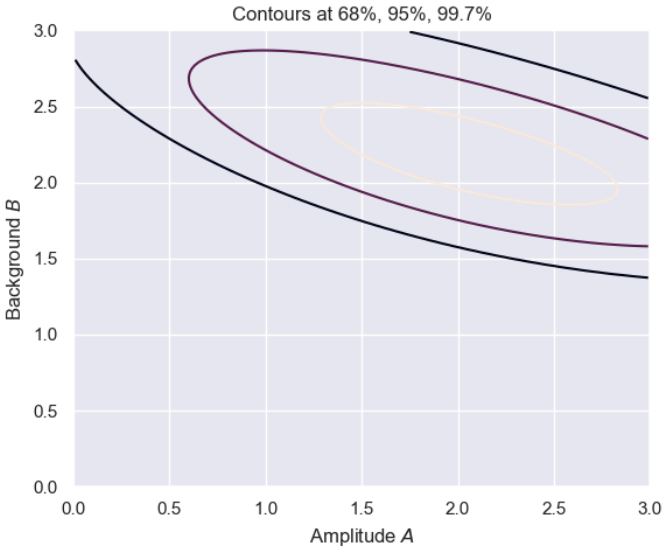

```
# Less data case: 15 bins and maximum expection of only 10 counts.

D_max = 10
databins = 15
delta_x = 1

fig = make_figs(databins, delta_x, D_max, flag=True)
```


Total counts = 147, # of bins = 15
Number of counts N
Measurement variable x
Background B
Amplitude A
marginalized over B
B = B_true
p(A|{N_k}, I)
Amplitude A
marginalized over A
p(B|{N_k}, I)
Background B

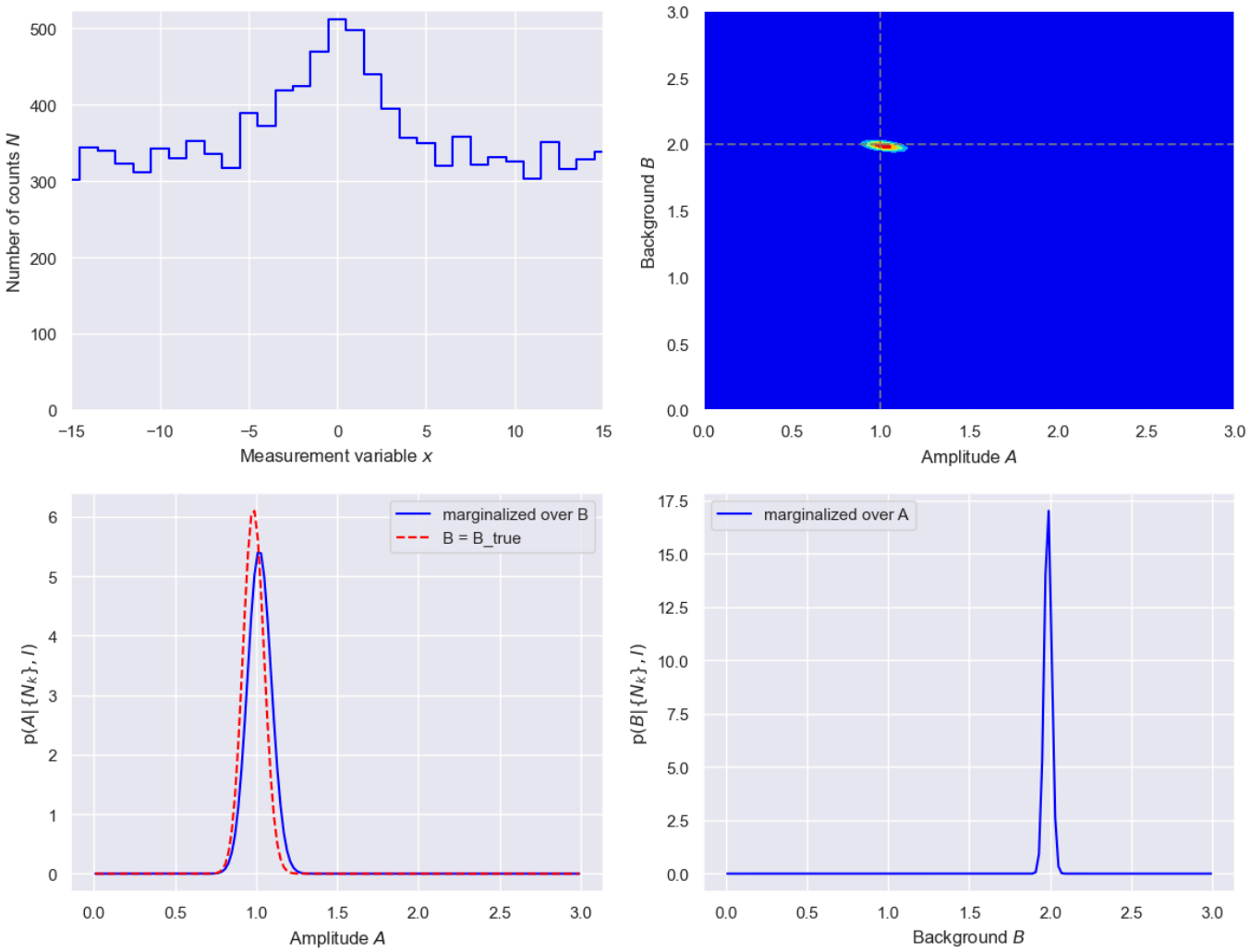


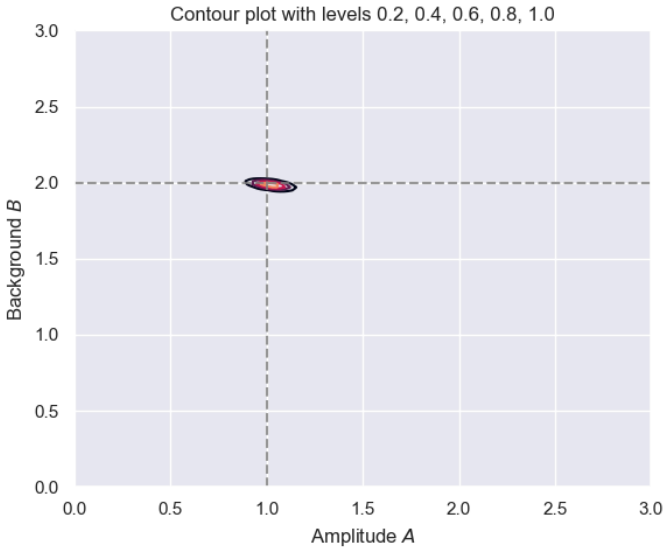


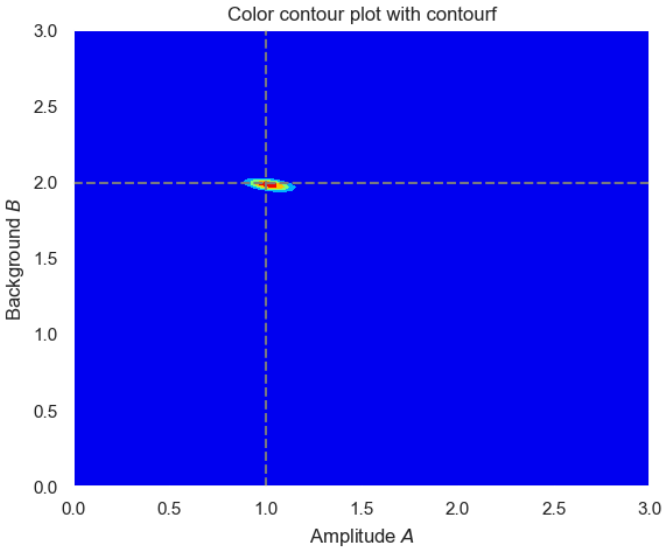


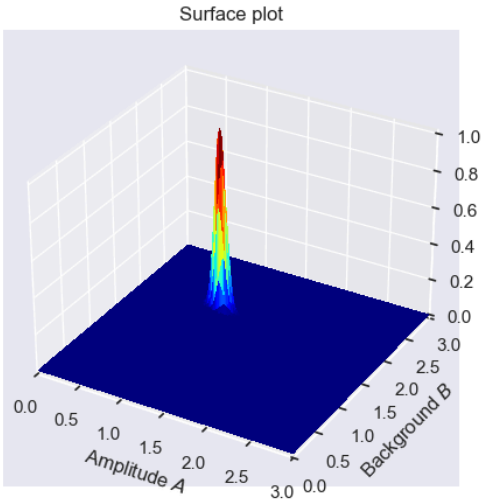


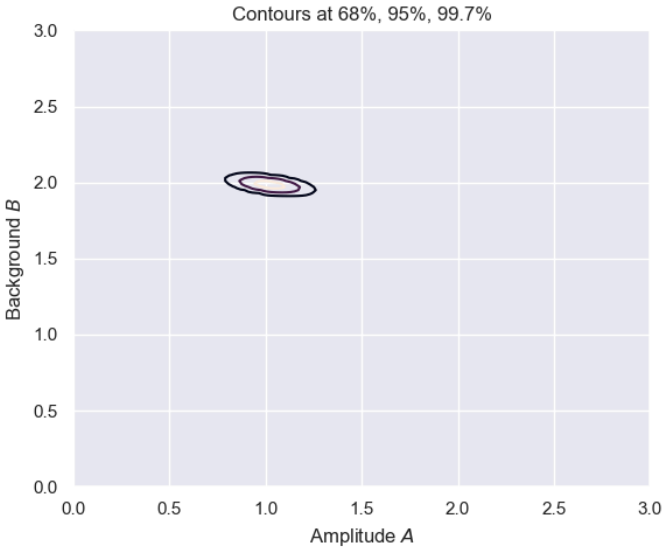

```
# Greater range case: 31 bins (with fixed bin width)
# and same maximum expection of 100 counts as in baseline case.

D_max = 500
databins = 31
delta_x = 1

fig = make_figs(databins, delta_x, D_max, flag=True)
```


Total counts = 11209, # of bins = 31
Number of counts N
Measurement variable x
Background B
Amplitude A
marginalized over B
B = B_true
p(A|{N_k}, I)
Amplitude A
marginalized over A
p(B|{N_k}, I)
Background B



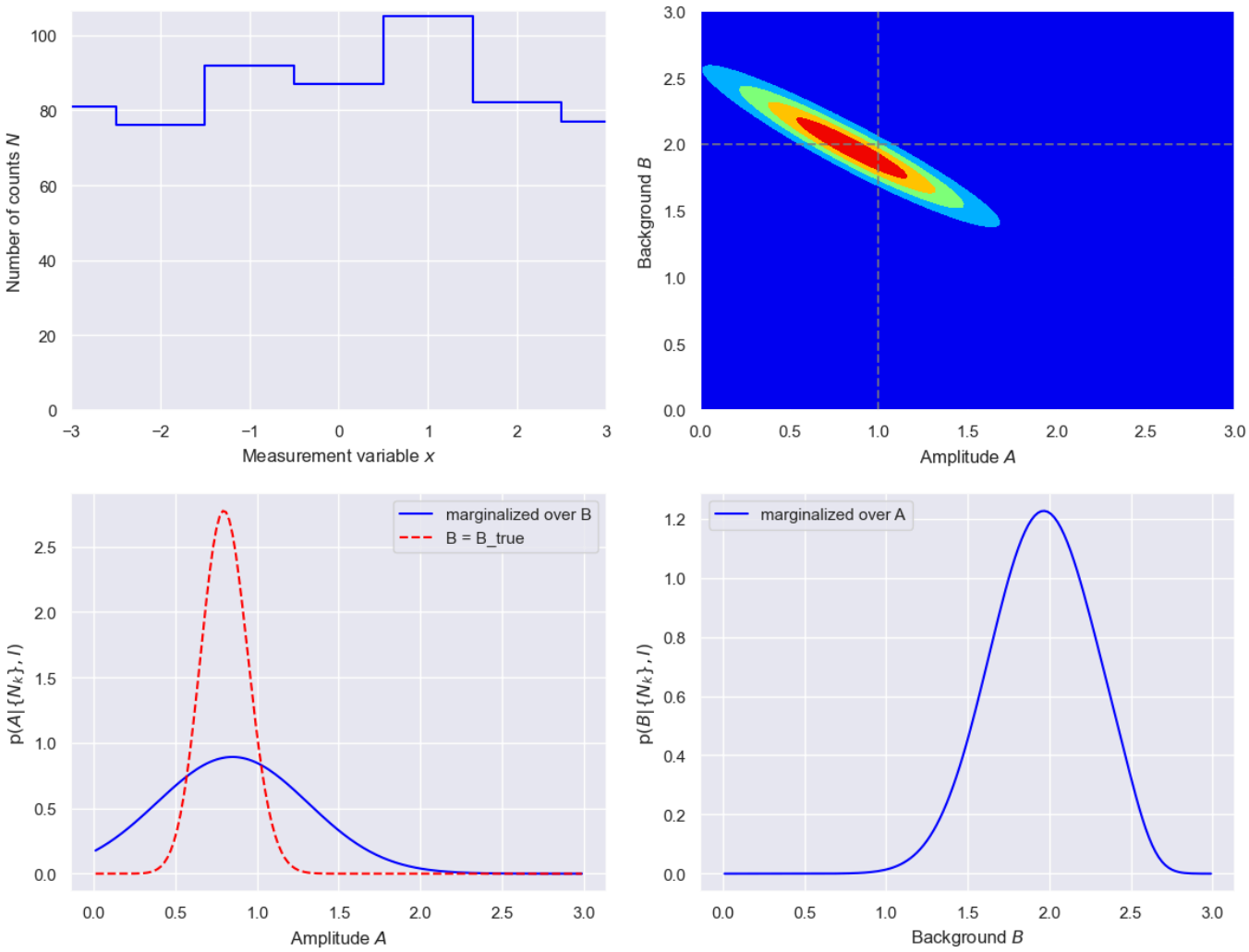


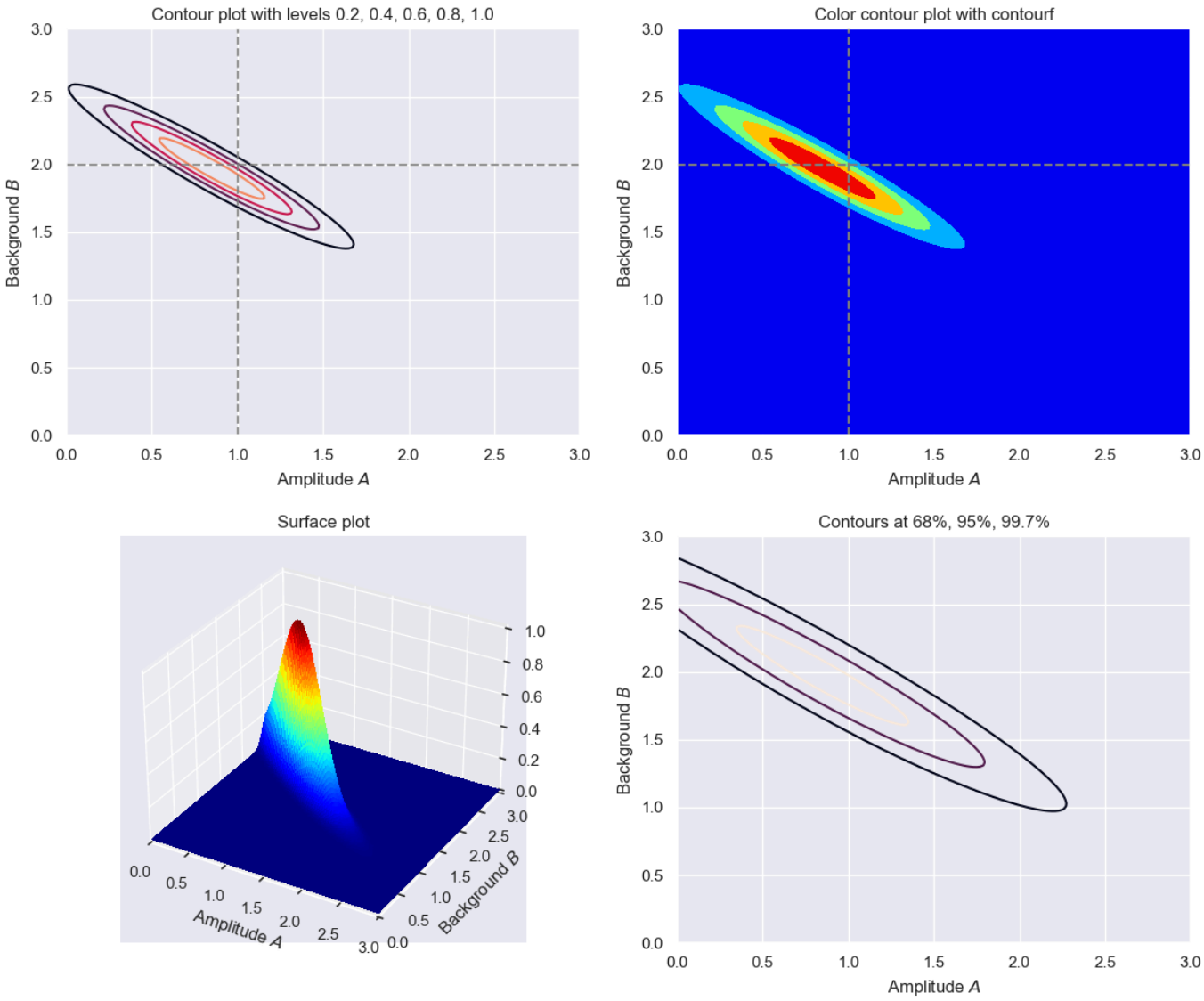





```
# Same as baseline and last case except now only 7 bins.

D_max = 100
databins = 7
delta_x = 1

fig = make_figs(databins, delta_x, D_max, flag=True)
```

Total counts = 600, # of bins = 7





9.5. 📥 Problem: fitting a straight line

Adapted from BayesianAstronomy.

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
import seaborn; seaborn.set('talk') # for plot formatting
```

The Data

Let's start by creating some data that we will fit with a straight line. We'll start with a constant standard deviation of σ on the y values and no error on x.

```
def make_data(intercept, slope, N=20, dy=5, rseed=10):
    """Given a straight line defined by intercept and slope:
          y = slope * x + intercept
       generate N points randomly spaced points from x=0 to x=100
       with Gaussian (i.e., normal) error with mean zero and standard
       deviation dy.

       Return the x and y arrays and an array of standard deviations.
    """
    rand = np.random.RandomState(rseed)
    x = 100 * rand.rand(N)  # choose the x values randomly in [0,100]
    y = intercept + slope * x  # This is the y value without noise
    y += dy * rand.randn(N)    # Add in Gaussian noise
    return x, y, dy * np.ones_like(x)  # return coordinates and error bars

intercept = 25.   # true intercept (called b elsewhere)
slope = 0.5       # true slope (called m elsewhere)
theta_true = [intercept, slope]  # put parameters in a true theta vector
x, y, dy = make_data(*theta_true, rseed=42)

fig, ax = plt.subplots(figsize=(8,8))
ax.errorbar(x, y, dy, fmt='o')
ax.set_xlabel(r'$x$')
ax.set_ylabel(r'$y$')
plot_title = rf'intercept $= {intercept:.1f}$, slope $= {slope:.1f}$, ' \
             + rf' $\sigma = {dy[0]:.1f}$'
ax.set_title(plot_title)
fig.tight_layout()
```

The Model

Next we need to specify a theoretical model. We're fitting a straight line to data, so we'll need a slope and an intercept; i.e.

$$y_{\text{th}}\ (x) = mx + b$$

where our parameter vector will be

$$\boldsymbol{\theta} = [b, m]$$

But this is only half the picture: what we mean by a "model" in a Bayesian sense is not only this expected value $y_{\text{th}}\ (x; \theta)$, but a **probability distribution** for our data. That is, we need an expression to compute the likelihood $p(D \mid \boldsymbol{\theta}, I)$ for our data as a function of the parameters $\boldsymbol{\theta}$ (I stands for all other information). Here D is the set of all (x, y) pairs that we know about (or measure).

[**Note:** At this stage we are (implicitly) assuming that our theoretical model is perfect. But it is not! We'll come back in a future notebook to talk about adding a theory error δy_{th} .]

We are given data with simple error bars, which imply that the probability for any *single* data point (labeled by i) is a normal distribution with mean given by the true value. That is,

$$y_i \sim \mathcal{N}(y_{\text{th}}\ (x_i; \boldsymbol{\theta}), \varepsilon_i^2)$$

or, in other words,

$$p(y_i \mid x_i, \boldsymbol{\theta}, I) = \frac{1}{\sqrt{2\pi\varepsilon_i^2}} \exp\left(\frac{-\left[y_i - y_{\text{th}}\ (x_i; \boldsymbol{\theta})\right]^2}{2\varepsilon_i^2}\right)$$

where ε_i are the (known) measurement errors indicated by the error bars. We can write out the full statistical model as

$$y = y_{\text{th}}\ + \delta y_{\text{th}}\ + \varepsilon,$$

(elsewhere we've used y_{exp} for the left side and recall we are taking δy_{th} to be negligible).

We have assumed that all the points are independent, so we can find the full likelihood by multiplying the individual likelihoods together:

$$p(D \mid \boldsymbol{\theta}, I) = \prod_{i=1}^{N} p(y_i \mid x_i, \boldsymbol{\theta}, I)$$

For convenience (and also for numerical accuracy) this is often expressed in terms of the log-likelihood:

$$\log p(D \mid \boldsymbol{\theta}, I) = -\frac{1}{2} \sum_{i=1}^{N} \left(\log(2\pi\varepsilon_i^2) + \frac{\left[y_i - y_{\text{th}}(x_i; \boldsymbol{\theta})\right]^2}{\varepsilon_i^2}\right)$$

Exercises

1. Write a Python function which computes the log-likelihood given a parameter vector $\boldsymbol{\theta}$, an array of errors ε, and an array of x and y values
2. Use tools in `scipy.optimize` to maximize this likelihood (i.e. minimize the negative log-likelihood). How close is this result to the input `theta_true` above? Is that expected or unexpected?
3. Change the random number seed to get different results and comment on how the maximum likelihood results fluctuate? How are size of the fluctuations related to the number of data points N and the data error standard deviation dy? (Try changing them!)

```
# Either write your code here or study the sample solution that follows.
```

One solution (how could these functions be improved?)

```
def log_likelihood(theta, x, y, dy):
    """
    Return the log of the likelihood function, given a 2D array theta of
    parameters (b, m) and the x, y coordinates with standard deviation dy.
    """
    b, m = theta
    y_th = b + m * x
    return -0.5 * np.sum(np.log(2 * np.pi * dy**2) + (y - y_th)**2 / dy**2)
```

```
from scipy import optimize

def minfunc(theta, x, y, dy):
    """
    Function to be minimized: minus the logarithm of the likelihood.
    """
    return -log_likelihood(theta, x, y, dy)



result = optimize.minimize(minfunc, x0=[0, 0], args=(x, y, dy))

# try without np.printoptions to see what you get
with np.printoptions(precision=3, suppress=True):
    print(f'result: {result.x}')
    print(f'input:  {theta_true}')
```

The Prior

We have computed the likelihood, now we need to think about the prior $p(\delta y_{\text{th}} \mid I)$.

So… what can we actually say about the slope and intercept before we fit our data?

There are several approaches to choosing priors that you'll come across in practice. Here we will just introduce some vocabulary and definitions and come back to details and options later.

1. **Conjugate priors.** A conjugate prior is a prior which, due to its mathematical relation to the likelihood, makes the result analytically computable.
2. **Empirical priors.** Empirical Priors are priors which are actually posteriors from previous studies of the same phenomenon. For example, it's common in Supernova cosmology studies to use the WMAP results as a prior: that is, to actually plug-in a *real result* and use the new data to improve on that. This situation is where Bayesian approaches really shine. For the linear fit, you might imagine that the x, y data is a more accurate version of a previous experiment, where we've found that the intercept is $\theta_0 = 50 \pm 30$ and the slope is $\theta_1 = 1.0 \pm 0.5$. In this case, we'd encode this prior knowledge in the prior distribution itself.
3. **Flat priors.** If you don't have an empirical prior, you might be tempted to simply use a *flat prior* – i.e. a prior that is constant between two reasonable limits (i.e. equal probability slopes from -1000 to +1000). The problem is that flat priors are not always non-informative! For example, a flat prior on the slope will effectively give a higher weight to larger slopes.
4. **Non-informative priors.** What we *really* want in cases where no empirical prior is available is a **non-informative prior**. Among other things, such a prior should not depend on the units of the data. Perhaps the most principled approach to choosing non-informative priors was the *principle of maximum entropy* advocated

by Jaynes (book). Similar in spirit is the commonly-used Jeffreys Prior, which in many cases of interest amounts to a "scale invariant" prior: a flat prior on the logarithm of the parameter. In the case of the linear slope, we often want a prior which does not artificially over-weight large slopes: there are a couple possible approaches to this (see http://arxiv.org/abs/1411.5018 for some discussion). For our situation, we use

$$p(m, b) \propto (1 + m^2)^{-3/2},$$

which, for lack of a better term, we will call a "symmetric prior" (because it's the same whether we're fitting $y = mx + b$ or $x = m'y + b'$, which is the requirement leading to the functional form given). This corresponds to a uniform prior in the variables $(\theta, b\cos\theta)$, where $\theta = \tan^{-1} m$ (calculate the Jacobian to check this).

5. **Weakly informative priors.** See Prior Choice Recommendations at the Stan github repository for a hierarchy of increasingly informative priors.

```
# see what happens if we sample from a uniform prior on the slope m
x_pts = np.linspace(-1, 1, 11)
fig = plt.figure(figsize=(8,8))
ax = fig.add_subplot(1, 1, 1)
ax.axis([-1., 1., -1., 1.])
for slope in np.linspace(0, 20, 100):   # 20 evenly spaced slopes m
    ax.plot(x_pts, slope * x_pts, '-k', linewidth=1)
ax.set_xlabel(r'$x$')
ax.set_ylabel(r'$y$')
ax.set_title(r'Uniform sampling of slopes $m$ in $y= mx + b$')
ax.set_aspect(1)
```

Implementation

Let's define two python functions to compute the options for our prior: we'll use both a (log) flat prior and a (log) symmetric prior. In general, we need not worry about the normalization of the prior or the likelihood, which makes our lives easier:

```
# Either write your code here or study the sample solution that follows.
```

One solution

Where would we edit the functions to produce properly normalized priors?

```
def log_flat_prior(theta):
    """Flat prior on value of slope m"""
    if np.all(np.abs(theta) < 1000):
        return 0 # log(1)
    else:
        return -np.inf  # log(0)

def log_symmetric_prior(theta):
    """Prior p(m) proportional to (1 + m^2)^{-3/2}"""
    if np.abs(theta[0]) < 1000:
        return -1.5 * np.log(1 + theta[1]**2)
    else:
        return -np.inf  # log(0)
```

With these defined, we now have what we need to compute the log posterior as a function of the model parameters. *You might be tempted to maximize this posterior in the same way that we did with the likelihood above, but this is not a Bayesian result!* The Bayesian result is a (possibly marginalized) posterior probability for our parameters. The mode of a probability distribution is perhaps slightly informative, but it is in no way a Bayesian result.

Next you will plot the posterior probability as a function of the slope and intercept.

Things to do:

1. Plot the posterior probability distribution for the slope and intercept, once for each prior. You might use `plt.contourf()` or `plt.pcolor()`. How different are the two distributions?
2. Modify the dataset – how do your conclusions about the sensitivity to choice of prior change if you have very few data points or very large errors?
3. If you finish this quickly, try adding 1-sigma and 2-sigma contours to your plot, keeping in mind that the probabilities are not normalized. You can add them to your plot with `plt.contour()`. Note: this is non-trivial, but there is a short solution possible.

```
# Either write your code here or study the sample solution that follows.
```

One solution (how could this solution be improved?)

We'll start by defining a function which takes a two-dimensional grid of likelihoods and returns 1, 2, and 3-sigma contours. This acts by sorting and normalizing the values and then finding the locations of the 0.682 , 0.952 , and 0.9972 cutoffs:

```
def contour_levels(grid):
    """Compute 68%, 95%, 99.7% credible-region contour levels for a gridded 2D posterior"""
    sorted_ = np.sort(grid.ravel())[::-1]
    pct = np.cumsum(sorted_) / np.sum(sorted_)
    cutoffs = np.searchsorted(pct, np.array([0.68, 0.95, 0.997]))
    return np.sort(sorted_[cutoffs])
```

Now we define a function to compute and plot the results of the Bayesian analysis:

```
def plot_results(x, y, dy,
                 slope_limits=(0.3, 0.7),
                 intercept_limits=(15, 35)):
    # 1. Evaluate the log probability on the grid (once for each prior)
    slope_range = np.linspace(*slope_limits)
    intercept_range = np.linspace(*intercept_limits)

    log_P1 = [[log_likelihood([b, m], x, y, dy) + log_flat_prior([b, m])
               for m in slope_range] for b in intercept_range]
    log_P2 = [[log_likelihood([b, m], x, y, dy) + log_symmetric_prior([b, m])
               for m in slope_range] for b in intercept_range]

    # For convenience, we normalize the pdf such that the maximum is 1
    P1 = np.exp(log_P1 - np.max(log_P1))
    P2 = np.exp(log_P2 - np.max(log_P2))

    # 2. Create two subplots and plot contours showing the results
    fig, ax = plt.subplots(1, 2, figsize=(16, 6),
                           sharex=True, sharey=True)

    ax[0].contourf(slope_range, intercept_range, P1, 100, cmap='Blues')
    ax[0].contour(slope_range, intercept_range, P1, contour_levels(P1),
                  colors='black')
    ax[0].set_title('Flat Prior')

    ax[1].contourf(slope_range, intercept_range, P2, 100, cmap='Blues')
    ax[1].contour(slope_range, intercept_range, P2, contour_levels(P2),
                  colors='black')
    ax[1].set_title('Symmetric Prior')

    # 3. Add grids and set axis labels
    for axi in ax:
        axi.grid('on', linestyle=':', color='gray', alpha=0.5)
        axi.set_axisbelow(False)
        axi.set_xlabel('slope')
        axi.set_ylabel('intercept')
```

```
plot_results(x, y, dy)
```

What do you conclude about how the form of the prior affects the final posterior in this case?

A general observation that often ends up being the case: for all the worrying about the effect of the prior, when you have enough data to constrain your model well, the prior has very little effect.

Let's use some different data and see what happens:

```
x2, y2, dy2 = make_data(*theta_true, N=3, dy=40, rseed=80)
plt.errorbar(x2, y2, dy2, fmt='o');
```

```
plot_results(x2, y2, dy2,
             slope_limits=(-2, 5),
             intercept_limits=(-300, 200))
```

Why in this case does the form of the prior have a clear effect?

This encodes exactly what you would scientifically expect: if you don't have very good data, it is unlikely to change your views of the world (which are of course encoded in the prior).

9.6. 📥 Problem: fitting a straight line II

Bayesian handling of nuisance parameters

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
```

The Data and the question

Let's start by defining some data that we will fit with a straight line. The following data is measured velocities and distances for a set of galaxies. We will assume that there is a constant standard deviation of $\sigma = 200$ km/sec on the y values and no error on x.

```
# Data from student lab observations;
# d0 = Galaxy distances in MPc
# v0 = Galaxy velocity in km/sec
d0 = np.array([6.75, 25, 33.8, 9.36, 21.8, 5.58, 8.52, 15.1])
v0 = np.array([462, 2562, 2130, 750, 2228, 598, 224, 971])
# Assumed exp. uncertainty
err_v0 = 200
```

```
x=d0; y=v0; dy=err_v0
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(1,1,1)
ax.errorbar(x, y, dy, fmt='o')
ax.set_xlabel(r'distance [MPc]')
ax.set_ylabel(r'velocity [km/sec]')
fig.tight_layout()
```

The question that we will be asking is:

> What value would you infer for the Hubble constant given this data?

We will make the prior assumption that the data can be fitted with a straight line (see also the 📥 Problem: fitting a straight line notebook). But we note that we are actually not interested in the offset of the straight line, but just its slope.

We will try three different approaches:

- Maximum likelihood estimate

- Single-parameter inference
- Full Bayesian analysis

As a final part of this notebook, we will also explore how the posterior belief from this analysis can feed into a second data analysis.

The Model

We follow the procedure outlined in 📥 Problem: fitting a straight line.

Thus, we're fitting a straight line to data,

$$y_M(x) = mx + b$$

where our parameter vector will be

$$\theta = [b, m].$$

But this is only half the picture: what we mean by a "model" in a Bayesian sense is not only this expected value $y_M(x;\theta)$, but a **probability distribution** for our data. That is, we need an expression to compute the likelihood $\mathrm{p}\left(D \mid \theta\right)$ for our data as a function of the parameters θ.

Here we are given data with simple error bars, which imply that the probability for any *single* data point is a normal distribution about the true value. That is,

$$y_i \sim \mathcal{N}(y_M(x_i;\theta), \sigma)$$

or, in other words,

$$\mathrm{p}\left(y_i \mid x_i, \theta\right) = \frac{1}{\sqrt{2\pi\varepsilon_i^2}} \exp\left(\frac{-\left[y_i - y_M(x_i;\theta)\right]^2}{2\varepsilon_i^2}\right)$$

where ε_i are the (known) measurement errors indicated by the error bars.

Assuming all the points are independent, we can find the full likelihood by multiplying the individual likelihoods together:

$$\mathrm{p}\left(D \mid \theta\right) = \prod_{i=1}^{N} \mathrm{p}\left(x_i, y_i \mid \theta\right)$$

For convenience (and also for numerical accuracy) this is often expressed in terms of the log-likelihood:

$$\log \mathrm{p}\left(D \mid \theta\right) = -\frac{1}{2}\sum_{i=1}^{N}\left(\log(2\pi\varepsilon_i^2) + \frac{\left[y_i - y_M(x_i;\theta)\right]^2}{\varepsilon_i^2}\right)$$

Step 1: Maximum likelihood estimate

```
# Log likelihood
def log_likelihood(theta, x, y, dy):
    y_model = theta[0] + theta[1] * x
    return -0.5 * np.sum(np.log(2 * np.pi * dy ** 2) +
                         (y - y_model) ** 2 / dy ** 2)
```

Use tools in `scipy.optimize` to maximize this likelihood (i.e. minimize the negative log-likelihood).

```
from scipy import optimize

def minfunc(theta, x, y, dy):
    """
    Function to be minimized: minus the logarithm of the likelihood
    """
    return -log_likelihood(theta, x, y, dy)

result = optimize.minimize(minfunc, x0=[0, 0], args=(x, y, dy))
```

The output from 'scipy.optimize' contains the set of parameters, and also the inverse of the hessian matrix (that measures the second-order curvature of the optimum). The inverse hessian is related to the covariance matrix. Very often you see the square root of the diagonal elements of this matrix as uncertainty estimates. We will not discuss this measure here, but refer to the highly recommended review: Error estimates of theoretical models: a guide.

```
# Print the MLE and the square-root of the diagonal elements of the inverse hessian
print(f'Maximum Likelihood Estimate (MLE):')
ndim = len(result.x)
theta_MLE=result.x
err_theta_MLE = np.array([np.sqrt(result.hess_inv[i,i]) for i in range(ndim)])
for i in range(ndim):
    print(f'... theta[{i}] = {theta_MLE[i]:>5.1f} +/- {err_theta_MLE[i]:>5.1f}')
```

Step 2: Single-parameter model

As we are not interested in the offset parameter, we might be tempted to fix its value to the most-likely estimate and then infer our knowledge about the slope from a single-parameter model. You have probably realized by now that this is not the Bayesian way of doing the analysis, but since this is a rather common way of handling nuisance parameters, we will still try it.

```
offset = theta_MLE[0]
```

Let's define the log-likelihood for the case that the offset is fixed. It will be a function of a single free parameter: the slope.

```
# Log likelihood
def log_likelihood_single(slope, x, y, dy, offset=0.):
    y_model = offset + slope * x
    return -0.5 * np.sum(np.log(2 * np.pi * dy ** 2) +
                         (y - y_model) ** 2 / dy ** 2)
```

Next we will plot the log-likelihood (left panel) and the likelihood (right panel) pdfs as a function of the slope. We normalize the peak of the likelihood to one

```
slope_range = np.linspace(60, 100, num=1000)
log_P1 = [log_likelihood_single(slope, x, y, dy,offset=offset) for slope in slope_range]
log_P1_1 = log_P1 - np.max(log_P1)

fig,ax = plt.subplots(1, 2, figsize=(12,6),sharex=True)

ax[0].plot(slope_range,log_P1_1,'-k');
ax[0].set_title('log likelihood')
ax[0].set_xlabel('slope')
ax[1].plot(slope_range,np.exp(log_P1_1),'-k');
ax[1].set_title('likelihood')
ax[1].set_xlabel('slope')
```

```
def contour_levels(grid,sigma):
    _sorted = np.sort(grid.ravel())[::-1]
    pct = np.cumsum(_sorted) / np.sum(_sorted)
    cutoffs = np.searchsorted(pct, np.array(sigma) )
    return _sorted[cutoffs]

P1 = np.exp(log_P1 - np.max(log_P1))
sigma_contours = contour_levels(P1,0.68)

# Find the max likelihood and the 68% contours
slope_max = slope_range[P1==1.][0]
err_slope_min = np.min(slope_range[P1>sigma_contours])
err_slope_max = np.max(slope_range[P1>sigma_contours])

# The error will be symmetric around the max
err_slope = (err_slope_max - err_slope_min) / 2

fig, ax = plt.subplots(1, 1, figsize=(12, 8))
ax.plot(slope_range,P1,'-k')
ax.set_xlabel('slope')
ax.vlines([err_slope_min,slope_max,err_slope_max],0,[sigma_contours,1.,sigma_contours])
ax.fill_between(slope_range, P1, where=P1>=sigma_contours, interpolate=True, alpha=0.2);

print('Single parameter estimate')
print(f'... slope = {slope_max:>5.1f} +/- {err_slope:>5.1f}')
```

Step 3: Full Bayesian approach

You might not be surprised to learn that we underestimate the uncertainty of the slope since we are making the assumption that we know the value of the offset (by fixing it to a specific estimate).

We will now repeat the data analysis, but with the full model and with marginalization on the posterior.

Let's use the (log) symmetric prior, which is the scale-invariant one.

```
def log_prior(theta):
    # symmetric prior for the slope, and normal pdf (mean=0, variance=200) for the intercept
    return - 0.5* theta[0]**2 / dy**2 - 1.5 * np.log(1 + theta[1] ** 2)
```

With these defined, we now have what we need to compute the log posterior as a function of the model parameters.

```
def log_posterior(theta, x, y, dy):
    return log_prior(theta) + log_likelihood(theta, x, y, dy)
```

Next we will plot the posterior probability as a function of the slope and intercept.

We will illustrate the use of MCMC sampling for obtaining the posterior pdf, which also offers a very convenient way of performing the marginalization

```
import emcee
print('emcee sampling (version: )', emcee.__version__)

ndim = 2  # number of parameters in the model
nwalkers = 50  # number of MCMC walkers
nsteps = 2000 # steps per walker

print(f'{nwalkers} walkers: {nsteps} samples each')

# initialize walkers
starting_guesses = np.random.randn(nwalkers, ndim)

sampler = emcee.EnsembleSampler(nwalkers, ndim, log_posterior, args=[x, y, dy])
%time sampler.run_mcmc(starting_guesses, nsteps)
print("done")
```

```
# sampler.get_chain() is of shape (nsteps, nwalkers, ndim)
# Let us flatten all walker chains together, giving shape (nsteps*nwalkers, ndim)
# Then make a scatter plot
emcee_trace = sampler.get_chain(flat=True).T
fig = plt.figure(figsize=(8,8))
ax = fig.add_subplot(1,1,1)
ax.plot(emcee_trace[0], emcee_trace[1], ',k', alpha=0.1);
```

Our choice of starting points were not optimal. It takes some time for the MCMC chains to converge. Let us study the traces.

```
fig, ax = plt.subplots(ndim, sharex=True,figsize=(10,6))
for i in range(ndim):
    # one line per walker; get_chain() has shape (nsteps, nwalkers, ndim)
    ax[i].plot(sampler.get_chain()[:, :, i], '-k', alpha=0.2);
```

```
# We choose a warm-up time
nwarmup = 200 # warm up

# sampler.get_chain() is of shape (nsteps, nwalkers, ndim)
# we'll throw-out the warmup points and flatten the walkers.
# Note: the samples and the log-probabilities must be extracted in the same way
# so that they stay matched element by element.
emcee_trace = sampler.get_chain(discard=nwarmup, flat=True).T
emcee_lnprob = sampler.get_log_prob(discard=nwarmup, flat=True)
```

Let us create some convenience tools for plotting, including a machinery to extract 1-, 2-, 3-sigma contour levels.

We will later use the ‘corner’ package to achieve such visualization.

```
def compute_sigma_level(trace1, trace2, nbins=20):
    """From a set of traces, bin by number of standard deviations"""
    L, xbins, ybins = np.histogram2d(trace1, trace2, nbins)
    L[L == 0] = 1E-16
    logL = np.log(L)

    shape = L.shape
    L = L.ravel()

    # obtain the indices to sort and unsort the flattened array
    i_sort = np.argsort(L)[::-1]
    i_unsort = np.argsort(i_sort)

    L_cumsum = L[i_sort].cumsum()
    L_cumsum /= L_cumsum[-1]

    xbins = 0.5 * (xbins[1:] + xbins[:-1])
    ybins = 0.5 * (ybins[1:] + ybins[:-1])

    return xbins, ybins, L_cumsum[i_unsort].reshape(shape)

def plot_MCMC_trace(ax, xdata, ydata, trace, scatter=False, **kwargs):
    """Plot traces and contours"""
    xbins, ybins, sigma = compute_sigma_level(trace[0], trace[1])
    ax.contour(xbins, ybins, sigma.T, levels=[0.683, 0.955, 0.997], **kwargs)
    if scatter:
        ax.plot(trace[0], trace[1], ',k', alpha=0.1)
    ax.set_xlabel(r'$\theta_0$')
    ax.set_ylabel(r'$\theta_1$')

# Convenience function to extract the peak position of the mode
def max_of_mode(sampler_object):
    max_arg = np.argmax(sampler_object.get_log_prob(flat=True))
    return(sampler_object.get_chain(flat=True)[max_arg])
```

```
fig = plt.figure(figsize=(8,8))
ax = fig.add_subplot(1,1,1)
plot_MCMC_trace(ax, x, y, emcee_trace, scatter=True,colors='k');
max_mode_theta=max_of_mode(sampler)
with np.printoptions(precision=3):
    print(f'Max posterior is at: {max_mode_theta}')
```

Marginalization

Next we will perform the marginalization over the offset parameter using the samples from the posterior pdf.

Furthermore, the extraction of a 68% credible region (not to be confused with the frequentist *confidence interval*) is made simple since the posterior is well described by a single mode.

```
# Sort the samples according to the log-probability.
# Note that we want to have the sorted by increasing -log(p)
sorted_lnprob = -np.sort(-emcee_lnprob)
# In this sorted list we then keep 1-sigma volume of the samples
# (note 1-sigma = 1-exp(-0.5) ~ 0.393 for 2-dim pdf). See
# https://corner.readthedocs.io/en/latest/pages/sigmas.html
# We then identify what log-prob this corresponds to
log_prob_max = sorted_lnprob[0]
level_1sigma = 1-np.exp(-0.5)
log_prob_cutoff = sorted_lnprob[int(level_1sigma*nwalkers*(nsteps-nwarmup))]
# From the list of samples that have log-prob larger than this cutoff,
# we then find the smallest and largest value for the slope parameter.
# Here we simply ignore the values for the offset parameter (this is marginalization when
having MCMC samples).

slope_samples = emcee_trace[1,:]
# Mode
bayesian_slope_maxprob = slope_samples[emcee_lnprob==log_prob_max][0]
# Mean
bayesian_slope_mean = np.sort(slope_samples)[int(0.5*nwalkers*(nsteps-nwarmup))]
# 68% CR
bayesian_CR_slope_min = np.min(slope_samples[emcee_lnprob>log_prob_cutoff])
bayesian_CR_slope_max = np.max(slope_samples[emcee_lnprob>log_prob_cutoff])
```

```
print('Bayesian slope parameter estimate')
print(f'... slope = {bayesian_slope_mean:>6.2f} ',\
      f'(-{bayesian_slope_mean-bayesian_CR_slope_min:>4.2f},',\
      f'+{bayesian_CR_slope_max-bayesian_slope_mean:>4.2f})')
```

```
# Alternatively we can use corner
import corner
fig, ax = plt.subplots(2,2, figsize=(10,10))
corner.corner(emcee_trace.T,labels=[r"$\theta_0$", r"$\theta_1$"],
                      quantiles=[0.16, 0.5, 0.84],fig=fig,show_titles=True
                      );
```

We can use the parameter samples to create corresponding samples of our model. Finally, we plot the mean and 1-sigma contours of these samples.

```
def plot_MCMC_model(ax, xdata, ydata, trace, yerr=0):
    """Plot the linear model and 2sigma contours"""
    ax.errorbar(xdata, ydata, yerr, fmt='o')

    alpha, beta = trace[:2]
    xfit = np.linspace(0, 50, 5)
    yfit = alpha[:, None] + beta[:, None] * xfit
    mu = yfit.mean(0)
    sig = yfit.std(0)

    ax.plot(xfit, mu, '-k')
    ax.fill_between(xfit, mu - sig, mu + sig, color='lightgray')

    ax.set_xlabel('x')
    ax.set_ylabel('y')
```

```
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(1,1,1)
plot_MCMC_model(ax,x,y,emcee_trace,dy)
```

Caution:

- Might we have underestimated the error bars on the experimental data?
- And/or is some of the data “bad” (are there outliers)?

More on this later.

Summary

```
print('{0:>25s}: {1:>3.1f} +/- {2:>3.1f}'.format('Max Likelihood (1-sigma)',
theta_MLE[1],err_theta_MLE[1]))
print('{0:>25s}: {1:>3.1f} +/- {2:>3.1f}'.format('Fixed offset (1-sigma)',slope_max,
err_slope))
print('{0:>25s}: {1:>3.1f} (+{2:>3.1f};-{3:>3.1f})'.format('Full Bayesian (68%
CR)',bayesian_slope_mean, bayesian_CR_slope_max-bayesian_slope_mean,bayesian_slope_mean-
bayesian_CR_slope_min))
```

Breakout questions

In the Bayesian analysis we had to specify our prior assumption on the value for the offset.

- *Is this a feature or a deficiency of the Bayesian approach? Discuss!*

What happens if we modify this prior assumption?

- *Redo the analysis with a very broad, uniform prior on the offset. How is the inference affected?*
- *Redo the analysis with a very narrow, normal prior on the offset. How is the inference affected?*

Step 4: Error propagation

The Bayesian approach offers a straight-forward approach for dealing with (known) systematic uncertainties; namely by marginalization.

Systematic error example

The Hubble constant acts as a galactic ruler as it is used to measure astronomical distances according to $v = H_0 x$. An error in this ruler will therefore correspond to a systematic uncertainty in such measurements.

Suppose that a particular galaxy has a measured recessional velocity $v_{\rm measured} = (100 \pm 5) \times 10^3$ km/sec. Also assume that the Hubble constant H_0 is known from the analysis performed above in Step 3. Determine the posterior pdf for the distance to the galaxy assuming:

1. A fixed value of H_0 corresponding to the mean of the previous analysis.
2. Using the sampled posterior pdf for H_0 from the above analysis.

```
vm=100000
sig_vm=5000
```

We assume that we can write

$$v_{\rm measured} = v_{\rm theory} + \delta v_{\rm exp},$$

where $v_{\rm theory}$ is the recessional velocity according to our model, and $\delta v_{\rm exp}$ represents the noise component of the measurement. We know that $\delta v_{\rm exp}$ can be described by a Gaussian pdf with mean 0 and standard deviation $\sigma_v = 5 \times 10^3$ km/sec. Note that we have also assumed that our model is perfect, i.e. $\delta v_{\rm theory}$ is negligible.

In the following, we also assume that the error in the measurement in v is uncorrelated with the uncertainty in H_0.

Through application of Bayes' rule we can readily evaluate the posterior pdf $p(x|D,I)$ for the distance x to the galaxy.

Case 1: Fixed H_0

$$\begin{aligned} p(x|D,I) &\propto p(D|x,I)p(x|I) \\ &= \frac{1}{\sqrt{2\pi}\sigma_v} \exp\left(-\frac{(v_{\rm measured} - v_{\rm theory})^2}{2\sigma_v^2}\right) p(x|I) \\ &= \begin{cases} \frac{1}{\sqrt{2\pi}\sigma_v} \exp\left(-\frac{(v_{\rm measured} - H_0 x)^2}{2\sigma_v^2}\right) & \text{with } \; x \in [x_{\rm min}, x_{\rm max}] \\ 0 & \text{otherwise } \; , \end{cases} \end{aligned} \tag{9.2}$$

where $p(x|I)$ is the prior for the distance, which we have assumed to be uniform, i.e. $p(x|I) \propto 1$ in some (possibly large) region $[x_{\rm min}, x_{\rm max}]$.

```
def x_with_fixedH(x,H0,vmeasured=vm,vsigma=sig_vm,xmin=0,xmax=10000):
    # Not including the prior
    x_posterior = np.exp(-(vmeasured-H0*x)**2/(2*vsigma**2))
    return x_posterior
```

Case 2: Using the inferred pdf for H_0

Here we use marginalization to obtain the desired posterior pdf $p(x|D,I)$ from the joint distribution of $p(x,H_0|D,I)$

$$p(x|D,I) = \int_{-\infty}^{\infty} dH_0 p(x,H_0|D,I).$$

Using Bayes' rule, the product rule, and the fact that H_0 is independent of x we find that

$$p(x|D,I) \propto p(x|I) \int dH_0 p(H_0|I) p(D|x,H_0,I),$$

which means that we have expressed the quantity that we want (the posterior for x) in terms of quantities that we know.

The pdf $p(H_0|I)$ is known via its N samples $\{H_i\}$ generated by the MCMC sampler.

This means that we can approximate

$$p(x|D,I) = \int dH_0 p(H_0|I) p(D|x,H_0,I) \approx \frac{1}{N} \sum_{i=1}^{N} p(D|x,H_0^{(i)},I)$$

where we have used $p(x|I) \propto 1$ and where $H_0^{(i)}$ is drawn from $p(H_0|I)$.

```
x_arr = np.linspace(800,2000,1200)

xposterior_fixedH = x_with_fixedH(x_arr,bayesian_slope_mean)
xposterior_fixedH /= np.sum(xposterior_fixedH)

xposterior_pdfH = np.zeros_like(x_arr)
for H0 in slope_samples:
    xposterior_pdfH += x_with_fixedH(x_arr,H0)
xposterior_pdfH /= np.sum(xposterior_pdfH)
```

```
fig, ax = plt.subplots(1,1, figsize=(8,6))
ax.plot(x_arr,xposterior_fixedH);
ax.plot(x_arr,xposterior_pdfH,'--');

print("The mean and 68% DoB of the inferred distance is")
for ix, xposterior in enumerate([xposterior_fixedH,xposterior_pdfH]):
    # Mean
    x_mean = np.min(x_arr[np.cumsum(xposterior)>0.5])
    # 68% DoB
    x_min = np.min(x_arr[np.cumsum(xposterior)>0.16])
    x_max = np.min(x_arr[np.cumsum(xposterior)>0.84])
    print(f"... Case {ix+1}: x_mean = {x_mean:.0f}; 68% DoB [-{x_mean-x_min:.0f},+{x_max-
x_mean:.0f}]")
```

9.7. 📥 Problem: Gaussian noise and averages II

Adapted from Sivia: *Data Analysis: A Bayesian Tutorial*

Here we'll take a fresh look at a simple parameter-estimation problem using sampling.

This notebook was adapted from a broad introduction to Bayesian statistics put together by Christian Forssen for the 2019 TALENT school on "Learning from Data".

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

# Not really needed, but nicer plots
import seaborn as sns
sns.set()
sns.set_context("talk")
```

```
# Modules needed for example: emcee is for MCMCsampling, corner for plotting
from scipy import stats
import emcee
import corner
```

Let us consider the problem of estimating the mean and the variance of a normal distribution that is associated with a collection of random variables. The normal distribution $$p(x|\mu,\sigma) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right),$$ is often used as a theoretical model to describe the noise associated with experimental data.

Why is a normal (Gaussian) distribution so often a good statistical model?

Let us assume that we have a series of M measurements $D \equiv \{x_k\} = (x_1, \ldots, x_M)$, that are samples from a normal $\mathcal{N}(\mu, \sigma^2)$ population, from which we would like to learn the approximate values of the parameters μ and σ. The standard frequentist approach to this problem is the maximum likelihood method. The Bayesian approach is to compute the posterior distribution for the model parameters μ and σ.

Here we'll use Python to generate some toy data to demonstrate the two approaches to the problem.

```
# Generating some data
np.random.seed(10)  # for repeatability

mu_true = 10   # true peak position
sigma_true = 1 # true standard deviation
M = 100 # number of measurements
D = stats.norm.rvs(mu_true, sigma_true, size=M)  # M measurements (samples)
```

Side note: try shift-tab-tab to get documentation!

Look at the array `D`.

```
D
```

```
array([11.3315865 , 10.71527897,  8.45459971,  9.99161615, 10.62133597,
        9.27991444, 10.26551159, 10.10854853, 10.00429143,  9.82539979,
       10.43302619, 11.20303737,  9.03493433, 11.02827408, 10.22863013,
       10.44513761,  8.86339779, 10.13513688, 11.484537  ,  8.92019511,
        8.02227172,  8.2566277 , 10.26607016, 12.38496733, 11.12369125,
       11.67262221, 10.09914922, 11.39799638,  9.72875201, 10.61320418,
        9.73268281,  9.45069099, 10.1327083 ,  9.52385799, 11.30847308,
       10.19501328, 10.40020999,  9.66236766, 11.25647226,  9.2680305 ,
       10.66023155,  9.64912811,  9.06056664,  9.51066278,  9.19540886,
        9.78730236,  9.66085975, 10.31216994, 10.56515267,  9.85257974,
        9.97409466, 10.2890942 ,  9.46012093, 10.70816002, 10.84222474,
       10.2035808 , 12.39470366, 10.91745894,  9.88772753,  9.63781955,
        9.76781774,  9.4982711 , 11.12878515,  9.30218997,  9.91887782,
        9.47070392, 11.04618286,  8.58144397,  9.63750082,  9.87809431,
       10.31935642, 10.4609029 ,  9.78421011, 10.98907246, 10.31475378,
       12.46765106,  8.49167851, 10.62060066,  8.95486746,  9.20199118,
       11.98508459, 11.74481415,  8.14381452,  9.7772263 ,  9.93415215,
        7.86828789,  9.95116949, 10.39334122, 10.21726515,  8.00560623,
       11.10770823, 10.24454398,  9.93808797,  9.24610704, 10.71195902,
       10.91826915,  9.51790686, 10.08958761, 10.82699862,  8.04548788])
```

Frequentist approach to Gaussian parameter estimation

We'll start with the classical frequentist **maximum likelihood** approach. The probability of a single measurement D_i having a value x_i is given by

$$p(x_i \mid \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[\frac{-(x_i - \mu)^2}{2\sigma^2}\right],$$

where μ, σ are the *true* values for the model parameters.

We construct the **likelihood function** by computing the product of the probabilities for each data point:

$$\mathcal{L}(D \mid \mu, \sigma) = \prod_{i=1}^{M} p(x_i \mid \mu, \sigma)$$

Because the value of the likelihood can become very small, it is often more convenient to instead compute the log-likelihood. Furthermore, when we are looking for the maximum of the likelihood, we might just as well maximize the log-likelihood. Combining the previous two equations and computing the log, we have

$$\log \mathcal{L} = -\frac{1}{2} \sum_{i=1}^{M} \left[\log(2\pi\sigma^2) + \frac{(x_i - \mu)^2}{\sigma^2} \right]$$

What we'd like to do is find μ_0, σ_0 such that the likelihood (or log likelihood) is maximized. For this simple problem, the maximization can be computed analytically (i.e. by setting $\partial \log \mathcal{L}/\partial\mu|_{\mu_0,\sigma_0} = \partial \log \mathcal{L}/\partial\sigma|_{\mu_0,\sigma_0} = 0$). This results in the following *maximum-likelihood estimates* of the true parameters:

$$\mu_0 = \frac{1}{M} \sum_{i=1}^{M} x_i$$

$$\sigma_0^2 = \frac{1}{M} \sum_{i=1}^{M} (x_i - \mu_0)^2$$

In principle, we should also compute the second derivatives to make sure that this point represents a maximum rather than a minimum or a saddle point. However, in agreement with intuition, μ_0 is simply the mean of the observed data. These results are fairly simple calculations; let's evaluate them for our toy dataset:

```
# Estimators: sample mean and (sqrt of) sample variance
# Sometimes the *unbiased* estimator for the sample variance is used
# with (M-1) degrees of freedom...
mu_est = D.sum()/M
sigma_est = np.sqrt(((D-mu_est)**2).sum()/M)
print("""
      (mu,sigma)_true = {0:.2f}, {1:.2f}

      Sample estimators:
      (mu,sigma)_0  = {2:.2f}, {3:.2f} (based on {4} measurements)
      """.format(mu_true, sigma_true, mu_est, sigma_est, M))
```

```
      (mu,sigma)_true = 10.00, 1.00

      Sample estimators:
      (mu,sigma)_0  = 10.08, 0.97 (based on 100 measurements)
```

Bayesian approach to Gaussian parameter estimation

The Bayesian approach begins and ends with probabilities (pdfs). It recognizes that what we fundamentally want to compute is our knowledge of the parameters in question, i.e. in this case,

$$p(\mu, \sigma \mid D, I)$$

Note that this formulation of the problem is *fundamentally contrary* to the frequentist philosophy, which says that *probabilities have no meaning for model parameters* like μ, σ. Nevertheless, within the Bayesian philosophy this is perfectly acceptable.

To compute this result, Bayesians next apply Bayes' Theorem, here with the hypothesis being the Gaussian model expressed in terms of the model parameters

$$p(\mu, \sigma \mid D, I) = \frac{p(D \mid \mu, \sigma, I)\, p(\mu, \sigma \mid I)}{p(D \mid I)}$$

Though Bayes' theorem is where Bayesians get their name, it is not this law itself that is controversial, but the Bayesian *interpretation of probability* implied by the term $p(\mu, \sigma \mid D, I)$.

Let's take a look at each of the terms in this expression:

- $p(\mu, \sigma \mid D, I)$: The **posterior**, or the probability of the model parameters given the data: this is the result we want to compute.
- $p(D \mid \mu, \sigma, I)$: The **likelihood**, which is proportional to the $\mathcal{L}(D \mid \mu, \sigma)$ in the frequentist approach, above.
- $p(\mu, \sigma \mid I)$: The **model prior**, which encodes what we knew about the model prior to the application of the data D.
- $p(D \mid I)$: The **data probability**, which in practice amounts to simply a normalization term.

If we set the prior $p(\mu, \sigma \mid I) \propto 1$ (a *flat prior*), we find

$$p(\mu, \sigma \mid D, I) \propto \mathcal{L}(D|\mu, \sigma)$$

and the Bayesian probability is maximized at precisely the same value as the frequentist result! So despite the philosophical differences, we see that (for this simple problem at least) the Bayesian and frequentist point estimates are equivalent.

But what about the prior?

You'll noticed that we glossed over something here: the prior, $p(\mu, \sigma)$. The choice of prior will be discussed repeatedly in the course, but we can already note that it allows inclusion of other information into the computation. This feature becomes very useful in cases where multiple measurement strategies are being combined to constrain a single model. The necessity to specify a prior, however, is one of the more controversial pieces of Bayesian analysis.

A frequentist will point out that the prior is problematic when no true prior information is available. Though it might seem straightforward to use a **noninformative prior** like the flat prior mentioned above, there are some surprisingly subtleties involved. It turns out that in many situations, a truly noninformative prior does not exist! Frequentists point out that the subjective choice of a prior which necessarily biases your result has no place in statistical data analysis.

A Bayesian would counter that frequentism doesn't solve this problem, but simply skirts the question. Frequentism can often be viewed as simply a special case of the Bayesian approach for some (implicit) choice of the prior: a Bayesian would say that it's better to make this implicit choice explicit, even if the choice might include some subjectivity.

Applying the Bayesian approach

Leaving these philosophical debates aside for the time being, let's address how Bayesian results are generally computed in practice. For a one parameter problem like the one considered here, it's as simple as computing the posterior probability $p(\mu, \sigma \mid D, I)$ as a function of μ, σ: this is the distribution reflecting our knowledge of the parameter μ, σ. But as the dimension of the model grows, this direct approach becomes increasingly intractable. For this reason, Bayesian calculations often depend on sampling methods such as Markov Chain Monte Carlo (MCMC).

We won't go into the details of the theory of MCMC here (again, we will be discussing this topic during the course). Instead we will show a practical example of applying an MCMC approach using Dan Foreman-Mackey's excellent emcee package. Keep in mind here that the goal is to *generate a set of points drawn from the posterior probability distribution*, and to use those points to determine the answer we seek.

To perform this MCMC, we start by defining Python functions for the prior $p(\mu, \sigma \mid I)$, the likelihood $p(D \mid \mu, \sigma, I)$, and the posterior $p(\mu, \sigma \mid D, I)$, noting that none of these need be properly normalized. Our model here is two-dimensional so we'll define the model in terms of an array of parameters θ, which in this case is $\theta = [\mu, \sigma]$:

```
min_theta=np.array([0,0])
max_theta=np.array([100,10])
volume_theta=np.prod(max_theta-min_theta)

def log_prior(theta):
    '''Log prior for parameter array theta'''
    assert len(theta)==2, "Parameter vector must have length two."
    # flat prior
    if np.logical_and(min_theta<=theta, theta<=max_theta).all():
        return np.log(1/volume_theta)
    else:
        return -np.inf

def log_likelihood(theta, X):
    '''Log likelihood for data X given parameter array theta'''
    try:
        return -0.5 * np.sum( ( (X - theta[0]) / theta[1] )** 2 ) \
               - 0.5*len(X)*np.log(2*np.pi*theta[1]**2)
    except ValueError:
        return -np.inf

def log_posterior(theta, X):
    '''Log posterior for data X given parameter array theta'''
    return log_prior(theta) + log_likelihood(theta, X)
```

Now we set up the problem, including generating some random starting guesses for the multiple chains of points.

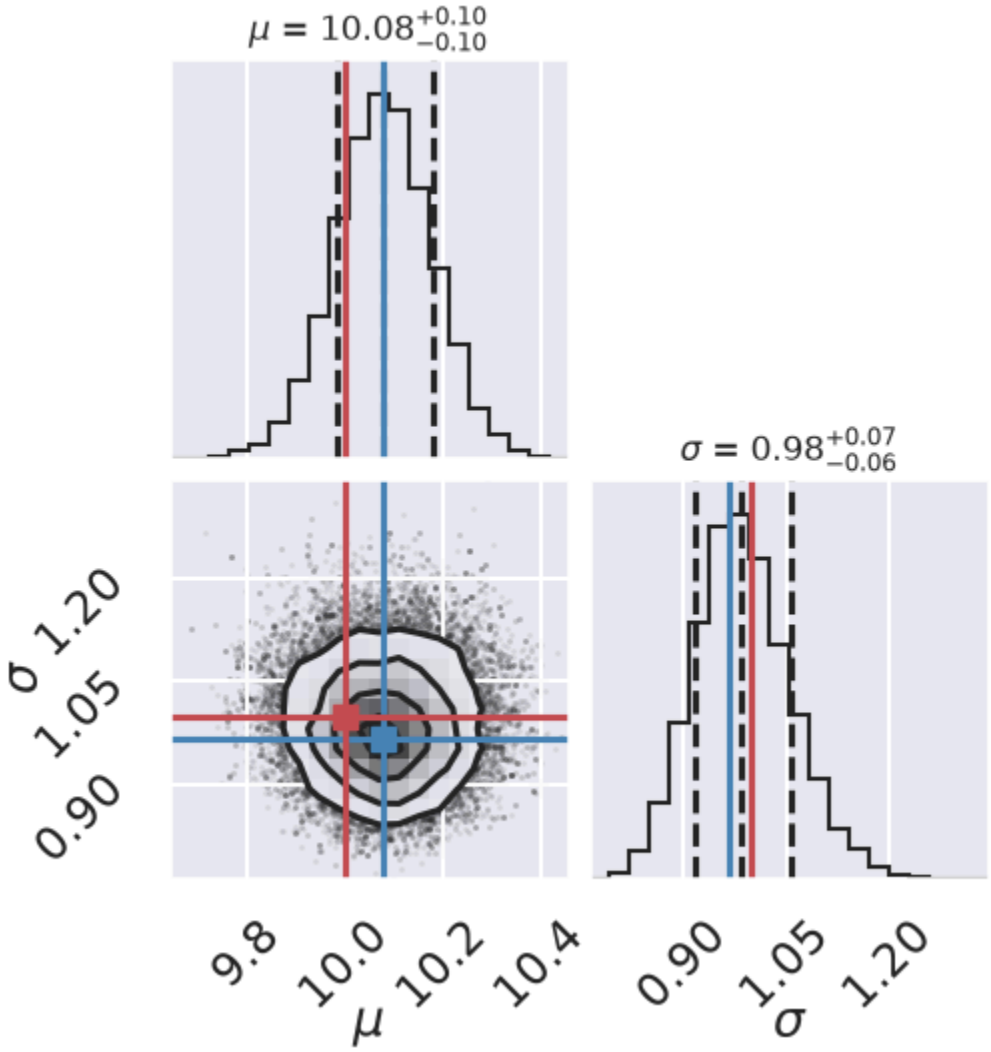
```
ndim = 2  # number of parameters in the model
nwalkers = 50  # number of MCMC walkers
nburn = 1000  # "burn-in" period to let chains stabilize
nsteps = 1000  # number of MCMC steps to take

# we'll start at random locations within the prior volume
starting_guesses = min_theta + \
                   (max_theta - min_theta) * np.random.rand(nwalkers,ndim)

print("MCMC sampling using emcee (affine-invariant ensamble sampler) with {0}
walkers".format(nwalkers))
sampler = emcee.EnsembleSampler(nwalkers, ndim, log_posterior, args=[D])

# "burn-in" period; save final state and then reset
state = sampler.run_mcmc(starting_guesses, nburn)
sampler.reset()

# sampling period
sampler.run_mcmc(state, nsteps)

print("Mean acceptance fraction: {0:.3f} (in total {1} steps)"
                .format(np.mean(sampler.acceptance_fraction),nwalkers*nsteps))

# discard burn-in points and flatten the walkers; the shape of samples is (nwalkers*nsteps,
ndim)
samples = sampler.get_chain(flat=True)
```

```
MCMC sampling using emcee (affine-invariant ensamble sampler) with 50 walkers
```

```
Mean acceptance fraction: 0.719 (in total 50000 steps)
```

If this all worked correctly, the array *samples* should contain a series of draws from the posterior. Let's plot them using a so called corner plot (to be discussed in much more detail during the course).

```
# make a corner plot with the posterior distribution
fig = corner.corner(samples, labels=["$\mu$", "$\sigma$"],
                    truths=[mu_est, sigma_est],quantiles=[0.16, 0.5, 0.84],
                    show_titles=True, title_kwargs={"fontsize": 12})

# With some manual efforts, we can add the maximum-likelihood estimate from
#  the frequentist analysis.
maxlike_results = (mu_true,sigma_true)

# First, extract the axes
axes = np.array(fig.axes).reshape((ndim, ndim))

# Then, loop over the diagonal
for i in range(ndim):
    ax = axes[i, i]
    ax.axvline(maxlike_results[i], color="r")

# And finally, loop over the histograms
for yi in range(ndim):
    for xi in range(yi):
        ax = axes[yi, xi]
        ax.axvline(maxlike_results[xi], color="r")
        ax.axhline(maxlike_results[yi], color="r")
        ax.plot(maxlike_results[xi], maxlike_results[yi], "sr")
```



The lower left panel in the corner plot shows the joint probability distribution for the two model parameters. The two plots on the diagonal show marginalized distributions (with one parameter integrated out) for the two model parameters. The blue lines indicate the *true* values of the model parameters, while the title and the dashed lines indicate the most probable value as well as a 68% confidence interval. We will be discussing these representative measures of a posterior distribution during the course.

The frequentist maximum-likelihood estimate from before is indicated by the red lines (that we added manually to the plot). As you can see, for this particular setup the maximum-likelihood estimate agrees with the most probable value from the MCMC sampling of the Bayesian posterior distribution.

The pure Bayesian result for a problem like this, however, would be to report the posterior distribution itself (i.e. its representative sample), and leave it at that. That is, in pure Bayesianism the answer to a question is not a single number with error bars; the answer is the posterior distribution over the model parameters!

Discussion

Now, you might come away with the impression that the Bayesian method is unnecessarily complicated, and in this case it certainly is. Using an Affine Invariant Markov Chain Monte Carlo Ensemble sampler to characterize a two-dimensional normal distribution is a bit like using the Death Star to destroy a beach ball, but we did this here because it demonstrates an approach that can scale to complicated posteriors in many, many dimensions, and can provide nice results in more complicated situations where an analytic likelihood approach is not possible.

9.8. 📥 Problem: 2D radioactive lighthouse location using MCMC

As before, a radioactive source that emits gamma rays randomly in time but uniformly in angle is placed at (x_0, y_0). The gamma rays are detected on the x-axis and these positions are saved, x_k, $k = 1, 2, \cdots, N$. Given these observed positions, the problem is to estimate the location of the source.

Unlike before, we will not assume that y_0 is known. Your task is to estimate both x_0 and y_0. Since we are using Bayesian methods, this means finding the joint posterior for x_0 and y_0, given the data on where the gamma rays were detected.

You will combine the results and Python code from the Radioactive Lighthouse Problem and Gaussian noise and averages II notebooks, sampling the posterior using `emcee` and plotting it using `corner`.

Learning goals:

- Be able to re-use markdown and Python from existing notebooks to perform similar tasks (even if not understanding all details); e.g., generating data, sampling via MCMC, making plots.
- Successfully apply the basic ideas of Bayesian statistics: Bayes theorem, priors, sampling of posteriors.
- Successfully analyze results (with hints).
- Get further practice with markdown.

Note: you shouldn't need to recalculate anything; focus on the notebook ingredients.

Expressions

The posterior we want is:

$$p(x_0, y_0 | \{x_k\}, I) \stackrel{?}{=}$$

Using LaTeX *, fill in the right side of the equation for Bayes' rule as it applies in this case.*

Then add below the expression for the likelihood in this case (replace the dots with the correct form and fill in the right side):

$$p(\{x_k\} | \cdots) \stackrel{?}{=}$$

Describe in words what you will use for the prior:

Do you need to find an expression for the denominator pdf? Explain.

Python imports

You will need to import emcee and corner. These packages must be installed in your python environment.

```
# Copy to here all of the Python imports you think will be relevant
```

Generating the data

Copy-and-paste here the code from Radioactive Lighthouse Problem used to generate the x_k points. Note that you have control over where the true position is but also the size of the dataset. You will want to adjust both later. It is recommended to add the code that plots the distribution of the data (to verify it is the same as before), but this is not required.

```
# Add code here to generate the dataset.  Start with 500 points.
```

PDFs for applying Bayes' rule

Adapt code from the two notebooks to express the logarithms of the prior, likelihood, and posterior for this case.

```
# Add code for the log_prior, log_likelihood, and log_posterior
```

Run MCMC

Copy code to run `emcee` *and make any necessary changes.*

```
# Add the code to run emcee
```

Generate figures

Note: you do not need to do the maximum likelihood estimates that were added to the corner plot for the Gaussian noise case. You can skip that part of the code.

```
# Add the code to make a corner plot
```

Analysis

Summarize the results for the following investigations. You can simply add to the cell containing the questions or else start new cells.

1. *Run the notebook 10 times (which will generate 10 different sets of random data). Record the values of x_0 and y_0 and their uncertainties given on the corner plot for each run. Is the actual spread of values consistent with the given uncertainties?*

2. *Increase the number of data points by a factor of 10. By how much does the width of the posterior shrink? Is this expected?*

3. *What does increasing the number of MC steps per walker do?*

4. *Try changing* `x0_true` *and* `y0_true`*. Does it still work?*

9.9. Compare Gaussian noise sampling to lighthouse analysis

Here we observe that Gaussian noise sampling is carried out just like the radioactive lighthouse analysis from exercise notebook 📥 Problem: Radioactive lighthouse (which we assume you have worked through). Jump to the Bayesian approach in the exercise notebook 📥 Problem: Gaussian noise and averages II. The goal is to sample a posterior $p(\boldsymbol{\theta}|D, I)$

$$p(\mu, \sigma|D, I) \leftrightarrow p(x_0, y_0|X, I)$$

where D on the left are the x points and $D = X$ on the right are the $\{x_k\}$ where scintillation flashes are detected.

What do we need? From Bayes' theorem, we need

$$\begin{aligned} \text{likelihood:} \quad & p(D|\mu, \sigma, I) \leftrightarrow p(D|x_0, y_0, I) \\ \text{prior:} \quad & p(\mu, \sigma|I) \leftrightarrow p(x_0, y_0|I) \end{aligned}$$

You are generalizing the functions for log PDFs and the plotting of posteriors that are in 📥 Problem: Radioactive lighthouse. Note the functions for log-prior and log-likelihood in 📥 Problem: Gaussian noise and averages II. Here $\boldsymbol{\theta} = [\mu, \sigma]$ is a vector of parameters; cf. $\boldsymbol{\theta} = [x_0, y_0]$.

Let's step through the essential set up for `emcee`.

- It is best to create an environment that will include `emcee` and `corner`.

> **Hint**
>
> Nothing in the `emcee` sampling part needs to change!

- Basically we are doing 50 random walks in parallel to explore the posterior. Where the walkers end up will define our samples of $\mu, \sigma \Longrightarrow$ the histogram *is* an approximation to the (unnormalized) joint posterior.
- Plotting is also the same, once you change labels and `mu_true`, `sigma_true` to `x0_true`, `y0_true`. (And skip the `maxlike` part.)

Maximum likelihood here is the frequentist estimate $\longrightarrow$ this is an optimization problem. And you can read off marginalized estimates for μ and σ.

> **Question**
>
> Are μ and σ correlated or uncorrelated?
>
> > **Answer**
> >
> > They are *uncorrelated* because the contour ellipses in the joint posterior have their major and minor axes parallel to the μ and σ axes. Note that the fact that they look like circles is just an accident of the ranges chosen for the axes; if you changed the σ axis range by a factor of two, the circle would become flattened.

Bottom line: the two analyses are completely analogous.

10. Overview of Part II: Advanced Bayesian methods

In this part we extend our introduction to Bayesian methods to include more advanced techniques and examples.

The chapters in this part are

- Discrepancy models with GPs addresses an important topic: accounting for the errors in our model (we often call this the "theoretical error"). All models are approximate to some degree and failing to account for the discrepancy can lead to erroneous and often over-confident results.
- Assigning probabilities looks at a variety of methods to specify prior PDFs, including symmetry invariance and maximum entropy.
- Dealing with outliers explores several ways to extend linear regression to treat outlier data.
- Bayes goes linear: History matching describes a powerful technique that starts with the statistical model.
- Multi-model inference is a large topic with many facets. One approach to dealing with multiple models for the same data is Model Selection, which gives a probabilistic ranking among candidate models. Another set of

approaches, Model averaging and mixing, combine (“mix”) results from the models rather than seeking the best, with the goal of more informed inference.

11. Assigning probabilities

In this chapter we look at how to assign probabilities, usually in the context of Bayesian priors, and introduce and explore maximum entropy.

Good references for additional information are Chapter 5 in Sivia [SS06] and Chapters 4 and 8 from Gregory [Gre05].

11.1. Indifferences and translation groups

Discrete permutation invariance

- Consider a six-sided dice
- How do we assign $p_i \equiv p(X_i|I)$, $i \in \{1,2,3,4,5,6\}$?
- We do know $\sum_i p(X_i|I) = 1$
- Invariance under labeling $\Rightarrow p(X_i|I) = 1/6$
  - provided that the prior information I says nothing that breaks the permutation symmetry (e.g., we might know that the dice are not fair).

Location invariance

Indifference to a constant shift x_0 for a location parameter x implies that

$$p(x|I)dx \approx p(x+x_0|I)d(x+x_0) = p(x+x_0|I)dx, \tag{11.1}$$

in the allowed range.

Location invariance implies that

$$p(x|I) = p(x+x_0|I) \quad \Rightarrow \quad p(x|I) = \text{constant}. \tag{11.2}$$

- Provided that the prior information I says nothing that breaks the symmetry.
- The pdf will be zero outside the allowed range (specified by I).

Scale invariance

Indifference to a re-scaling λ of a scale parameter x implies that

$$p(x|I)dx \approx p(\lambda x|I)d(\lambda x) = \lambda p(\lambda x|I)dx, \tag{11.3}$$

in the allowed range.

Invariance under re-scaling implies that

$$p(x|I) = \lambda p(\lambda x|I) \quad \Rightarrow \quad p(x|I) \propto 1/x. \tag{11.4}$$

- Provided that the prior information I says nothing that breaks the symmetry.
- The pdf will be zero outside the allowed range (specified by I).
- This prior is often called a *Jeffrey's prior*; it represents a complete ignorance of a scale parameter within an allowed range.
- It is equivalent to a uniform pdf for the logarithm: $p(\log(x)|I) = \text{constant}$
  - as can be verified with a change of variable $y = \log(x)$, see lecture notes on error propagation.

Checkpoint question

Can you provide alternative evidence for the scale invariance result?

Answer

scale invariance $\longrightarrow p(x|I) \propto 1/x$

- First check that it works: $p(x|I) = \lambda p(\lambda x|I) \Longrightarrow \frac{c}{x} = \frac{\lambda c}{\lambda x} = \frac{c}{x}$. Check!
- Now a more general proof: assume $p(x|I) \propto x^\alpha$. Then $x^\alpha = \lambda(\lambda x)^\alpha = \lambda^{1+\alpha}x^\alpha \Longrightarrow$ $\alpha = -1$. Check!
- Still more general: set $\lambda = 1 + \epsilon$ with $\epsilon \ll 1$, and solve to $\mathcal{O}(\epsilon)$: $p(x) = (1+\epsilon)(p(x) + \epsilon\frac{dp}{dx})$ $\Longrightarrow p(x) + x\frac{dp}{dx} = 0$

$$\Longrightarrow \int_{p(x_0)}^{p(x)} \frac{dp}{p} = \int_{x_0}^{x} \frac{dx'}{x'} \Longrightarrow \log\frac{p(x)}{p(x_0)} = \log\frac{x_0}{x} \Longrightarrow p(x) = \left(\frac{p(x_0)}{x_0}\right)\frac{1}{x}$$

so $p(x) \propto 1/x$.

Example: Straight-line model

Consider the theoretical model

$$y_{\rm th}(x) = \theta_1 x + \theta_0. \tag{11.5}$$

- Would you consider the intercept θ_0 a location or a scale parameter, or something else?
- Would you consider the slope θ_1 a location or a scale parameter, or something else?

Consider also the statistical model for the observed data $y_i = y_{\rm th}(x_i) + \epsilon_i$, where we assume independent, Gaussian noise $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$.

- Would you consider the standard deviation σ a location or a scale parameter, or something else?

Symmetry invariance

- In fact, by symmetry indifference we could as well have written the linear model as $x_{\rm th}(y) = \theta_1' y + \theta_0'$
- We would then equate the probability elements for the two models

$$p(\theta_0, \theta_1|I) d\theta_0 d\theta_1 = q(\theta_0', \theta_1'|I) d\theta_0' d\theta_1'. \tag{11.6}$$

- The transformation gives $(\theta_0', \theta_1') = (-\theta_1^{-1}\theta_0, \theta_1^{-1})$.

This change of variables implies that

$$q(\theta_0', \theta_1'|I) = p(\theta_0, \theta_1|I) \left| \frac{d\theta_0 d\theta_1}{d\theta_0' d\theta_1'} \right|, \tag{11.7}$$

where the (absolute value of the) determinant of the Jacobian is

$$\left| \frac{d\theta_0 d\theta_1}{d\theta_0' d\theta_1'} \right| = \mathrm{abs} \left(\begin{vmatrix} \frac{\partial \theta_0}{\partial \theta_0'} & \frac{\partial \theta_0}{\partial \theta_1'} \\ \frac{\partial \theta_1}{\partial \theta_0'} & \frac{\partial \theta_1}{\partial \theta_1'} \end{vmatrix} \right) = \frac{1}{(\theta_1')^3}. \tag{11.8}$$

- In summary we find that $\theta_1^3 p(\theta_0, \theta_1|I) = p(-\theta_1^{-1}\theta_0, \theta_1^{-1}|I)$.
- This functional equation is satisfied by

$$p(\theta_0, \theta_1|I) \propto \frac{1}{\left(1 + \theta_1^2\right)^{3/2}}. \tag{11.9}$$

▼ **Hide code cell source**

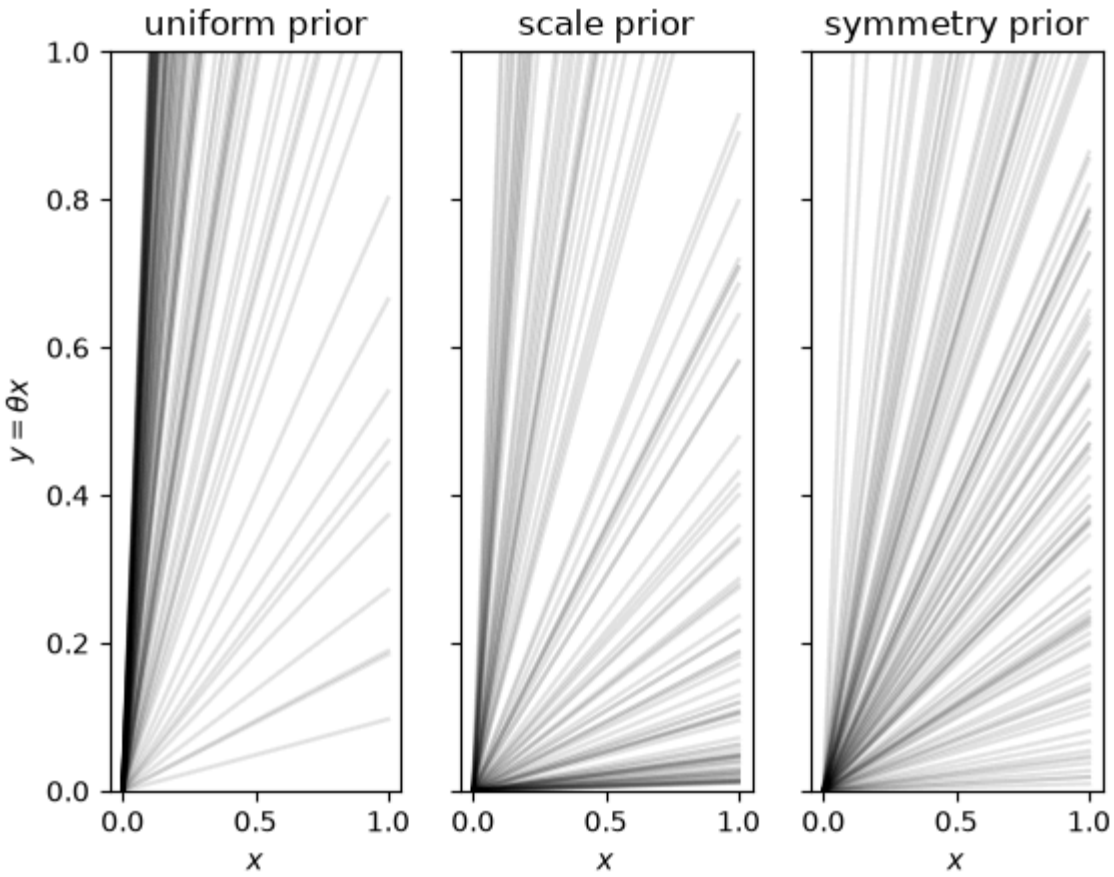


Fig. 11.1 100 samples of straight lines with fixed intercept equal to 0 and slopes sampled from three different prior pdfs. Note in particular the prior preference for large slopes that results from using a uniform pdf.

Straight-line example with MCMC sampling

We will define three different priors for the straight line model. Using always a flat prior U(-100,100) for the intercept, and a non-zero pdf range -100 <= slope <= 100. For the scale-invariant (Jeffreys) prior we must also exclude a small region around zero slope (here 0.01 < |slope|) since 1/|slope| is not normalizable otherwise.

Let us create 1000 samples from each prior pdf and plot the resulting sample of straight lines. Since the intercept is uniformly distributed in all three prior alternatives, we will just consider straight lines with intercept 0 since it makes it easier to compare the distribution of slopes.

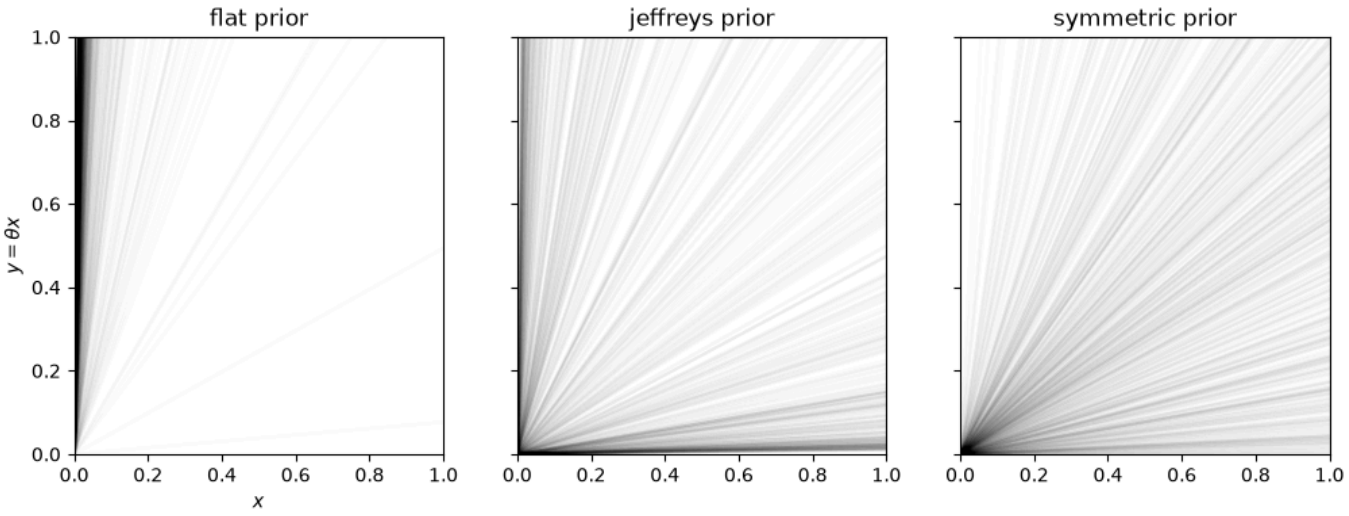
▼ **Hide code cell source**

```
MCMC sampling of log_flat_prior using emcee with 10 walkers

Mean acceptance fraction: 0.827  (in total 100000 steps)

MCMC sampling of log_jeffreys_prior using emcee with 10 walkers
```

```
Mean acceptance fraction: 0.586  (in total 100000 steps)
```

```
MCMC sampling of log_symmetric_prior using emcee with 10 walkers
```

```
Mean acceptance fraction: 0.776  (in total 100000 steps)
```



11.2. The principle of maximum entropy

Having dealt with ignorance, let us move on to more enlightened situations.

Consider a die with the usual six faces that was rolled a very large number of times. Suppose that we were only told that the average number of dots was 2.5. What (discrete) pdf would we assign? I.e. what are the probabilities $\{p_i\}$ that the face on top had i dots after a single throw?

The available information can be summarized as follows

$$\sum_{i=1}^{6} p_i = 1, \qquad \sum_{i=1}^{6} i p_i = 2.5 \tag{11.10}$$

This is obviously not a normal die, with uniform probability $p_i = 1/6$, since the average result would then be 3.5. But there are many candidate pdfs that would reproduce the given information. Which one should we prefer?

It turns out that there are several different arguments that all point in a direction that is very familiar to people with a physics background. Namely that we should prefer the probability distribution that maximizes an entropy measure, while fulfilling the given constraints.

It will be shown below that the preferred pdf $\{p_i\}$ is the one that maximizes

$$Q\left(\{p_i\};\lambda_0,\lambda_1\right)=-\sum_{i=1}^{6}p_i\log(p_i)+\lambda_0\left(1-\sum_{i=1}^{6}p_i\right)+\lambda_1\left(2.5-\sum_{i=1}^{6}ip_i\right), \tag{11.11}$$

where the constraints are included via the method of Lagrange multipliers.

The entropy of Scandinavians

Let's consider another pdf assignment problem. This is originally the *kangaroo problem* (Gull and Skilling, 1984), but translated here into a local context. The problem is stated as follows:

Information: :
70% of all Scandinavians have blonde hair, and 10% of all Scandinavians are left handed. Question: :
On the basis of this information alone, what proportion of Scandinavians are both blonde and left handed?

We note that for any one given Scandinavian there are four distinct possibilities:

1. Blonde and left handed (probability p_1).
2. Blonde and right handed (probability p_2).
3. Not blonde and left handed (probability p_3).
4. Not blonde and right handed (probability p_4).

The following 2x2 contingency table

	Left handed	**Right handed**
Blonde	p_1	p_2
Not blonde	p_3	p_4

can be written in terms of a single variable x due to the normalization condition $\sum_{i=1}^{4}p_i=1$, and the available information $p_1+p_2=0.7$ and $p_1+p_3=0.1$

	Left handed	**Right handed**
Blonde	$0\leq x\leq 0.1$	$0.7-x$
Not blonde	$0.1-x$	$0.2+x$

But which choice of x is preferred?

Here we will introduce the monkey argument to decide how to assign x. We will then show that this argument is equivalent to the principle of maximum entropy, which can be formulated for more general scenarios and used to derive a number of commonly used pdfs.

The monkey argument

The monkey argument is a model for assigning probabilities to M different (discrete) alternatives that satisfy some constraints described by I. The argument goes as follows:

1. Imagine a very large number of monkeys throwing N balls into M equally sized boxes. The final number of balls in box i is n_i.
   - The normalization condition is satisfied via $N = \sum_{i=1}^{M} n_i$.
   - The fraction of balls in each box gives a possible assignment for the corresponding probability $p_i = n_i/N$.
   - The distribution of balls $\{n_i\}$ divided by N is therefore a candidate pdf $\{p_i\}$.
2. After one round the monkeys have distributed their (huge number of) balls over the M boxes.
   - The resulting pdf might not be consistent with the constraints in I, however, in which case it must be rejected as a possible candidate.
   - The candidate pdf is recorded by an independent observer in the scenario that the constraints are in fact fulfilled.
3. After many such rounds, some distributions will be found to come up more often than others.
   - The one that appears most frequently (and satisfies I) would be a sensible choice for $p(\{p_i\}|I)$.
   - Since our ideal monkeys have no agenda of their own to influence the distribution, this most favoured distribution can be regarded as the one that best represents our given state of knowledge.
   - No bananas are allowed!

Now, let us see how this preferred solution corresponds to the pdf with the largest `entropy`. Remember in the following that N (and n_i) are considered to be very large numbers ($N/M \gg 1$)

- There are M^N different ways to distribute the balls.
- The micro-states corresponding to a particular distribution $\{n_i\}$ are all connected to the same pdf $\{p_i\}$. Therefore, the frequency of a given pdf is given by

$$F(\{p_i\}) = \frac{\text{number of ways of obtaining} \qquad \{n_i\}}{M^N} \tag{11.12}$$

- The number of micro-states, $W(\{n_i\})$, in the nominator is equal to $N!/\prod_{i=1}^{M} n_i!$.
- We express the logarithm of this number using the Stirling approximation for factorials of large numbers, $\log(n!) \approx n\log(n) - n$, and finding a cancellation of $N - \sum_i n_i$.

$$\log(W(\{n_i\})) = \log(N!) - \sum_{i=1}^{M} \log(n_i!) \approx N\log(N) - \sum_{i=1}^{M} n_i \log(n_i), \tag{11.13}$$

- Therefore, the logarithm of the frequency of a given pdf is

$$\log(F(\{p_i\})) \approx -N\log(M) + N\log(N) - \sum_{i=1}^{M} n_i \log(n_i) \tag{11.14}$$

Substituting $p_i = n_i/N$, and using the normalization condition finally gives

$$\log(F(\{p_i\})) \approx -N\log(M) - N\sum_{i=1}^{M} p_i \log(p_i) \tag{11.15}$$

Recall that the preferred pdf is the one that appears most frequently, i.e., that maximizes the above expression. We further note that N and M are constants such that the preferred pdf is given by the $\{p_i\}$ that maximizes

$$S = -\sum_{i=1}^{M} p_i \log(p_i). \tag{11.16}$$

You might recognise this quantity as the *entropy* measure from statistical mechanics. The interpretation of entropy in statistical mechanics is the measure of uncertainty that remains about a system after its observable macroscopic properties, such as temperature, pressure and volume, have been properly taken into account. For a given set of macroscopic variables, the entropy measures the degree to which the probability of the system is spread out over different possible microstates. Specifically, entropy is a logarithmic measure of the number of micro-states with significant probability of being occupied $S = -k_B \sum_i p_i \log(p_i)$, where k_B is the Boltzmann constant.

Why maximize the entropy?

There are a few different arguments for why the entropy should be maximized when assigning probability distributions given some limited information:

1. Information theory: maximum entropy=minimum information (Shannon, 1948).
2. Logical consistency (Shore & Johnson, 1960).
3. Uncorrelated assignments related monotonically to S (Skilling, 1988).

Consider the third argument. Let us check it empirically in the context of the problem of hair colour and handedness of Scandinavians. We are interested in determining $p_1 \equiv p(L, B|I) \equiv x$, the probability that a Scandinavian is both left-handed and blonde. However, in this simple example we can immediately realize that the assignment $p_1 = 0.07$ is the only one that implies no correlation between left-handedness and hair color. Any joint probability smaller than 0.07 implies that left-handed people are less likely to be blonde, and any larger vale indicates that left-handed people are more likely to be blonde.

Uncorrelated assignments

Unless you have specific information about the existence of a correlation, you should better not build it into the assignment of the probability distribution.

As argued, any assignment $p_1 \neq 0.07$ corresponds to the existence of a correlation that was not explicitly specified in the provided information.

Question

Can you show why $p_1 < 0.07$ and $p_1 > 0.07$ corresponds to left-handedness and blondeness being dependent variables?

Let us now empirically consider a few variational functions of $\{p_i\}$ and see if any of them gives a maximum that corresponds to the uncorrelated assignment $x = 0.07$. Note that $x = 0.07$ implies $p_1 = 0.07$, $p_2 = 0.63$, $p_3 = 0.03$, $p_4 = 0.27$. A few variational functions and their prediction for x are shown in the following table.

Variational function	argmax x	Implied correlation
$-\sum_i p_i \log(p_i)$	0.070	None
$\sum_i \log(p_i)$	0.053	Negative
$-\sum_i p_i^2 \log(p_i)$	0.100	Positive
$-\sum_i \sqrt{p_i(1-p_i)}$	0.066	Negative

The assignment based on the entropy measure is the only one that respects this lack of correlations.

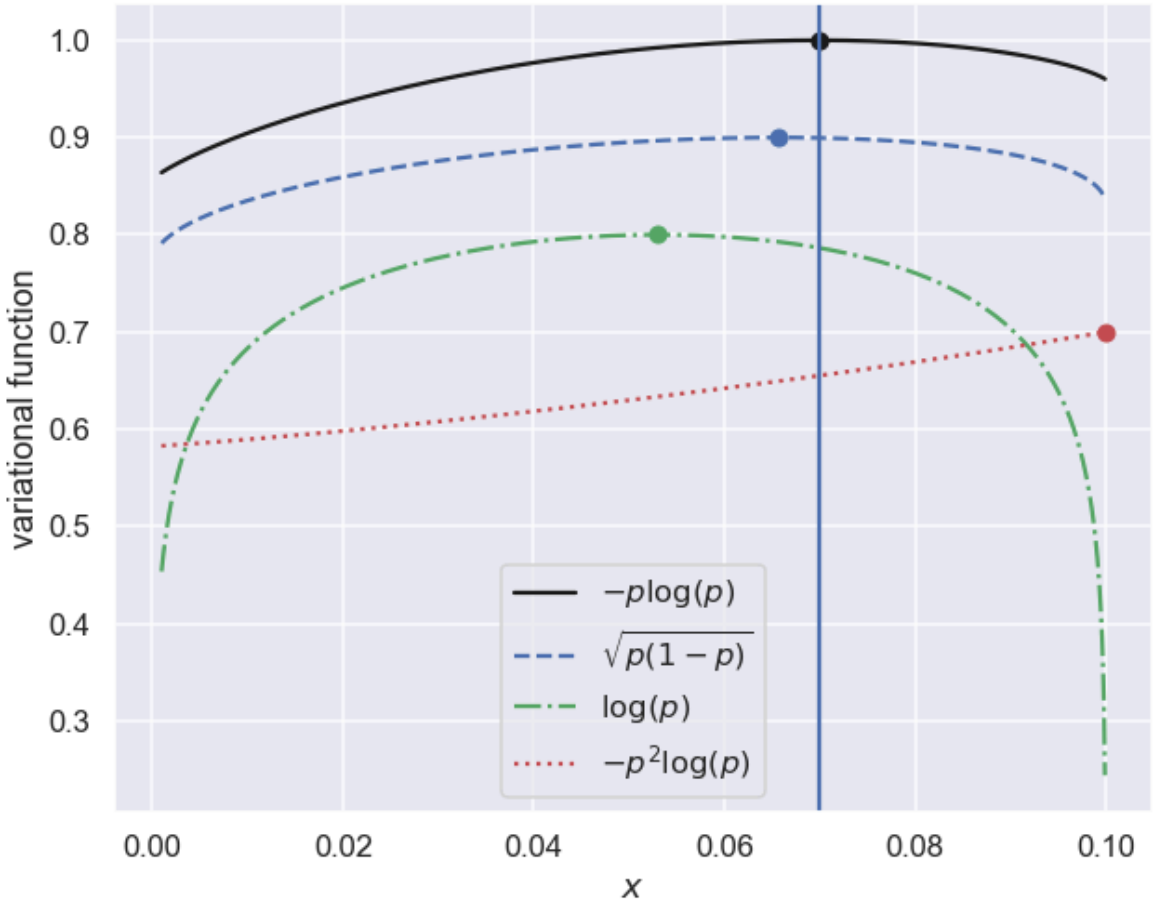


Fig. 11.2 Four different variational functions $f(\{p_i\})$. The optimal x for each one is shown by a circle. The uncorrelated assignment $x = 0.07$ is shown by a vertical line.

Continuous case

Let us return to the monkeys, but now with different probabilities for each bin. Then

$$S = -\sum_{i=1}^{M} p_i \log\left(\frac{p_i}{m_i}\right), \tag{11.17}$$

which is often known as the *Shannon-Jaynes entropy*, or the *Kullback number*, or the *cross entropy* (with opposite sign).

Jaynes (1963) has pointed out that this generalization of the entropy, including a *Lebesgue measure* m_i, is necessary when we consider the limit of continuous parameters.

$$S[p] = -\int p(x) \log\left(\frac{p(x)}{m(x)}\right). \tag{11.18}$$

- In particular, $m(x)$ ensures that the entropy expression is invariant under a change of variables $x \to y = f(x)$.
- Typically, the transformation-group (invariance) arguments are appropriate for assigning $m(x) = \text{constant}$.
- However, there are situations where other assignments for m represent the most ignorance. For example, in counting experiments one might assign $m(N) = M^N/N!$ for the number of observed events N and a very large number of intervals M.

Derivation of common pdfs using MaxEnt

The principle of maximum entropy (MaxEnt) allows incorporation of further information, e.g. constraints on the mean, variance, etc, into the assignment of probability distributions.

In summary, the MaxEnt approach aims to maximize the Shannon-Jaynes entropy and generates smooth functions.

Mean and the Exponential pdf

Suppose that we have a pdf $p(x|I)$ that is normalized over some interval $[x_{\min}, x_{\max}]$. Assume that we have information about its mean value, i.e.,

$$\langle x \rangle = \int x p(x|I) dx = \mu. \tag{11.19}$$

Based only on this information, what functional form should we assign for the pdf that we will now denote $p(x|\mu)$?

Let us use the principle of MaxEnt and maximize the entropy under the normalization and mean constraints. We will use Lagrange multipliers, and we will perform the optimization as a limiting case of a discrete problem; explicitly, we will maximize

$$Q = -\sum_i p_i \log\left(\frac{p_i}{m_i}\right) + \lambda_0\left(1 - \sum_i p_i\right) + \lambda_1\left(\mu - \sum_i x_i p_i\right). \tag{11.20}$$

Setting $\partial Q/\partial p_j = 0$ we obtain

$$p_j = m_j \exp\left[-(1+\lambda_0)\right] \exp\left[-\lambda_1 x_j\right]. \tag{11.21}$$

With a uniform measure $m_j = \text{constant}$ we find (in the continuous limit) that

$$p(x|\mu) = \mathcal{N} \exp\left[-\lambda_1 x\right]. \tag{11.22}$$

The normalization constant (related to λ_0) and the remaining Lagrange multiplier, λ_1, can easily determined by fulfilling the two constraints.

Assuming, e.g., that the normalization interval is $x \in [0, \infty[$ we obtain

$$\int_0^\infty p(x|\mu)dx = 1 = \left[-\frac{\mathcal{N}}{\lambda_1} e^{-\lambda_1 x}\right]_0^\infty = \frac{\mathcal{N}}{\lambda_1} \quad \Rightarrow \quad \mathcal{N} = \lambda_1. \tag{11.23}$$

The constraint for the mean then gives

$$\mu = \lambda_1 \int_0^\infty x e^{-\lambda_1 x} dx = \lambda_1 \frac{1!}{\lambda_1^2} = \frac{1}{\lambda_1} \quad \Rightarrow \quad \lambda_1 = \frac{1}{\mu}. \tag{11.24}$$

So that the properly normalized pdf from MaxEnt principles becomes the exponential distribution

$$p(x|\mu) = \frac{1}{\mu} \exp\left[-\frac{x}{\mu}\right]. \tag{11.25}$$

Variance and the Gaussian pdf

Suppose that we have information not only on the mean μ but also on the variance

$$\left\langle (x-\mu)^2 \right\rangle = \int (x-\mu)^2 p(x|I) dx = \sigma^2. \tag{11.26}$$

The principle of MaxEnt will then result in the continuum assignment

$$p(x|\mu,\sigma) \propto \exp\left[-\lambda_1 (x-\mu)^2\right]. \tag{11.27}$$

Assuming that the limits of integration are $\pm\infty$ we can determine both the normalization coefficient and the Lagrange multiplier. After some integration this results in the standard Gaussian pdf

$$p(x|\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}}\exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]. \tag{11.28}$$

The normal distribution

This indicates that the normal distribution is the most honest representation of our state of knowledge when we only have information about the mean and the variance.

Notice. These arguments extend easily to the case of several parameters. For example, considering $\{x_k\}$ as the data $\{D_k\}$ with error bars $\{\sigma_k\}$ and $\{\mu_k\}$ as the model predictions, this allows us to identify the least-squares likelihood as the pdf which best represents our state of knowledge given only the value of the expected squared-deviation between our predictions and the data

$$p\left(\{x_k\}|\{\mu_k,\sigma_k\}\right) = \prod_{k=1}^{N}\frac{1}{\sigma_k\sqrt{2\pi}}\exp\left[-\frac{(x_k-\mu_k)^2}{2\sigma_k^2}\right]. \tag{11.29}$$

If we had convincing information about the covariance $\langle(x_i-\mu_i)\,(x_j-\mu_j)\rangle$, where $i \neq j$, then MaxEnt would assign a correlated, multivariate Gaussian pdf for $p\left(\{x_k\}|I\right)$.

Poisson distribution

The constraints are normalization and a known mean:

$$\int_0^\infty dx\, p(x) = 1, \quad x \geq 0$$
$$\int_0^\infty dx\, xp(x) = \mu$$

$\Longrightarrow$ maximize

$$Q(p;\lambda_0,\lambda_1) = -\int dx\, p(x)\log\Big(\frac{p(x)}{m(x)}\Big) + \lambda_0\big(1-\int dx\, p(x)\big) + \lambda_1\big(\mu - \int dx\, xp(x)\big),$$

with uniform $m(x)$. The maximization is again straightforward:

$$\begin{aligned}
&\frac{\delta Q}{\delta p(x)} = -\log\frac{p(x)}{1} - \frac{p(x)}{p(x)} - \lambda_0 - \lambda_1 x = 0\\
&\Longrightarrow \log p(x) = -(1+\lambda_0) - \lambda_1 x\\
&\Longrightarrow p(x) = e^{-1+\lambda_0}e^{-\lambda_1 x}
\end{aligned}$$

Finally, we determine λ_0 and λ_1 from the constraints:

$$e^{1+\lambda_0}\int_0^\infty dx\, e^{-\lambda_1 x} = e^{-(1+\lambda_0)}\frac{1}{\lambda_1} = 1 \quad \Longrightarrow \quad \lambda_1 = e^{-(1+\lambda_0)}$$
$$\int_0^\infty dx\, \underbrace{e^{-(1+\lambda_0)}}_{\lambda_1}\underbrace{e^{-\lambda_1 x}}_{1/\lambda_1^2} x = \mu \quad \Longrightarrow \quad \lambda_1 = \frac{1}{\mu}.$$

Substituting we get the Poisson distribution:

$$p(x) = \frac{1}{\mu}e^{-x/\mu}.$$

Log normal distribution

Suppose the constraint is on the variance of $\ln x$, i.e.,

$$\int p(x)\left[\log\left(\frac{x}{x_0}\right)\right]^2 dx = \sigma^2$$

Change variables to $y = \log(x/x_0)$. What is the constraint in terms of y?

Now maximize the entropy, subject to this constraint, and, of course, the normalization constraint.

You should obtain the log-normal distribution:

$$p(x) = \frac{1}{\sqrt{2\pi}x\sigma}\exp\left[-\frac{\ln^2(x/x_0)}{2\sigma^2}\right].$$

When do you think it would make sense to say that we know the variance of $\log(x)$, rather than the variance of x itself?

l1-norm

Finally, we take the constraint on the mean absolute value of $x - \mu$: $\langle|x-\mu|\rangle = \epsilon$.

This constraint is written as:

$$\int p(x)\,|x-\mu|\,dx = \epsilon.$$

Use the uniform measure, and go through the steps once again, to show that:

$$p(x) = \frac{1}{2\epsilon}\exp\left(-\frac{|x-\mu|}{\epsilon}\right).$$

Try other examples!

11.3. Maximum Entropy for reconstructing a function from its moments

Here we use Maximum Entropy to reconstruct some simple functions from their moments, using the formulation by Mead and Papanicolaou, J. Math. Phys. 24, 2404 (1984). The context is answering the question: If we know some (or all) of the moments of a one-dimensional distribution $p(x)$, can we determine $p(x)$? This question was asked in Section 4.4 and we repeat the definitions from there.

First, note that we referred to $p(x)$ as a "distribution". This means that we assume it is positive definite and normalized to 1 when integrated over its domain of support S (e.g., $(-\infty, \infty)$ for a Gaussian or $[0, 1]$ for a Beta distribution). Thus $p(x)$ is a candidate for a probability distribution function (pdf) and we will refer to it as such in what follows.

We define the moments m_k of $p(x)$ for integer k as

$$m_k = \int_S x^k\, p(x)\, dx, \qquad k = 0, 1, 2, \ldots,$$

where S denotes the support of the pdf. So $m_0 = 1$, the mean μ is m_1, the variance σ^2 is $m_2 - m_1^2$, and so on.

If we have only a finite number $N+1$ of moments, this is an underdetermined inverse problem. We solve it by maximizing the entropy associated with $p(x)$ subject to the condition that the first $N+1$ moments be equal to the true moments m_k, $k = 0, 1, \ldots, N$. To carry this out we introduce $N+1$ Lagrange multipliers λ_k and maximize the functional $S = S[p]$ defined by

$$S = -\int_a^b [p(x)\ln p(x) - p(x)]dx + \sum_{k=0}^{N} \lambda_k \Big(\int_a^b x^k p(x) dx - m_k \Big).$$

We take the functional derivative with respect to $p(x)$ and set it equal to zero, yielding

$$\frac{\delta S}{\delta p(x)} = 0 \implies p(x) = p_N(x) = \exp\Big(-\lambda_0 - \sum_{k=1}^{N} \lambda_k x^k\Big),$$

which will be our expression when we know the λ_ks. We have $N+1$ nonlinear equations for those $N+1$ unknown after taking partial derivatives with respect to the λ_ks and setting them to zero, yielding

$$\int_a^b x^k p_N(x) dx = m_k, \quad k = 0, 1, \ldots, N.$$

We will assume that $p(x)$ is normalized, i.e., that $m_0 = 1$. This implies that the $k = 0$ equation can be used to solve for λ_0, which defines the partition function (and a Boltzmann factor as the integrand):

$$e^{\lambda_0} = \int_a^b dx\, \exp\Big(-\sum_{k=1}^{N} \lambda_k x^k\Big) \equiv Z.$$

We perform a Legendre transformation to go from the free energy $\ln Z$ to the effective potential $\Gamma = \Gamma(\lambda_1, \lambda_2, \ldots, \lambda_N)$

$$\Gamma = \ln Z + \sum_{k=1}^{N} m_k \lambda_k.$$

Given initial guesses for the λs, we will simply numerically minimize Γ to find the λs and then evaluate $p_N(x)$.

Benchmark case

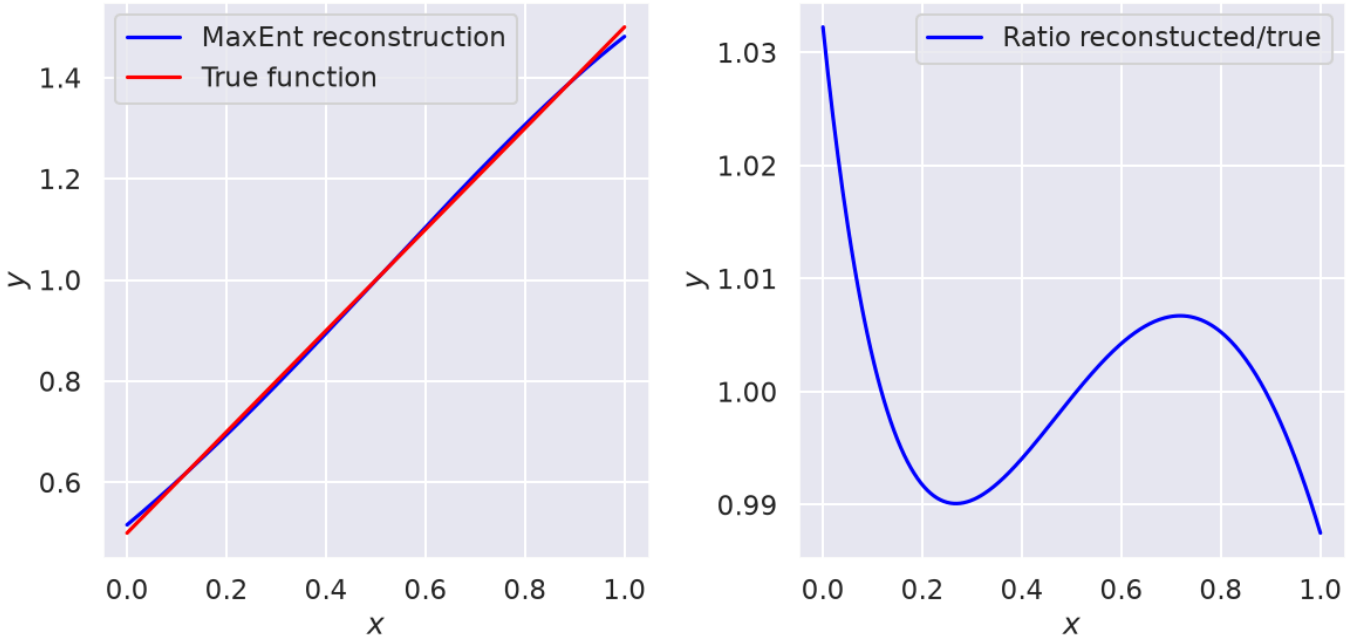
As a benchmark, we consider the (normalized) distribution $p(x) = x + 1/2$ in the domain $0 \le x \le 1$. For $N = 2$ to $N = 5$, we minimize the effective potential, starting from $\lambda_k = 1$ for all k, and then reconstruct and plot the result, comparing to the exact result.

▼ **Hide code cell source**

N=2 moments

▼ **Hide code cell source**

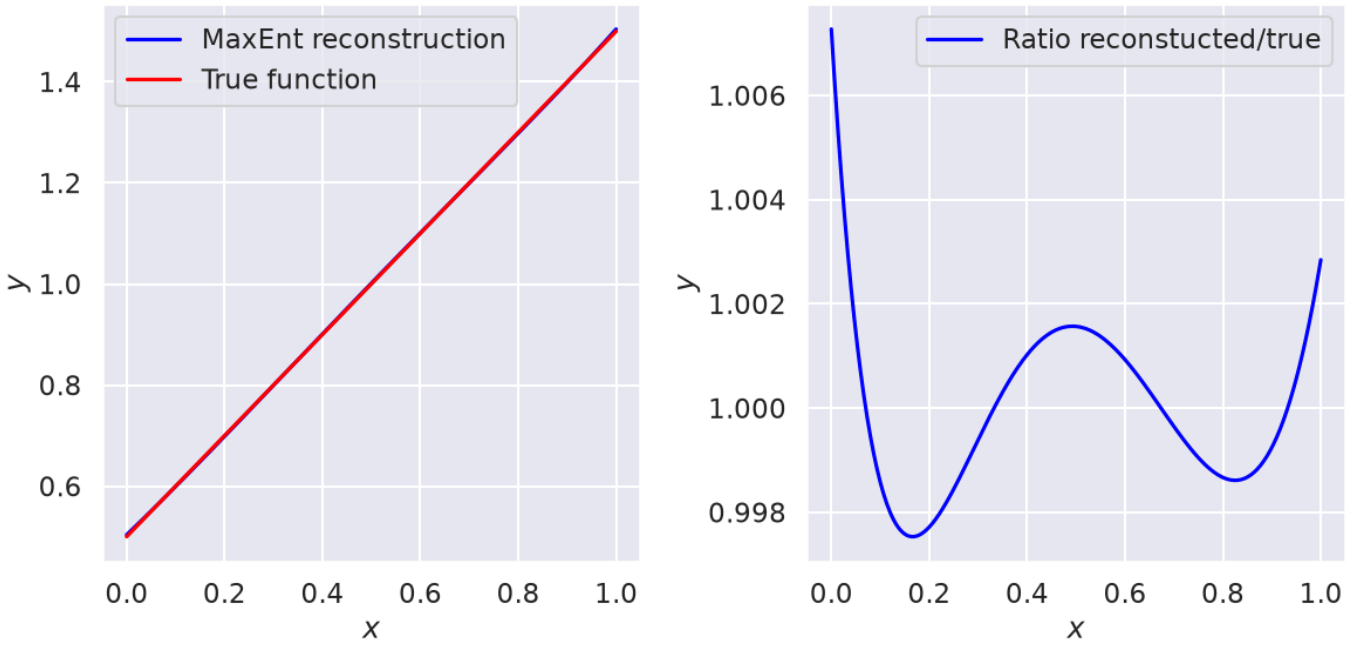
```
Moments:  [0.58333333 0.41666667]
Minimized Lagrange multipliers lambdas:  [-1.58929037  0.53499052]
```



N=3 moments

▼ **Hide code cell source**

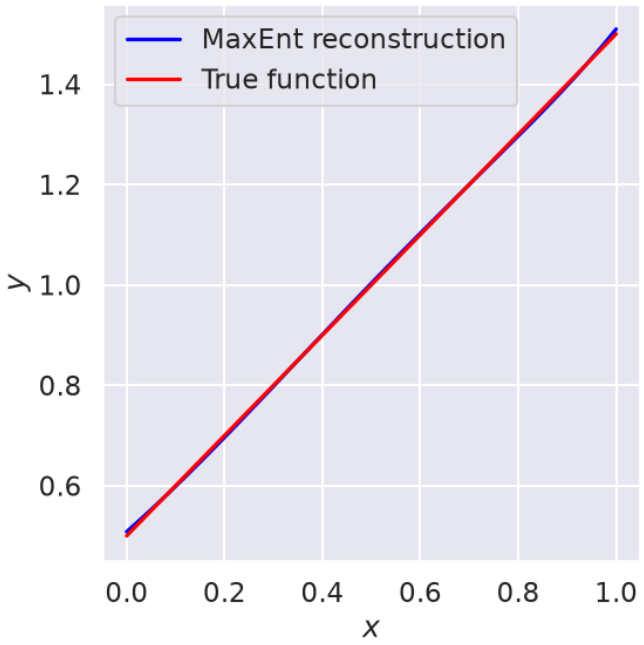
```
Moments:  [0.58333333 0.41666667 0.325     ]
Minimized Lagrange multipliers lambdas:  [-1.84592699  1.13225838 -0.38053871]
```

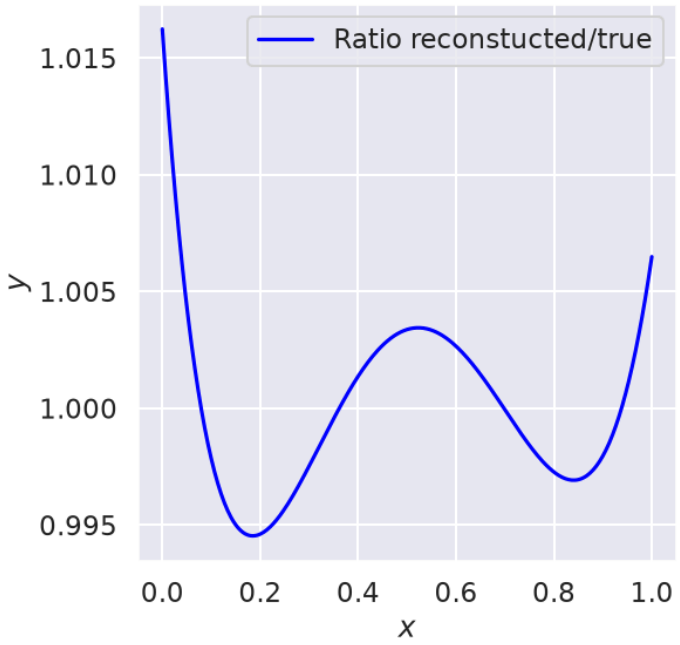


N=4 moments

▼ **Hide code cell source**

```
Moments:  [0.58333333 0.41666667 0.325      0.26666667]
Minimized Lagrange multipliers lambdas:  [-1.69941869  0.54752565  0.45580342 -0.39288493]
```

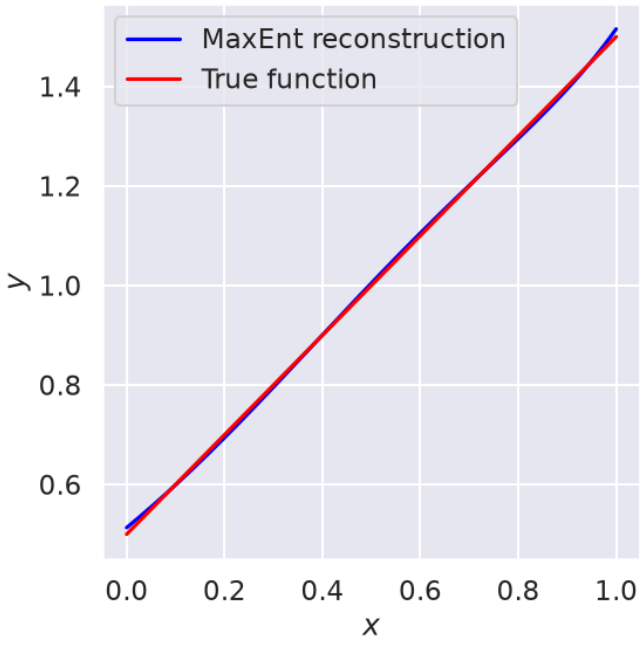




N=5 moments

▼ **Hide code cell source**

```
Moments:  [0.58333333 0.41666667 0.325      0.26666667 0.22619048]
Minimized Lagrange multipliers lambdas:  [-1.57580825  0.23066237  0.48944599  0.13622227
-0.3634522 ]
```



Checkpoint question

Does the result improve from $N = 2$ to $N = 3$? From $N = 3$ to $N = 4$? From $N = 4$ to $N = 5$?

Answer

There is clear improvement from $N = 2$ to $N = 3$, but not beyond that. Note that at N=4 the minimum found is: -0.04278640499674846, but there is also: -0.04279159257882259. How could you persuade Python to find this slightly better minimum? Does it help?

Exercises

Exercise 11.1

Copy the code defined here (open Show code cell source) to a Jupyter notebook (or write your own version) to produce a MaxEnt reconstruction of $y(x) = 2x$ for $0 \leq x \leq 1$.

Exercise 11.2

MaxEnt doesn't even care about essential singularities. Compute the moments for $y(x) = 1/\mu_0 \exp(-1/x)$ and reconstruct it. Note that you need to choose the constant μ_0 so that y can be interpreted as a probability distribution (i.e., normalized). [Copy the code defined here (open Show code cell source) to a Jupyter notebook (or write your own version)].

Exercise 11.3

MaxEnt does care about some things, though. Repeat the last exercise but for $P(x) = 4x - 1$ and see what happens. Can you figure out why MaxEnt failed for this function?

12. Dealing with outliers

In this chapter, we describe and illustrate several approaches to deal with outliers. By **outliers** in data, we mean data points that significantly differ from the pattern of other observations in a set of data. They may arise from unaccounted for or underestimated errors, or just be anomalous occurrences.

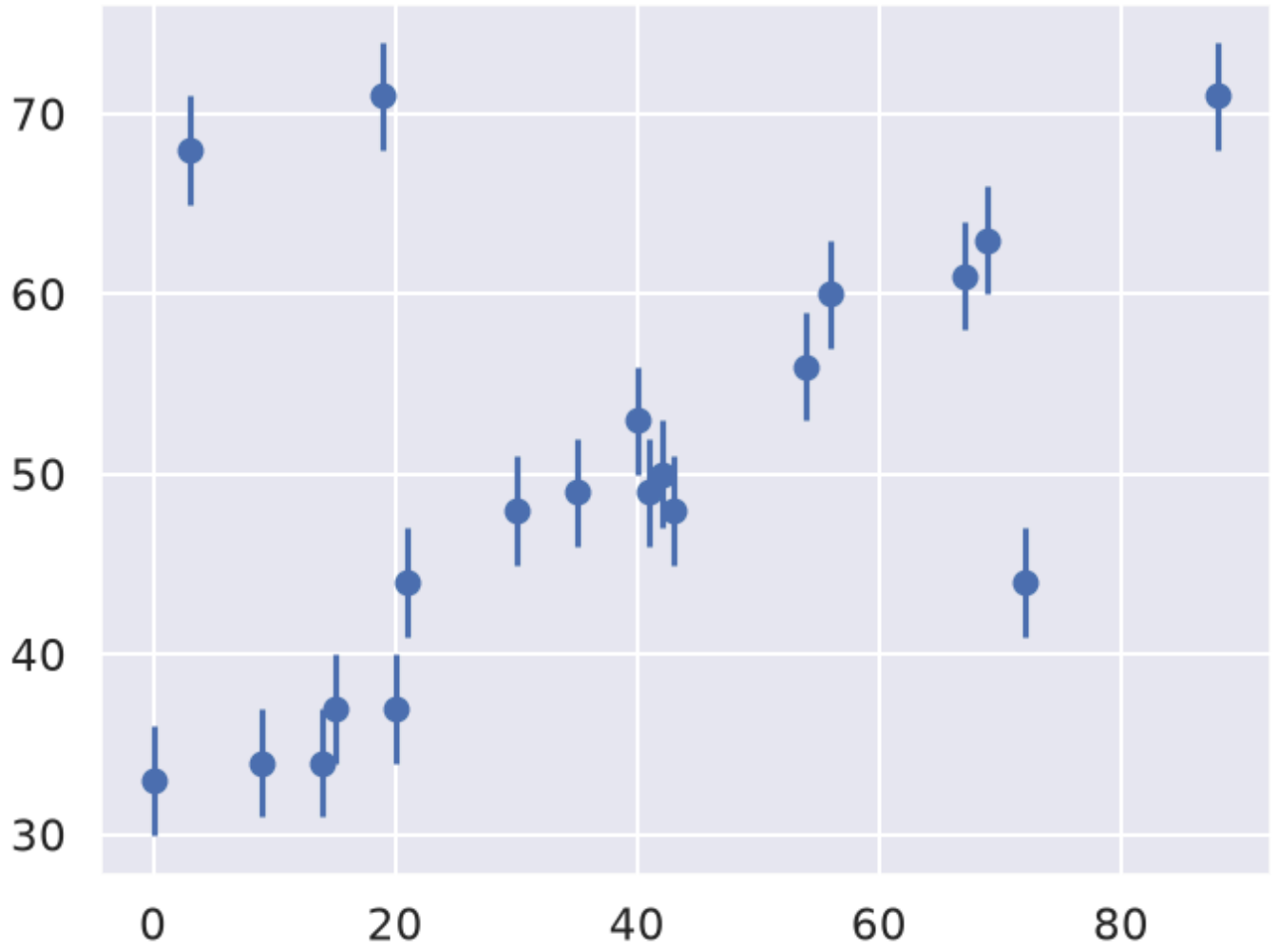
12.1. Setup: linear regression with data outliers

We will use as our canonical example a set of data that mostly lies close to a straight line, and is expected to lie on a straight line, but has several points that are far away. (This example and subsequent discussion was adapted from the blog post Frequentism and Bayesianism II: When Results Differ.)

Sample dataset

Consider the following plotted dataset, relating the observed variables x and y, with errors on y. (You will have the opportunity to explore different datasets and errors in 📥 Demo: Illustrating approaches to outliers.)

▼ Hide code cell source



Our task is to find a line of best-fit to the data. It's clear upon visual inspection that there are some outliers among these points, but how do we deal with them. In the subsequent sections we will consider both frequentist and Bayesian approaches. But first we need to set up our model.

The Model

Here we review the procedure for fitting a straight line to data, emphasizing the role of the statistical model. We take as our model ("M" here stands for "Model"; elsewhere we use "th" for theory),

$$y_M(x;\boldsymbol{\theta}) = \theta_0 + \theta_1 x,$$

where our parameter vector will be

$$\boldsymbol{\theta} = [\theta_0, \theta_1].$$

But this is only half the picture: what we mean by a “model” in a Bayesian sense is not only this expected value $y_M(x;\boldsymbol{\theta})$, but a **probability distribution** for our data. That is, we need an expression to compute the likelihood $p(D \mid \boldsymbol{\theta}, I)$ for our data as a function of the parameters $\boldsymbol{\theta}$.

The Bayesian statistical model we employ is that the distribution of data D is the sum of the distributions for the model, the model discrepancy, and the observation error:

$$D = M + \delta M + \delta D.$$

The model discrepancy term δM is in general important to consider, but we will neglect it here (e.g., assuming it is much smaller than δD).

To establish δD, we note that here we are given data with simple error bars, which imply that the probability for any *single* data point is a normal distribution about the true value. In this example, the errors are specified by a single parameter σ_0 (again, we neglect δM). That is, $y_i = D_i$ is given by

$$y_i \sim \mathcal{N}(y_M(x_i;\boldsymbol{\theta}), \sigma_0^2)$$

or, in functional form,

$$p(y_i \mid x_i, \boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi\sigma_0^2}} \exp\left(\frac{-\left[y_i - y_M(x_i;\boldsymbol{\theta})\right]^2}{2\sigma_0^2}\right)$$

The (known) variance of the measurement errors, σ_0, is indicated in the figure below by the error bars.

Assuming all the points are independent, we can find the full likelihood by multiplying the individual likelihoods together:

$$p(D \mid \boldsymbol{\theta}, I) = \prod_{i=1}^{N} p(y_i \mid x_i, \boldsymbol{\theta}, I).$$

For convenience (and also for numerical accuracy) this is often expressed in terms of the log-likelihood:

$$\begin{aligned} \log p(D \mid \boldsymbol{\theta}, I) &= -\frac{1}{2}\sum_{i=1}^{N}\left(\log(2\pi\sigma_0^2) + \frac{\left[y_i - y_M(x_i;\theta)\right]^2}{\sigma_0^2}\right) \\ &= \text{constant} \quad -\sum_{i=1}^{N}\frac{\left[y_i - y_M(x_i;\boldsymbol{\theta})\right]^2}{2\sigma_0^2}. \end{aligned} \tag{12.1}$$

We often define the *residuals* (the notation z_i is also common)

$$R_i = \left[y_i - y_M(x_i;\boldsymbol{\theta})\right]/\sigma_0,$$

so that the relevant chi-square sum is $\chi^2 = \sum_{i=1}^{N} R_i^2$.

The summation term that appears in this log-likelihood is often known as the *loss function*. This particular loss function is known as a *squared loss* or *chi-squared*; but as you can see it can be derived from the Gaussian log likelihood.

12.2. Frequentist methods to deal with outliers

In this section and the next we will use the data from Setup: linear regression with data outliers. The theoretical model is

$$y_M(x;\boldsymbol{\theta}) = \theta_0 + \theta_1 x,$$

where our parameter vector to be determined is

$$\boldsymbol{\theta} = [\theta_0, \theta_1].$$

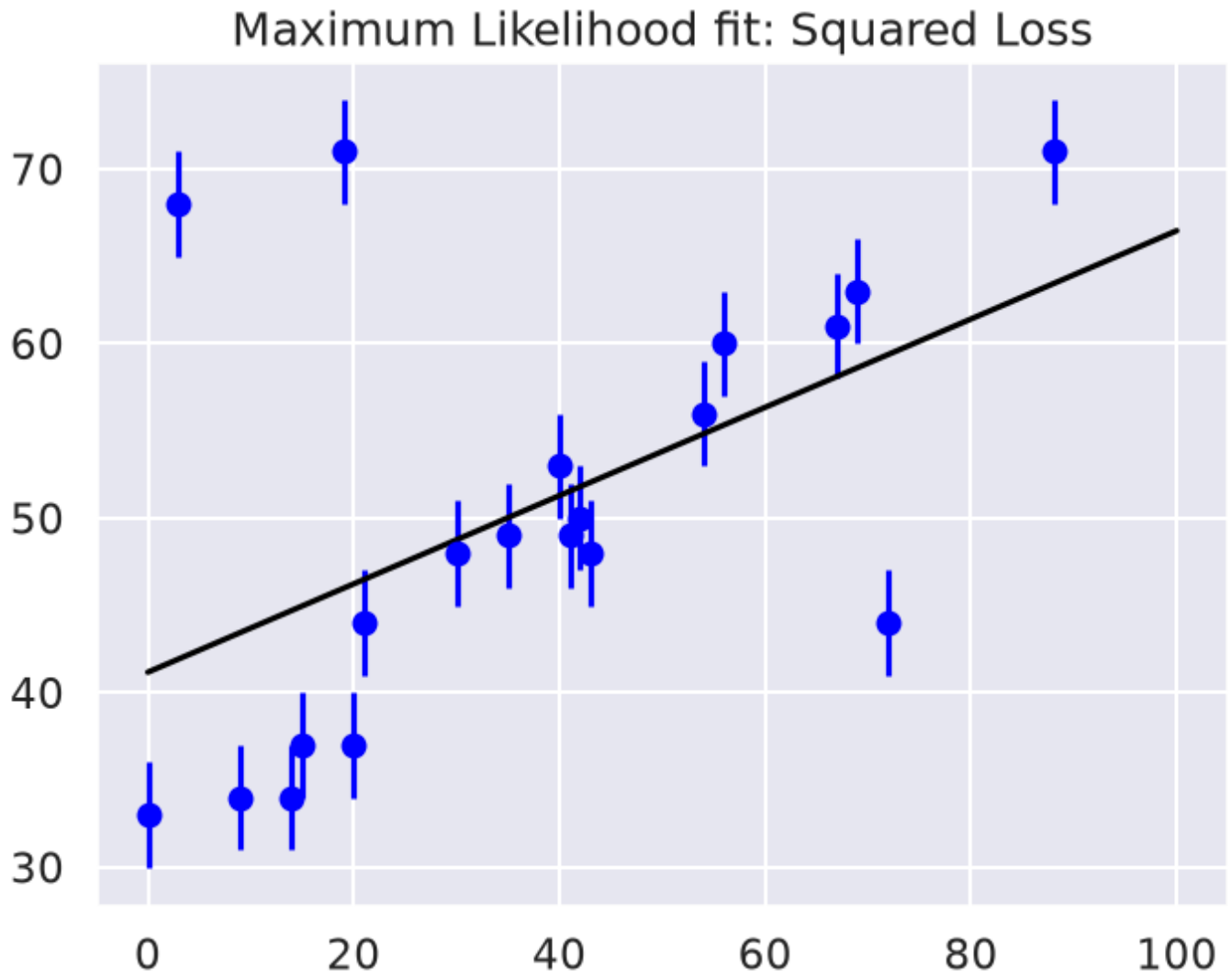
Standard Likelihood Approach

We can maximize the likelihood (or, equivalently, minimize the loss) to find $\boldsymbol{\theta}$ within a frequentist paradigm. For a flat prior in $\boldsymbol{\theta}$, the maximum of the Bayesian posterior will yield the same result. (Note that there are good arguments based on the principle of maximum entropy that a flat prior is not the best choice here; we'll ignore that detail for now, as it's a small effect for this problem.)

For simplicity, we'll use scipy's `optimize` package to minimize the loss (in the case of squared loss, this computation can be done more efficiently using matrix methods, but we'll use numerical minimization for simplicity here).

▼ **Hide code cell source**

```
Maximum likelihood estimate (MLE): theta0 = 41.2, theta1 = 0.25
```



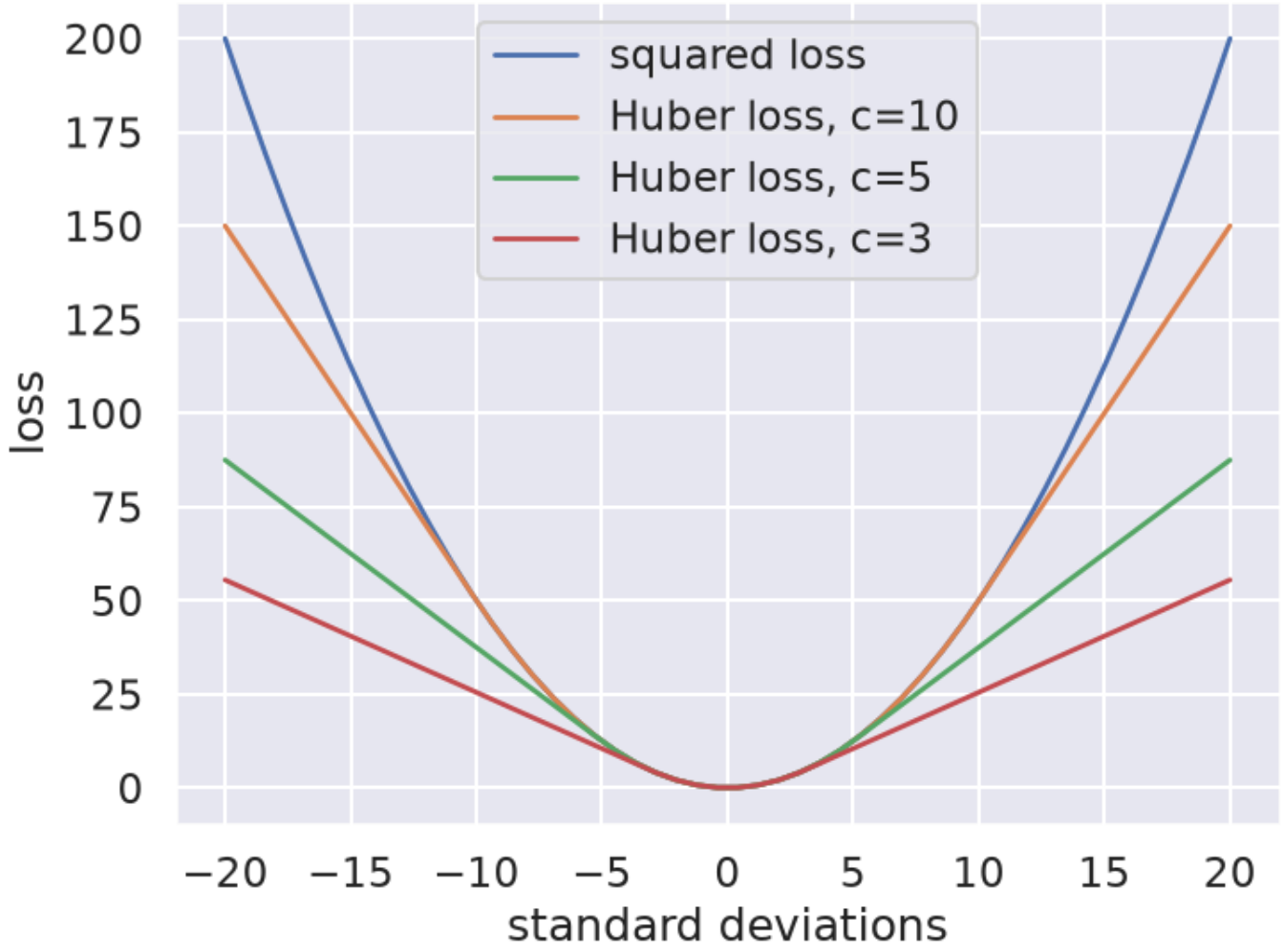
It's clear on examination that the **outliers are exerting a disproportionate influence on the fit**. This is due to the nature of the squared loss function. If you have a single outlier that is, say 10 standard deviations away from the fit, its contribution to the loss will out-weigh that of 25 points which are 2 standard deviations away!

Clearly the squared loss is overly sensitive to outliers, and this is causing issues with our fit. One way to address this within the frequentist paradigm is to simply adjust the loss function to be more robust.

Frequentist Correction for Outliers: Huber Loss

The variety of possible loss functions is quite literally infinite, but one relatively well-motivated option is the Huber loss. The Huber loss defines a critical value at which the loss curve transitions from quadratic to linear. Here is a plot that compares the Huber loss to the standard squared loss for several critical values c:

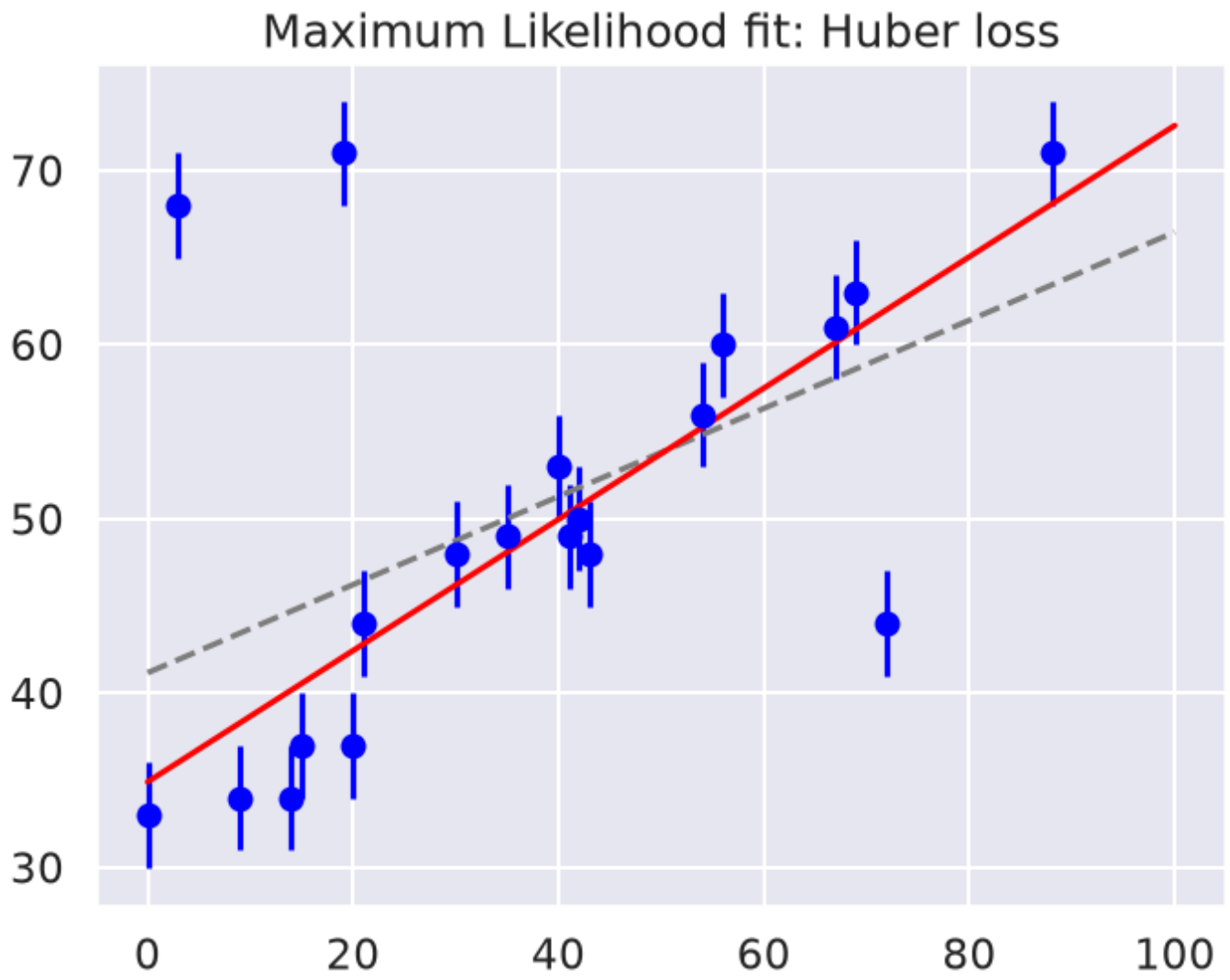
▼ **Hide code cell source**



The Huber loss is equivalent to the squared loss for points that are well-fit by the model, but reduces the loss contribution of outliers. For example, a point 20 standard deviations from the fit has a squared loss of 200, but a c=3 Huber loss of just over 55. Let's see the result of the best-fit line using the Huber loss rather than the squared loss. We'll plot the squared loss result as a dashed gray line for comparison:

▼ **Hide code cell source**

```
Huber: theta0 = 34.9, theta1 = 0.38
```



By eye, this seems to have worked as desired: the fit is much closer to our intuition! *However, a Bayesian might point out that the motivation for this new loss function is a bit suspect: as we showed, the squared-loss can be straightforwardly derived from a Gaussian likelihood.*

- The Huber loss seems a bit *ad hoc*: where does it come from?
- How should we decide what value of c to use?
- Is there any good motivation for using a linear loss on outliers, or should we simply remove them instead?
- How might this choice affect our resulting model?

12.3. Bayesian methods to deal with outliers

Bayesian approaches to erratic data

We'll consider four Bayesian approaches to outliers:

1. A conservative model (treat all data points as equally possible outliers).
2. Good-and-bad data model.

3. The Cauchy formulation.
4. Many nuisance parameters.

First, let's preview our approaches by considering what is problematic about a Gaussian model for the observation errors when we have outliers.

Why are outliers problematic with a Gaussian model?

As noted already, with a Gaussian log likelihood,

$$\log p(D \mid \boldsymbol{\theta}, I) = \text{constant} \quad - \sum_{i=1}^{N} \frac{\left[y_i - y_M(x_i; \boldsymbol{\theta})\right]^2}{2\sigma_0^2},$$

in units of σ_0 it costs a lot to have an outlier y_j far from $y_M(x_j; \boldsymbol{\theta})$ (i.e., to have an outlier with respect to the current model). This distorts the fit of the mean value at x_j, namely $y_M(x_j; \boldsymbol{\theta})$. Let's see how that distortion works analytically.

Suppose we first calculate the log likelihood with N points, none of which are outliers. Then the maximum likelihood parameters $\widehat{\boldsymbol{\theta}}$ are such that

$$\frac{\partial}{\partial \theta_n} \log p(D \mid \boldsymbol{\theta}, I) \bigg|_{\boldsymbol{\theta}=\widehat{\boldsymbol{\theta}}} = 0 \quad \text{for each} \quad \theta_n \in \boldsymbol{\theta}.$$

Now hold $\widehat{\boldsymbol{\theta}}$ fixed but add a new outlier point (x_j, y_j). What happens to the maximum likelihood condition? Only the new term in the sum will contribute because the derivatives of the other will sum to zero when $\boldsymbol{\theta} = \widehat{\boldsymbol{\theta}}$. So now

$$\begin{aligned} \frac{\partial}{\partial \theta_n} \log p(D \mid \boldsymbol{\theta}, I) \bigg|_{\boldsymbol{\theta}=\widehat{\boldsymbol{\theta}}} &= \frac{\partial}{\partial \theta_n} \left[-\frac{1}{2} \frac{\left(y_j - y_M(x_j; \boldsymbol{\theta})\right)^2}{\sigma_0^2} \right] \bigg|_{\boldsymbol{\theta}=\widehat{\boldsymbol{\theta}}} \\ &= -\frac{y_j - y_M(x_j; \widehat{\boldsymbol{\theta}})}{\sigma_0^2} \frac{\partial y_M(x_j; \boldsymbol{\theta})}{\partial \theta_n} \bigg|_{\boldsymbol{\theta}=\widehat{\boldsymbol{\theta}}}, \end{aligned}$$

where only the first factor depends on y_j. Thus this factor will grow without bound:

$$\frac{y_j - y_M(x_j; \widehat{\boldsymbol{\theta}})}{\sigma_0^2} \overset{y_j \to \infty}{\longrightarrow} \infty,$$

and therefore the parameters in $\widehat{\boldsymbol{\theta}}$ are pulled more and more by the single outlier point as the magnitude of the outlier grows. This influence will be the case for any theoretical model, but is intrinsic to the normal distribution for the likelihoo. How to avoid it?

One possibility is to switch to a heavier-tailed distribution that doesn't have this property of unbounded influence. For example, replace the normal distribution by Student's t-distribution (here this is any i):

$$y_i \sim t_\nu(\mu, \sigma) \quad \Longrightarrow \quad p(y_i|\mu,\sigma^2) \propto \Big[1 + \frac{(y_i-\mu)^2}{\nu\sigma^2}\Big],$$

with $\mu = y_M(x_i;\boldsymbol{\theta})$ and $\sigma = \sigma_0$ here. Then

$$\log p(y_i|\mu,\sigma^2) = \text{const.} \;\; - \frac{\nu+1}{2}\log\Big(1 + \frac{(y_i-\mu)^2}{\nu\sigma^2}\Big),$$

and the contribution of y_j to the minimization of the likelihood is (substituting for μ and σ)

$$\begin{aligned}\frac{\partial}{\partial\theta_n}\log p(y_j|y_M(x_j;\boldsymbol{\theta}),\sigma_0^2)\Big|_{\boldsymbol{\theta}=\widehat{\boldsymbol{\theta}}} &= \frac{\nu+1}{2}\,\frac{1}{1+(y_j-y_M(x_j;\widehat{\boldsymbol{\theta}}))^2/\nu\sigma_0^2}\,\frac{2(y_j-y_M(x_i;\widehat{\boldsymbol{\theta}}))}{\nu\sigma_0^2} \\ &= \frac{\nu+1}{\nu\sigma_0^2 + (y_j-y_M(x_j;\widehat{\boldsymbol{\theta}}))^2}\,(y_j-y_M(x_i;\widehat{\boldsymbol{\theta}})).\end{aligned}$$

The t-distribution becomes a Gaussian distribution as $\nu \to \infty$, and we see that in that case the derivative will go to infinity like $y_j - y_M(x_j;\widehat{\boldsymbol{\theta}})$, as we saw earlier. But for any finite ν, the $\big(y_j - y_M(x_j;\widehat{\boldsymbol{\theta}})\big)^2$ factor in the denominator with dominate over the $\nu\sigma_0^2$ factor, and the derivative will vanish as $y_j \to \infty$. In other words, the influence on the parameters from the outlier will go away.

In the next section we will see how a heavy-tailed distribution, such as the Student t distribution, can naturally arise in a Bayesian formulation when the uncertainty is itself uncertain.

Approach no. 1: A conservative formulation

Assuming that the specified error bars, σ_0, can be viewed as a recommended lower bound, we can construct a more conservative posterior through a marginal likelihood (that is, we introduce σ as a supplementary parameter for what might be the true error, then integrate over it):

$$p(D_i|\boldsymbol{\theta},\sigma_0,I) = \int_0^\infty p(D_i|\boldsymbol{\theta},\sigma,I)\,p(\sigma|\sigma_0)\,d\sigma,$$

Note that σ_0 doesn't appear in the first pdf in the integrand and $\boldsymbol{\theta}$ doesn't appear in the second pdf. The prior is a variation of Jeffrey's prior for a scale parameter (which would be $1/\log(\sigma_1/\sigma_0) \times (1/\sigma)$ for $\sigma_0 \le \sigma < \sigma_1$ and zero otherwise),

$$p(\sigma|\sigma_0,I) = \frac{\sigma_0}{\sigma^2},$$

for $\sigma \ge \sigma_0$ and zero otherwise. (This enables us to do the integral up to infinity.) Conservative here means that we treat all data points with suspicion (i.e., we do not single out any points).

The likelihood for a single data point D_i, given by $(x_i, y_i, \sigma_i = \sigma_0)$, is then

$$
\begin{aligned}
p(D_i|\boldsymbol{\theta}, \sigma_0, I) &= \int_{\sigma_0}^{\infty} \frac{1}{\sqrt{2\pi}\sigma} e^{-\sigma_0^2 R_i^2/2\sigma^2} \, \frac{\sigma_0}{\sigma^2} \, d\sigma \\
&= \frac{\sigma_0}{\sqrt{2\pi}} \int_0^{1/\sigma_0} t \, e^{-t^2 \sigma_0^2 R_i^2/2} \, dt \\
&= \frac{1}{\sigma_0 \sqrt{2\pi}} \left[\frac{1 - \exp(-R_i^2/2)}{R_i^2} \right],
\end{aligned}
\tag{12.2}
$$

with R_i the residual as defined above.

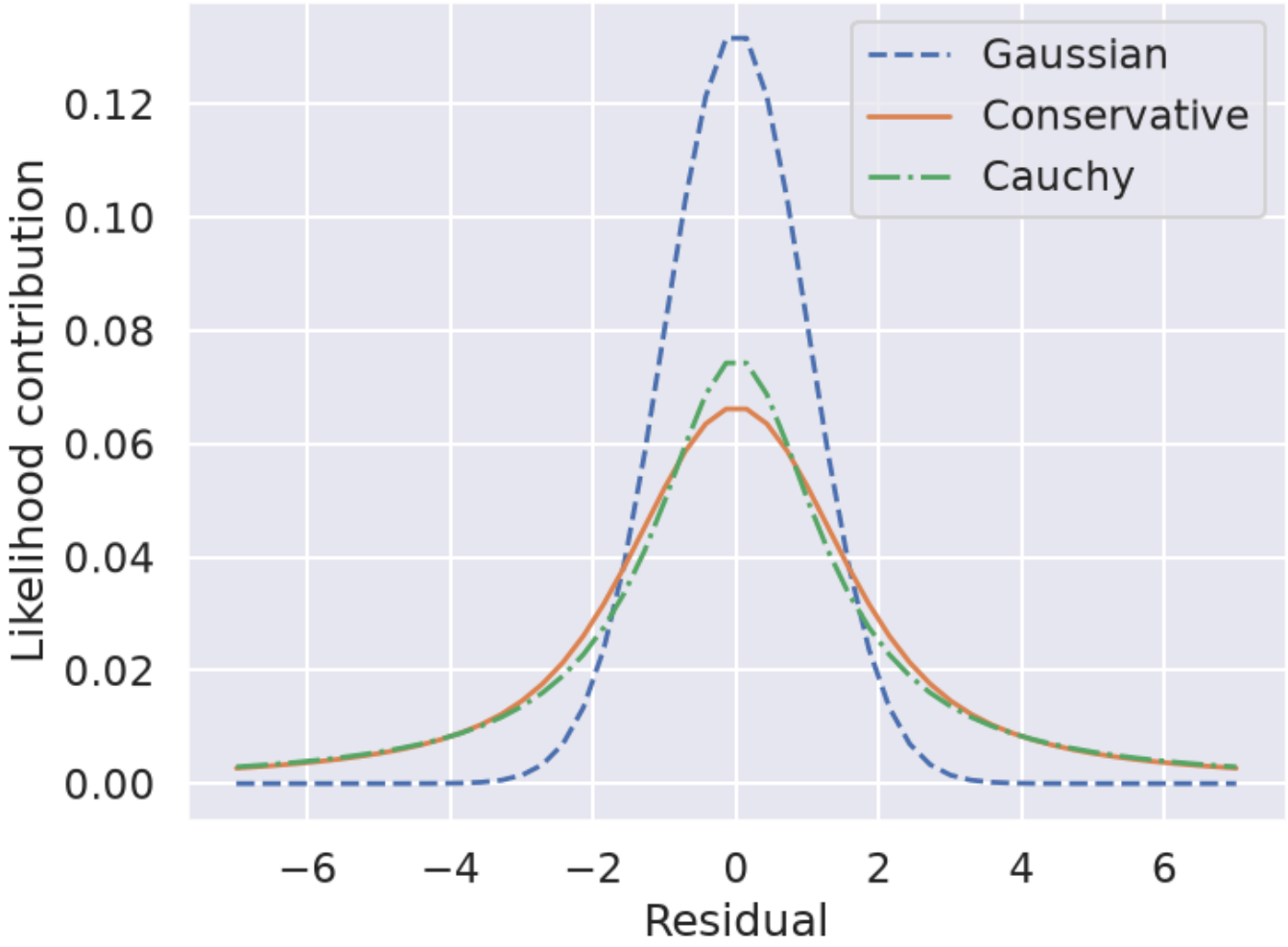
Treating the measurement noise as independent, and assigning a uniform prior for the model parameters, we find the log-posterior pdf

$$
\log\left[p(\boldsymbol{\theta}|D, I)\right] = \text{constant} \quad + \sum_{i=1}^{N} \log\left[\frac{1 - \exp(-R_i^2/2)}{R_i^2} \right].
$$

We'll also consider a Cauchy distribution for comparison.

▼ **Hide code cell source**

```
/usr/share/miniconda3/envs/2025-book-env/lib/python3.11/site-packages/arviz/__init__.py:50:
FutureWarning:
ArviZ is undergoing a major refactor to improve flexibility and extensibility while maintaining
a user-friendly interface.
Some upcoming changes may be backward incompatible.
For details and migration guidance, visit:
https://python.arviz.org/en/latest/user_guide/migration_guide.html
  warn(
```



So by marginalizing over the error for each data point, we introduce long tails into the likelihood. The likelihood function is close to a Cauchy distribution in the tails, which is a particular case of a Student t distribution. (Remember the radioactive lighthouse!)

Now we can sample using this conservative posterior and compare to the others.

If we wanted to get a general Student's t-distribution, we can do an integration of the variance over an inverse-gamma distribution or, equivalently, integrate over the inverse of the variance with a gamma distribtion. An animation of the integration leading to a t-distribution is created in Student's t-distribution from Gaussians.

Approach no. 2: Good-and-bad data

In this approach, we are less pessimistic about the data. In particular, we allow for two possibilities:
a) the datum and its error are reliable;
b) the datum is bad and the error should be larger by a (large) factor γ.

We implement this with the pdf for σ:

$$p(\sigma|\sigma_0, \beta, \gamma, I) = \beta\,\delta(\sigma - \gamma\sigma_0) + (1-\beta)\,\delta(\sigma - \sigma_0)\ ,$$

where β and γ represent the frequency of quirky measurements and their severity. If we use Gaussian pdfs for the two cases, then the likelihood for a datum D_i after integrating over σ is

$$p(D_i|\theta,\sigma_0,\beta,\gamma,I) = \frac{1}{\sigma_0\sqrt{2\pi}}\left\{\frac{\beta}{\gamma}\,e^{-R_i^2/2\gamma^2} + (1-\beta)\,e^{-R_i^2/2}\right\}$$

We won't provide code for this approach here but leave it to an exercise (see Sivia, Ch. 8.3.2 for more details). However, we will generalize the idea to the extreme in approach #4 below.

Approach no. 3: The Cauchy formulation

In this approach, we assume $\sigma \approx \sigma_0$ but allow it to be either narrower or wider:

$$p(\sigma|\sigma_0,I) = \frac{2\sigma_0}{\sqrt{\pi}\sigma^2}e^{-\sigma_0^2/\sigma^2}\ .$$

Marginalizing σ (using $\sigma = 1/t$) gives the Cauchy form likelihood (this is a special case of the Student t distribution):

$$p(D_i|\boldsymbol{\theta},\sigma_0,I) = \frac{1}{\sigma_0\pi\sqrt{2}[1+R_i^2(\boldsymbol{\theta})/2]}\ .$$

The log posterior is

$$L(\boldsymbol{\theta}) = \text{constant} \quad -\sum_{i=1}^{N}\log\left[1+\frac{R_i^2(\boldsymbol{\theta})}{2}\right]$$

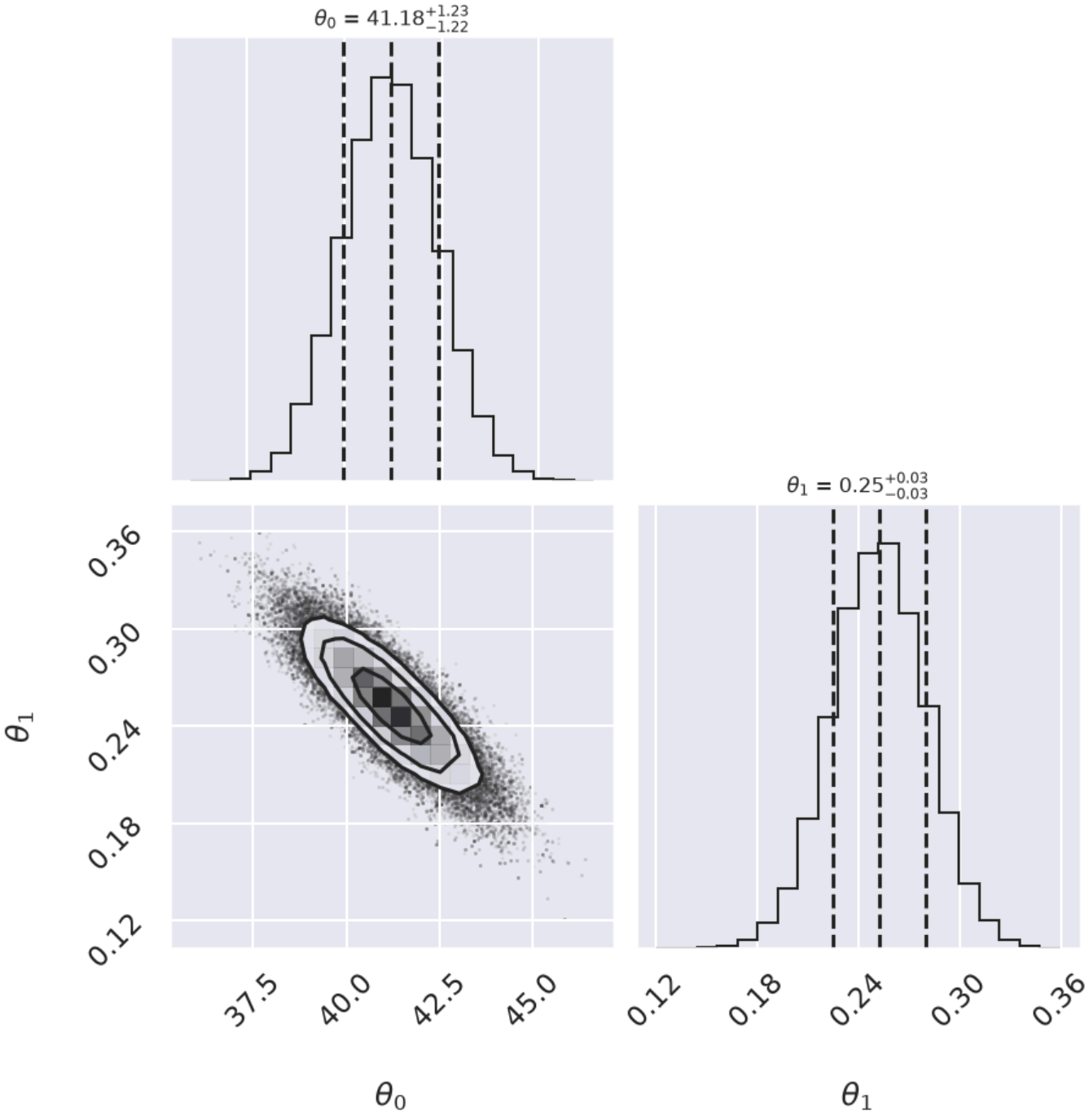
Sampling to compare methods

We'll use the `emcee` sampler to generate MCMC samples of our posteriors.

▼ **Hide code cell source**

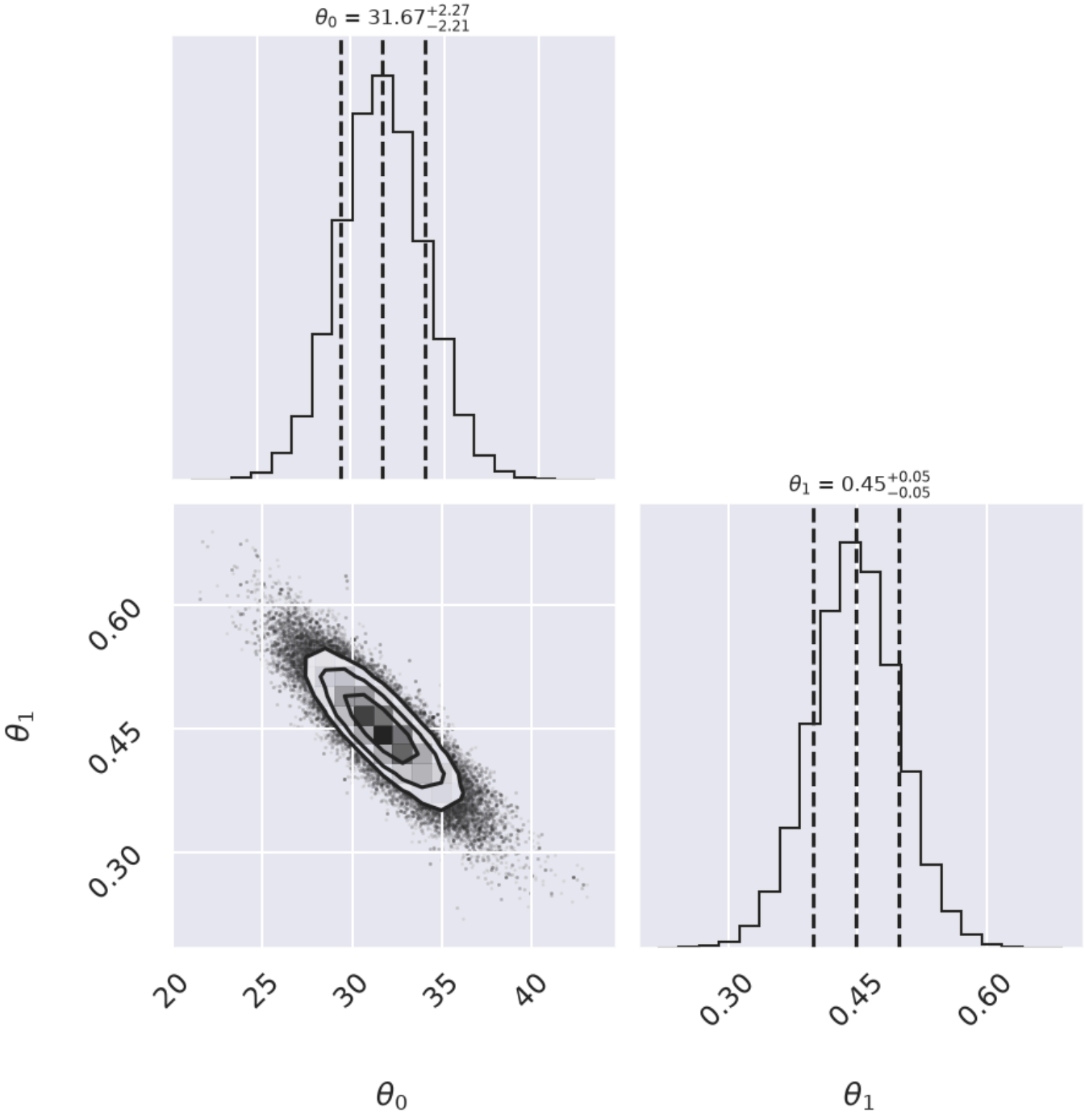
```
emcee sampling (version: ) 3.1.6
10 walkers:
Log posterior: Std Gaussian
... EMCEE sampler performing 1000 warnup iterations.


... EMCEE sampler performing 10000 samples.
```

```
CPU times: user 3.62 s, sys: 849 µs, total: 3.62 s
Wall time: 3.62 s
done
```



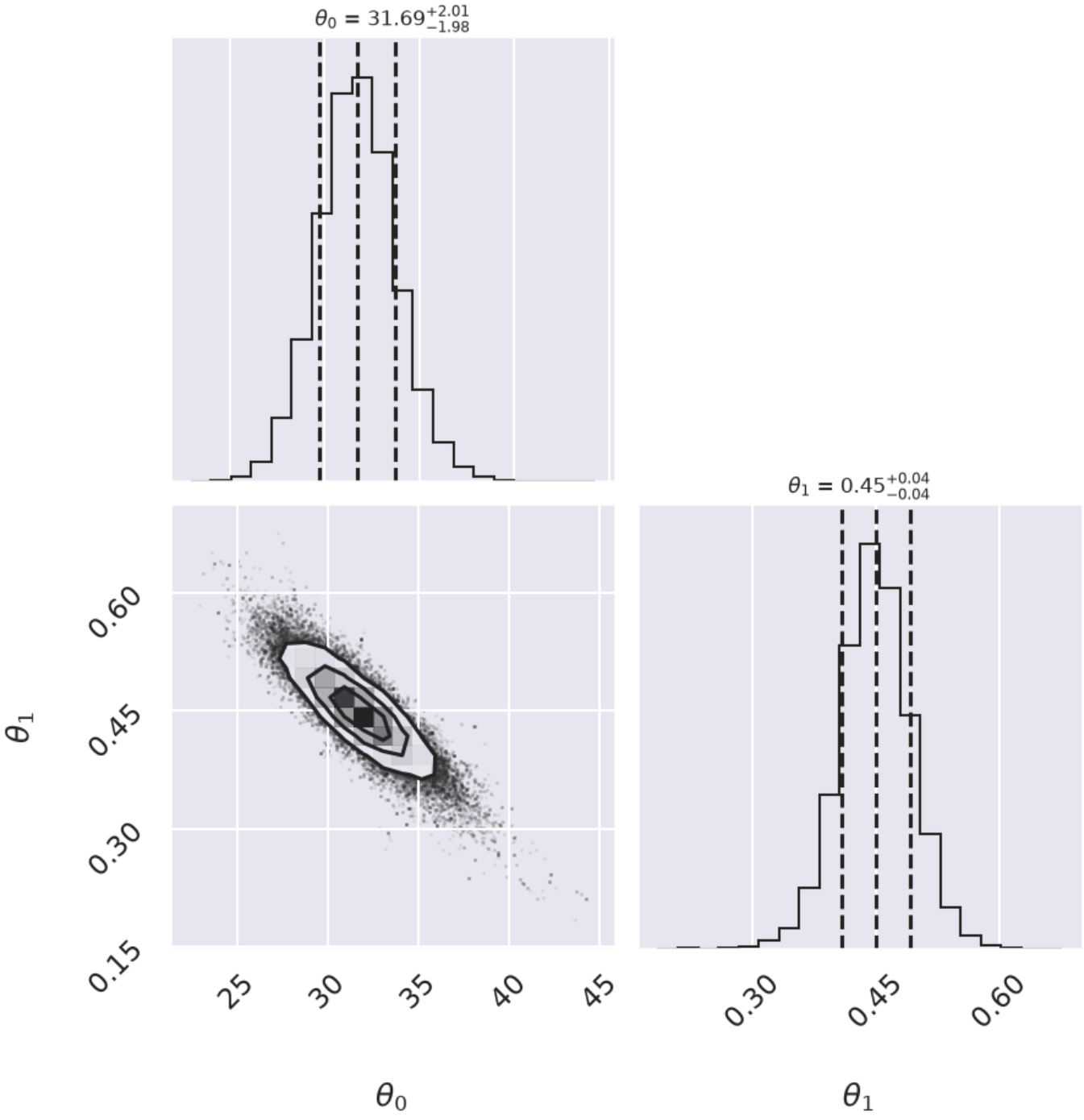

```
Log posterior: Conservative
... EMCEE sampler performing 1000 warnup iterations.


... EMCEE sampler performing 10000 samples.
```

```
CPU times: user 4.07 s, sys: 1.94 ms, total: 4.07 s
Wall time: 4.07 s
done
```

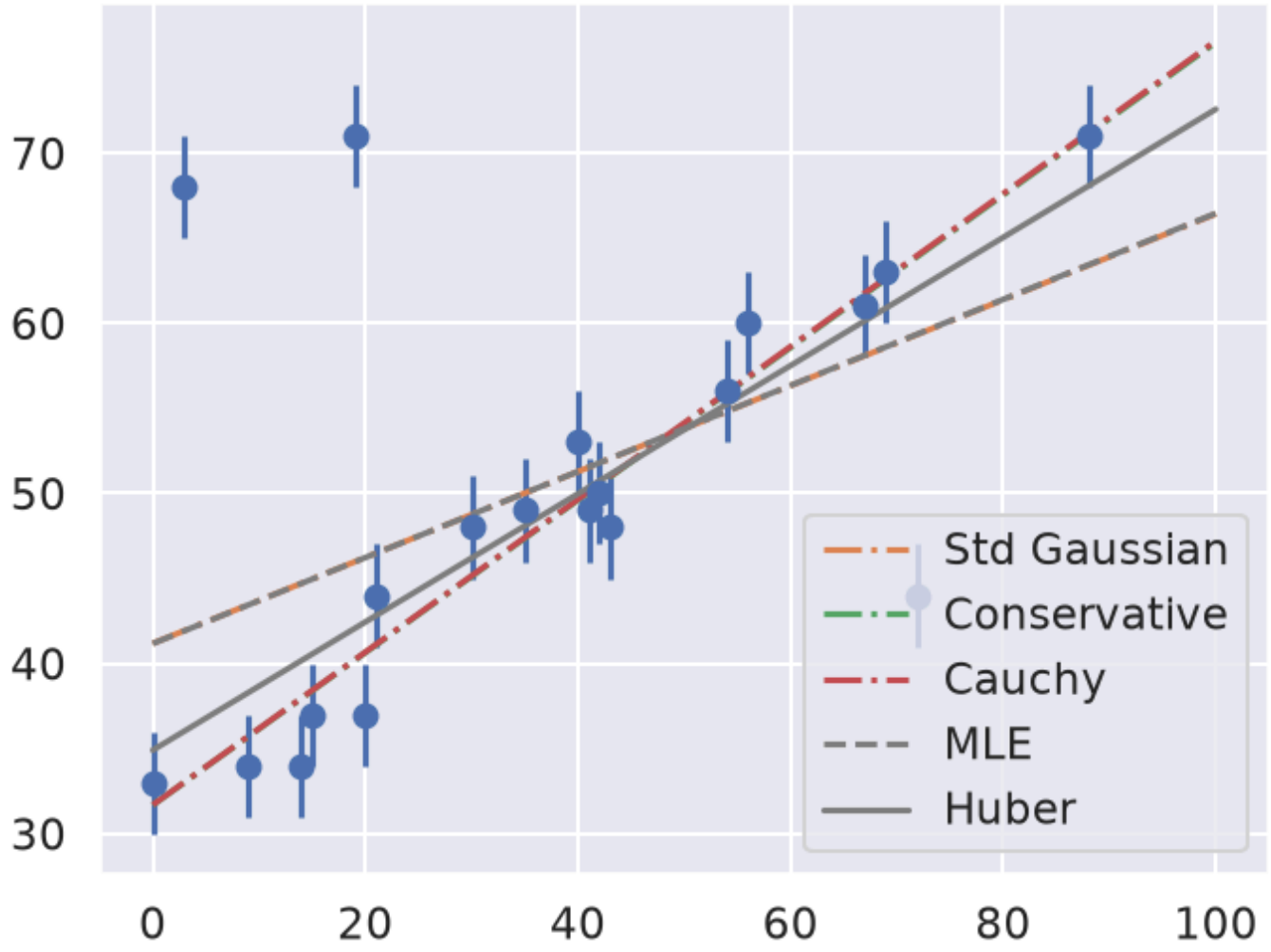



```
Log posterior: Cauchy
... EMCEE sampler performing 1000 warnup iterations.


... EMCEE sampler performing 10000 samples.
```

```
CPU times: user 3.99 s, sys: 971 µs, total: 3.99 s
Wall time: 3.99 s
done
```



Summary:	Mean offset	68% CR	Mean slope	68% CR
Std Gaussian	41.18	-1.22,+1.23	0.252	-0.027,+0.027
Conservative	31.67	-2.21,+2.27	0.448	-0.050,+0.049
Cauchy	31.69	-1.98,+2.01	0.449	-0.042,+0.041



In this plot we see a comparison of five approaches. The standard Gaussian and MLE lines are the same, showing the deleterious effect of the outliers. The Huber loss approach gives a reasonable fit (by eye). The conservative and Cauchy fits are indistinguishable and similar but not identical to the Huber loss fit. Which do you think is better?

Approach no. 4: Many nuisance parameters

The Bayesian approach to accounting for outliers generally involves *modifying the model* so that the outliers are accounted for. For this data, it is abundantly clear that a simple straight line is not a good fit to our data. So let's propose a more complicated model that has the flexibility to account for outliers. One option is to choose a mixture between a signal and a background:

$$p(\{y_i\} \mid \{x_i\}, \{e_i\}, \boldsymbol{\theta}, \{g_i\}, \sigma_B) = \frac{g_i}{\sqrt{2\pi e_i^2}} \exp\left[\frac{-\left(\hat{y}(x_i \mid \boldsymbol{\theta}) - y_i\right)^2}{2e_i^2}\right] + \frac{1-g_i}{\sqrt{2\pi\sigma_B^2}} \exp\left[\frac{-\left(\hat{y}(x_i \mid \boldsymbol{\theta}) - y_i\right)^2}{2\sigma_B^2}\right]$$

What we've done is expanded our model with some nuisance parameters: $\{g_i\}$ is a series of weights which range from 0 to 1 and encode for each point i the degree to which it fits the model. $g_i = 0$ indicates an outlier, in which

case a Gaussian of width σ_B is used in the computation of the likelihood. This σ_B can also be a nuisance parameter, or its value can be set at a sufficiently high number, say 50.

Our model is much more complicated now: it has 22 free parameters rather than 2, but the majority of these can be considered nuisance parameters, which can be marginalized-out in the end, just as we marginalized (integrated) over p in the Billiard example. Let's construct a function which implements this likelihood. We'll use the emcee package to explore the parameter space.

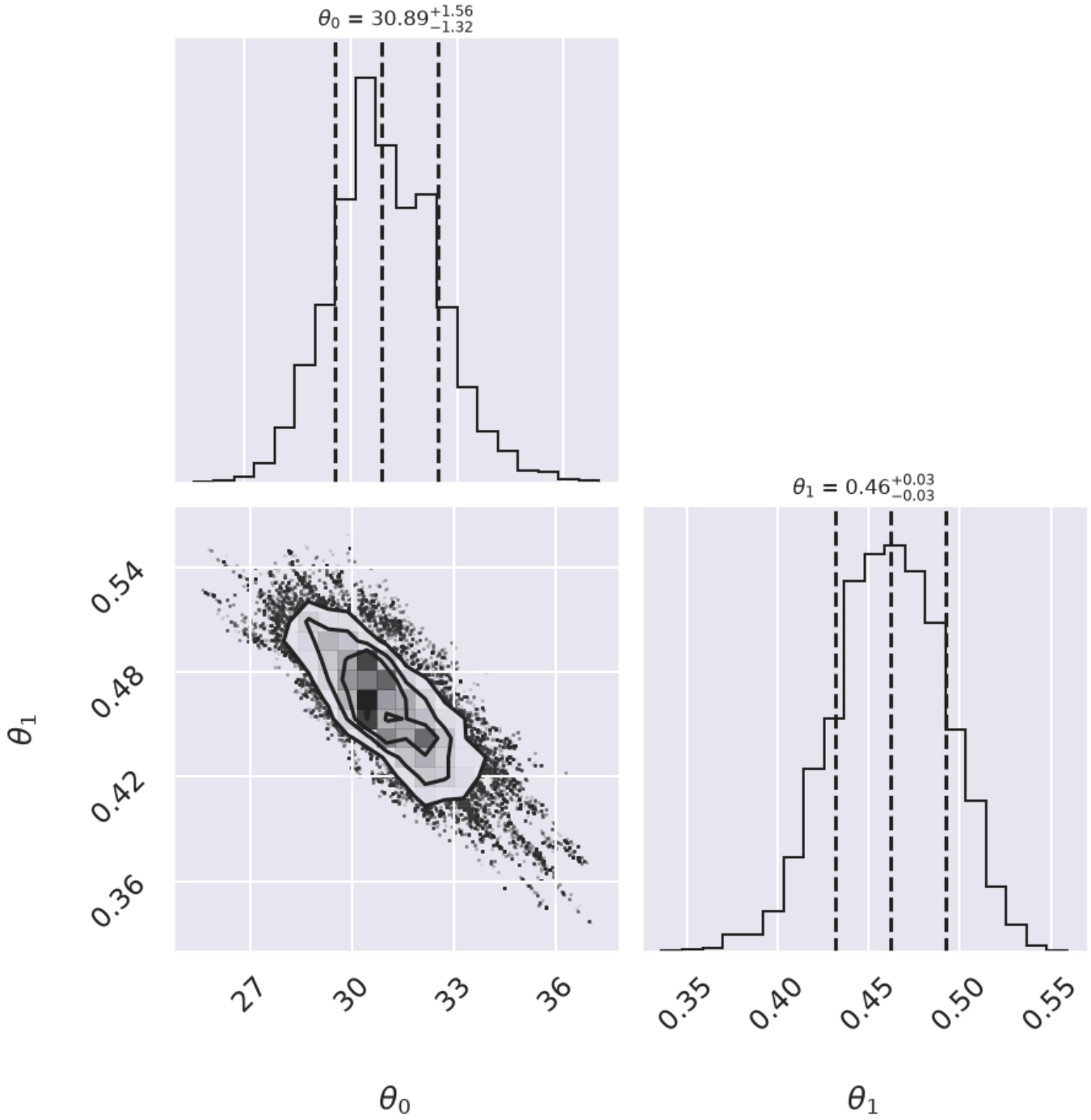
To actually compute this, we'll start by defining functions describing our prior, our likelihood function, and our posterior:

Now we'll run the MCMC sampler to explore the parameter space:

Once we have these samples, we can exploit a very nice property of the Markov chains. Because their distribution models the posterior, we can integrate out (i.e. marginalize) over nuisance parameters simply by ignoring them!

We can look at the (marginalized) distribution of slopes and intercepts by examining the first two columns of the sample:

▼ **Hide code cell source**

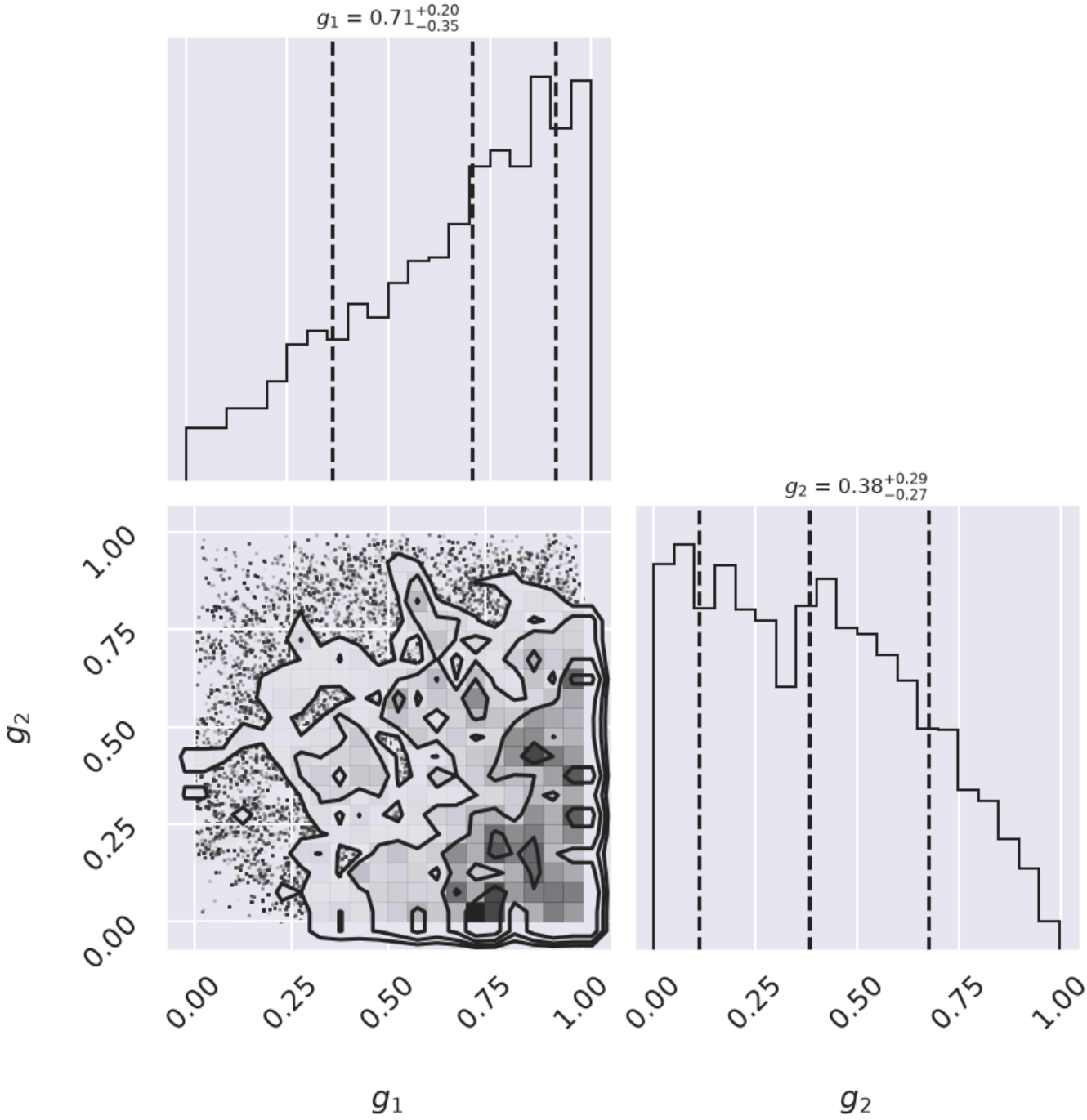


We see a distribution of points near a slope of $\sim 0.4 - 0.5$, and an intercept of $\sim 29 - 34$. We'll plot this model over the data below, but first let's see what other information we can extract from this trace.

One nice feature of analyzing MCMC samples is that the choice of nuisance parameters is completely symmetric: just as we can treat the $\{g_i\}$ as nuisance parameters, we can also treat the slope and intercept as nuisance parameters! Let's do this, and check the posterior for g_1 and g_2, the outlier flag for the first two points:

▼ **Hide code cell source**

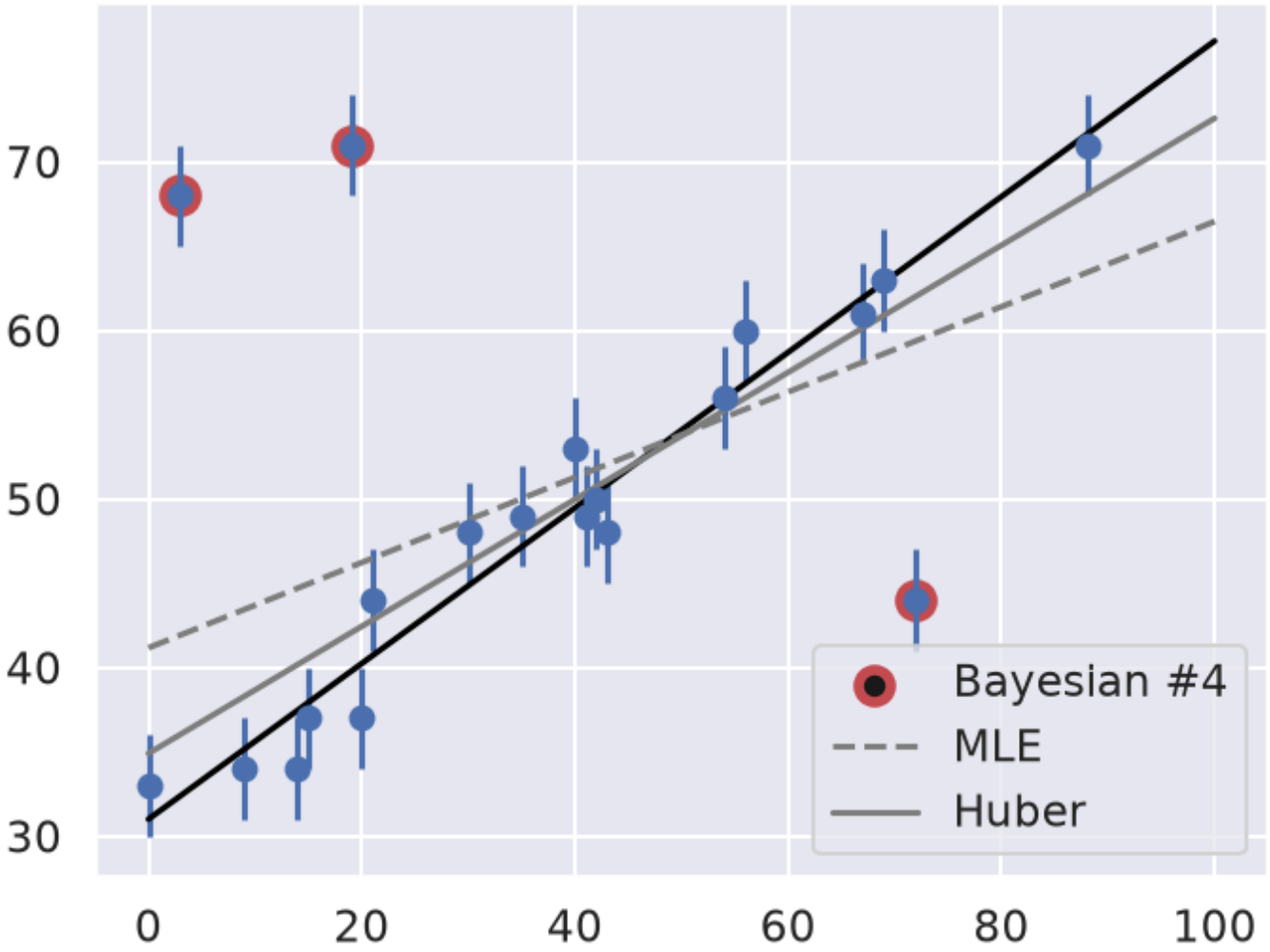
```
g1 mean: 0.65
g2 mean: 0.39
```



There is not an extremely strong constraint on either of these, but we do see that $(g_1, g_2) = (1, 0)$ is slightly favored: the means of g_1 and g_2 are greater than and less than 0.5, respectively. If we choose a cutoff at $g = 0.5$, our algorithm has identified g_2 as an outlier.

Let's make use of all this information, and plot the marginalized best model over the original data. As a bonus, we'll draw red circles to indicate which points the model detects as outliers:

▼ **Hide code cell source**



The result, shown by the dark line, matches our intuition! Furthermore, the points automatically identified as outliers are the ones we would identify by hand. For comparison, the gray lines show the two previous approaches: the simple maximum likelihood and the frequentist approach based on Huber loss.

Discussion

Here we've dived into linear regression in the presence of outliers. A typical Gaussian maximum likelihood approach fails to account for the outliers, but we were able to correct this in the frequentist paradigm by modifying the loss function, and in the Bayesian paradigm by adopting a mixture model with a large number of nuisance parameters.

Both approaches have their advantages and disadvantages: the frequentist approach here is relatively straightforward and computationally efficient, but is based on the use of a loss function which is not particularly well-motivated. The Bayesian approach is well-founded and produces very nice results, but it is much more intensive in both coding time and computational time. It also requires a specification of a prior that must be motivated. The advantage is that this prior choice can be tested and compared to alternatives. While you will often see this choice described in the literature (mostly older literature now) as "subjective", in fact it is a more objective approach because of the ability to evaluate it!

12.4. 📥 Demo: Illustrating approaches to outliers

In this demo we re-apply the frequentist and Bayesian methods illustrated in the last two sections to account for outliers in an analysis of a linear fit to noisy data. The difference here is that you are able to generate new datasets to test the robustness of the results.

(Adapted from the blog post Frequentism and Bayesianism II: When Results Differ).

Example: good data / bad data

In the previous sections we used a fixed dataset relating the observed variables x and y, with the error of y stored in e. Here we will generate new datasets with the function `generate_line_data_with_outliers`, which takes parameters that let you adjust the (random) generation of points with a specified number of outliers.

```
%matplotlib inline

import numpy as np
from scipy import optimize
import matplotlib.pyplot as plt
import seaborn as sns; sns.set(); sns.set_context("talk")

import emcee
import corner
```

```
import numpy as np

def generate_line_data_with_outliers(
    num=20,
    x_min=0,
    x_max=100,
    a=0.5,
    b=30.0,
    sig0=3.0,
    n_outlier=3,
    outlier_scale=10.0,
    seed=None,
    integer_x=True,
    sort_x=True,
):
    """
    Generate data for a straight-line model with Gaussian noise and outliers.

    Model:
        y = a*x + b + Gaussian noise

    Outliers:
        n_outlier randomly selected points have an additional displacement
        that is forced to be relatively far from the line.

        The displacement has random sign and magnitude approximately
        outlier_scale * sig0.

    Parameters
    ----------
    num : int
        Number of data points.
    x_min, x_max : float
        Range for random x values.
    a, b : float
        Parameters of the line y = a*x + b.
    sig0 : float
        Standard deviation of the normal observational noise.
    n_outlier : int
        Number of points to turn into outliers.
    outlier_scale : float
        Typical outlier displacement in units of sig0.
    seed : int or None
        Random seed for reproducibility.
    integer_x : bool
        If True, generate integer x values.
    sort_x : bool
        If True, sort the data by increasing x.

    Returns
    -------
    x : ndarray
        x values.
    y : ndarray
        noisy y values, including outliers.
    e : ndarray
        error bars, all equal to sig0.
    outlier_indices : ndarray
        indices of the points selected as outliers.
    """

    rng = np.random.default_rng(seed)

    if n_outlier > num:
```

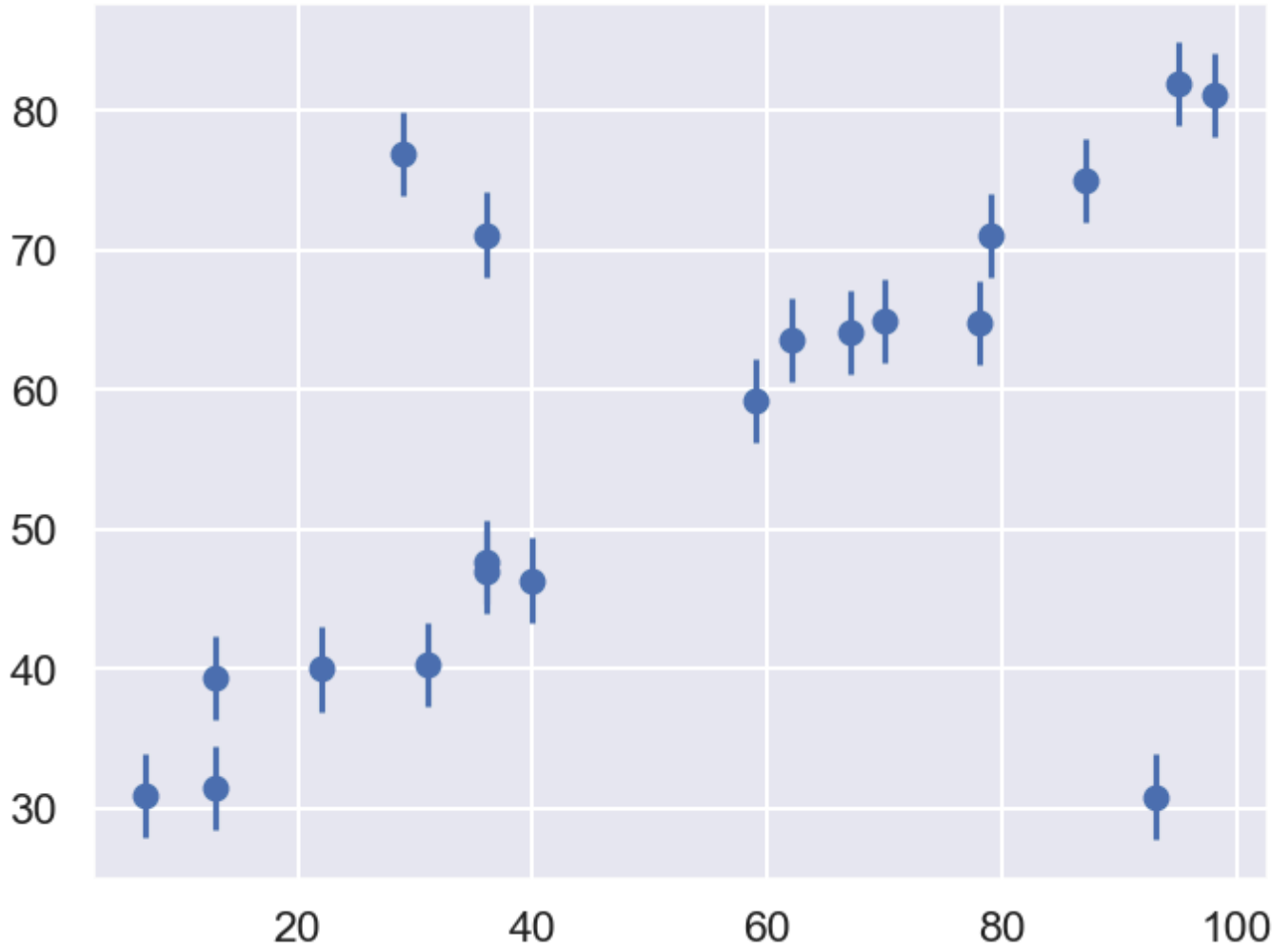
```
        raise ValueError("n_outlier cannot be larger than num.")

    # Generate x values
    if integer_x:
        x = rng.integers(x_min, x_max + 1, size=num)
    else:
        x = rng.uniform(x_min, x_max, size=num)

    # Optionally sort x values
    if sort_x:
        order = np.argsort(x)
        x = x[order]

    # Gaussian noise about the line
    y_true = a * x + b
    y = y_true + rng.normal(loc=0.0, scale=sig0, size=num)

    # Choose which points will become outliers
    outlier_indices = rng.choice(num, size=n_outlier, replace=False)

    # Add large outlier displacements.
    # Each outlier is shifted either up or down.
    signs = rng.choice([-1, 1], size=n_outlier)

    # The absolute displacement is centered around outlier_scale * sig0.
    # Taking abs() helps prevent accidentally tiny outlier shifts.
    magnitudes = np.abs(
        rng.normal(
            loc=outlier_scale * sig0,
            scale=0.5 * outlier_scale * sig0,
            size=n_outlier
        )
    )

    y[outlier_indices] += signs * magnitudes

    # Error bars
    e = sig0 * np.ones_like(y, dtype=float)

    return x, y, e, sig0, outlier_indices
```

```
# Generate a dataset. Note that seed can be omitted to
#   get a new dataset everytime this cell is run.
# We don't print out the outlier_indices by default, but you can
#  add a print statement to check against the final result.
x, y, e, sig0, outlier_indices = generate_line_data_with_outliers(
    num=20,
    x_min=0,
    x_max=100,
    a=0.5,
    b=30.0,
    sig0=3.0,
    n_outlier=3,
    outlier_scale=10.0,
    seed=101
)

np.set_printoptions(precision=0, suppress=True)  # Set 3 decimal places
print("x = ", x)
print("y = ", y)
print("e = ", e)
```

```
x =  [ 7 13 13 22 29 31 36 36 36 40 59 62 67 70 78 79 87 93 95 98]
y =  [31. 39. 32. 40. 77. 40. 48. 71. 47. 46. 59. 64. 64. 65. 65. 71. 75. 31.
 82. 81.]
e =  [3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3.]
```

We'll visualize this data below:

```
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(1,1,1)
ax.errorbar(x, y, e, fmt='o');
```



Our task is to find a line of best-fit to the data. The statistical model for the data is the same as introduced at the beginning of the chapter.

Standard Likelihood Approach

For simplicity, we'll use scipy's `optimize` package to minimize the loss (in the case of squared loss, this computation can be done more efficiently using matrix methods, but we'll use numerical minimization for simplicity here).

```
def residuals(theta, x=x, y=y, sigma0=sig0):
    """
    Residuals between data y (a vector at points x) and the theoretical model,
    which here is a straight line with theta[0] = b and theta[1] = m.
    """
    delta_y = y - theta[0] - theta[1] * x
    return delta_y / sigma0


def log_posterior_gaussian(theta):
    """
    Returns the logarithm of the posterior, with a standard chi^2 likelihood
    in terms of the residuals, with Gaussian errors as specified and a
    uniform prior assumed for theta between 0 and 100.
    """
    if (all(theta > 0) and all(theta < 100)):
        return -0.5 * np.sum(residuals(theta)**2)
    else:
        return -np.inf  # recall log(0) = -inf
```

```
def squared_loss(theta, x=x, y=y, sigma0=sig0):
    """Loss function is sum of squared residuals divided by 2, which
        is what we usually call chi^2 / 2.
    """
    delta_y = y - theta[0] - theta[1] * x
    return np.sum(0.5 * (delta_y / sigma0) ** 2)

# Find the maximum likelihood estimate (MLE) for theta by minimizing the
#  square_loss function using scipy.optimize.fmin. (Minimizing chi^2 gives
#  the same result as maximizing e^{chi^2/2}.)
theta_MLE = optimize.fmin(squared_loss, [0, 0], disp=False)
print(f"MLE: theta0 = {theta_MLE[0]:.1f}, theta1 = {theta_MLE[1]:.2f}")

# Plot the MLE fit versus the data
xfit = np.linspace(0, 100)
yfit = theta_MLE[0] + theta_MLE[1] * xfit

fig, ax = plt.subplots(1, 1, figsize=(8,6))
ax.errorbar(x, y, sig0, fmt='o', color='blue')
ax.plot(xfit, yfit, color='black')
ax.set_title('Maximum Likelihood fit: Squared Loss');
```

```
MLE: theta0 = 37.2, theta1 = 0.36
```

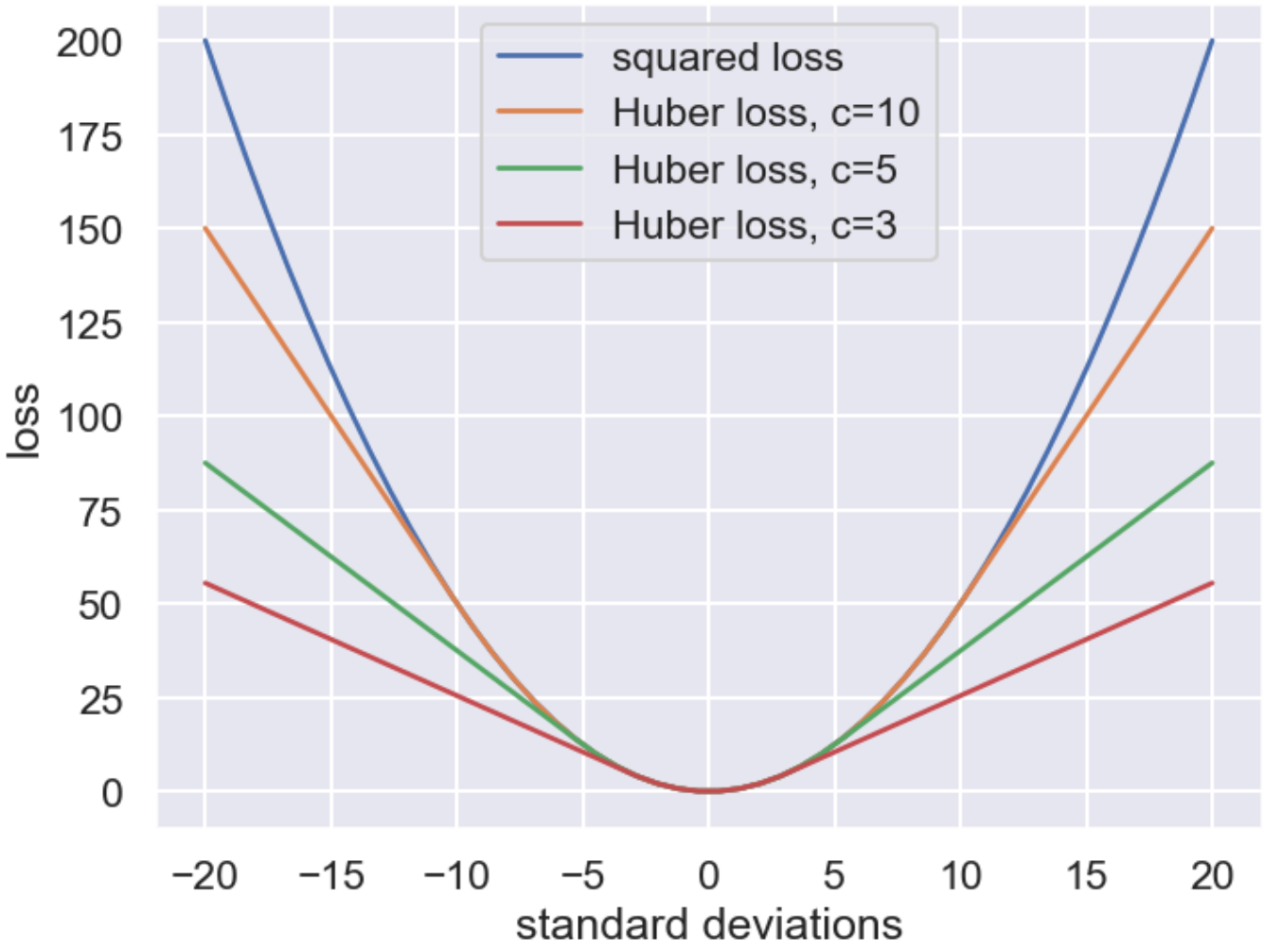
- Are the outliers exerting a disproportionate influence on the fit for this dataset?
- If so, what is it about the squared loss function that makes this likely to happen.

In most cases we will find that the squared loss is overly sensitive to outliers, and this is causing issues with our fit. One way to address this within the frequentist paradigm is to simply adjust the loss function to be more robust.

Frequentist Correction for Outliers: Huber Loss

The variety of possible loss functions is quite literally infinite, but one relatively well-motivated option is the Huber loss. The Huber loss defines a critical value at which the loss curve transitions from quadratic to linear. Let's create a plot which compares the Huber loss to the standard squared loss for several critical values c:

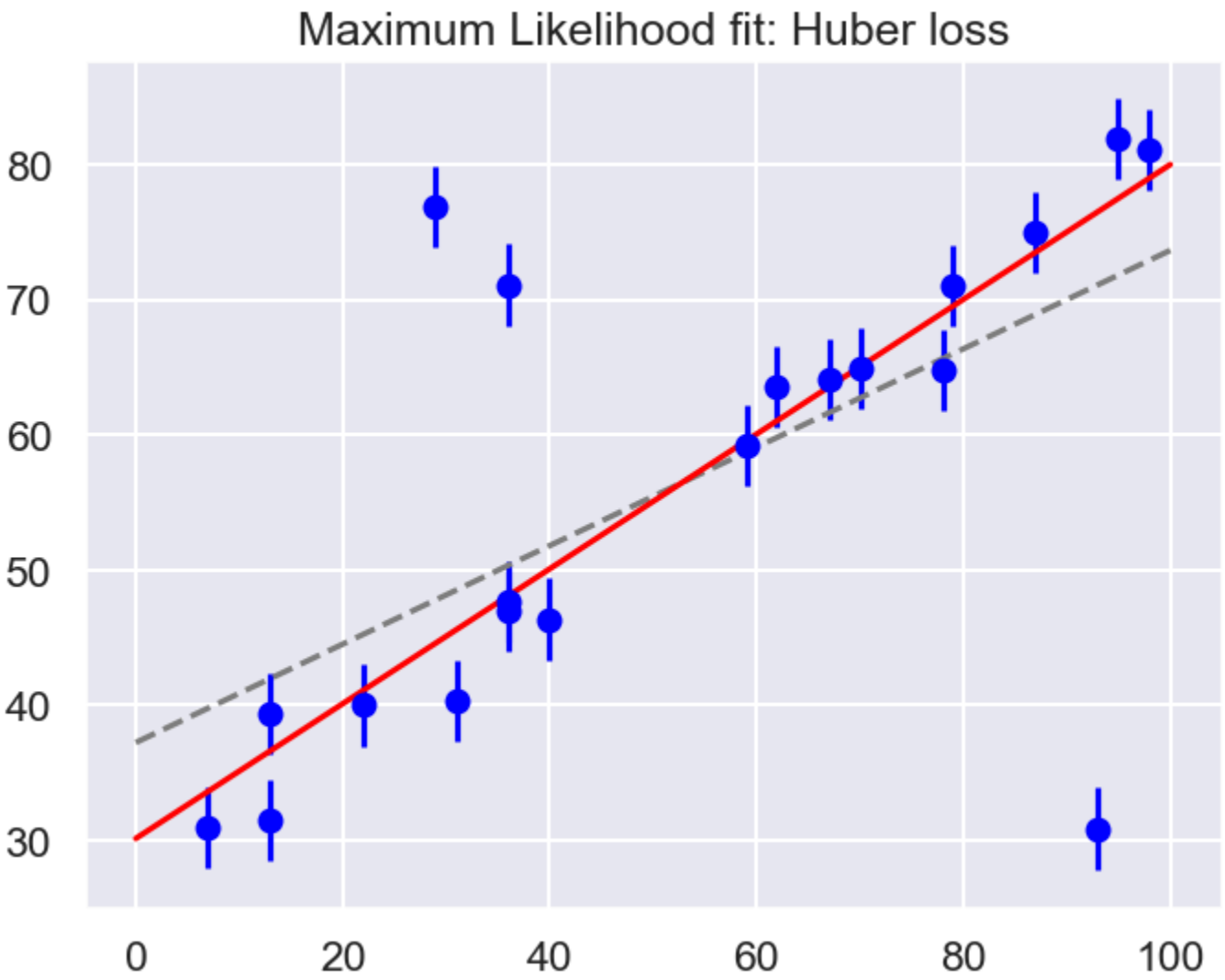
```
t = np.linspace(-20, 20)

def huber_loss(t, c=3):
    """
    Returns either a squared lost function or a linear (abolute value) loss
     function, depending on whether the |argument| is < c or >= c.
    """
    return ((abs(t) < c) * 0.5 * t ** 2
            + (abs(t) >= c) * -c * (0.5 * c - abs(t)))

fig, ax = plt.subplots(1, 1, figsize=(8,6))
ax.plot(t, 0.5 * t ** 2, label="squared loss", lw=2)
for c in (10, 5, 3):
    ax.plot(t, huber_loss(t, c), label=f"Huber loss, c={c}", lw=2)
ax.set(ylabel='loss', xlabel='standard deviations')
ax.legend(loc='best');
```



The Huber loss is equivalent to the squared loss for points that are well-fit by the model, but reduces the loss contribution of outliers. For example, a point 20 standard deviations from the fit has a squared loss of 200, but a c=3 Huber loss of just over 55. Let's see the result of the best-fit line using the Huber loss rather than the squared loss. We'll plot the squared loss result as a dashed gray line for comparison:

```
def total_huber_loss(theta, x=x, y=y, sigma0=sig0, c=3):
    return huber_loss((y - theta[0] - theta[1] * x) / sigma0, c).sum()

# minimize the total Huber loss for c=3
theta2 = optimize.fmin(total_huber_loss, [0, 0], disp=False)
print(f"Huber: theta0 = {theta2[0]:.1f}, theta1 = {theta2[1]:.2f}")

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(1,1,1)
ax.errorbar(x, y, sig0, fmt='o', color='blue')
ax.plot(xfit, theta_MLE[0] + theta_MLE[1] * xfit, color='gray',ls='--')
ax.plot(xfit, theta2[0] + theta2[1] * xfit, color='red')
ax.set_title('Maximum Likelihood fit: Huber loss');
```

```
Huber: theta0 = 30.1, theta1 = 0.50
```



- By eye, has this worked as desired? Namely, is the fit much closer to our intuition?
- As noted in an earlier section, a Bayesian might point out that the motivation for this new loss function is a bit suspect: as we showed, the squared-loss can be straightforwardly derived from a Gaussian likelihood.
    - The Huber loss seems a bit *ad hoc*: where does it come from?

- How should we decide what value of c to use?
- Is there any good motivation for using a linear loss on outliers, or should we simply remove them instead?
- How might this choice affect our resulting model?

Bayesian approaches to erratic data

We'll consider four Bayesian approaches to outliers:

1. A conservative model (treat all data points as equally possible outliers).
2. Good-and-bad data model.
3. The Cauchy formulation.
4. Many nuisance parameters.

Approach no. 1: A conservative formulation

Assuming that the specified error bars, σ_0, can be viewed as a recommended lower bound, we can construct a more conservative posterior through a marginal likelihood (that is, we introduce σ as a supplementary parameter for what might be the true error, then integrate over it):

$$p(D_i|\boldsymbol{\theta}, \sigma_0, I) = \int_0^\infty p(D_i|\boldsymbol{\theta}, \sigma, I)\, p(\sigma|\sigma_0)\, d\sigma,$$

Note that σ_0 doesn't appear in the first pdf in the integrand and $\boldsymbol{\theta}$ doesn't appear in the second pdf. The prior is a variation of Jeffrey's prior for a scale parameter (which would be $1/\log(\sigma_1/\sigma_0) \times (1/\sigma)$ for $\sigma_0 \le \sigma < \sigma_1$ and zero otherwise),

$$p(\sigma|\sigma_0, I) = \frac{\sigma_0}{\sigma^2},$$

for $\sigma \ge \sigma_0$ and zero otherwise. (This enables us to do the integral up to infinity.) Conservative here means that we treat all data points with suspicion (i.e., we do not single out any points).

The likelihood for a single data point D_i, given by $(x_i, y_i, \sigma_i = \sigma_0)$, is then

$$\begin{aligned} p(D_i|\boldsymbol{\theta}, \sigma_0, I) &= \int_{\sigma_0}^\infty \frac{1}{\sqrt{2\pi}\sigma} e^{-\sigma_0^2 R_i^2/2\sigma^2}\, \frac{\sigma_0}{\sigma^2}\, d\sigma \\ &= \frac{\sigma_0}{\sqrt{2\pi}} \int_0^{1/\sigma_0} t\, e^{-t^2\sigma_0^2 R_i^2/2}\, dt \\ &= \frac{1}{\sigma_0\sqrt{2\pi}} \left[\frac{1 - \exp(-R_i^2/2)}{R_i^2}\right], \end{aligned} \tag{12.3}$$

with R_i the residual as defined above.

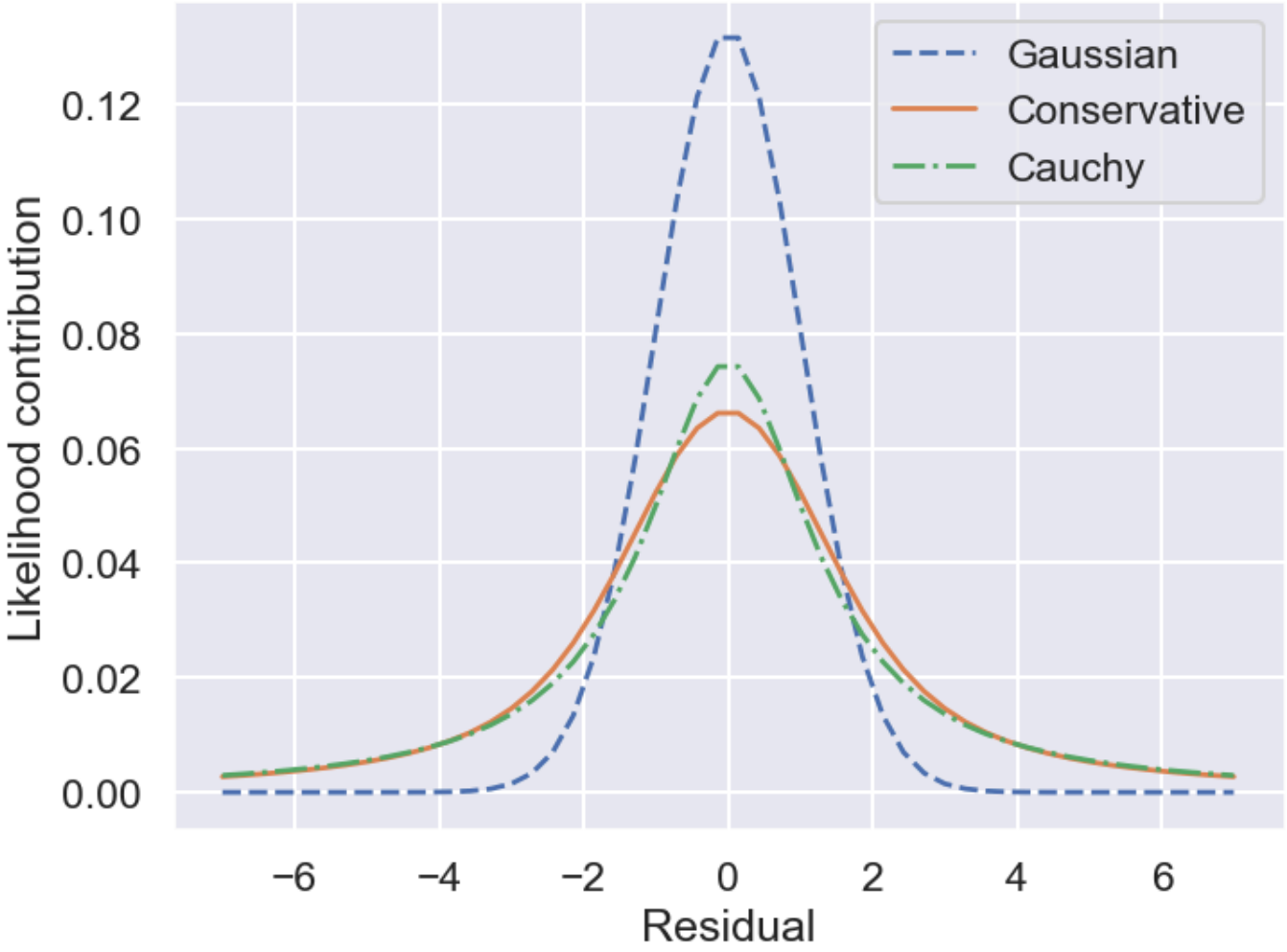
Treating the measurement noise as independent, and assigning a uniform prior for the model parameters, we find the log-posterior pdf

$$\log\left[p(\boldsymbol{\theta}|D,I)\right] = \text{constant} \quad + \sum_{i=1}^{N} \log\left[\frac{1-\exp(-R_i^2/2)}{R_i^2}\right].$$

We'll also consider a Cauchy distribution for comparison.

```
def single_gaussian_likelihood(R_i, sigma_0):
    """
    This likelihood is a zero-mean gaussian with standard deviation sigma_0.
    """
    rsq = R_i**2
    return np.exp(-rsq / 2.) / (sigma_0 * np.sqrt(2 * np.pi ))

def single_conservative_likelihood(R_i, sigma_0):
    """
    This likelihood is the result of marginalizing over the standard
    deviation sigma from sigma0 to infinity with a prior sigma0 / sigma^2.
    """
    rsq = R_i**2
    return (1-np.exp(-rsq / 2.)) / (sigma_0 * np.sqrt(2. * np.pi) * rsq)

def single_cauchy_likelihood(R_i, sigma_0):
    """
    This likelihood is a Cauchy distribution.
    """
    rsq = R_i**2
    return 1 / (sigma_0 * np.pi * np.sqrt(2) * (1. + rsq / 2.))
```

```
r = np.linspace(-7, 7)

fig, ax = plt.subplots(1, 1, figsize=(8,6))

ax.plot(r, single_gaussian_likelihood(r,sig0), label="Gaussian",
        lw=2, ls='--')
ax.plot(r, single_conservative_likelihood(r,sig0), label="Conservative",
        lw=2, ls='-')
ax.plot(r, single_cauchy_likelihood(r,sig0), label="Cauchy",
        lw=2, ls='-.')
ax.set(ylabel='Likelihood contribution',xlabel='Residual')
ax.legend(loc='best');
```



So by marginalizing over the error for each data point, we introduce long tails into the likelihood. The likelihood function is close to a Cauchy distribution in the tails, which is a particular case of a Student t distribution. (Remember the radioactive lighthouse!)

```
# Conservative error likelihood
def log_posterior_conservative(theta):
    """
    Log posterior with uniform prior for theta and a Gaussian likelihood
    """
    if (all(theta > 0) and all(theta < 100)):
        r2 = residuals(theta)**2
        return np.sum( np.log((1-np.exp(-r2/2))/r2) )
    else:
        return -np.inf  # recall log(0) = -inf
```

Now we can sample using this conservative posterior and compare to the others.

If we wanted to get a general Student t distribution, we can do an integration of the variance over an inverse-gamma distribution or, equivalently, integrate over the inverse of the variance with a gamma distribtion. An animation of the latter is created in Student's t-distribution from Gaussians.

Approach no. 2: Good-and-bad data

In this approach, we are less pessimistic about the data. In particular, we allow for two possibilities:
a) the datum and its error are reliable;
b) the datum is bad and the error should be larger by a (large) factor γ.

We implement this with the pdf for σ:

$$p(\sigma|\sigma_0,\beta,\gamma,I) = \beta\,\delta(\sigma-\gamma\sigma_0) + (1-\beta)\,\delta(\sigma-\sigma_0)\ ,$$

where β and γ represent the frequency of quirky measurements and their severity. If we use Gaussian pdfs for the two cases, then the likelihood for a datum D_i after integrating over σ is

$$p(D_i|\theta,\sigma_0,\beta,\gamma,I) = \frac{1}{\sigma_0\sqrt{2\pi}}\left\{\frac{\beta}{\gamma}\,e^{-R_i^2/2\gamma^2} + (1-\beta)\,e^{-R_i^2/2}\right\}$$

We won't code this approach here but leave it to an exercise (see Sivia, Ch. 8.3.2 for more details). We will generalize this approach to the extreme in approach #4 below.

Approach no. 3: The Cauchy formulation

In this approach, we assume $\sigma \approx \sigma_0$ but allow it to be either narrower or wider:

$$p(\sigma|\sigma_0,I) = \frac{2\sigma_0}{\sqrt{\pi}\sigma^2}e^{-\sigma_0^2/\sigma^2}\ .$$

Marginalizing σ (using $\sigma = 1/t$) gives the Cauchy form likelihood (this is a special case of the Student t distribution):

$$p(D_i|\boldsymbol{\theta},\sigma_0,I) = \frac{1}{\sigma_0\pi\sqrt{2}[1+R_i^2(\boldsymbol{\theta})/2]}\ .$$

The log posterior is

$$L(\boldsymbol{\theta}) = \text{constant} \quad -\sum_{i=1}^{N}\log\left[1+\frac{R_i^2(\boldsymbol{\theta})}{2}\right]$$

```
# Cauchy likelihood
def log_posterior_cauchy(theta):
    """
    Log posterior with Cauchy likelihood and uniform prior for theta
    """
    if (all(theta > 0) and all(theta < 100)):
        R_sq = residuals(theta)**2
        return - np.sum( np.log(1 + R_sq/2) )
    else:
        return -np.inf  # recall log(0) = -inf
```

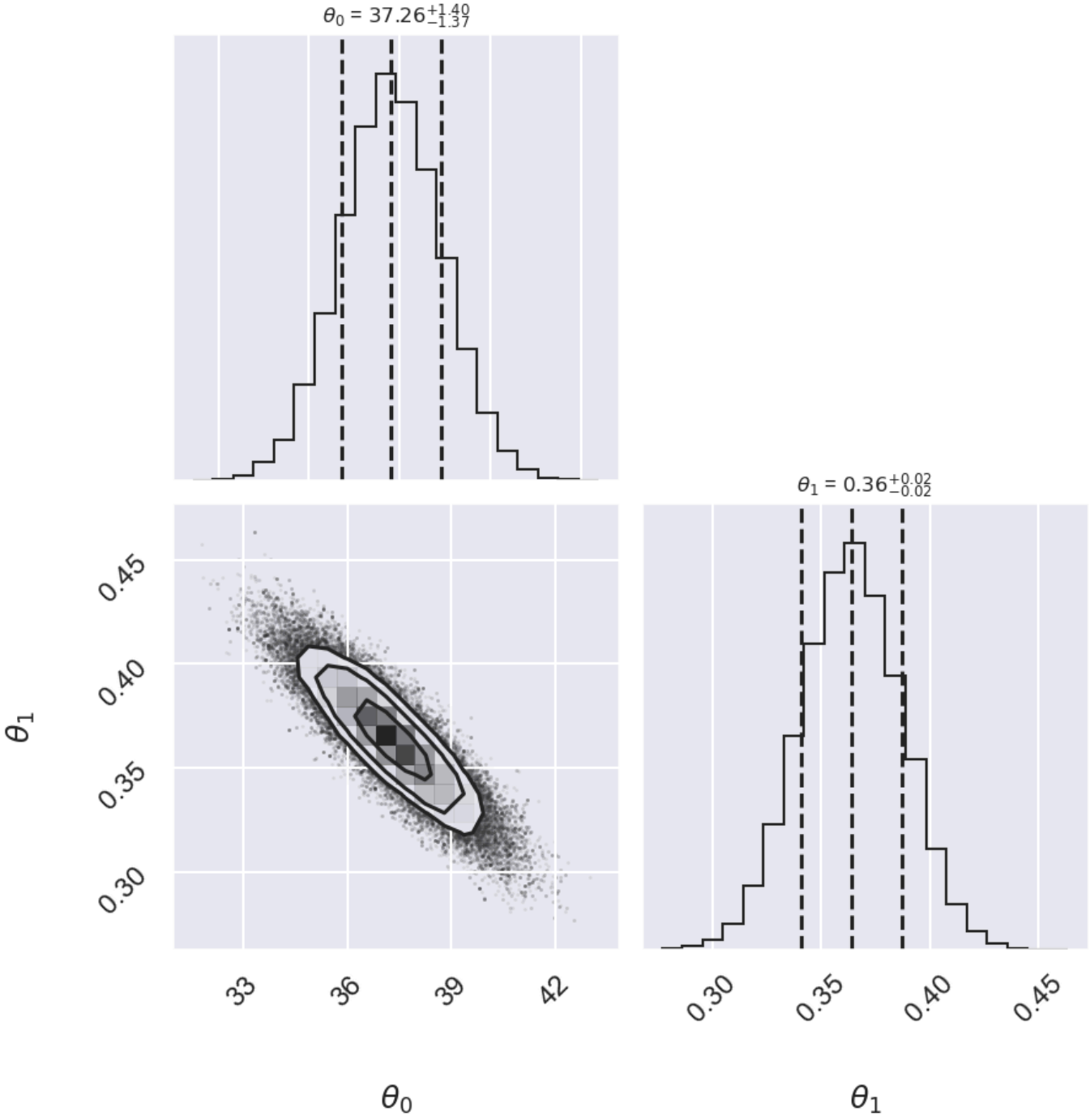
Sampling to compare methods

We'll use the `emcee` sampler to generate MCMC samples of our posteriors.

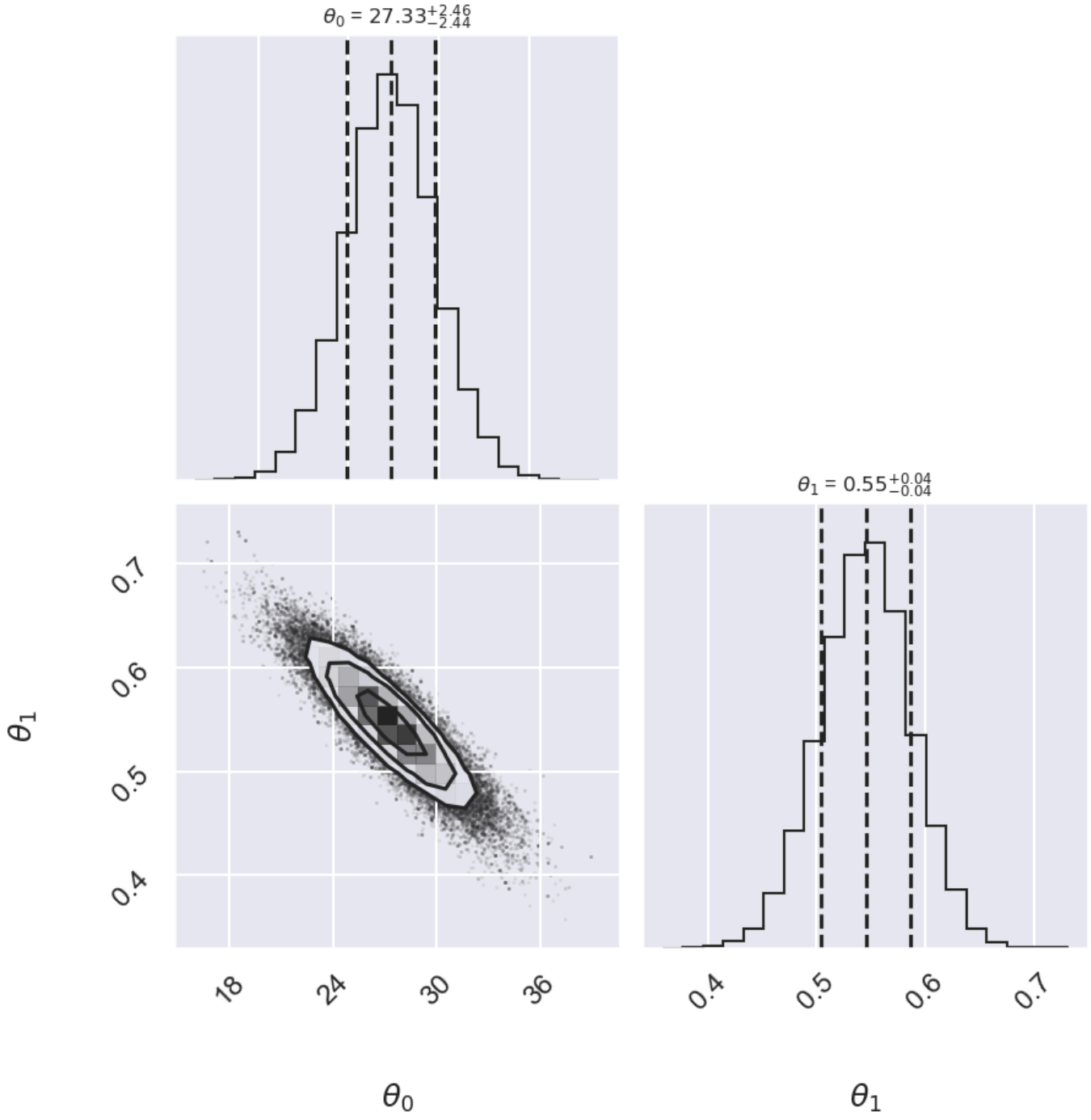
```
print('emcee sampling (version: )', emcee.__version__)

ndim = 2 # number of parameters in the model
nwalkers = 10  # number of MCMC walkers
nwarmup = 1000  # "burn-in" period to let chains stabilize
nsteps = 10000  # number of MCMC steps to take

print(f'{nwalkers} walkers:')

# Starting guesses close to the MLE
starting_guesses = np.abs(np.random.normal(1, 1, (nwalkers, 2)))
starting_guesses[:,0] += theta_MLE[0]
starting_guesses[:,1] /= 10
starting_guesses[:,1] += theta_MLE[1]

logps = [log_posterior_gaussian, log_posterior_conservative,
         log_posterior_cauchy]
approaches = ['Std Gaussian', 'Conservative','Cauchy']
mean_68CR = []

for ilogp,logp in enumerate(logps):
    print(f"Log posterior: {approaches[ilogp]}")
    # Sample the posterior distribution
    sampler = emcee.EnsembleSampler(nwalkers, ndim, logp)

    # Warm-up
    if nwarmup > 0:
        print(f'... EMCEE sampler performing {nwarmup} warnup iterations.')
        pos, prob, state = sampler.run_mcmc(starting_guesses, nwarmup)
        sampler.reset()
    else:
        pos = starting_guesses

    # Perform iterations, starting at the final position from the warmup.
    print(f'... EMCEE sampler performing {nsteps} samples.')
    %time sampler.run_mcmc(pos, nsteps)
    print("done")

    samples = sampler.get_chain(flat=True)
    lnposts = sampler.get_log_prob(flat=True)

    # Extract mean and 68% CR
    th0_mcmc, th1_mcmc = map(lambda v: (v[1], v[2]-v[1], v[1]-v[0]),
                             zip(*np.percentile(samples, [16, 50, 84],
                                                axis=0)))
    mean_68CR.append((th0_mcmc,th1_mcmc))

    # make a corner plot with the posterior distribution
    fig, ax = plt.subplots(2,2, figsize=(10,10))
    corner.corner(samples,labels=[r"$\theta_0$", r"$\theta_1$"],
                      quantiles=[0.16, 0.5, 0.84],fig=fig,
                      show_titles=True, title_kwargs={"fontsize": 12});
    plt.show()
```

```
emcee sampling (version: ) 3.1.6
10 walkers:
Log posterior: Std Gaussian
... EMCEE sampler performing 1000 warnup iterations.
... EMCEE sampler performing 10000 samples.
CPU times: user 3.62 s, sys: 18.3 ms, total: 3.64 s
```

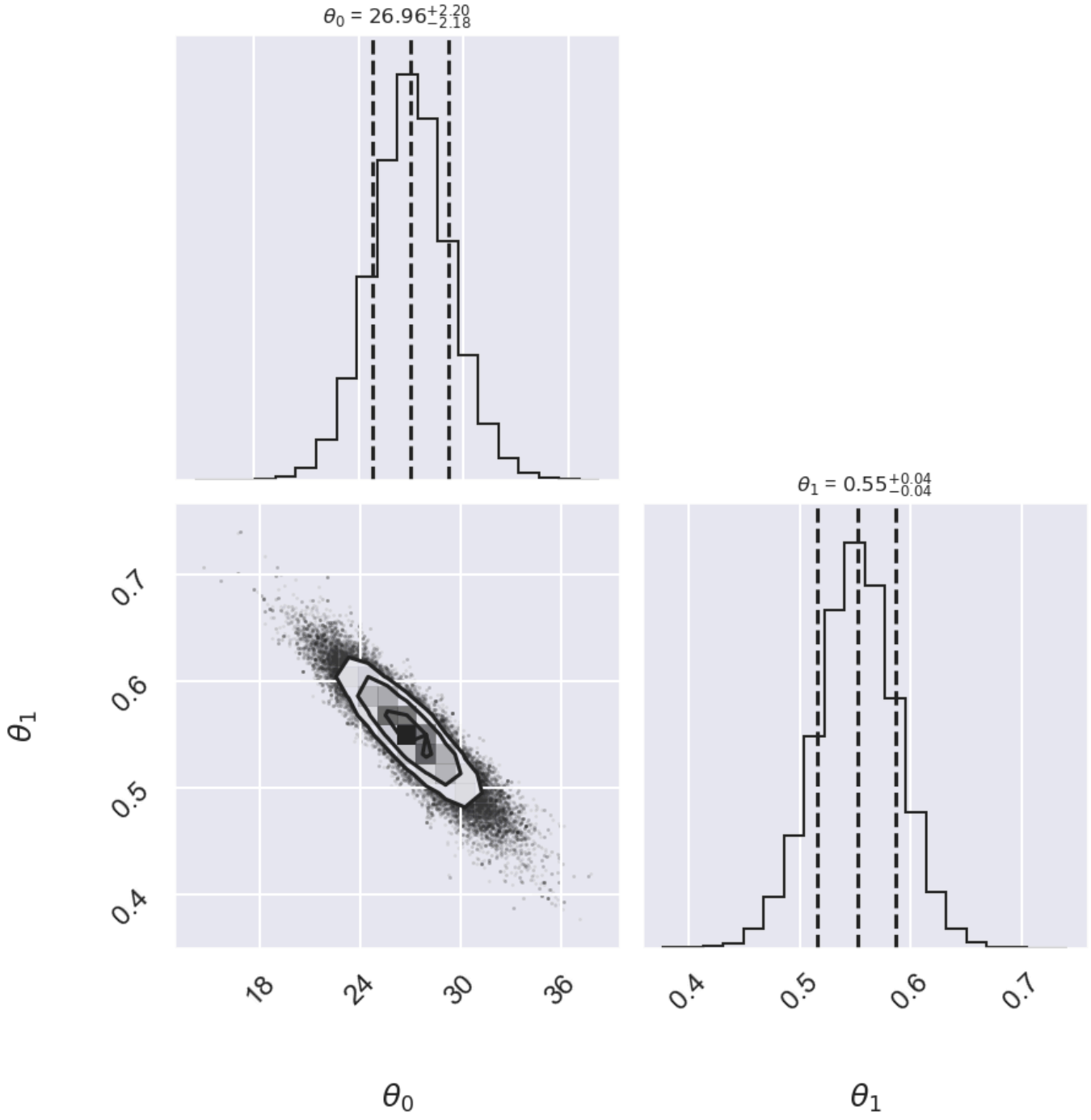
```
Wall time: 3.64 s
done
```




```
Log posterior: Conservative
... EMCEE sampler performing 1000 warnup iterations.
... EMCEE sampler performing 10000 samples.
CPU times: user 4.21 s, sys: 40.5 ms, total: 4.25 s
Wall time: 4.27 s
done
```

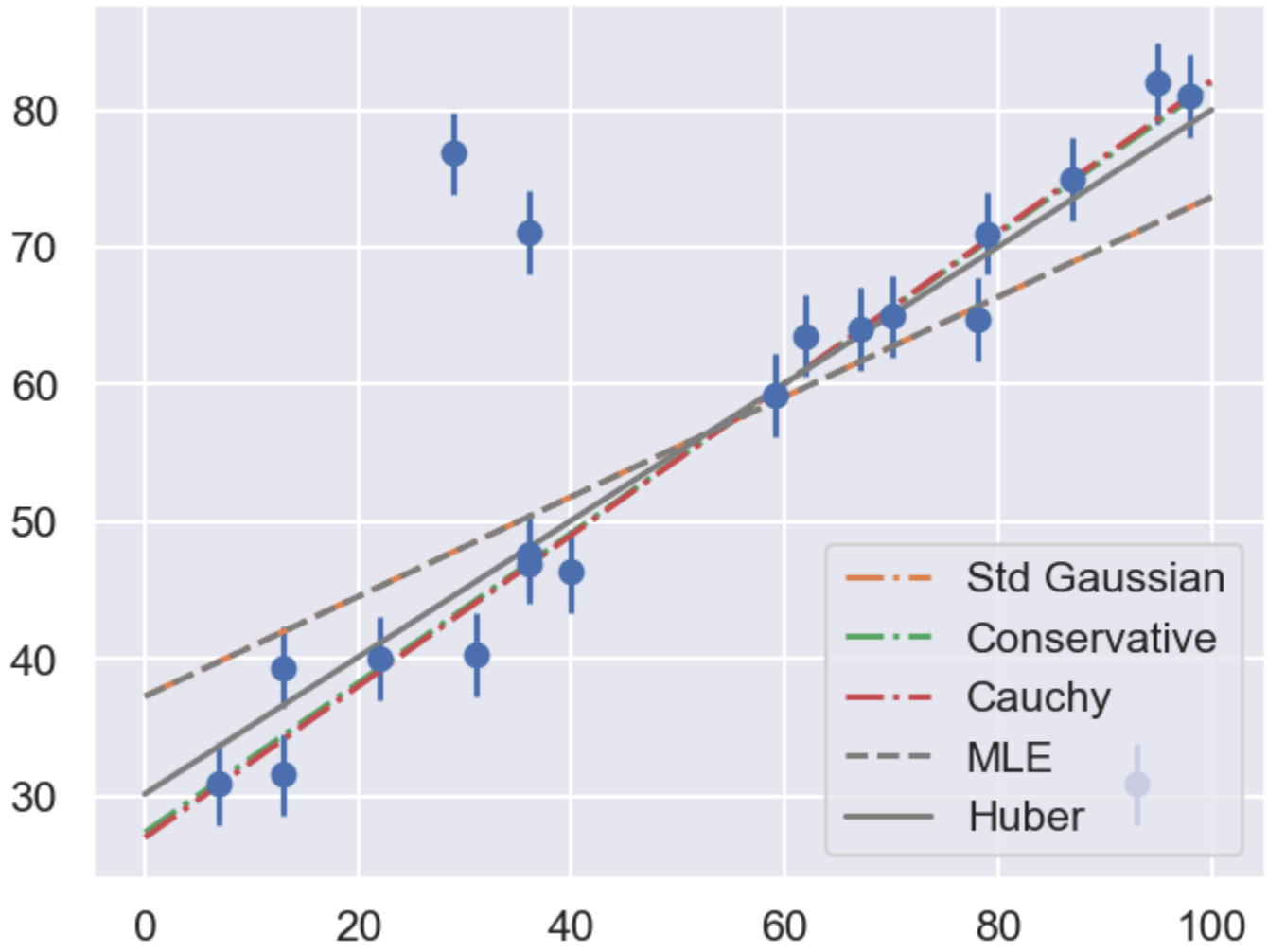



```
Log posterior: Cauchy
... EMCEE sampler performing 1000 warnup iterations.
... EMCEE sampler performing 10000 samples.
CPU times: user 4.04 s, sys: 35.2 ms, total: 4.08 s
Wall time: 4.08 s
done
```


$\theta_0 = 26.96^{+2.20}_{-2.18}$
$\theta_1 = 0.55^{+0.04}_{-0.04}$
θ_1
0.7
0.6
0.5
0.4
18
24
30
36
0.4
0.5
0.6
0.7
θ_0
θ_1

```
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(1,1,1)
ax.errorbar(x, y, sig0, fmt='o')

print("Summary:             Mean offset  68% CR     Mean slope     68% CR")
for i,approach in enumerate(approaches):
    ((th0,th0pos,th0neg),(th1,th1neg,th1pos)) = mean_68CR[i]
    print(f"{approach:>20s}    {th0:5.2f}   -{th0neg:4.2f},+{th0pos:4.2f}",\
          f"    {th1:5.3f}    -{th1neg:5.3f},+{th1pos:5.3f}")
    ax.plot(xfit, th0 + th1 * xfit, label=approach,  ls='-.')

ax.plot(xfit, theta_MLE[0] + theta_MLE[1] * xfit,
        color='gray',ls='--',label='MLE')
ax.plot(xfit, theta2[0] + theta2[1] * xfit,
        color='gray',ls='-',label='Huber')
ax.legend(loc='best');
```

```
Summary:             Mean offset  68% CR     Mean slope     68% CR
        Std Gaussian    37.26   -1.37,+1.40     0.364    -0.023,+0.023
        Conservative    27.33   -2.44,+2.46     0.546    -0.040,+0.042
              Cauchy    26.96   -2.18,+2.20     0.552    -0.035,+0.036
```



Comment on the comparison between the approaches. Repeat with different datasets. Do the results depend strongly on the dataset?

Approach no. 4: Many nuisance parameters

The Bayesian approach to accounting for outliers generally involves *modifying the model* so that the outliers are accounted for. For this data, it is abundantly clear that a simple straight line is not a good fit to our data. So let's propose a more complicated model that has the flexibility to account for outliers. One option is to choose a mixture between a signal and a background:

$$p(\{y_i\} \mid \{x_i\}, \{e_i\}, \boldsymbol{\theta}, \{g_i\}, \sigma_B) = \frac{g_i}{\sqrt{2\pi e_i^2}} \exp\left[\frac{-\left(\hat{y}(x_i \mid \boldsymbol{\theta}) - y_i\right)^2}{2e_i^2}\right] + \frac{1-g_i}{\sqrt{2\pi\sigma_B^2}} \exp\left[\frac{-\left(\hat{y}(x_i \mid \boldsymbol{\theta}) - y_i\right)^2}{2\sigma_B^2}\right]$$

What we've done is expanded our model with some nuisance parameters: $\{g_i\}$ is a series of weights which range from 0 to 1 and encode for each point i the degree to which it fits the model. $g_i = 0$ indicates an outlier, in which case a Gaussian of width σ_B is used in the computation of the likelihood. This σ_B can also be a nuisance parameter, or its value can be set at a sufficiently high number, say 50.

Our model is much more complicated now: it has 22 free parameters rather than 2, but the majority of these can be considered nuisance parameters, which can be marginalized-out in the end, just as we marginalized (integrated) over p in the Billiard example. Let's construct a function which implements this likelihood. We'll use the emcee package to explore the parameter space.

To actually compute this, we'll start by defining functions describing our prior, our likelihood function, and our posterior:

```
# theta will be an array of length 2 + N, where N is the number of points
# theta[0] is the intercept, theta[1] is the slope,
# and theta[2 + i] is the weight g_i

def log_prior(theta):
    #g_i needs to be between 0 and 1
    if (all(theta[2:] > 0) and all(theta[2:] < 1)):
        return 0
    else:
        return -np.inf  # recall log(0) = -inf

def log_likelihood(theta, x, y, e, sigma_B):
    dy = y - theta[0] - theta[1] * x
    g = np.clip(theta[2:], 0.001, 0.999)  # g<0 or g>1 leads to NaNs in logarithm
    logL1 = np.log(g) - 0.5 * np.log(2 * np.pi * e**2) - 0.5 * (dy / e)**2
    logL2 = np.log(1 - g) - 0.5 * np.log(2 * np.pi * sigma_B**2) \
            - 0.5 * (dy / sigma_B)**2
    return np.sum(np.logaddexp(logL1, logL2))

def log_posterior(theta, x, y, e, sigma_B):
    return log_prior(theta) + log_likelihood(theta, x, y, e, sigma_B)
```

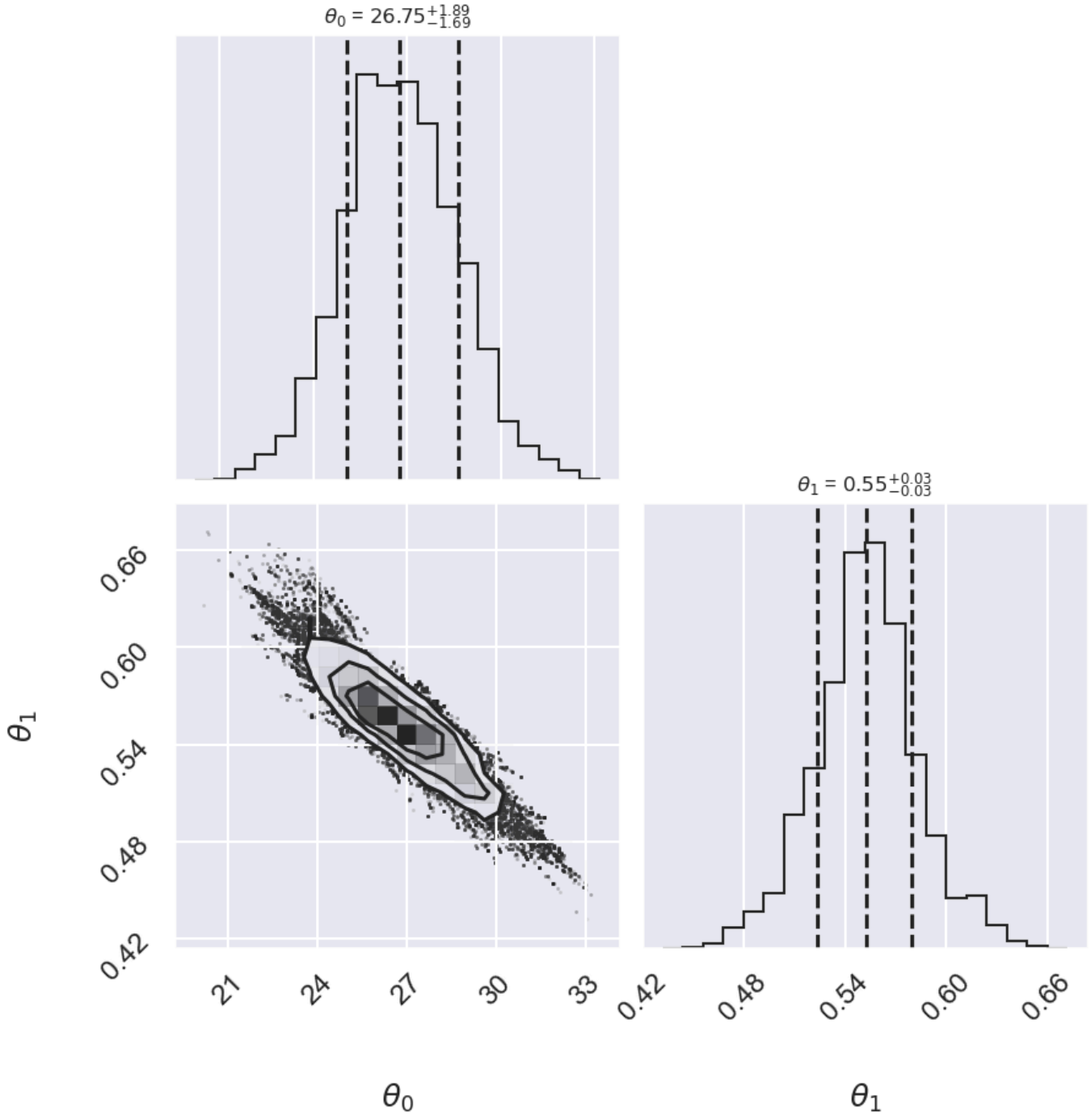
Now we'll run the MCMC sampler to explore the parameter space:

```
# Note that this step will take a few minutes to run!

ndim = 2 + len(x)  # number of parameters in the model
nwalkers = 50  # number of MCMC walkers
nburn = 10000  # "burn-in" period to let chains stabilize
nsteps = 15000  # number of MCMC steps to take

# set theta near the maximum likelihood, with
np.random.seed(0)
starting_guesses = np.zeros((nwalkers, ndim))
starting_guesses[:, :2] = np.random.normal(theta_MLE, 1, (nwalkers, 2))
starting_guesses[:, 2:] = np.random.normal(0.5, 0.1, (nwalkers, ndim - 2))

sampler = emcee.EnsembleSampler(nwalkers, ndim, log_posterior,
                                args=[x, y, sig0, 50])
sampler.run_mcmc(starting_guesses, nsteps)

samples = sampler.get_chain(discard=nburn, flat=True)
```

Once we have these samples, we can exploit a very nice property of the Markov chains. Because their distribution models the posterior, we can integrate out (i.e. marginalize) over nuisance parameters simply by ignoring them!

We can look at the (marginalized) distribution of slopes and intercepts by examining the first two columns of the sample:

```
fig, ax = plt.subplots(2,2, figsize=(10,10))
# plot a corner plot with the posterior distribution
# Note that the intercept and the slope correspond to
#  the first two entries in the parameter array.
fig = corner.corner(samples[:,:2], labels=[r"$\theta_0$", r"$\theta_1$"],
                    quantiles=[0.16, 0.5, 0.84],fig=fig,
                    show_titles=True, title_kwargs={"fontsize": 12})
```



We see a distribution of points for the slope and the intercept. We'll plot this model over the data below, but first let's see what other information we can extract from this trace.

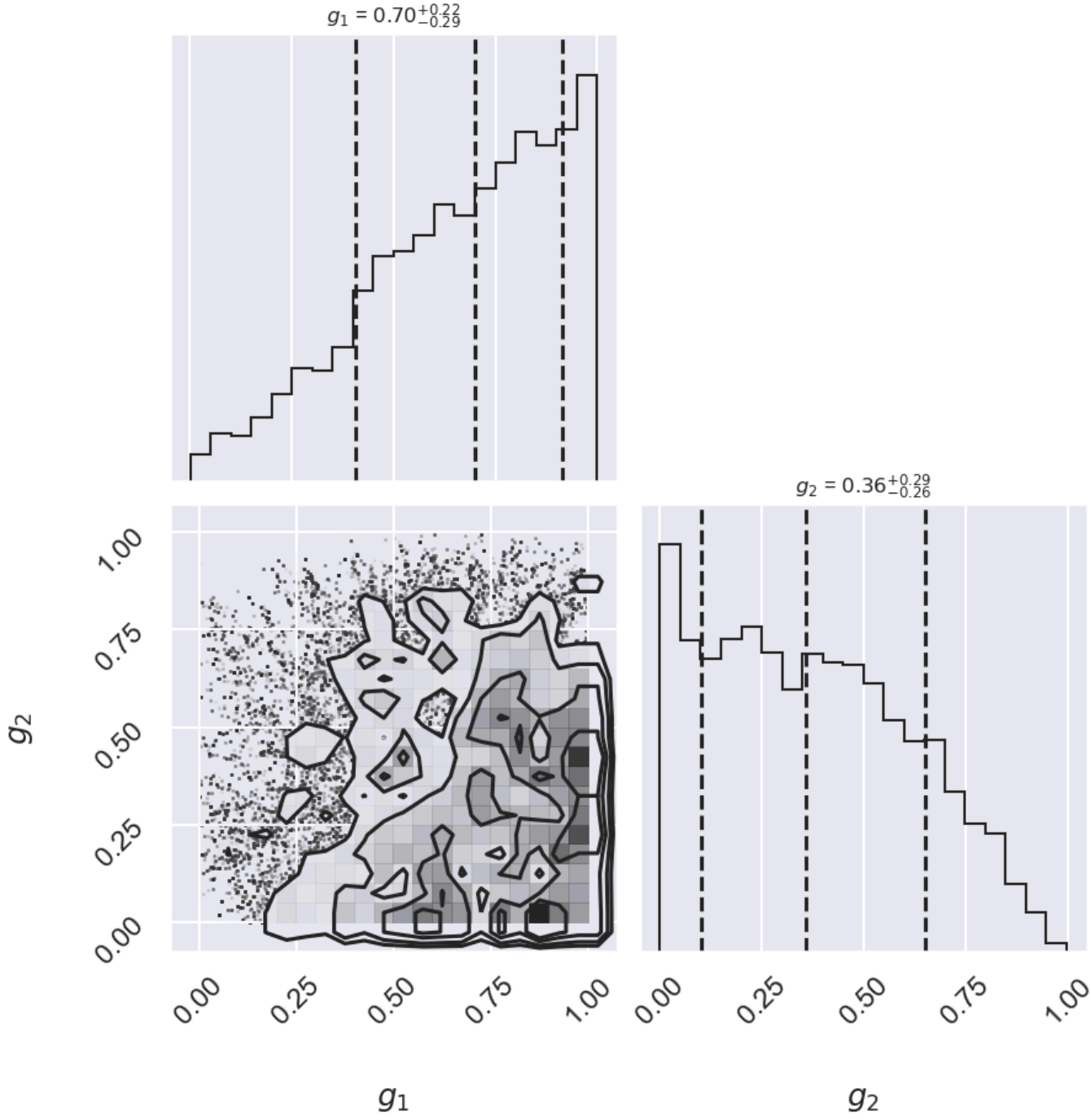
One nice feature of analyzing MCMC samples is that the choice of nuisance parameters is completely symmetric: just as we can treat the $\{g_i\}$ as nuisance parameters, we can also treat the slope and intercept as nuisance parameters! Let's do this, and check the $\{g_i\}$ posterior for two points that you choose. Pick one to be a good point and one that looks like an outlier.

```
# Print out the mean value of g_i with its index (starts from 2)
print(r' index  mean(g_i)')
for i in range(2, len(x)+2):
   print(f" {i:>3}      {samples[:,i].mean():0.3f}")
```

```
 index  mean(g_i)
   2      0.640
   3      0.524
   4      0.610
   5      0.662
   6      0.375
   7      0.628
   8      0.604
   9      0.376
  10      0.626
  11      0.646
  12      0.641
  13      0.668
  14      0.661
  15      0.655
  16      0.577
  17      0.693
  18      0.574
  19      0.346
  20      0.640
  21      0.671
```

```
# Choose i1 and i2 from the list above: one good point,
#  one potential outlier
i1 = 5
i2 = 6

fig, ax = plt.subplots(2,2, figsize=(10,10))
# plot a corner plot with the posterior distribution
# for two entries in the samples.
subset_samples = samples[:, [i1, i2]]
fig = corner.corner(subset_samples, labels=[r"$g_1$", r"$g_2$"],
                    quantiles=[0.16, 0.5, 0.84],fig=fig,
                    show_titles=True, title_kwargs={"fontsize": 12})

print(f"g1 mean: {samples[:, i1].mean():.2f}")
print(f"g2 mean: {samples[:, i2].mean():.2f}")
```

```
g1 mean: 0.66
g2 mean: 0.37
```

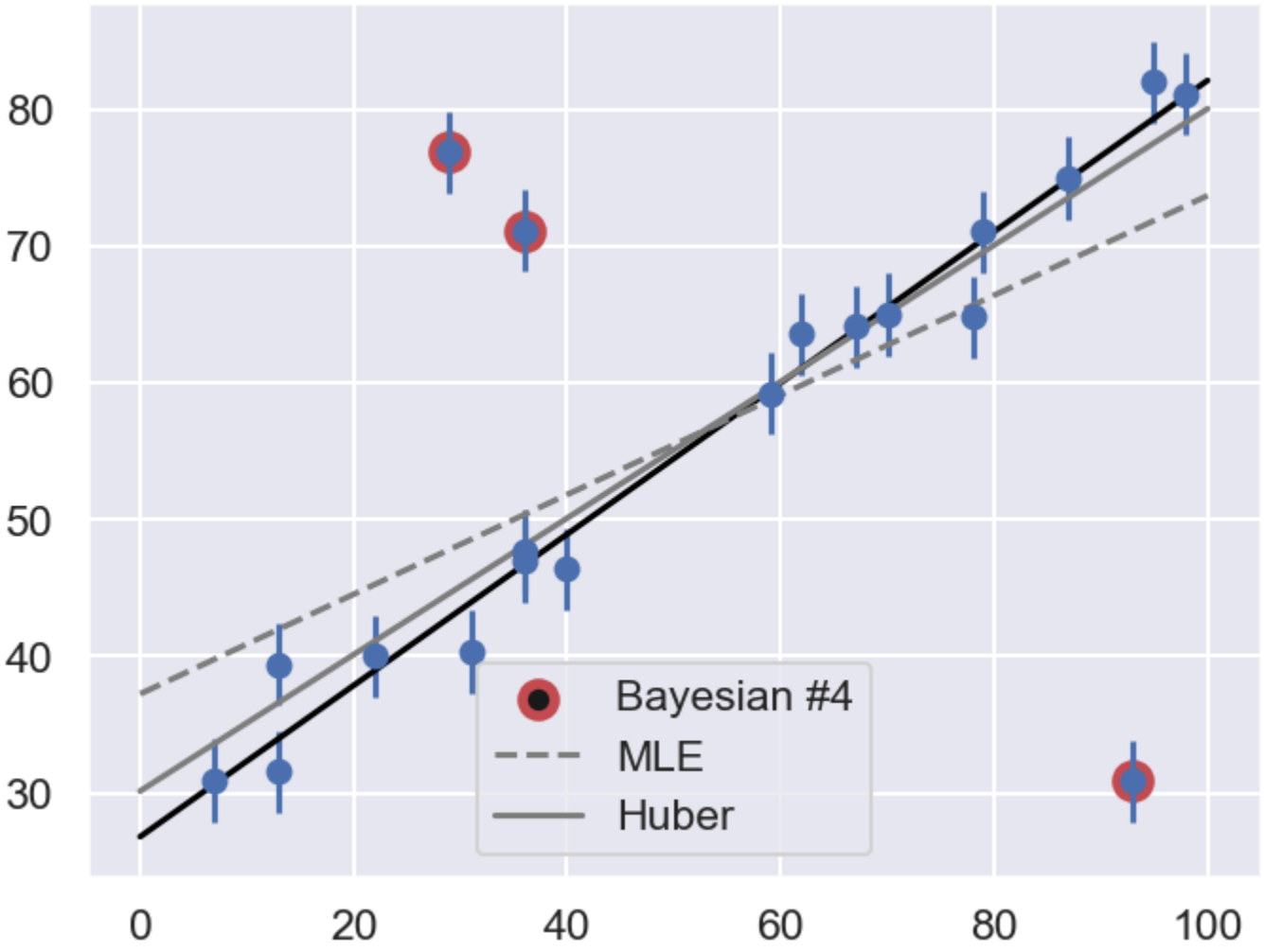


There may not be extremely strong constraint on either of the points, but generally good points have means greater than 0.5 and outlier points have means less than 0.5. If we choose a cutoff at $g = 0.5$, our algorithm will identify the outliers.

Let's make use of all this information, and plot the marginalized best model over the original data. As a bonus, we'll draw red circles to indicate which points the model detects as outliers.

```
theta3 = np.mean(samples[:, :2], 0)
g = np.mean(samples[:, 2:], 0)
outliers = (g < 0.5)
```

```
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(1,1,1)
ax.errorbar(x, y, sig0, fmt='o')

ax.plot(xfit, theta3[0] + theta3[1] * xfit, color='black')
ax.scatter(x[outliers], y[outliers],
           marker='o', s=150, edgecolors='r', linewidths=4, c='k',
           label='Bayesian #4')

ax.plot(xfit, theta_MLE[0] + theta_MLE[1] * xfit,
        color='gray',ls='--',label='MLE')
ax.plot(xfit, theta2[0] + theta2[1] * xfit,
        color='gray',ls='-',label='Huber')
ax.legend(loc='best');
```



- Does the result, shown by the dark line, match your intuition?
- Are the points automatically identified as outliers the ones you would identify by hand?
- Make a comparison with the gray lines, which show the two previous approaches: the simple maximum likelihood and the frequentist approach based on Huber loss.

12.5. Demo: Student's t-distribution from Gaussians

Overview: not a Gaussian distribution

Physicists often assume that most distributions are Gaussian. But sometimes they are Student's t-distributions.

David C. Bailey, in the paper *Not Normal: the uncertainties of scientific measurements*, analyses 41000 measurements of 3200 quantities from medicine, nuclear and particle physics, and many interlaboratory comparisons. He finds "Uncertainty-normalized differences between multiple measurements of the same quantity are consistent with heavy-tailed Student's t-distributions that are often almost Cauchy, far from a Gaussian Normal bell curve".

Appendix A of *Rigorous constraints on three-nucleon forces in chiral effective field theory from fast and accurate calculations of few-body observables* by Wesolowski et al. (arXiv:2104.0441) explains why, in linear parameter estimation problems with variance estimation, one finds that the parameters are typically t distributed after one marginalizes over the variance.

In short, if you are uncertain of your uncertainty (variance), and being a good Bayesian you integrate over the distribution of variances, you will get a t distribution. So imagine a mixture of normal distributions with a common mean but with the precision (the inverse of the variance) distributed according to a gamma distribution (or the variance is distributed with an inverse gamma distribution).

Emergence of a Student's t-distribution

As a visualization, Rasmus Bååth in this blog entry created an animation by making draws from a normal distribution whose width (standard deviation) "jumps around". Here we have created a version of this animation that also includes the sampling of the variance. (Note: instead of variance sampling we could have alternatively used conjugate priors.)

A t distribution with mean μ, scale s, and degrees of freedom ν

$$y \sim \mathrm{t}(\nu, \mathrm{loc} = \mu, \mathrm{scale} = s)$$

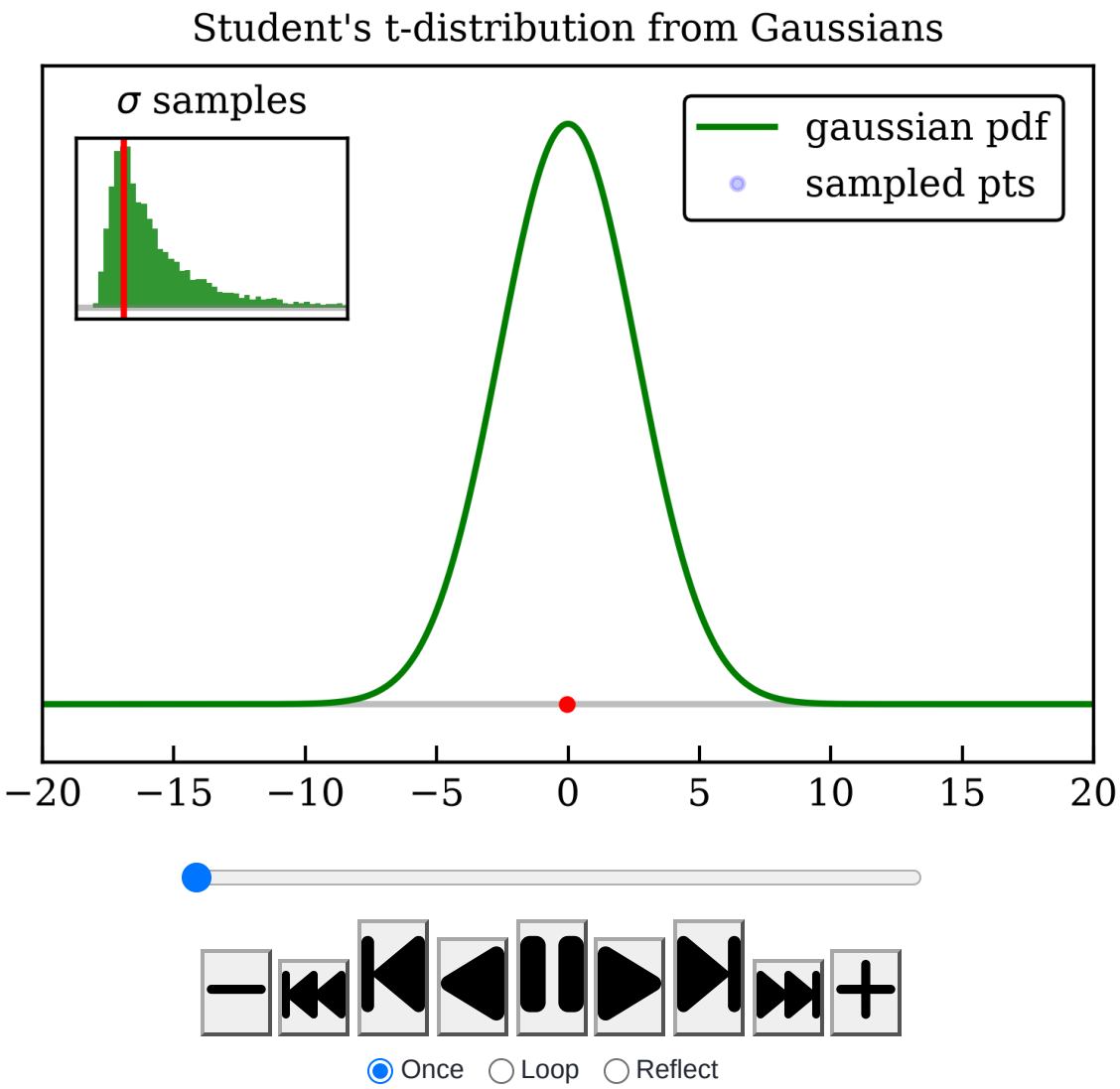
can be created from

$$y \sim \mathrm{Normal}(\mu, \sigma)$$

where σ comes from

$$\frac{1}{\sigma^2} \sim \mathrm{Gamma}(a = \nu/2, \mathrm{scale} = 1/(s^2\nu/2))$$

▼ **Hide code cell source**



13. Bayes goes linear: History matching

> “Those who cannot remember the past are condemned to repeat it.”
>
> —George Santayana

In this chapter we will introduce Bayes linear methods and history matching in particular.

13.1. Bayes linear methods

Bayes linear statistics

> Bayes linear methods are subjective statistical analyses based on expectation and covariance structures, rather than on distributional assumptions.
>
> The use of the word linear refers to Bruno de Finetti's arguments that probability theory is a linear theory (he argued against the more common measure theory approach).

The full Bayesian approach requires a complete, probabilistic enumeration of all possible outcomes. In practice. for scientific applications, this usually translates to the use of probability distributions to capture all prior knowledge as well as the statistical model that goes into the data likelihood. Such a specification can often be largely arbitrary.

Bayes linear methodology is similar in spirit to the standard Bayes analysis, but is constructed so as to avoid much of the burden of specification and computation of a fully Bayesian analysis. The aim is to develop a methodology which allows to state and analyse relatively small, carefully chosen collections of quantitative judgements that are within our ability to specify in a meaningful way.

We will in particular restrict ourselves to specifying mean values and covariances for a set $\mathcal{Z} = \{Z_1, Z_2, \ldots, Z_M\}$ of (random) quantities, for which we shall make statements of uncertainty. For each $Z_i, Z_j \in \mathcal{Z}$ we specify

1. The expectation $\mathbb{E}[Z_i]$, giving a simple quantification of our belief as to the magnitude of Z_i;
2. The variance Var (Z_i), quantifying our uncertainty or degree of confidence in our judgements of the magnitude of Z_i;
3. The covariance Cov (Z_i, Z_j), expressing a judgement on the relationship between the quantities, quantifying the extent to which observation on Z_j may (linearly) influence our belief as to the size of Z_i.

This restricted collection of belief statements then replaces the full specification of $p(\mathcal{Z})$.

For a thorough reference on Bayes linear methods, see the textbook *"Bayes Linear Statistics: Theory and Methods"** by Michael Goldstein and David Wooff [GW07].

Pukelsheim's three-sigma rule

While avoiding the full probabilistic treatment, it is still possible to make some quantitative statements using expectation values and (co)variances as ingredients in general probability inequalities. The most famous one is Pukelsheim's three-sigma rule which we will now discuss. For a full derivation, the reader is encouraged to study Pukelheim's paper from 1994 [Puk94] that also contains the original references. The original bound is due to Vysochanskii and Petunin (1980, 1983).

Consider a random varable X with mean value $\mu = \mathbb{E}[X]$ and variance $\sigma^2 = \text{Var}\ (X)$. A very general inequality was provided by Bienaymé (1853) and Chebyshev (1867) about the probability that X falls outside of a radius $r > 0$,

$$\mathbb{P}(|X - \mu| \geq r) \leq \frac{\sigma^2}{r^2}. \tag{13.1}$$

This is a rough bound. For example, we find that the probability of finding X outside of $r = 3\sigma$ is smaller than or equal to $1/9 \approx 11\%$.

It is possible to find a tighter constraint with an additional assumption. For a unimodal distribution, the probability density is non-decreasing up to the mode, and non-increasing after. For this class, the bound becomes more than halved

$$\mathbb{P}(|X-\mu| \geq r) \leq \frac{4}{9}\frac{\sigma^2}{r^2} \quad \text{for } r > \sqrt{\frac{8}{3}}\sigma \approx 1.63\sigma. \tag{13.2}$$

In particular, for $r = 3\sigma$ we get the celebrated Pukelsheim's three-sigma rule.

Pukelsheim's three-sigma rule

The probability for a random quantity X to fall away from its mean μ by more than three standard deviations 3σ is at most 5%,

$$\mathbb{P}(|X-\mu| \geq 3\sigma) \leq \frac{4}{81} < 0.05,$$

given that X is described by a unimodal probability density function.

13.2. Iterative history matching

In these lectures we will introduce a very powerful, iterative approach known as history matching [H+22, VGB14, VGB10, VLG+18]. The power of this method lies in the iterative confrontation of a computational model with a set of (historical) data within the Bayes linear framework. Its applicability rests on the ability to solve the model with small computational cost at low fidelities, e.g. using emulators (see Bayes goes fast: Emulator overview).

Let us start with the statistical model (4.11), which we repeat below, that relates data with model predictions via random quantities that describe uncertainties in data and model, respectively

$$\mathcal{D} = M(\boldsymbol{\theta}) + \delta\mathcal{D} + \delta M.$$

This relation, and our specification of probability distributions for the random quantities, usually leads to the data likelihood $p\left(\mathcal{D} \mid \boldsymbol{\theta}, I\right)$ that enters our Bayesian analysis. Here, however, we recognize the risk of arbitrariness when making such assignments. Instead, we rely on Bayes linear statistics and just specify the expectation values and covariances.

To be specific, let us consider a single observable Z_i with the corresponding model prediction $M_i(\boldsymbol{\theta})$. With $\mathbb{E}\left[\delta Z_i\right] = \mathbb{E}\left[\delta M_i\right] = 0$ and independent data and model uncertainties—Var $\left(\delta Z_i\right) = \sigma^2_{Z,i}$ and Var $\left(\delta M_i\right) = \sigma^2_{M,i}$, respectively—we would have

$$\begin{aligned} \mathbb{E}\left[Z_i\right] &= M_i(\boldsymbol{\theta}), \\ \text{Var}\ \left(Z_i\right) &= \sigma^2_{Z,i} + \sigma^2_{M,i}. \end{aligned} \tag{13.3}$$

The aim of history matching is to identify the set $\mathcal{Q}(z)$ of parameterizations $\boldsymbol{\theta}$, for which the evaluation of a model $M(\boldsymbol{\theta})$ yields an acceptable—or at least not implausible—match to a set of observations z. The set Q will then

translate into a non-implausible volume in the multi-dimensional parameter space. Let us stress that $\mathcal{Z}$ is the set of observables while $z = \{z_1, z_2, \ldots z_M\}$ is the actual set of (numerical) observations.

History matching has been employed in various studies involving complex computer models ranging from effects of climate modeling to systems biology, epidemiology, and nuclear physics.

We introduce the individual implausibility measure

$$I_i^2(\boldsymbol{\theta}) = \frac{|M_i(\boldsymbol{\theta}) - z_i|^2}{\text{Var}\ (\delta Z_i + \delta M_i)}, \tag{13.4}$$

which is a function over the input parameter space and quantifies the (mis-)match between our model output $M_i(\boldsymbol{\theta})$ and the observation z_i for an observable in the target set $\mathcal{Z}$. Note that we can employ emulation of our model as long as we quantify the expected precision of our emulator (in terms of a variance) and include that in the total variance of the denominator.

A common choice is to employ a maximum implausibility measure to make the distinction between implausible and non-implausible parameter samples. Specifically, we consider a particular value for $\boldsymbol{\theta}$ as implausible if

$$I_M(\boldsymbol{\theta}) \equiv \max_{z_i \in z} I_i(\boldsymbol{\theta}) > c_I. \tag{13.5}$$

We can choose $c_I \equiv 3.0$, appealing to Pukelheim's three-sigma rule [Puk94], or we can use other cutoff values depending on scientific need. An alternative approach is to consider the second or third highest implausibility values, denoted I_{2M} and I_{3M} respectively, ensuring that errors in uncertainty variance estimates do not lead to unfairly labeling parameter samples as implausible.

In accordance with the assumptions leading to (4.11), the variance in the denominator of (13.4) is a sum of independent squared errors. Generalizations of these assumptions are straightforward if additional information on error covariances or possible inaccuracies in our error model would become available.

An important strength of the history matching is that we can proceed iteratively, excluding regions of input space by imposing cutoffs on implausibility measures that can include *additional* observables Z_i and corresponding model outputs M_i with possibly refined emulators as the parameter volume is reduced. The history matching process is designed to be independent of the order in which observables are included. This is an important feature as it allows for efficient choices regarding such orderings.

The iterative history matching proceeds in waves according to a straightforward strategy that can be summarized as follows:

Algorithm 13.1 (The history-matching algorithm)

1. At wave j: Evaluate a set of model runs over the current NI volume $\mathcal{Q}_j$ using a space-filling design of sample values for the parameter inputs $\boldsymbol{\theta}$. Choose a rejection strategy based on implausibility measures for a set $\mathcal{Z}_j$ of informative observables.
2. Construct or refine emulators for the model predictions across $\mathcal{Q}_j$.
3. The implausibility measures are then calculated over $\mathcal{Q}_j$ using the emulators, and implausibility cutoffs are imposed. This defines a new, smaller non-implausible volume $\mathcal{Q}_{j+1}$ which should satisfy $\mathcal{Q}_{j+1} \subset \mathcal{Q}_j$.
4. Unless (a) computational resources are exhausted, or (b) all considered points in the parameter space are deemed implausible, we may include additional informative observables in the considered set $\mathcal{Z}_{j+1}$, and return to step 1.

5. If 4(a) is true we generate a number of acceptable runs from the final non-implausible volume $\mathcal{Q}_{\text{final}}$, sampled according to scientific need.

The strength of the history-matching scheme lies within its ability to achieve an iterative volume reduction. It is the removal of implausible regions that is the main goal. There are several ways in which that non-implausible volume may continue to shrink in subsequent waves:

- The emulator is often refined as we reduce the parameter volume. Thereby, the variance in the denominator of (13.4) will decrease and the resolving power of the implausibility metric will increase.
- The density of samples will increase as we reduce the parameter volume which allows to better identify the non-implausible volume. Furthermore, a larger density of samples allows to employ additional confirmation of non-implausible volumes from optical depths (which indicate the density of non-implausible samples; see Eqs. (25) and (26) in [VLG+18]).
- Additional observables may be introduced in later waves. The parameter space is then further constrained by model predictions for these new observables.
- One or more parameters can be set as *inactive* during early waves and then become activated as new observables are being introduced. There might be situations where the modeling output is independent of certain parameters, or it could be that the output is just not very sensitive to certain parameter variations. The modeling or emulation can then be performed without these parameters with an additional uncertainty that reflects the missing sensitivity. This is a well known emulation procedure called inactive parameter identification [VGB10].
- The mapping of non-implausible samples to a non-implausible volume might be more sophisticated in later waves. The easiest mapping is from the non-implausible range found for each individual parameter to a hyperrectangle. This mapping allows a very strainghtforward space-filling design in the next wave. However, we would completely ignore possible correlation structures. A better approach is then to identify rotated hyperrectangles, or ellipsoids, that encompass the non-implausible samples. See for example Ref. [May22].

What next?

The non-implausible samples do not offer a straightforward probabilistic interpretation. The author would just be comfortable with the following weak statement

$$\mathbb{P}\left(\boldsymbol{\theta} \notin Q(z) \mid \mathcal{Z}, M, I\right) \leq \text{implausible} \quad ,$$

where I contains all definitions of implausibility that have been employed through the analysis. What then are they good for?

The following is a (possible incomplete) list of possible uses of a history-matching analysis

1. The non-implausible samples summarise the parameter region of interest, and can directly aid insight regarding interdependencies between parameters induced by the match to observed data. This region is also where we would expect the bulk of the posterior distribution to reside.
2. Consequently, the non-implausible samples offer good starting points for a subsequent Bayesian analysis (assuming that we dare to make fully probabilistic statements concerning unknown quantities). The convergence of MCMC sampling will be much faster if it is initialized in the region of highest probability mass.
3. Furthermore, the efficient search of large, high-dimensional volumes can aid in identifying multiple regions of interest. The non-implausible samples do not necessarily reside in a single, connected domain but might represent multiple modes of a probability distribution.

4. Implausible regions will correspond to small likelihood densities. The history-matching procedure is therefore well suited to be combined with Sampling/Importance Resampling, see also Ref. [JForssen22].

14. Model Selection

So far, we have been concerned with the problem of parameter estimation. In studying the linear relationship between two quantities, for example, we discussed how to infer the slope and the offset of the associated straight-line model. Often, however, there is a question as to whether another functional form (such as quadratic or cubic) might be a more appropriate model. In this lecture, we will consider the broad class of scientific problems when there is uncertainty as to which one of a set of alternative models is most suitable. In the Bayesian terminology these can be labeled as **Model Selection** problems and we will discuss them in some depth.

> **Note**
>
> Good references for model selection are Sivia, chapter 4 [SS06], *Bayesian Model Selection and Model Averaging* by Wasserman, and chapter 7 of BDA3 [GCS+13].

One of the main objectives in science is that of inferring the truth of one or more hypotheses about how some aspect of nature works. Because we are always in a state of incomplete information, we can never prove any hypothesis (theory) is true.

We will start, however, in Section 14.1 with a brief discussion on sampling theory and the frequentist approach to **hypothesis testing**. This will involve the introduction of the P-value or significance measure—quantities that are often misinterpreted even by scientists themselves. See, for example, the following comment published in Nature (March 20, 2019): Scientists rise up against statistical significance.

In Bayesian model selection we provide a gentle introduction through the story of Dr. A and Prof. B and then in Evidence ratios for an expansion we step through the prototypical problem of different orders of an expansion, for which model selection can be analyzed analytically. Methods for Evidence calculations are next and we finish by surveying Information criteria, which are computationally light approximations to the evidence.

There are also several problems in Problems for Part II that apply and supplement the presentation in this chapter. See Problem: Model selection basics, Problem: Evidence calculation for EFT expansions, and Problem: How many lines are there?.

14.1. Frequentist hypothesis testing

Recall that in frequentist statistics, probability statements are restricted to random variables, see *Aside: Bayesian epistemology. A hypothesis can not be considered a random variable, and therefore we are restricted to a much more indirect approach when trying to infer its truth, or rather when attempting to falsify it.

Basic idea

The standard sampling theory approach to hypothesis testing is to construct a statistical test. The basic idea is described in the following.

Frequentist hypothesis testing. The sampling theory hypothesis test is designed to compare a selected statistic from the measured data with expected results from a very large number of hypothetical repeated measurements

under the assumption that a chosen null hypothesis ($\mathcal{H}_0$) is true.

P-values: when all you can do is falsify

The null hypothesis is rejected if the data are very unexpected under $\mathcal{H}_0$. Note that the null hypothesis is never accepted, it can only be rejected. Somewhat perversely then, you should really choose the null hypothesis to be the boring thing you hope is not true, e.g., if you are looking for the Higgs boson then the null hypothesis is that there is no Higgs boson, and you are checking how unlikely it is that the data you saw would've been generated under that hypothesis. Note also that you are *not* supposed to compare the improbability of the data under the null hypothesis with how probable they would be under an alternative hypothesis ($\mathcal{H}_A$).

To perform frequentist hypothesis testing you should pick a level of proof you are comfortable with. For the Higgs boson (and for many other particle physics experiments) it is "5 sigma''. *How do you think we convert this statement to a probability?* [Hint: refer to a Gaussian distribution.] One minus the resulting probability is called the P-value. We will denote it $p_{\rm crit}$. There is nothing God-given about it. It is a standard (like "beyond a reasonable doubt") that has been agreed upon in a research community for determining that something is (likely) going on.

You then take data and compute $p(D|\text{null hypothesis})$. This probability is often summarized via a statistics, such as the χ^2 statistic, which is chosen so that its distribution can be easily computed given the truth of the null hypothesis. In the second half of this section we will discuss the use of the χ^2 statistic to compute this probability, but for now we keep the discussion general.

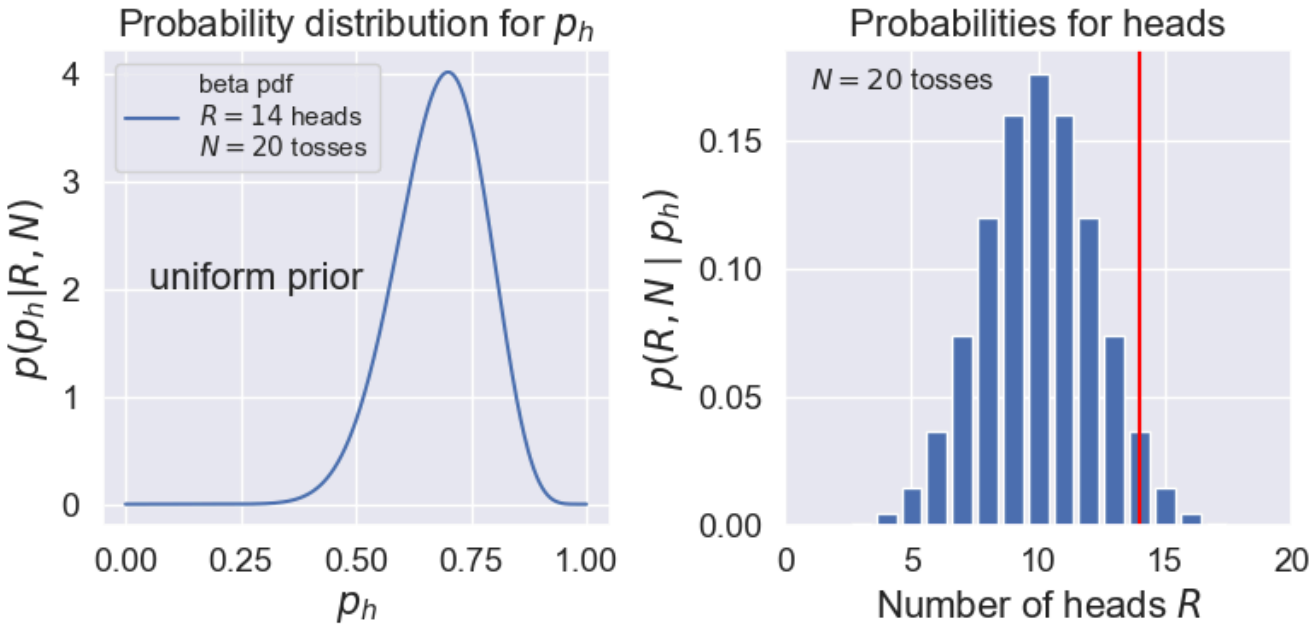
If $p(D|\text{null hypothesis}) < p_{\rm crit}$ then you conclude that the "the null hypothesis is rejected at the $p_{\rm crit}$ significance level''. This would mean that the chosen statistic, when computed for the observed data, falls at a spot on the distribution that is more improbable than the pre-defined threshold. We reiterate that, if the opposite happens, and $p(D|\text{null hypothesis}) > p_{\rm crit}$, you cannot conclude that the null hypothesis is true. It just means "no effect was observed".

Contrast Bayesian and significance analyses for coin flipping

Suppose we do a coin flipping experiment where we toss a coin 20 times and it comes up heads 14 times. Let's compare how we might analyze this with Bayesian methods to how we might do a significance analysis with P-values.

Bayesian analysis. Following our study in Demo: Widgetized coin tossing, let's calculate the probability of heads, which we call p_h, given data $D = \{R \text{ heads }, N \text{ tosses }\}$.

- Let's assume a uniform prior (encoded as a beta function with $\alpha = 1$, $\beta = 1$).
- We found that this can be expressed as a beta function, which we can calculate in Python using $p(p_h|D) = p(D|p_h)p(p_h) \longrightarrow$ `scipy.stats.beta.pdf(p_h,1+R,1+N-R)`. For $N = 20$, $R = 14$, this is shown on the left below.
- Now we can answer many questions, such as: what is the probability that p_h is between 0.49 and 0.51? The answer comes from integrating our PDF over this interval (i.e., the area under the curve).
- Note that in the Bayesian analysis, the data is given while the probability of heads is a random value.



Significance test. Here we will try to answer the question: Is the coin fair? We'll follow the discussion in Wikipedia under P-value titled "Testing the fairness of a coin".

- We interpret "fair" as meaning $p_h = 0.5$. Our null hypothesis is that the coin is fair and $p_h = 0.5$.
- We plot the probabilities for getting R heads in $N = 20$ tosses using the binomial probability mass distribution on the right above.
- We decide on the significance $p_{\text{crit}} = 0.05$.

Checkpoint question

p_{crit} and p_h are both probabilities. Explain what they are the probabilities of and how they are different.

Answer

p_h is the probability that the coin comes up heads on any one toss. p_{crit} is the "unlikeliness" threshold we have chosen at which we will declare that the observed number of heads is too unlikely to have been generated by a fair coin. (Rejecting the null hypothesis.)

We need to find the probability of getting data *at least as extreme* as $D = \{R \text{ heads }, N \text{ tosses }\}$. For our example, that means adding up the binomial probabilities for $R = 14, 15, \ldots, 20$, which is 0.058. (This is called a "one-tailed test". If we wanted to consider deviations favoring tails as well, we would would add as well the probabilities for $R = 0, 1, \ldots, 6$, so 0.115 in total. This would be a "two-tailed test".) Comparing to $p_{\text{crit}} = 0.05$, we find the p-value is greater than p_{crit}, so the null hypothesis is not rejected at the 95% level. (Note that we say "not rejected" as opposed to "accepted".) If we had gotten 15 heads instead we would have rejected the null hypothesis. We emphasize that in this frequentist analysis, the *data* is random while p_h is fixed (although unknown).

Exercise 14.1 (Dealing with rejection)

Verify that if we had gotten 15 heads in $N = 20$ tosses that we would have rejected the null hypothesis.

Exercise 14.2 (How many tosses will it take?)

Look at Demo: Widgetized coin tossing. Pick a p_{crit}-value. If $p_h = 0.4$, work out how many coin tosses it would take to reject the null hypothesis that it's a fair coin ($p_h = 0.5$) at this significance level.

Hypothesis testing with the chi-squared statistic

A very common statistic to use is the χ^2 measure. A good example is found in Gregory, ch 7.2.1, with the measurements of flux density from a distant galaxy over a period of 6000 days. The main steps of the presented analysis are the following:

- Choose as a null hypothesis that the galaxy has an unknown, but constant, flux density. If we can reject this hypothesis at e.g. the 95% confidence level, then this is sometimes taken as support for the opposite view that the radio emission is variable. But remember that all we have really established is that the hypothesis of constant flux density is rejected at the chosen confidence level.
- In this example, it is assumed that the measurement errors are independent and identically distributed (**iid**) according to a normal distribution with a fixed standard deviation σ that is known beforehand.
- The χ^2 statistic from the data set is evaluated (x_i is the data and $\bar{x}$ is the average from the sample)

$$\chi^2 = \sum_{i=1}^{n} \frac{(x_i - \bar{x})^2}{\sigma^2}. \tag{14.1}$$

- In our example we had 15 data points, but we are using them first to estimate the mean μ. Therefore, we lose one degree of freedom and are left with 14. This number will determine the form of the χ^2 distribution that will be used for comparison with our actual χ^2 statistic.
- The question of how unlikely is this value of χ^2 is by convention interpreted in terms of the area in the tail of the χ^2 distribution beyond this line. This is the P-value or significance of the test, under these data.
- In some cases, a two-sided statistic should be considered instead.

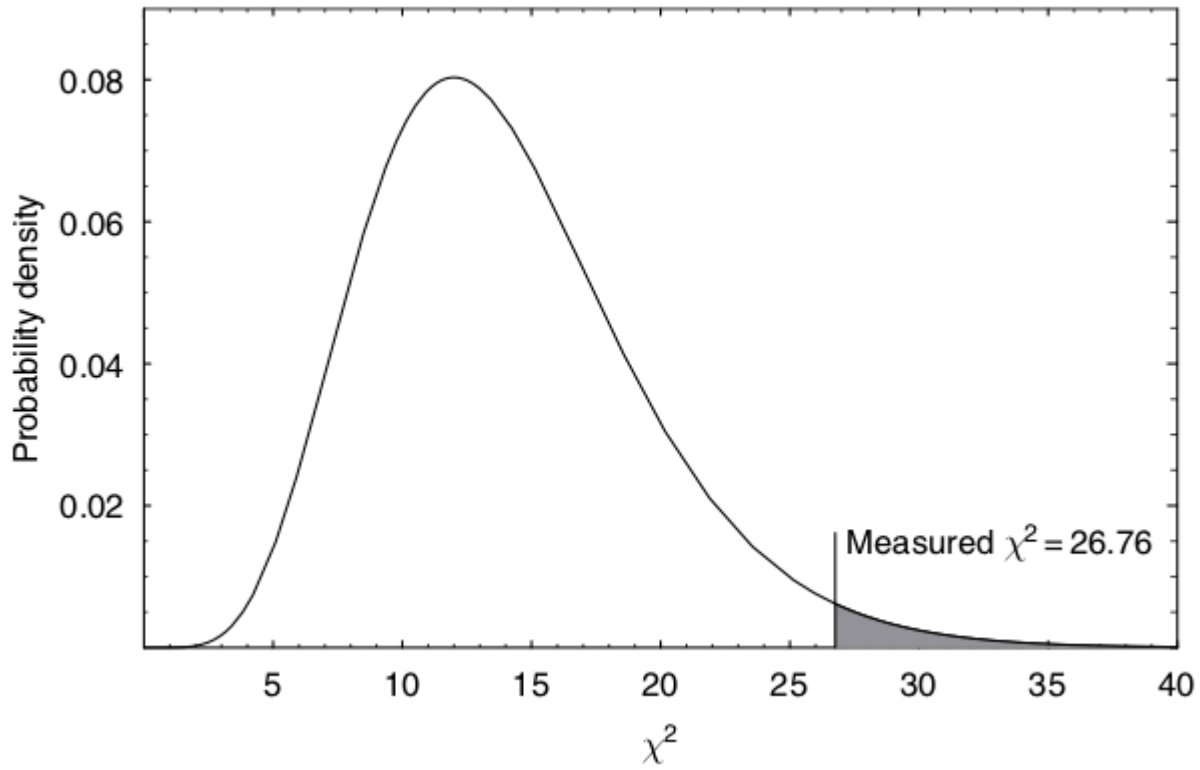


Fig. 14.1 The χ^2 distribution for 14 degrees of freedom. The value computed from the measurements of flux density from a galaxy is indicated by a vertical line. The shaded area is the P-value. It is 2% in this particular example so we would reject the null hypothesis with 98% confidence. (Gregory [Gre05], Fig. 7.2)

At the point of performing this comparison, and making a final statement, the sampling theory school divides itself into two camps:

1. One camp uses the following protocol: first, before looking at the data, pick the significance level of the test (e.g. 5%), and determine the critical value of χ^2 above which the null hypothesis will be rejected. (The significance level is the fraction of times that the statistic χ^2 would exceed the critical value, if the null hypothesis were true.) Then, compare the actual χ^2 with the critical value, and declare the outcome of the test, and its significance level (which was fixed beforehand). This was the school articulated above when the P-value was first introduced.
2. The second camp looks at the data, finds χ^2, then looks in the table of χ^2-distributions for the significance level, P, for which the observed value of χ^2 would be the critical value. The result of the test is then reported by giving this value of p, which is the fraction of times that a result as extreme as the one observed, or more extreme, would be expected to arise if the null hypothesis were true.

χ^2/dof for model assessment and comparison

Many physicists learn to judge whether a fit of a model to data is good or to pick out the best fitting model among several by evaluating the χ^2/dof (dof = “degrees of freedom”) for a given model and comparing the result to one. Here χ^2 is the sum of the squares of the residuals (data minus model predictions) divided by variance of the error at each point:

$$\chi^2 \equiv \sum_{i=1}^{N_{\text{data}}} \frac{\left(y_i - f(x_i;\hat{\boldsymbol{\theta}})\right)^2}{\sigma_i^2},$$

where y_i is the i^{th} data point, $f(x_i;\hat{\boldsymbol{\theta}})$ is the prediction of the model for that point using the best fit for the parameters, $\hat{\boldsymbol{\theta}}$, and σ_i is the error bar for that data point. The degrees-of-freedom (dof), often denoted by ν, is the number of data points minus number of fitted parameters:

$$\nu = N_{\text{data}} - N_{\text{fit parameters}}.$$

The rule of thumb is generally that $\chi^2/\text{dof} \gg 1$ means a poor fit and $\chi^2/\text{dof} < 1$ indicates overfitting. *Where does this come from and under what conditions is it a statistically valid thing to analyze fit models this way?* (Note that in contrast to the Bayesian evidence, we are not assessing the model in general, but a particular fit to the model identified by $\hat{\boldsymbol{\theta}}$.)

Underlying this use of χ^2/dof is a special case of the familiar statistical model

$$y_{\text{expt}} = y_{\text{th}} + \delta y_{\text{expt}} + \delta y_{\text{th}}$$

in which the theory is $y_{\text{th},i} = f(x_i;\hat{\boldsymbol{\theta}})$, the experimental error is *independent* Gaussian distributed noise with mean zero and standard deviation σ_i, that is $\delta y_{\text{expt}} \sim \mathcal{N}(0,\Sigma)$ with $\Sigma_{ij} = \sigma_i^2\delta_{ij}$, and δy_{th} is neglected (i.e., no model discrepancy is included). As a consequence

$$y_{\text{expt}} \sim \mathcal{N}\left(f(x_i;\hat{\boldsymbol{\theta}}),\Sigma\right).$$

The likelihood (and the posterior, with an implicitly assumed uniform prior) is then proportional to $e^{-\chi^2(\hat{\boldsymbol{\theta}})/2}$.

According to this model, each term in χ^2 before being squared is a draw from a *standard* normal distribution. In this context, “standard” means that the distribution has mean zero and variance 1. This is exactly what happens when we take as the random variables $\left(y_i - f(x_i;\hat{\boldsymbol{\theta}})\right)/\sigma_i$. But the sum of the squares of k *independent* standard normal random variables has a known distribution, called the χ^2 distribution with k degrees of freedom. So the sum of the normalized residuals squared should be distributed (if you generated many sets of them) as a χ^2 distribution. How many degrees of freedom? This should be the number of independent pieces of information. But we have found the fitted parameters $\hat{\boldsymbol{\theta}}$ by minimizing χ^2, i.e., by setting $\partial\chi^2(\boldsymbol{\theta})/\partial\theta_j$ for $j = 1,\ldots,N_{\text{fit parameters}}$, which means $N_{\text{fit parameters}}$ constraints. Therefore the number of dofs is given by $\nu = N_{\text{data}} - N_{\text{fit parameters}}$.

Now what do we do with this information? We only have one draw from the (supposed) χ^2 distribution. But if that distribution is narrow, we should be close to the mean. The mean of a χ^2 distribution with $k = \nu$ dofs is ν, with variance 2ν. So if we’ve got a good fit (and our statistical model is valid), then χ^2/ν should be close to one. If it is much larger, then the conditions are not satisfied, so the model doesn’t work. If it is smaller, than the failure implies that the residuals are too small, meaning overfitting.

But we should expect fluctuations, i.e., we shouldn’t always get the mean (or the mode, which is $\nu - 2$ for $\nu \geq 0$). If ν is large enough, then the distribution is approximately Gaussian and we can use the standard deviation / dof or

$\sqrt{2\nu}/\nu = \sqrt{2/\nu}$ as an expected width around one. One might use two or three times σ as a range to consider. If there are 1000 data points, then $\sigma \approx 0.045$, so $0.91 \leq \chi^2/\mathrm{dof} \leq 1.09$ would be an acceptable range at the 95% confidence level (2σ). With fewer data points this range grows significantly. **So in making the comparison of χ^2/ν to one, be sure to take into account the width of the distribution, i.e., consider $\pm\sqrt{2/\nu}$ roughly.** For a small number of data points, do a better analysis!

What can go wrong? Lots! See "Do's and Don'ts of reduced chi-squared" for a thorough discussion. But we've made assumptions that might not hold: that the data is Gaussian distributed and independent, that the data is dominated by experimental and not theoretical errors, and that the constraints from fitting are linearly independent. We've also assumed a lot of data and we've ignored informative (or, more precisely, non-uniform) priors.

14.2. Bayesian model selection

The Story of Dr. A and Prof. B

[Reproduced, with some modifications, from Sivia, 2006].

> *Dr. A has a theory; Prof. B also has a theory, but with an adjustable parameter λ. Whose theory should we prefer on the basis of data D?*— Jefferys (1939), Gull (1988), Sivia (2006)

It is clear that we need to evaluate the posterior probabilities for A and B being correct to ascertain the relative merit of the two theories. If the ratio of the posterior probabilities,

$$\text{posterior ratio} \quad = \frac{p(A|D,I)}{p(B|D,I)} \tag{14.2}$$

is very much greater than one, then we will prefer A's theory; if it is very much less than one, then we prefer that of B; and if it is of order unity, then the current data is insufficient to make an informed judgement.

To estimate the odds, let us start by applying Bayes' theorem to both the numerator and the denominator; this gives

$$\frac{p(A|D,I)}{p(B|D,I)} = \frac{p(D|A,I)p(A|I)}{p(D|B,I)p(B|I)} \tag{14.3}$$

because the term $p(D|I)$ cancels out, top and bottom. As usual, probability theory warns us immediately that the answer to our question depends partly on what we thought about the two theories before the analysis of the data. To be fair, we might take the ratio of the prior terms, on the far right of Eq. (14.3), to be unity; a harsher assignment could be based on the past track records of the theorists! To assign the probabilities involving the experimental measurements, $p(D|A,I)$ and $p(D|B,I)$, we need to be able to compare the data with the predictions of A and B: the larger the mismatch, the lower the corresponding probability. This calculation is straightforward for Dr A, but not for Prof B; the latter cannot make predictions without a value for λ.

To circumvent this difficulty, we can use the sum and product rule to relate the probability we require to other pdfs which might be easier to assign. In particular, marginalization and the product rule allow us to express $p(D|B,I)$ as

$$p(D|B,I) = \int d\lambda\, p(D,\lambda|B,I) = \int d\lambda\, p(D|\lambda,B,I)p(\lambda|B,I). \tag{14.4}$$

The first term in the integral $p(D|\lambda,B,I)$, where the value of λ is given, is now just an ordinary likelihood function; as such, it is on a par with $p(D|A,I)$. The second term is B's prior pdf for λ; the onus is, therefore, on the theorist to articulate his or her state of knowledge, or ignorance, before getting access to the data.

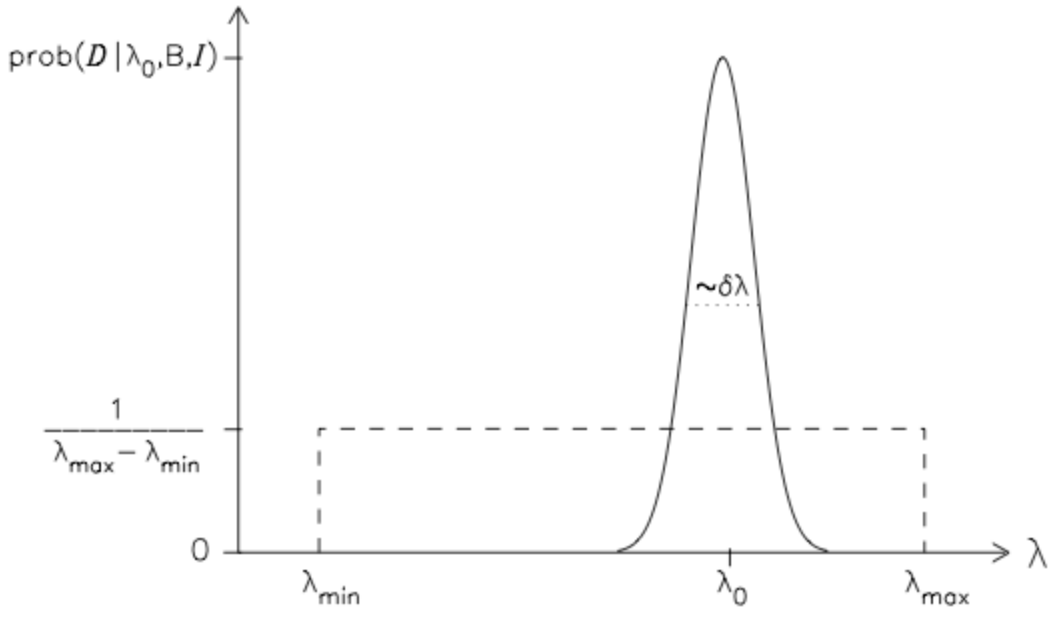
To proceed further analytically, let us make some simplifying approximations. Assume that, a priori, Prof B is only prepared to say that λ must lie between the limits $\lambda_{\min}$ and $\lambda_{\max}$; we can then naively assign a uniform prior within this range:

$$p(\lambda|B,I) = \frac{1}{\lambda_{\max} - \lambda_{\min}} \quad \text{for } \lambda_{\min} \le \lambda \le \lambda_{\max}, \tag{14.5}$$

and zero otherwise. Let us also take it that there is a value λ_0 which yields the closest agreement with the measurements; the corresponding probability $p(D|\lambda_0,B,I)$ will be the maximum of B's likelihood function. As long as this adjustable parameter lies in the neighbourhood of the optimal value, $\lambda_0 \pm (\delta\lambda)$, we would expect a reasonable fit to the data; this can be represented by the Gaussian pdf

$$p(D|\lambda,B,I) = p(D|\lambda_0,B,I)\exp\left[-\frac{(\lambda-\lambda_0)^2}{2(\delta\lambda)^2}\right]. \tag{14.6}$$

The assignments of the prior (14.5) and the likelihood (14.6) are illustrated in the figure below. We may note that, unlike the prior pdf $p(\lambda|B,I)$, B's likelihood function need not be normalized with respect to λ; in other words, $p(D|\lambda_0,B,I)$ need not equal $1/(\delta\lambda)\sqrt{2\pi}$. This is because the λ in $p(D|\lambda,B,I)$ appears in the conditioning statement, whereas the normalization requirement applies to quantities to the left of the '|' symbol.



In the evaluation of $p(D|B,I)$, we can make use of the fact that the prior does not depend explicitly on λ; this enables us to take $p(\lambda|B,I)$ outside the integral in Eq. (14.4).

$$p(D|B,I) = \frac{1}{\lambda_{\max} - \lambda_{\min}} \int_{\lambda_{\min}}^{\lambda_{\max}} d\lambda \, p(D|\lambda, B, I), \tag{14.7}$$

having set the limits according to the specified range. Assuming that the sharp cut-offs do not cause a significant truncation of the Gaussian pdf of the likelihood, its integral will be equal to $(\delta\lambda)\sqrt{2\pi}$ times $p(D|\lambda_0, B, I)$. The troublesome term then reduces to

$$p(D|B,I) = \frac{1}{\lambda_{\max} - \lambda_{\min}} p(D|\lambda_0, B, I)(\delta\lambda)\sqrt{2\pi}. \tag{14.8}$$

Substituting this into Eq. (14.3), we finally see that the ratio of the posteriors required to answer our original question decomposes into the product of three terms:

$$\frac{p(A|D,I)}{p(B|D,I)} = \frac{p(A|I)}{p(B|I)} \times \frac{p(D|A,I)}{p(D|\lambda_0, B, I)} \times \frac{\lambda_{\max} - \lambda_{\min}}{(\delta\lambda)\sqrt{2\pi}}. \tag{14.9}$$

The first term on the right-hand side reflects our relative prior preference for the alternative theories; to be fair, let us take it to be unity. The second term is a measure of how well the best predictions from each of the models agree with the data; with the added flexibility of his adjustable parameter, this maximum likelihood ratio can only favour B. The goodness-of-fit, however, cannot be the only thing that matters; if it was, we would always prefer more complicated explanations. Probability theory tells us that there is, indeed, another term to be considered. As assumed earlier in the evaluation of the marginal integral of Eq. (14.4), the prior range $\lambda_{\max} - \lambda_{\min}$ will generally be much larger than the uncertainty $\pm(\delta\lambda)$ permitted by the data. As such, the final term in Eq. (14.9) acts to penalize B for the additional parameter; for this reason, it is often called an Ockham factor. That is to say, we have naturally encompassed the spirit of Ockham's Razor: *'Frustra fit per plura quod potest fieri per pauciora'* or, in English, *'it is vain to do with more what can be done with fewer'* .

Although it is satisfying to quantify the everyday guiding principle attributed to the thirteenth-century Franciscan monk William of Ockham (or Occam, in Latin), that we should prefer the simplest theory which agrees with the empirical evidence, we should not get too carried away by it. After all, what do we mean by the simpler theory if alternative models have the same number of adjustable parameters? In the choice between Gaussian and Lorentzian peak shapes, for example, both are defined by the position of the maximum and their width. All that we are obliged to do, and have done, in addressing such questions is to adhere to the rules of probability.

While accepting the clear logic leading to Eq (14.9), many people rightly worry about the question of the limits $\lambda_{\min}$ and $\lambda_{\max}$. Jeffreys (1939) himself was concerned and pointed out that there would be an infinite penalty for any new parameter if the range was allowed to go to $\pm\infty$. Stated in the abstract, this would appear to be a severe limitation. In practice, however, it is not generally such a problem: since the analysis is always used in specific contexts, a suitable choice can usually be made on the basis of the relevant background information. Even in uncharted territory, a small amount of thought soon reveals that our state of ignorance is always far from the $\pm\infty$ scenario. If λ was the coupling constant (or strength) for a possible fifth force, for example, then we could put an upper bound on its magnitude because everybody would have noticed it by now if it had been large enough! We should also not lose sight of the fact that the precise form of Eq (14.9) stems from our stated simplifying approximations; if these are not appropriate, then Eqs. (14.3) and (14.4) will lead us to a somewhat different formula.

In most cases, our relative preference for A or B is dominated by the goodness of the fit to the data; that is to say, the maximum likelihood ratio in eqn (14.9) tends to overwhelm the contributions of the other two terms. The

Ockham factor can play a crucial role, however, when both theories give comparably good agreement with the measurements. Indeed, it becomes increasingly important if B's theory fails to give a significantly better fit as the quality of the data improves. In that case, $(\delta\lambda)$ continues to become smaller but the ratio of best-fit likelihoods remains close to unity; according to Eq. (14.9), therefore, A's theory is favoured ever more strongly. By the same token, the Ockham effect disappears if the data are either few in number, of poor quality or just fail to shed new light on the problem at hand. This is simply because the posterior ratio of Eq. (14.9) is then roughly equal to the complementary prior one, since the empirical evidence is very weak; hence, there is no inherent preference for A's theory unless it is explicitly encoded in $p(A|I)/p(B|I)$. This property can be verified formally by going back to Eqs. (14.3), (14.6) and (14.7), and considering the poor-data limit in which $(\delta\lambda) \gg \lambda_{\max} - \lambda_{\min}$ and $p(D|\lambda_0, B, I) \approx p(D|A, I)$.

One adjustable parameter each

Some further interesting features arise when we consider the case where Dr A also has one adjustable parameter; call it μ. If we make the same sort of probability assignments, and simplifying approximations, as for Prof B, then we find that

$$\frac{p(A|D,I)}{p(B|D,I)} = \frac{p(A|I)}{p(B|I)} \times \frac{p(D|\mu_0, A, I)}{p(D|\lambda_0, B, I)} \times \frac{\delta\mu(\lambda_{\max} - \lambda_{\min})}{\delta\lambda(\mu_{\max} - \mu_{\min})}. \tag{14.10}$$

This could represent the situation where we have to choose between a Gaussian and Lorentzian shape for a signal peak, but one associated parameter is not known. The position of the maximum may be fixed at the origin by theory, for example, and the amplitude constrained by the normalization of the data; A and B could then be the hypotheses favouring the alternative lineshapes, where $\delta\mu$ and $\delta\lambda$ are their related full-width-half-maxima. If we give equal weight to A and B before the analysis, and assign a similar large prior range for both μ and λ, then Eq. (14.10) reduces to

$$\frac{p(A|D,I)}{p(B|D,I)} \approx \frac{p(D|\mu_0, A, I)}{p(D|\lambda_0, B, I)} \times \frac{\delta\mu}{\delta\lambda}. \tag{14.11}$$

For data of good quality, the dominant factor will tend to be the best-fit likelihood ratio. If both give comparable agreement with the measurements, however, then the shape with the larger error-bar for its associated parameter will be favoured. At first sight, it might seem rather odd that the less discriminating theory can gain the upper hand. It appears less strange once we realize that, in the context of model selection, a larger 'error-bar' means that more parameter values are consistent with the given hypothesis; hence its preferential treatment.

One adjustable parameter each; different prior ranges

Finally, we can also consider the situation where Mr A and Mr B have the same physical theory but assign a different prior range for λ (or μ). Although Eq. (14.9) can be seen as representing the case when $(\mu_{\max} - \mu_{\min})$ is infinitesimally small, so that A has no flexibility, Eq. (14.10) is more appropriate when the limits set by both theorists are large enough to encompass all the parameter values giving a reasonable fit to the data. With equal initial weighting towards A and B, the latter reduces to

$$\frac{p(A|D,I)}{p(B|D,I)} = \frac{\lambda_{\max} - \lambda_{\min}}{\mu_{\max} - \mu_{\min}}. \tag{14.12}$$

because the best-fit likelihood ratio will be unity (since $\lambda_0 = \mu_0$) and $\delta\lambda = \delta\mu$. Thus, our analysis will lead us to prefer the theorist who gives the narrower prior range; this is not unreasonable as he must have had some additional insight to be able to predict the value of the parameter more accurately.

Comparison with parameter estimation

The dependence of the result in Eq. (14.9) on the prior range $(\lambda_{\max} - \lambda_{\min})$ can seem a little strange, since we haven't encountered such behaviour in the preceding chapters. It is instructive, therefore, to compare the model selection analysis with parameter estimation. To infer the value of λ from the data, given that B's theory is correct, we use Bayes' theorem:

$$p(\lambda|D,B,I) = \frac{p(D|\lambda,B,I)p(\lambda|B,I)}{p(D|B,I)}. \tag{14.13}$$

The numerator is the familiar product of a prior and likelihood, and the denominator is usually omitted since it does not depend explicitly on λ; hence this relationship is often written as a proportionality. From the story of Dr A and Prof B, however, we find that the neglected term on the bottom plays a crucial role in ascertaining the merit of B's theory relative to a competing alternative.

- In recognition of its new-found importance, the denominator in Bayes' theorem is sometimes called the **'evidence'** for B; it is also referred to as the 'marginal likelihood', the 'global likelihood' and the 'prior predictive'.
- Since all the components necessary for both parameter estimation and model selection appear in Eq. (14.13), we are not dealing with any new principles; the only thing that sets them apart is that we are asking different questions of the data.

A simple way to think about the difference between parameter estimation and model selection is to note that, to a good approximation, the former requires the location of the maximum of the likelihood function whereas the latter entails the calculation of its average value. As long as $\lambda_{\min}$ and $\lambda_{\max}$ encompass the significant region of $p(D|\lambda,B,I)$ around λ_0, the precise bounds do not matter for estimating the optimal parameter and need not be specified. Since the prior range defines the domain over which the mean likelihood is computed, due thought is necessary when dealing with model selection. Indeed, it is precisely this act of comparing 'average' likelihoods rather than 'maximum' ones which introduces the desired Ockham balance to the goodness- of-fit criterion. Any likelihood gain from a better agreement with the data, allowed by the greater flexibility of a more complicated model, has to be weighed against the additional cost of averaging it over a larger parameter space.

14.3. Evidence ratios for an expansion

Evidence ratios in general

Consider models M_1 and M_2, with the same dataset D. For the evidence $p(M_1|D,I)$ versus $p(M_2|D,I)$ there is no reference to a particular parameter set $\Longrightarrow$ it is a comparison between two models, not two fits. As already noted, in Bayesian model selection only a *comparison* makes sense. One does not deal with a hypothesis like:

"Model M_1 is correct." Here we will eventually take M_2 to be M_1 with one extra order in an expansion (one additional parameter).

To proceed, as is often the case, we first apply Bayes' theorem:

$$\frac{p(M_2|D,I)}{p(M_1|D,I)} = \frac{p(D|M_2,I)p(M_2,I)/p(D|I)}{p(D|M_1,I)p(M_1,I)/p(D|I)},$$

where the Bayes' theorem denominators cancel. We'll take the ratio $p(M_2|I)/p(M_1,I)$ to be one. Thus we have:

$$\frac{p(M_2|D,I)}{p(M_1|D,I)} = \frac{\int d\boldsymbol{a}_2\, p(D|\boldsymbol{a}_2,M_2,I)p(\boldsymbol{a}_2|M_2,I)}{\int d\boldsymbol{a}_1\, p(D|\boldsymbol{a}_1,M_1,I)p(\boldsymbol{a}_1|M_1,I)}, \tag{14.14}$$

where we've made the usual application of the product rule in the marginalization integral in numerator and denominator. Note that the integration is over the *entire* parameter space. This is difficult numerically because likelihoods are usually peaked but can have long tails that contribute to the integral (cf. averaging over the likelihood vs. finding the peak).

The special case of expansions

Consider the easiest example: $M_1 \to M_k$ and $M_2 \to M_{k+1}$, where k is the order in an expansion. The question is then: *Is going to a higher-order favored by the given data?* To further simplify, assume M_{k+1} has one additional parameter a' and assume the priors factorize. For example they are Gaussians:

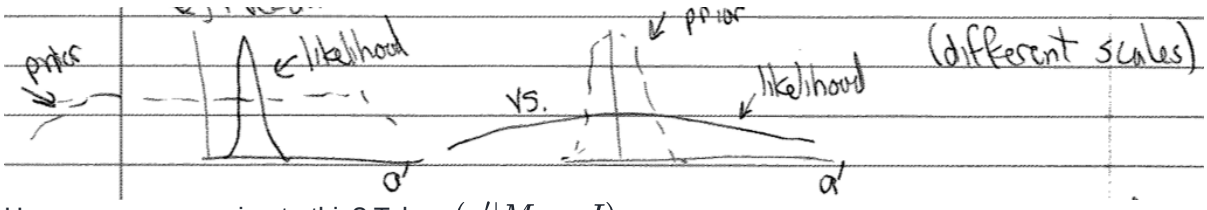
$$e^{-\boldsymbol{a}^2/2\bar{a}^2} = e^{-a_0^2/2\bar{a}^2} e^{-a_1^2/2\bar{a}^2} \cdots e^{-a_k^2/2\bar{a}^2}.$$

Then

$$p(\boldsymbol{a}_2|M_{k+1},I) = p(\boldsymbol{a}_1,a'|M_{k+1},I) = p(\boldsymbol{a}_1|M_{k+1},I)p(a'|M_{k+1},I)$$

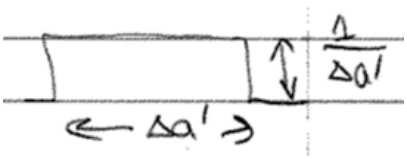
Consider cases …

i) values of a' that contribute to the integrand in the numerator of (14.14) are determined by the likelihood peaked region. E.g., recall



How can we approximate this? Take $p(a'|M_{k+1},I)$

Call the value of the likelihood peak $\hat{a}$ and the width $\delta a'$. So two different widths: the before-data $\Delta a'$ (width of prior) and the after-data $\delta a'$ (likelihood $\to$ posterior). Do the a' integral:

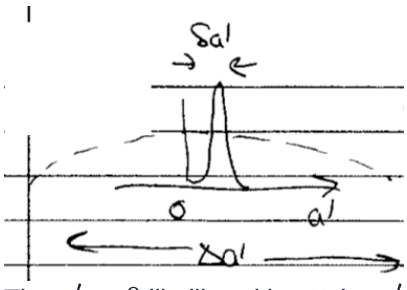


$$\implies \frac{p(D|M_{k+1},I)}{p(D|M_k)} \approx \frac{\delta a'}{\Delta a'} \frac{\int d\boldsymbol{a}_1\, p(D|\boldsymbol{a}_1,\hat{a}',M_{k+1},I)p(\boldsymbol{a}_1|M_{k+1},I)}{\int d\boldsymbol{a}_1\, p(D|\boldsymbol{a}_1,M_k,I)p(\boldsymbol{a}_1|M_k,I)}$$

with $\delta a'$ from the integral over a' (leaving the peak value $\hat{a}'$ in the numerator integral over $\boldsymbol{a}_1$) and $\Delta a'$ from $p(a'|M_{k+1},I)$.

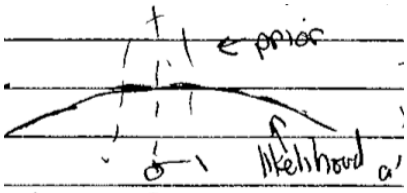
Therefore the ratio of the integrals is the gain in the likelihood from an extra parameter with value $\hat{a}$ (cf. $M_{k+1}(\hat{a}'=0)=M_k$). But also the "Occam factor" or "Occam penalty" $\delta a'/\Delta a'$. How much parameter space collapses in the face of data? We thought initially that a' could be anywhere in $\Delta a'$, but find after seeing the data that it is only in $\delta a'$. What a waste (less predictive) if $\delta a' \ll \Delta a'$! These factors play off each other: if we add a parameter to a nested model, we expect to gain because $\hat{a}'$ is more information (it could be $a'=0$ instead).

Now if this is the case:



The $a'=0$ likelihood is $\ll$ the $a'=\hat{a}'$ likelihood $\implies$ evidence ratio $\gg 1$ and inclusion of this parameter is highly favored. *Unless* you put a flat prior from near $-\infty$ to near $+\infty$. But we have a natural prior, so $\Delta a'$ is restricted.

ii) Now suppose:



Turn the analysis on its head. The dependence on a' is weak because of the width of the likelihood, so we can replace it in $p(D|\boldsymbol{a}_1,a',M_{k+1},I)$ by $\hat{a}$:

$$\implies \frac{p(D|M_{k+1},I)}{p(D|M_k)} \approx \frac{\int d\boldsymbol{a}_1\, p(D|\boldsymbol{a}_1,\hat{a}',M_{k+1},I)p(\boldsymbol{a}_1|M_{k+1},I)\int da' p(a'|M_{k+1},I)}{\int d\boldsymbol{a}_1\, p(D|\boldsymbol{a}_1,M_k,I)p(\boldsymbol{a}_1|M_k,I)}$$

The second integral in the numerator is now just a normalization integral, so it yields one. Further, we can take $\hat{a}' \approx 0$ because we are dominated by the prior. But M_{k+1} with $\hat{a}'=0$ is M_k! This means that the Bayes ratio in this situation goes to one rather than decreasing. The same argument applies to $k+1 \to k_2 \to \ldots \implies$ we have saturation of the a_k's.

Summary: a naturalness prior cuts down on wasted space in the parameter phase space that might be ruled out by data. Thus an EFT is a simpler model (in the model selection sense) than the same functional form with uncontrained or only weakly constrained LECs.

Checkpoint question

Predict based on your own experience: How does this behavior change if we have more data (higher energy) or more certain data?

Answer

[fill in your own answer!]

Evidence with linear algebra

Return to the notebook to look at the calculation of evidence with Python linear algebra. The integrals to calculate are Gaussian in multiple variables: $\boldsymbol{a} = (a_0, \ldots, a_k)$ plus $\bar{a}$. We can write them with matrices:

$$\chi^2 = [Y - A\boldsymbol{\theta}]^\intercal \, \boldsymbol{\Sigma}^{-1} \, [Y - A\boldsymbol{\theta}],$$

where

$$Y - A\boldsymbol{\theta} = \underbrace{\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}}_{N\times 1} - \underbrace{\begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^k \\ 1 & x_2 & x_2^2 & \cdots & x_2^k \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^k \end{pmatrix}}_{N\times(k+1)} \underbrace{\begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_k \end{pmatrix}}_{(k+1)\times 1}$$

and

$$\Sigma = \underbrace{\begin{pmatrix} \sigma_1^2 & \rho_{12}\sigma_1\sigma_{y_2} & \cdots & \rho_{1N}\sigma_1\sigma_N \\ & \sigma_2^2 & \cdots & \cdots \\ & & \ddots & \cdots \\ & & & \sigma_N^2 \end{pmatrix}}_{N\times N}.$$

Then from before we have χ^2_{MLE} when

$$\widehat{\boldsymbol{\theta}} = (A^\intercal \Sigma^{-1} A)^{-1} (A^\intercal \Sigma^{-1} Y).$$

Here we have a couple of options:

i) Use

$$\int e^{-\frac{1}{2}x^\intercal Mx + \boldsymbol{B}^\intercal x} dx = \sqrt{\det(2\pi M^{-1})} e^{\frac{1}{2}\boldsymbol{B}^\intercal M^{-1}\boldsymbol{B}}$$

where M is any symmetric matrix and $\boldsymbol{B}$ any vector. Derive this result by completing the square in the exponent (subtract and add $\frac{1}{2}\boldsymbol{B}^\intercal M^{-1}\boldsymbol{B}$).

ii) Use conjugacy. See the "conjugate prior" entry in Wikipedia for details. Apply this to Bayes' theorem with a Gaussian prior $p(\boldsymbol{\theta})$ with (μ_0, σ_0) and a Gaussian likelihood $p(D|\boldsymbol{\theta})$ with (μ, σ). Then $p(\boldsymbol{\theta}|D)$ is a Gaussian with

$$\tilde{\mu} = \frac{1}{\frac{1}{\sigma_0^2} + \frac{N}{\sigma^2}} \left(\frac{\mu_0}{\sigma_0^2} + \frac{\sum_{i=1}^N y_i}{\sigma^2} \right) \qquad \tilde{\sigma}^2 = \left(\frac{1}{\sigma_0^2} + \frac{N}{\sigma^2} \right)^{-1}$$

Checkpoint question

Check the $N \to \infty$ limit

Answer

Then the terms with μ_0 and σ_0 become negligible, and

$$\tilde{\mu} \underset{N\to\infty}{\longrightarrow} \frac{1}{N}\sum_{i=1}^N y_i \qquad \tilde{\sigma}^2 \underset{N\to\infty}{\longrightarrow} \sigma^2/N.$$

14.4. Evidence calculations

The actual computation of Bayesian evidences can be a challenging task. Recall that we often have knowledge of the posterior distribution only through sampling. In many cases, the simple Laplace method can be used to compute the evidence approximately, while in other cases we have to rely on special sampling algorithms such as nested sampling or parallel tempering with thermodynamic integration.

Laplace's method

The idea behind Laplace's method is simple, namely to use the Taylor expansion of a function around its peak to construct a Gaussian approximation which can be easily integrated. Let us consider an unnormalized density $P^*(\theta)$ that has a peak at a point θ_0. We are interested in the evidence, Z_P, which is given by the integral

$$Z_P = \int P^*(\theta) d^K\theta, \tag{14.15}$$

where we consider the general case in which θ is a K-dimensional vector of parameters.

We Taylor-expand the logarithm $\log P^*$ around the peak:

$$\log P^*(\theta) = \log P^*(\theta_0) - \frac{1}{2}(\theta-\theta_0)^T\Sigma^{-1}(\theta-\theta_0) + \ldots, \tag{14.16}$$

where $\Sigma^{-1} \equiv H$ is the (Hessian) matrix of second derivatives at the maximum

$$H_{ij} = -\frac{\partial^2}{\partial\theta_i\partial\theta_j}\log P^*(\theta)\bigg|_{\theta=\theta_0}. \tag{14.17}$$

By truncating the Taylor expansion at the second order we find that $P^*(\theta)$ is approximated by an unnormalized Gaussian,

$$P^*(\theta) \approx P^*(\theta_0)\exp\left[-\frac{1}{2}(\theta-\theta_0)^T\Sigma^{-1}(\theta-\theta_0)\right]. \tag{14.18}$$

The integral of this Gaussian is given by $P^*(\theta_0)$ times (the inverse of) its normalization constant. Thus we approximate Z_P by

$$Z_P \approx P^*(\theta_0)\sqrt{\frac{(2\pi)^K}{\det\Sigma^{-1}}}. \tag{14.19}$$

Predictions can then be made using this approximation. Physicists also call this widely-used approximation the saddle-point approximation.

Note, in particular, that if we consider a chi-squared likelihood: $P^*(\theta) = \mathcal{L}(D|\theta) = \exp\left(-\frac{1}{2}\chi^2(\theta)\right)$, then we get

$$Z_P \approx \exp\left(-\frac{1}{2}\chi^2(\theta_0)\right)\sqrt{\frac{(4\pi)^K}{\det\Sigma^{-1}}}, \tag{14.20}$$

where there is a factor $2^{K/2}$ that comes from the extra factor $1/2$ multiplying the covariance matrix Σ^{-1} and therefore appearing in all K eigenvalues.

Note that the Laplace approximation is basis-dependent: if θ is transformed to a nonlinear function $u(\theta)$ and the density is transformed to $P(u) = P(\theta)|d\theta/du|$ then in general the approximate normalizing constants Z_Q will be different. This can be viewed as a defect—since the true value Z_P is basis-independent in this approximation—or an opportunity, because we can hunt for a choice of basis in which the Laplace approximation is most accurate.

Normalization of a multivariate Gaussian

The fact that the normalizing constant of a Gaussian is given by

$$\int d^K\theta \exp\left[-\frac{1}{2}\theta^T\Sigma^{-1}\theta\right] = \sqrt{\frac{(2\pi)^K}{\det\Sigma^{-1}}}, \tag{14.21}$$

can be proved by making an orthogonal transformation into the basis u in which Σ is transformed into a diagonal matrix of eigenvalues λ_i. The integral then separates into a product of one-dimensional integrals, each of the form

$$\int du_i \exp\left[-\frac{1}{2}\lambda_i u_i^2\right] = \sqrt{\frac{2\pi}{\lambda_i}} \tag{14.22}$$

The product of the eigenvalues λ_i is the determinant of Σ^{-1}.

Correlations in the Laplace approximation

In the "fitting a straight-line" example you should find that the joint pdf for the slope and the intercept $[m, b]$ corresponds to a slanted ellipse. That result implies that the model parameters are (anti) **correlated**.

- *Try to understand the correlation that you find in this example.*

Let us explore correlations by studying the behavior of a bivariate pdf near the maximum where we employ the Laplace approximation (neglecting terms beyond the quadratic one in a Taylor expansion). We start by considering two independent parameters x and y, before studying the dependent case.

Two independent parameters

Independence implies that $p(x, y) = p_x(x)p_y(y)$. We will again consider the log-pdf $L(x, y) = \log\left(p(x, y)\right)$ which will then be

$$L(x, y) = L_x(x) + L_y(y). \tag{14.23}$$

At the mode we will have $\partial p/\partial x = \partial p_x/\partial x = \partial L_x/\partial x = 0$, and similarly $\partial L_y/\partial y = 0$.

The second derivatives will be

$$A \equiv \left.\frac{\partial^2 L_x}{\partial x^2}\right|_{x=x_0} < 0, \quad B \equiv \left.\frac{\partial^2 L_y}{\partial y^2}\right|_{y=y_0} < 0, \quad C \equiv \left.\frac{\partial L(x,y)}{\partial x \partial y}\right|_{x=x_0, y=y_0} = 0. \tag{14.24}$$

such that our approximated (log) pdf near the mode will be

$$L(x, y) = L(x_0, y_0) - \frac{1}{2}|A|(x - x_0)^2 - \frac{1}{2}|B|(y - y_0)^2. \tag{14.25}$$

We could visualize this bivariate pdf by plotting iso-probability contours. Or, equivalently, iso-log-probability contours which correspond to

$$|A|(x-x_0)^2 + |B|(y-y_0)^2 = \text{constant}. \tag{14.26}$$

This you should recognize as the equation for an ellipse with its center in (x_0, y_0), the principal axes corresponding to the x and y directions, and with the width and height parameters $\sigma_x = 1/\sqrt{|A|}$ and $\sigma_y = 1/\sqrt{|B|}$, respectively.

Two dependent parameters

For two dependent parameters we cannot separate $p(x, y)$ into a product of one-dimensional pdf:s. Instead, the Taylor expansion for the bivariate log-pdf $L(x, y)$ around the mode (x_0, y_0) gives

$$L(x,y) \approx L(x_0,y_0) + \frac{1}{2}(x-x_0 \quad y-y_0) H \begin{pmatrix} x-x_0 \\ y-y_0 \end{pmatrix}, \tag{14.27}$$

where H is the symmetric Hessian matrix

$$\begin{pmatrix} A & C \\ C & B \end{pmatrix}, \tag{14.28}$$

with elements

$$A = \left.\frac{\partial^2 L}{\partial x^2}\right|_{x_0,y_0} < 0, \quad B = \left.\frac{\partial^2 L}{\partial y^2}\right|_{x_0,y_0} < 0, \quad C = \left.\frac{\partial^2 L}{\partial x \partial y}\right|_{x_0,y_0} \neq 0. \tag{14.29}$$

- So in this quadratic approximation the contour is an ellipse centered at (x_0, y_0) with orientation and eccentricity determined by A, B, C.
- The principal axes are found from the eigenvectors of H.
- Depending on the skewness of the ellipse, the parameters are either (i) not correlated ($C = 0$), (ii) correlated, or (iii) anti-correlated.
- Take a minute to consider what that implies.

Let us be explicit. The Hessian can be diagonalized (we will also change sign)

$$-H = U \begin{pmatrix} a & 0 \\ 0 & b \end{pmatrix} U^{-1}, \tag{14.30}$$

where a, b are the (positive) eigenvalues of $-H$, and $U = \begin{pmatrix} a_x & b_x \\ a_y & b_y \end{pmatrix}$ is constructed from the eigenvectors.
Defining a new set of translated and linearly combined parameters

$$x' = a_x(x-x_0) + a_y(y-y_0) y' = b_x(x-x_0) + b_y(y-y_0) \tag{14.31}$$

we find that the log-pdf in the new coordinates becomes

$$\begin{aligned} L(x', y') &= L(0,0) - \frac{1}{2}\begin{pmatrix} x' & y' \end{pmatrix}\begin{pmatrix} a & 0 \\ 0 & b \end{pmatrix}\begin{pmatrix} x' \\ y' \end{pmatrix} \\ &= L(0,0) - \frac{1}{2}a(x')^2 - \frac{1}{2}b(y')^2. \end{aligned} \tag{14.32}$$

Checkpoint question

What has been achieved by this change of variables?

Answer

The joint pdf now factorizes, which implies that the transformed variables are independent.

Computing the Bayesian evidence

There are many possible challenges in calculating the evidence, including

- The likelihood may be sharply peaked in the prior range, but could have long tails and significant contributions to the required integrals;
- The likelihood could be multimodal;
- The posterior may only be significant on thin "sheets" in parameter space (cf. visualization of sampling).

Trotta [Tro08] gives a summary of methods (which is somewhat out-of-date in places):

1. Thermodynamic integration $\longrightarrow$ simulated annealing. The computational cost depends heavily on dimensionality of parameter space and on details of likelihood function. For example, cosmological applications require up to 10^7 likelihood evaluations (100 times MCMC-based parameter estimation). A solution is to use parallel tempering (more to follow!).
2. Nested sampling recasts multidimensional evidence integral into a one-dimensional integral, which is easy to evaluate numerically. Generall this takes $\sim 10^5$ likelihood evaluations. `multinest` and newer versions are more efficient still.
3. Approximations to the Bayes factor:
   - If models are nested: ask whether a new parameter is supported by data.
   - Laplace approximation may be good but be careful of priors.
   - Define the effective number of parameters (see BDA3 [GCS+13])
   - AIC, BIC, DIC, WAIC (summary to follow; see BDA3 for details)
   - The paper "Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC" by Vehtari, Gelman, and Gabry is a good (and reliable) source for theoretical and practical details on assessing and comparing the predictive accuracy of different models. Quote: "Cross-validation and information criteria are two approaches to estimating out-of-sample predictive accuracy using within-sample fits." The computations use the log-likelihood evaluated at posterior simulations of the parameters.

14.5. Information criteria

Information criteria are computationally much easier than full evaluations of the evidence. In this section we give a summary of some of the most common ICs and then give an example in some detail.

Summary of information criteria

AIC: *Akaiko Information Criteria* This is essentially a frequentist approach as it relies solely on the likelihood (no priors involved). The quantity to calculate is:

$$AIC = -2 \log p(D|\hat{\boldsymbol{\theta}}_{\text{MLE}}) + 2k,$$

where k is the number of free parameters and the probability distribution is the likelihood evaluated at the maximum likelihood values of the parameters. In practice one compares the resulting quantity between the models in question. It has the ingredients of evidence: an improved likelihood is balanced by a penalty for additional parameters. AIC is not well regarded by Bayesians.

BIC: *Bayesian Information Criteria* This is a Gaussian approximation to the Bayesian evidence in the limit of a large amount of data.

$$BIC = -2 \log p(D|\hat{\boldsymbol{\theta}}_{\text{MLE}}) + k \ln N$$

where k is the number of fitted parameters and N is the number of data points. The BIC implicitly assumes that the Occam penalty is negligible.

DIC: *Deviance Information Criteria* With the DIC, replace $\hat{\boldsymbol{\theta}}_{\text{MLE}}$ by $\hat{\boldsymbol{\theta}}_{\text{Bayes}}$, where the latter is the maximum of the posterior (as opposed to the maximum of the likelihood). Use an effective number of parameters:

$$p_{DIC} = 2 \log p(D|\hat{\boldsymbol{\theta}}_{\text{Bayes}}) - E[\log p(D|\boldsymbol{\theta})],$$

where the last term averages $\boldsymbol{\theta}$ over the posterior. Then

$$DIC = -2 \log p(D|\hat{\boldsymbol{\theta}}_{\text{Bayes}}) + 2p_{DIC}.$$

WAIC: Widely Applicable Information Criteria* This is favored by the authors of BDA-3 as being more fully Bayesian. The implementation is: given samples $s = 1$ to S,

$$WAIC = 2 \sum_{i=1}^{n_{\text{data}}} \left(\log \frac{1}{S} \sum_{s=1}^{S} p(y_i|\boldsymbol{\theta}^s) \right) - \frac{1}{S} \sum_{s=1}^{S} \log p(D_i|\boldsymbol{\theta}^s)$$

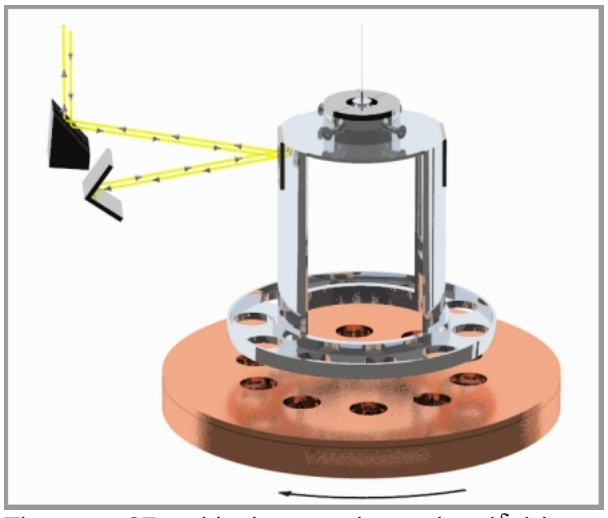
The WAIC averages over the posterior distribution.

Example: Information Criteria and Bayes factors

An example of applying Bayesian methods to perform model comparisons is "Model comparison tests of modified gravity from the Eöt-Wash experiment" by Krishak and Desai. They re-examine the claim in "Hints of Modified Gravity in Cosmos and in the Lab?"" by Perivolaropoulos and Kazantzidis, made using frequentist methods, that there is evidence in the data of the Eöt-Wash experiment that looks for modifications of Newton's Law of Gravity on sub-millimeter scales for a residual spatially oscillating signal in the data. This could point to a modification of general relativity (in particular, of some type of nonlocal gravity theory) or it could be due to statistical or systematic uncertainties in the data.

The experiment under consideration is a modern version of the classic torsion balance experiments to measure the force due to gravity. In particular, it is sensitive to departures from Newtonian gravity at sub-millimeter scales. The data analysis from the experimenters do not indicate signs of new physics, but the re-analysis by Perivolaropoulos and Kazantzidis claims that the residual data has signatures of an oscillating signal.

From https://www.npl.washington.edu/eotwash/inverse-square-law: "Below is a cartoon of one of our initial experimental devices which illustrates the technique we employ to measure gravity at short length scales. The pendulum ring, with 10 small holes bored into it, hangs by a thin tungsten fiber (typically 20 microns thick) and acts as the detector mass. The rotating plate just below it, with 10 similar holes bored into it, acts as the drive mass providing gravitational pull on the pendulum that twists it back and forth 10 times for every revolution of the plate. The pendulum twist is measured by shining a laser beam off of a mirror mounted above the ring. The induced twist (torque) on the pendulum at varying separation distances is then compared to a detailed Newtonian calculation of the system. For many of our measurements, the rotating attractor situated just below the detector ring actually consists of 2 disks. The upper disk has holes identical to those in the detector ring. The lower, thicker attractor disk also has a similar hole pattern bored into it, but these larger holes are rotated compared to those in the upper disk, so that they lie halfway between the holes in the upper disk. If the inverse-square law is correct, the lower holes are designed such that they produce a twist on the ring that just cancels the twist induced by the upper disk. However, if gravity changes strength at short distances as the theories suggest, the twist induced by the lower disk, which is farther from the ring, will no longer cancel the twist from the upper disk and we will see a clear twist signal. The actual situation is a bit more complicated; for ordinary gravity the cancellation is exact only for one particular separation between the ring and the attractor, but the variation of the magnitude of the twist with changing separation between the ring and the disks provides a clear 'signature' for any new gravitational or other short-range phenomena."



There are 87 residual torque data points ($\delta\tau$) between measured torques and expected Newtonian values. Perivolaropoulos and Kazantzidis made fits to three functions (models):

$$\delta\tau_1(\alpha', m', r) = \alpha' \qquad \text{offset Newtonian potential}$$
$$\delta\tau_2(\alpha', m', r) = \alpha' e^{-m'r}$$
$$\delta\tau_3(\alpha', m', r) = \alpha' \cos(m'r + \phi)$$

Figures of the data and some fits from figures in the two papers are shown here.

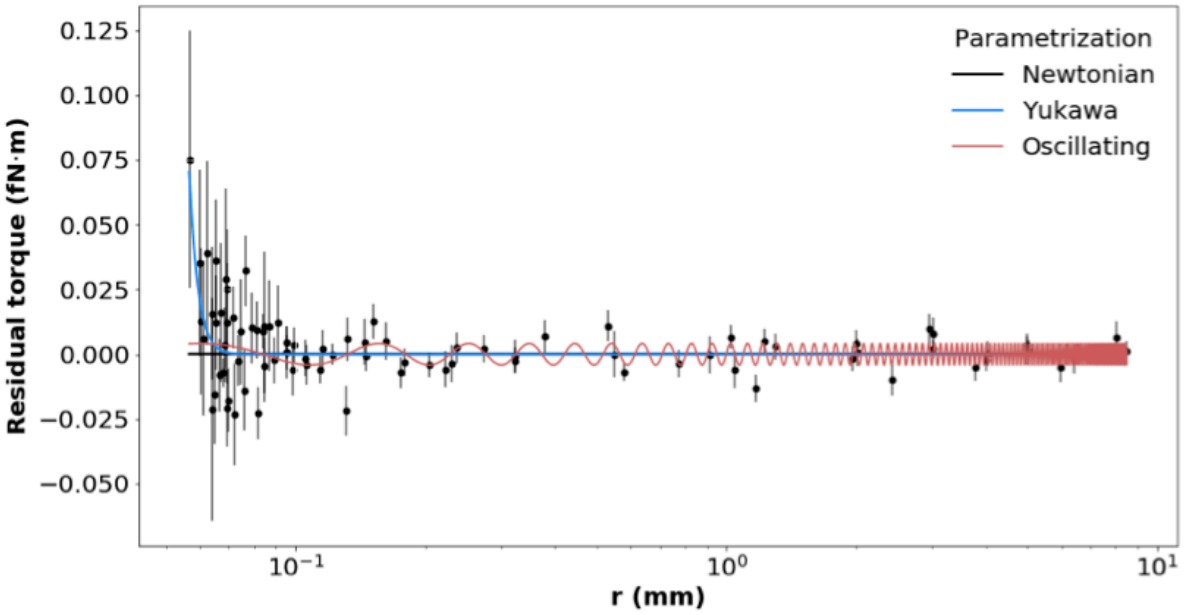


Figure 1. Plots showing the data points for the residual torque (obtained from P19) from the Eöt-Wash experiment, along with the best fits obtained for the three models using χ^2 minimization. The best-fit values for each of the three models are shown in table 1. We note that for the best-fit Yukawa model, except for the first few points, the best fit is indistinguishable by eye compared to the corresponding Newtonian fit.

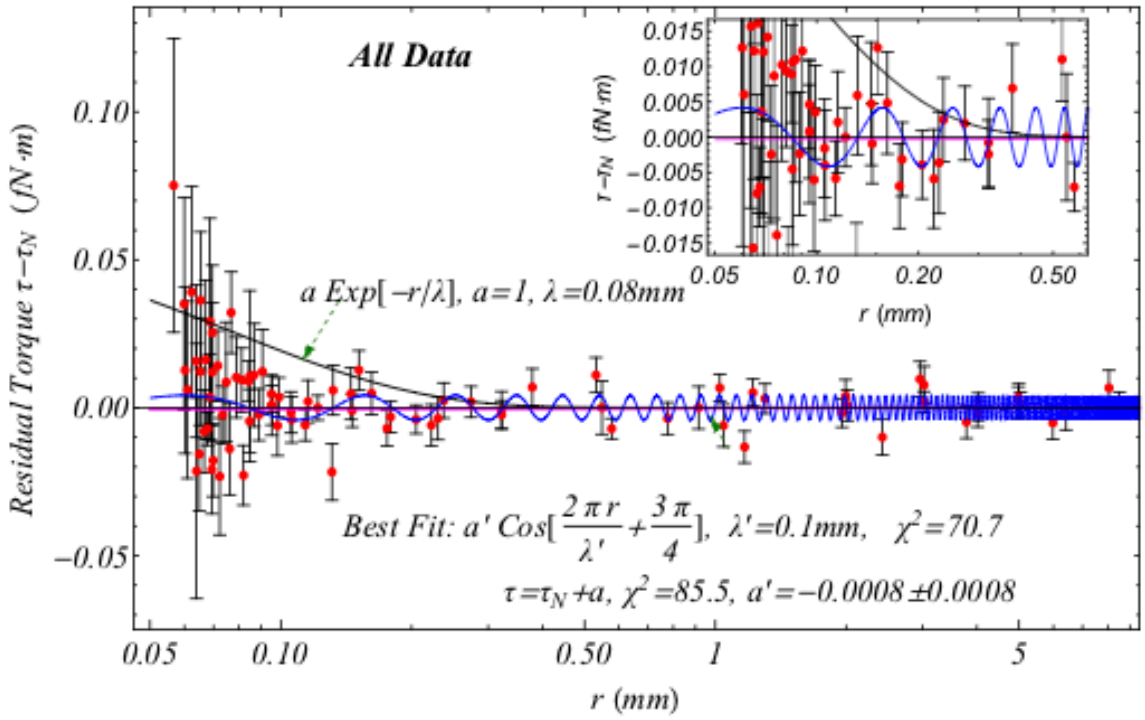


FIG. 19. The residual data (torques) that it was considered in Ref. [62]. The pink line denotes the best fit Yukawa parametrization, while the thin blue line describes the oscillating parametrization.

Next we have tables from Krishak and Desai showing the best-fit results and the adopted priors.

Parametrization	α' (fN · m)	m' (mm^{-1})	ϕ
Newtonian	$0.0001^{+0.0007}_{-0.0007}$	—	—
Yukawa	$(7.2 \pm 2.9) \times 10^7$	366.1^{+64}_{-47}	—
Oscillating (ϕ fixed)	$0.0042^{+0.0020}_{-0.0020}$	$65.29^{+0.93}_{-0.84}$	—
Oscillating (ϕ free)	$0.0042^{+0.0012}_{-0.0015}$	$65.28^{+0.20}_{-0.21}$	$(0.756^{+0.15}_{-0.15})\pi$

Table 1. The best-fit parameters obtained for the three parametrizations (eq. (2.1), eq. (2.2), eq. (2.3)) by χ^2 minimization.

Parameter	**Prior**	**Minimum**	**Maximum**
α' (eq. (2.1))	Uniform	-0.05	0.07
α' (eq. (2.2))	Uniform	0	1.0×10^8
α' (eq. (2.3))	Uniform	0	0.01
m' (Yukawa)	Uniform	0	500
m' (Oscillating)	Uniform	60	70
ϕ (Oscillating with phase free)	Uniform	0	2π

Table 2. List of priors used to evaluate the Bayesian evidence for each of the three different models. The units for α' and m' are in fN · m and mm^{-1} respectively.

Krishak and Desai applied AIC, BIC, WAIC, and calculated the Bayes factor (ratio of Bayesian evidences).

	Newtonian	Yukawa	Oscillating ($\phi = 3\pi/4$)	Oscillating (ϕ free)
$\mathrm{AIC_c}$ values	-571.3	-572.6	-584.0	-582.1
$\Delta\mathrm{AIC_c} = \mathrm{AIC_c(Newt.)} - \mathrm{AIC_c}$	—	1.2	12.7	10.5
BIC values	-568.9	-567.6	-579.2	-574.7
$\Delta\mathrm{BIC} = \mathrm{BIC(Newt.)} - \mathrm{BIC}$	—	-1.2	10.3	5.9
WAIC values	-571.4	-574.3	-583.6	-579.4
$\Delta\mathrm{WAIC} = \mathrm{WAIC(Newt.)} - \mathrm{WAIC}$	—	2.9	12.2	8.0
Bayes Factor	—	29	544	83

Table 3. Summary of results for the model comparison tests using Bayesian and information theoretic techniques. The Bayes factor (in this table) is the ratio of Bayesian evidence for the Yukawa or oscillating model to the same for Newtonian model. $\Delta\mathrm{AIC_c}$ (also BIC and WAIC) indicates the difference in $\mathrm{AIC_c}$/BIC between the Newtonian model and the Yukawa/oscillating model. When the phase (ϕ) is fixed at $3\pi/4$ similar to P19, all the tests used here, (viz. $\mathrm{AIC_c}$, BIC, WAIC, and Bayes factor) decisively favor the oscillating model compared to the Newtonian model. When ϕ is a free parameter, only $\mathrm{AIC_c}$ shows decisive evidence for the oscillating model; whereas BIC, WAIC, and Bayes factor now only show strong evidence for the same.

Conclusions were based on interpretive scales found in the literature. For example:

ΔBIC	Evidence against Model i
0 − 2	Not Worth More Than A Bare Mention
2 − 6	Positive
6 − 10	Strong
> 10	Very Strong

For interpreting the Bayes' factor the Jeffrey's scale is used: a value above 10 represents *strong* evidence in favor of the model in the numerator while a value above 100 represents *decisive* evidence. Krishak and Desai conclude that there is decisive support that the oscillating model with fixed phase (from the Perivolaropoulos and Kazantzidis fit) and strong evidence for varying phase. However, this statistical analysis does not distinguish between a physics origin and statistical effects in the data. Note: Krishak and Desai use nested sampling software to evaluate the evidence.

15. Discrepancy models with GPs

In Section 4.5 we introduced the mismatch term δM, typically referred to as a model discrepancy, in the statistical model

$$\mathcal{D} = M(\boldsymbol{\theta}) + \delta\mathcal{D} + \delta M. \tag{15.1}$$

In this chapter, we describe and illustrate an approach to model discrepancy that uses Gaussian processes (see Section 32) to model δM.

Most of our discussion of discrepancy models is based on the seminal papers of Kennedy and O'Hagan and of Brynjarsdóttir and O'Hagan. In this chapter we'll refer to these works using the abbreviations KOH and BOH:

- KOH = Kennedy and O'Hagan, *Bayesian calibration of computer models*
- BOH = Brynjarsdóttir and O'Hagan, *Learning about physical parameters: the importance of model discrepancy*. This content is particularly important if you are trying to extract the value of physical parameters from modeling data.

15.1. Introduction to discrepancy models

Comparisons between theoretical models and experimental data are at the heart of scientific inquiry. Theoretical models guide our understanding of complex systems by translating hypotheses into quantitative predictions that can be tested experimentally. Traditionally, a close fit between a model's predictions and measured data is interpreted as a sign of success, often implying that the model parameters capture the underlying physical processes. However, this paradigm assumes that the model fully represents the complexity of actual systems – an assumption that is rarely justified in practice.

All models have inherent limitations beyond their domains of validity, and using them beyond these regimes without accounting for theoretical uncertainties can lead to biased parameter estimates, reducing these parameters to mere "fitting variables" rather than meaningful physical quantities,. Moreover, discrepancies between certain

measurements and otherwise successful models sometimes lead researchers to assign lower weights to these data, thereby diminishing their utility and limiting the potential insights they can provide.

A model discrepancy framework employing Gaussian processes (or GPs, see Overview of Gaussian processes) was introduced in KOH and variations have been explored in many subsequent studies. In these approaches, the discrepancy between experimental data and theoretical model predictions, stemming from missing physics or approximations, is modeled using a GP. However, one persistent challenge is the need to constrain the GP's covariance kernel. For example, in BOH, the authors emphasized the importance of incorporating knowledge of the theory's validity at specific points in the input space (i.e., the domain in which observables are measured) so that both the GP and its derivative could be accurately constrained. In practice, however, specifying such accurate knowledge about the theory is often difficult.

In this chapter, we construct the GP covariance kernel based on only qualitative prior knowledge of the theory's domain of validity across the input space. This type of knowledge – for example, recognizing that "*the theory is more reliable in this regime than in that one*" – is typically easier to provide and often available. By leveraging this information, the framework prioritizes the accurate extraction of model parameters rather than simply optimizing the fit to the observables. We perform Bayesian parameter inference to simultaneously estimate both the model parameters and the GP hyperparameters, thereby quantifying uncertainties from both the experimental data and the theoretical model.

15.2. The KOH and BOH frameworks

We begin by setting up the model discrepancy framework following the Kennedy and O'Hagan framework. Let the i-th measurement for an observable y be denoted as

$$y(x_i) \sim \zeta(x_i) + \epsilon_i\,, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma_i^2)\,. \tag{15.2}$$

Here, the index i corresponds to the point in input space x where the i-th observation is made (e.g., a specific time in the ball drop experiment described below). Independent observation errors ϵ_i are assumed to follow Gaussian distributions with zero mean and standard deviation σ_i. Here, $\zeta(x_i)$ represents the true value of the observable at x_i. (We assume that x is made unitless by scaling it with a suitable reference scale x_0.)

We denote the prediction from a theoretical model for the observable y as $\eta(x, \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ is the vector of true but unknown model parameters. The model for $\eta(x, \boldsymbol{\theta})$ is then defined as:

$$\zeta(x) = \eta(x, \boldsymbol{\theta}) + \delta(x); \tag{15.3}$$

here $\delta(x)$ quantifies the discrepancy between the theoretical model prediction and the true system value at input setting x. Combining the above equations, we express the observation $y(x_i)$ as

$$y(x_i) \sim \eta(x_i, \boldsymbol{\theta}) + \delta(x_i) + \epsilon_i. \tag{15.4}$$

Thus, each observation is modeled as the sum of the model output (evaluated at the true $\boldsymbol{\theta}$), the model discrepancy δ at x_i, and the observational error ϵ_i. The statiscal model is summarized in Fig. 15.1.

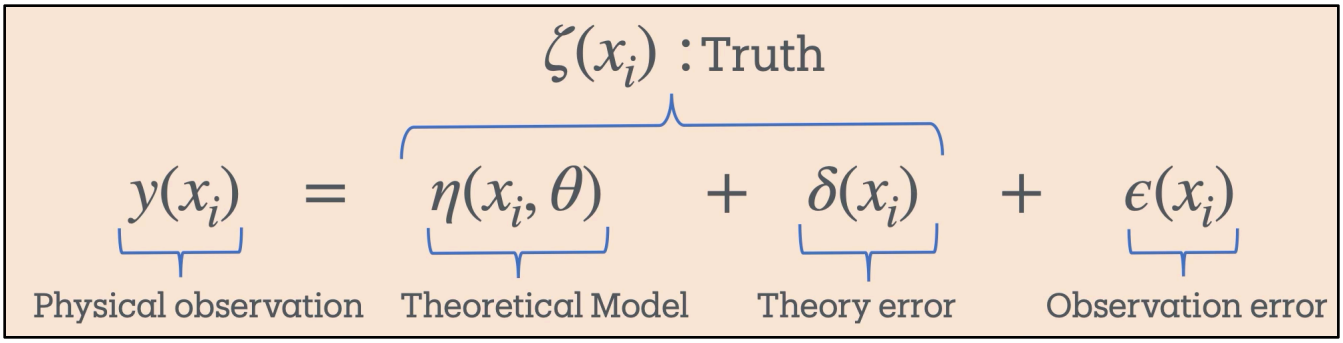


Fig. 15.1 Statistical model.

Following KOH, we represent $\delta(x)$ as a zero-mean Gaussian process (GP):

$$\delta(\cdot \mid \boldsymbol{\phi}) \sim \mathrm{GP}\left(\mathbf{0}, K(\cdot, \cdot \mid \boldsymbol{\phi})\right),$$

where $K(\cdot, \cdot \mid \boldsymbol{\phi})$ is the covariance kernel, which depends on a set of hyperparameters $\boldsymbol{\phi}$. A GP specifies a distribution over functions, with the covariance kernel K serving as the central object that defines the properties of the functions in the distribution. In this framework, we incorporate prior knowledge about the theoretical model's uncertainties through the choice of the covariance kernel. Adopting a Bayesian approach, we assign prior probability distributions to both the model parameters $\boldsymbol{\theta}$ and the GP hyperparameters $\boldsymbol{\phi}$, and update these to obtain their posterior distributions conditioned on the observations. In all examples discussed in this work, uniform priors (in appropriate ranges) are used for both the model parameters and the GP hyperparameters.

The particular K we use for the test case explored in the next section (the "ball-drop model") is motivated by the observations in BOH, in particular that to extract physical parameters it is necessary to adequately inform the model discrepancy GP. Here we use:

$$K(x_i, x_j) = \bar{c}^2 (x_i x_j)^r \exp\left(-\frac{\|x_i - x_j\|^2}{2\ell^2}\right) \tag{15.5}$$

The hyperparameters control the nature of the possible random draws, as illustrated in Fig. 15.2. The parameter $\bar{c}$, which is fixed at $\bar{c} = 1$ in the figure, controls the overall scale over which the random functions vary. The parameter ℓ is the correlation length between any two points; as $|x - x'|$ becomes greater than ℓ, the points become increasingly uncorrelated. Comparing $\ell = 1$ to $\ell = 3$ shows that the latter curves have a longer "wavelength". Finally, r is a parameter introduced to generate functions that increase with x for $r > 0$. It is the addition of the term with r that importantly informs the GP that the model is more reliable at small x than at large x.

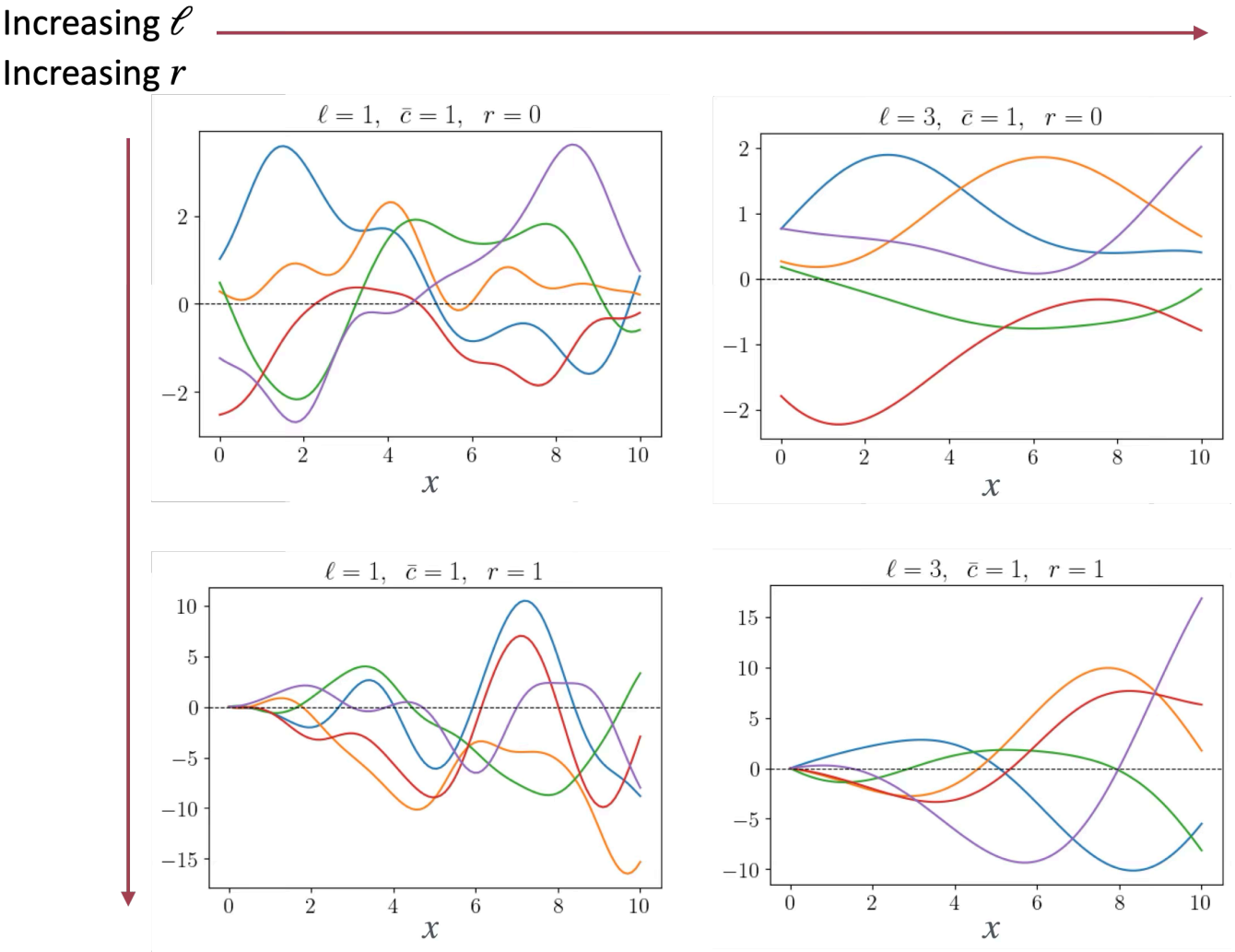


Fig. 15.2 Five draws from each of four GPs of the form in Eq. (15.5) with hyperparameters ℓ, $\bar{c}$, and r as listed. Note that with $r > 0$, the variance grows with x.

Having set up our ingredients (which will also apply to the real-world application example in Example: Discrepancy models for relativistic heavy-ion collisions), we now turn to an illustrative toy model: "The Ball-Drop Experiment".

Note

If sampled at any one input point x, the prior GP will yield a Gaussian distribution with mean and variance specified by the GP definition. In many cases, the prior GP is taken to be mean zero with a constant variance for all x. If sampled at any two input points x and x', the joint distribution will be a bivariate Gaussian, with covariance given by $K(x, x'|\phi)$.

15.3. The ball-drop model

Overview

We use the canonical “Ball Drop Experiment” (popularized by statistician Dave Higdon) to show how the BOH approach works for robust extraction of physical quantities from a calibration. The story is that a ball is dropped from a tower of height h_0 at initial time $t_0 = 0$ with initial velocity $v_0 = 0$. The height from the ground is recorded at discrete time points. The true data is generated based on a model with drag, and includes uncorrelated Gaussian measurement errors. The aim is to extract the value of acceleration due to gravity, g.

However, we seek a value for g using a theoretical model without air resistance. (In the demo notebook in Section 15.4 we consider only measurements of the height of the ball; a more general version with results shown below also has measurements of velocity and acceleration.) So the problem is that the data is noisy *and* our theoretical model (free fall without drag) is incorrect. How can we best formulate our analysis such that the extraction of g (and other observables) is robust? I.e., that we get a distribution for g that covers the true answer with plausible uncertainty.

Statistical formulation

We will follow the article by Higdon et.al. The statistical equation accounting for model discrepancy is:

$$y(x_i) = \eta(x_i;\boldsymbol{\theta}) + \delta(x_i;\boldsymbol{\phi}) + \epsilon(x_i;\boldsymbol{\sigma_\epsilon}), \quad i = 1,\ldots,n \tag{1}$$

where x is the input vector (time in the case of the ball drop experiment) of length n.

- We shall denote the input domain as $\{x_1,\cdots,x_n\} \in \chi_*$.
- The above equation should be understood as an equation which holds on the range (set of all possible outcomes) of the probability distributions. It means that a sample drawn at a given x_i from the probability distributions $\mathrm{pr}(y)$ should be equal to the sum of the sample drawn from each of the distributions $\mathrm{pr}(\eta|\boldsymbol{\theta})$, $\mathrm{pr}(\delta|\boldsymbol{\phi})$ and $\mathrm{pr}(\epsilon|\boldsymbol{\sigma_\epsilon})$.
- Note that for given x_i, $\mathrm{pr}(\eta(x_i)|\boldsymbol{\theta}) = \delta(\eta(x_i) - \eta(x_i;\boldsymbol{\theta}))$ is deterministic since the model is deterministic given $\boldsymbol{\theta}$ (note: δ is the Dirac delta function).

In Eq. (1), $y(x_i)$ are vectors of the measured observables (velocity and height) at each input point x_i.

- The error in the measurement of $y(x_i)$ is represented by $\epsilon(x_i;\boldsymbol{\sigma_\epsilon})$, which is assumed to be independently and identically distributed $\sim N(0,{\boldsymbol{\sigma_\epsilon}}^2)$ for each type of observable (velocity and height: $\{\sigma_v,\sigma_h\} \in \boldsymbol{\sigma_\epsilon}$).
- Note that $\{\sigma_v,\sigma_h\}$ are fixed constants since we have assumed the distribution for each type of observable to be iid's. We shall treat them as information `I' whenever we write any probability distribution $\mathrm{pr}(\bullet|\bullet,I)$, and will not state them explicitly.
- $\eta(x;\boldsymbol{\theta})$ denotes the observable predictions of the physics theory given the value of the parameters of the theory $\{g,\sigma_{v_0}\} \in \boldsymbol{\theta}$.
- The stochastic model $\mathrm{pr}(\delta(x)|\boldsymbol{\phi}) = GP[0,\kappa(x,x';\boldsymbol{\phi})]$, accounts for model discrepancy (discrepancy between the physics model and reality) where κ is covariance kernel of the Gaussian Process (GP) with $\{\bar{c}_v,\, l_v,\, \bar{c}_h,\, l_h\} \in \boldsymbol{\phi}$ representing the hyperparameters.
- We define $\{\boldsymbol{\theta},\boldsymbol{\phi}\} \in \boldsymbol{\alpha}$ as a vector containing $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$ for compact notation.

One first sees that a discrepancy model is essential; using a radial basis function (RBF) GP is standard (see Wikipedia RBF article) and one finds good predictions for times where there is data (this is KOH). But the

discrepancy model must be *more informed* for a robust extraction of g; this leads to using a nonstationary GP (in the spirit of BOH, as shown in Section 15.2.

Our simple test system has the ball dropped from a tower of height $h_0 = 60\,\text{m}$ at time $t = 0\,\text{s}$. Measurements of the ball's height above the ground, velocity, and acceleration are recorded at discrete time points until $t = 1$,s. (In this example $x = t$/(1,s) serves as the unitless input parameter, $0 \le x \le 1$.) The true theory, used to generate synthetic experimental data, incorporates a gravitational force $\mathbf{f}_G = -Mg\,\hat{\mathbf{z}}$ and a drag force given by $\mathbf{f}_D = -0.4v^2\,\hat{\mathbf{v}}$. Here, $M = 1\,\text{kg}$ is the mass of the ball, $g = 9.8\,\text{m/s}^2$ the acceleration due to gravity, and we set the initial velocity v_0 to $v_0 = 0\,\text{m/s}$.

We assume that the observational errors for height, velocity, and acceleration are independent and identically distributed Gaussian random variables (imagine they are measured with different instruments, so the data errors are uncorrelated).

Synthetic experimental data are generated by sampling from Gaussian distributions with specified standard deviations and with means given by the predictions of the true theory, which are determined by the equations of motion:

$$a = \frac{dv}{dt} = g - 0.4v|v|\,, \qquad \frac{dh}{dt} = -v. \tag{15.6}$$

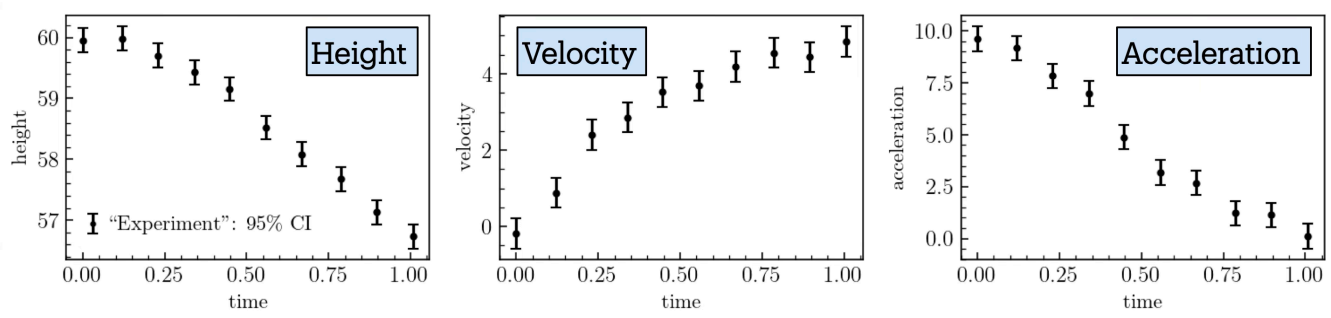


Fig. 15.3 Data for the ball drop experiment, including measurement errors.

We compare these synthetic data with a theoretical model that neglects the drag force and is governed by the simpler equations of motion:

$$v = v_0 + gt\,, \qquad h = h_0 - v_0 t - \frac{1}{2}gt^2.$$

Here $h_0 = 60\,\text{m}$ is fixed to the value used in generating the synthetic experimental data. The parameters g and v_0 are model parameters, which we infer using Bayesian parameter inference (in the demo notebook, for simplicity, only g is inferred). We incorporate our prior knowledge about the theory's domain using the two distinct GP kernels presented in Fig. 15.4. Since our model neglects drag forces, we are more confident in its predictions at earlier times (small x), when the velocity is lower, than at later times (larger x).

➡ Kernel I: $K(x_i, x_j \mid \phi) \equiv s^2 + \bar{c}^2 (x_i x_j)^r \exp\left(-\frac{\|x_i - x_j\|^2}{2\ell^2}\right)$. Encodes the belief: *model is more reliable at small x than at large x.*

➡ Kernel II: $K(x_i, x_j \mid \phi) \equiv \bar{c}^2 (x_i x_j)^r \exp\left(-\frac{\|x_i - x_j\|^2}{2\ell^2}\right)$. Encodes the belief: *model is correct at $x = 0$, but our confidence in its validity decreases as x increases*

Fig. 15.4 GP kernels `I` and `II` for the balldrop analysis.

In the following figures, we show results with successively more observables included. Fig. 15.5 has only height, Fig. 15.6 has height and velocity, and Fig. 15.7 has height, velocity and acceleration.

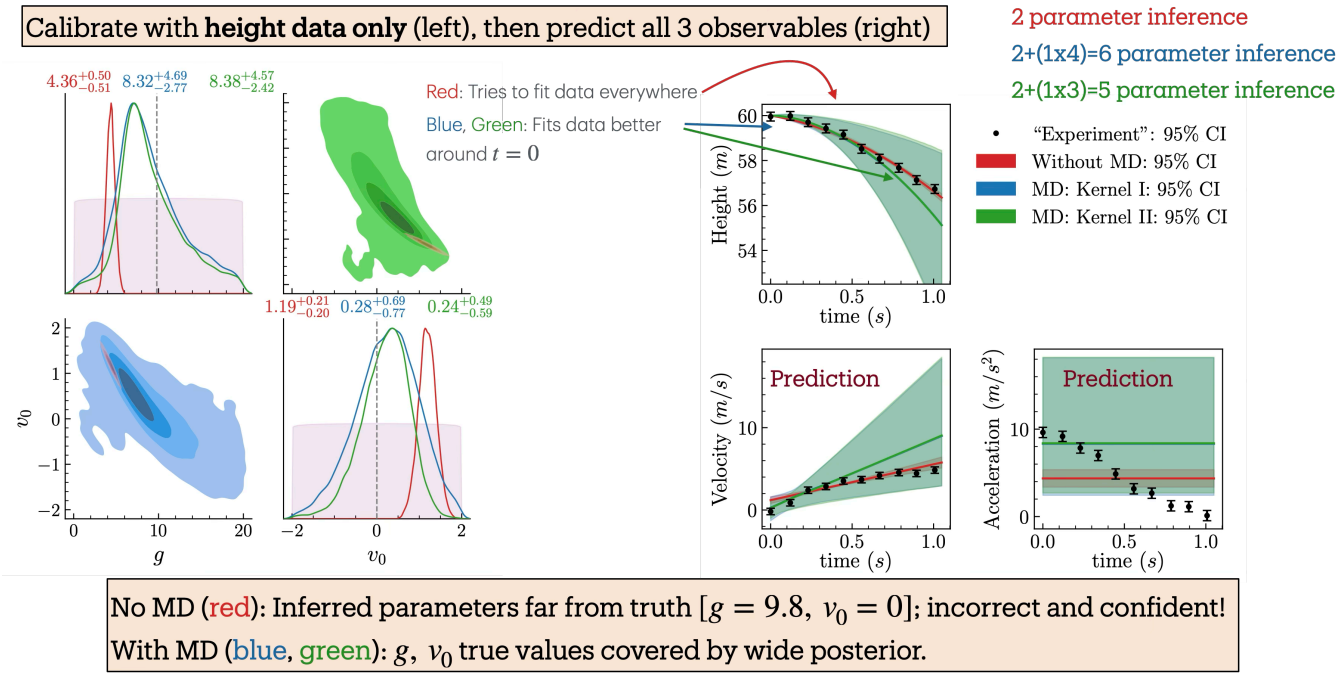


Fig. 15.5 Balldrop analysis with only the height measurements.

Fig. 15.5 shows both the inferred posteriors for the model parameters and the corresponding model predictions when only the height data is used. We perform Bayesian parameter inference under three scenarios: (i) without the model discrepancy (MD) term (red), (ii) with MD using GP-kernel `I` (blue), and (iii) with MD using GP-kernel `II` (green). The kernels are defined in Fig. 15.4; their hyperparameters are learned at the same time as the model parameters. Parameter inference is carried out by sequentially incorporating additional observables to examine their influence on the inferred posteriors. The posteriors and predictions for when height and velocity are included are shown in Fig. 15.6 and when height, velocity, and acceleration included are shown in Fig. 15.7.

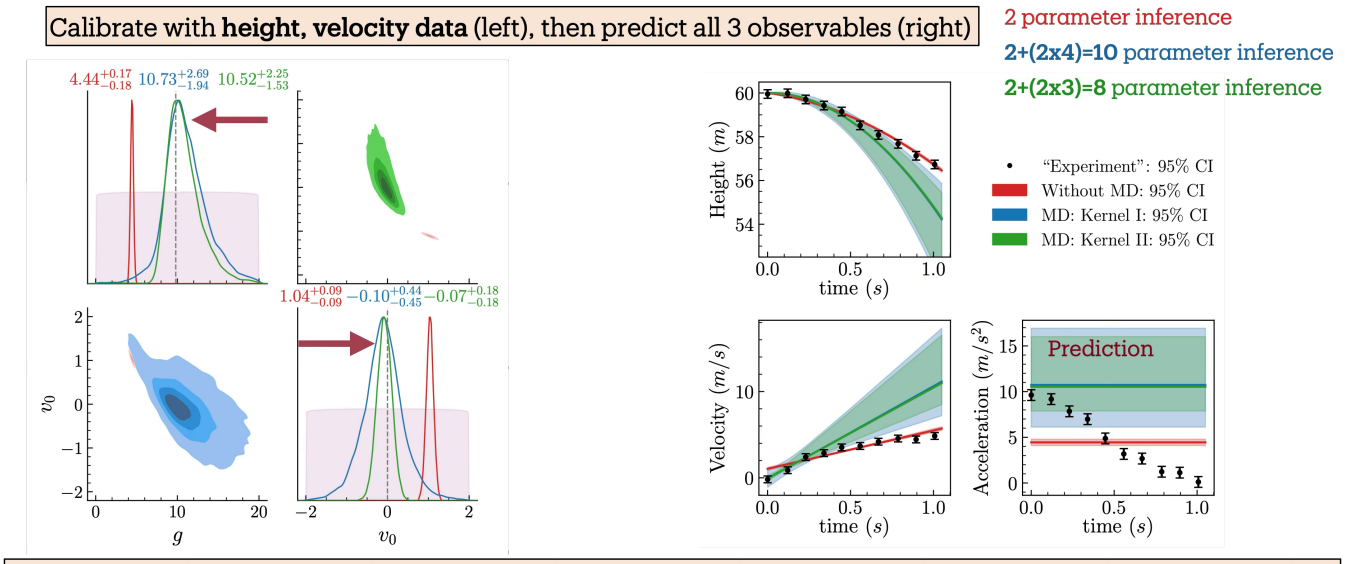


Fig. 15.6 Balldrop analysis with both the height and velocity measurements.

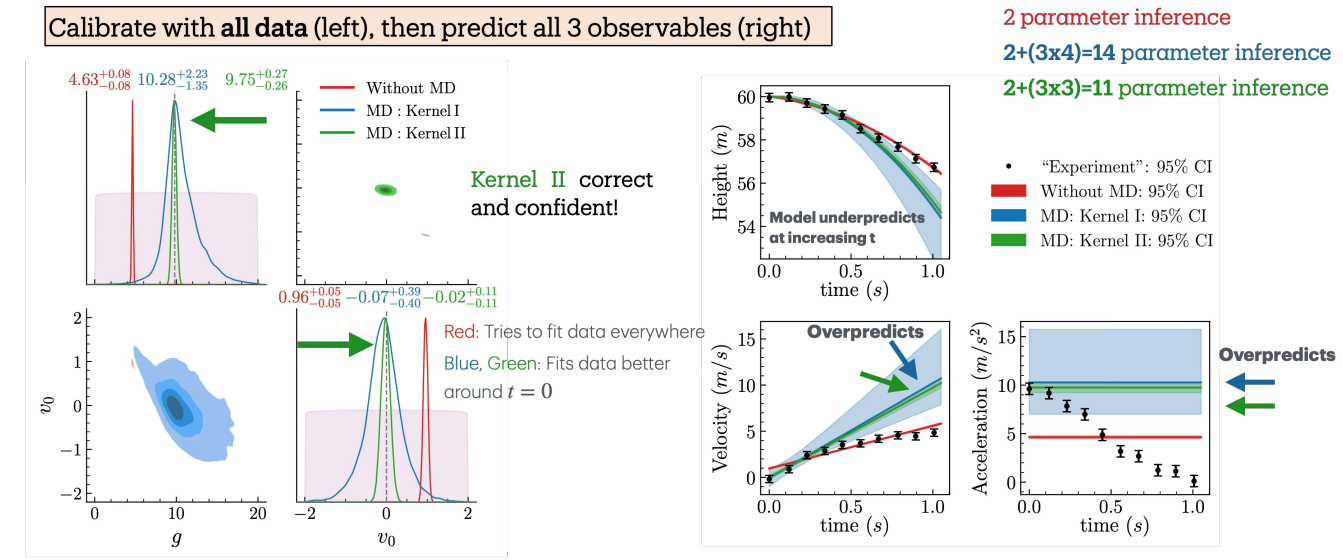


Fig. 15.7 Balldrop analysis with height, velocity, and acceleration measurements.

The posteriors incorporating MD (blue and green) consistently cover the true values and become narrower as more observables are included. In particular, the posteriors with MD using Kernel `II` (green), which encodes strong prior information about the theory's domain of validity, are consistently tighter, reflecting greater constraining power, yet remain consistent with those obtained using the more conservative Kernel `I` (blue). Remarkably, when acceleration data are included (Fig. 15.7), the posterior with Kernel `II` nearly recovers the true parameter values, even though the model predicts a constant acceleration that does not capture the observed decreasing trend in the data. In contrast, the posteriors from the inference without the MD term (red) deviate significantly from the truth and become increasingly narrow as more observables are added, resulting in model parameter estimates that are both incorrect *and* overconfident.

In the right panels of the figures, the synthetic experimental data are compared with the model predictions generated using the inferred parameter posteriors from the left panels. Notably, predictions with MD (blue, green) match the data at early times and deviate at later times, a behavior that directly results from incorporating prior knowledge about the theory's domain of applicability into the covariance kernel. In contrast, predictions without MD (red) prioritize achieving the best fit to the data, leading to inaccurate and overconfident parameter estimates.

15.4. 📥 Demo: Ball-drop experiment

This notebook has an implementation of the discrepancy model for the ball-drop experiment from the last section, simplified so that the height is the only observable that is measured.

This notebook was created by Dr. Sunil Jaiswal in July, 2025.

Theory for the truth

Consider the drag force on the ball (sphere) for the true model to behave as

$$\mathbf{f}_D = -(bv + cv^2)\,\hat{\mathbf{v}}\,, \tag{2}$$

where v is magnitude of velocity $\mathbf{v}$. The coefficients b and c are drag coefficients.

● Equation of motion:

$$m\frac{d\mathbf{v}}{dt} = m\mathbf{g} - b\mathbf{v} - cv^2\hat{\mathbf{v}}\,.$$

We have considerd the downward direction to be positive. Here $m = 1\,kg$ is the mass of the sphere and $g = 9.8\,m/s^2$ the acceleration due to gravity.

● The equation for velocity can be written as $m\frac{dv}{dt} = mg - bv - cv\sqrt{v^2}$. The last term is written such that it takes into account the direction of velocity in quadratic term. We need to solve the equations:

$$\frac{dv}{dt} = g - \frac{b}{m}v - \frac{c}{m}v|v|\,, \qquad \frac{dh}{dt} = -v\,.$$

By default we consider the ball to have the initial height $h_0 = 60\,m$ at initial time $t_0 = 0\,sec$ with initial velocity $v_0 = 0\,m/s^2$.

Setup sampling with emcee (and, if loaded, pocomc) samplers

```
import matplotlib.pyplot as plt
import emcee
import corner

#import pocomc   # uncomment here when using pocomc sampler
from multiprocess import Pool
from scipy.stats import uniform
import time
import os

# ================================ emcee sampler ==================================
def emcee_sampling(min_param, max_param, log_posterior,
                   nburn=2000, nsteps=2000, num_walker_per_dim=8, progress=False):
    """
    Function for sampling in parallel using emcee.

    Parameters:
      - min_param (list or array): minimum values parameters
      - max_param (list or array): maximum values parameters
      - log_posterior (callable): log posterior
      - nburn (int): "burn-in" period to let chains stabilize
      - nsteps (int): # number of MCMC steps to take after burn-in
      - num_walker_per_dim (int): number of walker per dimension
      - samples_save_dir (string): directory name in which the samples will be saved
      - progress (boolean): toggle the progress bar
    """
    if len(min_param) != len(max_param):
            raise ValueError(
                f"min_param and max_param must have the same length."
            )

    ndim = len(min_param)  # number of parameters in the model
    nwalkers = ndim * num_walker_per_dim  # number of MCMC walkers

    start_time = time.time()
    print(f"Starting MCMC sampling using emcee with {nwalkers} walkers in parallel...")

    # Generate starting guesses within the prior bounds
    starting_guesses =  min_param + (max_param - min_param) * np.random.rand(nwalkers,ndim)

    # Create a pool of workers using multiprocessing
    with Pool() as pool:
        # Pass the pool to the sampler
        sampler = emcee.EnsembleSampler(nwalkers,ndim,log_posterior,pool=pool)

        # "Burn-in" period
        pos, prob, state = sampler.run_mcmc(starting_guesses, nburn, progress=progress)
        sampler.reset()

        # Sampling period
        pos, prob, state = sampler.run_mcmc(pos, nsteps, progress=progress)

    # Collect samples
    samples = sampler.get_chain(flat=True)

    end_time = time.time()
    print(f"Sampling complete. Time taken: {(end_time - start_time)/60:.2f} minutes.")
    print(f"Mean acceptance fraction: {np.mean(sampler.acceptance_fraction):.3f} (in total
{nwalkers*nsteps} steps)")
    print(f"Samples shape: {samples.shape}")
```

```
    return samples


# ================================ pocoMC sampler ==================================
def pocomc_sampling(min_param, max_param, log_posterior,
                    n_effective=2000, n_active=800, n_steps=None,
                    # n_prior=2048, n_max_steps=200,
                    n_total=5000, n_evidence=5000, ncores = -1
                   ):
    """
    This function is based on PocoMC package (version 1.2.1).
    pocoMC is a Preconditioned Monte Carlo (PMC) sampler that uses
    normalizing flows to precondition the target distribution.

    Parameters:
      - min_param (list or array): Minimum values parameters
      - max_param (list or array): Maximum values parameters
      - log_posterior (callable): Log posterior

      - n_effective (int): The effective sample size maintained during the run. Default: 2000.
      - n_active (int): Number of active particles. Default: 800. Must be < n_effective.
      - n_steps (int): Number of MCMC steps after logP plateau. Default: None -> n_steps=n_dim.
                       Higher values lead to better exploration but increases computational
cost.
      - n_total (int): Total effectively independent samples to be collected. Default: 5000.
      - n_evidence (int): Number of importance samples used to estimate evidence. Default:
5000.
                          If 0, the evidence is not estimated using importance sampling.

      - ncores: Number of cores to use in sampling.
                  -1 (default): use all availaible cores
                   n (int): use n cores
    """

    # Generating prior range for pocomc
    priors = [uniform(lower, upper - lower) for lower, upper in zip(min_param, max_param)]
    prior = pocomc.Prior(priors)

    ndim = len(min_param)  # number of parameters in the model

    # ------------------------------------------------------------------------------------------
---
    # Try to get the allowed CPU count via affinity (works on Linux/Windows)
    try:
        available_cores = len(os.sched_getaffinity(0))# len(psutil.Process().cpu_affinity())
    except Exception:
        # On systems where affinity is not available (e.g., macOS), use total CPU count
        available_cores = os.cpu_count()

    # Validate that ncores is an integer and at least -1
    if not isinstance(ncores, int) or ncores < -1:
        raise ValueError("ncores must be an integer greater than or equal to -1. Specify '-1'
to use all available cores.")

    if ncores == -1:
        # Use all available cores
        num_cores = available_cores
    elif ncores > available_cores:
        # Warn if more cores are requested than available and then use available cores
        warnings.warn(
            f"ncores ({ncores}) exceeds available cores ({available_cores}). Using available
cores.",
            UserWarning
        )
        num_cores = available_cores
```

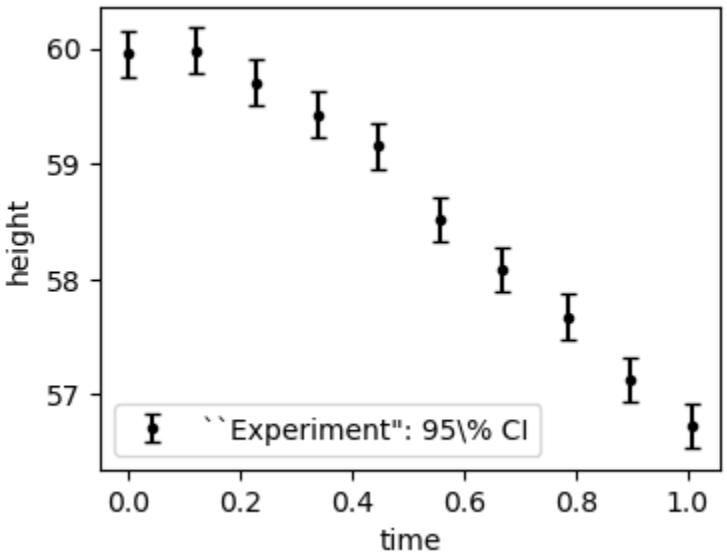
```
    else:
        num_cores = ncores
    # ------------------------------------------------------------------------------------
---

    start_time = time.time()

    sampler = pocomc.Sampler(prior=prior, likelihood=log_posterior,
                                n_effective=n_effective, n_active=n_active, n_steps=n_steps,
                                # dynamic=True, n_prior=n_prior, n_max_steps=n_max_steps,
                                pytorch_threads=None,  # use all availaible threads for
training
                                pool=num_cores
                                )

    print(f"Starting sampling using pocomc in {num_cores} cores...")

    sampler.run(n_total=n_total,
                n_evidence=n_evidence,
                progress=True)


    # Generating the posterior samples
    # samples, weights, logl, logp = sampler.posterior() # Weighted posterior samples
    samples, _, _ = sampler.posterior(resample=True)  # equal weights for samples

    # Generating the evidence
    # logz, logz_err = sampler.evidence() # Bayesian model evidence estimate and uncertainty
    # print("Log evidence: ", logz)
    # print("Log evidence error: ", logz_err)

    # logging.info('Writing pocoMC chains to file...')
    # chain_data = {'chain': samples, 'weights': weights, 'logl': logl,
    #                 'logp': logp, 'logz': logz, 'logz_err': logz_err}

    end_time = time.time()
    print(f"Sampling complete. Time taken: {(end_time - start_time)/60:.2f} minutes.")
    print(f"Samples shape: {samples.shape}")

    return samples
```

Generate “experimental” observations for height

```
import numpy as np
from scipy.integrate import odeint

def True_height(t, h0=60.0, v0=0.0,
                g=9.8, m=1.0, b=0.0, c=0.4):    # initial conditions
    """
    Height of a falling ball with linear + quadratic air drag.

    Parameters
    ----------
    t : float or 1-D array_like
        Time(s) at which to evaluate the height.
    h0, v0 : float, optional
        Initial height and velocity at time t = 0.
    g : float, optional
        Acceleration due to gravity (m s⁻²) (taken positive downward).
    m : float, optional
        Mass of the ball (kg).
    b, c : float, optional
        Linear and quadratic drag coefficients.

    Returns
    -------
    h : 1darray
        Height(s) at the requested time points (same shape as `t`).
    """

    def derivatives(y, _t):
        h, v = y
        dhdt = -v
        dvdt = g - (b*v)/m - (c*abs(v)*v)/m
        return [dhdt, dvdt]

    # Ensure we integrate from t0 out to the requested time(s)
    t = np.atleast_1d(t)
    times = np.concatenate(([t[0]], t))  # add t[0] to get solution at t=0
    sol = odeint(derivatives, [h0, v0], times)

    return sol[1:, 0]
```

```
np.random.seed(42)  # Set a random seed for reproducibility. Change 42 -> 0 for different seeds
every time.

# Generate an array of time where height are observed -->
t_obs = np.linspace(0.0, 1.0, 10)
t_noise = np.random.uniform(1e-3, 1e-2, size = t_obs.shape)  # random noise added to time.
helps in inversion of covariance matrix.
t_noise[0] = 0.0  # No noise for initial time
t_obs = np.round(t_obs + t_noise, decimals=4)


# Compute height of ball at these time steps -->
hexp_std = 0.1 # standard deviation of assumed gaussian noise as error of measurment
hexp_err = np.array([hexp_std]*len(t_obs))  # adding gaussian noise to observation of height
hexp_mean = np.random.normal(True_height(t_obs), hexp_err)    # observed height at input times
```

```
plt.figure(figsize=(4, 3))
plt.errorbar(t_obs, hexp_mean, yerr=[1.96*hexp_err, 1.96*hexp_err],
             fmt='o', markersize=3, capsize=3, color='black',
             label=f'``Experiment": 95\% CI')
plt.ylabel('height')
plt.xlabel('time')
plt.legend()

plt.show()
```



Define physics model

```
def model_height(t, g, v0=0.0, h0=60.0):
    return np.atleast_1d(h0  - v0 * t - g * t**2 / 2.0 )

# def model_height(t, g, v0, h0=60.0):
#     f = True_height(t, h0=60.0, v0=v0, g=g, m=1.0, b=0.0, c=0.0)
#     return np.atleast_1d(f)
# model_height(t=t_obs, g = 9.8)
```

Define model discrepancy class

```
from sklearn.preprocessing import StandardScaler
from scipy.spatial.distance import mahalanobis

class ModelDiscrepancy():
    def __init__(self, t_obs, hexp_mean, hexp_err, model_height, num_model_param, log_priors,
MD = False, kernel_func = None):

        self.t_obs = t_obs
        self.hexp_mean = hexp_mean
        self.hexp_err = hexp_err
        self.model_height = model_height
        self.num_model_param = num_model_param
        self.log_priors = log_priors
        self.MD = MD
        self.kernel_func = kernel_func

        if self.MD:
            self.log_priors_model = self.log_priors[:self.num_model_param]
            self.log_priors_MDGP = self.log_priors[self.num_model_param:]
        else:
            self.log_priors_model = self.log_priors

        # standardize experimental data  ----->
        self.scaler = StandardScaler()
        self.hexp_mean_scaled = self.scaler.fit_transform(self.hexp_mean.reshape(-1,
1)).ravel()

        self.offset = float(self.scaler.mean_[0])
        self.scale = float(self.scaler.scale_[0])

        self.hexp_err_scaled = self.hexp_err / self.scale
    #=========================================================================

    def log_likelihood(self, theta, phi=[]):
        """
        Calculate the log-likelihood of a dataset given the mean and covariance matrix.

        Parameters:
            theta: 1D NumPy array containing model parameters.
            phi: 1D NumPy array containing GP kernel hyperparameters.

        Returns:
            Log-likelihood of the dataset.
        """

        if self.num_model_param == 1:
            g = theta[0]
            h = model_height(t = self.t_obs, g = g)
        elif self.num_model_param == 2:
            g, v0 = theta[0], theta[1]
            h = self.model_height(t = self.t_obs, g = g, v0 = v0)

        hmodel_scaled = (h - self.offset) / self.scale

        cov_exp = np.diag(self.hexp_err_scaled**2)

        if self.MD == False:
            cov_total = cov_exp
        else:
            X = self.t_obs.reshape(-1, 1)
            cov_total = self.kernel_func(X, X, *phi) + cov_exp
```

```
        k = len(self.t_obs)  # number of observables

        md2 = mahalanobis(self.hexp_mean_scaled, hmodel_scaled, np.linalg.inv(cov_total))**2

        # calculate the log-likelihood
        log_likelihood_value = -0.5 * (md2 + np.log(np.linalg.det(cov_total)) + k * np.log(2.0
* np.pi) )

        return log_likelihood_value
    #=========================================================================

    def log_posterior(self, theta_and_phi):
        """
        Calculate the log posterior = sum of log priors + log likelihood.

        Parameters
        ----------
        theta_and_phi : np.ndarray (1D)
            Combined array of model params (theta) and discrepancy params (phi) if MD=True.

        Returns
        -------
        float
            The log posterior value.
        """
        theta_and_phi = np.atleast_1d(theta_and_phi)

        if self.MD:
            theta = theta_and_phi[:self.num_model_param]
            phi = theta_and_phi[self.num_model_param:]

        else:
            theta = theta_and_phi
            phi = []

        # ---------------------------------------
        # Calculate log-prior for model parameters
        # ---------------------------------------
        log_prior_val = 0.0
        for i, prior_func in enumerate(self.log_priors_model):
            lp = prior_func(theta[i])
            if lp <= -1e5:
                return -1e30
            log_prior_val += lp

        # -----------------------------------------------------
        # If MD ==True, accumulate log-prior for phi as well
        # -----------------------------------------------------
        if self.MD:
            for i, prior_func in enumerate(self.log_priors_MDGP):
                lp = prior_func(phi[i])
                if lp <= -1e5:
                    return -1e30
                log_prior_val += lp

        # ------------------------
        # Calculate log-likelihood
        # ------------------------
        log_likelihood_val = self.log_likelihood(theta, phi)

        return log_prior_val + log_likelihood_val
```

Define priors

```
from scipy.stats import beta

# Define function for general beta priors
def log_prior_param(param, alpha_val, beta_val, param_min, param_max):
    """
    Log prior for parameter based on a Beta distribution.
    """
    scale = param_max - param_min  # Rescaling factor

    if param_min < param < param_max:
        # Rescale param to the range [param_min, param_max]
        param_rescaled = (param - param_min) / scale

        prior = beta.pdf(param_rescaled, alpha_val, beta_val) / scale
        return np.log(prior)
    else:
        return -np.inf
```

```
model_param_bounds = np.array([[ 0.0,  20.0 ],
                               [-2.0 ,  2.0 ]])

# model_param_bounds = np.array([[ 0.0,  20.0 ]])  # When only model parameter is g -- That's
all the change required in code

num_model_param = len(model_param_bounds)

# Define a list of log_prior functions for model parameters
log_prior_theta = [
    lambda param, i=i: log_prior_param(
        param, alpha_val=1.01, beta_val=1.01,  # for flat priors
        param_min=model_param_bounds[i][0], param_max=model_param_bounds[i][1]
    )
    for i in range(num_model_param)
]
```

Without model discrepancy

```
# Instantiate class  -->
MDC = ModelDiscrepancy(t_obs=t_obs, hexp_mean=hexp_mean, hexp_err=hexp_err,
                       model_height=model_height, num_model_param=num_model_param,
                       log_priors=log_prior_theta, MD = False, kernel_func = None)
```

```
from scipy import optimize

guess = model_param_bounds.mean(axis=1)

# Find Maximum likelihood value for parameters -->
nll = lambda *args: - MDC.log_likelihood(*args)
result = optimize.minimize(nll, guess,  method = 'Powell')
max_lik = result['x']
print(result)
print("\nMaximum likelihood value for parameters:", max_lik,"\n")


# Find MAP values for parameters -->
nlp = lambda *args: - MDC.log_posterior(*args)
result = optimize.minimize(nlp, guess,  method = 'Powell')
MAP = result['x']
print(result)
print("\nMAP values for parameters:", MAP)
```

```
 message: Maximum number of function evaluations has been exceeded.
 success: False
  status: 1
     fun: -8.556378888999681
       x: [ 3.977e+00  1.354e+00]
     nit: 83
   direc: [[ 7.342e-03  7.656e-05]
           [ 9.371e-03 -8.352e-04]]
    nfev: 2000

Maximum likelihood value for parameters: [3.97682437 1.35379069]

 message: Maximum number of function evaluations has been exceeded.
 success: False
  status: 1
     fun: -4.17801963736677
       x: [ 3.978e+00  1.353e+00]
     nit: 83
   direc: [[ 7.276e-03  1.052e-04]
           [ 9.307e-03 -8.065e-04]]
    nfev: 2000

MAP values for parameters: [3.97794526 1.35323281]
```

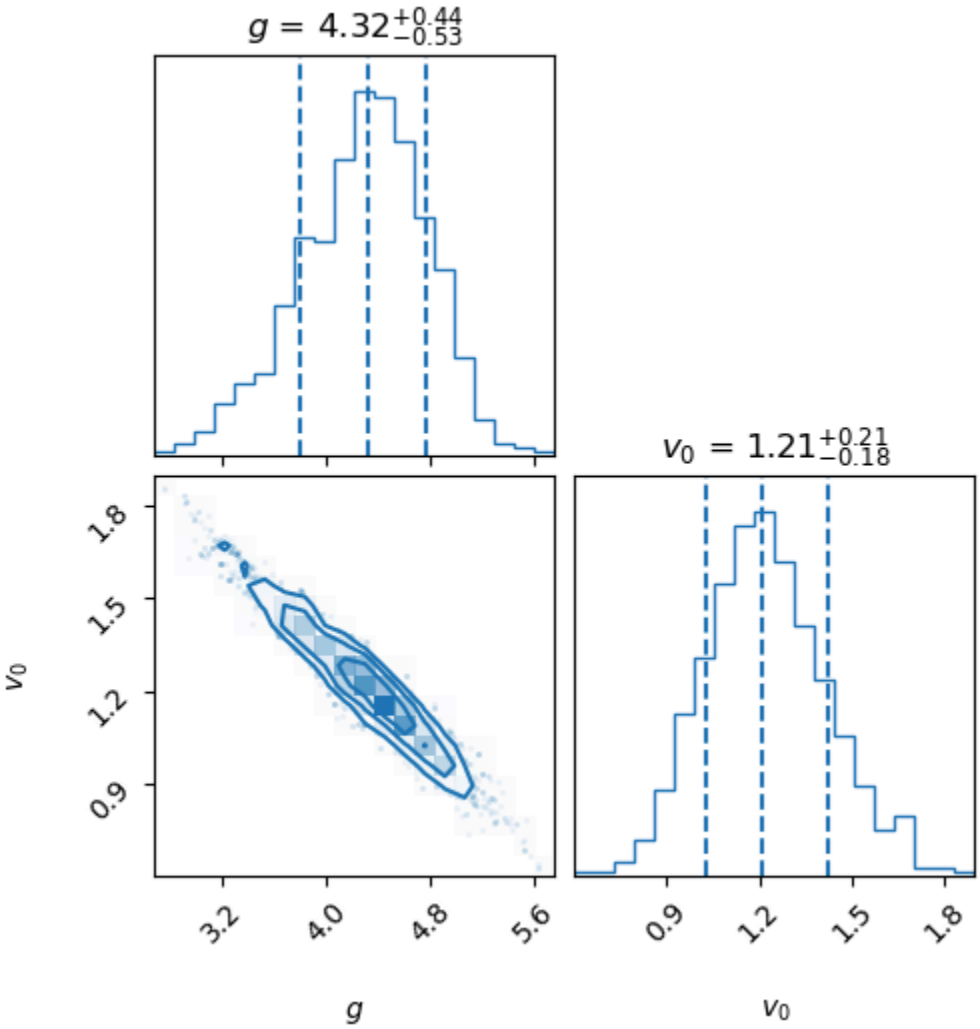
```
min_param = model_param_bounds[:,0]
max_param = model_param_bounds[:,1]
lab = [r'$g$', r'$v_0$']
```

Now do the sampling and plot the posteriors

The initial values for the the numbers of burn-in and sampling steps are initially rather low (for both this section and the next), which can lead to poorly converged posteriors. After your first analysis (to the end of the notebook), return and re-run with increased values (see suggested values in the comments; note that it will take a while to run).

```
num_burn = 200  # 2000 is a minimum
num_steps = 200  # 2000 is a minimum

samples_WMD = emcee_sampling(min_param = min_param, max_param = max_param,
                         log_posterior = MDC.log_posterior,
                         nburn = num_burn, nsteps = num_steps, num_walker_per_dim = 8)

fig = corner.corner(samples_WMD, labels=lab, color='C0', quantiles=[0.16, 0.5, 0.84],
                    show_titles=True, title_kwargs={"fontsize": 12}, hist_kwargs={'density':
True})
plt.show()
```

```
Starting MCMC sampling using emcee with 16 walkers in parallel...
Sampling complete. Time taken: 0.17 minutes.
Mean acceptance fraction: 0.735 (in total 3200 steps)
Samples shape: (3200, 2)
```



```
# # sampling with pocomc for consistency check --->

# n_effective_mltpl = 500
# n_active_mltpl = 150  # 0.3 x n_eff
# n_steps_mltpl = 2
# num_param = len(min_param)

# n_effective = n_effective_mltpl * num_param
# n_active = n_active_mltpl * num_param
# n_steps = n_steps_mltpl * num_param
# # print(n_effective, n_active, n_steps)

# n_total = 20000

# samples_WMD_poco = pocomc_sampling(min_param, max_param, MDC.log_posterior,
#                                    n_effective=n_effective, n_active=n_effective,
n_steps=n_steps,
#                                    n_total=n_total, n_evidence=n_total)

# fig = corner.corner(samples_WMD_poco, labels=lab, color='C0', quantiles=[0.16, 0.5, 0.84],
#                     show_titles=True, title_kwargs={"fontsize": 12}, hist_kwargs={'density':
True})
# plt.show()
```

With model discrepancy

```
from sklearn.gaussian_process.kernels import DotProduct, RBF

def MD_kernel(X1, X2, cbar, l, r, s):
    """
    Compute the covariance matrix using a custom kernel:
    K(X1, X2) = s^2 + cbar^2 * (X1 @ X2.T)^r * exp(-||X1 - X2||^2 / (2 * l^2)).

    This kernel serves as the core of the Model Discrepancy framework and should be modified
    to suit specific problems.

    Parameters:
        - X1 (numpy.ndarray): A matrix where each row represents an input vector.
        - X2 (numpy.ndarray): A matrix where each row represents an input vector.
        - cbar (float): The marginal variance parameter.
        - l (float): The length scale parameter for the RBF term.
        - r (float): The power applied to the dot product term.
        - s (float): A constant shift.

    Returns:
        - numpy.ndarray: A covariance matrix computed using the defined kernel.
    """
    # Compute the dot product term
    dot_product_term = np.dot(X1, X2.T) ** r if r > 0 else 1.0

    # Compute the RBF kernel term
    rbf_kernel = RBF(length_scale=l)
    rbf_term = rbf_kernel(X1, X2)

    # Combine terms
    cov_matrix =  s**2 + cbar**2 * dot_product_term * rbf_term
    return cov_matrix

# ============================================================================================
```

```
# Define hyperparameter bounds and priors
HP_bounds = np.array([[ 0.0,  10.0 ],     # for cbar
                      [ 0.0 ,  10.0 ],    # for l
                      [ 0.0 ,  10.0 ]     # for r
                      # [ 0.0 ,  10.0 ]   # for s
                     ])

# Define priors for GP hyperparmeters
log_prior_cbar = lambda param: log_prior_param(param, alpha_val=1.01, beta_val=1.01,
param_min=HP_bounds[0][0], param_max=HP_bounds[0][1])
log_prior_l =    lambda param: log_prior_param(param, alpha_val=1.01, beta_val=1.01,
param_min=HP_bounds[1][0], param_max=HP_bounds[1][1])
log_prior_r =    lambda param: log_prior_param(param, alpha_val=1.01, beta_val=1.01,
param_min=HP_bounds[2][0], param_max=HP_bounds[2][1])
# log_prior_s =    lambda param: log_prior_param(param, alpha_val=1.01, beta_val=1.01,
param_min=HP_bounds[3][0], param_max=HP_bounds[3][1])
```

```
kernel = lambda X1, X2, cbar, l, r: MD_kernel(X1, X2, cbar, l, r, s=0)
log_prior_phi = [log_prior_cbar, log_prior_l, log_prior_r]

log_priors_thetaphi = np.concatenate([log_prior_theta, log_prior_phi])
param_bounds = np.concatenate([model_param_bounds, HP_bounds])

# Instantiate class  -->
MDC = ModelDiscrepancy(t_obs=t_obs, hexp_mean=hexp_mean, hexp_err=hexp_err,
                       model_height=model_height, num_model_param=num_model_param,
                       log_priors=log_priors_thetaphi, MD = True, kernel_func = kernel)
```

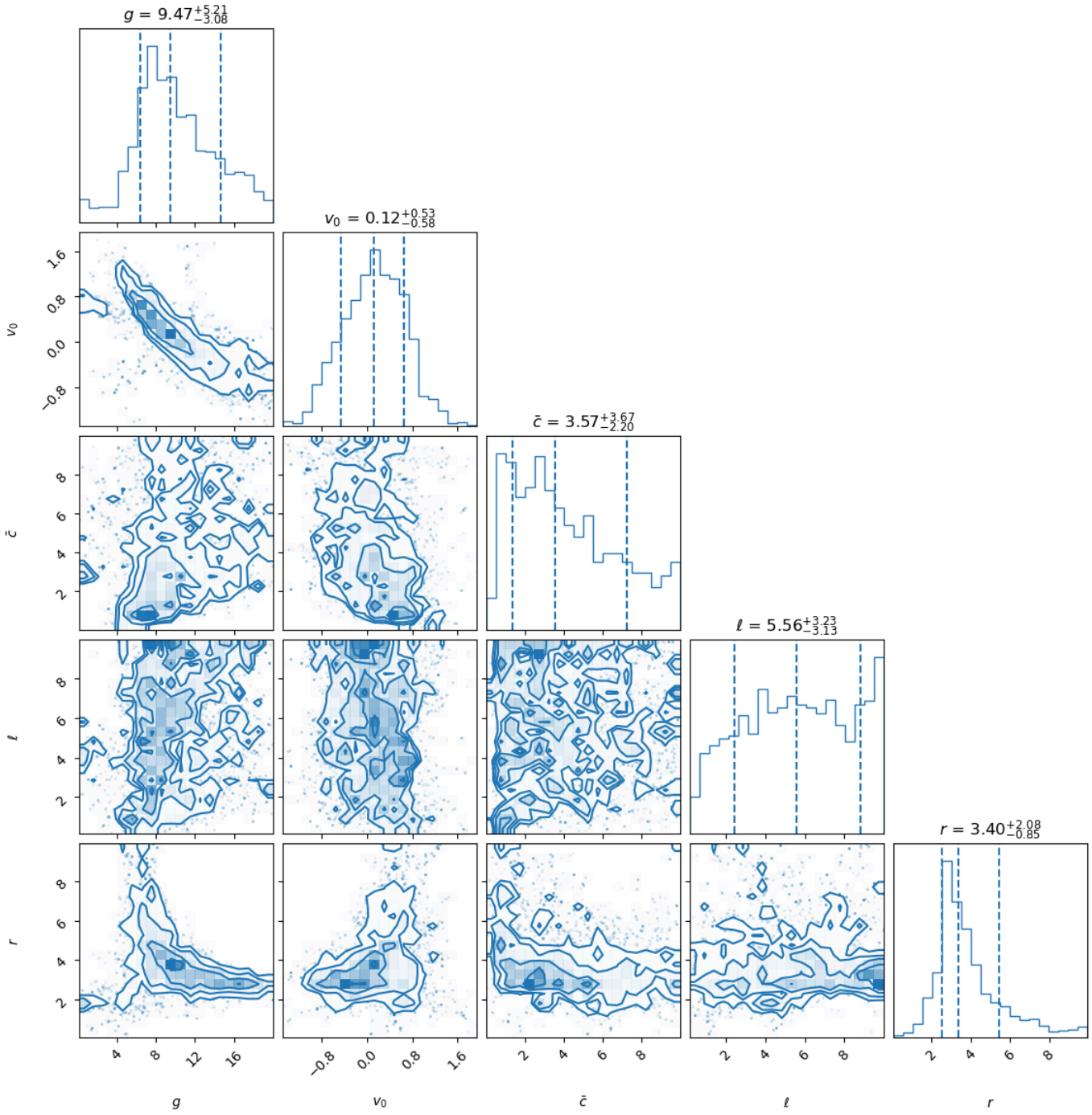
```
from scipy import optimize

# Find MAP values for parameters -->
guess = param_bounds.mean(axis=1)

nlp = lambda *args: - MDC.log_posterior(*args)
result = optimize.minimize(nlp, guess,  method = 'Powell')
MAP = result['x']

print(result)
print("\nMAP values for parameters:", MAP)
```

```
 message: Optimization terminated successfully.
 success: True
  status: 0
     fun: 1.130478145440673
       x: [ 6.949e+00  5.352e-01  6.789e-01  6.310e+00  6.134e+00]
     nit: 11
   direc: [[-7.990e-01  1.626e-01 ... -6.852e-01  5.009e-01]
           [ 3.777e-02  3.153e-02 ...  3.087e-01 -1.696e-01]
           ...
           [ 0.000e+00  0.000e+00 ...  1.000e+00  0.000e+00]
           [-3.678e-01  6.470e-02 ... -4.417e-01  8.013e-01]]
    nfev: 583

MAP values for parameters: [6.94940459 0.53522913 0.67887983 6.30979757 6.13444589]
```

```
min_param = param_bounds[:,0]
max_param = param_bounds[:,1]
lab = [r'$g$', r'$v_0$', r'$\bar{c}$', r'$\ell$', r'$r$', r'$s$']
```

```
num_burn = 200  # 2000 is a minimum
num_steps = 200  # 2000 is a minimum

samples_MD = emcee_sampling(min_param = min_param, max_param = max_param,
                         log_posterior = MDC.log_posterior,
                         nburn = num_burn, nsteps = num_steps, num_walker_per_dim = 8)

fig = corner.corner(samples_MD, labels=lab, color='C0', quantiles=[0.16, 0.5, 0.84],
                    show_titles=True, title_kwargs={"fontsize": 12}, hist_kwargs={'density':
True})
plt.show()
```

```
Starting MCMC sampling using emcee with 40 walkers in parallel...
Sampling complete. Time taken: 0.52 minutes.
Mean acceptance fraction: 0.357 (in total 8000 steps)
Samples shape: (8000, 5)
```



Things to do

- Compare the results from this notebook to the relevant figure from the section on “The ball-drop model”. Are they consistent?
- Summarize the difference between results without and with model discrepancy.
- Do you think you have enough samples? How can you tell? If you have time, increase the numbers and re-run to see if your observations change.
- (Advanced) Repeat using the pocomc sampler and comment on the comparison with emcee.

15.5. Example: Discrepancy models for relativistic heavy-ion collisions

In this section we summarize result for a real-world example. The figures are from two papers by Dr. Sunil Jaiswal and collaborators, which apply discrepancy models based on Gaussian processes to data from relativistic heavy-ion collision experiments. These models take the same form as for the ball-drop example. The goal is to extract meaningful distributions for the bulk and shear specific viscosities ("specific" in this context means normalized by the entropy density) and other experimental observables.

Bayesian model-data comparison incorporating theoretical uncertainties

Reference: S. Jaiswal, C. Shen, R.J. Furnstahl, U. Heinz, M.T. Pratola, "Bayesian model-data comparison incorporating theoretical uncertainties", Phys. Lett. B 870 (2025) 139946.

In the following examples, simulate collisions of Au on Au nuclei at $\sqrt{s_{\text{NN}}} = 200\,\text{GeV}$ at $20-30\%$ collision centrality are simulated using the `iEBE-MUSIC` simulation framework. These simulations are considered to represent the "truth" (i.e., the true results). We use them to generate synthetic experimental data with noise. The theoretical model to be compared with these synthetic data is the same `iEBE-MUSIC` model but with a different ansatz for the momentum distribution at particlization (assuming local thermal equilibrium f_{eq} instead of the Grad 14-moment momentum distributions for the particlized hadrons).

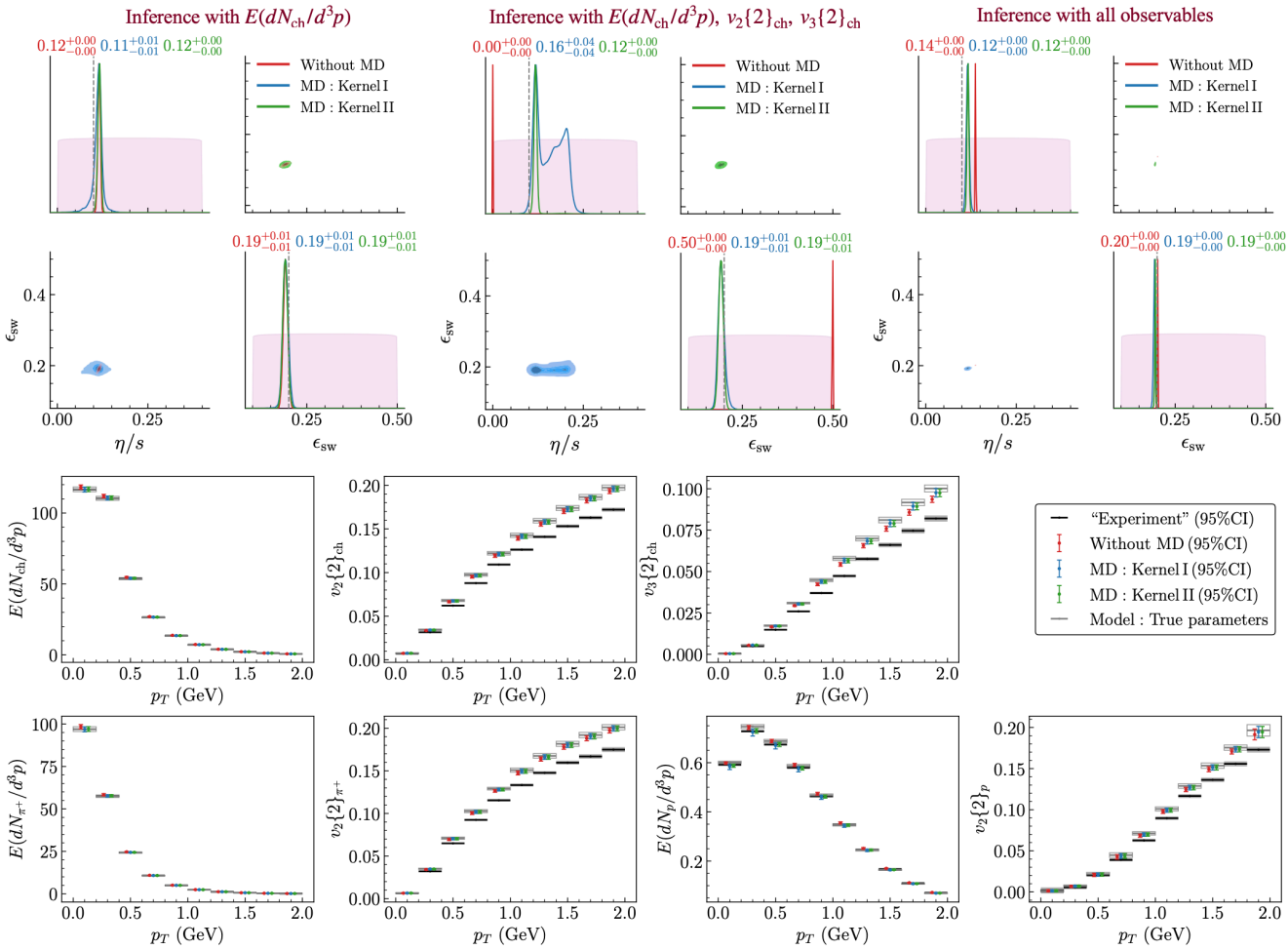


Fig. 15.8 Results for the two-parameter hydrodynamic simulation using synthetic data generated with fixed $\eta/s = 0.1$ and $\epsilon_{sw} = 0.2$GeV/fm ^{3}. Top row: Corner plots showing the posteriors for the inferred η/s and ϵ_{sw} (with median $\pm 68\%$ CI displayed on the diagonals) as more observables are included in the Bayesian inference (from left to right). Bottom rows: Model predictions based on inference using all observables.

Using only the $E(dN_{ch}/d^3p)$ measurements (top left in Fig. 15.8), all three cases (i)-(iii) yield posteriors that are statistically consistent with the truth. However, once the flow observables $v_2\{2\}_{ch}$ and $v_3\{2\}_{ch}$ are included (top middle), in the no model discrepancy (no-MD) case (red) the Bayesian fit clearly struggles to find optimal values for the parameters, producing significantly shifted posteriors that hit the edge of the prior range ("very wrong") while exhibiting exceedingly small uncertainties ("very confident"). Including MD using Kernel I (blue), the posteriors are once again statistically consistent with the truth, albeit with increased uncertainty ("much less confidence") for η/s. With Kernel II (green) the MD term captures the model imperfections for v_2 and v_3 much better, yielding posteriors similar to those obtained by ignoring the charged hadron anisotropic flow data. Once all available observables are included (top right), the inferred values for both model parameters agree with their true values, within significantly reduced uncertainties, as long as MD is accounted for (blue and green); without MD η/s is very confidently inferred to be about 40% larger than its true value.

The bottom panel of Fig. 15.8 compares the synthetic experimental data with their predictions from the calibrated model. Black boxes denote the median and 95% credible intervals (CI) of the experimental data, while open gray boxes show predictions from the model using the true parameter values, the difference highlighting the effects of model imperfections. The colored (red, blue, green) points and vertical bars show the median and 95% CI of the model predictions obtained from the inferred posteriors in the top right panel. Overall, predictions with MD (blue and

green) are closer to the model predictions with the true parameters while those without MD (red) align more closely with the data. This demonstrates that the MD framework prioritizes obtaining correct parameter estimates over merely fitting the observables.

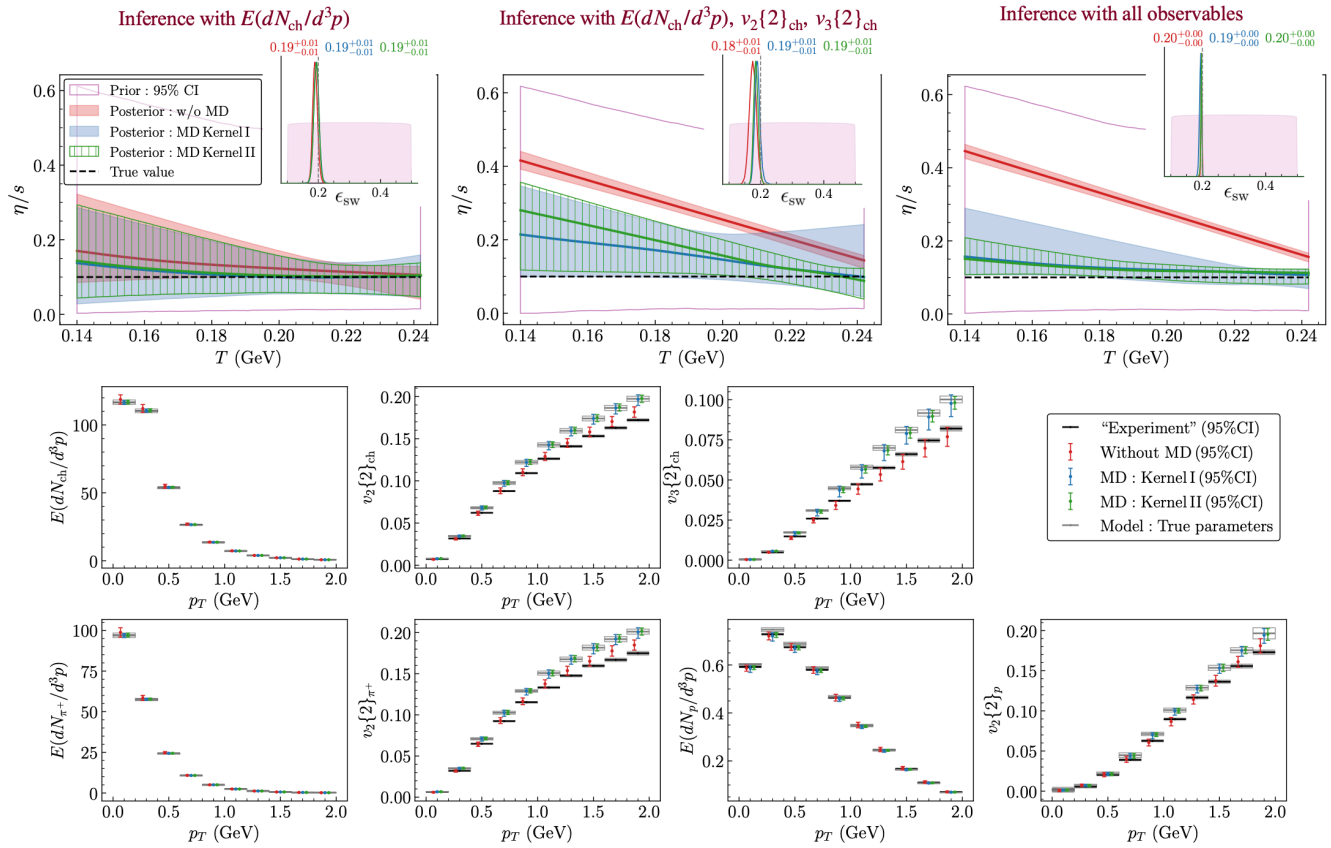


Fig. 15.9 Results for the five-parameter hydrodynamic simulation with parametrized η/s using synthetic data generated with fixed $\eta/s = 0.1$ and $\epsilon_{\rm sw} = 0.2\,\text{GeV/fm}^3$. Top row: Plots display the η/s posterior (median and $\pm 95\%$ CI) as a function of temperature, with the posterior for $\epsilon_{\rm sw}$ (median $\pm 68\%$ CI) shown in the inset, as additional observables are sequentially incorporated into the Bayesian inference (from left to right). Bottom rows: Model predictions based on inference using all observables.

When only the momentum distribution $E(dN_{\rm ch}\ /d^3p)$ is considered (top left), all three cases yield posteriors that are statistically compatible with the truth. However, for η/s the posteriors accounting for MD (blue and green) overlap significantly better with the true values than those without MD (red). Once $v_2\{2\}_{\rm ch}$ and $v_3\{2\}_{\rm ch}$ are added to the calibration data (top middle), none of the posteriors uniformly covers the truth any more. When not accounting for MD (red), the posterior credible intervals for η/s are very narrow (suggesting high confidence) but are completely inconsistent with the truth. Including MD helps a little but still infers 95% credible intervals that do not cover the true value of η/s over most of the temperature range. Apparently, the inference algorithm tries to compensate for the model's tendency to over-predict v_2 and v_3 by increasing η/s. When all observables (including the spectra and elliptic flows of identified pions and protons) are added to the calibration data (top right), all posteriors become narrower (i.e., more constrained). The overlap of the posteriors accounting for MD improves, whereas the posteriors that do not account for MD degrade further.

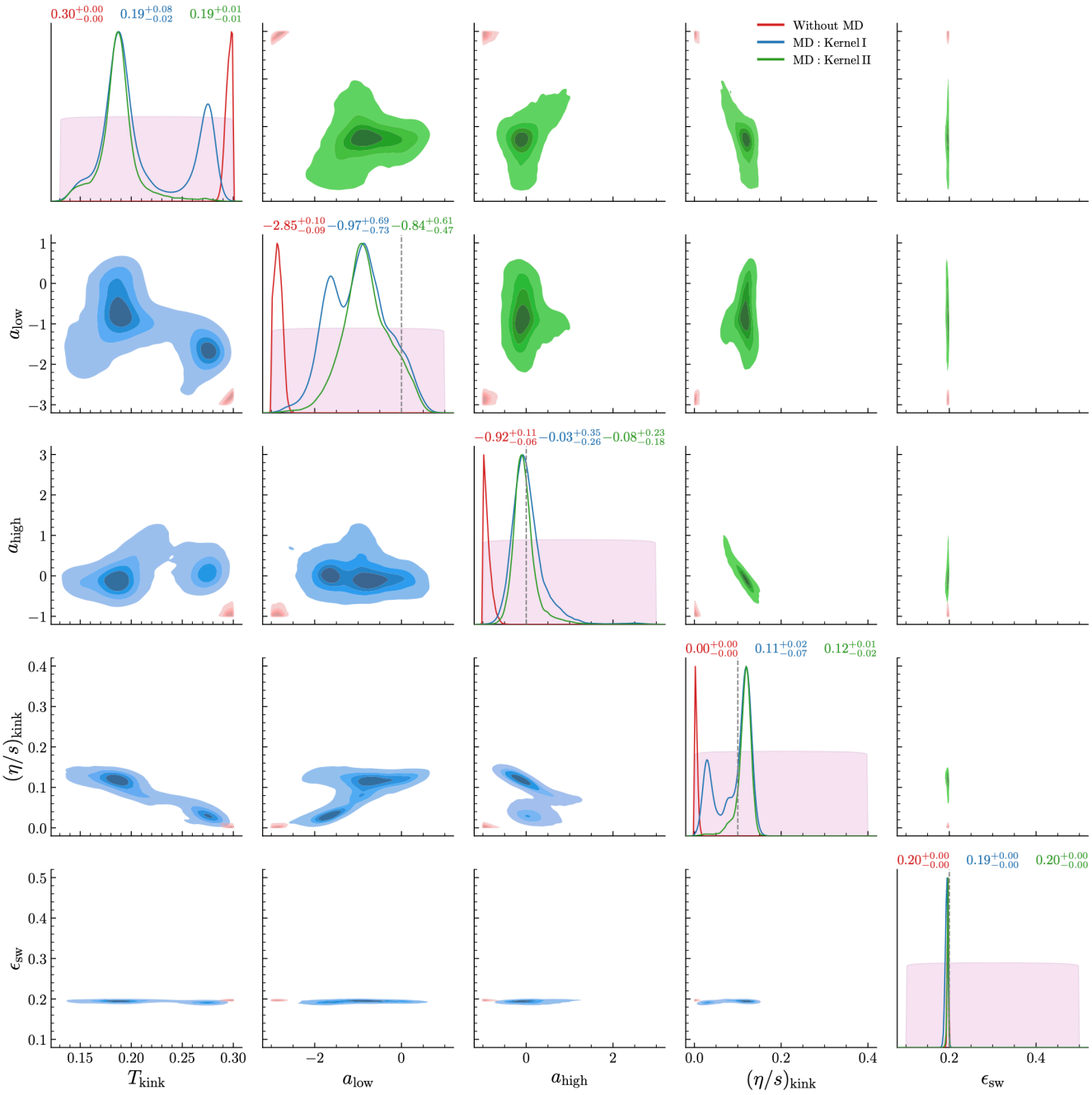


Fig. 15.10 Corner plot of all model parameters (with median $\pm 68\%$ CI displayed on the diagonals) for the five-parameter model with parametrized η/s, using synthetic data generated with fixed $\eta/s = 0.1$ and $\epsilon_{\mathrm{sw}} = 0.2\,\mathrm{GeV/fm}^3$, corresponding to the results shown in Fig. 15.9

The model predictions in the bottom row of Fig. 15.11 (obtained from the inferred posteriors in the top right panel) reinforce our observations made in the two-parameter case: Predictions that include MD do not attempt to overfit the data (black) but remain closer to the model predictions using the true parameter values (open gray). In contrast, the predictions without MD overfit the data, resulting in the incorrect inference for η/s observed in the top right panel.

Overall, the posteriors agree better with the true values (indicated by dashed gray vertical lines) when the model discrepancy (MD) term is included (for both Kernel I and Kernel II) compared to the no-MD case (red). Note that, since the generated synthetic experimental data do not depend on the parameter T_{kink}, its posterior should ideally

mirror the prior (shaded purple region). However, we observe that the posterior for T_{kink} peaks around a certain value even in the MD inference, although its width is significantly larger than in the no-MD case. This may be an artifact of the overly flexible model, where the MD cases struggle to disentangle the effects of the model parameter from the GP hyperparameters. Nonetheless, the resulting η/s posterior shown in Fig. 15.9 does not appear to depend crucially on this parameter.

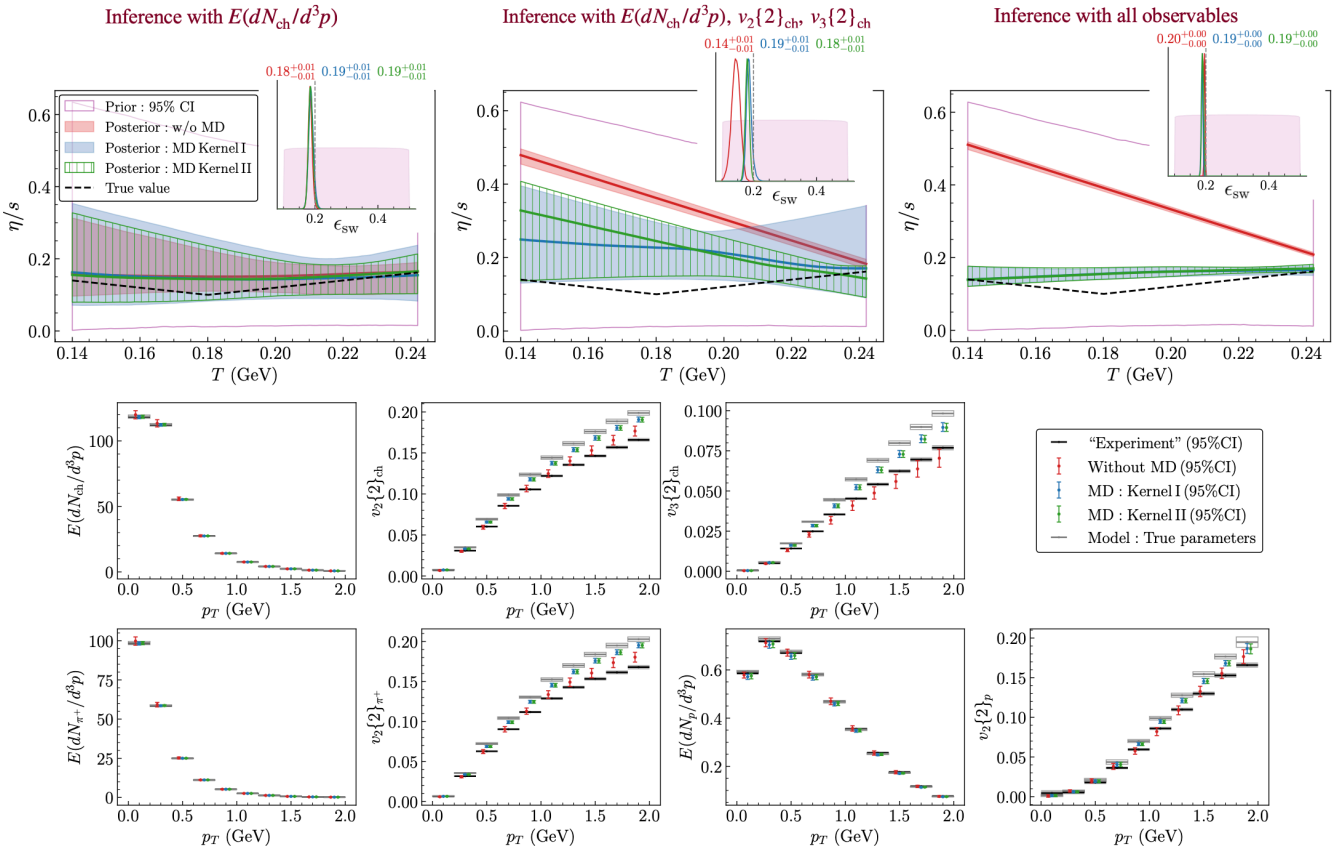


Fig. 15.11 Results for the five-parameter hydrodynamic simulation with parametrized η/s using synthetic data generated using a parametrization of η/s with $T_{\mathrm{kink}} = 0.18\,\mathrm{GeV}$, $a_{\mathrm{low}} = -1\,\mathrm{GeV}^{-1}$, $a_{\mathrm{high}} = 1\,\mathrm{GeV}^{-1}$, and $(\eta/s)_{\mathrm{kink}} = 0.1$, along with $\epsilon_{\mathrm{sw}} = 0.2\,\mathrm{GeV/fm}^3$. The plot layouts and legends are identical to those in the previous figure. The corner plot for all five model parameters, when all available observables are included, is provided below.

The results in Fig. 15.11 support the overall conclusions drawn from the preceding cases: inference including the MD term performs significantly better than inference without MD. Still, we observe that in this last example none of the cases (with or without accounting for MD) accurately captures the shape of $(\eta/s)(T)$. We suspect that the available data do not provide sufficient information to properly constrain it given the theoretical deficiencies (i.e. the neglect of viscous corrections in the particlization routine) of the model. Even with MD, simply knowing that the model is more reliable at small p_T than at large p_T is not enough to resolve the shape of $(\eta/s)(T)$. Nonetheless, the improvement over conventional Bayesian fits remains substantial.

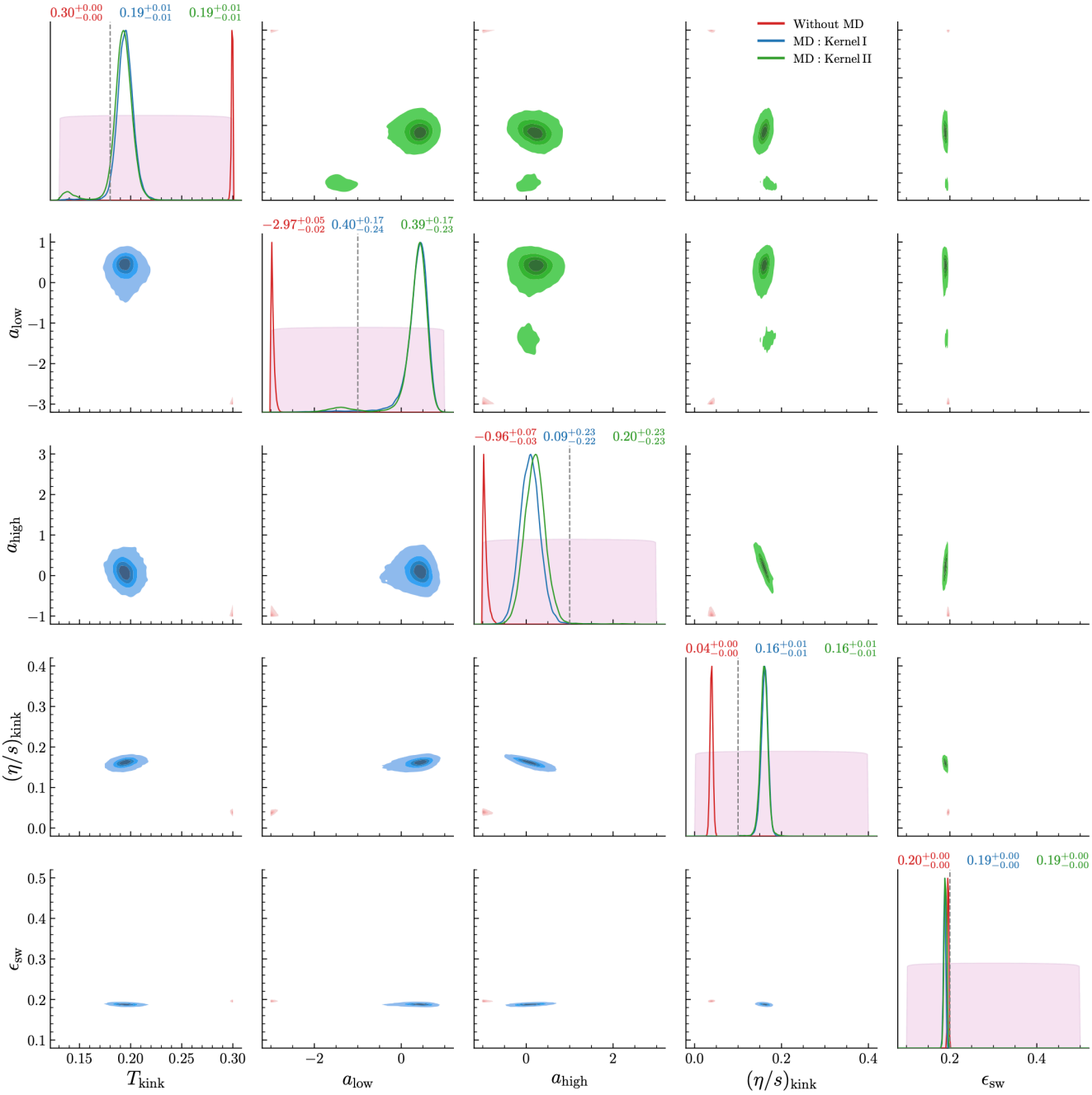


Fig. 15.12 Corner plot of all model parameters (with median $\pm 68\%$ CI displayed on the diagonals) for the five-parameter model with parametrized η/s, using synthetic data generated via the parametrization with $T_{\mathrm{kink}} = 0.18\,\mathrm{GeV}$, $a_{\mathrm{low}} = -1$, $a_{\mathrm{high}} = 1$, and $(\eta/s)_{\mathrm{kink}} = 0.1$, along with $\epsilon_{\mathrm{sw}} = 0.2\,\mathrm{GeV/fm}^3$. This plot corresponds to the results shown in Fig. 15.11.

Overall better agreement with the true values is observed when the model discrepancy (MD) term is included (blue, green) compared to the no-MD case (red). The inability of the posteriors to accurately capture the slopes a_{low} and a_{high} is reflected in the nearly flat shape of η/s as a function of temperature in Fig. 15.11 for the MD cases. We observe that the absence of increasing specific viscosity on either side of T_{kink}, as seen in the truth, is compensated by an overestimated posterior for $(\eta/s)_{\mathrm{kink}}$ (for blue, green). We attribute the framework's inability to accurately capture the posteriors for these parameters to a combination of limitations in the theoretical model, insufficient information in data to constrain the slopes, and weak prior information regarding the model's domain used in the kernel modeling.

Phenomenological constraints on QCD transport with quantified theory uncertainties

Reference: S. Jaiswal, “Phenomenological constraints on QCD transport with quantified theory uncertainties”, Phys.Lett.B 874 (2026) 140243.

In this work, a dramatic example of the value of informed Bayesian model discrepancies. The outcome is data-driven, state-of-the-art constraints on the temperature-dependent specific shear and bulk viscosities of the quark–gluon plasma from Pb–Pb collisions at $\sqrt{s_{\rm NN}}$ = 2.76 TeV. The approach is to perform global Bayesian calibration using the JETSCAPE multistage framework with two particlization ansatze, Grad 14-moment and first-order Chapman–Enskog, and quantify theoretical uncertainties via a centrality-dependent model discrepancy term. When theoretical uncertainties are neglected, the specific bulk viscosity and some model parameters inferred using the two ans¨atze exhibit clear tension. **Once theoretical uncertainties are quantified, the Grad and Chapman-Enskog posteriors for all model parameters become almost statistically indistinguishable and yield reliable, uncertainty-aware constraints.** Furthermore, the learned discrepancy identifies where each model falls short for specific observables and centrality classes, providing insight into model limitations.

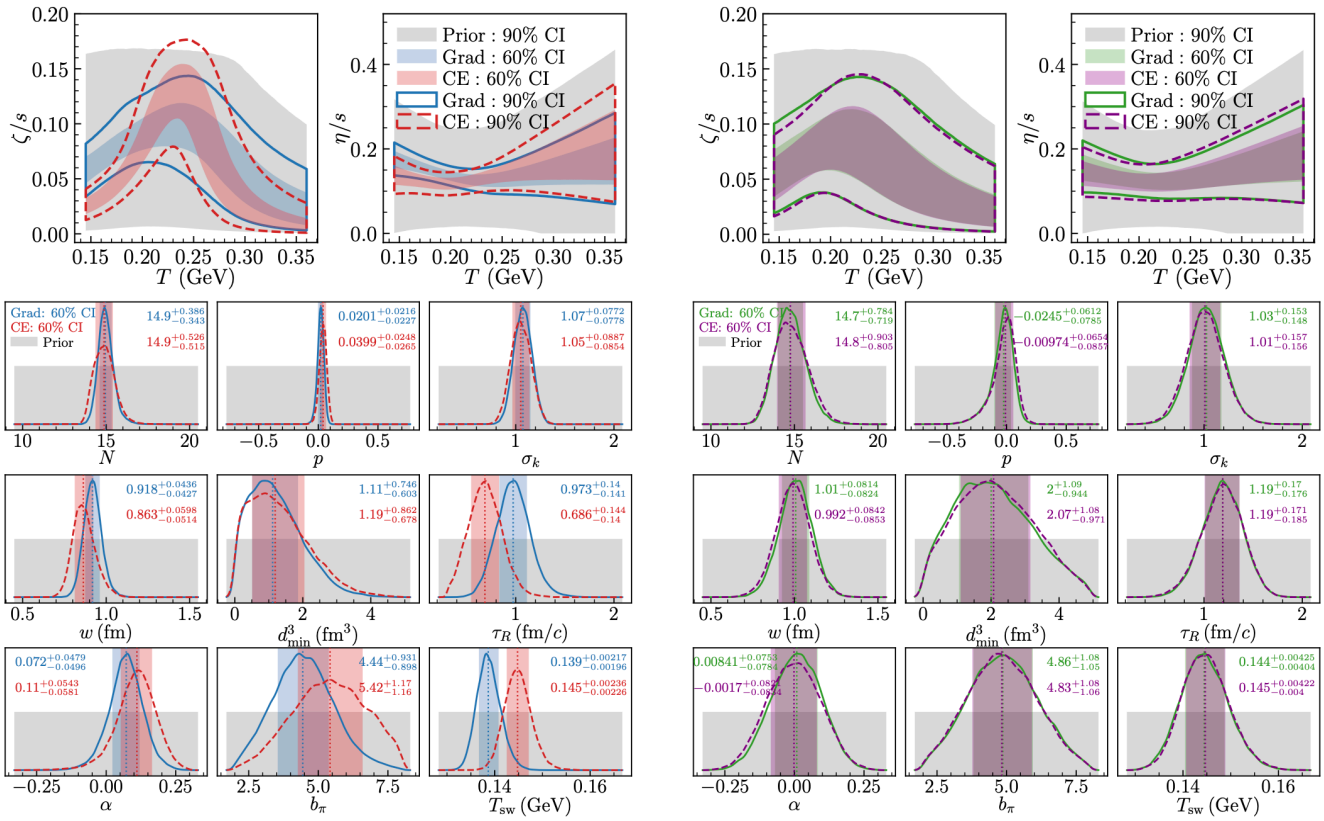


Fig. 15.13 Left: Parameter inference without accounting for theoretical uncertainty (w/o MD). Top row: Posteriors for the specific bulk and shear viscosities, with central 90% and 60% credible interval (CI) bands. The shaded gray regions denote the 90% prior intervals. Bottom rows: Solid and dashed curves show the posterior distributions for the remaining model parameters. Colored shaded regions show the central 60% CI, and shaded gray regions indicate the uniform prior range. In each subplot, a dotted vertical line marks the posterior median, and the printed value reports median $\pm 60\%$ CI. Right: Parameter inference accounting for theoretical uncertainty (w/ MD) with the same layouts and legends as on the left.

In brief summary of Fig. Fig. 15.13, the plots on the left in red and blue show parameters inferred from two different models (shrinkage from the gray priors show significant Bayesian updating), without discrepancy terms included:

there is significant tension between extracted parameters that are physical quantities. On the right, after model discrepancy is included for both models, there is almost miraculous agreement in the predictions.

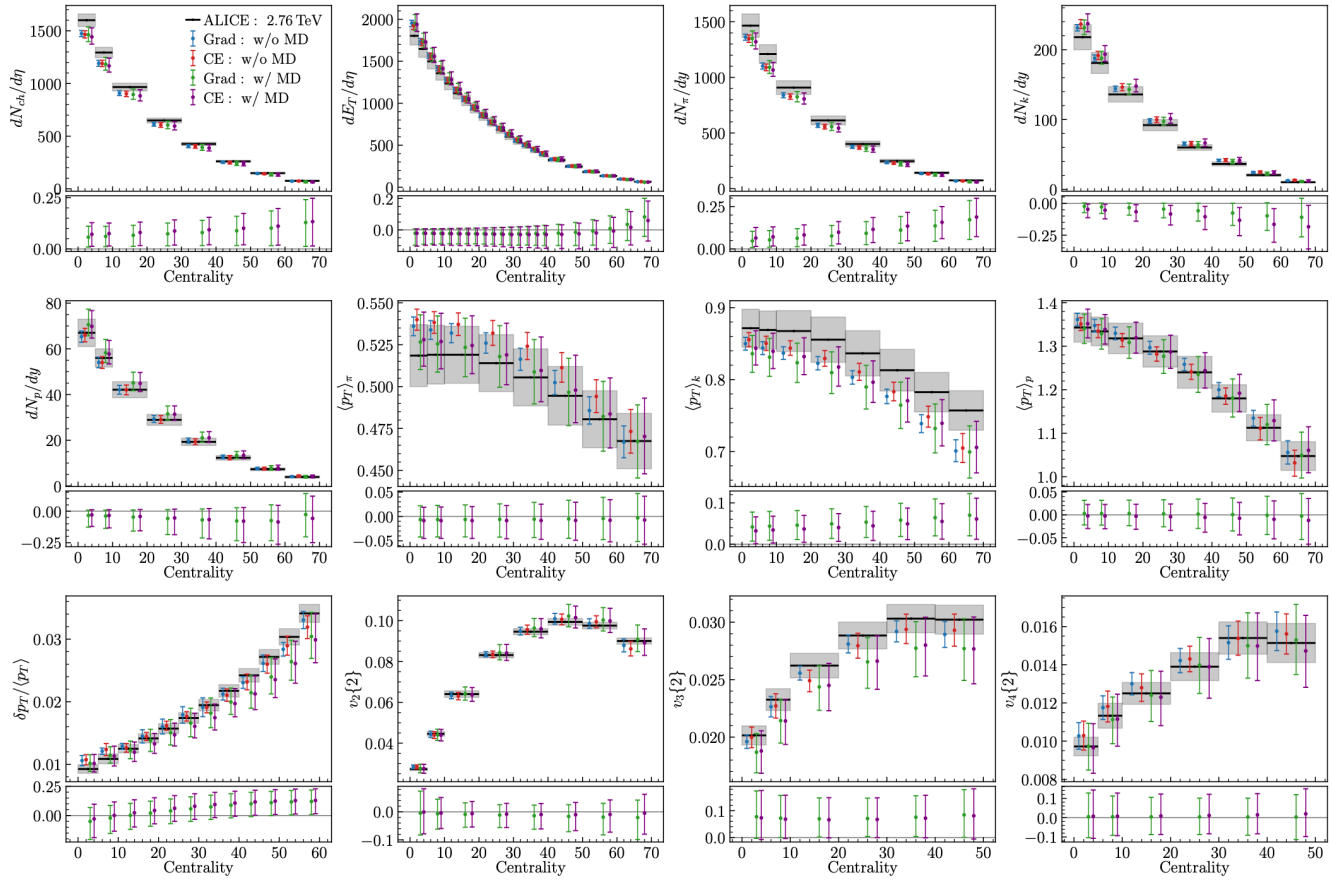


Fig. 15.14 Model predictions, obtained using the parameter posteriors shown above, are compared with ALICE data (black). Panels below each observable show the normalized model discrepancies from the w/ MD predictions.

Summary: Model–to-data comparison requires careful quantification of both experimental and theoretical uncertainties. Theoretical models have inherent limitations, and using them beyond their domains of validity without accounting for theory error can bias parameter estimates, reducing model parameters to mere fitting variables. In the work shown here, a Bayesian framework that quantifies theoretical uncertainties was used to constrain the temperature-dependent shear and bulk viscosities of the quark-gluon plasma from Pb–Pb collision data at $\sqrt{s_{\rm NN}}$ = 2.76 TeV. It was shown that, without accounting for theory errors, the multistage JETSCAPE framework with Grad and Chapman–Enskog particlization leads to tensions in the inferred transport coefficients and model parameters. Once the theoretical uncertainties are quantified, these tensions disappear and the Grad and CE posteriors become statistically indistinguishable across all shared parameters. This collapse of the model dependence yields reliable constraints on QGP viscosities in the temperature range $T \sim$ 150-350 MeV, where first-principles calculations remain limited. The learned discrepancy further provides insight into the limitations of the theoretical models.

16. Model averaging and mixing

In this chapter we discuss the challenge of combining the insights from a number of individual physics models to produce inference endowed with the physics models' collective wisdom. Section Bayesian inference in the multi-model setting provides the general setup for this problem, and introduces the crucial distinction between $\mathcal{M}$-closed and $\mathcal{M}$-open settings. Section Bayesian model averaging describes the standard Bayesian solution: Bayesian

Model Averaging (BMA); we then explain why BMA can only resolve the challenge in the $\mathcal{M}$-closed context. Section Using Bayesian model mixing to open the model space then articulates paths to generalize BMA to a more sophisticated Bayesian Model Mixing (BMM), wherein we combine information from different models in a more textured way than BMA accomplishes. We end with Section A tale of two models: contrasting BMA with BMM, which gives an example where BMM improves upon BMA by leveraging information on the local performance of two different models across the input domain.

16.1. Bayesian inference in the multi-model setting

Recall that our generic setup is that we have observations $\mathcal{D}$ consisting of pairs of inputs and outputs $(x_1, y_1), \ldots, (x_n, y_n)$ and want to, from these, predict quantities of interest $\mathbf{Q}$, which could be parameters, or interpolations or extrapolations, or even some totally new observable. In this section we further suppose we have several physics models f_k ($k = 1, \ldots, K$) that are purported to be a mapping from an x to a y. Each physics model takes in an input setting $x \in \mathcal{X}$ and a parameter setting $\theta_k \in \Theta_k$. The kth physics model is represented by $f_k(x, \theta_k)$, which should be considered a deterministic prediction of the observable at x once the model k and parameters θ_k are specified. One can build a model $\mathcal{M}_k$ for observables by combining a physics model with an error term ε that represents all uncertainties (systematic, statistical, computational):

$$\mathcal{M}_k : y_i = f_k(x_i, \theta_k) + \varepsilon_{i,k} \tag{16.1}$$

Usually, $\varepsilon_{i,k}$—the error of the ith observation in the kth model—is decomposed into a stochastic term modeling systematic discrepancy and an independent term. Note that the error does not always have to be an additive form, but we have displayed it as such for simplicity. Moreover, as written above, $\varepsilon_{i,k}$ depends on the physics model as well as on (hyper)parameters describing the statistical model, but this notation is suppressed as the dependence involves complex factors.

While different physics models may have different parameters, inference on multiple models involves dealing with a canonical parameter space Θ that spans all models of interest. We assume that for each k in $\{1, \ldots, K\}$, the model-specific parameter space Θ_k can be mapped to Θ via some (possibly non-invertible) map $\mathcal{T}_k : \Theta_k \mapsto \Theta$. After transformation, we say the parameters are in the canonical parameter space, and simply write our canonical parameter as $\theta \in \Theta$ since Θ is common to all models after the application of $\mathcal{T}_k$. We can think of this overall parameter space Θ as the union of the individual (transformed) model-specific parameters arising out of each model. For notational simplicity, the $\mathcal{T}_k$ function will be suppressed throughout this article, meaning θ is understood as $\mathcal{T}_k(\theta_k)$ when appropriate.

Our goal is to conduct inference on the values of θ as well as the error term $\varepsilon_{i,k}$ for each model using Bayesian inference. Three conceptual settings have been identified (see, e.g., \cite{Bernardo94}) where Bayesian inference on multiple models is applied: $\mathcal{M}-$closed, $\mathcal{M}-$open, and $\mathcal{M}-$complete. These three settings were originally motivated in the context of statistical model building. In the $\mathcal{M}-$closed case, one has “closed off” the need to introduce new models as it is known that the *perfect model* that represents the physical reality must be within the set of models being considered. Therefore, as data become more numerous and/or precise in the $\mathcal{M}-$closed case, that perfect model will become increasingly more likely, ultimately to the exclusion of all other models under consideration. In the $\mathcal{M}-$open case, one is open to introducing new models since the perfect model is not known. In the $\mathcal{M}-$complete case, we have decided that while we might introduce new models for the sake of accuracy, we would like to maintain inference on those in our original model set. We will not discuss this last case further.

The key distinction for inference in nuclear physics is between $\mathcal{M}-$closed, when the set of models is expected to include the perfect one, and $\mathcal{M}-$open, when we know that the set of models does not include the perfect one. We briefly outline the standard statistical solution for the $\mathcal{M}-$closed setting in the next section before moving on to

describing some potential approaches for the $\mathcal{M}-$open setting that is more interesting in the context of the BAND framework.

16.2. Bayesian model averaging

Historically, mixing together different statistical models has been done through Bayesian model averaging (BMA). BMA has been broadly applied in many areas of research including the physical and biological sciences, medicine, epidemiology, and political and social sciences. For a recent survey of BMA applications, we refer to \cite{Fragoso2018}. BMA is a framework where several competing (or alternative) models $\mathcal{M}_1, \ldots, \mathcal{M}_K$ are available. The BMA posterior density $p(\mathbf{Q}|\mathcal{D})$ corresponds to the linear combination of the posterior densities of the individual models:

$$p(\mathbf{Q}|\mathcal{D}) = \sum_{k=1}^{K} p(\mathbf{Q}|\mathcal{D}, \mathcal{M}_k)\, p(\mathcal{M}_k|\mathcal{D}). \tag{16.2}$$

If we pull through the typical inference, we can compute the first term $p(\mathbf{Q}|\mathcal{D}, \mathcal{M}_k)$ by

$$p(\mathbf{Q}|\mathcal{D}, \mathcal{M}_k) = \int_{\Theta} p(\mathbf{Q}|\mathcal{D}, \mathcal{M}_k, \theta) p(\theta|\mathcal{D}, \mathcal{M}_k) \mathrm{d}\theta. \tag{16.3}$$

The second term in (16.2), $p(\mathcal{M}_k|\mathcal{D})$, represents the posterior probability that the model k is correct. It can be computed as

$$p(\mathcal{M}_k|\mathcal{D}) = \frac{p(\mathcal{D}|\mathcal{M}_k) p(\mathcal{M}_k)}{\sum_{k=1}^{K} p(\mathcal{D}|\mathcal{M}_k) p(\mathcal{M}_k)} \tag{16.4}$$

where

$$p(\mathcal{D}|\mathcal{M}_k) = \int_{\Theta} p(\mathcal{D}|\mathcal{M}_k, \theta) p(\theta|\mathcal{M}_k) \mathrm{d}\theta.$$

The BMA posterior (16.2) for $\mathbf{Q}$ can then be obtained by using (16.3) and (16.4).

The posterior probability of model k being correct, $p(\mathcal{M}_k|\mathcal{D})$, accounts for the common physics assumptions or phenomenological properties being studied that may span many of these models. But this framing works by choosing a single model that is dominant over the entire model space. If a perfect model is explicitly considered, that is, if some $\mathcal{M}_k$ is correct, the corresponding term should dominate the sum in (\ref{eqn:posteriorBMA}). However, generic BMA can lead to misleading results when a perfect model is not included. One illustration is presented in Section A tale of two models: contrasting BMA with BMM. No nuclear physics models have access to an exact representation of reality; one only hopes some are usefully close to it. It is to be noted that while using an $\mathcal{M}-$closed approach may be problematic in many nuclear physics applications, there are nuclear physics cases when BMA can be useful~\cite{Jay2020}.

But, more generally, to be useful for nuclear physics, Bayesian inference methods should account for the relative performance of models among the different observables. Some early efforts in this direction include

\cite{kejzlar2019bayesian,Kejzlar2020} which consider multiple models which do not live on a common domain, resulting in some models being useful for prediction in certain physical regimes but not others.

16.3. Using Bayesian model mixing to open the model space

Suppose then, that no models are exactly correct through the domain of interest. To conceptualize this situation we introduce notation for the physical process $f_\star(\cdot, \theta)$, which gives the perfect (or oracle) model. That model's predictions are related to the experimental observations by:

$$y_i = f_\star(x_i, \theta) + \varepsilon_{i,\star}, \tag{16.5}$$

where the set of $\varepsilon_{i,\star}$'s represent the error between the perfect model and imperfect observations. Equation (\ref{eq:true_model}) is introduced purely for conceptual purposes. It is not practical because only an oracle has access to $f_\star(\cdot, \theta)$. Someone who knows $f_\star$ because they have direct access to the underlying reality of the universe would likely not be bothered with statistical inference—or with the scientific process at all. By presuming the $\mathcal{M}-$open scenario we invite the possibility that there is no k for which $f_\star(\cdot, \theta)$ is equivalent to $f_k(\cdot, \theta)$. The challenge is if that is true it breaks the statistical modeling principles that undergird the effectiveness of BMA as an inferential strategy.

The generalized alternative framework we now present does not attempt to weight models based on their performance across the entire input space. We say that such a generalized framework is an example of Bayesian model mixing (BMM). Our approach has connections to existing statistical literature such as \cite{goldstein2009reified} in addition to the single-model frameworks of \cite{KoH} and \cite{higdon2004combining}. Our objective is to establish different distributional assumptions beyond the assumption that any one model is perfect throughout the input space. We do this by constructing a model $\mathcal{M}_\dagger$ that combines the physics models to inform on the observations:

$$\mathcal{M}_\dagger : y_i = f_\dagger(x_i, \theta) + \varepsilon_{\dagger,i}, \text{ where } \quad f_\dagger(\cdot, \theta) \text{ is formed by combining } \qquad f_1(\cdot, \theta), \ldots, f_K(\cdot, \theta). \tag{16.6}$$

The \emph{supermodel} $f_\dagger$ is built to contain the collective wisdom of all existing models (this model was also termed reified in Ref.~\cite{goldstein2009reified}). One possible way to combine the models is BMA, where $f_\dagger(\cdot, \theta)$ has a prior distribution that is a point mass at each of $\{f_k(\cdot, \theta) : k = 1, \ldots, K\}$ that holds universally throughout the domain of interest. In BMM, we open up the possibility to combine the K models in more sophisticated ways. By mixing, one can form many potential inferences about $f_\dagger$, and—we hope—produce inferences using $f_\dagger$ that more closely resemble inferences produced by the oracle using $f_\star$.

The mixing approach would then give $p(\mathbf{Q}|\mathcal{D}) = p(\mathbf{Q}|\mathcal{D}, \mathcal{M}_\dagger)$. BMA is thus a particular special case of the BMM approach. The key to the BAND BMM framework is that $\mathcal{M}_\dagger$ accounts for underlying information present in the individual models. In the next subsection we present an example where such an $\mathcal{M}_\dagger$ is constructed in a way that takes into account the different places in the input domain $\mathcal{X}$ in which each of them is more accurate.

16.4. A tale of two models: contrasting BMA with BMM

Let us discuss a brief statistical example to unpack the sometimes subtle difference between BMA and BMM. This should not be considered a general assessment of the approaches, but instead an example to ground the concepts. For simplicity of presentation, we assume that we have two physics models: $f_1(\cdot, \theta)$ and $f_2(\cdot, \theta)$. We want to combine these two models to produce a model $f_\dagger$ that is as close to the perfect model $f_\star$ as possible. Since

perfection is not attainable we distinguish between $f_\star$, which we continue to use as a {\it gedankenmodel}, and $f_\dagger$ and try only to build the latter.

The first of the two models being mixed, f_1, is an imperfect model everywhere. Conceptually we imagine that, for all values of $x \in \{x_1, \ldots, x_n\}$, f_1 differs from $f_\star$ by a stochastic discrepancy a priori normally distributed with mean zero and some moderate variance. In contrast the second model, f_2, is such that there is a single observation, say the one at the first point x_1, for which $f_2(x_1, \theta) - f_\star(x_1, \theta)$ is potentially very large, i.e., here we think that the stochastic discrepancy is normally distributed with mean zero and an extremely large variance. But everywhere else the model is essentially perfect. We convert this information into Bayesian inference for $f_\dagger$ by saying that $f_1(x_i, \theta)$ given $f_\dagger(x_i, \theta)$ is normally distributed with mean $f_\dagger(x_i, \theta)$ and variance v_1. And that $f_2(x_1, \theta)$ given $f_\dagger(x_1, \theta)$ is normally distributed with mean $f_\dagger(x_1, \theta)$ and variance $v_2 \gg v_1$, while, for $j = 2, \ldots, n$, we have $f_2(x_j, \theta) = f_\dagger(x_j, \theta)$.

A BMA approach that acknowledges these model discrepancies expands the observed variance by the model error variance. We will assume each model has the same prior probability of being correct and the prior $p(\theta)$ on θ is given such that $p(\mathcal{M}_1, \theta) = p(\mathcal{M}_2, \theta) = \frac{1}{2} p(\theta)$. In terms of a posterior on the parameters, this implies that

$$
\begin{aligned}
p_{\text{BMA}}(\theta | \mathcal{D}) \propto p(\theta) \Bigg[& \prod_{i=1}^{n} \frac{1}{\sqrt{\sigma_i^2 + v_1}} \exp\left(-\frac{1}{2} \frac{(y_i - f_1(x_i, \theta))^2}{\sigma_i^2 + v_1} \right) \\
& + \frac{1}{\sqrt{\sigma_1^2 + v_2}} \exp\left(-\frac{1}{2} \frac{(y_1 - f_2(x_1, \theta))^2}{\sigma_1^2 + v_2} \right) \prod_{i=2}^{n} \frac{1}{\sigma_i} \exp\left(-\frac{1}{2} \frac{(y_i - f_2(x_i, \theta))^2}{\sigma_i^2} \right) \Bigg].
\end{aligned}
\tag{16.7}
$$

As mentioned previously, the BMA approach presumes that one model is correct throughout the entire domain of interest. If v_2 is truly extremely large, the BMA formalism will implement this presumption in the most extreme way possible. The spectacular failure of the second model at the first data point causes it to lose badly to the first model which just manages to be mediocre everywhere. That is, the expression for the posterior when $v_2 \to \infty$ becomes

$$
p_{\text{BMA}} \quad (\theta | \mathcal{D}) \propto \exp\left(-\frac{1}{2} \sum_{i=1}^{n} \frac{(y_i - f_1(x_i, \theta))^2}{\sigma_i^2 + v_1} \right) p(\theta).
$$

The model f_2 has no role in the BMA posterior because the BMA weights consider only the overall performance of the model over the entire domain of interest! But it seems unduly wasteful to discard the entirety of f_2 because it performs poorly in one small subset of the domain of interest.

Now we consider a BMM approach where we do not presume a single model is correct throughout the entire input space. One potential BMM approach obtains the distribution of $f_\dagger(x, \theta)$ by using standard Bayesian updating formulae to combine the probability distributions of $f_1(x, \theta)$ and $f_2(x, \theta)$ given $f_\dagger(x, \theta)$ with a Normally distributed prior on $f_\dagger$ having variance $v_\dagger$. Taking $v_\dagger \to \infty$, we have

$$
f_\dagger(x_i, \theta) \text{ is distributed as } \quad \begin{cases} \mathcal{N}\left(\frac{v_2 f_1(x_i, \theta) + v_1 f_2(x_i, \theta)}{v_1 + v_2}, \frac{v_1 v_2}{v_1 + v_2} \right) & \text{if } i = 1 \\ f_2(x_i, \theta) & \text{if } i = 2, \ldots, n. \end{cases}
\tag{16.8}
$$

This seems to use our inference on both f_1 and f_2 in an effective way. Pulling this into a posterior, we get that at $v_2 \to \infty$

$$p_{\text{BMM}} \quad (\theta|\mathcal{D}) \propto \exp\left(-\frac{1}{2}\frac{(y_1 - f_1(x_1,\theta))^2}{\sigma_1^2 + v_1} - \frac{1}{2}\sum_{i=2}^{n}\frac{(y_i - f_2(x_i,\theta))^2}{\sigma_i^2}\right)p(\theta).$$

Now both models are being used in their respective strong areas: the model f_2 is ignored only at a single point x_1 where it is very wrong and f_1 is ignored everywhere that f_2 provides a perfect result.

This example illustrates nicely that BMM can be a more effective tool for combining models than BMA. Although the example is simple we believe the concept it represents has wide applicability in NP applications where the models we want to mix perform well in different regions of the domain of interest.

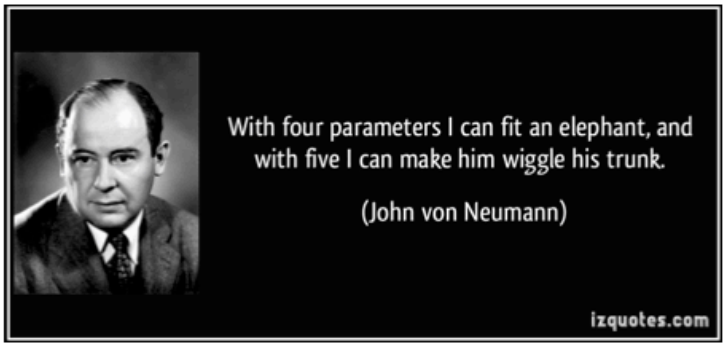
17. Problems for Part II

In this chapter we collect various problems to build intuition or gain practice in the topics covered in Part II.

- Problem: Model selection basics explores model selection via evidence calculations, using as a prototypical problem the number of terms in a Taylor series that provide the best to some (noisy) data in a finite range.
- Problem: Evidence calculation for EFT expansions has a series of tasks looking at evidence for expansions based on effective field theories.
- Problem: How many lines are there? explores model selection with a problem to infer how many signal peaks there are in a noisy set of data.

17.1. Problem: Evidence calculation for EFT expansions

Famous von Neumann quote



Attributed to John von Neumann by Enrico Fermi, as quoted by Freeman Dyson in "A meeting with Enrico Fermi" in Nature **427** (22 January 2004).

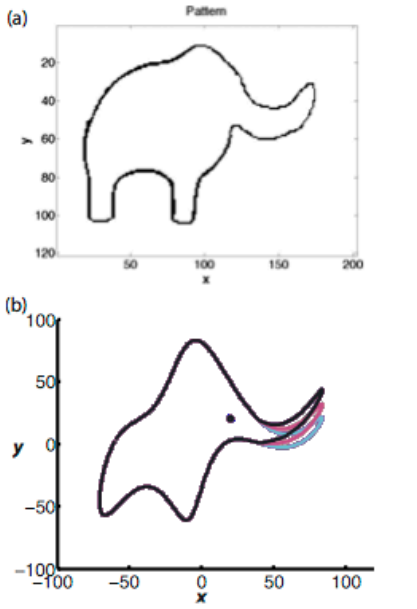


Fig. 1. (a) Outline of an elephant. (b) Three snapshots of the wiggling trunk.

"Drawing an elephant with four *complex* parameters," J. Mayer, K. Khairy, and J. Howard, Am. J. Phys. **78**, 648 (2010).

The toy model

Here we continue to explore aspects of Bayesian statistical analysis using toy models for effective field theories (EFTs), namely Taylor series of some specified functions. In this notebook we are exploring the evidence for how many coefficients in the EFT expansion are determined by the given data.

Let's first review the function we are using as a toy model, taken from *Bayesian parameter estimation for effective field theories*:

$$g(x) = \left(\frac{1}{2} + \tan\left(\frac{\pi}{2}x\right)\right)^2$$

represents the true, underlying theory. It has a Taylor expansion

$$g(x) = 0.25 + 1.57x + 2.47x^2 + 1.29x^3 + \cdots$$

Our model for an EFT for this "theory" is

$$g_{\text{th}}(x) \equiv \sum_{i=0}^{k} a_i x^i \ .$$

In another problem, our general task was to fit 1, 2, 3, … of the parameters a_i and to analyze the results.

The statistical model (recap)

Here we are given data with simple error bars, which imply that the probability for any *single* data point is a normal distribution about the true value. That is,

$$y_i \sim \mathcal{N}(y_M(x_i;\theta), \varepsilon_i)$$

or, in other words,

$$p(x_i \mid y_i, \theta) = \frac{1}{\sqrt{2\pi\varepsilon_i^2}} \exp\left(\frac{-[y_i - y_M(x_i;\theta)]^2}{2\varepsilon_i^2}\right)$$

where ε_i are the (known) measurement errors indicated by the error bars.

Assuming all the points are independent, we can find the full likelihood by multiplying the individual likelihoods together:

$$p(D \mid \theta) = \prod_{i=1}^{N} p(x_i, y_i|\theta)$$

For convenience and numerical accuracy, this is usually expressed in terms of the log-likelihood:

$$\log p(D \mid \theta) = -\frac{1}{2}\sum_{i=1}^{N}\left(\log(2\pi\varepsilon_i^2) + \frac{[y_i - y_M(x_i;\theta)]^2}{\varepsilon_i^2}\right)$$

We consider two priors for the coefficients. The first is a Gaussian prior that encodes naturalness through the parameter $\bar{a}$:

$$p(\mathbf{a} \mid \bar{a}, I) = \left(\frac{1}{\sqrt{2\pi}\bar{a}}\right)^{k+1} \exp\left(-\frac{\mathbf{a}^2}{2\bar{a}^2}\right)$$

with $\bar{a}$ taken to be fixed (at $\bar{a}_{\text{fix}} = 5$ usually). That is, the prior pdf for $\bar{a}$ is

$$p(\bar{a}) = \delta(\bar{a} - \bar{a}_{\rm fix}) \ .$$

(In more recent work, a conjugate prior was used for $\bar{a}$, which simplifies the calculations.)

The second is an uninformative uniform prior that we take to be a constant (cutting it off only at very high values, which may not even be needed). Given likelihood and prior, the posterior pdf by Bayes' Theorem is

$$p(\mathbf{a} \mid D, k, k_{\rm max}, I) = \frac{p(D \mid \mathbf{a}, k, k_{\rm max}, I)\ p(\mathbf{a} \mid I)}{p(D \mid k, k_{\rm max}, I)}$$

We have focused previously on calculating this posterior to find distributions for the coefficients $\theta = \{a_0, a_1, \cdots, a_k\}$. Furthermore, up to now we have ignored the denominator, which is the *evidence*, because we didn't need to calculate it independently. Now we will calculate it.

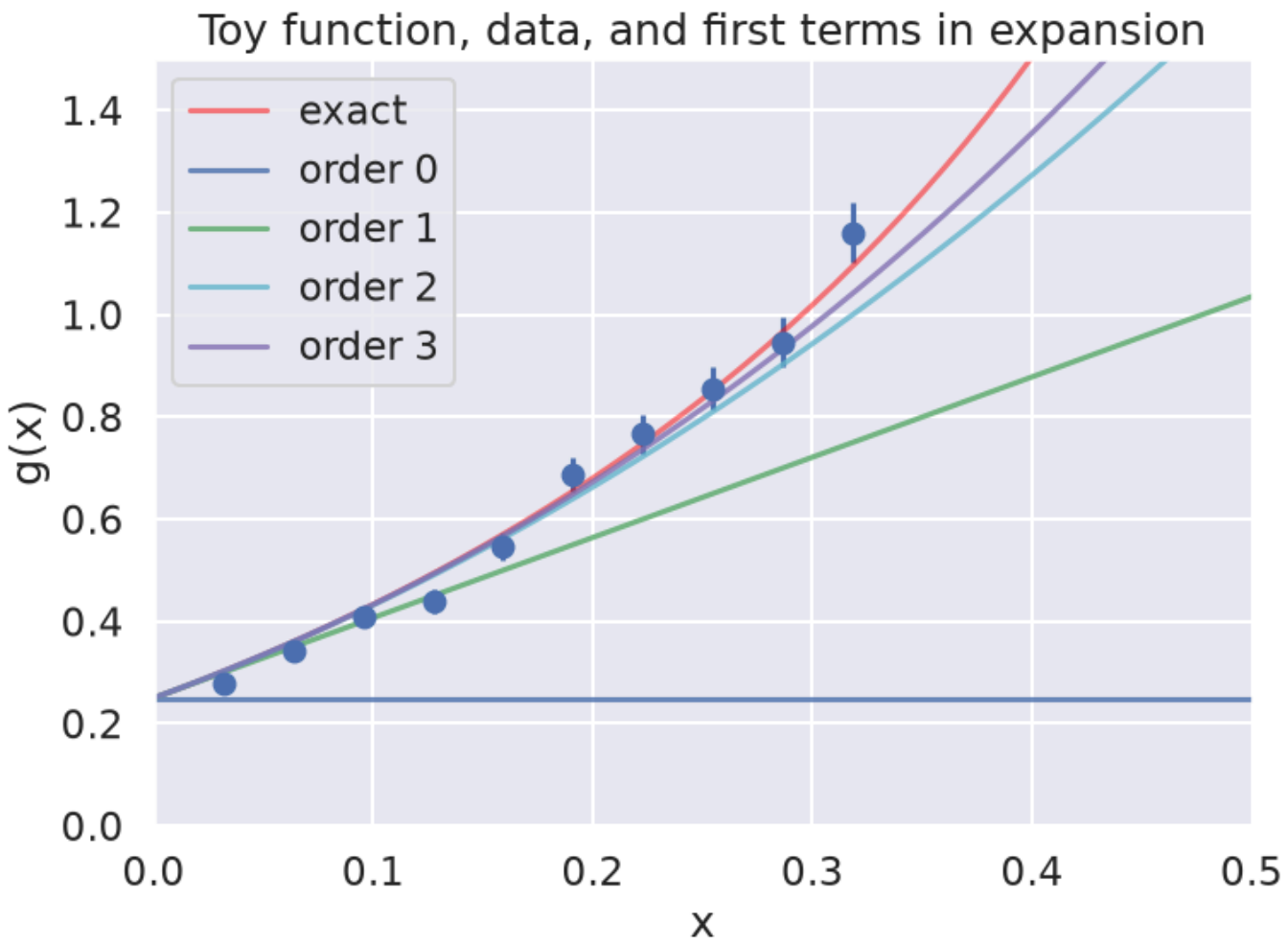
```
%matplotlib inline
import matplotlib.pyplot as plt
import matplotlib as mpl
import numpy as np
import seaborn; seaborn.set("talk") # for plot formatting
import scipy.stats as stats
from scipy import linalg

from cycler import cycler
cmap = mpl.colormaps['viridis']
```

The Data and the true result

Let's start by defining the exact function and the data for the toy model.

```
def g_fun(x):
    """
    Toy function to model an EFT expansion.  It has a Taylor expansion about
    x=0 with a radius of convergence of 1.
    """
    return (0.5 + np.tan(np.pi * x / 2.))**2

def y_model(x_pts, theta, orders=None):
    """
    Returns the evaluation of the theoretical model at all x values in the
    numpy array x_pts, using orders coefficients from theta (defaults to all).
    """
    if orders is None:   # allow for not using the full theta vector
        orders = len(theta)

    return np.array( [ np.sum(
                               [theta[i] * x**i for i in range(orders)]
                             ) for x in x_pts ] )
```

```
theta_true = np.array([0.25, 1.5707963, 2.4674011, 1.2919282, 4.0587121,
                       1.275082, 5.67486677])

# Generate data points as described in the paper; remember these are relative
#  errors, so multiply the percent by the data at each x.
x_max = 1./np.pi   # we'll eventually test sensitivity to x_max
num_x_pts = 10
x_data_pts = np.linspace(x_max/num_x_pts, x_max, num_x_pts) # don't start at x=0
eta = 0.05         # specified relative uncertainty is 5%

# Here we generate new (different) data points with every run
y_data_pts = g_fun(x_data_pts) * \
                 (1. + stats.norm.rvs(0., eta, size=len(x_data_pts)) )
# *** The following is the exact data with errors from the paper ***
#y_data_pts = np.array([0.31694, 0.33844, 0.42142, 0.57709, 0.56218, \
#                       0.68851, 0.73625, 0.87270, 1.0015, 1.0684])

dy_data = eta * y_data_pts

# Always make a figure to check your data!
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(1,1,1)
ax.errorbar(x_data_pts, y_data_pts, dy_data, fmt='o')
ax.set_xlabel(r'x')
ax.set_ylabel(r'g(x)')
ax.set_xlim(0, 0.5)
ax.set_ylim(0, 1.5)

x_pts_all = np.arange(0., 1., .01)
ax.plot(x_pts_all, g_fun(x_pts_all), color='red', alpha=0.5, label='exact')
ax.set_title('Toy function, data, and first terms in expansion')

n_dim = 4
colors = ['b', 'g', 'c', 'm', 'k']
for order in range(n_dim):
    ax.plot(x_pts_all, y_model(x_pts_all, theta_true[:n_dim], order+1),
            label=f'order {order:d}', color=colors[order], alpha=0.8)

ax.legend()
fig.tight_layout()
```



Evidence calculation

Now we seek to reproduce and understand Figure 8 in the paper *Bayesian parameter estimation for effective field theories*, which shows that the evidence for the model expansion up to order $k_{\rm max}$ *saturates* (i.e., increases up to a maximum and then flattens out close to that value). This is in contrast to the more typical expectation from evidence calculations that lead to a definite peak.

The evidence can be expressed by marginalization as an integral over *all possible* $\mathbf{a}$. (The notation with k and $k_{\rm max}$ is for consistency with the paper; for our purposes today consider this as the evidence for an expansion up to order $k = k_{\rm max}$.)

$$p(D \mid k \leq k_{\rm max}, k_{\rm max}, I) = \int d\bar{a} \int d\mathbf{a}\, p(D \mid \mathbf{a}, k = k_{\rm max}, k_{\rm max}, I)\; p(\mathbf{a} \mid \bar{a}, I)\; p(\bar{a} \mid I)$$

If you have a question about how this equations comes about, please ask!

The first term in the integrand is the likelihood, which we saw above is a multivariate Gaussian and, in the present case with independent points, it is very simple, just the product of one-dimensional Gaussians. If we take the case of a Gaussian prior for $\mathbf{a}$ and the fixed (delta function) prior for $\bar{a}$, we can do the $\bar{a}$ integral for free and the remaining integral for the evidence can be done analytically.

Evidence using linear algebra and Gaussian integrals

If we write the multivariate Gaussians in the evidence in matrix form, we can use the basic formula for integration:

$$\int e^{-\frac{1}{2}x^T Ax + B^T x}\, d^n x = \sqrt{\det(2\pi A^{-1})}\; e^{\frac{1}{2}B^T A^{-1} B}$$

where x and B are n-dimensional vectors and A is an $n \times n$ matrix, with n the number of data points. The x_i integrations are from $-\infty$ to $+\infty$.

Evidence using conjugate prior

The usefulness of a conjugate prior is in carrying out a Bayesian update without having to do any calculation. Recall yet again how Bayes theorem tells us how to update (the information I will be implicit in the following):

$$p(\theta \mid D) = \frac{p(D \mid \theta)\, p(\theta)}{p(D)}$$

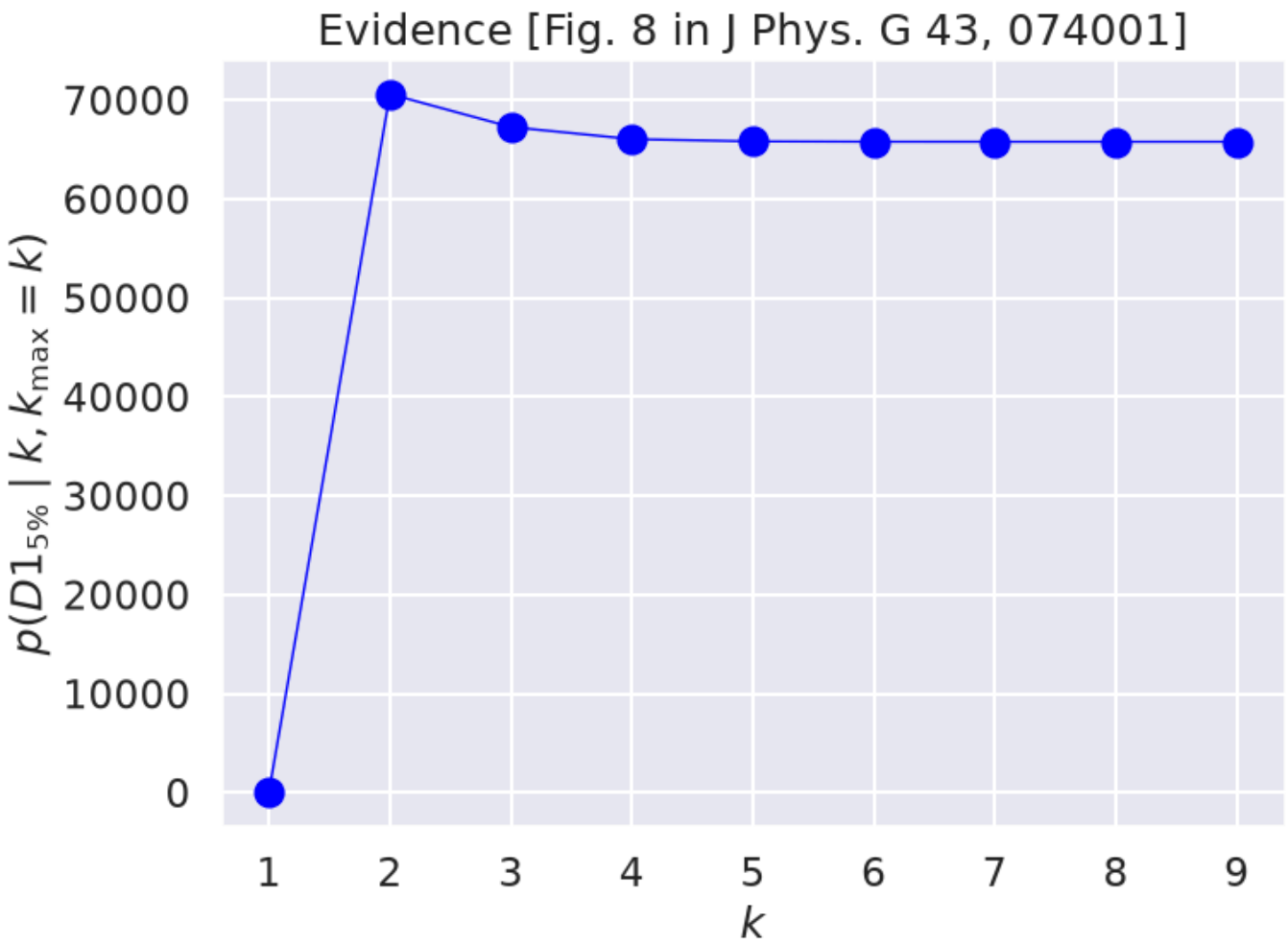
If $p(\theta)$ is a conjugate prior to the likelihood, the updating consists solely of changing the parameters that specify the prior pdf.

The most complete table of conjugate priors out in the wild seems to be the Wikipedia webpage Conjugate Prior. Take a look!

```
def make_matrices(x_pts, y_pts, dy_pts, k_order, a_bar):
    """
    Construct and return the matrices we'll need to calculate the evidence.
    We have only one observable for now, so d is omitted.
    """
    m = k_order + 1   # number of coefficients is one more than the order

    A_mat = np.array( [[x**i for x in x_pts] for i in range(m)] ).T
    Sigma_mat = np.diag(dy_pts**2)
    Vp_mat = a_bar**2 * np.eye(m)
    y_vec = y_pts

    return A_mat, Sigma_mat, Vp_mat, y_vec

def gaussian_norm(cov_mat):
    """Return the normalization factor for Gaussians.
       You can decide whether to use a covariance matrix or its inverse."""
    return 1. / np.sqrt(linalg.det(2. * np.pi * cov_mat))
```

```
# step through the orders
k_max = 10
k_orders = range(k_max)

evidence = np.zeros(k_max)
for k_order in k_orders:
    a_bar = 5.
    A_mat, Sigma_mat, Vp_mat, y_vec = make_matrices(x_data_pts, y_data_pts,
                                                    dy_data, k_order, a_bar)

    Sigma_mat_inv = linalg.inv(Sigma_mat)

    Lambda_mat = A_mat.T @ Sigma_mat_inv @ A_mat + linalg.inv(Vp_mat)
    Lambda_mat_inv = linalg.inv(Lambda_mat)

    Vp_mat_inv = linalg.inv(Vp_mat)

    a_hat = Lambda_mat_inv @ A_mat.T @ Sigma_mat_inv @ y_vec
    #print(f'k = {k_order}  a[k] = {a_hat:.2f}')
    print(k_order, a_hat[0:min(k_order,4)])

    chisq_min = (y_vec - A_mat @ a_hat).T @ Sigma_mat_inv @ \
                (y_vec - A_mat @ a_hat)

    evidence[k_order] = np.sqrt(linalg.det(2.*np.pi*Lambda_mat_inv)) \
                        * gaussian_norm(Sigma_mat) * np.exp(-chisq_min / 2.) \
                        * gaussian_norm(Vp_mat) \
                        * np.exp(- a_hat.T @ Vp_mat_inv @ a_hat / 2.)

fig, ax = plt.subplots(figsize=(8,6))
ax.plot(k_orders[1:], evidence[1:], color='blue', marker='o',
        linestyle='solid', linewidth=1, markersize=12)
ax.set_title('Evidence [Fig. 8 in J Phys. G 43, 074001]')
ax.set_xlabel(r'$k$')
ax.set_ylabel(r'$p(D1_{5\%} \mid k, k_{\rm max}=k)$')

fig.tight_layout()
```

```
0 []
1 [0.17194546]
2 [0.23734761 1.21543097]
3 [0.2349924  1.30412978 4.11522205]
4 [0.23465663 1.31476486 4.05347489 1.71813913]
5 [0.23461665 1.31593216 4.04787064 1.71159193]
6 [0.23461194 1.31606295 4.04731395 1.71080924]
7 [0.23461138 1.31607803 4.04725519 1.7107133 ]
8 [0.23461132 1.31607978 4.04724882 1.71070154]
9 [0.23461131 1.31607998 4.04724813 1.71070013]
```



Things for you to do:

- *What do you expect to happen if you increase the range of data (set by `x_max` at the upper end)? Do it and comment on the plots.*
- *What do you expect to happen if you increase the number of data points (set by `num_x_pts`)? Do it and comment on the plots.*
- *What do you expect to happen if you change (first decrease, then increase) the relative error at each point? Do it and comment on the plots.*
- *What happens if you comment the definition of `y_data_pts` that uses the exact data from the paper and instead generate the noise randomly. Does the pattern of the evidence change? Does the magnitude of the evidence change?* **Note: in the current version the exact data has already been commented out.**

Notes

- The simple expression for $\hat{a}$ here, which minimizes χ^2 (or, equivalently, maximizes the likelihood), analytically reproduces the results we worked hard for earlier to get by sampling. The point of the latter exercise was to illustrate in a checkable problem how to do sampling, not because it was required in this case.
- Show that you get the same results.
- *What can you say about the sampling (e.g., accuracy)?*

17.2. Problem: Model selection basics

For this problem, your task is simply to work through this notebook and answer the questions, modifying the code as needed.

Bayesian evidence:

Please see the notes on model selection for background and the discussion of “The story of Dr. A and Prof. B from Chapter 4 of Sivia’s book. There is also a summary of Laplace’s method for approximating evidence factors.

Import of modules

```
%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set("talk")  # for plot formatting

from scipy import optimize
```

What order polynomial?

Throughout the rest of this section, we will use data that was generated from a “true model” where x and y satisfy the following:

$$y_i = x_i \sin(x_i) + \epsilon_i,$$

where $0 \leq x_i \leq 3$ and the noise is drawn from a normal distribution $\epsilon_i \sim \mathcal{N}(0, \sigma_0)$. The values for 20 regularly spaced points with $\sigma_0 = 0.1$ are shown below.

```
#------------------------------------------------------------
# Define our functional form
def true_func(x):
    """The true function behind the data."""
    return np.sin(x) * x

def func(x, dy):
    """Returns the true function evaluated at x with Gaussian noise
        with standard deviation dy.
    """
    return np.random.normal(true_func(x), dy)

#------------------------------------------------------------
# generate the (noisy) data
np.random.seed()  # Use np.random.seed() for new random numbers.
                  # Use np.random.seed(42) for repeatable random numbers.

num_data = 20           # number of data points
x_min = 0; x_max = 3.   # specify the data range

# Build x and y arrays with noisy data
x = np.linspace(x_min, x_max, num_data+2)[1:-1]  # exclude end points
sig0 = 0.1  # Later: try 0.5 or higher or 0.01
y = func(x, sig0)
```

```
# Plot the current data set
fig, ax = plt.subplots(1,1, figsize=(8,6))
ax.errorbar(x, y, sig0, fmt='o', color='blue', label='data');
ax.plot(x, true_func(x), color='red', label='true')
ax.set(xlabel='x',ylabel='y')
ax.legend()
fig.tight_layout()
```

Assume that we have a multiple of models, with varying degree of sophistication. In this example we use *polynomials* of different orders to represent models of increasing complexity and with an increasing number of model parameters.

> Our task is to find which model finds the most support in the *given* data.

It is clear that a more complicated model with more free parameters should be able to fit the data more closely. But what is the **evidence** in the data for such a complicated model? Finding the answer to this question is a task for a Bayesian, and the problem is generally known as *Model selection*.

Below, we will use an approximate way of computing the Bayesian evidence, namely the Laplace method. In some cases one can also use conjugate priors to simplify the computation of the evidence factor. Or one can use certain sampling methods to compute the evidence numerically (this is generally quite difficult; we will try it in Mini-project 2b). The highlight will be the comparison of different models using the evidences to extract odds-ratios.

The Model

See the text for more details on fitting polynomials.

In general, we're fitting a M-degree polynomial to data,

$$y_M(x) = \sum_{i=0}^{M} \theta_i x^i$$

where we use θ to denote our parameter vector of length $M+1$.

Assuming all the points are independent, we can find the full log likelihood by adding the individual likelihoods together:

$$\begin{aligned} \log p(D \mid \theta, I) &= -\frac{1}{2} \sum_{i=1}^{N} \left(\log(2\pi\sigma_0^2) + \frac{[y_i - y_M(x_i;\theta)]^2}{\sigma_0^2} \right) \\ &= \text{constant} \quad - \sum_{i=1}^{N} \frac{[y_i - y_M(x_i;\theta)]^2}{2\sigma_0^2} \end{aligned}$$

We often define the residuals

$$R_i = [y_i - y_M(x_i;\theta)]/\sigma_0,$$

so that the relevant chi-square sum reads $-\sum_{i=1}^{N} R_i^2/2$.

```
def residuals(theta, x=x, y=y, sigma0=sig0):
    """Calculate residuals using the numpy function polyval"""
    dy = y - np.polyval(theta, x)
    return dy / sigma0

def log_likelihood(theta):
    """ Standard likelihood with Gaussian errors as specified using defaults.
        Uniform prior for theta is implied.
    """
    return -0.5 * np.sum(residuals(theta)**2)
```

Maximum likelihood fits

We can maximize the likelihood to find θ within a frequentist paradigm. Let us start with a linear fit:

```
degree = 1
# The numpy polyfit function fits a least squares polynial to given data.
#  Using different weights at different x points is an option, but not used
#  here just because we have constant sigma.
theta_hat = np.polyfit(x, y, degree)

x_fit = np.linspace(0, x_max, 1000)
y_fit = np.polyval(theta_hat, x_fit)
```

Rather than just plotting this fit, we will compare several different models in the figure below.

```
def fit_degree_n(degree, ax):
    """Fit a polynomial of order 'degree', return the chi-squared,
       and plot in axes 'ax'.
    """
    theta_hat = np.polyfit(x, y, degree)
    x_fit = np.linspace(x_min, x_max, 1000)
    y_fit = np.polyval(theta_hat, x_fit)


    ax.errorbar(x, y, sig0, fmt='o', color='blue')  # data
    ax.plot(x_fit, true_func(x_fit), color='red', alpha=.5)  # true function

    # add an annotation for the degree of the polynomial fit to the data
    ax.text(0.03, 0.96, f"d = {degree}", transform=plt.gca().transAxes,
            ha='left', va='top',
            bbox=dict(ec='k', fc='w', pad=10))

    ax.plot(x_fit, y_fit, '-k')
    ax.set_xlabel('$x$')
    ax.set_ylabel('$y$');

    return -2. * log_likelihood(theta_hat)  # chi_squared

#------------------------------------------------------------
# First figure: plot points with a linear fit
nrows = 1; ncols = 3;
fig = plt.figure(figsize=(6*ncols, 6*nrows))  # scale figure space

num_plots = nrows * ncols
degrees = np.zeros(num_plots, dtype=int)
chi_sqs_dof = np.zeros(num_plots)
print('degree  chi^2/dof')
for i in range(num_plots):
    ax = fig.add_subplot(nrows, ncols, i+1)
    degrees[i] = i
    dof = len(x) - (degrees[i])
    chi_sqs_dof[i] = fit_degree_n(i, ax) / dof
    print(f'  {degrees[i]:d}     {chi_sqs_dof[i]:8.3f}')
fig.tight_layout()
```

Things to do and Questions to answer

- *Change the degree of the polynomial that is used for the fit so that the multi-panel figure is 3 by 3.*
- *Plot the computed chi-squared value per degree of freedom as a function of the degree of the polynomial. Is it decreasing, or is there a peak? How do you interpret the result?*
- *For which degree polynomials would you say that you're underfitting the data?*
- *For which degree polynomials would you say that you're overfitting the data?*

Bayesian evidence

Let us try the Bayesian approach and actually compute the evidence for these different models. We will use the Laplace method for computing the norm of the posterior distribution (i.e. approximating it as a single Gaussian).

We use simple uniform priors for the model parameters:

$$p(\theta_i|I) = \begin{cases} \frac{1}{\theta_{\max}-\theta_{\min}} & \text{for } \theta_{\min} \leq \theta_i \leq \theta_{\max}, \\ 0 & \text{otherwise} \end{cases},$$

which means that the posterior will be

$$p(\theta|D,I) = \frac{1}{(\theta_{\max}-\theta_{\min})^K} \frac{1}{\sqrt{(2\pi)\sigma_0^2}^N} \exp\left(-\chi^2/2\right),$$

within the allowed prior region for the K parameters and zero elsewhere.

Assuming that the peak of the Gaussian is located at θ^*, well inside the prior region; we can easily approximate the integral

$$Z_p = \int d^K\theta \, p(\theta|D,I),$$

using Laplace's method (see Model selection). With this particular choice of prior, and again under the assumption that the cut at the edges does not change the integral over the multidimensional integral, we get

$$Z_p \approx \frac{1}{(\theta_{\max}-\theta_{\min})^K} \exp\left(-\chi^2(\theta^*)/2\right) \frac{\sqrt{(2\pi)^K}}{\sqrt{\det(\Sigma^{-1})}},$$

where $\Sigma_{ij}^{-1} = \partial^2\chi^2/\partial\theta_i\partial\theta_j$ (i.e. the Hessian) evaluated at the maximum θ^*. We removed the constant factor $\sqrt{(2\pi)\sigma_0^2}^N$ since it will be the same for all models.

Note that for this linear regression problem we can get all these quantities (θ^*, Σ) via linear algebra. See, e.g., the Jupyter Book Follow-up to EFT evidence or Hogg's nice paper: Data analysis recipes: Fitting a model to data. Below, we will use `numpy.polyfit` to extract the relevant quantities.

```
# We use a uniform prior for all parameters in [theta_min, theta_max]
theta_max = 1000
theta_min = -1000
prior_range = theta_max - theta_min
```

```
degree_max = 8
evidence = np.zeros(degree_max+1)
print("Degree  P*          Best fit parameters: ")
for ideg,deg in enumerate(range(degree_max+1)):
    theta_hat, Cov = np.polyfit(x, y, deg, cov='unscaled')
    if not (np.all(theta_hat < theta_max) and np.all(theta_hat > theta_min)):
        print("Outside of prior range")
    P_star = np.exp(log_likelihood(theta_hat))
    H = np.linalg.inv(Cov)
    evidence[ideg] = P_star * np.sqrt((2*np.pi)**deg / np.linalg.det(H)) / prior_range**deg
    print (f'   {deg}    {P_star:.2e}  ',('{:5.2f} '*len(theta_hat)).format(*theta_hat))
```

```
# plot the evidence

d = np.arange(0, degree_max+1)

fig,ax = plt.subplots(figsize=(8, 6))
ax.plot(d,evidence,'o-')

ax.set_xlabel('polynomial degree')
ax.set_ylabel('evidence');

fig,ax = plt.subplots(figsize=(8, 6))
ax.semilogy(d,evidence,'o-')

ax.set_xlabel('polynomial degree')
ax.set_ylabel('evidence');
```

Explorations: Things to do and Questions to answer

- *Can you see the transition from underfit to overfit in the figures?*
- *What would you say is the degree of polynomial that is supported by the data?*
- *Describe the results after you change the number of the generated data? E.g., try* `num_data` *= 10.*
- *Describe the results after you change the range of the generated data? E.g., try* `x_max` *= 1.*
- *Describe the results after you change the error of the generated data? E.g., try* `sig0` *= 0.5.*

We see sensitivity to the prior in the peak of the evidence with the degree of the polynomial in our model. What about the scale-invariant prior advocated by Minka? Another possibility advanced by Trotta (Bayes in the sky) is a multi-normal Gaussian distribution with zero mean and Fisher information matrix as covariance. These are worth trying (but you don't need to do them for Mini-Project IIa).

Odds-ratios

Quoting Table 1 from the well-known paper by Trotta: Bayes in the sky: Bayesian inference and model selection in cosmology we can quantify an empirical scale for evaluating the strength of evidence when comparing two models:

Roberto Trotta

Table 1. Empirical scale for evaluating the strength of evidence when comparing two models, $\mathcal{M}_0$ versus $\mathcal{M}_1$ (so–called "Jeffreys' scale"). Threshold values are empirically set, and they occur for values of the logarithm of the Bayes factor of $|\ln B_{01}| = 1.0$, 2.5 and 5.0. The right–most column gives our convention for denoting the different levels of evidence above these thresholds. The probability column refers to the posterior probability of the favoured model, assuming non–committal priors on the two competing models, i.e. $p(\mathcal{M}_0) = p(\mathcal{M}_1) = 1/2$ and that the two models exhaust the model space, $p(\mathcal{M}_0|d) + p(\mathcal{M}_1|d) = 1$.

$\lvert\ln B_{01}\rvert$	Odds	Probability	Strength of evidence
< 1.0	$\lesssim 3:1$	< 0.750	Inconclusive
1.0	$\sim 3:1$	0.750	Weak evidence
2.5	$\sim 12:1$	0.923	Moderate evidence
5.0	$\sim 150:1$	0.993	Strong evidence

Here, the ratio of the evidences of model M_0 and M_1 is given by,

$$B_{01} = \frac{p(\text{data}|M_0)}{p(\text{data}|M_1)} \,,$$

which is also called *Bayes factor*. That means $|\ln B_{01}| \equiv |\ln p(\text{data}|M_0) - \ln p(\text{data}|M_1)|$ is the relevant quantity for estimating the strength of evidence of the two models (see first and last column of the table).

Question

- *Below we have created a table of odds-ratios between pairs of the different-order polynomial models, given that the ratio of prior probabilities for the different models is unity. Interpret the table based on Trotta's chart.*

```
# Odds ratio table
print('Calculating the logarithm of the Bayes ratio of degree n to n+1')
print('  n   n+1   log(Bn,n+1)')
for ideg,deg in enumerate(range(degree_max)):
    odds_ratio = evidence[ideg] / evidence[ideg+1]
    print(f'  {deg}   {deg+1}      {(np.log(odds_ratio)):.2f}')
```

Cross validation: do this section for full points

This section will introduce the frequentist tool of cross validation (CV). This approach is used extensively within machine-learning as a way to handle overfitting and underfitting, bias and variance.

```
# Select the cross-validation points
ncross=5
index_cv = np.random.choice(range(len(x)), ncross, replace=False)

x_cv=x[index_cv]
y_cv=y[index_cv]
```

```
# The training data is then
x_train = np.delete(x,index_cv)
y_train = np.delete(y,index_cv)
```

```
# Plot training and CV errors as a function of polynomial degree d
degree_max = 13
d = np.arange(0, degree_max+1)
training_err = np.zeros(d.shape)
crossval_err = np.zeros(d.shape)

fig,ax = plt.subplots(figsize=(8, 6))
for i in range(len(d)):
    p = np.polyfit(x_train, y_train, d[i])
    training_err[i] = np.sqrt(np.sum((np.polyval(p, x_train) - y_train) ** 2)
                              / len(y_train))
    crossval_err[i] = np.sqrt(np.sum((np.polyval(p, x_cv) - y_cv) ** 2)
                              / len(y_cv))

ax.plot(d, crossval_err, '--k', label='cross-validation')
ax.plot(d, training_err, '-k', label='training')
ax.plot(d, sig0 * np.ones(d.shape), ':k')

ax.set_xlim(0, degree_max)
# You might need to change the y-scale if you make modifications to the training data
ax.set_ylim(0, 0.8)

ax.set_xlabel('polynomial degree')
ax.set_ylabel('rms error')
ax.legend(loc='best');
```

Answer these questions

- *Can you see the transition from underfit to overfit in this figure?*
- *What would you say is the degree of polynomial that is supported by the data?*
- *Try changing the size of the cross-validation and training sets. Does the conclusions become more/less clear?*
- *Do the results change between different runs with the same number of CV samples? If so, why?*
- *K-fold cross validation is a popular variant of CV. It addresses some issues with the sensitivity to the actual choice of which data is used for training and validation. What do you think that it means, and what is the possible drawback if you have a computational expensive model?*

- *Leave-one-out is another variant. For linear regression problems, this type of cross-validation can actually be performed without having to do multiple fits. What do you think that it means?*

- *It is common to divide the data into a training set, a cross-validation set, and a test set. What do you think is the purpose of having three different sets?*

17.3. Problem: How many lines are there?

The overall problem goal is to determine how many signal peaks there are in a noisy set of data. We want to use parameter estimation and model selection (via parallel tempering) to determine what we can about the peaks. Model selection addresses how many peaks while parameter estimation addresses what are the positions and amplitudes of the peaks.

Learning goals:

- Use advanced Monte Carlo sampling to generate posterior probability distributions and analyze the results.
- Compute an evidence ratio and explain what it means.
- Employ Bayesian evidence for model selection in the context of a prototype for an experimental spectral measurement.

A spectral line problem

See e.g. section 4.2 in Sivia for a similar problem formulation. In short, we have data from a spectroscopy experiment that supposedly shows a number of spectral lines. The ideal spectrum, as a function of a scaled coordinate x, can be expressed as

$$G(x) = \sum_{j=1}^{M} A_j f(x, x_j),$$

where A_j is the amplitude of the jth line, and x_j represents its position. If all the spectral lines were Gaussians of width W, for example, then

$$f(x, x_j) = \frac{1}{\sqrt{2\pi W^2}} \exp\left[-\frac{(x - x_j)^2}{2W^2} \right]$$

The measurement also includes a background signal which we take to be constant $B(x) = B$.

We use the label 'k' to enumerate the bins $\{x_k\}$ (do not confuse x_k and x_j). The spectrum according to our model is therefore

$$F_k \equiv F(x_k) = G(x_k) + B.$$

The experimental data is denoted $\{D_k\}$. This data also includes measurement errors ε that are assumed to be independent and identically distributed (IID) normal with the same variance $\sigma_{\rm exp}$. The measured data is then related to the ideal spectrum by

$$D_k \equiv D(x_k) = G(x_k) + B + \varepsilon.$$

The task is to infer how many spectral lines (M) are in the experimental data, and their positions (x_j) and amplitudes (A_j). The magnitude of the background is not known before the analysis except that it must be ≤ 1.

Using scaled parameters the range of peak positions is $[0, 1]$, and the range of amplitudes as well as background is also $[0, 1]$.

Solution strategy:

Our model with M lines has $2M + 1$ model parameters that we denote by the vector $\vec{\alpha}$. These are the amplitudes, the positions, and the constant background. We order them as follows:

$$\vec{\alpha} = (A_0, x_0, A_1, x_1, \ldots, B).$$

The background strength is a *nuisance parameter* in the sense that we're not really interested in its value, we just need to marginalize over it.

Subtasks (put your answers here):

For the first two, see Sivia 4.2 if you get stuck.

1. *Formulate the problem of how many lines and what are the model parameters in Bayesian language.*

2. *Using the expressions from step 1, derive the approximate posterior probability:*

$$p(M|\{D_k\}, I) \propto \frac{M!(4\pi)^M}{[x_{\rm max} - x_{\rm min}]^M [A_{\rm max} - A_{\rm min}]^M \sqrt{\det(\boldsymbol{\nabla}\boldsymbol{\nabla}\chi^2)}} \exp\left(-\frac{\chi^2_{\rm min}}{2}\right).$$

   Where is the Ockham factor?

3. **Bonus:** *Explain how you might compute this probability numerically using Python. For this imagined numerical implementation you should consider that some model parameters enter non-linearly in your likelihood. (Note: you do not need to implement this calculation.)*

4. *Generate data using the code snippet below, but plan to change values to explore the dependence of the analysis on the number and relative position of peaks, noise, background, etc. Note that the random seed is initialized to specific values in this notebook. You should start with this example, but should then run the data generator several times to observe the degree of fluctuation.*

5. *First let's see what happens with ordinary sampling to perform parameter estimation. Run the code below that uses emcee (our favorite MCMC sampler) for first one and then two signal peaks (`numpeaks = 1` then `2`). Comment on the posterior pdf (note that this is the full posterior now, not an approximation) in the two cases for the first corner plot and then the second. What is the difference between these plots? (Hint: what is the reason the first plot is multimodal when the model has two peaks?)*

6. *Run the parallel tempering code to compute the log evidence for a model with M peaks, looking at M from 1 to 4. Fill in your results in the table here. What are your conclusions about the number of peaks based on the evidence? Does the evidence have a clear maximum or does it saturate?*

numpeaks	log evidence
1	
2	
3	
4	

7. **For full points:** *Repeat the parallel tempering calculation and discussion but for data with three peaks and smaller width and noise.*

numpeaks	log evidence
1	
2	
3	
4	

Import statements

```
%matplotlib inline
import numpy as np
import scipy.stats, scipy

import matplotlib.pyplot as plt
import seaborn as sns
sns.set() # set default plot styles

import emcee
import ptemcee
import corner
print(f'Running on emcee v{emcee.__version__} and ptemcee v{ptemcee.__version__}')
```

Generate data

Parameters known before the analysis (explore different values for these as requested)

```
# Width W and noise sigma_exp
W = 0.12  # to try: switch to 0.10 # The width of the spectral lines
sigma_exp = 0.4  # to try: switch to 0.2 # Variance of IID experimental errors
```

```
# Fixed prior ranges (Don't change these)
xmin=0; xmax=1;
Amin=0; Amax=1;
Bmax=1.;
```

Parameters that should be learned from the data

```
np.random.seed(3) # Keep a definite number to reproduce the same data

# Number of lines
M = 2

# Amplitudes of the spectral lines
A0 = 10**np.random.uniform(low=-.5, high=0, size=M)

# Positions of the spectral lines
X0 = np.ones(M)*np.random.uniform(low=.1, high=.4, size=1)

# Special treatment to place the lines close together
for i in np.arange(1,M):
    X0[i] += np.random.normal(loc=.3,scale=.1)
    X0[i] = min(0.95,X0[i])
    X0[i] = max(0.05,X0[i])

B = np.random.uniform(0.05, 0.8) # A constant background signal

print("True values:   Peak amplitude     Peak position")
print("-----------    --------------     -------------")
for iM in range(M):
    print(f"     peak #{iM}             {A0[iM]:.2f}              {X0[iM]:.2f}  ")
print(f" Background:             {B:.2f}")
```

```
# Define a truths array that will be of length 2*M + 1
truths = np.empty((2*X0.size + 1,), dtype=X0.dtype)
truths[-1] = B
truths[0:-1:2] = A0
truths[1:-1:2] = X0

# Define a labels list with parameter names
labels = []
for iM in range(M):
    labels.append(f"$A_{iM}$")
    labels.append(f"$X_{iM}$")
labels.append(f"$B$")
```

```
def f(x,x0,w):
    '''Simple Gaussian line shape. Position x0, width w.'''
    return np.exp(- (x-x0)**2 / (2 * w**2)) / np.sqrt(2*np.pi*w**2)

def G(x,amplitudes,positions,width=W):
    '''Ideal spectrum composed of N spectral lines with input amplitudes
    and positions (arrays of length N), and a single, fixed width.'''
    ideal = np.zeros_like(x)
    for a,x0 in zip(amplitudes,positions):
        ideal += a * f(x,x0,width)
    return ideal
```

```
x=np.linspace(xmin,xmax,100)
plt.plot(x,G(x,A0,X0,W))
plt.xlabel('$x$ [x-unit]')
plt.ylabel('$y$ [y-unit]');
```

Assert:

1. unknown constant background ($B \leq 1$), unknown number of spectral lines with unknown amplitudes and positions.
2. known, natural width (W) of the spectral lines
3. known variance ($\sigma_{\rm exp}$) for the IID normal experimental errors
4. known and relevant interval: peak positions within [xmin=0, xmax=1] in position space, peak amplitudes within [Amin=0, Amax=1],

```
print(f'Natural width of spectral lines:                W = {W}')
print(f'Variance for IID normal exp errors:              s = {sigma_exp}')
print(f'Relevant range in position space:    [xmin, xmax] = [{xmin:.1f}, {xmax:.1f}]')
print(f'Relevant range for peak amplitudes:  [Amin, Amax] = [{Amin:.1f}, {Amax:.1f}]')
```

```
# Generate experimental data
np.random.seed(42) # For reproducibility.  Set 42 to None for different values.
Ndata = 100 # Number of data points
xk = np.linspace(xmin, xmax, Ndata)
sk=np.ones_like(xk) * sigma_exp # We assume that the variance is the same for all k
ek = np.random.normal(loc=0.0, scale=sigma_exp, size=Ndata)
Dk = G(xk,A0,X0,W) + B + ek
# Data saved in a file (but also printed below)
data = [xk,Dk,sk]
```

```
plt.errorbar(xk,Dk,yerr=sk,fmt='ok',ecolor='gray')
plt.xlabel('$x$ [x-unit]')
plt.ylabel('$y$ [y-unit]');
```

Parameter estimation with your favorite MCMC sampler (emcee here!)

We will use an MCMC to draw samples from the posterior in order to solve the model fitting problem. We will use the emcee package, which requires us to first define functions which compute the prior, likelihood, and posterior

under each model. The model is determined by the number of peaks, which in turn defines the dimension of the model (an amplitude and a location to determine for each peak, plus the background amplitude).

emcee

```
# Uniform priors for parameters

def log_prior(alpha):
    '''Logarithm of prior.

    Flat priors over predetermined ranges
    The size of alpha determines the model.
    Amplitudes are elements 0, 2, 4, ..., [0:-1:2]
    Positions are elements 1,3,5, ...     [1:-1:2]
    Background is the last element.    [-1]
    '''

    # All parameters must be positive
    if np.any(alpha < 0):
        return -np.inf  # log(0)
    # All amplitudes are assumed to be smaller than Amax
    elif np.any(alpha[0:-1:2] > Amax):
        return -np.inf
    # The positions are assumed to be in the [xmin, xmax] range
    elif np.any(alpha[1:-1:2] > xmax):
        return -np.inf
    # The background signal is assumed to be smaller than Bmax
    elif alpha[-1] > Bmax:
        return -np.inf
    else:
        return 0 # log(1), note that it is not properly normalized
```

```
def model_func(alpha, x=xk, width=W):
    '''Computes the model ideal spectrum Fk'''
    _amplitudes = alpha[0:-1:2]
    _positions = alpha[1:-1:2]
    _background = alpha[-1]
    return G(x,_amplitudes,_positions,width) + _background

def log_likelihood(alpha, data=data):
    '''Logarithm of likelihood function.

    Returns the negative sum of the squared residuals, which is
    equivalent to a chi-square measure or the logarithm of a
    gaussian likelihood function.
    '''
    x, y, sigma_y = data
    model_y = model_func(alpha,x=x)
    residuals = (model_y - y) / sigma_y
    return -0.5 * np.sum(residuals**2)
```

```
def log_posterior(alpha, data=data):
    '''Logarithm of posterior.'''
    _alpha = np.asarray(alpha)
    return log_prior(_alpha) + log_likelihood(_alpha, data=data)
```

```
numpeaks=1 # Number of peaks in the *model*. *** You set this ***
           #  Note how numpeaks determines ndim below.

nwalkers=50
nburn=1000
nsteps=5000

ndim = numpeaks*2 + 1  # this determines the model dimension

starting_guesses = (np.random.rand(nwalkers, ndim) * \
                    np.tile(np.ones(ndim), (nwalkers, 1)))
sampler = emcee.EnsembleSampler(nwalkers, ndim, log_posterior, args=[data])
sampler.run_mcmc(starting_guesses, nsteps)
trace_unordered = sampler.get_chain(discard=nburn, flat=True)
```

```
# First a corner plot without the ordered samples
figure = corner.corner(trace_unordered)
```

```
trace = np.copy(trace_unordered)

for i, sample in enumerate(trace_unordered):
    # Sort by proposed peak position since (x0,x1,...) is the same as (x1,x0,...)
    x_sort = np.argsort(sample[1:-1:2])*2+1
    sample_sort = np.empty((2*x_sort.size + 1,), dtype=x_sort.dtype)
    sample_sort[-1]=len(sample_sort)-1
    sample_sort[1:-1:2] = x_sort
    sample_sort[0:-1:2] = x_sort-1
    trace[i] = sample[sample_sort]
```

```
if numpeaks==2:
    truths_corner = truths
else:
    truths_corner = None

# Define a labels list with parameter names
labels_corner = []
for iM in range(numpeaks):
    labels_corner.append(f"$A_{iM}$")
    labels_corner.append(f"$X_{iM}$")
labels_corner.append(f"$B$")

figure = corner.corner(trace[:,:],labels=labels_corner,
                       quantiles=[0.16, 0.5, 0.84],truths=truths_corner,
                       show_titles=True, title_fmt='.3f', title_kwargs={"fontsize": 12})
```

```
params = np.percentile(trace_unordered, [16, 50, 84], axis=0)
with np.printoptions(precision=2):
    print(f"True values:", truths)
    print(f"       MAP:", params[1,:])
```

```
plt.errorbar(xk, Dk, yerr=sk, fmt='ok', ecolor='gray')
x = np.linspace(xmin,xmax,100)
plt.plot(x, G(x,params[1,0:-1:2], params[1,1:-1:2]) + params[1,-1])
plt.xlabel('$x$ [x-unit]')
plt.ylabel('$y$ [y-unit]');
```

Using parallel tempering: ptemcee

```
numpeaks = 1 # Number of peaks in the *model* (step through 1, 2, 3, 4)
             #  Note how this determines ndim below.
```

```
# Now we can construct a sampler object that will drive the PTMCMC;
# in the first pass we chose to use 21 temperatures
# (the default is for each temperature to increase by a factor of sqrt(2),
# so the highest temperature will be T=1024, resulting in an effective
# \sigma_T=32\sigma=3.2, which is about the separation of our modes).

#ntemps = 21
#temps = np.array([np.sqrt(2)**i for i in range(ntemps)])

# This did not work well, so we modified the temperature ladder to improve
#  the integration for evidence calculation:
#  need more low temperatures, i.e. finer grid near beta = 1.
ntemps_lo = 8
ntemps_hi = 21
temps_lo = np.array([2**(i/8.) for i in range(ntemps_lo)])
temps_hi = np.array([np.sqrt(2)**i for i in range(ntemps_hi)])
temps = np.concatenate((temps_lo,temps_hi[temps_hi>max(temps_lo)]))
ntemps=len(temps)

betas=1/temps

# Let us use 10 walkers in the ensemble at each temperature:
ndim = numpeaks*2 + 1  # this determines the model dimension
nwalkers = max(10,ndim*2+2)

nburnin=1000
nsteps=5000
nthin = 10 # only record every nthin iteration

nthreads = 1
```

```
p0 = np.random.uniform(size=(ntemps, nwalkers, ndim))
```

```
sampler = ptemcee.Sampler(nwalkers, ndim, log_likelihood, log_prior, ntemps,
                          threads=nthreads, betas=betas)
```

```
#sampler=PTSampler(ntemps, nwalkers, ndim, log_likelihood, log_prior,
#                  threads=nthreads, betas=betas)

#Making the sampling multi-threaded is as simple as adding the threads=Nthreads
# argument to PTSampler. We could have modified the temperature ladder using the
# betas optional argument (which should be an array of \beta=1/T values).

#First, we run the sampler for 1000 burn-in iterations:
print("Running burn-in phase")
for p, lnprob, lnlike in sampler.sample(p0, iterations=nburnin):
    pass
sampler.reset()

print("Running MCMC chains")
#Now we sample for nwalkers*niterations, recording every nthin-th sample:

for p, lnprob, lnlike in sampler.sample(p, iterations=nsteps, thin=nthin):
    pass
```

```
pt_sampler_T0 = sampler.chain[0,...].reshape(-1,ndim)
figure = corner.corner(pt_sampler_T0)

# Extract the axes
axes = np.array(figure.axes).reshape((ndim, ndim))
```

Perform thermodynamic integration from PT sampler

```
# From PT sampler
(lnZ, dlnZ) = sampler.log_evidence_estimate()
print(f"The log evidence is {lnZ:.3f} +/- {dlnZ:.3f}")
print("NOTE: This thermodynamic integration output from ptemcee",\
      "may not be very reliable due to poor numerical integration.")
print("Better do it yourself to check (see below).")
```

```
# Array with beta values (1/T)
betas = sampler.betas
avg_lnl = np.zeros_like(betas) # averages of log likelihood for different beta
var_lnl = np.zeros_like(betas) # variances of log likelihood for different beta
for ib,b in enumerate(betas):
    avg_lnl[ib] = np.mean(sampler.loglikelihood[ib,...].reshape(-1))
    var_lnl[ib] = np.var(sampler.loglikelihood[ib,...].reshape(-1))

betas0 = np.concatenate((sampler.betas, np.array([0])))
dbetas = np.diff(betas0)
```

```
plt.plot(betas,avg_lnl,'ko-')
ax=plt.gca()
ymin=np.ones_like(betas)*min(avg_lnl)
ax.fill_between(betas, ymin,avg_lnl,facecolor='green', alpha=0.1)
fig=plt.gcf()
#ax.set_ylim([-100,-30]);
ax.set_xlim([0.,1.05]);
ax.set_xlabel(r'$\beta$');
ax.set_ylabel(r'$\ln\left(Z(\beta)\right)$');
```

```
print("Integrate using trapezoid approximation with N={} points".format(ntemps))
print(f"from {betas0[0]:.4e} to {betas0[-1]:.4e}")
lnzl = -np.trapz(avg_lnl, x=betas)
print(f"\nWe find log-evidence = {lnzl:7.4f}")
```

18. Overview of Part III: MCMC sampling

> “Any one who considers arithmetical methods of producing random digits is, of course, in a state of sin.”

—John von Neumann (1951)

We have already seem examples of the sampling of PDFs in previous chapters. Here we look in depth at Markov chain Monte Carlo (MCMC), which is the workhorse of sampling methods. We will give an overview of both the theory and practice, considering first the Random Walk Metropolis-Hastings algorithm and then other more efficient samplers.

Chapters in this part:

- Intuition for MCMC gives a general motivation for MCMC sampling, builds intuition through visualizations and analogies to statistical mechanics, introduces the Metropolis-Hastings algorithm, and provides some basic examples, such as an application to Poisson processes.
- Details of MCMC provides formal and detailed discussion on stochastic processes in general, Markov chains, and Metropolis-Hastings MCMC.
- Markov Chain Monte Carlo in practice looks at MCMC in practice, with convergence tests and other diagnostics.
- Advanced sampling algorithms introduces a handful of useful and more advanced sampling algorithms such as Hamiltonian Monte Carlo (HMC), ensamble and slice sampling, parallel tempering, importance resampling, and nested sampling.
- State-of-the-art MCMC implementations provides several demo notebooks for selected state-of-the-art implementations of advanced sampling algorithms.

19. Intuition for MCMC

Intuition for Markov Chain Monte Carlo (MCMC) includes:

- Why MCMC?, which identifies problems in Bayesian inference where MCMC is useful or necessary; addresses multi-dimensional integration in general and then using Monte Carlo specfically; discusses sampling from a PDF; and provides a lead-in to MCMC.
- The Metropolis–Hastings algorithm, lays out the Metropolis-Hastings algorithm; provides intuition for detailed balance and why the MH algorithm satisfies the conditions; and provides a visual demonstration that a possibly non-intuitive feature of MH, namely repeating configurations in the MC chain when a proposal is rejected, is actually essential.

- Visualizations of MCMC points to online javascript simulations that are invaluable for building intuition about how MCMC works (here for MH but later for Hamiltonian MC) and highlights some of the features from the simulations.
- Demo: Sampling of a Poisson distribution provides a full MH MCMC example to play with, prompted by a series of question.
- Demo: Random walk and sampling is another example to build intuition about MH sampling; autocorrelation is introduced.

There will be lead-ins in various places in this chapter to more details presented in the subsequent chapters.

19.1. Why MCMC?

General problems in Bayesian inference

We have been emphasizing that in the Bayesian approach, (almost) everything is a PDF. Given a PDF, we might be interested in:

1. The exploration of the PDF;
    - Where is the probability most concentrated and where is there low probability?
    - Are there correlations between parameters?
    - Are there multiple modes (local maxima in the PDF)?
    - What do the *marginal* PDFs look like? (I.e., we have a situation where we want to integrate (marginalize) over a subset of parameters $\boldsymbol{\theta}_B$ to find a posterior for the rest $\boldsymbol{\theta}_A$. That is, $\boldsymbol{\theta} = \{\boldsymbol{\theta}_A, \boldsymbol{\theta}_B\}$. For example, marginalizing over parameters for the width of a signal and other parameters characterizing our model for the Higgs mass.)
2. The exploration of posterior predictions with a function $g(\boldsymbol{\theta})$;
    - Finding the **posterior predictive distribution**

$$\{g(\boldsymbol{\theta}) \ : \ \boldsymbol{\theta} \sim p(\boldsymbol{\theta})\} \tag{19.1}$$

      (Recall that this notation means “the distribution of $g(\boldsymbol{\theta})$ given the distribution of $\boldsymbol{\theta}$”.)

    - The evaluation of **expectation values** (as discussed in Expectation values and moments)

$$\mathbb{E}\left[g(\boldsymbol{\theta})\right] = \int d\boldsymbol{\theta}\, g(\boldsymbol{\theta}) p(\boldsymbol{\theta}) \tag{19.2}$$

The first set of tasks can be addressed with a method to collect a set of samples $\{\boldsymbol{\theta}_i\}_{i=1}^N$, where the samples are distributed according to $p(\boldsymbol{\theta})$. The process of generating a random sample according to a probability distribution is commonly called a “draw”. Such samples can then be histogrammed and plotted to provide an approximate representation of $p(\boldsymbol{\theta})$.

In particular, it should be noted that marginal distributions can easily be obtained from these samples by ignoring the values of parameters that we are integrating over. For example, the marginal distribution $p(\theta_1, \theta_2)$ of $p(\boldsymbol{\theta})$ with $\boldsymbol{\theta}$ a D-dimensional vector is represented by the samples

$$\{(\theta_1, \theta_2)_i\}_{i=1}^N,$$

where $(\theta_1, \theta_2)_i$ are the first two elements of the $\boldsymbol{\theta}_i$ sample. This procedure of ignoring the other parameter values can be understood since the process of marginalization is basically a projection of the multi-dimensional PDF onto the chosen axes.

The computation of multi-dimensional integrals such as in Eq. (19.2) is very challenging in high dimensions, but high-dimensional integrals appear frequently in science and applied mathematics. They are only analytically solvable in some special cases and it is therefore an important and general problem to be able to solve. It turns out that the ability to draw samples from $p(\boldsymbol{\theta})$ will make it possible to compute very efficient, numerical approximations of such integrals also in very high dimensions.

Numerical integration

There are several methods to perform numerical integration. Quadrature is the method of choice for small numbers of dimensions, while Monte Carlo integration becomes very powerful in high dimensions.

Quadrature methods

For small dimensions, classical numerical integration using quadrature is often the method of choice. For example, the one-dimensional integral

$$I = \int d\theta \, f(\theta), \tag{19.3}$$

is approximated by a weighted average of the function f evaluated at a number of quadrature points $\{\theta_i\}_{i=1}^N$

$$I \approx \sum_{i=1}^{N} w_i f(\theta_i), \tag{19.4}$$

where the different quadrature schemes are distinguished by using different sets of design points and weights. Familiar examples of such schemes are the trapezoid rule, Simpson's rule, and Gaussian quadrature (the former two rely on evenly space quadrature points while the latter exploits the freedom to choose the spacing).

We note, however, that the number of design points that is required for a fixed precision grows exponentially with dimension. Assuming an integration scheme with N design points in one dimension, the same coverage in D dimensions requires N^D design points, which will be impractically large unless D is sufficiently small. This exponential increase in the number of required function evaluations with the dimensionality of the problem is often called the "curse of dimensionality" and is the main motivation for Monte Carlo integration.

Monte Carlo integration

The most straightforward implementation of Monte Carlo integration provides an estimate of the D-dimensional integral

$$I = \int_{V_D} d\boldsymbol{\theta} f(\boldsymbol{\theta}) \approx I_N \equiv \frac{V_D}{N} \sum_{i=1}^{N} f(\boldsymbol{\theta}_i), \tag{19.5}$$

where $\{\boldsymbol{\theta}_i\}_{i=1}^N$ are N uniformly distributed points in the D dimensional volume V_D of the parameter space. (By "uniformly distributed" here we don't mean equally spaced, but distributed randomly according to a uniform distribution.) The convergence of this expression will not be proven here, but is a consequence of the law of large numbers. Furthermore, one can show that the error in this approximation (usually) decreases as $1/\sqrt{N}$ regardless of the dimensionality D, which is the main reason for its usefulness when facing complicated, high-dimensional integrals. To be specific, the variance of I_N can be estimated by the sample variance

$$\sigma_N^2 = \frac{1}{N-1} \sum_{i=1}^{N} \left(f(\boldsymbol{\theta}_i) - \langle f \rangle\right)^2, \tag{19.6}$$

where $\langle f \rangle$ is the sample mean, and becomes

$$\text{Var}\,(I_N) = V_D^2 \frac{\sigma_N^2}{N}. \tag{19.7}$$

Caution

$\boldsymbol{\theta}_i$ is the i^{th} set of $\boldsymbol{\theta}$ values, not a component of $\boldsymbol{\theta}$.

Intuition: naive MC in one dimension

In one dimension, the average value of a function g on $[a, b]$ is the number

$$\overline{g} = \frac{1}{b-a} \int_a^b g(\theta)\, d\theta.$$

We can *estimate* $\overline{g}$ by averaging g over N samples $\theta_i \sim \mathcal{U}(a, b)$, i.e., drawn uniformly on $[a, b]$. The integral then becomes

$$I = \int_a^b g(\theta)\, d\theta = (b-a)\, \overline{g} \approx \frac{b-a}{N} \sum_{i=1}^{N} g(\theta_i),$$

which is Eq. (19.5) with "volume" $V_D = b - a$ in $D = 1$ dimension.

Since $1/(b-a)$ is the uniform PDF on $[a, b]$, $\overline{g} = \mathbb{E}_{\mathcal{U}}\left[g(\theta)\right]$ is an expectation value. The same idea therefore works for any PDF $p(\theta)$ that we can draw samples $\theta_i \sim p$ from:

$$\mathbb{E}_p\left[f(\theta)\right] = \int f(\theta)\, p(\theta)\, d\theta \approx \frac{1}{N}\sum_{i=1}^{N} f(\theta_i).$$

There is no volume factor here because p is normalized; in the uniform case $p(\theta) = 1/(b-a) = 1/V_D$. Drawing samples from a general p is what MCMC will do for us.

Does this always work? No! The error decreases as $1/\sqrt{N}$ (central limit theorem) only if f has a finite variance under the sampling distribution, Var $_p\left(f(\theta)\right) < \infty$. This fails for Cauchy-like tails. In the radioactive lighthouse problem (📥 Problem: Radioactive lighthouse) the flash positions have neither a finite mean nor a finite variance, so their sample average does not converge to the lighthouse position.

Example 19.1 (Monte Carlo estimation of π)

A standard example of Monte Carlo integration is the estimation of π using the (circular) step function

$$f(x,y) = \begin{cases} 1 & \text{if } x^2 + y^2 \leq 1 \\ 0 & \text{otherwise} \end{cases}, \tag{19.8}$$

such that the integral

$$I = \int_{-1}^{1} dx \int_{-1}^{1} dy\, f(x,y) = \pi. \tag{19.9}$$

One can estimate I using N samples $\{(x_i, y_i)\}_{i=1}^{N}$ from the bivariate, uniform distribution $\mathcal{U}\left([-1,1]^2\right)$ such that

$$I \approx I_N = \frac{4}{N}\sum_{i=1}^{N} f(x_i, y_i), \tag{19.10}$$

according to Eq. (19.5). In the code cell below we implement this Monte Carlo integration using Python and show that the (relative) error of this numerical simulation of π indeed decreases as $1/\sqrt{N}$.

```
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

num_samples=[int(10**i) for i in np.linspace(1,5,50)]
I_num = []
I_rel_error = []
for num in num_samples:
    max_I_num, max_err = 0., -1.
    # We will do ten simulations and find the largest error that is made
    for iexp in range(10):
        # `num` bivariate samples from a uniform distribution [-1,1]**2
        samples = stats.uniform.rvs(loc=-1,scale=2,size=(2,num))
        # Boolean array with elements that are true if the sample is inside the unit circle
        # This corresponds to the unit circle step function
        inside = samples[0,:]**2 + samples[1,:]**2 <= 1
        # We can evaluate the Monte Carlo sum
        iexp_I_num = 4.0 * np.sum(inside) / num
        # We keep the result with the largest error that we find in all experiments
        if np.abs(iexp_I_num-np.pi) > max_err:
            max_err = np.abs(iexp_I_num-np.pi)
            max_I_num = iexp_I_num

    I_num.append(max_I_num)
    # And the (absolute value) of the relative error
    I_rel_error.append(np.abs(I_num[-1]-np.pi)/np.pi)

num_samples = np.array(num_samples)
I_rel_error = np.array(I_rel_error)

fig, ax = plt.subplots(nrows=1, ncols=1, **{"figsize":(5,4)})
ax.loglog(num_samples, I_rel_error, 'bo',label=r'$(I_N-\pi) / \pi$')
ax.loglog(num_samples, 1/np.sqrt(num_samples),'r--',label=r'$1/\sqrt{N}$')
ax.set_xlabel(r'$N$')
ax.set_ylabel('rel. error')
ax.legend(loc='best');

from myst_nb import glue
glue("MC-integration_fig", fig, display=False)
```

▼ Hide code cell output

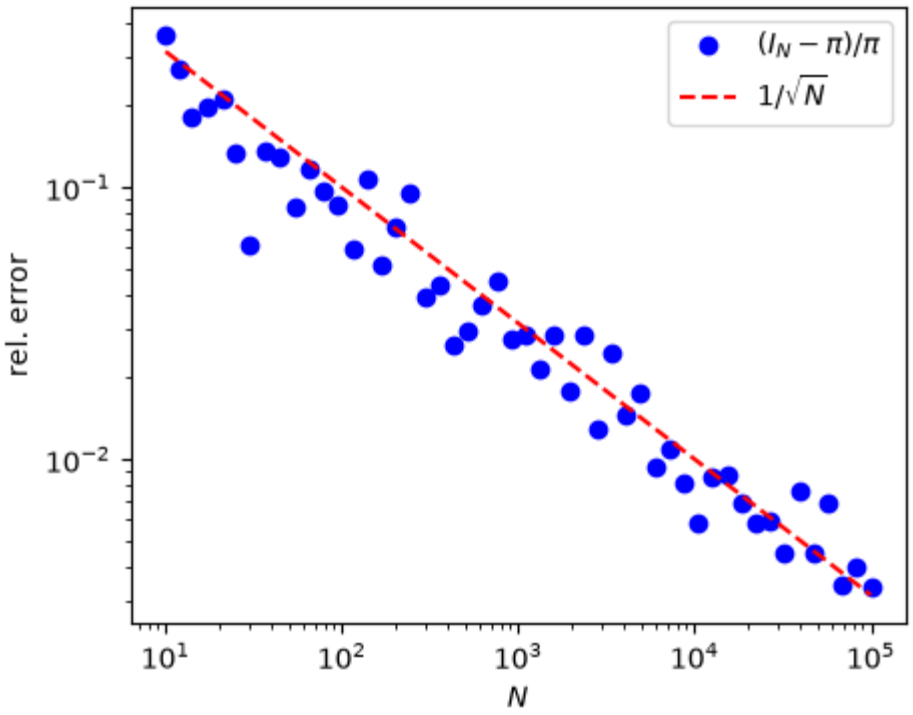


Fig. 19.1 Relative error of the Monte Carlo integration (19.10) as a function of the number of samples, demonstrating the $1/\sqrt{N}$ scaling.

Despite the favorable $1/\sqrt{N}$-scaling, the problem of straight-forward Monte Carlo integration is that you will often evaluate the integrand at locations where it is small. This affects the precision of the approximation due to the volume factor and because the sample variance is large. It would be more beneficial to focus the sampling to regions that contribute significantly to the integral.

Monte Carlo integration in statistics

Indeed, it turns out that significantly better precision can be achieved for integrals such as (19.2), that appear in statistical computations. Given that we have a method to collect samples $\{\boldsymbol{\theta}_i\}_{i=1}^N$ from $p(\boldsymbol{\theta})$, we find that

$$I = \int_{V_D} d\boldsymbol{\theta}\, g(\boldsymbol{\theta}) p(\boldsymbol{\theta}) \approx I_N \equiv \frac{1}{N} \sum_{i=1}^{N} g(\boldsymbol{\theta}_i), \tag{19.11}$$

where you should note that the volume factor has disappeared and that you only evaluate $g(\boldsymbol{\theta})$ at the sampled positions where the PDF $p(\boldsymbol{\theta})$ is usually large.

Sampling from a PDF

We have seen that the ability to draw samples from a probability distribution is very useful. If $p(\boldsymbol{\theta})$ is of a standard form then it is straightforward to sample from it using available algorithms, most of which are based on nonlinear transformations of uniformly distributed random numbers. For example, a set of samples $\{u_i\}_{i=1}^N$ from the one-

dimensional standard uniform distribution $\mathcal{U}([0,1])$ can be transformed to a set of samples $\{\theta_i\}_{i=1}^N$ from the standard Gaussian distribution $\mathcal{N}(0,1)$ via the transformation

$$\theta_i = F_{\mathcal{N}}^{-1}(u_i), \tag{19.12}$$

where $F_{\mathcal{N}}^{-1}$ is the *inverse* distribution function of the standard normal distribution. That is, we interpret u_i as a probability sample and use the above transformation to find the solution to $u_i = F(\theta_i)$.

This works well for many standard distributions. However, for the generation of random samples from nonstandard, arbitrary distributions (such as the ones that we might encounter in a realistic Bayesian analysis), standard algorithms (such as the inverse transformation above) are usually not applicable. This is where custom-built Markov chains enter.

Markov Chain Monte Carlo (MCMC)

The key problem with the naive MC integration approach is that too much time is wasted sampling regions where $p(\boldsymbol{\theta}|D,I)$ is very small. If, for one parameter, the fraction of significant strength is 10^{-1}, in an M-parameter problem the fraction of the volume is 10^{-M}, which gets very small rapidly with increasing M. This necessitates *importance sampling*, which reweighs the integrand to more appropriately distribution points. This is accomplished by the adaptive Monte Carlo integration program VEGAS, but it is difficult to accomplish in general.

The bottom line is that it is not feasible to draw a series of independent random samples from $p(\boldsymbol{\theta}|D,I)$ for larger sizes of $\boldsymbol{\theta}$. Remember, independent means if $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \ldots$ is the series, knowing $\boldsymbol{\theta}_i$ doesn't tell us anything about $\boldsymbol{\theta}_{i+1}$ (or any other $\boldsymbol{\theta}$). *But consecutive samples don't need to be independent.* We just need to generate $p(\boldsymbol{\theta}|D,I)$ in the correct proportions; e.g., so it approximates $p(\boldsymbol{\theta}|D,I)$ when the samples are histogrammed.

Solution: Do a *random walk* (diffusion) in the parameter space of $\boldsymbol{\theta}$, so that the probability for being in a region is proportional to $p(\boldsymbol{\theta}|D,I)$ for that region. The basic plan is:

- $\boldsymbol{\theta}_{i+1}$ follows from $\boldsymbol{\theta}_i$ by a transition probability ("kernel") $\Longrightarrow p(\boldsymbol{\theta}_{i+1}|\boldsymbol{\theta}_i)$.
- The transition probability is assumed to be "time independent", so the same $p(\boldsymbol{\theta}_{i+1}|\boldsymbol{\theta}_i)$ holds no matter when you do it $\Longrightarrow$ this is a *Markov chain* and the method is called Markov Chain Monte Carlo or MCMC.
- Once we have a representative set of N vectors $\{\boldsymbol{\theta}_i\}$, then any expectation value of a function f of $\boldsymbol{\theta}$, which is the integral of $f(\boldsymbol{\theta})p(\boldsymbol{\theta}|D,I)$ over $\boldsymbol{\theta}$, is given simply by the average $\langle f\rangle = \frac{1}{N}\sum_i f(\boldsymbol{\theta}_i)$.

We can think of the sampled distribution as a sum of delta functions, whose normalization is trivial:

$$p(\boldsymbol{\theta}|D,I) \to \frac{1}{N}\sum_{i=1}^{N}\delta(\boldsymbol{\theta}-\boldsymbol{\theta}_i)$$

Checkpoint exercise

Check that with the delta functions we reproduce normalization of p and the rule for $\langle f\rangle$.

Answer

$$\int d\boldsymbol{\theta}\, p(\boldsymbol{\theta}|D,I) \to \frac{1}{N}\sum_{i=1}^{N}\int d\boldsymbol{\theta}\, \delta(\boldsymbol{\theta}-\boldsymbol{\theta}_i) = \frac{1}{N}\sum_{i=1}^{N} 1 = 1,$$

$$\langle f\rangle = \int d\boldsymbol{\theta}\, f(\boldsymbol{\theta}) p(\boldsymbol{\theta}|D,I) \to \frac{1}{N}\sum_{i=1}^{N}\int d\boldsymbol{\theta}\, f(\boldsymbol{\theta})\delta(\boldsymbol{\theta}-\boldsymbol{\theta}_i) = \frac{1}{N}\sum_{i=1}^{N} f(\boldsymbol{\theta}_i).$$

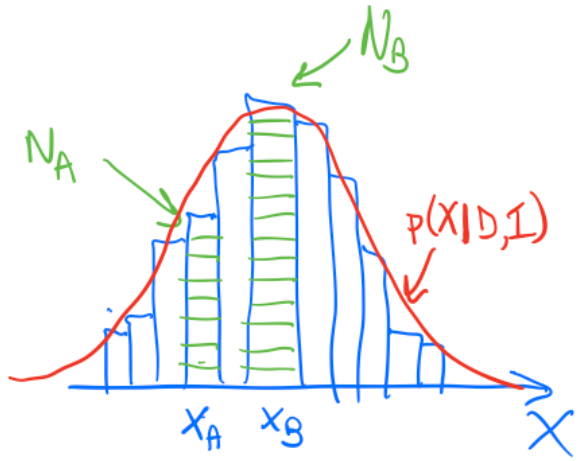
Intuition for integration

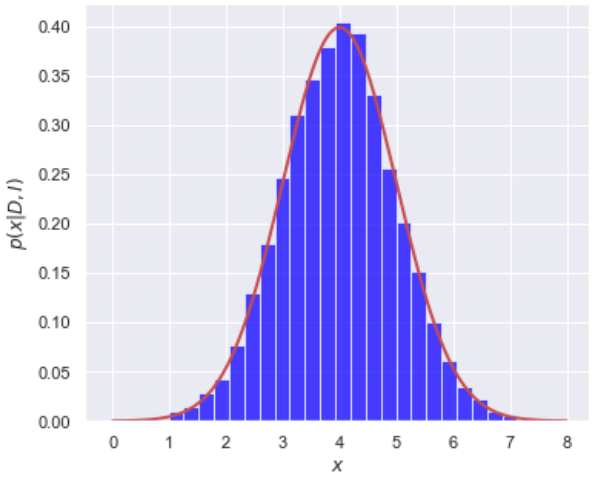
Here we compare two classes of integration methods: 1. Approximation by Riemann sum (or trapezoid rule). 1. Approximation using MCMC samples.

Suppose we want to calculate the expectation value of a function $f(x)$ where the distribution of x is $p(x|D,I)$ and we will restrict the domain of x to the positive real axis. So

$$\langle f\rangle = \int_0^\infty f(x)p(x|D,I)dx \tag{19.13}$$

and we'll imagine $p(x|D,I)$ is like the red lines in the figures here.





In method 1, we divide the x axis into bins of width δx, let $x_i = i \cdot \Delta x$, $i = 0, \ldots, N-1$. Then

$$\langle f \rangle \approx \sum_{i=0}^{N-1} f(x_i)\, p(x_i|D,I)\, \Delta x.$$

The histogram bar height tells us the strength of $p(x_i|D,I)$ and we have a similar discretization for $f(x)$.

In method 2, we *sample* $p(x|D,I)$ to obtain a representative set of x values, which we'll denote by $\{x_j\}$, $j = 0, \ldots, N'-1$. If I histogram $\{x_j\}$ it looks like the posterior, as seen in the figures. To approximate the expectation value, we simply use:

$$\langle f \rangle \approx \frac{1}{N'} \sum_{j=0}^{N'-1} f(x_j).$$

In method 1, the function $p(x|D,I)$ weights $f(x)$ at x_i by multiplying by the value of $p(x_i|D,I)$, so strong weighting near the peak of p and weak weighting far away. The amount of the weighting is given (approximately) by the height of the corresponding histogram bar. In method 2, we have similar weighting of $f(x)$ near to and far from the peak of p, but instead of this being accomplished by multiplying by $p(x_j|D,I)$, there are more x_j values near the peak than far away, in proportion to $p(x_j|D,I)$. In the end it is the same weight!

Intuition for marginalization of a nuisance parameter

Return to the expectation value of x in Eq. (19.13), but now suppose we introduce a nuisance parameter y into the distribution,

$$p(x|D,I) = \int_0^\infty dy\, p(x,y|D,I), \tag{19.14}$$

and now we sample $p(x,y|D,I)$ using MCMC to obtain samples $\{(x_j,y_j)\}$, $j = 0, \ldots, N'-1$.

If we had a function of x and y, say $g(x,y)$, that we wanted to take the expectation value of, we could use our samples as usual:

$$\langle g \rangle \approx \frac{1}{N'} \sum_{j=0}^{N'-1} g(x_j,y_j). \tag{19.15}$$

But suppose we just have $f(x)$, so we want to integrate out the y dependence; e.g., going backwards in (19.14)? How do we do that with our samples $\{(x_j,y_j)\}$?

$$\begin{aligned} \langle f \rangle &= \int_0^\infty dx f(x) \int_0^\infty dy\, p(x,y|D,I) \\ &\approx \frac{1}{N'} \sum_{j=0}^{N'-1} f(x_j). \end{aligned} \tag{19.16}$$

Equivalently we can note that $f(x)$ is just a special case of $g(x, y)$ with no y dependence. Then (19.15) gives the same formula for $\langle f \rangle$ as (19.16).

So to marginalize, we just use the x values in each (x_j, y_j) pair. **I.e., we ignore the y_j column!**

19.2. The Metropolis–Hastings algorithm

Basic structure of the Metropolis-Hastings algorithm

Let's first elaborate on the structure of the historically very important Metropolis algorithm.

1. Given $\boldsymbol{\theta}_i$, *propose* a value for $\boldsymbol{\theta}_{i+1}$, call it the "candidate" $\boldsymbol{\phi}$, sampled from $q(\boldsymbol{\phi}|\boldsymbol{\theta}_i)$. This q could take many forms, so for concreteness imagine it as a multivariate normal distribution with mean given by $\boldsymbol{\theta}_i$ and variance $\boldsymbol{\sigma}^2$ (to be specified). This entails a decreased probability to step further away from the current sample and $\boldsymbol{\sigma}$ sets scale of the step size.
2. Decide whether or not to accept candidate $\boldsymbol{\phi}$ for $\boldsymbol{\theta}_{i+1}$. Here we'll use the *Metropolis* condition (later we'll see other ways that may be better). This approach dates from the 1950's in physics but didn't become widespread in statistics until around 1990. With its adoption, it finally enabled Bayesian methods to be much more widely applied.

Metropolis condition

Calculate Metropolis ratio r given current $\boldsymbol{\theta}_i$ and proposed candidate $\boldsymbol{\phi}$:

$$r = \frac{p(\boldsymbol{\phi}|D, I)q(\boldsymbol{\theta}_i|\boldsymbol{\phi})}{p(\boldsymbol{\theta}_i|D, I)q(\boldsymbol{\phi}|\boldsymbol{\theta}_i)}$$

The distribution q may be symmetric: $q(\boldsymbol{\theta}_1|\boldsymbol{\theta}_2) = q(\boldsymbol{\theta}_2|\boldsymbol{\theta}_1)$. If so, then this is called "Metropolis". If not, then it is called "Metropolis-Hastings".

Decision:

- If $r \geq 1$, set $\boldsymbol{\theta}_{i+1} = \boldsymbol{\phi} \Longrightarrow$ **always accept**
- If $r < 1$, so less probable, *don't always reject!* **Accept with probability** r by sampling a Uniform (0,1) distribution. (Because $0 \leq r \leq 1$ for this case we can take r as a probability.)
- In practice we carry out the $r < 1$ case by getting a sample $U \sim$ Uniform $(0, 1)$. If $U \leq r$, then $\boldsymbol{\theta}_{i+1} = \boldsymbol{\phi}$, else $\boldsymbol{\theta}_{i+1} = \boldsymbol{\theta}_i$.

Note that the last case means you *do* have a θ_{i+1}, but it is the same as θ_i, so you have multiple copies and the chain always continues to grow. We further note that the acceptance probability is the minimum of 1 and r, which makes the (pseudo-)code particularly simple:

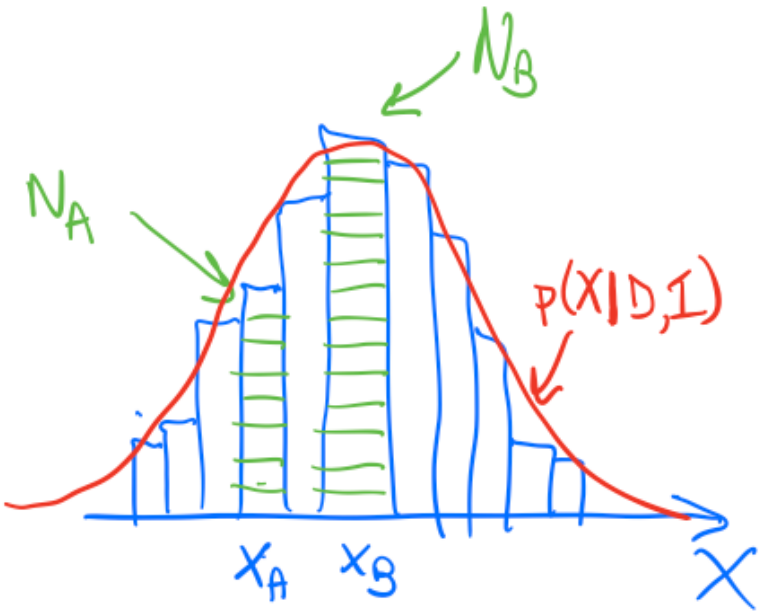
Algorithm pseudo-code:

1. Initialize $\boldsymbol{\theta}_i$, set $i = 0$.
2. Repeat
   {
   Obtain new candidate $\boldsymbol{\phi}$ from $q(\boldsymbol{\phi}|\theta_i)$ Calculate r Sample $U \sim$ uniform $(0, 1)$ If $U \leq r$ set $\boldsymbol{\theta}_{i+1} = \boldsymbol{\phi}$,

else set $\boldsymbol{\theta}_{i+1} = \boldsymbol{\theta}_i$.
$i++$
}

It's important to do the `else` step for $U > r$ (see below).

Intuition for detailed balance and the MH algorithm



Imagine the schematic histogram here is the accumulated distribution of walkers after some time. The red line is the posterior we are sampling, $p(X|D,I)$ (we're using X instead of $\boldsymbol{\theta}$ for a change of pace), so it is looking like we have an equilibrated distribution (more or less). This doesn't mean the distribution is static as we continue to sample; walkers will continue to roam around and we'll accumulate more green boxes at different X values. But if our Metropolis-Hastings algorithm is correctly implemented, the histogram shape should be *stationary* at the correct posterior, i.e., it should not change with time except for fluctuations.

After equilibrium we look at the chain and note there are N_A green boxes at X_A and N_B green boxes at X_B. Let's consider each of the X_A boxes in turn in the chain. Each of the boxes at X_A had a chance with the next Monte Carlo step to go to X_B. Similarly, each of the boxes at X_B had a chance to go to X_A. For a steady-state situation, we need the number of actual moves in each direction to be the same. This means that the two *rates* of all such transitions are equal; this is what we mean by "detailed balance".

Checkpoint question

What if the only moves accepted were those that went uphill (i.e., to higher probability density)? What would happen to N_A and N_B over time? Is this stationary?

Answer

If only uphill moves were accepted, then N_B would monotonically increase while N_A would eventually monotonically decrease (it would get some input at first from lower probability values of X but eventually those would all be gone). So all of the boxes would end up at X_B. This is stationary but not at the posterior we are sampling!

Ok, so suppose $p(X|X')$ is the transition probability, that we'll move from X' to X in one step. For $X \neq X'$ this is the probability to propose X times the probability to accept it, which we denote $A(X' \to X)$:

$$p(X|X') = q(X|X')\, A(X' \to X), \qquad X \neq X'. \tag{19.17}$$

The remaining probability is for staying at X' (a rejected step).

Checkpoint question

In terms of $p(X_A|X_B)$, $p(X_B|X_A)$, N_A, and N_B, what is the condition that the exchanges between A and B cancel out?

Answer

$$N_A \cdot p(X_B|X_A) = N_B \cdot p(X_A|X_B)$$

Checkpoint question

How are N_A and N_B related to the total N and the posteriors $p(X_A|D,I)$ and $p(X_B|D,I)$?

Answer

$$N_A \propto N \cdot p(X_A|D,I) \qquad N_B \propto N \cdot p(X_B|D,I)$$

Now let's put it together.

Checkpoint question

What is the ratio of $p(X_B|X_A)$ to $p(X_A|X_B)$ in terms of $p(X_A|D,I)$ and $p(X_B|D,I)$?

Answer

$$\frac{p(X_B|X_A)}{p(X_A|X_B)} = \frac{N_B}{N_A} = \frac{p(X_B|D,I)}{p(X_A|D,I)}$$

Now how do we realize a rule that satisfies the condition we just derived? We actually have a lot of freedom in doing so and the Metropolis choice is just one possibility. Let's verify that it works. The Metropolis algorithm uses a symmetric proposal, $q(X|X') = q(X'|X)$, so q cancels in the ratio of transition probabilities (19.17) and only the ratio of acceptance probabilities remains. The Metropolis choice is

$$A(X' \to X) = \begin{cases} \dfrac{p(X|D,I)}{p(X'|D,I)}\,, & \text{if } p(X|D,I) \le p(X'|D,I) \\ 1\,. & \text{if } p(X|D,I) > p(X'|D,I) \end{cases}$$

Let's check cases and see if it works. Here is a chart:

	$A(X_A \to X_B)$	$A(X_B \to X_A)$
$p(X_B\|D,I) < p(X_A\|D,I)$	*[fill in here]*	*[fill in here]*
$p(X_A\|D,I) < p(X_B\|D,I)$	*[fill in here]*	*[fill in here]*

Checkpoint question

Fill in the chart based on the Metropolis algorithm we are using and verify that the ratio of $A(X_A \to X_B)$ to $A(X_B \to X_A)$, which for symmetric q equals the ratio of $p(X_B|X_A)$ to $p(X_A|X_B)$, agrees with the answer derived above.

Answer

	$A(X_A \to X_B)$	$A(X_B \to X_A)$
$p(X_B\|D,I) < p(X_A\|D,I)$	$\dfrac{p(X_B\|D,I)}{p(X_A\|D,I)}$	1
$p(X_A\|D,I) < p(X_B\|D,I)$	1	$\dfrac{p(X_A\|D,I)}{p(X_B\|D,I)}$

Dividing the 2nd by the 3rd columns for the 2nd and 3rd rows each gives the correct result!

Ok, so it works. What if we have an asymmetric proposal distribution, $q(X|X') \neq q(X'|X)$? Inserting Eq. (19.17) into the balance condition gives

$$N_A\, q(X_B|X_A)\, A(X_A \to X_B) = N_B\, q(X_A|X_B)\, A(X_B \to X_A),$$

so that

$$\frac{A(X_A \to X_B)}{A(X_B \to X_A)} = \frac{p(X_B|D,I)\, q(X_A|X_B)}{p(X_A|D,I)\, q(X_B|X_A)} \equiv r,$$

which is the Metropolis–Hastings ratio for a move from X_A to X_B. The reverse move has ratio $1/r$. The choice $A = \min(1, r)$ therefore satisfies the condition: for $r < 1$ the left-hand side is $r/1$, and for $r \geq 1$ it is $1/(1/r) = r$. This is the Metropolis–Hastings algorithm as summarized at the beginning of this section.

Checkpoint question

Why do you think that this property is called detailed balance? Can you make an analogy with thermodynamic equilibrium for a collection of hydrogen atoms?

Answer

At an equilibrium temperature T, the different atoms will be distributed in different energy states according to the Boltzmann distribution. But equilibrium does not mean static; there are constant excitation and de-excitation transitions among the hydrogen atoms. Detailed balance requires that the forward and reverse rates of each particular microscopic transition are equal.

Summary: Metropolis algorithm for $p(\boldsymbol{\theta}|D,I)$ (or any other posterior).

- start with inital point $\boldsymbol{\theta}_0$ (for each walker)
- **begin repeat:** given $\boldsymbol{\theta}_i$, propose $\boldsymbol{\phi}$ from $q(\boldsymbol{\phi}|\boldsymbol{\theta}_i)$
- calculate r:

$$r = \frac{p(\boldsymbol{\phi}|D,I)}{p(\boldsymbol{\theta}_i|D,I)}\left[\frac{q(\boldsymbol{\theta}_i|\boldsymbol{\phi})}{q(\boldsymbol{\phi}|\boldsymbol{\theta}_i)}\right]$$

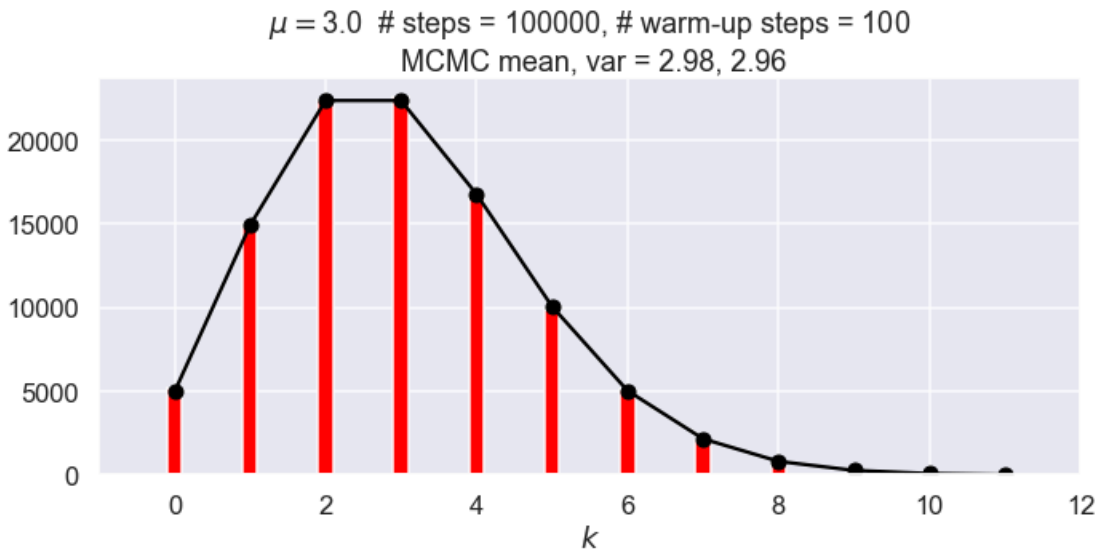
where the left factor compares posteriors and the right factor is a correction for r if the q pdf is not symmetric: $q(\boldsymbol{\phi}|\boldsymbol{\theta}_i) \neq q(\boldsymbol{\theta}_i|\boldsymbol{\phi})$

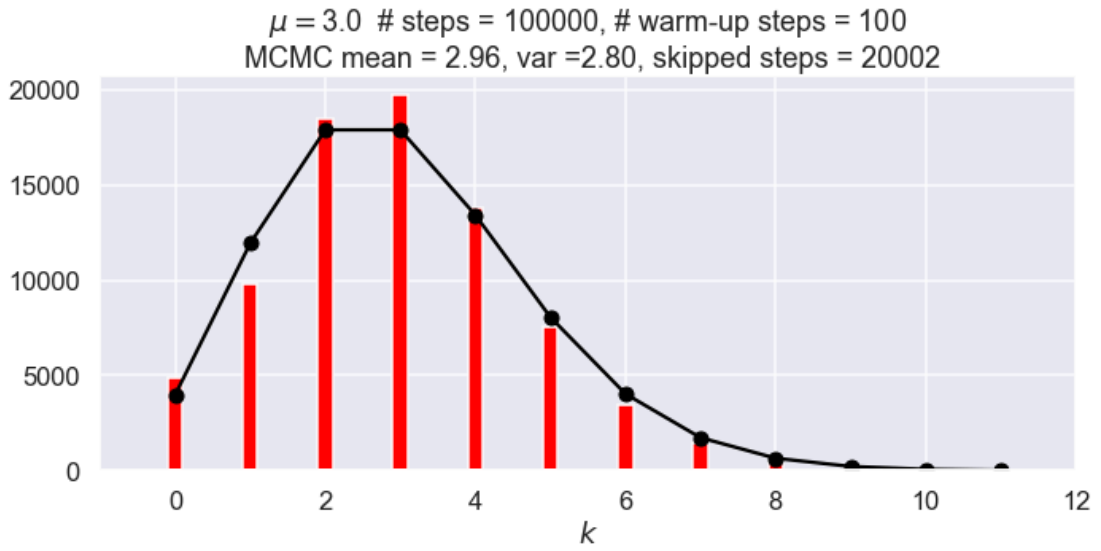
- decide whether to keep $\boldsymbol{\phi}$:
  - if $r \geq 1$, set $\boldsymbol{\theta}_{i+1} = \boldsymbol{\phi}$ (accept)
  - else draw $u \sim$ uniform $(0,1)$
    - if $u \leq r$, $\boldsymbol{\theta}_{i+1} = \boldsymbol{\phi}$ (accept)
    - else $\boldsymbol{\theta}_{i+1} = \boldsymbol{\theta}_i$ (add another copy of $\boldsymbol{\theta}_i$ to the chain)
- **end repeat** when "converged".

Key questions: When are you converged? How many "warm-up" or "burn-in " steps to skip?

Repeating configurations

What if the $\boldsymbol{\theta}_{i+1} = \boldsymbol{\theta}_i$ step is not implemented as it should be? I.e., what if the chain is only incremented if the step is accepted? You might find that this is more intuitive to you. But consider an empirical demonstration: see the figures below from the Demo: Sampling of a Poisson distribution with 100,000 steps. The first one follows the Metropolis algorithm and adds the same step if the candidate is rejected; the second one does not. Not keeping the repeated steps invalidates the Markov chain conditions $\Longrightarrow$ the wrong stationary distribution is reached (it is not wrong by a lot but is is noticeable and is seen every time the sampling is run).





19.3. Visualizations of MCMC

Chi Feng visualizations

There are excellent javascript visualizations of MCMC sampling available on the web. A particularly effective set of interactive demos was created by Chi Feng, which are included in this section (with some modifications). These demos range from random walk Metropolis-Hastings (MH) to Adaptive MH to Hamiltonian Monte Carlo (HMC) to No-U-Turn Sampler (NUTS) to Metropolis-adjusted Langevin Algorithm (MALA) to Hessian-informed HMC (H2MC), to Stein Variational Gradient Descent (SVGD) to Nested Sampling with RadFriends (RadFriends-NS). We'll start with the random walk MH, which is the algorithm discussed in Section 19.2.

References for the visualizations

[1] H. Haario, E. Saksman, and J. Tamminen, An adaptive Metropolis algorithm (2001)

[2] M. D. Hoffman, A. Gelman, The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo (2011)

[3] G. O. Roberts, R. L. Tweedie, Exponential Convergence of Langevin Distributions and Their Discrete Approximations (1996)

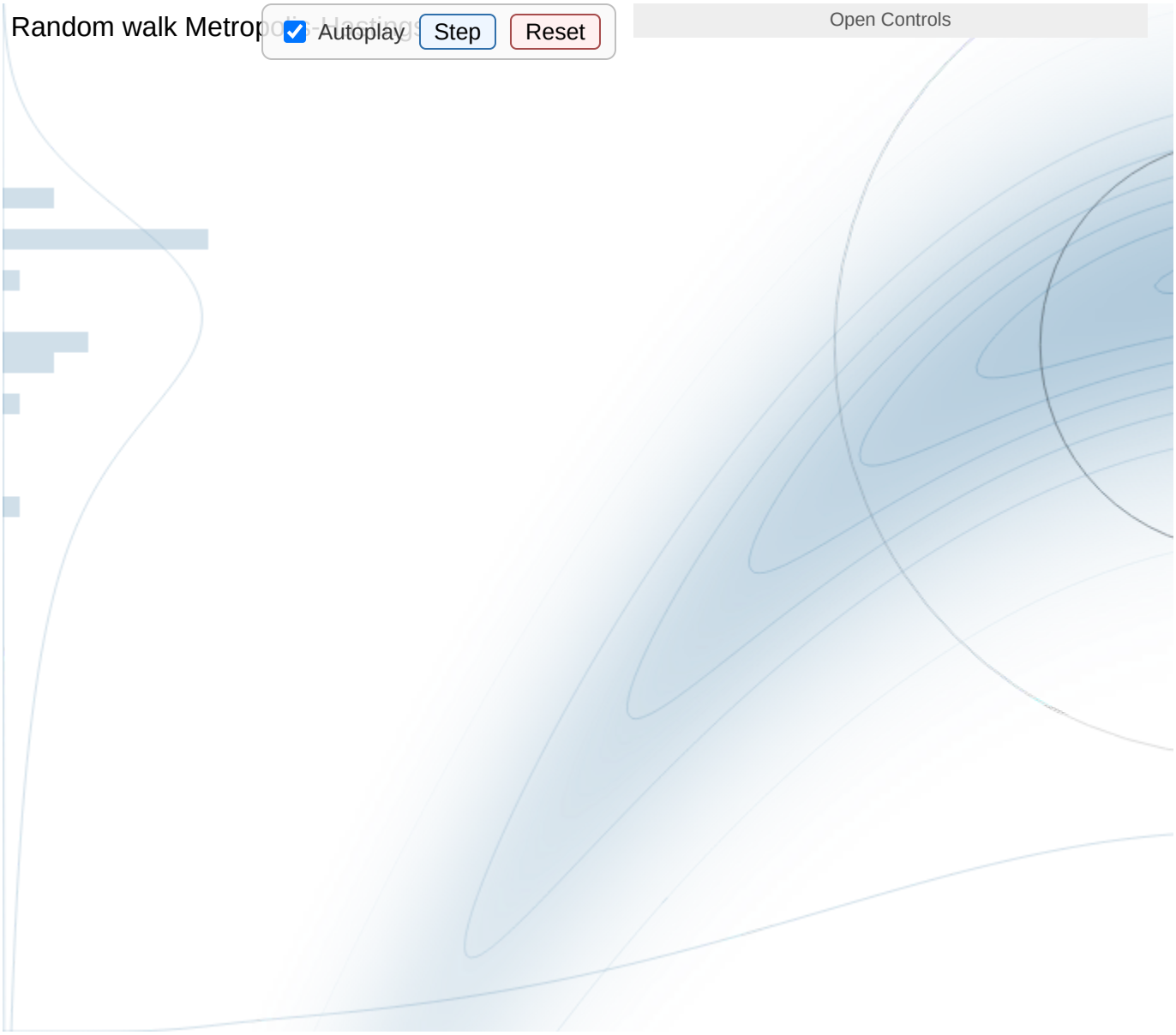
[4] Li, Tzu-Mao, et al. Anisotropic Gaussian mutations for metropolis light transport through Hessian-Hamiltonian dynamics ACM Transactions on Graphics 34.6 (2015): 209.

[5] Q. Liu, et al. Stein Variational Gradient Descent: A General Purpose Bayesian Inference Algorithm Advances in Neural Information Processing Systems. 2016.

[6] J. Buchner A statistical test for Nested Sampling algorithms Statistics and Computing. 2014.

[7] Cajo J. F. ter Braak & Jasper A. Vrugt Differential Evolution Markov Chain with snooker updater and fewer chains Statistics and Computing. 2008.

Exploring the Metropolis-Hastings (MH) simulation



Controls for the MCMC simulations

If you uncheck the `Autoplay` box, you can use the `Step` button to see the algorithm carried out one step at a time. Use the `Reset` button to clear the sampled points. Select `Open Controls` when you want to make a change to one of the settings. Note the various `Simulation options` (and other options) when the controls are open. For now, switch the `Target distribution` to `standard`. This distribution is a two-dimensional Gaussian (just the product of two one-dimensional Gaussians). After making changes, use `Close Controls` to avoid obscuring the simulation.

When looking at the visualizations, remember the basic structure of the MH algorithm:

1. Make a random proposal for new parameter values.
2. Accept or reject the proposal based on a Metropolis criterion.

Checkpoint question

Is the `standard` distribution correlated? How do you know from the simulation?

Answer

The distribution is uncorrelated. The accumulated joint posterior has horizontally oriented ellipses (circles if the scales are equal). They would be slanted if there were correlations.

Here are some comments and observations on the basic MH simulation:

- An uncolored arrow indicates a proposal, which is accepted (green) or rejected (red).

Checkpoint question

What happens when a proposal is rejected? You can see the result by unchecking the `Autoplay` box and using the `Step` button. Look at the histogram as you press the button to get either a green or red arrow.

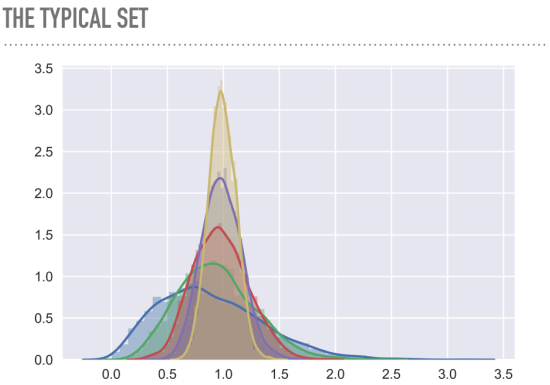
Answer

With a rejection, the point should be added to the existing set of samples. This means that if the arrow is green (accepted proposal), then the histogram bin at the new point should go up by one. If the arrow is red (rejected proposal), then the histogram bin at the old point should go up by one (this is easy to see if you first `Reset` so there are not many points yet.)

- Notice that the direction and the length of the proposal arrow varies and are, in fact, chosen randomly from a distribution. The direction is sampled uniformly.
- The MH MCMC seems to do ok on sampling such a simple distribution, as indicated by how well the projected posteriors get filled in.
- But it is *diffusing*, i.e., a random walk, which is not so efficient. A more complicated shape can cause problems:
  - MH can spend a lot of time exploring over and over again the same regions;
  - if not specially tuned, many proposals can be rejected (red arrows).
- Try the `donut` target distribution, which is much trickier.
  - Notice that the projected one-dimensional posteriors don't seem to be so complex, but this is a difficult topology.
  - Is this shape realistic? When there are many parameters (a high-dimensional space), this is analogous to a common target distribution. The probability *mass* concentrates in this shape.

Note on donuts in high dimensions

- Look at the average radius of points sampled from multivariate Gaussians as a function of the dimension.
- blue is one dimensional, green is two dimensional, … , yellow is six dimensional.
- Imagine yellow as a 6-dimensional *shell* $\Longrightarrow$ the *analog* is a two-dimensional donut.



Problem: we are constantly looking for the right step size, which is big enough to explore the space, but small enough to not get rejected too often.

- High dimensions is a big space! It is hard to stay in a region of high probability while also exploring enough of the full space (in a reasonable time).
- Try adjusting the proposal σ (there is a Gaussian proposal with variance σ^2) $\Longrightarrow$ try this on donut: to get a reasonable rate of green arrows you need excellent step size tuning.

Challenges in MCMC sampling

Use the pulldown menu to try the different target distributions available. The `banana` distribution is generally difficult to sample while the `multimodal` distribution is in general very tough to sample effectively.

Checkpoint question

What makes the various distributions difficult to sample effectively?

Hint

We see several general classes of problematic PDFs:

- Correlated distributions that are very narrow in certain directions.
- Donut or banana shapes.
- Multimodal distributions.

What is the difficulty with each of these?

Answer

- Correlated distributions: scaled parameters are needed.
- Donut or banana shapes: very low acceptance ratios.
- Multimodal distributions: might easily get stuck in local region of high probability and completely miss other regions.

19.4. Demo: Sampling of a Poisson distribution

This notebook was adapted from Example 1, section 12.2 in Gregory's *Bayesian Logical Data Analysis for the Physical Sciences* [Gre05].

The Poisson discrete random variable from `scipy.stats` is defined by (see documentation)

$$p(k|\mu) = \frac{\mu^k e^{-\mu}}{k!} \quad \text{for} \;\; k \geq 0 \; .$$

where k is an integer and μ is called the shape parameter. The mean and variance of this distribution are both equal to μ. (Note: Gregory uses a different notation, but we'll stay consistent with scipy.stats.)

By "sampling a Poisson distribution" we mean we will obtain a set of k values: $\{k_1, k_2, k_3, \ldots\}$ that follow this distribution. That is, for a particular k_i, the probability to get that value should be what we get by evaluating $p(k_i|\mu)$. We know we've succeeded if we make a histogram of our set of k_is and it looks like $p(k|\mu)$ (scaled to line up or else our histogram needs to be normalized to one). We've already seen sampled the Poisson distribution through a `scipy.stats` script in the Part I exercise notebook Amplitude of a signal in the presence of background.

Here we will sample it using a method generically called Markov chain Monte Carlo or MCMC. A Markov chain starts with some initial value, and then each successive value is generated from the previous one. But it is not deterministic: when the new value is chosen, there is a random number involved. The particular version of MCMC used here is called Metropolis-Hasting. You may be familiar with this from a statistical mechanics class, where it is typically applied to the Ising model.

We'll do the Metropolis-Hasting sampling as follows:

1. Choose an initial k (call it k_0), having already fixed μ.
2. Given k_i, sample a uniform random number x from 0 to 1 (so $x \sim U(0,1)$) and propose $k' = k_i + 1$ if the $x > 0.5$, otherwise propose $k' = \max(k_i - 1)$ (meaning: step to the left, but don't go below zero; this is an asymetric proposal!).
3. Compute the Metropolis ratio $r = p(k'|\mu) \; / \; p(k_i|\mu)$ using the discrete Poisson distribution.
4. Given another uniform random number $y \sim U(0,1)$, $k_{i+1} = k'$ if $y \leq r$, else $k_{i+1} = k_i$ (i.e., keep the same value for the next k).
5. Repeat 2.-4. until you think you have enough samples of k.
6. When graphing the posterior or calculating averages, skip the first values until the sampling has equilibrated (this is generally called the "burn-in" or "warm-up").

In practice we'll carry this out by generating all our uniform random numbers at the beginning using `scipy.stats.uniform.rvs`.

```
%matplotlib inline

import numpy as np
from scipy.special import factorial

# We'll get our uniform distributions from stats, but there are other ways.
import scipy.stats as stats

import matplotlib.pyplot as plt
```

```
def poisson(k, mu):
    """
    Returns a Poisson distribution value for k with mean mu
    """
    return mu**k * np.exp(-mu) / factorial(k)
```

In the following we have the steps 1-6 defined above marked in the `MCMC_poisson` function. *Step through the implementation and ask questions about what you don't understand.*

```
def MCMC_poisson(mu, k0, num_steps, warm_up_steps=100):
    """
    1. Set mu and k0 by passed variables.
    Also pass num_steps, the number of MCMC steps we'll take
    """

    # generate the two sets of uniform random numbers we'll need for 2. and 4.
    uniform_1 = stats.uniform.rvs(size=num_steps)
    uniform_2 = stats.uniform.rvs(size=num_steps)

    _k_array = np.zeros(num_steps, dtype=int)
    _k_array[0] = k0

    # 5. Loop through steps 2-4
    for i in range(num_steps-1):  # num_steps-1 so k_array[i+1] is always defined
        # 2. Propose a step
        k_now = _k_array[i]
        if uniform_1[i] > 0.5:
            kp = k_now + 1          # step to the right
        else:
            kp = max(0, k_now - 1)  # step to the left, but don't go below zero

        # 3. Calculate Metropolis ratio
        metropolis_r = poisson(kp, mu) / poisson(k_now, mu)
        # 4. Accept or reject
        if uniform_2[i] <= metropolis_r:
            _k_array[i+1] = kp
        else:
            _k_array[i+1] = k_now

    # 6. Choose how many steps to skip (100 by default, but optionally passed)

    # Return the full chain, but also the mean and std after the warm-up
    return _k_array, np.mean(_k_array[warm_up_steps:]), np.std(_k_array[warm_up_steps:])
```

Test the `MCMC_poisson` function

```
# test the function
mu = 3.
k0 = 10.

num_steps = 1000  # number of MCMC steps we'll take
warm_up_steps = 100
k_array, mean, sd = MCMC_poisson(mu, k0, num_steps, warm_up_steps)
print(mean, sd)
```

Trace plot

- MCMC trace: a plot of the value at successive MC steps. Notice the fluctuations; it stays reasonably close to the value of μ but still can jump high.
- Note the outliers at the beginning: the sampling needs to *equilibrate*. This is called the warm-up (or "burn-in") time. *What would you estimate for the warm-up time?*
- *Does the trace show that the space is being explored (rather than being confined to one region)?*
- *Predict how the trace to behave for different μ and try it.*

```
# Plot the trace
fig = plt.figure(figsize=(10,6))

# Plot the trace (that means k_i for i=0 to num_steps)
ax_trace = fig.add_subplot(1, 1,1)
ax_trace.plot(range(num_steps), k_array, color='blue')
ax_trace.set_ylim(0, 12)
ax_trace.set_xlabel(r'$i$')
ax_trace.set_ylabel(r'$k_i$')
trace_title = rf'MCMC trace for Poisson distribution with $\mu = ${mu:.1f}'
ax_trace.set_title(trace_title);
```

Histogram

The histogram shows how well we are doing. Rerun the test multiple times and characterize the fluctuations.

- *What do you observe about the histogram?*
- *Does the sampled historgram agrees with (scaled) exact Poisson pdf (that is what success looks like)? Is it normalized? Compare 1000 to 100,000 samples.*

```
fig = plt.figure(figsize=(10,6))
# Plot the Poisson distribution
ax_plot = fig.add_subplot(1, 1, 1)
bin_num = 12
n_pts = range(bin_num)

# Scale exact result to the histogram, accounting for warm_up_steps
poisson_pts = [(num_steps - warm_up_steps) * poisson(n, mu) for n in n_pts]

# Plot the exact distribution
ax_plot.plot(n_pts, poisson_pts, marker='o', color='black')
# Histogram k_i beyond the warm-up period
ax_plot.hist(k_array[warm_up_steps:], bins=n_pts,
             align='left', rwidth=0.2, color='red')
ax_plot.set_xlim(-1, bin_num)
ax_plot.set_xlabel(r'$k$')
plot_title = rf'$\mu = ${mu:.1f}  # steps = {num_steps:d},' \
               + rf' # warm-up steps = {warm_up_steps:d},'  \
               + rf' mean = {mean:.3f}'
ax_plot.set_title(plot_title)

fig.tight_layout()
```

```
# Check the mean and standard deviations from the samples against exact
print(f' MCMC mean = {np.mean(k_array[warm_up_steps:]):.3f}')
print(f'Exact mean = {stats.poisson.mean(mu=mu):.3f}')
print(f'   MCMC sd = {np.std(k_array[warm_up_steps:]):.3f}')
print(f'  Exact sd = {stats.poisson.std(mu=mu):.3f}')
```

Questions and things to do

How do you expect the accuracy of the estimate of the mean scales with the number of points? How would you test it?

Record values of the MCMC mean for `num_steps` = 1000, 4000, and 16000, running each 10-20 times. Explain what you find.

Calculate the mean and standard deviations of the means you found (using `np.mean()` and `np.std())`. Explain your results.

Predict what you will find from 10 runs at `num_steps` = 100,000. What did you actually find?

Try a log-log plot

```
num_trials = 20
sd_array = []
steps_array = []

for i in range(0, 7):  # range(0, 9)  # range(3, 9) # range(0, 7)
    mu = 3.
    k0 = 10.
    num_steps = 1000 * 2**(i)  # number of MCMC steps we'll take
    warm_up_steps = 100

    mean_temp_array = []
    for j in range(num_trials):
        k_array, mean, sd = MCMC_poisson(mu, k0, num_steps, warm_up_steps)
        mean_temp_array.append(mean)

    sd_array.append(np.std(mean_temp_array))
    steps_array.append(num_steps - warm_up_steps)
```

```
for i in range(len(steps_array)):
    print(f'{steps_array[i]} {sd_array[i]:.3e}')
print(sd_array)
```

```
slope, intercept = np.polyfit(np.log(steps_array), np.log(sd_array), 1)
print(f'slope = {slope:.2f}')
```

```
fig,ax = plt.subplots(1,1)
ax.loglog(steps_array,1/np.sqrt(steps_array)*np.sqrt(steps_array[-1])*sd_array[-1],label=r'$1/\sqr
ax.loglog(steps_array,sd_array,'o',label=r'$\sigma(N)$')
ax.plot(steps_array, np.exp(intercept) * steps_array**slope, label=r'linear fit')
ax.set_xlabel(r'$N$')
ax.set_ylabel(r'$\sigma$')
ax.legend(loc='best');
```

Repeat with a new incorrect MCMC function

In MCMC_poisson_no_repeat the current position is *not* added to the chain (as a repeated entry) if the proposed move is not accepted. Compare the posterior to what is obtained earlier with a large number of steps.

```
def MCMC_poisson_no_repeat(mu, k0, num_steps, warm_up_steps=100):
    """
    1. Set mu and k0 by passed variables.
    Also pass num_steps, the number of MCMC steps we'll take
    """

    # generate the two sets of uniform random numbers we'll need for 2. and 4.
    uniform_1 = stats.uniform.rvs(size=num_steps)
    uniform_2 = stats.uniform.rvs(size=num_steps)

    _k_array = np.zeros(num_steps, dtype=int)
    _k_array[0] = k0

    index = 0
    skipped = 0
    # 5. Loop through steps 2-4
    for i in range(num_steps-1):  # num_steps-1 so k_array[i+1] is always defined
        # 2. Propose a step
        k_now = _k_array[index]
        if uniform_1[i] > 0.5:
            kp = k_now + 1          # step to the right
        else:
            kp = max(0, k_now - 1)  # step to the left, but don't go below zero

        # 3. Calculate Metropolis ratio
        metropolis_r = poisson(kp, mu) / poisson(k_now, mu)
        # 4. Accept or reject
        if uniform_2[i] <= metropolis_r:
            _k_array[index+1] = kp
            index += 1
        else:
            # _k_array[i+1] = k_now
            # index +=1
            skipped += 1
            pass

    # 6. Choose how many steps to skip (100 by default, but optionally passed)

    # Return the full chain, but also the mean and std after the warm-up
    return _k_array, np.mean(_k_array[warm_up_steps:-skipped]), np.std(_k_array[warm_up_steps:-
skipped]), skipped
```

```
# test the (incorrect) function (with same parameters as above)
# mu = 3.
# k0 = 10.

#num_steps = 100000  # number of MCMC steps we'll take
#warm_up_steps = 100
k_array_no_repeat, mean_no_repeat, sd_no_repeat, skipped = MCMC_poisson_no_repeat(mu, k0,
num_steps, warm_up_steps)
print(mean_no_repeat, sd_no_repeat)
```

```
fig = plt.figure(figsize=(10,6))
# Plot the Poisson distribution
ax_plot = fig.add_subplot(1, 1, 1)
bin_num = 12
n_pts = range(bin_num)

# Scale exact result to the histogram, accounting for warm_up_steps
poisson_pts_no_repeat = [(num_steps - warm_up_steps - skipped) * poisson(n, mu) for n in n_pts]

# Plot the exact distribution
ax_plot.plot(n_pts, poisson_pts_no_repeat, marker='o', color='black')
# Histogram k_i beyond the warm-up period
ax_plot.hist(k_array_no_repeat[warm_up_steps:-skipped], bins=n_pts,
             align='left', rwidth=0.2, color='red')
ax_plot.set_xlim(-1, bin_num)
ax_plot.set_xlabel(r'$k$')
plot_title = rf'$\mu = ${mu:.1f}  # steps = {num_steps:d},' \
               + rf' # warm-up steps = {warm_up_steps:d},'  \
               + rf' mean = {mean:.3f}' \
               + f' skipped steps = {skipped:d}'
ax_plot.set_title(plot_title)

fig.tight_layout()
```

```
# Check the mean and standard deviations from the samples against exact
print(f' MCMC mean = {np.mean(k_array_no_repeat[warm_up_steps:-skipped]):.3f}')
print(f'Exact mean = {stats.poisson.mean(mu=mu):.3f}')
print(f'   MCMC sd = {np.std(k_array_no_repeat[warm_up_steps:-skipped]):.3f}')
print(f'  Exact sd = {stats.poisson.std(mu=mu):.3f}')
```

19.5. Demo: Random walk and sampling

Adapted from the TALENT course on Learning from Data: Bayesian Methods and Machine Learning (York, UK, June 10-28, 2019). The original notebook was by Christian Forssen.

The second part of this exercise was inspired by the post "MCMC sampling for dummies" in the excellent blog "While my MCMC gently samples" by Thomas Wiecki.

Import of modules

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

# Not really needed, but nicer plots
import seaborn as sns
sns.set()
sns.set_context("talk")
```

```
# Modules needed for this exercise
from scipy.stats import norm
from scipy.stats import cauchy
```

Part 1: Random walk in [-5,5] region

We will make a random walk in the bounded region [-5,5] according to the following algorithm:

1. Start in some position
2. Draw a random number from a normal distribution centered at the current position and a width that we set beforehand.
3. Move to the proposed position unless it is outside of the region boundaries (in which case we stay at the current position, adding it again to the list of visited positions)

Map this problem onto the Metropolis algorithm.

- *What is $q(\phi|\boldsymbol{\theta}_i)$?*
- *What is $p(\boldsymbol{\theta}_i|D,I)$?*

```
def sampler(start_pos, no_of_samples=4, proposal_width=.2):
    samples=[]
    current_position = start_pos
    for i in range(no_of_samples):
        # suggest new position
        proposed_position = norm(current_position, proposal_width).rvs()

        accept = (np.abs(proposed_position)<5)

        if accept:
            # Update position
            current_position = proposed_position

        samples.append(current_position)

    return np.array(samples)
```

Collect a large number of samples according to the above procedure. Plot a histogram and the trace of the collected samples.

```
# for reproducibility
np.random.seed()
samples = sampler(0.,no_of_samples=10000, proposal_width=0.4)

fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(10, 4))
ax1.hist(samples, density=True, histtype='stepfilled', alpha=0.2,
         range=(-5.2,5.2),bins=21)
ax1.set(xlabel=r'$x$', ylabel='prob. dens.', title='samples histogram')
ax2.plot(samples)
ax2.set(xlabel='iteration', ylabel='mu', title='trace of samples');
```

Questions

Change `np.random.seed(10)` to `np.random.seed()` so that a different pseudo-random number sequence every time you re-run.

- *What distribution does the probability density as represented by the histogram resemble?*

- *Try increasing and decreasing the number of samples. How do the fluctuations between different runs change?*
- *Study the algorithm closely and find the line where the current position is added again to the list of samples if the proposed position is outside the hard-wall boundary. What happens if you change this so that the proposed step is just ignored and no position gets added to the list of samples for that iteration?*
- *What happens if you try with another proposal distribution, e.g. a uniform one?*
- *What happens if you decrease the width of the proposal distribution?*

When answering the last question you probably did find that the proposal changed quite drastically when using the same number of samples, but taking much shorter steps. This is a consequence of the collected samples being *correlated*.

We can define the autocorrelation function $\rho(h)$ from the list of samples X, where X_t is sample t and X_{t+h} is a shifted version of X_t with h being called the *lag*

$$\rho(h) = \frac{\sum_{\text{overlap}} \left[(X_t - \bar{X})(X_{t+h} - \bar{X})\right]}{\sqrt{\sum_{\text{overlap}}(X_t - \bar{X})^2}\sqrt{\sum_{\text{overlap}}(X_{t+h} - \bar{X})^2}}$$

The summation is carried out over the subset of samples that overlap.

```
def autocorrelation(chain, max_lag=100):
    dimension = len(chain)
    acors = np.empty(max_lag+1)
    if max_lag > len(chain)/5:
        warnings.warn('max_lag is more than one fifth the chain length')
    # Create a copy of the chain with average zero
    chain1d = chain - np.average(chain)
    for lag in range(max_lag+1):
        unshifted = None
        shifted = chain1d[lag:]
        if 0 == lag:
            unshifted = chain1d
        else:
            unshifted = chain1d[:-lag]
        normalization = np.sqrt(np.dot(unshifted, unshifted))
        normalization *= np.sqrt(np.dot(shifted, shifted))
        acors[lag] = np.dot(unshifted, shifted) / normalization
    return acors
```

Study the behavior of the autocorrelation function (ACF) $\rho(h)$

- *First, take a minute to get a feeling for the definition of the ACF. Try to describe the operation that is performed in words.*
- *What happens when the proposal width is large/small/intermediate?*
- *What does it mean when the autocorrelation is large/small?*
- It is often observed that $\rho(h)$ is roughly exponential so that we can define an autocorrelation time τ according to

$$\rho(h) \sim \exp(-h/\tau).$$

- *Extract τ for some different choices of the proposal width.*

```
import warnings
```

```
acors = autocorrelation(samples,max_lag=500)
fig, ax = plt.subplots()
ax.plot(acors);
ax.set(xlabel='lag', ylabel='autocorrelation', ylim=(-.2, 1));
```

Part 2: MCMC sampling of a Lorentzian pdf using the random walk Metropolis algorithm

In the previous example we performed a random walk and accepted all steps unless they were outside a hard-wall boundary. This corresponds to a situation where all positions within that region is equally probable, and all positions outside have probability zero.

- What pdf does that situation correspond to?

Now, let us try another situation in which we have some function that tells us the (possibly unnormalized) probability of a given position. We will assume a known, specific form of this pdf, namely a Lorentzian (Cauchy) distribution, but it might just as well be some very complicated function that can only be evaluated numerically. All that is needed is some function that, for each position in the parameter space, returns a probability.

Let us start by studying the pdf that we will be sampling from using a random walk (using the Metropolis algorithm outlined below).

```
# Draw a number of random samples from the standard Cauchy
r = cauchy.rvs(size=1000)
plt.hist(r, density=True, histtype='stepfilled', alpha=0.2,
         range=(-10,10),bins=21);
```

Questions:

This histogram corresponds to a finite sample from the pdf of a standard Cauchy (Lorentzian)

$$p(x|\alpha=0,\beta=1) = \frac{1}{\pi(1+x^2)},$$

with mean $\alpha = 0$ and FWHM $2\beta = 2$.

- How does this pdf compare with a standard normal distribution $\mathcal{N}(x;\mu=0,\sigma^2=1)$?
- What is the function of the optional argument ‘density=True’ in the ‘hist’ plot function call in the cell above?
- What happens if you remove the optional ‘range’ and ‘bins’ arguments from the ‘hist’ plot function call in the cell above?
- Why? How would this look when plotting a histogram for random samples from a standard normal distribution?

Let us turn the posterior into a callable function. We will deliberately remove the normalization to make the point that sampling can be made for an unnormalized pdf:

```
def posterior_function(x):
    return 1 / (1+x**2)

def normalized_posterior_function(x):
    return 1 / (np.pi * (1+x**2))
```

Now on to the sampling. At first, you define the starting parameter position (can be randomly chosen), let's fix it arbitrarily to:

```
current_position = 1.
```

Then, you propose to move (jump) from that position somewhere else (that's the Markov part). You can be very dumb or very sophisticated about how you come up with that proposal. The Metropolis sampler is very dumb and just takes a sample from a proposal distribution (here we again choose a normal distribution) centered around your current position (i.e. `current_position`) with a certain standard deviation (`proposal_width`) that will determine how far you propose jumps (here we're using `scipy.stats.norm`):

```
proposal_position = norm(current_position, proposal_width).rvs()
```

Next, you evaluate whether that's a good place to jump to or not. We quantify this by computing the probability of the proposed position in parameter space, i.e. evaluating the posterior pdf at the proposed position. Usually you would use log probabilities but we omit this here.

```
# Nominator of Bayes' formula
p_current = posterior(current_position)
p_proposal = posterior(proposed_position)
```

Up until now, we essentially have a hill-climbing algorithm that would just propose movements into random directions and only accept a jump if the `proposed_position` has higher likelihood than `current_position`. Eventually we'll get to `x = 0` (or close to it) from where no more moves will be possible. However, we want to get a posterior pdf so we'll also have to sometimes accept moves into the other direction. The key trick is by dividing the two probabilities,

```
p_accept = p_proposal / p_current
```

we get an acceptance probability. Note that the acceptance probability is obtained by dividing the posterior of proposed parameter setting by the posterior of the current parameter setting. This implies that the posterior pdf does not necessarily need to be normalized, the normalization factor will anyway be canceled out.

You can see that if `p_proposal` is larger, that probability will be `> 1` and we'll definitely accept. However, if `p_current` is larger, say twice as large, there'll be a 50% chance of moving there which we will decide by drawing a random number:

```
accept = np.random.rand() < p_accept

if accept:
    # Update position
    current_position = proposed_position
```

This simple procedure gives us samples from the posterior.

```
def sampler(posterior_func, no_of_samples=4, start_position=.5,
            proposal_width=1., plot=False):
    current_position = start_position
    samples = [current_position]
    for i in range(no_of_samples):
        # suggest new position
        proposed_position = norm(current_position, proposal_width).rvs()

        # Compute posteriors of current and proposed position
        p_current = posterior_func(current_position)
        p_proposal = posterior_func(proposed_position)

        # Acceptance probability
        # Note that this part will be modified in the Metropolis-Hastings algorithm
        # for which we also consider asymmetric proposal distributions
        p_accept = p_proposal / p_current

        # Accept proposal?
        accept = np.random.rand() < p_accept

        # Visualization
        if plot:
            assert no_of_samples < 11, "Too many samples for visualization"
            plot_proposal(posterior_func, current_position, p_current,
                          proposed_position, p_proposal, accept, samples, i)

        if accept:
            # Update position
            current_position = proposed_position

        samples.append(current_position)

    return np.array(samples)

# Function to display
def plot_proposal(posterior_func, current_position, p_current,
                  proposed_position, p_proposal, accepted, trace, i):
    from copy import copy
    trace = copy(trace)
    fig, (ax1, ax2, ax3) = plt.subplots(ncols=3, figsize=(12, 5))
    fig.suptitle('Iteration %i' % (i + 1))
    x = np.linspace(-5, 5, 5000)
    color = 'g' if accepted else 'r'

    # Plot posterior
    ax1.plot(x, posterior_func(x))
    ax1.plot([current_position] * 2, [0, p_current], marker='o', color='b')
    ax1.plot([proposed_position] * 2, [0, p_proposal], marker='o', color=color)
    ax1.annotate("", xy=(proposed_position, 0.2), xytext=(current_position, 0.2),
                 arrowprops=dict(arrowstyle="->", lw=2.))
    ax1.set(ylabel='Probability Density', \
            title='current: posterior(mu=%.2f) = %.2f\nproposal: posterior(mu=%.2f) = %.2f' \
            % (current_position, p_current, proposed_position, p_proposal))

    if accepted:
        trace.append(proposed_position)
    else:
        trace.append(current_position)

    # Posterior histogram
    ax2.plot(x, normalized_posterior_function(x)) # properly normalized
    sns.histplot(trace, kde=False, ax=ax2)
    ax2.axvline(current_position, color='b', linestyle='--',
                label='current position')
```

```
    ax2.axvline(proposed_position, color=color, linestyle='--',
                label='proposed position')
    ax2.annotate("", xy=(proposed_position, 0.2), xytext=(current_position, 0.2),
                 arrowprops=dict(arrowstyle="->", lw=2.))


    ax3.plot(trace)
    ax3.set(xlabel='iteration', ylabel='position', title='trace')
    plt.tight_layout()
```

To visualize the sampling, we'll create plots for some quantities that are computed. Each row below is a single iteration through our Metropolis sampler.

The first column displays our unnormalized posterior distribution. This is for visualization only, normally we would not be able to plot a nice curve to show the posterior. Here, we plug in our x proposals. The vertical lines represent our current position in blue and our proposed position in either red or green (rejected or accepted, respectively).

The second column is our posterior distribution. Here we are displaying the normalized posterior as the blue curve compared to the normalized histogram of samples (green bars) and the move that was just proposed.

The third column is our trace (i.e. the posterior samples of visited positions that we're generating). Note that we store a sample at each iteration, irrespective of whether the propsal was accepted or rejected. In the latter situation, we keep the previous position and the line just stays constant.

Note that we always accept moves to relatively more likely x values (in terms of their posterior density), but only sometimes to relatively less likely x values, as can be seen already in the first iteration, and later in iterations 6, 7, and 8 (the iteration number can be found at the top center of each row).

```
np.random.seed(123)
samples = sampler(posterior_function, no_of_samples=8, start_position=.5, proposal_width=1.,
plot=True);
```

Now the magic of MCMC is that you just have to do that for a long time, and the samples that are generated in this way come from the posterior distribution of your model. There is a rigorous mathematical proof that guarantees this, which we won't go into detail here.

To get a sense of what this produces, lets draw a lot of samples and plot them.

```
samples = sampler(posterior_function, no_of_samples=15000, start_position=1.)
```

```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(10, 4))

ax1.plot(samples)
ax.set(xlabel='sample', ylabel='x');

ax2.hist(samples[500:], density=True, histtype='stepfilled', alpha=0.2,
         range=(-10,10), bins=21, label='samples')
x = np.linspace(-10., 10., 200)
ax2.plot(x, normalized_posterior_function(x), 'g', label='posterior')
ax2.set(xlabel='position', ylabel='belief');
ax2.legend();
plt.tight_layout()
```

The plot in the left panel is usually called the trace. To get an approximation of the posterior (the reason why we're doing all this), we simply take the histogram of this trace. That histogram is shown in the right panel which then represents our **belief** in the probability density for different `positions`. For comparison we also show (green line) the properly normalized posterior pdf, for which we had a known analytical form in this case.

Questions / tasks

- Add functionality to extract the acceptance rate (i.e. the fraction of proposed steps that are accepted).
- Print the acceptance rate and plot the autocorrelation function / autocorrelation time when performing the following studies:
- Based mainly on empirical studies, Roberts, Gelman, and Gilks (1997) recommend calibrating the acceptance rate to about 25% for a high-dimensional model, and to about 50% for models of one or two dimensions. How can you calibrate the acceptance rate of your MCMC sampler?
- Try to sample from some other posterior pdf.
- What happens when you change the proposal distribution:
  - Using a different proposal distribution?
  - What happens if you make it asymmetric around the 'current_position'?
  - What happens if you stick to the normal distribution as a proposal distribution, but change its width?
- How would you define that a MCMC algorithm works? What is the property that you are after?
- The Metropolis algorithm works because it has three properties:
  - It is *irreducible*
  - It is *aperiodic*
  - It is *positive recurrent*

  What do you think that these properties mean?

Optional task: log probabilities

Since one often encounters very small probabilities when sampling from some pdf it is usually a good idea to work with log probabilities instead.

- Modify the code above to employ log-probabilities instead.

Part 3: Detailed balance and the Metropolis-Hastings algorithm

Questions / tasks

- Write an expression for the Metropolis acceptance probability, $\alpha(X_t, X_{t+1})$, that describes the probability of accepting a proposed step from X_t to X_{t+1}. Use the notation $p(X_t|D, I)$ for the probability of drawing sample X_t from the posterior and analogous for $p(X_{t+1}|D, I)$.

Let us introduce the notation $q(X_{t+1}|X_t)$ for the probability (pdf) for proposing a step from position X_t to X_{t+1}.

- What does $q(X_{t+1}|X_t)$ correspond to in the above example?

From these two quantities, $q(X_{t+1}|X_t)$ and $\alpha(X_t, X_{t+1})$, it is possible to define a transition kernel

$$p(X_{t+1}|X_t) = q(X_{t+1}|X_t)\alpha(X_t, X_{t+1})$$

- Why do you think that is called a *transition kernel*?

Our definition of the acceptance probability implies *detailed balance*

$$p(X_t|D, I)p(X_{t+1}|X_t) = p(X_{t+1}|D, I)p(X_t|X_{t+1}),$$

- Why do you think that this property is called detailed balance? Can you make an analogy with thermodynamic equilibrium for e.g. a collection of hydrogen atoms?

In this exercise we have considered the Metropolis algorithm, proposed by Metropolis *et al.* in 1953. Hastings (1970) generalized the algorithm to include asymmetric proposal distributions. The single consequence of this generalization is a modification of the acceptance probability $\alpha(X_t, X_{t+1})$.

- How is the acceptance probability $\alpha(X_t, X_{t+1})$ modified when allowing for asymmetric proposal distributions?
- Try to make this change in the code above and try with both symmetric and asymmetric proposal distributions. For an asymmetric proposal distribution you could use a log normal pdf (which is available from scipy.stats).

20. Details of MCMC

Here we provide formal and detailed discussions on stochastic processes in general, including random walk and Gaussian process examples, and then on Markov Chain Monte Carlo and the Metropolis Hasting algorithm.

- Stochastic processes introduces the concept of a stochastic process, with code examples and several exercises.
- Example: Random walk provides a basic example.
- Special case: Gaussian process introduces the Gaussian process, which we will revisit in detail in Overview of Gaussian processes.
- Markov chains has a detailed introduction to Markov chains, with examples.
- Derivation of the Metropolis-Hastings algorithm provides a complete definition and some exercises.

20.1. Stochastic processes

> “Stochastic, or random, processes is the dynamic side of probability. What differential equations is to calculus, stochastic processes is to probability.”

—Robert P. Dobrow, *Introduction to Stochastic Processes with R*

A stochastic (random) process is simply one in which outcomes are uncertain. By contrast, a deterministic process always produces the same result for a given input. While functions and differential equations are often used to describe deterministic processes, we can use random variables and probability distribution functions to describe stochastic ones.

Stochastic processes are very useful for the modeling of systems that have inherent randomness. This is certainly true in physics, and not the least for quantum systems. We will not delve deep into this topic in these notes. You might encounter stochastic modeling in the future when studying, for example, statistical physics.

Instead, our specific goal is to become familiar with a special kind of stochastic processes known as Markov chains. Having defined and studied general properties, we will find that such chains can be designed to produce samples from a stationary probability distribution of our choice.

Discuss

Why is it so useful in Bayesian inference to be able to produce samples from a user-specified probability distrbution?

Example 20.1 (An exponential growth model)

Consider a simple exponential growth model to describe a population of bacteria in a culture with a food source. In a deterministic model we assert that the population grows at a fixed rate λ. The growth rate is then proportional to the population size as described by the differential equation

$$\frac{dy}{dt} = \lambda y(t). \tag{20.1}$$

With the initial population $y(0) = y_0$, this equation is solved by the exponential function

$$y(t) = y_0 \exp(\lambda t), \tag{20.2}$$

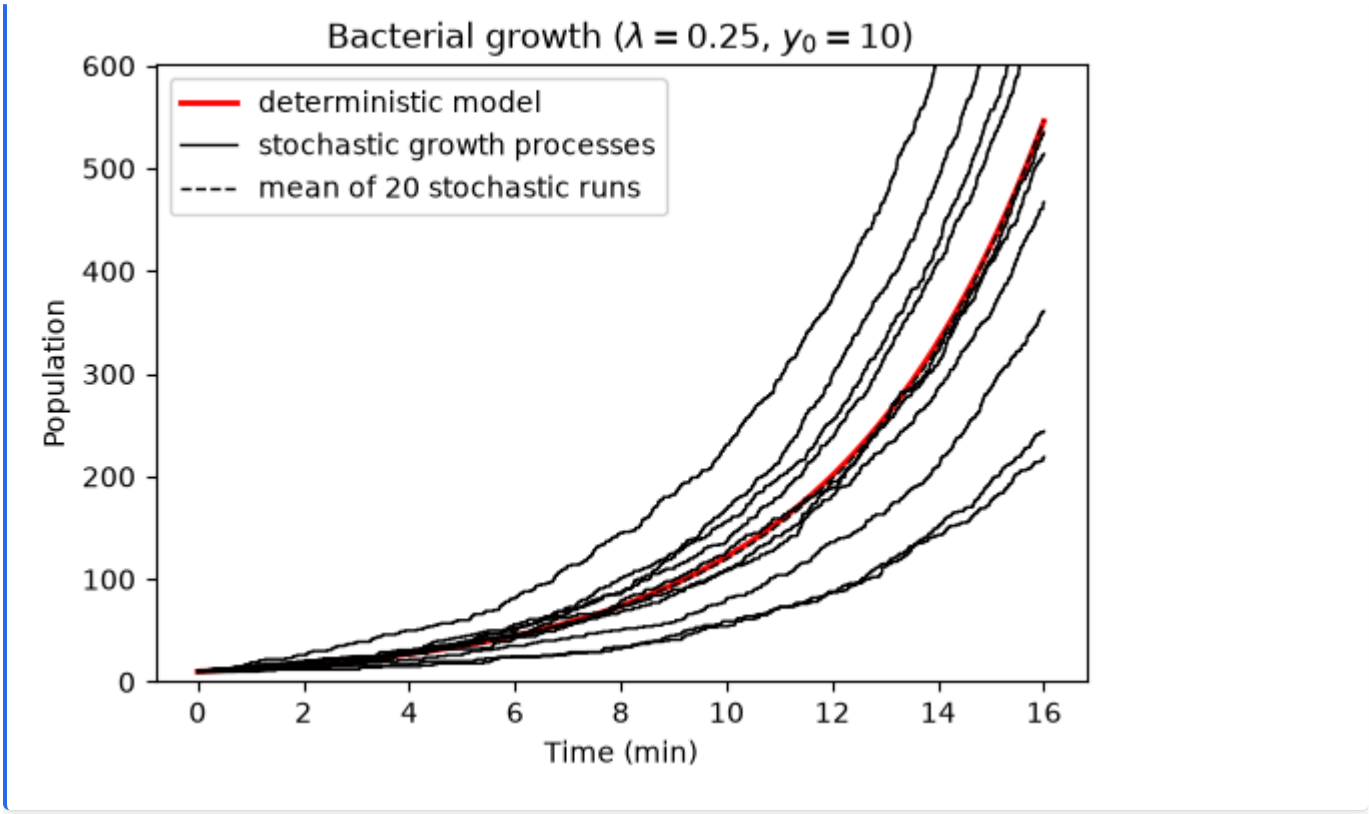
from which the population is precisely determined. For example, with $y_0 = 10$ and $\lambda = 0.25$ per minute we find that the population has almost doubled four times after 11 minutes (since $y(11)/y_0 = 15.6 \approx 2^4$).

This model, however, does not take into account the randomness of the reproduction of individual organisms. This can be considered in a stochastic growth model where we assign the probability $\lambda \Delta t$ for a single bacteria to split once during a (very short) time interval Δt. This random process can be simulated using a short python script (click below to show the code). Ten different realizations of the stochastic model are visualized in Fig. 20.1 and compared with the deterministic one. Questions that might be of scientific interest include:

- What is the expected number of bacteria at time t?
- What is the probability that the number of bacteria will exceed some threshold value after t minutes?
- What is the distribution of the time it takes for the population to double in size?

▼ Hide code cell content

```
'65'
```



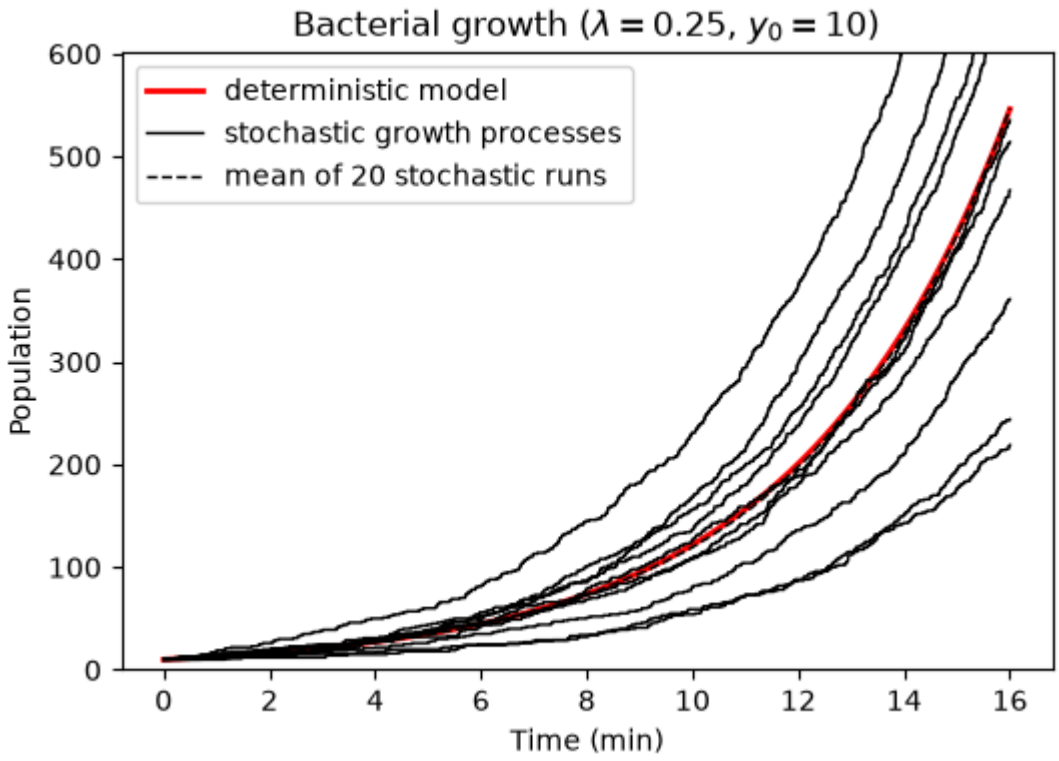


Fig. 20.1 Growth of a bacteria population. The deterministic exponential growth curve is plotted against a number of realizations of a stochastic growth process. We find for example that **65**% of the processes exceed 100 bacteria after ten minutes.

Definition of a stochastic process

Using probability theory, a stochastic process X is a family

$$\{X_t \ : \ t \in T\}, \tag{20.3}$$

of random variables X_t indexed by some set T. All the random variables take values in a common state space, S. The state space can be discrete or continuous. Although we might use discrete state spaces in some of the examples, one should realize that most examples in physics belong to the continuous type.

Furthermore, the family is known as **discrete process** if the indices take discrete values $t \in \{0, 1, 2, \ldots\}$. This will in fact be the situation in the practical application of **Markov chain Monte Carlo** applications that we are aiming for. On the other hand, many physical stochastic processes are **continuous** and will have $T \in \mathbb{R}$ or $t \in [0, \infty)$. In either case we think of a stochastic process as a family of variables that evolve as time passes, although in general T might be any kind of ordering index and is not restricted to physical time.

Due to the randomness, different runs of the stochastic process will generate different sequences of outputs. An observer recording many such sequences generated from the same stochastic process would need to measure all the following conditional probabilities to fully describe the process:

$$\begin{aligned} &p_{X_0}(x_0), \\ &p_{X_1|X_0}(x_1|x_0), \\ &p_{X_2|X_0,X_1}(x_2|x_0,x_1), \\ &\vdots \\ &p_{X_n|X_0,X_1,\ldots,X_{n-1}}(x_n|x_0,x_1,\ldots,x_{n-1}). \end{aligned} \tag{20.4}$$

Example 20.2 (Conditional probabilities of a stochastic process)

Below is an example of a stochastic process (implemented using an abstract python class `StochasticProcess`).

Note that the process indeed is defined by:

1. the distribution of the first random variable X_0, which is included in the definition of class method `start`, and
2. the conditional distributions for subsequent variables (as defined in the class method `update`).

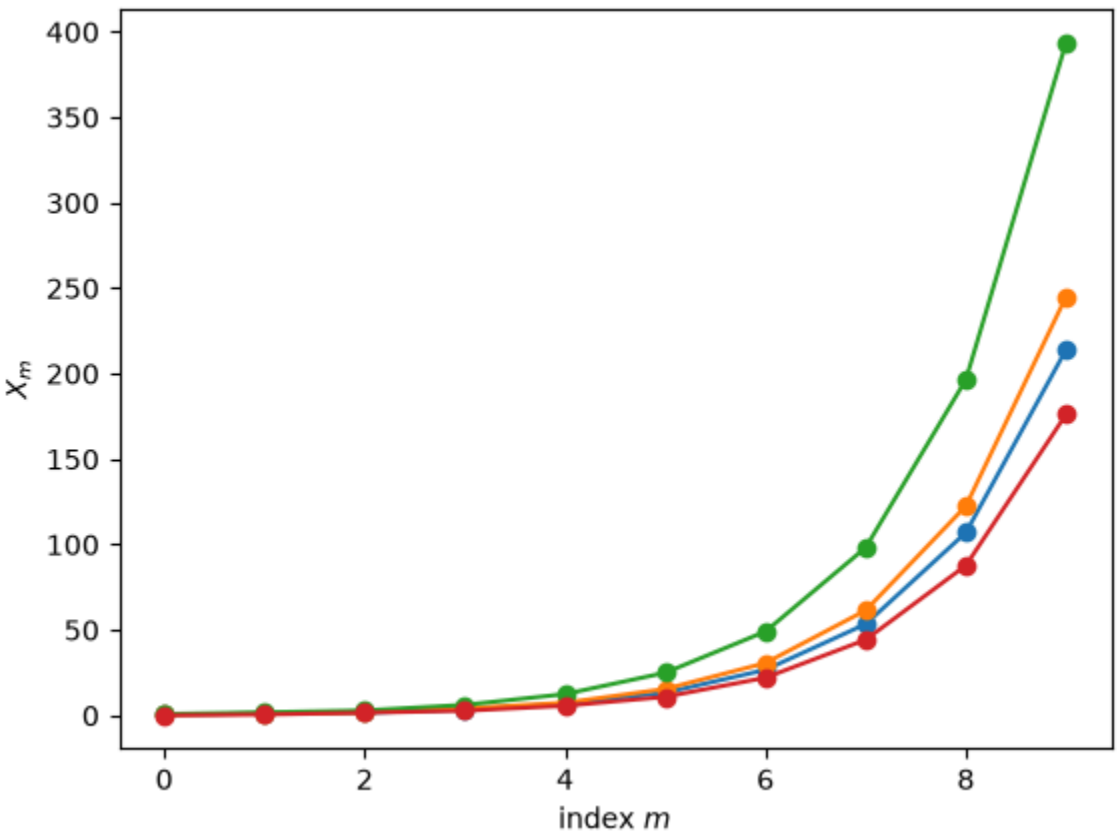
Two relevant visualizations are also included in the example below

- Plots of the first variables from a few runs.
- A corner plot of bivariate and univariate marginal distributions.

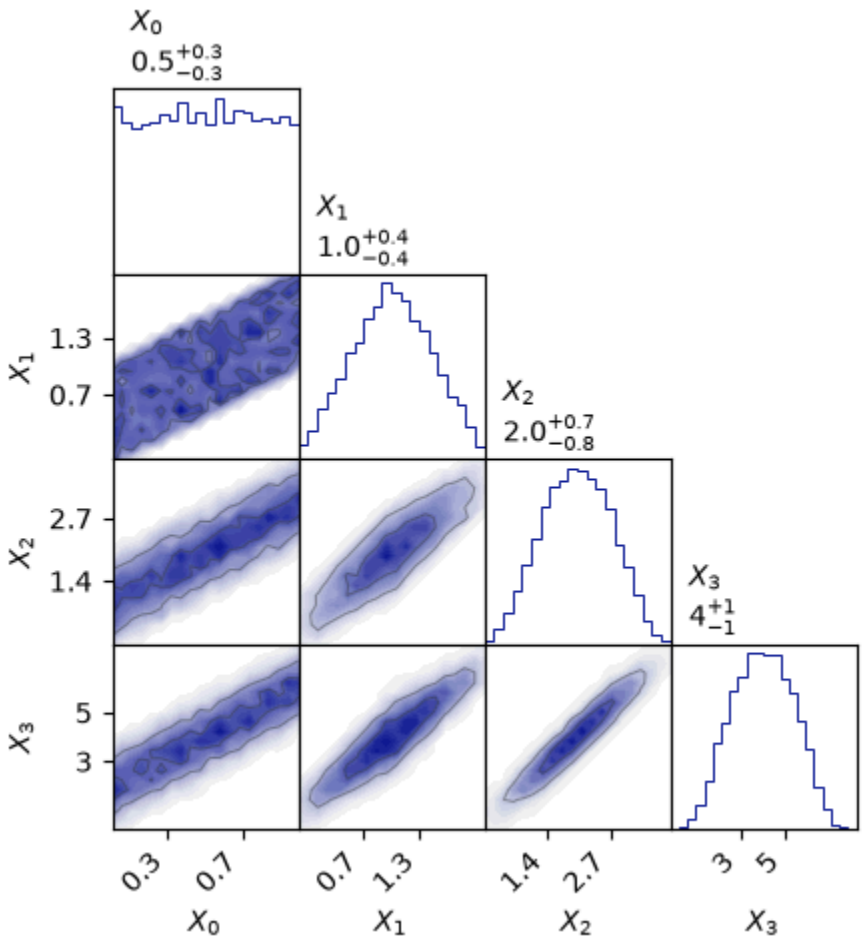
```
import numpy as np
import sys
import os
# Adding ../assets/ to the python module search path
sys.path.insert(0, os.path.abspath('../assets/'))

from StochasticProcess.StochasticProcess import StochasticProcess as SP

class StochasticProcessExample(SP):
    def start(self,random_state):
        return random_state.uniform()

    def update(self, random_state, history):
        return np.sum(history) + random_state.uniform()
```

```
# Create an instance of the class. It is possible to provide a seed
# for the random state such that results are reproducible.
example = StochasticProcessExample(seed=1)

# The following method call produces 4 runs, each of length 10.
example.create_multiple_processes(10,4)
```

```
# and here plotted
fig_runs, ax_runs = example.plot_processes()
```



```
# Pretty corner plots generated with `prettyplease`
# https://github.com/svisak/prettyplease
import prettyplease as pp

# Here we instead create 10,000 runs (to collect statistics), each of length 4.
example.create_multiple_processes(4,10000)
fig_corner = pp.corner(example.sequence.T,labels=[fr'$X_{{{i}}}$' for i in range(13)])
```



Exercise 20.1

Can you understand the initialization and update rules of Example 20.2?

Exercise 20.2

For each panel of the pairplot above:

- What conditional distribution of random variables does each panel show?
- What random variables are marginalized out, if any?
- Explain how the pairplot answers the question “Are X_3 and X_0 independent?”?

Hints:

- The top-left panel shows $p\left(X_0\right)$, with no marginalization since X_0 starts the random sequence.
- The independence of X_3 and X_0 would imply $p\left(X_3 \mid X_0\right) = p\left(X_3\right)$.

Exercise 20.3

Define your own stochastic process by creating your own initial and update methods within the StochasticProcess class. Explore the conditional probabilities by creating many chains and making plots.

Solutions to selective exercises

Here are answers and solutions to selected exercises.

Solution to Exercise 20.1

- The initial variable is from a uniform distribution: $p_{X_0}(x_0) = \mathcal{U}([0,1])$.
- Variable X_n is obtained from $p_{X_n|X_0,X_1,\ldots,X_{n-1}}(x_n|x_0,x_1,\ldots,x_{n-1}) = \mathcal{U}([0,1]) + \sum_{i=0}^{n-1} x_i$

Solution to Exercise 20.2

The histograms on the diagonal show the distributions $p(X_n)$ which are marginalized over $X_{n-1},\ldots,X_0$. The off-diagonal plots show the joint distributions $p(X_i,X_j)$ which are marginalized over $X_{n-1},\ldots,X_0$ with $n = \max(i,j)$ and excluding $X_{\min(i,j)}$. Note that these are distributions of random variables X_n, not random values x_n.

More explicitly:

- $p(X_0)$ has no correlated random variables marginalized out since it starts the sequence.
- $p(X_1)$ is marginalized over X_0.
- $p(X_2)$ is marginalized over X_1, X_0.
- $p(X_3)$ is marginalized over X_2, X_1, X_0.
- $p(X_0, X_1)$ has no correlated random variables marginalized out.
- $p(X_1, X_2)$ is marginalized over X_0.
- $p(X_2, X_3)$ is marginalized over X_0, X_1.
- $p(X_1, X_3)$ is marginalized over X_0, X_2.
- $p(X_0, X_3)$ is marginalized over X_1, X_2.
- $p(X_0, X_2)$ is marginalized over X_1, X_3.

The dependence of X_3 on X_0 is not directly observed since $p(X_3 \mid X_0)$ is not shown. However, from Eq. (39.2) we have that $p(X_3 \mid X_0) = p(X_0, X_3)/p(X_0)$. And since $p(X_0)$ is uniform, we find that $p(X_3 \mid X_0) \neq p(X_3)$. In other words, X_3 "remembers" X_0.

Solution to Exercise 20.3

Here's one possible solution. It is a slightly more general random walk.

```
class exerciseSP(SP):
    def start(self,random_state):
        return random_state.uniform()

    def update(self, random_state, history):
      return history[-1] + random_state.uniform()
```

20.2. Example: Random walk

One of the simplest random processes is the so called *random walk*. This process can arise in many ways. A standard example is the coin-flip casino game in which a gambler G starts with a fortune a. The croupier tosses a (possibly biased) coin. Each time heads appears, the gambler gets one euro (let us assume that the game takes place in Monte Carlo). If tails appears, the player loses one euro.

The player's fortune S_n after n tosses is

$$S_n = a + \sum_{i=1}^{n} X_i, \tag{20.5}$$

where X_i is a random variable with possible outcomes $\{-1, 1\}$. The probability mass function is $\{p(X_i = -1) = p, p(X_i = 1) = 1 - p\}$, where an unbiased coin would have $p = 0.5$. The fortune S_n can be considered a random walk stochastic process. A number of different scenarios can be considered by imposing external constraints on the gambler's fortune (such as the termination of the game when $S_n = 0$).

Example 20.3 (Simple random walk)

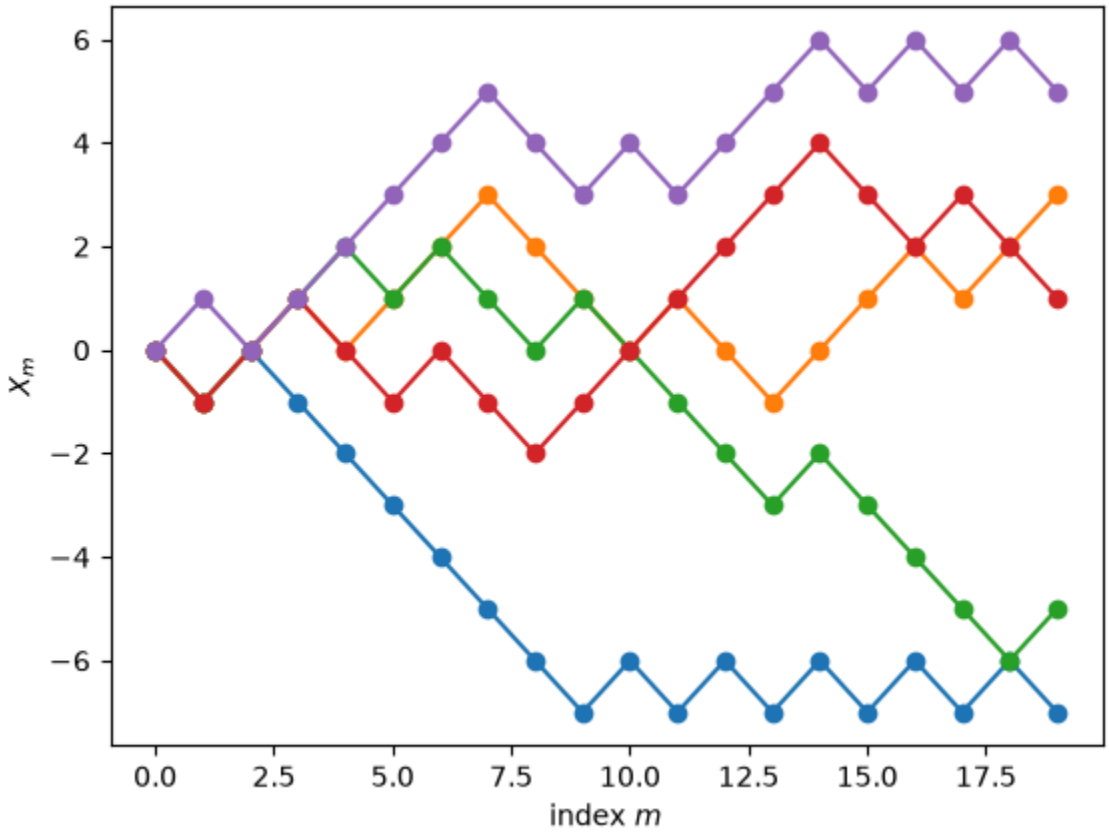
In the following code example (again using the `StochasticProcess` class) we create a large number of such processes starting at $X_0 = 0$. Here we investigate the questions:

- What is the mean position of the random walk after n steps?
- What is the average distance traveled after n steps?

Can you formulate these questions as some moments of a probability distribution?

```
import numpy as np
import sys
import os
sys.path.insert(0, os.path.abspath('../assets/'))
from StochasticProcess.StochasticProcess import StochasticProcess as SP

class randomwalk(SP):
    def start(self,random_state):
        return 0.

    def update(self, random_state, history):
        step = np.sign(random_state.uniform()-0.5)
        return history[-1]+step

test=randomwalk(seed=1)
test.create_multiple_processes(20,5)
test.plot_processes()

test.create_multiple_processes(101,5000)

for step in [9,16,25,36,49,64,81,100]:
    mean = test.sequence[step,:].mean()
    std = test.sequence[step,:].std()
    print(f'After {step:>3} steps: mean = {mean:5.3f}, std = {std:6.2f}')
```

```
After   9 steps: mean = -0.021, std =   3.00
After  16 steps: mean = -0.004, std =   4.02
After  25 steps: mean = 0.017, std =   4.99
After  36 steps: mean = -0.043, std =   6.01
After  49 steps: mean = -0.015, std =   7.00
After  64 steps: mean = 0.047, std =   8.01
After  81 steps: mean = 0.112, std =   9.19
After 100 steps: mean = 0.013, std =  10.27
```



These walks can be made more general by allowing the steps X_n to have some continuous distribution on the reals, and it can be placed in a multi-dimensional space with both positions and steps in $\mathbb{R}^p$. A particularly well-known random walk process involves the motion of a particle in a fluid where it is subjected to multiple collisions. This gives a so called *Brownian motion*.

20.3. Special case: Gaussian process

The joint density function for a multivariate normal distribution was presented in Eq. (40.5) for $\boldsymbol{x} = (x_1, x_2, \ldots, x_k)$ corresponding to random variables $X_1, X_2, \ldots X_k$. This normal distribution is completely determined by its mean vector, $\boldsymbol{\mu}$, and covariance matrix, $\boldsymbol{\Sigma}$, where elements $\mu_i = \mathbb{E}[X_i]$ and $\Sigma_{ij} = \text{Cov}\ (X_i, X_j)$.

The multivariate normal distribution has the remarkable property that all marginal and conditional distributions are normal, and specified by the corresponding subsets of the mean vector and covariance matrix.

A Gaussian process extends the multivariate normal distribution to a stochastic process with a continuous time index T.

Definition 20.1 (Gaussian process)

A Gaussian process $(X(t))_{t\geq 0}$ is a stochastic process with a continuous-time index $t \in [0, \infty)$ if each finite-dimensional vector of random variables

$$X(t_1), X(t_2), \ldots, X(t_k)$$

has a multivariate normal distribution for $0 \leq t_1 < \ldots < t_k$.

A Gaussian process is completely determined by its *mean function* $\mu(X(t))$ and *covariance function* $C(X(s), X(t))$ for $s, t \geq 0$.

A Gaussian process is stationary if the mean function $\mu(X(t))$ is constant for all t and if the covariance function fulfils $C(X(s), X(t)) = C(X(s+h), X(t+h))$ with $h \geq 0$. Note that stationarity is not a requirement for a Gaussian process.

We return to discuss Gaussian processes in detail in Overview of Gaussian processes.

20.4. Markov chains

> "Proceeding by guesswork."
>
> —Plato (attr.), on the meaning of stochastic

Many relevant stochastic processes share the following property: Conditional on their value at step n, the future values do not depend on the previous values. Such processes can be said to have a very short memory and are known as Markov chains. Let us for simplicity in notation focus on discrete processes where the time index takes integer values.

Definition 20.2 (Markov chains)

The stochastic process X is a Markov chain if it has the **Markov property**

$$\mathbb{P}_{X_n|X_0,X_1,\ldots,X_{n-1}}(i_n|i_0, i_1, \ldots, i_{n-1}) = \mathbb{P}_{X_n|X_{n-1}}(i_n|i_{n-1}) \tag{20.6}$$

for a countable (discrete) sample space. For continuous variables we formulate this property using conditional distribution functions

$$P_{X_n|X_0,X_1,\ldots,X_{n-1}}(x_n|x_0, x_1, \ldots, x_{n-1}) = P_{X_n|X_{n-1}}(x_n|x_{n-1}) \tag{20.7}$$

An alternative, expressive formulation is

> Conditional on the present, the future of a Markov chain does not depend on the past.

An observer of a Markov process would only have to measure the one-step conditional probability distributions

$$\begin{aligned} &P_{X_0}(x_0), \\ &P_{X_1|X_0}(x_1|x_0), \\ &P_{X_2|X_1}(x_2|x_1), \\ &\vdots \\ &P_{X_n|X_{n-1}}(x_n|x_{n-1}), \end{aligned} \tag{20.8}$$

to understand the process.

Stationary processes

An important subset of Markov processes are **stationary** which implies that the conditional probability distribution does not depend on the position in time. In combination with the Markov property, this can be viewed as a process with very *short-term memory* and *no sense of time*. An observer of a stationary Markov process only needs to measure

$$\begin{aligned} &P_{X_0}(x_0), \\ &P_{X_n|X_{n-1}}(x_n|x_{n-1}), \end{aligned} \tag{20.9}$$

for any n, to have a complete description. Here is a more formal definition.

Definition 20.3 (Stationary processes)

Given a countable (discrete) sample space S, the Markov chain X is stationary if

$$\mathbb{P}_{X_{n+1}|X_n}(j|i) = \mathbb{P}_{X_1|X_0}(j|i) \equiv T(i,j) \tag{20.10}$$

for all n, i, j. Here we have also introduced the (stationary) **transition density** T, which here becomes an $|S| \times |S|$ matrix with elements $T(i,j)$.

For continuous variables we use (conditional) distribution functions and we introduce the continuous transition density

$$P_{X_{n+1}|X_n}(x_j|x_i) = P_{X_1|X_0}(x_j|x_i) \equiv T(x_i, x_j). \tag{20.11}$$

Terminology

Markov chains with this property are also known as *(time-)homogeneous*. This meaning of “stationary” should not be confused with the *stationary distribution* of a chain, which is introduced below.

Be aware since the transition density is sometimes denoted $T(x_j \leftarrow x_i)$ or even $T_{j,i}$.

Exercise 20.4 (Stochastic matrix)

The transition density T is a so called *stochastic matrix*. What properties must it have? (consider the discrete case with matrix elements $T(i,j)$).

Exercise 20.5 (Simple random walk)

- How can you see that the random walk introduced in Example 20.3 is a Markov process?
- Is it stationary?
- If so, what are the matrix elements of the transition density?

In the following we will only consider stationary Markov chains and we stress that these are fully defined by $P_{X_0}(x_0)$ and $T\left(x_i, x_j\right)$.

Example 20.4 (A simple Markov process)

Let us construct a simpe Markov process using again the abstract python class (`StochasticProcess`).

We initialize the chain with a uniform random variable $p_{X_0}(x_0) = \mathcal{U}\left([0,1]\right)$ and update it using the conditional probability density function

$$p_{X_{n+1}|X_n}\left(x_j|x_i\right) = \mathcal{U}\left(\left[\frac{x_i}{2}, \frac{x_i+1}{2}\right]\right). \tag{20.12}$$

The simulations in Fig. 20.2 and Fig. 20.3 includes two relevant visualizations:

- Plots of the first variables from a few runs.
- A corner plot of bivariate and univariate marginal distributions.

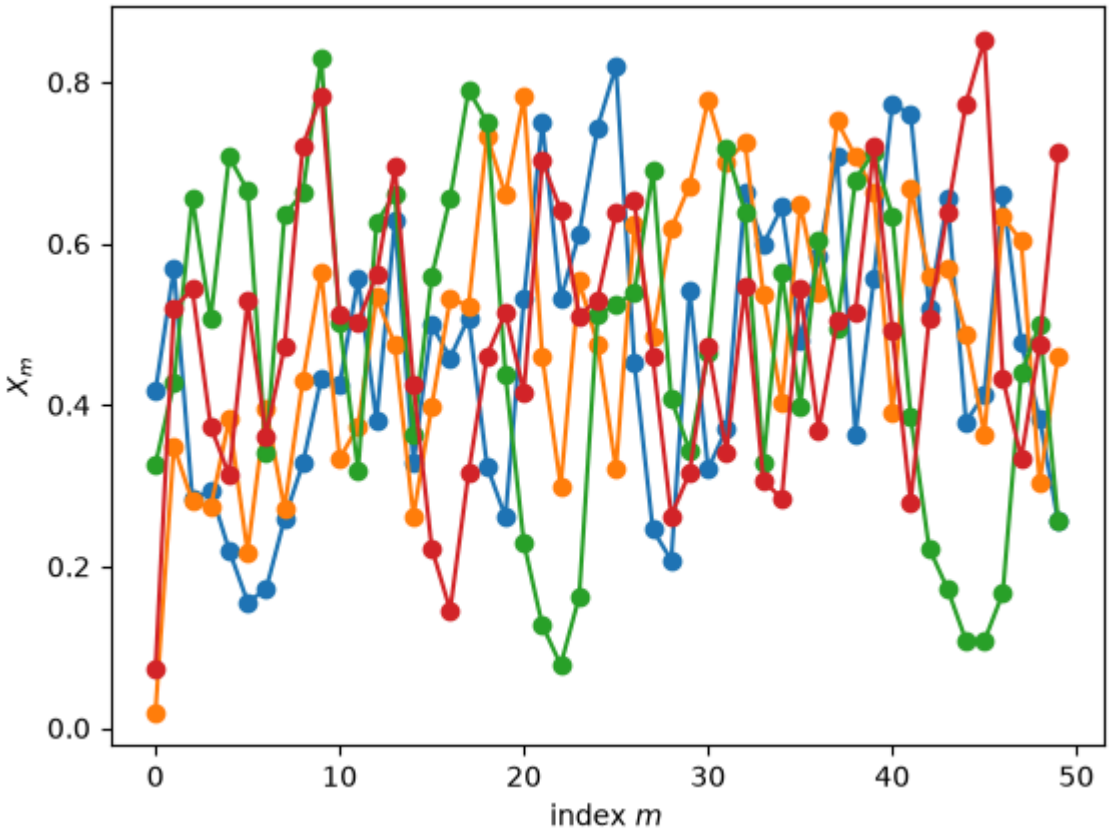


Fig. 20.2 First 50 variables in four realisations of the chain.

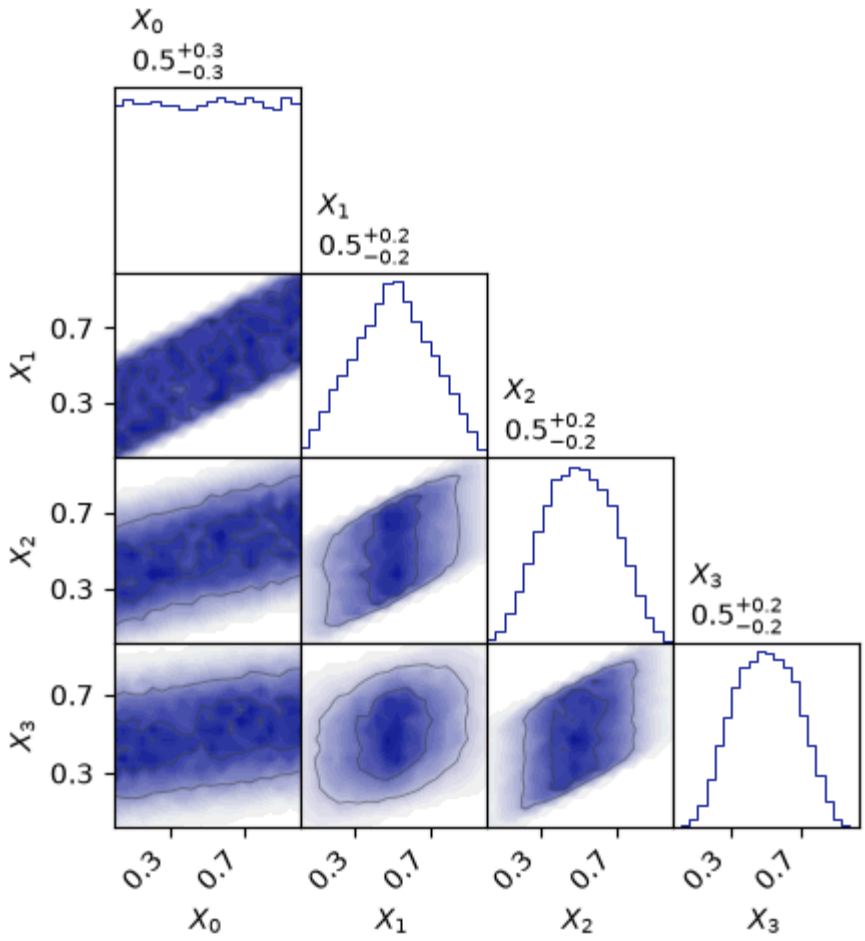


Fig. 20.3 Joint bivariate and univariate probability distributions. The statistics to produce these plots were gathered from 40,000 realisations of the Markov chains.

```
import numpy as np
import matplotlib.pyplot as plt
from myst_nb import glue

import sys
import os
# Adding ../assets/ to the python module search path
sys.path.insert(0, os.path.abspath('../assets/'))

from StochasticProcess.StochasticProcess import StochasticProcess as SP

class MarkovProcessExample(SP):
    def start(self,random_state):
        return random_state.uniform()

    def update(self, random_state, history):
        return 0.5 * ( history[-1] + random_state.uniform() )
```

```
# Create an instance of the class. It is possible to provide a seed
# for the random state such that results are reproducible.
example = MarkovProcessExample(seed=1)

# The following method calls produce 4 runs, each of length 50, and plots them.
example.create_multiple_processes(50,4)
fig_runs, ax_runs = example.plot_processes()
glue("MarkovProcessExample_runs_fig", fig_runs, display=False)
```

▼ **Hide code cell output**

```
# Marginal, joint distrubtions
import prettyplease as pp

# Here we instead create 40,000 runs (to collect statistics), each of length 13.
example.create_multiple_processes(13,40000)
# but we just plot the first four variables
fig_corner = pp.corner(example.sequence[:4,:].T,labels=[fr'$X_{{{i}}}$' for i in range(4)])
glue("MarkovProcessExample_corner_fig", fig_corner, display=False)
```

▼ **Hide code cell output**

Stationary and limiting distributions

We will now explore the long-time evolution of a Markov chain. We will, for simplicity, consider a countable sample space although our final application will be to continuous probability distributions.

First, we note that even the Markov process with its very short memory (dependence on just the previous value) will maintain an imprint of earlier positions. This imprint, however, will be finite. Let us explore the first few steps of the chain. The conditional probability of being in state k at position 2 given that we started in position i at $t = 0$ can be obtained using the product rule (39.3)

$$\begin{aligned}\mathbb{P}\left(X_2 = k \mid X_0 = i\right) &= \sum_j \mathbb{P}\left(X_2 = k, X_1 = j \mid X_0 = i\right) \\ &= \sum_j \mathbb{P}\left(X_2 = k \mid X_0 = i, X_1 = j\right)\mathbb{P}\left(X_1 = j \mid X_0 = i\right) \\ &= \sum_j \mathbb{P}\left(X_2 = k \mid X_1 = j\right)\mathbb{P}\left(X_1 = j \mid X_0 = i\right),\end{aligned} \tag{20.13}$$

where we have employed the Markov property in the last equality. We can now compare this with the probability of being in state k at position 2 without any condition on the starting position

$$
\begin{aligned}
\mathbb{P}\left(X_2 = k\right) &= \sum_{i,j} \mathbb{P}\left(X_2 = k, X_1 = j, X_0 = i\right) \\
&= \sum_{i,j} \mathbb{P}\left(X_2 = k \mid X_0 = i, X_1 = j\right) \mathbb{P}\left(X_1 = j, X_0 = i\right) \\
&= \sum_{j} \mathbb{P}\left(X_2 = k \mid X_1 = j\right) \mathbb{P}\left(X_1 = j\right),
\end{aligned}
\tag{20.14}
$$

where we have again used the product rule and the Markov property. In the final equality we also used the marginalization of the joint probability. The comparison of Eqs. (20.13) and (20.14) reveals that the two probabilites are not equal in general. They would be equal if $\mathbb{P}\left(X_1 = j \mid X_0 = i\right) = \mathbb{P}\left(X_1 = j\right)$, which is only fulfilled if the positions are in fact completely independent.

Exercise 20.6 (Remnant memory)

Can you visually verify the remnant memory by identifying the distributions (20.13) and (20.14) in Fig. 20.3 of Example 20.4?

Hint: The conditional distribution is actually not shown in Fig. 20.3 but is easy to obtain using the simple initial distribution and Eq. (39.2).

The remnant memory in the process from Example 20.4 is considered in Exercise 20.6. Conditional distributions $p\left(x_m \mid x_n\right)$ (for $m > n$) can be produced with the method `plot_conditional_distributions()` in the `StochasticProcess` class.

```
fig_X2givenX0,_,_,_,_ = example.plot_conditional_distributions([1,2],bins=20)
glue("X2givenX0_fig", fig_X2givenX0, display=False)
```

▼ **Hide code cell output**

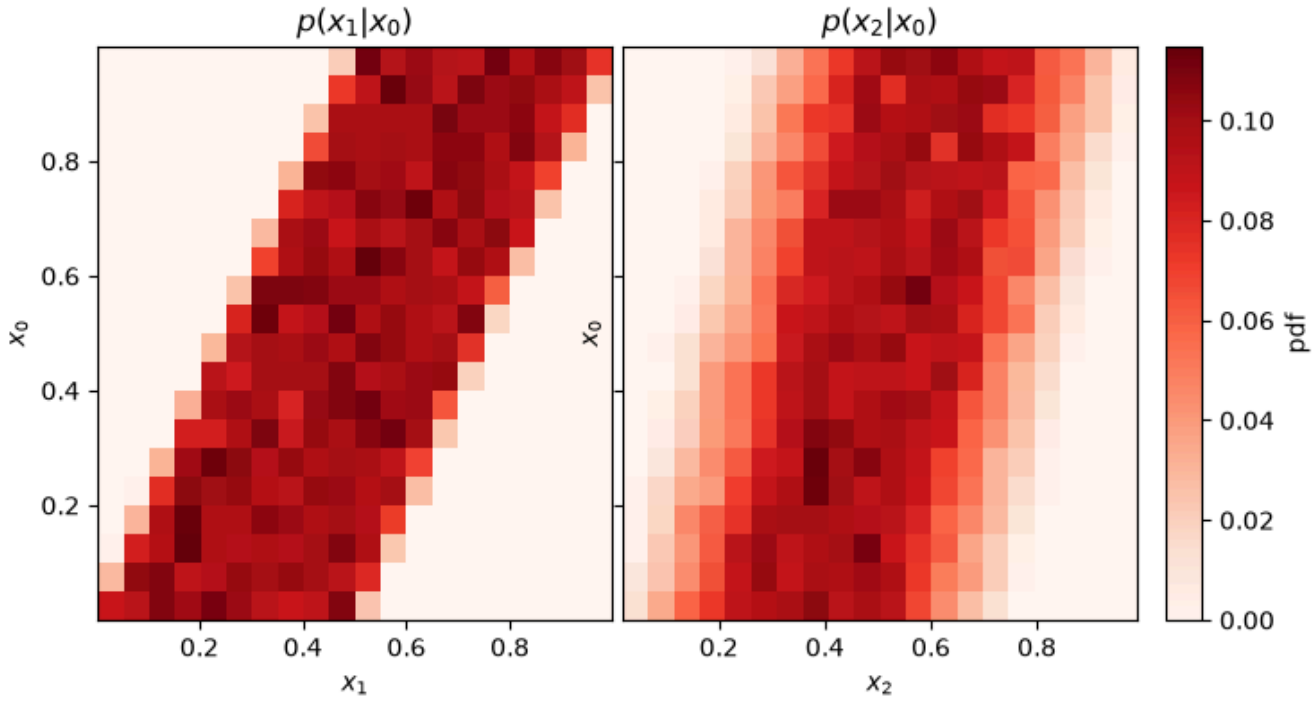


Fig. 20.4 The conditional distributions $p\left(x_1 \mid x_0\right)$ and $p\left(x_2 \mid x_0\right)$ for Example 20.4. Note that each of these is a one-dimensional distribution whose shape might depend on the value of x_0. This property is visualized here by histogramming the distributions. Each row corresponds to a bin of x_0 values and represents the corresponding $p\left(x_m \mid x_0\right)$ distribution. It is clear from this figure that these distributions are indeed dependent on the value of x_0.

Exercise 20.7 (Conditional distributions)

Consider Fig. 20.4.

- The speckling is due to using finite statistics to estimate the probability. How would this plot look different with infinite statistics?
- This left panel shows $p\left(x_1 \mid x_0\right)$. Would $p\left(x_2 \mid x_1\right)$ look any different?
- Are these conditional probabilities normalized along x_0? along $x_{1,2}$?
- How would this plot change if we changed the definition of `start()` in `MarkovProcessExample`?

We have shown that there is in fact some memory in a stationary Markov chain. However, it can be shown that a stationary Markov chain (with certain additional properties) will eventually reach an equilibrium where the distribution of X_n actually does settle down to a **limiting distribution**.

Definition 20.4 (Limiting distribution)

The vector π is called a limiting distribution of the Markov chain if it has entries $\left(\pi_j \ : \ j \in S\right)$ such that

1. $\pi_j \geq 0$ for all j
2. $\sum_j \pi_j = 1$
3. $\pi_j = \lim_{n\to\infty} T^n(i,j)$ for all i, j,

where T is the transition density of the Markov chain.

This definition is equivalent to the matrix equality

$$\pi = \lim_{n\to\infty} \alpha T^n, \tag{20.15}$$

where α is any intial distribution.

By the uniqueness of limits, the existence of a limiting distribution will imply that it is unique.

Assuming that π is a limiting distribution of a chain, then it is also a **stationary distribution** (also known as **equilibrium distribution**).

Definition 20.5 (Stationary distribution)

The vector π is called a stationary distribution of the Markov chain if it has entries $(\pi_j \ : \ j \in S)$ such that

1. $\pi_j \geq 0$ for all j
2. $\sum_j \pi_j = 1$
3. $\pi_j = \sum_i \pi_i T(i,j)$ for all j,

where T is the transition density of the Markov chain.

The last equality can be written on matrix form as

$$\pi = \pi T \tag{20.16}$$

and can be seen as an eigenvalue equation (with eigenvalue 1) where π is a *left* eigenvector.

A limiting distribution π is also a stationary one. According to Eq. (20.16) we just need to show that $\pi T = \pi$. See Exercise 20.12 to complete the proof.

This implies that the random variable X_n will be distributed according to π for large enough n and that it will be stationary as time passes.

A chain has a unique **stationary distribution** if it fulfills

irreducibility
all states *communicate* with each other which basically implies that all parts of the state space can be reached regardless of starting position;
and if all states are

positively recurrent
the probability of eventually returning to position i, having started from i, is 1 and the expected return time is finite for all i (for an irreducible chain with a finite state space this is always the case).
In addition, with the additional property of

aperiodicity
returns to any position i are not restricted to multiples of some period $d > 1$;
it can be shown that the stationary Markov chain will converge to this stationary distribution regardless of starting point. It is then also the unique **limiting distribution**. This is the so called **limit theorem**. Henceforth we will only deal with irreducible, positively recurrent, aperiodic chains unless otherwise indicated.

Example 20.5 (Stationary distribution of "A simple Markov process")

Let us consider again the Markov process from Example 20.4. We could see in Fig. 20.4 that there was some remnant memory of x_0 after two steps. However, according to the discussion above, this memory should eventually disappear and we will be left with the stationary (or *equilibrium*) distribution. This is clearly seen in Fig. 20.5.

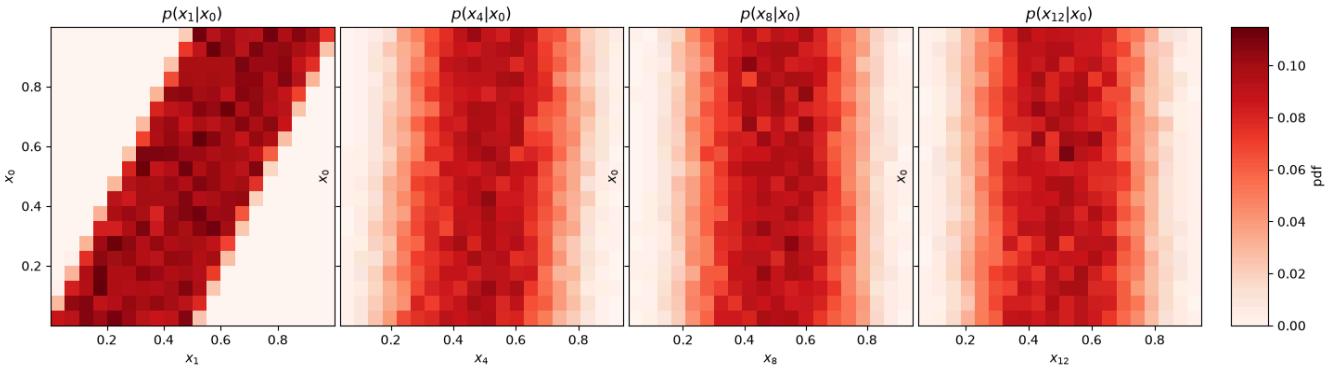
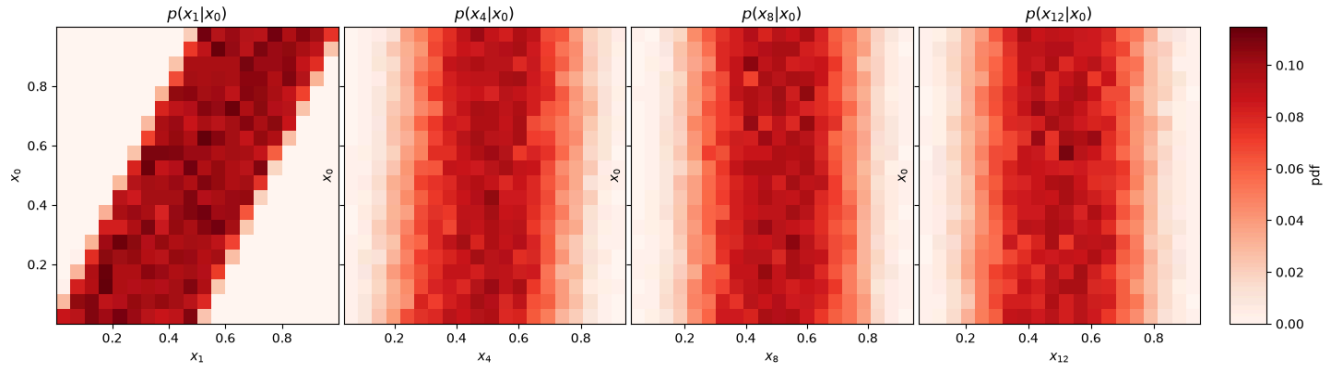


Fig. 20.5 The appearance of a limiting distribution already after ≈ 8–12 iterations.

▼ **Hide code cell content**



For practical purposes there are two issues to deal with:

- There is no way to know in advance how big n needs to be to reach equilibrium.
- Given a stationary Markov chain, we can generate samples from its equilibrium distribution; but how do we construct a chain to sample a specific distribution?

The latter problem is actually an example of an **inverse problem**. These are in general very difficult to solve. However, in this case there is a very clever, general class of solutions that we will look at in the next chapter.

Exercise 20.8 (Stationary distribution)

Show that Definition 20.5 implies that there is a stationary distribution π such that $\mathbb{P}(X_n = i) = \mathbb{P}(X_{n-1} = i) = \pi_i$

Reversibility

Another important property that some Markov chains can have is reversability. This property can also be seen as an invariance under reversal of time. Let us consider X, an irreducible, positive recurrent Markov chain of length N, $\{X_n : 0 \leq n \leq N\}$, with transition matrix T_X and stationary distribution $\boldsymbol{\pi}$. Suppose further that X_0 has the distribution $\boldsymbol{\pi}$ such that all X_n also have the same distribution.

Define the reversed chain Y with elements $Y_n = X_{N-n}$. One can show that this sequence will also be a Markov chain, and that its transition matrix T_Y will be given by

$$T_Y(i,j) = \mathbb{P}_{Y_{n+1}|Y_n}(j|i) = \frac{\pi_j}{\pi_i} T_X(j,i). \tag{20.17}$$

The chain Y is called the *time-reversal* of X, and we say that X is *reversible* if $T_X = T_Y$.

Considering a reversible chain with transition density T and stationary distribution $\boldsymbol{\pi}$, the property of reversibility can be formulated as a **detailed balance**

$$\pi_i T(i,j) = \pi_j T(j,i), \tag{20.18}$$

for all states i and j.

Exercise 20.9 (Reversibility)

- Are all stationary chains reversible?
- Are all reversible chains stationary?

Example 20.6 (A reversible Markov process)

We can construct a transition denisty that fulfills detailed balance by applying an update rule that is composed of a step proposal and an acceptance decision:

```
def update(self, random_state, history):
        propose = history[-1] + 0.5*(random_state.uniform()-0.5)
    if propose<0 or propose>1:
        return history[-1]
    else:
        return propose
```

This family of transition densities will be discussed in more detail in the next chapter. For now, we just simulate the above chain and study a set of chains in Fig. 20.6 and a subset of conditional probabilities in Fig. 20.7 (produced with the script below). Some relevant questions are listed in Exercise 20.10.

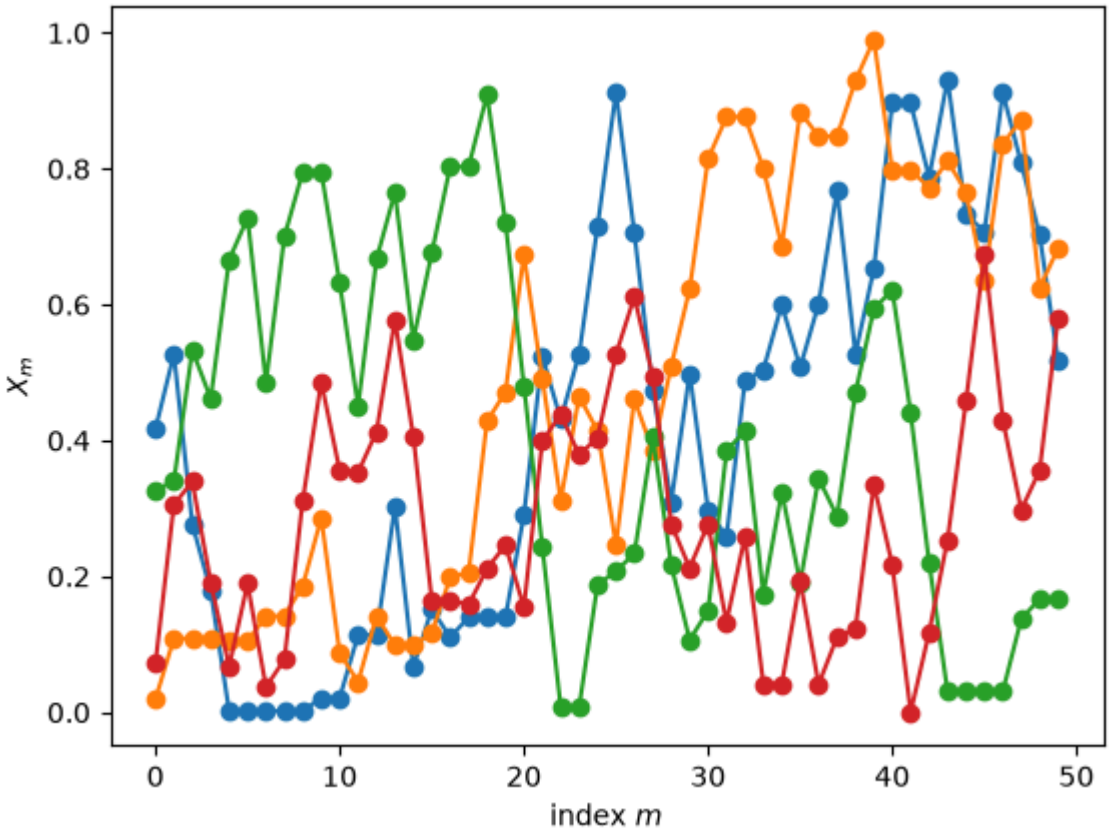


Fig. 20.6 Four chains produced with the reversible Markov chain.

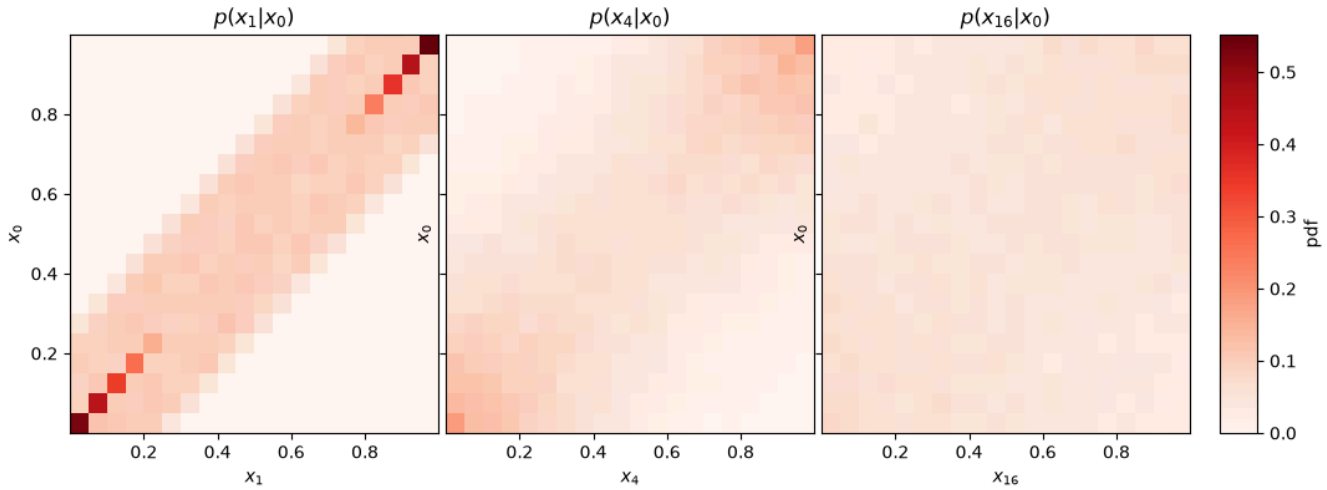
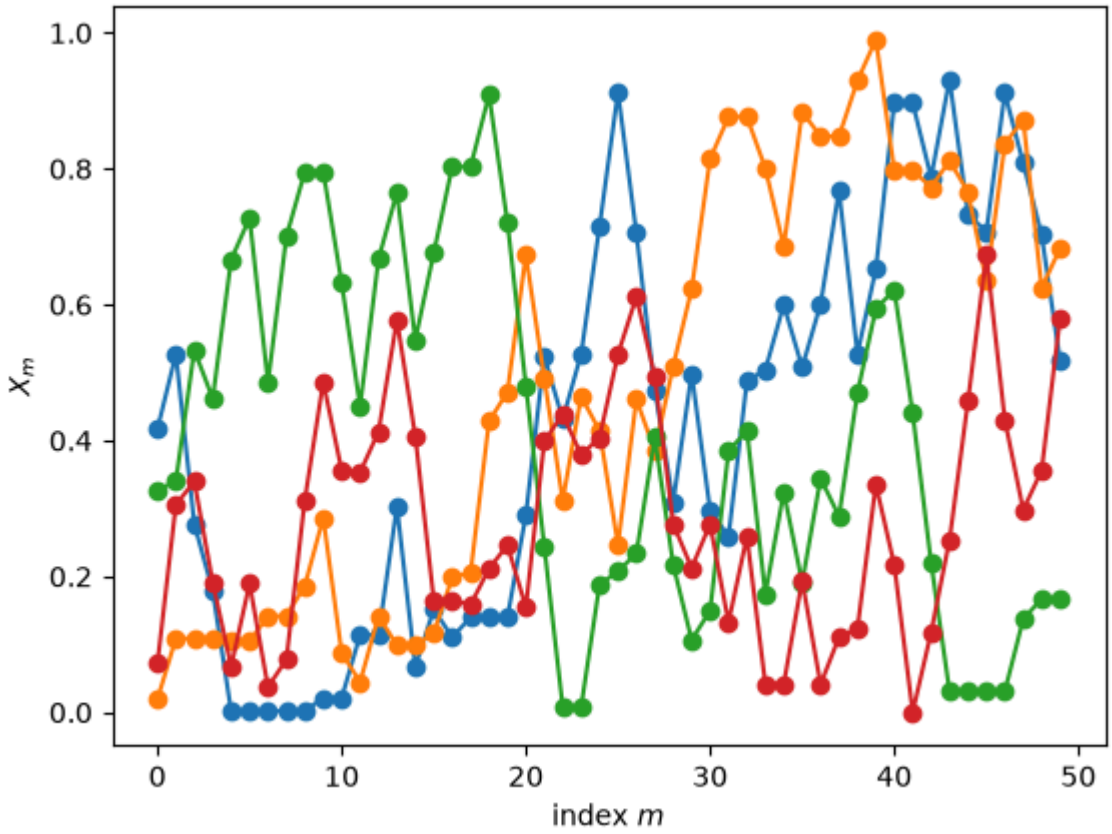
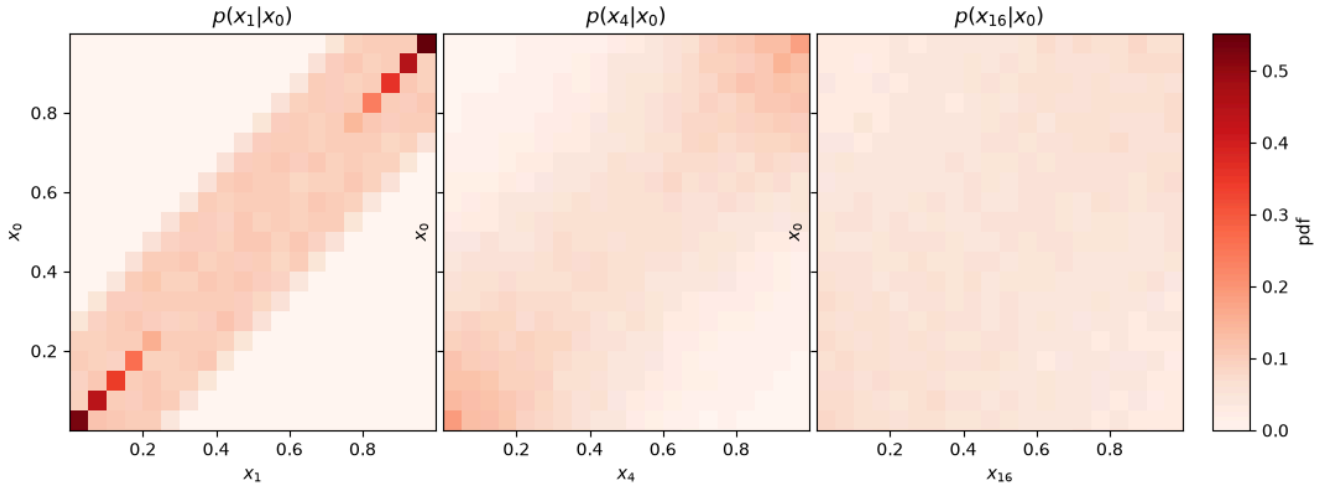


Fig. 20.7 Conditional probabilities for the reversible Markov chain. See the caption of Fig. 20.4 for explanations how to interpret these plots.

▼ **Hide code cell content**





Exercise 20.10 (A reversible Markov chain)

Consider Fig. 20.7.

- Where is the transition density reflected in this plot?
- Is this a reversible Markov chain?
- Can you guess what is the equilibrium distribution?

Metropolis design

A Markov chain that reaches an equilibrium with a stationary distribution π will have all subsequent outcomes distributed according to π. Here we consider discrete chains with a discrete sample space.

Sampling a distribution

Consider a sequence of outcomes $(i_0, i_1, i_2, \ldots)$ from a stationary Markov chain.

1. The first outcome, i_0, is in practice a sample from the initial probability distribution $\pi^{(0)}$. Note that $\pi_i^{(0)} = \mathbb{P}_{X_0}(i)$.
2. The second outcome, i_1, is a sample from $\pi^{(1)} = \pi^{(0)}T$. In practice, since we have the first outcome i_0, we obtain this second outcome as a sample from the conditional distribution for X_1 given X_0, i.e., a distribution with elements $\mathbb{P}_{X_1|X_0}(i|i_0) = T(i_0, i)$.
3. Continuing the process of drawing samples from conditional distributions we find that the n:th outcome i_n is a sample from $\pi^{(n)} = \pi^{(0)}T^n$, but in practice we obtain it as a draw from $\mathbb{P}_{X_n|X_{n-1}}(i|i_{n-1}) = T(i_{n-1}, i)$.

Dismissing the first n outcomes, for which we have not yet reached equilibrium, we have the sequence $(i_{n+1}, i_{n+2}, \ldots)$. These are, respectively, samples from $\pi^{(n+1)}, \pi^{(n+2)}, \ldots$. Assuming that the Markov chain has reached equilibrium, then all of these distributions are in fact the same. Therefore, our sequence of outcomes after equilibration are samples from the *same* distribution π.

Now we would like to *design* a Markov chain, via its transition matrix T, such that it has a desired probability distribution π as its limiting distribution. Let us first recapitulate some important facts concerning Markov chains.

1. A Markov chain is guaranteed to have a limiting distribution when the transition matrix T fulfills certain conditions (irreducibility and aperiodicity, see Stationary and limiting distributions).
2. A limiting distribution π is also a stationary distribution: $\pi = \pi T$.
3. A distribution π that fulfills detailed balance, $\pi_i T(i,j) = \pi_j T(j,i)$, is guaranteed to be a stationary distribution (this is shown in the proof below).

We utilize these facts in the so called Metropolis design of a Markov chain. The idea is to construct the transition matrix in a product form. Consequently, a transition from position i to a different position j happens in two steps: a new position j is proposed, and the proposal is then either accepted or rejected. The corresponding (conditional) probabilities are collected in a step proposal matrix S and an acceptance matrix A with elements

$$\begin{aligned} S(i,j) &= \mathbb{P}\left(\text{proposing next position } \quad j \;\middle|\; \text{current position is } \quad i\right), \\ A(i,j) &= \mathbb{P}\left(\text{accepting new position } \quad j \;\middle|\; \text{current position is } \quad i \text{ and } \quad j \text{ is proposed } \quad\right), \end{aligned} \tag{20.19}$$

where we think of outcomes of subsequent random variables as positions in the sample space. If the proposal is rejected, the chain stays at position i. This gives a transition matrix in product form

$$T(i,j) = S(i,j)A(i,j), \quad \text{for } \; i \neq j, \tag{20.20}$$

$$T(i,i) = S(i,i) + \sum_{j \neq i} S(i,j)\,(1 - A(i,j)). \tag{20.21}$$

An off-diagonal element is the probability that position j is proposed *and* accepted. The chain stays at position i either if this position is proposed (the first term of the diagonal element), or if a move to some other position j is proposed (probability $S(i,j)$) and rejected (probability $1 - A(i,j)$).

The question is now how to choose the acceptance probabilities such that the chain samples the desired distribution π.

Theorem 20.1 (The Metropolis design for obtaining a discrete limiting distribution)

Let π be a desired probability distribution with $\pi_i > 0$ for all positions i in a discrete sample space, and let S be a stochastic matrix (non-negative entries and row sums equal to one) with $S(i,j) > 0$ whenever $S(j,i) > 0$. A Markov chain with transition matrix T, whose off-diagonal elements are written in the product form $T(i,j) = S(i,j)A(i,j)$ with acceptance probabilities $A(i,j)$, can then be designed such that it has π as a stationary distribution. If the chain is also irreducible and aperiodic, then π is its unique limiting distribution.

Proof. We will prove the theorem in four steps:

1. *T must be a stochastic matrix.* We let a rejected proposal leave the chain at position i, which gives the diagonal elements in Eq. (20.21). Every proposed move from i to $j \neq i$ is then either accepted, with probability $S(i,j)A(i,j)$, or rejected, with probability $S(i,j)\,(1 - A(i,j))$. These two terms appear in the off-diagonal and the diagonal elements, respectively, and add up to $S(i,j)$. Summing over a row therefore gives

$$\begin{aligned}\sum_j T(i,j) &= \sum_{j\neq i} S(i,j)A(i,j) + \underbrace{S(i,i) + \sum_{j\neq i} S(i,j)\,(1 - A(i,j))}_{T(i,i)} \\ &= \sum_j S(i,j) = 1,\end{aligned}$$

   since S is a stochastic matrix. In addition, since $0 \leq A(i,j) \leq 1$ and $S(i,j) \geq 0$, all elements of T are non-negative.

2. *Detailed balance dictates acceptance probabilities.* We demand that T fulfills detailed balance with the target distribution π. That is, $\pi_i T(i,j) = \pi_j T(j,i)$. This is trivially fulfilled for $i = j$. For $i \neq j$ the product form (20.20) gives the condition

$$\pi_i S(i,j)A(i,j) = \pi_j S(j,i)A(j,i).$$

   If $S(i,j) = 0$, then also $S(j,i) = 0$ and the condition holds for any acceptance probabilities. Otherwise, it only constrains the ratio

$$\frac{A(i,j)}{A(j,i)} = \frac{\pi_j S(j,i)}{\pi_i S(i,j)} \equiv r_{ij},$$

   where r_{ij} is known as the *Metropolis ratio* for a move from i to j. Note that $r_{ji} = 1/r_{ij}$. The acceptance probabilities must be in $[0,1]$, and we want them as large as possible such that the chain moves efficiently. For each pair of positions we therefore set the larger of $A(i,j)$ and $A(j,i)$ equal to one:

   - If $r_{ij} \leq 1$, then $A(i,j) = r_{ij}$ and $A(j,i) = 1$.
   - If $r_{ij} > 1$, then $A(i,j) = 1$ and $A(j,i) = 1/r_{ij} = r_{ji}$.

   Both cases are summarized by the *Metropolis acceptance probability*

$$A(i,j) = \min(1, r_{ij}) = \min\left(1, \frac{\pi_j}{\pi_i}\frac{S(j,i)}{S(i,j)}\right). \tag{20.22}$$

3. *Detailed balance implies stationarity.* With this choice, $\pi_i T(i,j) = \pi_j T(j,i)$ for all i, j. Summing over i gives

$$\sum_i \pi_i T(i,j) = \sum_i \pi_j T(j,i) = \pi_j \sum_i T(j,i) = \pi_j,$$

   where the last step uses that the rows of T sum to one (step 1). This equality implies that $\pi = \pi T$ and consequently that π is a stationary distribution.
4. *Limiting distribution.* According to the limit theorem in Stationary and limiting distributions, an irreducible and aperiodic chain that has a stationary distribution converges to it from any starting point. (An irreducible chain with a stationary distribution is automatically positively recurrent.) The stationary distribution π is then the unique limiting distribution.

In practice, irreducibility must be ensured by the choice of the proposal matrix S, such that all positions can be reached. Aperiodicity is usually not a problem: an irreducible chain is aperiodic if $T(i,i) > 0$ for some i, which is the case as soon as some proposal can be rejected.

See Exercise 20.21 for an explicit example.

Additional exercises

Exercise 20.11 (Stationary two-state distribution)

Find the stationary distribution to the Markov chain with transition matrix

$$T = \begin{bmatrix} 1-p & p \\ q & 1-q \end{bmatrix}.$$

Exercise 20.12 (Limiting distribution)

(*i*) Assume that π is a limiting distribution of a Markov chain. Show that π is also stationary distribution.

(*ii*) Show by providing a counterexample that the converse of the above is not true.

Exercise 20.13 (Flip-flop)

Construct a transition matrix for a two-state system that will always flip state 1 $\leftrightarrow$ state 2.

- Does it have one or several stationary distributions?
- Is there a limiting distribution?

Exercise 20.14 (Gothenburg winter weather)

The winter weather in Gothenburg consists mostly of rain plus the occasional days of snow and clear skies. We might describe this winter weather as a Markov chain in which the conditional probabilities for tomorrow's weather just depends on the weather today. Here is a realistic transition matrix for a Gothenburg winter weather chain

$$T = \begin{array}{c} \\ s \\ r \\ c \end{array} \begin{array}{c} \begin{array}{ccc} s & r & c \end{array} \\ \begin{pmatrix} 0.2 & 0.6 & 0.2 \\ 0.1 & 0.8 & 0.1 \\ 0.1 & 0.6 & 0.3 \end{pmatrix} \end{array}$$

- What is the probability that it snows tomorrow given that it snows today?
- What is the minimum probability that it will rain tomorrow no matter what the weather today?
- Assume that the amount of precipitation is 2 mm if it snows, 5 mm if it rains, and 0 mm if the sky is clear. Given that the sky is clear today, what is the expected amount of precipitation tomorrow?
- The meterologist predicts 50% probability that it will snow tomorrow and 50% probability that it will rain. Find the probability that it will rain two days later (that is in three days from today).

Exercise 20.15 (Stationary Gothenburg winter weather)

Find the stationary distribution of the Gothenburg winter weather Markov chain from Exercise 20.14.

Test numerically that it is also a limiting distribution.

Exercise 20.16 (Is it reversible?)

Determine if the Markov chain with transition matrix

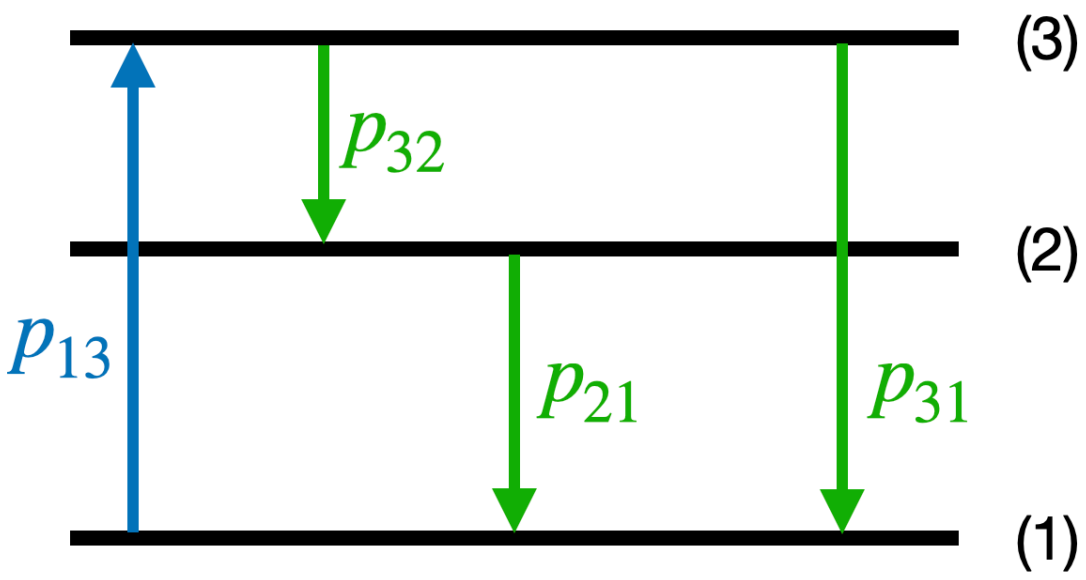
$$T = \begin{bmatrix} 0 & 2/5 & 3/5 \\ 1/2 & 1/4 & 1/4 \\ 1/2 & 1/6 & 1/3 \end{bmatrix}$$

is reversible.

Exercise 20.17 (Optical pumping)

In this exercise we will consider a simple quantum mechanical system that can be modeled using a Markov process.

Consider a quantum system that has three relevant energy levels labeled 1,2 and 3, see figure. We do not need to consider any quantum mechanics to solve this problem, but a few key properties are needed: An electron generally "wants" to be in the lowest possible energy state, and if it is excited to state 2 or 3 it will eventually decay to a state will lower energy within some time frame. The possible decays and their respective probabilities are also shown in the figure. However, by introducing an auxiliary driving mechanism, for example a laser, the electrons can transition from a lower energy level to a higher one. This is known as *optical pumping* and is depicted in the figure where electrons can go from state 1 to 3.



The probabilities are given by

$$p_{31} = 1/3, \quad p_{13} = 1/3, \quad p_{21} = 2/3, \quad p_{32} = 1/3.$$

The time evolution of this system can be modeled as a Markov process with state $\pi_n = (a_n^1, a_n^2, a_n^3)$ (at time t_n) which describes the distribution of the electrons among the three levels subject to the constraint $\sum_i a_n^i = 1$. An example is $\pi_n = (3/6, 2/6, 1/6)$ which describes that half of the particles are in the ground state, one third in the first excited state, and one sixth in the second excited state.

In our model the state will be updated at each time step (Δt) according to $\pi_{n+1} = \pi_n T$, where T is a matrix of transition probabilities where T_{ij} gives the probability that an electron in state i goes to state j.

We want to study the equilibrium distribution of electrons in this system, meaning that the state has evolved for a long time.

(*i*) Construct the transition matrix, T from the transition probabilities that are given in the figure.

(*ii*) Find the limiting distribution (if it exists).

(*iii*) Would a limiting distribution fulfill detailed balance? Why, why not?

(*iv*) (optional) Implement this process in the Python class `StochasticProcess` and simulate it for a few different starting configurations. Do you find the same limiting distribution as in the analytical calculation?

Exercise 20.18 (Detailed balance)

Prove Eq. (20.17) to show that a reversible Markov chain must fulfill detailed balance (20.18).

Solutions to exercises

Solution to Exercise 20.4 (Stochastic matrix)

The transition density matrix T has:

1. Non-negative entries; $T(i,j) \geq 0$ for all i, j.
2. Row sums equal to one; $\sum_j T(i,j) = 1$ for all i.

Solution to Exercise 20.5 (Simple random walk)

- The update rule only depends on the last variable that was sampled.
- Yes, it is stationary since the update rule does not depend on the position in the chain but only on the value of the last variable that was sampled.
- $T(i,j) = \begin{cases} 0.5 & j = i \pm 1, \\ 0 & \text{otherwise} \end{cases}$.

Solution to Exercise 20.6 (Remnant memory)

The marginal distribution $p(x_2)$ is shown in the third panel on the diagonal of Fig. 20.3. The fact that the initial distribution for X_0 is uniform implies that $p(x_2 \mid x_0) \propto p(x_2, x_0)$. The joint distribution on the right-hand side is shown in the third panel from the top in the fist column. The conditional distribution then corresponds to slices at different values of x_0. It is obvious that these slices do not look like $p(x_2)$ for all values of x_0. (See also Fig. 20.4.)

Solution to Exercise 20.7 (Conditional distributions)

- With infinite statistics, the diagonal band would be a uniform color since $p(x_1 \mid x_0)$ is uniform (within the diagonal limits shown) for each x_0.
- A plot of $p(x_2 \mid x_1)$, or any $p(x_{n+1} \mid x_n)$ would look identical since this is the definition of a stationary Markov chain.
- A conditional probability $p(\alpha \mid \beta)$ is normalized along α, with the value of β considered fixed. In other words, a `numpy` array of values `P[n,m]` tabulating $p(x_m \mid x_n)$ on a grid satisfies:

```
np.sum(P, axis=1) == 1
```

- This plot only shows the update rule and does not depend on how we chose an initial value for the Markov chain.

Solution to Exercise 20.8 (Stationary distribution)

Assume that $\mathbb{P}(X_{n-1} = i) = \pi_i$. Then

$$\begin{aligned}\mathbb{P}\left(X_n = i\right) &= \sum_j \mathbb{P}\left(X_n = i, X_{n-1} = j\right) \\ &= \sum_j \mathbb{P}\left(X_n = i \mid X_{n-1} = j\right)\mathbb{P}\left(X_{n-1} = j\right) \\ &= \sum_j \pi_j T(j,i) = \pi_i.\end{aligned}$$

Solution to Exercise 20.9 (Reversibility)

- A reversible chain is always stationary, but not vice versa.

Solution to Exercise 20.10 (A reversible Markov chain)

- The first panel, $p\left(x_1 \mid x_0\right)$, shows the transition density.
- A reversible Markov chain fulfills detailed balance, $\pi(x_n)p\left(x_{n+1} \mid x_n\right) = \pi(x_{n+1})p\left(x_n \mid x_{n+1}\right)$. With a uniform stationary distribution, as here, this implies a symmetric transition density $p\left(x_{n+1} \mid x_n\right) = p\left(x_n \mid x_{n+1}\right)$. Here we can study the first panel which indeed is symmetric under the interchange of its axes. In other words, mirroring the plot along the diagonal should not change its appearance.
- The equilibrium seem to provide samples from a uniform distribution $\mathcal{U}([0,1])$.

Solution to Exercise 20.11 (Stationary two-state distribution)

Solve the system of equations

$$\sum_i \pi_i T_{ij} = \pi_j, \quad \forall j$$

subject to the constraint $\pi_1 + \pi_2 = 1$. The solution is

$$\pi = \left(\frac{q}{p+q}, \frac{p}{p+q}\right).$$

Solution to Exercise 20.12 (Limiting distribution)

(*i*) By the assumption we have for any starting distribution α that

$$\pi = \lim_{n\to\infty} \alpha T^n.$$

This can be rewritten as

$$\lim_{n\to\infty} \alpha T^n = \left(\lim_{n\to\infty} \alpha T^{n-1}\right) T = \pi T.$$

(*ii*) The converse is not true. This can be shown by for example considering the transition matrix

$$T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

that has infinitely many stationary distributions (any distribution $(p, 1-p)$ is stationary). However, this distribution will not be the limit for $n \to \infty$ transitions from any initial distribution $(q, 1-q)$.

Solution to Exercise 20.13 (Flip-flop)

$$T = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

The distribution $\pi = (0.5, 0.5)$ is stationary. However, it is not a limiting distribution since it will not be reached from a random initial state.

Solution to Exercise 20.14 (Gothenburg winter weather)

- $\mathbb{P}(s \mid s) = T(s, s) = 0.2$
- $\min(\mathbb{P}(r \mid i)) = 0.6$ for $i \in (s, r, c)$.
- The probability distribution for the weather tomorrow (given that it is clear today) is $(0.1, 0.6, 0.3)$ with the ordered states (s, r, c). The expected amount of precipitation is therefore $\mathbb{E}[\text{amount precipitation}] = 0.1 \cdot 2 + 0.6 \cdot 5 + 0.3 \cdot 0 = 3.2$ mm.
- The initial distribution is $\pi_1 = (0.5, 0.5, 0)$. We therefore have that the distribution two days later is

$$\pi_3 = \pi_1 T^2 = (0.5, 0.5, 0) \begin{pmatrix} 0.12 & 0.72 & 0.16 \\ 0.11 & 0.76 & 0.13 \\ 0.11 & 0.72 & 0.17 \end{pmatrix} = (0.115, 0.74, 0.145).$$

  We find that the requested probability is 0.74.

Solution to Exercise 20.15 (Stationary Gothenburg winter weather)

Solve the system of equations

$$\sum_i \pi_i T_{ij} = \pi_j, \quad \forall j$$

subject to the constraint $\pi_1 + \pi_2 + \pi_3 = 1$. The solution is

$$\pi = \left(\frac{1}{9}, \frac{3}{4}, \frac{5}{36} \right).$$

A numerical test can be performed by evaluating a large number of transitions from different initial states.

Solution to Exercise 20.16 (Is it reversible?)

Solve the system of equations

$$\sum_i \pi_i T_{ij} = \pi_j, \quad \forall j$$

subject to the constraint $\pi_1 + \pi_2 + \pi_3 = 1$ to find the stationary distribution. The solution is

$$\pi = \left(\frac{1}{3}, \frac{4}{15}, \frac{2}{5}\right).$$

Check the detailed balance equations

$$\pi_i T_{ij} = \pi_j T_{ji}, \quad \forall i, j$$

All these hold, hence the chain is reversible.

Solution to Exercise 20.17 (Optical pumping)

(*i*)

- The T matrix can be constructed from the probabilities given, remembering that the rows of T must sum to one, which ensures that the probability of the electron going anywhere (including staying where is it) is one.

$$T = \begin{bmatrix} 1 - p_{13} & 0 & p_{13} \\ p_{21} & 1 - p_{21} & 0 \\ p_{31} & p_{32} & 1 - p_{31} - p_{32} \end{bmatrix} = \begin{bmatrix} 2/3 & 0 & 1/3 \\ 2/3 & 1/3 & 0 \\ 1/3 & 1/3 & 1/3 \end{bmatrix}$$

(*ii*)

- If there is a limiting stationary distribution, π, it satisfies the equation $\pi T = \pi$. In other words, π is a left eigenvector to T with eigenvalue one. This also implies that the column vector π^t is a right eigenvector to the transposed transition matrix T^t (again with eigenvalue one). Note that the eigenvalues of T and T^t are the same. We compute the eigenvalues of the matrix T.

  $$\det(T - \lambda I) = 0$$

  which will give one eigenvalue $\lambda_0 = 1$.

  Any transition matrix has an eigenvalue one since its rows sum to one. Here, the chain is also irreducible and aperiodic ($T_{11} > 0$), so there is a unique limiting distribution. To find the distribution we compute the right eigenvector of T^t corresponding to the $\lambda_0 = 1$ eigenvalue. We normalize it to have sum equal to one and get $\pi = \frac{1}{7}(4, 1, 2)$.

(*iii*)

- The detailed balance conditions are $\pi_i T_{ij} = \pi_j T_{ji}$. Since $T_{23} = 0$ and $T_{32} \neq 0$ for a non-zero stationary distribution these conditions are not satisfied.

Solution to Exercise 20.18 (Detailed balance)

Hint:

1. Express the conditional probability

$$\mathbb{P}_{Y_{n+1}|Y_0,Y_1,\ldots,Y_n}(i_{n+1}|i_0,i_1,\ldots,i_n)$$

   in terms of a ratio of joint probabilities.
2. Express in terms of X_i events.
3. Idenitfy elements of the stationary distribution and transition matrix elements.

20.5. Derivation of the Metropolis-Hastings algorithm

How can we collect random samples from an arbitrary probability distribution $p(\boldsymbol{\theta})$? In particular, how can we perform this task when we might not even have a closed form expression for the PDF? It is common in Bayesian inference that we find ourselves in the situation that we can evaluate $p(\boldsymbol{\theta})$ at any position $\boldsymbol{\theta}$, but we don't know beforehand if it will be large or small.

We will now show that it is possible to construct a Markov chain that has $p(\boldsymbol{\theta})$ as its stationary distribution. Once this chain has converged it will provide us with samples from $p(\boldsymbol{\theta})$.

As we have learned, the evolution of positions is described by the transition density T. The generalization to continuous variables of the stationary-chain equilibrium described in Eq. (20.16) is

$$p(\boldsymbol{\theta}) = \int_{\Omega'} p(\boldsymbol{\theta}')T(\boldsymbol{\theta}',\boldsymbol{\theta})d\boldsymbol{\theta}'. \tag{20.23}$$

This equilibrium condition can be replaced with the more stringent **detailed-balance** condition

$$p(\boldsymbol{\theta})T(\boldsymbol{\theta},\boldsymbol{\theta}') = p(\boldsymbol{\theta}')T(\boldsymbol{\theta}',\boldsymbol{\theta}), \tag{20.24}$$

which can be understood from the integral relation

$$\int d\boldsymbol{\theta}' p(\boldsymbol{\theta})T(\boldsymbol{\theta},\boldsymbol{\theta}') = \int d\boldsymbol{\theta}' p(\boldsymbol{\theta}')T(\boldsymbol{\theta}',\boldsymbol{\theta}), \tag{20.25}$$

that describes the balance between the transitions of probability density out of $\boldsymbol{\theta}$ and into $\boldsymbol{\theta}$. At equilibrium, these integrals must be equal.

The trick to achieve this for a certain $p(\boldsymbol{\theta})$ is to write T as a product of a simple step proposal function $S(\boldsymbol{\theta},\boldsymbol{\theta}')$, that provides probability densities for steps in $\boldsymbol{\theta}$-space, and a function $A(\boldsymbol{\theta},\boldsymbol{\theta}')$ that we will call the **acceptance probability**

$$T(\boldsymbol{\theta},\boldsymbol{\theta}') = A(\boldsymbol{\theta},\boldsymbol{\theta}')S(\boldsymbol{\theta},\boldsymbol{\theta}'). \tag{20.26}$$

The step proposal function can be almost anything, but a key property is that it should be easy to draw a sample from it. The probability for proposing a new position $\boldsymbol{\theta}'$ will depend on the current position $\boldsymbol{\theta}$

$$p\left(\text{proposing} \quad \boldsymbol{\theta}' \mid \text{current position} \quad = \boldsymbol{\theta}\right) = S(\boldsymbol{\theta}, \boldsymbol{\theta}') \tag{20.27}$$

For example, assuming that $\boldsymbol{\theta}$ is a vector in a D-dimensional parameter space we can take $S(\boldsymbol{\theta}, \boldsymbol{\theta}')$ to be a uniform distribution within a D-dimensional hypercube with side length Δ

$$S(\boldsymbol{\theta}, \boldsymbol{\theta}') = \begin{cases} \frac{1}{\Delta^D} & \text{if } |\boldsymbol{\theta}'_i - \boldsymbol{\theta}_i| \leq \frac{\Delta}{2} \text{ for all } \quad i \in \{1, \ldots, D\} \\ 0 & \text{otherwise} \end{cases} \tag{20.28}$$

Alternatively we can use a multivariate Gaussian distribution with a mean that corresponds to the current position $\boldsymbol{\theta}$ and a covariance matrix that we can specify.

The probability of actually accepting the proposed step from $\boldsymbol{\theta}$ to $\boldsymbol{\theta}'$ will then be given by $0 \leq A(\boldsymbol{\theta}, \boldsymbol{\theta}') \leq 1$.

The detailed balance condition (20.24) can be fulfilled by placing the following condition on the acceptance probability

$$\frac{A(\boldsymbol{\theta}, \boldsymbol{\theta}')}{A(\boldsymbol{\theta}', \boldsymbol{\theta})} = \frac{p(\boldsymbol{\theta}')S(\boldsymbol{\theta}', \boldsymbol{\theta})}{p(\boldsymbol{\theta})S(\boldsymbol{\theta}, \boldsymbol{\theta}')}. \tag{20.29}$$

The expression for A that fulfils this condition is

$$A(\boldsymbol{\theta}, \boldsymbol{\theta}') = \min\left(\frac{p(\boldsymbol{\theta}')S(\boldsymbol{\theta}', \boldsymbol{\theta})}{p(\boldsymbol{\theta})S(\boldsymbol{\theta}, \boldsymbol{\theta}')}, 1\right), \tag{20.30}$$

and holds the probability by which we decide whether to accept the proposed position $\boldsymbol{\theta}'$, or remain in the old one $\boldsymbol{\theta}$. The ratio that appears in the acceptance probability is called the Metropolis-Hastings ratio and will here be denoted r. We note that it gets further simplified for symmetric proposal functions (for which it is sometimes referred to as the Metropolis ratio)

$$r = \begin{cases} \frac{p(\boldsymbol{\theta}')S(\boldsymbol{\theta}', \boldsymbol{\theta})}{p(\boldsymbol{\theta})S(\boldsymbol{\theta}, \boldsymbol{\theta}')} & \text{for general proposal functions} \quad , \\ \frac{p(\boldsymbol{\theta}')}{p(\boldsymbol{\theta})} & \text{for symmetric proposal functions} \quad . \end{cases} \tag{20.31}$$

Note how the proposal function $S(\boldsymbol{\theta}, \boldsymbol{\theta}')$ in combination with the acceptance probability $A(\boldsymbol{\theta}, \boldsymbol{\theta}') = \min(r, 1)$ provides a simple process for generating samples in a Markov chain that has the required equilibrium distribution. It only requires simple draws of proposal steps, and the evaluation of p at these positions. It is stochastic due to the random proposal of steps and since the decision to accept a proposed step contains randomness via the acceptance probability.

Note

The proposal function must allow the chain to be irreducible, positively recurrent and aperiodic.

The basic structure of the Metropolis (and Metropolis-Hastings) algorithm is the following:

Algorithm 20.1 (The Metropolis-Hastings algorithm)

1. Initialize the sampling by choosing a starting point $\boldsymbol{\theta}_0$.
2. Collect samples by repeating the following:
   1. Given $\boldsymbol{\theta}_i$, *propose* a new point $\boldsymbol{\phi}$, sampled from the proposal distribution $S(\boldsymbol{\theta}_i, \boldsymbol{\phi})$. This proposal distribution could take many forms. However, for concreteness you can imagine it as a multivariate normal with mean vector given by $\boldsymbol{\theta}_i$ and (position-independent) variances $\boldsymbol{\sigma}^2$ specified by the user.
      - The propsal function will (usually) give a smaller probability for visiting positions that are far from the current position.
      - The width $\boldsymbol{\sigma}$ determines the average step size and is known as the proposal width.
   2. Compute the Metropolis(-Hastings) ratio r given by Eq. (20.31). For symmetric proposal distributions, with the simpler expression for the acceptance probability, this algorithm is known as the Metropolis algorithm.
   3. Decide whether or not to accept candidate $\boldsymbol{\phi}$ for $\boldsymbol{\theta}_{i+1}$.
      - If $r \geq 1$: accept the proposal position and set $\boldsymbol{\theta}_{i+1} = \boldsymbol{\phi}$.
      - If $r < 1$: accept the position with probability r by sampling a uniform $\mathcal{U}([0, 1])$ distribution (note that now we have $0 \leq r < 1$).
        - If $u \sim \mathcal{U}([0, 1]) \leq r$, then $\boldsymbol{\theta}_{i+1} = \boldsymbol{\phi}$ (accept);
        - else $\boldsymbol{\theta}_{i+1} = \boldsymbol{\theta}_i$ (reject).

      Note that the chain always grows since you add the current position again if the proposed step is rejected.
   4. Iterate until the chain has reached a predetermined length or passes some convergence tests.

- The Metropolis algorithm dates back to the 1950s in physics, but didn't become widespread in statistics until around 1990.
- It enabled Bayesian methods to become feasible.
- Note, however, that nowadays there are much more sophisticated samplers than the original Metropolis one.

Exercises

Exercise 20.19 (Metropolis sampling of a uniform distribution)

Show that the update rule implemented in Example 20.6 has the uniform distribution $\mathcal{U}(0, 1)$ as its limiting distribution.

Exercise 20.20 (Power-law distributions)

Discrete power-law distributions have the general form $p(i) \propto i^{\alpha}$. for some constant α. Unlike exponentially decaying distributions (such as the normal distribution and many others), power-law distributions have *fat tails*. They are therefore often used to model skewed data sets. Let

$$\pi_i = \frac{i^{-3/2}}{\sum_{k=1}^{\infty} k^{-3/2}}, \quad \text{for } \; i = 1, 2, \ldots$$

Implement a Metropolis algorithm to sample for $\boldsymbol{\pi}$.

Exercise 20.21 (The Metropolis algorithm for a discrete distribution)

Use the Metropolis algorithm as outlined in Theorem 20.1 to construct a transition matrix T for a Markov chain that gives the discrete limiting distribution

$$\pi = \left(\frac{1}{9}, \frac{3}{4}, \frac{5}{36}\right).$$

Assume that the step proposal matrix has $S(i,j) = 1/3$ for all i, j.

- Compute T by hand, or numerically using Python. Verify that it is a valid transition matrix and that the above distribution is stationary.
- Verify that π and T fulfills detailed balance.
- Note that this distribution is the limiting distribution found in Exercise 20.15. However, the transition matrices are not the same. Try to explain why. Can you construct other transition matrices with the same limiting distribution?
- Construct the Markov chain outcomes of random variable $\{X_n\}$ numerically. Plot the convergence of the discrete distribution for different starting distributions and see if the correct limiting distribution is obtained.

Solutions to exercises

Solution to Exercise 20.19 (Metropolis sampling of a uniform distribution)

Check the acceptance criterion.

Solution to Exercise 20.20 (Power-law distributions)

It is suggested to use $\{-1, 1\}$ as the proposal step (with $p = 0.5$ a step to the left). The acceptence fraction involves a ratio of probabilities, which removes the infinite sum in the denominator. Note also that the acceptance criterion must include the reflective boundary at $i = 1$.

Solution to Exercise 20.21 (The Metropolis algorithm for a discrete distribution)

- Construct the T-matrix

$$T = \begin{pmatrix} 0.33333333 & 0.33333333 & 0.33333333 \\ 0.04938272 & 0.88888889 & 0.0617284 \\ 0.26666667 & 0.33333333 & 0.4 \end{pmatrix}$$

and note that all elements are positive and that the row sums equal one. You can verify by hand that $\pi = \pi T$ with the given distribution.

- Eq. (20.18) gives nine equalities that should be checked.
- The stationary distribution π is a left eigenvector of T with eigenvalue $+1$. But there is no uniqueness theorem that dictates that there can only be one matrix with this eigenvalue/vector. It is easy to create other

transition matrices with the same limiting distribution by just changing the step proposal matrix.
- Generate the Markov chain and see that it converges

```
import numpy as np

t=np.array([[0.33333333, 0.33333333, 0.33333333],
 [0.04938272, 0.88888889, 0.0617284 ],
 [0.26666667, 0.33333333, 0.4       ]])

N = 40 # Number of steps
x0 = np.array([1,0,0]) # Start distribution

X = []
X.append(x0)
for i in range(1,N):
    X.append(np.matmul(X[i-1],t))

X = np.array(X)
print(X)
```

21. Markov Chain Monte Carlo in practice

Here we look at aspects of using MCMC in actual analyses: a workflow for MCMC sampling from preparation to analysis, and the diagnostics used to assess convergence.

21.1. Workflow for MCMC sampling

MCMC sampling is an enabling technology for modern Bayesian inference. It allows us to collect samples from complex target probability distributions that may be high-dimensional, out of reach for analytical treatment, and computationally costly to evaluate. Here we consider a possible workflow to make MCMC sampling efficient, to diagnose and address possible failure modes, and to make the sampling results interpretable:

Four-phase MCMC workflow

1. Preparatory phase
2. Warmup phase
3. Collection phase
4. Analysis phase

In this section we give a general overview of the four phases. Explicit examples, best practices, and more detailed considerations follow when we discuss specific sampling algorithms in Advanced sampling algorithms and their implementations in State-of-the-art MCMC implementations. The diagnostics referred to here are introduced in Diagnostics for MCMC sampling and demonstrated in Demo: MCMC Diagnostics. For an opinionated practitioner's account of MCMC in the physical sciences, see Hogg and Foreman-Mackey [HFM18].

The four phases are shown on the left in Fig. 21.1: (1) prepare for sampling by considering all available information about the target distribution; (2) warm up by tuning the sampler and letting the chains equilibrate; (3) collect as many samples as the computational budget allows; (4) analyze the results and provide Monte Carlo estimators for the quantities of interest. Although the phases are traversed in order, there are feedback loops within each phase,

and the diagnostic checkpoints can send us back to an earlier phase to start over. The samples collected during warmup are discarded and must not be mixed with those from the collection phase.

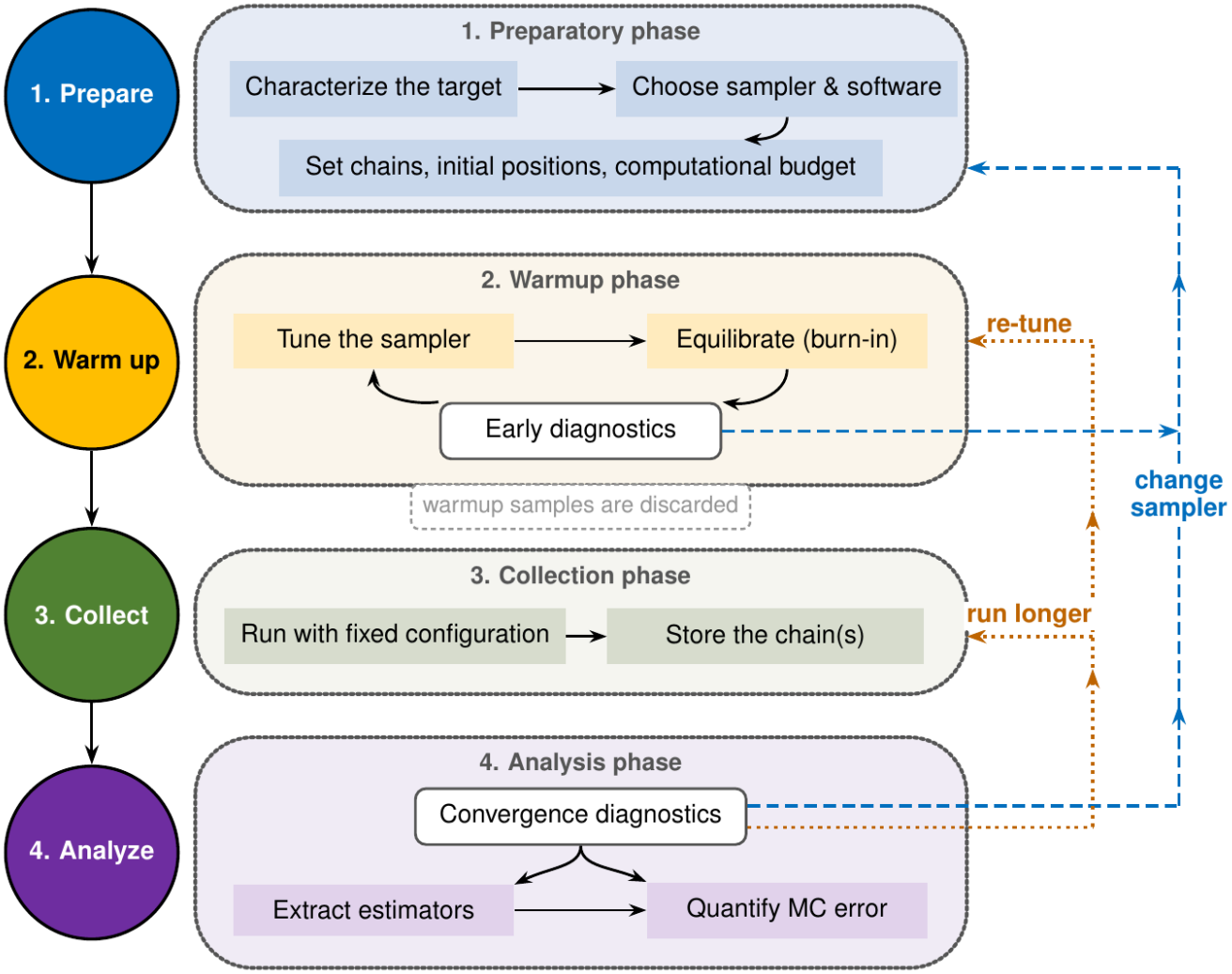


Fig. 21.1 The MCMC workflow. Coloured boxes are *actions* that belong to one phase, while white outlined boxes are *diagnostic checkpoints*, from which the dashed feedback paths depart. The short-dashed path returns to the warmup phase (re-tune or run longer), the long-dashed path to the preparatory phase (change sampler).

In the following subsections we expand upon each phase.

Phase 1: Preparatory Phase

In the preparatory phase we characterize the target distribution: its dimension, the cost of a single evaluation, and whether gradients are available. Any prior knowledge of its shape (the regions of high probability mass, strong correlations between parameters, possible multimodality) feeds into the choice of sampling algorithm, the number of chains, and the distribution of initial positions. Since the cost is usually dominated by the likelihood evaluation, the total computational budget sets a firm limit on the number of samples, and the algorithm should be chosen so that this budget buys as many effectively independent samples as possible. Note also that MCMC is a tool for sampling, not for optimization: if the goal is only to locate the mode of a distribution, an optimizer is the better choice [HFM18].

Two practical steps belong here as well. First, verify the implementation of the target before sampling: evaluate the log-probability at a few points, check that it returns $-\infty$ outside the support (and never NaN) and, when gradients are used, compare them with finite differences. Second, decide what will be recorded during the run. The convergence diagnostics of the later phases are computed from the parameter traces, but only if these are stored per chain and in step order; the log-probability of each sample comes for free and is worth keeping as well, both as an informal diagnostic and for later use.

We take an end-user perspective here: rather than engineering our own Markov transitions we rely on existing, well-tested sampling software, and the choice among these is discussed in State-of-the-art MCMC implementations.

Not every sampling algorithm is of the Markov chain type. Nested sampling (Nested sampling) instead evolves a population of live points that is successively compressed towards regions of higher likelihood; its primary output is the evidence, with weighted posterior samples as a by-product. There is no burn-in, or autocorrelation time in that scheme, so the workflow described here applies to it only in its preparatory and analysis phases, and it has its own convergence criteria.

Phase 2: Warmup phase

The warmup phase consists of two steps, tuning and equilibration, connected through a feedback loop with early diagnostics. In the tuning step we adapt the parameters of the sampling algorithm to the target distribution, as represented by the warmup samples. Which parameters these are depends on the algorithm: for the Metropolis–Hastings algorithm (Derivation of the Metropolis-Hastings algorithm) it is the step length and the functional form of the proposal distribution; for Hamiltonian Monte Carlo (Hamiltonian Monte Carlo) it is the mass matrix, the leapfrog step size, and the number of steps (which, with the step size, sets the integration time). Tuning is critical for performance. Tuning can also mean transforming the parameters: a linear transformation that removes strong correlations, or sampling $\log \sigma$ instead of a scale parameter σ, often helps any sampler more than adjusting its internal settings. Modern sampling software typically performs the adaptation of internal settings automatically during a designated warmup period, but the user still has to judge whether it succeeded. The early diagnostics used for this purpose are acceptance rates, trace plots, and preliminary estimates of the autocorrelation time.

The equilibration step, also known as burn-in, moves the chains from their initial positions, which may lie in an unrepresentative region of parameter space, into the region where the probability mass is concentrated. Starting several chains from dispersed positions makes this step more informative: trace plots and the Gelman–Rubin statistic $\hat{R}$ (Diagnostics for MCMC sampling) show whether the chains have forgotten where they started and have found the same region. If they have not, a longer warmup is needed; if they have, but the sampler moves too slowly, one returns to tuning; and if tuning cannot fix it, one returns to the preparatory phase and reconsiders the algorithm.

All warmup samples must be discarded. During tuning the transition kernel keeps changing, so the chain is not yet a Markov chain with a fixed stationary distribution, and its samples do not follow the target. The samples collected during equilibration do come from the final kernel, but from its transient regime rather than from its limiting distribution, so they would bias the Monte Carlo estimators. Neither kind may be mixed with the samples of the collection phase.

Phase 3: Collection phase

The collection phase is a long run with fixed sampler configuration, in which we collect as many samples as the computational budget allows. The chain positions are stored per chain and in step order, since this is what the convergence diagnostics of the final phase require; flattened arrays, with all chains pooled, are then used for constructing the Monte Carlo estimators. It is worth also storing the log-probability of every position. It comes for

free from the sampler and serves as an additional diagnostic: a chain stuck in a low-probability region, or a $\log p$ that keeps drifting upward, shows up there before it does in the parameters. It is also needed for identifying the MAP sample and for the evidence-related methods later in the book. Some samplers report further per-step quantities, such as divergent transitions in Hamiltonian Monte Carlo, that are genuinely diagnostic and cannot be reconstructed from the traces; these should be stored as well.

How long should the run be? Since a chain can appear converged while it has not yet discovered a part of the parameter space, the only real safeguard is to run longer than seems necessary. Geyer, in his introduction to the *Handbook of Markov Chain Monte Carlo* [Gey11], argues for one long run rather than many short ones:

> Worse, addiction to many short runs can keep one from running the sampler long enough to detect pseudo-convergence or other problems, such as bugs in the code. People who have used MCMC in complicated problems can tell stories about samplers that appeared to be converging until after weeks of running they discovered a new part of the state space and the distribution changed radically. If those people had thought it necessary to make hundreds of runs, none of them could have been several weeks long.

Geyer makes another recommendation, “only a bit facetious”, for those who write scientific papers based on MCMC sampling:

> [O]ne should start a run when the paper is submitted and keep running until referee’s reports arrive. This cannot delay the paper, and may detect pseudo-convergence.

Note that this advice concerns the length of each chain; it does not contradict the use of a handful of chains started from dispersed positions, which is what makes the between-chain diagnostics of the next phase possible.

Phase 4: Analysis phase

The analysis phase begins with the convergence diagnostics of Diagnostics for MCMC sampling, now applied to the full collection run. Between-chain comparisons and $\hat{R}$ test whether chains started from dispersed positions agree; trace plots and the log-probability trace reveal chains that are stuck or still drifting; and the autocorrelation time τ, which is only estimated reliably from long chains, tells us how many effectively independent samples we have, $N_{\mathrm{eff}} \approx N/\tau$. A long run also increases the chance of exposing problematic behaviour such as multimodality. Passing these checks is a prerequisite for everything that follows; failing them sends us back to the warmup phase (longer run, re-tuning) or, if the sampler is fundamentally unsuited, to the preparatory phase.

Once the samples are trusted, Monte Carlo estimators are formed from the pooled samples of all chains: posterior means and medians, credible intervals from quantiles, and marginal distributions of any subset of parameters, which are obtained simply by ignoring the other coordinates. A corner plot is nothing but the set of one- and two-dimensional marginals. Predictive distributions follow by pushing each sample through the model. The MAP estimate can be read off from the stored log-probabilities, although with finite samples it is only approximate and often less useful than the posterior summaries.

Three plots to make every time you run MCMC

Following Hogg and Foreman-Mackey [HFM18], three figures should accompany every MCMC analysis.

1. *Trace plots* of all chains expose the equilibration time, problems with the model or the sampler, and give a qualitative impression of convergence.

2. *Corner plots* show all one- and two-dimensional marginal distributions; they are remarkable for locating expected and unexpected parameter relationships, and often suggest reparametrizations that simplify the problem.
3. *Posterior predictive plots* compare predictions with a few dozen random samples to the observed data; they give a qualitative sense of how well the model fits and can also expose problems with sampling or convergence.

Every estimator should be reported with its Monte Carlo error, which for a mean is $\sigma/\sqrt{N_{\text{eff}}}$ rather than $\sigma/\sqrt{N}$. Interval endpoints and other tail quantities are more demanding for two reasons: their sampling variance is intrinsically larger than that of a mean, since few samples fall in the tails, and the autocorrelation time of a tail indicator is typically longer than that of the parameter itself, so the relevant N_{eff} is smaller [VGS+21]. A posterior mean is well determined by a few hundred effective samples; the endpoints of a 95% credible interval require thousands. Thinning the chain is not needed for these estimators, since correlated samples carry less information per sample but discarding them never adds information; it is justified only to save storage or to make plots legible.

Finally, the analysis phase hands over to model checking. Whether the posterior itself is reasonable, as judged by posterior predictive checks, sensitivity to the prior, and comparison with alternative models, is the subject of the Bayesian workflow in Bayesian research workflow and is answered with the same set of samples.

21.2. Diagnostics for MCMC sampling

We've seen that using MCMC leads to a Markov chain: a set of configurations of the parameters we are sampling. This chain enables inference because they are samples of the posterior of interest. MCMC sampling is notoriously difficult to validate. In this section we discuss the common convergence tests that enter the warmup and analysis phases of the workflow in Workflow for MCMC sampling. Keep in mind that none of them can prove convergence; they can only detect certain kinds of failure. Passing all tests is therefore necessary, but not sufficient.

Convergence tests for MCMC sampling

MCMC sampling can go wrong in (at least) three ways:

1. No convergence—The limiting distribution is not reached.
2. Pseudoconvergence—The chain seems stationary, but the limiting distribution is not actually reached.
3. Highly correlated samples—Not really wrong, but inefficient.

There is no unique solution to such problems, but they are typically addressed in three ways:

1. Design the simulation runs for monitoring of convergence. In particular, run multiple sequences of the Markov chains with starting points dispersed over the sampling space. This is difficult in high dimensions.
2. Monitor the convergence of individual sampling dimensions, as well as predicted quantities of interest, by comparing variations within and between the different sequences (chains). Here you are looking for stationarity (the running means are stable) and mixing (different sequences are sampling the same distribution). The Gelman-Rubin test (see below) is devised for this purpose. Fig. 21.2 illustrates the two failure modes (cf. Figure 11.3 in BDA3 [GCS+13]): on the left two chains that each look stationary but stay in separate regions of θ_0 (no

mixing), and on the right two chains that mix well but drift together, so that neither samples a stationary distribution (the θ_0 distribution keeps changing with MC steps).

▼ **Hide code cell source**

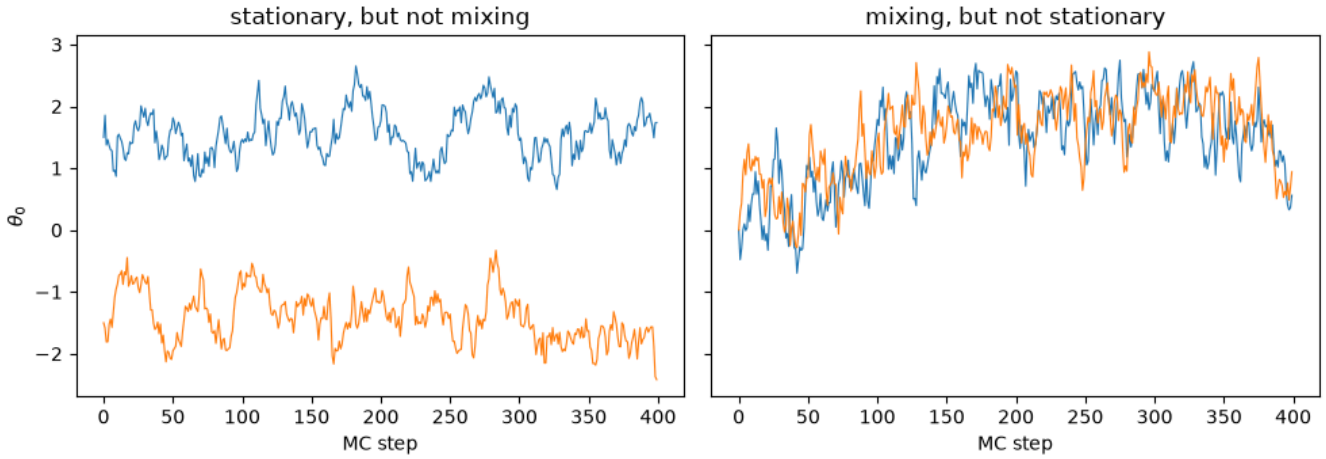


Fig. 21.2 Two ways in which MCMC chains can fail. Left: two chains that are each stationary but sample different regions of parameter space and never mix. Right: two chains that mix but are not stationary, since both drift with the number of MC steps. Only chains that are both stationary and mixing sample the target distribution.

3. Monitor the autocorrelation of the samples (see below), which determines how many effectively independent samples a chain of given length contains. Strong autocorrelation implies that the algorithm must be tuned, or that an altogether different sampling algorithm should be used. A first indicator is the *acceptance rate* of the sampler: for a random-walk Metropolis sampler an acceptance rate close to one means that the steps are too short (successive samples are almost identical), while a rate close to zero means that almost every proposal is rejected (the chain stands still). The optimal value depends on the algorithm: for random-walk Metropolis proposals values in the range 0.2–0.5 are typically desired in high dimensions; for Hamiltonian Monte Carlo the target is much higher, around 0.65–0.8, since its proposals are informed by the gradient; for the affine-invariant ensemble sampler its authors again recommend 0.2–0.5.

The diagnostics that will be discussed below are all univariate. They work perfectly when there is only one parameter to estimate. In fact, most convergence tests are performed with univariate diagnostics applied to each sampling dimension one by one.

Variance of the mean

Consider the sampling variance of a parameter mean value. If the N samples were independent draws from the posterior it would be

$$\text{Var}\left(\bar{\theta}\right) = \frac{\text{Var}\left(\theta\right)}{N}, \tag{21.1}$$

where N is the length of the chain. This quantity is capturing the simulation error of the mean rather than the underlying uncertainty of the parameter θ. MCMC samples are not independent, however, and we will see below how the formula must be corrected. We can visualize this by examining the moving average of our parameter trace. The trace is the sequence as a function of iteration number.

Autocorrelation

A challenge when doing MCMC sampling is that the collected samples can be *correlated*. This can be tested by computing the *autocorrelation function* and extracting the correlation time for a chain of samples.

Say that X is an array of N samples numbered by the index t. Then X_{+h} is a shifted version of X with elements X_{t+h}. The integer h is called the *lag*. Since we have a finite number of samples, the array X_{+h} will be h elements shorter than X.

Furthermore, $\bar{X}$ is the average value of X.

We can then define the autocorrelation function $\rho(h)$ from the list of samples.

$$\rho(h) = \frac{\sum_{t=0}^{N-h-1}\left[(X_t - \bar{X})(X_{t+h} - \bar{X})\right]}{\sqrt{\sum_{t=0}^{N-h-1}(X_t - \bar{X})^2}\sqrt{\sum_{t=0}^{N-h-1}(X_{t+h} - \bar{X})^2}} \tag{21.2}$$

The summation is carried out over the subset of samples that overlap. The autocorrelation is the overlap (scalar product) of the chain of samples (the trace) with a copy of itself shifted by the lag, as a function of the lag. If the lag is short so that nearby samples are close to each other (and have not moved very far) the product of these two vectors is large. If samples are independent, you will have both positive and negative numbers in the overlap that cancel each other.

The typical example of a highly correlated chain is a random walk with a too short proposal step length.

It is often observed that $\rho(h)$ is roughly exponential so that we can define an autocorrelation time τ_{a} according to $\rho(h) \sim \exp(-h/\tau_{\mathrm{a}})$.

The quantity that enters the error estimates below is instead the *integrated* autocorrelation time

$$\tau = 1 + 2\lim_{N\to\infty}\sum_{h=1}^{N}\rho(h).$$

For a purely exponential $\rho(h)$ the two are related by $\tau \approx 2\tau_{\mathrm{a}}$ (for $\tau_{\mathrm{a}} \gg 1$), so one should be careful about which one is quoted.

With autocorrelated samples, the sampling variance (21.1) becomes

$$\mathrm{Var}\left(\bar{\theta}\right) = \tau\frac{\mathrm{Var}\left(\theta\right)}{N}, \tag{21.3}$$

with $\tau \gg 1$ for highly correlated samples. This motivates us to define the effective sample size (ESS), or effective number of samples, as

$$N_{\mathrm{eff}} = N/\tau.$$

To keep N_{eff} high, we must collect many samples and keep the autocorrelation time small.

In practice τ must be estimated from the finite chain, and the naive sum of $\rho(h)$ over all lags does not work: the estimated $\rho(h)$ at large lag is pure noise that does not decay, so the sum keeps wandering. The standard remedy, due to Sokal, is to truncate the sum at a window M chosen self-consistently such that $M \gtrsim 5\tau$. This is what `emcee.autocorr.integrated_time` (and thus `sampler.get_autocorr_time()`) implements, averaging the autocorrelation functions of all walkers before integrating. The estimate is only trustworthy when the chain is much longer than the autocorrelation time, $N \gtrsim 50\tau$ as a rule of thumb; `emcee` raises an error otherwise. Finally, τ is a property of the *quantity* whose time series is analyzed, not of the chain alone: the autocorrelation time of a tail indicator $\mathbb{1}[\theta > q]$ is typically longer than that of θ itself, so tail quantities have a smaller N_{eff} than the per-parameter value suggests [VGS+21].

The Gelman-Rubin test

The Gelman-Rubin diagnostic was constructed to test for stationarity and mixing of different Markov chain sequences [GR92].

Algorithm 21.1 (The Gelman-Rubin diagnostic)

1. Collect $M > 1$ sequences (chains) of length $2N$.
2. Discard the first N draws of each sequence, leaving the last N iterations in the chain.
3. Calculate the within and between chain variance.
   - Within chain variance:

$$W = \frac{1}{M} \sum_{j=1}^{M} s_j^2$$

     where s_j^2 is the variance of each chain (after throwing out the first N draws).
   - Between chain variance:

$$B = \frac{N}{M-1} \sum_{j=1}^{M} (\bar{\theta}_j - \bar{\bar{\theta}})^2$$

     where $\bar{\theta}_j$ is the mean of chain j and $\bar{\bar{\theta}}$ is the mean of the M chain means.
4. Calculate the estimated variance of θ as the weighted sum of between and within chain variance.

$$\widehat{\mathrm{Var}}(\theta) = \left(1 - \frac{1}{N}\right) W + \frac{1}{N} B$$

5. Calculate the potential scale reduction factor.

$$\hat{R} = \sqrt{\frac{\widehat{\mathrm{Var}}(\theta)}{W}}$$

We want the $\hat{R}$ number to be close to 1 since this would indicate that the between chain variance is small. And a small between chain variance means that the chains are mixing around the same stationary distribution. Gelman

and Rubin [GR92] showed that stationarity has certainly not been achieved when $\hat{R}$ is greater than 1.1 or 1.2. Current practice is stricter and uses the *split* $\hat{R}$: each chain is cut into two halves that are treated as separate sequences (so M chains give $2M$ sequences), which also detects a chain that is still drifting, and convergence is only accepted for $\hat{R} < 1.01$ [VGS+21].

Two remarks are in order. First, the test is only meaningful for *independent* chains started from dispersed positions. The walkers of an ensemble sampler with the default stretch move are not independent (each proposal is constructed from the positions of the other walkers), so $\hat{R}$ across walkers of a single ensemble is not a valid diagnostic; one has to compare independent ensembles. With a Metropolis move, as in Demo: MCMC Diagnostics, each walker is an independent chain and the test can be applied across walkers. Second, like all the diagnostics discussed here $\hat{R}$ can give a false pass: chains that have all found the same mode of a multimodal posterior, but none of the others, will happily agree with each other.

21.3. Demo: MCMC Diagnostics

This notebook demonstrates some of the basic MCMC diagnostic tools.

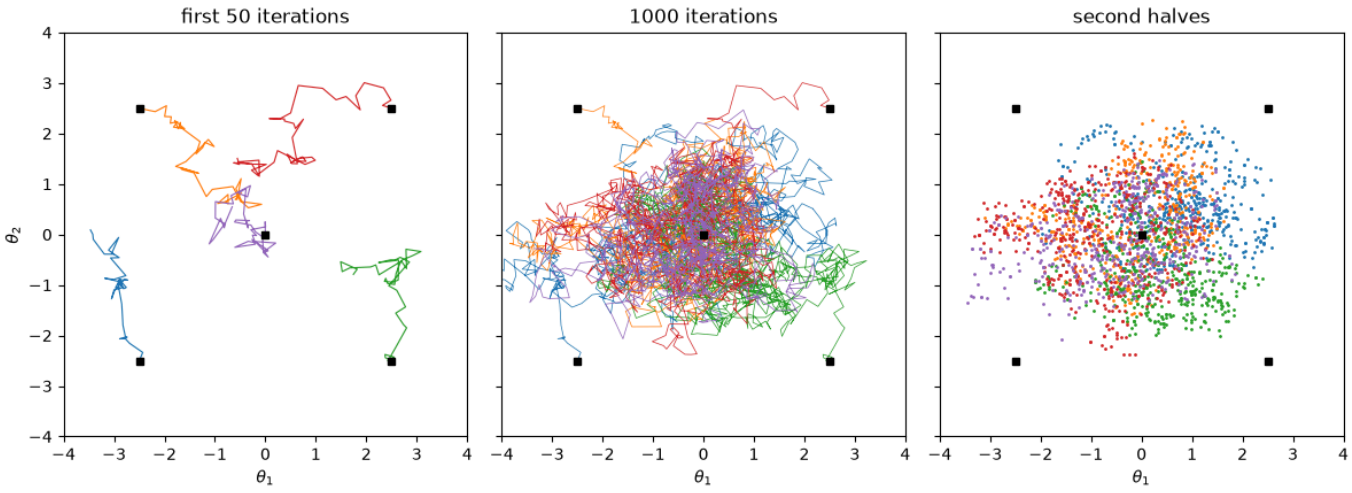
```
import numpy as np
from scipy.stats import norm, uniform

import matplotlib.pyplot as plt

import pandas as pd
import warnings
warnings.filterwarnings('ignore')
```

MCMC diagnostics: assessing convergence

From previous notebooks we know that MCMC sampling is a powerful tool for Bayesian inference. However, MCMC samples represent the posterior only once the chain has converged to its stationary distribution. Convergence is guaranteed in the limit of infinitely many steps, but a finite chain can still be far from stationary, or explore the posterior so slowly that its samples are strongly correlated. Since we cannot simply inspect the posterior, we rely on a set of numerical diagnostics that detect when something is wrong. None of them can prove convergence, but together they flag many problematic scenarios.

The figure generated with the following code shows five Metropolis chains sampling a bivariate normal distribution, started from overdispersed points. After 50 iterations the chains are still far from the target and from each other (left panel); after 1000 iterations they have mixed (middle panel), and the second halves of the chains look like samples from the target (right panel).

```python
# Our own version of BDA3 Fig. 11.1: five Metropolis chains sampling a
# bivariate unit normal, started from overdispersed points.
def metropolis(log_p, theta0, nsteps, proposal_width, rng):
    """Random-walk Metropolis with a Gaussian proposal; returns the chain."""
    theta = np.array(theta0, dtype=float)
    logp = log_p(theta)
    chain = np.empty((nsteps + 1, len(theta)))
    chain[0] = theta
    for i in range(1, nsteps + 1):
        proposal = theta + proposal_width * rng.standard_normal(len(theta))
        logp_prop = log_p(proposal)
        if np.log(rng.uniform()) < logp_prop - logp:
            theta, logp = proposal, logp_prop
        chain[i] = theta
    return chain

log_target = lambda th: -0.5 * th @ th   # bivariate unit normal
starts = [(-2.5, -2.5), (-2.5, 2.5), (2.5, -2.5), (2.5, 2.5), (0., 0.)]
rng = np.random.default_rng(1)
chains = [metropolis(log_target, th0, 1000, 0.2, rng) for th0 in starts]

fig, axs = plt.subplots(1, 3, figsize=(12, 4.5), sharex=True, sharey=True)
for ch in chains:
    axs[0].plot(ch[:51, 0], ch[:51, 1], '-', lw=0.8)
    axs[1].plot(ch[:, 0], ch[:, 1], '-', lw=0.5)
    axs[2].plot(ch[501:, 0], ch[501:, 1], '.', ms=2)
for ax, title in zip(axs, ['first 50 iterations', '1000 iterations', 'second halves']):
    ax.plot(*zip(*starts), 'ks', ms=4)   # starting points
    ax.set_title(title)
    ax.set_xlabel(r'$\theta_1$')
    ax.set_xlim(-4, 4); ax.set_ylim(-4, 4); ax.set_aspect('equal')
axs[0].set_ylabel(r'$\theta_2$')
fig.tight_layout()
```



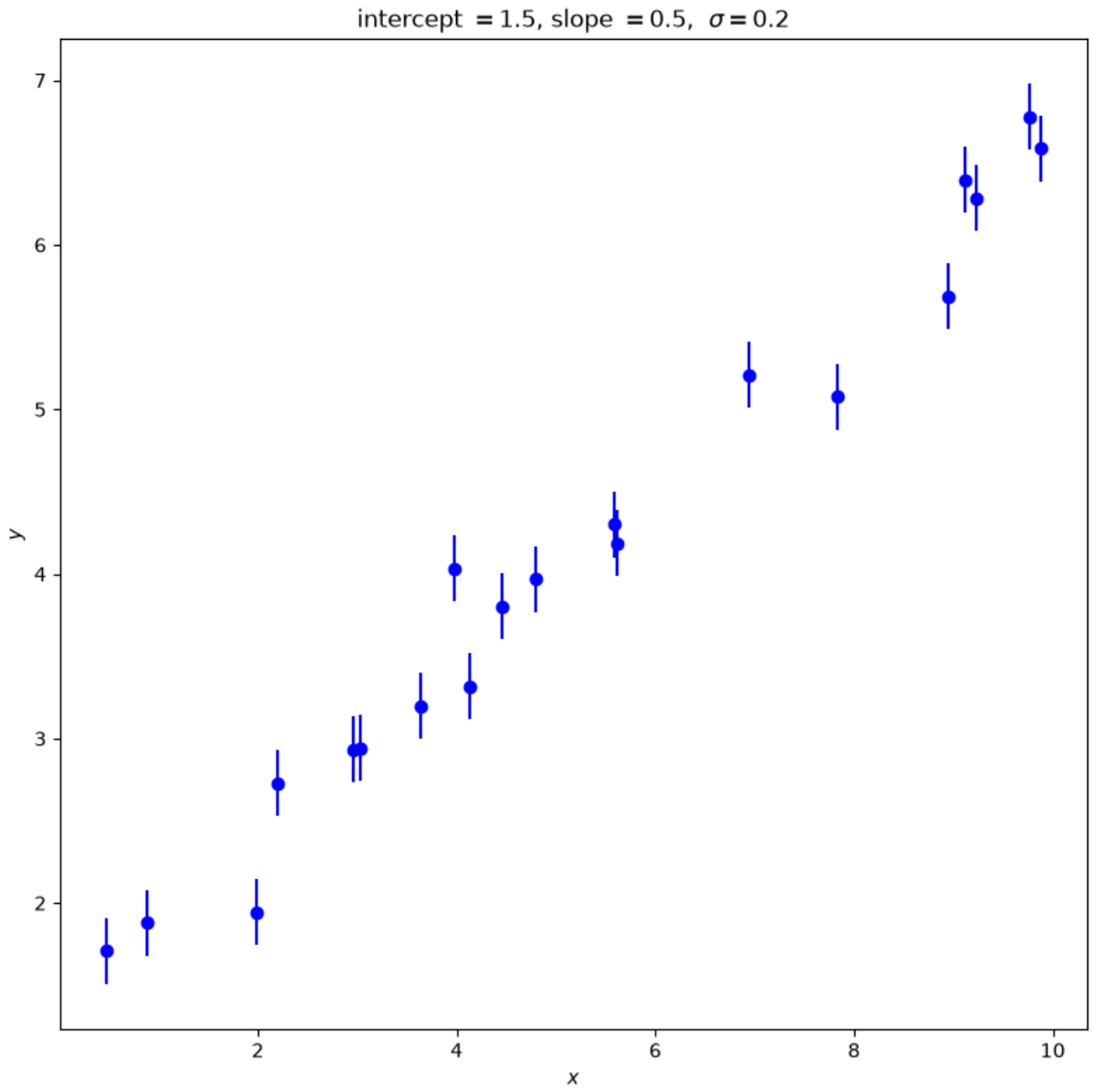
Fitting a straight line - revisited

Let us revisit the problem of inferring the parameters of a straight line. See also 📥 Problem: fitting a straight line and 📥 Problem: fitting a straight line II

The Data

We start by creating some data that we will fit with a straight line. The data is generated with a variance σ^2 on the y values and no error on x.

```
def make_data(intercept, slope, N_pts=20, dy=.2, rseed=None):
    """Given a straight line defined by intercept and slope:
          y = slope * x + intercept
       generate N_pts points randomly spaced points from x=0 to x=x_max
       with Gaussian (i.e., normal) error with mean zero and standard
       deviation dy.

       Unless rseed is specified as an integer, new random data will be
       generated each time.

       Return the x and y arrays and an array of standard deviations.
    """
    rand = np.random.RandomState(rseed)

    x_max = 10.
    x = x_max * rand.rand(N_pts)  # choose the x values randomly in [0,10]
    y = intercept + slope * x  # This is the y value without noise
    y += dy * rand.randn(N_pts)    # Add in Gaussian noise
    return x, y, dy * np.ones_like(x)  # return coordinates and error bars

# Fix the random seed so that the data and the MCMC runs are reproducible.
# (emcee draws from numpy's global random state unless given its own.)
seed = 2026
np.random.seed(seed)

# Specify the true parameters and make sample data
intercept = 1.5   # true intercept (called b elsewhere)
slope = 0.5       # true slope (called m elsewhere)
theta_true = [intercept, slope]  # put parameters in a true theta vector
x, y, dy = make_data(*theta_true, rseed=seed)

# Make a plot of the data
fig, ax = plt.subplots(figsize=(8,8))
ax.errorbar(x, y, dy, fmt='o', color='blue')
ax.set_xlabel(r'$x$')
ax.set_ylabel(r'$y$')
plot_title = rf'intercept $= {intercept:.1f}$, slope $= {slope:.1f}$, ' \
             + rf' $\sigma = {dy[0]:.1f}$'
ax.set_title(plot_title)
fig.tight_layout()
```



The Model

Next we need to specify a theoretical model. We're fitting a straight line to data, so we'll need a slope and an intercept; i.e.

$$y_{\text{th}}(x) = mx + b$$

where our parameter vector will be

$$\theta = [b, m]$$

But this is only half the picture: what we mean by a “model” in a Bayesian sense is not only this expected value $y_{\text{th}}\ (x;\theta)$, but a **probability distribution** for our data. That is, we need an expression to compute the likelihood $p(D \mid \theta, I)$ for our data as a function of the parameters θ (I stands for all other information). Here D is the set of all (x, y) pairs that we know about (or measure).

[Note: At this stage we are (implicitly) assuming that our theoretical model is perfect. But it is not! We’ll come back eventually to talk about adding a theory error δy_{th} .]

We are given data with simple error bars, which imply that the probability for any *single* data point (labeled by i) is a normal distribution with mean given by the true value. That is,

$$y_i \sim \mathcal{N}(y_{\text{th}}\ (x_i;\theta), \varepsilon_i^2)$$

or, in other words,

$$p(y_i \mid x_i, \theta, I) = \frac{1}{\sqrt{2\pi\varepsilon_i^2}} \exp\left(\frac{-\left[y_i - y_{\text{th}}\ (x_i;\theta)\right]^2}{2\varepsilon_i^2}\right)$$

where ε_i are the (known) measurement errors indicated by the error bars.

Assuming all the points are independent, we can find the full likelihood by multiplying the individual likelihoods together:

$$p(D \mid \theta, I) = \prod_{i=1}^{N} p(y_i \mid x_i, \theta, I)$$

For convenience (and also for numerical accuracy) this is often expressed in terms of the log-likelihood:

$$\log p(D \mid \theta, I) = -\frac{1}{2} \sum_{i=1}^{N} \left(\log(2\pi\varepsilon_i^2) + \frac{\left[y_i - y_M(x_i;\theta)\right]^2}{\varepsilon_i^2} \right)$$

```
# Log likelihood
def log_likelihood(theta, x, y, dy):
    """Return the log likelihood given the vector of parameters theta and
        numpy arrays for x, y, and dy (which is the standard deviation).
    """
    y_model = theta[0] + theta[1] * x
    return -0.5 * np.sum(np.log(2 * np.pi * dy ** 2) +
                         (y - y_model) ** 2 / dy ** 2)

# Let's use the (log) symmetric prior, which is the scale-invariant one.
# Uniform prior for the offset
def log_prior(theta):
    """Prior p(m) proportional to (1 + m^2)^{-3/2}"""
    if np.abs(theta[0]) < 1000:
        return -1.5 * np.log(1 + theta[1]**2)
    else:
        return -np.inf  # log(0)

def log_posterior(theta, x, y, dy):
    """Return the log posterior by evaluating the log prior and log
        likelihood.
       Probably should first check if the log prior is -np.inf before
        evaluating the log likelihood
    """
    return log_prior(theta) + log_likelihood(theta, x, y, dy)
```

We will use the emcee sampler, but in its Metropolis-Hastings mode. In particular, we invoke the sampler with `moves.GaussianMove(cov)`, which implements a Metropolis step using a Gaussian proposal with mean zero and covariance `cov`. The covariance `cov` could be a scalar, as it is here, or a vector or a matrix. See the relevant emcee manual page for further details and more general moves. The `stepsize` parameter is at our disposal to explore the consequences on convergence of it being too large or too small.

One consequence of this choice deserves emphasis. With `emcee`'s default *stretch move*, a walker's proposal is constructed from the positions of the other walkers, so the walkers in the ensemble are **not** independent chains. A Gaussian Metropolis proposal, in contrast, depends only on the walker's own current position. In this notebook each walker is therefore an independent Markov chain, and we will exploit this below when we treat the walkers as separate chains for convergence diagnostics. Do not assume this for an ensemble sampler in general.

Alternatively, you can use your own sampler here if you created one.

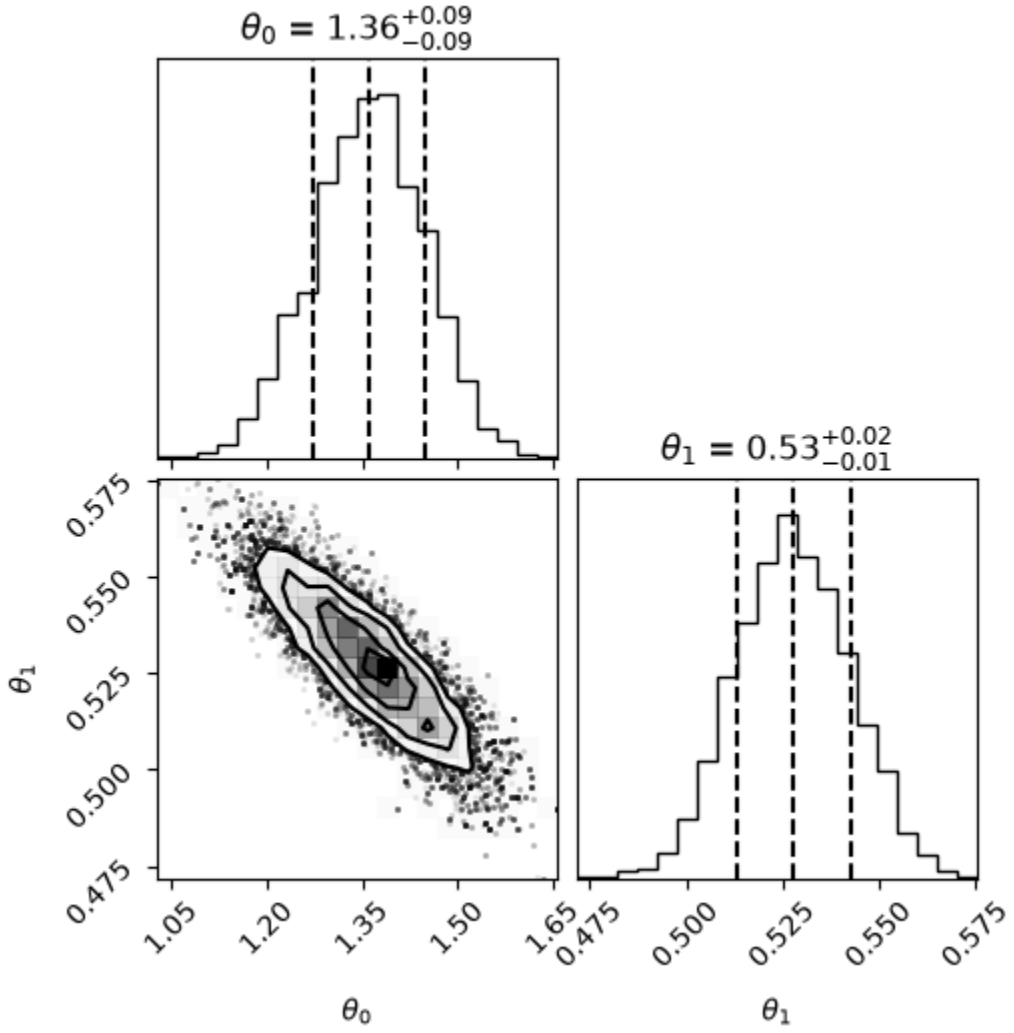
```
import emcee
import corner
print('emcee sampling (version: )', emcee.__version__)

ndim = 2  # number of parameters in the model
nwalkers = 10
nwarmup = 1000
nsteps = 5000

# MH-Sampler setup
stepsize = .005
cov = stepsize * np.eye(ndim)
p0 = np.random.rand(nwalkers,ndim)

# initialize the sampler
sampler = emcee.EnsembleSampler(nwalkers, ndim, log_posterior, args=[x, y, dy],
                                moves=emcee.moves.GaussianMove(cov))
```

```
emcee sampling (version: ) 3.1.6
```

To get the chains below we use `sampler.get_chain()`, which returns an array with the shape (# steps, # walkers, # dimensions). So 10 walkers taking 5000 steps each for a two-dimensional posterior (that is, $\boldsymbol{\theta}$ has two components) gives the shape (5000, 10, 2). We can combine the results from all the walkers with `sampler.get_chain(flat=True)`, which flattens the first two axes so that the array has the shape (# steps $\times$ # walkers, # dimensions). Note that the flattened samples are ordered step by step (all walkers at step 0, then all walkers at step 1, and so on). The flattened array is therefore fine for histograms and corner plots, but for trace plots we use the unflattened array and draw one line per walker. The log-posterior values are obtained in the same way with `sampler.get_log_prob()`.

```
# Sample the posterior distribution

# Warm-up
if nwarmup > 0:
    print(f'Performing {nwarmup} warmup iterations.')
    pos, prob, state = sampler.run_mcmc(p0, nwarmup)
    sampler.reset()
else:
    pos = p0

# Perform iterations, starting at the final position from the warmup.
print(f'MH sampler performing {nsteps} samples.')
%time sampler.run_mcmc(pos, nsteps)
print("done")

print(f"Mean acceptance fraction: {np.mean(sampler.acceptance_fraction):.3f}")

# get_chain() has shape (nsteps, nwalkers, ndim); flat=True merges the first two axes
samples = sampler.get_chain(flat=True)
samples_unflattened = sampler.get_chain()
lnposts = sampler.get_log_prob(flat=True)
lnposts_unflattened = sampler.get_log_prob()


# make a corner plot with the posterior distribution
fig = corner.corner(samples, quantiles=[0.16, 0.5, 0.84], labels=[r"$\theta_0$",
r"$\theta_1$"],
                    show_titles=True, title_kwargs={"fontsize": 12})
```

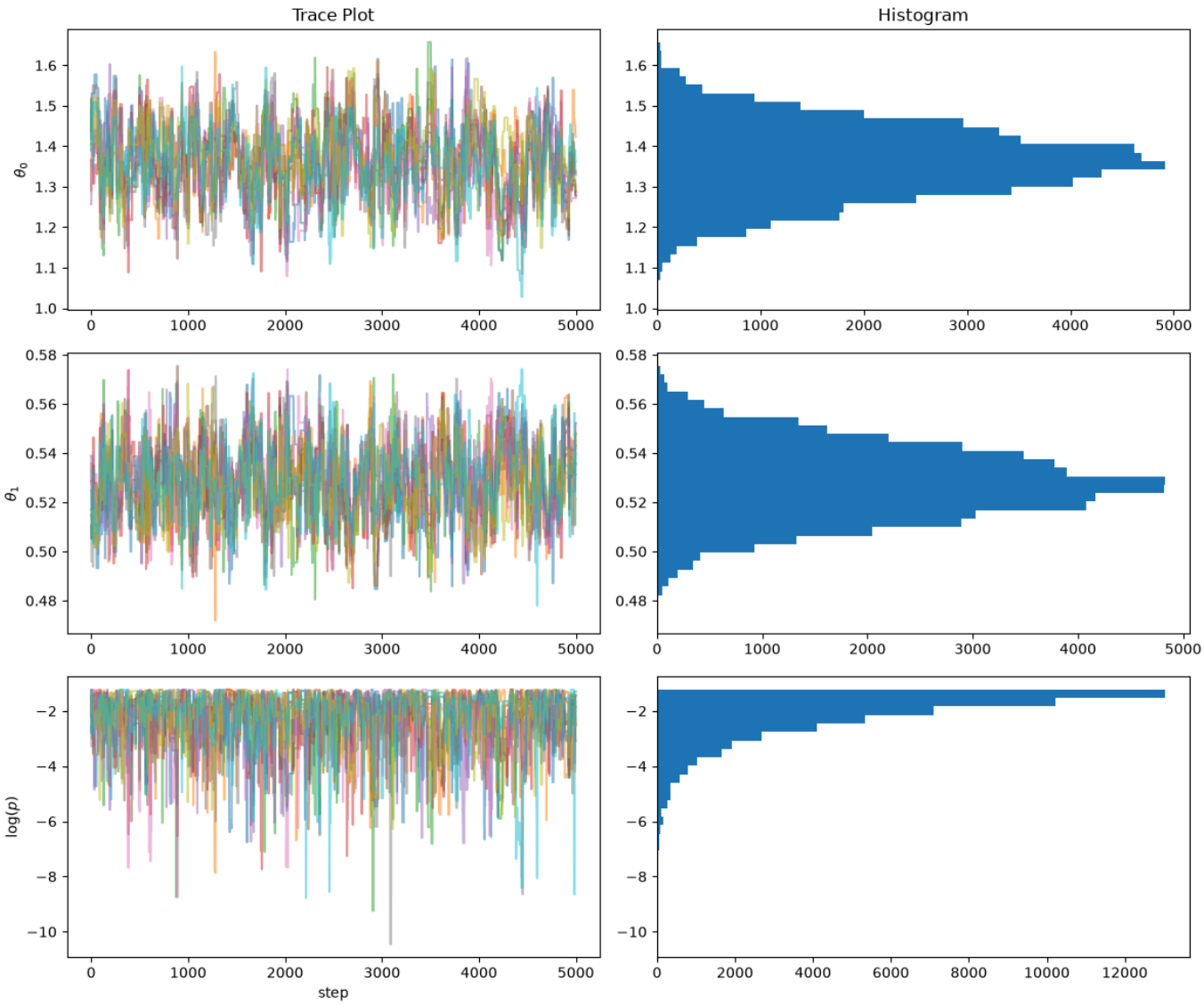
```
Performing 1000 warmup iterations.


MH sampler performing 5000 samples.


CPU times: user 1.59 s, sys: 617 µs, total: 1.6 s
Wall time: 1.59 s
done
Mean acceptance fraction: 0.113
```




```
print(samples.shape)
print(samples_unflattened.shape)
print(lnposts.shape)
print(lnposts_unflattened.shape)
```

```
(50000, 2)
(5000, 10, 2)
(50000,)
(5000, 10)
```

```
fix, ax = plt.subplots(3,2,figsize=(12,5*ndim))
for irow in range(ndim):
    # one line per walker
    ax[irow,0].plot(samples_unflattened[:,:,irow], alpha=0.5)
    ax[irow,0].set_ylabel(r'$\theta_{0}$'.format(irow))
    ax[irow,1].hist(samples[:,irow],orientation='horizontal',bins=30)

ax[2,0].plot(lnposts_unflattened, alpha=0.5)
ax[2,1].hist(lnposts,orientation='horizontal',bins=30)
ax[2,0].set_ylabel(r'$\log(p)$')
ax[2,0].set_xlabel('step')

ax[0,1].set_title('Histogram')
ax[0,0].set_title('Trace Plot')

plt.tight_layout()
```



How do we know this chain has converged to the posterior?

Credit to BDA3 by Gelman et al. and lecture notes by Rob Hicks

Standard Error of the Mean

This investigates the question how does the **mean** of θ deviate in our chain, and is capturing the *simulation error* of the mean rather than underlying uncertainty of our parameter θ:

$$SE(\bar{\theta}) = \frac{\text{Posterior Standard Deviation}}{\sqrt{N}}$$

where N is the chain length (the number of iterations in your chain).

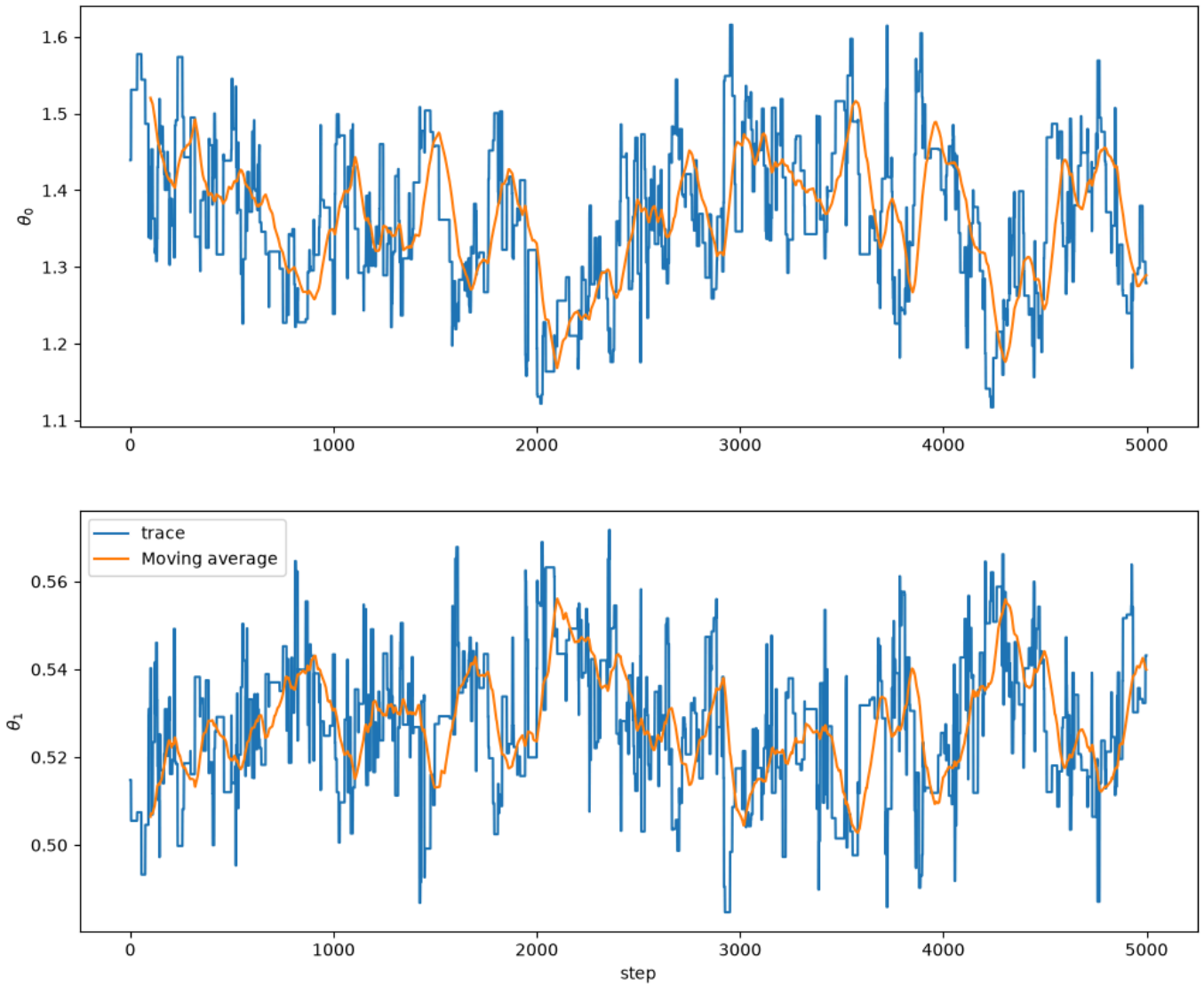
For our problem this is:

```
for irow in range(ndim):
    print(f"Standard Error of the Mean for theta_{irow}:
{samples[:,irow].std()/np.sqrt(samples.shape[0]):.1e}")
```

```
Standard Error of the Mean for theta_0: 3.9e-04
Standard Error of the Mean for theta_1: 6.5e-05
```

This is saying that very little of our posterior variation in θ is due to sampling error (that is good). We can visualize this by examining the moving average of a chain as we move through the iterations. Since each walker is an independent chain here (because of the Metropolis move, see above), we study a single walker (the flattened array interleaves the walkers and is not a time series):

```
# with the Gaussian Metropolis move each walker is an independent chain; shape (nsteps, ndim)
chain0 = samples_unflattened[:, 0, :]

fix, ax = plt.subplots(2,1,figsize=(12,10))
# pandas makes this easy:
df_chain = pd.DataFrame(chain0,columns=['theta0','theta1'])
df_chain['ma_theta0'] = df_chain.theta0.rolling(window=100,center=False).mean()
df_chain['ma_theta1'] = df_chain.theta1.rolling(window=100,center=False).mean()

ax[0].plot(np.arange(chain0.shape[0]),chain0[:,0],label=r'$\theta_0$')
ax[0].plot(np.arange(chain0.shape[0]),df_chain['ma_theta0'],label=r'Moving average')
ax[0].set_ylabel(r'$\theta_0$')

ax[1].plot(np.arange(chain0.shape[0]),chain0[:,1],label=r'trace')
ax[1].plot(np.arange(chain0.shape[0]),df_chain['ma_theta1'],label=r'Moving average')
ax[1].set_ylabel(r'$\theta_1$')
ax[1].set_xlabel('step')

plt.legend();
```

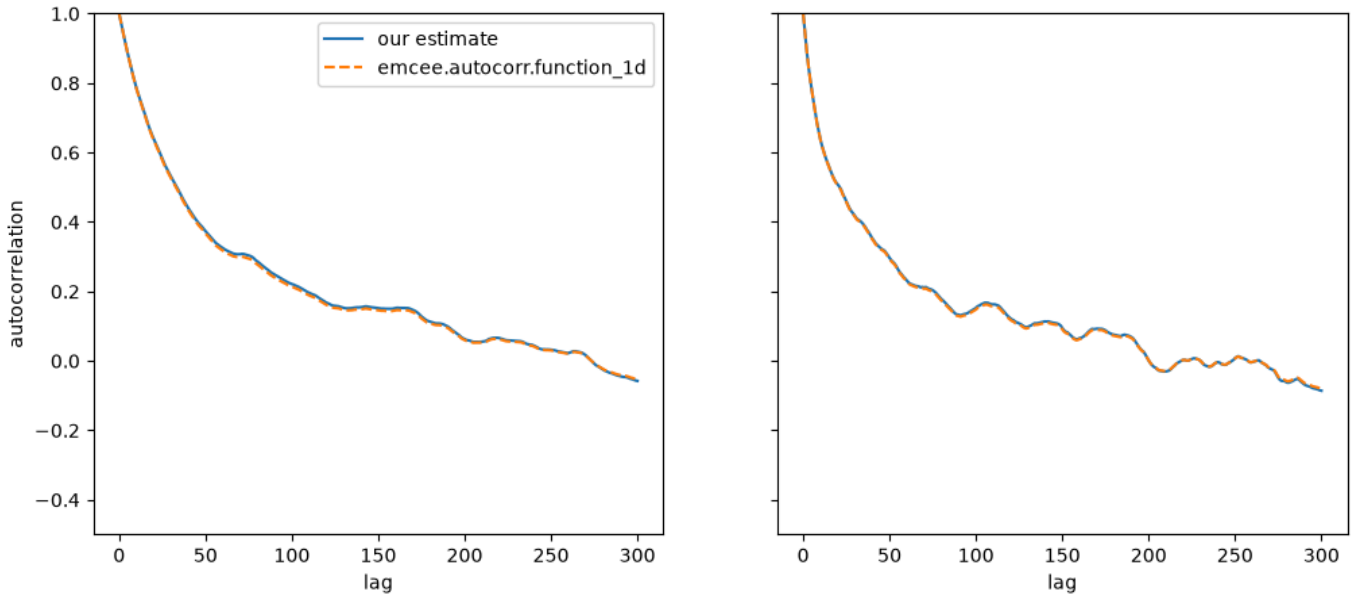


This is a good sign that our chain is stable, since both the individual samples of θ in our chain and the mean of the samples dance around a stable value of θ. The calculation above makes this more concrete. There are time series versions of this calculation that accounts for the fact that the chain is not iid.

Autocorrelation Plots

```
def autocorrelation(chain, max_lag=100):
    dimension = len(chain)
    acors = np.empty(max_lag+1)
    if max_lag > len(chain)/5:
        warnings.warn('max_lag is more than one fifth the chain length')
    # Create a copy of the chain with average zero
    chain1d = chain - np.average(chain)
    for lag in range(max_lag+1):
        unshifted = None
        shifted = chain1d[lag:]
        if 0 == lag:
            unshifted = chain1d
        else:
            unshifted = chain1d[:-lag]
        normalization = np.sqrt(np.dot(unshifted, unshifted))
        normalization *= np.sqrt(np.dot(shifted, shifted))
        acors[lag] = np.dot(unshifted, shifted) / normalization
    return acors
```

```
fig, ax = plt.subplots(1,2,sharey=True,figsize=(12,5))
for icol in range(ndim):
    # autocorrelation along a single walker's chain
    max_lag = 300
    acors = autocorrelation(chain0[:,icol],max_lag=max_lag)
    ax[icol].plot(acors, label='our estimate')
    # emcee's built-in estimate of the same function (FFT based)
    acors_emcee = emcee.autocorr.function_1d(chain0[:,icol])
    ax[icol].plot(acors_emcee[:max_lag+1], '--', label='emcee.autocorr.function_1d')
    ax[icol].set_xlabel('lag')
ax[0].set(ylabel='autocorrelation', ylim=(-.5, 1.));
ax[0].legend();
```



The two estimates agree (they differ only in the normalization of the lagged sums). A more compact summary is the *integrated autocorrelation time* τ, roughly the number of steps between effectively independent samples. `emcee` estimates it from all walkers with `sampler.get_autocorr_time()`. Note that the estimate is only considered reliable

when the chain is longer than about 50τ; for a shorter chain `emcee` raises an error unless `quiet=True`, in which case it warns and returns the estimate anyway.

```
tau = sampler.get_autocorr_time(quiet=True)
for irow in range(ndim):
    print(f"Integrated autocorrelation time for theta_{irow}: {tau[irow]:.0f} steps")
print(f"Effective number of independent samples: ~{nsteps*nwalkers/np.max(tau):.0f} (out of
{nsteps*nwalkers})")
```

```
Integrated autocorrelation time for theta_0: 50 steps
Integrated autocorrelation time for theta_1: 42 steps
Effective number of independent samples: ~993 (out of 50000)
```

Standard error of the mean, revisited

The standard error of the mean computed above assumed N *independent* samples. With correlated samples the effective number is only N/τ, so the error should be $\sigma\sqrt{\tau/N}$. We can check this directly: the walkers are independent chains with our Metropolis move (again, not for the default stretch move), so the scatter of the walker means measures the actual simulation error of a single-chain mean, and dividing by $\sqrt{n_{\text{walkers}}}$ gives the error of the combined mean.

```
N = samples.shape[0]
walker_means = samples_unflattened.mean(axis=0)   # shape (nwalkers, ndim)
for irow in range(ndim):
    sigma = samples[:,irow].std()
    se_naive = sigma / np.sqrt(N)
    se_tau = sigma * np.sqrt(tau[irow] / N)
    se_empirical = walker_means[:,irow].std(ddof=1) / np.sqrt(nwalkers)
    print(f"theta_{irow}: SE assuming independent samples = {se_naive:.1e},"
          f"  corrected with tau = {se_tau:.1e},"
          f"  from the spread of walker means = {se_empirical:.1e}")
```

```
theta_0: SE assuming independent samples = 3.9e-04,  corrected with tau = 2.8e-03,  from the
spread of walker means = 2.5e-03
theta_1: SE assuming independent samples = 6.5e-05,  corrected with tau = 4.2e-04,  from the
spread of walker means = 3.9e-04
```

Acceptance Rate for the MH Algorithm

Recall that we want the acceptance rate to be in the range .2 to .4. For our problem this paper suggests an acceptance rate of .234 for random walk MH.

Since the number of **new** members in the chain represent the number of acceptances, count changes in chain values and divide by total chain length to calculate acceptance rate:

```
print(f"Acceptance Rate is: {np.mean(sampler.acceptance_fraction):.3f}")
```

```
Acceptance Rate is: 0.113
```

The acceptance rate is helpful in describing convergence because it indicates a good level of “mixing” over the parameter space. The acceptance rate can be tuned via the proposal width after which we re-run our MH MCMC sampler.

> Note: modern software (like pymc and emcee) can auto-tune the proposal distribution to achieve a desired acceptance rate.

Gelman Rubin Diagnostic

If our MH MCMC Chain reaches a stationary distribution, and we repeat the exercise multiple times, then we can examine if the posterior for each chain converges to the same place in the distribution of the parameter space.

Steps:

1. Run multiple chains starting at different points (multiple walkers). Discard the warm-up for each.
2. Split each chain in two, with N iterations in each half chain. Call M the total number of chains now (twice the original number).
3. Calculate the within and between chain variance. This tests both mixing (if well-mixed, the separate parts of different chains should mix) and stationarity (two halves of each chain should be sampling the same distribution).

- Label the scalar parameter or expectation value being tested as ψ and label the simulated results as ψ_{ij}, where i runs from 1 to N within each chain and j labels the chain from 1 to M. Then we define:

$$\overline{\psi}_{\cdot j} \equiv \frac{1}{N}\sum_{i=1}^{N}\psi_{ij} \quad \text{and} \quad \overline{\psi}_{\cdot\cdot} \equiv \frac{1}{M}\sum_{j=1}^{M}\overline{\psi}_{\cdot j}$$

where $\overline{\psi}_{\cdot j}$ is the mean within chain j and $\overline{\psi}_{\cdot\cdot}$ is the average (mean) of these means across the M chains.

- Within chain variance:

$$W = \frac{1}{M}\sum_{j=1}^{M} s_j^2 \quad \text{where} \quad s_j^2 = \frac{1}{N-1}\sum_{i=1}^{N}(\psi_{ij} - \overline{\psi}_{\cdot j})^2 \ ,$$

with s_j^2 is the variance of each chain. So W is the mean of the in-chain variances. It is expected that W will *underestimate* the variance of ψ (which we’ll denote var (ψ) because an individual sequence (i.e., chain) with $N < \infty$ will not have run forever, so it will not have ranged over the full target distribution, so it will have less variability.

- Between chain variance:

$$B = \frac{N}{M-1}\sum_{j=1}^{M}(\overline{\psi}_{\cdot j} - \overline{\psi}_{\cdot\cdot})^2 \ .$$

There is an N in the numerator of B because it is from the variance of the within-sequence means $\overline{\psi}_{\cdot j}$, each of which is an average of N values ψ_{ij}.

4. Calculate the estimated variance of ψ as the weighted sum of within and between chain variance.

$$\hat{\text{var}}\,(\psi)^+ = \left(1 - \frac{1}{N}\right)W + \frac{1}{N}B \ .$$

This quantity is expected to *overestimate* var (ψ) but is unbiased under stationarity.

5. Calculate the potential scale reduction factor, $\hat{R}$, which is the factor by which the scale that characterizes the distribution for ψ at the current stage might be reduced if we increased each chain size N toward infinity:

$$\hat{R} = \sqrt{\frac{\hat{\text{var}}\,(\psi)}{W}}$$

Based on our expectations, this should be greater than 1 because the numerator overestimates var (ψ) and denominator underestimates it. But if it is close to 1, then it should mean that both chains are mixing around the stationary distribution.
Gelman and Rubin show that when $\hat{R}$ is greater than 1.1 or 1.2, we need longer runs.

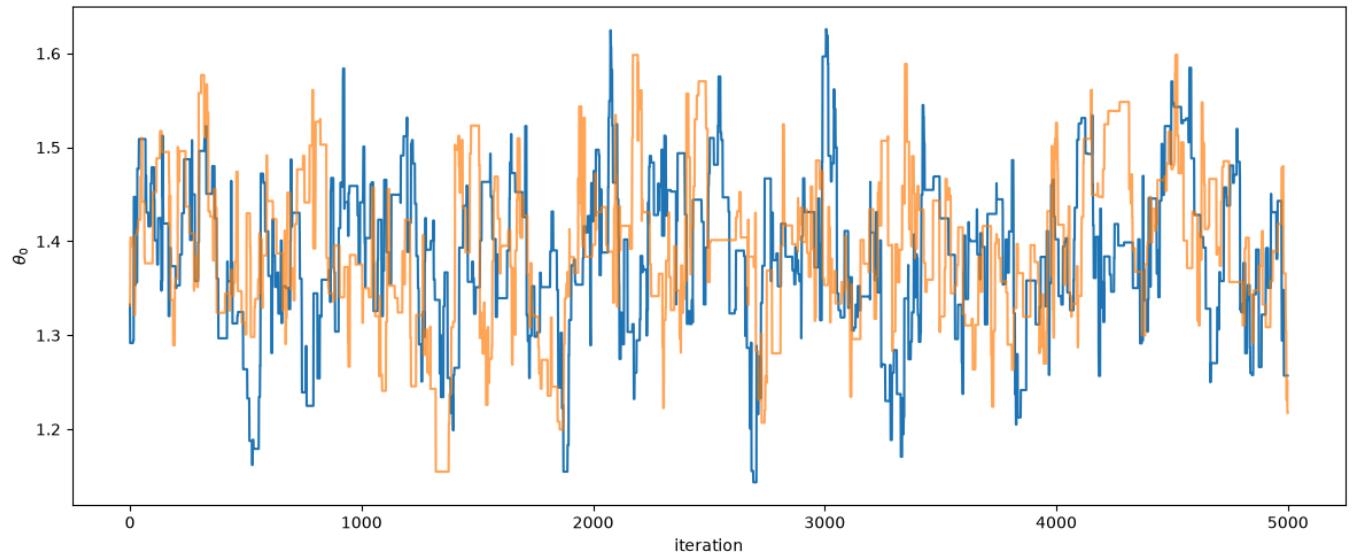
Let's run 2 chains:

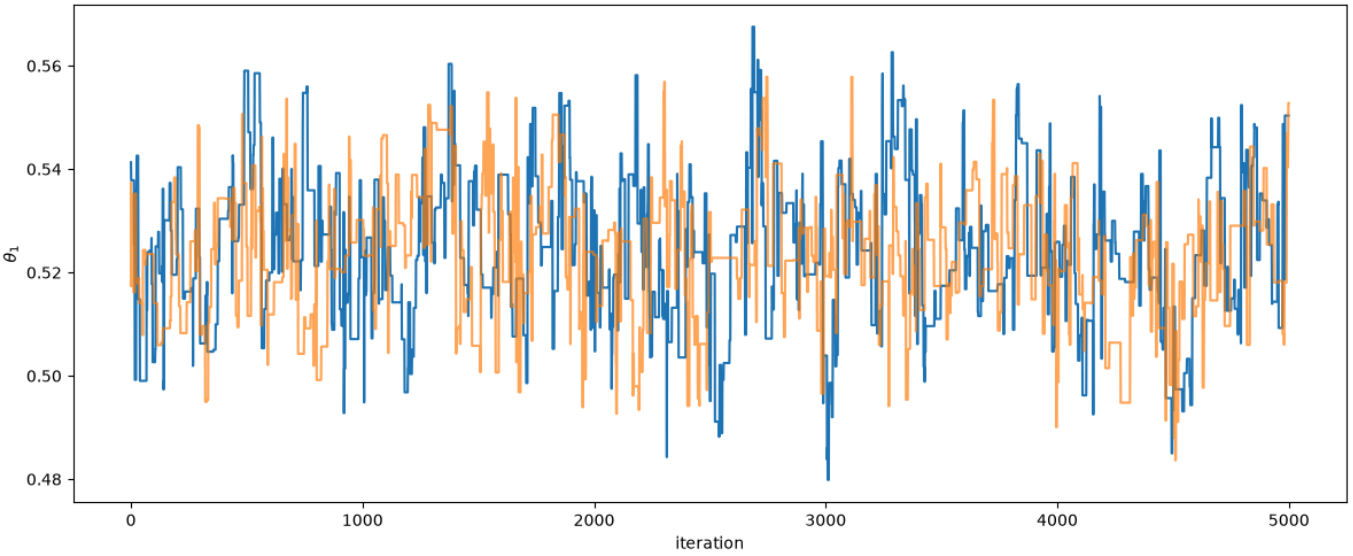
```
no_of_chains=2
chains=[]

for ichain in range(no_of_chains):
    sampler.reset()
    p0 = np.random.rand(nwalkers,ndim)
    # Warm-up
    if nwarmup > 0:
        print(f'Chain {ichain} performing {nwarmup} warmup iterations.')
        pos, prob, state = sampler.run_mcmc(p0, nwarmup)
        sampler.reset()
    else:
        pos = p0

    # Perform iterations, starting at the final position from the warmup.
    print(f'MH sampler {ichain} performing {nsteps} samples.')
    sampler.run_mcmc(pos, nsteps)
    print("done")
    print(f"Mean acceptance fraction: {np.mean(sampler.acceptance_fraction):.3f}")

    # With the Gaussian Metropolis move each walker is an independent chain
    # (not true for emcee's default stretch move!). We keep one walker (index 0)
    # as the chain from this run; shape (nsteps, ndim)
    chains.append(sampler.get_chain()[:, 0, :])
```

```
Chain 0 performing 1000 warmup iterations.


MH sampler 0 performing 5000 samples.


done
Mean acceptance fraction: 0.117
Chain 1 performing 1000 warmup iterations.


MH sampler 1 performing 5000 samples.


done
Mean acceptance fraction: 0.111
```

```
chain1 = chains[0]
chain2 = chains[1]
num_iter = chain1.shape[0]
```

```
fig, ax = plt.subplots(2, 1, figsize=(12,10))
for icol in range(ndim):
    ax[icol].plot(np.arange(num_iter), chain1[:,icol])
    ax[icol].plot(np.arange(num_iter), chain2[:,icol], alpha=.7)
    ax[icol].set_ylabel(fr'$\theta_{icol}$')
    ax[icol].set_xlabel('iteration')

fig.tight_layout()
```


1.6
1.5
1.4
1.3
1.2
θ_0
0
1000
2000
3000
4000
5000
iteration


0.56
0.54
0.52
0.50
0.48
θ_1
0
1000
2000
3000
4000
5000
iteration

```
# rewrite of Gelman-Rubin estimation
# we only want one of the variables
Nchain = int(nsteps / 2)  # maximum size of each half chain
Mchain = 4   # total number of chains
param = 0    # which parameter to use


def Gelman_Rubin_diagnostic_calc(chains, Nchain, Mchain=4, param=0):
    psi_chains = np.zeros((Mchain, Nchain))
    for icol in range(0, Mchain, 2):
        i = int(icol/2)
        psi_chains[icol,:] = np.array( chains[i] )[:Nchain, param]
        psi_chains[icol+1,:] = np.array( chains[i] )[Nchain:2*Nchain, param]

    psi_mean = np.array([chain.mean() for chain in psi_chains])
    psi_mean_all = psi_mean.sum() / Mchain

    var_chain = np.zeros(Mchain)
    for i in range(Mchain):
        var_chain[i] = 1./(Nchain - 1) * \
                           ((psi_chains[i] - psi_mean[i])**2).sum()

    W = var_chain.sum() / Mchain

    B = Nchain / (Mchain - 1) * \
          np.array([(mean - psi_mean_all)**2 for mean in psi_mean]).sum()

    var_theta = (1. - 1./Nchain) * W + 1./Nchain * B
    Rhat = np.sqrt(var_theta/W)
    print(fr"Nchain = {Nchain:4d}  Rhat = {Rhat:.3f}")

print(f"Gelman-Rubin Diagnostic for different chain lengths: ")
for Nchain in [50, 100, 200, 500, 1000, 2000]:
    Gelman_Rubin_diagnostic_calc(chains, Nchain, param=0)
```

```
Gelman-Rubin Diagnostic for different chain lengths:
Nchain =   50  Rhat = 1.166
Nchain =  100  Rhat = 1.071
Nchain =  200  Rhat = 1.013
Nchain =  500  Rhat = 1.065
Nchain = 1000  Rhat = 1.056
Nchain = 2000  Rhat = 1.003
```

To repeat: Gelman and Rubin show that when $\hat{R}$ is greater than 1.1 or 1.2, we need longer runs.

Univariate Approaches

The diagnostics we have discussed are all univariate (they work perfectly when there is only one parameter to estimate).

So most people examine univariate diagnostics *for each variable*, examine autocorrelation plots, acceptance rates and try to argue chain convergence based on that.

22. Advanced sampling algorithms

In this section we introduce a handful of useful and more advanced sampling algorithms such as Hamiltonian Monte Carlo (HMC), ensamble and slice sampling, parallel tempering, importance resampling, and nested sampling.

22.1. Ensemble sampling

> **Note**
>
> This section is under construction. The motivation below is in place; the description of the stretch move and of the ensemble sampler in practice will be added.

Motivation: sampling from correlated distributions

If our posterior has projections that are slanted, indicating correlations, and we are doing Metropolis-Hastings (MH) sampling, how do we decide on a step size? The problem is that we want to step differently in different directions: a step long enough to explore the long axis will lead to many rejections in the orthogonal direction. *So we do not want an isotropic step proposal!*

If we propose steps by

$$p(\boldsymbol{x}) = \frac{1}{\sqrt{(2\pi)^N |\Sigma|}} e^{-\boldsymbol{x}^\intercal \cdot \Sigma^{-1} \cdot \boldsymbol{x}},$$

then we don't need to take $\Sigma \propto \sigma^2 \mathbb{1}_N$!

- We have $N(N+1)/2$ parameters in Σ to "tune" to reduce the correlation time.
- However, this is increasingly difficult as N increases.

One improvement is to do a linear transformation of $\boldsymbol{\theta} \longrightarrow \boldsymbol{\theta}' = A\boldsymbol{\theta} + B$ such that $\boldsymbol{\theta}'$ is uncorrelated with similar σ_i's in each direction. Thus effectively to rotate the slanted ellipse.

Or one could use an "affine invariant" sampler such as `emcee`. An affine transformation is an invertible mapping from $\mathbb{R}^N \to \mathbb{R}^N$, namely $\boldsymbol{y} = A\boldsymbol{x} + B$, which is a combination of stretching, rotation, and translation. "Affine invariant" means that the sampler performs equally well on all affine transformations of a distribution. So `emcee` figures out how to make optimal steps. It does this by using the many walkers at time t, which have sampled the space, to construct an appropriate affine compatible update step for $t + 1$. This is one reason to make sure there are plenty of walkers.

22.2. Parallel tempering

Parallel tempering was particularly introduced to deal with multimodal distributions. This is analogous to the problem of *global* (as opposed to local) optimization, which can mean either minimizing or maximizing a function. It is a challenging problem: how do you jump from one region of high posterior density to another when there is a large region of low probability in between? This problem shows up for evidence calculations, because we need to integrate over the entire parameter space. In contrast, for parameter estimation, it may be sufficient to start walkers in the vicinity of the "best" (highest posterior) part of the posterior.

Parallel tempering is built on top of an MCMC sampler. The general idea is to simulate N *replicas* of a system, each at a different temperature. The temperature of a Metropolis-Hastings Markov chain specifies how likely it is to sample from a low-density part of the target distribution. At high temperature, large volumes of phase space are sampled roughly, while low temperature systems have precise sampling in a local region of the parameter space, where they can get stuck in a local energy minimum (meaning a local posterior maximum). Parallel tempering works by letting the systems at different temperatures exchange configurations, which enables the low T system access to a complete set of low-temperature regions.

The temperature enters in analogy to a Boltzmann factor $e^{-E/k_BT} = e^{-\beta E}$ where $\beta \equiv 1/k_BT$. Instead of E we have $-\log p_\beta(x)$ so

$$p_\beta(x) = e^{-\beta(-\log p(x))} = \big(p(x)\big)^\beta$$

We seek the posterior written with Bayes' theorem and normalization constant C:

$$p(\boldsymbol{\theta}|D,I) = Cp(D|\boldsymbol{\theta},I)p(\boldsymbol{\theta}|I)$$

and generalize to the temperature-dependent posterior p_β:

$$p_{\beta(\boldsymbol{\theta}|D,I)} \propto p(D|\boldsymbol{\theta},I)^\beta p(\boldsymbol{\theta}|I) \qquad 0 \leq \beta \leq 1$$

or

$$\log p_{\beta(\boldsymbol{\theta}|D,I)} = C + \beta \log p(D|\boldsymbol{\theta},I) + \log p(\boldsymbol{\theta}|I).$$

So the desired distribution is $\beta = 1$, and $\beta = 0$ is the prior alone. At large temperature, $\beta = 0$, and we sample the prior, which should encompass the full (accessible) space. As we approach $\beta = 1$ we fous increasingly on where the likelihood is large.

We will use $\beta \in \{1, \beta_2, \ldots, \beta_N\}$ running *in parallel*, but swapping members of the chains in such a way that detailed balance is preserved. The general sampling strategy is:

- at intervals, pick a pair of *adjacent* chains at random: β_i and β_{i+1};
- propose a swap of their current positions at this time t, namely exchange $\boldsymbol{\theta}_{t,i}$ and $\boldsymbol{\theta}_{t,i+1}$;
- accept this proposal with probability (note the is and $(i+1)$s):

$$r = \min\left\{1, \frac{p_\beta(\boldsymbol{\theta}_{t,i+1}|D,\beta_i,I)p_\beta(\boldsymbol{\theta}_{t,i}|D,\beta_{i+1},I)}{p_\beta(\boldsymbol{\theta}_{t,i}|D,\beta_i,I)p_\beta(\boldsymbol{\theta}_{t,i+1}|D,\beta_{i+1},I)}\right\}$$

This algorithm will preserve detailed balance.

We can make this expression more intuitive if we connect back to the physics analogy.

$$p_\beta(\boldsymbol{\theta}|D,I) \propto p(D|\boldsymbol{\theta},I)^\beta p(\boldsymbol{\theta}|I) \propto e^{-\beta(-\log p(D|\boldsymbol{\theta},I))p(\boldsymbol{\theta}|I)}$$

and $-\log p(D|\boldsymbol{\theta},I)$ is the potential energy $U(\boldsymbol{r})$ (by analogy). Then the priors drop out of the ratio in r (the prior is independent of β), leaving:

$$r = \min\left\{1, e^{-(\beta_i-\beta_{i+1})(U(\boldsymbol{r}_{i+1}-U(\boldsymbol{r}_i)))}\right\}$$

so the four terms in r arise from the Boltzman factors of the energy difference at the two temperatures.

Things to specify for the sampler: * n_s: propose a swap every n_s iterations, which is implemented by drawing a random number $U_1 \sim$ Uniform $[0,1]$ every iteration and proposing a swap if $U_1 \leq 1/n_s$. * The length and spacing of the temperature ladder.

Calculating the evidence

To calculate the *evidence* from parallel tempering, we can use *thermodynamic integration* see Ref. [GC04].

Define the temperature dependent evidence:

$$Z(\beta) \equiv \int d\boldsymbol{\theta}\, p(D|\boldsymbol{\theta},I)^\beta p(\boldsymbol{\theta}|I) = \int d\boldsymbol{\theta}\, e^{\beta \log p(D|\boldsymbol{\theta},I)} p(\boldsymbol{\theta}|I)$$

so we want $Z(1)$.

Now $Z(\beta)$ satisfies a differential equation:

$$\begin{aligned}\frac{d\log Z(\beta)}{d\beta} &= \frac{1}{Z(\beta)}\int d\boldsymbol{\theta}\big(\log p(D|\boldsymbol{\theta},I)\big)p(D|\boldsymbol{\theta},I)^\beta p(\boldsymbol{\theta}|I) \\ &\equiv \langle \log p(D|\boldsymbol{\theta},I\rangle_\beta,\end{aligned}$$

which is the average of the log likelihood at temperature β. So we can integrate over β (the first term is doesn't contribute if the prior is normalized):

$$\log Z(1) = \log Z(0) + \int_0^1 d\beta\, \langle \log p(D|\boldsymbol{\theta},I)\rangle_\beta,$$

$\Longrightarrow$ estimate from emcee samples by computing the average of $p(D|\boldsymbol{\theta}, I)$ within each chain and then evaluating the integral from a quadrature formula (e.g., Simpson's rule).

Example of parallel tempering

An example of parallel tempering is given in Demo: Multimodal distributions with the ptemcee and zeus samplers.

Comments:

- First we set up a bi-modal distribution (it was originally intended to be a surprise, so the code was hidden). Just two Gaussians with different amplitudes.
- A first sampling try with an ordinary Metropolis-Hastings (MH) sampler fails by only finding one mode.
- But then if one checks two chains, separate modes are found but the chains do not mix. With many walkers one might find multiple modes, but the relative normalizations would not work because each walker would not explore the space.
- The setup for `ptemcee` includes a temperature grid chosen so that numerically integrating over temperature for the evidence has a finer grid at low temperatures for greater accuracy.
- Note the corner plots for different temperatures and how the multimodal structure emerges.

22.3. Hamiltonian Monte Carlo

> "Why walk when you can flow."
>
> —Richard McElreath

Overview and intuition from simulations

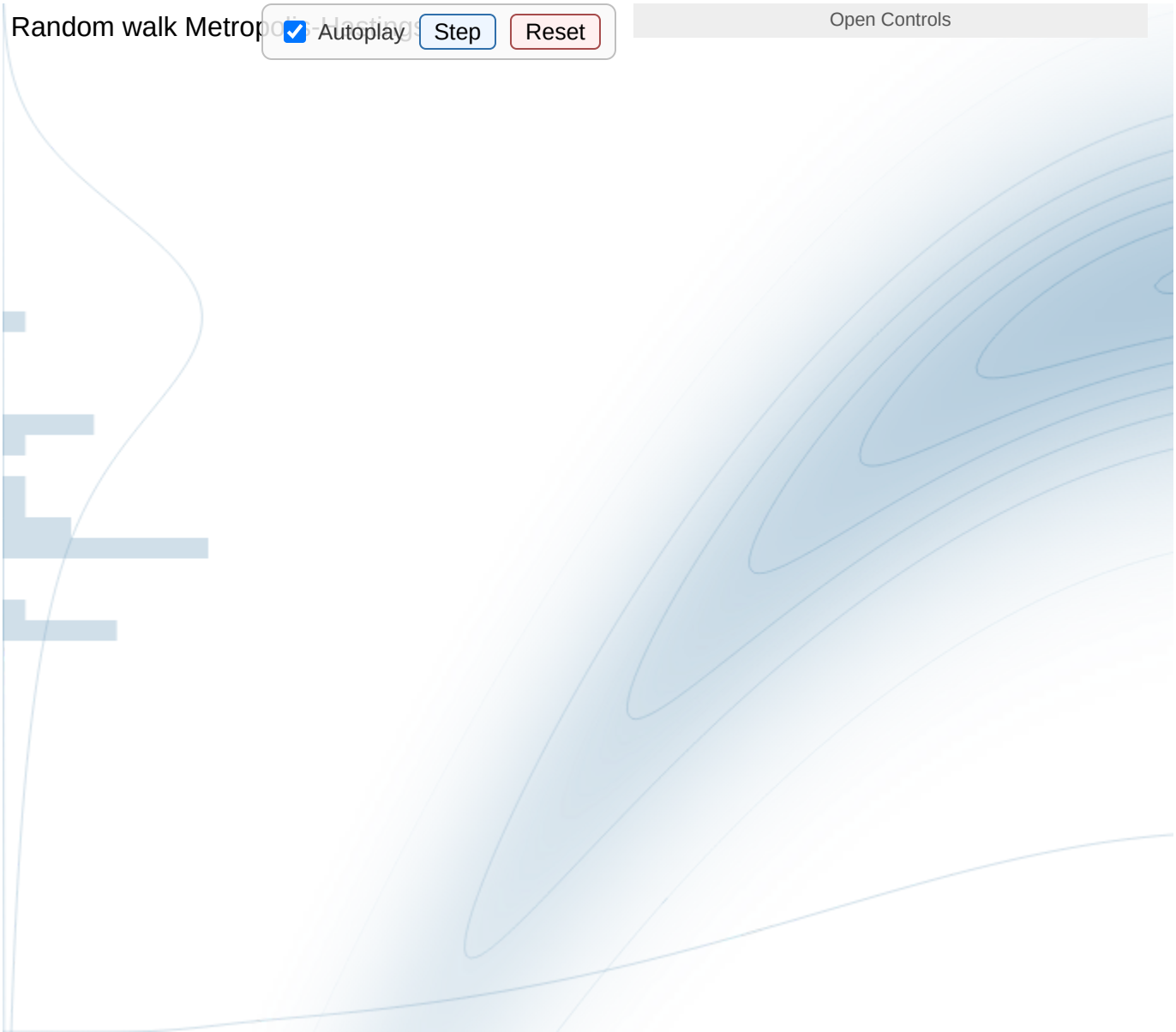
One of the most widespread techniques in contemporary samplers is Hamiltonian Monte Carlo, or HMC, which is a Metropolis method that uses gradient information to reduce the random-walk behavior and to collect effectively independent samples. It was introduced in lattice QCD by Duane et al. [DKPR87] and was originally named Hybrid Monte Carlo as they were combining molecular dynamics with the Metropolis MCMC algorithm. A very good and detailed review of HMC is written by Radford Neal and is contained in the Handbook of Markov Chain Monte Carlo [BGJM11].

To establish some intuition about HMC, we return to the excellent set of interactive demos by Chi Feng, which are reproduced here. You could start with the HMC demo or refresh your memory first on MH sampling.

Controls for the MCMC simulations

- The initial simulation uses MH sampling. To switch to HMC, select `Open Controls` and pull down the `Algorithm` menu and select `HamiltonianMC`. Note the `Algorithm Options`.

- After making changes, use `Close Controls` to avoid obscuring the simulation.
- For orientation, you might want to switch the `Target distribution` to `standard`. This distribution is a two-dimensional Gaussian (just the product of two one-dimensional Gaussians).
- If you uncheck the `Autoplay` box, you can use the `Step` button to see the algorithm carried out one step at a time.
- Use the `Reset` button to clear the sampled points.



Why HMC?

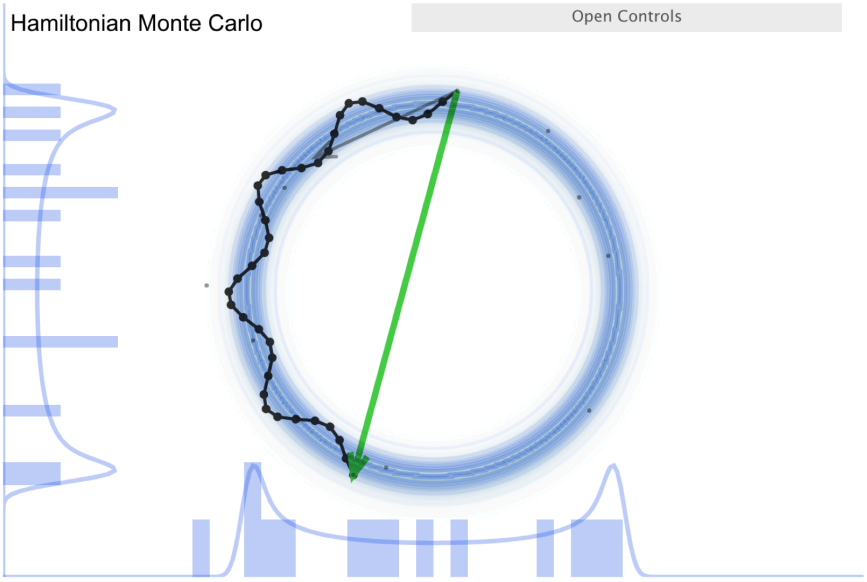
Let's summarize the argument for abandoning Metropolis-Hasting (MH) sampling in favor of HMC. First recall the random walk MH:

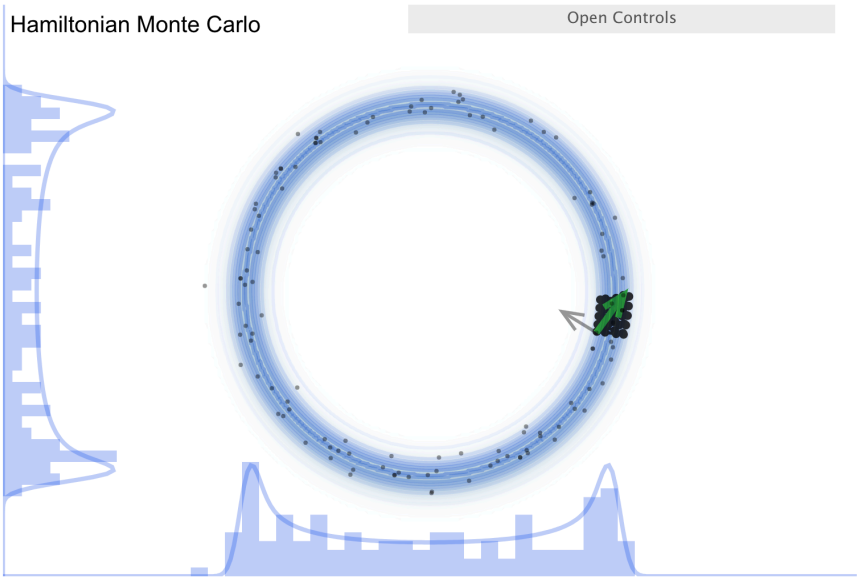
1. Make a random proposal for new parameter values (a step in parameter space, indicated in the visualization by an arrow).

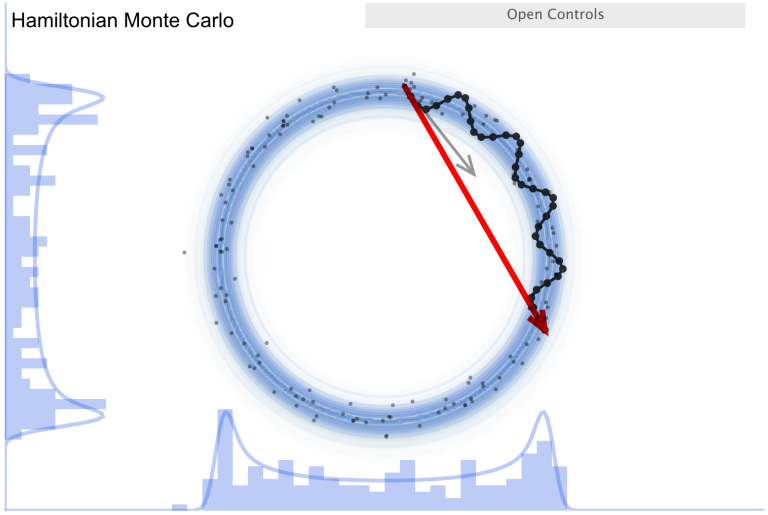
2. Aceept (green arrow) or reject (red arrow) based on a Metropolis criterion (which is not deterministic but has a random element). This is *diffusion* (i.e., a random walk), so it is not efficient in exploring the parameter space and needs special tuning to avoid too high a rejection rate. The donut shape in the simulation is common in higher dimensions and it is difficult to explore. (I.e., consider a multidimensional uncorrelated Gaussian distribution. In spherical coordinates the distribution is $\propto r^n e^{-r^2/2\sigma^2}$, so the marginalized distribution will be peaked away from $r = 0$.)

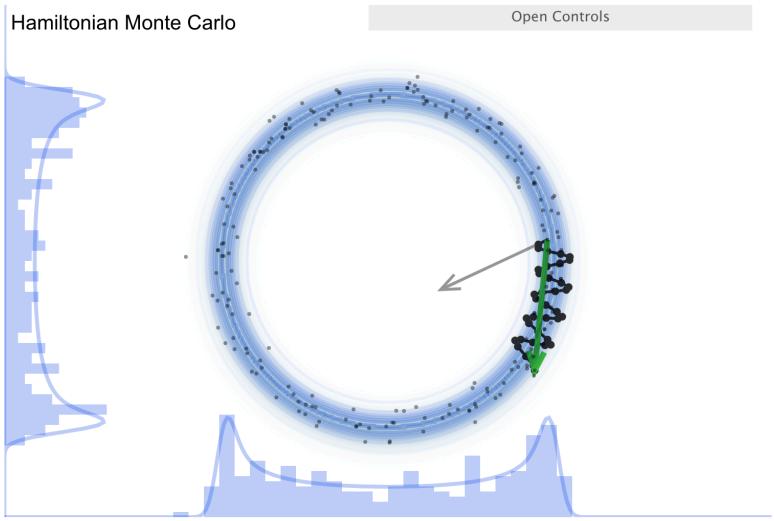
Now consider the HMC simulation. The idea is that we map our parameter vector $\boldsymbol{\theta}$ of length n to a particle in an n-dimensional space. The surface is an n-dimensional (inverted) bowl with the shape given by minus-log(target distribution), where the target distribution is the posterior. Now treat the system as frictionless and "flick" the particle in a random direction, so it travels across the bowl.

See the snapshots from the simulation in the figures below. The little gray arrow is the flick. After the particle travels some distance, there is a decision whether to accept; this is done as in MH. Most endpoints are within a high probability region, so a high percentage is accepted. Chains can get far from the starting point easily $\Longrightarrow$ efficient exploration of the full distribution. While more calculation along the path is needed, fewer samples are needed; this is typically a winning trade-off. Check the donut example: HMC works very well!









There is a further improvement called NUTS, which stands for “no-U-turn sampler” (there is a separate demo). The idea is to address the problem that HMC needs to be told how many steps to take before another random flick. Too few steps and the samples are too similar; too many steps are also too similar. NUTS *adaptively* finds a good number of steps. In particular, it simulates in *both* directions to figure out when the path turns around (i.e., U-turns) and stops there. There are other adaptive features; see the Stan documentation or the conceptual introduction by Michael Betancourt [Bet18] for details. Note that NUTS still has trouble with multimodal targets $\Longrightarrow$ can explore each high probability area, but has trouble going between them.

Return to the MCMC simulations above

- Turn `Autoplay` off and make multiple steps, noting how the HMC algorithm is realized.
- To try out NUTS and compare to vanilla HMC, open the controls and select `NaiveNUTS` from the `Algorithms` pulldown menu.

Implementation details

The basic idea behind HMC is to translate a PDF for the desired distribution into a potential energy function and to add a (fictitious!) momentum variable. In the Markov chain at each iteration, one resamples the momentum (the flick!), creates a proposal using classical Hamiltonian dynamics, and then does a Metropolis update. Thus, the state

space vector $\boldsymbol{x}$ is augmented by a conjugate momentum $\boldsymbol{p}$. Note that $\boldsymbol{x}$ will be the model parameters $\boldsymbol{\theta}$ (with d components) in our applications but here we will stick to $\boldsymbol{x}, \boldsymbol{p}$ to make the connection with Hamiltonian dynamics explicit.

The $2 \times d$ dimensional phase space is for a *Hamiltonian* defined as

$$H(\boldsymbol{x}, \boldsymbol{p}) = K(\boldsymbol{p}) + U(\boldsymbol{x}), \tag{22.1}$$

where the kinetic energy function $K(\boldsymbol{p})$ is a design choice (see below) while the potential energy function $U(\boldsymbol{x})$ will be defined as minus the log probability of the density (PDF) that we wish to sample

$$\begin{aligned} K(\boldsymbol{p}) &= \frac{1}{2} \boldsymbol{p}^T \mathcal{M}^{-1} \boldsymbol{p}, \\ U(\boldsymbol{x}) &= -\ln\left(p\left(\boldsymbol{x}\right)\right), \end{aligned} \tag{22.2}$$

where $\mathcal{M}$ is a user-defined mass matrix (positive definite and symmetric; typically diagonal as often proportional to the d-dimensional identity matrix) and where we use $p\left(\boldsymbol{x}\right)$ to denote the PDF that we want to sample. It could for example be a posterior distribution for some model parameters.

We can link $H(\boldsymbol{x}, \boldsymbol{p})$ to a probability distribution for $(\boldsymbol{x}, \boldsymbol{p})$ using the canonical Boltzmann distribution

$$p\left(\boldsymbol{x}, \boldsymbol{p}\right) = \frac{1}{Z} \exp\left(\frac{-H(\boldsymbol{x}, \boldsymbol{p})}{T}\right), \tag{22.3}$$

where Z is the normalization and T is a temperature (here serving to render the exponents dimensionless). Let us set $T = 1$ and use the expressions (22.2) to obtain the separable joint distribution

$$p\left(\boldsymbol{x}, \boldsymbol{p}\right) \propto p\left(\boldsymbol{x}\right) \exp\left(\frac{\boldsymbol{p}^T \mathcal{M}^{-1} \boldsymbol{p}}{2}\right). \tag{22.4}$$

Since we will use the Metropolis ratio during sampling, the normalization factor becomes irrelevant. The joint distribution is a produce of two (independent) distributions, the one for $\boldsymbol{p}$ becomes a Gaussian with our design choices and the one for $\boldsymbol{x}$ is the one that we are seeking. If we manage to efficiently sample from $p\left(\boldsymbol{x}, \boldsymbol{p}\right)$ then we can just ignore the $\boldsymbol{p}$ samples and we will be left with samples from $p\left(\boldsymbol{x}\right)$.

How do we sample from $p\left(\boldsymbol{x}, \boldsymbol{p}\right)$? Well, naturally we simulate Hamiltonian dynamics for a finite period of time. For Metropolis updates, using a proposal found by Hamiltonian dynamics, the acceptance probability will be one since H is kept invariant. Unfortunately, we will only be able to solve Hamilton's equations approximately and in practice we might not end up with perfect energy conservation.

Apart from energy conservation, two additional key properties of Hamiltonian dynamics—in the context of using it for MCMC sampling—are that it is time reversibile and that it preserves the volume in $(\boldsymbol{x}, \boldsymbol{p})$-space, a result known as Liouville's theorem. (If we take a cluster of points and follow their time evolution, the volume they occupy is unchanged.)

This is critical because a change in volume would mean we would have to make a nontrivial adjustment to the proposal because the normalization Z would change.

Hamilton's equations are

$$\begin{aligned}\frac{dx_i}{dt} &= \frac{\partial H}{\partial p_i} = \left(\mathcal{M}^{-1}\boldsymbol{p}\right)_i \\ \frac{dp_i}{dt} &= -\frac{\partial H}{\partial x_i} = \frac{\partial \ln(p\left(\boldsymbol{x}\right))}{\partial x_i}.\end{aligned} \tag{22.5}$$

Every HMC iteration begins with a sample of the momentum $\boldsymbol{p}$ and continues with the integration of Hamilton's equations for some total time t before the final Metropolis update and the (likely) acceptance of the new position.

In order to maintain energy conservation it is important to simulate the Hamiltonian dynamics using a short time step and to perform the integration such that the accumulation of numerical errors is avoided. The standard method in HMC is known as leapfrog integration and is a symplectic integrator where the discretization error for each time step is equally likely to be positive and negative, thus helping to preserve the total energy throughout the simulation. Leapfrog integration works as an update of the momentum with half a time step, followed by a full step for the position (using the new values for the momentum variables), and finally another half time step for the momentum using the newly updated position variables.

Leapfrog algorithm

The requirements for HMC are satisfied by the exact Hamilton's equations, but we are *approximating* the solution to these differential equations. This necessitates a *symplectic* (symmetry conserving) integration.

Ordinary Runge-Kutta-type ODE solvers won't work because they are not time-reversal invariant. E.g., consider the Euler method. Forward and backward integrations are different because the derivatives are calculated from different points. We need something like the Leapfrog algorithm (here is the 2nd order version):

$$\begin{aligned}\boldsymbol{p}_i(t+\epsilon/2) &= \boldsymbol{p}_i(t) - \frac{\epsilon}{2}\frac{\partial U}{\partial \boldsymbol{x}_i}\left(\boldsymbol{x}(t)\right) && \longleftarrow \text{ half step} \\ \boldsymbol{x}_i(t+\epsilon) &= \boldsymbol{x}_i(t) + \epsilon \boldsymbol{p}_i(t+\epsilon/2)/\mathcal{M}_i && \longleftarrow \text{ use intermediate } \boldsymbol{p} \\ \boldsymbol{p}_i(t+\epsilon) &= \boldsymbol{p}_i(t+\epsilon/2) - \frac{\epsilon}{2}\frac{\partial U}{\partial \boldsymbol{x}_i}\left(\boldsymbol{x}(t+\epsilon)\right) && \longleftarrow \text{ other half step}\end{aligned}$$

Also see Figures 1, and 3-6 in "MCMC using Hamiltonian dynamics" by Radford Neal for further illustrations.

Algorithm 22.1 (The Hamiltonian Monte Carlo method)

HMC can be used to sample from continuous distributions pn $\mathbb{R}^d$ for which both the density and the partial derivatives of the log of the density can be evaluated.

1. In the first step, new values for the momentum variables $\boldsymbol{p}$ are randomly drawn from their Gaussian distribution (independently of the current values of the position variables). The result is a state $(\boldsymbol{x}_i, \boldsymbol{p}_i)$. If M is diagonal, $\boldsymbol{p}_i$ will have mean zero and variance $\mathcal{M}_{ii}$. $\boldsymbol{x}_i$ is changed and $\boldsymbol{p}_i$ is from the correct conditional distribution given $\boldsymbol{x}$, so the canonical joint distribution is invariant.
2. In the second step, a Metropolis update is performed, simulating Hamiltonian dynamics to propose a new state.
    - Starting at the current state $(\boldsymbol{x}_i, \boldsymbol{p}_i)$, Hamilton's equations are integrated for L timesteps of length ε using the leapfrog method (or some other reversible method that preserves volume).

- The momentum variable at the end of this L-step trajectory are then negated, giving a proposed state $(\boldsymbol{x}_i^*, \boldsymbol{p}_i^*)$. The negation is is needed to make the Metropolis proposal symmetric, but does not matter in practice since $K(-\boldsymbol{p}) = K(\boldsymbol{p})$ and the momentum will be replaced before it is used again.
- The proposed state is accepted as the new state $(\boldsymbol{x}_{i+1}, \boldsymbol{p}_{i+1})$ with probability

$$\min\left(1, -\exp(-H(\boldsymbol{x}_i^*, \boldsymbol{p}_i^*)) + H\exp(-H(\boldsymbol{x}_i, \boldsymbol{p}_i))\right).$$

  Otherwise, the new state is a copy of the current state. Note that the probability distribution for $(\boldsymbol{x}, \boldsymbol{p})$ *jointly* is (almost) unchanged because energy is conserved, but in terms of $\boldsymbol{x}$ we get a very different probability density.

For successful application of HMC there are three hyperparameters that need to be tuned by the user:

1. The mass matrix $\mathcal{M}$;
2. The integration timestep ε;
3. The number of timesteps L.

The MontePython implementation of HMC, by Isak Svensson, was published in Ref. [SEkstromForssen22] and is publicly available at: svisak/montepython.git

More advanced versions, such as the No-U-Turn Samplers (NUTS) [HG+14], are also available and aim to simplify the process of tuning the hyperparameters.

22.4. Sampling / Importance Resampling

Note

This chapter is reproduced (with some adjustments) from *Bayesian probability updates using sampling/importance resampling: Applications in nuclear theory* by Weiguang Jiang and Christian Forssén in Front. in Phys., 10:1058809, 2022 [JForssen22] (with permission).

The use of MCMC in the statistical analysis of complex computer models typically requires massive computations. There are certainly situations where MCMC sampling is not ideal, or even becomes infeasible:

1. When the posterior is conditioned on some calibration data for which the model evaluations are very costly. Then a limited number of full likelihood evaluations can be afforded and the MCMC sampling becomes less likely to converge.
2. Bayesian posterior updates in which calibration data is added in several different stages. This typically requires that the MCMC sampling must be carried out repeatedly from scratch.
3. Model checking where the sensitivity to prior assignments is being explored. This is a second example of posterior updating.
4. The prediction of target observables for which model evaluations become very costly and the handling of a large number of MCMC samples becomes infeasible.

These are situations where one might want to use the Sampling/Importance Resampling (S/IR) method [BS06, SG92], which can exploit the previous results of model evaluations to allow posterior probability updates at a much smaller computational cost compared to the full MCMC method.

Brief review of the method

The basic idea of S/IR is to utilize the inherent duality between samples and the density (probability distribution) from which they were generated [SG92]. This duality offers an opportunity to indirectly recreate a density (that might be hard to compute) from samples that are easy to obtain.

Let us consider a target density $h(\boldsymbol{\theta})$. This could be a posterior PDF $p\left(\boldsymbol{\theta} \mid \mathcal{D}, I\right)$. Instead of attempting to directly collect samples from $h(\boldsymbol{\theta})$, as would be the goal in MCMC approaches, the S/IR method uses a detour. We first obtain samples from a simple (even analytic) density $g(\boldsymbol{\theta})$. We then resample from this finite set using a resampling algorithm to approximately recreate samples from the target density $h(\boldsymbol{\theta})$. There are (at least) two different resampling methods. In this paper we only focus on one of them called weighted bootstrap (more details of resampling methods can be found in Refs. [Rub88, SG92]).

Assuming we are interested in the target density $h(\boldsymbol{\theta}) = f(\boldsymbol{\theta}) \,/\, \int f(\boldsymbol{\theta})\, \mathrm{d}\boldsymbol{\theta}$, the procedure of resampling via weighted bootstrap can be summarized as follows:

Algorithm 22.2 (The Sampling/Importance Resampling method)

1. Generate the set $\{\boldsymbol{\theta}_i\}_{i=1}^n$ of samples from a sampling density $g(\boldsymbol{\theta})$.
2. Calculate $\omega_i = f(\boldsymbol{\theta}_i) \,/\, g(\boldsymbol{\theta}_i)$ for the n samples and define importance weights as: $q_i = \omega_i \,/\, \sum_{j=1}^n \omega_j$.
3. Draw N new samples $\{\boldsymbol{\theta}_i^*\}_{i=1}^N$ from the discrete distribution $\{\boldsymbol{\theta}_i\}_{i=1}^n$ with probability mass q_i on $\boldsymbol{\theta}_i$.
4. The set of samples $\{\boldsymbol{\theta}_i^*\}_{i=1}^N$ will then be approximately distributed according to the target density $h(\boldsymbol{\theta})$.

Intuitively, the distribution of $\boldsymbol{\theta}^*$ should be good approximation of $h(\boldsymbol{\theta})$ when n is large enough. Here we justify this claim via the cumulative distribution function of θ^* (for the one-dimensional case)

$$\begin{aligned} \mathrm{pr}(\theta^* \leq \mathrm{a}) = \sum_{i=1}^n q_i \cdot H(a-\theta_i) = \frac{\frac{1}{n}\sum_{i=1}^n \omega_i \cdot H(a-\theta_i)}{\frac{1}{n}\sum_{i=1}^n \omega_i} \\ \xrightarrow[n\to\infty]{} \frac{\mathbb{E}_g\left[\frac{f(\theta)}{g(\theta)} \cdot H(a-\theta)\right]}{\mathbb{E}_g\left[\frac{f(\theta)}{g(\theta)}\right]} = \frac{\int_{-\infty}^a f(\theta)\, d\theta}{\int_{-\infty}^{\infty} f(\theta)\, d\theta} = \int_{-\infty}^a h(\theta)\, d\theta, \end{aligned} \tag{22.6}$$

with $\mathbb{E}_g[X(\theta)] = \int_{-\infty}^{\infty} X(\theta) g(\theta)\, d\theta$ the expectation value of $X(\theta)$ with respect to $g(\theta)$, and H Heaviside step function such that

$$H(a-\theta) = \begin{cases} 1 & \text{if } \theta \leq a, \\ 0 & \text{if } \theta > a. \end{cases} \tag{22.7}$$

The above resampling method can be applied to generate samples from the posterior PDF $h(\boldsymbol{\theta}) = \mathrm{pr}(\boldsymbol{\theta}|\mathcal{D})$ in a Bayesian analysis. It remains to choose a sampling distribution, $g(\boldsymbol{\theta})$, which in principle could be any continuous density distribution. However, recall that $h(\boldsymbol{\theta})$ can be expressed in terms of an unnormalized distribution $f(\boldsymbol{\theta})$, and using Bayes' theorem we can set $f(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) p\left(\boldsymbol{\theta} \mid I\right)$. Thus, choosing the prior $p\left(\boldsymbol{\theta} \mid I\right)$ as the sampling distribution $g(\boldsymbol{\theta})$ we find that the importance weights are expressed in terms of the likelihood, $q_i = \mathcal{L}(\boldsymbol{\theta}_i) / \sum_{j=1}^n \mathcal{L}(\boldsymbol{\theta}_j)$. Assuming that it is simple to collect samples from the prior, the costly operation will be

the evaluation of $\mathcal{L}(\boldsymbol{\theta}_i)$. Here we make the side remark that an effective and computationally cost-saving approximation can be made if we manage to perform a pre-screening to identify (and ignore) samples that will give a very small importance weight. We also note that the above choice of $g(\boldsymbol{\theta}) = \mathrm{pr}(\boldsymbol{\theta})$ is purely for simplicity and one can perform importance resampling with any $g(\boldsymbol{\theta})$.

Example 22.1 (Illustration of S/IR)

Let us follow the above procedure in a simple example of S/IR to illustrate how to get samples from a posterior distribution. We consider a two-dimensional parametric model with $\boldsymbol{\theta} = (\theta_1, \theta_2)$. Given data $\mathcal{D}$ obtained under the model we have:

$$p\left(\theta_1, \theta_2 \mid \mathcal{D}\right) = \frac{\mathcal{L}(\theta_1, \theta_2) p\left(\theta_1, \theta_2 \mid I\right)}{\iint \mathcal{L}(\theta_1, \theta_2) p\left(\theta_1, \theta_2 \mid I\right) d\theta_1 d\theta_2}. \tag{22.8}$$

For simplicity and illustration, the joint prior distribution for θ_1, θ_2 is set to be uniform over the unit square as shown in Fig. 22.1a. In this example we also assume that the data $\mathcal{D}$ likelihood is described by a multivariate Student-t distribution

$$\mathcal{L}(\theta_1, \theta_2) = \frac{\Gamma[(\nu + p)/2]}{\Gamma(\nu/2)\nu^{p/2}\pi^{p/2}|\boldsymbol{\Sigma}|^{1/2}} \left[1 + \frac{1}{\nu}(\boldsymbol{\theta} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\boldsymbol{\theta} - \boldsymbol{\mu})\right]^{-(\mu+p)/2}, \tag{22.9}$$

where the dimension $p = 2$, the degrees of freedom $\nu = 2$, the mean vector $\boldsymbol{\mu} = (0.2, 0.5)$ and the scale matrix $\boldsymbol{\Sigma} = [[0.02, 0.005], [0.005, 0.02]]$.

The importance weights q_i are then computed for $n = 2000$ samples drawn from the prior (these prior samples are shown in Fig. 22.1a). The resulting histogram of importance weights is shown in Fig. 22.1b. Here the weights have been rescaled as $\tilde{q}_i = q_i / \max(\{q\})$ such that the sample with the largest probability mass corresponds to 1 in the histogram. We also define the effective number of samples, n_{eff}, as the sum of rescaled importance weights, $n_{\mathrm{eff}} = \sum_{i=1}^{n} \tilde{q}_i$. Finally, in Fig. 22.1c we show $N = 20,000$ new samples $\{\boldsymbol{\theta}_i^*\}_{i=1}^N$ that are drawn from the prior samples $\{\boldsymbol{\theta}_i\}_{i=1}^n$ according to the probability mass q_i for each $\boldsymbol{\theta}_i$. The blue and green contour lines represent (68% and 90%) credible regions for the resampled distribution and for the Student-t distribution, respectively. This result demonstrates that the samples generated by the S/IR method give a very good approximation of the target posterior distribution.

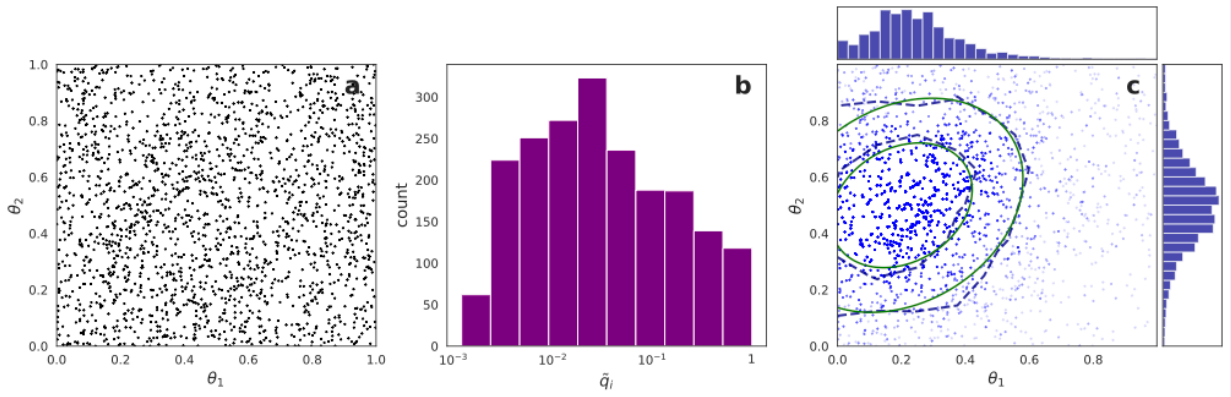


Fig. 22.1 Illustration of S/IR procedures. a. Samples $\{\boldsymbol{\theta}\}_{i=1}^{n}$ from a uniform prior in a unit square ($n = 2000$ samples are shown). b. Histogram of rescaled importance weights $\tilde{q}_i = q_i / \max(\{q\})$ where $q_i = \mathcal{L}(\boldsymbol{\theta}_i) / \sum_{j=1}^{n} \mathcal{L}(\boldsymbol{\theta}_j)$ with $\mathcal{L}(\boldsymbol{\theta})$ as in Eq. (22.9). The number of effective samples is $n_{\text{eff}} = 214.6$. Note that the samples are drawn from a unit square and that the tail of the target distribution is not covered. c. Samples $\{\boldsymbol{\theta}^*\}_{i=1}^{N}$ of the posterior (blue dots with 10% opacity) resampled from the prior samples with probability mass q_i. The contour lines for the 68% and 90% credible regions of the posterior samples (blue dashed) are shown and compared with those of the exact bivariate target distribution (green solid). Summary histograms of the marginal distributions for θ_1 and θ_2 are shown in the top and right subplots.

S/IR limitations

While the S/IR approach might provide a useful alternative to full MCMC sampling, there are some important limitations. In particular, users should be mindful of the effective number of samples. The more complex the likelihood function, the less effective is the use of a sampling distribution.

Unfortunately one can also envision more difficult scenarios in which S/IR could fail without any clear signatures. For example, if the prior has a very small overlap with the posterior there is a risk that many prior samples get a similar importance weight (such that the number of effective samples is large) but that one has missed the most interesting region. Prior knowledge of the posterior landscape is very useful to avoid possible failure scenarios that might not be signaled by the number of effective samples.

22.5. Slice sampling

The idea of slice sampling is well documented in two arXiv references for the ensemble slice sampler `zeus` (which is a particular implementation):

1. *Ensemble Slice Sampling: Parallel, black-box and gradient-free inference for correlated & multimodal distributions* by Minas Karamanis and Florian Beutler.
2. *zeus: A Python implementation of Ensemble Slice Sampling for efficient Bayesian parameter inference* by Minas Karamanis, Florian Beutler, and John A. Peacock.

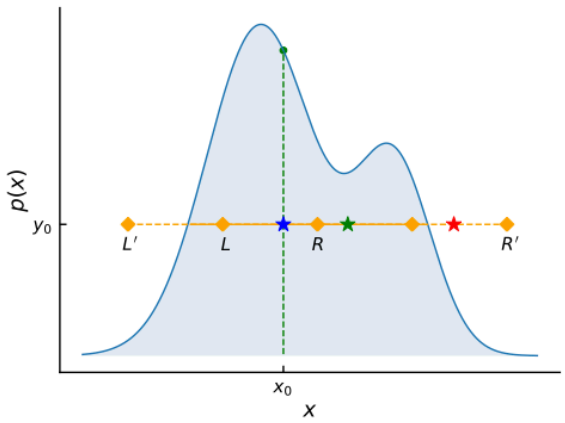


Fig. 1 The plot shows the univariate slice sampling method. Given an initial value x_0, a value y_0 is uniformly sampled along the vertical slice $(0, f(x_0))$ (green dashed line) thus defining the initial point (blue star). An interval (L, R) is randomly positioned horizontally around the initial point, and then it is expanded in steps of size $\mu = R - L$ until both of its ends L', R' are outside the slice. The new point (green star) is generated by repeatedly sampling uniformly from the expanded interval (L', R') until a point is found inside the slice. Points outside the slice (e.g. the red star) are used to shrink the interval (L', R') by moving L' or in this case R' to that point and accelerate the sampling procedure.

Fig. 22.2 Figure 1 from Ref. 1 above.

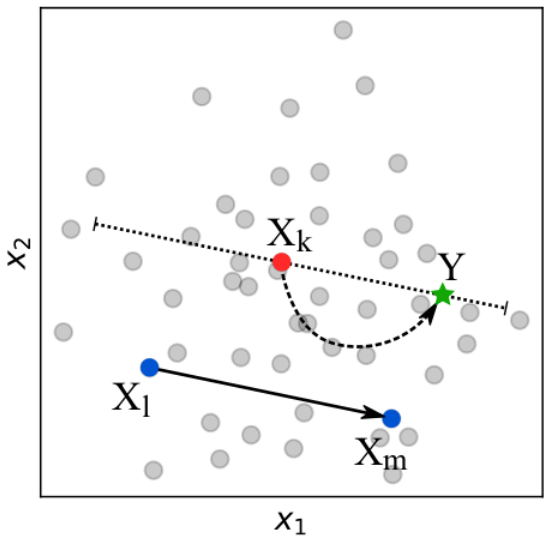


Figure 2. The figure illustrates the differential move in the context of Ensemble Slice Sampling. The walker X_k to be updated is shown in red. Two walkers, X_l and X_m, (blue) are uniformly selected from the complementary ensemble (grey). The approximate slice (dotted line) is constructed parallel to the two walkers X_l and X_m using the stepping-out procedure. The new position Y (green) of X_k is sampled using the shrinking procedure along the approximate slice.

Fig. 22.3 Figure 2 from Ref. 2 above.

Slice sampling in action for a one-dimensional distribution is shown in Fig. 22.2. The idea is that sampling from a distribution $p(x)$ is the same as uniform sampling from the *area* under the plot of $f(x) \propto p(x)$. E.g., the highest probability $p(x)$ is where the area is largest, and this is a simple proportionality. The height y is introduced with $0 < y < f(x)$ as an auxiliary variable and one samples the uniform joint probability $p(x, y)$. Then $p(x)$ is obtained by marginalizing over y (meaning just dropping y from the sample).

The blue star comes from uniformly sampling in y at the initial point x_0 to identify y_0. The "slice" is the set of x such that $y_0 < f(x)$. In the figure the interval initially from L to R is expanded to L' to R', which includes the full slice. When the final interval is found, a new x_1 is drawn uniformly from the intersection of the interval and the slice. The green star is an accepted new point x_1 (inside the slice) while the red star is a rejected point (outside the slice). There is one parameter to tune: the width μ of the interval from L to R that sets the original interval around x_0 and then the step size for expanding the interval to be from L' to R'. In `zeus` the value of μ is tuned by stochastic optimization.

The "ensemble" aspect of `zeus` is that there are multiple walkers, as in the ensemble sampler `emcee`. Fig. 22.3 illustrates how the ensemble of walkers is used to do what is called a "differential move". There are several choices for moves, including a global one that is effective for multi-modal distributions.

- A text case for parallel tempering (using `ptemcee`) vs. slice sampling (using `zeus`) in a very simple multi-modal example is in Demo: Multimodal distributions with the ptemcee and zeus samplers.
- The basic call for `zeus` compared to `emcee` and `PyMC` is illustrated in Comparing samplers for a simple problem.

22.6. Nested sampling

Prelude

Nested sampling (or NS) is a technique invented by John Skilling to estimate the Bayesian *evidence* (and we get a sampling of the posterior as a by-product, see below). Let us recall the ingredients of Bayes' theorem:

$$\overbrace{p\left(\boldsymbol{\theta} \mid \mathcal{D}, I\right)}^{\text{posterior}} = \frac{\overbrace{p\left(\mathcal{D} \mid \boldsymbol{\theta}, I\right)}^{\text{likelihood}} \times \overbrace{p\left(\boldsymbol{\theta} \mid I\right)}^{\text{prior}}}{\underbrace{p\left(\mathcal{D} \mid I\right)}_{\text{evidence}}} \equiv \frac{\mathcal{L}(\boldsymbol{\theta})\pi(\boldsymbol{\theta})}{\mathcal{Z}} \equiv p(\boldsymbol{\theta}),$$

where we have introduced a condensed notation for clarity in the following discussion. We can write $\mathcal{Z}$, the evidence (or marginal likelihood or normalizing constant), as

$$\mathcal{Z} = \int \mathcal{L}(\boldsymbol{\theta})\pi(\boldsymbol{\theta})\, d\boldsymbol{\theta}, \tag{22.10}$$

so, as expressed here, it is an integral in the often high-dimensional space of $\boldsymbol{\theta}$. You can think of the posterior as the shape of the integrand of this integral while $\mathcal{Z}$ is the magnitude of the integral. The usual MCMC sampling techniques based on importance sampling are designed to estimate the relative shape in terms of a representative sampling of points $\boldsymbol{\theta}_i$ found via a Markov chain. But finding the absolute normalization can be a much greater challenge.

To gain intuition about NS, first recall the standard integration method as a Riemannian sum: we divide space into small volumes (e.g., multi-dimensional cubes) and approximate the integral as a sum over the integrand evaluated in each volume times the volume (see the left panel of Fig. 22.4). The cost of such an integral grows exponentially with the dimension of the integral ("the curse of dimensionality"), which is why we turned to Monte Carlo integration in the first place. To meet this challenge a different way, NS transforms the problem into finding a statistical estimate of a *one-dimensional* integral.

Good references for NS include:

- [SS06] D.S. Sivia with J. Skilling, *"Data Analysis : A Bayesian Tutorial"*, Oxford University Press (2006).
- [A+22] G. Ashton et al., *"Nested sampling for physical scientists"*, Nature (2022).
- [Buc23] J. Buchner, *"Nested sampling methods"*, Statistical Surveys (2023).
- [ML26] L. Martino and L. Fernando, *"Nested Sampling: A Critical and Comprehensive Theoretical Guide"*, arXiv:2606.17916 (2026).
- See also method discussions in the documentation of NS implementations, such as dynesty and Ultranest.

Basic idea of NS

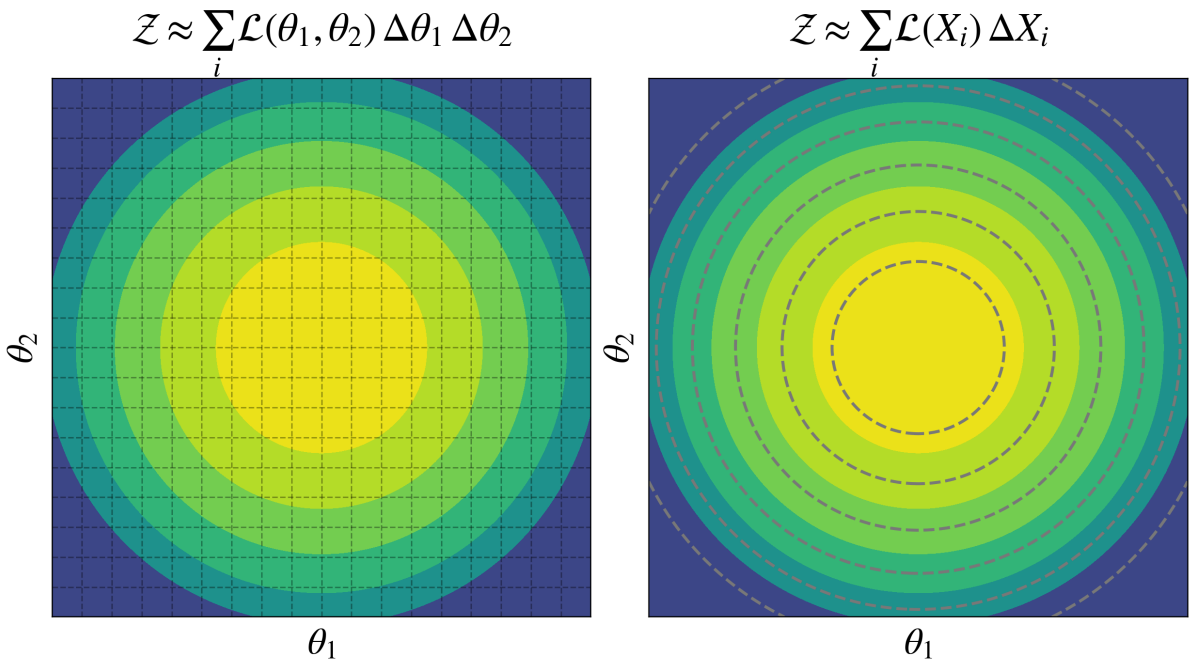


Fig. 22.4 On the left is a schematic for evaluating the evidence integral by summing over uniform small volumes in the multi-dimensional parameter space (here the two-dimensional area is $\Delta\theta_1\Delta\theta_2$). On the right is a schematic for evaluating the same integral but now by grouping all points with the same likelihood value into small volumes ΔX_i and summing over $\mathcal{L}(X_i)\Delta X_i$, which approximates (22.10).

A key observation leading to NS is that we can use other shapes beside cubes to estimate an integral, if volume elements are such that $p(\boldsymbol{\theta})$ is almost constant for any shape. In particular, we can group together into small volumes all points with the same value of the likelihood, as in the right panel of Fig. 22.4. To get to an expression taking advantage of this observation, we first split the evidence integral into an integration over likelihood values $\mathcal{L}^\star$, where for each $\mathcal{L}^\star$ we have an integral over the enclosed part of parameter space weighted by the prior:

$$\mathcal{Z} = \int^{+\infty} \left[\int_{\boldsymbol{\theta}:\mathcal{L}(\boldsymbol{\theta})>\mathcal{L}^\star} \pi(\boldsymbol{\theta})\, d\boldsymbol{\theta} \right] d\mathcal{L}^\star.$$

Here the integral in square brackets is the volume variable X, defined by

$$X(\mathcal{L}^\star) \equiv \int_{\boldsymbol{\theta}:\mathcal{L}(\boldsymbol{\theta})>\mathcal{L}^\star} \pi(\boldsymbol{\theta})\, d\boldsymbol{\theta}, \tag{22.11}$$

which is the prior-constrained volume enclosed by contour $\mathcal{L}^\star = \mathcal{L}(\boldsymbol{\theta})$ (which may consist of disconnected regions).

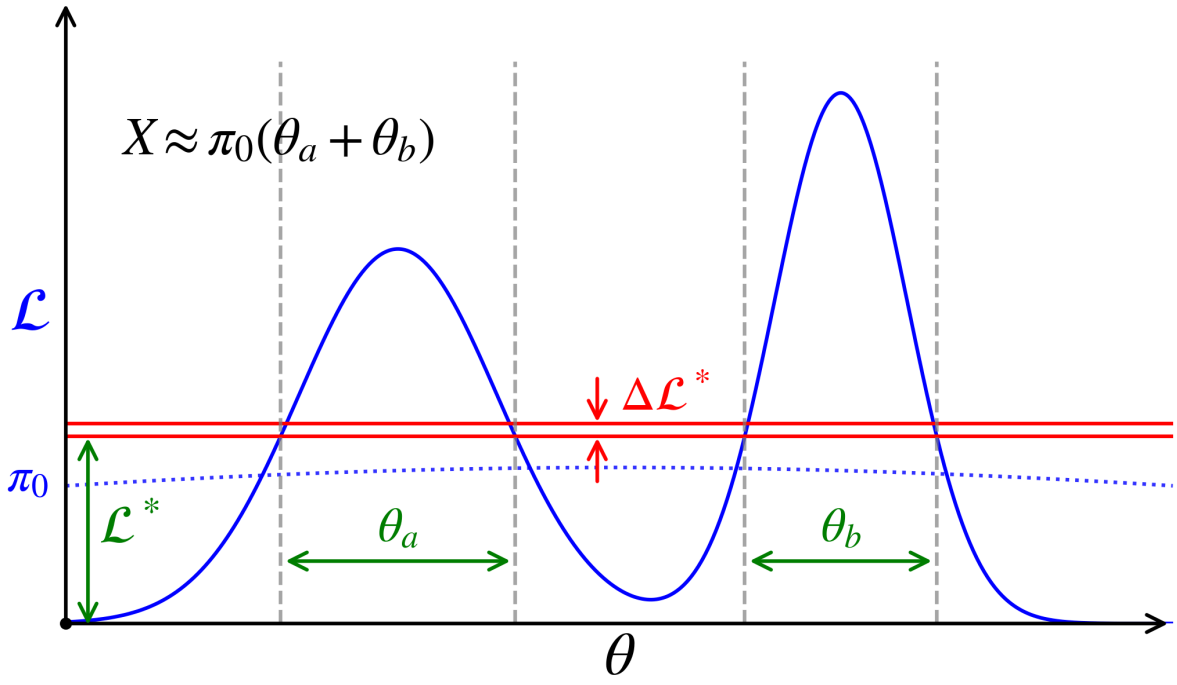


Fig. 22.5 Schematic example showing how the prior-weighted scalar volume X is determined at a given value of $\mathcal{L}^\star$. The vertical dashed lines indicate the limits in θ for which $\mathcal{L}(\boldsymbol{\theta}) > \mathcal{L}^\star$; in one dimensions these define the lengths θ_a and θ_b. The prior $\pi(\boldsymbol{\theta})$ is approximately constant (equal to π_0) in those regions, so the integral defining X is approximately $X \approx \pi_0(\theta_a + \theta_b)$. This piece contributes to the evidence integral as $X\Delta\mathcal{L}^\star$.

In Fig. 22.5 we show a one-dimensional example of how this works. As we step through each value of $\mathcal{L}^\star$ in the outer integral, this defines a region in θ where the likelihood satisfies $\mathcal{L}(\theta) > \mathcal{L}^\star$. In the example, the current region, which is disconnected, is defined by lengths θ_a and θ_b. In one dimension, the internal integration over θ for an approximately constant $\pi(\theta) \approx \pi_0$ yields $X \approx \pi_0(\theta_a + \theta_b)$. In two dimensions, this region would be the area defined by a slice at height $\mathcal{L}^\star$, in three dimensions it would be a 3D volume, and so on. Note that there is no problem with the likelihood being multimodal. Note also that X is monotonic in $\mathcal{L}^\star$ (this may be easiest to see if you visualize a line at $\mathcal{L}^\star$ starting well above the peak and being moved downward; then X always increases, even if $\pi(\boldsymbol{\theta})$ is not constant). We multiply X by the width $\Delta\mathcal{L}^\star$ to get the contribution to the evidence, and sum this up for all $\mathcal{L}^\star$ to get the total evidence.

When the threshold $\mathcal{L}^\star$ is zero, $X = 1$ (the entire prior space is included, and the prior is normalized). As the threshold increases toward the peak value, the allowed space shrinks, and X approaches zero. Because X is monotonic and decreases smoothly from 1 to 0, we can invert $X(\mathcal{L}^\star)$ to find $\mathcal{L}^\star(X)$. Integrating the evidence integral by parts yields the NS evidence identity,

$$\mathcal{Z} = \int^{+\infty} X(\mathcal{L}^\star)\, d\mathcal{L}^\star = \int_0^1 \mathcal{L}^\star(X)\, dX,$$

assuming $\mathcal{L}^\star(X)$, the inverse of $X(\mathcal{L}^\star)$, exists and $\mathcal{Z}$ is finite. If we can evaluate the iso-likelihood contour $\mathcal{L}_i \equiv \mathcal{L}(X_i)$ (dropping the $\star$) associated with samples from the prior volumes

$$1 > X_1 > X_2 > \cdots > X_N > 0,$$

then we can compute the evidence $\mathcal{Z} = \int_0^1 \mathcal{L}(X)\, dX \approx \sum_i \mathcal{L}_i \Delta X_i$ by basic numerical quadrature (e.g., the trapezoid rule). Computing the evidence using these "nested shells" is what gives Nested Sampling its name. The NS procedure is illustrated schematically in Fig. 22.6.

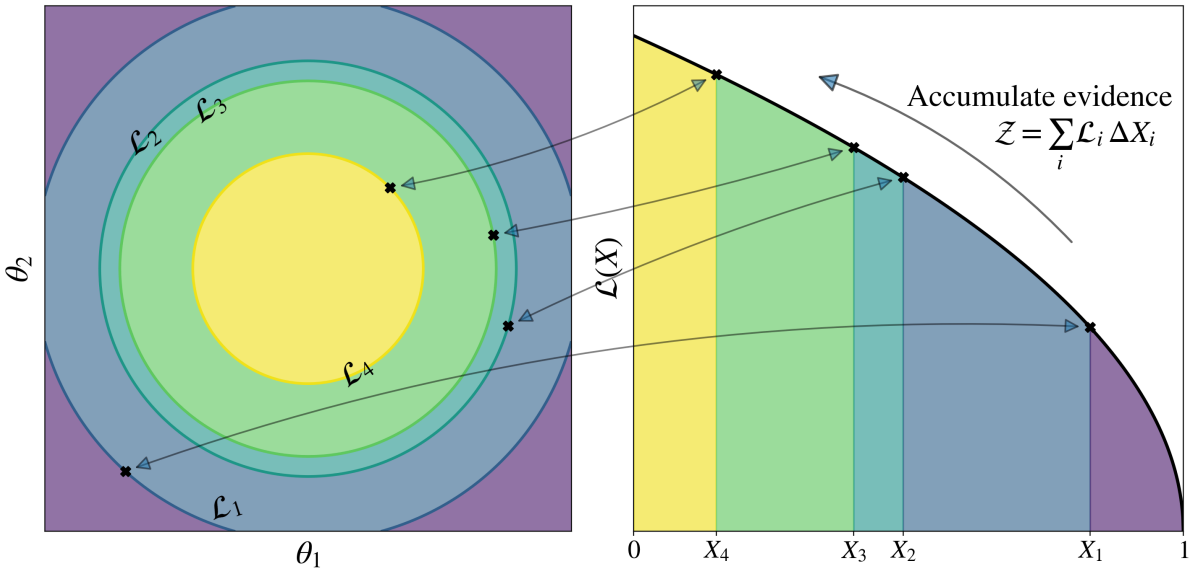


Fig. 22.6 Schematic representation of NS for a two dimensional problem. The samples X_i from the prior volumes at $\mathcal{L}_i$ satisfy $1 > X_1 > X_2 > X_3 > X_4 > 0$. The corresponding ΔX_is are multiplied by the $\mathcal{L}_i$s and summed to approximate the evidence: $\mathcal{Z} = \int_0^1 \mathcal{L}(X)\, dX \approx \sum_i \mathcal{L}_i \Delta X_i$.

This sounds great, but we realize that calculating the X_is directly involves a series of high-dimensional integrals that are exactly what we want to avoid! So we will estimate the X_is statistically.

NS in practice

In an implementation of NS one evolves a collection of so-called "live points" through the parameter space. We start with $n_{\rm live}$ points drawn from the prior. The prior should cover the full range of parameters being sampled. The likelihood is calculated for all of the live points, and the least likely (i.e., lowest value of $\mathcal{L}$) is removed and a new draw is made to replace the "dead" point. This lowest value is $\mathcal{L}^\star$. (We keep track of the dead points, which give an estimate of the posterior, see below).

Suppose we could (magically) know the prior volume X_i from (22.11) associated with each live point θ_i. If the current volume is X_{i-1}, then the variables $u_j = X_j/X_{i-1}$ would all be uniformly distributed in $[0, 1]$.

Checkpoint question

Intuitively, why is u_j uniformly distributed in $[0, 1]$?

Answer

Imagine dividing the prior into nested likelihood regions that have prior volumes $X = 0.1, 0.2, 0.3, \ldots, 1$. Then the region with $X(\theta) \equiv X(\mathcal{L}(\theta)) < 0.2$ has, by construction, 20% of the prior probability. So for a random draw of θ, the cumulative probability of the corresponding $X < 0.2$ is 0.2. And, generally, the cumulative probability for $X < x$ is x. This is the cumulative probability distribution for a uniform distribution of X.

If we have already eliminated points so that the current volume is X_{i-1}, then the uniform distribution is between 0 and X_{i-1}, so to make it uniform in $[0, 1]$ we define u_j with X_{i-1} divided out.

This result is independent of the details of the likelihood or the prior, because X has been defined in terms of enclosed prior probability, so it automatically turns prior draws into uniform random variables.

Removing the point with the lowest likelihood means removing the one with the highest X, so

$$t_i \equiv \frac{X_i}{X_{i-1}} = \max(u_1, \ldots, u_{n_{\text{live}}}),$$

which means (because the u_is are independent):

$$F(t_i < t) = t^{n_{\text{live}}} \quad \Longrightarrow \quad p(t) = n_{\text{live}} t^{n_{\text{live}}-1} \quad \Longrightarrow \quad t_i \sim \text{Beta}\ (n_{\text{live}}, 1),$$

where F is the cumulative distribution and $p(t)$ therefore follows as its derivative.

Checkpoint question

Why is this a Beta distribution?

Answer

The general Beta pdf is

$$p(t) = \frac{1}{B(a,b)} t^{a-1}(1-t)^{b-1}, \quad \text{where} \quad B(a,b) = \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)},$$

and $\Gamma(z)$ is the gamma function. Here $a = n_{\text{live}}$ and $b = 1$ to match $p(t) = n_{\text{live}} t^{n_{\text{live}}-1}$ and $B(n_{\text{live}}, 1) = 1/n_{\text{live}}$.

Checkpoint question

Why is t so close to 1? Does this make sense intuitively?

Hint

Imagine you are determining t from 50 points uniformly distributed between 0 and 1.

Answer

Intuitively it makes sense: with 50 points between 0 and 1 and you are taking the largest one, you expect it to be close to 1.

Mathematically, with $n_{\text{live}} = 50$, the density is $p(t) = 50t^{50}$ and any number not close to 1 raised to an exponent of 50 will be close to zero, so the strength is concentrated near 1.

This is the statistical shrinkage rule. So while there really is a sequence of X_is, they are irregular and random because the live points themselves were randomly drawn. The NS algorithm doesn't know these while it does know $\mathcal{L}_i$ (as noted above, calculating the X_i would be a difficult multidimensional integral of the type we are trying to avoid). So, in fact, representative values are used in practice:

$$X_i \approx e^{-i/n_{\text{live}}} \quad \text{or} \quad X_i = t_i X_{i-1}, \text{ with } t_i \sim \text{Beta } (n_{\text{live}}, 1).$$

The process is repeated over many iterations and the evidence accumulated from

$$\mathcal{Z} \approx \sum_i \mathcal{L}_i \cdot \Delta X_i,$$

where $\Delta X_i = X_{i-1} - X_i$. The iterations are stopped when the remaining evidence is negligible (various stopping criterion are used in different implementations of NS).

As a by-product, posteriors for $\boldsymbol{\theta}$ are acquired "for free" from the same dead points by assigning each sample its associated importance weight

$$\mathbb{P}(\boldsymbol{\theta}_i) = p(X_i) \approx \frac{\mathcal{L}_i \Delta X_i}{\mathcal{Z}}.$$

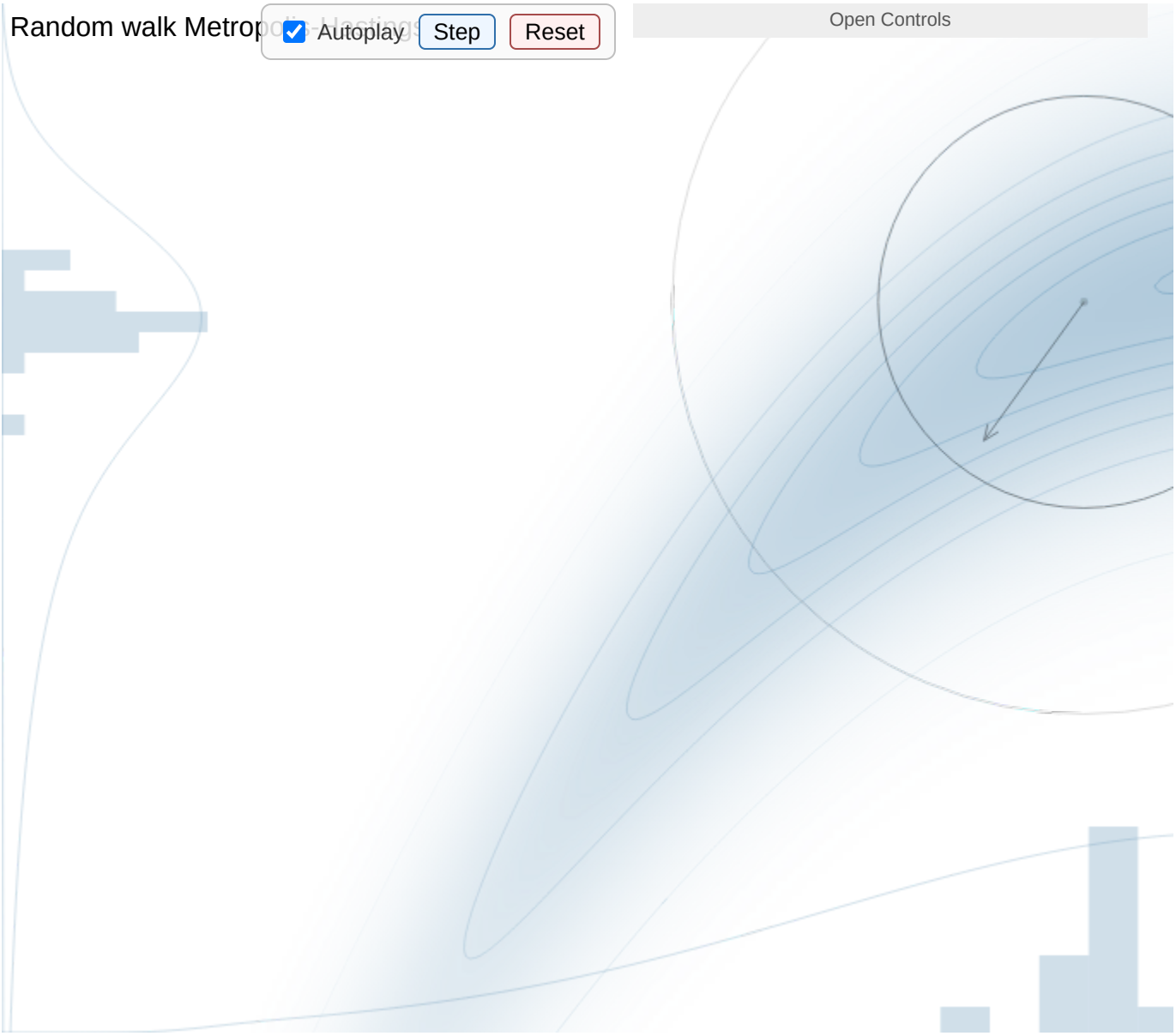
To determine replacement live points, there are different strategies used by different NS implementations. For example, to do region sampling, a geometric shape (e.g., an ellipse) is constructed around the live points and a new point is drawn from within the region, rejecting it if it falls outside the region. Below we examine in detail one such implementation.

Simulation of nested sampling

To establish some intuition about NS in practice, we return to the excellent set of interactive demos by Chi Feng. You might want to refresh your memory first on MH and HMC sampling before jumping into the nested sampling demo.

Controls for the MCMC simulations

- The initial simulation uses MH sampling. To switch to an implementation of nested sampling, select `Open Controls` and pull down the `Algorithm` menu and select `RadFriends-NS`. Note the `Algorithm Options`, where `numLivePoints` $= n_{\text{live}}$ is set.
- After making changes, use `Close Controls` to avoid obscuring the simulation.
- The initial `Target distribution` should be `banana`, which is a good choice to demonstrate the nested sampling algorithm.
- But for orientation, you might want to first switch the `Target distribution` to `standard`. This distribution is a two-dimensional Gaussian (just the product of two one-dimensional Gaussians).
- If you uncheck the `Autoplay` box, you can use the `Step` button to see the algorithm carried out one step at a time.
- Use the `Reset` button to clear the live points and start again with the full prior.



In standard MCMC (like Random Walk or HMC), the histograms accumulate sequentially. Every time a single particle takes a step, the algorithm records its coordinate and adds exactly one tally mark to the corresponding bin. Nested Sampling, however, does not follow a single particle. It tracks an entire population of active (live) points simultaneously, weighting their contributions based on the prior volume they represent.

- Weighting by Posterior Mass: Because Nested Sampling's goal is to compute the total integral ($\mathcal{Z}$), every point that gets killed/replaced carries a specific "weight" ($w_i = L_i \cdot \Delta X_i$), which represents how much total probability mass that point occupies.
- The Histogram Addition: In the visualization, the histograms are built by accumulating these weights. When a point is eliminated, its position along the X and Y axes is recorded into the histograms, multiplied by the fractional volume it leaves behind.

Notes for the `RadFriends-NS` simultation:

1. Initially, points are drawn from the entire parameter space (green) weighted by the prior. At any time, the number of points will match the setting of `numLivePoints`. You might want to first set it to a low number (10 is the least) and switch off autoplay so that you count each of the dots and watch what happens step by step.

2. In each step, the live point with the lowest likelihood (worst fit) is removed, and a better one sought. At each removal, the volume sampled by the live points shrinks. If you go step-by-step (`Autoplay` unchecked) with a small number of points, you can identify by eye the point with the lowest likelihood (it is easiest to start with the `standard` target distribution). When you take a step it will disappear. The green arrow visualizes the birth of a new, valid replacement point; it points from a randomly chosen live point (different from the one about to die) to the sampled next point, which will be added to the set of live points. There are red points to indicate potential next points that do not have a higher likelihood, and therefore are rejected as candidate live points.
3. To still sample from the prior, RadFriends creates ellipsoids around all live points (not just the one being removed) and samples from them. The ellipsoid size is determined by bootstrapping: Some points are randomly left out, and the ellipsoids have to be large enough so that they could have been sampled. This is repeated several times. In Ultranest, the ellipsoid shape is learnt as well. Other NS algorithms have other ways to do the sampling.
4. Nested sampling proceeds to the peak, keeping track of the likelihood. The volume becomes smaller and smaller. At some point, the remainder does not contribute any probability mass, and the exploration is finished.
5. The removed points are weighted by their likelihood and the volume they represent. These are the posterior samples (histograms).

Why do so many points appear initially in the histograms?

The instant appearance of many bins on Step 1 of the Rad-Friends-NS simulation is the result of a specific programmatic sequence in the rendering engine. Before the simulation officially begins iterating, the code checks the likelihood of all 40 randomly scattered points (assuming you are using the default $n_{\text{live}} = 40$. The visualization code treats this entire initial batch of 40 points as the starting baseline. Because a histogram cannot show "live" floating points (it can only record permanent data data-points), the code takes all n_{live} initial locations and puts them into the histogram array simultaneously as a single initialization event. You are seeing the birth of the entire population rendering all at once, not a single particle moving. Thus, the code pushes the coordinates of all n_{live} points into the histogram bin counters in a single clock cycle. If you have 40 points, up to 40 different bins across the X and Y axes instantly receive a value.

Once that initial batch of n_{live} points is registered into the visualization framework, the simulation settles into its actual loop:

- Step 2: One single worst point is removed. One new green arrow finds one replacement point. Therefore, only one new coordinate is added to the histogram.
- Step 3: Only one point is replaced. Only one contribution is added to a single bin.
- The massive burst of bins is a one-time initialization dump required to set up the population ecosystem on screen. Every step after that will only modify a single point at a time, making the progress look slow and incremental from that moment forward.

The Life Cycle of a Green Arrow in RadFriends-NS

- **The Replacement Goal:** The algorithm identifies the "worst" active point in the entire population (the point with the lowest likelihood density) and marks it to be destroyed.
- **The Originating End (The Tail):** To find a replacement, the algorithm randomly selects one of the other, surviving active points to act as a parent. The tail of the green arrow sits exactly on this surviving parent point.
- **The "Friend" Ball:** The algorithm draws a radius (the "Rad" in RadFriends) around that parent point to define a local neighborhood.
- **The Pointy End (The Tip):** The algorithm randomly samples a new coordinate inside that neighborhood. If this new coordinate has a higher likelihood than the "worst" point, it is accepted. The tip of the green arrow lands on this new location.

Summary of the Visual Mechanics

- **The Tail:** The surviving point chosen to spawn a new candidate.
- **The Tip:** The successful new point that instantly takes the place of the dead, lowest-likelihood point.
- **The Deleted Point:** The actual point being replaced vanishes from the screen elsewhere in the cluster without an arrow drawn directly to it.

Other notes:

- The RadFriends algorithm determines the radius of the proposal region dynamically. This resulting radius builds a local bounding region around each surviving point. By overlapping these regions across all active points, a collective “friend” zone envelope is formed that wraps around the entire high-density target landscape.
- A red arrow represents a failed proposal where a newly sampled point is rejected. This occurs because the algorithm failed to meet the strict criterion of Nested Sampling: every new point must have a higher likelihood density than the worst point in the current population. Just like a green arrow, a red arrow starts at the randomly selected parent point. The population remains unchanged, no points are replaced, and the algorithm tries again from a new random parent.

Multimodal sampling

Now switch the `Target distribution` to `multimodal`. Compare the performance of nested sampling to MH or HMC. We will compare several sampling techniques in finding multimodal posteriors and calculating evidence integrals in demo notebooks in the next chapter.

Summary of Nested Sampling

Comparison of Nested Sampling, Metropolis-Hastings, and HMC simulations

RadFriends-NS (Nested Sampling) differs fundamentally from algorithms like Metropolis-Hastings (MH) and Hamiltonian Monte Carlo (HMC) because it is designed to calculate a total volume (the Bayesian evidence) rather than just exploring a continuous path (or multiple paths in parallel).

Algorithm	Exploration Strategy	Behavior of Arrows	Best Used For
RadFriends-NS (RadFriends Nested Sampling)	Shrinks a contour to isolate high-density zones while keeping a population of active points.	**Radial clusters.** Green arrows radiate outward inside an adaptive bounding region (a "friend" ball) to replace the lowest-likelihood point.	Multi-modal distributions and calculating Bayesian evidence.
Random Walk MH (Metropolis-Hastings)	Takes blind, random steps in any direction from a single active point.	**Jagged, local paths.** Shows many red arrows (rejections) because random steps frequently land in low-probability areas.	Simple, low-dimensional distributions.
HMC (Hamiltonian Monte Carlo)	Uses physics simulation (gradient/slope) to glide smoothly across the distribution.	**Long, sweeping curves.** Green arrows form long, elegant trajectories with very few red arrows, easily moving between distant peaks.	Complex, high-dimensional, and continuous distributions.

Visualizing the Difference: NS vs. MCMC

Aspect	Traditional MCMC (e.g., HMC, Metropolis)	Nested Sampling (e.g., RadFriends-NS)
Mathematical Goal	Samples proportionally to the target density to map the shape of the posterior.	Directly calculates the absolute volume under the likelihood curve.
The Integration Issue	Cannot easily compute $\mathcal{Z}$ because it does not track the total volume of unvisited spaces.	Explicitly solves $\mathcal{Z}$ by systematically compressing the known prior volume.
Handling Phase Transitions	Often gets trapped on a single peak, missing entire "islands" of probability.	Compresses all modes uniformly, naturally isolating multiple peaks as the volume shrinks.

Every time you see a green arrow replace a point in the RadFriends simulation, the algorithm is taking one more step down the X-axis (from 1 toward 0). It locks in the likelihood of the rejected point ($\mathcal{L}_i$), shrinks the remaining volume estimate (X_i), and adds another rectangle to the grand total of the Bayesian Evidence ($\mathcal{Z}$).

Advantages of nested sampling

- Returns results for both model comparison and parameter inference at the same time.
- Successful in multi-model problems.
- Naturally self-tuning.

Physics connection: statistical mechanics

[This discussion is based on [A+22]]

We can make a connection between the evidence and the canonical ensemble partition function in statistical mechanics by defining an “energy”

$$E(\boldsymbol{\theta}) = -\log \mathcal{L}(\boldsymbol{\theta}) \quad \Longrightarrow \quad \mathcal{L}(\boldsymbol{\theta}) = e^{-E(\boldsymbol{\theta})},$$

where we think of $\boldsymbol{\theta}$ as defining a microstate. Then the partition function at inverse temperature β is

$$Z(\beta) = \int e^{-\beta E(\boldsymbol{\theta})} \pi(\boldsymbol{\theta})\, d\boldsymbol{\theta} = \int \mathcal{L}(\boldsymbol{\theta})^{\beta} \pi(\boldsymbol{\theta})\, d\boldsymbol{\theta},$$

where $\pi(\boldsymbol{\theta})\, d\boldsymbol{\theta}$ is the underlying phase-space measure. For $\beta = 1$ this is the evidence and with $\beta = 0$ the integrand is just the prior. In parallel tempering, we find a differential equation in β for $Z(\beta)$.

Nested sampling of parameters $\boldsymbol{\theta}$ presents a different perspective, which is also well aligned with statistical mechanics, but the analogies are to the *microcanonical ensemble*. The microcanonical ensemble has all states with $E(\boldsymbol{\theta}) = \epsilon$ where ϵ is a constant energy level. The volume of state space corresponding to a given energy ϵ is given by the density of states,

$$g(\epsilon) = \int \delta(E(\boldsymbol{\theta}) - \epsilon) \pi(\boldsymbol{\theta})\, d\boldsymbol{\theta}.$$

Taking the Laplace transform of the density of states g is the canonical partition function $\mathcal{Z}(\beta) = \int e^{-\beta E} g(E) dE$, which corresponds to the generalized evidence $\mathcal{Z}(\beta)$. As noted, the canonical ensemble describes thermodynamic states based on the inverse temperature β rather than the energy level ϵ. In the canonical ensemble, the Boltzmann distribution $p(\boldsymbol{\theta} \mid \beta) = \exp\{-\beta E(\boldsymbol{\theta})\} / \mathcal{Z}(\beta)$ characterizes the distribution of states.

NS in essence tracks the cumulative density of states via:

$$X(\mathcal{L}) = \int_{E \leq -\log \mathcal{L}} g(E)\, dE,$$

Thus, NS is a way of reconstructing the density-of-states information. Note that in NS, we don’t have a microcanonical *shell*, but rather the entire interior below a given E. The practical approximation to the evidence follows as

$$\mathcal{Z}(\beta) \approx \sum_i e^{-\beta E_i} \left(X_{i-1} - X_{i+1}\right)/2.$$

Correspondence between Bayesian inference and stat mech

Bayesian inference (NS)	Statistical mechanics
$\boldsymbol{\theta}$	configuration / microstate
$-\log \mathcal{L}(\boldsymbol{\theta})$	$E(\boldsymbol{\theta})$
$\pi(\boldsymbol{\theta})\, d\boldsymbol{\theta}$	reference phase-space measure
$X(\mathcal{L})$	cumulative density of states
$\mathcal{L} = e^{-E}$	Boltzmann factor at $\beta = 1$
$\mathcal{Z}$	partition function
$-\log \mathcal{Z}$	free energy at $\beta = 1$

During NS, states are generated from the prior constrained by an upper energy limit $\epsilon = -\log \mathcal{L}^\star$, which can be achieved with various techniques (see references). The information entropy of the constrained prior is the *volume entropy* $\log X(\epsilon)$. As NS progresses from one energy limit ϵ to the next $\epsilon' < \epsilon$, the volume entropy changes by $\Delta H = \log[X(\epsilon)/X(\epsilon')]$ at a rate that is constant on average: $\langle \Delta H \rangle = 1/n_{\text{live}}$.

The evidence is an energy-entropy competition, which is what determines equilibrium in statistical mechanics. In Bayesian terms, high likelihood favors a small region around an excellent fit while prior volume favors broader regions fit reasonably well. We recognize this as the tradeoff between better fit and the Occam penalty.

NS vs. thermodynamic integration

The NS procedure with energy as a control parameter can be contrasted with thermodynamic integration (see Parallel tempering), which works in the canonical ensemble and uses the inverse temperature β as a control parameter to weight each microstate. In contrast to the ensemble property β, a key advantage of $E(\boldsymbol{\theta})$ is that it can be evaluated for a single microstate $\boldsymbol{\theta}$. NS constructs a sequence of energy levels at runtime. The sequence is optimal in that it achieves constant thermodynamic speed because changes in the volume entropy are constant on average. Therefore, NS avoids having to design a good temperature schedule, as must be done in thermodynamic integration.

23. State-of-the-art MCMC implementations

In this chapter we present an (incomplete) list of state-of-the-art MCMC implementations and full probabilistic packages that are available in Python (and often other languages). An overview is presented in Overview: individual sampling libraries and full probabilistic packages followed by several demo notebooks.

23.1. Overview: individual sampling libraries and full probabilistic packages

Here we present an (incomplete) list of state-of-the-art MCMC implementations and packages that are available in Python (and often other languages). Several demo notebooks follow in subsequent sections.

Individual libraries for MCMC sampling

emcee:

emcee [FMHLG13] is an MIT licensed pure-Python implementation of Goodman & Weare's Affine Invariant Markov chain Monte Carlo (MCMC) Ensemble sampler [GW10]

pocoMC:

pocoMC From the documentation:

- `pocoMC` is a Python package for fast Bayesian posterior and model evidence estimation.
- It leverages the Preconditioned Monte Carlo (PMC) algorithm, offering significant speed improvements over traditional methods like MCMC and Nested Sampling.
- Ideal for large-scale scientific problems with expensive likelihood evaluations, non-linear correlations, and multimodality, pocoMC provides efficient and scalable posterior sampling and model evidence estimation.
- Widely used in cosmology and astronomy, `pocoMC` is user-friendly, flexible, and actively maintained."

zeus:

zeus is a Python implementation of the Ensemble Slice Sampling method. From the documentation:

- Fast & Robust Bayesian Inference,
- Efficient Markov Chain Monte Carlo (MCMC),
- Black-box inference, no hand-tuning,
- Excellent performance in terms of autocorrelation time and convergence rate,
- Scale to multiple CPUs without any extra effort,
- Automated Convergence diagnostics.

PyMultiNest:

PyMultiNest interacts with MultiNest [FHB09], a Nested Sampling Monte Carlo library.

ptemcee

ptemcee From the documentation: "`ptemcee`, pronounced "tem-cee", is fork of Daniel Foreman-Mackey's emcee to implement parallel tempering more robustly. As far as possible, it is designed as a drop-in replacement for emcee. If you're trying to characterise awkward, multi-modal probability distributions, then `ptemcee` is your friend. " This repository is archived (frozen) and unmaintained.

Full packages for inference (including sampling)

PyMC:

PyMC is a Python package for Bayesian statistical modeling and probabilistic machine learning which focuses on advanced Markov chain Monte Carlo and variational fitting algorithms.

PyStan:

PyStan provides an interface to Stan, a package for Bayesian inference using the No-U-Turn sampler, a variant of Hamiltonian Monte Carlo.

Bilby:

Bilby is a Bayesian inference library. From the documentation: “The aim of bilby is to provide a user-friendly interface to perform parameter estimation. It is primarily designed and built for inference of compact binary coalescence events in interferometric data, but it can also be used for more general problems.” Bilby provides a common interface to a wide range of samplers, including all of those listed in the first section above.

23.2. Demo: Multimodal distributions with the ptemcee and zeus samplers

In this Demo notebook we consider sampling a simple multimodal distribution. We use the `emcee` sampler as a baseline and compare to the performance of the `ptemcee` parallel tempering sampler and the `zeus` slice sampler.

Python imports

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns;  sns.set(context='talk')

import emcee
import ptemcee
print('ptemcee sampling (version: )', ptemcee.__version__)
import corner
import zeus
print('zeus sampling (version: )', zeus.__version__)

from mpl_toolkits.mplot3d import axes3d
from matplotlib import cm
```

```
ptemcee sampling (version: ) 1.0.0
zeus sampling (version: ) 2.5.4
```

Prior

Use a Gaussian prior centered at (0,0) with variance 10. So this is supposed to encompass the entire region where we expect significant posterior probability. In particular, in the example, it will include both Gaussian peaks. θ_0 and θ_1 are assumed to be uncorrelated.

```
mup = np.zeros(2)   # means
sigp=np.sqrt(10.)   # standard deviation
sigmap = np.diag([sigp**2, sigp**2]) # uncorrelated, so diagonal
sigmapinv = np.linalg.inv(sigmap)

# Normalization factor for the Gaussian
normp = 1/np.sqrt(np.linalg.det(2*np.pi*sigmap))
```

```
def log_prior(x):
    """
    Define the log prior using linear algebra, normp and sigmapinv defined globally.
    """
    dxp = x - mup
    return -dxp @ sigmapinv @ dxp /2 + np.log(normp)
```

Likelihood

```
def setup_modes(sig=0.2, ratio=3.):
    """
    Set up two Gaussians for the likelihood
    """
    # Means of the two Gaussian modes
    mu1 = np.ones(2)
    mu2 = -np.ones(2)

    # Width in each dimension (zero correlation)
    _sig = sig
    sigma1 = np.diag([_sig**2, _sig**2])
    sigma2 = np.diag([_sig**2, _sig**2])
    sigma1inv = np.linalg.inv(sigma1)
    sigma2inv = np.linalg.inv(sigma2)

    # amplitudes of the two modes and the corresponding norm factor
    _amp1 = ratio / (1+ratio)
    _amp2 = 1. / (1+ratio)
    norm1 = _amp1 / np.sqrt(np.linalg.det(2*np.pi*sigma1))
    norm2 = _amp2 / np.sqrt(np.linalg.det(2*np.pi*sigma2))
    return (mu1,mu2,sigma1inv,sigma2inv,norm1,norm2)
```

```
params_modes = setup_modes()  # use the defaults

# Define the log likelihood function
def log_likelihood(x, params=params_modes):
    """
    Define the log likelihoo function using linear algebra.
    """
    (mu1,mu2,sigma1inv,sigma2inv,norm1,norm2) = params # split out the parameters
    dx1 = x - mu1
    dx2 = x - mu2
    return np.logaddexp(-dx1 @ sigma1inv @ dx1 / 2 + np.log(norm1), \
                        -dx2 @ sigma2inv @ dx2 / 2 + np.log(norm2) )
```

Posterior

```
def log_posterior(x):
    """
    Return the log of the posterior, calling the log prior and log likelihood
    functions.
    """
    return log_prior(x) + log_likelihood(x)

@np.vectorize
def posterior(y,x):
    """
    Return the posterior by exponentiating the log prior and likelihood.
    """
    xv=np.array([x,y])
    return np.exp(log_likelihood(xv) + log_prior(xv))
```

MH Sampling and convergence

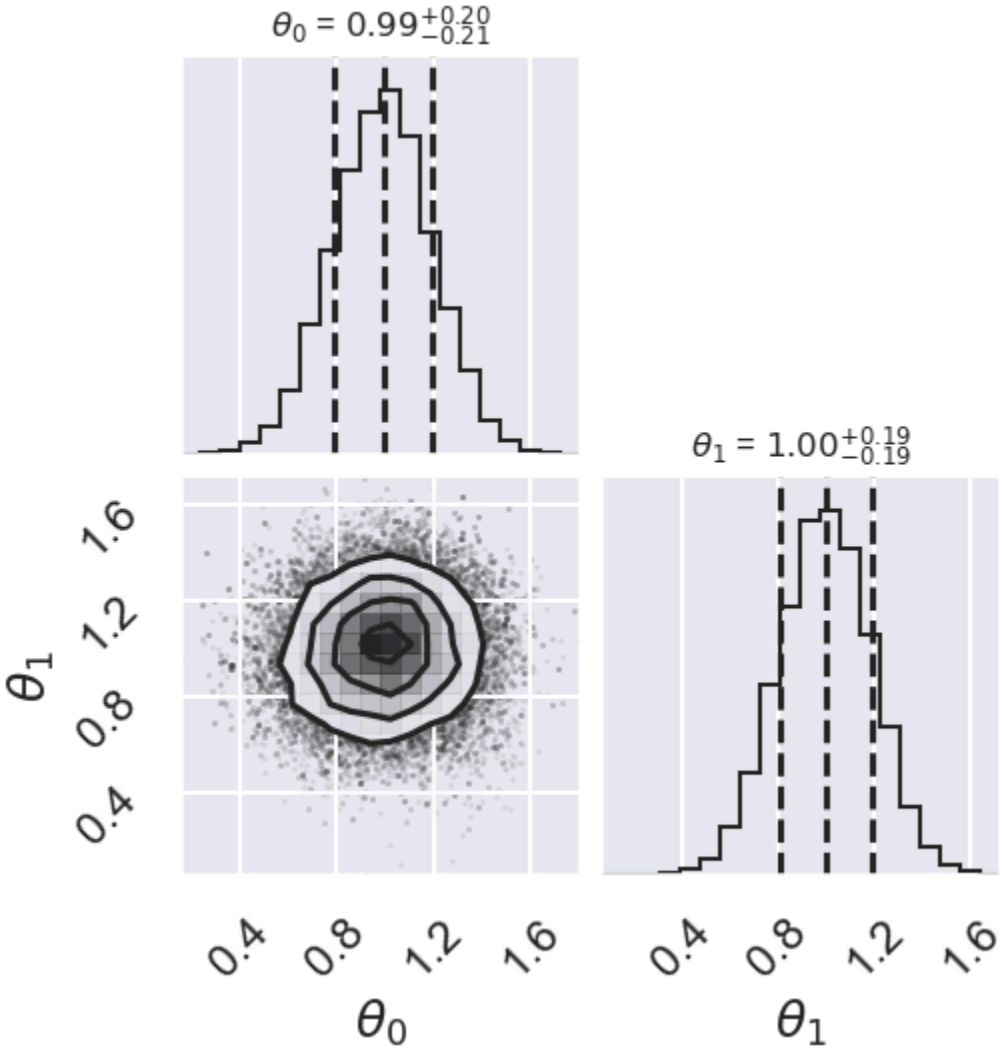
First do some basic sampling using the Metropolis-Hastings (MH) option in `emcee`. This is specified by `moves=emcee.moves.GaussianMove(cov)`. The starting positions are randomly distributed from 0 to 1 (not negative).

```
print('emcee sampling (version: )', emcee.__version__)

ndim = 2  # number of parameters in the model
nwalkers = 50
nwarmup = 1000
nsteps = 1000

# MH-Sampler setup
stepsize = .05
cov = stepsize * np.eye(ndim)
p0 = np.random.rand(nwalkers, ndim)   # uniform between 0 and 1

# initialize the sampler
sampler = emcee.EnsembleSampler(nwalkers, ndim, log_posterior,
                                moves=emcee.moves.GaussianMove(cov))
```

```
emcee sampling (version: ) 3.1.6
```

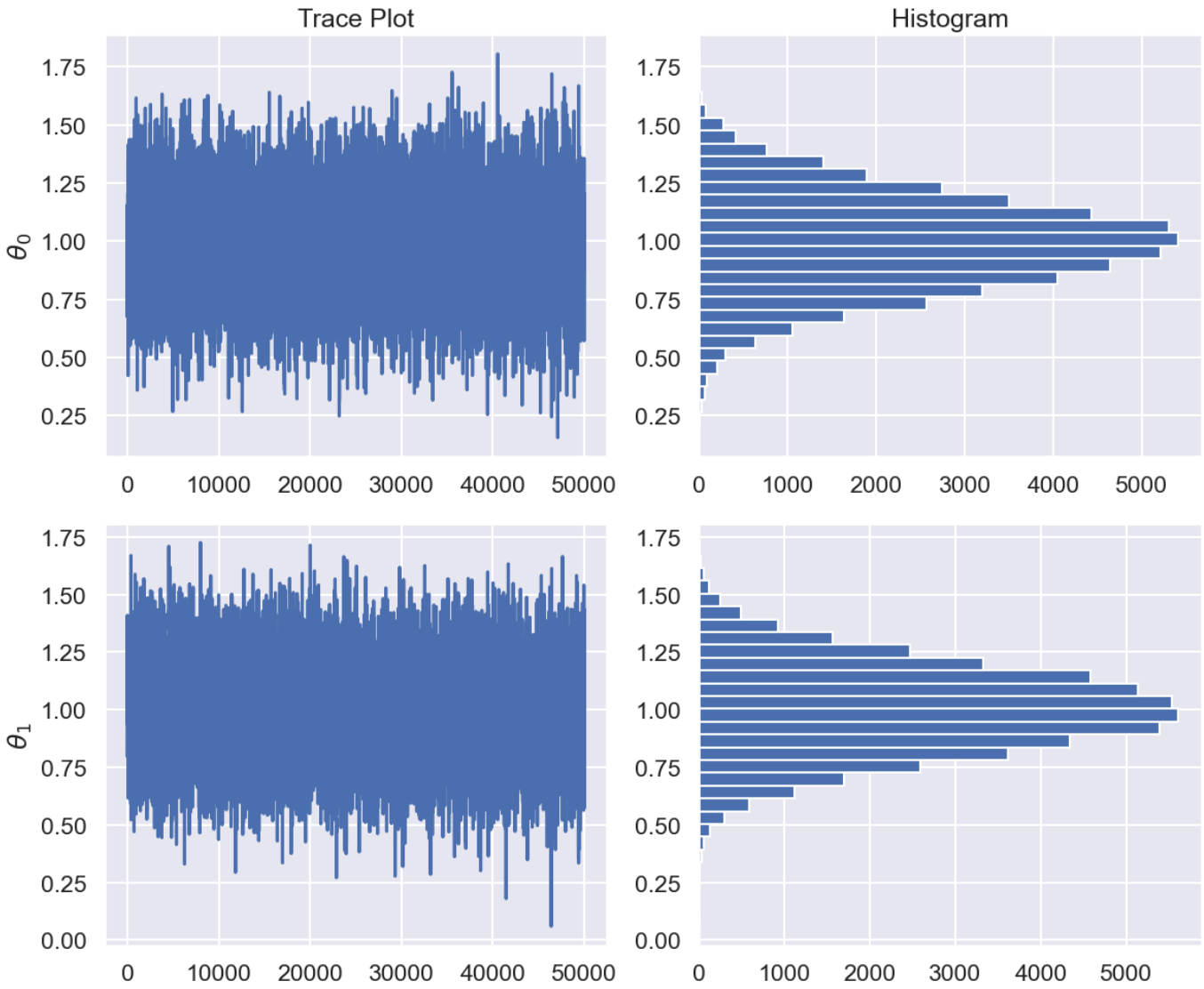
```
# Sample the posterior distribution

# Warm-up
if nwarmup > 0:
    print(f'Performing {nwarmup} warmup iterations.')
    pos, prob, state = sampler.run_mcmc(p0, nwarmup)
    sampler.reset()
else:
    pos = p0

# Perform iterations, starting at the final position from the warmup.
print(f'MH sampler performing {nsteps} samples.')
%time sampler.run_mcmc(pos, nsteps)
print("done")

print(f"Mean acceptance fraction: {np.mean(sampler.acceptance_fraction):.3f}")

# get_chain() has shape (nsteps, nwalkers, ndim); flat=True merges the first two axes
samples = sampler.get_chain(flat=True)
samples_unflattened = sampler.get_chain()
lnposts = sampler.get_log_prob(flat=True)
lnposts_unflattened = sampler.get_log_prob()


# make a corner plot with the posterior distribution
fig = corner.corner(samples, quantiles=[0.16, 0.5, 0.84], labels=[r"$\theta_0$",
r"$\theta_1$"],
                        show_titles=True, title_kwargs={"fontsize": 12})
```

```
Performing 1000 warmup iterations.
MH sampler performing 1000 samples.
CPU times: user 1.2 s, sys: 11 ms, total: 1.21 s
Wall time: 1.21 s
done
Mean acceptance fraction: 0.508
```



Question: *According to this MCMC sampling, what does the posterior look like?*

```
fix, ax = plt.subplots(2,2,figsize=(12,5*ndim))
for irow in range(ndim):
    # one line per walker
    ax[irow,0].plot(samples_unflattened[:,:,irow], alpha=0.5)
    ax[irow,0].set_ylabel(r'$\theta_{0}$'.format(irow))
    ax[irow,1].hist(samples[:,irow],orientation='horizontal',bins=30)

ax[0,1].set_title('Histogram')
ax[0,0].set_title('Trace Plot')

plt.tight_layout()
```




```
for irow in range(ndim):
    print(f"Standard Error of the Mean for theta_{irow}:
{samples[:,irow].std()/np.sqrt(samples.shape[0]):.1e}")
```

```
Standard Error of the Mean for theta_0: 9.2e-04
Standard Error of the Mean for theta_1: 8.6e-04
```

Check for between chain variations

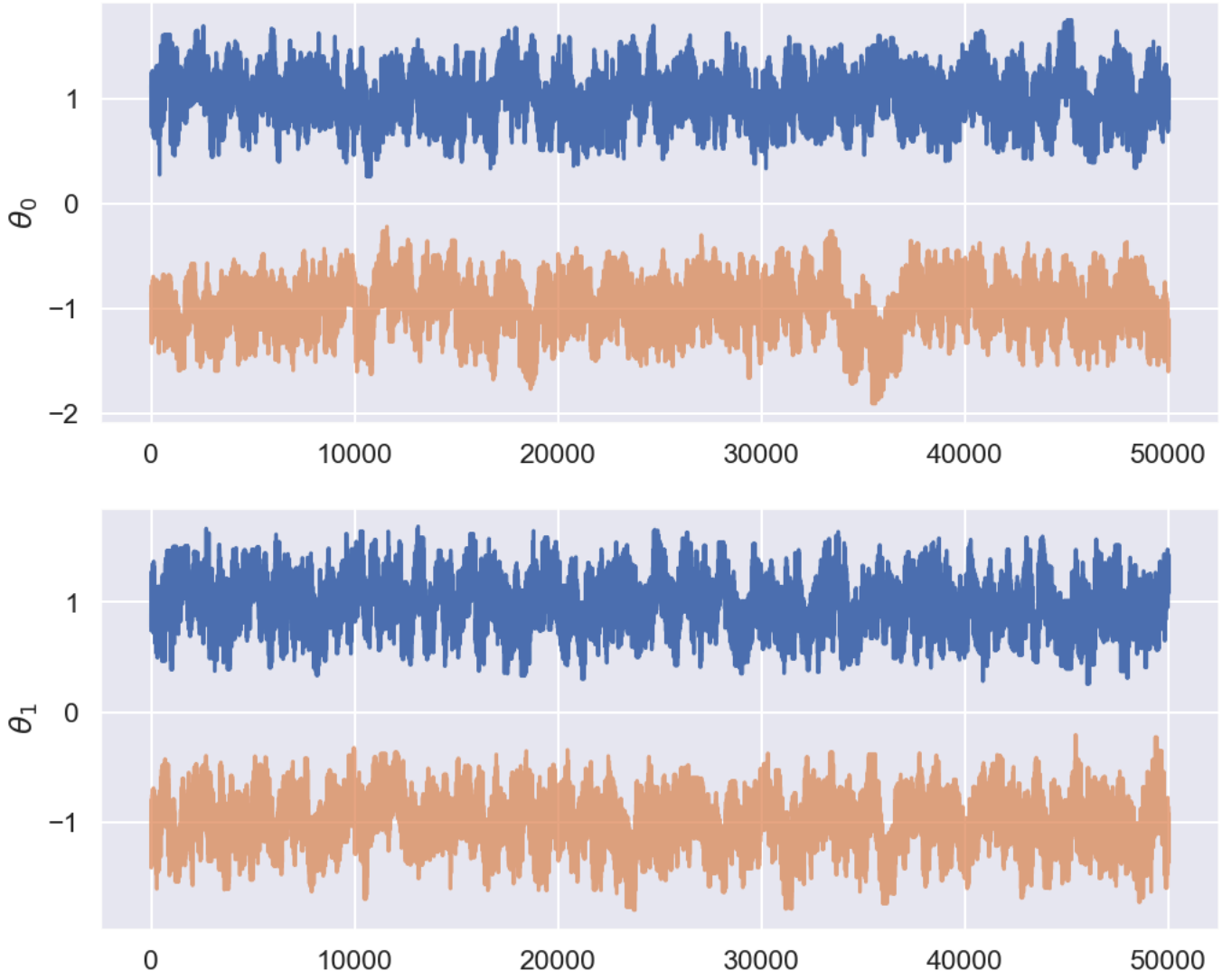
Note how the initial positions of the different walks are chosen here. What does `(-1)**chain` accomplish?

```
no_of_chains = 2
chains = []

for ichain in range(no_of_chains):
    sampler.reset()
    p0 = np.random.rand(nwalkers, ndim)/10 + (-1)**ichain
    # Warm-up
    if nwarmup > 0:
        print(f'Chain {ichain} performing {nwarmup} warmup iterations.')
        pos, prob, state = sampler.run_mcmc(p0, nwarmup)
        sampler.reset()
    else:
        pos = p0

    # Perform iterations, starting at the final position from the warmup.
    print(f'MH sampler {ichain} performing {nsteps} samples.')
    sampler.run_mcmc(pos, nsteps)
    print("done")
    print(f"Mean acceptance fraction: {np.mean(sampler.acceptance_fraction):.3f}")

    # Each walker of the GaussianMove sampler is an independent Metropolis chain.
    # We keep one walker (index 0) as the chain from this run; shape (nsteps, ndim)
    chains.append(sampler.get_chain()[:, 0, :])
```

```
Chain 0 performing 1000 warmup iterations.
MH sampler 0 performing 1000 samples.
done
Mean acceptance fraction: 0.504
Chain 1 performing 1000 warmup iterations.
MH sampler 1 performing 1000 samples.
done
Mean acceptance fraction: 0.512
```

```
chain1 = chains[0]
chain2 = chains[1]
```

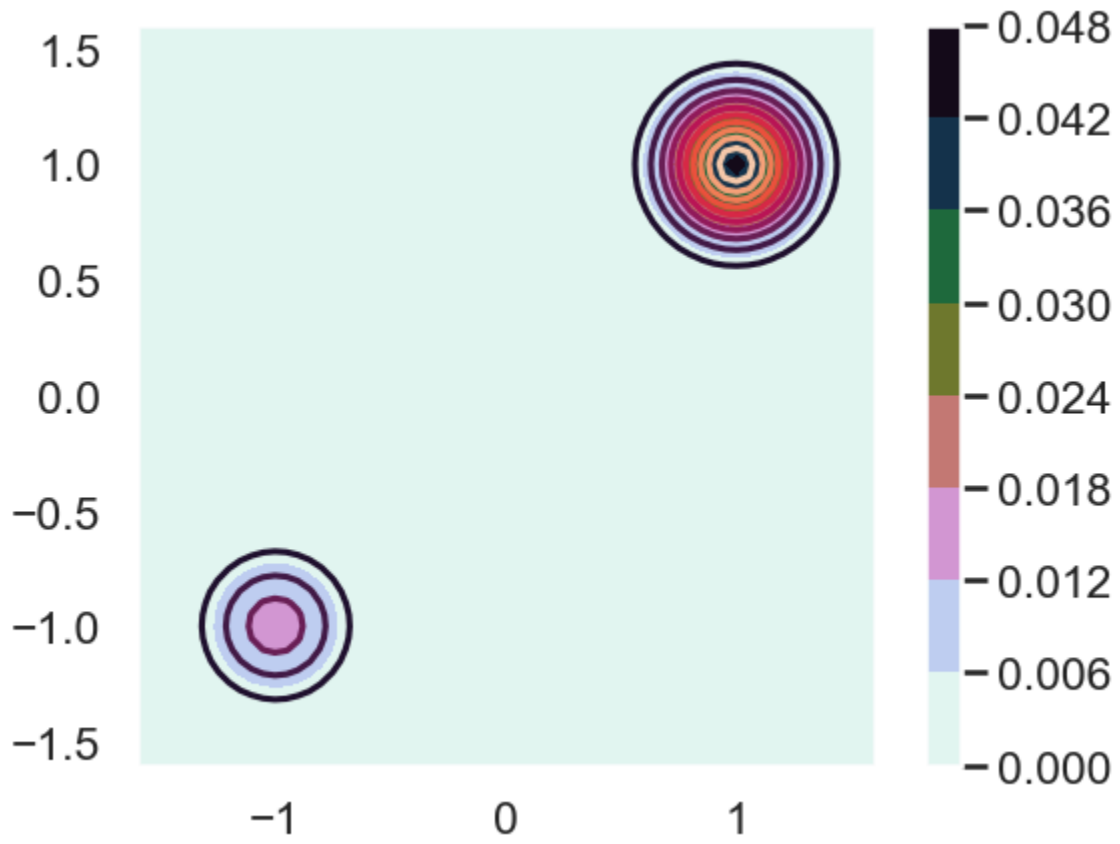
```
fix, ax = plt.subplots(2,1,figsize=(12,10))
for icol in range(ndim):
    ax[icol].plot(np.arange(chain1.shape[0]),chain1[:,icol])
    ax[icol].plot(np.arange(chain2.shape[0]),chain2[:,icol],alpha=.7)
    ax[icol].set_ylabel(r'$\theta_{0}$'.format(icol))
```



Question: *What do you conclude from the trace plots?*

Let's plot it! (Note how the `posterior` function is used here.)

```
fig = plt.figure()
ax = fig.gca()

# Make data.
X = np.arange(-4, 4, 0.05)
Y = np.arange(-4, 4, 0.05)
X, Y = np.meshgrid(X, Y)
Z = posterior(Y,X)

ax.set_xlim([-1.6,1.6])
ax.set_ylim([-1.6,1.6])
ax.contour(X, Y, Z, 10)
CS = ax.contourf(X, Y, Z, cmap=plt.cm.cubehelix_r);
cbar = plt.colorbar(CS)
ax.set_aspect('equal', 'box')
```



Parallel tempering Sampler

Now we repeat the sampling but with a parallel tempering MCMC (or PTMCMC) sampler. We use ptemcee.

Here we construct the sampler object that will drive the PTMCMC. Originally the temperature ladder was 21 temperatures separated by factors of 2; this is commented out below. To improve the accuracy of the integration for an evidence calculation, more lower temperatures were added, i.e., a finer grid near $\beta = 1$.

The highest temperature will be $T = 1024$, resulting in an effective $\sigma_T = 32\sigma = 3.2$, which is about the separation of our modes.

```
#ntemps = 21
#temps = np.array([np.sqrt(2)**i for i in range(ntemps)])

# Modified temperature ladder to improve the integration for evidence calculation
# need more low temperatures, i.e. finer grid near beta=1
ntemps_lo = 8
ntemps_hi = 21
temps_lo = np.array([2**(i/8.) for i in range(ntemps_lo)])
temps_hi = np.array([np.sqrt(2)**i for i in range(ntemps_hi)])
temps = np.concatenate((temps_lo,temps_hi[temps_hi>max(temps_lo)]))
ntemps=len(temps)

# Let us use 10 walkers in the ensemble at each temperature:
nwalkers = 10
ndim = 2

nburnin = 1000
niterations=1000
nthin = 10 # only record every nthin iteration

nthreads = 1  # threading didn't work with current set up (revisit)
```

```
betas=1/temps   # define the beta grid
```

Use the ptemsee sampler.

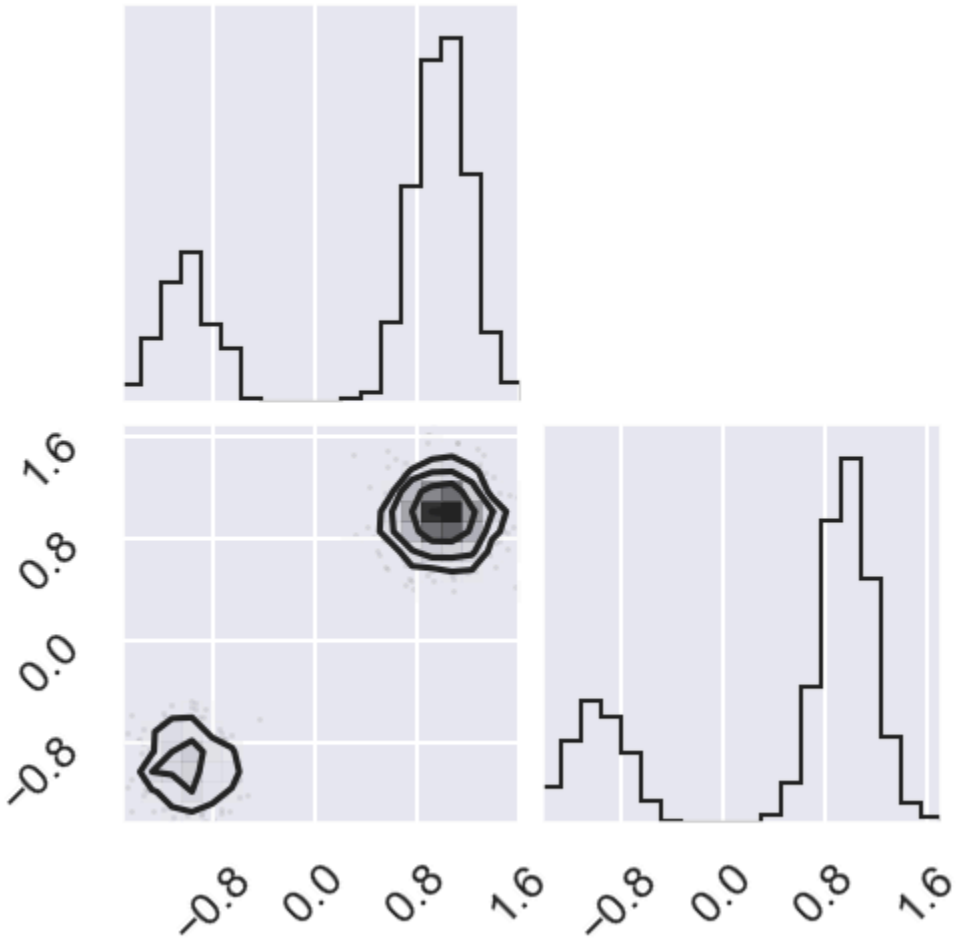
```
sampler = ptemcee.Sampler(nwalkers, ndim, log_likelihood, log_prior, ntemps,
                          threads=nthreads, betas=betas)

#Making the sampling multi-threaded is as simple as adding the threads=Nthreads
# argument to Sampler.

#First, we run the sampler for 1000 burn-in iterations:

# initial walkers are normally distributed with mean mup and standard deviation sigp.
p0 = np.random.normal(loc=mup, scale=sigp, size=(ntemps, nwalkers, ndim))

print("Running burn-in phase")
for p, lnprob, lnlike in sampler.sample(p0, iterations=nburnin):
    pass
sampler.reset()

print("Running MCMC chains")
#Now we sample for nwalkers*niterations, recording every nthin-th sample:

for p, lnprob, lnlike in sampler.sample(p,iterations=niterations, thin=nthin):
    pass
```

```
Running burn-in phase
Running MCMC chains
```

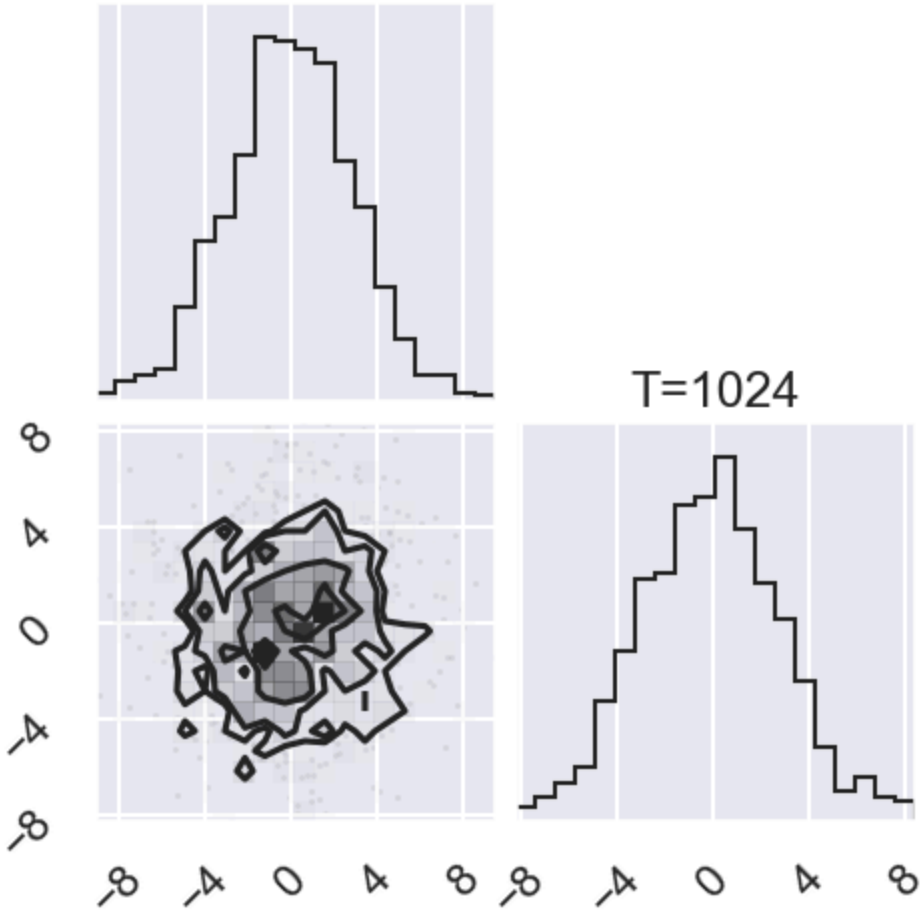
```
# The resulting samples (nwalkers*niterations/nthin of them)
# are stored as the sampler.chain property:

assert sampler.chain.shape == (ntemps, nwalkers, niterations/nthin, ndim)

# Chain has shape (ntemps, nwalkers, nsteps, ndim)
# Zero temperature mean:
mu0 = np.mean(np.mean(sampler.chain[0,...], axis=0), axis=0)
print(f"The zero temperature mean is: {mu0}")

(mu1,mu2,sigma1inv,sigma2inv,norm1,norm2)=params_modes
print("To be compared with likelihood distribution: ")
print(f"... peak 1: {mu1}, peak 2: {mu2}")
```

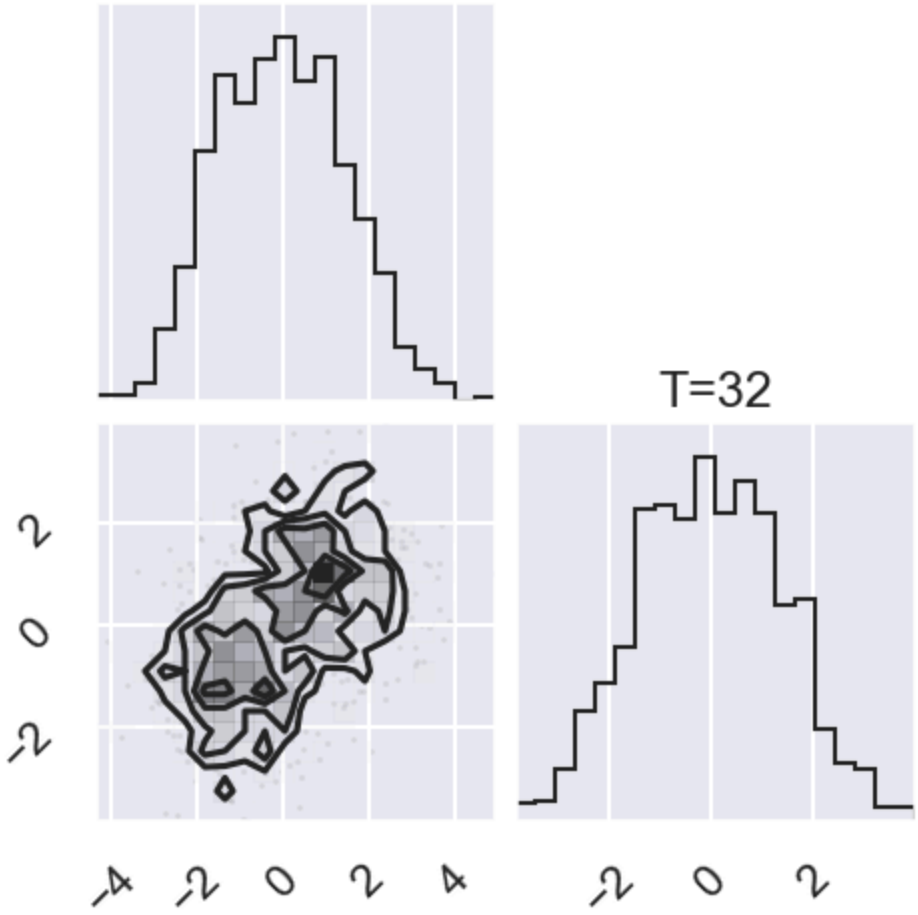
```
The zero temperature mean is: [0.46476976 0.47054044]
To be compared with likelihood distribution:
... peak 1: [1. 1.], peak 2: [-1. -1.]
```

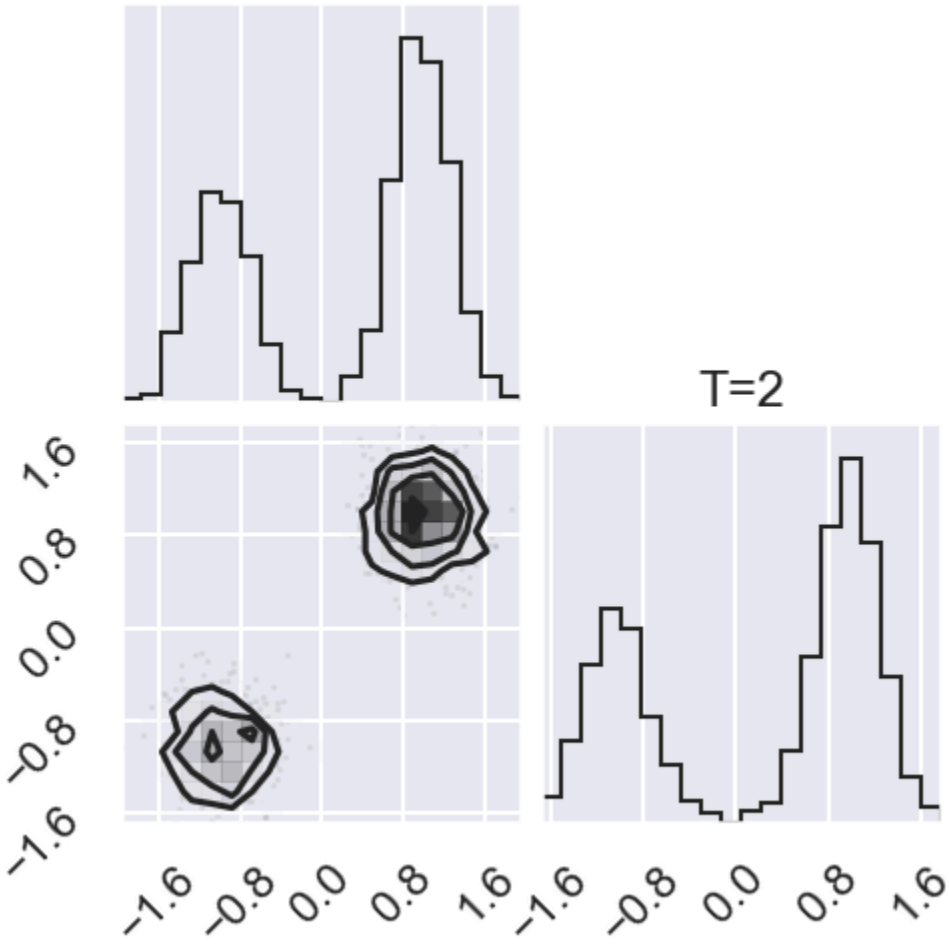
```
# Check the zero temperature corner plot
mcmc_data0 = sampler.chain[0,...].reshape(-1,ndim)
figure = corner.corner(mcmc_data0)

# Extract the axes
axes = np.array(figure.axes).reshape((ndim, ndim))
```

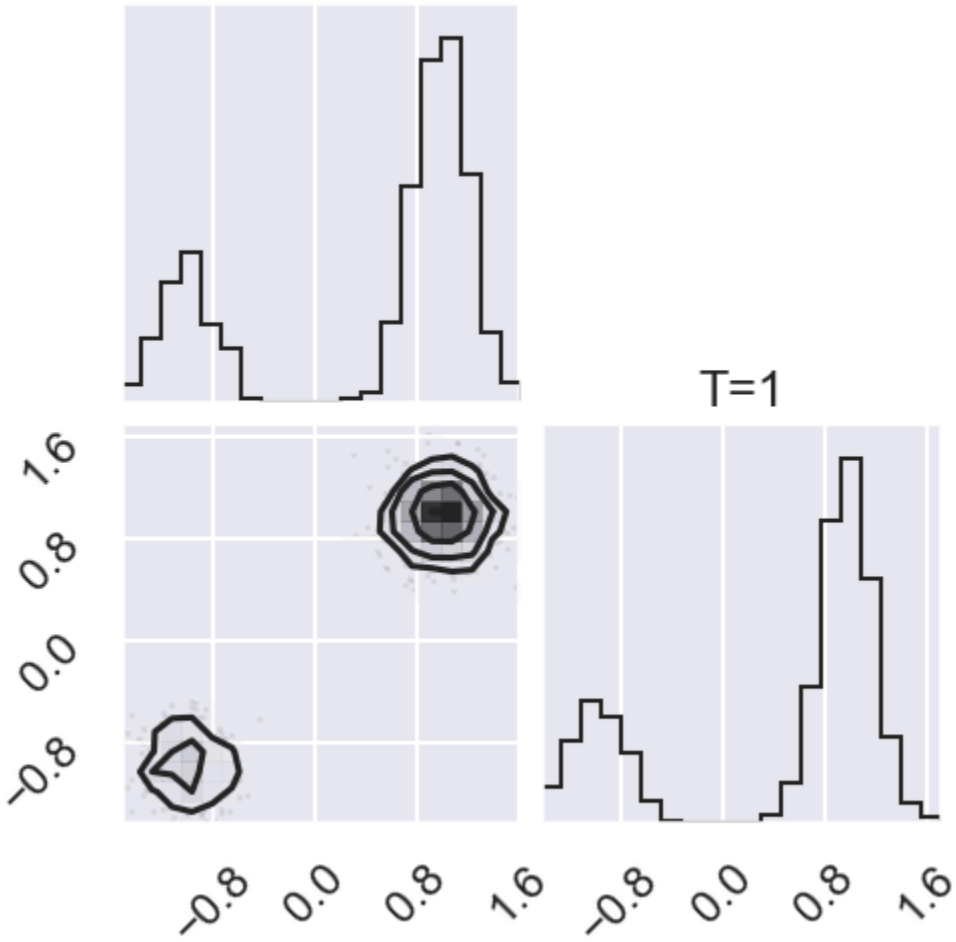



```
# Plot the posterior for four different temperatures
ntempered = np.array([ntemps-1,ntemps-11,8,0])

for nt in ntempered:
    mcmc_data_nt = sampler.chain[nt,...].reshape(-1,ndim)
    figure = corner.corner(mcmc_data_nt)
    # Extract the axes
    axes = np.array(figure.axes).reshape((ndim, ndim))
    axes[ndim-1,ndim-1].set_title('T=%i' %temps[nt])
```


T=1024
8
4
0
−4
−8
−8
−4
0
4
8
−8
−4
0
4
8


T=32
2
0
−2
−4
−2
0
2
4
−2
0
2


T=2
1.6
0.8
0.0
−0.8
−1.6
−1.6
−0.8
0.0
0.8
1.6
−1.6
−0.8
0.0
0.8
1.6



Now try it with slice sampler zeus!

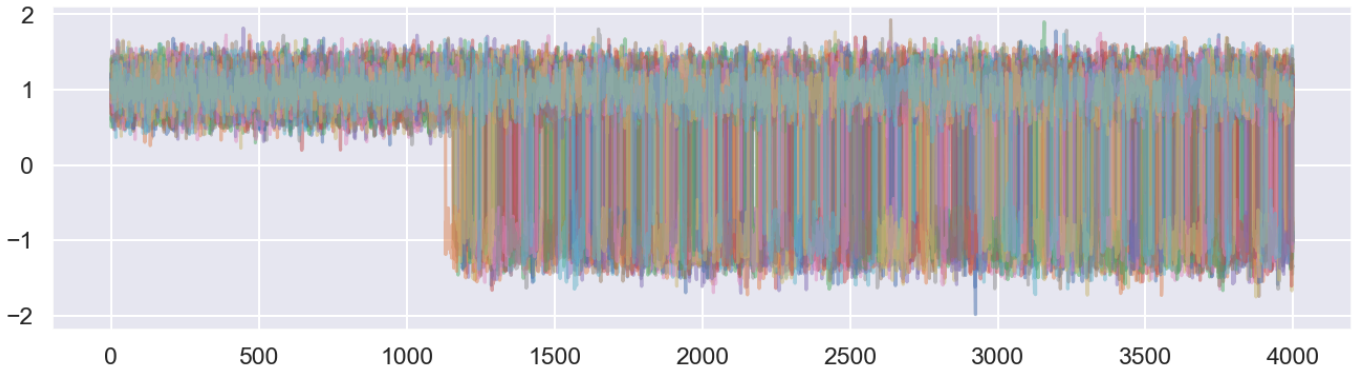
From the documentation at the Zeus webpage:

- `zeus` is a Python implementation of the Ensemble Slice Sampling method.
  - Fast & Robust Bayesian Inference,
  - Efficient Markov Chain Monte Carlo (MCMC),
  - Black-box inference, no hand-tuning,
  - Excellent performance in terms of autocorrelation time and convergence rate,
  - Scale to multiple CPUs without any extra effort.

```
ndim = 2  # number of parameters in the model
nwalkers = 50
nwarmup = 2000
nsteps = 2000

# MH-Sampler setup
stepsize = .05
cov = stepsize * np.eye(ndim)
p0 = np.random.rand(nwalkers, ndim)   # uniform between 0 and 1
```

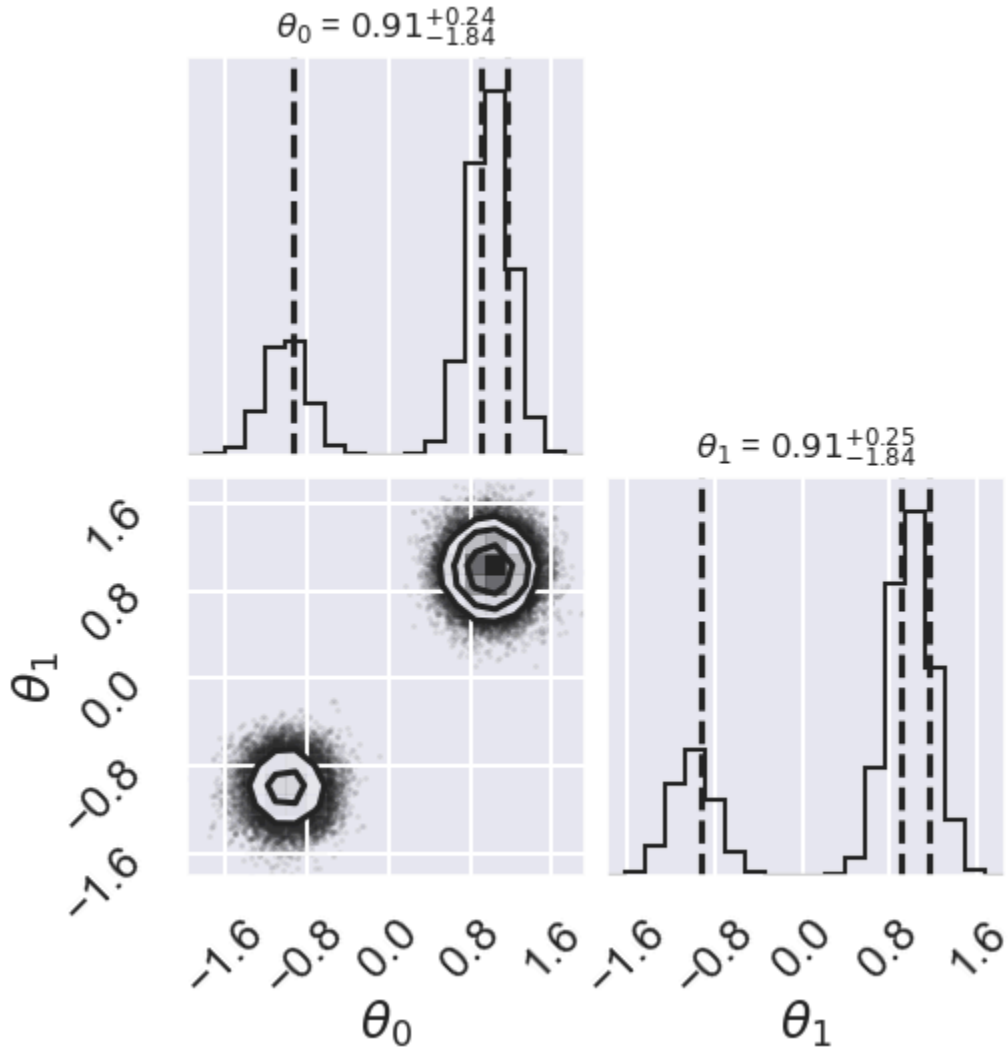
```
# Initialise the Ensemble Sampler using the default ``DifferentialMove``
sampler = zeus.EnsembleSampler(nwalkers, ndim, log_posterior)
# Run MCMC
p0 = 1 + np.random.rand(nwalkers, ndim)/10  # start near 1
start = p0
sampler.run_mcmc(start, nwarmup, progress=False)

# Get the burnin samples
burnin = sampler.get_chain()

# Set the new starting positions of walkers based on their last positions
start = burnin[-1]

# Initialise the Ensemble Sampler using the advanced ``GlobalMove``.
sampler = zeus.EnsembleSampler(nwalkers, ndim, log_posterior, moves=zeus.moves.GlobalMove())
# Run MCMC
sampler.run_mcmc(start, nsteps, progress=False)

# Get the samples and combine them with the burnin phase for plotting purposes
samples = sampler.get_chain()
total_samples = np.concatenate((burnin, samples))

# Plot the walker trajectories for the first parameter
plt.figure(figsize=(16,4))
plt.plot(total_samples[:,:,0],alpha=0.5)
plt.show()
```

```
Initialising ensemble of 50 walkers...
Sampling progress : 100%|████████████████████| 2000/2000 [00:15<00:00, 133.16it/s]
Initialising ensemble of 50 walkers...
/Users/furnstah/Dropbox/Anaconda/anaconda3/envs/2025-book-env/lib/python3.10/site-
packages/sklearn/mixture/_base.py:275: ConvergenceWarning: Best performing initialization did
not converge. Try different init parameters, or increase max_iter, tol, or check for degenerate
data.
  warnings.warn(
```



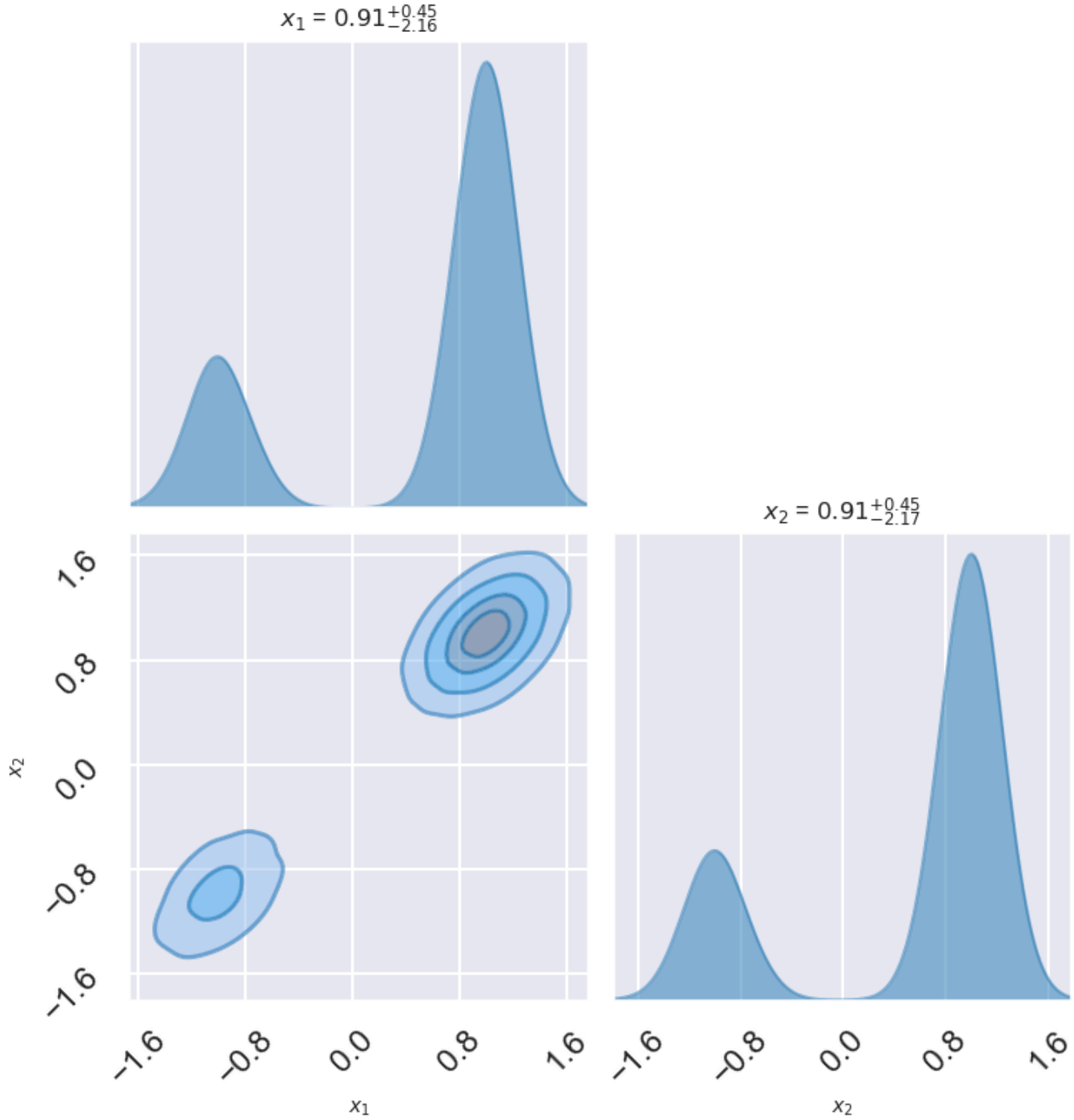

```
# Get the samples flattened
samples = sampler.get_chain(flat=True, discard=0, thin=1)
```

```
# Sample the posterior distribution

# make a corner plot with the posterior distribution
fig = corner.corner(samples, quantiles=[0.16, 0.5, 0.84], labels=[r"$\theta_0$",
r"$\theta_1$"],
                    show_titles=True, title_kwargs={"fontsize": 12})
```




```
# Try the zeus corner plot routine, using every 10
fig, axes = zeus.cornerplot(samples[::10], size=(8,8))
```

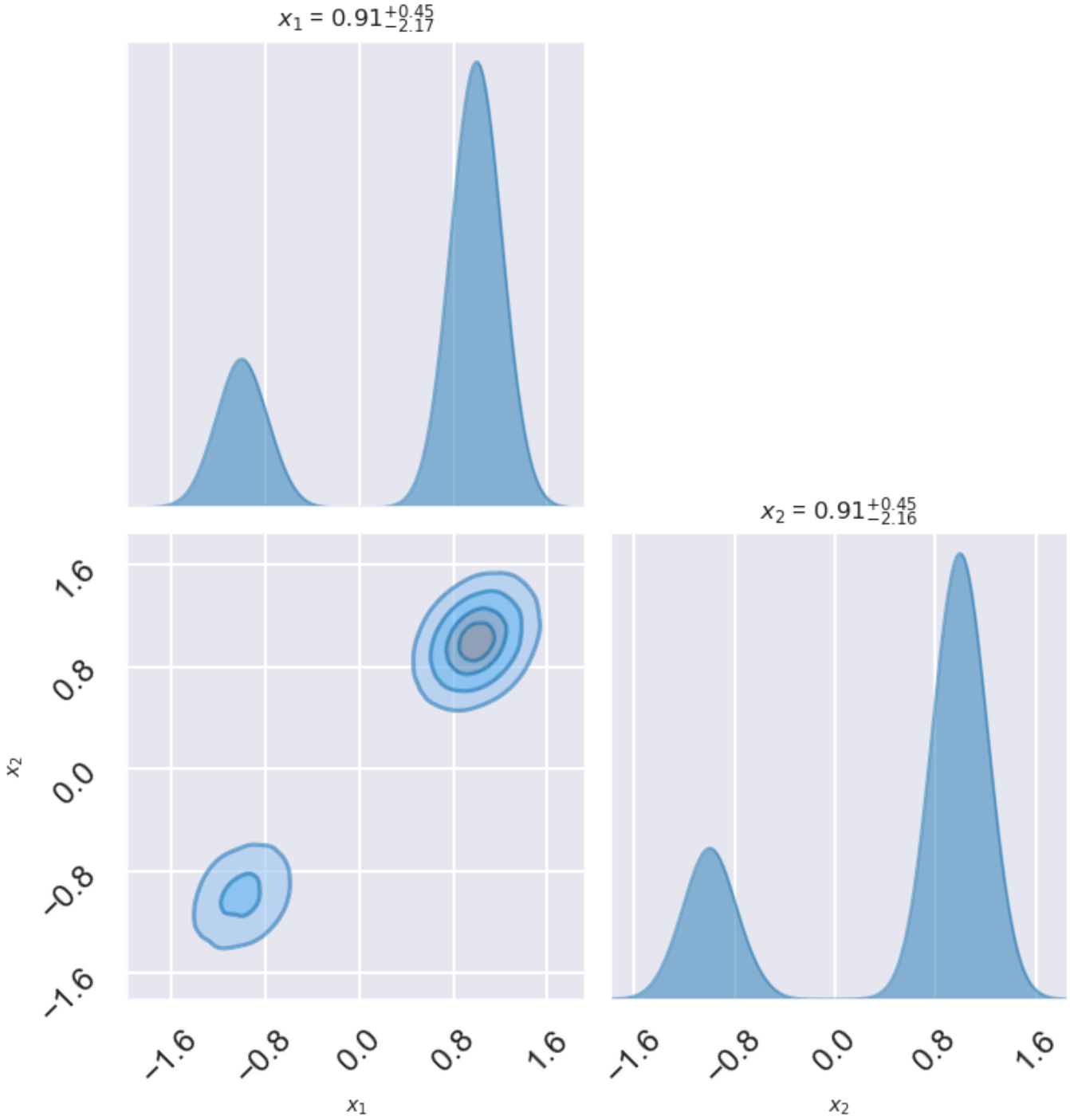
```
/Users/furnstah/Dropbox/Anaconda/anaconda3/envs/2025-book-env/lib/python3.10/site-
packages/seaborn/distributions.py:1176: UserWarning: The following kwargs were not used by
contour: 'linewidth'
  cset = contour_func(
/Users/furnstah/Dropbox/Anaconda/anaconda3/envs/2025-book-env/lib/python3.10/site-
packages/seaborn/distributions.py:1176: UserWarning: The following kwargs were not used by
contour: 'linewidth'
  cset = contour_func(
```




```
# Try the zeus corner plot routine, using all
fig, axes = zeus.cornerplot(samples, size=(8,8))
```

```
/Users/furnstah/Dropbox/Anaconda/anaconda3/envs/2025-book-env/lib/python3.10/site-
packages/seaborn/distributions.py:1176: UserWarning: The following kwargs were not used by
contour: 'linewidth'
  cset = contour_func(
/Users/furnstah/Dropbox/Anaconda/anaconda3/envs/2025-book-env/lib/python3.10/site-
packages/seaborn/distributions.py:1176: UserWarning: The following kwargs were not used by
```

```
contour: 'linewidth'
  cset = contour_func(
```



23.3. pocoMC

Demo to be added. See pocoMC.

23.4. Intro to PyMC

PyMC is a *probabilistic programming library* for Python. Let's quote from the documentation: "PyMC strives to make Bayesian modeling as simple and painless as possible, allowing users to focus on their problem rather than the methods." The list of features are (taken directly from the documentation page):

- **Modern**: Includes state-of-the-art inference algorithms, including MCMC (NUTS) and variational inference (ADVI).
- **User friendly**: Write your models using friendly Python syntax. Learn Bayesian modeling from the many example notebooks.
- **Fast**: Uses PyTensor as its computational backend to compile through C, Numba or JAX, run your models on the GPU, and benefit from complex graph-optimizations.
- **Batteries included**: Includes probability distributions, Gaussian processes, ABC, SMC and much more. It integrates nicely with ArviZ for visualizations and diagnostics, as well as Bambi for high-level mixed-effect models.
- **Community focused**: Ask questions on discourse, join MeetUp events, follow us on Twitter, and start contributing.

Overview of Intro to PyMC notebook

In the following demo notebook, we provide a hands-on, active-learning introduction to PyMC.

Demo: PyMC Introduction starts with sampling to find the posterior for μ, the mean of a distribution, given data that is generated according to a normal distribution with mean zero.

- Try changing the true μ, sampling sigma as well.
- The standard deviation is initially fixed at 1.

In the code, one specifies priors and then the likelihood. This is sufficient to specify the posterior for μ by Bayes' theorem (up to a normalization):

$$p(\mu|D,\sigma) \propto p(D|\mu,\sigma)p(\mu|\mu^0_\mu,\sigma^0_\mu),$$

- $p(\mu|D,\sigma)$ is the pdf to sample,
- D is the *observed* data,
- $p(D|\mu,\sigma)$ is the likelihood, which is specified last. Here it is $\sim \mathcal{N}(\mu,\sigma^2)$.
- μ^0_μ and σ^0_μ are hyperparameters specifying the priors for μ and σ. The statements for these priors appear first.

Looking at PyMC getting started notebook

Demo: PyMC Introduction starts with a brief overview and a list of features of PyMC. Note the comment about Theano being used to calculate gradients needed for HMC. The technique of "automatic differentiation" ensures machine precision results for these derivatives.

Consider the "Motivating Example" on linear regression model that predicts the outcome $Y \sim \mathcal{N}(\mu,\sigma^2)$ from a "linear model", meaning the expected result is a linear combination of the two input variables X_1 and X_2, $\mu = \alpha + \beta_1 X_1 + \beta_2 X_2$.

- This is what we have written elsewhere as

$$y_{\text{expt}} = y_{\text{th}} + \delta y_{\text{exp}}$$

  (no model discrepancy δy_{th}), with $y_{\text{expt}} \to Y$, $y_{\text{th}} \to \mu$, and δy_{exp} Gaussian noise with mean zero and variance σ^2.

- The basic workflow is: make data then encode the model in approximate statistical notation.
- Step through the line-by-line explanation of the model (more detail than here!).
  - Use the Python `with` for a context manager. Note that the first two statements could have been combined to be `with pm.Model() as basic_model:`.
  - Compare the code to first the priors for *unknown* parameters:

$$\alpha \sim \mathcal{N}(0, 100), \quad \beta_i \sim \mathcal{N}(0, 100), \quad \sigma \sim |\mathcal{N}(0, 1)|,$$

    Investigate the options for `Normal`, such as `shape` in the documentation for Probability Distributions in PyMC.

  - Then the encoding of $\mu = \alpha + \beta_1 X_1 + \beta_2 X_2$.
  - Finally the statement of the likelihood (always comes last) $Y \sim \mathcal{N}(\mu, \sigma^2)$. Note the use of `observed=Y`.
  - Read through all the background!. This and the other examples can serve as prototypes for your use of PyMC.

Your task: run through the notebook!

23.5. Demo: PyMC Introduction

A good starting point for notebooks with PyMC examples is the official documentation site: https://docs.pymc.io/. We've adapted some examples from that site here and in other notebooks.

Aside. Here is a good quote from Rob Hicks on HMC and No U-Turn:

"The idea: rather than blindly stumbling around the posterior, use the posterior gradient to skate around the gradient contour. As you skate closer to a drop-off (gradient is steep and probability is lower), potential energy decreases and kinetic energy increases (since energy is always conserved). When this happens the skater is turned back uphill and pushed from the precipice and skates on along a posterior likelihood contour. The No U-Turn sampler keeps skating until the skater tries to turn back towards the original point."

Imports

```
import numpy as np
import scipy.stats as stats

import arviz as az
import matplotlib.pyplot as plt

import pymc as pm

# Recommended: document what PyMC version we are using
print(f'Running on PyMC v{pm.__version__}')
```

```
Running on PyMC v5.10.3
```

Basic setup of a model

First we need to create a *model*, which will be an instance of the Model class. The model has references to all random variables (RVs) and computes the model log posterior (logp) and its gradients. We typically instantiate it using a with context. For example:

```
with pm.Model() as my_model:
    mu = pm.Normal('mu', mu=0, sigma=1)
    obs = pm.Normal('obs', mu=mu, sigma=1, observed=np.random.randn(100))
```

So `my_model` is an instance of the PyMC Model class, and we have set up a prior for `mu` in the form of a standard normal distribution (i.e., mean = 0 and standard deviation = 1). The last line sets up the likelihood, also distributed as a normal with observed data taken as 100 random draws from a standard normal distribution. The standard deviation sigma for the mu posterior is given. The goal will be to sample the posterior for mu.

Sampling

The main entry point to MCMC sampling algorithms is via the `pm.sample()` function. By default, this function tries to auto-assign the right sampler(s) and auto-initialize if you don't pass anything.

As you can see, on a continuous model, `PyMC` assigns the NUTS sampler, which is very efficient even for complex models. `PyMC` also runs variational inference (i.e. `ADVI`) to find good starting parameters for the sampler. Here we draw 1000 samples from the posterior and allow the sampler to adjust its parameters in an additional 500 iterations. These 500 samples are discarded by default:

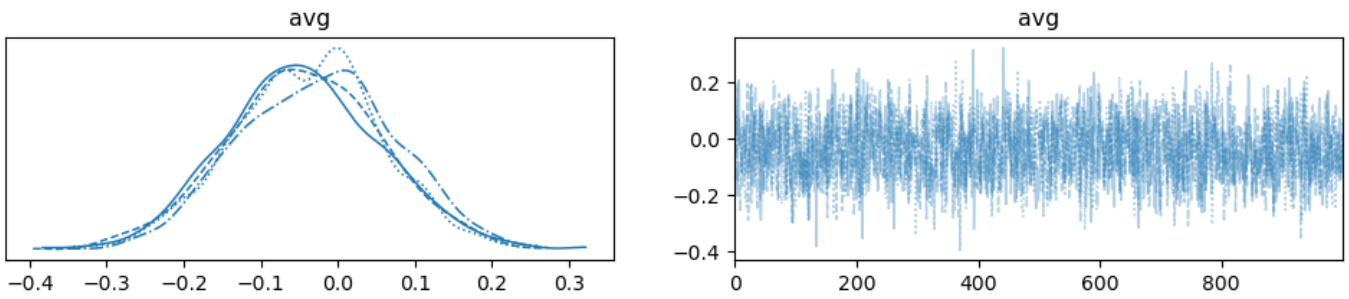
```
with pm.Model() as my_NUTS_model:
    avg = pm.Normal('avg', mu=0, sigma=1)

    model = avg

    obs = pm.Normal('obs', mu=model, sigma=1, observed=np.random.randn(100))

    trace_NUTS = pm.sample(1000, tune=1000, return_inferencedata=True)
```

```
Auto-assigning NUTS sampler...
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [avg]
```

100.00% [8000/8000 00:00<00:00 Sampling 4 chains, 0 divergences]

```
Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 28 seconds.
```

```
with my_NUTS_model:
    az.plot_trace(trace_NUTS);
```



Available samplers

`PyMC` offers a variety of samplers, found in pm.step_methods:

```
list(filter(lambda x: x[0].isupper(), dir(pm.step_methods)))
```

```
['BinaryGibbsMetropolis',
 'BinaryMetropolis',
 'CategoricalGibbsMetropolis',
 'CauchyProposal',
 'CompoundStep',
 'DEMetropolis',
 'DEMetropolisZ',
 'HamiltonianMC',
 'LaplaceProposal',
 'Metropolis',
 'MultivariateNormalProposal',
 'NUTS',
 'NormalProposal',
 'PoissonProposal',
 'STEP_METHODS',
 'Slice',
 'UniformProposal']
```

Commonly used step-methods besides NUTS include Metropolis and Slice. The claim is that for almost all continuous models, `NUTS` should be preferred. There are hard-to-sample models for which NUTS will be very slow

causing many users to use Metropolis instead. This practice, however, is rarely successful. NUTS is fast on simple models but can be slow if the model is very complex or it is badly initialized. In the case of a complex model that is hard for NUTS, Metropolis, while faster, will have a very low effective sample size or not converge properly at all. A better approach is to instead try to improve initialization of NUTS, or reparameterize the model.

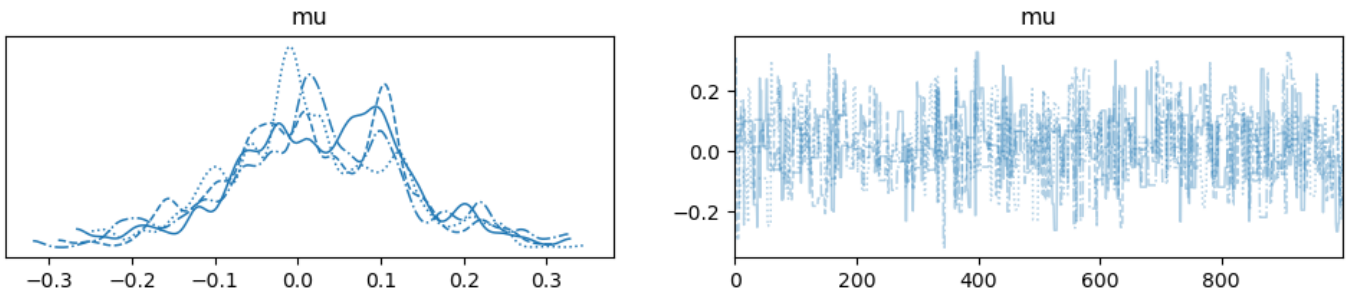
For completeness, other sampling methods can be passed to sample. Here is an example (Metropolis-Hastings):

```
with pm.Model() as my_Metropolis_model:
    mu = pm.Normal('mu', mu=0, sigma=1)
    obs = pm.Normal('obs', mu=mu, sigma=1, observed=np.random.randn(100))

    step = pm.Metropolis()
    trace_MH = pm.sample(1000, step=step, return_inferencedata=True)

    az.plot_trace(trace_MH);
```

```
Multiprocess sampling (4 chains in 4 jobs)
Metropolis: [mu]
```

100.00% [8000/8000 00:00<00:00 Sampling 4 chains, 0 divergences]

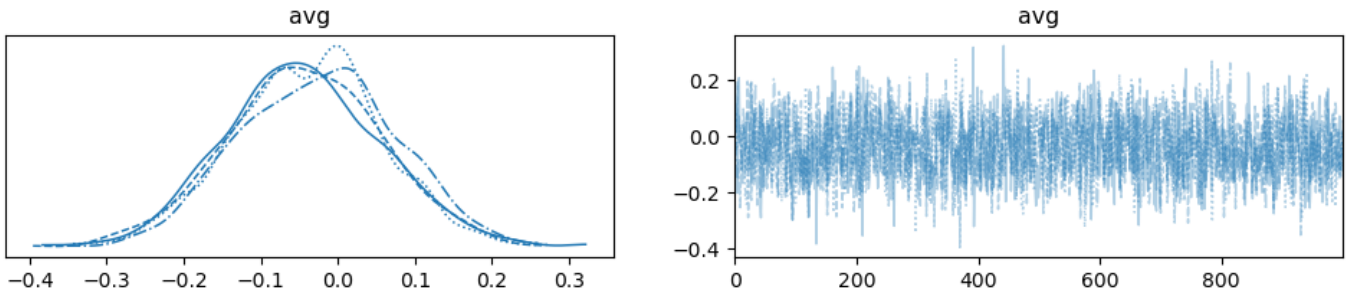
```
Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 28 seconds.
The rhat statistic is larger than 1.01 for some parameters. This indicates problems during sampling. See https://arxiv.org/abs/1903.08008 for details
```



Analyze sampling results

The most common used plot to analyze sampling results is the so-called trace-plot, now invoked with a arViz `plot_trace` command:

```
with my_NUTS_model:
    az.plot_trace(trace_NUTS);
```

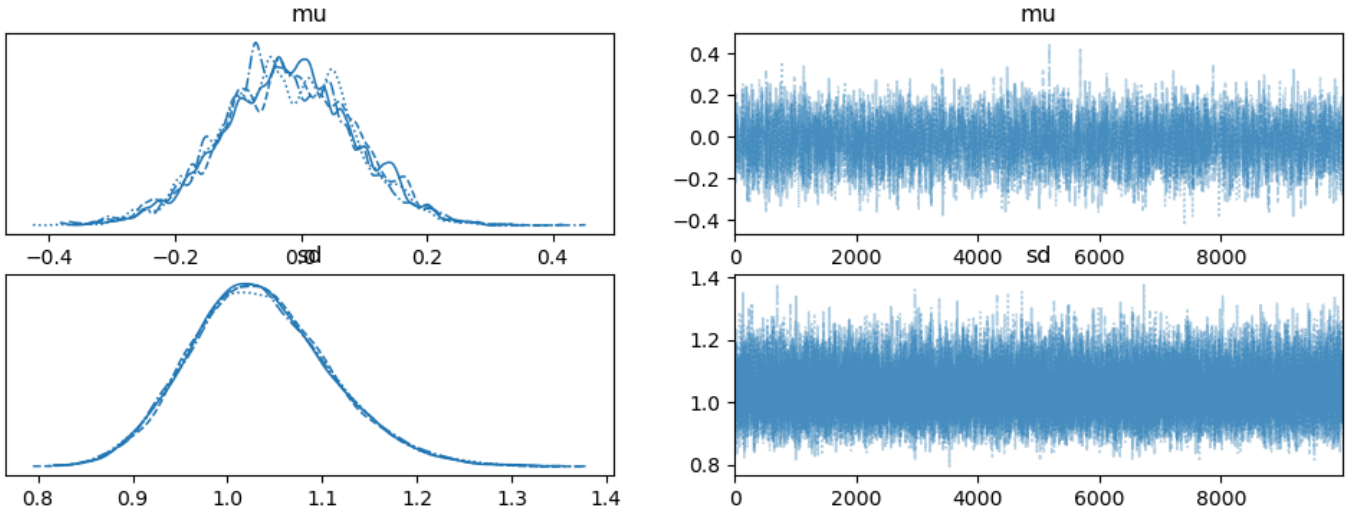



```
with pm.Model() as model:
    mu = pm.Normal('mu', mu=0, sigma=1)
    sd = pm.HalfNormal('sd', sigma=1)
    obs = pm.Normal('obs', mu=mu, sigma=sd, observed=np.random.randn(100))

    step1 = pm.Metropolis(vars=[mu])
    step2 = pm.Slice(vars=[sd])
    trace_2_samplers = pm.sample(10000, step=[step1, step2], cores=4,
                                 return_inferencedata=True)

    az.plot_trace(trace_2_samplers);
```

```
Multiprocess sampling (4 chains in 4 jobs)
CompoundStep
>Metropolis: [mu]
>Slice: [sd]
```

100.00% [44000/44000 00:06<00:00 Sampling 4 chains, 0 divergences]

```
Sampling 4 chains for 1_000 tune and 10_000 draw iterations (4_000 + 40_000 draws total) took 34 seconds.
```



Diagnostics

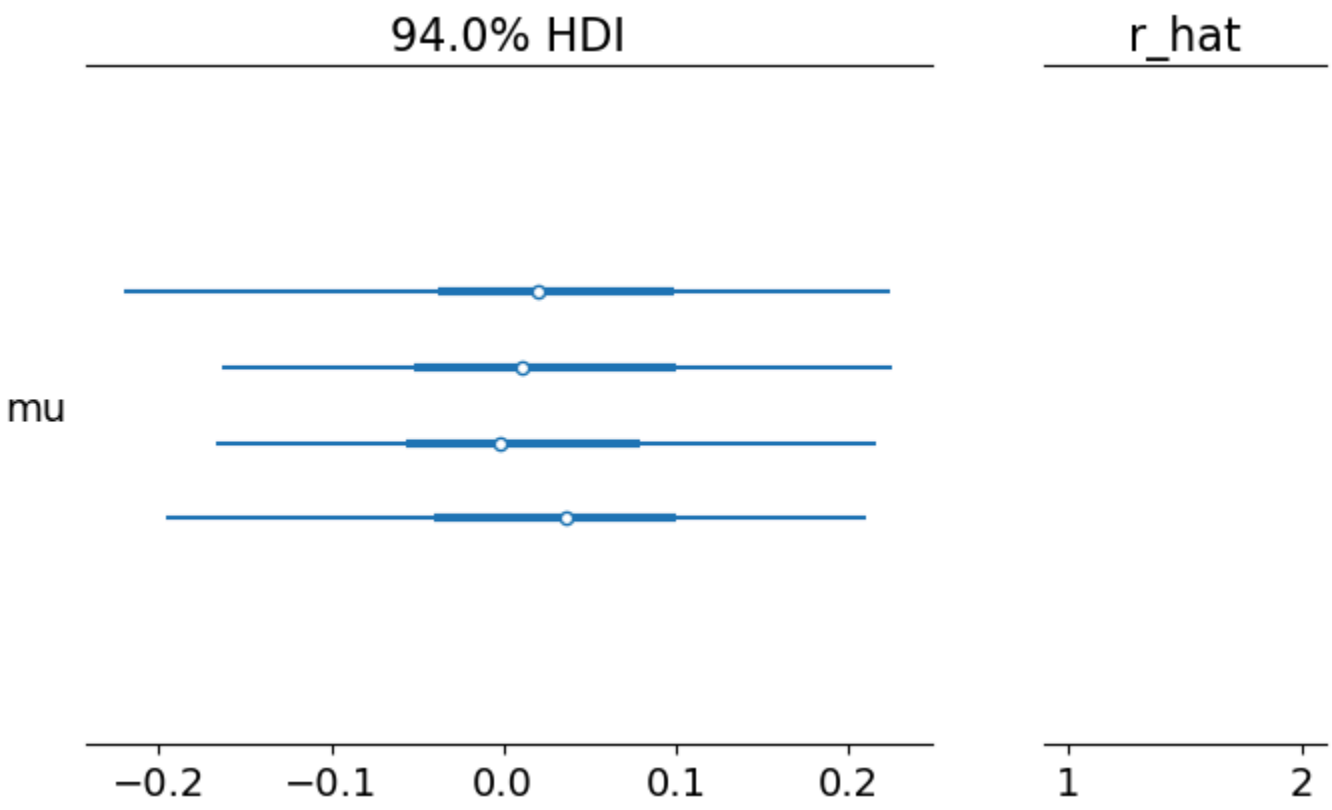
```
with pm.Model() as model:
    display(az.summary(trace_2_samplers))
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
mu	-0.023	0.104	-0.209	0.179	0.001	0.001	5621.0	5657.0	1.0
sd	1.034	0.074	0.902	1.179	0.000	0.000	36668.0	29073.0	1.0

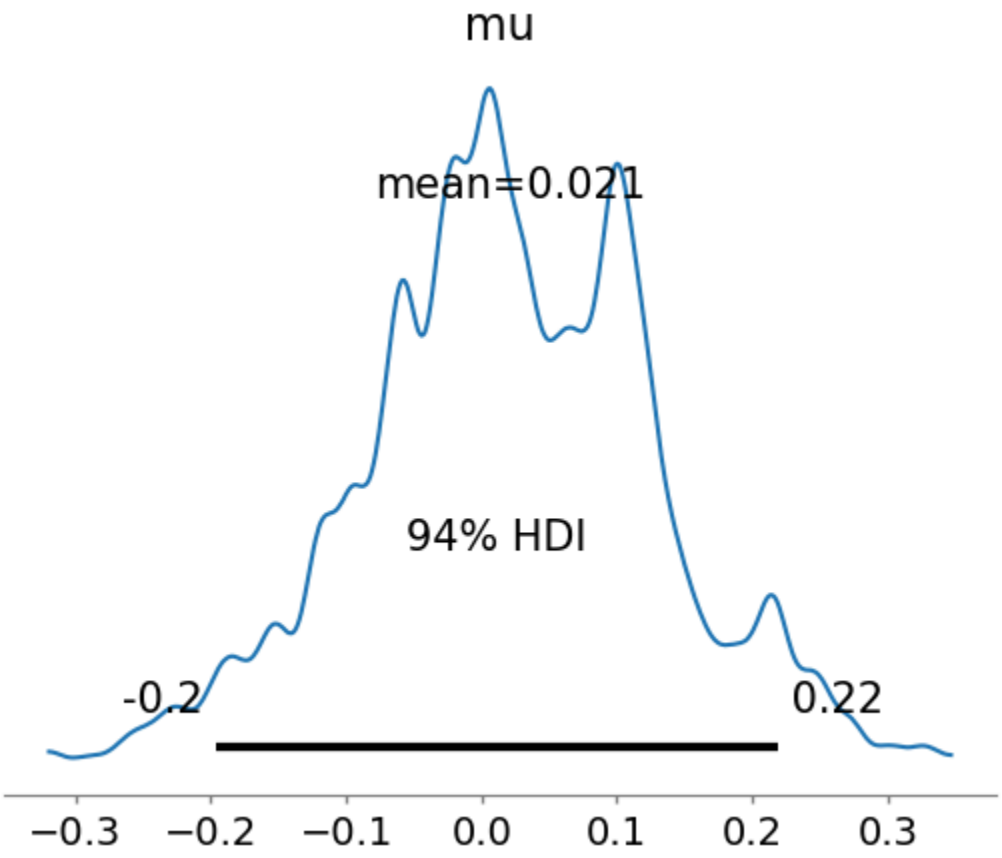
```
with my_Metropolis_model:
    display(az.summary(trace_MH))
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
mu	0.021	0.107	-0.196	0.219	0.004	0.004	604.0	588.0	1.02

```
with my_Metropolis_model:
    az.plot_forest(trace_MH, r_hat=True);
```




```
with pm.Model() as my_Metropolis_model:
    pm.plot_posterior(trace_MH);
```

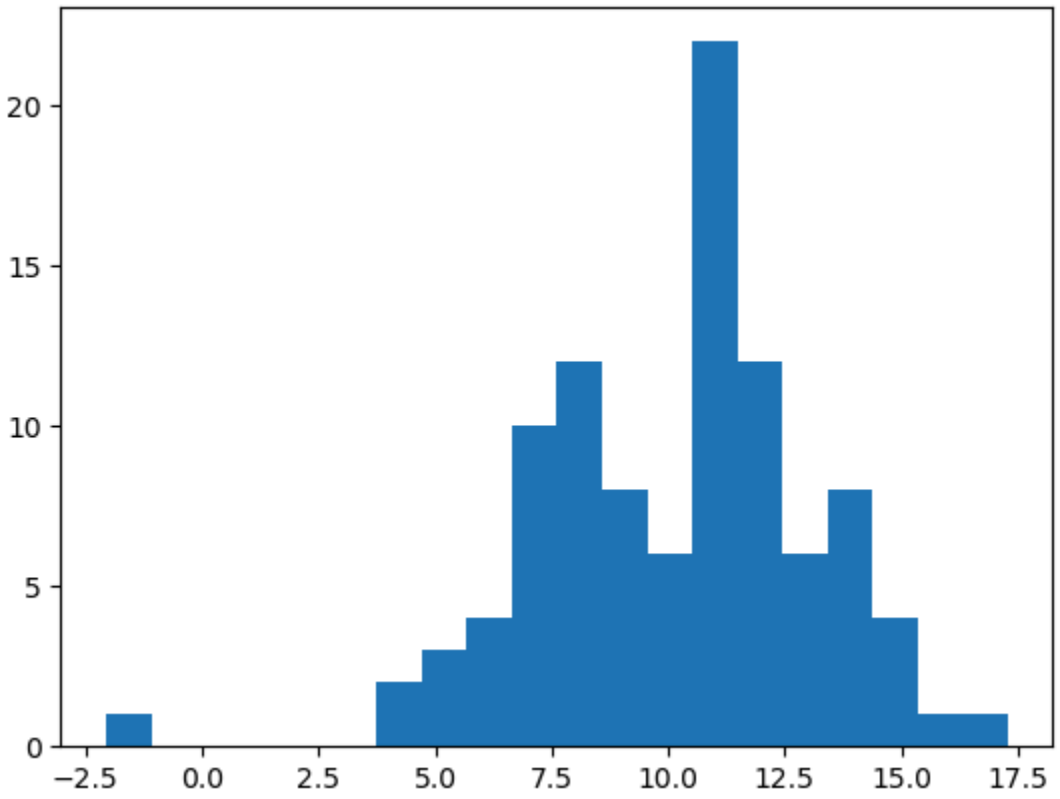


Examples from Rob Hicks

See https://rlhick.people.wm.edu/stories/bayesian_7.html. We also have a notebook from his Bayesian 8 “story”.

We start with a very simple one parameter model and then move to slightly more complicated settings:

```
sigma = 3.  # standard deviation
mu = 10.    # mean
num_samples = 100  # 10**6

# sample from a normal distribution
data = stats.norm(mu, sigma).rvs(num_samples)

# plot a histogram of the sampled data
num_bins = 20
plt.hist(data, bins=num_bins)
plt.show()
```



Run the previous cell a few times to see the fluctuations. Crank up the number of samples to `10**6` to see a smoother histogram.

PyMC implementation

We instantiate a Model with a descriptions of priors and the likelihood. Here, `mu` is defined to be a random variable (we want to sample this variable by generating a Markov chain) and we provide a prior distribution with associated hyper-parameters for it. The likelihood function is chosen to be Normal, with one parameter to be estimated (mu), and we use known σ (denoted as `sigma`). Our "dependent variable" is given by `observed=data`, where `data` is generated above and shown in the histogram. So we our implementing Bayes theorem in the form:

$$\text{pr}\left(\mu|\sigma, \textbf{data}\ \right) \propto \text{pr}\left(\textbf{data}\ |\mu, \sigma\right) \times \text{pr}\left(\mu|\mu_{\mu}^{0}, \sigma_{\mu}^{0}\right) \tag{23.1}$$

```
# parameters for the prior on mu
mu_prior = 8.
sigma_prior = 1.5  # Note this is our prior on the std of mu

# Could do this instead as:
#   basic_model = pm3.Model()
#   with basic_model:

with pm.Model() as basic_model:

    # Prior for unknown model parameters (mean and sd of the normal pdf)
    mu = pm.Normal('Mean of Data', mu_prior, sigma_prior)

    # Likelihood (sampling distribution) of observations
    data_in = pm.Normal('Y_obs', mu=mu, sigma=sigma, observed=data)
```

Next we define how the Markov chain will be constructed. The example we are following set `startvals` to be the MAP and used a Metropolis step method. There always seems to be a complaint with the latest pyMC about using find_MAP to start the sampler.

```
chain_length = 10000

with basic_model:
    # obtain starting values via MAP (maximum a posteriori)
    startvals = pm.find_MAP(model=basic_model)  # model here is optional
    print(startvals)

    # instantiate sampler
    step = pm.Metropolis()   # Metropolis-Hastings

    # draw 10000 posterior samples for each chain (4 chains by default?)
    trace = pm.sample(draws=chain_length, step=step, start=startvals,
                      return_inferencedata=True)
```

100.00% [3/3 00:00<00:00 logp = -277.42, ||grad|| = 23.157]

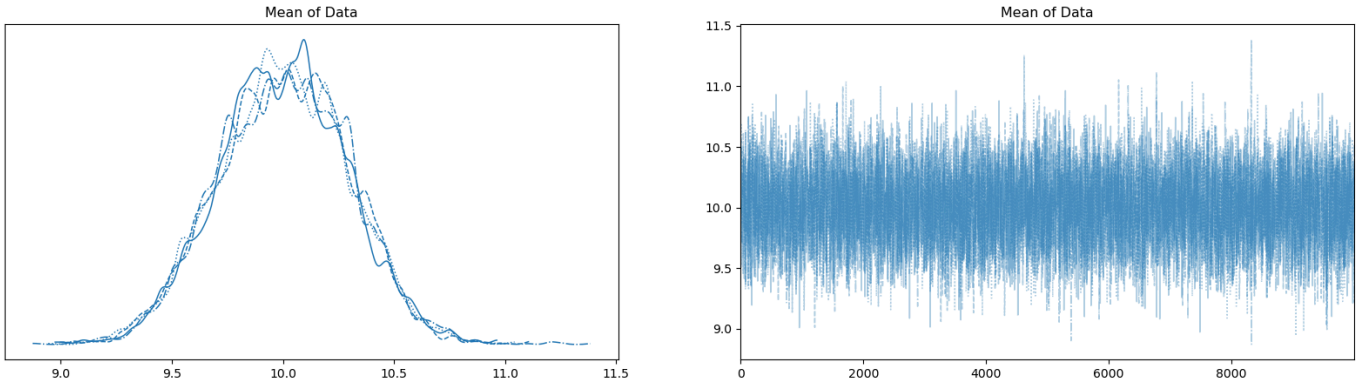
```
{'Mean of Data': array(10.00394922)}
```

```
/var/folders/1s/3wbqg65x5k79l5ww8fg25yvh0000gn/T/ipykernel_15896/2949430162.py:12:
FutureWarning: The `start` kwarg was renamed to `initvals` and can now do more. Please check the
docstring.
  trace = pm.sample(draws=chain_length, step=step, start=startvals,
Multiprocess sampling (4 chains in 4 jobs)
Metropolis: [Mean of Data]
```

100.00% [44000/44000 00:14<00:00 Sampling 4 chains, 0 divergences]

```
Sampling 4 chains for 1_000 tune and 10_000 draw iterations (4_000 + 40_000 draws total) took 41
seconds.
```

```
# Plot the two chains
with basic_model:
    az.plot_trace(trace, figsize=(20,5));
```




```
# Summary information on the Markov chains
with basic_model:
    display(az.summary(trace))
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
Mean of Data	10.003	0.291	9.461	10.536	0.003	0.002	8321.0	9544.0	1.0

Remember that what we are generating is a posterior for the mean given the data and our (assumed) knowledge of the standard deviation.

So for the summary info we get the mean and standard deviation (sd) of the distribution, with an estimate of the Monte Carlo error. What does hpd stand for? “Highest posterior density” 2.5 and 97.5 are percentages, so one talks of a 95% hpd interval in this case.

From an answer online: “You create the parameter trace plots to make sure that your a priori distribution is well calibrated which is indicated by your parameters having sufficient state changes as the MCMC algorithm runs.”

“All the results are contained in the trace variable. This is a pymc results object. It contains some information that we might want to extract at times. Note that this data structure changed between pymc3 and pymc.

```
with basic_model:
    display(trace)
```

arviz.InferenceData

▸ posterior

▸ sample_stats

▸ observed_data

This was set up when we initiated our model (in specifying the prior for mu). With the variable names, we can extract chain values for each variable using trace.posterior[''].

```
trace.posterior['Mean of Data']
```

xarray.DataArray 'Mean of Data' (**chain**: 4, **draw**: 10000)

```
array([[10.24839369, 10.24839369, 10.24839369, ...,  9.94009705,
         9.85834927,  9.85834927],
       [ 9.50584998,  9.50584998,  9.50584998, ..., 10.06524448,
        10.06524448, 10.06524448],
       [ 9.50583452,  9.50583452,  9.60683078, ..., 10.13090017,
        10.13090017, 10.10012361],
       [ 9.91478375,  9.91478375,  9.91478375, ...,  9.5187798 ,
         9.5187798 ,  9.21585145]])
```

▼ Coordinates:

chain	(chain)	int64	0 1 2 3
draw	(draw)	int64	0 1 2 3 4 ... 9996 9997 9998 9999

▶ Indexes: (2)

▶ Attributes: (0)

Is this one chain or both chains? Check the length! Looks like both.

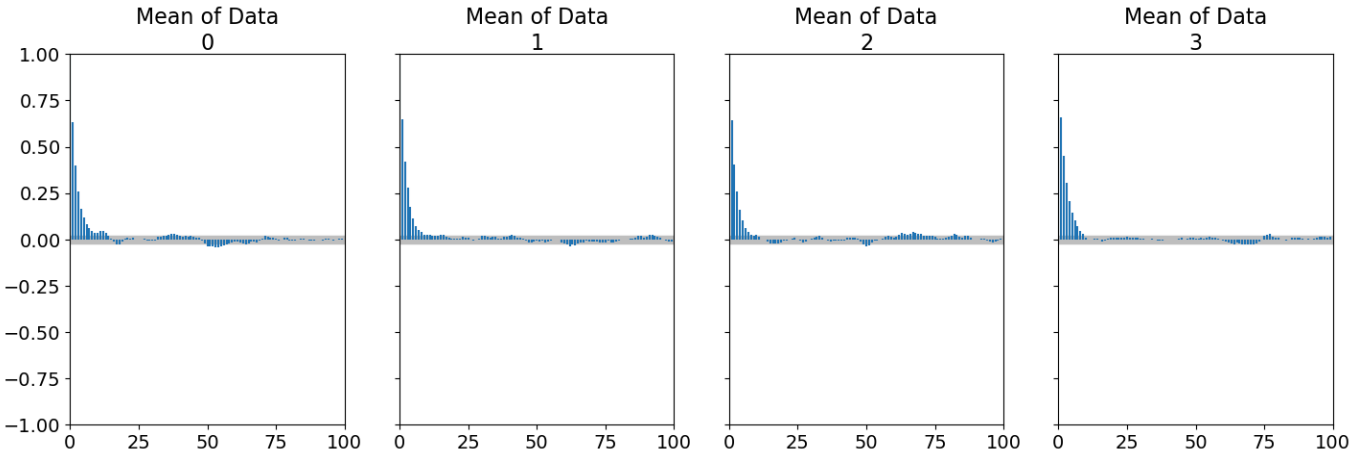
```
print(len(trace.posterior['Mean of Data']))
print(trace.posterior['Mean of Data'].shape)
```

```
4
(4, 10000)
```

Now for diagnostics.

Autocorrelation plots

```
with basic_model:
    az.plot_autocorr(trace, figsize=(17,5));
```



What do we see here? An autocorrelation time around 10 or so.

Acceptance rate

First we have to flatten the samples to make it easier to compute the acceptance rate. But we can only flatten after we have converted to a regular array from a “Dataarray”.

```
samples=trace.posterior['Mean of Data'].values.flatten()
```

```
accept = np.sum(samples[1:] != samples[:-1])
print(accept)
print("Acceptance Rate: ", accept/samples.shape[0])
```

```
13590
Acceptance Rate:  0.33975
```

That looks like we have to work harder than one might have expected. It is taking the array of results and comparing each point to the previous one and including it in the sum if it is different. So if there wasn’t an acceptance, then the point remains the same. The ratio to the full length is the acceptance rate. Maybe we should define a function here instead.

```
def acceptance_rate(trace_array):
    """Calculate how many times the entry in the trace array changed compared
       to the total length.
    """
    changed = np.sum(trace_array[1:] != trace_array[:-1])
    total_length = trace_array.shape[0]
    return changed / total_length
```

```
acceptance_rate(samples)
```

```
0.33975
```

InferenceData object

```
# parameters for the prior on mu
mu_prior = 8.
sigma_prior = 1.5  # Note this is our prior on the std of mu

# Could do this instead as:
#   basic_model = pm3.Model()
#   with basic_model:

with pm.Model() as basic_model_alt:

    # Prior for unknown model parameters (mean and sd of the normal pdf)
    mu = pm.Normal('Mean of Data', mu_prior, sigma_prior)

    # Likelihood (sampling distribution) of observations
    data_in = pm.Normal('Y_obs', mu=mu, sigma=sigma, observed=data)
```

```
chain_length = 10000

with basic_model_alt:
    # obtain starting values via MAP (maximum a posteriori)
    startvals = pm.find_MAP(model=basic_model)  # model here is optional
    print(startvals)

    # instantiate sampler
    step = pm.Metropolis()   # Metropolis-Hastings

    # draw 10000 posterior samples for each chain (4 chains by default?)
    trace_inferencedata = pm.sample(draws=chain_length, step=step, start=startvals,
                                    return_inferencedata=True)
```

100.00% [3/3 00:00<00:00 logp = -277.42, ||grad|| = 23.157]

```
{'Mean of Data': array(10.00394922)}
```

```
/var/folders/1s/3wbqg65x5k79l5ww8fg25yvh0000gn/T/ipykernel_15896/1454651347.py:12:
FutureWarning: The `start` kwarg was renamed to `initvals` and can now do more. Please check the
docstring.
  trace_inferencedata = pm.sample(draws=chain_length, step=step, start=startvals,
Multiprocess sampling (4 chains in 4 jobs)
Metropolis: [Mean of Data]
```

100.00% [44000/44000 00:23<00:00 Sampling 4 chains, 0 divergences]

```
Sampling 4 chains for 1_000 tune and 10_000 draw iterations (4_000 + 40_000 draws total) took 50
seconds.
```

```
with basic_model_alt:
    display(trace_inferencedata.sample_stats)
```

xarray.Dataset

▸ Dimensions: (**chain**: 4, **draw**: 10000)

▾ Coordinates:

chain	(chain)	int64	0 1 2 3
draw	(draw)	int64	0 1 2 3 4 ... 9996 9997 9998 9999

▾ Data variables:

accepted	(chain, draw)	float64	0.0 0.0 1.0 1.0 ... 0.0 0.0 1.0 0.0
accept	(chain, draw)	float64	4.643e-18 0.0003382 ... 0.06293
scaling	(chain, draw)	float64	1.0 1.0 1.0 1.0 ... 1.0 1.0 1.0 1.0

▸ Indexes: (2)

▾ Attributes:

created_at :	2024-03-05T02:13:08.106552
arviz_version :	0.17.0
inference_library :	pymc
inference_library…	5.10.3
sampling_time :	50.17578411102295
tuning_steps :	1000

Gelman Rubin Diagnostic (quoted verbatim from the Hicks notebook)

If our MH MCMC Chain reaches a stationary distribution, and we repeat the excercise multiple times, then we can examine if the posterior for each chain converges to the same place in the distribution of the parameter space.

Steps:

1. Run $M > 1$ Chains of length $2 \times N$.
2. Discard the first N draws of each chain, leaving N iterations in the chain.
3. Calculate the within and between chain variance.
   - Within chain variance: $W = \frac{1}{M} \sum_{j=1}^{M} s_j^2$ where s_j^2 is the variance of each chain (after throwing out the first N draws).
   - Between chain variance: $B = \frac{N}{M-1} \sum_{j=1}^{M} (\bar{\theta_j} - \bar{\bar{\theta}})^2$ where $\bar{\bar{\theta}}$ is the mean of each of the M means.
4. Calculate the estimated variance of θ as the weighted sum of between and within chain variance.
   $\hat{var}(\theta) = \left(1 - \frac{1}{N}\right) W + \frac{1}{N} B$
5. Calculate the potential scale reduction factor. $\hat{R} = \sqrt{\frac{\hat{var}(\theta)}{W}}$

We want this number to be close to 1. Why? This would indicate that the between chain variance is small. This makes sense, if between chain variance is small, that means both chains are mixing around the stationary distribution. Gelmen and Rubin show that when $\hat{R}$ is greater than 1.1 or 1.2, we need longer burn-in.

Trying without `find_MAP`, i.e., not specifying `start` in `pm.sample`.

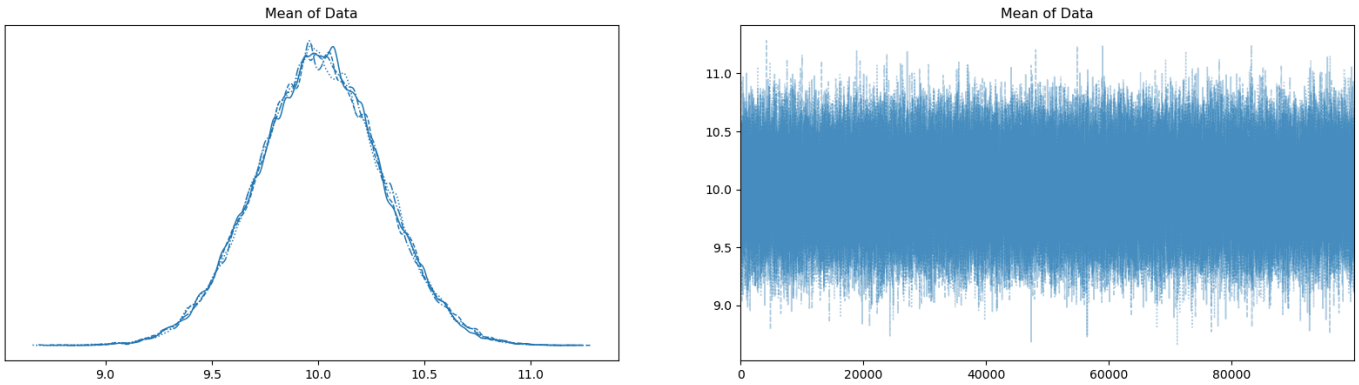
```
chain_length = 100000

with basic_model:
    # obtain starting values via MAP
    #startvals = pm.find_MAP(model=basic_model)
    #print(startvals)
    # instantiate sampler
    step = pm.Metropolis()

    # draw 5000 posterior samples
    trace = pm.sample(chain_length, step=step, return_inferencedata=True)
```

```
Multiprocess sampling (4 chains in 4 jobs)
Metropolis: [Mean of Data]
```

100.00% [404000/404000 00:25<00:00 Sampling 4 chains, 0 divergences]

```
Sampling 4 chains for 1_000 tune and 100_000 draw iterations (4_000 + 400_000 draws total) took
53 seconds.
```

```
# Plot the two chains
with basic_model:
    az.plot_trace(trace, figsize=(20,5));
```



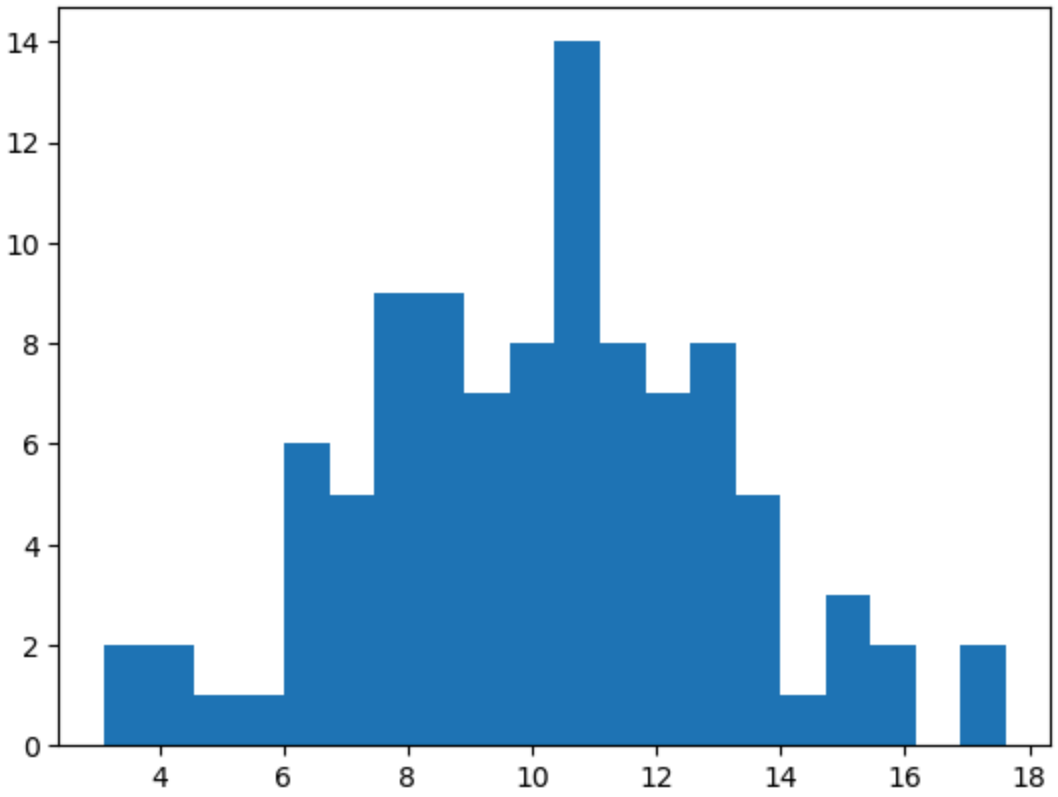
“The diagnostics we have discussed are all univariate (they work perfectly when there is only 1 parameter to estimate). Other diagnostics have been derived for the multivariate case, but these are useful only when using Gibbs Samplers or other specialized versions of Metropolis-Hastings.

So most people examine univariate diagnostics for each variable, examine autocorrelation plots, acceptance rates and try to argue chain convergence based on that- unless they are using Gibbs or other specialized samplers.”

In-class exercise

Let’s try to modify the code below to estimate sigma as well as the mean (first three cells are original code, copied from above):

```
sigma = 3.  # standard deviation
mu = 10.    # mean
num_samples = 100  # 10**6

# sample from a normal distribution
data = stats.norm(mu, sigma).rvs(num_samples)


# plot a histogram of the sampled data
num_bins = 20
plt.hist(data, bins=num_bins)
plt.show()
```




```
# parameters for the prior on mu
mu_mean_prior = 8.
mu_sd_prior = 1.5  # Note this is our prior on the std of mu

with pm.Model() as basic_model:

    # Priors for unknown model parameters
    mu = pm.Normal('Mean of Data', mu_mean_prior, mu_sd_prior)

    # Likelihood (sampling distribution) of observations
    data_in = pm.Normal('Y_obs', mu=mu, sigma=sigma, observed=data)
```

```
chain_length = 10000

with basic_model:
    # obtain starting values via MAP
    startvals = pm.find_MAP(model=basic_model)
    print(startvals)
    # instantiate sampler
    step = pm.Metropolis()

    # draw 10000 posterior samples
    trace = pm.sample(chain_length, step=step, start=startvals,
                      return_inferencedata=True)
```

100.00% [3/3 00:00<00:00 logp = -277.69, ||grad|| = 24.375]

```
{'Mean of Data': array(10.10937835)}
```

```
/var/folders/1s/3wbqg65x5k79l5ww8fg25yvh0000gn/T/ipykernel_15896/1249115293.py:11:
FutureWarning: The `start` kwarg was renamed to `initvals` and can now do more. Please check the
docstring.
  trace = pm.sample(chain_length, step=step, start=startvals,
Multiprocess sampling (4 chains in 4 jobs)
Metropolis: [Mean of Data]
```

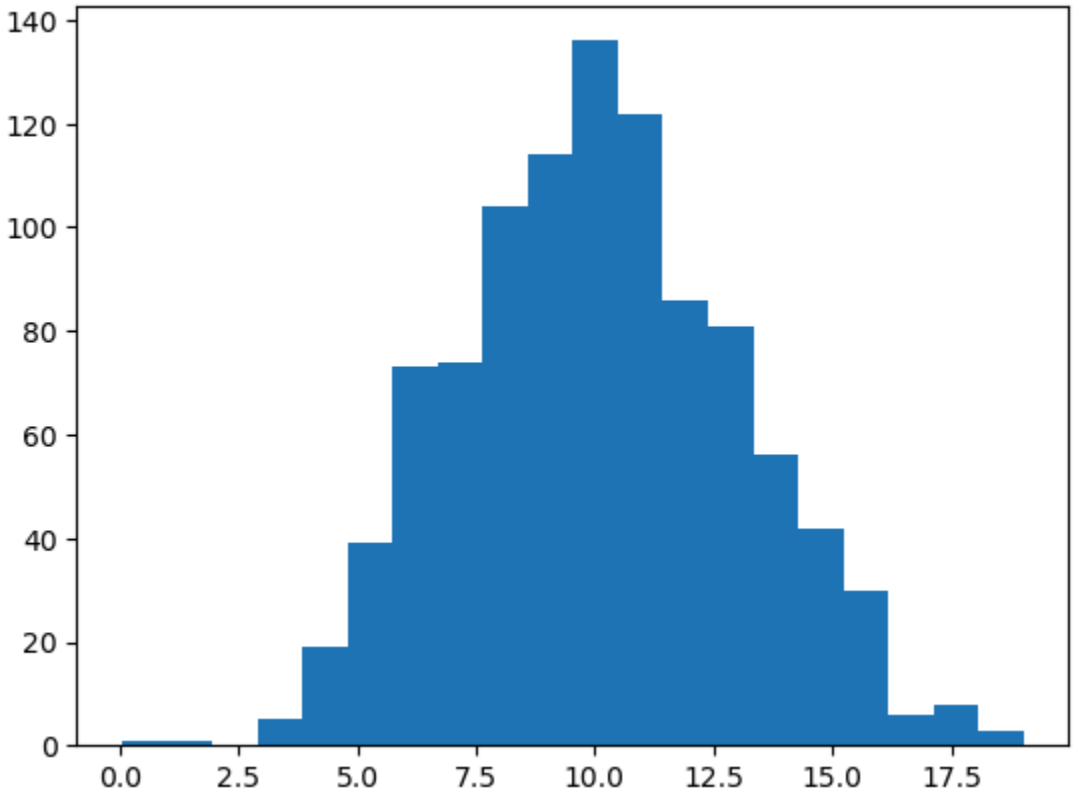
100.00% [44000/44000 00:02<00:00 Sampling 4 chains, 0 divergences]

```
Sampling 4 chains for 1_000 tune and 10_000 draw iterations (4_000 + 40_000 draws total) took 30
seconds.
```

```
# NOTE: currently there is an issue with geweke. Try again in the future.
# score=pm.geweke(trace, first=0.1, last=0.5, intervals=20)
# plt.scatter(score[0]['Mean of Data'][:,0],score[0]['Mean of Data'][:,1],
#             marker = 'o', s=100)
# plt.axhline(-1.98, c='r')
# plt.axhline(1.98, c='r')
# plt.ylim(-2.5,2.5)
# plt.xlim(0-10,.5*trace['Mean of Data'].shape[0]/2+10)
# my_title = 'Geweke Plot Comparing first 10% and Slices of the Last 50%' +\
#            ' of Chain\nDifference in Mean Z score'
# plt.title(my_title)
# plt.show()
```

Ok, we're trying it! (New code starts here.)

```
sigma = 3.  # standard deviation
mu = 10.    # mean
num_samples = 1000  # 100 # 10**6

# sample from a normal distribution
data = stats.norm(mu, sigma).rvs(num_samples)


# plot a histogram of the sampled data
num_bins = 20
plt.hist(data, bins=num_bins)
plt.show()
```




```
# parameters for the prior on mu
mu_mean_prior = 8.
mu_sd_prior = 1.5  # Note this is our prior on the std of mu

sigma_mean_prior = 1.
sigma_sd_prior = 1.

with pm.Model() as two_param_model:

    # Priors for unknown model parameters
    mu = pm.Normal('Mean of Data', mu_mean_prior, mu_sd_prior)
    sigma = pm.Normal('SD of Data', sigma_mean_prior, sigma_sd_prior)

    # Likelihood (sampling distribution) of observations
    data_in = pm.Normal('Y_obs', mu=mu, sigma=sigma, observed=data)
```

```
chain_length = 10000

with two_param_model:
    # obtain starting values via MAP
    startvals = pm.find_MAP(model=two_param_model)
    print(startvals)
    # instantiate sampler
    step = pm.Metropolis()

    # draw 10000 posterior samples
    trace_two_param = pm.sample(chain_length, step=step, start=startvals,
                                return_inferencedata=True)
```

100.00% [12/12 00:00<00:00 logp = -2,503.2, ||grad|| = 0.035828]

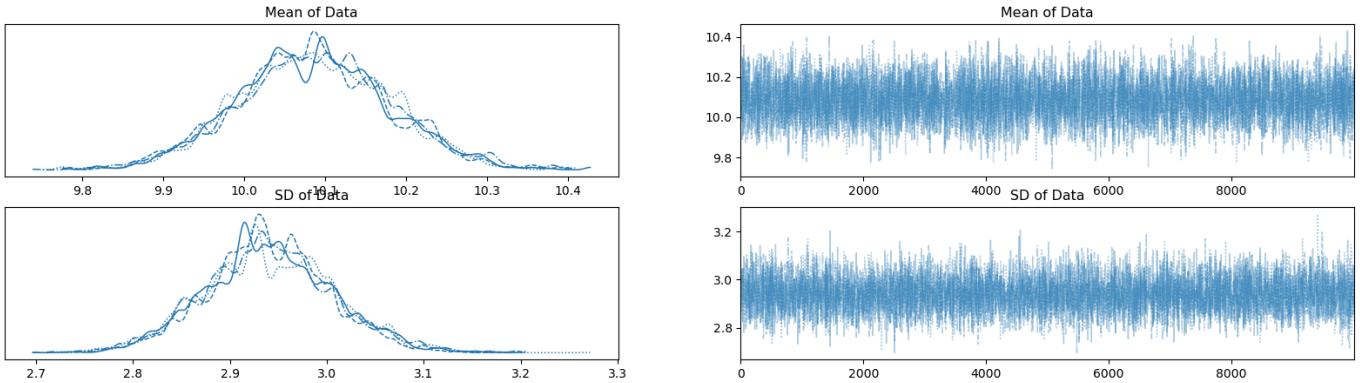
```
{'Mean of Data': array(10.08854313), 'SD of Data': array(2.93392367)}
```

```
/var/folders/1s/3wbqg65x5k79l5ww8fg25yvh0000gn/T/ipykernel_15896/2453781259.py:11:
FutureWarning: The `start` kwarg was renamed to `initvals` and can now do more. Please check the
docstring.
  trace_two_param = pm.sample(chain_length, step=step, start=startvals,
Multiprocess sampling (4 chains in 4 jobs)
CompoundStep
>Metropolis: [Mean of Data]
>Metropolis: [SD of Data]
```

100.00% [44000/44000 00:03<00:00 Sampling 4 chains, 0 divergences]

```
Sampling 4 chains for 1_000 tune and 10_000 draw iterations (4_000 + 40_000 draws total) took 31
seconds.
```

```
with two_param_model:
    az.plot_trace(trace_two_param, figsize=(20,5));
```



```
# NOTE: currently there is an issue with geweke. Try again in the future.
# score=pm.geweke(trace_two_param, first=0.1, last=0.5, intervals=20)
# plt.scatter(score[0]['Mean of Data'][:,0],score[0]['Mean of Data'][:,1],
#             marker = 'o', s=100)
# plt.axhline(-1.98, c='r')
# plt.axhline(1.98, c='r')
# plt.ylim(-2.5,2.5)
# plt.xlim(0-10,.5*trace['Mean of Data'].shape[0]/2+10)
# my_title = 'Geweke Plot Comparing first 10% and Slices of the Last 50%' +\
#            ' of Chain\nDifference in Mean Z score'
# plt.title(my_title)
# plt.show()
```

```
# NOTE: currently there is an issue with geweke. Try again in the future.
# score=pm.geweke(trace_two_param, first=0.1, last=0.5, intervals=20)
# plt.scatter(score[0]['SD of Data'][:,0],score[0]['SD of Data'][:,1], marker = 'o', s=100)
# plt.axhline(-1.98, c='r')
# plt.axhline(1.98, c='r')
# plt.ylim(-2.5,2.5)
# #plt.xlim(0-10,.5*trace['SD of Data'].shape[0]/2+10)
# plt.title('Geweke Plot Comparing first 10% and Slices of the Last 50% of Chain\nDifference in
SD Z score')
# plt.show()
```

23.6. Bilby

Demo to be added. See bilby.

23.7. Comparing samplers for a simple problem

We'll use the previous notebook on "Parameter estimation example: Gaussian noise and averages".

We used the affine-invariant sampler standalone `emcee` before. Here we'll compare to using another standalone sampler, `zeus`, and the probabilistic programming language `PyMC`.

Original background

Adapted from Sivia: *Data Analysis: A Bayesian Tutorial*

Here we'll take a look at a simple parameter-estimation problem. We will compare the frequentist and Bayesian approaches. This problem is an extended version of Example 2 in Ch 2.3 of the book by Sivia. This short book is very readable and is highly recommended.

This notebook was adapted from a broad introduction to Bayesian statistics put together by Christian Forssen for the 2019 TALENT school on "Learning from Data".

Common set up and generating data for all samplers

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

from scipy import stats

# Not really needed, but nicer plots
# import seaborn as sns
# sns.set()
# sns.set_context("talk")
```

Let us consider the problem of estimating the mean and the variance of a normal distribution that is associated with a collection of random variables. The normal distribution $$p(x|\mu,\sigma) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$ is often used as a theoretical model to describe the noise associated with experimental data.

Why is a normal (Gaussian) distribution so often a good statistical model?

Let us assume that we have a series of M measurements $D \equiv \{x_k\} = (x_1, \ldots, x_M)$, that are samples from a normal $\mathcal{N}(\mu, \sigma^2)$ population, from which we would like to learn the approximate values of the parameters μ and σ. The standard frequentist approach to this problem is the maximum likelihood method. The Bayesian approach is to compute the posterior distribution for the model parameters μ and σ.

Here we'll use Python to generate some toy data to demonstrate the two approaches to the problem.

```
# Generating some data
np.random.seed(10)  # for repeatability

mu_true = 10   # true peak position
sigma_true = 1 # true standard deviation
M = 100 # number of measurements
D = stats.norm.rvs(mu_true, sigma_true, size=M)  # M measurements (samples)
```

Side note: try shift-tab-tab to get documentation!

Look at the array `D`*. Are the number of entries in the "tails" what you would expect?*

```
D
```

```
array([11.3315865 , 10.71527897,  8.45459971,  9.99161615, 10.62133597,
        9.27991444, 10.26551159, 10.10854853, 10.00429143,  9.82539979,
       10.43302619, 11.20303737,  9.03493433, 11.02827408, 10.22863013,
       10.44513761,  8.86339779, 10.13513688, 11.484537  ,  8.92019511,
        8.02227172,  8.2566277 , 10.26607016, 12.38496733, 11.12369125,
       11.67262221, 10.09914922, 11.39799638,  9.72875201, 10.61320418,
        9.73268281,  9.45069099, 10.1327083 ,  9.52385799, 11.30847308,
       10.19501328, 10.40020999,  9.66236766, 11.25647226,  9.2680305 ,
       10.66023155,  9.64912811,  9.06056664,  9.51066278,  9.19540886,
        9.78730236,  9.66085975, 10.31216994, 10.56515267,  9.85257974,
        9.97409466, 10.2890942 ,  9.46012093, 10.70816002, 10.84222474,
       10.2035808 , 12.39470366, 10.91745894,  9.88772753,  9.63781955,
        9.76781774,  9.4982711 , 11.12878515,  9.30218997,  9.91887782,
```

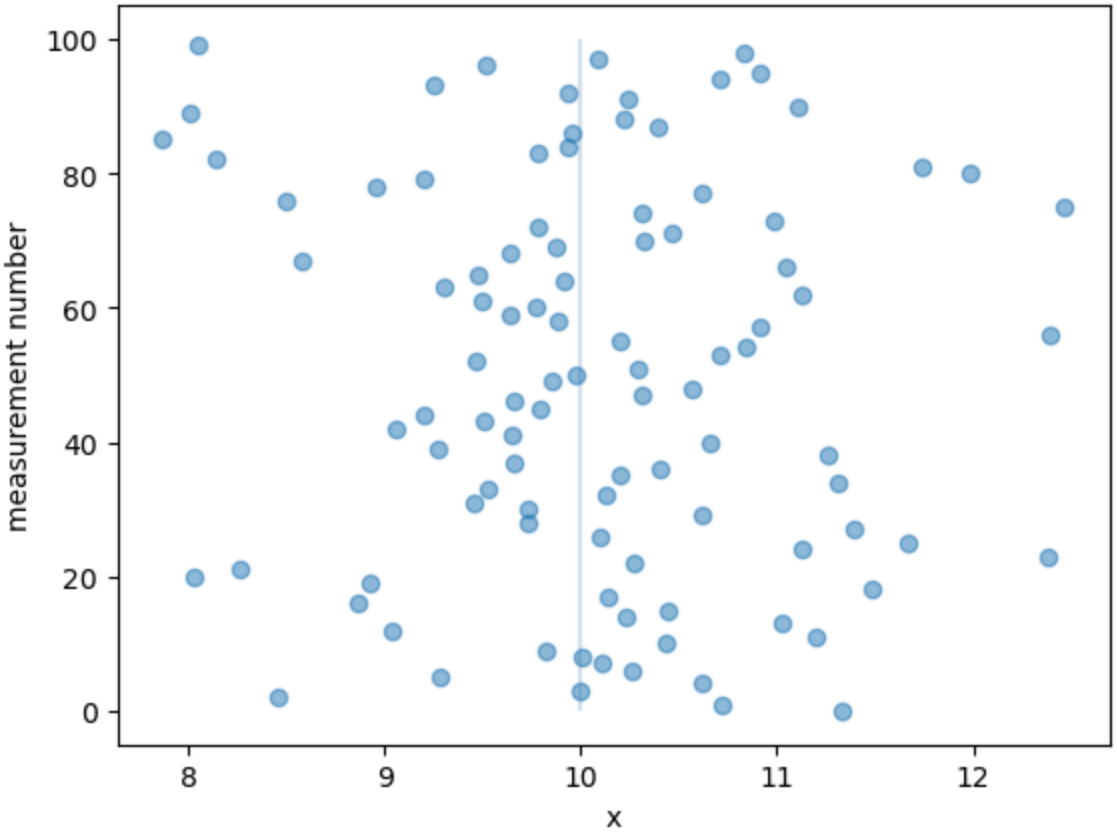
```
 9.47070392, 11.04618286,  8.58144397,  9.63750082,  9.87809431,
10.31935642, 10.4609029 ,  9.78421011, 10.98907246, 10.31475378,
12.46765106,  8.49167851, 10.62060066,  8.95486746,  9.20199118,
11.98508459, 11.74481415,  8.14381452,  9.7772263 ,  9.93415215,
 7.86828789,  9.95116949, 10.39334122, 10.21726515,  8.00560623,
11.10770823, 10.24454398,  9.93808797,  9.24610704, 10.71195902,
10.91826915,  9.51790686, 10.08958761, 10.82699862,  8.04548788])
```

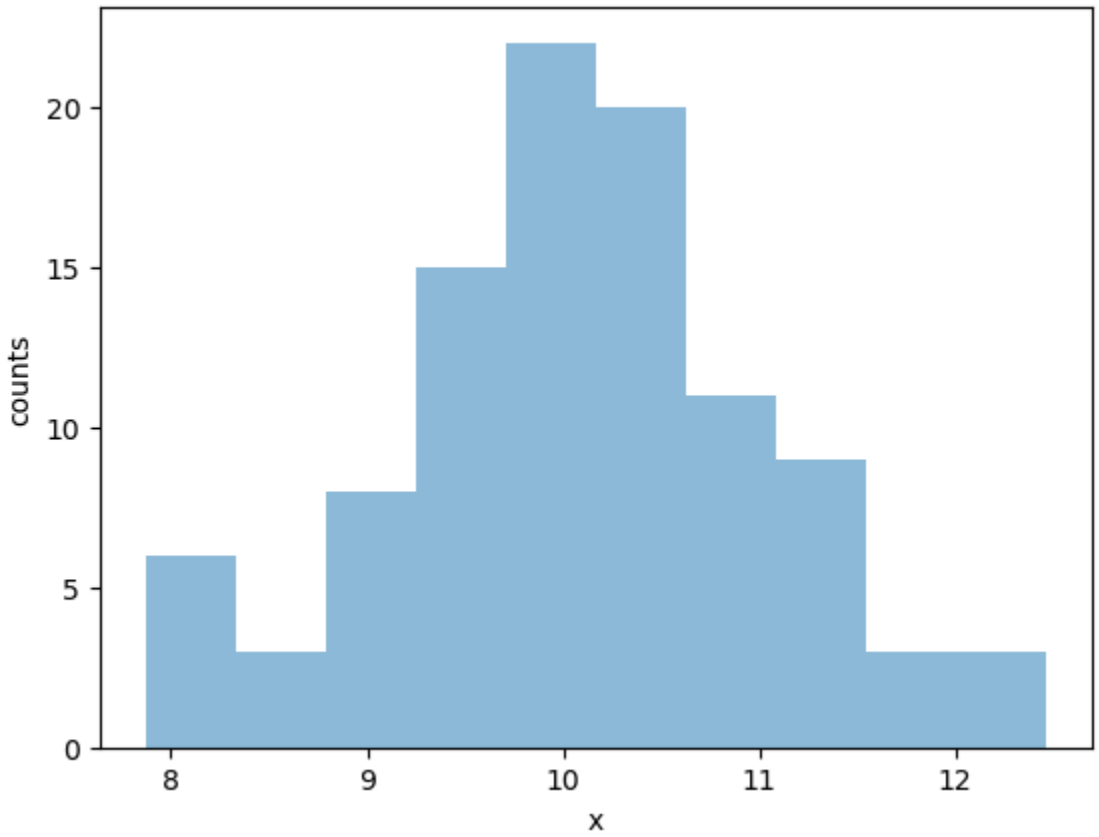
Hint: Roughly how many entries do you expect to find two sigma away from the mean?

Let's verify your answer with visualizations!

First, let's make two simple visualizations of the "measured" data: a scatter plot and a histogram

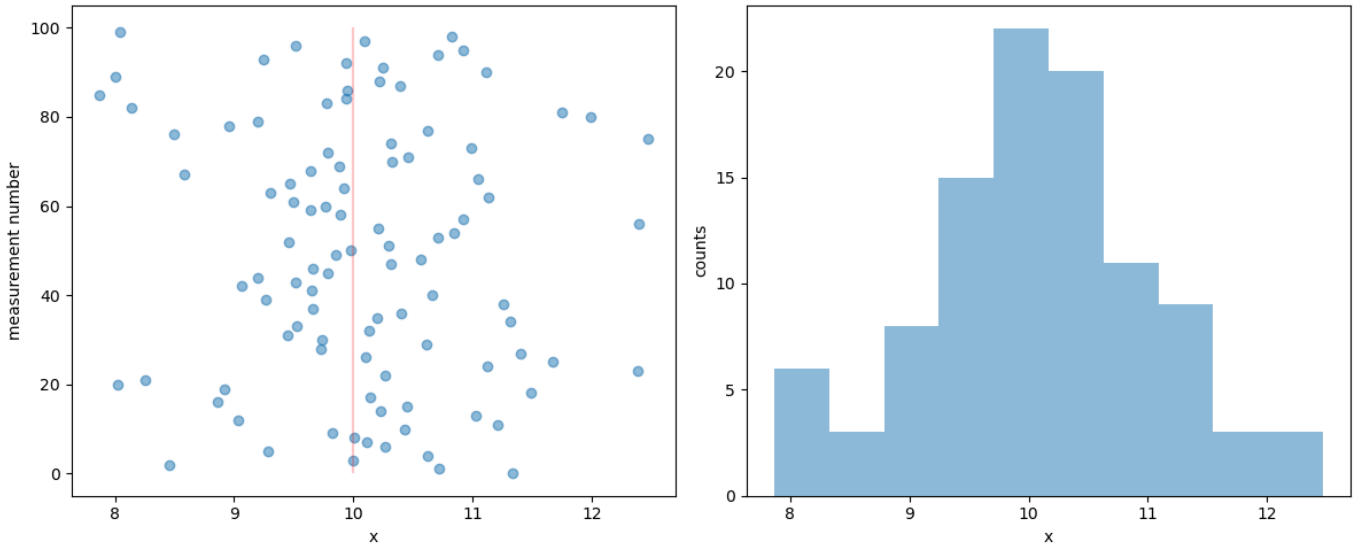
```
fig1, ax1 = plt.subplots()
ax1.scatter(D, np.arange(M), alpha=0.5)
ax1.vlines([mu_true], 0, M, alpha=0.2)
ax1.set_xlabel("x"); ax1.set_ylabel("measurement number");

fig2, ax2 = plt.subplots()
ax2.hist(D,alpha   =0.5)
#ax1.vlines([mu_true], 0, M, alpha=0.2)
ax2.set_xlabel("x"); ax2.set_ylabel("counts");
```





`Matplotlib` alternative: one figure with two subplots using an array of axes.

```
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12,5))
# An alternative keeping the same axis names as before would be:
#  fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(12,5))

axes[0].scatter(D, np.arange(M), alpha=0.5)
axes[0].vlines([mu_true], 0, M, color='red', alpha=0.2)
axes[0].set_xlabel("x"); axes[0].set_ylabel("measurement number");

axes[1].hist(D,alpha=0.5)
axes[1].set_xlabel("x"); axes[1].set_ylabel("counts");

fig.tight_layout()
```



What can you conclude about the tails?

Change np.random.seed(1) to a different number so that a different set of random numbers is generated.

Frequentist approach to Gaussian parameter estimation

We'll start with the classical frequentist **maximum likelihood** approach. The probability of a single measurement D_i having a value x_i is given by

$$p(x_i \mid \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[\frac{-(x_i - \mu)^2}{2\sigma^2}\right],$$

where μ, σ are the *true* values for the model parameters.

We construct the **likelihood function** by computing the product of the probabilities for each data point:

$$\mathcal{L}(D \mid \mu, \sigma) = \prod_{i=1}^{M} p(x_i \mid \mu, \sigma)$$

Because the value of the likelihood can become very small, it is often more convenient to instead compute the log-likelihood. Furthermore, when we are looking for the maximum of the likelihood, we might just as well maximize the log-likelihood. Combining the previous two equations and computing the log, we have

$$\log \mathcal{L} = -\frac{1}{2} \sum_{i=1}^{M} \left[\log(2\pi\sigma^2) + \frac{(x_i - \mu)^2}{\sigma^2} \right]$$

What we'd like to do is find μ_0, σ_0 such that the likelihood (or log likelihood) is maximized. For this simple problem, the maximization can be computed analytically (i.e. by setting $\partial \log \mathcal{L}/\partial\mu|_{\mu_0,\sigma_0} = \partial \log \mathcal{L}/\partial\sigma|_{\mu_0,\sigma_0} = 0$). This results in the following *maximum-likelihood estimates* of the true parameters:

$$\mu_0 = \frac{1}{M} \sum_{i=1}^{M} x_i$$

$$\sigma_0^2 = \frac{1}{M} \sum_{i=1}^{M} (x_i - \mu_0)^2$$

In principle, we should also compute the second derivatives to make sure that this point represents a maximum rather than a minimum or a saddle point. However, in agreement with intuition, μ_0 is simply the mean of the observed data. These results are fairly simple calculations; let's evaluate them for our toy dataset:

```
# Estimators: sample mean and (sqrt of) sample variance
# Sometimes the *unbiased* estimator for the sample variance is used
# with (M-1) degrees of freedom...
mu_est = D.sum()/M
sigma_est = np.sqrt(((D-mu_est)**2).sum()/M)
print("""
      (mu,sigma)_true = {0:.2f}, {1:.2f}

      Sample estimators:
      (mu,sigma)_0  = {2:.2f}, {3:.2f} (based on {4} measurements)
      """.format(mu_true, sigma_true, mu_est, sigma_est, M))
```

```
      (mu,sigma)_true = 10.00, 1.00

      Sample estimators:
      (mu,sigma)_0  = 10.08, 0.97 (based on 100 measurements)
```

Aside: using `fstrings` by putting an `f` in front of the quotes.

```
# Estimators: sample mean and (sqrt of) sample variance
# Sometimes the *unbiased* estimator for the sample variance is used with
# (M-1) degrees of freedom...
mu_est = D.sum()/M
sigma_est = np.sqrt(((D-mu_est)**2).sum()/M)
print(f"""
      (mu,sigma)_true = {mu_true:.2f}, {sigma_true:.2f}

      Sample estimators:
      (mu,sigma)_0 = {mu_est:.2f}, {sigma_est:.2f} (based on {M} measurements)
      """)
```

```
(mu,sigma)_true = 10.00, 1.00

Sample estimators:
(mu,sigma)_0 = 10.08, 0.97 (based on 100 measurements)
```

Bayesian approach to Gaussian parameter estimation

The Bayesian approach begins and ends with probabilities (pdfs). It recognizes that what we fundamentally want to compute is our knowledge of the parameters in question, i.e. in this case,

$$p(\mu, \sigma \mid D, I)$$

Note that this formulation of the problem is *fundamentally contrary* to the frequentist philosophy, which says that *probabilities have no meaning for model parameters* like μ, σ. Nevertheless, within the Bayesian philosophy this is perfectly acceptable.

To compute this result, Bayesians next apply Bayes' Theorem, here with the hypothesis being the Gaussian model expressed in terms of the model parameters

$$p(\mu, \sigma \mid D, I) = \frac{p(D \mid \mu, \sigma, I)\, p(\mu, \sigma \mid I)}{p(D \mid I)}$$

Though Bayes' theorem is where Bayesians get their name, it is not this law itself that is controversial, but the Bayesian *interpretation of probability* implied by the term $p(\mu, \sigma \mid D, I)$.

Let's take a look at each of the terms in this expression:

- $p(\mu, \sigma \mid D, I)$: The **posterior**, or the probability of the model parameters given the data: this is the result we want to compute.
- $p(D \mid \mu, \sigma, I)$: The **likelihood**, which is proportional to the $\mathcal{L}(D \mid \mu, \sigma)$ in the frequentist approach, above.
- $p(\mu, \sigma \mid I)$: The **model prior**, which encodes what we knew about the model prior to the application of the data D.
- $p(D \mid I)$: The **data probability**, which in practice amounts to simply a normalization term.

If we set the prior $p(\mu, \sigma \mid I) \propto 1$ (a *flat prior*), we find

$$p(\mu, \sigma \mid D, I) \propto \mathcal{L}(D|\mu, \sigma)$$

and the Bayesian probability is maximized at precisely the same value as the frequentist result! So despite the philosophical differences, we see that (for this simple problem at least) the Bayesian and frequentist point estimates are equivalent.

But what about the prior?

You'll noticed that we glossed over something here: the prior, $p(\mu, \sigma)$. The choice of prior will be discussed repeatedly in the course, but we can already note that it allows inclusion of other information into the computation. This feature becomes very useful in cases where multiple measurement strategies are being combined to constrain a single model. The necessity to specify a prior, however, is one of the more controversial pieces of Bayesian analysis.

A frequentist will point out that the prior is problematic when no true prior information is available. Though it might seem straightforward to use a **noninformative prior** like the flat prior mentioned above, there are some surprisingly subtleties involved. It turns out that in many situations, a truly noninformative prior does not exist! Frequentists point out that the subjective choice of a prior which necessarily biases your result has no place in statistical data analysis.

A Bayesian would counter that frequentism doesn't solve this problem, but simply skirts the question. Frequentism can often be viewed as simply a special case of the Bayesian approach for some (implicit) choice of the prior: a Bayesian would say that it's better to make this implicit choice explicit, even if the choice might include some subjectivity.

Applying the Bayesian approach

Leaving these philosophical debates aside for the time being, let's address how Bayesian results are generally computed in practice. For a one parameter problem like the one considered here, it's as simple as computing the posterior probability $p(\mu, \sigma \mid D, I)$ as a function of μ, σ: this is the distribution reflecting our knowledge of the parameter μ, σ. But as the dimension of the model grows, this direct approach becomes increasingly intractable. For this reason, Bayesian calculations often depend on sampling methods such as Markov Chain Monte Carlo (MCMC).

We won't go into the details of the theory of MCMC here (again, we will be discussing this topic during the course). Instead we will show a practical example of applying an MCMC approach using Dan Foreman-Mackey's excellent emcee package. Keep in mind here that the goal is to *generate a set of points drawn from the posterior probability distribution*, and to use those points to determine the answer we seek.

To perform this MCMC, we start by defining Python functions for the prior $p(\mu, \sigma \mid I)$, the likelihood $p(D \mid \mu, \sigma, I)$, and the posterior $p(\mu, \sigma \mid D, I)$, noting that none of these need be properly normalized. Our model here is two-dimensional so we'll define the model in terms of an array of parameters θ, which in this case is $\theta = [\mu, \sigma]$:

```
min_theta=np.array([0,0])
max_theta=np.array([100,10])
volume_theta=np.prod(max_theta-min_theta)

def log_prior(theta):
    '''Log prior for parameter array theta'''
    assert len(theta)==2, "Parameter vector must have length two."
    # flat prior
    if np.logical_and(min_theta<=theta, theta<=max_theta).all():
        return np.log(1/volume_theta)
    else:
        return -np.inf

def log_likelihood(theta, X):
    '''Log likelihood for data X given parameter array theta'''
    try:
        return -0.5 * np.sum( ( (X - theta[0]) / theta[1] )** 2 ) \
               - 0.5*len(X)*np.log(2*np.pi*theta[1]**2)
    except ValueError:
        return -np.inf

def log_posterior(theta, X):
    '''Log posterior for data X given parameter array theta'''
    return log_prior(theta) + log_likelihood(theta, X)
```

Now we set up the problem, including generating some random starting guesses for the multiple chains of points.

First sample with `emcee`

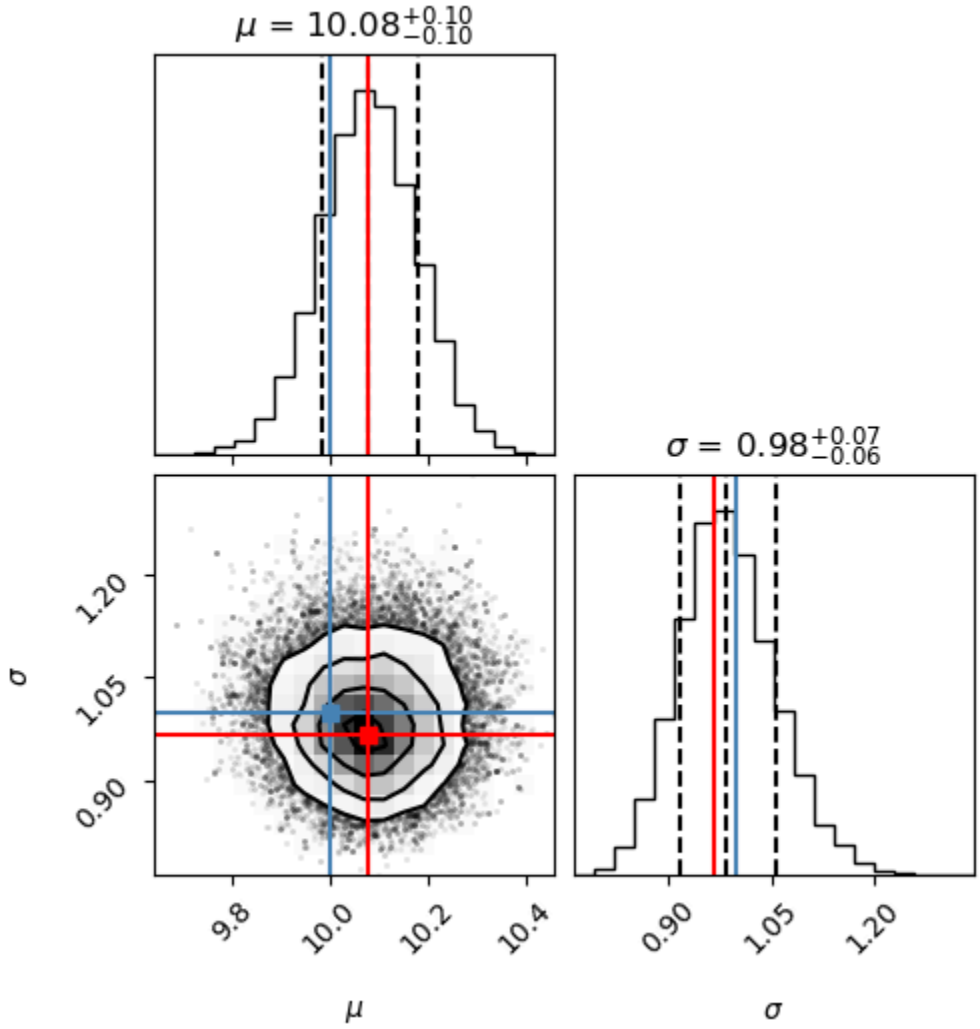
```
# Modules needed for example: emcee is for MCMCsampling, corner for plotting
import emcee
import corner  # can be used with any sampler
```

```
ndim = 2  # number of parameters in the model
nwalkers = 50  # number of MCMC walkers
nburn = 1000  # "burn-in" period to let chains stabilize
nsteps = 1000  # number of MCMC steps to take

# we'll start at random locations within the prior volume
starting_guesses = min_theta + \
                   (max_theta - min_theta) * np.random.rand(nwalkers,ndim)

print("MCMC sampling using emcee (affine-invariant ensamble sampler) with {0}
walkers".format(nwalkers))
sampler = emcee.EnsembleSampler(nwalkers, ndim, log_posterior, args=[D])

# "burn-in" period; save final positions and then reset
pos, prob, state = sampler.run_mcmc(starting_guesses, nburn)
sampler.reset()

# sampling period
sampler.run_mcmc(pos, nsteps)

print("Mean acceptance fraction: {0:.3f} (in total {1} steps)"
                .format(np.mean(sampler.acceptance_fraction),nwalkers*nsteps))

# discard burn-in points and flatten the walkers; the shape of samples is (nwalkers*nsteps,
ndim)
samples = sampler.get_chain(flat=True)
```

```
MCMC sampling using emcee (affine-invariant ensamble sampler) with 50 walkers
Mean acceptance fraction: 0.719 (in total 50000 steps)
```

If this all worked correctly, the array *samples* should contain a series of draws from the posterior. Let's plot them using a so called corner plot (to be discussed in much more detail during the course).

```
# make a corner plot with the posterior distribution
fig = corner.corner(samples, labels=["$\mu$", "$\sigma$"],
                    truths=[mu_true, sigma_true],quantiles=[0.16, 0.5, 0.84],
                    show_titles=True, title_kwargs={"fontsize": 12})

# With some manual efforts, we can add the maximum-likelihood estimate from
#  the frequentist analysis.
maxlike_results = (mu_est,sigma_est)

# First, extract the axes
axes = np.array(fig.axes).reshape((ndim, ndim))

# Then, loop over the diagonal
for i in range(ndim):
    ax = axes[i, i]
    ax.axvline(maxlike_results[i], color="r")

# And finally, loop over the histograms
for yi in range(ndim):
    for xi in range(yi):
        ax = axes[yi, xi]
        ax.axvline(maxlike_results[xi], color="r")
        ax.axhline(maxlike_results[yi], color="r")
        ax.plot(maxlike_results[xi], maxlike_results[yi], "sr")
```



The lower left panel in the corner plot shows the joint probability distribution for the two model parameters. The two plots on the diagonal show marginalized distributions (with one parameter integrated out) for the two model parameters. The blue lines indicate the *true* values of the model parameters, while the title and the dashed lines indicate the most probable value as well as a 68% confidence interval. We will be discussing these representative measures of a posterior distribution during the course.

The frequentist maximum-likelihood estimate from before is indicated by the red lines (that we added manually to the plot). As you can see, for this particular setup the maximum-likelihood estimate agrees with the most probable value from the MCMC sampling of the Bayesian posterior distribution.

The pure Bayesian result for a problem like this, however, would be to report the posterior distribution itself (i.e. its representative sample), and leave it at that. That is, in pure Bayesianism the answer to a question is not a single number with error bars; the answer is the posterior distribution over the model parameters!

Now sample with `zeus`

zeus is a slice sampler. See https://zeus-mcmc.readthedocs.io/en/latest/index.html.

```
import zeus
```

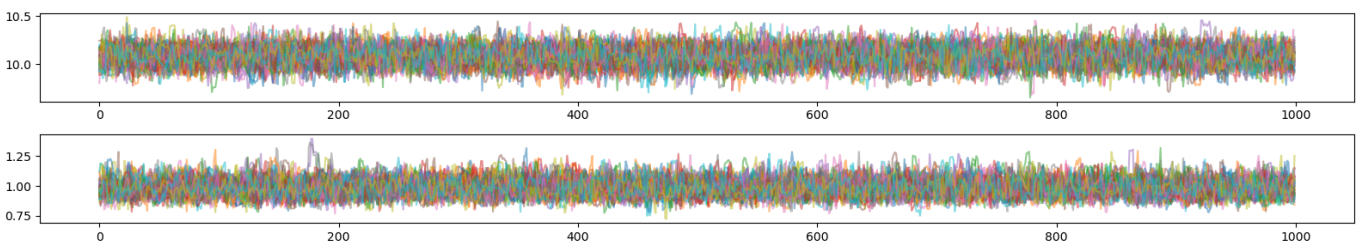
```
ndim = 2  # number of parameters in the model
nwalkers = 50  # number of MCMC walkers
nburn = 1000  # "burn-in" period to let chains stabilize
nsteps = 1000  # number of MCMC steps to take


# we'll start at random locations within the prior volume
starting_guesses = min_theta + \
                   (max_theta - min_theta) * np.random.rand(nwalkers,ndim)


# Do the burn-in with nburn steps
sampler = zeus.EnsembleSampler(nwalkers, ndim, log_posterior, args=[D])
sampler.run_mcmc(starting_guesses, nburn)

# Get the burn-in samples
burnin = sampler.get_chain()

# reset the start
starting_guesses = burnin[-1]

# now do the post-burn-in sampling with nsteps steps
sampler = zeus.EnsembleSampler(nwalkers, ndim, log_posterior, args=[D])
sampler.run_mcmc(starting_guesses, nsteps)
```

```
Initialising ensemble of 50 walkers...
Sampling progress : 100%|████████████████████| 1000/1000 [00:12<00:00, 78.43it/s]
Initialising ensemble of 50 walkers...
Sampling progress : 100%|████████████████████| 1000/1000 [00:05<00:00, 194.39it/s]
```

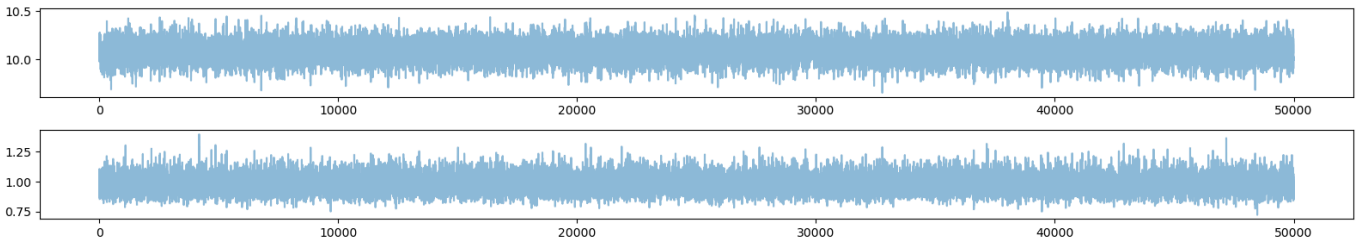
```
chain = sampler.get_chain(flat=True, discard=0, thin=1)
chain.shape
```

```
(50000, 2)
```

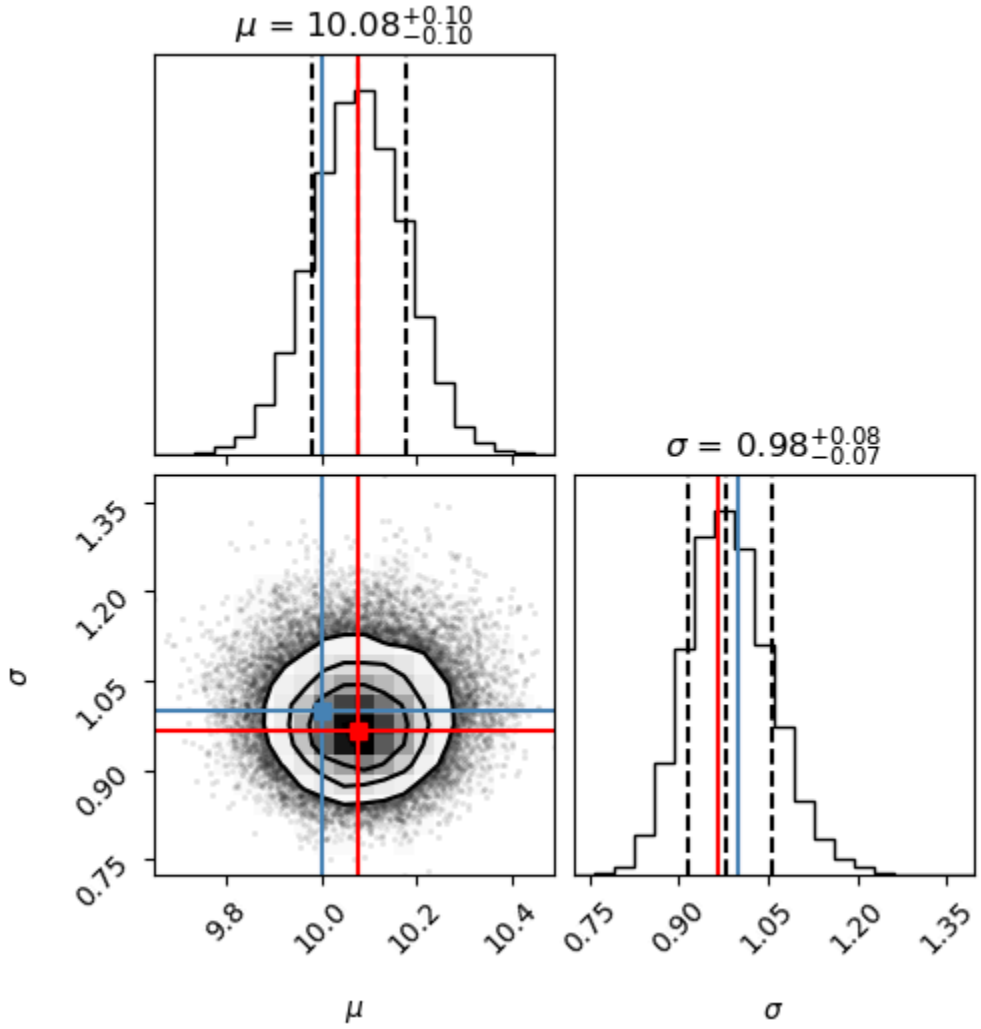
```
plt.figure(figsize=(16,1.5*ndim))
for n in range(ndim):
    plt.subplot2grid((ndim, 1), (n, 0))
    plt.plot(sampler.get_chain()[:,:,n],alpha=0.5)
    #plt.plot(chain[:,n],alpha=0.5)
    #plt.axhline(y=mu[n])
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(16,1.5*ndim))
for n in range(ndim):
    plt.subplot2grid((ndim, 1), (n, 0))
    #plt.plot(sampler.get_chain()[:,:,n],alpha=0.5)
    plt.plot(chain[:,n],alpha=0.5)
    #plt.axhline(y=mu[n])
plt.tight_layout()
plt.show()
```

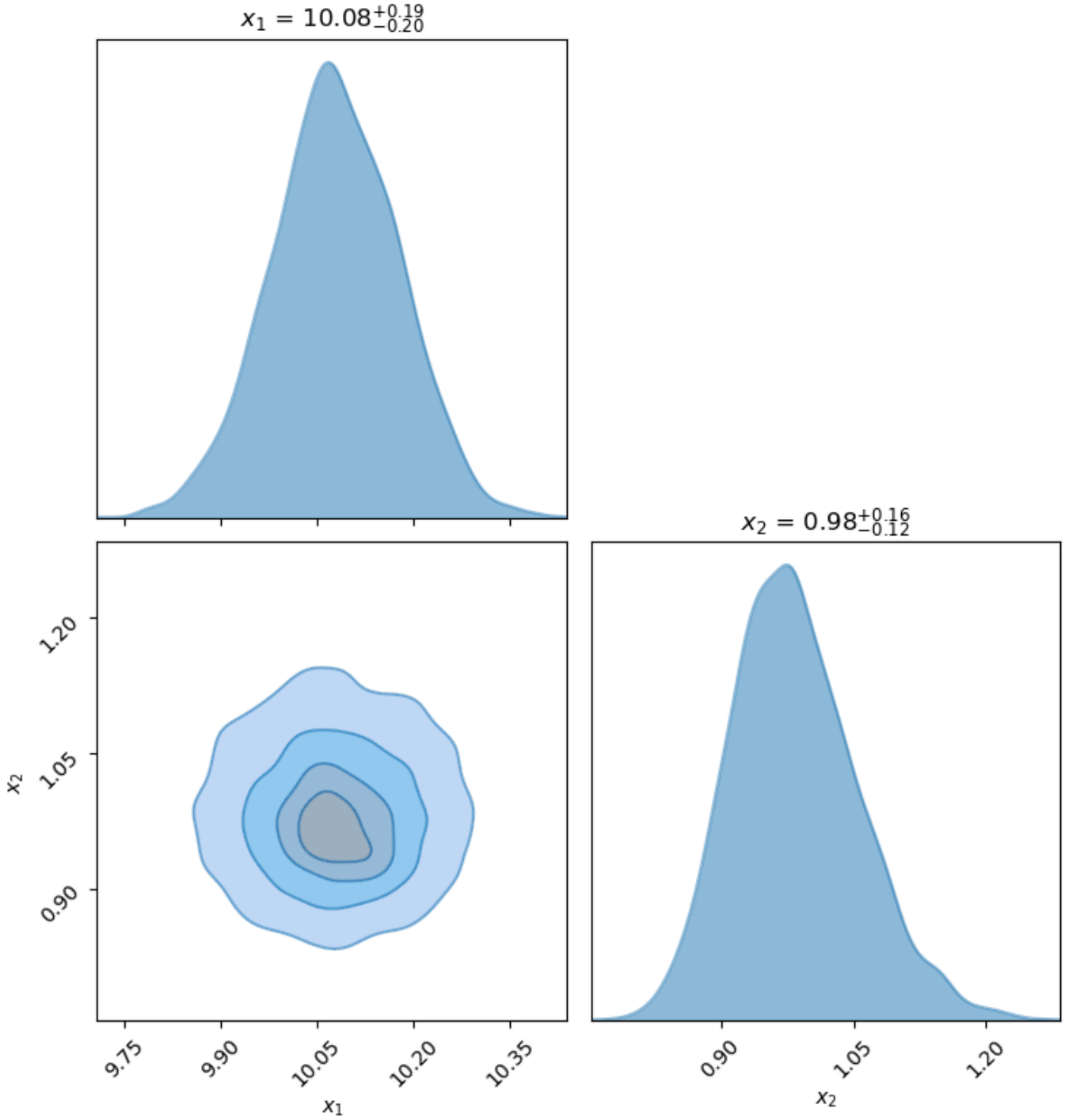



```
# make a corner plot with the posterior distribution
fig = corner.corner(chain, labels=["$\mu$", "$\sigma$"],
                    truths=[mu_true, sigma_true], quantiles=[0.16, 0.5, 0.84],
                    show_titles=True, title_kwargs={"fontsize": 12})

# With some manual efforts, we can add the maximum-likelihood estimate from
#  the frequentist analysis.
maxlike_results = (mu_est,sigma_est)

# First, extract the axes
axes = np.array(fig.axes).reshape((ndim, ndim))

# Then, loop over the diagonal
for i in range(ndim):
    ax = axes[i, i]
    ax.axvline(maxlike_results[i], color="r")

# And finally, loop over the histograms
for yi in range(ndim):
    for xi in range(yi):
        ax = axes[yi, xi]
        ax.axvline(maxlike_results[xi], color="r")
        ax.axhline(maxlike_results[yi], color="r")
        ax.plot(maxlike_results[xi], maxlike_results[yi], "sr")
```




```
# Try the zeus corner plot routine, using every 10
fig, axes = zeus.cornerplot(chain[::10], size=(8,8))
```

```
/Users/furnstah/Dropbox/Anaconda/anaconda3/envs/8820-env/lib/python3.10/site-
packages/seaborn/distributions.py:1176: UserWarning: The following kwargs were not used by
contour: 'linewidth'
  cset = contour_func(
/Users/furnstah/Dropbox/Anaconda/anaconda3/envs/8820-env/lib/python3.10/site-
packages/seaborn/distributions.py:1176: UserWarning: The following kwargs were not used by
contour: 'linewidth'
  cset = contour_func(
```




```
# If we use all the points, this will take a long time
# fig, axes = zeus.cornerplot(chain, size=(8,8))
```

Now sample with PyMC

The setup here is different.

```
import pymc as pm
import arviz as az

print('Running on PyMC v{}'.format(pm.__version__))
```

```
Running on PyMC v5.10.4
```

```
Gaussian_model = pm.Model()

with Gaussian_model:

    # Priors for mu and sigma
    mu = pm.Uniform("mu", lower=min_theta[0], upper=max_theta[0])
    sigma = pm.Uniform("sigma", lower=min_theta[0], upper=max_theta[0])

    # Likelihood (sampling distribution) of observations
    Y_obs = pm.Normal("Y_obs", mu=mu, sigma=sigma, observed=D)
```

```
map_estimate = pm.find_MAP(model=Gaussian_model)

map_estimate
```

100.00% [29/29 00:00<00:00 logp = -154.5, ||grad|| = 65.04]

```
{'mu_interval__': array(-2.18843153),
 'sigma_interval__': array(-4.62896866),
 'mu': array(10.07941624),
 'sigma': array(0.9670395)}
```

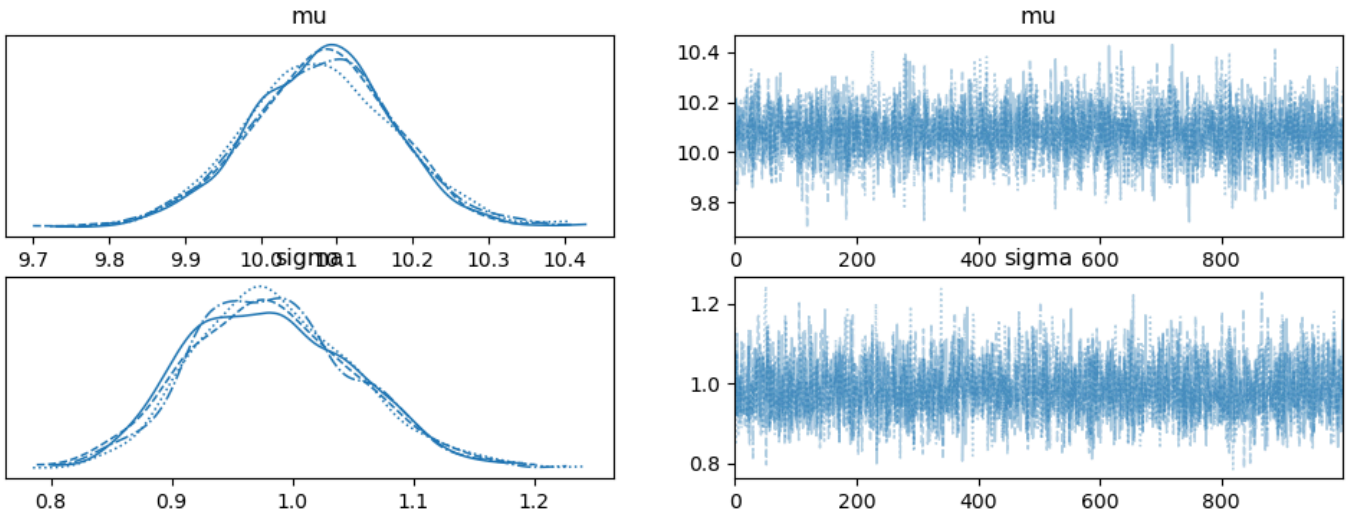
```
with Gaussian_model:
    trace_NUTS = pm.sample(1000, tune=1000, return_inferencedata=True)
    az.plot_trace(trace_NUTS);
    display(az.summary(trace_NUTS))
```

```
Auto-assigning NUTS sampler...
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [mu, sigma]
```

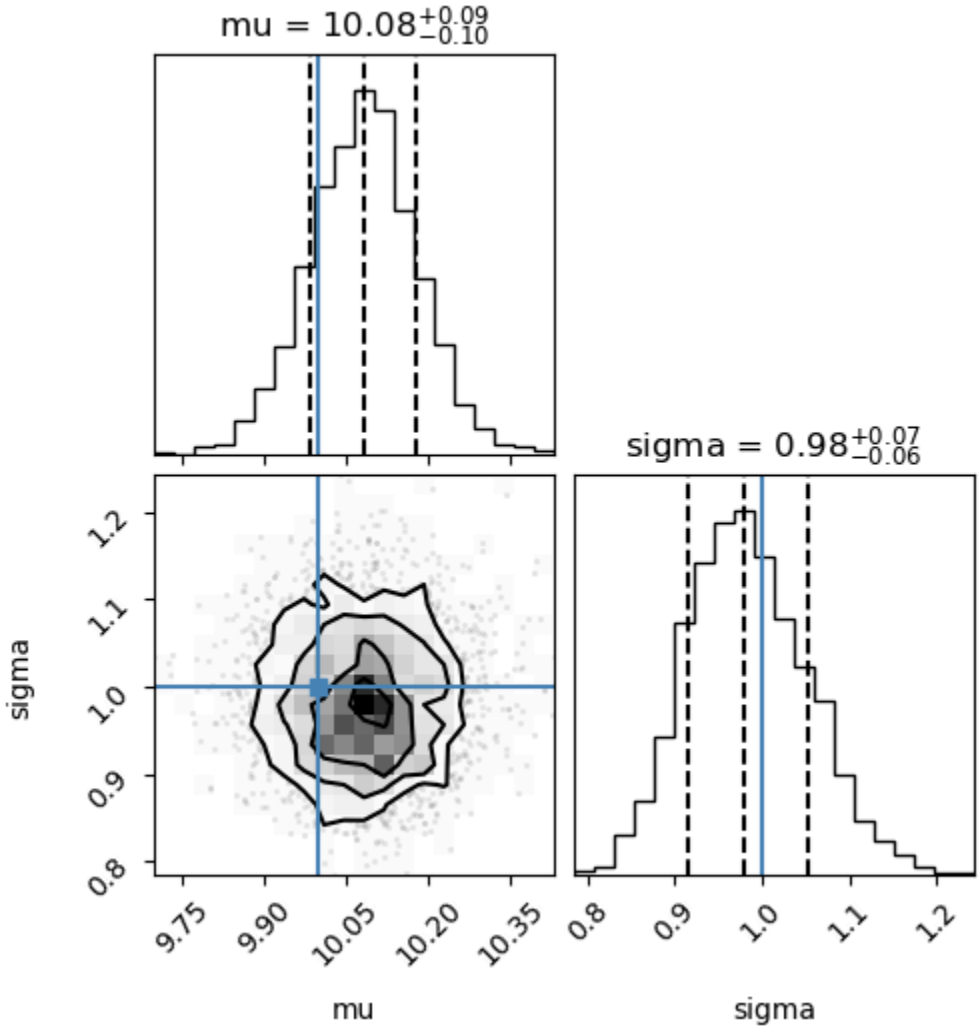
100.00% [8000/8000 00:00<00:00 Sampling 4 chains, 0 divergences]

```
Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 20 seconds.
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
mu	10.080	0.098	9.891	10.262	0.002	0.001	3970.0	2471.0	1.0
sigma	0.984	0.069	0.862	1.116	0.001	0.001	4161.0	3015.0	1.0

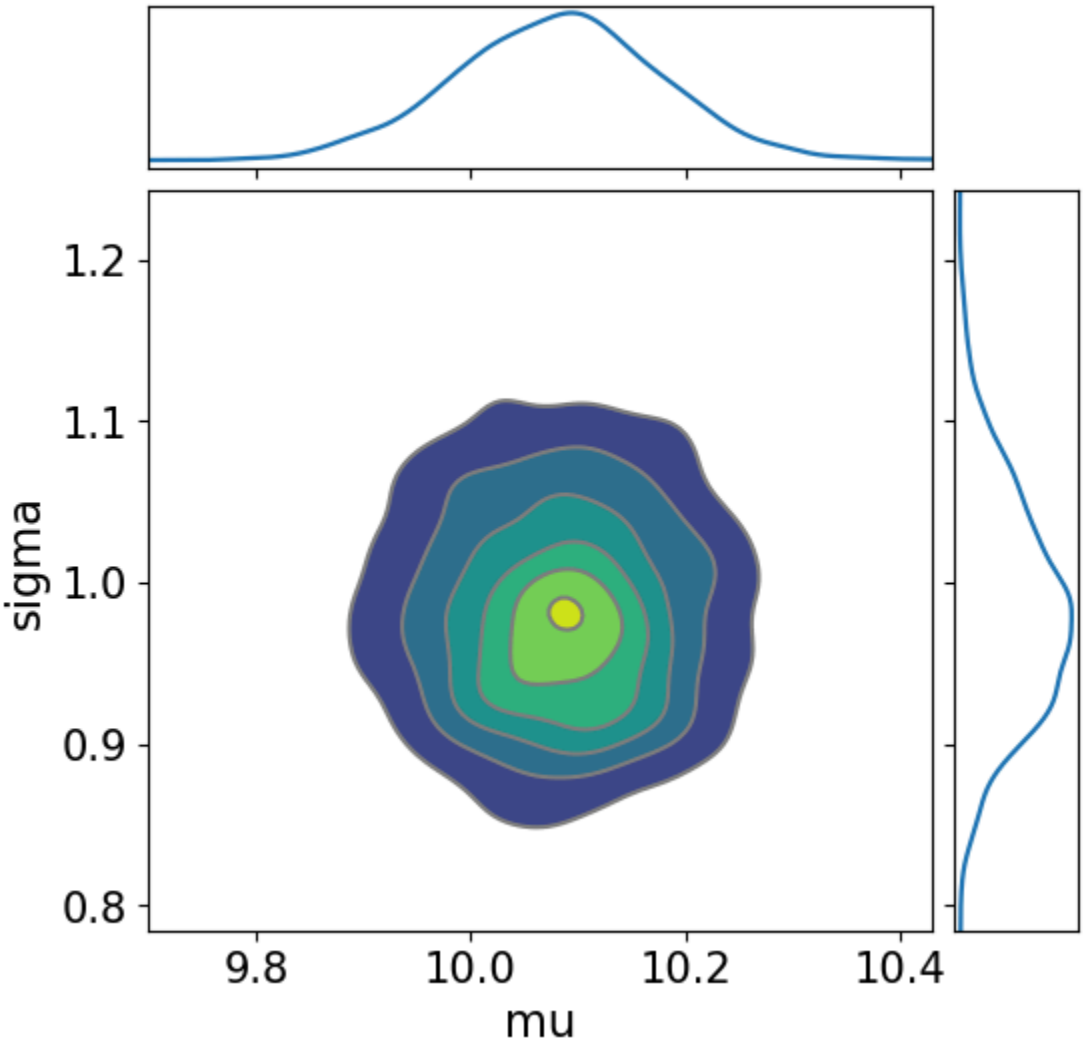



```
pymc_names = ["mu", "sigma"]
corner.corner(trace_NUTS, var_names=pymc_names, truths=[mu_true, sigma_true], quantiles=[0.16,
0.5, 0.84],
              show_titles=True);
```




```
with Gaussian_model:
    az.plot_pair(trace_NUTS,
            kind='kde', marginals=True,
            colorbar=True,
            figsize=(6, 6))
```

```
/Users/furnstah/Dropbox/Anaconda/anaconda3/envs/8820-env/lib/python3.10/site-
packages/arviz/plots/pairplot.py:232: FutureWarning: The return type of `Dataset.dims` will be
changed to return a set of dimension names in future, in order to be more consistent with
`DataArray.dims`. To access a mapping from dimension names to lengths, please use
`Dataset.sizes`.
  gridsize = int(dataset.dims["draw"] ** 0.35)
```



Discussion

Now, you might come away with the impression that the Bayesian method is unnecessarily complicated, and in this case it certainly is. Using any of these Markov Chain Monte Carlo samplers to characterize a two-dimensional normal distribution is a bit like using the Death Star to destroy a beach ball, but we did this here because it demonstrates an approach that can scale to complicated posteriors in many, many dimensions, and can provide nice results in more complicated situations where an analytic likelihood approach is not possible.

24. Overview of Part IV: Machine learning basics

> Machine learning (ML) is the study of computer algorithms that can improve automatically through experience and by the use of data. … A subset of machine learning is closely related to computational statistics, which focuses on making predictions using computers; but not all machine learning is statistical learning. … Machine learning and statistics are closely related fields in terms of methods, but

> distinct in their principal goal: statistics draws population inferences from a sample, while machine learning finds generalizable predictive patterns.
>
> —Wikipedia article on Machine Learning

> Machine learning (ML) is a type of artificial intelligence (AI) that allows software applications to become more accurate at predicting outcomes without being explicitly programmed to do so. Machine learning algorithms use historical data as input to predict new output values.
>
> —Ed Burns, TechTarget

We will be selective in our coverage, focusing on aspects that will ultimately connect to our Bayesian perspective. The chapters here are:

- Machine learning: Overview gives a general overview of machine learning: types of learning, data handling and ethical aspects, model validation, and data bias and fairness.
- Logistic Regression introduces binary and multi-class classification, with first machine-learning examples.
- Artificial neural networks covers the architecture of artificial neural networks (ANNs), the mathematical model of feed-forward networks, and training with back-propagation, with PyTorch demonstration notebooks.
- *Convolutional neural networks (or CNNs) are networks designed for image data.
- Problems for Part IV.

A non-empirical, field-theory-based treatment of ANNs that expands around the large-width limit is presented in Part V, see ANNs in the large-width limit.

25. Machine learning: Overview

Machine learning is about learning from data using computer programs. The algorithms in these programs are constructed to process data to find patterns or to make predictions or recommend actions. The automatic aspect of this process opens a wide range of application areas for machine learning techniques and algorithms. In particular, machine learning is used in computer technology but there has also been a swift and amazing development in the last decades that impact domains in the natural sciences, humanities, social sciences, medicine, law, etc. Indeed, almost all possible disciplines are affected. The applications are incredibly many, from self-driving cars to solving high-dimensional differential equations or complicated quantum mechanical many-body problems. Machine learning is perceived by many as a *disruptive technology*, i.e., implying that it will change our society.

Statistics, data science and machine learning form important fields of research in modern science. They describe how to learn and make predictions from data, as well as allowing us to find correlations between features in (large) data sets. Such big data sets now appear frequently in essentially all disciplines, from the traditional Science, Technology, Engineering and Mathematics (STEM) fields to Life Science, Law, Education research, the Humanities and the Social Sciences. Having a solid understanding of data analysis and machine learning is thus becoming central to scientific computing in many fields, and competences and skills within the fields of machine learning and scientific computing are nowadays strongly requested by many potential employers.

Machine learning is a subfield of computer science, and is closely related to computational statistics. It evolved from the study of pattern recognition in artificial intelligence (AI) research, and has made contributions to AI tasks like computer vision, natural language processing and speech recognition. Many of the methods we will study are also strongly rooted in basic mathematics and physics research.

Ideally, machine learning represents the science of giving computers the ability to learn without being explicitly programmed. The idea is that there exist generic algorithms which can be used to find patterns in a broad class of data sets without having to write code specifically for each problem. The algorithm will build its own logic based on the data. You should however always keep in mind that machines and algorithms are to a large extent developed by humans. The insights and knowledge we have about a specific system, play a central role when we develop a specific machine learning algorithm.

Three main ingredients of machine learning

A machine learning model learns from available training data. Thus, one can usually identify three ingredients of basically any machine learning application:

The data
The first, and often most important, ingredient is the data set. The data is usually split into *training, validation, and test sets*.
The mathematical model
The second ingredient is a model, which can be a function of some parameters. The complexity of the model can often be varied.
The learning algorithm
The final ingredient is the algorithm that is used for learning. A specific component of the *learning algorithm* is often a so-called *cost function*, which allows us to present an estimate on how good our model is in reproducing the data.
We will encounter several examples (and variations) of these building blocks in the next few chapters.

Machine learning is an extremely rich field, in spite of its young age. Increases in computational capabilities have been followed by developments of methods and techniques for analyzing and handling large data sets—relying heavily on statistics, computer science and mathematics, but also importing ideas from other disciplines such as physics. The field is rather new and developing rapidly. Popular software packages (many of them written in Python) are freely available at their respective GitHub sites and involve developer communities measured in the thousands or more. And the number of code developers and contributors keeps increasing.

Python for machine learning

Some very popular Python libraries for machine learning and probabilistic programming are

- Scikit-learn,
- PyTorch,
- TensorFlow and Keras (Keras 3 can also run on top of JAX or PyTorch),
- JAX,
- PyMC,
- NumPyro and Pyro.

In addition, Python is an excellent programming language for prototype construction and for data visualization (with extensive galleries of existing examples).

Still, it should be noted that Python is notoriously slow. Therefore, analysis codes tend to utilize compiled languages (C++,C, Fortran) and possibly hardware acceleration for the heavy computations.

Not all machine learning algorithms and methods can be given a rigorous mathematical justification, thereby opening up opportunities for experimenting, trial and error, and exciting new developments. However, a solid command of linear algebra, multivariate analysis, probability theory, statistical data analysis, Bayesian inference, understanding errors and Monte Carlo methods are central elements in a proper understanding of machine learning applications.

25.1. Types of Machine Learning

The approaches to machine learning are many, but are often split into two main categories. In *supervised learning* we claim to know the system under investigation and we use the computer to deduce the strengths of relationships and dependencies. On the other hand, *unsupervised learning* is used for finding patterns and relationship in data sets without any prior knowledge of the system. Some authors also operate with a third category, namely *reinforcement learning*. This is a paradigm of learning inspired by behavioral psychology, where learning is achieved by trial-and-error using a system of rewards and punishments. Here we will focus mainly on algorithms for supervised learning.

Another way to categorize machine learning is to consider the desired output. What kind of inference are you performing with your data? Is the aim to classify a result into categories, to predict a continuous response variable, or to simply observe patterns within the data? Let's briefly introduce different types of tasks:

Regression algorithms

aims to find a functional relationship between input and output (predictor and response). The goal is often to construct a function that maps input data to continuous output values. These algorithms also require labeled output.

Classification algorithms

are used to predict splitting of a data set into separate classes; binary or multiple. The outputs are discrete and represent target classes. Classification algorithms often undergo supervised training, which means they require labeled true output data.

Clustering algorithms

can also be used for classification or simply to observe data patterns. By observing structures of data within the feature space, clustering algorithms aim to identify clusters. Some algorithms of this type don't require output labels, making them unsupervised algorithms.

Dimensionality reduction algorithms

focuses on decreasing the number of features from your data set, identifying the most important predictor variables and preventing your models from overfitting. They are usually unsupervised.

Generative models

aims to find patterns and structures within a data set and turn these into a generative model that that can be used to create new data.

Sometimes, the data collection process automatically provides the labels used in supervised learning, but in some cases the labeling is in itself a painstaking task that involves manual labor.

In the natural sciences, where we often confront scientific models with observations, there is certainly a large interest in regression algorithms. However, there are also many examples where other classes of machine-learning algorithms are being used.

25.2. Data handling, machine learning and ethical aspects

The data collection (or selection) process in a machine learning pipeline has important ethical aspects. Undetected biases in the data set will propagate via the learning algorithm to the final model predictions. In physics, the model might be rather well understood, and we might have good control over the data generation process, but we should still develop a sound ethical attitude to the use, processing and analysis of data.

Another pressing ethical aspects deals with our approach to the scientific process. In particular, it is of utmost importance that scientific results are reproducible. In fact, reproducibility should be imprinted as part of the dialectics of science. Nowadays, with version control software like Git and various online repositories like Github, Gitlab etc, we can easily make our codes and data sets openly and easily accessible to a wide community. This service helps almost automagically to make our science reproducible. The large open-source development communities involved in Scikit-Learn, Tensorflow, Keras, etc, are all excellent examples of this. The codes can be tested and improved continuosly, helping thereby our scientific community at large in developing data analysis and machine learning tools. It is much easier today to gain traction and acceptance for making your science reproducible. From a societal stand, this is an important element since many of the developers are employees of large public institutions like universities and research labs.

Let us also add a disclaimer concerning the fantastic progress of machine learning technology. Even though we may dream of computers developing some kind of higher learning capabilities, at the end it is we (yes you reading these lines) who end up constructing and instructing, via various algorithms, the machine learning approaches.

For self-driving vehicles, where the standard machine learning algorithms discussed here enter into the software, there are stages where the human programmer must make choices. As an example, all carmakers have the safety of the driver and the accompanying passengers as their utmost priority. Consider the scenario where the programmer has to construct an *if* statement that decides in an accident scenario between crashing into a truck or steering into a group of bicyclists.

This leads to serious ethical aspects. Who is entitled to make such choices? Keep in mind that many of the algorithms you will encounter in this series of lectures, or that you will hear about later, are indeed based on simple programming instructions. And you are very likely to be one of the people who end up writing such a code. Thus, developing a sound ethical attitude is much needed. The example of the self-driving cars is just one of infinitely many cases where we have to make choices. Other domains where applications might have serious ethical aspects include the financial sector, law, and medicine.

We do not have the answers here, nor will we venture into a deeper discussions of these aspects, but you should think over these topics in a more overarching way. A statistical data analysis with its dry numbers and graphs meant to guide the eye, does not necessarily reflect the truth, whatever that is. As a scientist, and after a university education, you are supposedly a highly qualified citizen, with an improved critical view and understanding of the scientific method, and perhaps some deeper understanding of the ethics of science at large. Use these insights. You owe it to our society.

25.3. Model validation

In this lecture we will continue to explore linear regression and we will encounter several concepts that are common for machine learning methods. These concepts are:

- Model validation
- Overfitting and underfitting
- Bias-variance-tradeoff
- Regularization
- Model hyperparameters

The lecture is based and inspired by material in several good textbooks: in particular chapter 4 in Hands-On Machine Learning with Scikit-Learn and TensorFlow [Geron17] by Aurelien Geron, chapter 5 in the Python Data Science Handbook [Van16] by Jake VanderPlas and chapters 2-4 in The Elements of Statistical Learning [HTF09] by Trevor Hastie et al.

Over- and underfitting

Overfitting and underfitting are common problems in data analysis and machine learning. Both extremes are illustrated in Fig. 25.1.

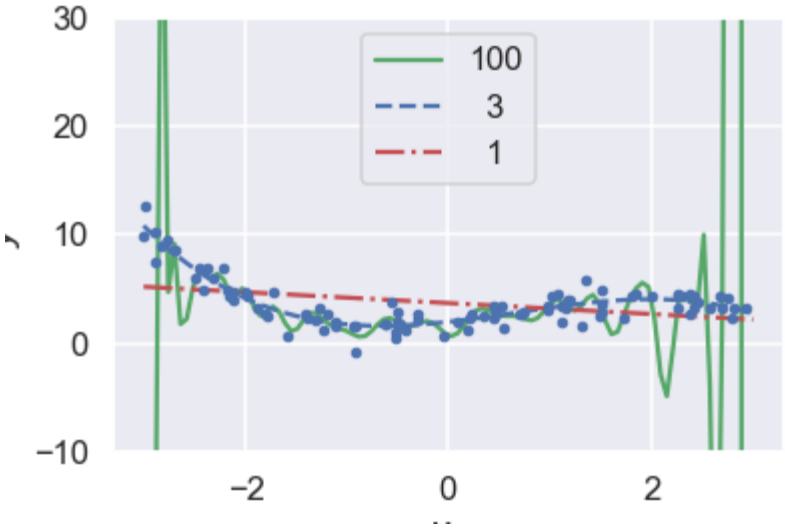


Fig. 25.1 The first-order polynomial model is clearly underfitting the data, while the very high degree model is overfitting it trying to reproduce variations that are clearly noise.

The following quote from an unknown source provides a concise definition of overfitting and underfitting:

> A model overfits if it fits noise as much as data and underfits if it considers variability in data to be noise while it is actually not.

The question is then: How do we detect these problems and how can we reduce them.

We can detect over- and underfitting by employing holdout sets, also known as *validation* sets. This means that we only use a fraction of the data for training the model, and save the rest for validation purposes.

I.e. we optimize the model parameters to best fit the training data, and then measure e.g. the mean-square error (MSE) of the model predictions for the validation set. The function that is used for measuring the performance of the model prediction is called an error function, and is here denoted $E(\hat{\boldsymbol{y}}, \boldsymbol{y})$. We stress that this function does not necessarily have to be the same as the cost function that is used for learning. The latter can incorporate other desirable features such as regularization to avoid overfitting (see below), or some domain-specific constraints. The error function, on the other hand, should focus on measuring the property of the prediction that is most important for the application at hand. Default choices for regression and classification tasks, respectively, are

Task	Error function	Measures
Regression	$E(\hat{y}, y) = (\hat{y} - y)^2$	Squared error
Classification	$E(\hat{y}, y) = 1 - \delta_{\hat{y},y}$	Misclassification

where $1 - \delta_{\hat{y},y} = 0$ if $\hat{y} = y$ (i.e. correct classification) and $= 1$ if $\hat{y} \neq y$ (misclassified).

Evaluating the error function on either the training set, or the validation set, gives relevant metrics that can be used to monitor under- or overfitting

$$
\begin{aligned}
E_{\text{train}} &= \frac{1}{N} \sum_{i=1}^{N} E(\hat{y}(\boldsymbol{x}_i), y_i), \quad \text{for } (\boldsymbol{x}_i, y_i) \in \mathcal{T} \\
E_{\text{val}} &= \frac{1}{N_{\text{val}}} \sum_{i=1}^{N_{\text{val}}} E(\hat{y}(\boldsymbol{x}_i), y_i), \quad \text{for } (\boldsymbol{x}_i, y_i) \in \textbf{data}_{\text{ val}}.
\end{aligned}
\tag{25.1}
$$

Training and validation scores

Several training and validation scores are commonly used in machine learning applications. First we have the **Mean-Squared Error** (MSE)

$$
\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}(\boldsymbol{x}_i) - y_i \right)^2, \tag{25.2}
$$

where we have N training data and our model is a function of the parameter vector $\boldsymbol{\theta}$. Note that this is the metric that we are minimizing when solving the normal equation Eq. (43.12).

Furthermore, we have the **mean absolute error** (MAE) defined as.

$$
\text{MAE} = \frac{1}{N} \sum_{i=1}^{N} \left| \hat{y}(\boldsymbol{x}_i) - y_i \right|, \tag{25.3}
$$

And the $R2$ score, also known as *coefficient of determination* is

$$\mathrm{R2} = 1 - \frac{\sum_{i=1}^{N} \left(\hat{y}(\boldsymbol{x}_i) - y_i\right)^2}{\sum_{i=1}^{N} \left(y_i - \bar{y}\right)^2}, \tag{25.4}$$

where $\bar{y} = \frac{1}{N}\sum_{i=1}^{N} y_i$ is the mean of the data. This metric therefore represents the proportion of variance (of **data**) that has been explained by the independent variables in the model.

An underfit model has a *high bias*, which means that it gives a rather poor fit and the error function will be rather large in average. This will be true for both the training and the validation sets.

An overfit model will depend sensitively on the choice of training data. A different split into training and validation sets will give very different predictions. We therefore say that the model displays a *high variance*. While the overfit model usually reproduces training data very well (low bias), it does not generalize well and gives a poor reproduction of validation data. The average values for the error function are therefore very different for the training and validation sets.

Another sign of overfitting is the appearance of very large fit parameters that are needed for the fine tunings of cancellations of different terms in the model. Polynomial fits of different orders to the data in Fig. 25.1 give the following root-mean-square parameter values

$$\theta_{\mathrm{rms}} \equiv \sqrt{\frac{1}{p}\sum_{i=0}^{p-1}\theta_i^2} = \|\boldsymbol{\theta}\|_2/\sqrt{p}. \tag{25.5}$$

order	θ_{rms}
1	3.0e-01
3	1.2e+00
100	6.3e+12

Note

In some cases a learning algorithm makes good predictions despite fitting to the noise in training data. This is called “benign overfitting”. See, for example, Bartlett et al., *Benign Overfitting in Linear Regression*. A recent explanation of this is Chatterji and Long, *Foolish Crowds Support Benign Overfitting*. From their abstract: “Our analysis exposes the benefit of an effect analogous to the ‘wisdom of the crowd’, except here the harm arising from fitting the *noise* is ameliorated by spreading it among many directions - the variance reduction arises from a *foolish* crowd.”

Regularization: Ridge and Lasso

Assuming that overfitting is characterized by large fit parameters, we can attempt to avoid this scenario by *regularizing* the model parameters. We will introduce two kinds of regularization: Ridge and Lasso. In addition, so

called elastic net regularization is also in use and basically corresponds to a linear combination of the Ridge and Lasso penalty functions.

Let us remind ourselves about the expression for the standard Mean Squared Error (MSE) which we used to define our cost function and the equations for the ordinary least squares (OLS) method. That is our optimization problem is

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}\in\mathbb{R}^p}{\operatorname{argmin}} \frac{1}{n}\Big\{ \left(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}\right)^T \left(\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}\right)\Big\}. \tag{25.6}$$

or we can state it as

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}\in\mathbb{R}^p}{\operatorname{argmin}} \frac{1}{n}\sum_{i=0}^{n-1} \left(y_i-\tilde{y}_i\right)^2 = \underset{\boldsymbol{\theta}\in\mathbb{R}^p}{\operatorname{argmin}} \frac{1}{n}||\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}||_2^2, \tag{25.7}$$

where we have used the definition of a norm-2 vector, that is

$$||\boldsymbol{\epsilon}||_2^2 = \sum_i \epsilon_i^2. \tag{25.8}$$

By minimizing the above equation with respect to the parameters $\boldsymbol{\theta}$ we could then obtain an analytical expression for the parameters $\boldsymbol{\theta}$. We can add a regularization parameter λ by defining a new cost function to be minimized, that is

$$C_{\lambda,2}\left(\boldsymbol{\theta}\right) \equiv ||\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}||_2^2 + \lambda||\boldsymbol{\theta}||_2^2 \tag{25.9}$$

(the normalization factor $1/n$ of the MSE has been absorbed into λ),

which leads to the *Ridge regression* minimization problem where we constrain the parameters via $||\boldsymbol{\theta}||_2^2$ and the optimization equation becomes

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}\in\mathbb{R}^p}{\operatorname{argmin}} C_{\lambda,2}\left(\boldsymbol{\theta}\right). \tag{25.10}$$

Alternatively, *Lasso regularization* can be performed by defining

$$C_{\lambda,1}\left(\boldsymbol{\theta}\right) \equiv ||\boldsymbol{y}-\boldsymbol{X}\boldsymbol{\theta}||_2^2 + \lambda||\boldsymbol{\theta}||_1. \tag{25.11}$$

Here we have defined the norm-1 as

$$||\boldsymbol{\epsilon}||_1 = \sum_i |\epsilon_i|. \tag{25.12}$$

Lasso stands for least absolute shrinkage and selection operator.

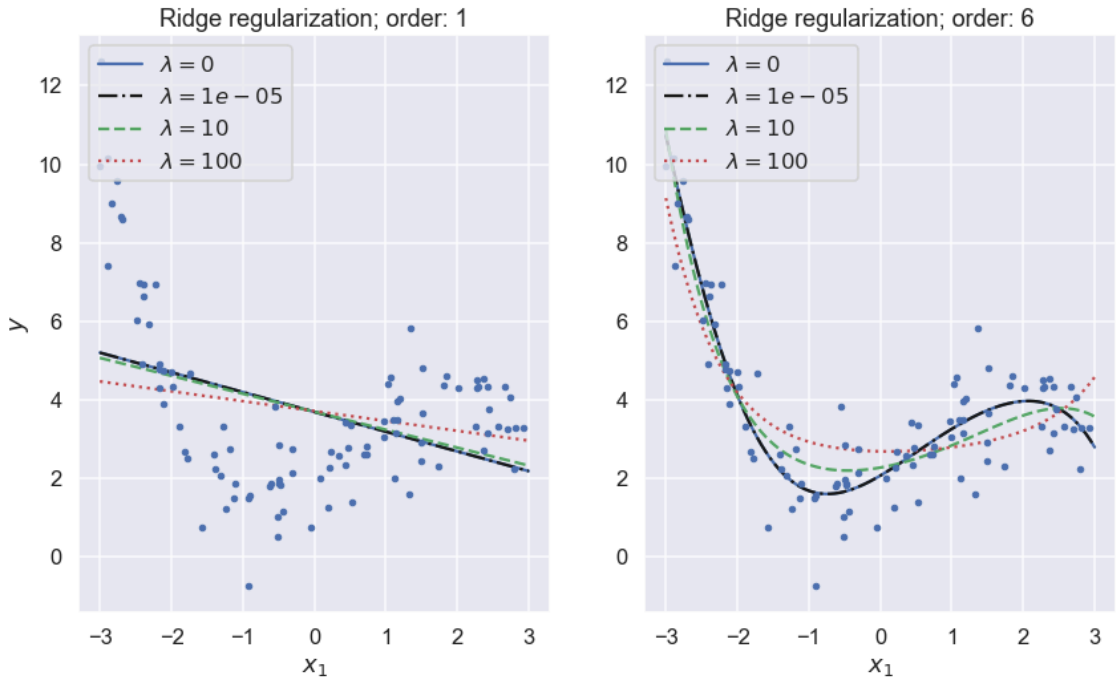


Fig. 25.2 Ridge regularization with different penalty parameters λ for different polynomial models of a noisy data set.

More on Ridge Regression

Using the matrix-vector expression for Ridge regression,

$$C_{\lambda,2}(\boldsymbol{\theta}) = (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta})^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}) + \lambda\boldsymbol{\theta}^T\boldsymbol{\theta}, \tag{25.13}$$

by taking the derivatives with respect to $\boldsymbol{\theta}$ we obtain then a slightly modified matrix inversion problem which for finite values of λ does not suffer from singularity problems. We obtain

$$\boldsymbol{\theta}^{\mathrm{Ridge}} = \left(\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{I}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y}, \tag{25.14}$$

with $\boldsymbol{I}$ being a $p \times p$ identity matrix

We see that Ridge regression is nothing but the standard OLS with a modified diagonal term added to $\boldsymbol{X}^T\boldsymbol{X}$. The consequences, in particular for our discussion of the bias-variance are rather interesting. Ridge regression imposes a constraint on the model parameters

$$\sum_{i=0}^{p-1}\theta_i^2 \leq t, \tag{25.15}$$

with t a finite positive number.

For more discussions of Ridge and Lasso regression, see: Wessel van Wieringen's [vW15] article or Mehta et al's [MBW+19] article.

The bias-variance tradeoff

We will discuss the bias-variance tradeoff in the context of continuous predictions such as regression. However, many of the intuitions and ideas discussed here also carry over to classification tasks.

Consider observations that are related to the true data-generating process f via experimental noise. For a specific observation i we can write

$$y_i = f(\boldsymbol{x}_i) + \epsilon_i, \tag{25.16}$$

where ϵ_i is an irreducible error described by a random variable. That is, even if we could find the data-generating process f we would not reproduce the data more accurately than ϵ permits. Let us assume that these errors are i.i.d. with expectation value $\mathbb{E}\left[\epsilon\right] = 0$ and variance Var $\left(\epsilon\right) = \sigma_\epsilon^2$[1].

Our model $\hat{y}(\boldsymbol{x})$ is an approximation to the function $f(\boldsymbol{x})$. We will arrive at the final model after calibration (training) and there is always a risk of either under- or overfitting. Let us now consider the following scenario:

- We make a prediction with our trained model at a new point $\boldsymbol{x}^\odot$. We use the shorthand notation $\hat{y}^\odot \equiv \hat{y}(\boldsymbol{x}^\odot)$.
- This prediction should eventually be compared with a future observation $y^\odot \equiv y(\boldsymbol{x}^\odot) = f(\boldsymbol{x}^\odot) + \epsilon^\odot \equiv f^\odot + \epsilon^\odot$
- Specifically, we are interested in the prediction error, $y^\odot - \hat{y}^\odot = f^\odot + \epsilon^\odot - \hat{y}^\odot$, to judge the predictive power of our model.

What can we say about this prediction error? It obviously depends on the inherent noise (ϵ), which is random, but also on the model complexity and how it was calibrated. Imagine the following repeated model calibrations:

1. Draw a size n sample, $\mathcal{T}_n = \{(\boldsymbol{x}_j, y_j), j = 1 \dots n\}$
   - Train our model $\hat{y}$ using $\mathcal{T}_n$.
   - Make the prediction at $\boldsymbol{x}^\odot$ and evaluate $y^\odot - \hat{y}^\odot$
2. Repeat this multiple times, using different sets of data $\mathcal{T}_n$ to fit your model. What is the expectation value $\mathbb{E}\left[\left(y^\odot - \hat{y}^\odot\right)^2\right]$?

Theorem 25.1 (The bias-variance tradeoff)

Consider a fixed input $\boldsymbol{x}^\odot$ and the prediction $\hat{y}^\odot$ of a model that has been trained on a randomly drawn training set. The prediction is compared with a future observation $y^\odot = f^\odot + \epsilon^\odot$, where the noise $\epsilon^\odot$ has zero mean and variance σ_ϵ^2, and is independent of the training data. The bias-variance tradeoff theorem states that the expected squared prediction error (with the expectation taken over the randomness in both training sets and the noise $\epsilon^\odot$) can be expressed as a sum

$$\mathbb{E}\left[\left(y^\odot - \hat{y}^\odot\right)^2\right] = \left(f^\odot - \mathbb{E}\left[\hat{y}^\odot\right]\right)^2 + \text{Var}\ \left(\hat{y}^\odot\right) + \sigma_\epsilon^2. \tag{25.17}$$

The first of the three terms represents the square of the bias of the machine-learning model, which can be thought of as the error caused by a lack of flexibility (underfitting) that inhibits our model to fully recreate f. The

second term represents the variance of the model predictions, which can be thought of as its sensitivity to the choice of training data (overfitting). Finally, the last term comes from the irreducible error ϵ that is inherent in the observations.

Proof. There are two sources of randomness: the training set, which makes the prediction $\hat{y}^\odot$ a random variable, and the noise $\epsilon^\odot$ of the future observation. The true value $f^\odot = f(\boldsymbol{x}^\odot)$ is a fixed number, and so is the expected prediction $\mathbb{E}\left[\hat{y}^\odot\right]$.

1. *Split the error.* Subtracting and adding $\mathbb{E}\left[\hat{y}^\odot\right]$, we write the prediction error as a sum of two terms

$$y^\odot - \hat{y}^\odot = \underbrace{\left(f^\odot - \mathbb{E}\left[\hat{y}^\odot\right] + \epsilon^\odot\right)}_{a} + \underbrace{\left(\mathbb{E}\left[\hat{y}^\odot\right] - \hat{y}^\odot\right)}_{b}.$$

   Squaring and using the linearity of expectation values gives

$$\mathbb{E}\left[\left(y^\odot - \hat{y}^\odot\right)^2\right] = \mathbb{E}\left[a^2\right] + \mathbb{E}\left[b^2\right] + 2\mathbb{E}\left[ab\right].$$

2. *First term.* Since $f^\odot - \mathbb{E}\left[\hat{y}^\odot\right]$ is a fixed number, we have

$$\begin{aligned}\mathbb{E}\left[a^2\right] &= \left(f^\odot - \mathbb{E}\left[\hat{y}^\odot\right]\right)^2 + 2\left(f^\odot - \mathbb{E}\left[\hat{y}^\odot\right]\right)\mathbb{E}\left[\epsilon^\odot\right] + \mathbb{E}\left[(\epsilon^\odot)^2\right] \\ &= \left(f^\odot - \mathbb{E}\left[\hat{y}^\odot\right]\right)^2 + \sigma_\epsilon^2,\end{aligned}$$

   where we used $\mathbb{E}\left[\epsilon^\odot\right] = 0$, which also implies that $\mathbb{E}\left[(\epsilon^\odot)^2\right] = \text{Var}\ \left(\epsilon^\odot\right) = \sigma_\epsilon^2$. This term is the squared bias plus the irreducible error.
3. *Second term.* By the definition of the variance,

$$\mathbb{E}\left[b^2\right] = \mathbb{E}\left[\left(\hat{y}^\odot - \mathbb{E}\left[\hat{y}^\odot\right]\right)^2\right] = \text{Var}\ \left(\hat{y}^\odot\right).$$

4. *Mixing term.* Splitting a into its fixed and random parts gives

$$\mathbb{E}\left[ab\right] = \left(f^\odot - \mathbb{E}\left[\hat{y}^\odot\right]\right)\mathbb{E}\left[\mathbb{E}\left[\hat{y}^\odot\right] - \hat{y}^\odot\right] + \mathbb{E}\left[\epsilon^\odot\left(\mathbb{E}\left[\hat{y}^\odot\right] - \hat{y}^\odot\right)\right].$$

   The first part vanishes since $\mathbb{E}\left[\mathbb{E}\left[\hat{y}^\odot\right] - \hat{y}^\odot\right] = \mathbb{E}\left[\hat{y}^\odot\right] - \mathbb{E}\left[\hat{y}^\odot\right] = 0$. The second part vanishes because the noise $\epsilon^\odot$ of the future observation is independent of the training data, and therefore of $\hat{y}^\odot$. The expectation value then factorizes into $\mathbb{E}\left[\epsilon^\odot\right]\mathbb{E}\left[\mathbb{E}\left[\hat{y}^\odot\right] - \hat{y}^\odot\right] = 0$.

Adding the three terms gives Eq. (25.17).

The tradeoff between bias and variance is illustrated in Fig. 25.3 and Fig. 25.4.

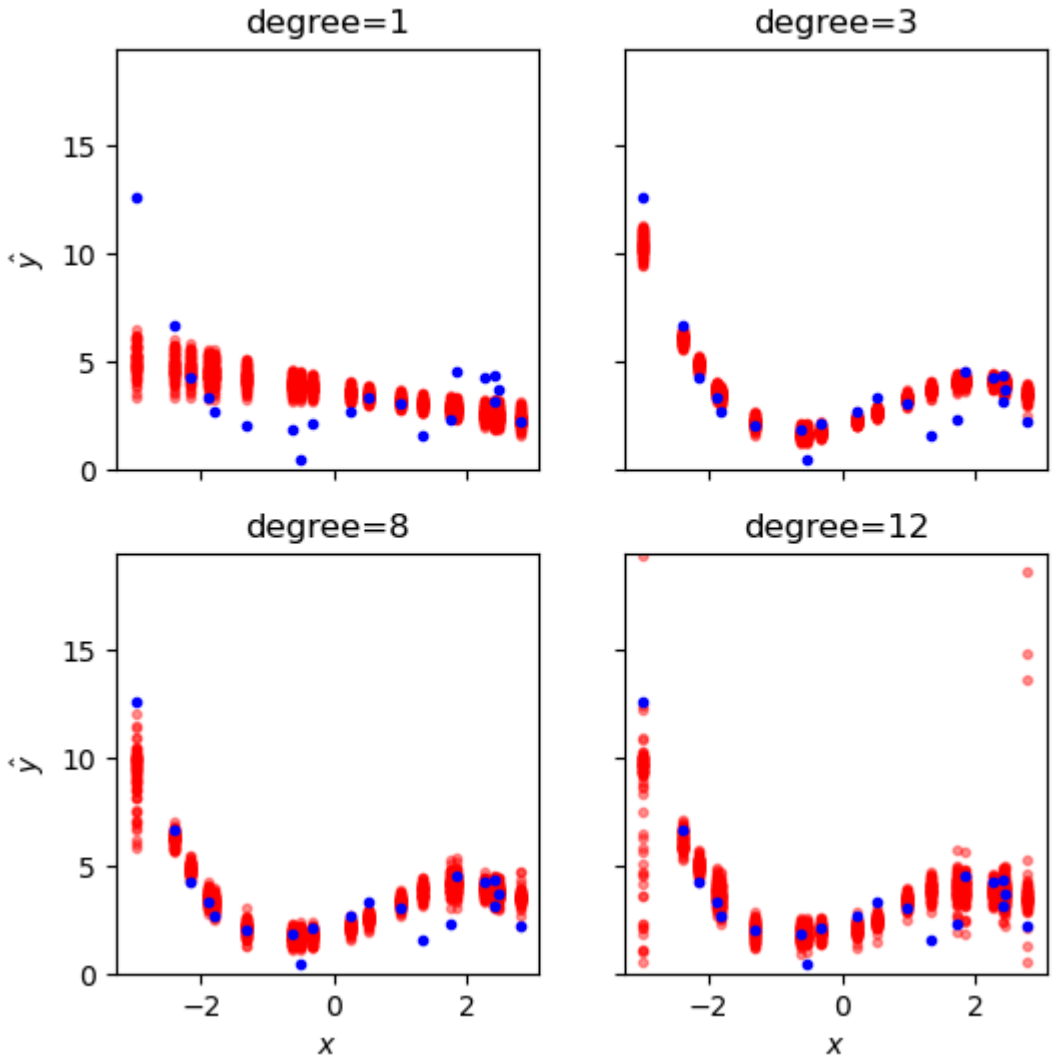


Fig. 25.3 The bias-variance tradeoff illustrated for different polynomial models from low- to high-complexity. For each model complexity (degree) we have trained several versions of the machine-learning model using resampled training data. For each test datum (blue dots) we show the different model predictions. Underfit models have large average errors (bias) while overfit ones have large prediction variance.

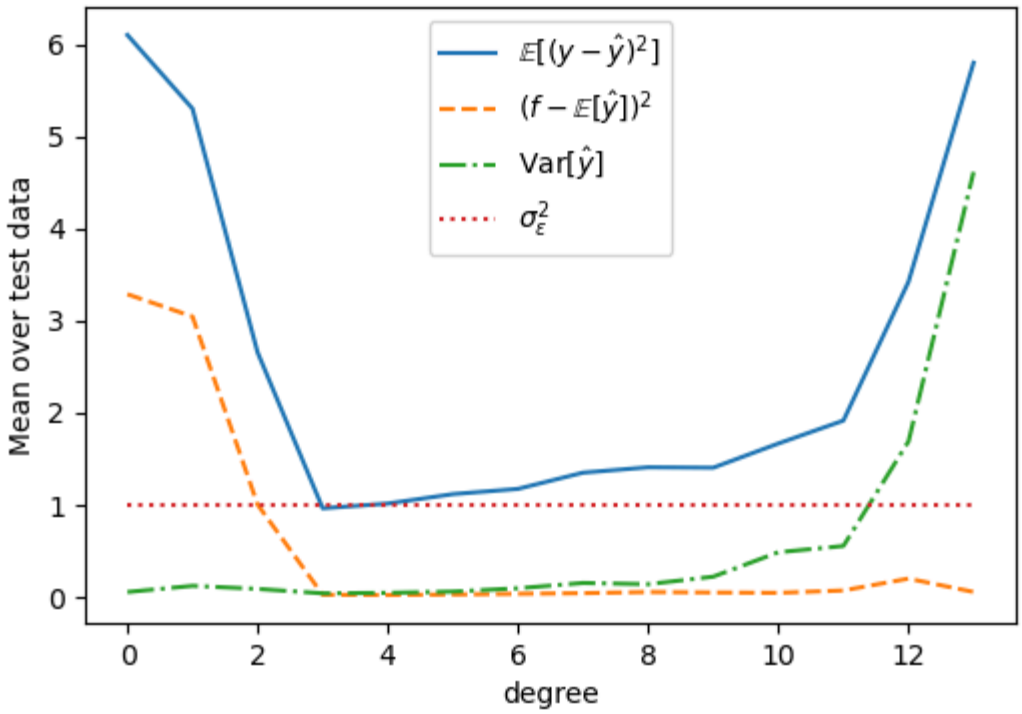


Fig. 25.4 The bias-variance tradeoff illustrated for different polynomial models of our noisy data set. The squared errors for each degree represent the mean results of the test data.

Remarks on bias and variance

The bias-variance tradeoff summarizes the fundamental tension in machine learning, particularly supervised learning, between the complexity of a model and the amount of training data needed to train it. Since data is often limited, in practice it is often useful to use a less-complex model with higher bias, that is a model whose asymptotic performance is worse than another model because it is easier to train and less sensitive to sampling noise arising from having a finite-sized training dataset (smaller variance). In general, more flexible statistical methods have higher variance.

The above equations tell us that in order to minimize the expected validation error, we need to select a statistical learning method that simultaneously achieves low variance and low bias. Note that variance is inherently a nonnegative quantity, and squared bias is also nonnegative. Hence, we see that the prediction error can never lie below Var $(\epsilon) \equiv \sigma_\epsilon^2$, the irreducible error.

Model validation

In order to make an informed choice for these hyperparameters we need to validate that our model provides a good fit to the training data while not becoming overfit. This important process is known as *model validation*. It most often involves splitting the data into two sets: the training set and the validation set.

The model is then trained on the first set of data, while its performance is monitored on the validation set (by computing your choice of performance score/error function).

Caution

Why is it important not to train and evaluate the model on the same data?

Learning curves

The performance of the model will depend on the amount of data that is used for training. When using iterative optimization approaches, such as gradient descent, it will also depend on the number of training iterations. In order to monitor this dependence one usually plots a *learning curve*.

Learning curves are plots of the model performance on both the training and the validation sets, measured by some performance metric such as the mean squared error. This measure is plotted as a function of the size of the training set, or alternatively as a function of the number of training iterations. One defines an *epoch* of training as an update of the model parameters where the entire batch of training data has been used exactly once. An epoch can therefore consist of multiple updates via a random traverse of all training data one by one, or split into mini-batches.

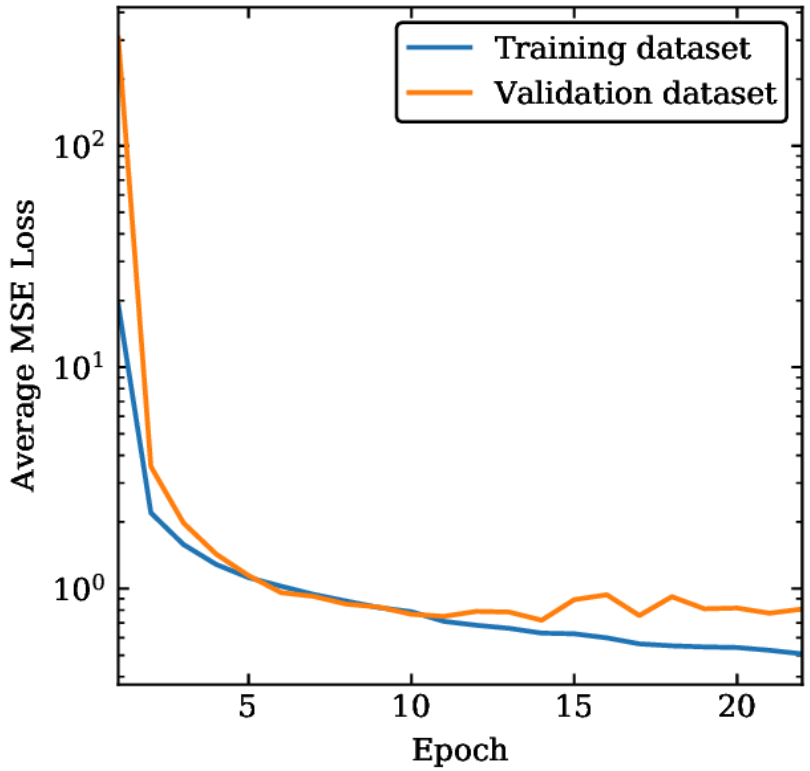


Fig. 25.5 Learning curve for a machine-learning model. The average mean-squared error for the model evaluated on training data and on validation data are shown as a function of the number of training epochs.

Several features in the learning curves shown in Fig. 25.5 deserve to be mentioned:

1. The performance on both training and validation sets is very poor in the beginning of the training.
2. The error on the training set then decreases steadily as more training iterations are performed.
3. The training error decreases quickly in the beginning, and much more slowly as the model parameters are becoming optimized.
4. The validation error is initially very high but decreases as the model becomes better.

5. The validation error eventually reaches a plateau. If the training proceeds much longer than this, the model can become overfit and the validation error will eventually start increasing.

For models with a high degree of complexity there will usually be a gap between the curves which implies that the model performs significantly better on the training data than on the validation set. This is a sign of high variance, see the discussion of the *bias-variance tradeoff* above.

Cross-validation

Cross-validation is a strategy to find model hyperparameters that yield a model with good prediction performance. A common practice is to hold back some subset of the data from the training of the model and then use this holdout set to check the model performance.

One of these two data sets, called the *training set*, plays the role of **original** data on which the model is built. The second of these data sets, called the *validation set*, plays the role of the **novel** data and is used to evaluate the prediction performance (often operationalized as the log-likelihood or the prediction error: MSE or R2 score) of the model built on the training data set. This procedure (model building and prediction evaluation on training and validation set, respectively) is done for a collection of possible choices for the hyperparameters. The parameter that yields the model with the best prediction performance is to be preferred.

The validation set approach is conceptually simple and is easy to implement. But it has two potential drawbacks:

- The validation estimate of the test error rate can be highly variable, depending on precisely which observations are included in the training set and which observations are included in the validation set. There might be data points that are critical for training the model, and the performance metric will be very bad if those happen to be excluded from the training set.
- In the validation approach, only a subset of the observations, those that are included in the training set rather than in the validation set are used to fit the model. Since statistical methods tend to perform worse when trained on fewer observations, this suggests that the validation set error rate may tend to overestimate the test error rate for the model fit on the entire data set.

To reduce the sensitivity on a particular data split, one can perform several different splits. For each split the model is fit using the training data and evaluated on the corresponding validation set. The hyperparameter that performs best on average (in some sense) is then selected.

To avoid unbalanced influence of important training samples one can use k-fold cross-validation to structure the data splitting. The samples are divided into k more or less equally sized, exhaustive and mutually exclusive subsets. At each fold one of these subsets plays the role of the validation set while the union of the remaining subsets constitutes the training set. Such a splitting warrants a balanced representation of each sample in both training and validation set over the splits. Still the division into the k subsets involves a degree of randomness. This may be fully excluded when choosing $k = n$. This particular case is referred to as leave-one-out cross-validation (LOOCV).

Algorithm 25.1 (k-fold cross-validation)

1. Define a range of interest for the model hyperparameter(s) $\boldsymbol{\lambda}$ and create a set $\{\boldsymbol{\lambda}_m\}_{m=1}^{M}$ that spans this range.
2. Divide the training data set **data** into k exhaustive and mutually exclusive subsets **data** $_i \subset$ **data** for $i = 1, \ldots, k$, and **data** $_i \cap$ **data** $_j = \emptyset$ for $i \neq j$.
3. For $m \in \{1, \ldots, M\}$:

1. For $i \in \{1, \ldots, k\}$:
    - Use $\textbf{data}_i$ as the validation set and all other data $\textbf{data}_{-i} = \textbf{data} \setminus \textbf{data}_i$ as the training set.
    - Train the model with hyperparameter(s) $\boldsymbol{\lambda}_m$ using the training set $\textbf{data}_{-i}$, which will give a best fit $\boldsymbol{\theta}_i^*(\boldsymbol{\lambda}_m)$.
    - Evaluate the prediction performance with $\boldsymbol{\theta}_i^*(\boldsymbol{\lambda}_m)$.
2. Average the prediction performances on all folds. This is known as the *cross-validated error* $\mathrm{CV}_k(\boldsymbol{\lambda}_m)$.

4. The hyperparameter(s) that minimizes the cross-validated error is the best choice.

$$\boldsymbol{\lambda}^* = \underset{\boldsymbol{\lambda}}{\operatorname{argmin}}\, \mathrm{CV}_k(\boldsymbol{\lambda}).$$

5. Fix $\boldsymbol{\lambda} = \boldsymbol{\lambda}^*$ and train the model on all data $\textbf{data}$.

Exercises

Exercise 25.1 ($k = 1$ NN training error)

What is E_{train} (25.1) for binary classification using kNN with $k = 1$?

Exercise 25.2 (kNN model complexity)

Draw a simple figure to illustrate the under-and overfitting regions of a kNN model. With model complexity on the x-axis, where would you put the extremes $k = 1$ and $k = N$ (with N being the number of training data)? In other words, what would be the effective number of parameters in this model?

Exercise 25.3 (Study of model bias and variance)

Perform your own study of the bias-variance tradeoff by revisiting the linear regression problem in Ordinary linear regression in practice.

- Create one validation set and ten different training sets using different random number generators (`rng` argument) in the `data_generating_process_measurement()` function.
- Train linear regression models of different polynomial degrees for each training set. Evaluate both the training error and the validation error for each model.
- Plot all ten degree-0 models in a single figure (using the same color for each line, but make them slightly transparent such that they don't block the view) together with the validation data and with the true, underlying model. Plot also the ten degree-5 models in the same figure (in a different color). Now try to understand why the former family can be described as having large bias but small variance, whereas the latter has large variance.

Connect these findings to the concept of predictive power.

Exercise 25.4 (Implement k-fold cross validation)

Use either the linear regression example, or the binary classification one, to implement k-fold cross validation. Try to use the results to determine an optimal degree of the polynomial for linear regression, or the choice of k for the kNN classifier.

Exercise 25.5 (Large training error)

- Explain why E_{train} is a poor metric for estimating the prediction error.
- Consider the situation when E_{train} is larger than the prediction error that you are aiming for. Explain why it then becomes a waste of time to implement cross validation or other techniques to estimate the actual prediction error.
- What could help to reduce the training error?

Solutions to exercises

Solution to Exercise 25.1 ($k = 1$ NN training error)

$E_{\text{train}} = 0$ for kNN models with $k = 1$.

Solution to Exercise 25.2 (kNN model complexity)

The effective number of parameters in a kNN model goes something like N/k. Think about how many effective parameters the training data corresponds to in the extreme situations with $k = 1$ and $k = N$. In between, you can imagine non-overlapping regions of k sized clusters.

In effect, this implies that the $k = 1$ model is the one with the largest number of effective parameters and hence the most complex one (explaining the overfitting), whereas the $k = N$ model has effectively a single parameter and will underfit.

Solution to Exercise 25.3 (Study of model bias and variance)

You should see small variations between the degree-0 models since they just capture the means of the respective training sets. However, it also implies that they in average reproduce both training and validation data poorly.

The higher degree models will show much larger variations (since they will be overfitting to the different sets of training data). They will all perform well on their respective training set, but poorly on the validation set.

Solution to Exercise 25.4 (Implement k-fold cross validation)

Note you are interested in $\text{CV}_k(\lambda)$ where the hyperparameter is $\lambda = p$ for the linear regression example, or $\lambda = k$ for the kNN classifier.

Solution to Exercise 25.5 (Large training error)

- E_{train} will always decrease with increasing model complexity as it is tuned to the data (and higher model complexity implies more model flexibility).
- The error on new data, E_{new}, is in general larger than E_{train}. This is the generalization gap. So it is better to invest your time in improving the model, or collecting more data, before actually attempting to estimate the prediction error.
- See the above answer.

[1] Remember that $\mathbb{E}[t]$ denotes the expectation value for the random variable t. Furthermore, the variance is given by Var $(t) = \mathbb{E}\left[(t - \mathbb{E}[t])^2\right]$.

25.4. Data bias and fairness in machine learning

Part of this lecture is based on the chapter **Ethics in machine learning** [Sum21] by David Sumpter in Machine Learning: A First Course for Engineers and Scientists [LWahlstromLSchon21]. There are several guidelines for Fairness and Ethics in the context of Artificial Intelligence and Machine Learning. In particular, we refer to the Ethics Guidelines for Trustworthy Artificial Intelligence by the High-Level Expert Group on AI set up by the European Commission.

Fairness and error functions

We have explained how a cost function is used for training a machine learning model, and how an error function can be used to assess its performance. The choices of these functions might appear a technical issue, but it should be understood that they might also affect the perceived fairness of the model.

Before turning to a real-world example, let us introduce the so called confusion matrix. This tool is mainly used in the evaluation of binary classifiers, although the idea can be extended also to multiclass problems. In short, the confusion matrix separates the validation data into four groups depending on y (the observation) and $\hat{y}$ (the model prediction). Here we use the labels $y = 1$ for positive/presence and $y = -1$ for negative/absence.

	$y = -1$	$y = 1$	**total**
$\hat{y} = -1$	True Negative (TN)	False Negative (FN)	N*
$\hat{y} = 1$	False Positive (FP)	True Positive (TP)	P*
total	N	P	n

The entries in this table should be replaced with the actual numbers from the validation set. This matrix provides more information than just the accuracy of the model as it also reveals how it fails when it does. Note that P(N) denotes the total number of positive(negative) outputs in the data set, while P*(N*) is the total number of positive(negative) class predictions made by the model.

False positive predictions (FP) are also known as *Type I* errors, while false negatives (FN) are called *Type II* errors. Depending on the application, one of these types of errors is typically more serious than the other one. Consider for example a machine-learning model to diagnose a rare disease, or one that predicts whether a young offender is likely to commit future crimes, which type of error has the largest negative consequence in each situation?

There is an extensive terminology connected with the confusion matrix. Let us just highlight a few metrics that might be relevant in different circumstances:

- The misclassification rate = (FN + FP) / n.
- The false positive rate (fall-out) = FP / N
- The false negative rate (miss rate) = FN / P
- The precision = TP / P* = TP / (TP + FP)

The fact that the choice of relevant metric depends on the problem reveals an important point: *There is no single function for measuring fairness*. In fact, as a machine-learning engineer you should not make claims that your model is fair, but rather make every effort to explain how the model performs in different aspects related to fairness.

Example: The Compas algorithm

(from Sumpter 2021 [Sum21])

The Compas algorithm was developed by a private company, Northpointe, to help with criminal sentencing decisions. The model used logistic regression with input variables including age at first arrest, years of education, and questionnaire answers about family background, drug use, and other factors to predict an output variable as to whether the person would reoffend. Race was not included in the model. Nonetheless, when tested – as part of a study by Julia Angwin and colleagues at Pro-Publica on an independently collected data set, the model gave different predictions for black defendants than for white. The results are reproduced below in the form of a confusion matrix for re-offending over the next two years.

Black defendants	**Did not reoffend ($y = -1$)**	**Reoffended ($y = 1$)**
Lower risk ($\hat{y} = -1$)	$\mathrm{TN} = 990$	$\mathrm{FN} = 532$
Higher risk ($\hat{y} = 1$)	$\mathrm{FP} = 805$	$\mathrm{TP} = 1369$

White defendants	**Did not reoffend ($y = -1$)**	**Reoffended ($y = 1$)**
Lower risk ($\hat{y} = -1$)	$\mathrm{TN} = 1139$	$\mathrm{FN} = 461$
Higher risk ($\hat{y} = 1$)	$\mathrm{FP} = 349$	$\mathrm{TP} = 505$

Angwin and her colleagues pointed out that the false positive rate for black defendants, 805/(990 + 805) = 44.8%, is almost double that of white defendants, 349/(349 + 1 139) = 23.5%. This difference cannot be accounted for simply by overall reoffending rates: although this is higher for black defendants (at 51.4% arrested for another offence within two years), when compared to white defendants (39.4%), these differences are smaller than the differences in false positive rates. On this basis, the model is clearly unfair. The model is also unfair in terms of true positive rate (recall). For black defendants, this is 1 369/(532 + 1369) = 72.0% versus 505/(505 + 461) = 52.3% for white defendants. White offenders who go on to commit crimes are more likely to be classified as lower risk.

In response to criticism about the fairness of their method, the company Northpointe countered that in terms of performance, the precision (positive predictive value) was roughly equal for both groups: 1 369/(805 + 1369) = 63.0% for black defendants and 505/(505 + 349) = 59.1% for white. In this sense the model is fair, in that it has the same performance for both groups. Moreover, Northpointe argued that it is precision which is required, by law, to be equal for different categories. Again this is the problem we highlighted above, but now with serious repercussions for the people this algorithm is applied to: black people who won't later reoffend are more likely to be classified as high risk than white people.

Limitations of training data

Bias in data is an error that occurs when certain elements of a dataset are overweighted or overrepresented. Biased datasets don't accurately represent ML model's use case, which leads to skewed outcomes, systematic prejudice, and low accuracy.

Often, the erroneous result discriminates against a specific group or groups of people. For example, data bias reflects prejudice against age, race, culture, or sexual orientation. In a world where AI systems are increasingly used everywhere, the danger of bias lies in amplifying discrimination.

It takes a lot of training data for machine learning models to produce viable results. If you want to perform advanced operations (such as text, image, or video recognition), you need millions of data points. Poor or incomplete data as well as biased data collection & analysis methods will result in inaccurate predictions because the quality of the outputs is determined by the quality of the inputs.

If you think data bias is the problem of recent times, it's not. Dating back to 1988, British Medical Journal described a case from St George's Hospital Medical School.

The school developed a program that aimed to reduce the amount of work when selecting candidates for interviews. The school believed it would eliminate human error and inconsistencies.

But the Commission for Racial Equality found the school guilty of discrimination against women and people with non-European sounding names. In fact, the program reproduced the bias that already existed in the system.

Data bias types in machine learning, including examples

If you want to build a fair AI project and use data ethically, you have to know the types of data bias in machine learning to spot them before they wreck your ML model.

However, data bias in machine learning doesn't only result from skewed data. There are far more reasons such a bias occurs. Let's take a deeper look (list from 8 types of data bias that can wreck your machine learning models by Joanna Kamińska)

Systemic biases

Systemic bias occurs when certain social groups are favored and others are devalued.

The National Institute of Standards and Technology (NIST) provides a good example of such bias in their recent special publication on managing bias in AI - the infrastructure for daily living (in most cases) isn't adjusted to the needs of disabled people.

The reason behind it is institutional and stems from the underrepresentation of disabled people in studies. So, the biggest problem with systemic bias is that it is stealthily hidden in the world and thus overlooked.

Another example of systemic bias comes from St George's Hospital Medical School. This type of bias results from wrong procedures and practices in an organization. Although engineers don't want to discriminate against any group of people, the bias is already rooted in the system.

Automation bias

Have you ever used a digital tool that, based on artificial intelligence, suggested an action? Automation bias occurs when you take this AI-based recommendation and use it before verifying if the information was right.

For instance, a data scientist depends on the analytics platform's suggestions to remove or modify specific data despite the recommendations worsening the quality of this data.

Selection bias

Randomization is the process that balances out the effects of uncontrollable factors - variables in a data set that are not specifically measured and can compromise results. In data science, selection bias occurs when you have data that aren't properly randomized. If your dataset isn't properly randomized, it means the sample isn't representative - it doesn't truly reflect the analyzed population.

For instance, when building models for healthcare exploration, a data scientist operates only on white patients. This data sample isn't reflecting the entire population because it doesn't take into account BIPOC (Black, Indigenous, and people of color) patients.

This case also overlaps with racial bias – when data skews in favor of a particular group of people from specific demographics.

Overfitting and underfitting the data

In machine learning, overfitting occurs when a model is so flexible, relative to the amount of training data, that it begins to learn from the noise and inaccurate data entries in the data set. Machine learning models have trouble predicting new data based on the training data because this noise cannot be applied to new data.

When a machine learning model fails to capture the underlying trend of the data (because it is too simple), underfitting occurs. In this case, it indicates that the model or algorithm is not fitting the data well enough.

Reporting Biases

A reporting bias is the inclusion of only a subset of results in an analysis, which typically only covers a small fraction of evidence. Reporting bias can take many forms. An example would be analyzing data based on studies found in citations of other studies (citation bias), excluding reports not written in the scientist's native language (language bias), or choosing studies with positive findings rather than negative findings (publication bias) & more.

As an example, a sentiment analysis model can be trained to predict whether a book review on a popular website is positive or negative. The vast majority of reviews in the training data set reflect extreme opinions (reviewers who either adored or despised a book). This was because people were less likely to review a book they did not feel strongly about. Because of this, the model is less likely to accurately predict sentiment of reviews that use more subtle language to describe a book.

Overgeneralization Bias

When a person applies something from one event to all future events, it is overgeneralization. In the field of data science, whenever you assume that what you see in your dataset is also what would be seen in another dataset, you are overgeneralizing.

Group Attribution Biases

Group attribution biases refer to the human tendency to assume that an individual's characteristics are always determined by the beliefs of the group, or that a group's decisions are influenced by the feelings of all its members. The group attribution bias manifests itself when you give preference to your own group (in-group bias) or when you stereotype members of groups you don't belong to (out-group bias).

For example, engineers might be predisposed to believe that applicants who attended the same school as they did are better qualified for a job when training a résumé-screening model for software developers.

Implicit Biases

Implicit biases occur when we make assumptions based on our personal experiences. Implicit bias manifests itself as attitudes and stereotypes we hold about others, even when we are unaware of it.

We might look for information that would support our beliefs and hypotheses and disregard information that doesn't. You may be more likely to continue testing machine learning models until you obtain results that support your hypothesis (confirmation bias).

When there is not enough data or the data is not representative, you end up with sample bias. For instance, if your training data only features male doctors, the system may conclude that all doctors are male.

Existing stereotypes and poor measurement can creep in data at the stage of data collection. Diverse and representative datasets are crucial for machine learning. When you don't have enough original training data, which is often the case, synthetic data can be used to supplement it.

Ethics guidelines

Although there are methods to reach for fairness, the most important lesson here is *awareness*. Machine-learning engineers have a responsibility to be aware of model limitations and to describe and discuss them with users.

The Ethics Guidelines for Trustworthy Artificial Intelligence by the High-Level Expert Group on AI set up by the European Commission identifies ethical principles that must be respected in the development, deployment and use of AI systems:

Ethical principles

- Develop, deploy and use AI systems in a way that adheres to the ethical principles of: respect for human autonomy, prevention of harm, fairness and explicability. Acknowledge and address the potential tensions between these principles.
- Pay particular attention to situations involving more vulnerable groups such as children, persons with disabilities and others that have historically been disadvantaged or are at risk of exclusion, and to situations which are characterised by asymmetries of power or information, such as between employers and workers, or between businesses and consumers.
- Acknowledge that, while bringing substantial benefits to individuals and society, AI systems also pose certain risks and may have a negative impact, including impacts which may be difficult to anticipate, identify or measure (e.g. on democracy, the rule of law and distributive justice, or on the human mind itself.) Adopt adequate measures to mitigate these risks when appropriate, and proportionately to the magnitude of the risk.

They also list seven requirements that AI systems should meet.

Requirements for AI systems

- Ensure that the development, deployment and use of AI systems meets the seven key requirements for Trustworthy AI:
    1. human agency and oversight,

2. technical robustness and safety,
3. privacy and data governance,
4. transparency,
5. diversity, non-discrimination and fairness,
6. environmental and societal well-being and
7. accountability.

- Consider technical and non-technical methods to ensure the implementation of those requirements.

26. Logistic Regression

In linear regression our main interest was centered on learning the coefficients of a functional fit (say a polynomial) in order to be able to predict the response of a continuous variable on some unseen data. The fit to the continuous variable $y^{(i)}$ is based on some independent variables $\boldsymbol{x}^{(i)}$. Linear regression resulted in analytical expressions for standard ordinary Least Squares or Ridge regression (in terms of matrices to invert) for several quantities, ranging from the variance and thereby the confidence intervals of the parameters $\boldsymbol{w}$ to the mean squared error. If we can invert the product of the design matrices, linear regression gives then a simple recipe for fitting our data.

Classification problems, however, are concerned with outcomes taking the form of discrete variables (i.e. categories). We may for example, on the basis of DNA sequencing for a number of patients, like to find out which mutations are important for a certain disease; or based on scans of various patients' brains, figure out if there is a tumor or not; or given a specific physical system, we'd like to identify its state, say whether it is an ordered or disordered system (typical situation in solid state physics); or classify the status of a patient, whether she/he has a stroke or not and many other similar situations.

The most common situation we encounter when we apply logistic regression is that of two possible outcomes, normally denoted as a binary outcome, true or false, positive or negative, success or failure etc.

26.1. Optimization and Deep learning

Logistic regression will also serve as our stepping stone towards neural network algorithms and supervised deep learning. For logistic learning, the minimization of the cost function leads to an optimization problem that is non-linear in the parameters $\boldsymbol{w}$. This optimization (how to find reliable minima of a multi-variable function) is a key challenge for all machine learning algorithms. This leads us back to the family of gradient descent methods encountered in the chapter on Mathematical optimization. These methods are the working horses of basically all modern machine learning algorithms.

We note also that many of the topics discussed here on logistic regression are also commonly used in modern supervised deep learning models, as we will see later.

Basics and notation

We consider the case where the dependent variables (also called the responses, targets, or outcomes) are discrete and only take values from $k = 1, \dots, K$ (i.e. K classes).

The goal is to predict the output classes from the design matrix $\boldsymbol{X} \in \mathbb{R}^{n \times p}$ made of n samples, each of which carries p features or predictors. The primary goal is to identify the classes to which new unseen samples belong.

Notation

We will use the following notation:

- $\boldsymbol{x}$: independent (input) variables, typically a vector of length p. A matrix of N instances of input vectors is denoted $\boldsymbol{X}$, and is also known as the *design matrix*. The input for machine-learning applications are often referred to as *features*.
- t: dependent, response variable, also known as the target. For binary classification the target $t^{(i)} \in \{0, 1\}$. For K different classes we would have $t^{(i)} \in \{1, 2, \ldots, K\}$. A vector of N targets from N instances of data is denoted $\boldsymbol{t}$.
- $\mathcal{D}$: is the data, where $\mathcal{D}^{(i)} = \{(\boldsymbol{x}^{(i)}, t^{(i)})\}$.
- $\boldsymbol{y}$: is the output of our classifier that will be used to quantify probabilities $p_{t=C}$ that the target belongs to class C.
- $\boldsymbol{w}$: will be the parameters (weights) of our classification model.

26.2. Binary classification

Let us specialize to the case of two classes only, with outputs $t^{(i)} \in \{0, 1\}$. That is

$$t^{(i)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \text{no} \\ \text{yes} \end{bmatrix}. \tag{26.1}$$

The perceptron

Before moving to the logistic model, let us try to use a linear regression model to classify these two outcomes. We could use a linear model

$$\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{w}, \tag{26.2}$$

where $\tilde{\boldsymbol{y}}$ is a vector representing the possible outcomes, $\boldsymbol{X}$ is our $n \times p$ design matrix and $\boldsymbol{w}$ are the model parameters.

Note however that our outputs $\tilde{y}^{(i)} \in \mathbb{R}$ take values on the entire real axis. Our targets $t^{(i)}$, however, are discrete variables.

One simple way to get a discrete output is to have sign functions that map the output of a linear regressor to values $y^{(i)} \in \{0, 1\}$, $y^{(i)} = f(\tilde{y}^{(i)}) = \frac{\text{sign}(\tilde{y}^{(i)})+1}{2}$, which will map to one if $\tilde{y}^{(i)} \geq 0$ and zero otherwise. Historically this model is called the *perceptron* in the machine learning literature.

The perceptron is an example of a "hard classification" model. We will encounter this model when we discuss neural networks as well. Each datapoint is deterministically assigned to a category (i.e $y^{(i)} = 0$ or $y^{(i)} = 1$). In many cases, it is favorable to have a *soft* classifier that outputs the probability of a given category rather than a single value. For example, given $\boldsymbol{x}^{(i)}$, the classifier outputs the probability of being in a category k.

The logistic function

Logistic regression is the simplest example of the use of such a soft classifier. Let us assume that we have two classes such that $t^{(i)}$ is either 0 or 1.

In logistic regression, we will use the *logistic* function

$$y(\boldsymbol{x};\boldsymbol{w}) = y(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{1+e^z}, \tag{26.3}$$

with the so-called *preactivation* $z = z(\boldsymbol{x};\boldsymbol{w})$. This function is no longer linear in the model parameters $\boldsymbol{w}$. It is an example of an S-shaped or *sigmoid* function. Its inverse, $z = \ln\left[y/(1-y)\right]$, is known as the *logit* (or log-odds) function.

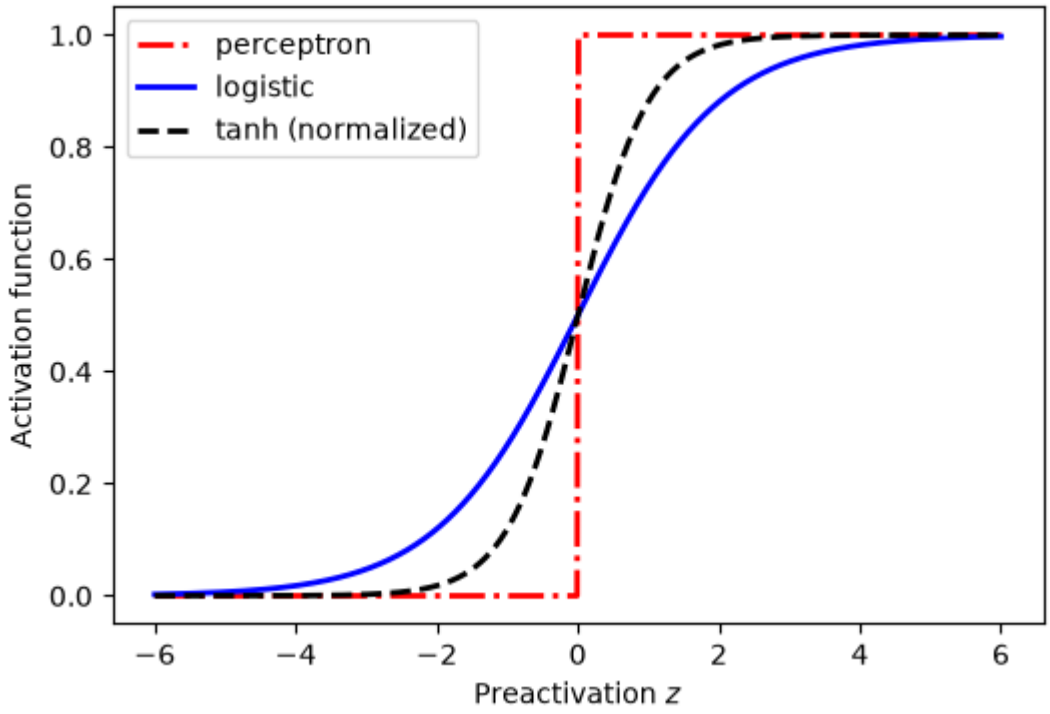
We let $y^{(i)}$ give the probability that a data point $\boldsymbol{x}^{(i)}$ belongs to category $t^{(i)} = 1$,

$$\mathbb{P}\left(t^{(i)} = 1 | \boldsymbol{x}^{(i)}, \boldsymbol{w}\right) = y(\boldsymbol{x}^{(i)};\boldsymbol{w}). \tag{26.4}$$

- Most frequently one uses $z = z(\boldsymbol{x},\boldsymbol{w}) \equiv \boldsymbol{x}\cdot\boldsymbol{w}$.
- It is common to introduce also a bias, or threshold, weight w_0. This can be accommodated by prepending a constant feature $x_0^{(i)} = 1$ for all i.
- Note that $1 - y(z) = y(-z)$.
- The sigmoid function can be motivated in several different ways:
  - In information theory this function represents the probability of a signal $s = 1$ rather than $s = 0$ when transmission occurs over a noisy channel.
  - It can be seen as an artificial neuron that mimics aspects of its biological counterpart.

Standard activation functions

▼ **Hide code cell content**



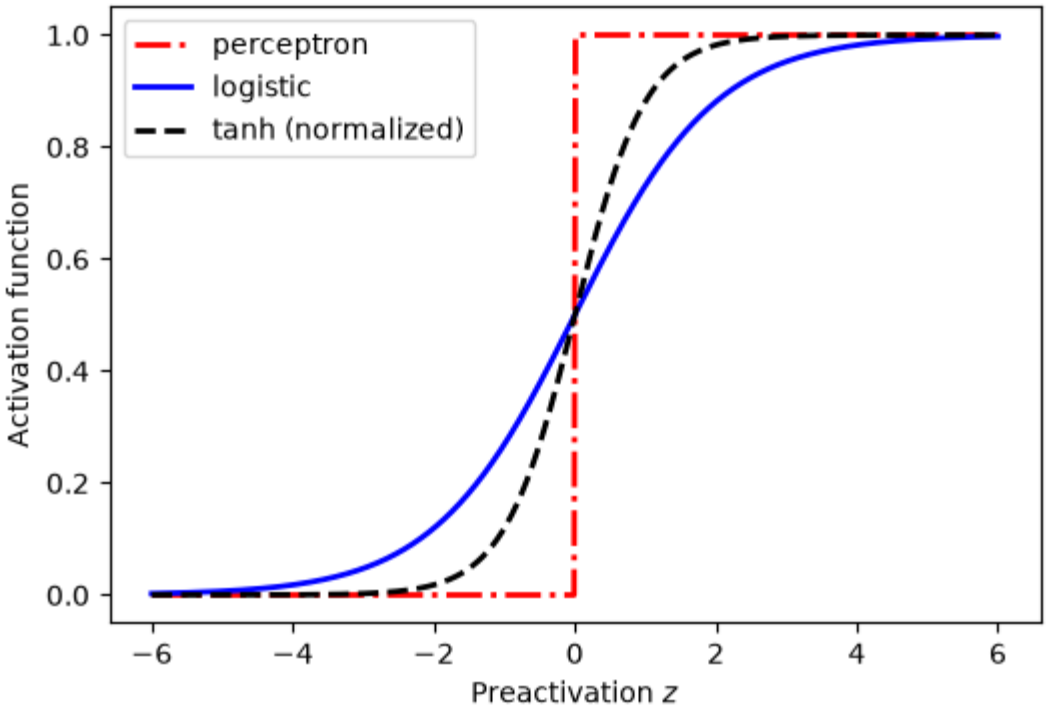


Fig. 26.1 The sigmoid, step,and (normalized) tanh functions; three common classifier functions used in classification and neural networks. In these lecture notes we use the letter z to denote the preactivation.

A binary classifier with two parameters

We assume now that we have two classes with $t^{(i)}$ being either 0 or 1. Furthermore we assume also that we have only two parameters w_0, w_1 and the features $\boldsymbol{x}^{(i)} = \{1, x^{(i)}\}$ defining the activation function. I.e., there is a single independent (input) variable x. We can produce probabilities from the classifier output $y^{(i)}$

$$\mathbb{P}(t^{(i)} = 1|x^{(i)}, \boldsymbol{w}) = y(z^{(i)}) = \frac{\exp\left(w_0 + w_1 x^{(i)}\right)}{1 + \exp\left(w_0 + w_1 x^{(i)}\right)},$$
$$\mathbb{P}(t^{(i)} = 0|x^{(i)}, \boldsymbol{w}) = 1 - \mathbb{P}(t^{(i)} = 1|x^{(i)}, \boldsymbol{w}) = \frac{1}{1 + \exp\left(w_0 + w_1 x^{(i)}\right)},$$

where $\boldsymbol{w} = (w_0, w_1)$ are the weights we wish to extract from training data.

Determination of weights

Among ML practitioners, the prevalent approach to determine the weights in the activation function(s) is by minimizing some kind of cost function using some version of gradient descent. As we will see this usually corresponds to maximizing a likelihood function with or without a regularizer.

In this course we will obviously also advocate (or at least make aware of) the more probabilistic approach to learning about these parameters.

Maximum likelihood

In order to define the total likelihood for all possible outcomes from a dataset $\mathcal{D} = \{(x^{(i)}, t^{(i)},)\}$, with the binary labels $t^{(i)} \in \{0, 1\}$ and where the data points are drawn independently, we use the binary version of the Maximum Likelihood Estimation (MLE) principle. We express the likelihood in terms of the product of the individual probabilities of a specific outcome $t^{(i)}$, that is

$$\mathcal{L} = \mathbb{P}(\mathcal{D}|\boldsymbol{w}) = \prod_{i=1}^{N} \left[p(t^{(i)} = 1|x^{(i)}, \boldsymbol{w})\right]^{t^{(i)}} \left[1 - p(t^{(i)} = 1|x^{(i)}, \boldsymbol{w}))\right]^{1-t^{(i)}}$$

The **cost/loss** function (to be minimized) is then defined as the negative log-likelihood

$$\mathcal{C}(\boldsymbol{w}) = -L \equiv -\log(\mathcal{L}) = -\sum_{i=1}^{N} \Big(t^{(i)} \log \mathbb{P}(t^{(i)} = 1|x^{(i)}, \boldsymbol{w}) + (1 - t^{(i)}) \log \Big[1 - \mathbb{P}(t^{(i)} = 1|x^{(i)}, \boldsymbol{u}$$ (26.5)

The cost function rewritten as cross entropy

Using the definitions of the probabilities we can rewrite the **cost/loss** function as

$$\mathcal{C}(\boldsymbol{w}) = -\sum_{i=1}^{N} \Big(t^{(i)} \log y(x^{(i)}, \boldsymbol{w}) + (1 - t^{(i)}) \log \Big[1 - y(x^{(i)}, \boldsymbol{w}) \Big] \Big), \tag{26.6}$$

which can be recognised as the relative entropy between the empirical probability distribution $(t^{(i)}, 1 - t^{(i)})$ and the probability distribution predicted by the classifier $(y^{(i)}, 1 - y^{(i)})$. Therefore, this cost function is known in statistics as the **cross entropy**.

Using specifically the logistic sigmoid activation function with two weights, and reordering the logarithms, we can rewrite the log-likelihood to obtain

$$\mathcal{C}(\boldsymbol{w}) = -\sum_{i=1}^{N} \Big[t^{(i)}(w_0 + w_1 x^{(i)}) - \log \Big(1 + \exp\left(w_0 + w_1 x^{(i)}\right) \Big) \Big]. \tag{26.7}$$

MLE should give the weights $\boldsymbol{w}^*$ that minimizes this cost function.

Regularization

In practice, just as for linear regression, one often supplements the cross-entropy cost function with additional regularization terms, usually L_1 and L_2 regularization. This introduces hyperparameters into the classifier.

In particular, Ridge regularization is obtained by defining another cost function

$$\mathcal{C}_W(\boldsymbol{w}) \equiv \mathcal{C}(\boldsymbol{w}) + \alpha E_W(\boldsymbol{w}) \tag{26.8}$$

where $E_W(\boldsymbol{w}) = \frac{1}{2} \sum_j w_j^2$ and α is known as the *weight decay*.

Question

Can you motivate why α is known as the weight decay?

Hint: Recall the origin of this regularizer from a Bayesian perspective.

Minimizing the cross entropy

The cross entropy is a convex function of the weights $\boldsymbol{w}$ and, therefore, any local minimizer is a global minimizer.

Minimizing this cost function (here without regularization term) with respect to the two parameters w_0 and w_1 we obtain

$$\frac{\partial \mathcal{C}(\boldsymbol{w})}{\partial w_0} = -\sum_{i=1}^{N}\left(t^{(i)} - \frac{\exp\left(w_0 + w_1 x^{(i)}\right)}{1+\exp\left(w_0 + w_1 x^{(i)}\right)}\right) = -\sum_{i=1}^{N}\left(t^{(i)} - y^{(i)}\right),$$

$$\frac{\partial \mathcal{C}(\boldsymbol{w})}{\partial w_1} = -\sum_{i=1}^{N}\left(t^{(i)}x^{(i)} - x^{(i)}\frac{\exp\left(w_0 + w_1 x^{(i)}\right)}{1+\exp\left(w_0 + w_1 x^{(i)}\right)}\right) = -\sum_{i=1}^{N} x^{(i)}\left(t^{(i)} - y^{(i)}\right).$$

A more compact expression

Let us now define a vector $\boldsymbol{t}$ with n elements $t^{(i)}$, an $N \times 2$ matrix $\boldsymbol{X}$ which contains the $(1, x^{(i)})$ predictor variables, and a vector $\boldsymbol{y}$ of the outputs $y^{(i)} = y(x^{(i)}, \boldsymbol{w})$. We can then express the first derivative of the cost function in matrix form

$$\frac{\partial \mathcal{C}(\boldsymbol{w})}{\partial \boldsymbol{w}} = -\boldsymbol{X}^T(\boldsymbol{t} - \boldsymbol{y}). \tag{26.9}$$

A learning algorithm

Notice. Having access to the first derivative we can define an *on-line learning rule* as follows:

- For each input i (possibly permuting the sequence in each epoch) compute the error $e^{(i)} = t^{(i)} - y^{(i)}$.
- Adjust the weights in a direction that would reduce this error: $\Delta w_j = \eta e^{(i)} x_j^{(i)}$. The parameter η is called the *learning rate*.
- Perform multiple passes through the data, where each pass is known as an *epoch*. The computation of outputs $\boldsymbol{y}$ given a set of weights $\boldsymbol{w}$ is known as a *forward pass*, while the computation of gradients and adjustment of weights is called *back-propagation*.

You will recognise this learning algorithm as *stochastic gradient descent*.

Alternatively, one can perform *batch learning* for which multiple instances are combined into a batch, and the weights are adjusted following the matrix expression stated above. At the end, one hopes to have reached an optimal set of weights.

Extending to more features

Within a binary classification problem, we can easily expand our model to include multiple input features. Our activation function is then (with p features)

$$z(\boldsymbol{x}^{(i)}, \boldsymbol{w}) = w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)} + \cdots + w_p x_p^{(i)}. \tag{26.10}$$

Defining $\boldsymbol{x}^{(i)} \equiv [1, x_1^{(i)}, x_2^{(i)}, \ldots, x_p^{(i)}]$ and $\boldsymbol{w} = [w_0, w_1, \ldots, w_p]$ we get

$$\mathbb{P}(t^{(i)} = 1 | \boldsymbol{w}, \boldsymbol{x}^{(i)}) = \frac{\exp\left(\boldsymbol{w} \cdot \boldsymbol{x}^{(i)}\right)}{1 + \exp\left(\boldsymbol{w} \cdot \boldsymbol{x}^{(i)}\right)}. \tag{26.11}$$

26.3. Extending to more classes

Until now we have focused on binary classification involving just a decision between two classes. Suppose we wish to extend to K classes. We will then introduce K outputs $\boldsymbol{y}^{(i)} = \{y_1^{(i)}, y_2^{(i)}, \ldots, y_K^{(i)}\}$.

Question

Actually, we would only need $K - 1$ outputs to create a soft classifier for K classes. Why?

Let us for the sake of simplicity assume we have only one feature. The preactivations are (suppressing the index i)

$$z_1 = w_{1,0} + w_{1,1} x_1, \tag{26.12}$$

$$z_2 = w_{2,0} + w_{2,1} x_1, \tag{26.13}$$

and so on until the class K:th class

$$z_K = w_{K,0} + w_{K,1} x_1, \tag{26.14}$$

and the model is specified in terms of these K linear functions $z_j^{(i)}$, known as log-odds or **logits**, that are transformed into class probabilities $y_j^{(i)} = y(z_j^{(i)})$.

Class probabilities: The Softmax function

The transformation of the multiple outputs, as described above, to probabilities for belonging to any of K different classes can be achieved via the so-called *Softmax* function.

The Softmax function is used in various multiclass classification methods, such as multinomial logistic regression (also known as softmax regression), multiclass linear discriminant analysis, naive Bayes classifiers, and artificial neural networks. Specifically, the predicted probability for the k:th class given a sample vector $\boldsymbol{x}^{(i)}$ and a weighting vector $\boldsymbol{w}$ is (with one independent variable):

$$\mathbb{P}(t^{(i)} = k | \boldsymbol{x}^{(i)}, \boldsymbol{w}) = \frac{\exp\left(w_{k,0} + w_{k,1} x_1^{(i)}\right)}{\sum_{l=1}^{K} \exp\left(w_{l,0} + w_{l,1} x_1^{(i)}\right)}. \tag{26.15}$$

which means that the discrete set of probabilities is properly normalized.

Our earlier discussions were all specialized to the case with two classes only. It is easy to see from the above that what we derived earlier is compatible with these equations.

26.4. Machine Learning: First Examples

In this section we consider two simple, but powerful, machine learning approaches: the k-nearest neighbor algorithm and the linear model (already encountered in the first part on scientific inference).

We will first consider the regression problem using the linear model, which by now should be a rather familiar task. However, we will dress it in the machine learning language.

Then we will consider a simple binary classification problem with two predictor variables. The linear model makes strong assumptions concerning structure and thereby yields stable but possibly inaccurate predictions. The method of k nearest neighbors makes very mild structural assumptions which yields predictions that are often accurate but the results are unstable and strongly dependent on the training data.

Linear regression

Regression modeling deals with the description of a **response** variable(s) $\boldsymbol{y}$ and how it varies as function of some **predictor** variable(s) $\boldsymbol{x}$. In machine learning the predictor variables are often known as *features*.

Here we will consider the special case in which there is a single response variable but several features.

When performing a regression analysis we will have access to a set of training data $\mathcal{T}$ that consists of:

- N cases, indexed by $i = 1, 2, \ldots, N$

For each case there is a

- response variable y_i (observation);
- vector of features $\boldsymbol{x}_i$ (input).

Features
The key to a successful linear regression analysis is to identify the most relevant features. In physics, these would correspond to a set of basis functions. It is common to include a bias term as a feature, that is we might start the feature indexing at zero and assign $x_0 = 1$.
Assume that there are p features, including the bias. We will use the (possibly confusing) notation

- $\boldsymbol{x}_i = \left[x_{0,i}, x_{1,i}, \ldots, x_{p-1,i}\right]$, with $x_{0,i} = 1 \, \forall i$.

Following Linear models we will apply the linear machine learning model

$$\hat{\boldsymbol{y}}\left(\boldsymbol{w}\,;\,\boldsymbol{x}_i\right) = \boldsymbol{x}_i \cdot \boldsymbol{w}, \tag{26.16}$$

where $\boldsymbol{w}$ is a $p \times 1$ column vector of model parameters. These are known as weights in this context. We might also simplify the notation and write $\hat{y}_i = \hat{\boldsymbol{y}}\left(\boldsymbol{x}_i\right)$.

A machine learning regression analysis aims at finding the model parameters $\boldsymbol{w}$ such that a selected cost function is minimized. This optimization step is the learning part of the model. The cost function is supposed to measure the performance of the model

The p features for the N cases in the training data set are collected in the $N \times p$ design matrix $\mathbf{X}$ such that we get the simple matrix form

$$\hat{\boldsymbol{y}} = \mathbf{X}\boldsymbol{w} \tag{26.17}$$

The most common metric to quantify the performance of the model is the mean-squared error (MSE)

$$\mathrm{MSE}(\boldsymbol{w}) = \frac{1}{N}\sum_{i=1}^{N}\left(y_i - \hat{y}_i\right)^2. \tag{26.18}$$

Using this metric as a cost function, $C(\boldsymbol{w})$, and using the more compact matrix-vector notation we have

$$C(\boldsymbol{w}) = \frac{1}{N}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}). \tag{26.19}$$

In general, the minimization of the cost function is an optimization problem that must be approached with numerical techniques such as *gradient descent*. For linear regression, however, the learning can be achieved analytically. The minimum of Eq. (26.19) is found by solving the normal equation, and if $\boldsymbol{X}^T\boldsymbol{X}$ is invertible we have the optimal model parameters

$$\boldsymbol{w}^* = \left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y}. \tag{26.20}$$

Training scores

The mean-squared error (26.18) is just one possible choice for the cost function. In general, such metrics are also known as **training scores**.

Other choices are the **mean absolute error** (MAE), defined as

$$\mathrm{MAE}(\boldsymbol{w}) = \frac{1}{N}\sum_{i=1}^{N}\left|y_i - \hat{y}_i\right|, \tag{26.21}$$

and the $R2$ score, also known as *coefficient of determination*

$$\mathrm{R2}(\boldsymbol{w}) = 1 - \frac{\sum_{i=1}^{N}\left(y_i - \hat{y}_i\right)^2}{\sum_{i=1}^{N}\left(y_i - \bar{y}\right)^2}, \tag{26.22}$$

where $\bar{y} = \frac{1}{N}\sum_{i=1}^{N} y_i$ is the mean of the data.

For classification tasks one might also use the misclassification cost function (see Exercise 26.2) or cross entropy. Misclassification is the relative number of misclassified instances in a data set and is therefore the same as $1 - \alpha$, where α is the accuracy.

Example 26.1 (Binary classification)

Fig. 26.2 shows a scatterplot of training data that is a function of two predictor variables $\boldsymbol{x}_i = (x_1, x_2)_i$. The output y is a class that can be either Blue or Red.

The procedure and code that generate this data is included in the hidden code cell below. However, here we assume that we don't know the underlying data-generating process. Our aim is to construct machine learning algorithms that can be trained on the collected data and make predictions for future data. The task can then be expressed as: Develop a computer program that can learn from labeled data $\mathcal{T} = \{\boldsymbol{x}_i, y_i\}_{i=1}^N$ and make a prediction of the class $y^{\odot}$ that a new input $\boldsymbol{x}^{\odot}$ belongs to.

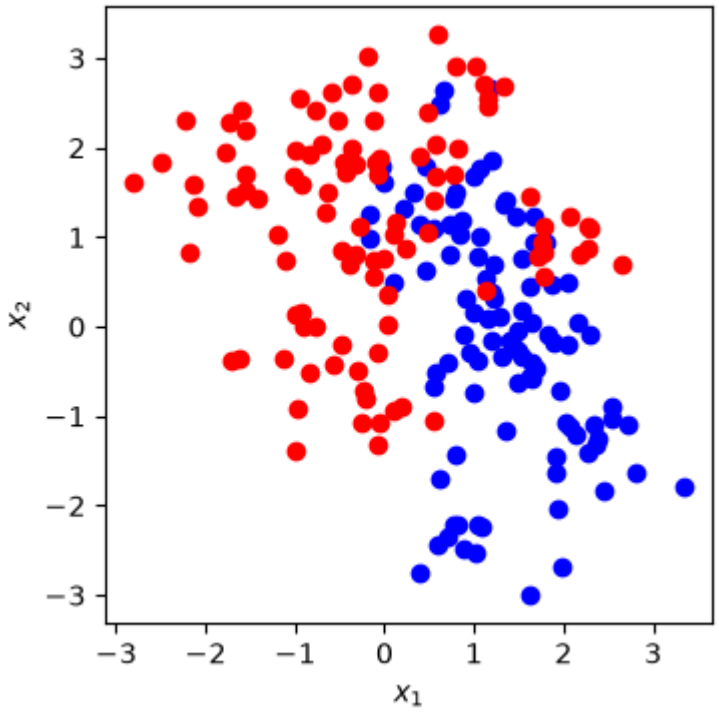
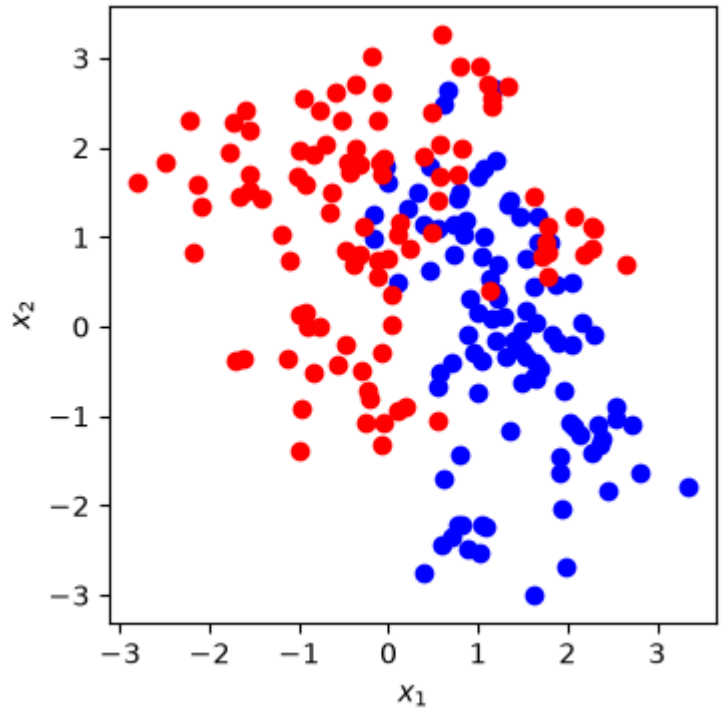


Fig. 26.2 Labeled training data for a binary classification problem.

▼ **Hide code cell content**



Linear classification

Let's consider an application of the linear model for the binary classification task of Example 26.1. We label these classes as 0 for Blue and 1 for Red such that $y_i \in \{0, 1\}$. Let us first introduce a linear model whose output we label as z

$$\boldsymbol{z} = \boldsymbol{X}\boldsymbol{w}.$$

We again include a bias feature in the design matrix $\boldsymbol{X}$ such that $\boldsymbol{w} = [w_0, w_1, w_2]^T$. We note that each model output z is a continuous variable. How could this be used for the task of a binary classifier?

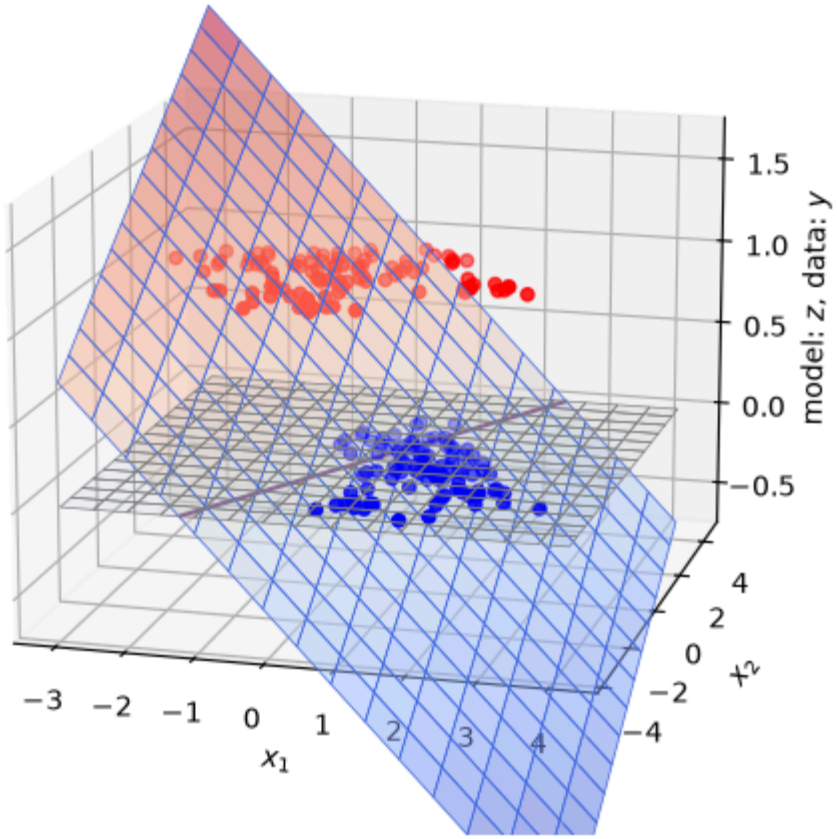


Fig. 26.3 This plane is the linear regression (ordinary least squares) fit of a model $z = w_0 + w_1 x_1 + w_2 x_2$ to the training data of Fig. 26.2. For visualization we indicate the line where it crosses the $z = 0$ plane and use red (blue) color to emphasize regions with positive (negative) z-value.

The perceptron classifier

We can achieve a mapping from the continuous output z of the linear regressor to a discrete one $\hat{y} \in \{0, 1\}$ using the *perceptron* model which is usually formulated using the sign function as

$$\hat{y} = \frac{\text{sign}(z) + 1}{2}. \tag{26.23}$$

A binary classifier, providing a discrete output, can then be obtained according to the simple rule

$$\hat{y} = \hat{y}(z) = \begin{cases} \text{Red} & \text{if } z > 0 \\ \text{Blue} & \text{if } z < 0. \end{cases}$$

The two predicted classes will be separated by a *decision boundary* $\{\boldsymbol{x} \,:\, \boldsymbol{x}\boldsymbol{w}^* = 0\}$. This boundary becomes a straight line in this case.

The soft classifier

In many cases, it is more favorable to have a "soft" classifier that outputs a probability for an output to belong to a given category rather than providing a hard decision. Logistic regression is the most common example of such a soft classifier. In logistic regression, the probability that input $\boldsymbol{x}$ belongs to the class $c = 1$ is given by the *logistic* function (an example of an S-shaped or *sigmoid* function),

$$\hat{y}(z) = \mathbb{P}(y(\boldsymbol{x}) = 1) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}, \tag{26.24}$$

where the so-called *preactivation* $z = \boldsymbol{x} \cdot \boldsymbol{w}$.

- Note that $1 - \hat{y}(z) = \hat{y}(-z)$.
- The sigmoid function can be motivated in several different ways. E.g. in information theory this function represents the probability of a signal $s = 1$ rather than $s = 0$ when transmission occurs over a noisy channel.

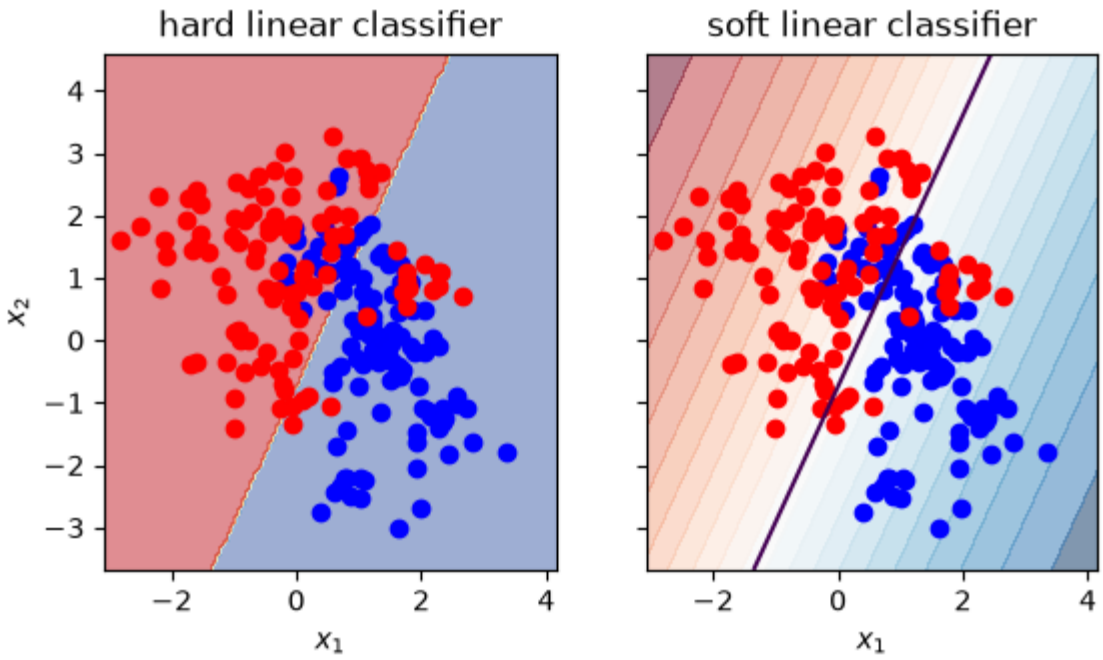


Fig. 26.4 Results of the hard (perceptron) and soft (sigmoid) linear classifier on the data from Example 26.1. The model predictions for the hard classifier are represented by the red and blue regions on either side of the sharp decision boundary (left panel). For the soft classifier (right panel) the colors represent the model prediction for the probability of belonging to the red class. The black line marks the decision boundary where the model gives no preference for either class $\mathbb{P}(\hat{y}^{\odot} = \text{Red}\ ) = 0.5$. See also Fig. 26.3 for the corresponding preactivation z from the linear regression model.

k nearest neighbors classification

Nearest-neighbor methods use the closest observations in the training data $\mathcal{T}$ to make a prediction. The definition of "closest" is usually the Euclidean distance in the input space, i.e., the length $|\boldsymbol{x}^{\odot} - \boldsymbol{x}_i|$. Specifically, the k nearest neighbor prediction for a target would be given by the average

$$\tilde{y}(\boldsymbol{x}^{\odot}) = \frac{1}{k} \sum_{\boldsymbol{x}_i \in N_k(\boldsymbol{x}^{\odot})} y_i, \tag{26.25}$$

where $N_k(\boldsymbol{x}^{\odot})$ is the neighborhood of $\boldsymbol{x}^{\odot}$ defined by the k nearest points $\boldsymbol{x}_i$ in the training data set. Forced to make a hard decision we would use the majority vote and assign $\hat{y}^{\odot} = 1$ if $\tilde{y}(\boldsymbol{x}^{\odot}) > 0.5$, and $\hat{y}^{\odot} = 0$ if $\tilde{y}(\boldsymbol{x}^{\odot}) \leq 0.5$.

As shown in Fig. Fig. 26.5 there are much fewer misclassifications in the training data set when using the k nearest neighbor method, than when using the linear classifier. However, one should also note that none(!) of the training data will be misclassified when using $k = 1$, while the number will be an increasing function of k. Is it always best to use $k = 1$ nearest neighbors? No, that cannot be. One realizes that the decision boundaries will vary greatly when using a different training set, giving very different predictions for a validation set of data. This is an example of overfitting, but this time in the context of a classification problem.

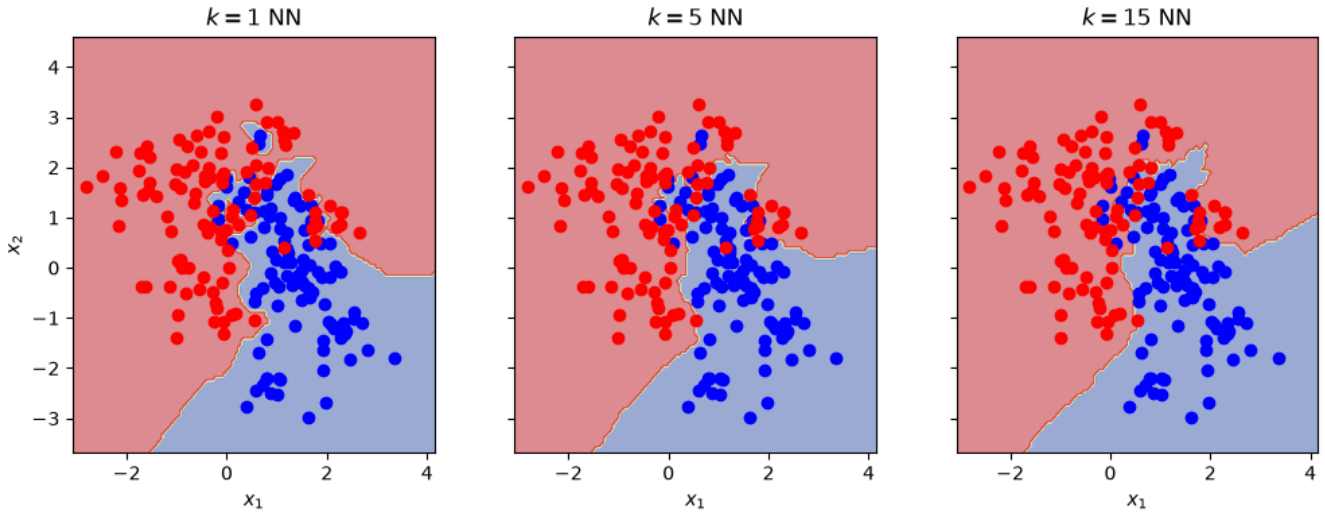


Fig. 26.5 Results of kNN classifier, with $k = 1, 5, 15$, on the data from Example 26.1. With $k = 1$ we will trivially reproduce all training data, but the model will be heavily overfitted to the actual set of training data. The $k = 15$ model will misclassify some of the training data but will (probably) generalize better when classifying new data.

Data normalization

The k-nearest neighbor classifier is an example of a method for which it becomes very important to normalize the input data. Imagine that one of the two inputs, x_1, varies in a range $[100, 1000]$ whereas the other one, x_2, is limited to $[0, 1]$. It is then obvious that the Euclidean distance will be dominated by the distance in x_1 and our model will be almost insensitive to x_2.

We can solve this by normalizing the input parameters by some transformation and scaling. For example, we could consider instead $(\tilde{x}_1, x_2)$, where $\tilde{x}_1 \equiv \frac{x_1 - 100}{900}$. Now both parameters will be in the $[0, 1]$ range and the method will work much better.

Another common normalization scheme is known as standardization

$$\tilde{x}_i = \frac{x_i - \bar{x}_i}{\sigma_i}, \tag{26.26}$$

where $\bar{x}_i$ and σ_i are the mean and the standard deviation, respectively, for each input variable evaluated over the training data set.

Data normalization is a common procedure before any machine learning task. In fact, it is common to also normalize the output. However, remember that:

- Use only the training data for normalization. You will build a bias into the training procedure if you also include validation or test data into the computation of, e.g., the data mean and standard deviation.
- Apply the same normalization on any new data (including validation data).
- Don't forget to transform back before presenting results.

Code examples: binary classifiers

Data normalization

```
def standardize_data(train_data,verbose=False):
    "Standardize data to mean zero and std=1."
    sample_mean=train_data.mean(axis=0)
    sample_std=train_data.std(axis=0)
    if verbose:
        print('mean (before standardization) = ',sample_mean)
        print(' std (before standardization) = ',sample_std)
    return (train_data - sample_mean) / sample_std, sample_mean, sample_std

# Apply to our training data set
std_train_data, sample_mean, sample_std = standardize_data(train_data,verbose=True)
```

```
mean (before standardization) =  [0.56877654 0.49681303 0.5       ]
 std (before standardization) =  [1.21440984 1.39206493 0.5       ]
```

Note in particular that we normalize both input and output. This implies that the transformed data $(\tilde{\boldsymbol{x}}, \tilde{\boldsymbol{y}})$ has mean zero and variance one, and consequently that $\tilde{y}_i \in \{-1, 1\}$.

Linear classifier(s)

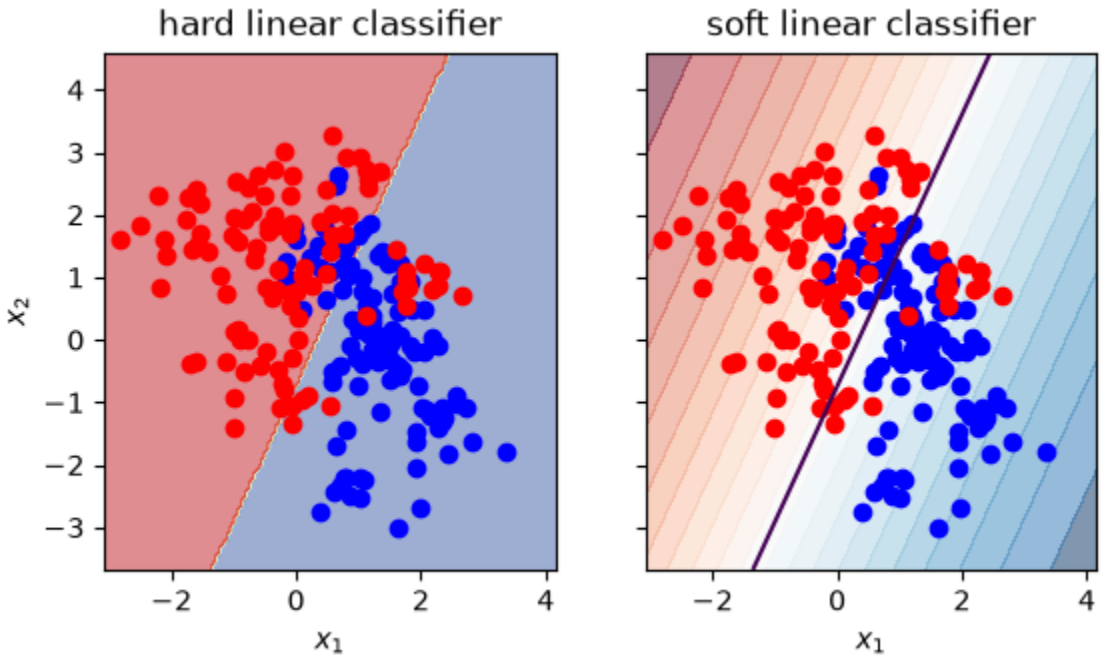
Create and fit the linear model.

```
# Set up design matrix
# correct shape, column 0 will be the bias feature
X = np.ones((len(std_train_data),3))
# column 1 corresponds to x1
X[:,1] = std_train_data[:,0]
# column 2 corresponds to x2
X[:,2] = std_train_data[:,1]

# Targets (responses)
y = std_train_data[:,2]

#ols estimator for model parameter theta
ols_cov   = np.linalg.inv(np.matmul(X.T,X))
ols_xTd   = np.matmul(X.T,y)
ols_theta = np.matmul(ols_cov,ols_xTd)

print(f'theta_ols \t{ols_theta}')
```

```
theta_ols       [-1.40436100e-17 -5.06798079e-01  2.67244070e-01]
```

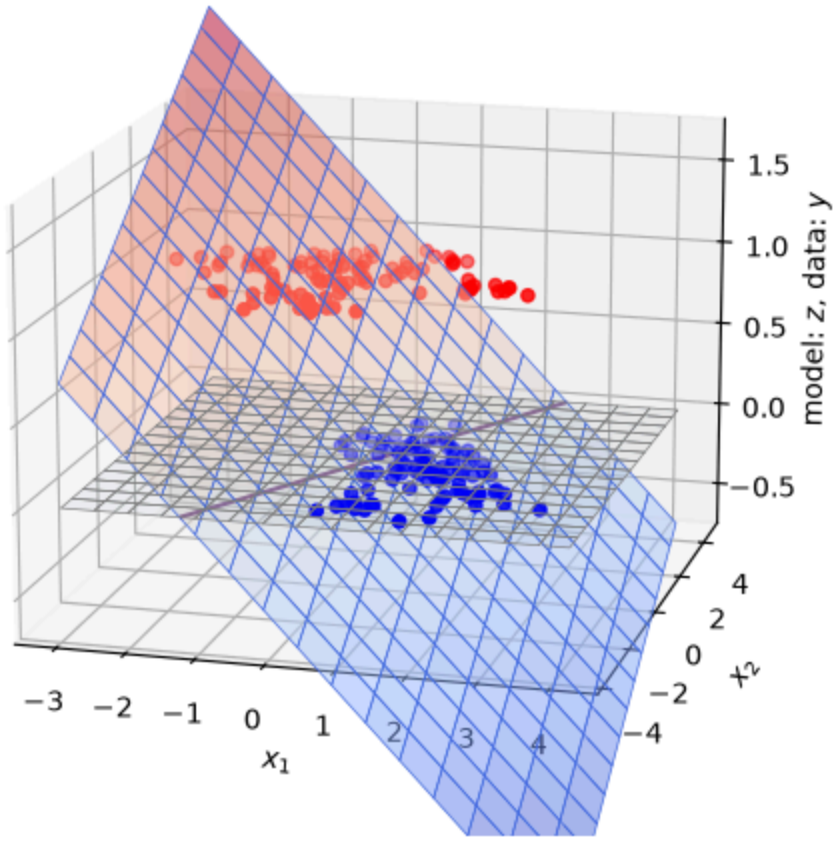
Make predictions using both hard and soft classifiers on a grid. Note how we use the standardized data for model training (above) and predictions (below), while we transform the input coordinates back to the original scale for the plot.

```
# Grid for predictions
x1, x2 = np.mgrid[-3:3:.05, -3:3:.05]
x1x2_grid = np.dstack((x1, x2))

# predictions of the linear classifier on the grid
# liner model output
z_grid = ols_theta[0] + ols_theta[1]*x1x2_grid[:,:,0] + ols_theta[2]*x1x2_grid[:,:,1]
# perceptron (hard classifier) prediction
yhat_hard_grid = np.sign(z_grid)
# sigmoid (soft classifier) prediction
yhat_soft_grid = 1/(1+np.exp(-z_grid))

# Final results are transformed back to original coordinates
x1_orig = sample_std[0]*x1 + sample_mean[0]
x2_orig = sample_std[1]*x2 + sample_mean[1]

fig_linear_classifier,axs = plt.subplots(1,2,sharey=True)

ax=axs[0]
ax.set_ylabel(r'$x_2$')
im = ax.contourf(x1_orig, x2_orig, yhat_hard_grid,cmap=plt.cm.RdYlBu_r,alpha=0.5);
ax.set_aspect('equal', 'box')
ax.set_title('hard linear classifier')

ax=axs[1]
im = ax.contourf(x1_orig, x2_orig, yhat_soft_grid,levels=20,cmap=plt.cm.RdBu_r,alpha=0.5,
vmin=0.0, vmax=1.0, extend='both');
ax.contour(x1_orig, x2_orig, yhat_soft_grid,levels=[0.5])

for iclass, color in enumerate(classes):
    x_data = train_data[train_data[:,2]==iclass,:2]
    for ax in axs[:2]:
        ax.set_xlabel(r'$x_1$')
        ax.scatter(x_data[:,0],x_data[:,1],color=color)
ax.set_aspect('equal', 'box')
ax.set_title('soft linear classifier');

glue("fig_linear_classifier", fig_linear_classifier, display=False);
```



For visualization we also make a 3D plot of the surface $z = z(x_1, x_2) = w_0 + w_1 x_1 + w_2 x_2$ that results from the linear regression. This is the plane that minimizes the sum of squared distances to the data.

```
from mpl_toolkits.mplot3d import axes3d

fig_linear_classifier_plane = plt.figure()
ax = fig_linear_classifier_plane.add_subplot(projection='3d')

# Plot the 3D surface
ax.plot_surface(x1_orig, x2_orig, z_grid, edgecolor='royalblue', lw=0.5, rstride=8, cstride=8,
alpha=0.5, cmap=plt.cm.coolwarm)

# Plot the x1x2 surface
ax.plot_surface(x1_orig, x2_orig, np.zeros_like(x1_orig), edgecolor='grey', lw=0.5, rstride=8,
cstride=8, alpha=0.05)

for iclass, color in enumerate(classes):
    x_data = train_data[train_data[:,2]==iclass,:2]
    ax.scatter(x_data[:,0],x_data[:,1], zs=iclass, zdir='z',color=color)

ax.contour(x1_orig, x2_orig, z_grid,levels=[0.0],)

ax.set_xlabel(r'$x_1$')
ax.set_ylabel(r'$x_2$')
ax.set_zlabel(r'model: $z$, data: $y$')

ax.set_zlim(-0.75, 1.75)

ax.view_init(elev=15., azim=-75, roll=0)
fig_linear_classifier_plane.tight_layout()
glue("fig_linear_classifier_plane", fig_linear_classifier_plane, display=False);
```

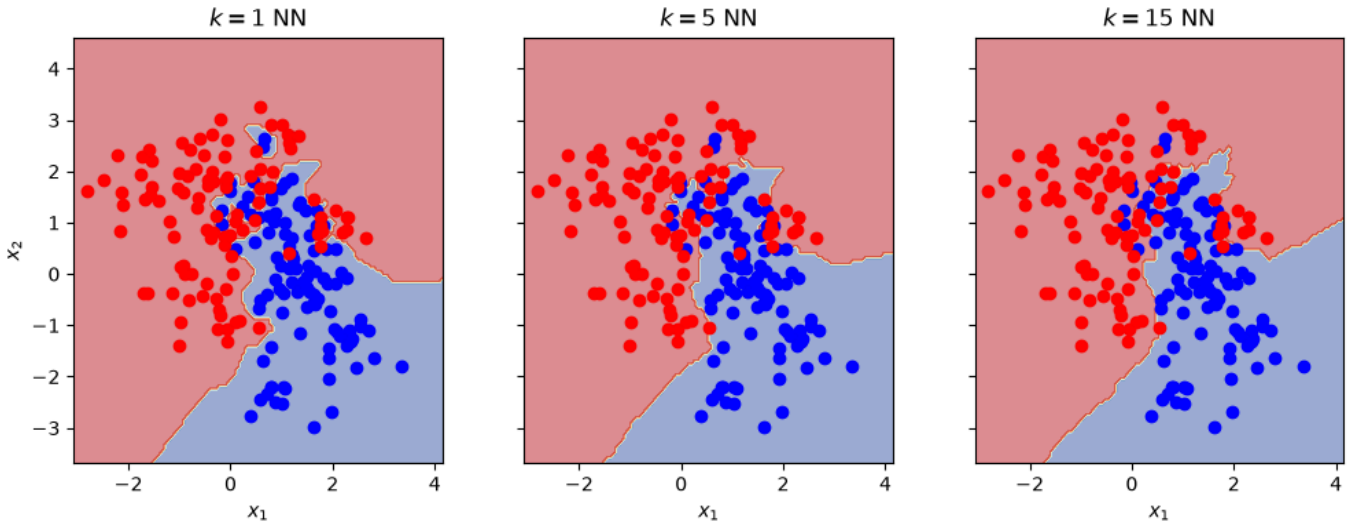


kNN classifier

```
# calculate the Euclidean distance between two vectors
def euclidean_distance(pos1, pos2):
    return np.linalg.norm(pos1-pos2)

def knn_classifier(xstar,k=5):
    y_train = std_train_data[:,2]
    # compute distances to all training data
    distances = []
    for x_train in std_train_data[:,:2]:
        distances.append(euclidean_distance(xstar,x_train))
    # sort targets by closest distance
    y_sorted = [y for _, y in sorted(zip(distances, y_train))]
    # majority vote (targets are either +1 or -1)
    try:
        return np.sign(np.sum(y_sorted[:k]))
    except TypeError:
        # allow k to be list-like
        return [np.sign(np.sum(y_sorted[:ki])) for ki in k]
```

We will explore three different settings with $k = 1, 5, 15$.

```
k_list=[1,5,15]
yhat_knn_grid = np.zeros((len(x1),len(x2),len(k_list)))
for ix1 in range(x1x2_grid.shape[0]):
    for ix2 in range(x1x2_grid.shape[1]):
        yhat_knn_grid[ix1,ix2,:] = np.array(knn_classifier(x1x2_grid[ix1,ix2,:],k=k_list))

fig_kNN_classifier,axs = plt.subplots(1,len(k_list),figsize=(12,4),sharey=True)
axs[0].set_ylabel(r'$x_2$')
for ik,k in enumerate(k_list):
    ax=axs[ik]
    ax.set_xlabel(r'$x_1$')
    im = ax.contourf(x1_orig, x2_orig,\
        yhat_knn_grid[:,:,ik],cmap=plt.cm.RdYlBu_r,alpha=0.5);
    for iclass, color in enumerate(classes):
        x_data = train_data[train_data[:,2]==iclass,:2]
        ax.scatter(x_data[:,0],x_data[:,1],color=color)
    ax.set_aspect('equal', 'box')
    ax.set_title(rf'$k={k}$ NN')

glue("fig_kNN_classifier", fig_kNN_classifier, display=False);
```



Exercises

Exercise 26.1 (Sigmoid decision boundary)

Consider the sigmoid classifier (26.24) with the preactivation determined by the scalar product of predictor variables and model parameters and the output providing the probability $\mathbb{P}(y(\boldsymbol{x}) = 1)$.

- Explain why the sigmoid classifier can be considered a non-linear model.
- Explain why the decision boundary of a sigmoid classifier is still a straight line despite the non-linear dependence on the model parameters.

Exercise 26.2 (Misclassification cost function)

The performance of a hard classifier can be evaluated using a misclassification cost function

$$C(\boldsymbol{w}) = \frac{1}{N}\sum_{i=1}^{N}(1 - \delta_{\hat{y}_i, y_i}), \tag{26.27}$$

where each term evaluates to zero if the classification is correct.

- What would be different in our training when using such a cost function?
- Do you expect the parameters to be A) the same, B) similar, C) very different compared to the model where the mean-squared error is used to define the cost function?

Exercise 26.3 (Validation errors)

Generate 2000 validation data with the code used in the binary classification Example 26.1. Evaluate the misclassification error (26.27) for both the training data and this new validation data using:

- The hard linear classifier
- The kNN classifier with different $k \in [1, 20]$.

Comment on the results.

Exercise 26.4 (kNN for regression)

How could you use the kNN model for a regression problem? In particular, how would you decide the prediction $\hat{y}^{\odot}$ for a specific $\boldsymbol{x}^{\odot}$?

Exercise 26.5 ($R2$ score)

The $R2$ score can be said to describe the fraction of variations in the data that is predictable from the independent variables using the model.

- Try to understand this statement.
- What is the value of the $R2$ score for a perfect model?
- What does it mean when $R2 \to 0$?

Solutions to exercises

Solution to Exercise 26.1 (Sigmoid decision boundary)

The mathematical model for the sigmoid outputs is not linear in the parameters. The decision boundary corresponds to $\hat{y} = 0.5$, which implies $e^{-z} = 1$, or $\boldsymbol{x} \cdot \boldsymbol{w} = 0$. This is a straight line.

Solution to Exercise 26.4 (kNN for regression)

You could use the same algorithm but your training data would have continuous responses y_i and the model must give a continuos variable $\hat{y}^{\odot}$ as the prediction. One way would be to use the average

$$\hat{y}(\boldsymbol{x}^{\odot}) = \frac{1}{k} \sum_{\boldsymbol{x}_i \in N_k(\boldsymbol{x}^{\odot})} y_i,$$

Solution to Exercise 26.5 ($R2$ score)

For a perfect model we would have $y_i = \hat{y}_i$ for all i and therefore $R2 = 1$.

A model that simply predicts the mean of the training data, $\hat{y}^{\odot} = \bar{y}$, would be one that is completely independent of the independent variables. It would also give $R2 = 0$ when evaluated over to the training data. Negative values for $R2$ would mean that your model is even worse than that simple mean-value model.

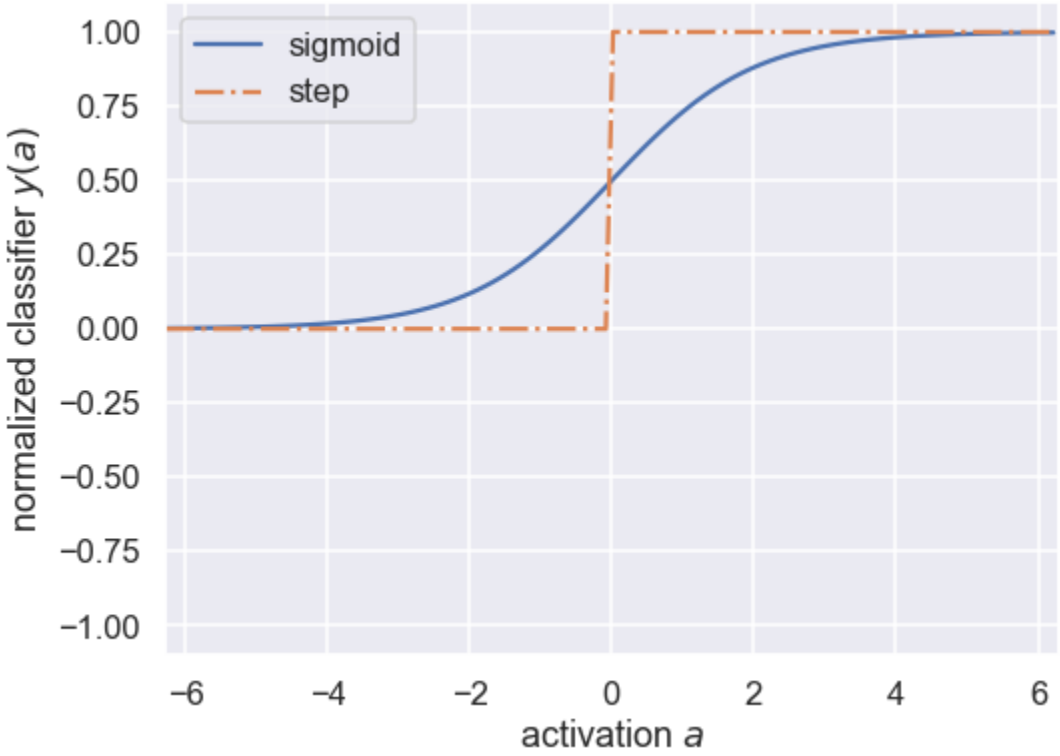
26.5. Exercise: Logistic Regression and neural networks

```
%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt

import seaborn as sns
sns.set()
sns.set_context("talk")
```

Examples of classifier functions used in logistic regression and neural networks

The following code plots the sigmoid and the step function, two common classifier functions used in neural networks (and logistic regression).

```
z = np.arange(-2*np.pi, 2*np.pi, .1)
sigma_fn = np.vectorize(lambda z: 1/(1+np.exp(-z)))
sigma = sigma_fn(z)

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111)
ax.plot(z, sigma, label='sigmoid')

# Step Function
step_fn = np.vectorize(lambda z: 1.0 if z >= 0.0 else 0.0)
step = step_fn(z)
ax.plot(z, step, '-.', label='step')

ax.set_ylim([-1.1, 1.1])
ax.set_xlim([-2*np.pi,2*np.pi])
ax.set_ylabel('normalized classifier $y(z)$')
ax.set_xlabel(r'preactivation $z$')
ax.legend(loc='best');
```



Exercise

- Add the tanh function.
- Add the ReLU, leaky ReLU, and ELU activation functions (find the functional forms online)

A simple classification problem

`scikit-learn` includes various random sample generators that can be used to build artificial datasets of controlled size and complexity.

For example, `make_moons` generates two overlapping half cricles with optional Gaussian noise.

```
from sklearn import datasets, linear_model
```

Logistic regression using `scikit-learn`

```
np.random.seed(0)
X, y = datasets.make_moons(200, noise=1.20)
```

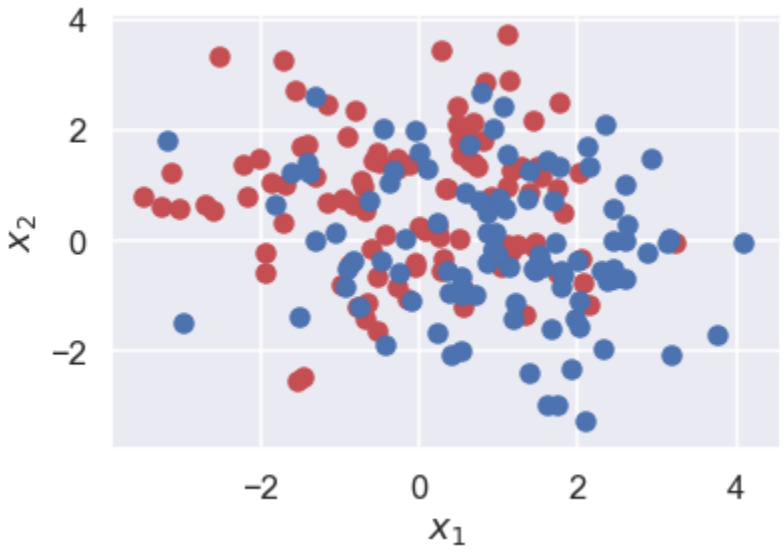
```
X.shape
```

```
(200, 2)
```

```
y.shape
```

```
(200,)
```

```
fig,ax=plt.subplots(1,1)
ax.scatter(X[y==0,0],X[y==0,1],c='r')
ax.scatter(X[y==1,0],X[y==1,1],c='b')
ax.set_xlabel(r'$x_1$')
ax.set_ylabel(r'$x_2$');
```




```
clf = linear_model.LogisticRegressionCV(cv=5,penalty='l2')
clf.fit(X, y)
```

```
LogisticRegressionCV(Cs=10, class_weight=None, cv=5, dual=False,
                     fit_intercept=True, intercept_scaling=1.0, l1_ratios=None,
                     max_iter=100, multi_class='auto', n_jobs=None,
                     penalty='l2', random_state=None, refit=True, scoring=None,
                     solver='lbfgs', tol=0.0001, verbose=0)
```

```
clf.coef_
```

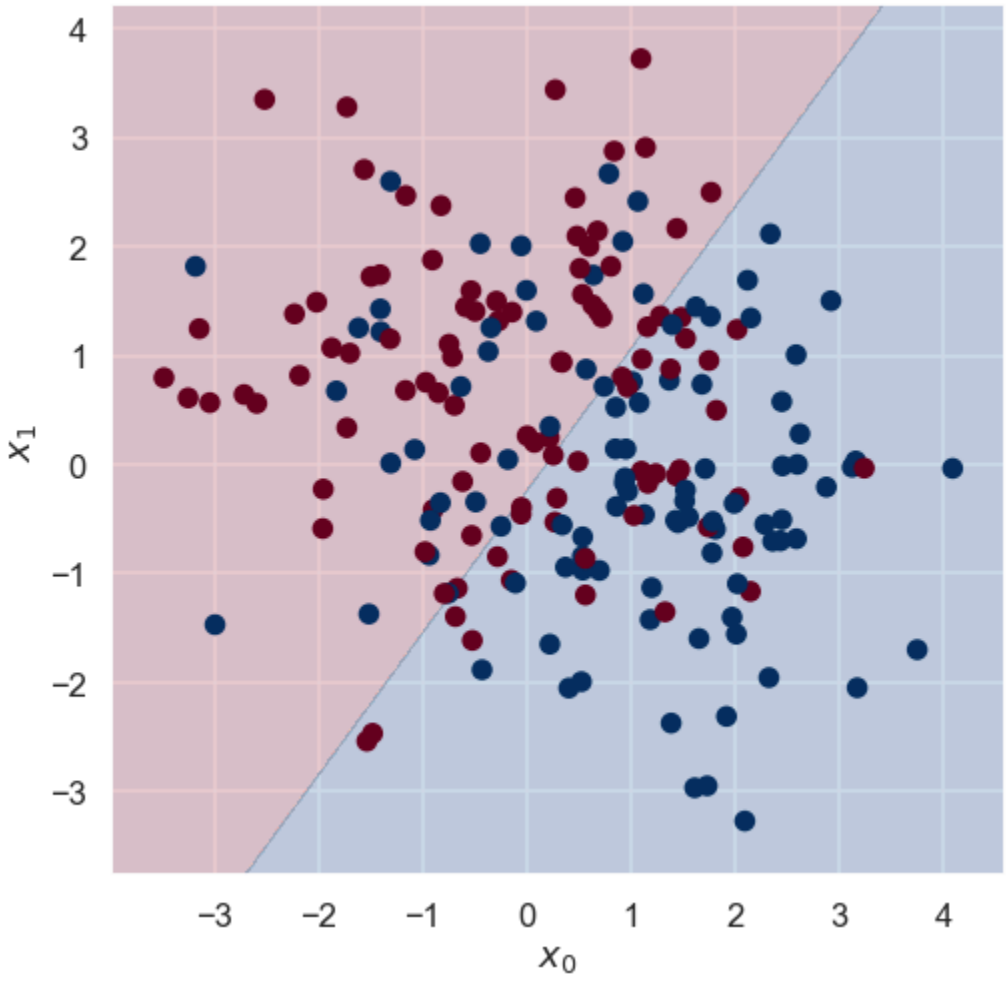
```
array([[ 0.18505795, -0.14192397]])
```

```
clf.intercept_
```

```
array([-0.03432889])
```

```
# Helper functions to visualize the data and the decision boundary
#
def visualize(X, y, clf,ax=[]):
    plot_decision_boundary(lambda x: clf.predict(x), X, y, ax=ax)

def plot_decision_boundary(pred_func, X, y,ax=[]):
    # Set min and max values and give it some padding
    x0_min, x0_max = X[:, 0].min() - .5, X[:, 0].max() + .5
    x1_min, x1_max = X[:, 1].min() - .5, X[:, 1].max() + .5
    h = 0.01
    # Generate a grid of points with distance h between them
    xx0, xx1 = np.meshgrid(np.arange(x0_min, x0_max, h), np.arange(x1_min, x1_max, h))
    # Predict the function value for the whole gid
    Z = pred_func(np.c_[xx0.ravel(), xx1.ravel()])
    Z = Z.reshape(xx0.shape)
    # Plot the contour and training examples
    if ax:
        ax.contourf(xx0, xx1, Z, cmap=plt.cm.RdBu, alpha=0.2)
        ax.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdBu)
    else:
        plt.contourf(xx0, xx1, Z, cmap=plt.cm.RdBu, alpha=0.2)
        plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdBu)
```

```
fig,ax = plt.subplots(figsize=(8,8))
visualize(X, y, clf,ax)
ax.set_xlabel(r'$x_0$')
ax.set_ylabel(r'$x_1$');
```



Sub-task

- Generate 200 new samples for use as test data.
- What is the accuracy of your binary classifier on
  - The training data
  - The test data?

Exercise: Develop your own logistic regression binary classifier

Implement your own logistic regression binary classifier by modifying the function definitions below

1. Define the sigmoid activation function.
2. Define the feed-forward function that returns the output of the neuron given some input and weights.
3. Define the learning algorithm that:
   - use the cross entropy cost function defined in the lecture. Add an L2 regularizer with `alpha=0.1`
   - computes gradients of the cross entropy cost function (i.e. back propagation)
   - modifies the parameters with a learning rate parameter `eta = 0.01`.
   - returns the new weights.

Then:

1. Train the binary classifier on the data set. Perform rather many iterations.
2. Plot the decision boundary and compare with the `scikit-learn` implementation above.

```
def sigmoid(z):
    '''
    Sigmoid activation function

    Args:
        z (array[float]): preactivation

    Returns:
        y (float): the output of the neuron
    '''
    # Add code here (remove the dummy lines)
    y = None
    return y

def single_neuron(x, w):
    """
    Single neuron prediction.

    Args:
        x (array[float]): input to the neuron
        w (array[float]):

    Returns:
        y (float): the output of the neuron
    """
    if len(np.array([x1[0],x2[1]]).shape)==1:
        x = np.append(1, x)
    else:
        m = len(x)
        x = np.c_[np.ones((m, 1)), x]
    assert(len(w)==x.shape[-1])

    z = np.dot(x,w)
    return sigmoid(z)

def single_neuron_binary_classifier(x, t, iters=10000, alpha=0.1, eta0=0.01):
    """
    Makes predictions for a single neuron binary classifier

    Args:
        x (array[float]): an array of input data
        t (float): target output for each data points
        iters (int): number of iterations to apply gradient descent
        alpha (float): a rescaling parameter for the weights
        eta (float): learning rate

    Return
        w (array[float]): the trained weights of the classifier
    """
    # Insert code here (remove the dummy lines)
    w = None
    return w
```

Sub-tasks and follow-up questions

- Plot the decision boundary $p(t = 1|x_1, x_2, w_0^*, w_1^*) = 0.5$, where w^* are the optimized weights.
- Why does it correspond to a straight line?
- What is the accuracy for the training set?
- Create a test set. What is the accuracy on that?
- What would be needed to construct a better classifier that could handle the complicated class boundary?

Exercise: Create a neural net binary classifier

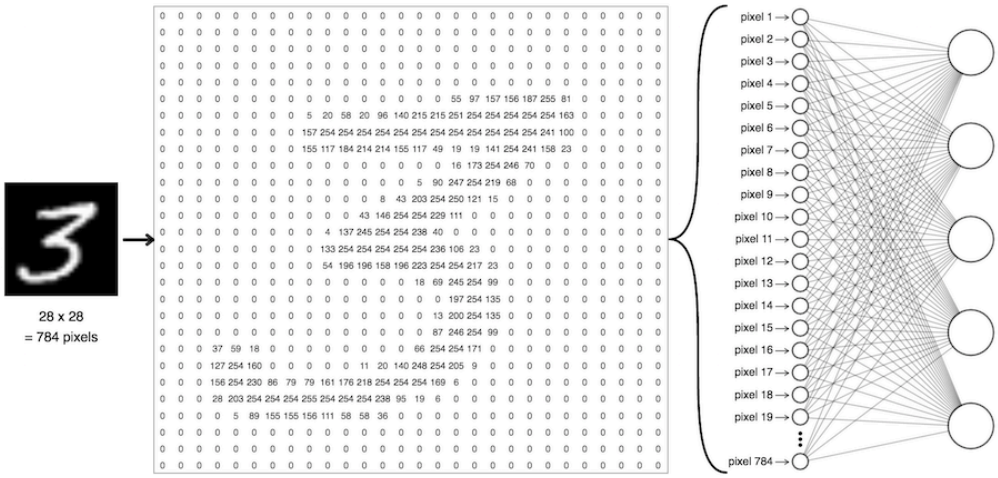
Import and use tensorflow to create a (non-linear) binary classifier

1. Build a keras sequential model as in the `demo-NeuralNet.ipynb` example. A single hidden layer is sufficient.
2. Print a summary of your model. How many parameters does it have?
3. Train the binary classifier on the training data set. Try with different activation functions and different number of epochs of training.
4. Plot the decision boundary and compare with the Logistic Regression implementations above.
5. What is the accuracy on the test set?

```
# Install TensorFlow by updating the conda environment
# Download the latest version of the environment.yml file
# (with tensorflow on the last line)
# Then run:
# conda env update -f /path/to/environment.yml

import tensorflow as tf
```

27. Artificial neural networks

Artificial neural networks are computational systems that can be trained to perform tasks by learning from examples, generally without having to be programmed with any task-specific rules. Its functionality is supposed to mimic a biological neural system, wherein neurons interact by sending signals to each other. In the computer, these signals are generated by mathematical functions in nodes organized in connected layers. All layers can contain an arbitrary number of nodes, and each connection is represented by a weight variable.



The chapter is organized as follows:

- Neural network types and architecture introduces the terminology, the main types of neural networks and their architecture, and discusses some limitations of supervised learning with deep networks.
- Feed-forward networks: mathematical model formulates fully-connected feed-forward networks mathematically. It introduces the matrix-vector notation, common activation functions, and the ingredients of the learning algorithm, and ends with exercises.
- Neural networks: Backpropagation derives the back-propagation algorithm for computing the gradient of the cost function, and discusses vanishing and exploding gradients.
- Demo: Feed-forward neural network for a function in PyTorch trains a network in PyTorch to approximate a simple one-dimensional function.
- Demo: MNIST classifier in PyTorch builds a PyTorch classifier for handwritten digits (MNIST).

27.1. Neural network types and architecture

Terminology

When describing a neural network algorithm we typically need to specify three key ingredients:

Architecture

The architecture specifies the topology (the number and connectivity of nodes) and parameters that are involved in the network. For example, the parameters involved in a neural network could be the weights that multiply signals between the neurons, including bias weights, and other parameters determining the form of the activition functions.

Activation rule

Most neural network models have short time-scale dynamics. Local (possibly node-specific) rules define how the activities change in response to signals from other nodes. Typically the activation rules are functions of the weights in the network and possibly other activation-function hyperparameters.

Learning algorithm

The learning algorithm specifies the way in which the neural network's weights are optimized during training. This learning is usually viewed as taking place on a longer time scale than the one of the activity rule dynamics. Usually the learning rule involve iterative updates that depend on the activities of the neurons. It may also depend on target values supplied by a teacher and on the current values of the weights.

Artificial neurons

The field of artificial neural networks has a long history of development and is closely connected with the advancement of computer science and computers in general. A model of artificial neurons was first developed by McCulloch and Pitts in 1943 to study signal processing in the brain. Their work has later been refined by others. The general idea is to mimic the functionality of the neural network in the human brain. This biological system is composed of billions of neurons that communicate with each other by sending electrical signals. Each neuron

accumulates incoming signals which must exceed an activation threshold to trigger the neuron and yield an output. If the threshold is not overcome, the neuron remains inactive, i.e. has zero output.

This behaviour has inspired a simple mathematical model for an artificial neuron.

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) = f(z), \tag{27.1}$$

where the bias b is sometimes denoted w_0. Here, the signal y of an artificial neuron is the output of an activation function, which takes as input a weighted sum of signals $x_1, \ldots, x_n$ received from n connected artificial neurons.

Conceptually, it is helpful to divide neural networks into four different categories:

1. general purpose neural networks, including deep neural networks (DNN) with several hidden layers, for supervised learning,
2. neural networks designed specifically for image processing, the most prominent example of this class being Convolutional Neural Networks (CNNs),
3. neural networks for sequential data such as Recurrent Neural Networks (RNNs), and
4. neural networks for unsupervised learning such as Deep Boltzmann Machines.

Artificial neural networks of all these types have found numerous applications in the natural sciences. For example, they have been applied to detect phase transitions in Ising and Potts models, lattice gauge theories, and classify different phases of polymers. They have been used to simulate solutions to numerous differential equations such as the Navier-Stokes equation in weather forecasting. Deep learning has also found interesting applications in quantum physics. For example, in quantum information theory, it has been shown that one can perform gate decompositions with the help of neural networks.

The scientific applications are certainly not limited to the natural sciences. In fact, there is a plethora of applications in essentially all disciplines, from the humanities to life science and medicine. However, the real expansion has been into the tech industry and other private sectors.

Neural network types

An artificial neural network is a computational model that consists of layers of connected neurons. We will refer to these computational units as nodes.

A wide variety of different kinds of neural networks have been developed. Most of them are constructed with an input layer, an output layer, and *hidden layers* in between. All layers contain a number of nodes, and connections between nodes are associated with weight variables.

Neural networks (also called neural nets) can be used as nonlinear models for supervised learning. As we will see, neural networks can be viewed as powerful extensions of supervised learning methods such as linear and logistic regression.

Feed-forward neural networks

The feed-forward neural network (FFNN) was the first and simplest type of artificial neural network that was devised. In this type of network, the information moves in one direction, namely forward through the layers from

input to output.

In Fig. 27.1 the nodes are represented by circles, while the arrows display the connections between the nodes and show the direction of information flow. Additionally, each arrow corresponds to a weight variable. In this network, every node in one layer is connected to *all* nodes in the subsequent layer, making this a so-called *fully-connected* FFNN.

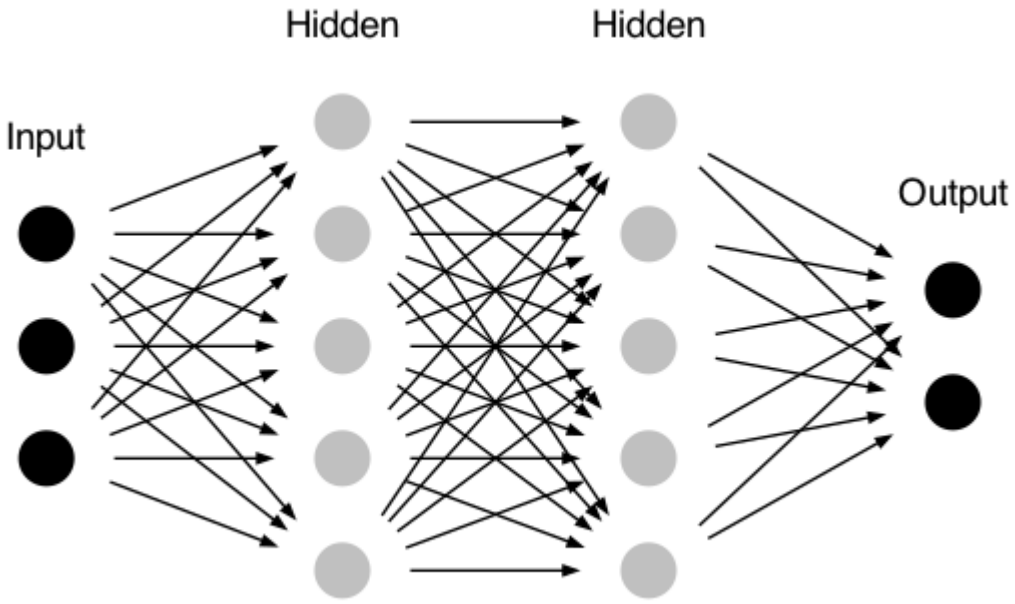


Fig. 27.1 A FFNN with two hidden layers. In addition to the weights associated with the connection arrows there can also be a bias weight connected with each of the active nodes in the hidden and output layers.

Convolutional Neural Network

A different variant of FFNNs are *convolutional neural networks* (CNNs). These networks have a connectivity pattern inspired by the animal visual cortex. Individual neurons in the visual cortex only respond to stimuli from small sub-regions of the visual field, called a receptive field. A similar connectivity of CNNs makes them well-suited to exploit the strong spatial correlations present in images. The response of each node in a CNN is expressed mathematically as a convolution operation.

Convolutional neural networks emulate the behaviour of neurons in the visual cortex by enforcing a *local* connectivity pattern between nodes of adjacent layers: Each node in a convolutional layer is connected only to a subset of the nodes in the previous layer, in contrast to the fully-connected FFNN. Often, CNNs consist of several convolutional layers that learn local features of the input, with a fully-connected layer at the end, which gathers all the local data and produces the outputs. They have wide applications in image and video recognition.

Recurrent neural networks

So far we have only mentioned artificial neural networks where information flows only in the forward direction. *Recurrent neural networks*, on the other hand, have connections between nodes that form directed *cycles*. This creates a form of internal memory which is able to capture information on what has been calculated before. The

output becomes dependent on previous computations. Recurrent neural networks can therefore make use of sequential information. An example of sequential information is sentences, making recurrent neural networks especially well-suited for text and speech recognition.

Feedback networks

Artificial neural networks that can perform unsupervised learning typically require some sort of feedback mechanism. Two famous examples of feedback networks are *Hopfield networks* and *Boltzmann machines*. Due to strong analogies with quantum spin systems, the functionality of these networks can be understood with arguments from statistical physics. John Hopfield and Geoffrey Hinton were awarded the 2024 Nobel Prize in physics for these and related foundational contributions to machine learning with artificial neural networks.

Neural network architecture

In the following sections we restrict ourselves to fully-connected FFNNs. The term *multilayer perceptron* is used ambiguously in the literature, sometimes loosely to mean any FFNN, sometimes strictly to refer to networks composed of multiple layers of perceptron nodes (with step-function activation). A general FFNN, however, consists of:

1. An input layer that distributes the input signals to the active layers.
2. A multilayer network structure with one or more hidden layers, and one output layer.
3. the input nodes pass values to the first hidden layer, its nodes pass the information on to the second and so on until the output layer produces the final output.

The number of layers correspond to the *network depth*. As a convention we primarily count the number of active layers (i.e., those that are associated with activation signals) when specifying the depth. Consequently, we denote a network with one layer of input units plus one layer of hidden units and one layer of output units as a two-layer network. A network with two layers of hidden units is called a three-layer network, etc.

The number of nodes can be different in each layer and is usually known as the *layer width*. Note, however, the number of nodes in the input and output layers are dictated by the task, or mapping, that the network should perform. For example, a network that is constructed to describe a relation $y = y(x_1, x_2)$ will have two input nodes and one output node.

The hidden layers are not linked to observables but play an important role in learning and describing complex relations.

Why deep neural networks?

According to the universal approximation theorem [Cyb89], a feed-forward neural network with just one hidden layer containing a finite number of neurons can approximate a continuous multidimensional function to arbitrary accuracy. The proof of this theorem assumes that the activation function for the hidden layer is a **non-constant, bounded and monotonically-increasing continuous function**. The theorem thus states that simple neural networks provide flexible models for a wide variety of interesting functions. The multilayer, feedforward architecture has proven to be easier to train and really gives neural networks the potential of being universal approximators.

Limitations of supervised learning with deep networks

Like all statistical methods, supervised learning using neural networks has important limitations. This is especially important when one seeks to apply these methods to physics problems. As all machine-learning algorithms, neural networks are not a universal solution.

As physicists you should always maintain the ambition of learning about the model itself. Often, the same or better performance on a task can be achieved by identifying the most relevant features.

Here we list some of the important limitations of supervised neural network based models.

- **Need labeled data**. All supervised learning methods require labeled data. Often, labeled data is harder to acquire than unlabeled data (e.g. one must pay for human experts to label images).
- **Deep neural networks are extremely data hungry**. They perform best when data is plentiful. This is extra problematic for supervised methods where the data must also be labeled. The utility of deep neural networks is extremely limited if data is hard to acquire or the datasets are small (hundreds to a few thousand samples). In this case, the performance of other methods that utilize hand-engineered features can exceed that of deep neural networks.
- **Homogeneous data**. Almost all deep neural networks deal with homogeneous data of one type. It is very hard to design architectures that mix and match data types (i.e., some continuous and some discrete variables). In some applications, mixed data types is what is required. So called ensemble models, such as random forests or gradient-boosted trees, are better suited to handle mixed data types.
- **Many problems are not just about prediction**. In the natural sciences we are often interested in learning about the underlying process that generates the data. In this case, it is often difficult to cast hypotheses in a supervised learning setting. While the problems are related, it is possible to make good predictions with a *wrong* model. The model might or might not be useful for understanding the underlying scientific principles.

27.2. Feed-forward networks: mathematical model

In this section we formulate the fully-connected feed-forward networks of Neural network types and architecture mathematically. We introduce the notation for signals, weights and biases that is used throughout the following sections, and the linear-algebra form that is used in practice.

Mathematical model

In a FFNN, the *inputs* to the nodes in one layer are the *outputs* of the nodes in the preceding layer.

Starting from the first hidden layer, we compute a weighted sum z_j^1 of the input signals $\boldsymbol{x} = (x_1, \ldots, x_n)$, for each node j

$$z_j^1 = \sum_{i=1}^{n} w_{ij}^1 x_i + b_j^1, \tag{27.2}$$

where w_{ij}^1 are the linear weights of this node and b_j^1 is known as a bias, or threshold, weight.

The value, z_j^1, is known as the *preactivation* and becomes the argument to the activation function $f^1(z)$ of node j. For simplicity we assume that all nodes in one layer have the same activation function. The output from node j in

the first layer then becomes

$$y_j^1 = f^1(z_j^1) = f^1\left(\sum_{i=1}^{n} w_{ij}^1 x_i + b_j^1\right). \tag{27.3}$$

When the output of all the nodes in the first hidden layer are computed, the preactivations of the subsequent layer can be calculated and so forth. Propagating signals from layer to layer we eventually reach node j in layer l. Its output signal is produced via its activation function $f^l(z_j^l)$

$$y_j^l = f^l(z_j^l) = f^l\left(\sum_{i=1}^{N_{l-1}} w_{ij}^l y_i^{l-1} + b_j^l\right), \tag{27.4}$$

where N_{l-1} is the number of nodes in layer $l-1$.

The output layer of an L-layer network (i.e., with $L-1$ hidden layers) is layer L. For example, the final output from node i of a three-layer network reads

$$y_i^3 = f^3\left[\sum_{k=1}^{N_2} w_{ki}^3 f^2\left(\sum_{j=1}^{N_1} w_{jk}^2 f^1\left(\sum_{n=1}^{N_0} w_{nj}^1 x_n + b_j^1\right) + b_k^2\right) + b_i^3\right], \tag{27.5}$$

with $N_0 = n$ the number of inputs. Deeper networks simply add further levels of nesting. This illustrates that the outputs $\boldsymbol{y} = (y_1^L, \ldots, y_m^L)$ are the dependent variables and the input $\boldsymbol{x}$ are the independent variables.

This confirms that a FFNN, despite its quite convoluted mathematical form, is nothing more than a non-linear mapping. For example it can map real-valued vectors $\boldsymbol{x} \in \mathbb{R}^n \rightarrow \boldsymbol{y} \in \mathbb{R}^m$.

Furthermore, the flexibility and universality of a neural network can be illustrated by realizing that the expression is essentially a nested sum of scaled activation functions leading to

$$\boldsymbol{y} = \boldsymbol{y}(\boldsymbol{x}; \boldsymbol{W}), \tag{27.6}$$

where $\boldsymbol{W}$ contains the weights and biases which are the model parameters.

Matrix-vector notation

We can introduce a much more convenient matrix-vector notation for all quantities in a FFNN.

In particular, we represent all signals as layer-wise row vectors $\boldsymbol{y}^l$ so that the i-th element of each vector is the output y_i^l of node i in layer l.

We have that $\boldsymbol{W}^l$ is an $N_{l-1} \times N_l$ matrix, while $\boldsymbol{b}^l$ and $\boldsymbol{y}^l$ are $1 \times N_l$ row vectors. With this notation, the sum becomes a matrix-vector multiplication, and we can write the preactivations of layer l as

$$\boldsymbol{z}^l = \boldsymbol{y}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l, \tag{27.7}$$

where $\boldsymbol{y}^0 \equiv \boldsymbol{x}$. The outputs of this layer then become $\boldsymbol{y}^l = f^l(\boldsymbol{z}^l)$, where we should understand element-wise application of the activation function.

For N data instances stacked as the rows of a matrix, the same expression reads $\boldsymbol{Z}^l = \boldsymbol{Y}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l$, with the bias added to each row. In code this is `Y @ W + b`, with `Y = X` for the first layer.

As an example we consider the outputs of the second layer. Assuming three nodes in the first layer we get

$$\boldsymbol{y}^2 = f^2(\boldsymbol{y}^1\boldsymbol{W}^2 + \boldsymbol{b}^2) = f^2\left(\begin{bmatrix} y_1^1, y_2^1, y_3^1 \end{bmatrix} \begin{bmatrix} w_{11}^2 & w_{12}^2 & w_{13}^2 \\ w_{21}^2 & w_{22}^2 & w_{23}^2 \\ w_{31}^2 & w_{32}^2 & w_{33}^2 \end{bmatrix} + \begin{bmatrix} b_1^2, b_2^2, b_3^2 \end{bmatrix}\right). \tag{27.8}$$

This is not just a convenient and compact notation, but also a useful and intuitive way to think about FFNNs. The output is calculated by a sequence of matrix-vector multiplications and vector additions used as input to activation functions.

Activation rules

A property that characterizes a neural network, other than its connectivity, is the choice of activation function(s). In general, these are non-linear functions. In fact, a FFNN with only linear activation functions would imply that each layer simply performs a linear transformation of its inputs and the output would be an unnecessarily complicated linear mapping of the input.

Logistic and Hyperbolic activation functions. The archetypical example of activation functions is the family of S-shaped (sigmoid) functions. In particular, the logistic sigmoid function is

$$f_{\text{sigmoid}}(z) = \frac{1}{1 + e^{-z}}, \tag{27.9}$$

and the *hyperbolic tangent* function is

$$f_{\tanh}(z) = \tanh(z) \tag{27.10}$$

Noisy networks

Both the sigmoid and tanh activation functions imply that signals will be non-zero everywhere. Even when nodes are inactive, they transmit small, but nonzero, signals. This leads to inefficiencies in both feed-forward and back-propagation operations.

The ambition to completely silence inactive neurons leads to the family of *rectifier activation functions*. The Rectifier Linear Unit (ReLU) uses the following activation function

$$f_{\mathrm{ReLU}}(z) = \max(0, z). \tag{27.11}$$

However, this type of activation function gives zero gradients, $f'(z) = 0$ for $z < 0$, which makes weight updates during training much less efficient. To solve this problem, known as dying ReLU neurons, practitioners often use modified versions of the ReLU function, such as the leaky ReLU or the exponential linear unit (ELU) function

$$f_{\mathrm{ELU}}(z) = \begin{cases} \alpha\left(\exp\left(z\right) - 1\right) & z < 0, \\ z & z \geq 0. \end{cases} \tag{27.12}$$

The final layer of a neural network has to use an activation function that produces a relevant output signal for the task at hand. For example, the output nodes could employ a linear activation function to give continuous outputs for regression tasks, or a softmax function to provide classification probabilities.

Learning algorithm

The learning phase (network training) aims to optimize the weights to minimize a problem and data-specific cost function. This task involves multiple choices

1. Choosing a cost function, $C(\mathcal{T}, \boldsymbol{W})$, that describes how to compare model outputs with targets from a training data set $\mathcal{T} = \{(\boldsymbol{x}_i, \boldsymbol{t}_i)\}_{i=1}^{N}$. Each pair consists of an input $\boldsymbol{x}_i$ and the target $\boldsymbol{t}_i$ that the network output $\boldsymbol{y}(\boldsymbol{x}_i; \boldsymbol{W})$ should reproduce.
    - Common cost-function choices include: Mean-squared error (MSE) or Mean-absolute error (MAE) for regression; Cross-entropy or Accuracy for classification.
    - Weight regularization or random node dropout to avoid overfitting.
    - The incorporation of physics (model) knowledge in the construction of a more relevant cost function.
2. Optimization algorithm.
    - Back-propagation (see Neural networks: Backpropagation) can be used to compute the gradient of the cost function with respect to the weights. It corresponds to repeated use of the chain rule on the activation functions while traversing the different layers backwards.
    - The gradient evaluated after each parameter update can be used to iteratively search for a (local) optimum in the high-dimensional space of weights. Popular gradient descent optimizers include:
        - Standard stochastic gradient descent (SGD), possibly with batches.
        - Momentum SGD (to incorporate a moving average)
        - AdaGrad (with per-parameter learning rates)
        - RMSprop (adapting the learning rates based on RMS gradients)
        - Adam (a combination of momentum and RMSprop that employs estimates of both the first and second moments of weight gradients).
3. Splits of data
    - Training data: used for training.
    - Validation data: used to monitor learning and to adjust hyperparameters.
    - Test data: for final test of perfomance.

General remarks on training:

- It is critical that neither validation nor test data is used for adjusting the network weights.

- Data is often split into batches such that gradients are computed for a batch of data rather than for the full set all at once.
- A full pass of all data is known as an epoch. A validation score is often evaluated at the end of each epoch.
- Additional “hyperparameters” that can be tuned include:
  - The number of epochs (overfitting eventually).
  - The batch size (interplay between stochasticity and efficiency).
  - Learning rates.

In conclusion, there are many options when designing and training an neural network.

Exercises

Exercise 27.1 (Weights and signal propagation of a simple neural network)

Consider a neural network with two nodes in the input layer, one hidden layer with three nodes, and an output layer with one node (see Fig. 27.2). All active nodes (i.e., those that map input signals to an output via an activation function) employ the same activation function $f(z) = 1/(1 + e^{-z})$ and include a bias weight when computing the preactivation z.

- a) How many free (trainable) parameters does this neural network have?
- b) Write an explicit expression for the output signal y from the output node as a function of its weights $(w_{11}^2, w_{12}^2, w_{13}^2)$, bias weight b_1^2, and the signals (y_1^1, y_2^1, y_3^1) coming from the nodes in the hidden layer.
- c) Write a vector expression for the preactivations (z_1^1, z_2^1, z_3^1) of the three nodes in the hidden layer given a vector of inputs (x_1, x_2) and a matrix that contains the weights of the hidden layer and a vector that contains the bias weights. Be explicit how the weights are ordered in the matrix such that the preactivation vector can be obtained with matrix-vector operations.
- d) How can the above expression be generalized to handle a scenario when there are four instances of input data?

$$\boldsymbol{X} = \begin{pmatrix} x_1^{(1)} & x_2^{(1)} \\ x_1^{(2)} & x_2^{(2)} \\ x_1^{(3)} & x_2^{(3)} \\ x_1^{(4)} & x_2^{(4)} \end{pmatrix}$$

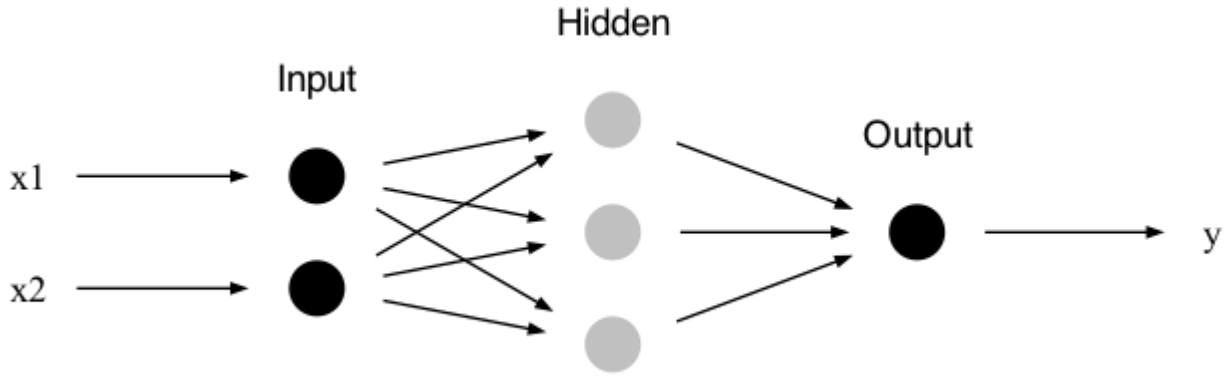


Fig. 27.2 A simple two-layer neural network.

Exercise 27.2 (Weights and signal propagation of a wide neural network)

Consider a neural network with three nodes in the input layer, one hidden layer with one hundred nodes, and an output layer with two nodes. All nodes in the hidden layer employ the same activation function $f(z) = 1/(1 + e^{-z})$ and include a bias weight when computing the preactivation z. The nodes in the output layer are linear.

- a) How many free (trainable) parameters does this neural network have?
- b) Introduce relevant matrices and vectors to write down an expression for the output signal $\boldsymbol{y} = (y_1, y_2)$ given an input $\boldsymbol{x} = (x_1, x_2, x_3)$.
- c) How would this expression have to be modified to handle a scenario when there are N instances of input data, for which one would like to compute the corresponding network outputs?

Exercise 27.3 (Linear signals)

Consider a neural network with p nodes in the input layer, one hidden layer with L nodes, and an output layer with a single node. All nodes in the hidden layer employ the same activation function $f(z) = 1/(1 + e^{-z})$ and include a bias weight when computing the preactivation z. The node in the output layer is linear.

- a) Consider small signals and weights such that the output from the nodes in the hidden layer can be approximated by linear functions in the input $\boldsymbol{x} = (x_1, x_2, \ldots, x_p)$. Write an expression for the output from one of these nodes.
- b) Show that the final output from the network also becomes linear in the inputs and relate the coefficients of this expression to the weights of the network.

Exercise 27.4 (Expected signal)

Consider a single node with p input signals, including a bias weight when computing the preactivation z, and with a sigmoid activation function $f(z) = 1/(1 + e^{-z})$. All weights are initialized as independent draws from a standard normal distribution (zero mean and unit variance).

Assume that you would initialize a large number of instances of this kind of node. Each one will have different weights. For each one, you propagate the same input signal $\boldsymbol{x}$ and measure the node's preactivation z and output y.

- a) How would the distribution $p\left(z\right)$ of the measured preactivations look like?
- b) How would the distribution $p\left(y\right)$ of the measured outputs look like?
- c) Study the form of $p\left(y\right)$ for the two different limits $|\boldsymbol{x}| \ll 1$ (also with $w_0 \ll 1$) and $|\boldsymbol{x}| \gg 1$.

Hints

Solutions to exercises

Solution to Exercise 27.1 (Weights and signal propagation of a simple neural network)

- a) Trainable parameters are in the active nodes which, in this case, are the nodes in the hidden layer and the output layer. There are three weights per node in the hidden layer (two linear ones multiplying the two input signals and one bias) and four weights in the output node (three linear weights and one bias). Therefore, in total, there are $3 \times 3 + 4 = 13$.
- b) $y = 1/(1 + e^{-z})$, where $z = w_{11}^2 y_1^1 + w_{12}^2 y_2^1 + w_{13}^2 y_3^1 + b_1^2$.
- c) $\boldsymbol{z}^1 = \boldsymbol{x} \cdot \boldsymbol{W}^1 + \boldsymbol{b}^1$, where $\boldsymbol{z}^1 = (z_1^1, z_2^1, z_3^1)$, $\boldsymbol{x} = (x_1, x_2)$, $\boldsymbol{b}^1 = (b_1^1, b_2^1, b_3^1)$ are all row vectors and $\boldsymbol{W}^1 = \begin{pmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \end{pmatrix}$ is a 2×3 matrix.
- d) It would look the same $\boldsymbol{Z}^1 = \boldsymbol{X} \cdot \boldsymbol{W}^1 + \boldsymbol{b}^1$ but $\boldsymbol{Z}^1$ would now be a 4×3 matrix with each row containing the hidden layer node preactivations for the corresponding instance of input data (row in $\boldsymbol{X}$). Applying the activation function $f(z)$ to each element of this matrix would give the corresponding output signals $\boldsymbol{Y}^1$ as another 4×3 matrix.

Solution to Exercise 27.2 (Weights and signal propagation of a wide neural network)

- a) There are $100 \times (3 + 1) + 2 \times (100 + 1) = 602$ trainable parameters.
- b) $\boldsymbol{y} = \boldsymbol{z} = \boldsymbol{y}^1 \boldsymbol{W}^2 + \boldsymbol{b}^2$, where the output layer parameters appear in the $\boldsymbol{W}^2$ matrix and the $\boldsymbol{b}^2$ vector. Furthermore, $\boldsymbol{y}^1 = 1/(1 + e^{-\boldsymbol{z}^1})$ and $\boldsymbol{z}^1 = \boldsymbol{x}\boldsymbol{W}^1 + \boldsymbol{b}^1$.

  The weight matrices and the bias vectors of layer L have elements w_{ij}^L and b_j^L, respectively, and will have the sizes: dim $(\boldsymbol{W}^1) = 3 \times 100$, dim $(\boldsymbol{b}^1) = 1 \times 100$, dim $(\boldsymbol{W}^2) = 100 \times 2$, dim $(\boldsymbol{b}^2) = 1 \times 2$.
- c) In this case, the input, $\boldsymbol{X}$, becomes an $N \times 3$ matrix. The linear algebra expressions would look the same with the data instances representing one more dimension. That is $\boldsymbol{Y} = \boldsymbol{Y}^1 \boldsymbol{W}^2 + \boldsymbol{b}^2$, with dim $(\boldsymbol{Y}) = N \times 2$, and $\boldsymbol{Y}^1 = f(\boldsymbol{Z}^1)$. Here, $\boldsymbol{Z}^1 = \boldsymbol{X}\boldsymbol{W}^1 + \boldsymbol{b}^1$ is now an $N \times 100$ matrix with each row containing the hidden layer node preactivations for the corresponding instance of input data (row in $\boldsymbol{X}$). Furthermore, signals $\boldsymbol{Y}^1$ and preactivations $\boldsymbol{Z}^2$ are both matrices with N rows. Note, however, that the weight and bias matrices are the same.

Solution to Exercise 27.3 (Linear signals)

In the following, superscripts denote the layer with $l = 1$ the hidden layer and $l = 2$ the output layer. Correspondingly, propagating signals are row vectors such that w_{ij}^l is the weight i of node j in layer l (i.e., it multiplies incoming signal i, $i = 0$ corresponding to the bias).

- a) The output of node 1: $y^1(z^1) = \frac{1}{1+1-z^1+\mathcal{O}((z^1)^2)} = \frac{1}{2}\left(1 + \frac{z^1}{2} + \mathcal{O}((z^1)^2)\right)$.
  With $z^1 = w^1_{01} + \sum_{i=1}^p w^1_{i1} x_i$ we find

$$y^1 = \frac{1}{2} + \frac{1}{4}\left(w^1_{01} + \sum_{i=1}^p w^1_{i1} x_i\right).$$

- b) The final output is $y = w^2_{01} + \sum_{l=1}^L w^2_{l1} y^1_l$. The signals y^1_l from the hidden layer are given by (a). We arrive at the linear signal

$$y = k_0 + k_1 x_1 + k_2 x_2 + \ldots + k_p x_p$$

  with

$$k_0 = w^2_{01} + \frac{1}{4}\sum_{l=1}^L w^2_{l1}(2 + w^1_{0l}),$$
$$k_i = \frac{1}{4}\sum_{l=1}^L w^2_{l1} w^1_{il}.$$

Solution to Exercise 27.4 (Expected signal)

The input is $\boldsymbol{x} = (x_1, x_2, \ldots, x_p)$ and the node's weights are $(w_0, w_1, w_2, \ldots, w_p)$ with w_0 the bias.

- a) The preactivation z will be normally distributed

$$p\,(z) = \mathcal{N}(\mu_z, \sigma_z^2),$$

  with mean $\mu_z = 0$ and variance $\sigma_z^2 = 1 + |\boldsymbol{x}|^2$.
- b) The distribution of the output can be obtained via a transformation $y = 1/(1+e^{-z})$, or rather $z = \ln(y/(1-y))$. One finds

$$\begin{aligned} p_Y(y) &= p_Z(z(y)) \left|\frac{dz}{dy}\right| \\ &= \frac{1}{\sqrt{2\pi\sigma_z^2}} \frac{1}{y(1-y)} \exp\left[-\frac{(\ln(y/(1-y)))^2}{2\sigma_z^2}\right] \end{aligned}$$

- c) For small signals the preactivation will be small and the response will be approximately linear. This means that the final output will be a linear combination of normal-distributed weights, which is another normal distribution. The mode, however, will be at $y = 0.5$.
  For very large signals you will find an output distribution that is sharply peaked at $y = 0$ and $y = 1$. That means that you should expect the node to be either very quiet or very noisy, but hardly anything in between (the expectation value will still be $\mathbb{E}\,[y] = 0.5$, but Std $(y) \to 0.5$.)

27.3. Neural networks: Backpropagation

As we have seen, the final output of a feed-forward network can be expressed in terms of basic matrix-vector multiplications. The unknown quantities are the weights w_{ij}^l and biases b_j^l, and we need an algorithm for changing them so that our errors are as small as possible. This leads us to the famous back propagation algorithm [RumelhartHintonWilliams86].

Deriving the back propagation equations for a feed-forward network

The questions we want to ask are how changes in the biases and the weights of our network change the cost function, and how we can use the final output to modify the weights.

Notation

We use the notation of Feed-forward networks: mathematical model. Consider an L-layer network, i.e., $L-1$ hidden layers and an output layer $l=L$. Node j in layer l has the preactivation

$$z_j^l = \sum_{i=1}^{N_{l-1}} w_{ij}^l y_i^{l-1} + b_j^l, \tag{27.13}$$

and the output $y_j^l = f^l(z_j^l)$. Here N_{l-1} is the number of nodes in layer $l-1$, and w_{ij}^l connects node i in layer $l-1$ to node j in layer l. The input layer performs no computation; we simply set $\boldsymbol{y}^0 \equiv \boldsymbol{x}$. The network output is $\boldsymbol{y}^L$.

In matrix-vector form, with signals as row vectors and $\boldsymbol{W}^l$ an $N_{l-1} \times N_l$ matrix,

$$\boldsymbol{z}^l = \boldsymbol{y}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l, \qquad \boldsymbol{y}^l = f^l(\boldsymbol{z}^l). \tag{27.14}$$

Cost function

To derive the equations, let us start with a plain regression problem. For a single data instance with targets $\boldsymbol{t} = (t_1, \ldots, t_{N_L})$ we use the cost function

$$\mathcal{C} = \frac{1}{2}\sum_{j=1}^{N_L} \left(y_j^L - t_j\right)^2. \tag{27.15}$$

For a batch of training data $\{(\boldsymbol{x}_i, \boldsymbol{t}_i)\}$ (see Learning algorithm), the cost is a sum over instances and so are its gradients. Other cost functions can also be considered; only the derivative $\partial \mathcal{C}/\partial y_j^L$ changes.

Derivatives and the chain rule

From the definition of the preactivation z_j^l we have

$$\frac{\partial z_j^l}{\partial w_{ij}^l} = y_i^{l-1}, \qquad \frac{\partial z_j^l}{\partial b_j^l} = 1, \qquad \frac{\partial z_j^l}{\partial y_i^{l-1}} = w_{ij}^l. \tag{27.16}$$

The output of a node depends only on its own preactivation, $\partial y_j^l / \partial z_j^l = f^{l\prime}(z_j^l)$. For the sigmoid function, $f(z) = 1/(1+e^{-z})$, this derivative is particularly simple

$$f'(z_j^l) = f(z_j^l)\left[1 - f(z_j^l)\right] = y_j^l\left(1 - y_j^l\right). \tag{27.17}$$

The error signal δ

The key quantity is the derivative of the cost function with respect to the preactivation of node j in layer l

$$\delta_j^l \equiv \frac{\partial \mathcal{C}}{\partial z_j^l}. \tag{27.18}$$

Since w_{ij}^l and b_j^l affect the cost only through z_j^l, the chain rule immediately gives the gradients

$$\frac{\partial \mathcal{C}}{\partial w_{ij}^l} = y_i^{l-1}\,\delta_j^l, \qquad \frac{\partial \mathcal{C}}{\partial b_j^l} = \delta_j^l. \tag{27.19}$$

The weight gradient is the product of the signal entering the connection and the error signal at its end. In matrix form, $\partial \mathcal{C}/\partial \boldsymbol{W}^l = (\boldsymbol{y}^{l-1})^T \boldsymbol{\delta}^l$, which is an outer product with the same $N_{l-1} \times N_l$ shape as $\boldsymbol{W}^l$.

Output layer

At the output layer the error signal follows directly from the cost function

$$\delta_j^L = \frac{\partial \mathcal{C}}{\partial y_j^L}\frac{\partial y_j^L}{\partial z_j^L} = \left(y_j^L - t_j\right) f^{L\prime}(z_j^L), \tag{27.20}$$

or, using the Hadamard (element-wise) product $\odot$,

$$\boldsymbol{\delta}^L = \frac{\partial \mathcal{C}}{\partial \boldsymbol{y}^L} \odot f^{L\prime}(\boldsymbol{z}^L). \tag{27.21}$$

The first factor measures how fast the cost changes with output j. If the cost does not depend much on a particular output node, then δ_j^L will be small. The second factor measures how fast the activation function changes at z_j^L, which is a cheap by-product of the forward pass.

> **Linear output for regression**
>
> Regression networks usually have a linear output layer, $f^L(z) = z$. Then $f^{L\prime} = 1$ and the output error is simply the residual, $\boldsymbol{\delta}^L = \boldsymbol{y}^L - \boldsymbol{t}$.

Two consequences of the gradient expressions are worth noting. When the incoming signal y_i^{l-1} is small, the gradient with respect to w_{ij}^l is also small and the weight learns slowly. The same happens when a sigmoid node saturates, i.e., when its output approaches 0 or 1, since then $f'(z) \approx 0$.

Back-propagating the error

We need one more equation: the error signal in layer l expressed in terms of the errors in layer $l+1$. Since z_j^l affects the cost through all preactivations z_k^{l+1} of the next layer, the chain rule gives

$$\delta_j^l = \sum_{k=1}^{N_{l+1}} \frac{\partial \mathcal{C}}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial z_j^l} = \sum_{k=1}^{N_{l+1}} \delta_k^{l+1} \frac{\partial z_k^{l+1}}{\partial z_j^l}. \tag{27.22}$$

With $z_k^{l+1} = \sum_i w_{ik}^{l+1} y_i^l + b_k^{l+1}$ and $y_i^l = f^l(z_i^l)$ we find $\partial z_k^{l+1}/\partial z_j^l = w_{jk}^{l+1} f^{l\prime}(z_j^l)$, so that

$$\delta_j^l = f^{l\prime}(z_j^l) \sum_{k=1}^{N_{l+1}} w_{jk}^{l+1}\, \delta_k^{l+1}, \qquad \text{i.e.,} \qquad \boldsymbol{\delta}^l = \left(\boldsymbol{\delta}^{l+1} \big(\boldsymbol{W}^{l+1}\big)^T \right) \odot f^{l\prime}(\boldsymbol{z}^l). \tag{27.23}$$

The error signal travels backwards through the same weights as the forward signal, but with the transposed matrix.

The back-propagation algorithm

The equations above provide the gradient of the cost function with respect to all weights and biases.

Summary.

1. Set $\boldsymbol{y}^0 = \boldsymbol{x}$.
2. Forward pass: for $l = 1, 2, \dots, L$ compute and store $\boldsymbol{z}^l = \boldsymbol{y}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l$ and $\boldsymbol{y}^l = f^l(\boldsymbol{z}^l)$.
3. Output error: $\boldsymbol{\delta}^L = \dfrac{\partial \mathcal{C}}{\partial \boldsymbol{y}^L} \odot f^{L\prime}(\boldsymbol{z}^L)$.
4. Backward pass: for $l = L-1, L-2, \dots, 1$ compute $\boldsymbol{\delta}^l = \big(\boldsymbol{\delta}^{l+1}(\boldsymbol{W}^{l+1})^T\big) \odot f^{l\prime}(\boldsymbol{z}^l)$.
5. Gradient-descent update for all layers $l = 1, 2, \dots, L$

$$\boldsymbol{W}^l \leftarrow \boldsymbol{W}^l - \eta\big(\boldsymbol{y}^{l-1}\big)^T \boldsymbol{\delta}^l, \qquad \boldsymbol{b}^l \leftarrow \boldsymbol{b}^l - \eta\,\boldsymbol{\delta}^l. \tag{27.24}$$

The parameter η is the learning rate. In practice one uses stochastic gradient descent with mini-batches and an outer loop that steps through multiple epochs of training. With N instances stacked as rows of $\boldsymbol{Y}^{l-1}$ and $\boldsymbol{\Delta}^l$, the batch gradients are $\partial\mathcal{C}/\partial\boldsymbol{W}^l = (\boldsymbol{Y}^{l-1})^T\boldsymbol{\Delta}^l$ and $\partial\mathcal{C}/\partial\boldsymbol{b}^l =$ the sum of the rows of $\boldsymbol{\Delta}^l$. The matrix product performs the sum over the batch.

Weight conventions in code

The row-vector convention $\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{W} + \boldsymbol{b}$ matches `X @ W + b` in NumPy. Note that PyTorch's `nn.Linear` stores the transposed matrix, with shape $N_l \times N_{l-1}$, such that `layer.weight.T` corresponds to $\boldsymbol{W}^l$.

Learning challenges

The back-propagation algorithm works by going from the output layer to the input layer, propagating the error gradient. The learning algorithm uses these gradients to update each parameter with a Gradient Descent (GD) step.

Unfortunately, the gradients often get smaller and smaller as the algorithm progresses down to the first hidden layers. Each step of the backward pass multiplies by a factor $f'(z_j^l)$, which is at most $1/4$ for the sigmoid. As a result, the GD update step leaves the lower layer connection weights virtually unchanged, and training never converges to a good solution. This is known in the literature as **the vanishing gradients problem**.

In other cases, the opposite can happen, namely that the gradients grow bigger and bigger. The result is that many of the layers get large updates of the weights and the learning algorithm diverges. This is the **exploding gradients problem**, which is mostly encountered in recurrent neural networks. More generally, deep neural networks suffer from unstable gradients, different layers may learn at widely different speeds.

27.4. Demo: Feed-forward neural network for a function in PyTorch

See the PyTorch website for tutorials and other documentation. In this notebook we'll rely on ChatGPT 4 to generate code for a PyTorch neural network for a function of a single variable (this particular network is a "multilayer perceptron"). Other than the questions to ChatGPT, all code and documentation was provided by ChatGPT. (Slight tweaks were made to change the format of the output from training and to increase the range of the trained function to test extrapolation.)

Installation of PyTorch

To install PyTorch via Anaconda, use the following conda command:

```
conda install pytorch torchvision -c pytorch
```

Verification of PyTorch installation (suggested in PyTorch website)

```
import torch
x = torch.rand(5, 3)
print(x)
```

```
tensor([[0.1380, 0.9330, 0.0819],
        [0.4911, 0.3917, 0.8913],
        [0.2078, 0.3373, 0.7952],
        [0.8604, 0.9642, 0.1106],
        [0.4804, 0.9332, 0.5428]])
```

Result should be similar to (but not the same; these are random numbers!):

```
tensor([[0.3380, 0.3845, 0.3217],
        [0.8337, 0.9050, 0.2650],
        [0.2979, 0.7141, 0.9069],
        [0.1449, 0.1132, 0.1375],
        [0.4675, 0.3947, 0.1426]])
```

Telling ChatGPT 4 to make a network for a function of one variable

To ChatGPT: *Write a neural network in PyTorch to learn a function of one variable. It should have three hidden layers, each with a width of 10 neurons.*

To create a neural network in PyTorch with the specifications you described, we'll first need to import the necessary modules from PyTorch, particularly torch and torch.nn. The network will consist of three hidden layers, each with 10 neurons, and since it's for learning a function of one variable, the input and output layers will each have one neuron.

Here's how you can define such a network using PyTorch:

```python
import torch
import torch.nn as nn

# Define the neural network class inheriting from nn.Module
class OneVariableNet(nn.Module):
    def __init__(self):
        super(OneVariableNet, self).__init__()
        # Define the first hidden layer (input layer to first hidden layer)
        self.hidden1 = nn.Linear(1, 10)  # 1 input, 10 outputs
        # Define the second hidden layer
        self.hidden2 = nn.Linear(10, 10) # 10 inputs, 10 outputs
        # Define the third hidden layer
        self.hidden3 = nn.Linear(10, 10) # 10 inputs, 10 outputs
        # Define the output layer
        self.output = nn.Linear(10, 1)   # 10 inputs, 1 output

        # Define activation function, you can use others like ReLU or LeakyReLU
        self.activation = nn.Tanh()

    def forward(self, x):
        # Forward pass through each layer
        x = self.activation(self.hidden1(x))
        x = self.activation(self.hidden2(x))
        x = self.activation(self.hidden3(x))
        x = self.output(x)
        return x

# Create an instance of the network
net = OneVariableNet()

# Print the network structure
print(net)
```

```
OneVariableNet(
  (hidden1): Linear(in_features=1, out_features=10, bias=True)
  (hidden2): Linear(in_features=10, out_features=10, bias=True)
  (hidden3): Linear(in_features=10, out_features=10, bias=True)
  (output): Linear(in_features=10, out_features=1, bias=True)
  (activation): Tanh()
)
```

Explanation:

1. **Network Structure:** This network begins with an input layer that accepts a single feature (since it's a function of one variable). It then passes through three hidden layers, each with 10 neurons, and finally outputs a single value, which is typical for regression tasks.
2. **Activation Function:** I used the Tanh activation function for the hidden layers, which helps introduce non-linearity to the model, allowing it to learn more complex patterns. You can substitute this with other functions like ReLU if preferred, which might offer benefits in terms of training speed and avoiding vanishing gradients in some cases.
3. **Output Layer:** The final output layer does not use an activation function since we're presumably predicting a continuous variable (regression task). If you're working on a classification task, you might need a suitable activation function like sigmoid or softmax for binary or multi-class classification, respectively.
4. **Initialization and Usage:** After defining the network, you can create an instance and use it for training or inference as needed. Training this network would involve defining a loss function and an optimizer, then running a training loop to fit the model to your data.

This setup should work well for basic experiments and learning purposes. If you want to train this model on actual data, you'll need to set up a training loop with loss computation and optimization steps.

Now ask ChatGPT to use this code to learn a specified function in a specified region

To ChatGPT: *Write code to use this code to learn the function* `f(x) = sin(x)**2` *from x = 0 to x = 2, with 20 randomly spaced training points. Then test it with 50 equally spaced testing points.*

To accomplish the task of training a neural network on the function `f(x) = sin(x)**2` and testing it on new data points, we'll follow these steps:

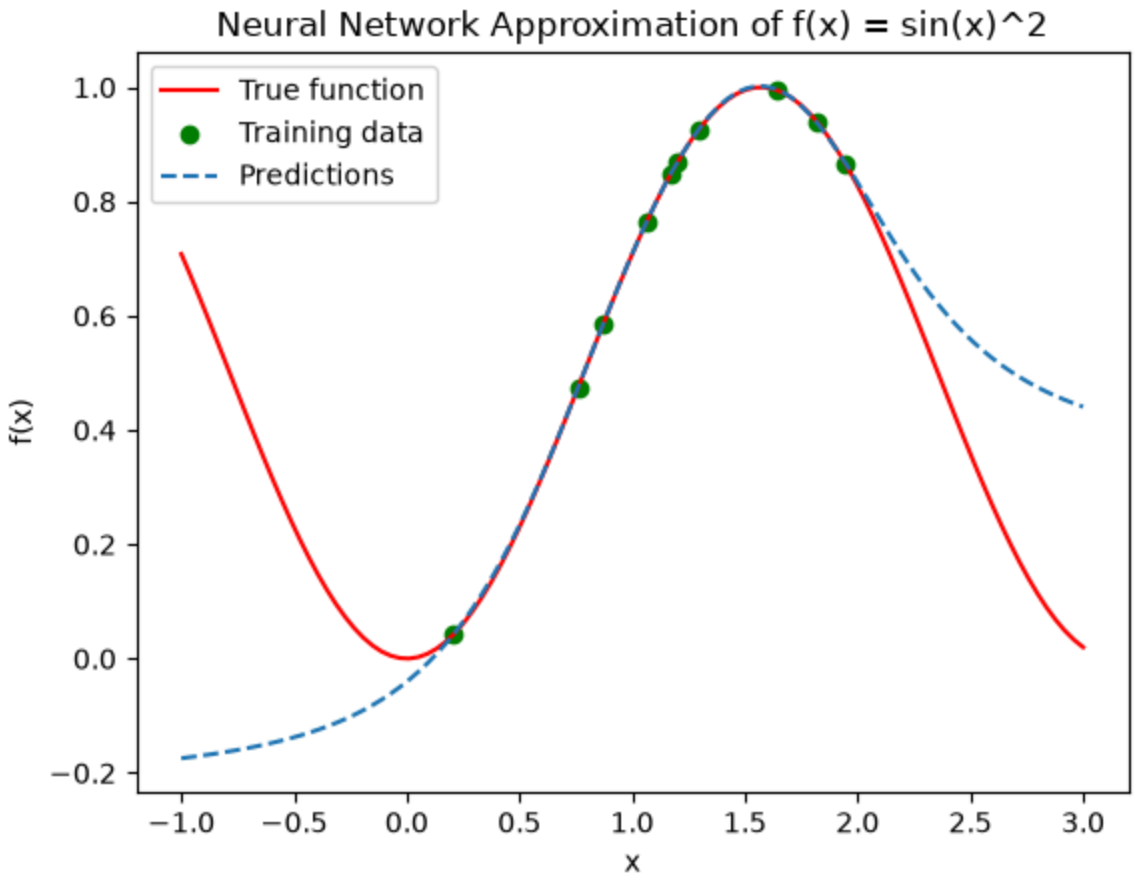
1. Data Preparation: Generate training data and testing data as specified.
2. Network Training: Implement the training loop using a suitable optimizer and loss function.
3. Testing: Evaluate the network on the testing data and possibly visualize the results.

First, we'll need to define our dataset. We'll generate 20 random training points in the range [0, 2] and 50 equally spaced testing points in the same range.

Here's the complete code to do this, including both training and testing phases:

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt

class OneVariableNet(nn.Module):
    def __init__(self):
        super(OneVariableNet, self).__init__()
        self.hidden1 = nn.Linear(1, 10)
        self.hidden2 = nn.Linear(10, 10)
        self.hidden3 = nn.Linear(10, 10)
        self.output = nn.Linear(10, 1)
        self.activation = nn.Tanh()

    def forward(self, x):
        x = self.activation(self.hidden1(x))
        x = self.activation(self.hidden2(x))
        x = self.activation(self.hidden3(x))
        x = self.output(x)
        return x

# Create an instance of the network
net = OneVariableNet()

# Loss function and optimizer
criterion = nn.MSELoss()
optimizer = optim.Adam(net.parameters(), lr=0.01)

# Generate training data
x_train = 2 * torch.rand(10, 1)  # Randomly spaced points
y_train = torch.sin(x_train)**2  # f(x) = sin(x)^2

# Training loop
epochs = 2000
for epoch in range(epochs):
    optimizer.zero_grad()
    outputs = net(x_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 100 == 0:
        print(f'Epoch [{epoch + 1}/{epochs}], Loss: {loss.item():.4e}')

# Generate testing data
x_test = torch.linspace(-1, 3, 100).unsqueeze(1)  # Equally spaced points
y_test = torch.sin(x_test)**2  # f(x) = sin(x)^2
with torch.no_grad():
    predictions = net(x_test)

# Plot the results
plt.plot(x_test.numpy(), y_test.numpy(), label='True function', color='r')
plt.scatter(x_train.numpy(), y_train.numpy(), label='Training data', color='g')
plt.plot(x_test.numpy(), predictions.numpy(), label='Predictions', linestyle='--')
plt.legend()
plt.title('Neural Network Approximation of f(x) = sin(x)^2')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.show()
```

```
Epoch [100/2000], Loss: 3.3517e-03
Epoch [200/2000], Loss: 2.4322e-03
Epoch [300/2000], Loss: 8.1125e-04
Epoch [400/2000], Loss: 6.2598e-04


Epoch [500/2000], Loss: 4.1802e-04
Epoch [600/2000], Loss: 2.0139e-04
Epoch [700/2000], Loss: 5.6264e-05
Epoch [800/2000], Loss: 1.1275e-05


Epoch [900/2000], Loss: 4.2630e-06
Epoch [1000/2000], Loss: 2.9764e-06
Epoch [1100/2000], Loss: 2.3020e-06
Epoch [1200/2000], Loss: 1.7996e-06


Epoch [1300/2000], Loss: 1.4168e-06
Epoch [1400/2000], Loss: 1.1255e-06
Epoch [1500/2000], Loss: 9.0461e-07
Epoch [1600/2000], Loss: 7.3771e-07


Epoch [1700/2000], Loss: 5.8402e-06
Epoch [1800/2000], Loss: 6.1580e-07
Epoch [1900/2000], Loss: 5.2188e-07
Epoch [2000/2000], Loss: 1.5325e-06
```



Explanation:

- **Network Setup:** The neural network is set up as defined previously.
- **Data Generation:** Random training data is generated within the specified range and the function values are computed as `sin(x)**2`
- **Training Process:** The network is trained using the Adam optimizer and mean squared error (MSE) loss function over 2000 epochs. Adjustments can be made to the learning rate or number of epochs based on how the loss progresses.
- **Testing and Visualization:** The network's predictions are generated for the testing dataset and plotted against the true function values. The training data is also plotted to show how well the network learned to approximate the function within the training range.

This code should provide a clear demonstration of how well the neural network can learn and generalize the function `f(x) = sin(x)**2`.

To ChatGPT: *Explain each of the steps in the for loop over epochs.*

In the neural network training code that uses the `OneVariableNet`, the for-loop over epochs is a critical part where the actual training of the model takes place. Let's break down each step inside this training loop to understand how the model is trained epoch by epoch:

```
epochs = 2000
for epoch in range(epochs):
    optimizer.zero_grad()  # Step 1
    outputs = net(x_train)  # Step 2
    loss = criterion(outputs, y_train)  # Step 3
    loss.backward()  # Step 4
    optimizer.step()  # Step 5

    if (epoch + 1) % 100 == 0:
        print(f'Epoch [{epoch + 1}/{epochs}], Loss: {loss.item():.4f}')
```

Detailed Explanation of Each Step:

1. **Zero the gradients (`optimizer.zero_grad()`)**:
   - **Purpose**: Before the new forward pass, you need to zero out the gradients from the previous pass, because PyTorch accumulates gradients on subsequent backward passes. If this is not done, you would be accumulating gradients across epochs which generally is not desired (except in specific cases).
   - **Effect**: This resets the gradient of all model parameters to zero, ensuring that the backpropagation of this pass starts with a clean slate.
2. **Perform a forward pass (`outputs = net(x_train)`)**:
   - **Purpose**: This computes the predicted outputs (`outputs`) for the given input data (`x_train`). It involves passing the data through the network and calculating the output of each layer sequentially until the final layer.
   - **Effect**: The model uses its current state (weights and biases) to predict the output based on the input data.
3. **Calculate the loss (`loss = criterion(outputs, y_train)`)**:
   - **Purpose**: After obtaining the predictions, this step computes the loss, which is a measure of how far the network's predictions are from the actual target values (`y_train`). The loss function used here is the Mean Squared Error Loss (MSE), which is typical for regression tasks.
   - **Effect**: The calculated loss provides a quantitative measure of the model's performance. A lower loss indicates better model predictions.
4. **Backward pass to compute gradients (`loss.backward()`)**:
   - **Purpose**: This step involves backpropagation. By calling `backward()` on the loss tensor, gradients are computed for every model parameter (weights and biases) that has `requires_grad=True`. This is done by tracing the loss back through the network, calculating the gradient of the loss with respect to each parameter.
   - **Effect**: Updates the gradient of each parameter. These gradients will guide how the weights should be changed to minimize the loss.
5. **Update the weights (`optimizer.step()`)**:
   - **Purpose**: This step applies the gradients to adjust the weights of the network. The specific way the weights are updated depends on the optimizer used (Adam, in this case). Adam adjusts the learning rate dynamically for each parameter.
   - **Effect**: Updates the model's weights to potentially lower the loss in subsequent epochs. This is where the model learns by incrementally improving its predictions.
6. **Printing the loss (Optional Debugging/Logging)**:
   - **Purpose**: Regularly printing the loss gives you insight into how well the training is progressing. By observing the loss decrease over epochs, you can gauge the effectiveness of your training loop and hyperparameters.

- **Effect**: Helps in monitoring the training process and deciding when the model has sufficiently learned or if there might be issues like overfitting or underfitting.

This loop is executed for a fixed number of iterations (`epochs`), and each iteration of this loop is intended to incrementally improve the model's performance by adjusting its parameters based on the calculated loss and its gradients.

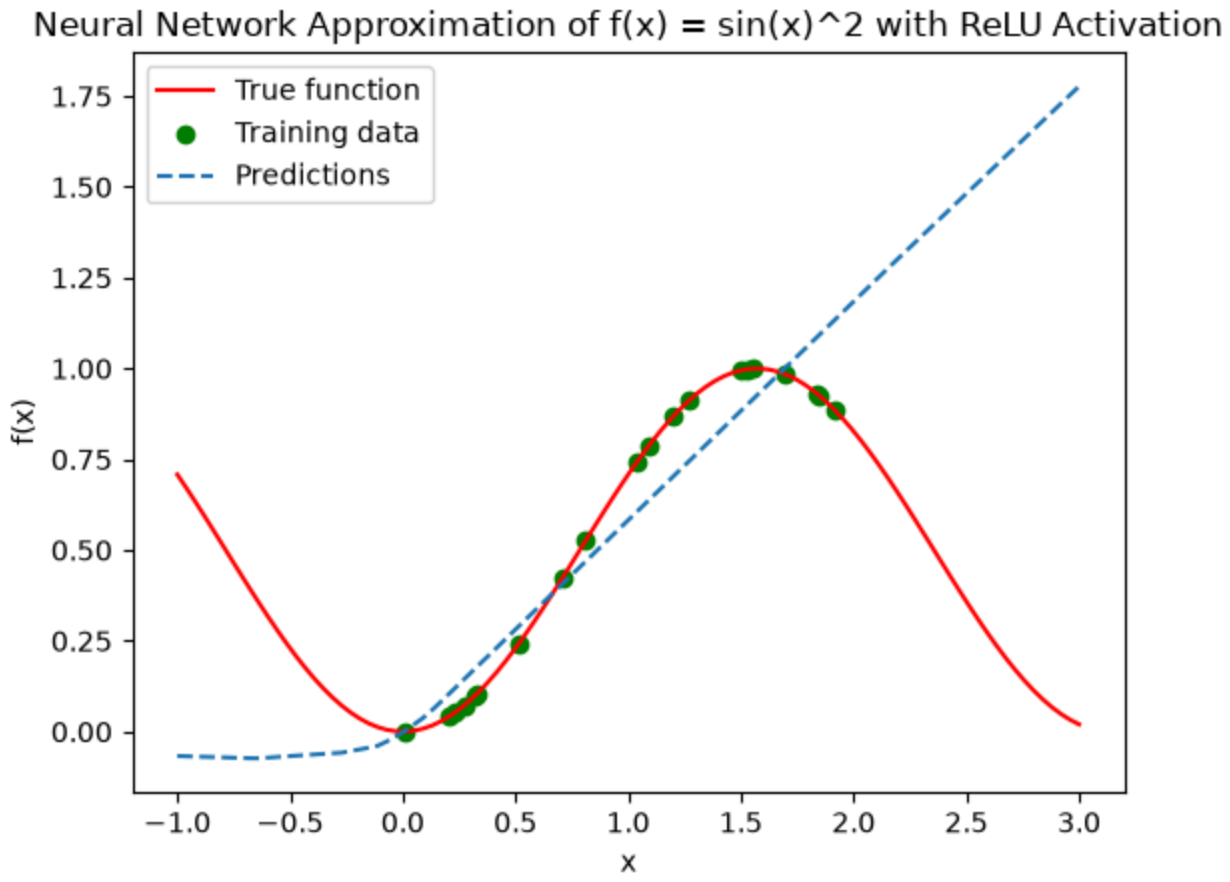
To ChatGPT: *What is the same code with a ReLu activation function?*

To modify the previously described neural network so that it uses the ReLU (Rectified Linear Unit) activation function instead of the Tanh activation function, you simply need to replace the Tanh activation instances with ReLU in the network definition. ReLU is often preferred in deep networks due to its simplicity and effectiveness at avoiding vanishing gradient issues.

Here's how you can update the code to use ReLU:

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt

class OneVariableNet(nn.Module):
    def __init__(self):
        super(OneVariableNet, self).__init__()
        self.hidden1 = nn.Linear(1, 10)
        self.hidden2 = nn.Linear(10, 10)
        self.hidden3 = nn.Linear(10, 10)
        self.output = nn.Linear(10, 1)
        self.activation = nn.ReLU()  # Changed from nn.Tanh() to nn.ReLU()

    def forward(self, x):
        x = self.activation(self.hidden1(x))
        x = self.activation(self.hidden2(x))
        x = self.activation(self.hidden3(x))
        x = self.output(x)
        return x

# Create an instance of the network
net = OneVariableNet()

# Loss function and optimizer
criterion = nn.MSELoss()
optimizer = optim.Adam(net.parameters(), lr=0.01)

# Generate training data
x_train = 2 * torch.rand(20, 1)  # Randomly spaced points
y_train = torch.sin(x_train)**2  # f(x) = sin(x)^2

# Training loop
epochs = 2000
for epoch in range(epochs):
    optimizer.zero_grad()
    outputs = net(x_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 100 == 0:
        print(f'Epoch [{epoch + 1}/{epochs}], Loss: {loss.item():.4e}')

# Generate testing data
x_test = torch.linspace(-1, 3, 100).unsqueeze(1)  # Equally spaced points
#x_test = torch.linspace(0, 2, 50).unsqueeze(1)  # Equally spaced points
y_test = torch.sin(x_test)**2  # f(x) = sin(x)^2
with torch.no_grad():
    predictions = net(x_test)

# Plot the results
plt.plot(x_test.numpy(), y_test.numpy(), label='True function', color='r')
plt.scatter(x_train.numpy(), y_train.numpy(), label='Training data', color='g')
plt.plot(x_test.numpy(), predictions.numpy(), label='Predictions', linestyle='--')
plt.legend()
plt.title('Neural Network Approximation of f(x) = sin(x)^2 with ReLU Activation')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.show()
```

```
Epoch [100/2000], Loss: 1.3591e-02
Epoch [200/2000], Loss: 1.3558e-02
Epoch [300/2000], Loss: 1.3557e-02
Epoch [400/2000], Loss: 1.3557e-02


Epoch [500/2000], Loss: 1.3557e-02
Epoch [600/2000], Loss: 1.3557e-02
Epoch [700/2000], Loss: 1.3557e-02
Epoch [800/2000], Loss: 1.3557e-02


Epoch [900/2000], Loss: 1.3557e-02
Epoch [1000/2000], Loss: 1.3557e-02
Epoch [1100/2000], Loss: 1.3557e-02
Epoch [1200/2000], Loss: 1.3557e-02


Epoch [1300/2000], Loss: 1.3557e-02
Epoch [1400/2000], Loss: 1.3557e-02
Epoch [1500/2000], Loss: 1.3557e-02
Epoch [1600/2000], Loss: 1.3557e-02


Epoch [1700/2000], Loss: 1.3557e-02
Epoch [1800/2000], Loss: 1.3557e-02
Epoch [1900/2000], Loss: 1.3557e-02
Epoch [2000/2000], Loss: 1.3557e-02
```



Changes and Considerations:

- **Activation Function:** We replaced `nn.Tanh()` with `nn.ReLU()`. ReLU activation function might result in different learning characteristics, particularly in how the network fits non-linear functions like `sin(x)**2`. ReLU provides linear responses for positive values and zero for negative values, which might affect how well it models the curve compared to Tanh.
- Learning Characteristics: Networks with ReLU activation might learn faster in some cases due to simpler gradient computations. However, in cases where the function involves negative inputs or outputs that need modeling around zero, Tanh might capture dynamics better because of its symmetric behavior around zero.

This updated code uses ReLU, and you might observe different results in how the model learns and generalizes the function `f(x) = sin(x)**2`. Experimenting with different activation functions can provide insights into their impact on model performance and behavior.

Information from ChatGPT about backpropagation

To ChatGPT: *What algorithm is being used for backpropagation?*

In the provided code, the algorithm being used for backpropagation is not explicitly specified because it's inherently handled by PyTorch's automatic differentiation engine, which is built into the `backward()` function. However, the optimization algorithm that leverages this backpropagation to update the weights is Adam (Adaptive Moment Estimation).

Here's a breakdown of the key components:

Automatic Differentiation

- **Automatic Differentiation:** PyTorch uses a mechanism known as automatic differentiation to compute gradients. This is performed by the `backward()` method on the final loss tensor. This method calculates the gradients of the loss with respect to all tensors that have `requires_grad=True`. These tensors typically represent the weights and biases of the model.
- **Backpropagation:** Under the hood, when you call `loss.backward()`, PyTorch computes the gradient of the loss with respect to all parameters (weights) in the network by tracing back from the loss through the computational graph. Each operation in the graph has a predefined way of calculating gradients in a backward pass, effectively implementing the chain rule. This process is a practical implementation of the backpropagation algorithm.

Optimization Algorithm

- **Adam Optimizer:** The optimizer used here, `optim.Adam`, is an algorithm for first-order gradient-based optimization of stochastic objective functions, based on adaptive estimates of lower-order moments. Adam combines the advantages of two other extensions of stochastic gradient descent, specifically:
  - Adaptive Gradient Algorithm (AdaGrad) that maintains a per-parameter learning rate that improves performance on problems with sparse gradients (like natural language and computer vision problems).
  - Root Mean Square Propagation (RMSProp) that also maintains per-parameter learning rates that are adapted based on the average of recent magnitudes of the gradients for the weight (this means it works well on online and non-stationary problems).

Why Adam?

- **Efficiency:** Adam is computationally efficient and has little memory requirement.
- **Adaptive Learning Rate:** It handles sparse gradients on noisy problems very well.
- **Suitability for Problems:** Adam is generally well suited for problems that are large in terms of data and/or parameters.
- **Hyperparameters:** It has few hyperparameters that require tuning (like the learning rate, the exponential decay rates for the moment estimates).

In essence, while the backpropagation algorithm is the method used to compute the gradients, Adam is the algorithm that uses these gradients to update the weights iteratively in order to minimize the loss function.

Information from ChatGPT about tensor operations

To ChatGPT: *How do you write mathematical expressions using pytensor?*

In PyTorch, writing mathematical expressions involves using a combination of PyTorch's tensor operations, which are very similar to those found in NumPy but are optimized for automatic differentiation and can run on GPUs. Here's how you can utilize PyTorch to handle various mathematical expressions:

Basic Mathematical Operations

Here are some basic operations:

1. **Addition**

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
c = a + b  # Element-wise addition
print(c)
```

```
tensor([5, 7, 9])
```

2. **Subtraction**

```
d = b - a  # Element-wise subtraction
print(d)
```

```
tensor([3, 3, 3])
```

3. **Multiplication** (element-wise)

```
e = a * b  # Element-wise multiplication
print(e)
```

```
tensor([ 4, 10, 18])
```

4. **Division**

```
f = b / a  # Element-wise division
print(f)
```

```
tensor([4.0000, 2.5000, 2.0000])
```

5. **Matrix multiplication**

```
x = torch.tensor([[1, 2], [3, 4]])
y = torch.tensor([[5, 6], [7, 8]])
z = torch.matmul(x, y)  # Matrix multiplication
# or using the "@" operator
z = x @ y
print(z)
```

```
tensor([[19, 22],
        [43, 50]])
```

Complex Expressions

For more complex mathematical expressions, such as those involving powers, roots, logarithms, trigonometric functions, and other operations, PyTorch provides a wide range of functions:

```
import torch

x = torch.tensor([1, 2, 3], dtype=torch.float32)

# Power
x_squared = x.pow(2)  # or x ** 2

# Square root
x_sqrt = torch.sqrt(x)

# Exponential
x_exp = torch.exp(x)

# Logarithm
x_log = torch.log(x)

# Trigonometric functions
x_sin = torch.sin(x)
x_cos = torch.cos(x)

# Hyperbolic functions
x_sinh = torch.sinh(x)
x_cosh = torch.cosh(x)
```

Example of a Complex Expression

If you want to compute something more complex, like the following expression:

$$f(x) = \sin(x^2) + \log(x) \cdot \cos(x)$$

Here's how you would implement it in PyTorch:

```
import torch

# Assuming x is a tensor of positive numbers
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)

# Compute the function
f = torch.sin(x**2) + torch.log(x) * torch.cos(x)
print(f)
```

```
tensor([ 0.8415, -1.0453, -0.6755], grad_fn=<AddBackward0>)
```

Gradient Computations

A key feature of PyTorch is its ability to automatically compute derivatives, which is crucial for machine learning algorithms like backpropagation. Here's how you can compute the gradient of f with respect to x:

```
f.sum().backward()  # Compute the gradients (derivative of sum of f wrt x)
print(x.grad)       # Access the gradients
```

```
tensor([ 1.6209, -3.4529, -5.9518])
```

This example assumes that f is not a scalar, so we use f.sum() to create a scalar context for backward(), which computes the total derivative.

Utilizing GPU Acceleration

To leverage GPU acceleration, you would typically move your tensors and models to the GPU:

```
if torch.cuda.is_available():
    x = x.to('cuda')
    # Perform your operations on x here
```

This approach to writing mathematical expressions in PyTorch enables both flexibility and powerful integration with deep learning models, allowing for both simple and advanced computations, all while being able to backpropagate errors through complex computational graphs efficiently.

Information from ChatGPT about initialization

To ChatGPT: *How are these neural networks initialized?*

Neural networks in frameworks like PyTorch and TensorFlow are initialized using specific strategies for setting the initial weights and biases of the network before training begins. Proper initialization is crucial because it can significantly affect the convergence and performance of the network during training.

Common Initialization Methods

Here are some of the most common methods used for initializing neural networks:

1. **Zero Initialization:**
   - Weights and biases are set to zero.
   - Generally not recommended except for biases because it can lead to problems where neurons in the same layer learn the same features during training (symmetry breaking issue).
2. **Random Initialization:**
   - Weights are set to small random values, often drawn from a normal or uniform distribution.
   - Helps in breaking symmetry so that each neuron can learn different things.

    - Too small values can lead to the vanishing gradients problem, and too large values can lead to exploding gradients.
3. **Uniform and Normal Distribution:**
    - **Uniform Distribution:** Weights are initialized uniformly at random within a specified range (e.g., $[-y, y]$ where y is a small value).
    - **Normal Distribution:** Weights are initialized according to a Gaussian distribution, usually with a mean of 0 and a small standard deviation.
4. **Xavier/Glorot Initialization:**
    - Designed specifically for keeping the scale of gradients roughly the same in all layers.
    - For a layer with n_{in} inputs and n_{out} outputs, weights are initialized from a distribution with zero mean and a variance of $2/(n_{\text{in}} + n_{\text{out}})$.
    - Works well with sigmoid and tanh activation functions.
5. **He Initialization:**
    - Similar to Xavier initialization but designed for layers with ReLU activation functions.
    - Weights are initialized from a normal distribution with a mean of 0 and a variance of 2 / n_in or a uniform distribution scaled by sqrt(6 / n_in), where n_in is the number of input units.
    - Helps prevent issues related to the non-linear nature of ReLU activations at initialization.

PyTorch Default Initialization

In PyTorch, different types of layers are initialized differently by default:

- **Linear layers** (`nn.Linear`): By default, the weights and biases are initialized from a uniform distribution $U(-\sqrt{k}, \sqrt{k})$ where $k = 1/(\text{number of input features} \quad)$.
- **Convolutional layers** (`nn.Conv2d`): These use Kaiming uniform initialization by default, which is a form of He initialization suitable for layers followed by ReLU activations.

Customizing Initialization in PyTorch

You can also customize how layers are initialized in PyTorch. Here's a simple example where we set the weights of a linear layer with He initialization and set biases to zero:

```
import torch
import torch.nn as nn
import torch.nn.init as init

class MyModel(nn.Module):
    def __init__(self):
        super(MyModel, self).__init__()
        self.linear = nn.Linear(10, 5)

        # Custom weights initialization
        init.kaiming_normal_(self.linear.weight, mode='fan_out', nonlinearity='relu')
        init.constant_(self.linear.bias, 0)  # setting biases to zero

model = MyModel()
print(model)
```

```
MyModel(
  (linear): Linear(in_features=10, out_features=5, bias=True)
)
```

This flexibility allows you to experiment with different initialization methods to find what works best for your specific application or to adapt to different activation functions used in the network.

Information from ChatGPT about the default initialization

To ChatGPT: *Which initialization method was used in the latest code you provided above?*

In the example of the `OneVariableNet` class I provided earlier, the network uses PyTorch's default initialization methods for the layers defined within it. For `nn.Linear` layers in PyTorch, the default initialization is done with a uniform distribution over a range derived from the number of input and output units of the layer. Specifically:

1. **Weights:** Initialized from a uniform distribution $U(-\sqrt{k}, \sqrt{k})$ where $k = 1/(\text{number of input features})$.
2. **Biases:** Also typically initialized from the same uniform distribution, unless otherwise specified or customized.

This is a good default choice for many situations, especially when just starting out with a new neural network model, because it provides a reasonable compromise between too large and too small initial weights, helping to ensure that gradients are neither too small (causing vanishing gradients) nor too large (causing exploding gradients) at the start of training.

Information from ChatGPT about how to switch to normal distributions to intialize

To ChatGPT: *Give the `OneVariableNet` class initialized with a normal distribution.*

To customize the `OneVariableNet` class such that all the weights (and optionally biases) of the layers are initialized from a normal distribution, you can use PyTorch's `torch.nn.init` functions. In this case, we will use `normal_` to initialize the weights and biases.

Here's how you can modify the `OneVariableNet` class to initialize its parameters with a normal distribution, specifying a mean and a standard deviation:

```
import torch
import torch.nn as nn
import torch.nn.init as init

class OneVariableNet(nn.Module):
    def __init__(self):
        super(OneVariableNet, self).__init__()
        self.hidden1 = nn.Linear(1, 10)
        self.hidden2 = nn.Linear(10, 10)
        self.hidden3 = nn.Linear(10, 10)
        self.output = nn.Linear(10, 1)
        self.activation = nn.ReLU()  # or any other activation

        # Initialize weights and biases with a normal distribution
        self.init_weights()

    def init_weights(self):
        # Initialize weights with a normal distribution
        for m in self.modules():
            if isinstance(m, nn.Linear):
                init.normal_(m.weight, mean=0.0, std=0.1)
                init.constant_(m.bias, 0)  # Initialize biases to zero or use normal
distribution as well

    def forward(self, x):
        x = self.activation(self.hidden1(x))
        x = self.activation(self.hidden2(x))
        x = self.activation(self.hidden3(x))
        x = self.output(x)
        return x

# Create an instance of the network
net = OneVariableNet()
print(net)
```

```
OneVariableNet(
  (hidden1): Linear(in_features=1, out_features=10, bias=True)
  (hidden2): Linear(in_features=10, out_features=10, bias=True)
  (hidden3): Linear(in_features=10, out_features=10, bias=True)
  (output): Linear(in_features=10, out_features=1, bias=True)
  (activation): ReLU()
)
```

Explanation:

1. `init_weights` **Method:** This method iterates through all modules (layers) in the network. If the module is an instance of `nn.Linear` (a linear layer), it initializes the weights using a normal distribution with specified mean and standard deviation (mean=0.0, std=0.1 in this example). Biases are set to zero using `init.constant_`. You can also choose to initialize biases from a normal distribution if desired.
2. **Initialization in Constructor:** The `init_weights` method is called within the constructor (`__init__`) of the class, ensuring that all weights and biases are initialized properly when an instance of `OneVariableNet` is created.

This approach lets you control the initialization process in depth, adapting it as needed for different network architectures or experiments. Adjusting the mean and standard deviation of the normal distribution can help in fine-tuning the initialization process according to specific needs of your training dynamics.

27.5. Demo: MNIST classifier in PyTorch

This notebook is a PyTorch rewrite of the original TensorFlow/Keras demo.
It trains a simple fully-connected network on MNIST and reproduces:

- data loading
- model definition (Flatten → Dense(128, ReLU) → Dropout(0.2) → Dense(10))
- training for 10 epochs
- accuracy on the test set
- probability predictions for a sample
- the same plotting helpers for predictions

```
# Imports
import os, math, numpy as np


import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt

print('You have PyTorch version:', torch.__version__)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)
torch.manual_seed(1234)
```

```
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
You have PyTorch version: 2.6.0
Using device: cpu
```

```
<torch._C.Generator at 0x111a171f0>
```

```
# Data: MNIST via torchvision (returns tensors in [0,1])
batch_size = 128

transform = transforms.ToTensor()  # simple scaling to [0,1]
train_dataset = datasets.MNIST(root='../assets', train=True, download=True,
transform=transform)
test_dataset  = datasets.MNIST(root='../assets', train=False, download=True,
transform=transform)

x_train = train_dataset.data.numpy()
y_train = train_dataset.targets.numpy()
x_test  = test_dataset.data.numpy()
y_test  = test_dataset.targets.numpy()

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2)
test_loader  = DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=2)

print('Train:', len(train_dataset), 'Test:', len(test_dataset))
```

```
Train: 60000 Test: 10000
```

```python
# Model: Flatten → Linear(784,128) → ReLU → Dropout(0.2) → Linear(128,10)
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 128),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, 10)
        )
    def forward(self, x):
        return self.net(x)

model = MLP().to(device)
sum(p.numel() for p in model.parameters()), model
```

```
(101770,
 MLP(
   (net): Sequential(
     (0): Flatten(start_dim=1, end_dim=-1)
     (1): Linear(in_features=784, out_features=128, bias=True)
     (2): ReLU()
     (3): Dropout(p=0.2, inplace=False)
     (4): Linear(in_features=128, out_features=10, bias=True)
   )
 ))
```

```
# Training
criterion = nn.CrossEntropyLoss()  # expects raw logits
optimizer = torch.optim.Adam(model.parameters())
epochs = 10

history = {'accuracy': [], 'loss': []}

for epoch in range(1, epochs+1):
    model.train()
    running_loss = 0.0
    correct, total = 0, 0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * xb.size(0)

        # compute training accuracy
        preds = logits.argmax(dim=1)
        correct += (preds == yb).sum().item()
        total += yb.size(0)

    epoch_loss = running_loss / total
    epoch_acc = correct / total
    history['loss'].append(epoch_loss)
    history['accuracy'].append(epoch_acc)
    print(f'Epoch {epoch:2d}/{epochs} - loss: {epoch_loss:.4f} - acc: {epoch_acc:.4f}')

history
```

```
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch  1/10 - loss: 0.4442 - acc: 0.8808
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch  2/10 - loss: 0.2119 - acc: 0.9388
```

```
# Evaluation on test set
model.eval()
correct, total, test_loss = 0, 0, 0.0
with torch.no_grad():
    for xb, yb in test_loader:
        xb, yb = xb.to(device), yb.to(device)
        logits = model(xb)
        loss = criterion(logits, yb)
        test_loss += loss.item() * xb.size(0)
        preds = logits.argmax(dim=1)
        correct += (preds == yb).sum().item()
        total += yb.size(0)

test_acc = correct / total
test_loss = test_loss / total
print(f'\nTest accuracy: {test_acc:5.3f}')
test_acc
```

```
# Predict class probabilities for the first test image (to mirror Keras softmax output)
softmax = nn.Softmax(dim=1)
model.eval()
with torch.no_grad():
    sample = test_dataset[0][0].unsqueeze(0).to(device)  # 1x1x28x28
    probs = softmax(model(sample)).cpu().numpy()[0]

probs, probs.sum(), probs.argmax()
```

```
# Helper plotting functions (ported from the Keras version)
def plot_image(i, predictions_array, true_label, img):
    predictions_array, true_label, img = predictions_array, int(true_label), img
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])

    plt.imshow(img, cmap=plt.cm.binary)
    predicted_label = int(np.argmax(predictions_array))
    color = 'blue' if predicted_label == true_label else 'red'
    plt.xlabel(f"{predicted_label} {100*np.max(predictions_array):2.0f}% (true: {true_label})",
color=color)

def plot_value_array(i, predictions_array, true_label):
    predictions_array, true_label = predictions_array, int(true_label)
    plt.grid(False)
    plt.xticks(range(10))
    plt.yticks([])
    thisplot = plt.bar(range(10), predictions_array)
    predicted_label = int(np.argmax(predictions_array))
    thisplot[predicted_label].set_color('red')
    thisplot[true_label].set_color('green')
```

```
# Plot a grid of test images with predicted probabilities
num_rows = 5
num_cols = 3
num_images = num_rows * num_cols
plt.figure(figsize=(2*2*num_cols, 2*num_rows))

# Precompute probabilities for first num_images
with torch.no_grad():
    xs = torch.stack([test_dataset[i][0] for i in range(num_images)]).to(device)
    probs_batch = nn.Softmax(dim=1)(model(xs)).cpu().numpy()

for i in range(num_images):
    img, label = test_dataset[i]
    plt.subplot(num_rows, 2*num_cols, 2*i+1)
    plot_image(i, probs_batch[i], label, img.squeeze().numpy())
    plt.subplot(num_rows, 2*num_cols, 2*i+2)
    plot_value_array(i, probs_batch[i], label)

plt.tight_layout()
plt.show()
```

28. *Convolutional neural networks

Convolutional Neural Networks (CNNs) are very similar to ordinary Neural Networks, but are particularly adopted for image recognition.

- They are made up of layers that have learnable weights and biases.
- The inputs are operated on with dot products, typically followed by a non-linear activation function.
- The whole network still expresses a single differentiable score function: from the raw image pixels on one end to class scores at the other.
- And they still have a loss function (for example Softmax) on the last (fully-connected) layer.
- Learning takes place via back propagation, gradient descent, etc.

What is the difference? **CNN architectures make the explicit assumption that the inputs are images, which allows us to encode certain properties into the architecture. These then make the forward function more efficient to implement and vastly reduce the amount of parameters in the network.**

Here we provide only a superficial overview.

28.1. Regular NNs don’t scale well to full images

As an example, consider an image of size $32 \times 32 \times 3$ (32 wide, 32 high, 3 color channels), so a single fully-connected neuron in a first hidden layer of a regular Neural Network would have $32 \times 32 \times 3 = 3072$ weights. This amount still seems manageable, but clearly this fully-connected structure does not scale to larger images. For example, an image of more respectable size, say $200 \times 200 \times 3$, would lead to neurons that have $200 \times 200 \times 3 = 120,000$ weights.

We could have several such neurons, and the parameters would add up quickly! Clearly, this full connectivity is wasteful and the huge number of parameters would quickly lead to possible overfitting.

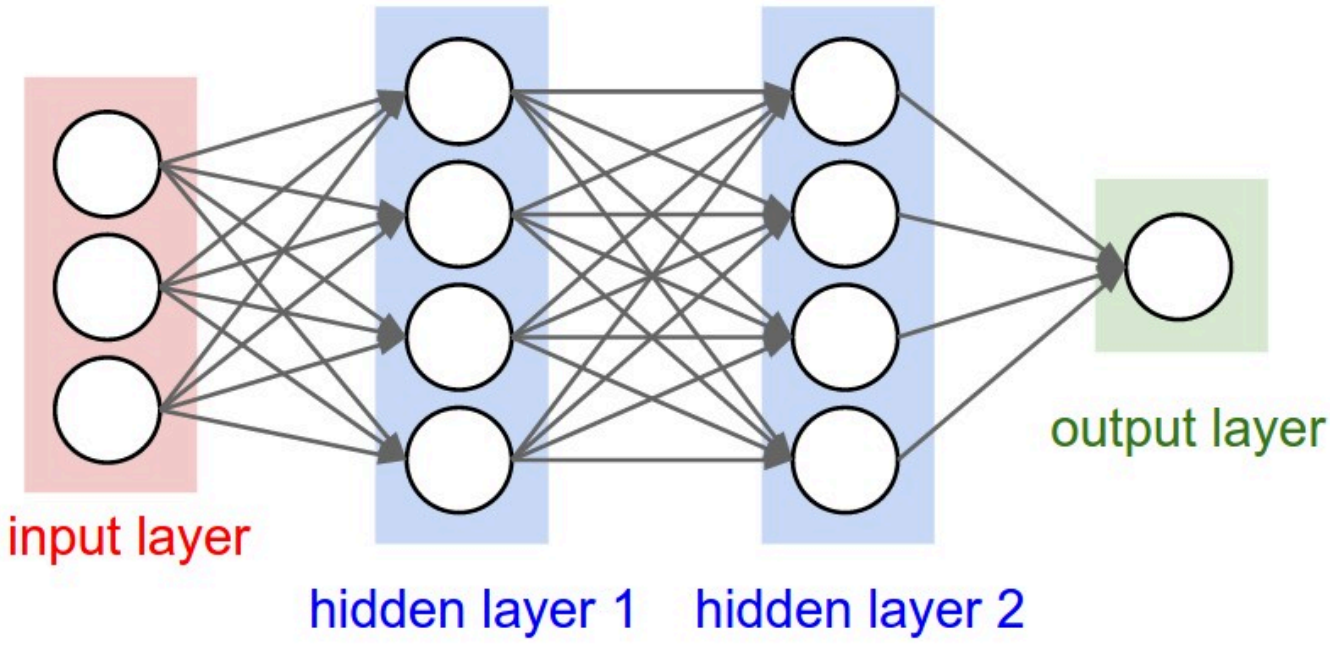


Fig. 28.1 A three-layer Neural Network.

28.2. 3D volumes of neurons

CNNs take advantage of the fact that the input consists of images and they constrain the architecture in a more sensible way.

In particular, unlike a regular Neural Network, the layers of a CNN have neurons arranged in 3 dimensions: width, height, depth. (Note that the word depth here refers to the third dimension of an activation volume, not to the depth of a full Neural Network, which can refer to the total number of layers in a network.)

To understand it better, the above example of an image with an input volume of activations has dimensions $32 \times 32 \times 3$ (width, height, depth respectively).

The neurons in a layer will only be connected to a small region of the layer before it, instead of all of the neurons in a fully-connected manner. Moreover, the final output layer could for this specific image have dimensions $1 \times 1 \times 10$, because by the end of the CNN architecture we will reduce the full image into a single vector of class scores, arranged along the depth dimension.

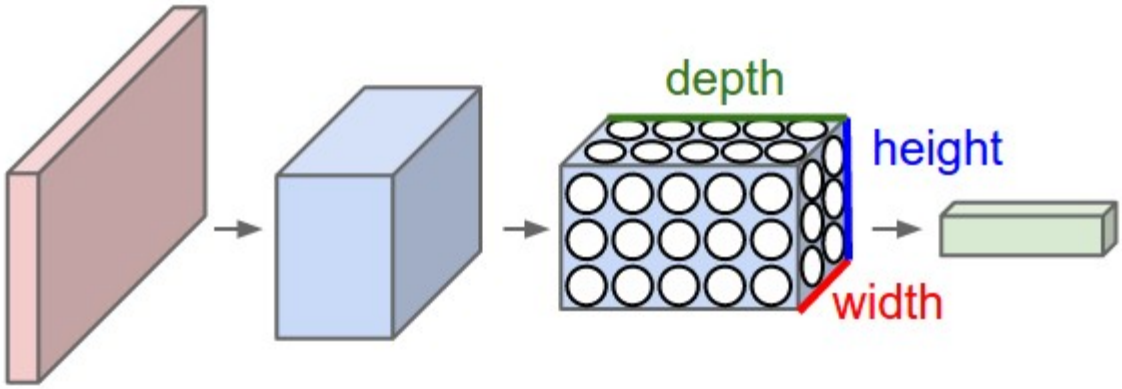


Fig. 28.2 A CNN arranges its neurons in three dimensions (width, height, depth), as visualized in one of the layers. Every layer of a CNN transforms the 3D input volume to a 3D output volume of neuron activations. In this example, the red input layer holds the image, so its width and height would be the dimensions of the image, and the depth would be 3 (Red, Green, Blue channels).

28.3. Layers used to build CNNs

A simple CNN is a sequence of layers, and every layer of a CNN transforms one volume of activations to another through a differentiable function. We use three main types of layers to build CNN architectures: Convolutional Layer, Pooling Layer, and Fully-Connected Layer (exactly as seen in regular Neural Networks). We will stack these layers to form a full CNN architecture.

The layers of a convolutional neural network arrange neurons in 3D: width, height and depth.
The input image is typically a square matrix of depth 3.

A **convolution** is performed on the image which outputs a 3D volume of neurons. The weights to the input are arranged in a number of 2D matrices, known as **filters**.

Each filter slides along the input image, taking the dot product between each small part of the image and the filter, in all depth dimensions. This is then passed through a non-linear function, typically the **Rectified Linear (ReLu)** function, which serves as the activation of the neurons in the first convolutional layer. This is further passed through a **pooling layer**, which reduces the size of the convolutional layer, e.g. by taking the maximum or average across some small regions, and this serves as input to the next convolutional layer.

Example: CNN architecture

A simple CNN for image classification could have the architecture:

- **INPUT** ($32 \times 32 \times 3$) will hold the raw pixel values of the image, in this case an image of width 32, height 32, and with three color channels R,G,B.
- **CONV** (convolutional)layer will compute the output of neurons that are connected to local regions in the input, each computing a dot product between their weights and a small region they are connected to in the input volume. This may result in volume such as $[32 \times 32 \times 12]$ if we decided to use 12 filters.
- **RELU** layer will apply an elementwise activation function, such as the $max(0, x)$ thresholding at zero. This leaves the size of the volume unchanged ($[32 \times 32 \times 12]$).
- **POOL** (pooling) layer will perform a downsampling operation along the spatial dimensions (width, height), resulting in volume such as $[16 \times 16 \times 12]$.

- **FC** (i.e. fully-connected) layer will compute the class scores, resulting in volume of size $[1 \times 1 \times 10]$, where each of the 10 numbers correspond to a class score, such as among the 10 categories of the MNIST images we considered above . As with ordinary Neural Networks and as the name implies, each neuron in this layer will be connected to all the numbers in the previous volume.

Systematic reduction

By systematically reducing the size of the input volume, through convolution and pooling, the network should create representations of small parts of the input, and then from them assemble representations of larger areas. The final pooling layer is flattened to serve as input to a hidden layer, such that each neuron in the final pooling layer is connected to every single neuron in the hidden layer. This then serves as input to the output layer, e.g. a softmax output for classification.

28.4. Transforming images

CNNs transform the original image layer by layer from the original pixel values to the final class scores.

Observe that some layers contain parameters and other don't. In particular, the CNN layers perform transformations that are a function of not only the activations in the input volume, but also of the parameters (the weights and biases of the neurons). On the other hand, the RELU/POOL layers will implement a fixed function. The parameters in the CONV/FC layers will be trained with gradient descent so that the class scores that the CNN computes are consistent with the labels in the training set for each image.

Example: The MNIST dataset

The MNIST dataset consists of grayscale images with a pixel size of 28×28, meaning we require $28 \times 28 = 724$ weights to each neuron in the first hidden layer.

If we were to analyze images of size 128×128 we would require $128 \times 128 = 16384$ weights to each neuron. Even worse if we were dealing with color images, as most images are, we have an image matrix of size 128×128 for each color dimension (Red, Green, Blue), meaning 3 times the number of weights $= 49152$ are required for every single neuron in the first hidden layer.

Setting it up

It means that to represent the entire dataset of images, we require a 4D matrix or **tensor**. This tensor has the dimensions:

$$\left(n_{\text{inputs}},\, n_{\text{pixels,width}},\, n_{\text{pixels,height}},\, n_{\text{pixels,depth}}\right). \tag{28.1}$$

28.5. CNNs in brief

In summary:

- A CNN architecture is in the simplest case a list of layers that transform the image volume into an output volume (e.g. holding the class scores)
- There are a few distinct types of layers (e.g. CONV/FC/RELU/POOL)
- Each layer accepts an input 3D volume and transforms it to an output 3D volume through a differentiable function
- Each layer may or may not have parameters (e.g. CONV/FC do, RELU/POOL don't)
- Each layer may or may not have additional hyperparameters (e.g. CONV/FC/POOL do, RELU doesn't)

For more material on convolutional networks, we strongly recommend the slides of CS231 which is taught at Stanford University. Furthermore, Michael Nielsen's book Neural Networks and Deep Learning is a very good read, in particular chapter 6 which deals with CNNs.

28.6. What is a convolutional neural network?

Notes by Alberto Garcia (2021)

A convolutional neural network (CNN) is a machine learning algorithm that is useful for classifying images. It takes in an image as an input, assigns importance (through weights) to various features of the image such as objects, and learns how to differentiate between the images.

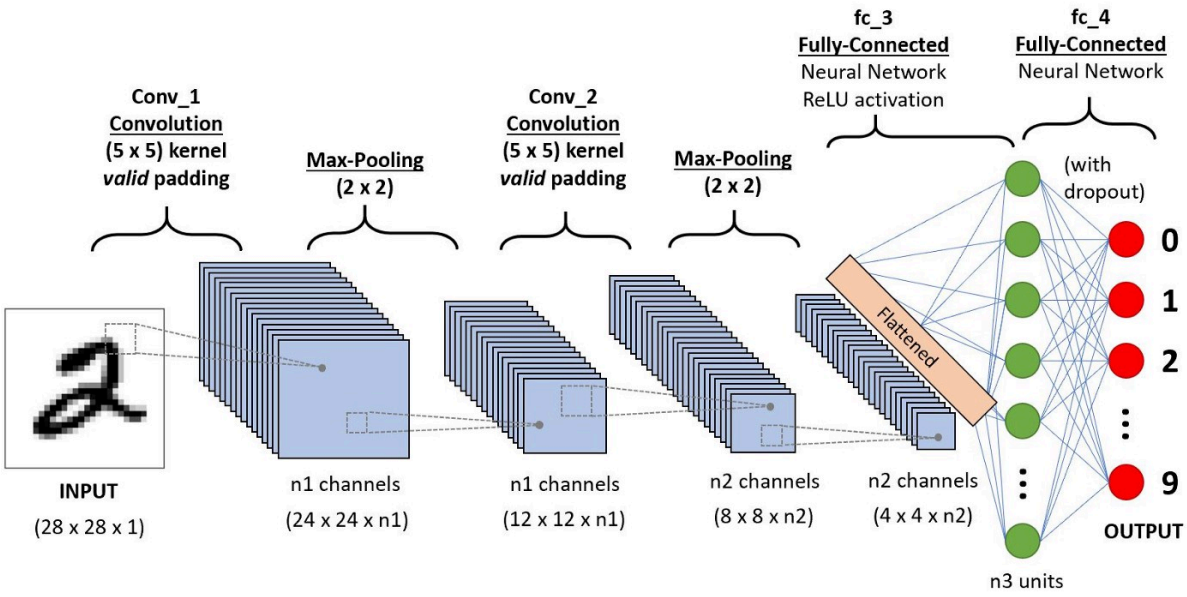


Image taken from useful links #1.

- Some useful links:
  1. https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53
  2. https://heartbeat.fritz.ai/classification-with-tensorflow-and-dense-neural-networks-8299327a818a#:~:text=What is a dense neural network%3F&text=Each neuron in a layer,those in the next layer.
  3. https://towardsdatascience.com/types-of-convolution-kernels-simplified-f040cb307c37
  4. https://towardsdatascience.com/a-beginners-guide-to-convolutional-neural-networks-cnns-14649dbddce8#:~:text=In convolutional networks%2C multiple filters,%2C say%2C a dark edge.

Why are traditional neural networks not enough?

Although neural networks (NNs) are a great tool for making predictions, there are a few reasons why it is not wise to use NNs when dealing with images:

- Multi-layer perceptrons use one perceptron per input. For images, one input would correspond to one pixel. If the image is 224x224, that is ~50,000 inputs. If the image is in color, that is x3 pixels, so 224x224x3 ~150,000. This means we'll have ~150,000 weights per neuron that need to be trained…can lead to overfitting and slow training process.
- NNs are not translationally invarient. If an important feature changes location in pictures, the NN will try to correct for that change. This leads to weights not being trained properly.

Why use convolutional neural networks?

- The influence of neighboring pixels is analyzed by something called a filter. This serves in reducing the dimensions of the weights by picking the important overall feature in sections of the image.
- The pooling layer also serves as a way to reduce the dimensions.
- CNNs are translationally invarient since they do not care exactly where the feature is in the image, but if the feature exists.

Different layers of a CNN

Convolutional layer:

This layer is used to extract features from the input image by use of a filter **whose elements are weights that undergo a training process**. These filters are typically 3x3 or 5x5 matrices that get convoluted with the image pixels. Even number filters are avoided since we want our feature maps to end up with a center cell. If it doesn't there can be problems moving to the next layer. **We will discuss greyscale images**.

- The size of the filters are much smaller than the size of the input image. The process of obtaining feature maps involves taking the scalar (dot) product between the image and the filter. By applying the same filter to an image, it allows the filter to discover features anywhere in that image (the translation invariance that was mentioned above). We can conclude that the filter allows us to see if a feature is present as opposed to where it is in the image.
- The filters themselves are built from random numbers (from my current understanding) and get updated as the network is trained. There are certain filters that correspond to certain operations like edge detection and image sharpening. The output of a filter is a feature map. These are used to predict the class of each image. The number of filters used can be chosen. An example of the convolution operation is

$$\text{image} = \begin{pmatrix} 3 & 0 & 1 & 5 \\ 2 & 6 & 2 & 4 \\ 2 & 4 & 1 & 0 \\ 2 & 3 & 1 & 4 \end{pmatrix}, \qquad \text{filter} = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

- First matrix element:

$$(3)(-1)+(0)(0)+(1)(1)+(2)(-2)+(6)(0)+(2)(2)+(2)(-1)+(4)(0)+(1)(1)=-3$$

- The outputted feature map is:

$$\text{feature map} \quad = \text{image} \quad * \text{filter} \quad = \begin{pmatrix} -3 & -3 \\ -3 & -9 \end{pmatrix}$$

- Once the convolution happens, the feature maps are stacked up. This represents the new image. These feature maps are then passed through an activation function which decides whether a certain feature is present anywhere in the image.
- If we think about tradicional neural networks and the transition from input to hidden layer, the operation is

$$\vec{X}_{\text{train}} \cdot \vec{W} + b_0 w_0.$$

- We have a similar operation in the convolutional layer of a CNN. In this case, the elements of $\vec{X}_{\text{train}}$ is composed of the pixels of the input image and the elements of $\vec{W}$ are the values of the filter. These get multipled in form of a scalar product, get summed up, and get passed into the activation function. In turn, each element of the feature map will go through the activation function. This amounts to the operation

$$h_{\text{out}} = f\big(\vec{X}_{\text{train}} \cdot \vec{W} + b_0 w_0\big).$$

- As mentioned above, there are certain filters that pick out certain features. **We do not concern ourselves with picking any specific filter**. The point of the convolutional layer is to **train** the filters to recognize feature. The network is forced to learn how to properly extract features in order to minimize the loss. This is similar to neural networks where we compare the prediction to the true value. The error has to be below some threshold in order for the training to be complete. The completed “product” is a CNN with many filters that built in such a way that they pick out the most important features of an image **in tandem** to properly classify it.

How do we choose parameters for the convolutional layer?

Examples:
network.add(layers.Conv2D(32,(3,3), activation=‘relu’, input_shape=(64,64,3)))
network.add(layers.Conv2D(64, (5,5), activation=‘relu’))

- Input shape:
  - This corresponds to the size of the input image. In the example above the input image is size 64x64 and is in color.

- Size of filter:
  - You want the filter to be small to pick up as many details as possible. Let’s start with the smallest choice, evaluate it’s use, and increase the size:
    1. 1x1

        - In picking a 1x1 filter, we will get a feature map that is essentially the same as the image and will not get information about neighboring pixels. This filter would be of no help!
    2. 2x2
        - Even dimensional filters are generally **not preferred**. The reason is that odd-sized filters are symmetrical about the middle element. An even-sized filter will not have this symmetry and can lead to distortions.
  - This leaves us with 3x3 and 5x5 being the smallest. These are the sizes used widely throughout CNNs.
  - Interesting addition: In the ImageNet Recognition challenge, Google introduced a CNN where they replaced the 3x3 convolutional layer with a 1x3 and 3x1 layer. So they split up the process into a series of one dimensional operations.

- Number of filters:
  - There is no set way of choosing the number of filters. This is a hyperparameter that one must play with to see which is best for the dataset. The numbers chosen are usually powers of 2: 32, 63, 128, 256, 512,… etc. This has to do with how many threads are in GPUs. Normally a group is composed of 32 threads. If you have a convolutional layer with 40 filters, it will need 64 threads (2 groups). At that point you might as well take up all the threads possible.

- Activation function:
  - Will be discussed in the next section.
- More information can be found at:
  1. https://machinelearningmastery.com/convolutional-layers-for-deep-learning-neural-networks/
  2. https://towardsdatascience.com/deciding-optimal-filter-size-for-cnns-d6f7b56f9363
  3. https://medium.com/@himanshuxd/activation-functions-sigmoid-relu-leaky-relu-and-softmax-basics-for-neural-networks-and-deep-8d9c70eed91e
  4. https://medium.com/analytics-vidhya/how-to-choose-the-size-of-the-convolution-filter-or-kernel-size-for-cnn-86a55a1e2d15
  5. https://stackoverflow.com/questions/51103639/optimal-number-of-filters-in-a-convolutional-network

Activation function

- Once we have a stack of feature maps, each value of the map is passed through a function known as the activation function. It is important to choose a non-linear activation function since the data one is passing into the network is usually non-linear. This will allow for the CNN to generalize better.
- A function is chosen in order to set boundaries on the values passed through. This forces the values into a certain range and reduces the chances of the weights blowing up. The most used activation function in CNN (according to multiple articles) is the ReLU (rectified linear unit). The main reason for their usage is that they are cheap computationally and they throw out negative numbers. You'll either get a 0 or 1 when computing the gradients, which make the training portion a lot faster compared to using sigmoid or tanh.

$$\text{ReLU function} \quad \longrightarrow \quad f(x) = \begin{cases} 0, & \text{for} \quad x < 0 \\ x, & \text{for} \quad x \geq 0 \end{cases}$$

- More information can be found at:
  1. https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/

Pooling layer

- The function of the pooling layer is to reduce the size of the feature maps in order to reduce the amount of parameters needed, thus decreasing the computational time. These layers will operate on each feature map independently. Values from the feature maps are selected and are used as inputs for the subsequent layers. There are different ways to select these values. The common way of doing it is max pooling. This grabs the largest value using a pre-determined size. In addition, you can pick a stride. This tells the filter how to move across the feature map. For example, let's consider a pooling layer with size 2x2 and stride 2

$$\text{feature map} = \begin{pmatrix} 1 & 1 & 2 & 4 \\ 5 & 6 & 7 & 8 \\ 3 & 2 & 1 & 0 \\ 1 & 2 & 3 & 4 \end{pmatrix}, \longrightarrow 2x2 \text{ pooling filter} \longrightarrow \text{output} = \begin{pmatrix} 6 & 8 \\ 3 & 4 \end{pmatrix}$$

How do we choose the size of the pooling layer?

- The size of the pooling layer is usually chosen to be 2x2 or 3x3. The reason is because the point of the pooling layer is to reduce the size of the feature maps. As we increase our pooling size, we decrease the resolution. That's why it is best to stick with 2x2 or 3x3 pooling size to not lose too many details.

Different types of pooling

- There are three popular ways of pooling: max, min, and average. Max pooling takes the maximum value in the matrix sub-block and throws away the rest. Min pooling takes the minimum value and throws away the rest. Average pooling averages all the values and takes that as the element. Which is better to use?
    1. Min pooling:
        - Typically not good to use since you'll be taking the smallest, least important pixel. This can usually just be noise.
    2. Average pooling:
        - Not a bad choice to pick. This should give you a good scope of the image. The only issue is that the image will be smoothed (smeared) out and the sharp features will not be identified.
    3. Max pooling:
        - This is the best and mostly commonly used choice because it chooses the most important features. The image is already being downsized by pooling so we need to make sure to choose the pixels that really capture the essential features of the image. These are usually the ones with the brightest (largest) number. This is not always the case! It all depends on your data.
- More information can be found at:
    1. https://machinelearningmastery.com/pooling-layers-for-convolutional-neural-networks/
    2. https://medium.com/@bdhuma/which-pooling-method-is-better-maxpooling-vs-minpooling-vs-average-pooling-95fb03f45a9#:~:text=Average pooling method smooths out,lighter pixels of the image.

Multiple color channels

- So far we have only discussed a greyscale picture. All pictures are three-dimenional inputs since they have pixels along the image and a color. For the case of greyscale the dimensions would be (for example) 32x32x1. This means it is effectively two-dimensional. When we have an image that is in color, the filter is three-dimensional, i.e. 32x32x3. So the CNN operate over some volume. This means that the filter will also be three-dimensional, as well as the rest of the pooling layer.

Flattening layer

- The flatten layer prepares a vector to be passed into the fully connected layer by transforming a two-dimensional matrix into a vector that is then fed into the dense layers. For example,

$$\text{output} = \begin{pmatrix} 6 & 8 \\ 3 & 4 \end{pmatrix} \longrightarrow \text{flattening layer} \longrightarrow \text{vector} = \begin{pmatrix} 6 \\ 8 \\ 3 \\ 4 \end{pmatrix}$$

- This will now be the inputs for a neural network.

Dense layer

- A dense layer is another phrase for a fully connected layer. This layer is a fully connected neural network. Each neuron in the network receives an input from all the neurons present in the previous layer (hence why they are called dense). A dense layer provides features from all the combinations of features from the previous layer.

Output layer

- This layer outputs the prediction. The function used in classification problems is usually the Softmax function. Softmax makes output sum to one so we obtain probabilities.

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Categorical crossentropy loss

- Defined as

$$\text{Loss} = -\sum_{i=1}^{\text{output size}} y_i \ln(\hat{y}_i)$$

where y_i is the target value and $\hat{y}_i$ is the scalar value in the model output.

- For example

$$\text{target} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \text{prediction} = \begin{pmatrix} 0.5 \\ 0.3 \\ 0.2 \end{pmatrix} \rightarrow \text{corresponding} \rightarrow \begin{pmatrix} \text{dog} \\ \text{cat} \\ \text{monkey} \end{pmatrix}$$

- Loss total = loss for dog + loss for cat + loss for monkey

$$\text{Loss} = -1 * \ln(0.5) - 0 * \ln(0.3) - 0 * \ln(0.2)$$

- This type of loss function is to see how distinguishable two discrete probability distributions are from each other.
- More information can be found at:
  1. https://peltarion.com/knowledge-center/documentation/modeling-view/build-an-ai-model/loss-functions/categorical-crossentropy
  2. https://machinelearningmastery.com/cross-entropy-for-machine-learning/
  3. https://towardsdatascience.com/cross-entropy-for-classification-d98e7f974451

28.7. Demo: Image recognition with Convolutional Neural Networks

Copyright 2019 The TensorFlow Authors.

```
#@title Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
# https://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
```

Convolutional Neural Network (CNN)

View on TensorFlow.org Run in Google Colab View source on GitHub Download notebook

This tutorial demonstrates training a simple Convolutional Neural Network (CNN) to classify CIFAR images. Because this tutorial uses the Keras Sequential API, creating and training our model will take just a few lines of code.

Import TensorFlow

```
from __future__ import absolute_import, division, print_function, unicode_literals

try:
  # %tensorflow_version only exists in Colab.
  %tensorflow_version 2.x
except Exception:
  pass
import tensorflow as tf

from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
```

Download and prepare the CIFAR10 dataset

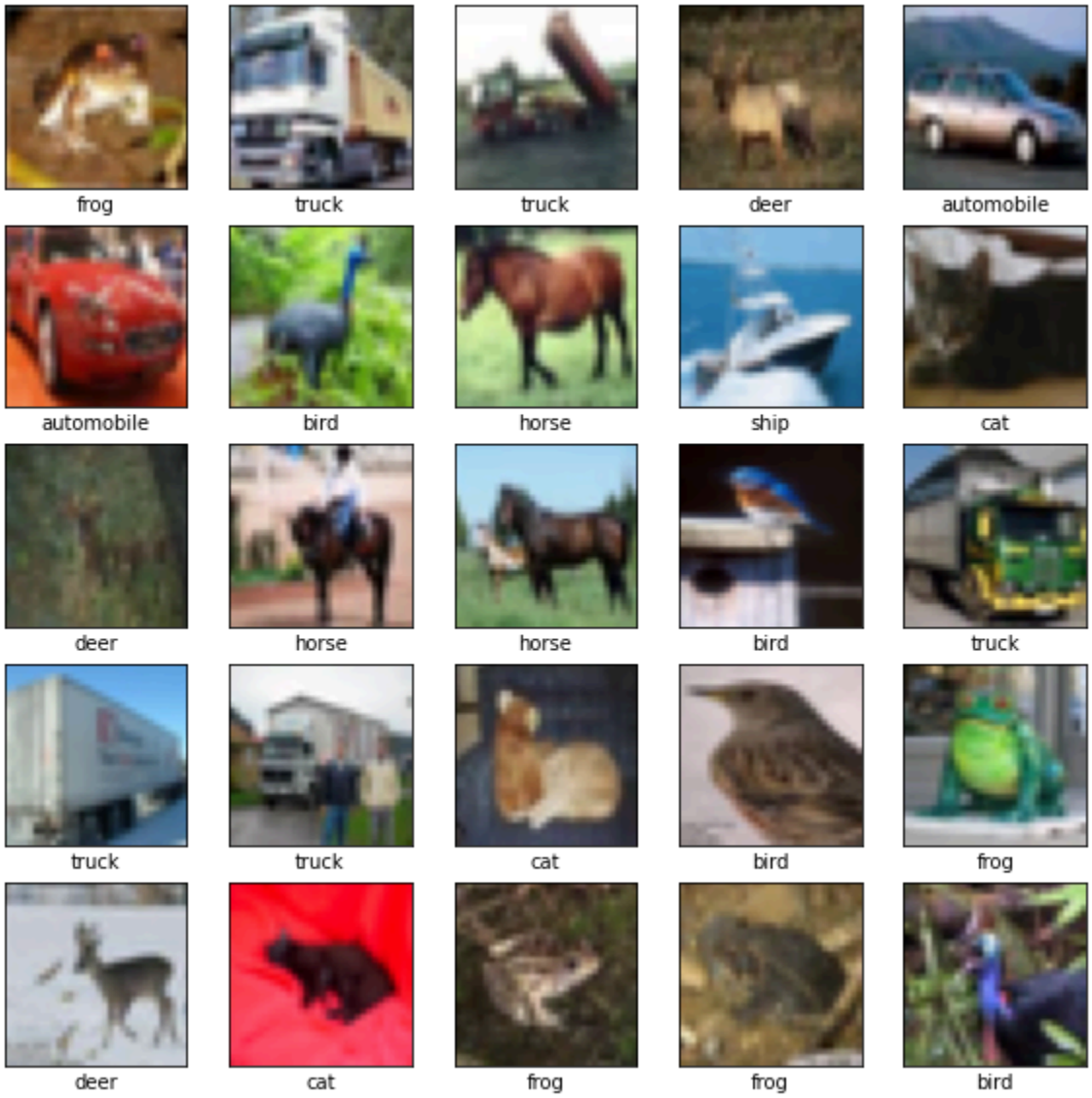
The CIFAR10 dataset contains 60,000 color images in 10 classes, with 6,000 images in each class. The dataset is divided into 50,000 training images and 10,000 testing images. The classes are mutually exclusive and there is no overlap between them.

```
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()

# Normalize pixel values to be between 0 and 1
train_images, test_images = train_images / 255.0, test_images / 255.0
```

Verify the data

To verify that the dataset looks correct, let's plot the first 25 images from the training set and display the class name below each image.

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i], cmap=plt.cm.binary)
    # The CIFAR labels happen to be arrays,
    # which is why you need the extra index
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```



Create the convolutional base

The 6 lines of code below define the convolutional base using a common pattern: a stack of Conv2D and MaxPooling2D layers.

As input, a CNN takes tensors of shape (image_height, image_width, color_channels), ignoring the batch size. If you are new to these dimensions, color_channels refers to (R,G,B). In this example, you will configure our CNN to process inputs of shape (32, 32, 3), which is the format of CIFAR images. You can do this by passing the argument `input_shape` to our first layer.

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

Let's display the architecture of our model so far.

```
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 30, 30, 32)        896
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 15, 15, 32)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 13, 13, 64)        18496
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 6, 6, 64)          0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 4, 4, 64)          36928
=================================================================
Total params: 56,320
Trainable params: 56,320
Non-trainable params: 0
_________________________________________________________________
```

Above, you can see that the output of every Conv2D and MaxPooling2D layer is a 3D tensor of shape (height, width, channels). The width and height dimensions tend to shrink as you go deeper in the network. The number of output channels for each Conv2D layer is controlled by the first argument (e.g., 32 or 64). Typically, as the width and height shrink, you can afford (computationally) to add more output channels in each Conv2D layer.

Add Dense layers on top

To complete our model, you will feed the last output tensor from the convolutional base (of shape (3, 3, 64)) into one or more Dense layers to perform classification. Dense layers take vectors as input (which are 1D), while the current output is a 3D tensor. First, you will flatten (or unroll) the 3D output to 1D, then add one or more Dense layers on top. CIFAR has 10 output classes, so you use a final Dense layer with 10 outputs and a softmax activation.

```
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
```

Here's the complete architecture of our model.

```
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 30, 30, 32)        896
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 15, 15, 32)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 13, 13, 64)        18496
```

```
max_pooling2d_1 (MaxPooling2 (None, 6, 6, 64)          0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 4, 4, 64)          36928
_________________________________________________________________
flatten (Flatten)            (None, 1024)              0
_________________________________________________________________
dense (Dense)                (None, 64)                65600
_________________________________________________________________
dense_1 (Dense)              (None, 10)                650
=================================================================
Total params: 122,570
Trainable params: 122,570
Non-trainable params: 0
_________________________________________________________________
```

As you can see, our (3, 3, 64) outputs were flattened into vectors of shape (576) before going through two Dense layers.

Compile and train the model

Note The training of the model can take some time (depending on the power of your computer).

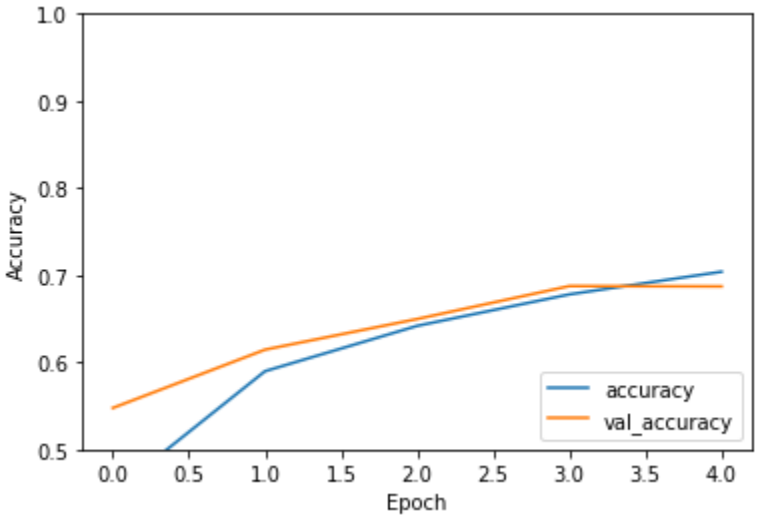
```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=5,
                    validation_data=(test_images, test_labels))
```

```
Epoch 1/5
1563/1563 [==============================] - 50s 32ms/step - loss: 1.5135 - accuracy: 0.4504 -
val_loss: 1.2607 - val_accuracy: 0.5476
Epoch 2/5
1563/1563 [==============================] - 60s 38ms/step - loss: 1.1570 - accuracy: 0.5898 -
val_loss: 1.0883 - val_accuracy: 0.6145
Epoch 3/5
1563/1563 [==============================] - 64s 41ms/step - loss: 1.0094 - accuracy: 0.6418 -
val_loss: 1.0038 - val_accuracy: 0.6497
Epoch 4/5
1563/1563 [==============================] - 66s 42ms/step - loss: 0.9154 - accuracy: 0.6780 -
val_loss: 0.9063 - val_accuracy: 0.6875
Epoch 5/5
1563/1563 [==============================] - 73s 46ms/step - loss: 0.8399 - accuracy: 0.7040 -
val_loss: 0.9099 - val_accuracy: 0.6871
```

Evaluate the model

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')

test_loss, test_acc = model.evaluate(test_images,  test_labels, verbose=2)
```

```
313/313 - 3s - loss: 0.9099 - accuracy: 0.6871
```




```
print(test_acc)
```

```
0.6870999932289124
```

Our simple CNN has achieved a test accuracy of over around 70%. Not bad for a few lines of code! For another CNN style, see an example using the Keras subclassing API and a `tf.GradientTape` here.

Making predictions

from https://www.tensorflow.org/tutorials/keras/classification

```
# Helper libraries
import numpy as np
import matplotlib.pyplot as plt
```

Make predictions

With the model trained, you can use it to make predictions about some images.

```
predictions = model.predict(test_images)
```

Here, the model has predicted the label for each image in the testing set. Let’s take a look at the first prediction:

```
predictions[0]
```

```
array([1.6858752e-03, 1.6222546e-04, 4.9888561e-03, 8.4230685e-01,
       7.5447030e-04, 2.0084806e-02, 1.1800880e-01, 1.0761167e-05,
       1.1810401e-02, 1.8694594e-04], dtype=float32)
```

A prediction is an array of 10 numbers. They represent the model’s “confidence” that the image corresponds to each of the 10 different articles of clothing. You can see which label has the highest confidence value:

```
np.argmax(predictions[0])
```

```
3
```

So, the model is most confident that this image is an ankle boot, or `class_names[9]`. Examining the test label shows that this classification is correct:

```
test_labels[0]
```

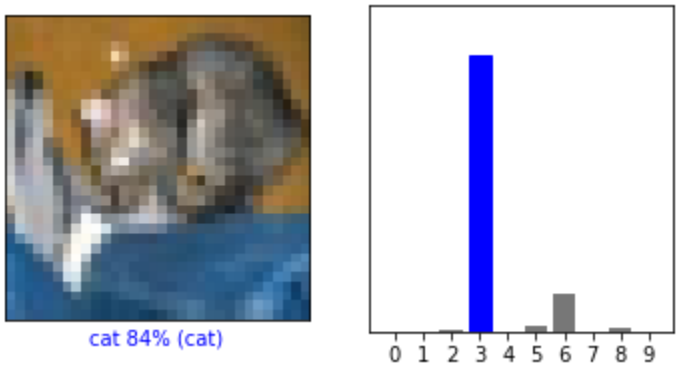
```
array([3], dtype=uint8)
```

Graph this to look at the full set of 10 class predictions.

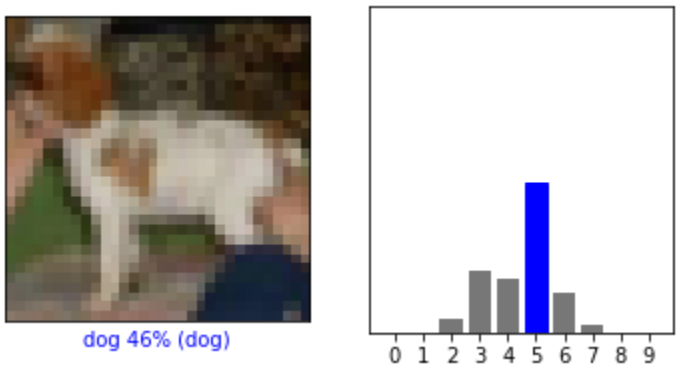
```
def plot_image(i, predictions_array, true_label, img):
  predictions_array, true_label, img = predictions_array, true_label[i], img[i]
  plt.grid(False)
  plt.xticks([])
  plt.yticks([])

  plt.imshow(img, cmap=plt.cm.binary)

  predicted_label = np.argmax(predictions_array)
  if predicted_label == true_label:
    color = 'blue'
  else:
    color = 'red'

  plt.xlabel("{} {:2.0f}% ({})".format(class_names[predicted_label],
                                100*np.max(predictions_array),
                                class_names[true_label]),
                                color=color)

def plot_value_array(i, predictions_array, true_label):
  predictions_array, true_label = predictions_array, true_label[i]
  plt.grid(False)
  plt.xticks(range(10))
  plt.yticks([])
  thisplot = plt.bar(range(10), predictions_array, color="#777777")
  plt.ylim([0, 1])
  predicted_label = np.argmax(predictions_array)

  thisplot[predicted_label].set_color('red')
  thisplot[true_label].set_color('blue')
```

Let's look at the 0th image, predictions, and prediction array. Correct prediction labels are blue and incorrect prediction labels are red. The number gives the percentage (out of 100) for the predicted label.

```
i = 0
plt.figure(figsize=(6,3))
plt.subplot(1,2,1)
plot_image(i, predictions[i], test_labels.reshape(-1), test_images)
plt.subplot(1,2,2)
plot_value_array(i, predictions[i],  test_labels.reshape(-1))
plt.show()
```

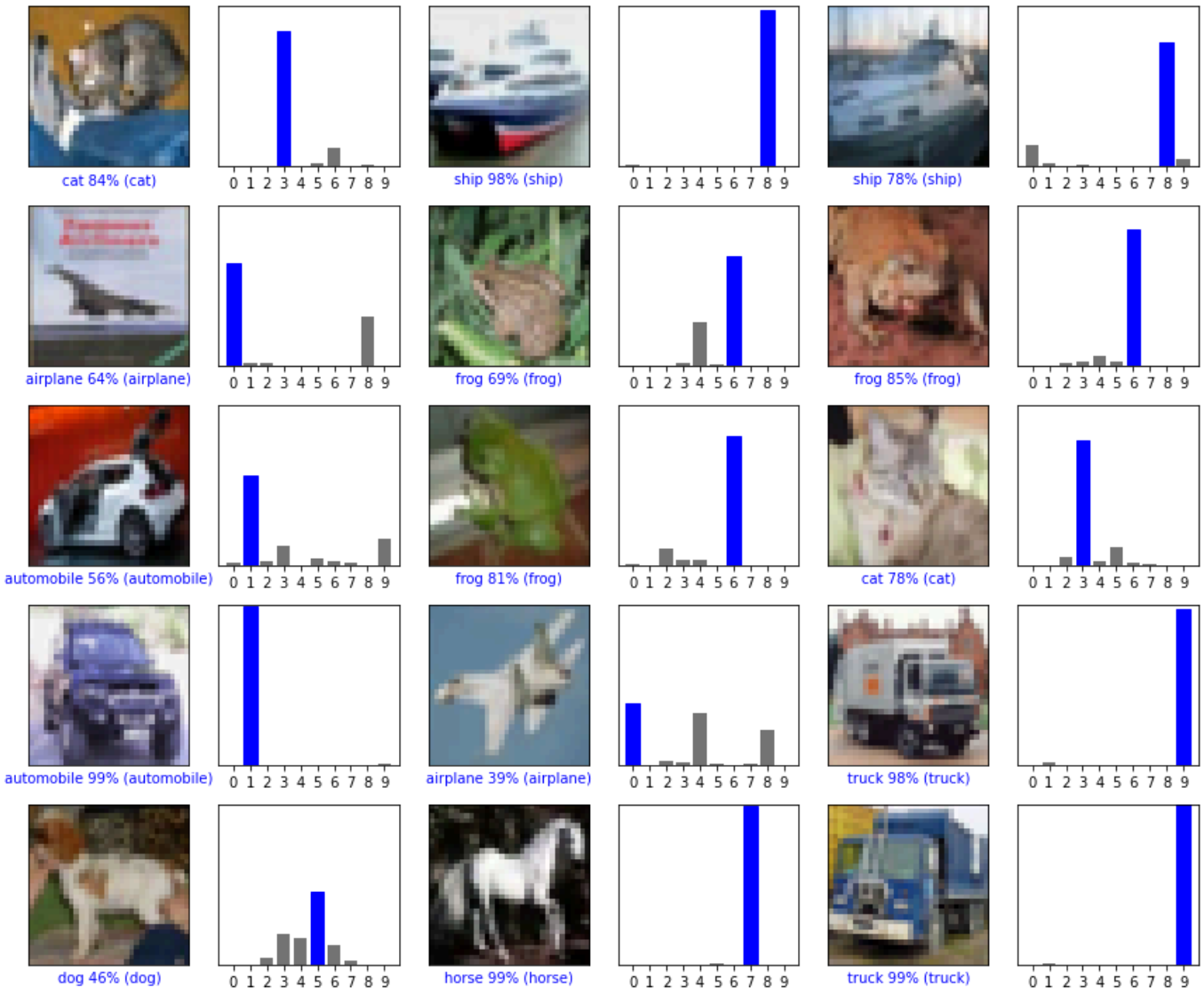


```
i = 12
plt.figure(figsize=(6,3))
plt.subplot(1,2,1)
plot_image(i, predictions[i], test_labels.reshape(-1), test_images)
plt.subplot(1,2,2)
plot_value_array(i, predictions[i],  test_labels.reshape(-1))
plt.show()
```



Let's plot several images with their predictions. Note that the model can be wrong even when very confident.

```
# Plot the first X test images, their predicted labels, and the true labels.
# Color correct predictions in blue and incorrect predictions in red.
num_rows = 5
num_cols = 3
num_images = num_rows*num_cols
plt.figure(figsize=(2*2*num_cols, 2*num_rows))
for i in range(num_images):
  plt.subplot(num_rows, 2*num_cols, 2*i+1)
  plot_image(i, predictions[i], test_labels.reshape(-1), test_images)
  plt.subplot(num_rows, 2*num_cols, 2*i+2)
  plot_value_array(i, predictions[i], test_labels.reshape(-1))
plt.tight_layout()
```



28.8. Demo: CNN in PyTorch

This notebook rewrites the original TensorFlow CNN for **CIFAR-10** using PyTorch. It keeps the same overall workflow: load data → define CNN → train → evaluate → predict → visualize.

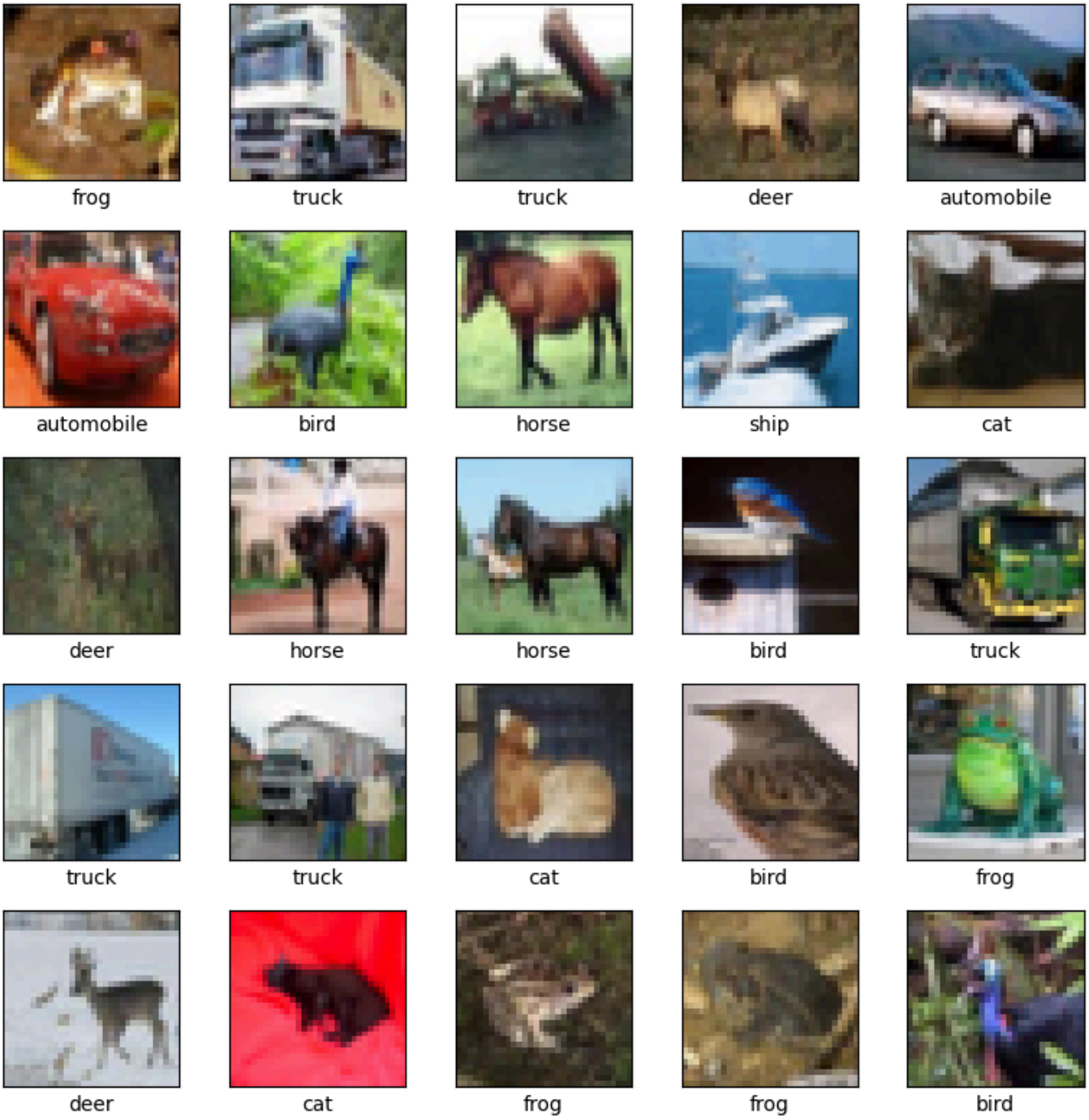
```
# Imports
import numpy as np
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
```

```
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
```

```
# Reproducibility & device
SEED = 1234
np.random.seed(SEED)
torch.manual_seed(SEED)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('PyTorch:', torch.__version__)
print('Device :', device)
```

```
PyTorch: 2.6.0
Device : cpu
```

```
# Data: CIFAR-10
# Keras normalized images to [0,1]. We'll match that here.
transform = transforms.Compose([
    transforms.ToTensor()
])

train_ds = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
test_ds  = datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)

batch_size = 128
train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,  num_workers=2)
test_loader  = DataLoader(test_ds,  batch_size=batch_size, shuffle=False, num_workers=2)

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

# Visualize a 5x5 grid as in the TF notebook
fig = plt.figure(figsize=(8,8))
for i in range(25):
    img, label = train_ds[i]
    npimg = img.permute(1,2,0).numpy()  # C,H,W -> H,W,C
    ax = plt.subplot(5,5,i+1)
    plt.xticks([]); plt.yticks([]); plt.grid(False)
    plt.imshow(npimg)
    ax.set_xlabel(class_names[label])
plt.tight_layout(); plt.show()
```


frog
truck
truck
deer
automobile
automobile
bird
horse
ship
cat
deer
horse
horse
bird
truck
truck
truck
cat
bird
frog
deer
cat
frog
frog
bird

```
# Model: small CNN analogous to the Keras example
# TF model had Conv2D(32,3x3) + MaxPool(2), Conv2D(64,3x3) + MaxPool(2), Conv2D(64,3x3),
Flatten, Dense(64), Dense(10)
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),  # 32x32 -> 16x16
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),  # 16x16 -> 8x8
            nn.Conv2d(64, 64, kernel_size=3, padding=1), nn.ReLU(),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64*8*8, 64), nn.ReLU(),
            nn.Linear(64, 10)  # logits
        )
    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

model = CNN().to(device)
print(model)
```

```
CNN(
  (features): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU()
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=4096, out_features=64, bias=True)
    (2): ReLU()
    (3): Linear(in_features=64, out_features=10, bias=True)
  )
)
```

```
# Training
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters())

EPOCHS = 10
history = {'loss': [], 'acc': []}

for epoch in range(1, EPOCHS+1):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * xb.size(0)
        pred = logits.argmax(1)
        correct += (pred == yb).sum().item()
        total += yb.size(0)
    epoch_loss = running_loss/total
    epoch_acc  = correct/total
    history['loss'].append(epoch_loss); history['acc'].append(epoch_acc)
    print(f'Epoch {epoch:2d}/{EPOCHS} - loss: {epoch_loss:.4f} - acc: {epoch_acc:.4f}')
```

```
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch  1/10 - loss: 1.6254 - acc: 0.4105
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch  2/10 - loss: 1.2615 - acc: 0.5495
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch  3/10 - loss: 1.0960 - acc: 0.6105
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch  4/10 - loss: 0.9883 - acc: 0.6495
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch  5/10 - loss: 0.8943 - acc: 0.6849
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
```

```
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch  6/10 - loss: 0.8234 - acc: 0.7078
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch  7/10 - loss: 0.7647 - acc: 0.7300
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch  8/10 - loss: 0.7199 - acc: 0.7481
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch  9/10 - loss: 0.6607 - acc: 0.7714
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Epoch 10/10 - loss: 0.6203 - acc: 0.7821
```

```
# Evaluate
model.eval()
correct, total, running_loss = 0, 0, 0.0
with torch.no_grad():
    for xb, yb in test_loader:
        xb, yb = xb.to(device), yb.to(device)
        logits = model(xb)
        loss = criterion(logits, yb)
        running_loss += loss.item() * xb.size(0)
        pred = logits.argmax(1)
        correct += (pred == yb).sum().item()
        total += yb.size(0)
test_loss = running_loss/total
test_acc = correct/total
print(f'\nTest loss: {test_loss:.4f}  Test acc: {test_acc:.4f}')
```

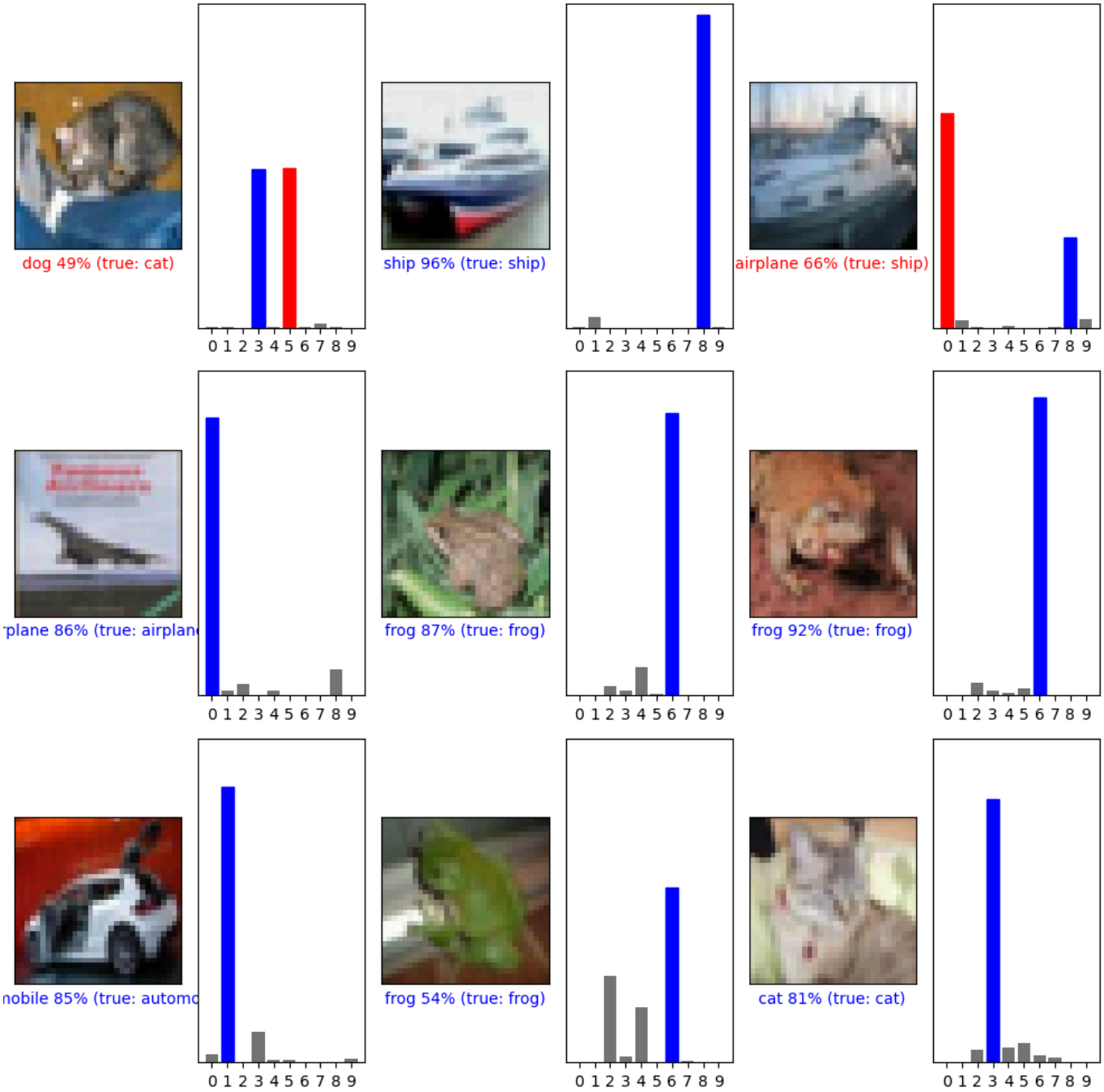
```
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.

Test loss: 0.8114  Test acc: 0.7196
```

```
# Predict probabilities for the test set (to mirror Keras 'predict')
softmax = nn.Softmax(dim=1)
model.eval()
all_probs = []
with torch.no_grad():
    for xb, _ in test_loader:
        xb = xb.to(device)
        probs = softmax(model(xb)).cpu().numpy()
        all_probs.append(probs)
import numpy as np
predictions = np.vstack(all_probs)
print('Predictions shape:', predictions.shape)
```

```
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Predictions shape: (10000, 10)
```

```
# Plot helpers adapted from the TF notebook
import numpy as np

def plot_image(i, predictions_array, true_label, img):
    true_label = int(true_label)
    plt.grid(False); plt.xticks([]); plt.yticks([])
    plt.imshow(img)
    predicted_label = int(np.argmax(predictions_array))
    color = 'blue' if predicted_label == true_label else 'red'
    plt.xlabel(f'{class_names[predicted_label]} {100*np.max(predictions_array):2.0f}% (true:
{class_names[true_label]})',
               color=color)

def plot_value_array(predictions_array, true_label):
    true_label = int(true_label)
    plt.grid(False); plt.xticks(range(10)); plt.yticks([])
    thisplot = plt.bar(range(10), predictions_array, color="#777777")
    plt.ylim([0, 1])
    predicted_label = int(np.argmax(predictions_array))
    thisplot[predicted_label].set_color('red')
    thisplot[true_label].set_color('blue')
```

```
# Display a few predictions as in the TF notebook
fig = plt.figure(figsize=(10,10))
for i in range(9):
    img, label = test_ds[i]
    npimg = img.permute(1,2,0).numpy()
    probs = predictions[i]
    ax1 = plt.subplot(3, 2*3, 2*i+1)
    plot_image(i, probs, label, npimg)
    ax2 = plt.subplot(3, 2*3, 2*i+2)
    plot_value_array(probs, label)
plt.tight_layout(); plt.show()
```



28.9. Demo: MNIST CNN in PyTorch (Binder-ready)

This notebook is a **convolutional neural network (CNN)** version of the earlier MLP demo, kept Binder-friendly and classroom-ready. It adds convolution/pooling layers for better accuracy.

```
# --- Setup & Reproducibility ---------------------------------------------------
import os, time, numpy as np
import torch

SEED = int(os.environ.get("SEED", 1234))
rng = np.random.default_rng(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("PyTorch:", torch.__version__)
print("Device :", device)
```

```
PyTorch: 2.6.0
Device : cpu
```

```
# --- Imports -------------------------------------------------------------------
import matplotlib.pyplot as plt
from torch import nn
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms
```

```
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
```

```
# --- Data: MNIST ---------------------------------------------------------------
# Normalize to standard MNIST mean/std for CNN training
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_full = datasets.MNIST(root="../assets", train=True, download=True, transform=transform)
test_ds    = datasets.MNIST(root="../assets", train=False, download=True, transform=transform)

n_val = 10000
n_train = len(train_full) - n_val
train_ds, val_ds = random_split(train_full, [n_train, n_val],
generator=torch.Generator().manual_seed(SEED))

batch_size = 128  # good default; Binder-safe with num_workers=0
train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,  num_workers=0,
pin_memory=False)
val_loader   = DataLoader(val_ds,   batch_size=batch_size, shuffle=False, num_workers=0,
pin_memory=False)
test_loader  = DataLoader(test_ds,  batch_size=batch_size, shuffle=False, num_workers=0,
pin_memory=False)

print(f"Train/Val/Test sizes: {len(train_ds)}/{len(val_ds)}/{len(test_ds)}")
```

```
Train/Val/Test sizes: 50000/10000/10000
```

```python
# --- Model: A small CNN -------------------------------------------------------
# Architecture: (1x28x28) -> Conv(32, 3x3) -> ReLU -> Conv(64, 3x3) -> ReLU -> MaxPool(2)
#               -> Dropout(0.25) -> Flatten -> Linear(64*12*12, 128) -> ReLU
#               -> Dropout(0.5) -> Linear(128, 10)
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),          # 28 -> 14
            nn.Dropout(0.25),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64*14*14, 128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(128, 10)        # raw logits
        )
    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

model = CNN().to(device)
print(model)
print("Total parameters:", sum(p.numel() for p in model.parameters()))
```

```
CNN(
  (features): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (3): ReLU(inplace=True)
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (5): Dropout(p=0.25, inplace=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=12544, out_features=128, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=128, out_features=10, bias=True)
  )
)
Total parameters: 1625866
```

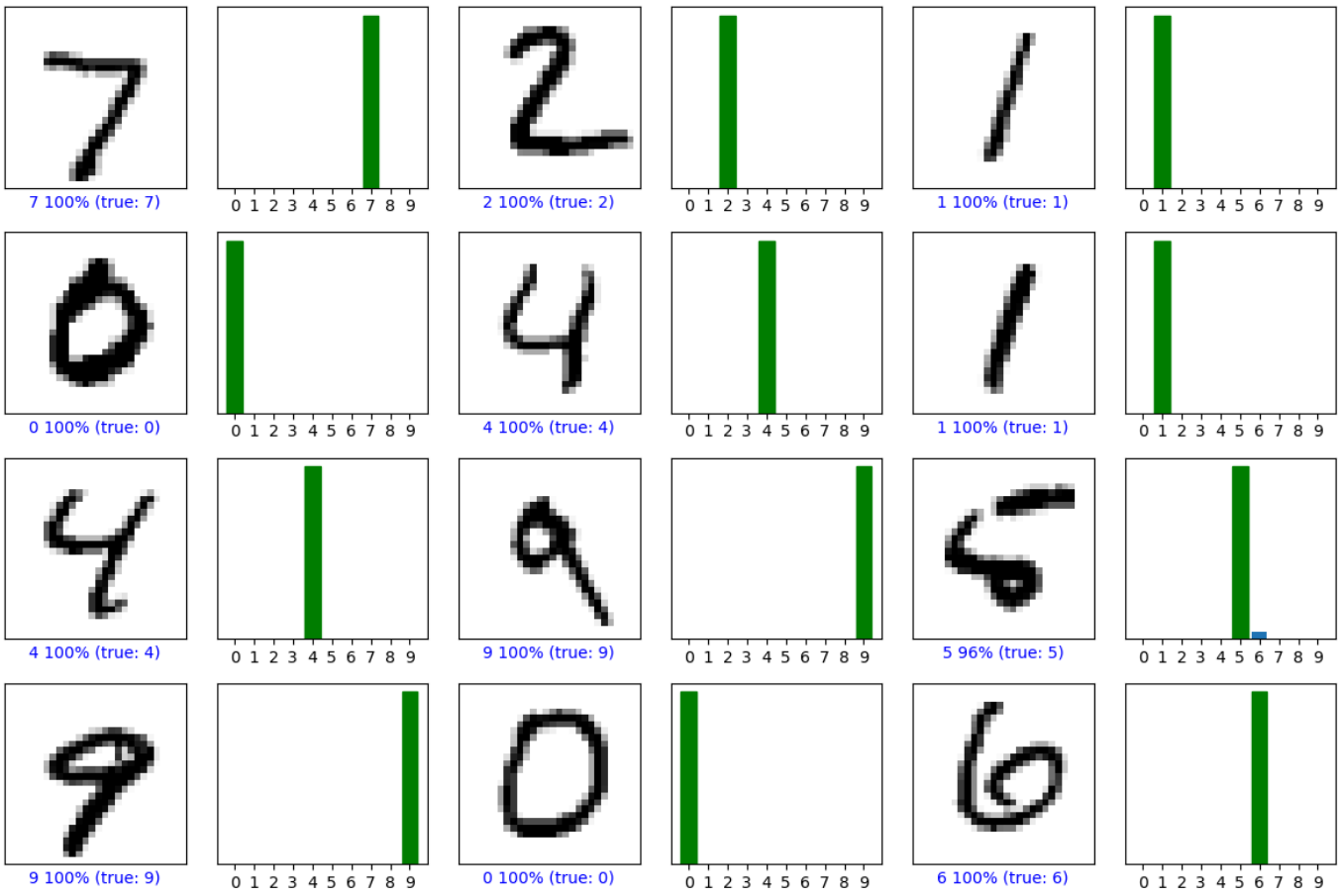
```
# --- Training Loop ------------------------------------------------------------
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

def run_epoch(loader, train=True):
    if train:
        model.train()
    else:
        model.eval()
    total, correct, running_loss = 0, 0, 0.0
    t0 = time.time()
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        if train:
            optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        if train:
            loss.backward()
            optimizer.step()
        running_loss += loss.item() * xb.size(0)
        pred = logits.argmax(1)
        correct += (pred == yb).sum().item()
        total += yb.size(0)
    elapsed = time.time() - t0
    return running_loss/total, correct/total, elapsed

EPOCHS = 8  # CNN learns faster; 8 epochs is plenty for a demo
history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}
for epoch in range(1, EPOCHS+1):
    tr_loss, tr_acc, tr_t = run_epoch(train_loader, train=True)
    va_loss, va_acc, va_t = run_epoch(val_loader,   train=False)
    history["train_loss"].append(tr_loss); history["train_acc"].append(tr_acc)
    history["val_loss"].append(va_loss);   history["val_acc"].append(va_acc)
    print(f"Epoch {epoch:2d}/{EPOCHS} | "
          f"train loss {tr_loss:.4f} acc {tr_acc:.4f} ({tr_t:.1f}s) | "
          f"val loss {va_loss:.4f} acc {va_acc:.4f} ({va_t:.1f}s)")
```

```
Epoch  1/8 | train loss 0.2532 acc 0.9236 (91.1s) | val loss 0.0724 acc 0.9782 (3.4s)
Epoch  2/8 | train loss 0.0909 acc 0.9734 (55.5s) | val loss 0.0517 acc 0.9854 (3.3s)
Epoch  3/8 | train loss 0.0698 acc 0.9793 (52.3s) | val loss 0.0475 acc 0.9872 (3.2s)
Epoch  4/8 | train loss 0.0556 acc 0.9827 (51.7s) | val loss 0.0400 acc 0.9895 (3.3s)
Epoch  5/8 | train loss 0.0494 acc 0.9847 (52.7s) | val loss 0.0402 acc 0.9890 (3.2s)
Epoch  6/8 | train loss 0.0416 acc 0.9869 (51.6s) | val loss 0.0395 acc 0.9893 (3.2s)
Epoch  7/8 | train loss 0.0383 acc 0.9877 (52.5s) | val loss 0.0349 acc 0.9912 (3.7s)
Epoch  8/8 | train loss 0.0321 acc 0.9896 (53.8s) | val loss 0.0437 acc 0.9890 (3.2s)
```

```python
# --- Test Set Evaluation ----------------------------------------------------
model.eval()
total, correct, running_loss = 0, 0, 0.0
with torch.no_grad():
    for xb, yb in test_loader:
        xb, yb = xb.to(device), yb.to(device)
        logits = model(xb)
        loss = criterion(logits, yb)
        running_loss += loss.item() * xb.size(0)
        pred = logits.argmax(1)
        correct += (pred == yb).sum().item()
        total += yb.size(0)
test_loss = running_loss/total
test_acc = correct/total
print(f"\nTest loss {test_loss:.4f} | Test accuracy {test_acc:.4f}")
```

```
Test loss 0.0322 | Test accuracy 0.9901
```

```
# --- Prediction & Visualization ----------------------------------------------
import numpy as np
softmax = nn.Softmax(dim=1)

# Inverse normalization for display
inv_mean, inv_std = 0.1307, 0.3081
def denorm(x):
    return (x * inv_std + inv_mean).clamp(0, 1)

def predict_probs(x_batch):
    model.eval()
    with torch.no_grad():
        return softmax(model(x_batch)).cpu().numpy()

def plot_image(predictions_array, true_label, img):
    import matplotlib.pyplot as plt
    predictions_array, true_label, img = predictions_array, int(true_label), img
    plt.grid(False); plt.xticks([]); plt.yticks([])
    plt.imshow(img, cmap=plt.cm.binary)
    predicted_label = int(np.argmax(predictions_array))
    color = 'blue' if predicted_label == true_label else 'red'
    plt.xlabel(f"{predicted_label} {100*np.max(predictions_array):2.0f}% (true: {true_label})",
color=color)

def plot_value_array(predictions_array, true_label):
    import matplotlib.pyplot as plt
    predictions_array, true_label = predictions_array, int(true_label)
    plt.grid(False); plt.xticks(range(10)); plt.yticks([])
    bars = plt.bar(range(10), predictions_array)
    bars[np.argmax(predictions_array)].set_color('red')
    bars[true_label].set_color('green')

# Gallery
num_rows, num_cols = 4, 3
num_images = num_rows * num_cols
fig = plt.figure(figsize=(2*2*num_cols, 2*num_rows))

imgs, labels = [], []
for i in range(num_images):
    img, label = test_ds[i]
    imgs.append(img)
    labels.append(label)
xb = torch.stack(imgs).to(device)
probs = predict_probs(xb)

for i in range(num_images):
    img = denorm(imgs[i]).squeeze().cpu().numpy()  # de-normalize for nicer display
    label = labels[i]
    ax1 = plt.subplot(num_rows, 2*num_cols, 2*i+1)
    plot_image(probs[i], label, img)
    ax2 = plt.subplot(num_rows, 2*num_cols, 2*i+2)
    plot_value_array(probs[i], label)
plt.tight_layout(); plt.show()
```



Notes

- The CNN uses **two conv layers (32→64)** with 3×3 kernels, **ReLU**, a **2×2 max-pool**, then two FC layers.
- We normalize MNIST to `(mean=0.1307, std=0.3081)` which helps training; images are **denormalized for display**.
- `EPOCHS=8` is chosen to keep Binder runs snappy while still achieving strong accuracy (>99% is typical).

29. Problems for Part IV

In this chapter we collect various problems.

30. Overview of Part V: Probabilistic machine learning and emulation

In this part we focus on machine learning that we define from a Bayesian perspective as methods that provide probability distributions over functions trained on data (defined broadly). We can think conceptually of the discussion of machine learning in Part IV as applying to *point estimates* while now we will consider *distributions*.

The chapters are:

- Bayesian neural networks (or BNNs) build on a probabilistic interpretation of ANNs.
- Overview of Gaussian processes (or GPs) are distributions over functions.
- ANNs in the large-width limit illustrates how a controlled expansion around neural networks in the large-width limit can be formalized (the infinite-width limit of an ANN is a GP!).
- Bayesian optimization is a strategy to optimize computationally expensive functions using a surrogate (often a GP) for the function of interest.
- Dimensionality reduction and emulators gives an overview of singular value decompositions (SVD), which is a key component of dimensional reduction methods such as principal component analysis (PCA). It also covers emulators, which are fast & accurate computer models of expensive theoretical simulations. Bayesian inference often requires a large number of samples of the simulations, e.g., for parameter estimation, which could be prohibitive unless we can use emulators to generate the samples. The particular focus is on reduced basis model (RBM) emulators.
- Problems for Part V.

31. Bayesian neural networks

The introduction part of this lecture is inspired by the chapter *"Learning as Inference"* in the excellent book Information Theory, Inference, and Learning Algorithms by David MacKay [Mac03].

Some python libraries that are relevant for Bayesian Neural Networks (and part of the general trend towards Probabilistic Programming in Machine Learning) are:

- PyTorch, which is what we use in this course. The demonstration at the end of this lecture builds a Bayesian neural network with nothing but `torch.nn`.
- Pyro, a probabilistic programming language built on PyTorch.
- Tensorflow Probability
- PyMC, the successor to PyMC3.

31.1. Probabilistic neural network

We use y to denote the output from the neural network. This output could, in general, be a vector. The neural network can therefore be seen as a non-linear mapping $y(\boldsymbol{x};\boldsymbol{w})$: $x \in \mathbb{R}^p \rightarrow y \in \mathbb{R}^m$. The training of the network implies feeding it with training data and finding the sets of weights and biases that minimizes a loss function that has been selected for that particular problem. We will now be interested in probabilistic machine-learning models for which predictions are represented by probability distributions.

A Bayesian binary classifier

Consider first a classification problem where the single output y of the final network layer is a real number $\in [0,1]$ that indicates the (discrete) probability for input $\boldsymbol{x}$ belonging to either class $t=1$ or $t=0$:

$$\begin{aligned} p_{t=1} &\equiv p(t=1|\boldsymbol{w},\boldsymbol{x}) = y \\ p_{t=0} &\equiv p(t=0|\boldsymbol{w},\boldsymbol{x}) = 1-y, \end{aligned} \tag{31.1}$$

The output y is in fact the *parameter* of a Bernoulli distribution over t. For a classification task, an optimized-weights network with a single output therefore delivers a complete probability distribution for the datum. The distribution width, $y(1-y)$, is already built in. For a regression problem, a probabilistic model might need to output more than one parameter to specify probability distributions for target data. We return to this in Bayesian neural networks in practice.

A simple binary classifier can be trained by minimizing the loss function

$$C_W(\boldsymbol{w}) = C(\boldsymbol{w}) + \alpha E_W(\boldsymbol{w}), \tag{31.2}$$

made up of an error function

$$C(\boldsymbol{w}) = -\sum_n \left[t^{(n)} \log(y(\boldsymbol{x}^{(n)}, \boldsymbol{w})) + (1 - t^{(n)}) \log(1 - y(\boldsymbol{x}^{(n)}, \boldsymbol{w})) \right], \tag{31.3}$$

where $t^{(n)}$ is the training data, and the regularizer

$$E_W(\boldsymbol{w}) = \frac{1}{2} \sum_i w_i^2, \tag{31.4}$$

is designed to avoid overfitting. The error function can be interpreted as minus the log likelihood, with the likelihood

$$p(\mathcal{D}|\boldsymbol{w}) \propto \exp\left[-C(\boldsymbol{w})\right]. \tag{31.5}$$

Similarly the regularizer can be interpreted in terms of a log prior probability distribution over the parameters. With the quadratic E_W given above, the corresponding prior distribution is a Gaussian with variance $\sigma_W^2 = 1/\alpha$ and $1/Z_W = (\alpha/2\pi)^{K/2}$, where K is the number of parameters in w.

$$p(\boldsymbol{w}|\alpha) = \frac{1}{Z_W(\alpha)} \exp\left[-\alpha E_W\right]. \tag{31.6}$$

The objective function $C_W(w)$ then corresponds to a (unnormalized) negative log-posterior for the parameters $\boldsymbol{w}$ given the data

$$p(\boldsymbol{w}|\mathcal{D}, \alpha) = \frac{p(D|\boldsymbol{w}) p(\boldsymbol{w}|\alpha)}{p(\mathcal{D}|\alpha)} = \frac{1}{Z_M} \exp[-C_W(\boldsymbol{w})]. \tag{31.7}$$

We show the evolution of this probability distribution for a sequence of an increasing number of training data (N) in Fig. 31.1. The targets are either 0 or 1, as indicated by red and blue markers. The network parameters $\boldsymbol{w}$ that are found by minimizing $C_W(\boldsymbol{w})$ can be interpreted as the most probable parameter vector $\boldsymbol{w}^*$.

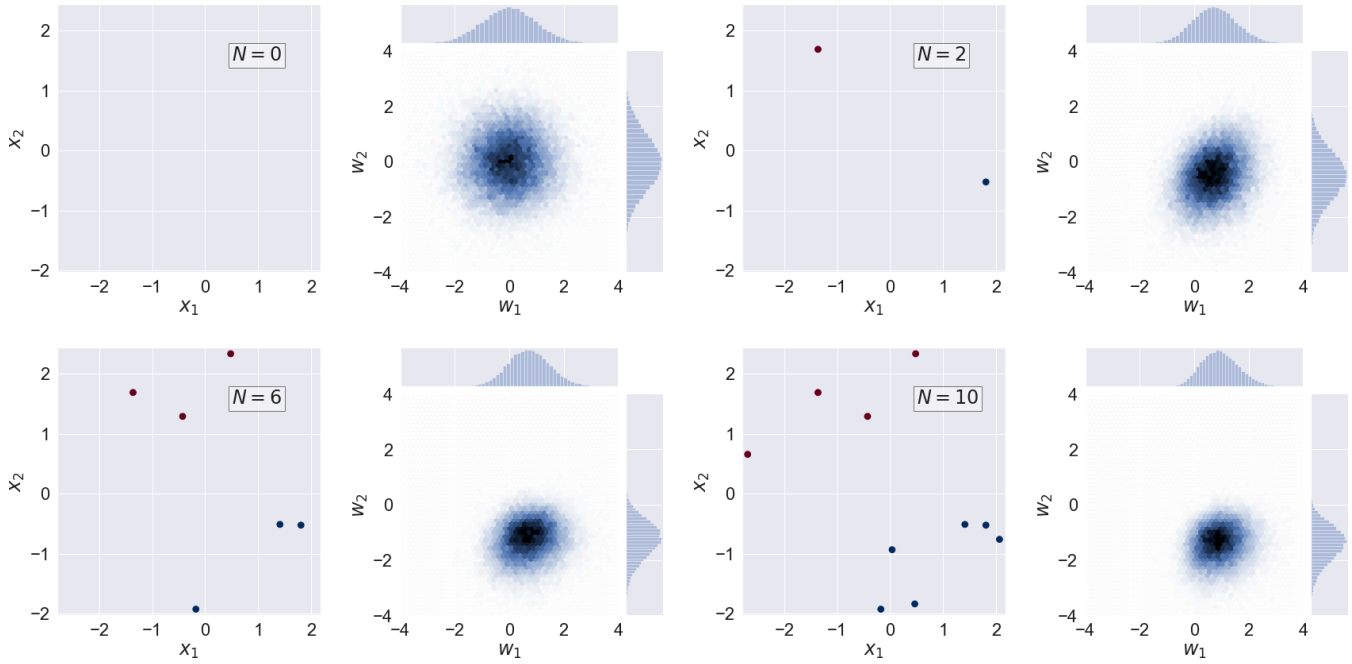


Fig. 31.1 Scatter plot of training data and the corresponding bivariate posterior PDF for the neuron weights $p(w_1, w_2|\mathcal{D}, \alpha)$ (i.e. marginalized over the bias w_0) for a sequence of $N = 0, 2, 6, 10$ training data.

Types of uncertainty

In the following, we will use the full Bayesian approach and consider the information that is contained in the actual probability distribution. In fact, there are different types of uncertainty that will be addressed:

Epistemic uncertainties

correspond to what we do not know about the model, such as the “true” values of model parameters and, more drastically, whether the model class is able to represent the truth at all. The defining property is that this uncertainty **is reduced by more data of the same kind**. More training data narrows the posterior for the weights.

Aleatoric uncertainties

appear as a result of inherent noise in the data. They enter through the likelihood function and are therefore part of the Bayesian treatment from the start. The defining property is that this uncertainty **cannot be reduced with more data of the same quality**. More measurements pin down the size of the scatter more precisely, but the scatter itself remains. Aleatoric is derived from the Latin *alea* or dice, referring to a game of chance.

The machine-learning literature works with this two-way split of uncertainty in which model discrepancy, that we have discussed extensively in the context of scientific modeling, has no slot of its own. Conceptually it belongs with the epistemic uncertainties, since it is a statement about the model rather than about the data. Operationally, however, it is almost always absorbed into the aleatoric term. A network that reports a predictive uncertainty width will fit that width to whatever residual scatter remains. Such a machine-learning model has no way of telling whether the scatter comes from noise in the data or from its own inability to follow them. Obviously, it is worth knowing which type of uncertainty you are looking at, because the remedies differ — more data, a better measurement instrument, or a better model.

	what the uncertainty is about	reduced by more data of the same kind?	usual label in the ML literature
Parameter uncertainty	our incomplete knowledge of the parameters $\boldsymbol{w}$	yes	**epistemic**
Model discrepancy	the model class not containing the truth	no — only a better model helps	*see above*
Observational noise	randomness in the measurement process itself	no	**aleatoric**

This is also why the physicist's *statistical* and *systematic* errors do not map onto the two labels. A statistical experimental error is aleatoric. A systematic experimental error, such as a calibration offset, is not reduced by repeating the measurement, but it *is* reducible by a better calibration. It is epistemic in the sense used here. Model discrepancy, meanwhile, is systematic in origin yet turns up in the aleatoric slot. The aleatoric/epistemic distinction is a statement about what more data of the same kind can fix; it is not a relabelling of statistical and systematic.

31.2. Bayesian inference for network weights

A Bayesian neural network can be viewed as probabilistic model in which we want to infer $p(y|\boldsymbol{x}, \mathcal{D})$ where $\mathcal{D} = \left\{\boldsymbol{x}^{(i)}, y^{(i)}\right\}$ is a given training dataset.

Maximum likelihood and MAP estimates

We construct the likelihood function $p(\mathcal{D}|\boldsymbol{w}) = \prod_i p(y^{(i)}|\boldsymbol{x}^{(i)}, \boldsymbol{w})$ which is a function of parameters $\boldsymbol{w}$. Maximizing the likelihood function gives the maximum likelihood estimate (MLE) of $\boldsymbol{w}$. The usual optimization objective during training is the negative log likelihood. For a categorical distribution this is the *cross entropy* error function, for a Gaussian distribution this is proportional to the *sum of squares* error function. MLE can lead to severe overfitting though.

Multiplying the likelihood with a prior distribution $p(\boldsymbol{w})$ is, by Bayes theorem, proportional to the posterior distribution $p(\boldsymbol{w}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w})$. Maximizing $p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w})$ gives the maximum a posteriori (MAP) estimate of $\boldsymbol{w}$. Computing the MAP estimate has a regularizing effect and can prevent overfitting. The optimization objectives here are the same as for MLE plus a regularization term coming from the log prior.

The posterior predictive distribution

Both MLE and MAP give point estimates of parameters. If we instead had a full posterior distribution over parameters we could make predictions that take weight uncertainty into account. This is covered by the posterior predictive distribution $p(y|\boldsymbol{x}, \mathcal{D}) = \int p(y|\boldsymbol{x}, \boldsymbol{w})p(\boldsymbol{w}|\mathcal{D})d\boldsymbol{w}$ in which the parameters have been marginalized out. This is equivalent to averaging predictions from an ensemble of neural networks weighted by the posterior probabilities of their parameters $\boldsymbol{w}$.

Returning to the binary classification problem, $y^{(n+1)}$ corresponds to the probability $p_{t^{(n+1)}=1}$ and a Bayesian prediction of a new datum $y^{(n+1)}$ will correspond to a pdf and involves *marginalizing* over the weight and bias parameters

$$p(y^{(n+1)}|x^{(n+1)}, D, \alpha) = \int d\boldsymbol{w}\, p(y^{(n+1)}|x^{(n+1)}, w, \alpha) p(w|D, \alpha), \tag{31.8}$$

where we have also included the weight decay hyperparameter α from the prior (regularizer). Marginalization could, of course, also be performed over this parameter.

Example: the Bayesian binary classifier

We show an example of such inference, comparing the point estimate $y(x; w^*, \alpha)$ and the Bayesian approach, in Fig. 31.2.

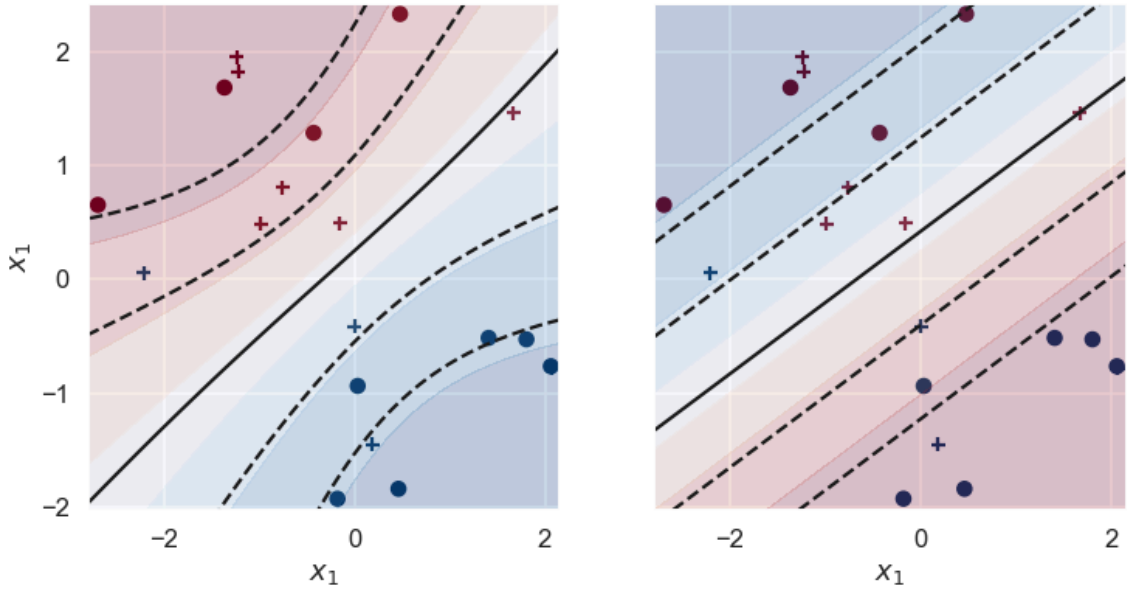


Fig. 31.2 The predictions for a Bayesian (left panel) and regular (right panel) binary classifier that has been learning from ten training data (circles) with a weight decay $\alpha = 1.0$. The decision boundary ($y = 0.5$, i.e. the preactivation $z = 0$) is shown together with the levels 0.12,0.27,0.73,0.88 (corresponding to the preactivation $z = \pm 1, \pm 2$). Test data is shown as plus symbols.

The Bayesian classifier is based on sampling a very large ensemble of single neurons with different parameters. The distribution of these samples will be proportional to the posterior pdf for the parameters. The decision boundary shown in the figure is obtained as the mean of the predictions of the sampled neurons evaluated on a grid. It is clear that the Bayesian classifier is more uncertain about its predictions in the lower left and upper right corners, where there is little training data.

This becomes even more clear when we plot the standard deviation of the predictions of the Bayesian classifier in Fig. 31.3.

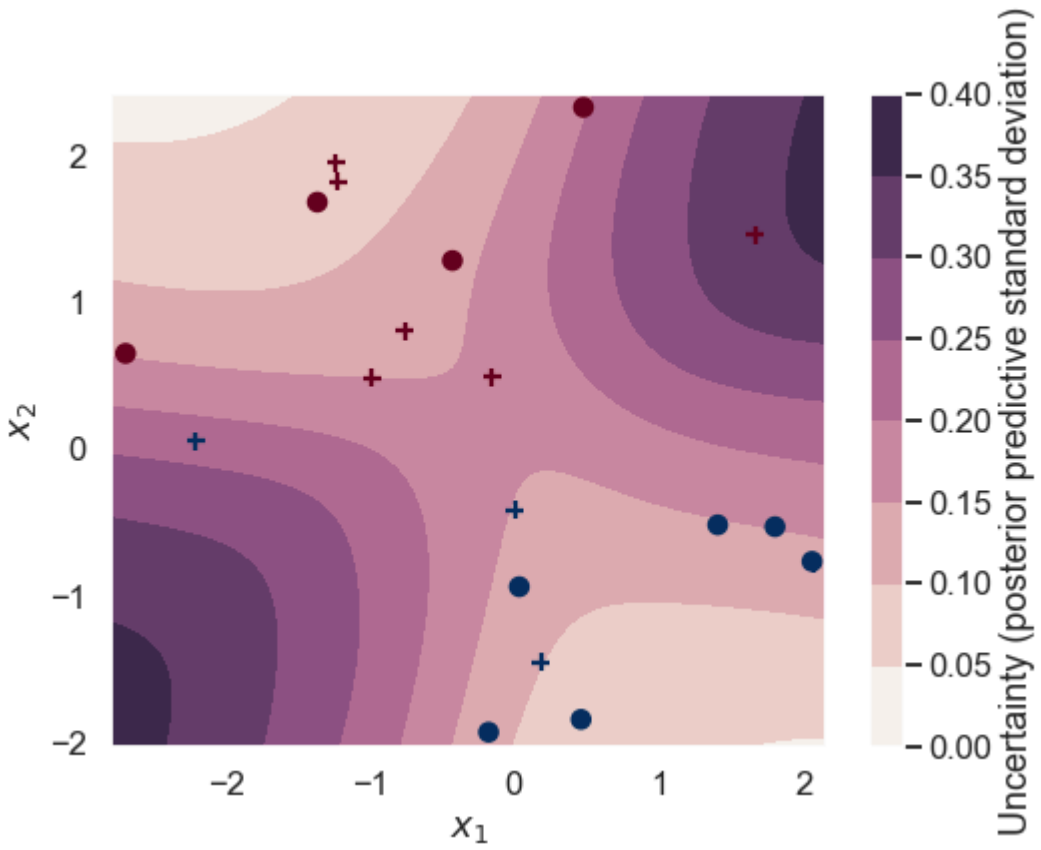


Fig. 31.3 The standard deviation of the class label predictions for a Bayesian binary classifier.

The predictions are rather certain along a diagonal line (close to the training data). Note that the interpretation of the prediction in the center of Fig. 31.3 (near $x_1, x_2 = 0, 0$) is the following: The Bayesian binary classifier predicts a probability of ~ 0.5 for this point in the input parameter space to belong to class 1 (i.e. the decision is very uncertain). The Bayesian classifier is also very certain about this uncertainty (the standard deviation is small).

In contrast, predictions for points in the upper left or lower right corners are very certain about the class label (and there is little uncertainty about this certainty).

This is the distinction of Probabilistic neural network made concrete. The value $y \approx 0.5$ at the centre is **aleatoric**: even with the weights known exactly, the label of a point sitting on the decision boundary is a coin toss, and no amount of further training data would change that. The *spread* of y across the sampled weights is **epistemic**: it measures how much the classifier's opinion depends on which weights we happen to draw, and it shrinks wherever the training data pin the boundary down. The two figures show that the two are independent of each other. Near the origin the aleatoric uncertainty is at its largest while the epistemic uncertainty is small; in the upper left and lower right corners it is the other way round. That is precisely why they are worth reporting separately: they answer different questions, and only one of them would be reduced by collecting more data.

Note also that the aleatoric part required no extra machinery here. The output y is the parameter of a Bernoulli distribution, so it carries its own width. In a Bayesian neural network approach to a regression problem this is no longer the case. The width has to be supplied explicitly, either as a fixed number or as a further output of the network.

31.3. Bayesian neural networks in practice

How can we compute the marginalization integral for neural networks with thousands of parameters?

In short, there are three different approaches:

1. **Sampling methods**, e.g. MCMC (this approach would be exact as the number of samples $\rightarrow \infty$);
2. **Deterministic approximate methods**, for example using Gaussian approximations with the Laplace method;
3. **Variational methods**.

The first two have been discussed previously in these notes in the general context of Bayesian inference. In the following, we will focus on the variational methods.

Variational inference for Bayesian neural networks

Bayesian neural networks differ from plain neural networks in that their weights are assigned a probability distribution instead of a single value or point estimate. These probability distributions describe the uncertainty in weights and can be used to estimate uncertainty in predictions.

Unfortunately, full inference of the weight posterior $p(\boldsymbol{w}|\mathcal{D})$ for neural networks is usually intractable due to the large dimensionality. Instead we can attempt to approximate the true posterior with a proxy distribution $q(\boldsymbol{w}|\boldsymbol{\theta})$ with variational parameters that we want to estimate.

Training a Bayesian neural network via variational inference implies learning the parameters of the proxy distribution rather than learning the real posterior.

This can be done by minimizing the Kullback-Leibler divergence between $q(\boldsymbol{w}|\boldsymbol{\theta})$ and the true posterior $p(\boldsymbol{w}|\mathcal{D})$ w.r.t. $\boldsymbol{\theta}$.

The specific goal is then to replace $p(\boldsymbol{w}|\mathcal{D})$, which we don't know, with the known proxy distribution $q(\boldsymbol{w}|\boldsymbol{\theta}^*)$, where $\boldsymbol{\theta}^*$ is the optimal set of variational parameters.

The Kullback-Leibler divergence

The KL divergence is a numeric measure of the difference between two distributions. For two probability distributions $q(\boldsymbol{w})$ and $p(\boldsymbol{w})$, the KL divergence in a continuous case,

$$D_{\mathrm{KL}}(q||p) = \int d\boldsymbol{w}\, q(\boldsymbol{w}) \log \frac{q(\boldsymbol{w})}{p(\boldsymbol{w})} \equiv \mathbb{E}_q \left[\log\, q(\boldsymbol{w}) - \log\, p(\boldsymbol{w})\right] \tag{31.9}$$

As we can see, the KL divergence corresponds to the expected log difference between two distributions with respect to distribution q. It is a non-negative quantity and it is equal to zero only when the two distributions are identical.

Intuitively there are three scenarios:

- if both q and p are high at the same positions, then we are succeeding;
- if q is high where p is low, we pay a price;
- if q is low we don't care about p (because of the expectation).

The divergence measure is not symmetric, i.e., $D_{\mathrm{KL}}(p||q) \neq D_{\mathrm{KL}}(q||p)$. In fact, it is possibly more natural to reverse the arguments and compute $D_{\mathrm{KL}}(p||q)$. However, we choose $\mathrm{KL}(q||p)$ so that we can take expectations with respect to the known $q(\boldsymbol{w})$ distribution. In addition, the minimization of this KL divergence will encourage the fit to concentrate on plausible parameters since

$$\begin{aligned} D_{\mathrm{KL}}(q||p) &= \int d\boldsymbol{w}\, q(\boldsymbol{w}|\boldsymbol{\theta}) \log \frac{q(\boldsymbol{w}|\boldsymbol{\theta})}{p(\boldsymbol{w}|\mathcal{D})} \\ &= -\int d\boldsymbol{w}\, q(\boldsymbol{w}|\boldsymbol{\theta}) \log\, p(\boldsymbol{w}|\mathcal{D}) + \int d\boldsymbol{w}\, q(\boldsymbol{w}|\boldsymbol{\theta}) \log\, q(\boldsymbol{w}|\boldsymbol{\theta}). \end{aligned} \tag{31.10}$$

To minimize the first term we have to avoid putting probability mass into regions of implausible parameters. To minimize the second term we have to maximize the entropy of the variational distribution q as this term corresponds to its negative entropy.

The evidence lower bound

Let us rewrite the posterior pdf $p(\boldsymbol{w}|\mathcal{D})$ using Bayes theorem

$$\begin{aligned} D_{\mathrm{KL}}(q||p) &= \int d\boldsymbol{w}\, q(\boldsymbol{w}|\boldsymbol{\theta}) \left[\log\, q(\boldsymbol{w}|\boldsymbol{\theta}) - p(\mathcal{D}|\boldsymbol{w}) - p(\boldsymbol{w}) + p(\mathcal{D})\right] \\ &= \mathbb{E}_q\left[\log\, q(\boldsymbol{w}|\boldsymbol{\theta})\right] - \mathbb{E}_q\left[\log\, p(\mathcal{D}|\boldsymbol{w})\right] - \mathbb{E}_q\left[\log\, p(\boldsymbol{w})\right] + \log\, p(\mathcal{D}). \end{aligned} \tag{31.11}$$

Note that the logarithm of the last term has no dependence on $\boldsymbol{w}$ and the integration of q will just give one since it should be a properly normalized pdf. This term is then the log marginal likelihood (or model evidence). Furthermore, since the KL divergence on the left hand side is bounded from below by zero we get the **Evidence Lower Bound** (ELBO)

$$\log\, p(\mathcal{D}) \geq -\mathbb{E}_q\left[\log\, q(\boldsymbol{w}|\boldsymbol{\theta})\right] + \mathbb{E}_q\left[\log\, p(\mathcal{D}|\boldsymbol{w})\right] + \mathbb{E}_q\left[\log\, p(\boldsymbol{w})\right] \equiv J_{\mathrm{ELBO}}(\boldsymbol{\theta}) \tag{31.12}$$

Variational inference was originally inspired by work in statistical physics, and with that analogy, $-J_{\mathrm{ELBO}}(\boldsymbol{\theta})$ is also called the **variational free energy** and sometimes denoted $\mathcal{F}(\mathcal{D}, \boldsymbol{\theta})$.

The task at hand is therefore to find the set of parameters $\boldsymbol{\theta}^*$ that maximizes $J_{\mathrm{ELBO}}(\boldsymbol{\theta})$. The hardest term to evaluate is obviously the expectation of the log-likelihood

$$\mathbb{E}_q\left[\log\, p(\mathcal{D}|\boldsymbol{w})\right] = \sum_{i=1}^{N} \mathbb{E}_q\left[\log\, p(y^{(i)}|\boldsymbol{x}^{(i)}, \boldsymbol{w})\right]. \tag{31.13}$$

This problem constitutes a new and active area of research in machine learning and it permeates well with the overarching theme of this course. We will end by giving two pointers to further readings on this subject.

Bayes by Backprop

The well-cited paper Weight Uncertainty in Neural Networks (*Bayes by Backprop*) [BCKW15] has been well described in the blog entry by Martin Krasser. The main points of this blog entry are reproduced below with some

modifications and some adjustments of notation.

The variational free energy

All three terms in equation (31.12) are expectations w.r.t. the variational distribution $q(\boldsymbol{w}|\boldsymbol{\theta})$. In this paper they use the variational free energy $\mathcal{F}(\mathcal{D}, \boldsymbol{\theta}) \equiv -J_{\mathrm{ELBO}}(\boldsymbol{\theta})$ as a cost function (since it should be *minimized*). This quantity can be approximated by drawing Monte Carlo samples $\boldsymbol{w}^{(i)}$ from $q(\boldsymbol{w}|\boldsymbol{\theta})$.

$$\mathcal{F}(\mathcal{D}, \boldsymbol{\theta}) \approx \frac{1}{N} \sum_{i=1}^{N} \Big[\log\, q(\boldsymbol{w}^{(i)}|\boldsymbol{\theta}) - \log\, p(\boldsymbol{w}^{(i)}) - \log\, p(\mathcal{D}|\boldsymbol{w}^{(i)}) \Big] \tag{31.14}$$

In the example used in the blog post, they use a Gaussian distribution for the variational posterior, parameterized by $\boldsymbol{\theta} = (\boldsymbol{\mu}, \boldsymbol{\sigma})$ where $\boldsymbol{\mu}$ is the mean vector of the distribution and $\boldsymbol{\sigma}$ the standard deviation vector. The elements of $\boldsymbol{\sigma}$ are the elements of a diagonal covariance matrix which means that weights are assumed to be uncorrelated. Instead of parameterizing the neural network with weights $\boldsymbol{w}$ directly, it is parameterized with $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ and therefore the number of parameters are doubled compared to a plain neural network.

Network training

A training iteration consists of a forward-pass and and backward-pass. During a forward pass a single sample is drawn from the variational posterior distribution. It is used to evaluate the approximate cost function defined by equation (31.14). The first two terms of the cost function are data-independent and can be evaluated layer-wise, the last term is data-dependent and is evaluated at the end of the forward-pass. During a backward-pass, gradients of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are calculated via backpropagation so that their values can be updated by an optimizer.

The reparameterization trick

Since a forward pass involves a stochastic sampling step we have to apply the so-called *re-parameterization trick* for backpropagation to work. The trick is to sample from a parameter-free distribution and then transform the sampled $\boldsymbol{\epsilon}$ with a deterministic function $t(\boldsymbol{\mu}, \boldsymbol{\sigma}, \boldsymbol{\epsilon})$ for which a gradient can be defined. In the blog post they choose $\boldsymbol{\epsilon}$ to be drawn from a standard normal distribution i.e. $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ and the function t is taken to be $t(\boldsymbol{\mu}, \boldsymbol{\sigma}, \boldsymbol{\epsilon}) = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$, i.e., it shifts the sample by mean $\boldsymbol{\mu}$ and scales it with $\boldsymbol{\sigma}$ where $\odot$ is element-wise multiplication.

For numerical stability the network is parametrized with $\boldsymbol{\rho}$ instead of $\boldsymbol{\sigma}$ and $\boldsymbol{\rho}$ is transformed with the softplus function to obtain $\boldsymbol{\sigma} = \log(1 + \exp(\boldsymbol{\rho}))$. This ensures that $\boldsymbol{\sigma}$ is always positive. As prior, a scale mixture of two Gaussians is used $p(\boldsymbol{w}) = \pi \mathcal{N}(\boldsymbol{w}|0, \sigma_1^2) + (1 - \pi)\mathcal{N}(\boldsymbol{w}|0, \sigma_2^2)$ where σ_1, σ_2 and π are shared parameters. Their values are learned during training (which is in contrast to the paper where a fixed prior is used).

The demonstration notebook of this lecture, Demo: Bayes by Backprop in PyTorch, implements this in `PyTorch` for a one-dimensional regression problem in which the true noise level is known and varies with x, so that the learned aleatoric term can be checked against the truth, and in which the training data have a gap, so that the epistemic term has somewhere to grow.

31.4. Demo: Bayes by Backprop in PyTorch

The previous section derived the variational free energy and described *Bayes by Backprop*. Here we implement it in `PyTorch`, and use it on a one-dimensional regression problem where we know the right answer.

The point of the demo is to see the **two different uncertainties** that a Bayesian neural network gives you, and to see that they behave differently:

- the **aleatoric** part is the noise in the data. It is irreducible: more data would pin it down more precisely but would not make it smaller.
- the **epistemic** part is our uncertainty about the weights. It is large where we have no data and shrinks where we have plenty.

See the general discussion on epistemic and aleatoric uncertainties in Types of uncertainty.

We will build a data set in which both are visible at a glance: the noise grows from left to right, and there is a gap in the middle with no data at all.

The data

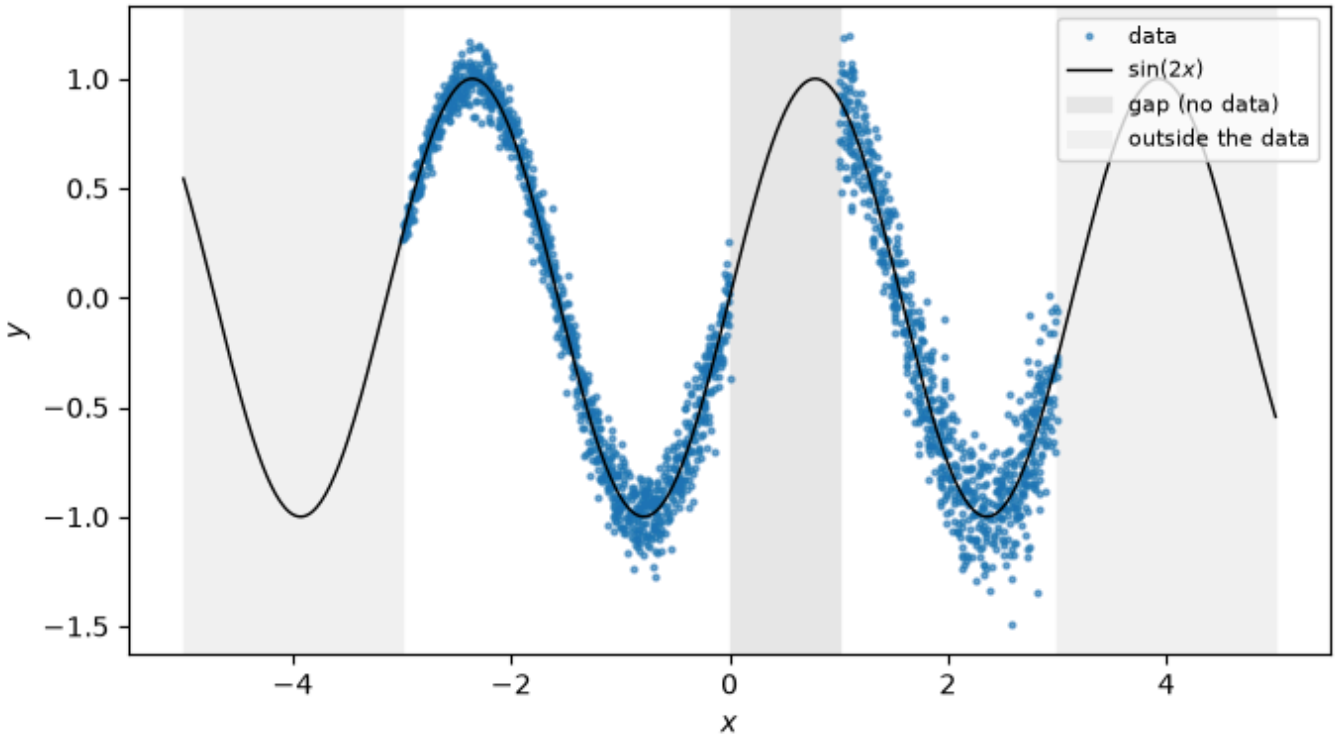
We generate data with

$$y = \sin(2x) + \varepsilon(x), \qquad \varepsilon(x) \sim \mathcal{N}\left(0, \sigma_{\text{true}}(x)^2\right),$$

with a noise level that grows linearly across the interval. Both $\sin(2x)$ and $\sigma_{\text{true}}(x)$ are known to us, so at the end we can check what the network learned against the truth. This is a luxury we never have in a real problem, and exactly why a demo is worth exploring.

We sample x uniformly from $[-3, 3]$ but leave $[0, 1]$ unsampled to study how our model performs for interpolation.

```
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(1)
rng = np.random.default_rng(1)


def f_true(x):
    return np.sin(2.0 * x)


def sigma_true(x):
    """The noise level grows from left to right."""
    return 0.05 + 0.15 * (x + 3.0) / 6.0


# Sample x everywhere except the gap [0, 1]
n = 2000
x = rng.uniform(-3.0, 3.0, size=4 * n)
x = x[(x < 0.0) | (x > 1.0)][:n]
y = f_true(x) + rng.normal(0.0, sigma_true(x))

xt = torch.tensor(x, dtype=torch.float32).unsqueeze(1)
yt = torch.tensor(y, dtype=torch.float32).unsqueeze(1)

xgrid = np.linspace(-5.0, 5.0, 500)          # note: wider than the data
xgrid_t = torch.tensor(xgrid, dtype=torch.float32).unsqueeze(1)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(x, y, '.', ms=4, alpha=0.6, label='data')
ax.plot(xgrid, f_true(xgrid), 'k-', lw=1, label=r'$\sin(2x)$')
ax.axvspan(0.0, 1.0, color='0.9', label='gap (no data)')
ax.axvspan(-5.0, -3.0, color='0.95')
ax.axvspan(3.0, 5.0, color='0.95', label='outside the data')
ax.set_xlabel('$x$'); ax.set_ylabel('$y$'); ax.legend(loc='upper right', fontsize=8)
fig.tight_layout()
```



From variational inference to Bayes by Backprop

Variational inference (see The evidence lower bound) is a *strategy* to address the hard Bayesian inference problem: replace the intractable $p(\boldsymbol{w}|\mathcal{D})$ by a tractable family of distributions $q(\boldsymbol{w}|\boldsymbol{\theta})$ and pick the member of that family that maximises the ELBO, equation (31.12). It leaves two questions open:

- *Which family* q should we use?
- *How* do we maximise the ELBO, which contains an expectation over q that has no closed form?

Bayes by Backprop answers both so that training reduces to ordinary backpropagation. Choice 1 answers the first question while choices 2 and 3 answer the second.

1. **The family is a diagonal Gaussian.** Every weight is independently normal with its own mean and width (the *mean-field* assumption). The KL term then becomes a sum of one-dimensional analytic expressions. The price is that a diagonal q cannot capture the correlations of the true posterior, and mean-field methods tend to *underestimate* the posterior width in parameter space, although the predictive variance can still come out too large [ORSF26].
2. **The expectation over q is estimated with one sample per gradient step.** A single draw of $\boldsymbol{w}$ gives a noisy but unbiased estimate; the noise averages out over many steps, as mini-batch noise does.
3. **The draw is made differentiable by the reparameterisation trick.** Writing $w = \mu + \sigma\epsilon$ with $\epsilon \sim \mathcal{N}(0, 1)$ moves the randomness into ϵ, which carries no parameters, so gradients flow to μ and σ.

Together these turn “maximise the ELBO” into “call `.backward()`”, hence the name.

Note that the optimiser adjusts $\boldsymbol{\theta} = (\mu, \rho)$, the parameters of the *distribution over* the weights, not the weights themselves. The network below has twice as many numbers to learn as its deterministic counterpart, and none of them is a weight.

A Bayesian linear layer

Every weight and bias gets a variational posterior $q(w|\theta) = \mathcal{N}(w; \mu, \sigma^2)$, parameterised by μ and ρ with $\sigma = \log(1 + e^{\rho})$ so that σ stays positive. A forward pass draws one sample using the reparameterisation trick,

$$w = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

so that gradients flow to μ and ρ through a deterministic function of ϵ.

The layer also has to report its contribution to the KL divergence $D_{\mathrm{KL}}(q\|p)$, equation (31.9). For two diagonal Gaussians this is analytic:

$$D_{\mathrm{KL}} = \sum_{j} \left[\log \frac{\sigma_p}{\sigma_j} + \frac{\sigma_j^2 + \mu_j^2}{2\sigma_p^2} - \frac{1}{2} \right].$$

We take the prior $p(w) = \mathcal{N}(0, \sigma_p^2)$ with $\sigma_p^2 = 2/n_{\text{in}}$, which is the `relu` initialisation distribution.

```
class BayesLinear(nn.Module):
    """Linear layer with q(w) = N(w; mu, sigma^2) and sigma = softplus(rho)."""

    def __init__(self, n_in, n_out):
        super().__init__()
        self.prior_sigma = (2.0 / n_in) ** 0.5
        self.w_mu = nn.Parameter(torch.randn(n_out, n_in) * self.prior_sigma)
        self.b_mu = nn.Parameter(torch.zeros(n_out))
        # softplus(-5) is about 0.007: start almost deterministic and let the
        # data widen the posterior. Starting wide often fails to train at all.
        self.w_rho = nn.Parameter(torch.full((n_out, n_in), -5.0))
        self.b_rho = nn.Parameter(torch.full((n_out,), -5.0))

    def forward(self, x):
        w = self.w_mu + F.softplus(self.w_rho) * torch.randn_like(self.w_rho)
        b = self.b_mu + F.softplus(self.b_rho) * torch.randn_like(self.b_rho)
        return F.linear(x, w, b)

    def kl(self):
        total = 0.0
        for mu, rho in ((self.w_mu, self.w_rho), (self.b_mu, self.b_rho)):
            sigma = F.softplus(rho)
            total = total + (torch.log(self.prior_sigma / sigma)
                             + (sigma ** 2 + mu ** 2) / (2 * self.prior_sigma ** 2)
                             - 0.5).sum()
        return total
```

The network

The architecture is chosen empirically. Two hidden layers of width 16 (about 340 weights and biases, each with a mean and a width to learn) is enough to follow $\sin(2x)$ and the growing noise level. Smaller networks underfit the mean near the edges, and the residual then lands in the aleatoric term. Much larger networks do not help either: the KL term penalises every parameter that deviates from the prior, and the fit degrades with too many parameters such that the epistemic width becomes large everywhere. The output layer produces **two** numbers rather than one: the

mean $\mu(x)$ and the log-variance $\log v(x)$ of the predictive distribution at that x. Letting the network output the width is what allows the aleatoric term to depend on the input, which it must here, since σ_{true} grows with x.

Clamping the log-variance is used to avoid sending it to $-\infty$ due to a single unlucky batch early in training, which would cause the run to never recover.

```
class BayesMLP(nn.Module):
    def __init__(self, width=16):
        super().__init__()
        self.h1 = BayesLinear(1, width)
        self.h2 = BayesLinear(width, width)
        self.out = BayesLinear(width, 2)      # mean and log-variance

    def forward(self, x):
        x = torch.tanh(self.h1(x))
        x = torch.tanh(self.h2(x))
        o = self.out(x)
        return o[:, 0:1], o[:, 1:2].clamp(-10.0, 2.0)

    def kl(self):
        return self.h1.kl() + self.h2.kl() + self.out.kl()
```

The cost

We minimise the variational free energy $\mathcal{F} = -J_{\text{ELBO}}$. Per data point,

$$\frac{\mathcal{F}}{N} = \mathbb{E}_q\left[-\log p(y|x,w)\right] + \frac{D_{\text{KL}}(q\|p)}{N},$$

where the expectation is estimated with a single weight sample per step — one sample is noisy but unbiased, and the noise averages out over many steps.

Two things to be careful about. The KL term is divided by the size of the **data set**, not the size of the mini-batch: it is a single penalty shared over all N points, and getting this wrong reweights prior against likelihood by the ratio of the two. And with a predicted variance the likelihood is Gaussian, so

$$-\log p(y|x,w) = \tfrac{1}{2}\left[\log v(x) + \frac{(y-\mu(x))^2}{v(x)}\right] + \text{const}\ ,$$

which reduces to the familiar squared error only when v is held fixed.

```python
def gaussian_nll(y, mean, log_var):
    return 0.5 * (log_var + (y - mean) ** 2 / log_var.exp()).mean()


net = BayesMLP()
opt = torch.optim.Adam(net.parameters(), lr=0.01)

N = len(xt)
for step in range(8000):
    mean, log_var = net(xt)
    loss = gaussian_nll(yt, mean, log_var) + net.kl() / N     # F / N
    opt.zero_grad(); loss.backward(); opt.step()
    if (step + 1) % 2000 == 0:
        print(f'step {step + 1:5d}   F/N = {loss.item():8.4f}   '
              f'KL/N = {(net.kl() / N).item():7.4f}')
```

```
step  2000   F/N =  -1.0919   KL/N =  0.3515

step  4000   F/N =  -1.1636   KL/N =  0.3304

step  6000   F/N =  -1.1094   KL/N =  0.3159

step  8000   F/N =  -1.1551   KL/N =  0.3052
```

The predictive distribution

The quantity we actually want is

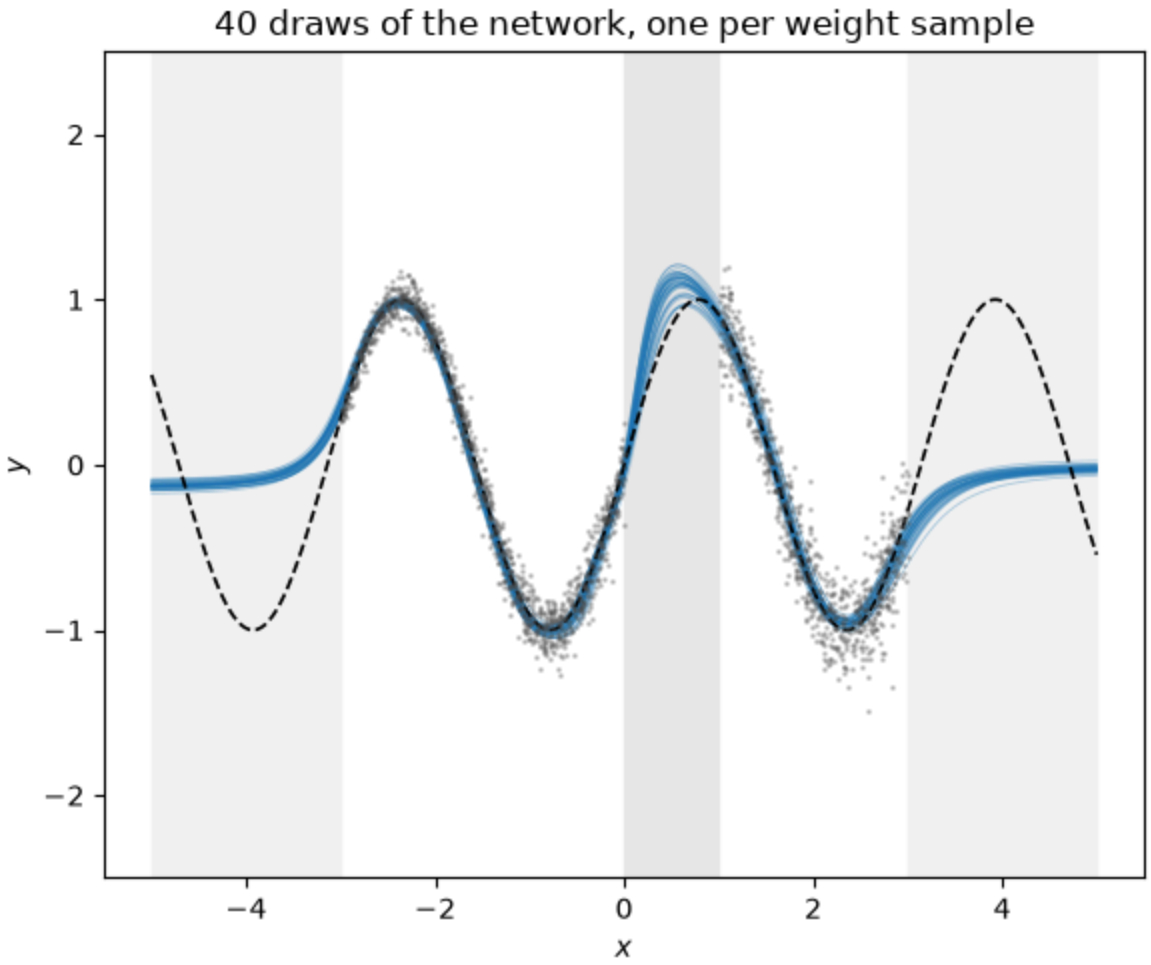
$$p(y|x,\mathcal{D}) = \int dw\, p(y|x,w)\, q(w|\theta^*),$$

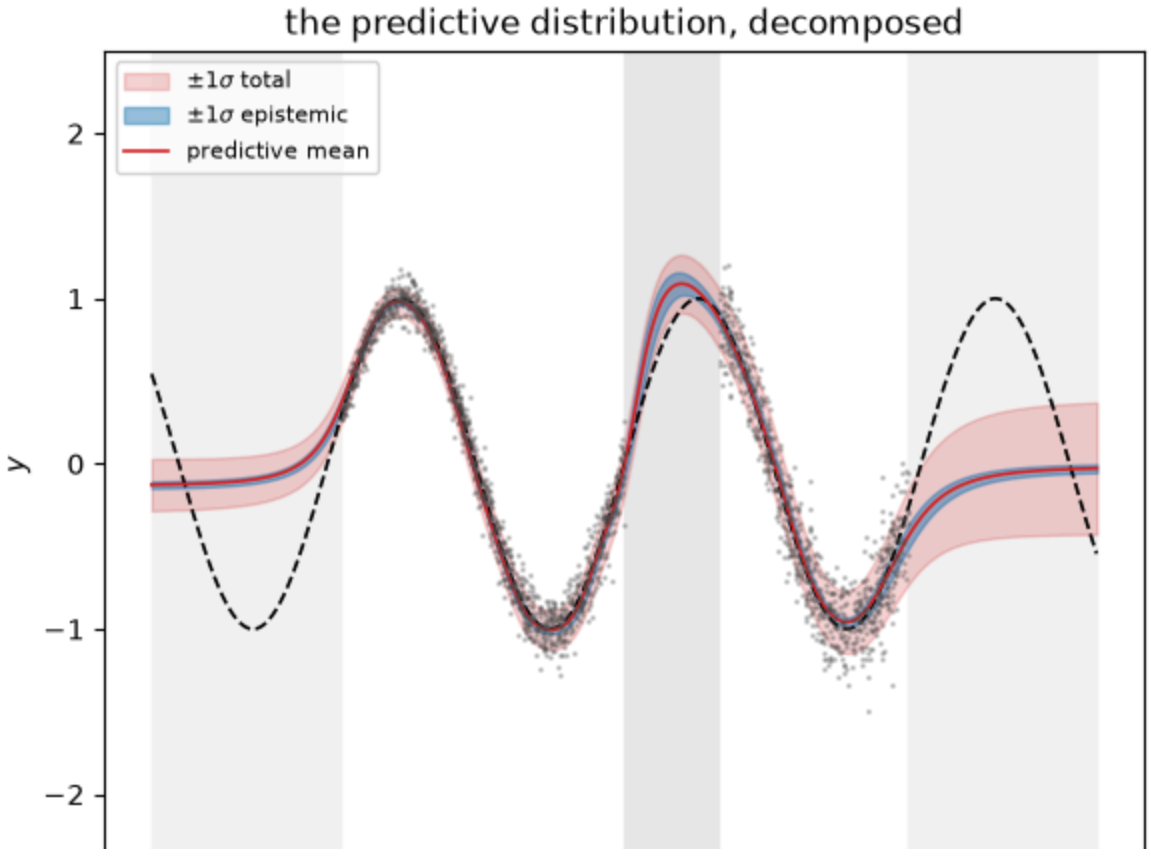
which we estimate by drawing S weight samples. Each sample w_s gives a mean $\mu_s(x)$ and a variance $v_s(x)$, and the **law of total variance** splits the result into the two pieces exactly:

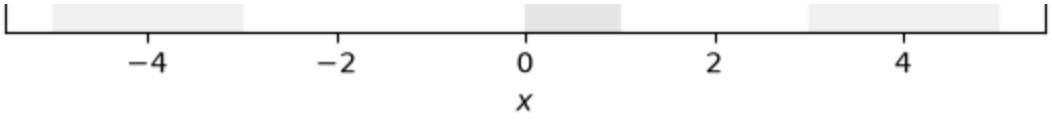
$$\underbrace{\mathrm{Var}\left[y|x,\mathcal{D}\right]}_{\text{total}} = \underbrace{\mathbb{E}_q\left[v_w(x)\right]}_{\text{aleatoric}} + \underbrace{\mathrm{Var}_q\left[\mu_w(x)\right]}_{\text{epistemic}}.$$

In words: average the *variances* to get the aleatoric part, and take the variance of the *means* to get the epistemic part. Both come out of the same S forward passes.

```
@torch.no_grad()
def predictive(net, x, n_samples=200):
    """Predictive mean, epistemic std and aleatoric std."""
    means, variances = [], []
    for _ in range(n_samples):
        m, lv = net(x)
        means.append(m); variances.append(lv.exp())
    means = torch.stack(means); variances = torch.stack(variances)
    return (means.mean(0).squeeze(),            # predictive mean
            means.std(0).squeeze(),             # epistemic:  std of the means
            variances.mean(0).squeeze().sqrt())  # aleatoric: mean of the variances


mu, epi, ale = predictive(net, xgrid_t)
mu, epi, ale = mu.numpy(), epi.numpy(), ale.numpy()
tot = np.sqrt(epi ** 2 + ale ** 2)

fig, axs = plt.subplots(2, 1, figsize=(6, 10))#, sharey=True, sharex=True)

# (a) draws of the function itself: one curve per weight sample
with torch.no_grad():
    for _ in range(40):
        m, _ = net(xgrid_t)
        axs[0].plot(xgrid, m.squeeze().numpy(), 'C0-', lw=0.6, alpha=0.35)
axs[0].plot(xgrid, f_true(xgrid), 'k--', lw=1.2, label=r'true $\sin(2x)$')
axs[0].plot(x, y, '.', ms=2, alpha=0.25, color='0.3')
axs[0].set_title('40 draws of the network, one per weight sample')

# (b) the same information as a band, split into its two parts
axs[1].fill_between(xgrid, mu - tot, mu + tot, color='C3', alpha=0.20,
                    label=r'$\pm 1\sigma$ total')
axs[1].fill_between(xgrid, mu - epi, mu + epi, color='C0', alpha=0.45,
                    label=r'$\pm 1\sigma$ epistemic')
axs[1].plot(xgrid, f_true(xgrid), 'k--', lw=1.2)
axs[1].plot(xgrid, mu, 'C3-', lw=1.2, label='predictive mean')
axs[1].plot(x, y, '.', ms=2, alpha=0.25, color='0.3')
axs[1].set_title('the predictive distribution, decomposed')
axs[1].legend(loc='upper left', fontsize=8)
axs[1].set_ylabel('$y$'); axs[1].set_ylim(-2.5, 2.5)

for a in axs:
    a.axvspan(0.0, 1.0, color='0.9', zorder=0)
    a.axvspan(-5.0, -3.0, color='0.95', zorder=0)
    a.axvspan(3.0, 5.0, color='0.95', zorder=0)
    a.set_xlabel('$x$')
axs[0].set_ylabel('$y$'); axs[0].set_ylim(-2.5, 2.5)
fig.tight_layout()
```


40 draws of the network, one per weight sample
2
1
0
−1
−2
y
−4
−2
0
2
4
x


the predictive distribution, decomposed
±1σ total
±1σ epistemic
predictive mean
2
1
0
−1
−2
y



Three things to look at, one of which is a warning.

1. **Inside the data** the individual draws lie almost on top of each other and the blue epistemic band is thin: many points constrain the weights there. The red total band is wider due to the noise. Note that it widens towards the right, following σ_{true}.
2. **In the gap** $[0, 1]$ the draws visibly fan out and the epistemic band roughly doubles. Nothing pinned the weights down there, so different draws of $\boldsymbol{w}$ disagree about what the function does in between. This is the honest statement "I have not been told", and it is the thing an ordinary network trained on the squared error cannot give you: that would be a single curve, equally confident in the gap as in the middle of the data.
3. **Outside** $[-3, 3]$, however, the draws *converge* again and the epistemic band gets narrower, not wider. This is not the network becoming certain; it is the `tanh` units saturating. A unit saturates once $|wx + b| \gg 1$, which for a scalar input is unavoidable at large $|x|$, so every sampled network outputs a near-constant there and the constants differ little. Swap `tanh` for `relu` and the extrapolation becomes linear instead of flat, with a different but equally arbitrary confidence.

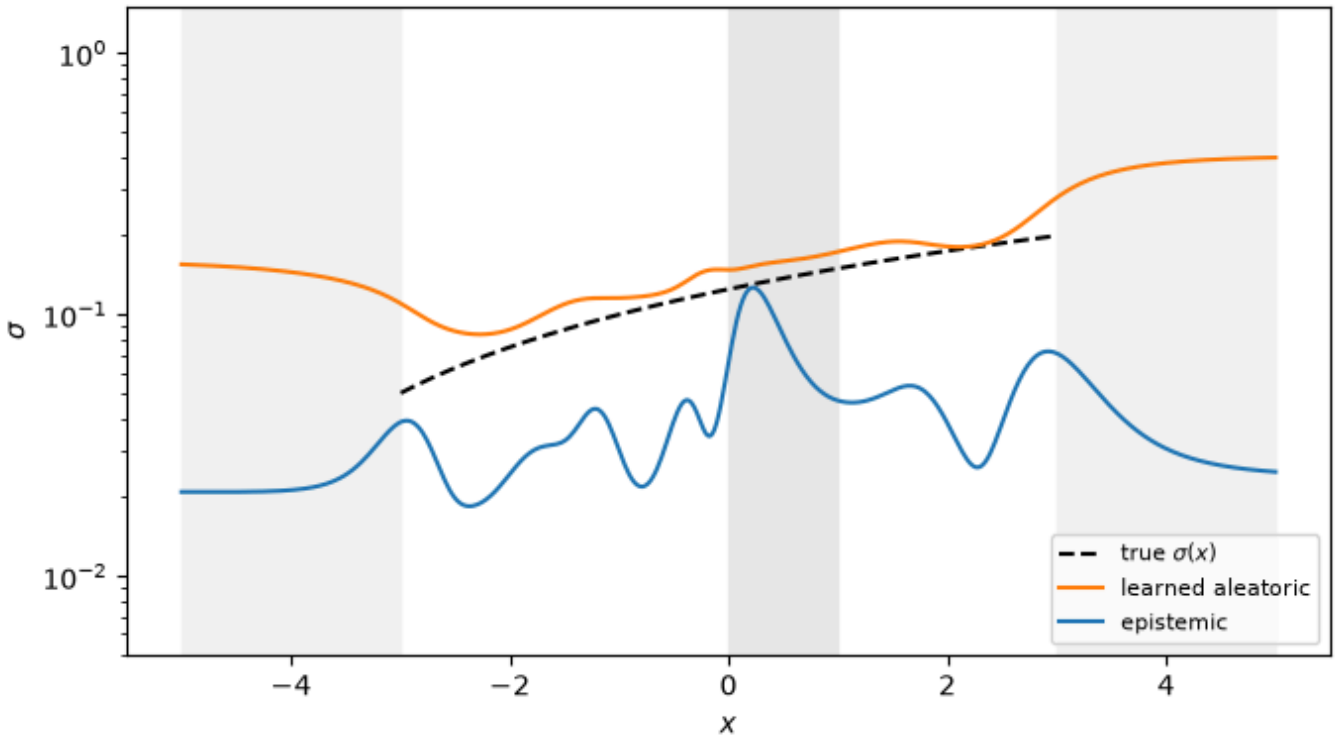
The third point is particularly important. *How a network behaves far from its training data is a property of the architecture, not of the Bayesian treatment*. Compare with a Gaussian process (Demo: Intuition for Gaussian process), where the posterior far from the data reverts to the prior: the mean goes to zero and the variance to the kernel amplitude, both dictated by the kernel we chose. A Bayesian neural network does the same, except that its prior over functions is defined implicitly by the architecture and the weight prior, and is harder to read off. Epistemic uncertainty beyond the data is therefore only as trustworthy as that prior, and mean-field variational inference narrows it further [ORSF26]. It remains a useful warning about gaps *between* data.

Did it learn the right noise?

Because we generated the data, we can check the aleatoric term against the truth. This is the part that would be invisible in a real problem.

```
fig, ax = plt.subplots(figsize=(7, 4))
# sigma_true is only defined on the interval the data were drawn from
data_range = (xgrid >= -3.0) & (xgrid <= 3.0)
ax.plot(xgrid[data_range], sigma_true(xgrid[data_range]), 'k--', lw=1.5,
        label=r'true $\sigma(x)$')
ax.plot(xgrid, ale, 'C1-', label='learned aleatoric')
ax.plot(xgrid, epi, 'C0-', label='epistemic')
ax.axvspan(0.0, 1.0, color='0.9', zorder=0)
ax.axvspan(-5.0, -3.0, color='0.95', zorder=0)
ax.axvspan(3.0, 5.0, color='0.95', zorder=0)
ax.set_xlabel('$x$'); ax.set_ylabel(r'$\sigma$')
ax.set_yscale('log'); ax.set_ylim(5e-3, 1.5)
ax.legend(fontsize=8, loc='lower right')
fig.tight_layout()

inside = (xgrid > -3.0) & (xgrid < 3.0) & ~((xgrid > 0.0) & (xgrid < 1.0))
print(f'where there is data: mean |learned - true| noise = '
      f'{np.abs(ale[inside] - sigma_true(xgrid[inside])).mean():.3f}')
print(f'epistemic sigma, inside the data: {epi[inside].mean():.3f}')
print(f'epistemic sigma, in the gap:      '
      f'{epi[(xgrid > 0.2) & (xgrid < 0.8)].mean():.3f}')
```

```
where there is data: mean |learned - true| noise = 0.022
epistemic sigma, inside the data: 0.038
epistemic sigma, in the gap:      0.087
```



Over most of the interval the learned aleatoric term tracks $\sigma_{\text{true}}(x)$ and reproduces its growth from left to right: the network worked out not just *that* the data are noisy but *where* they are noisier, which is the whole reason for letting the width depend on the input.

Where it does not track is also instructive. Towards the left edge, where the true noise is smallest, the learned width sits well above it. The noise model is not at fault. The fit is slightly worse there, and a Gaussian likelihood can only account for a residual it cannot remove by calling it noise. **A learned aleatoric term absorbs model discrepancy.**

Notice which term does what. Across the gap it is the **epistemic** curve that rises; the aleatoric one simply interpolates, because a predicted noise level is a fitted function like any other and has no obligation to behave sensibly where it has seen nothing. Whenever you report a single combined error bar you lose this distinction, and with it the ability to say whether more data would help.

Finally, outside the data range the epistemic curve falls rather than rises, for the reason discussed above: `tanh` saturation, not certainty.

What to take away

- A Bayesian neural network by variational inference is a small change to an ordinary one: weights carry a distribution, the forward pass samples from it, and the cost gains a KL term.
- The predictive spread separates cleanly, by the law of total variance, into a part that reflects noise in the data and a part that reflects ignorance about the weights. They behave differently and should be reported separately.

- Mean-field variational inference is known to *underestimate* posterior width: it assumes the weights are independent, which they are not. Treat the epistemic band as a useful warning rather than a calibrated interval, and check calibration whenever you can.

32. Overview of Gaussian processes

In this chapter we introduce and demonstrate Gaussian processes (GPs).

Good references for GPs are *Gaussian processes for machine learning* by Rasmussen and William; *The Kernel Cookbook* by Duvenaud; Chapter 21 of *Bayesian Data Analysis, 3rd Edition* by Gelman et al.; and Melendez et al., Phys. Rev. C **100**, 044001 (2019), arXiv:1904.10581.

The sections here are:

- Demo: Intuition for Gaussian process, which points to several websites with Gaussian process visualizations and tasks to build intuition;
- Basic info on GPs, which places GPs in the context of stochastic processes and provides a first exposure to the mathematical form of GPs and its use for interpolation and regression;
- More details on Gaussian processes, which distinguishes parametric and non-parametric inference
- Scikit-learn demo notebooks uses demonstrations with a popular library (scikit-learn)

32.1. Demo: Intuition for Gaussian process

GP sandbox widget

The widget here provides a Gaussian process (GP) visualization and some things to try to help build intuition about GPs. A GP defines a distribution over *functions*, so in the simulation we will see draws of random functions, in analogy to drawing random variables from a normal distribution (recall that “normal” and “Gaussian” are synonymous here). Our principal goal is to experience how GPs look in practice and how they are used for interpolation or regression.

The defining feature of an ensemble of GP function draws is that at *any* fixed point, the histogram of the function values at that point would approximate a Gaussian distribution; while for any two points the function values would be draws from a bivariate Gaussian distribution with a proscribed covariance matrix; and so on with more points being draws from a multivariate Gaussian distribution. The covariance in each case between any two points is specified by a *kernel* evaluated at those points.

Gaussian Process Intuition Demo

Click in the plot to add training points. Use the controls to change the kernel, hyperparameters, and whether the draws are from the prior or posterior.

Kernel RBF (SE)

ℓ (length scale) 1.00

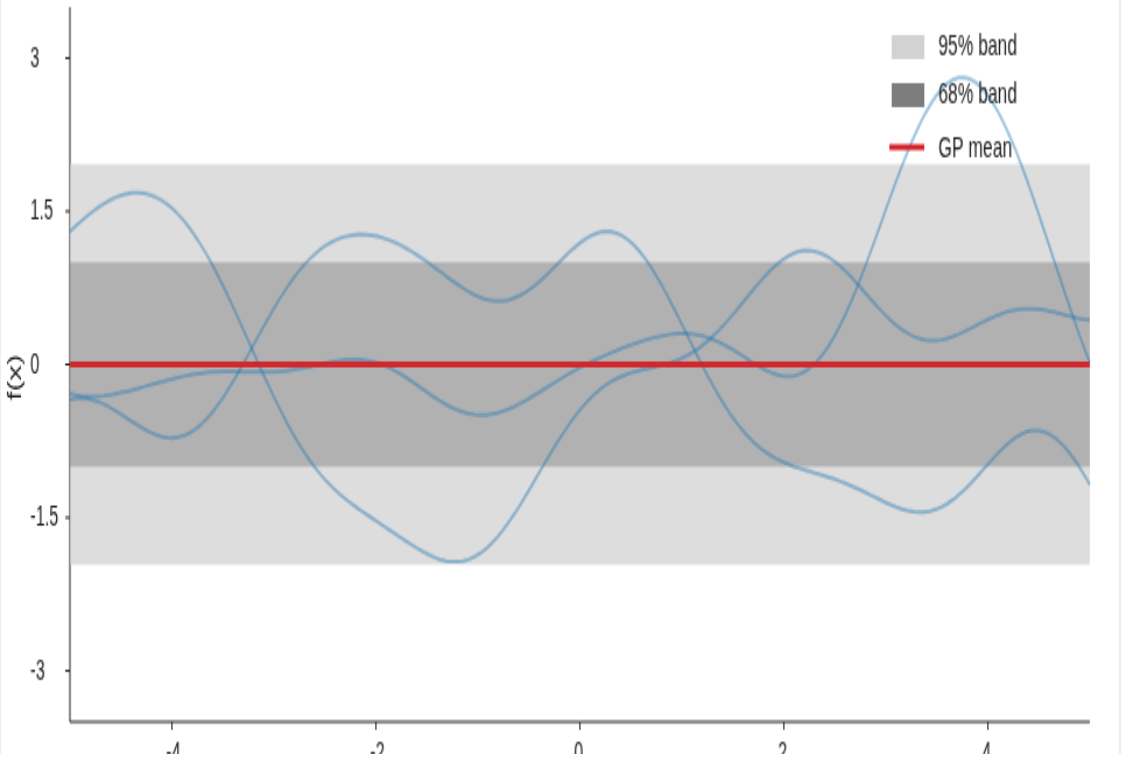
σ_f (signal std) 1.00

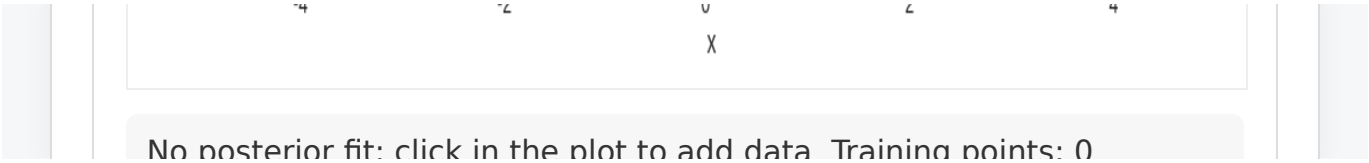
σ_n (noise std) 0.100

samples 3 Draw from posterior [prior]

Show mean & bands Optimize hyperparameters

Draw samples | Clear points | Reseed RNG





No posterior fit: click in the plot to add data. Training points: 0

Things to try

1. Drawing curves from a GP
   - Press the `Draw samples` button. These are draws from a GP, with each draw a function plotted from $x = -5$ to $x = +5$.
   - Turn the `Show mean & bands` button on and off. What is the value of the mean? What are the bands?
   - Change the number of realizations (using the `# samples slider`) to 1, 5, 10, maximum allowed. Look at any fixed x value, draw a vertical line (in your mind!), and visualize the histogram; does it look like a Gaussian distribution? What is the mean and standard deviation?
2. Changing the hyperparameters
   - Try changing ℓ `(length scale)`, which is the *correlation length*. What does this (hyper)parameter control?
   - Now try different values for the noise standard deviation and the signal standare deviation. What do each control?
   - Predict how changing the covariance and length scale will affect the draws and then try out various combinations. Do the draws change as predicted?
   - Try adjusting the noise. What happens?
3. Adding points
   - Add observations by clicking in the plot.
   - What happens to the GP mean (solid red line)?
   - What happens if you increase σ_n?
   - What if you uncheck the `Draw from posterior [prior]` box? What does it mean that you are showing the prior?
   - Takeaway: when points are added, the class of possible functions is reduced to those that pass near the points; the closeness is random scaling with the noise amplitude.
4. Trying different kernels
   - Explore the kernels under the `Kernel` pulldown (clear any point first).
   - What changes about the GP function draws?
   - What is the difference between the two Matern kernels?
5. Optimizing hyperparameters
   - Add at least one point. What does LML (*log marginal likelihood*) indicate?
   - If you adjust the hyperparameters by hand, what happens to the LML? Record the lowest LML and the corresponding values of the hyperparameters.
   - Now check the `Optimize hyperparameters` box. How does the LML compare to the lowest value you found by hand?

32.2. Basic info on GPs

Overview

Here we make use of the paper by Melendez et al., Phys. Rev. C **100**, 044001 (2019), arXiv:1904.10581 as an introduction to the definition and practical use of GPs. The reader will also find in this article a novel application of GPs to model discrepancy.

A stochastic *process* is a collection of random variables (RVs) indexed by time or space. I.e., at each time or at each space point there is a random variable. A *Gaussian* process (GP) is a stochastic process with definite relationships (correlations!) between the RVs. In particular, any finite subset (say at x_1, x_2, x_3) has a multivariate normal distribution. Thus, a GP is the natural generalization of multivariate random variables to infinite (countably or continuous) index sets. They look like random functions, but with characteristic degrees of smoothness, correlation lengths, and ranges.

Gaussian processes (GPs) are often used for *nonparametric regression*. Compare fitting a polynomial, where the basis functions are $1, x, x^2, \ldots, x^d$ and the coefficients are the parameters. This is *parametric regression*. So with GPs we do not have the corresponding set of parameters for basis functions. But there are parameters that specify the GP itself. Besides regression, the other most common applications of GPs are to *interpolation*. To carry out either of these we need to fit the GP parameters.

Specifying a GP

A GP is specified by a mean function $m(x)$ [often written $\mu(x)$] and a positive semidefinite covariance function, also called a *kernel*, $\kappa(x, x')$ where $x \in \mathbb{R}^d$ and d is the dimension of the parameter space. (We will generally suppress the d dependence of κ.) If we know $m(x)$ and $\kappa(x, x')$, then a GP $f(x)$ is denoted

$$f(x) \sim \mathcal{GP}[m(x), \kappa(x, x')]$$

in the same way a normal distribution is $g(x) \sim \mathcal{N}(\mu, \sigma^2)$.

While the definition has continuous, infinite-dimensional x, in practice we use a finite number of points:

$$\boldsymbol{x} = \{x_i\}_{i=1}^N \quad \text{and} \quad \boldsymbol{f} = \{f(x_i)\}_{i=1}^N$$

where N is the number of input "points" (which are actually vectors in general). Defining

$$\boldsymbol{m} = m(\boldsymbol{x}) \in \mathbb{R}^N \quad \text{and} \quad K = \kappa(\boldsymbol{x}, \boldsymbol{x}') \in \mathbb{R}^{N \times N},$$

we can write that any subset of inputs form a multivariate Gaussian:

$$\implies \boldsymbol{f}|\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{m}, K)$$

which can serve as the definition of a GP, as already mentioned. As usual, we say that "$\boldsymbol{f}$ is conditional on $\boldsymbol{x}$". The mean function is the *a priori* "best guess" for f. If there are no features, this mean function is often taken to be zero. Our specification of the kernel tells us what K is.

A general multivariate Gaussian distribution with mean vector $\boldsymbol{\mu}$ and covariance matrix Σ has the functional form

$$p(\boldsymbol{x}|\boldsymbol{\mu},\Sigma) = \frac{1}{\sqrt{\det(2\pi\Sigma)}} e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\intercal \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}.$$

For the bivariate case:

$$\boldsymbol{\mu} = \begin{pmatrix}\mu_x \\ \mu_y\end{pmatrix} \quad \text{and} \quad \Sigma = \begin{pmatrix}\sigma_x^2 & \rho\sigma_x\sigma_y \\ \rho\sigma_x\sigma_y & \sigma_y^2\end{pmatrix} \quad \text{with } \; 0 \leq \rho^2 < 1,$$

where Σ is manifestly positive definite. The contours of equal probability are ellipses. You can think of the bivariate case with strong correlations ($|\rho|$ close to one) as belonging to two points sufficiently close together on a single curve: the smoothness of the function tells us that the contour lines in a corner plot should be close to flat.

Kernels are the covariance functions that, given two points in the N-dimensional space, say $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$, return the covariance between $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$. Consider these vectors to be one-dimensional for simplicity, so we have x_1 and x_2 . Then the commonly used RBF (radial basis function) kernel is

$$K_{\text{RBF}}(x_1, x_2) = \sigma^2 e^{-(x_1-x_2)^2/2\ell^2}.$$

Compare this to

$$\Sigma = \sigma^2 \begin{pmatrix}1 & \rho \\ \rho & 1\end{pmatrix}.$$

The diagonals have $x_1 = x_2$ while $\rho = e^{-(x_1-x_2)^2/2\ell^2}$. So when x_1 and x_2 are close compared to ℓ then the values of the sample at x_1 and x_2 are highly correlated. When x_1 and x_2 are far apart, $\rho \to 0$ and they become independent. **Thus, ℓ plays the role of a correlation length.**

Using the GP

So how do we use this GP? Let's assume we already know $\boldsymbol{\theta}$, the set of hyperparameters. And suppose we know the value of the function f at a set of $\boldsymbol{x}_t$ points $\Longrightarrow$ this is our *training set*. Therefore partition the inputs into N_t training and N_e evaluation (test) points; the latter are our predictions:

$$\boldsymbol{x} = [\boldsymbol{x}_t\, \boldsymbol{x}_e]^\intercal \quad \text{and} \quad \boldsymbol{f} = f(\boldsymbol{x}) = [\boldsymbol{f}_t\, \boldsymbol{f}_e]^\intercal.$$

There are corresponding vectors $\boldsymbol{m}_t$ and $\boldsymbol{m}_e$ and covariance matrices.

The *definition* of a GP says the joint distribution of $\boldsymbol{f}_t$, $\boldsymbol{f}_e$ is

$$\begin{pmatrix}\boldsymbol{f}_t \\ \boldsymbol{f}_e\end{pmatrix} | \boldsymbol{x}, \boldsymbol{\theta} \sim \mathcal{N}\left[\begin{pmatrix}m_t \\ m_e\end{pmatrix}, \begin{pmatrix}K_{tt} & K_{te} \\ K_{et} & K_{ee}\end{pmatrix}\right],$$

where

$$K_{tt} = \kappa(\boldsymbol{x}_t, \boldsymbol{x}_t)$$
$$K_{ee} = \kappa(\boldsymbol{x}_e, \boldsymbol{x}_e)$$
$$K_{te} = \kappa(\boldsymbol{x}_t, \boldsymbol{x}_e) = K_{et}^\mathsf{T}$$

Then manipulation of matrices tells us that

$$\boldsymbol{f}_e | \boldsymbol{x}_e, \boldsymbol{x}_t, \boldsymbol{f}_t, \boldsymbol{\theta} \sim \mathcal{N}(\widetilde{m}_t, \widetilde{K}_{tt})$$

where

$$\widetilde{m}_e = m_e + K_{et} K_{tt}^{-1} (\boldsymbol{f}_t - \boldsymbol{m}_t)$$
$$\widetilde{K}_{ee} \equiv K_{ee} - K_{et} K_{tt}^{-1} K_{te}.$$

So the updated mean function $\widetilde{m}_e$ is our best prediction for $\boldsymbol{x}_e$ and $\widetilde{K}_{ee}$ is the variance (this determines the error band).

This makes sense:

$$p(\boldsymbol{f}_e, \boldsymbol{f}_t) = p(\boldsymbol{f}_e | \boldsymbol{f}_t) \times p(\boldsymbol{f}_t) \quad \Longrightarrow \quad p(\boldsymbol{f}_e | \boldsymbol{f}_t) = p(\boldsymbol{f}_e, \boldsymbol{f}_t) / p(\boldsymbol{f}_t)$$

and the ratio of Gaussians is also a Gaussian $\Longrightarrow$ we only need to figure out the matrices!

We can make draws from $\mathcal{N}(\widetilde{m}_e, \widetilde{K}_{ee})$ and plot them. With a dense enough grid, we will have lines that go through the $\boldsymbol{x}_t$ points and have bands in between, which grow larger when further from points. If we have Gaussian white noise at each point, then $K_{tt} \to K_{tt} + \sigma_n^2 I_{N_t}$ (if the variance of the noise is the same; otherwise it is a diagonal matrix with entries given by the separate variances).
In general, even if the data is perfect we need to add some noise (called adding a “nugget”) for numerical reasons: K_{tt}^{-1} will very probably be unstable without it. For predictions with noise, then $K_{ee} \to K_{ee} + \sigma_n^2 I_{N_e}$.

Checkpoint question

How do we make draws from a multivariate Gaussian (normal) distribution if we know how to make draws from $\mathcal{N}(0, 1)$?

Answer

Suppose $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, where $\boldsymbol{x}$ and $\boldsymbol{\mu}$ are dimension d and $\boldsymbol{\Sigma}$ is $d \times d$. Then we want to make draws with the pdf

$$p(\boldsymbol{x} | \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{\det 2\pi\boldsymbol{\Sigma}}} e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\mathsf{T} \boldsymbol{\Sigma}^{-1} (\boldsymbol{x}-\boldsymbol{\mu})}.$$

The procedure is:

- Generate $\boldsymbol{u} \sim \mathcal{N}(0, I_d)$ from d independent draws of $\mathcal{N}(0,1)$. This yields $\boldsymbol{u} = (u_0, u_1, \ldots, u_{d-1})^\intercal$.
- Find L such that $\boldsymbol{\Sigma} = LL^\intercal$ (using a Cholesky decomposition).
- Then $\boldsymbol{x} = \boldsymbol{\mu} + L\boldsymbol{u}$ has the desired distribution.

Check: What is the expectation value of $\boldsymbol{x}$, given $E[\boldsymbol{u}] = 0$?

$$E[\boldsymbol{x}] = E[\boldsymbol{\mu} + L\boldsymbol{u}] = \mu + LE[\boldsymbol{u}] = \boldsymbol{\mu}$$

which is the correct mean. Note that these manipulations used that expectation values are linear.

Now try for $\boldsymbol{\Sigma}$:

$$\begin{aligned}\boldsymbol{\Sigma} &\stackrel{?}{=} E[(\boldsymbol{x} - \boldsymbol{\mu})(\boldsymbol{x}^\intercal - \boldsymbol{\mu}^\intercal)] = E[(L\boldsymbol{u})(L\boldsymbol{u})^T] \\ &= E[L\boldsymbol{u}\boldsymbol{u}^\intercal L^\intercal] = LE[\boldsymbol{u}\boldsymbol{u}^\intercal] = LI_d L^\intercal = \boldsymbol{\Sigma}\end{aligned}$$

so it works!

Let's check that the formulas work for the special case of one training, one evaluation (test) point. We will also take the mean to be zero to avoid clutter. Then

$$\begin{pmatrix} f_t \\ f_e \end{pmatrix} \sim \mathcal{N}\left[\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \Sigma_{tt} & \Sigma_{te} \\ \Sigma_{et} & \Sigma_{ee} \end{pmatrix}\right],$$

where

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma_t^2 & \rho\sigma_t\sigma_e \\ \rho\sigma_t\sigma_e & \sigma_e^2 \end{pmatrix}.$$

The claim is that

$$f_e | f_t \sim \mathcal{N}(\Sigma_{et}\Sigma_{tt}^{-1} f_t, \Sigma_{ee} - \Sigma_{et}\Sigma_{tt}^{-1}\Sigma_{te})$$

i.e., that it is a Gaussian distribution with mean $\Sigma_{et}\Sigma_{tt}^{-1} f_t$ and variance $\Sigma_{ee} - \Sigma_{et}\Sigma_{tt}^{-1}\Sigma_{te}$. Try it explicitly:

$$p(f_e|f_t) = p(f_e, f_t)/p(f_e) = \frac{\frac{1}{\sqrt{\det(2\pi\boldsymbol{\Sigma})}} e^{-(f_t\ f_e)\,\boldsymbol{\Sigma}^{-1}\begin{pmatrix} f_t \\ f_e \end{pmatrix}}}{\frac{1}{\sqrt{2\pi\Sigma_{tt}}} e^{-f_t^2\Sigma_{tt}^{-1}}}$$

All of the Σ_{ij}'s are just scalars now and $\Sigma_{te} = \Sigma_{et}$.

$$\boldsymbol{\Sigma}^{-1} = \frac{1}{\det \boldsymbol{\Sigma}} \begin{pmatrix} \Sigma_{ee} & -\Sigma_{et} \\ -\Sigma_{te} & \Sigma_{tt} \end{pmatrix} = \frac{1}{\Sigma_{tt}\Sigma_{ee} - \Sigma_{te}\Sigma_{et}} \begin{pmatrix} \Sigma_{ee} & -\Sigma_{et} \\ -\Sigma_{te} & \Sigma_{tt} \end{pmatrix},$$

which yields

$$\begin{pmatrix} f_t & f_e \end{pmatrix} \boldsymbol{\Sigma}^{-1} \begin{pmatrix} f_t \\ f_e \end{pmatrix} = \frac{(-\Sigma_{ee} f_t^2 + 2\Sigma_{et} f_t f_e - \Sigma_{tt} f_e^2)}{\det \boldsymbol{\Sigma}}.$$

So if we gather the Σ_{ee} parts, we find to complete the square we need the combination $(f_e - \Sigma_{et}\Sigma_{tt}^{-1} f_t)$, which identifies the mean, and then the variance part comes out as advertised.

32.3. More details on Gaussian processes

Inference using Gaussian processes

Assume that there is a set of input vectors with independent, predictor, variables

$$\boldsymbol{X}_N \equiv \{\boldsymbol{x}^{(i)}\}_{i=1}^N \tag{32.1}$$

and a set of target values

$$\boldsymbol{t}_N \equiv \{t^{(i)}\}_{i=1}^N. \tag{32.2}$$

- Note that we will use the symbol t to denote the target, or response, variables in the context of Gaussian Processes. Here we will consider single, scalar outputs $t^{(i)}$. The extension to vector outputs $\boldsymbol{t}^{(i)}$ is straightforward.
- Furthermore, we will use the subscript N to denote a set of N vectors (or scalars): $\boldsymbol{X}_N$ ($\boldsymbol{t}_N$),
- … while a single instance i is denoted by a superscript: $\boldsymbol{x}^{(i)}$ ($t^{(i)}$).

Notation

We will use the notation $\mathcal{D}_N = [\boldsymbol{X}_N, \boldsymbol{t}_N]$ for the data.

- Note that a target is always associated with an input vector. When our focus is on the prediction of targets, we sometimes write $p(\boldsymbol{t}_N)$ without including $\boldsymbol{X}_N$ explcitly.

We will consider two different *inference problems*:

1. The prediction of a *new target* $t^{(N+1)}$ given the data $\mathcal{D}_N$ and a new input $\boldsymbol{x}^{(N+1)}$.
2. The inference of a *model function* $y(\boldsymbol{x})$ from the data $\mathcal{D}_N$.

The former can be expressed with the pdf

$$p\left(t^{(N+1)}|\mathcal{D}_N, \boldsymbol{x}^{(N+1)}\right) \tag{32.3}$$

while the latter can be written using Bayes' formula (in these notes we will not be including information I explicitly in the conditional probabilities)

$$p\left(y(\boldsymbol{x})|\mathcal{D}_N\right) = \frac{p\left(\mathcal{D}_N|y(\boldsymbol{x})\right)p\left(y(\boldsymbol{x})\right)}{p\left(\mathcal{D}_N\right)} \tag{32.4}$$

The inference of a function will obviously also allow to make predictions for new targets. However, we will need to consider in particular the second term in the numerator, which is the **prior** distribution on functions assumed in the model.

- This prior is implicit in parametric models with priors on the parameters.
- The idea of Gaussian process modeling is to put a prior directly on the **space of functions** without parameterizing $y(\boldsymbol{x})$.
- A Gaussian process can be thought of as a generalization of a Gaussian distribution over a finite vector space to a **function space of infinite dimension**.
- Just as a Gaussian distribution is specified by its mean and covariance matrix, a Gaussian process is specified by a **mean and covariance function**.

Gaussian process. A Gaussian process is a stochastic process (a collection of random variables indexed by time or space), such that every finite collection of those random variables has a multivariate normal distribution

References:

1. Gaussian Processes for Machine Learning, Carl Edward Rasmussen and Chris Williams [RW05], online version.
2. GPy: a Gaussian Process (GP) framework written in python, from the Sheffield machine learning group.

Parametric approach

Let us express $y(\boldsymbol{x})$ in terms of a model function $y(\boldsymbol{x};\boldsymbol{\theta})$ that depends on a vector of model parameters $\boldsymbol{\theta}$.

For example, using a set of basis functions $\{\phi_h(\boldsymbol{x})\}_{h=1}^H$ with linear weights $\boldsymbol{\theta} \in \mathbb{R}^H$ we have

$$y(\boldsymbol{x},\boldsymbol{\theta}) = \sum_{h=1}^{H} \theta_h \phi_h(\boldsymbol{x}) \tag{32.5}$$

Notice. The basis functions can be non-linear in $\boldsymbol{x}$ such as Gaussians (aka *radial basis functions*). Still, this constitutes a linear model since $y(\boldsymbol{x},\boldsymbol{\theta})$ depends linearly on the parameters $\boldsymbol{\theta}$.

The inference of model parameters should be a well-known problem by now. We state it in terms of Bayes theorem

$$p\left(\boldsymbol{\theta}|\mathcal{D}_N\right) = \frac{p\left(\mathcal{D}_N|\boldsymbol{\theta}\right)p\left(\boldsymbol{\theta}\right)}{p\left(\mathcal{D}_N\right)} \tag{32.6}$$

Having solved this inference problem (note that the likelihood is Gaussian, cf linear regression) a prediction can be made through marginalization

$$p\left(t^{(N+1)}|\mathcal{D}_N, \boldsymbol{x}^{(N+1)}\right) = \int d^H\boldsymbol{\theta} p\left(t^{(N+1)}|\boldsymbol{\theta}, \boldsymbol{x}^{(N+1)}\right) p\left(\boldsymbol{\theta}|\mathcal{D}_N\right). \tag{32.7}$$

Here it is important to note that the final answer does not make any explicit reference to our parametric representation of the unknown function $y(\boldsymbol{x})$.

Assuming that we have a fixed set of basis functions and Gaussian prior distributions (with zero mean) on the weights $\boldsymbol{\theta}$ we will show that:

- The joint pdf of the observed data given the model $p(\mathcal{D}_N)$, is a multivariate Gaussian with mean zero and with a covariance matrix that is determined by the basis functions.
- This implies that the conditional distribution $p(t^{(N+1)}|\mathcal{D}_N, \boldsymbol{X}_{N+1})$, is also a multivariate Gaussian whose mean depends linearly on $\boldsymbol{t}_N$.

Sum of normally distributed random variables.

If X and Y are independent random variables that are normally distributed (and therefore also jointly so), then their sum is also normally distributed. i.e., $Z = X + Y$ is normally distributed with its mean being the sum of the two means, and its variance being the sum of the two variances.

Consider the linear model and define the $N \times H$ design matrix $\boldsymbol{R}$ with elements

$$R_{nh} \equiv \phi_h\left(\boldsymbol{x}^{(n)}\right). \tag{32.8}$$

Then $\boldsymbol{y}_N = \boldsymbol{R}\boldsymbol{\theta}$ is the vector of model predictions, i.e.

$$y^{(n)} = \sum_{h=1}^{H} R_{nh}\theta_h. \tag{32.9}$$

Assume that we have a Gaussian prior for the linear model weights $\boldsymbol{\theta}$ with zero mean and a diagonal covariance matrix

$$p(\boldsymbol{\theta}) = \mathcal{N}\left(\boldsymbol{\theta}; 0, \sigma_\theta^2\boldsymbol{I}\right). \tag{32.10}$$

Now, since y is a linear function of $\boldsymbol{\theta}$, it is also Gaussian distributed with mean zero. Its covariance matrix becomes

$$\boldsymbol{Q} = \langle \boldsymbol{y}_N\boldsymbol{y}_N^T\rangle = \langle \boldsymbol{R}\boldsymbol{\theta}\boldsymbol{\theta}^T\boldsymbol{R}^T\rangle = \sigma_\theta^2\boldsymbol{R}\boldsymbol{R}^T, \tag{32.11}$$

which implies that

$$p(\boldsymbol{y}_N) = \mathcal{N}\left(\boldsymbol{y}_N; 0, \sigma_\theta^2 \boldsymbol{R}\boldsymbol{R}^T\right). \tag{32.12}$$

This will be true for any set of points $\boldsymbol{X}_N$; which is the defining property of a **Gaussian process**.

- What about the target values $\boldsymbol{t}_N$?

Well, if $t^{(n)}$ is assumed to differ by additive Gaussian noise, i.e.,

$$t^{(n)} = y^{(n)} + \varepsilon^{(n)}, \tag{32.13}$$

where $\varepsilon^{(n)} \sim \mathcal{N}\left(0, \sigma_\nu^2\right)$; then $\boldsymbol{t}_N$ also has a Gaussian distribution

$$p(\boldsymbol{t}_N) = \mathcal{N}\left(\boldsymbol{t}_N; 0, \boldsymbol{C}\right), \tag{32.14}$$

where the covariance matrix of this target distribution is given by

$$\boldsymbol{C} = \boldsymbol{Q} + \sigma_\nu^2 \boldsymbol{I} = \sigma_\theta^2 \boldsymbol{R}\boldsymbol{R}^T + \sigma_\nu^2 \boldsymbol{I}. \tag{32.15}$$

The covariance matrix as the central object

The covariance matrices are given by

$$Q_{nn'} = \sigma_\theta^2 \sum_h \phi_h\left(\boldsymbol{x}^{(n)}\right)\phi_h\left(\boldsymbol{x}^{(n')}\right), \tag{32.16}$$

and

$$C_{nn'} = Q_{nn'} + \delta_{nn'}\sigma_\nu^2. \tag{32.17}$$

This means that the correlation between target values $t^{(n)}$ and $t^{(n')}$ is determined by the points $\boldsymbol{x}^{(n)}$, $\boldsymbol{x}^{(n')}$ and the behaviour of the basis functions.

Non-parametric approach: Mean and covariance functions

In fact, we don't really need the basis functions and their parameters anymore. The influence of these appear only in the covariance matrix that describes the distribution of the targets, which is our key object. We can replace the parametric model altogether with a **covariance function** $C(\boldsymbol{x}, \boldsymbol{x}')$ which generates the elements of the covariance matrix

$$Q_{nn'} = C\left(\boldsymbol{x}^{(n)}, \boldsymbol{x}^{(n')}\right), \tag{32.18}$$

for any set of points $\left\{\boldsymbol{x}^{(n)}\right\}_{n=1}^{N}$.

Note, however, that $\boldsymbol{Q}$ must be positive-definite. This constrains the set of valid covariance functions.

Once we have defined a covariance function, the covariance matrix for the target values will be given by

$$C_{nn'} = C\left(\boldsymbol{x}^{(n)}, \boldsymbol{x}^{(n')}\right) + \sigma_\nu^2 \delta_{nn'}. \tag{32.19}$$

A wide range of different covariance contributions can be constructed. These standard covariance functions are typically parametrized with hyperparameters $\boldsymbol{\alpha}$ so that

$$C_{nn'} = C\left(\boldsymbol{x}^{(n)}, \boldsymbol{x}^{(n')}, \boldsymbol{\alpha}\right) + \delta_{nn'}\Delta\left(\boldsymbol{x}^{(n)}; \boldsymbol{\alpha}\right), \tag{32.20}$$

where Δ is often included as a flexible white noise component. It is usually good practice to include a small white noise term, $\Delta = \sigma_\nu^2 \ll 1$, even if your data has negligible errors, since it helps with numerical stability and making sure that your covariance matrix is positive definite.

Stationary kernels

The most common types of covariance functions are stationary, or translationally invariant, which implies that

$$C\left(\boldsymbol{x}, \boldsymbol{x}', \boldsymbol{\alpha}\right) = D\left(\boldsymbol{x} - \boldsymbol{x}'; \boldsymbol{\alpha}\right), \tag{32.21}$$

where the function D is often referred to as a *kernel*. Note that the $\left(\boldsymbol{x} - \boldsymbol{x}'\right)$-dependence must be such that the kernel is symmetric.

A very standard kernel is the RBF (radial basis function, but also known as Exponentiated Quadratic or Gaussian kernel) which is differentiable infinitely many times (hence, very smooth),

$$C_{\mathrm{RBF}}(\mathbf{x}, \mathbf{x}'; \boldsymbol{\alpha}) = \exp\left[-\frac{1}{2}\sum_{i=1}^{p}\frac{(x_i - x_i')^2}{l_i^2}\right] \tag{32.22}$$

where p denotes the dimensionality of the input space (i.e., $\mathbf{x} \in \mathbb{R}^p$). The hyperparameters of the RBF kernel, $\boldsymbol{\alpha} = \bar{l}$, are known as the correlation length(s). Sometimes, a single correlation length $l_i = l$ is used for all dimensions.

Different kernels can be combined to build the most relevant covariance function. For example, the RBF kernel is often multiplied by a constant kernel (which then becomes known as signal noise) followed by the addition of a diagonal white noise kernel. This would result in

$$C_(\mathbf{x}, \mathbf{x}'; \boldsymbol{\alpha}) = \sigma_f^2 \exp\left[-\frac{1}{2}\sum_{i=1}^{p}\frac{(x_i - x_i')^2}{l_i^2}\right] + \sigma_\nu^2 \delta_{\mathbf{x},\mathbf{x}'}, \tag{32.23}$$

with the complete set of hyperparameters $\boldsymbol{\alpha} = \{\sigma_\nu^2, \sigma_f^2, \vec{l}\}$ known as the white noise variance, signal variance, and RBF correlation length(s).

GP models for regression

Let us return to the problem of predicting $t^{(N+1)}$ given $\boldsymbol{t}_N$. The independent variables $\boldsymbol{X}_{N+1}$ are also given, but will be omitted from the conditional pdfs below.

The joint density is

$$p\left(t^{(N+1)}, \boldsymbol{t}_N\right) = p\left(t^{(N+1)}|\boldsymbol{t}_N\right)p\left(\boldsymbol{t}_N\right) \quad \Rightarrow \quad p\left(t^{(N+1)}|\boldsymbol{t}_N\right) = \frac{p\left(t^{(N+1)}, \boldsymbol{t}_N\right)}{p\left(\boldsymbol{t}_N\right)}. \tag{32.24}$$

First, let us note that $\boldsymbol{t}_{N+1} = \left\{\boldsymbol{t}_N, t^{(N+1)}\right\}$ such that $p\left(t^{(N+1)}, \boldsymbol{t}_N\right) = p\left(\boldsymbol{t}_{N+1}\right)$.

Since both $p\left(\boldsymbol{t}_{N+1}\right)$ and $p\left(\boldsymbol{t}_N\right)$ are Gaussian distributions, then the conditional distribution, obtained by the ratio, must also be a Gaussian. Let us use the notation $\boldsymbol{C}_{N+1}$ for the $(N+1)\times(N+1)$ covariance matrix for $\boldsymbol{t}_{N+1}$. This implies that

$$p\left(\boldsymbol{t}_{N+1}\right) \propto \exp\left[-\frac{1}{2}\left(\boldsymbol{t}_N, t^{(N+1)}\right)\boldsymbol{C}_{N+1}^{-1}\begin{pmatrix}\boldsymbol{t}_N \\ t^{(N+1)}\end{pmatrix}\right] \tag{32.25}$$

Summary

The prediction of the (Gaussian) pdf for $t^{(N+1)}$ requires an inversion of the covariance matrix $\boldsymbol{C}_{N+1}$.

Elegant linear algebra tricks to obtain $\boldsymbol{C}_{N+1}^{-1}$

Let us split the $\boldsymbol{C}_{N+1}$ covariance matrix into four different blocks

$$\boldsymbol{C}_{N+1} = \begin{pmatrix}\boldsymbol{C}_N & \boldsymbol{k} \\ \boldsymbol{k}^T & \kappa\end{pmatrix}, \tag{32.26}$$

where $\boldsymbol{C}_N$ is the $N \times N$ covariance matrix (which depends on the positions $\boldsymbol{X}_N$), $\boldsymbol{k}$ is an $N \times 1$ vector (that describes the covariance of $\boldsymbol{X}_N$ with $\boldsymbol{x}^{(N+1)}$), while κ is the single diagonal element obtained from $\boldsymbol{x}^{(N+1)}$.

We can use the partitioned inverse equations (Barnett, 1979) to rewrite $\boldsymbol{C}_{N+1}^{-1}$ in terms of $\boldsymbol{C}_N^{-1}$ and $\boldsymbol{C}_{N+1}$ as follows

$$\boldsymbol{C}_{N+1}^{-1} = \begin{pmatrix} \boldsymbol{M}_N & \boldsymbol{m} \\ \boldsymbol{m}^T & \mu \end{pmatrix}, \tag{32.27}$$

where

$$\begin{aligned} \mu &= \left(\kappa - \boldsymbol{k}^T \boldsymbol{C}_N^{-1} \boldsymbol{k}\right)^{-1} \\ \boldsymbol{m} &= -\mu \boldsymbol{C}_N^{-1} \boldsymbol{k} \\ \boldsymbol{M}_N &= \boldsymbol{C}_N^{-1} + \frac{1}{\mu} \boldsymbol{m}\boldsymbol{m}^T. \end{aligned} \tag{32.28}$$

Question

Check that the dimensions of the different blocks are correct.

The prediction for $t^{(N+1)}$ is a Gaussian

$$p\left(t^{(N+1)} | \boldsymbol{t}_N\right) = \frac{p\left(\boldsymbol{t}_{N+1}\right)}{p\left(\boldsymbol{t}_N\right)} \propto \frac{\exp\left[-\frac{1}{2}\left(\boldsymbol{t}_N, t^{(N+1)}\right) \boldsymbol{C}_{N+1}^{-1} \begin{pmatrix} \boldsymbol{t}_N \\ t^{(N+1)} \end{pmatrix}\right]}{\exp\left[-\frac{1}{2} \boldsymbol{t}_N^T \boldsymbol{C}_N^{-1} \boldsymbol{t}_N\right]}, \tag{32.29}$$

which can be written as a univariate Gaussian (see below).

A new target prediction using a GP

The prediction for $t^{(N+1)}$ is a Gaussian

$$p\left(t^{(N+1)} | \boldsymbol{t}_N\right) = \frac{1}{Z} \exp\left[-\frac{\left(t^{(N+1)} - \hat{t}^{(N+1)}\right)^2}{2\sigma_{\hat{t}_{N+1}}^2}\right]. \tag{32.30}$$

The mean and variance are obtained from (32.29) after some algebra

$$\begin{aligned} \text{mean}: \quad & \hat{t}^{(N+1)} = \boldsymbol{k}^T \boldsymbol{C}_N^{-1} \boldsymbol{t}_N \\ \text{variance}: \quad & \sigma_{\hat{t}_{N+1}}^2 = \kappa - \boldsymbol{k}^T \boldsymbol{C}_N^{-1} \boldsymbol{k}. \end{aligned} \tag{32.31}$$

This implies that we can make a prediction for the Gaussian pdf of $t^{(N+1)}$ (meaning that we predict its value with an associated uncertainty) for an N^3 computational cost (the inversion of an $N \times N$ matrix).

In fact, since the prediction only depends on the N available data we might as well predict several new target values at once. Consider $\boldsymbol{t}_M = \{t^{(N+i)}\}_{i=1}^M$ so that

$$\boldsymbol{C}_{N+M} = \begin{pmatrix} \boldsymbol{C}_N & \boldsymbol{k} \\ \boldsymbol{k}^T & \boldsymbol{\kappa} \end{pmatrix}, \tag{32.32}$$

where $\boldsymbol{k}$ is now an $N \times M$ matrix and $\boldsymbol{\kappa}$ an $M \times M$ matrix.

Many new target predictions using a GP

The prediction for M new targets $\boldsymbol{t}_M$ becomes a multivariate Gaussian

$$p\left(\boldsymbol{t}_M | \boldsymbol{t}_N\right) = \frac{1}{Z} \exp\left[-\frac{1}{2}\left(\boldsymbol{t}_M - \hat{\boldsymbol{t}}_M\right)^T \boldsymbol{\Sigma}_M^{-1}\left(\boldsymbol{t}_M - \hat{\boldsymbol{t}}_M\right)\right], \tag{32.33}$$

where the $M \times 1$ mean vector and $M \times M$ covariance matrix are

$$\begin{aligned} \hat{\boldsymbol{t}}_M &= \boldsymbol{k}^T \boldsymbol{C}_N^{-1} \boldsymbol{t}_N \\ \boldsymbol{\Sigma}_M &= \boldsymbol{\kappa} - \boldsymbol{k}^T \boldsymbol{C}_N^{-1} \boldsymbol{k}. \end{aligned} \tag{32.34}$$

Choosing the GP model hyperparameters

Predictions can be made once we have

1. Chosen an appropriate covariance function.
2. Determined the hyperparameters.
3. Evaluated the relevant blocks in the covariance function and inverted $\boldsymbol{C}_N$.

How do we determine the hyperparameters $\boldsymbol{\alpha}$? Well, recall that

$$p\left(\boldsymbol{t}_N\right) = \frac{1}{Z_N} \exp\left[-\frac{1}{2} \boldsymbol{t}_N^T \boldsymbol{C}_N^{-1} \boldsymbol{t}_N\right]. \tag{32.35}$$

This pdf is basically a data likelihood.

- The simplest approach is therefore to find the set of hyperparameters $\boldsymbol{\alpha}^*$ that maximizes the data likelihood, i.e.,

$$\boldsymbol{\alpha}^* = \underset{\alpha}{\operatorname{argmin}}\, \boldsymbol{t}_N^T \boldsymbol{C}_N^{-1}(\alpha) \boldsymbol{t}_N \tag{32.36}$$

- A Bayesian approach would be to assign a prior to the hyperparameters and seek a posterior pdf $p(\boldsymbol{\alpha}|\boldsymbol{t}_N)$ instead which is then propagated using marginalization

$$p\left(t^{(N+1)} | \boldsymbol{t}_N\right) = \int d\boldsymbol{\alpha} p\left(t^{(N+1)}, \boldsymbol{\alpha} | \boldsymbol{t}_N\right) = \int d\boldsymbol{\alpha} p\left(t^{(N+1)} | \boldsymbol{t}_N, \boldsymbol{\alpha}\right) p(\boldsymbol{\alpha} | \boldsymbol{t}_N) \tag{32.37}$$

The optimization approach is absolutely dominating the literature on GP regression. The data likelihood is maximized with respect to the hyperparameters and the resulting covariance function is then used for regression. The second approach gives a better quantification of the uncertainties, but is more computationally demanding.

32.4. Demo: Gaussian processes

Gaussian processes as high-dimensional Gaussian distributions

A Gaussian process (GP) is a collection of random variables, $(Y_x)_{x \geq 0}$ (labeled by a continuous index x) such that any finite number of such variables has a joint, multivariate Gaussian distribution. In fact, this implies that the GP is a continuous-time stochastic process although we use x instead of t as our index variable.

We will typically consider any number $n = 1, 2, \ldots$ of indices with $0 \leq x_1 < x_2, < \ldots < x_n$.

For any n, the GP is completely determined by its mean vector $\boldsymbol{\mu}$, with $\mu_i = \mathbb{E}(Y_i)$, and covariance matrix $\boldsymbol{\Sigma}$, with $\Sigma_{ij} = \text{Cov }(Y_i, Y_j)$, for $i, j \in 1, \ldots, n$ and $Y_k \equiv Y(x_k)$.

Although a GP can formally involve any valid covariance matrix, we will restrict ourselves to positive covariances $\Sigma_{ij} > 0$. Furthermore, we will usually have correlations that become weaker with increasing distance in the index variable, i.e.,

$$\rho_{ij} < \rho_{ik}, \text{ for } \ x_i < x_j < x_k.$$

(recall that $\rho_{ij} = \frac{\Sigma_{ij}}{\sigma_i \sigma_j}$, where $\sigma_i^2 = \Sigma_{ii}$).

Multivariate normal distributions

Consider the set of random variables

$$Y_1, Y_2, \ldots, Y_k.$$

These have a multivariate normal (Gaussian) distribution if the linear combination

$$Z = a_1 Y_1 + a_2 Y_2 + \ldots + a_k Y_k$$

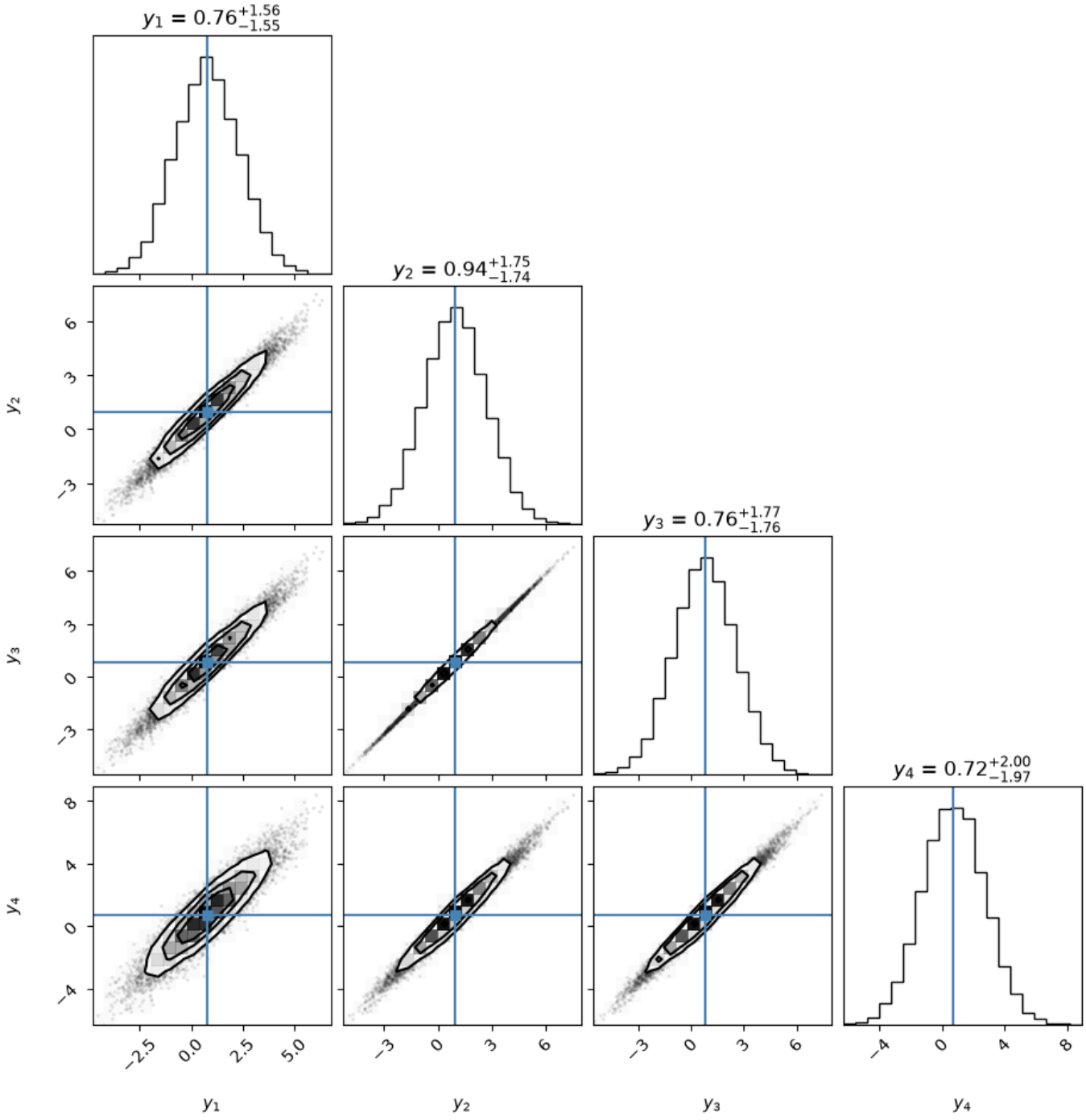
has a univariate normal distribution for all real numbers $(a_1, a_2, \ldots, a_k)$. The multivariate normal distribution is completely specified by the mean vector $\boldsymbol{\mu}$ and the covariance matrix $\boldsymbol{\Sigma}$. In fact, the random variable $Z \sim \mathcal{N}(\mu_Z, \sigma_Z^2)$ with

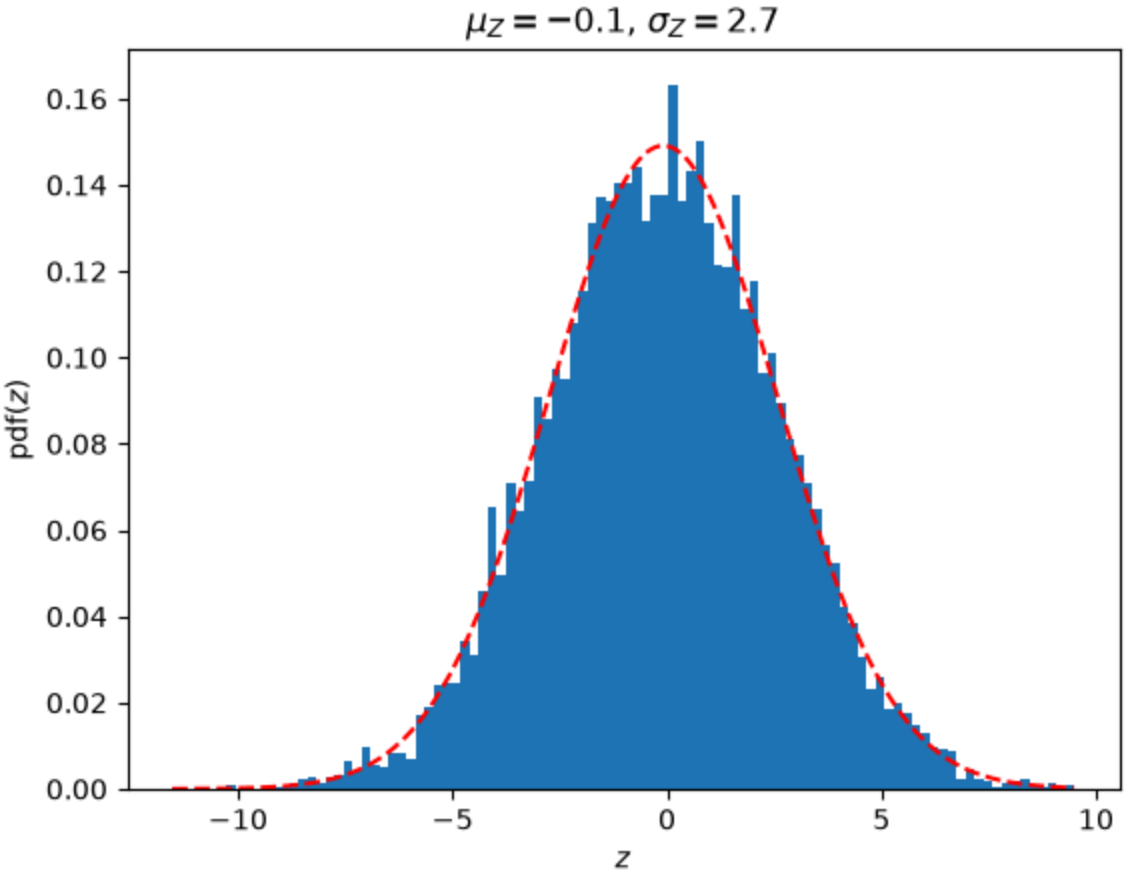
$$\mu_Z = \sum_{i=1}^{k} a_i \mu_i$$

$$\sigma_Z^2 = \sum_{i,j=1}^{k} a_i a_j \Sigma_{i.j}$$

Make a corner plot with samples from bi- and univariate marginals. Note that corner shows a histogram and iso-probability contours in the densest region (and samples outside). Then plot the histogram of samples together with the analytical distribution.

▼ **Hide code cell source**

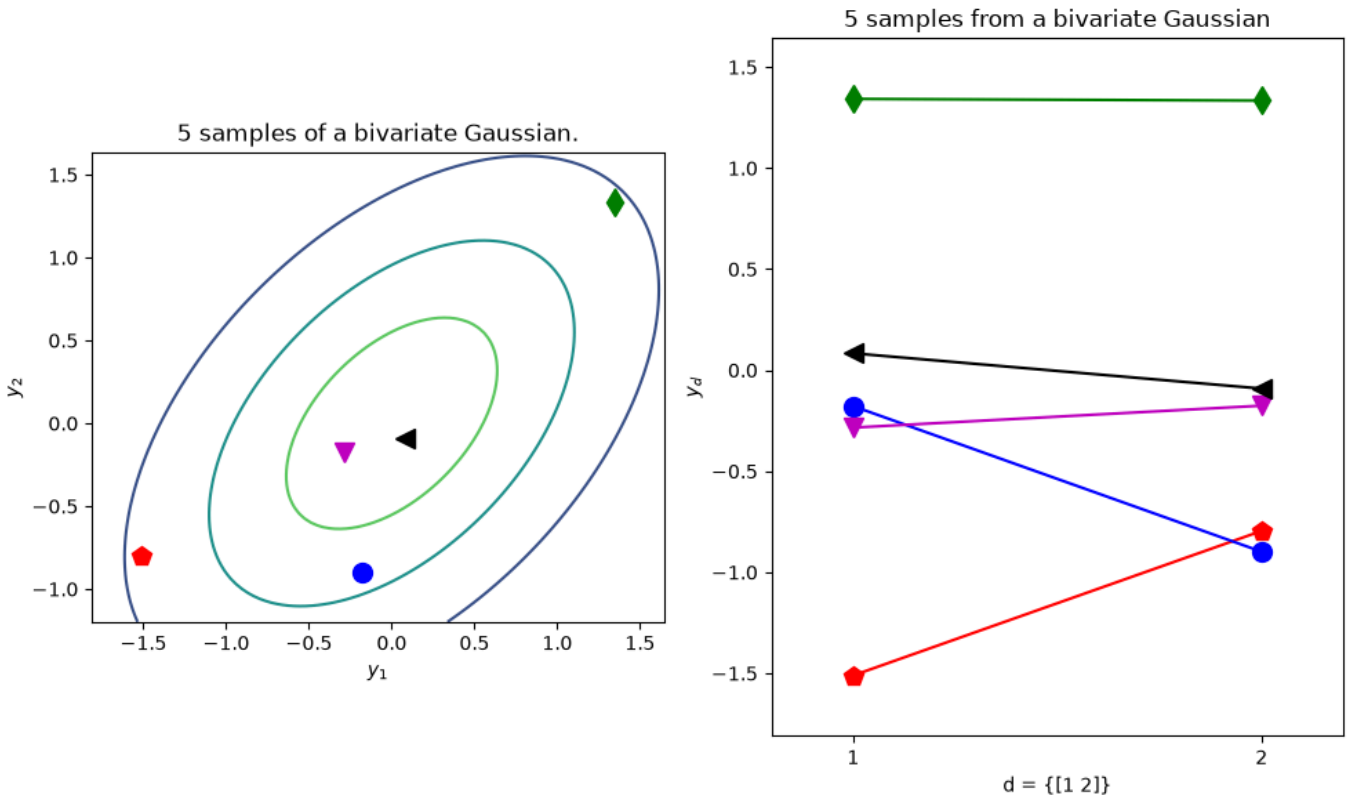

$y_1 = 0.76^{+1.56}_{-1.55}$
$y_2 = 0.94^{+1.75}_{-1.74}$
$y_3 = 0.76^{+1.77}_{-1.76}$
$y_4 = 0.72^{+2.00}_{-1.97}$
y_1
y_2
y_3
y_4



Sampling and plotting multivariate Gaussians

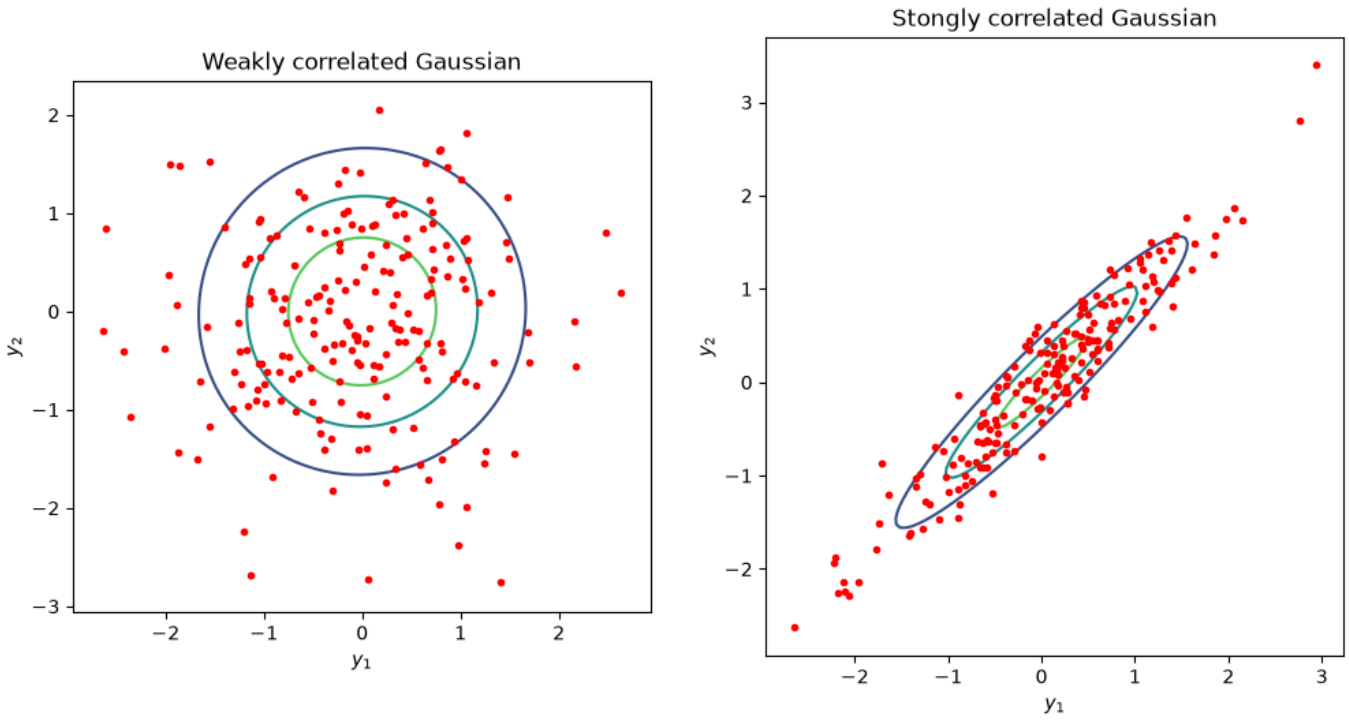
Test two different ways of plotting a bivariate Gaussian.

▼ **Hide code cell source**



Repeat as before, but now we'll plot many samples from two kinds of Gaussians: one which with strongly correlated dimensions and one with weak correlations

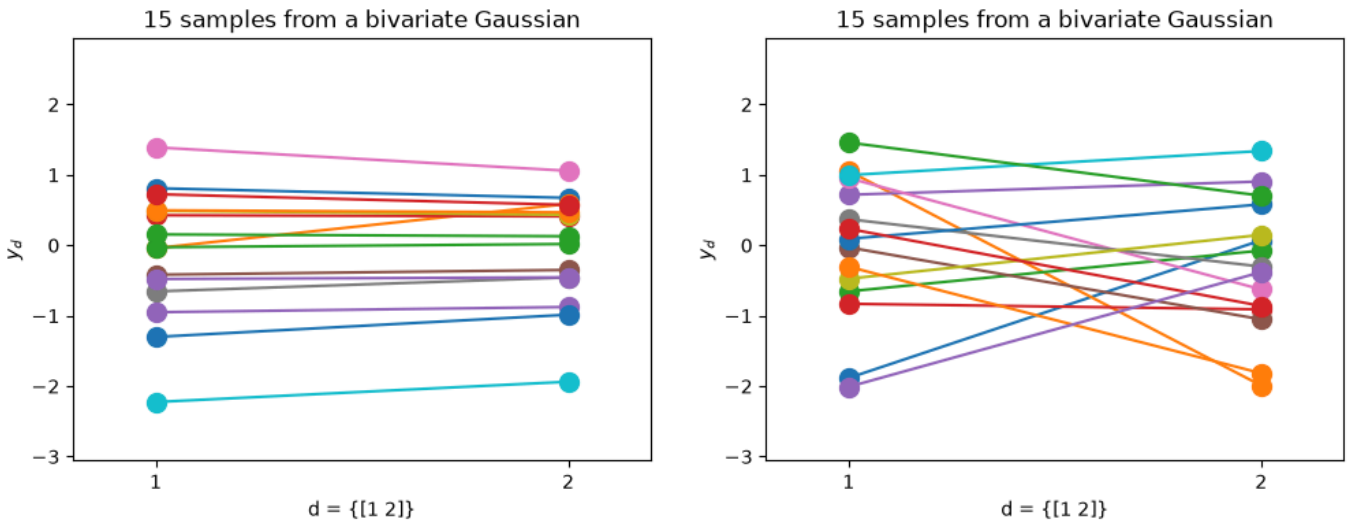
▼ **Hide code cell source**



But let's plot them as before dimension-wise…

Which one is which?

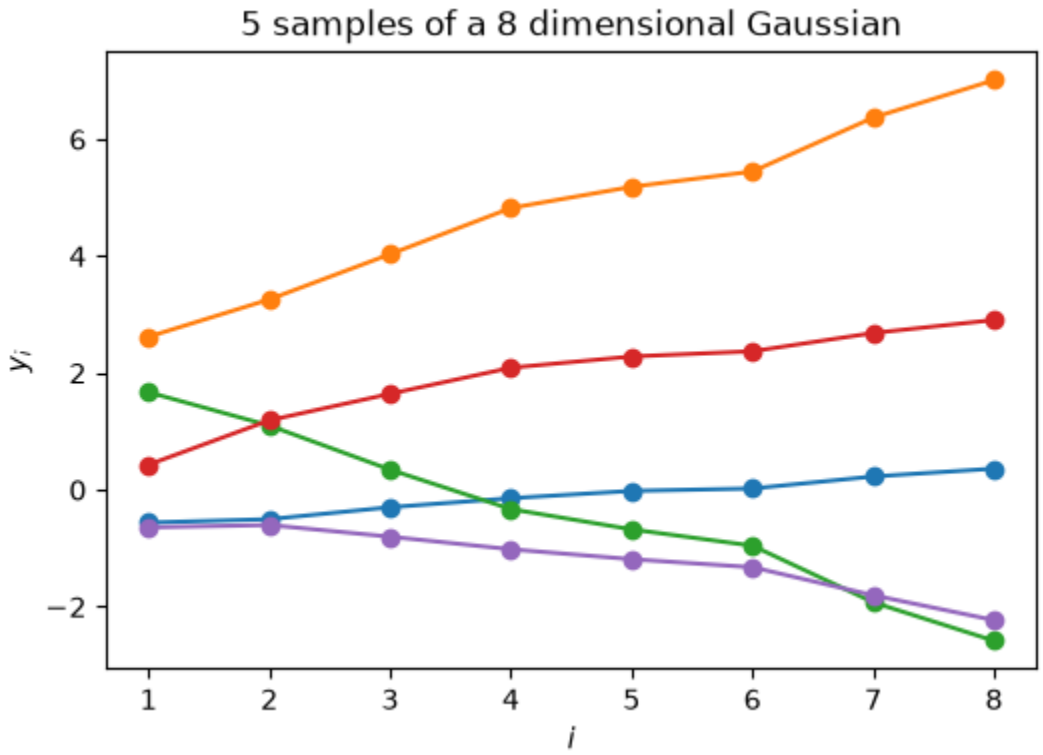
▼ **Hide code cell source**



- The strongly correlated Gaussian gives more “horizontal” lines in the dimension-wise plot.
- More importantly, by using the dimension-wise plot, we are able to plot Gaussians which have more than two dimensions. Below we plot N samples from a D=8-dimensional Gaussian.
- Because I don’t want to write down the full 8x8 covariance matrix, I define a “random” one through a mathematical procedure that is guaranteed to give me back a positive definite and symmetric matrix (i.e. a valid covariance). More on this later.

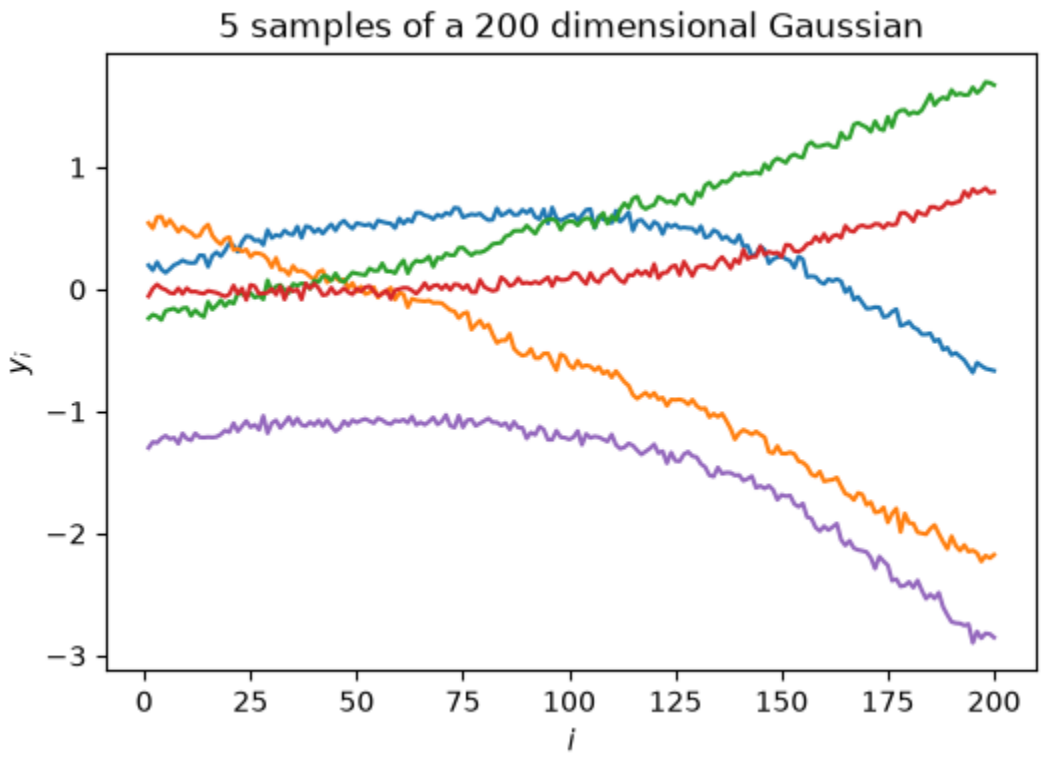
▼ **Hide code cell source**

```
Covariance matrix:
[[2.02 2.09 2.12 2.15 2.16 2.17 2.19 2.21]
 [2.09 2.45 2.69 2.89 2.98 3.07 3.27 3.42]
 [2.12 2.69 3.1  3.48 3.66 3.83 4.23 4.55]
 [2.15 2.89 3.48 4.05 4.33 4.59 5.22 5.73]
 [2.16 2.98 3.66 4.33 4.65 4.96 5.72 6.33]
 [2.17 3.07 3.83 4.59 4.96 5.32 6.19 6.9 ]
 [2.19 3.27 4.23 5.22 5.72 6.19 7.38 8.35]
 [2.21 3.42 4.55 5.73 6.33 6.9  8.35 9.54]]
Eigenvalues all larger than zero
[35.57  2.75  0.19  0.    0.    0.    0.    0.  ]
```



Taking this even further, we can plot samples from a 200-dimensional Gaussian in the dimension-wise plot.

▼ **Hide code cell source**



We see that each sample now starts looking like a “smooth” curve (notice that the random variables are highly correlated). This illustrates why a GP can be seen as an *infinite dimensional multivariate Gaussian* that can be used as a *prior over functions*.

Mean and covariance function

Similarly to how a D-dimensional Gaussian is parameterized by its mean vector and its covariance matrix, a GP is parameterized by a mean *function* and a covariance *function*. These can be written as functions of the index variable x. We'll assume (without loss of generality) that the mean function is $\mu(x) = \mathbf{0}$. As for the covariance function, $C(x, x')$, it is a function that receives as input two locations x, x' belonging to the input domain, i.e. $x, x' \in \mathcal{X}$, and returns the value of their co-variance.

In this way, if we have a *finite* set of input locations we can evaluate the covariance function at every pair of locations and obtain a covariance matrix $\mathbf{C}$. We write:

$$\mathbf{C} = C(\mathbf{x}, \mathbf{x}').$$

Covariance functions, aka kernels

We saw above their role for creating covariance matrices from training inputs, thereby allowing us to work with a finite domain when it is potentially infinite.

We'll see below that the covariance function is what encodes our assumption about the GP. By selecting a covariance function, we are making implicit assumptions about the shape of the function we wish to encode with the GP, for example how smooth it is.

Even if the covariance function has a parametric form, combined with the GP it gives us a nonparametric model. In other words, the covariance function is specifying the general properties of the GP function we wish to encode, and not a specific parametric form for it.

Below we define two very common covariance functions: The RBF (also known as Exponentiated Quadratic or Gaussian kernel) which is differentiable infinitely many times (hence, very smooth),

$$C_{\text{RBF}}(\mathbf{x}_i, \mathbf{x}_j) = \sigma_f^2 \exp\left(-\frac{1}{2\ell^2}\sum_{q=1}^{Q}(x_{i,q} - x_{j,q})^2\right)$$

where Q denotes the dimensionality of the input space. Its parameters are: the *lengthscale*, ℓ and the signal variance σ_f^2. We also note that this kernel is *stationary*, meaning that it only depends on the distance $|\mathbf{x}_i - \mathbf{x}_j|$.

There are many other kinds of kernels. An example of a non-stationary one is the linear kernel:

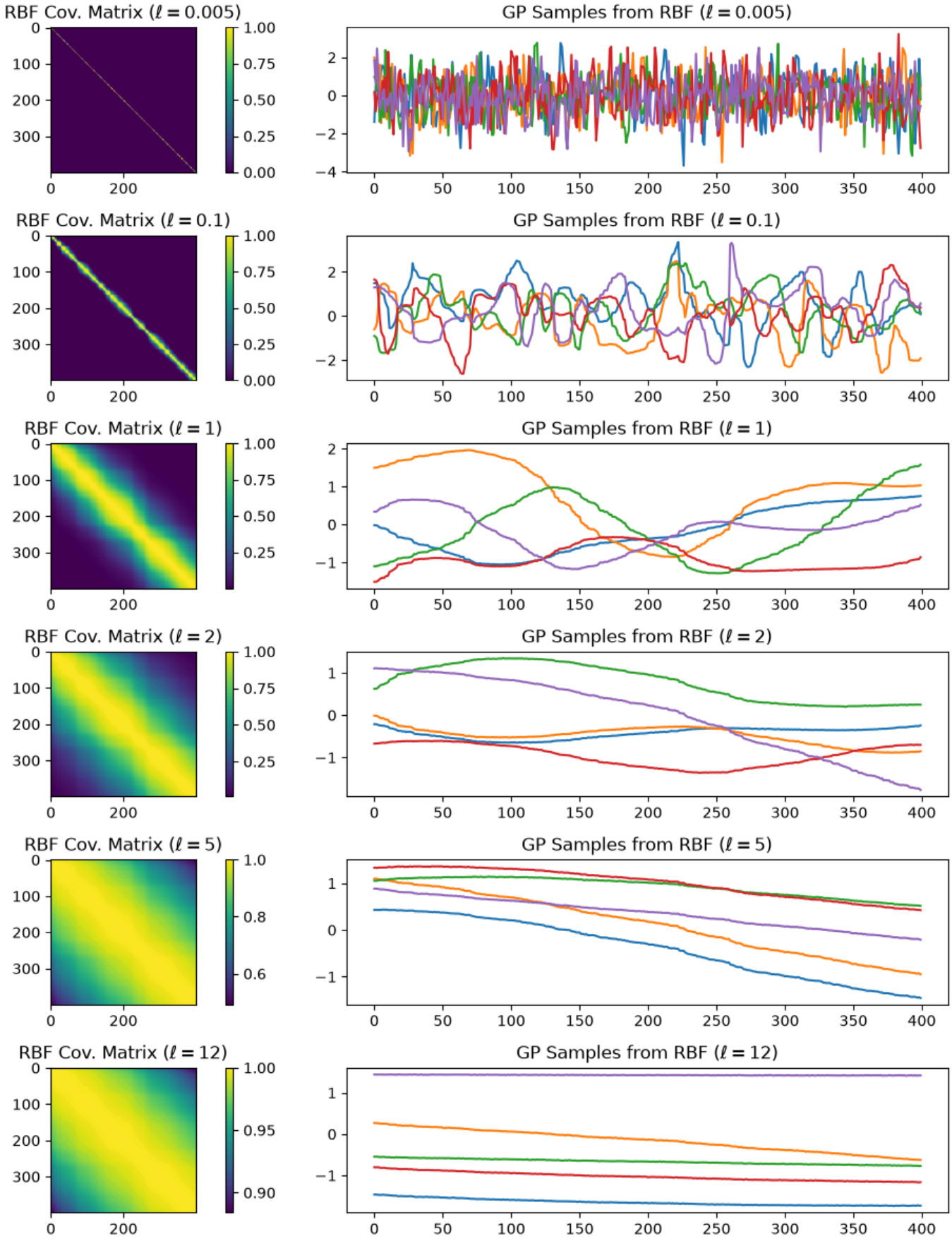
$$C_{\text{lin}}(\mathbf{x}_i, \mathbf{x}_j) = \sigma^2 \mathbf{x}_i^T \mathbf{x}_j$$

Below, we will implement and investigate them.

The RBF kernel (a.k.a Gaussian)

Given hyperparameters α, we plot the resulting covariance matrix and samples from a GP with this covariance function.

▼ **Hide code cell source**

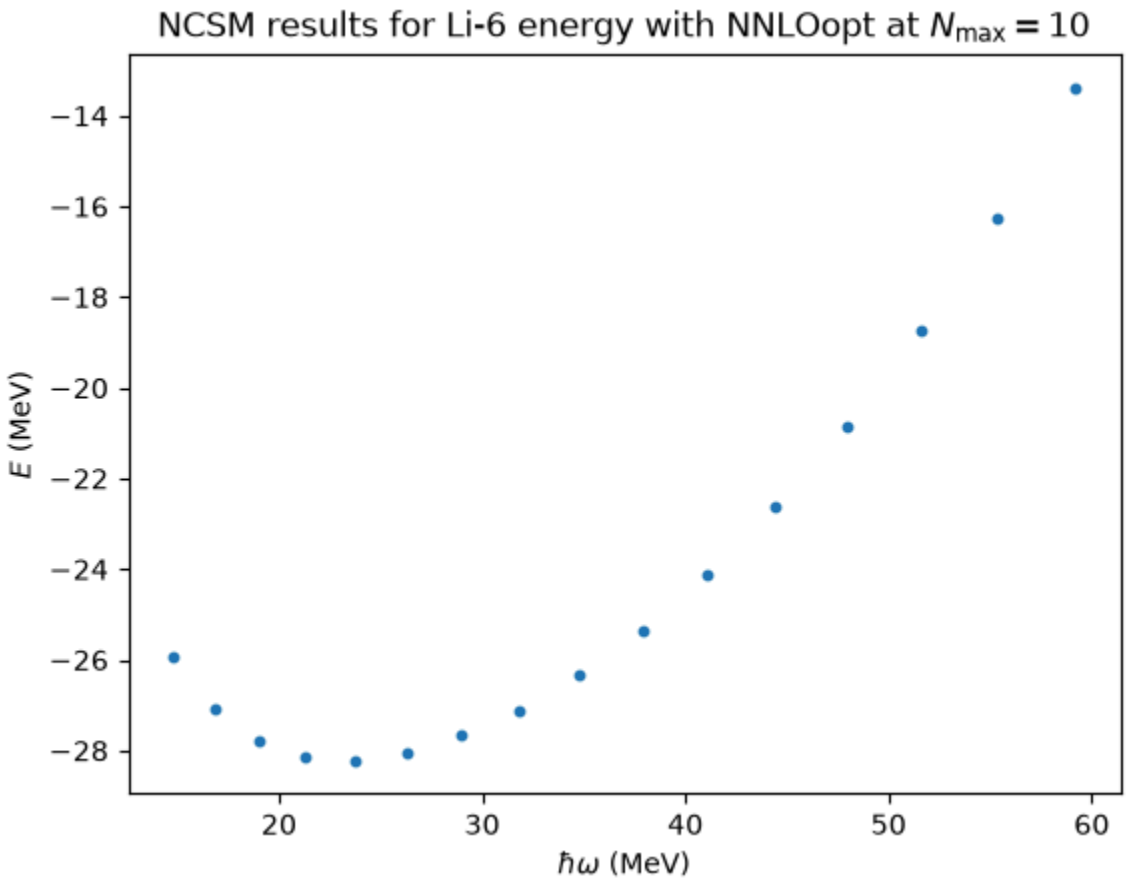

RBF Cov. Matrix ($\ell = 0.005$)
GP Samples from RBF ($\ell = 0.005$)
RBF Cov. Matrix ($\ell = 0.1$)
GP Samples from RBF ($\ell = 0.1$)
RBF Cov. Matrix ($\ell = 1$)
GP Samples from RBF ($\ell = 1$)
RBF Cov. Matrix ($\ell = 2$)
GP Samples from RBF ($\ell = 2$)
RBF Cov. Matrix ($\ell = 5$)
GP Samples from RBF ($\ell = 5$)
RBF Cov. Matrix ($\ell = 12$)
GP Samples from RBF ($\ell = 12$)

Example: GP models for regression

No-core shell model $\hbar\omega$ dependence

Import some NCSM data from *Phys. Rev. C 97, 034328 (2018)*.

▼ **Hide code cell source**

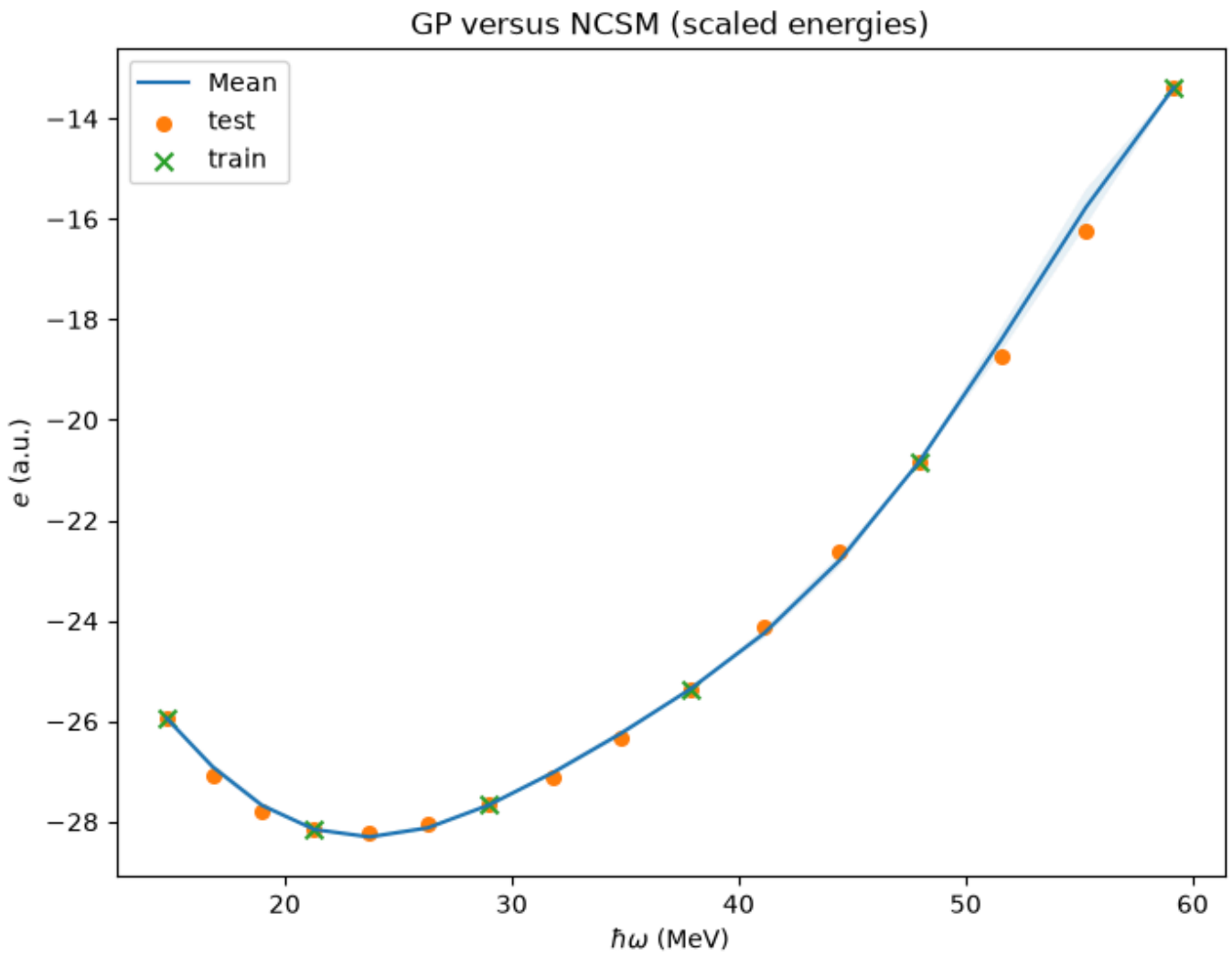


Test our GP by training on every third data point and validate on the rest.

▼ **Hide code cell source**

```
k1 : 2.24**2
k1__constant_value : 5.0
k1__constant_value_bounds : (1e-05, 100000.0)
k2 : RBF(length_scale=10)
k2__length_scale : 10.0
k2__length_scale_bounds : (1e-05, 100000.0)
       l = 15.329
 sigma_f = 1.384
sigma_nu = 1e-06
```

▼ **Hide code cell source**



Print the largest ratios (predict-true)/predict for the validation data.

▼ **Hide code cell source**

```
Validation result: Ratio true/predict in [0.9930,1.0303]
```

32.5. Scikit-learn demo notebooks

The Gaussian Process for Machine Learning page on the scikit-learn website is a great source of code and documentation and examples for GPs.

Here we have adapted their demonstration notebooks for:

- One-dimensional GP regression. Compares noise-free (interpolation) and noisy (regression) for a one-dimensional function (which can be easily changed). An RBF kernel is the default, but this is exchangeable for

any of the standard sklearn kernels. A maximum likelihood fit determines the hyperparameters (so it might fail to find a good solution, but the hyperparameter values are given so this can be diagnosed).

- Prior and posterior Gaussian process for different kernels. This example illustrates the prior and posterior of the Scikit-learn class `GaussianProcessRegressor` with different kernels. Mean, standard deviation, and 5 samples are shown for both prior and posterior distributions.

We also have additional demo notebooks

- Demonstration: Gaussian Process Regression, which builds an RBF-based kernel (with signal scale and noise term), fits the GP on a subset (e.g., every 3rd point), predicts mean and uncertainty on a target grid or the full input, plots mean ±2σ and data, and computes simple validation metrics.
- Exercise: Gaussian Processes, which build RBF kernels with signal variance and length-scale, fit GaussianProcessRegressor with a white-noise term, predict posterior mean and uncertainty, plot mean ±2σ and data, examine setting hyperparameters explicitly vs. optimizing by LML.
- Gaussian Processes Exercises, which build RBF kernels and visualize samples, fit a GP to 1D data (train/test split), plot the posterior mean and ±2σ band, apply the workflow to a small dataset.

One-dimensional GP regression

A simple one-dimensional regression example computed in two different ways:

1. A noise-free case
2. A noisy case with known noise-level per datapoint

In both cases, the kernel's parameters are estimated using the maximum likelihood principle.

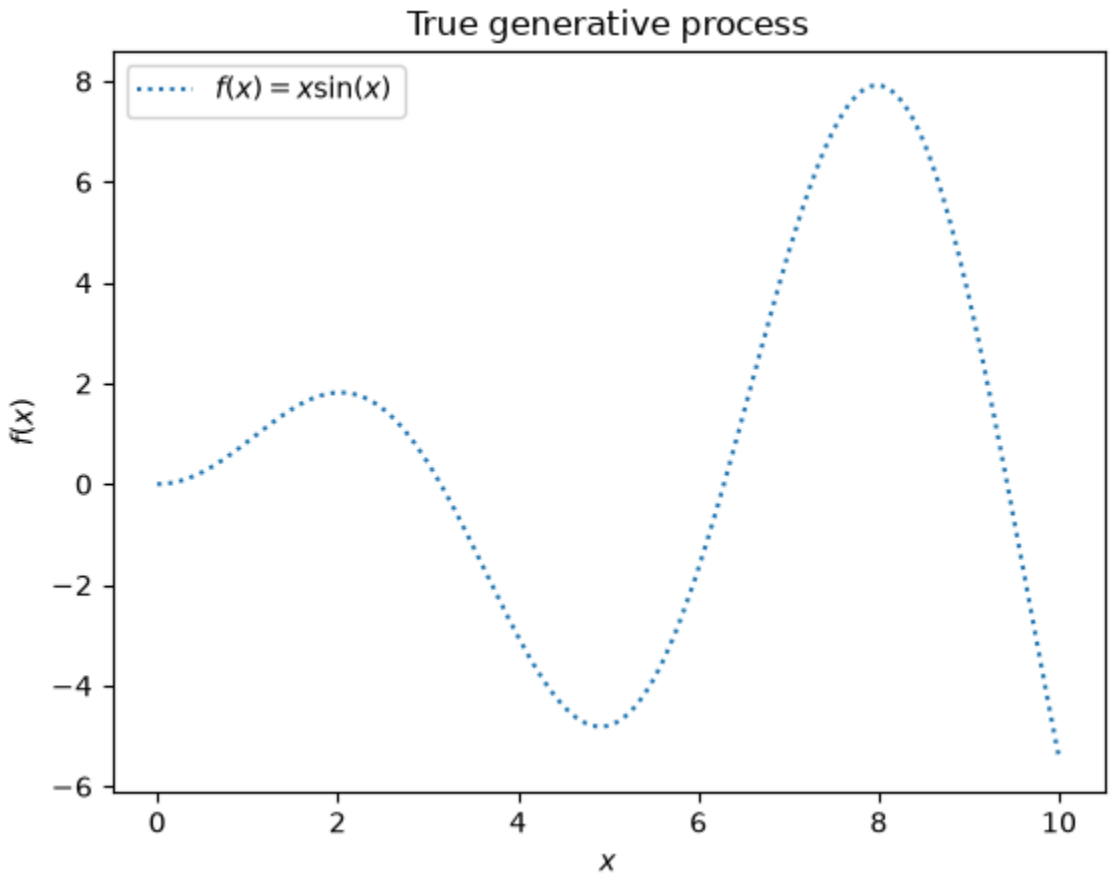
The figures illustrate the interpolating property of the Gaussian Process model as well as its probabilistic nature in the form of a pointwise 95% confidence interval.

Note that `alpha` is a parameter to control the strength of the Tikhonov regularization on the assumed training points' covariance matrix.

Dataset generation

We will start by generating a synthetic dataset. The true generative process is defined as $f(x) = x \sin(x)$ (but you can change the function as desired).

▼ **Hide code cell source**



We will use this dataset in the next experiment to illustrate how Gaussian Process regression is working.

Example with noise-free target

In this first example, we will use the true generative process without adding any noise. For training the Gaussian Process regression, we will only select few samples.

▼ **Hide code cell source**

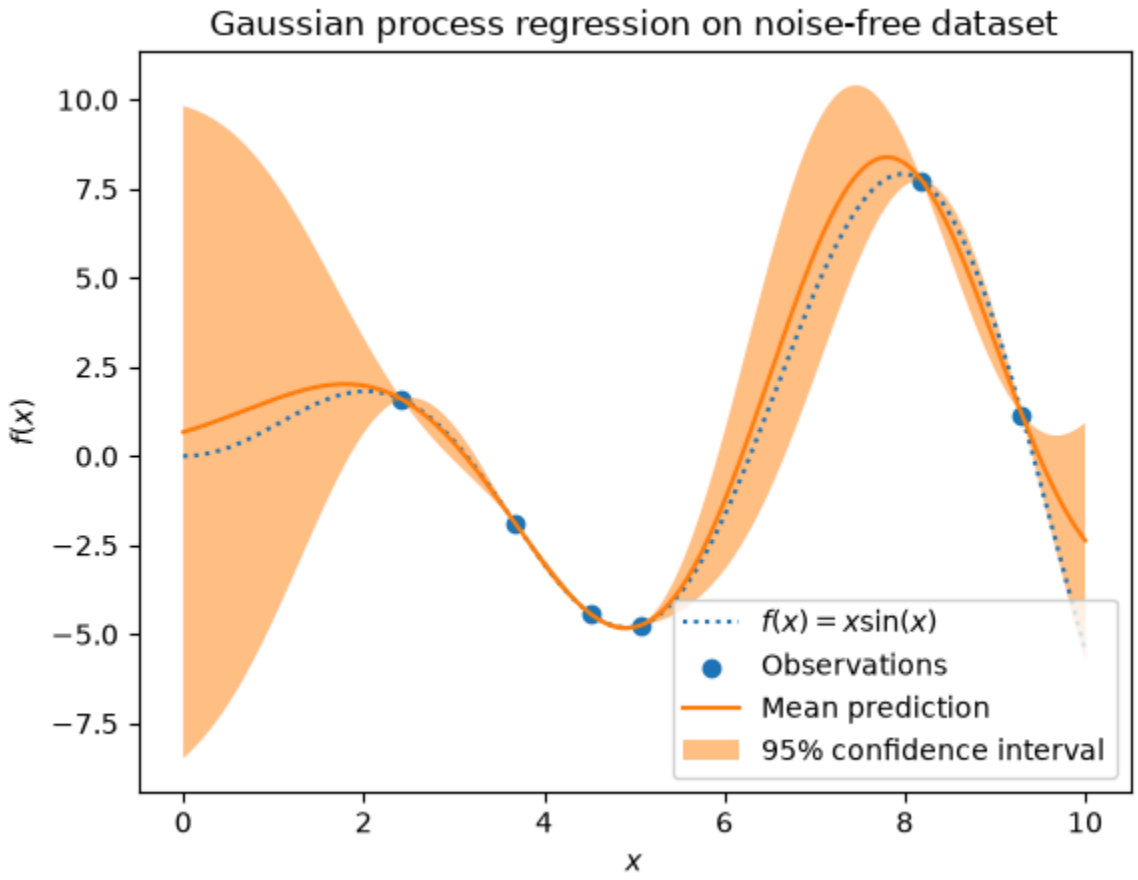
Now, we fit a Gaussian process on these few training data samples. We will use a radial basis function (RBF) kernel and a constant parameter to fit the amplitude.

▼ **Hide code cell source**

```
5.02**2 * RBF(length_scale=1.43)
```

After fitting our model, we see that the hyperparameters of the kernel have been optimized. Now, we will use our kernel to compute the mean prediction of the full dataset and plot the 95% confidence interval.

▼ **Hide code cell source**



We see that for a prediction made on a data point close to the one from the training set, the 95% confidence has a small amplitude. Whenever a sample falls far from training data, our model's prediction is less accurate and the model prediction is less precise (higher uncertainty).

Example with noisy targets

We can repeat a similar experiment adding an additional noise to the target this time. It will allow seeing the effect of the noise on the fitted model.

We add some random Gaussian noise to the target with an arbitrary standard deviation.

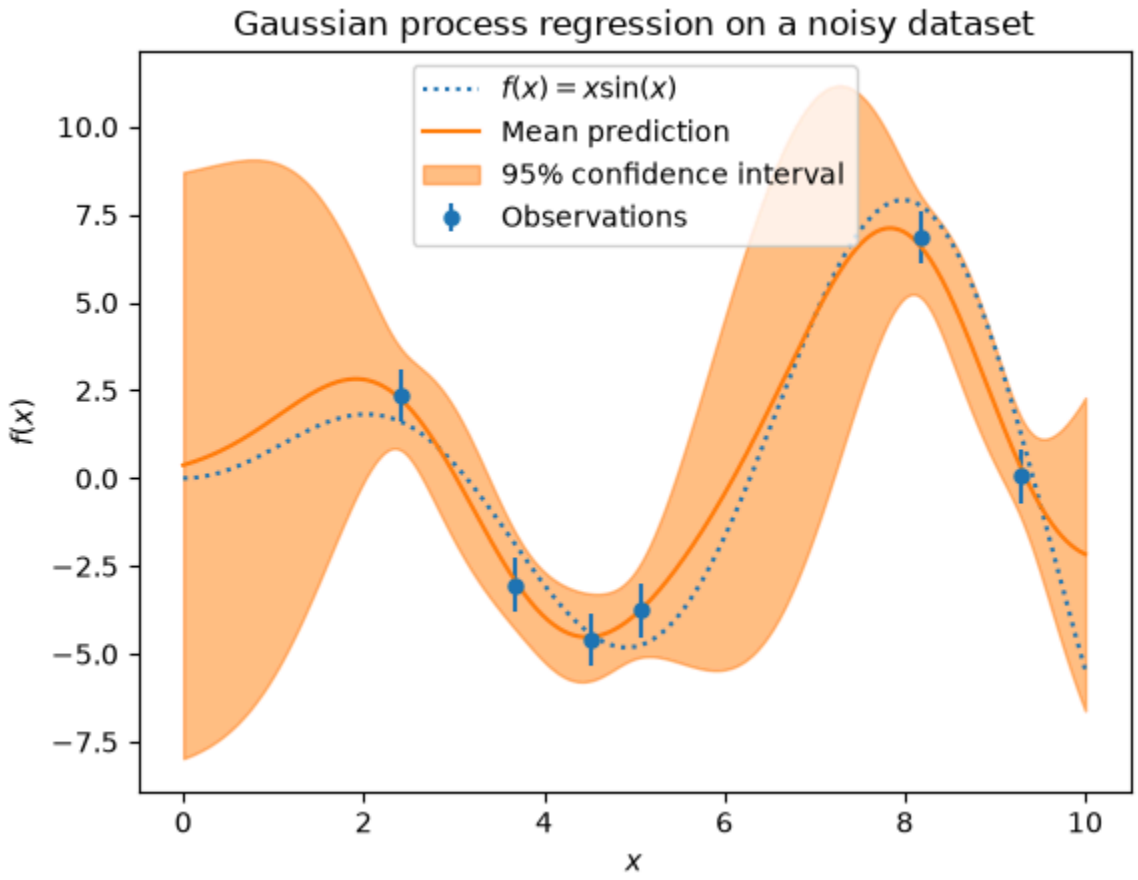
▼ **Hide code cell source**

We create a similar Gaussian process model. In addition to the kernel, this time, we specify the parameter `alpha` which can be interpreted as the variance of a Gaussian noise.

▼ **Hide code cell source**

Let's plot the mean prediction and the uncertainty region as before.

▼ **Hide code cell source**



The noise affects the predictions close to the training samples: the predictive uncertainty near to the training samples is larger because we explicitly model a given level target noise independent of the input variable.

Prior and posterior Gaussian process for different kernels

This example illustrates the prior and posterior of the class `GaussianProcessRegressor` with different kernels. Mean, standard deviation, and 5 samples are shown for both prior and posterior distributions.

Here, we only give some illustration. To know more about kernels' formulation, refer to the sklearn users guide.

```
# Authors: Jan Hendrik Metzen <jhm@informatik.uni-bremen.de>
#          Guillaume Lemaitre <g.lemaitre58@gmail.com>
# License: BSD 3 clause
```

Helper function

Before presenting each individual kernel available for Gaussian processes, we will define an helper function allowing us plotting samples drawn from the Gaussian process.

This function will take a `GaussianProcessRegressor` model and will drawn sample from the Gaussian process. If the model was not fit, the samples are drawn from the prior distribution while after model fitting, the samples are drawn from the posterior distribution.

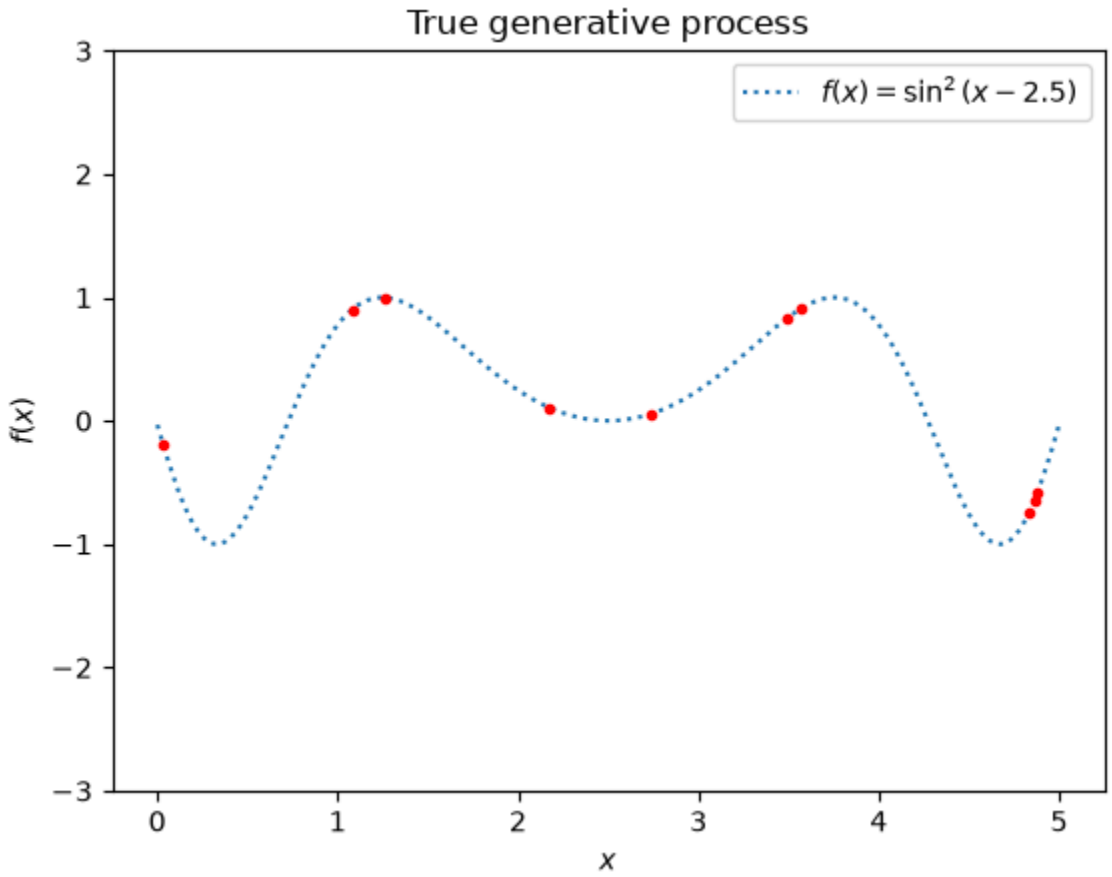
```
import matplotlib.pyplot as plt
import numpy as np


def plot_gpr_samples(gpr_model, n_samples, ax):
    """Plot samples drawn from the Gaussian process model.

    If the Gaussian process model is not trained then the drawn samples are
    drawn from the prior distribution. Otherwise, the samples are drawn from
    the posterior distribution. Be aware that a sample here corresponds to a
    function.

    Parameters
    ----------
    gpr_model : `GaussianProcessRegressor`
        A :class:`~sklearn.gaussian_process.GaussianProcessRegressor` model.
    n_samples : int
        The number of samples to draw from the Gaussian process distribution.
    ax : matplotlib axis
        The matplotlib axis where to plot the samples.
    """
    x = np.linspace(0, 5, 100)
    X = x.reshape(-1, 1)

    y_mean, y_std = gpr_model.predict(X, return_std=True)
    y_samples = gpr_model.sample_y(X, n_samples)

    for idx, single_prior in enumerate(y_samples.T):
        ax.plot(
            x,
            single_prior,
            linestyle="--",
            alpha=0.7,
            label=f"Sampled function #{idx + 1}",
        )
    ax.plot(x, y_mean, color="black", label="Mean")
    ax.fill_between(
        x,
        y_mean - y_std,
        y_mean + y_std,
        alpha=0.1,
        color="black",
        label=r"$\pm$ 1 std. dev.",
    )
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_ylim([-3, 3])
```

Dataset and Gaussian process generation

We will create a training dataset that we will use in the different sections.

```
rng = np.random.RandomState(4)
X_train = rng.uniform(0, 5, 10).reshape(-1, 1)
y_train = np.sin((X_train[:, 0] - 2.5) ** 2)
n_samples = 5
```

```
X_true = np.linspace(start=0, stop=5, num=1_000).reshape(-1, 1)
y_true = np.squeeze(np.sin((X_true - 2.5) ** 2))

plt.plot(X_true, y_true, label=r"$f(x) = \sin^2(x-2.5)$", linestyle="dotted")
plt.plot(X_train,y_train,'.', color='red')
plt.legend()
plt.xlabel("$x$")
plt.ylabel("$f(x)$")
plt.ylim(-3,3)
_ = plt.title("True generative process")
```

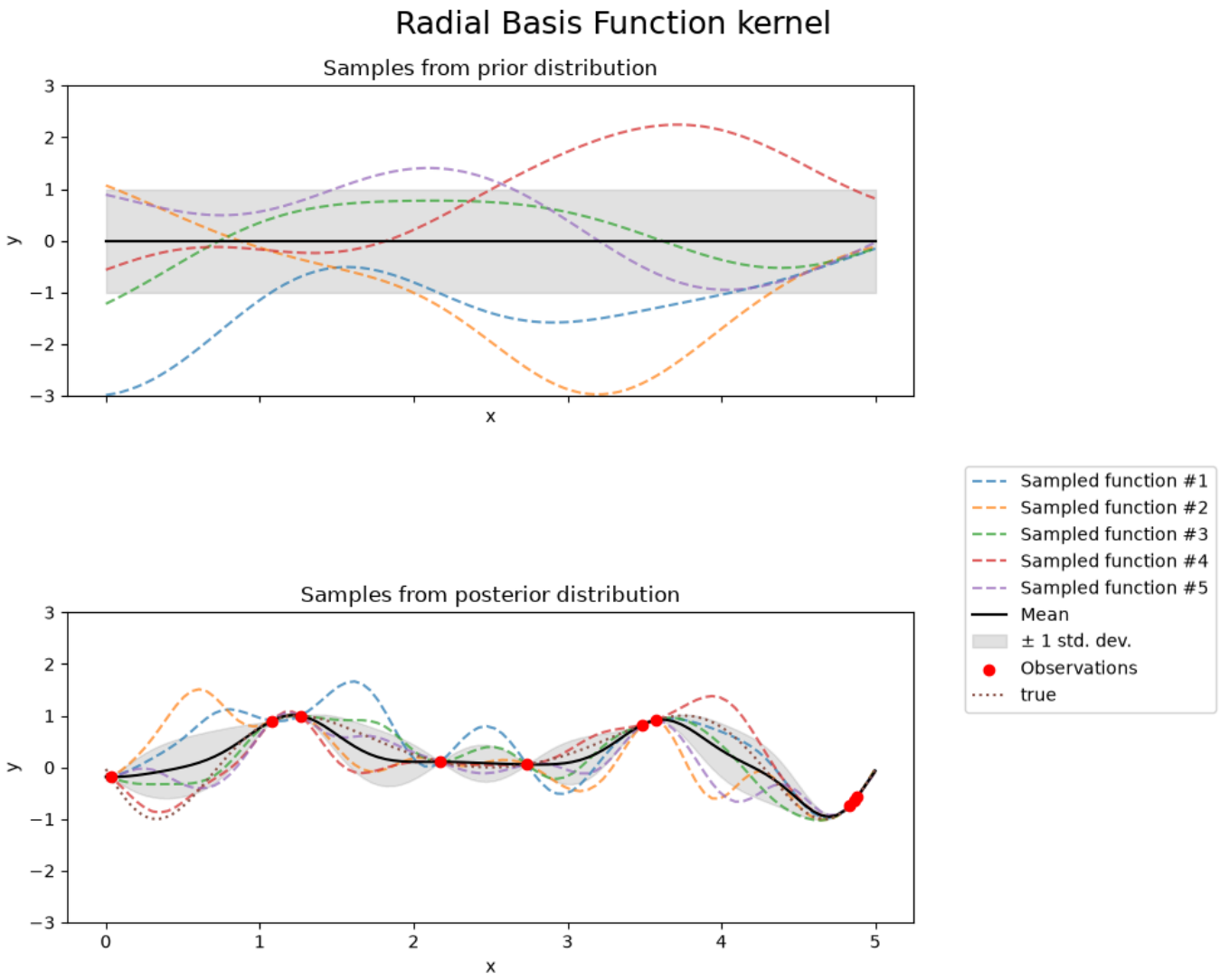


Kernel cookbook

In this section, we illustrate some samples drawn from the prior and posterior distributions of the Gaussian process with different kernels.

Radial Basis Function kernel

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF

kernel = 1.0 * RBF(length_scale=1.0, length_scale_bounds=(1e-1, 10.0))
gpr = GaussianProcessRegressor(kernel=kernel, random_state=0)

fig, axs = plt.subplots(nrows=2, sharex=True, sharey=True, figsize=(10, 8))

# plot prior
plot_gpr_samples(gpr, n_samples=n_samples, ax=axs[0])
axs[0].set_title("Samples from prior distribution")

# plot posterior
gpr.fit(X_train, y_train)
plot_gpr_samples(gpr, n_samples=n_samples, ax=axs[1])
axs[1].scatter(X_train[:, 0], y_train, color="red", zorder=10, label="Observations")
axs[1].plot(X_true, y_true, label="true", linestyle="dotted")
axs[1].legend(bbox_to_anchor=(1.05, 1.5), loc="upper left")
axs[1].set_title("Samples from posterior distribution")

fig.suptitle("Radial Basis Function kernel", fontsize=18)
plt.tight_layout()
```




```
print(f"Kernel parameters before fit:\n{kernel})")
print(
    f"Kernel parameters after fit: \n{gpr.kernel_} \n"
    f"Log-likelihood: {gpr.log_marginal_likelihood(gpr.kernel_.theta):.3f}"
)
```

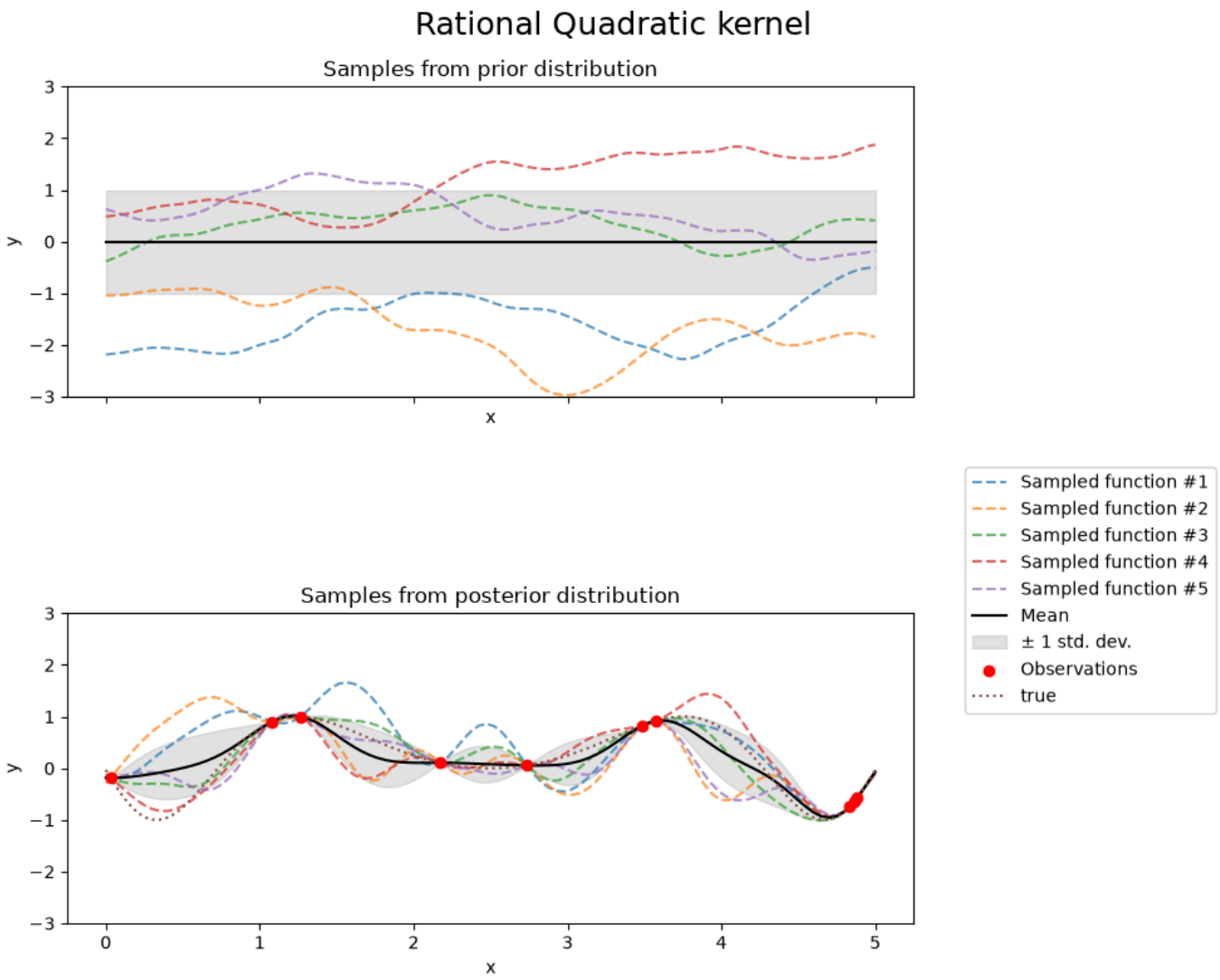
```
Kernel parameters before fit:
1**2 * RBF(length_scale=1))
Kernel parameters after fit:
0.594**2 * RBF(length_scale=0.279)
Log-likelihood: -0.067
```


```
Kernel parameters before fit:
1**2 * RBF(length_scale=1))
Kernel parameters after fit:
0.594**2 * RBF(length_scale=0.279)
Log-likelihood: -0.067
```

Rational Quadratic kernel

```
from sklearn.gaussian_process.kernels import RationalQuadratic

kernel = 1.0 * RationalQuadratic(length_scale=1.0, alpha=0.1, alpha_bounds=(1e-5, 1e15))
gpr = GaussianProcessRegressor(kernel=kernel, random_state=0)

fig, axs = plt.subplots(nrows=2, sharex=True, sharey=True, figsize=(10, 8))

# plot prior
plot_gpr_samples(gpr, n_samples=n_samples, ax=axs[0])
axs[0].set_title("Samples from prior distribution")

# plot posterior
gpr.fit(X_train, y_train)
plot_gpr_samples(gpr, n_samples=n_samples, ax=axs[1])
axs[1].scatter(X_train[:, 0], y_train, color="red", zorder=10, label="Observations")
axs[1].plot(X_true, y_true, label="true", linestyle="dotted")
axs[1].legend(bbox_to_anchor=(1.05, 1.5), loc="upper left")
axs[1].set_title("Samples from posterior distribution")

fig.suptitle("Rational Quadratic kernel", fontsize=18)
plt.tight_layout()
```

```
/usr/share/miniconda3/envs/2025-book-env/lib/python3.11/site-
packages/sklearn/gaussian_process/_gpr.py:530: RuntimeWarning: covariance is not symmetric
positive-semidefinite.
  y_samples = rng.multivariate_normal(y_mean, y_cov, n_samples).T
```



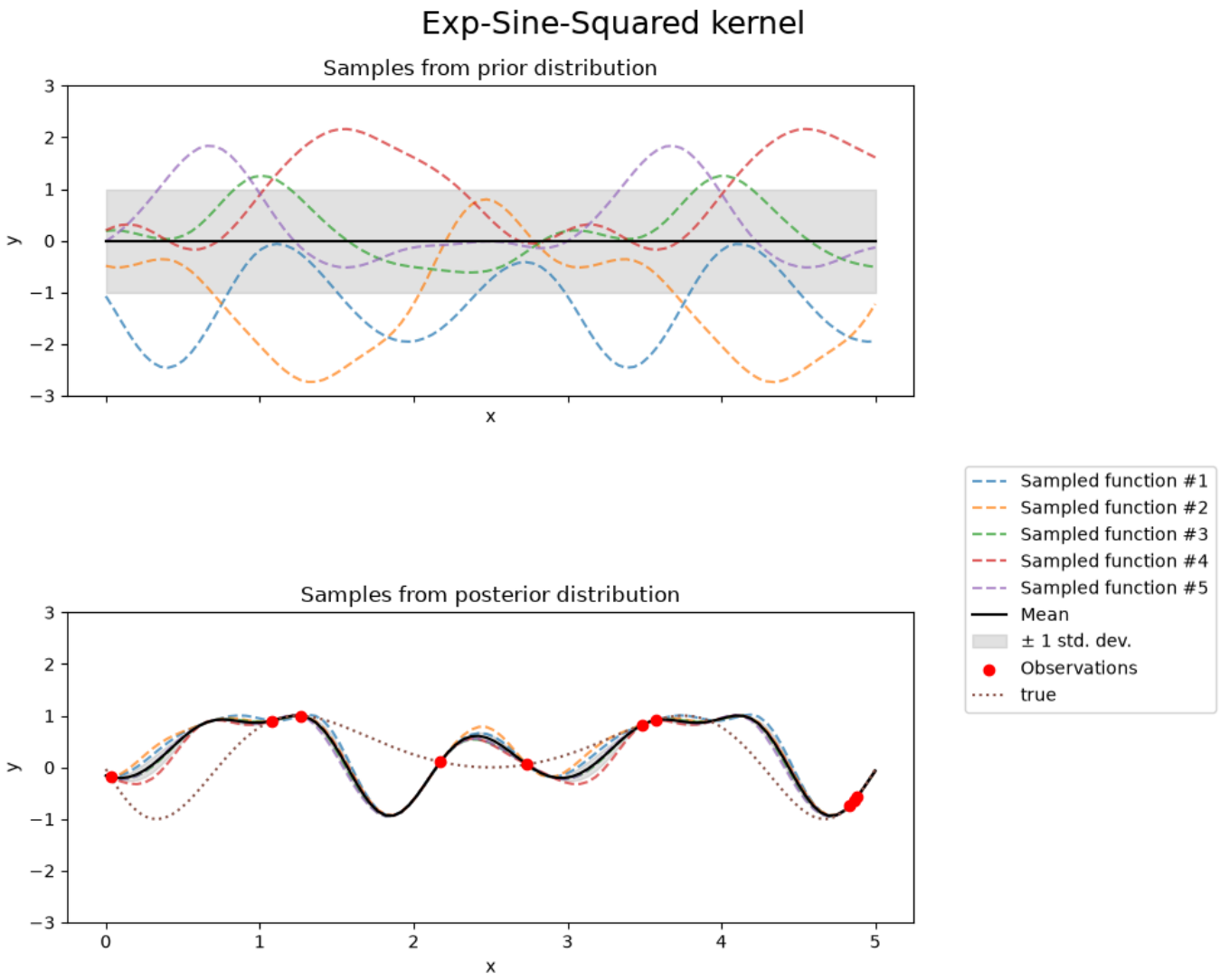

```
print(f"Kernel parameters before fit:\n{kernel})")
print(
    f"Kernel parameters after fit: \n{gpr.kernel_} \n"
    f"Log-likelihood: {gpr.log_marginal_likelihood(gpr.kernel_.theta):.3f}"
)
```

```
Kernel parameters before fit:
1**2 * RationalQuadratic(alpha=0.1, length_scale=1))
Kernel parameters after fit:
0.594**2 * RationalQuadratic(alpha=7.78e+07, length_scale=0.279)
Log-likelihood: -0.067
```

```
Kernel parameters before fit:
1**2 * RationalQuadratic(alpha=0.1, length_scale=1))
Kernel parameters after fit:
0.594**2 * RationalQuadratic(alpha=3.93e+05, length_scale=0.279)
Log-likelihood: -0.067
```

Exp-Sine-Squared kernel

```
from sklearn.gaussian_process.kernels import ExpSineSquared

kernel = 1.0 * ExpSineSquared(
    length_scale=1.0,
    periodicity=3.0,
    length_scale_bounds=(0.1, 10.0),
    periodicity_bounds=(1.0, 10.0),
)
gpr = GaussianProcessRegressor(kernel=kernel, random_state=0)

fig, axs = plt.subplots(nrows=2, sharex=True, sharey=True, figsize=(10, 8))

# plot prior
plot_gpr_samples(gpr, n_samples=n_samples, ax=axs[0])
axs[0].set_title("Samples from prior distribution")

# plot posterior
gpr.fit(X_train, y_train)
plot_gpr_samples(gpr, n_samples=n_samples, ax=axs[1])
axs[1].scatter(X_train[:, 0], y_train, color="red", zorder=10, label="Observations")
axs[1].plot(X_true, y_true, label="true", linestyle="dotted")
axs[1].legend(bbox_to_anchor=(1.05, 1.5), loc="upper left")
axs[1].set_title("Samples from posterior distribution")

fig.suptitle("Exp-Sine-Squared kernel", fontsize=18)
plt.tight_layout()
```




```
print(f"Kernel parameters before fit:\n{kernel})")
print(
    f"Kernel parameters after fit: \n{gpr.kernel_} \n"
    f"Log-likelihood: {gpr.log_marginal_likelihood(gpr.kernel_.theta):.3f}"
)
```

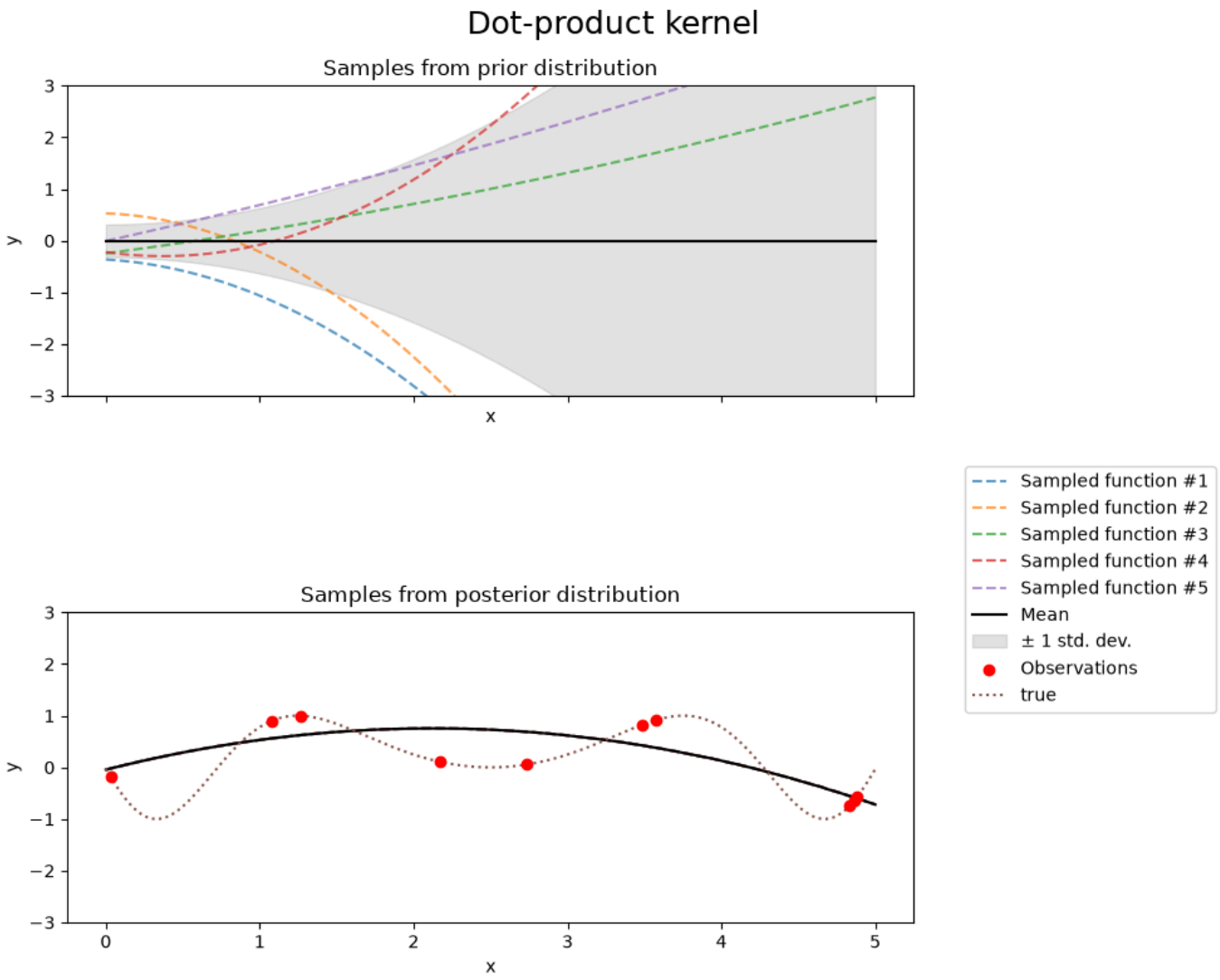
```
Kernel parameters before fit:
1**2 * ExpSineSquared(length_scale=1, periodicity=3))
Kernel parameters after fit:
0.799**2 * ExpSineSquared(length_scale=0.791, periodicity=2.87)
Log-likelihood: 3.394
```


```
Kernel parameters before fit:
1**2 * ExpSineSquared(length_scale=1, periodicity=3))
Kernel parameters after fit:
0.799**2 * ExpSineSquared(length_scale=0.791, periodicity=2.87)
Log-likelihood: 3.394
```

Dot-product kernel

```
from sklearn.gaussian_process.kernels import ConstantKernel, DotProduct

kernel = ConstantKernel(0.1, (0.01, 10.0)) * (
    DotProduct(sigma_0=1.0, sigma_0_bounds=(0.1, 10.0)) ** 2
)
gpr = GaussianProcessRegressor(kernel=kernel, random_state=0)

fig, axs = plt.subplots(nrows=2, sharex=True, sharey=True, figsize=(10, 8))

# plot prior
plot_gpr_samples(gpr, n_samples=n_samples, ax=axs[0])
axs[0].set_title("Samples from prior distribution")

# plot posterior
gpr.fit(X_train, y_train)
plot_gpr_samples(gpr, n_samples=n_samples, ax=axs[1])
axs[1].scatter(X_train[:, 0], y_train, color="red", zorder=10, label="Observations")
axs[1].plot(X_true, y_true, label="true", linestyle="dotted")
axs[1].legend(bbox_to_anchor=(1.05, 1.5), loc="upper left")
axs[1].set_title("Samples from posterior distribution")

fig.suptitle("Dot-product kernel", fontsize=18)
plt.tight_layout()
```

```
/usr/share/miniconda3/envs/2025-book-env/lib/python3.11/site-
packages/sklearn/gaussian_process/_gpr.py:667: ConvergenceWarning: lbfgs failed to converge
after 2 iteration(s) (status=2):
ABNORMAL:

You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
  _check_optimize_result("lbfgs", opt_res)
```



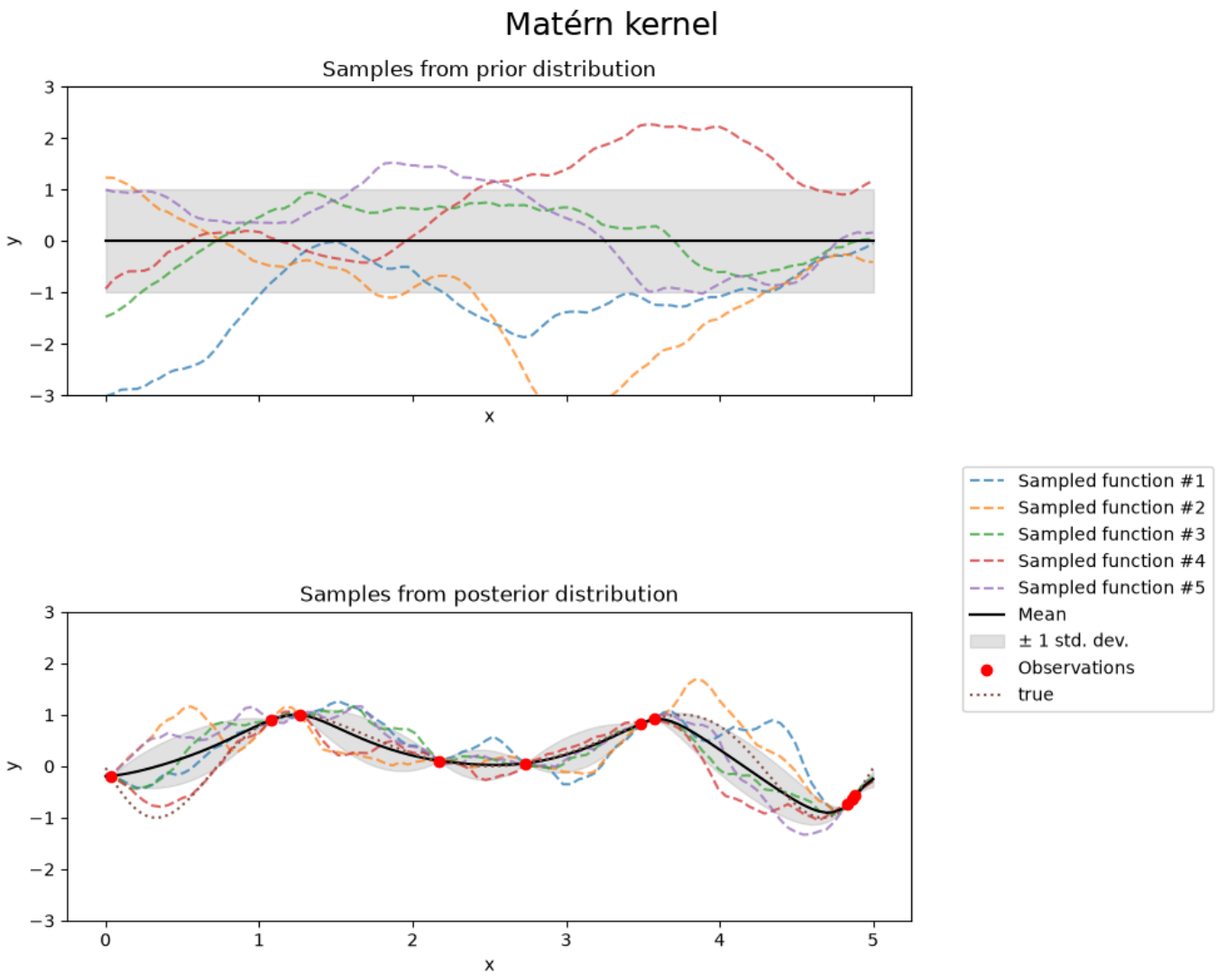

```
print(f"Kernel parameters before fit:\n{kernel})")
print(
    f"Kernel parameters after fit: \n{gpr.kernel_} \n"
    f"Log-likelihood: {gpr.log_marginal_likelihood(gpr.kernel_.theta):.3f}"
)
```

```
Kernel parameters before fit:
0.316**2 * DotProduct(sigma_0=1) ** 2)
Kernel parameters after fit:
2.36**2 * DotProduct(sigma_0=0.134) ** 2
Log-likelihood: -7943459458.877
```

```
Kernel parameters before fit:
0.316**2 * DotProduct(sigma_0=1) ** 2)
Kernel parameters after fit:
0.997**2 * DotProduct(sigma_0=10) ** 2
Log-likelihood: -7858765344.362
```

Matérn kernel

```
from sklearn.gaussian_process.kernels import Matern

kernel = 1.0 * Matern(length_scale=1.0, length_scale_bounds=(1e-1, 10.0), nu=1.5)
gpr = GaussianProcessRegressor(kernel=kernel, random_state=0)

fig, axs = plt.subplots(nrows=2, sharex=True, sharey=True, figsize=(10, 8))

# plot prior
plot_gpr_samples(gpr, n_samples=n_samples, ax=axs[0])
axs[0].set_title("Samples from prior distribution")

# plot posterior
gpr.fit(X_train, y_train)
plot_gpr_samples(gpr, n_samples=n_samples, ax=axs[1])
axs[1].scatter(X_train[:, 0], y_train, color="red", zorder=10, label="Observations")
axs[1].plot(X_true, y_true, label="true", linestyle="dotted")
axs[1].legend(bbox_to_anchor=(1.05, 1.5), loc="upper left")
axs[1].set_title("Samples from posterior distribution")

fig.suptitle("Matérn kernel", fontsize=18)
plt.tight_layout()
```



```
print(f"Kernel parameters before fit:\n{kernel})")
print(
    f"Kernel parameters after fit: \n{gpr.kernel_} \n"
    f"Log-likelihood: {gpr.log_marginal_likelihood(gpr.kernel_.theta):.3f}"
)
```

```
Kernel parameters before fit:
1**2 * Matern(length_scale=1, nu=1.5))
Kernel parameters after fit:
0.609**2 * Matern(length_scale=0.484, nu=1.5)
Log-likelihood: -1.185
```


```
Kernel parameters before fit:
1**2 * Matern(length_scale=1, nu=1.5))
Kernel parameters after fit:
0.609**2 * Matern(length_scale=0.484, nu=1.5)
Log-likelihood: -1.185
```


Demonstration: Gaussian Process Regression

This notebook uses **scikit-learn** with the workflow:

- build an RBF-based kernel (with signal scale and noise term),
- fit the GP on a subset (e.g., every 3rd point),
- predict mean and uncertainty on a target grid or the full input,
- plot mean ±2σ and data,
- compute simple validation metrics.

```
# Imports
import numpy as np
import matplotlib.pyplot as plt

from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C, WhiteKernel

np.random.seed(1234)
```

```
x = np.linspace(0, 10, 60)
f = np.sin(x) + 0.2*np.cos(3*x)
y = f + 0.2*np.random.randn(x.size)

# Ensure shapes (n,1) and (n,)
X = np.asarray(x).reshape(-1, 1)
y = np.asarray(y).reshape(-1)
print('Data shapes -> X:', X.shape, ' y:', y.shape)
```

```
Data shapes -> X: (60, 1)  y: (60,)
```

```
# --- Train/validation split: every 3rd point for train ---
idx = np.arange(X.shape[0])
train_mask = (idx % 3 == 0)
test_mask  = ~train_mask

X_train, y_train = X[train_mask], y[train_mask]
X_test,  y_test  = X[test_mask],  y[test_mask]

print('Train size:', X_train.shape[0], ' Test size:', X_test.shape[0])
```

```
Train size: 20  Test size: 40
```

```
# --- Kernel: signal variance * RBF(length_scale) + white noise ---
kernel = C(1.0, (1e-3, 1e3)) * RBF(length_scale=1.0, length_scale_bounds=(1e-1, 1e3)) +
WhiteKernel(noise_level=1e-3, noise_level_bounds=(1e-6, 1e1))

gpr = GaussianProcessRegressor(kernel=kernel, normalize_y=True, random_state=1234,
n_restarts_optimizer=9)
print('Initial kernel:', gpr.kernel)
```

```
Initial kernel: 1**2 * RBF(length_scale=1) + WhiteKernel(noise_level=0.001)
```

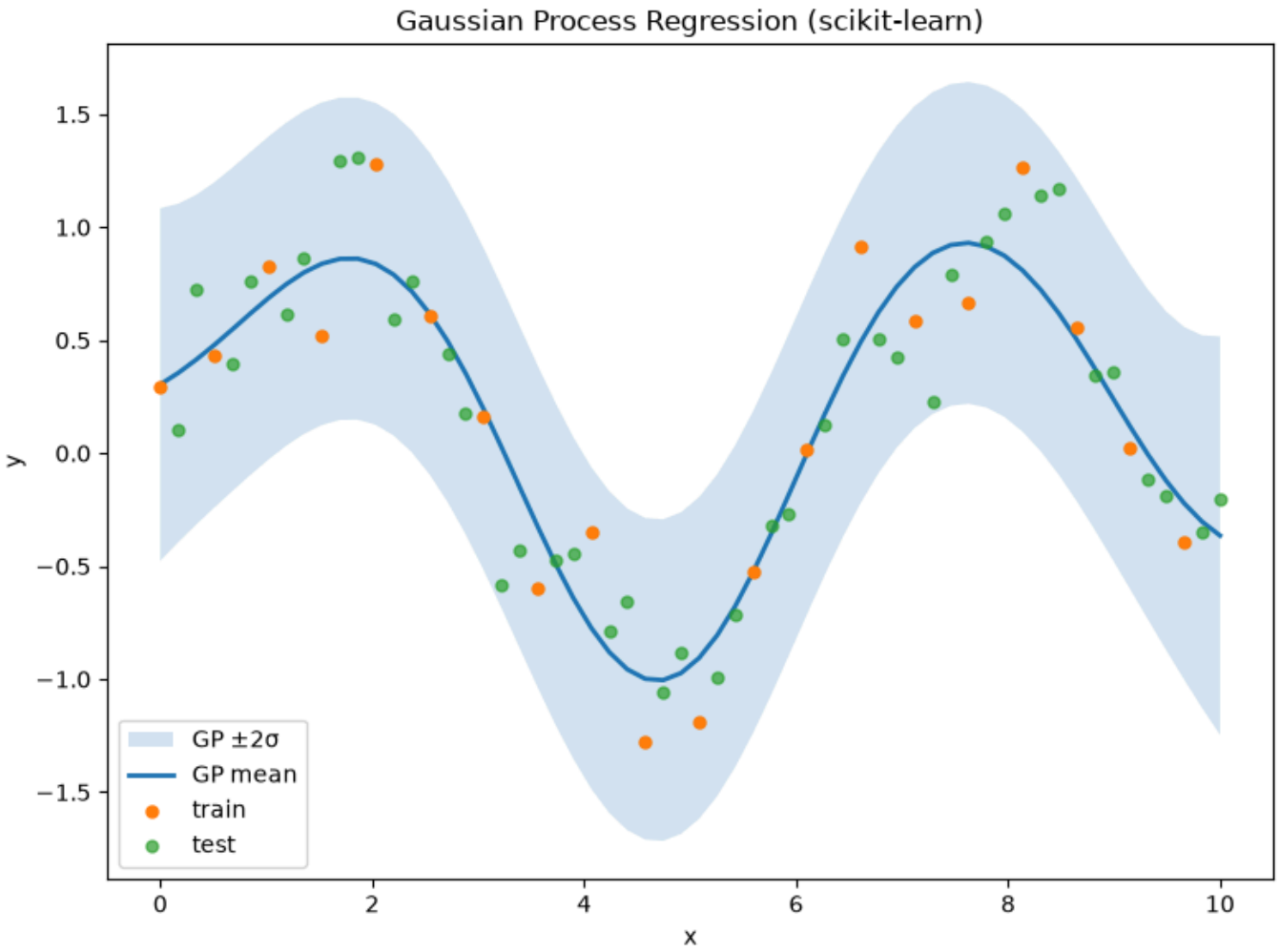
```
# --- Fit (replaces GPy: m.optimize(...)) ---
gpr.fit(X_train, y_train)
print('\nOptimized kernel:', gpr.kernel_)
```

```
Optimized kernel: 1.03**2 * RBF(length_scale=1.36) + WhiteKernel(noise_level=0.193)
```

```
# --- Predict on full X (replaces: yp, vp = m.predict(xp)) ---
y_mean, y_std = gpr.predict(X, return_std=True)

# For test set metrics
from sklearn.metrics import r2_score, mean_absolute_percentage_error
y_pred_test, y_std_test = gpr.predict(X_test, return_std=True)
print('\nValidation:')
print('  R^2  (test):', r2_score(y_test, y_pred_test))
try:
    print('  MAPE (test):', mean_absolute_percentage_error(y_test, y_pred_test))
except Exception:
    pass
```

```
Validation:
  R^2  (test): 0.8596309559662767
  MAPE (test): 0.44632613843402247
```

```
# --- Plot (replaces: m.plot(...)) ---
plt.figure(figsize=(8,6))

# 95% confidence band
plt.fill_between(X.ravel(),
                 y_mean - 2*y_std,
                 y_mean + 2*y_std,
                 alpha=0.2, label='GP ±2σ')

plt.plot(X.ravel(), y_mean, lw=2, label='GP mean')
plt.scatter(X_train.ravel(), y_train, s=25, label='train', zorder=3)
plt.scatter(X_test.ravel(),  y_test,  s=25, label='test',  alpha=0.7, zorder=2)

plt.xlabel('x')
plt.ylabel('y')
plt.title('Gaussian Process Regression (scikit-learn)')
plt.legend()
plt.tight_layout()
plt.show()
```



Exercise: Gaussian Processes

This exercise uses scikit-learn and covers:

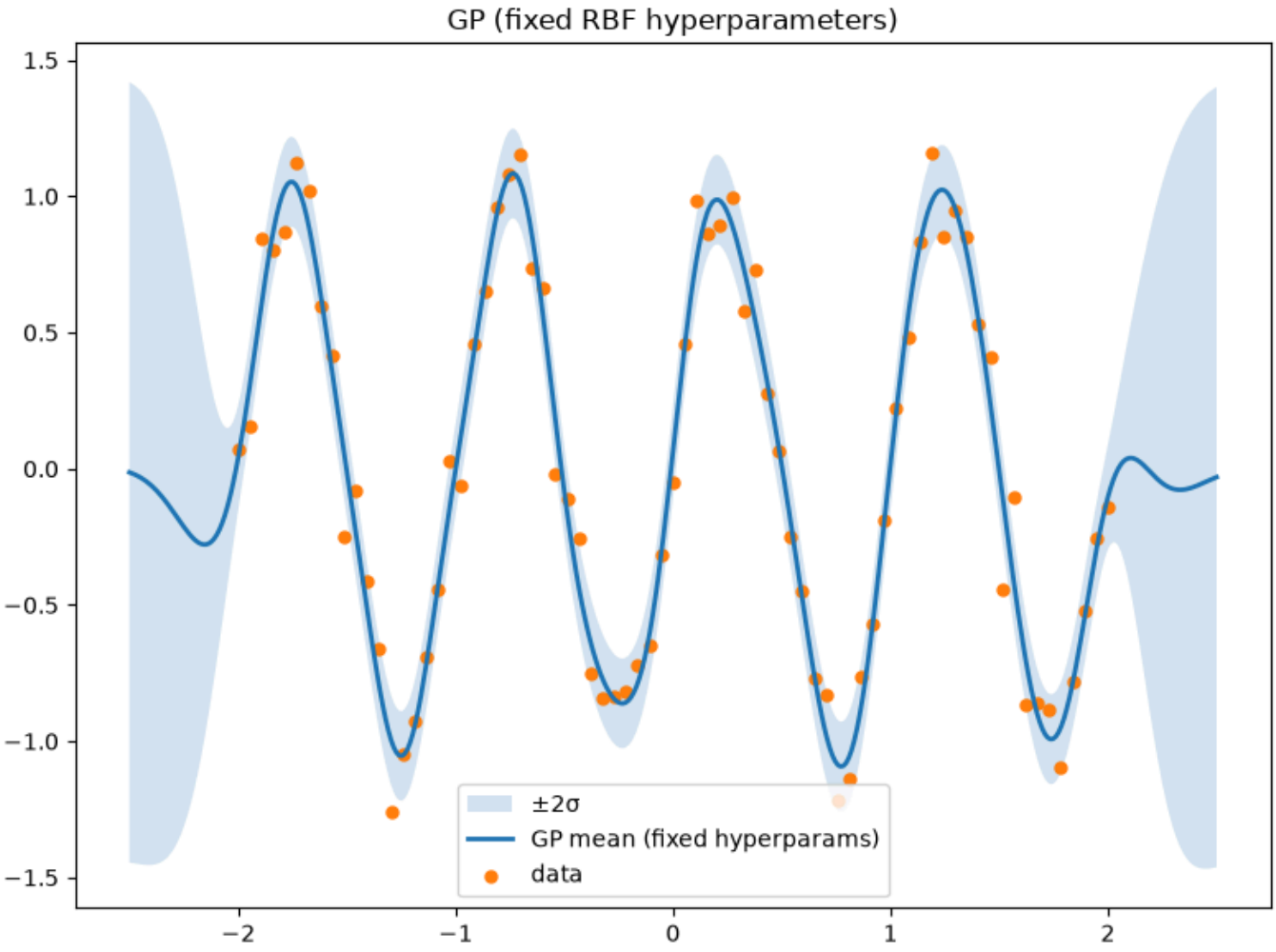
- building RBF kernels with signal variance and length-scale,
- fitting `GaussianProcessRegressor` with a white-noise term,
- predicting posterior mean and uncertainty,
- plotting mean ±2σ and data,
- setting hyperparameters explicitly vs. optimizing by LML.

```
# Imports
import numpy as np
import matplotlib.pyplot as plt

from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C, WhiteKernel
np.random.seed(1234)
```

```
# Synthetic 1D regression data (adapter-friendly)
# If your upstream notebook already defines X,y, this block can be skipped or adapted.
n = 75
X = np.linspace(-2, 2, n).reshape(-1,1)
f = np.sin(2*np.pi*X).ravel()
y = f + 0.15*np.random.randn(n)

print('Data shapes:', X.shape, y.shape)
```

```
Data shapes: (75, 1) (75,)
```

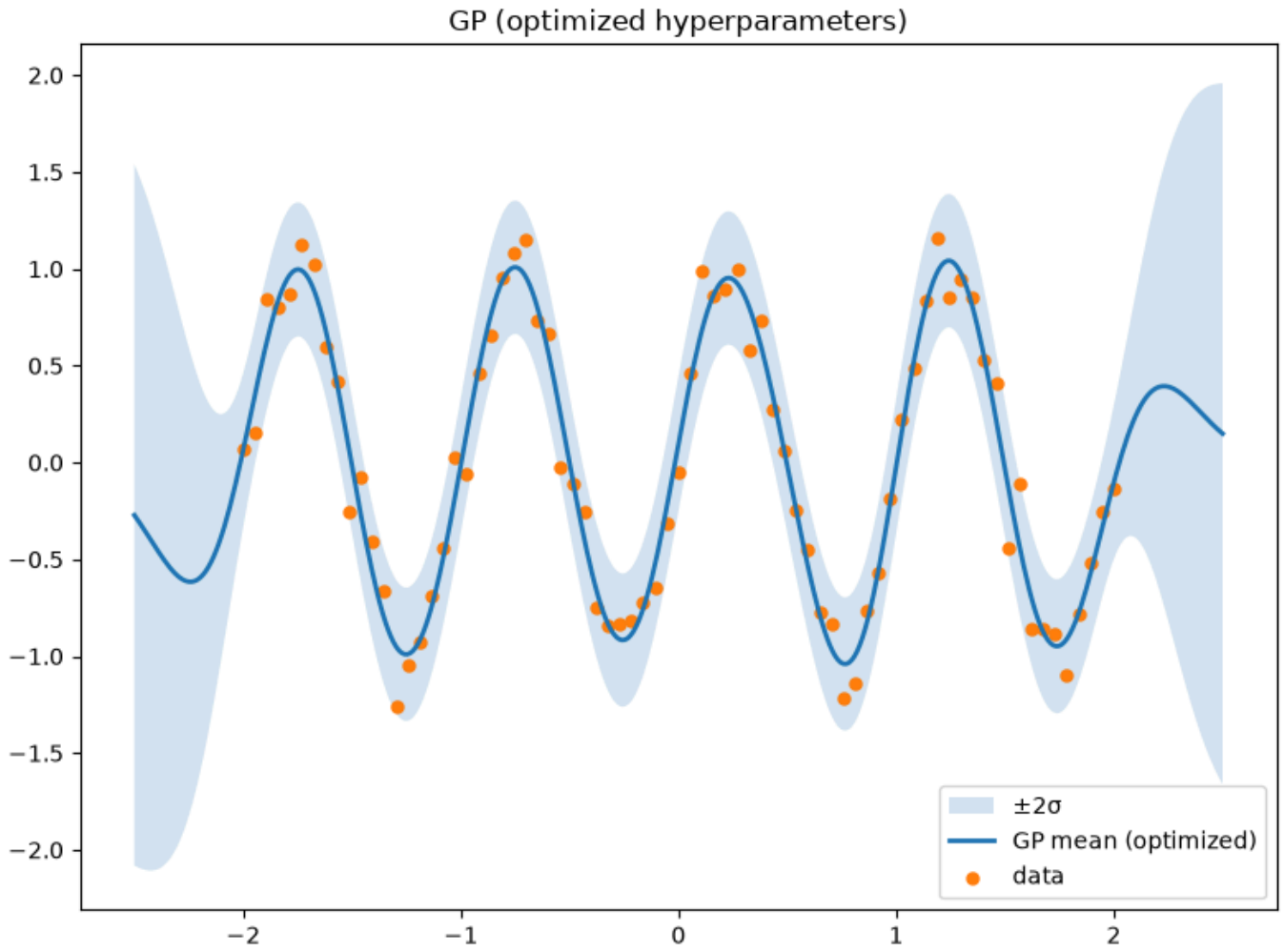
```
# Fixed hyperparameters (like setting m['rbf.lengthscale']=...,
m['Gaussian_noise.variance']=...)
sigma_f2 = 1.0
ell = 0.2
sigma_n2 = 1e-2

kernel_fixed = C(sigma_f2, constant_value_bounds='fixed') * RBF(length_scale=ell,
length_scale_bounds='fixed') \
               + WhiteKernel(noise_level=sigma_n2, noise_level_bounds='fixed')

gpr_fixed = GaussianProcessRegressor(kernel=kernel_fixed, normalize_y=True, optimizer=None)  #
no optimization
gpr_fixed.fit(X, y)

Xg = np.linspace(X.min()-0.5, X.max()+0.5, 400).reshape(-1,1)
m_fixed, s_fixed = gpr_fixed.predict(Xg, return_std=True)

plt.figure(figsize=(8,6))
plt.fill_between(Xg.ravel(), m_fixed-2*s_fixed, m_fixed+2*s_fixed, alpha=0.2, label='±2σ')
plt.plot(Xg, m_fixed, lw=2, label='GP mean (fixed hyperparams)')
plt.scatter(X, y, s=25, label='data')
plt.title('GP (fixed RBF hyperparameters)')
plt.legend(); plt.tight_layout(); plt.show()
```




```
# Optimized hyperparameters (analog of m.optimize())
kernel0 = C(1.0, (1e-3, 1e3)) * RBF(length_scale=0.5, length_scale_bounds=(1e-3, 1e3)) \
          + WhiteKernel(noise_level=1e-2, noise_level_bounds=(1e-6, 1e1))

gpr = GaussianProcessRegressor(kernel=kernel0, normalize_y=True, n_restarts_optimizer=5,
random_state=1234)
print('Initial kernel:', gpr.kernel)

gpr.fit(X, y)
print('\nOptimized kernel:', gpr.kernel_)

m_opt, s_opt = gpr.predict(Xg, return_std=True)

plt.figure(figsize=(8,6))
plt.fill_between(Xg.ravel(), m_opt-2*s_opt, m_opt+2*s_opt, alpha=0.2, label='±2σ')
plt.plot(Xg, m_opt, lw=2, label='GP mean (optimized)')
plt.scatter(X, y, s=25, label='data')
plt.title('GP (optimized hyperparameters)')
plt.legend(); plt.tight_layout(); plt.show()
```

```
Initial kernel: 1**2 * RBF(length_scale=0.5) + WhiteKernel(noise_level=0.01)
```

```
Optimized kernel: 1.31**2 * RBF(length_scale=0.277) + WhiteKernel(noise_level=0.0475)
```




```python
# Optional: full predictive covariance
m_cov, K_cov = gpr.predict(Xg, return_cov=True)
print('Predictive covariance shape:', K_cov.shape)
```

```
Predictive covariance shape: (400, 400)
```

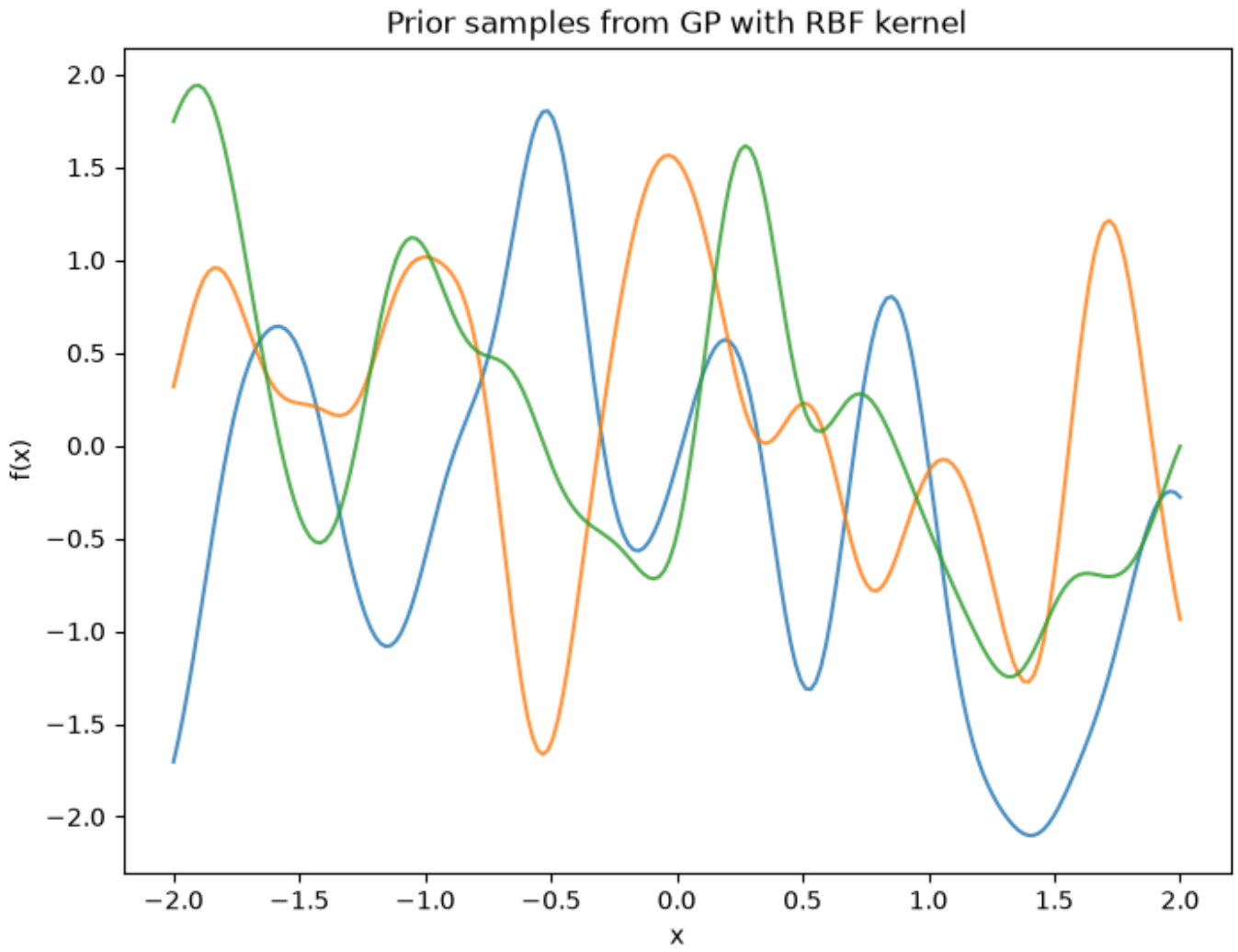
Gaussian Processes Exercises

Tasks include:

- building RBF kernels and visualizing samples,
- fitting a GP to 1D data (train/test split),
- plotting the posterior mean and ±2σ band,
- applying to a small dataset.

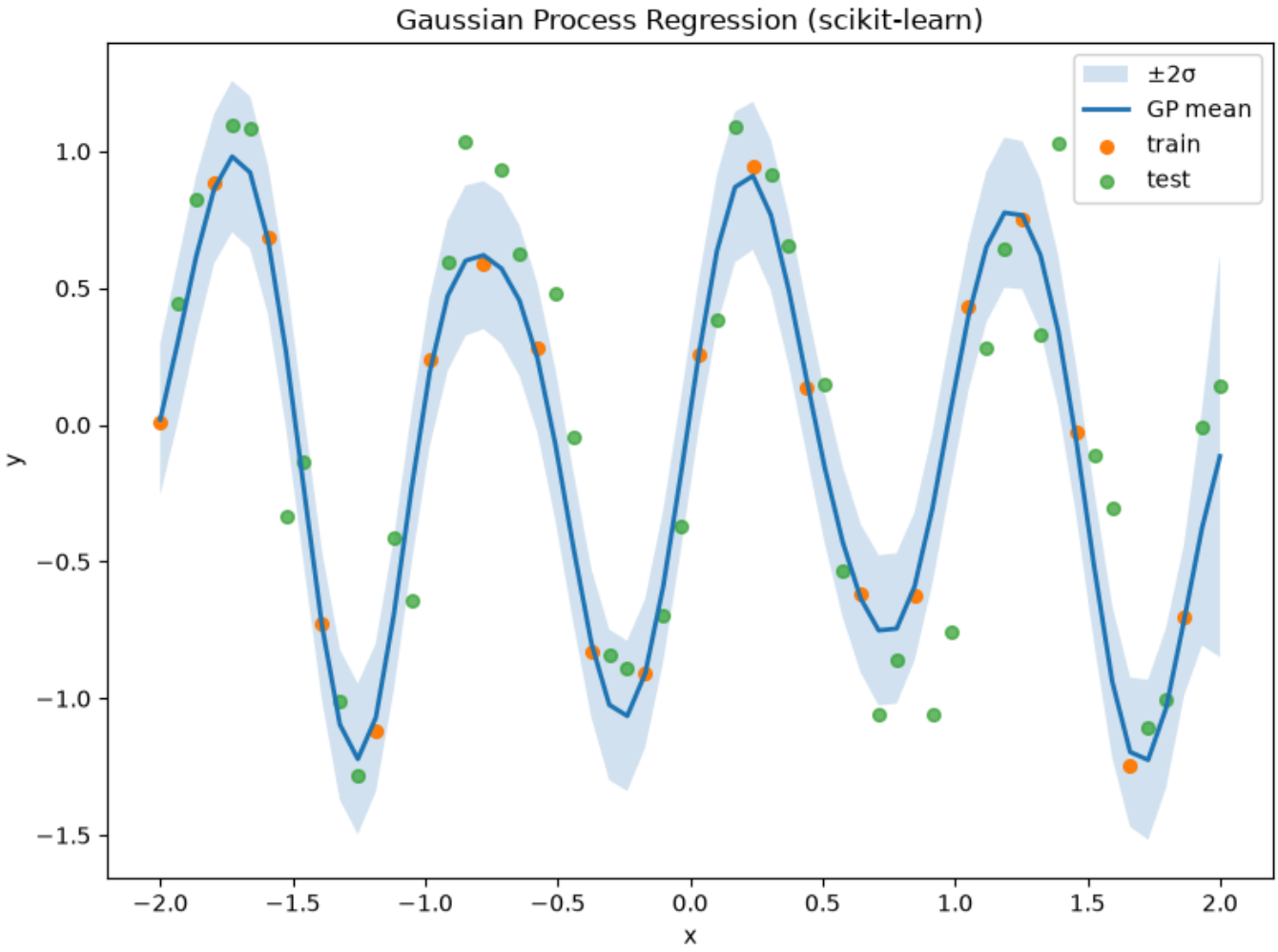
```
# Imports
import numpy as np
import matplotlib.pyplot as plt

from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C, WhiteKernel
from sklearn.metrics import r2_score, mean_absolute_error

np.random.seed(1234)
```

```
# --- Kernel construction & draws from the prior ---
n = 200
X_demo = np.linspace(-2, 2, n).reshape(-1,1)
var, theta = 1.0, 0.2
kernel_demo = C(var) * RBF(length_scale=theta)

# Prior samples: draw f ~ GP(0, K) on the grid
from sklearn.metrics.pairwise import rbf_kernel
K = var * rbf_kernel(X_demo, X_demo, gamma=1.0/(2*theta**2))
samples = np.random.multivariate_normal(mean=np.zeros(n), cov=K + 1e-9*np.eye(n), size=3)

plt.figure(figsize=(8,6))
plt.plot(X_demo, samples.T, alpha=0.8)
plt.title("Prior samples from GP with RBF kernel")
plt.xlabel("x"); plt.ylabel("f(x)")
plt.show()
```


Prior samples from GP with RBF kernel
f(x)
x
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

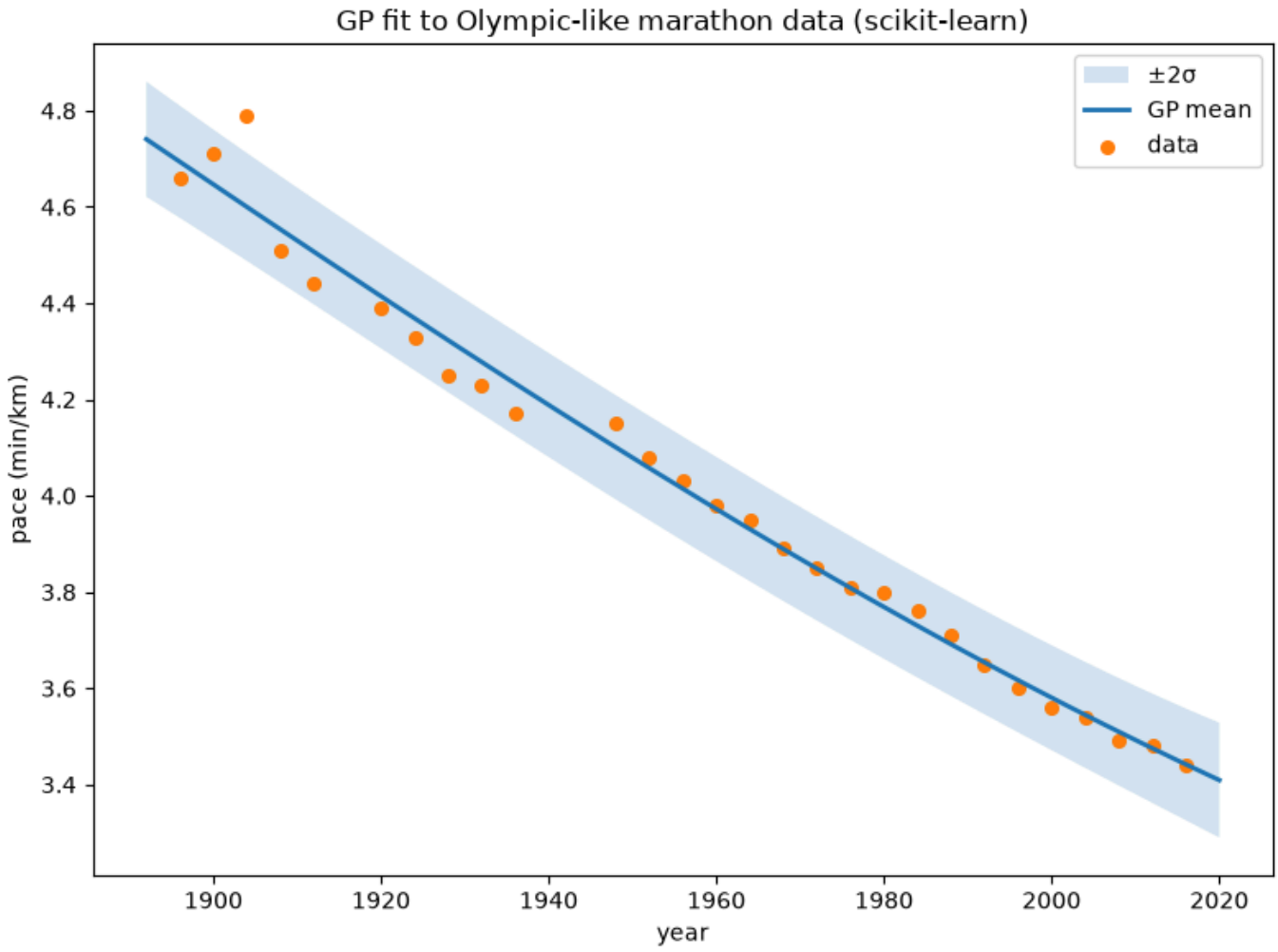
```
# --- Fit GP on subset and predict on full grid ---
X = np.linspace(-2, 2, 60).reshape(-1,1)
f = np.sin(2*np.pi*X).ravel()
y = f + 0.25*np.random.randn(X.shape[0])

# Train/test split: every 3rd point for training to mimic original
idx = np.arange(X.shape[0])
mask_tr = (idx % 3 == 0)
mask_te = ~mask_tr
X_tr, y_tr = X[mask_tr], y[mask_tr]
X_te, y_te = X[mask_te], y[mask_te]

# Kernel: signal * RBF + noise
kernel = C(1.0, (1e-3, 1e3)) * RBF(length_scale=1.0, length_scale_bounds=(1e-3, 1e3)) +
WhiteKernel(noise_level=1e-2, noise_level_bounds=(1e-6, 1e1))

gpr = GaussianProcessRegressor(kernel=kernel, normalize_y=True, n_restarts_optimizer=9,
random_state=1234)
print("Initial kernel:", gpr.kernel)

gpr.fit(X_tr, y_tr)
print("\nOptimized kernel:", gpr.kernel_)

# Predict on full X (posterior mean/std)
y_mean, y_std = gpr.predict(X, return_std=True)

# Evaluate on held-out points
y_pred_te, y_std_te = gpr.predict(X_te, return_std=True)
print("\nTest R^2:", r2_score(y_te, y_pred_te))
print("Test MAE:", mean_absolute_error(y_te, y_pred_te))

# Plot
plt.figure(figsize=(8,6))
plt.fill_between(X.ravel(), y_mean-2*y_std, y_mean+2*y_std, alpha=0.2, label="±2σ")
plt.plot(X, y_mean, lw=2, label="GP mean")
plt.scatter(X_tr, y_tr, s=30, label="train")
plt.scatter(X_te, y_te, s=30, label="test", alpha=0.7)
plt.xlabel("x"); plt.ylabel("y")
plt.title("Gaussian Process Regression (scikit-learn)")
plt.legend(); plt.tight_layout(); plt.show()
```

```
Initial kernel: 1**2 * RBF(length_scale=1) + WhiteKernel(noise_level=0.01)



Optimized kernel: 1.12**2 * RBF(length_scale=0.22) + WhiteKernel(noise_level=0.0204)

Test R^2: 0.7846195170516508
Test MAE: 0.2832273891509585
```


Gaussian Process Regression (scikit-learn)
±2σ
GP mean
train
test
1.0
0.5
0.0
−0.5
−1.0
−1.5
y
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
x

```
# --- Example dataset  ---

# Small inline sample approximating a downward trend (year vs pace)
years =
np.array([1896,1900,1904,1908,1912,1920,1924,1928,1932,1936,1948,1952,1956,1960,1964,1968,1972,197
pace  =
np.array([4.66,4.71,4.79,4.51,4.44,4.39,4.33,4.25,4.23,4.17,4.15,4.08,4.03,3.98,3.95,3.89,3.85,3.8
Xo, Yo = years, pace

kernel_o = C(1.0, (1e-3, 1e3)) * RBF(length_scale=20.0, length_scale_bounds=(1e-1, 1e4))
+ WhiteKernel(noise_level=1e-3, noise_level_bounds=(1e-6, 1e1))
gpr_o = GaussianProcessRegressor(kernel=kernel_o, normalize_y=True, n_restarts_optimizer=5,
random_state=1234)

gpr_o.fit(Xo, Yo)
print("Optimized kernel (Olympic-like):", gpr_o.kernel_)

# Prediction grid
Xg = np.linspace(Xo.min()-4, Xo.max()+4, 400).reshape(-1,1)
Ym, Ys = gpr_o.predict(Xg, return_std=True)

plt.figure(figsize=(8,6))
plt.fill_between(Xg.ravel(), Ym-2*Ys, Ym+2*Ys, alpha=0.2, label="±2σ")
plt.plot(Xg, Ym, lw=2, label="GP mean")
plt.scatter(Xo, Yo, s=30, label="data")
plt.xlabel("year"); plt.ylabel("pace (min/km)")
plt.title("GP fit to Olympic-like marathon data (scikit-learn)")
plt.legend(); plt.tight_layout(); plt.show()
```

```
Optimized kernel (Olympic-like): 5.22**2 * RBF(length_scale=277) +
WhiteKernel(noise_level=0.0179)
```



33. ANNs in the large-width limit

Machine learning methods such as artificial neural networks (ANNs) are offering physicists new ways to explore and understand a wide range of physics systems, as well as improving existing solution methods [MBW+19]. ANNs are commonly treated as black boxes that are empirically optimized. Given their growing prominence as a tool for physics research, it is desirable to have a framework that allows for a more structured analysis based on an understanding of how they work. Several authors have proposed a field theory approach to analyze and optimize neural networks using a combination of methods from quantum field theory (QFT) and Bayesian statistics [Hal21, HMS21, Rob21, RYH22]. It is based on expanding about the large-width limit of ANNs and is known as ANNFT.

Dan Roberts, in an essay entitled “Why is AI hard and Physics simple?” [Rob21], claims that the principle of *sparsity* means that methods of theoretical physics and associated physical intuition can be powerful in understanding machine learning. Roberts interprets Wigner’s observations about “The Unreasonable Effectiveness of Mathematics in the Natural Sciences” as “the laws of physics have an (unreasonable?) lack of algorithmic complexity”. The key idea is that many neural network architectures (including the common ones) have a well-defined limit when the network width (which is the number of neurons in each layer) is taken to infinity. In particular, they reduce to Gaussian processes (GPs), and with gradient-based training they evolve in a clear way as linear models according to the so-called neural tangent kernel (or NTK).

This infinite width limit by itself is not an accurate model for actual deep-learning networks, because it is too limited in what it can learn from the data. But there is a way to describe *finite-width* effects systematically using the correspondence with field theories (statistical or quantum). In this correspondence, the infinite-width limit is associated with free (non-interacting) theories that can be corrected *perturbatively* for finite width as weakly interacting theories. The concepts of effective (field) theories carry over as well, as the information propagation through the layers of a deep neural network can be understood in terms of a renormalization group (RG) flow. A fixed point analysis motivates strategies for tuning the network to criticality, which deals with gradient problems (blowing up and going to zero).

33.1. Sparsity

Roberts argues how the most generic theory in a quantum field theory framework without guiding principles would start with all combinations of interactions between the particles in the theory but that applying several guiding principles of sparsity will drastically reduce the algorithmic complexity. He first discusses why AI is hard by considering all possible n-pixel images that take on two values (e.g., black or white), so that there are 2^n different images. If each of these images has one of two possible labels (e.g., cat/not-cat or spin-up/spin-down), then the images can be labeled 2^{2^n} ways. This is a really big number. If the labels don't correlate with image *features*, then you can't do better than memorizing images with their labels.

Roberts follows by discussing why physics is simple; these are the arguments that lead him to paraphrase Wigner. In particular, he analyzes the degrees of freedom (dofs) within a QFT-like framework. When thinking about the associated counting of possible terms in the Hamiltonian, it is convenient to have the Ising model in mind as a concrete and familiar example (i.e., imagine a lattice of spins). The starting point is a generic theory with all interactions between all spins; two at a time, three at a time, and so on. Generically the strength of the couplings for these interactions would all be different. So is there are N spins in the Ising model lattice, with no restriction on interactions, there will be 2^N possible independent interaction terms in the Hamiltonian. (I.e., a given spin is there or not in any term.) This Hamiltonian would have no predictive power because you would have to do every experiment to determine the interaction strengths, with no correlation between experiments to enable predictions.

The highest level of sparsity is to limit the number of degrees of freedom that contribute to any interaction; this is what we observe in nature. If the limit is k degrees of freedom, with $k \ll N$, then the dominant term when summing interactions with $N \gg 1$ is

$$\binom{N}{k} \sim N^k/k!,$$

i.e., now the number of parameters is just polynomial in N. This is called k-*locality*. But this is too general still, because we also observe *spatial locality*: N dofs interact only with neighbors (e.g., nearest or next-to-nearest). This mean in d dimensions the number of parameters scales like the number of neighbors, meaning of order Nd. Finally, we also observe *translational invariance*. This implies that the strength of any local interaction is the same everywhere (electric charge or the Ising model coupling). Now the number of parameters is order unity.

A summary of the sparsity counting of dofs is:

$$2^{\mathcal{O}(N)} \xrightarrow[\text{locality}]{k} \mathcal{O}(N^k) \xrightarrow[\text{locality}]{\text{spatial}} \mathcal{O}(N) \xrightarrow[\text{invariance}]{\text{translational}} \mathcal{O}(1).$$

This is a spectacular reduction in the dofs! A consequence is that we have structure to possible physics theories. Sparsity enables us to learn physics and accounts for why physics can provide simple explanations for apparently complicated phenomena.

What does this imply about AI? Because machine learning *does* work, this implies that deep learning and the like will lead to sparse descriptions. We can try to use our a physics framework to organize that description.

33.2. Neural networks in the large-width limit

The big picture is that the neural network acts like a parameterized function

$$f(x;\boldsymbol{\theta}),$$

where the input vector is x (we don't use boldface or arrows for this vector) and $\boldsymbol{\theta}$ is the collective vector of all the parameters (weights and biases). In order to understand this we need a framework and an approximation scheme. Effective (field) theory and large-width limit of the network is a compelling path.

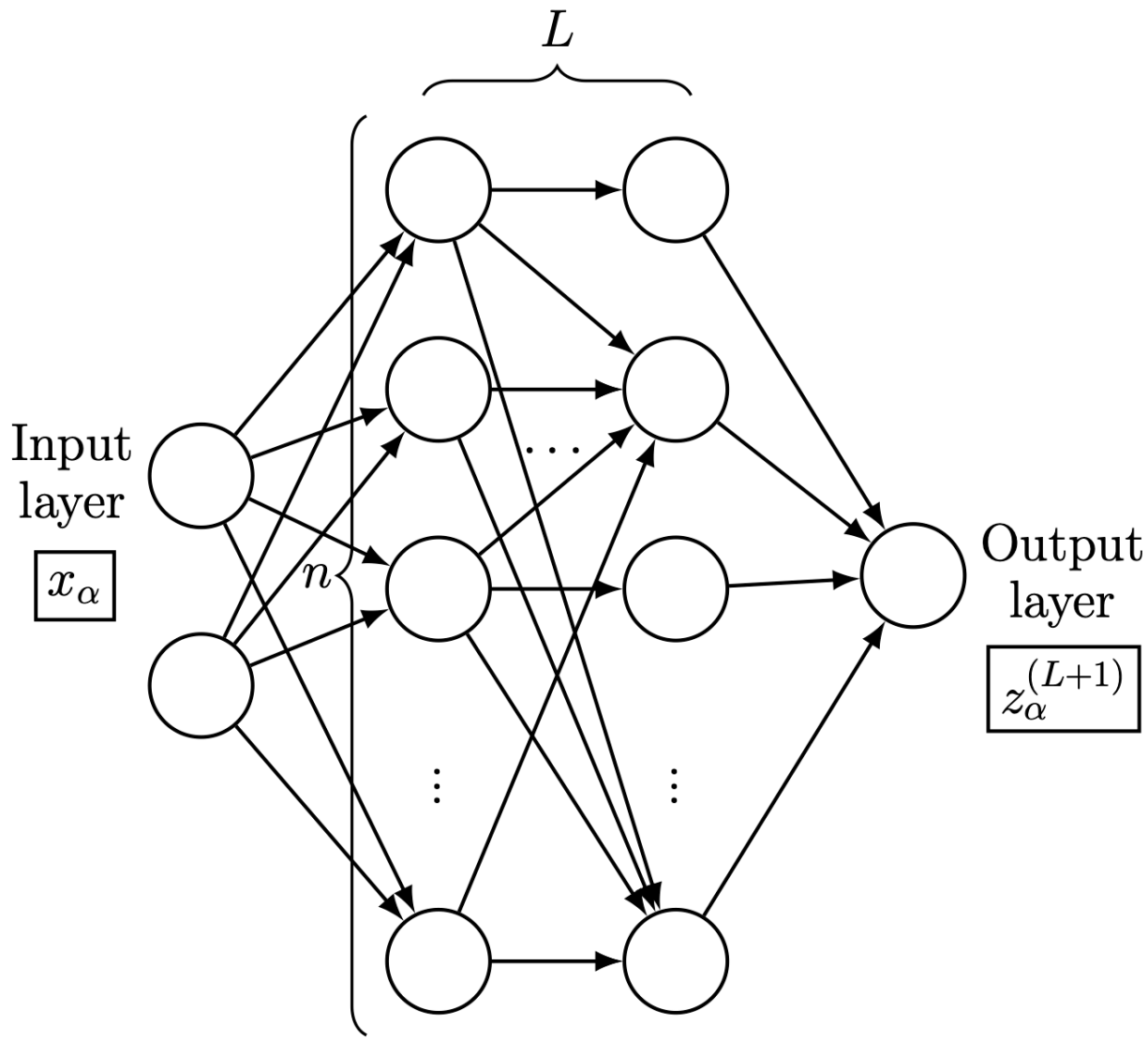


Fig. 33.1 Schematic of a feed-forward neural network with two inputs x_α and one output $z_\alpha^{(L+1)}$, where α is an index running over the dataset $\mathcal{D}$. The width is n and the depth is $L+1$, with L being the number of hidden layers. The neurons are totally connected (some lines are omitted here for clarity). For the examples shown in this chapter, the inputs are the number of protons Z and neutrons N and the output is the mass (or, equivalently, the binding energy) of the corresponding nuclide (see Fig. 33.2).

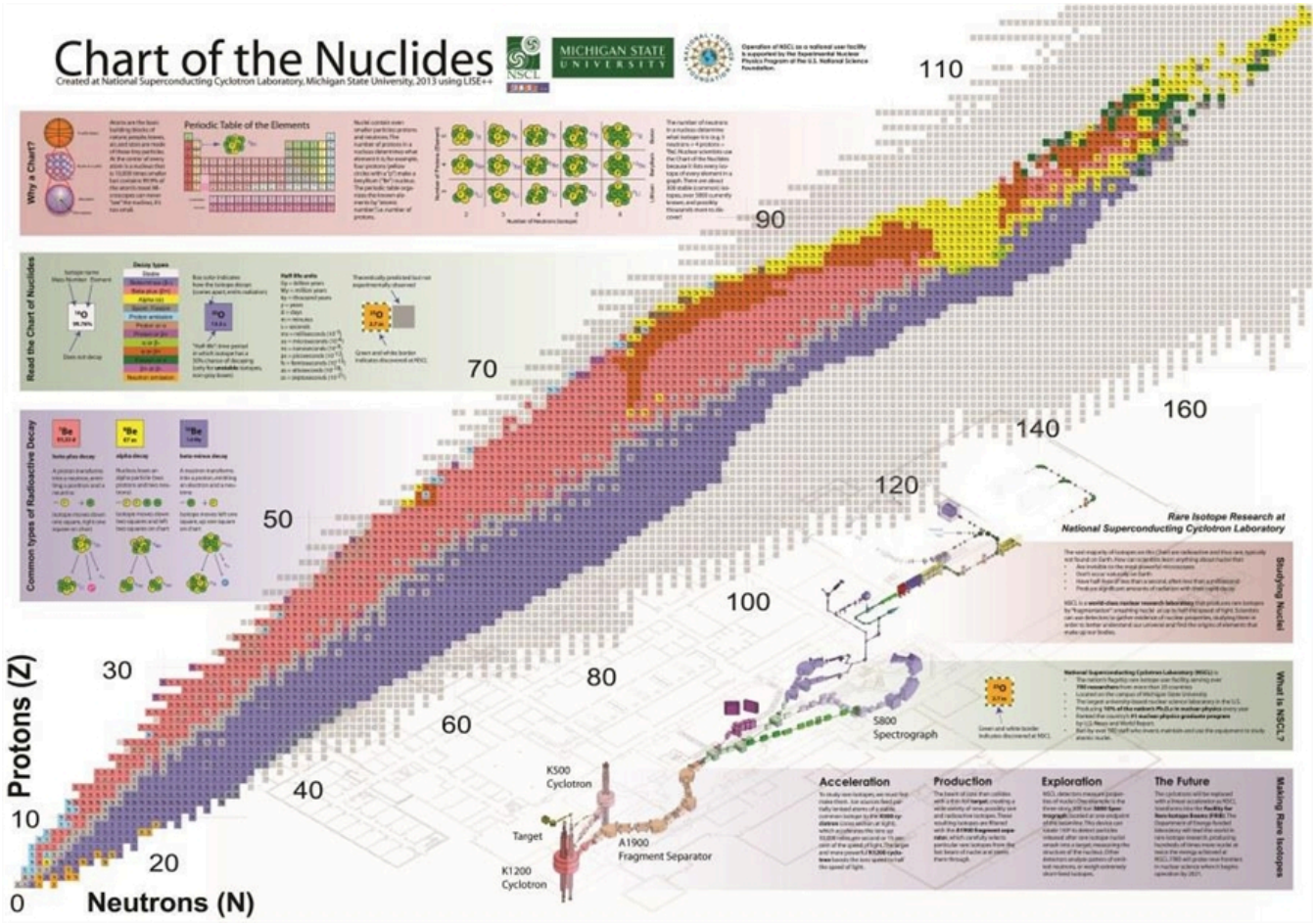


Fig. 33.2 Chart of the nuclides.

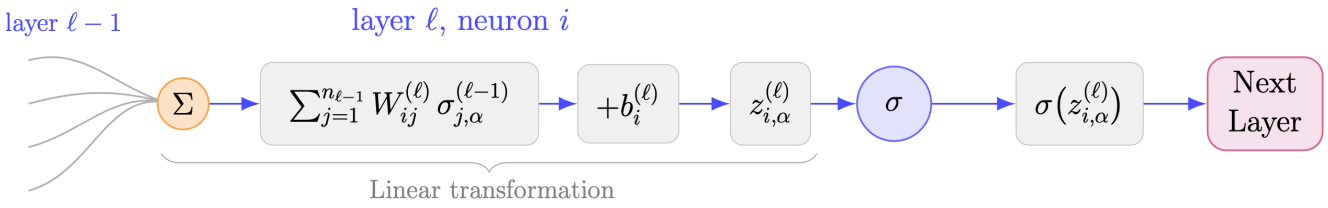


Fig. 33.3 Schematic of a neuron within a neural network. In the layer ℓ, the product between the weight matrix $W^{(\ell)}$ and the previous layers' output $\sigma^{(\ell-1)}$ is summed over the width of the prior layer $n_{\ell-1}$, and a bias vector $b^{(\ell)}$ is added, forming the preactivation $z^{(\ell)}$. The activation function σ acts on the preactivation to yield the output $\sigma\left(z^{(\ell)}\right)$ (also notated $\sigma^{(\ell)}$) to be passed to the next layer.

For a given initialization of the network parameters $\boldsymbol{\theta}$, the preactivations and the output function f are uniquely determined. However, a repeated random initialization of $\boldsymbol{\theta}$ induces a probability distribution on preactivations and f. Explicit expressions for these probability distributions in the limit of large width n can be derived, with the ratio of depth to width revealed as an expansion parameter for the distribution's action. The output distribution in a given layer ℓ refers to the distribution of preactivations $z_{j,\alpha}^{(\ell)}$. This choice allows for parallel treatment of all other layers with the function distribution in the output layer.

To achieve the desired functional relationship between $x_{i,\alpha}$ and $y_{j,\alpha}$, a neural network is trained by adjusting the weights and biases in each layer according to the gradients of the loss function $\mathcal{L}\big(f(x,\boldsymbol{\theta}),y\big)$. The loss function quantifies the difference between the network output $f(x,\boldsymbol{\theta})$ and the output data y, and the goal of training is to minimize $\mathcal{L}\big(f(x,\boldsymbol{\theta}),y\big)$. The gradients of $\mathcal{L}\big(f(x,\boldsymbol{\theta}),y\big)$ with respect to the network parameters $\boldsymbol{\theta}$ depend on the size of the output, as well as the architecture hyperparameters. It is known empirically that poor choices of architecture and/or initialization lead to exploding/vanishing gradients in the absence of adaptive learning algorithms, and this was problematic for training networks for many years.

The ANN output function $f(x,\boldsymbol{\theta})$ for any fixed $\boldsymbol{\theta}$ (that is, specified weights and biases) is a deterministic function. But, as already stressed, with repeated initializations of the ANN with $\boldsymbol{\theta}$ drawn from random distributions, the network acquires a probability distribution over functions $p\left(f(x)|\mathcal{D}\right)$. Again, $\mathcal{D}$ is used to represent the data set of inputs and outputs that the network is finding a relationship between. In the pre-training case, this quantity simply refers to the inputs x_α for the network.

Review of the central limit theorem (CLT)

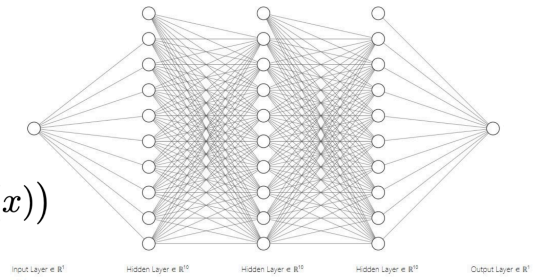
Canonical statement of CLT: "The Central Limit Theorem (CLT) states that the distribution of the sample means of a large number of i.i.d. (independent and identically distributed) random variables will approximate a normal distribution, regardless of the original population's distribution."

For simplicity, assume one-dimensional input *x* and initialize with gaussians (no training).

First layer: $z_i^{(1)}(x) = b_i^{(1)} + W_{i1}^{(1)}x, \quad i = 1,\dots,n$

Other layers: $z_i^{(\ell+1)}(x) = b_i^{(\ell+1)} + \sum_{j=1}^{n_\ell} W_{ij}^{(\ell+1)}\sigma\big(z_j^{(\ell)}(x)\big)$

Important: if y is iid, then g(y) is iid!



Generalized CLT: In the large-width limit, the joint probability *p(f(x), f(x'))* is a bivariate Gaussian with kernel function determined by the activation function and layers. Similarly with more input points. So distribution over functions is a GP!

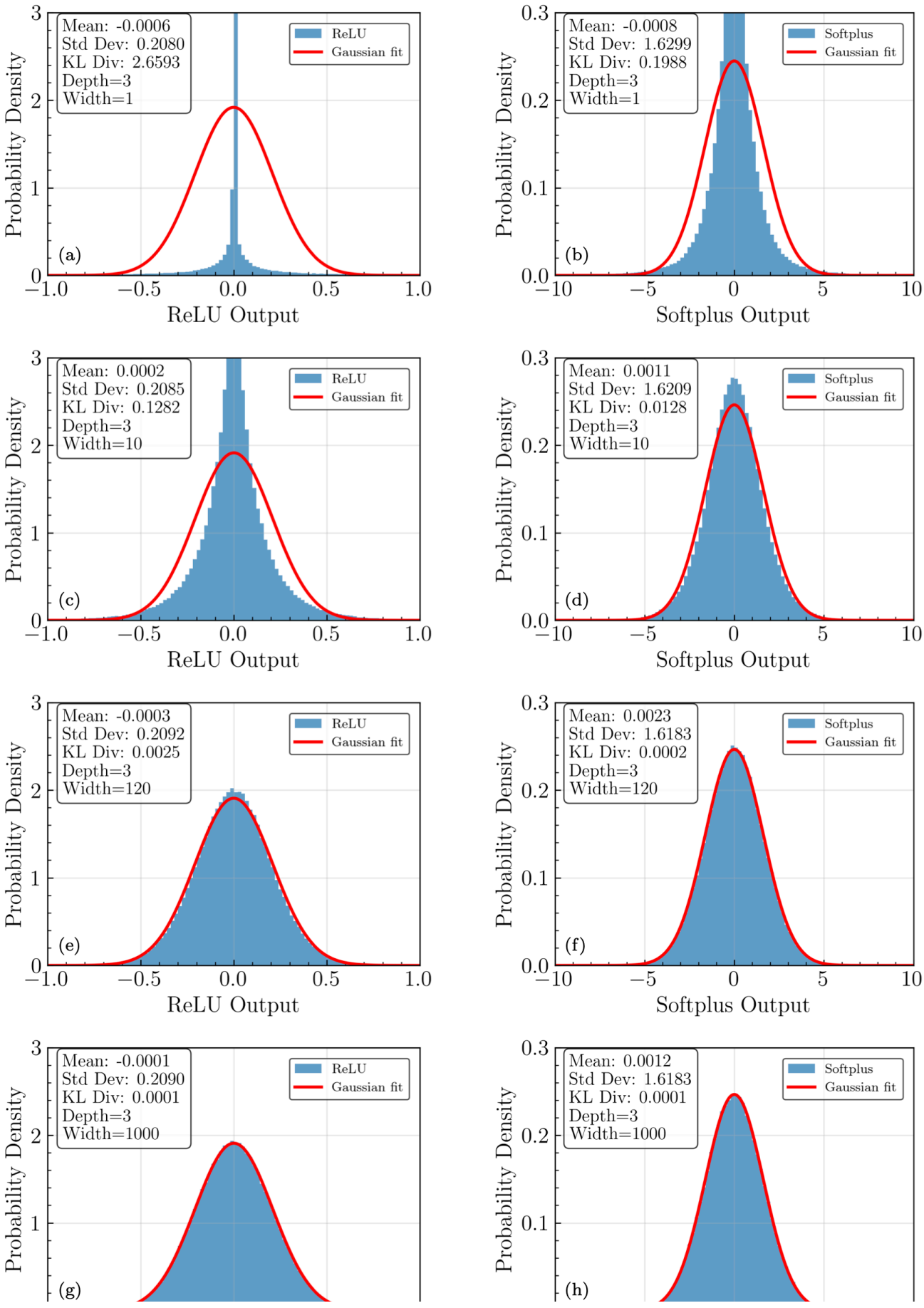

Mean: -0.0006
Std Dev: 0.2080
KL Div: 2.6593
Depth=3
Width=1
ReLU
Gaussian fit
(a)
Probability Density
ReLU Output
Mean: -0.0008
Std Dev: 1.6299
KL Div: 0.1988
Depth=3
Width=1
Softplus
Gaussian fit
(b)
Probability Density
Softplus Output
Mean: 0.0002
Std Dev: 0.2085
KL Div: 0.1282
Depth=3
Width=10
ReLU
Gaussian fit
(c)
Probability Density
ReLU Output
Mean: 0.0011
Std Dev: 1.6209
KL Div: 0.0128
Depth=3
Width=10
Softplus
Gaussian fit
(d)
Probability Density
Softplus Output
Mean: -0.0003
Std Dev: 0.2092
KL Div: 0.0025
Depth=3
Width=120
ReLU
Gaussian fit
(e)
Probability Density
ReLU Output
Mean: 0.0023
Std Dev: 1.6183
KL Div: 0.0002
Depth=3
Width=120
Softplus
Gaussian fit
(f)
Probability Density
Softplus Output
Mean: -0.0001
Std Dev: 0.2090
KL Div: 0.0001
Depth=3
Width=1000
ReLU
Gaussian fit
(g)
Probability Density
Mean: 0.0012
Std Dev: 1.6183
KL Div: 0.0001
Depth=3
Width=1000
Softplus
Gaussian fit
(h)
Probability Density

Fig. 33.4 Final layer ANN output distributions from repeated sampling at the same input point, each plot being a different width. There are two hidden layers. ReLU activation is on the left; Softplus activaiton is on the right. The widths vary from 1 up to 1000 neurons per layer.

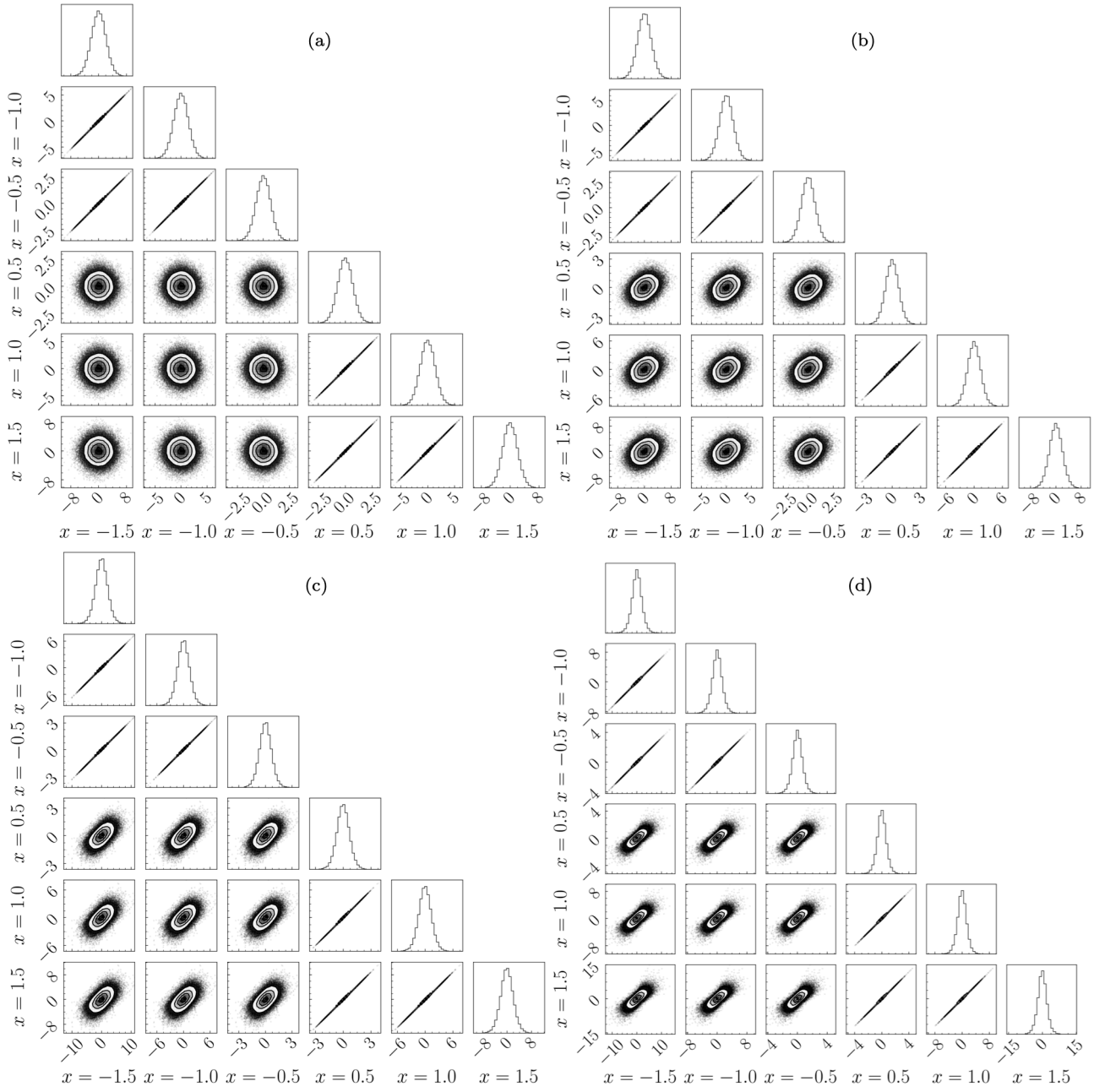


Fig. 33.5 Corner plots for ReLU activation function outputs. The corner plots all feature ReLU networks of a fixed width of 120 neurons, and depths of (a) 1, (b) 2, © 4, and (d) 8. The addition of hidden layers introduces correlations to the network output distributions.

- Fix the sample size n
- Take the mean of the sample drawn N times
- Histogram it
- Try n = 1, 2, 10

- Width is set by n, not N
- If n=1 → distribution
- If n=10 → close to $\mathcal{N}$

Non-gaussian tails are averaged out!

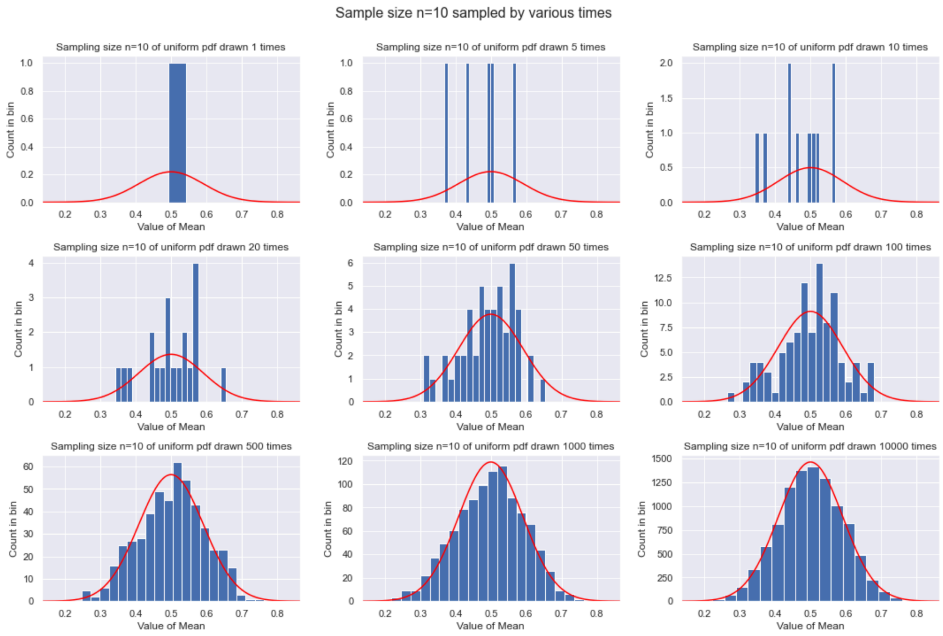


Fig. 33.6 Intuition for the central limit theorem (CLT).

Summary: What is the prior distribution $p(f(x;\theta)|I)$?

- The network's weights and biases $\boldsymbol{\theta}$ are initialized as independent and identically distributed random variables (zero mean, we can take as Gaussian).
- Neural network of given architecture induces a probability distribution of preactivations z at a given layer ℓ.
- The preactivation of final (output) layer is $f(x)$.
- The Central Limit Theorem kicks in as the width n goes to infinity, so these distributions should become (correlated) Gaussians, i.e., Gaussian processes.
- Claim: if we back off from this limit, neural networks can be studied using perturbation theory, with an expansion parameter: depth/width.

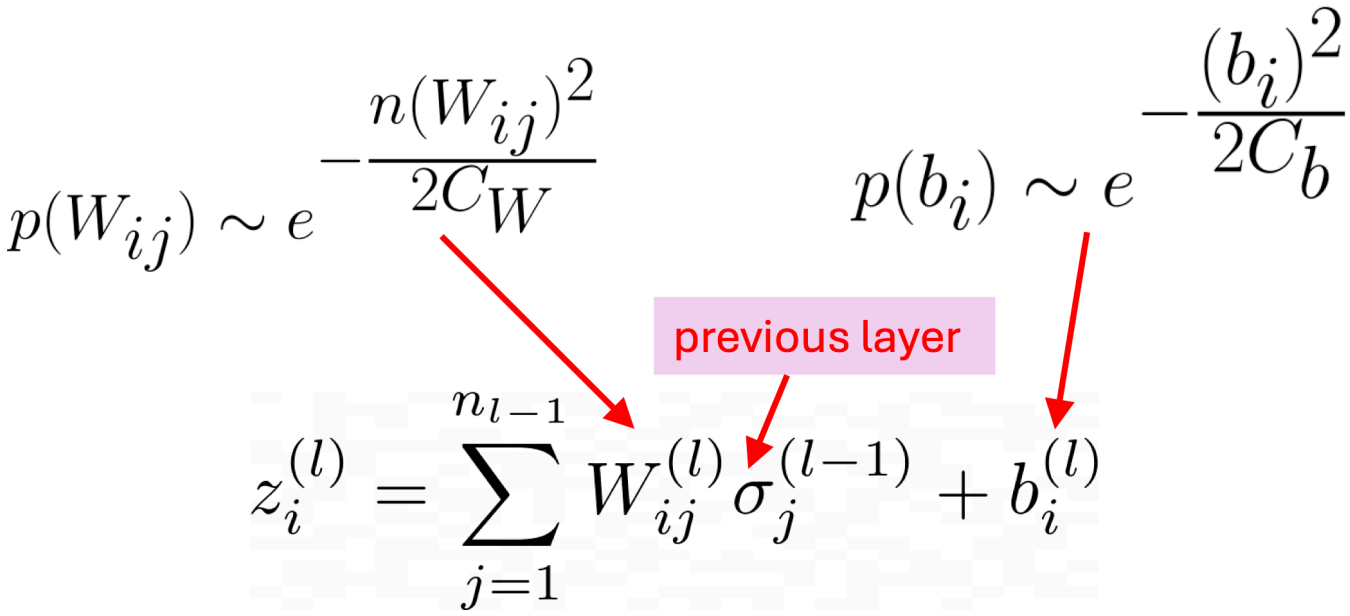


Fig. 33.7 Initialization distributions

33.3. ANNFT: key ideas

Effective theory for ANNs: Overview of key ideas

Roberts and Yaida (2021)

Ensembles of ANNs ➔ **Distributions over random functions** ➔ Path integrals in field theory (FT)

$$\underbrace{\int d\theta\, P(\theta)\, e^{\int d^d x\, J(x) f(x)}}_{\text{parameter space}} \Longrightarrow Z[J] = \left\langle e^{\int d^d x\, J(x) f(x)} \right\rangle \Longleftarrow \underbrace{\int \mathcal{D}f\, e^{-S[f] + \int d^d x\, J(x) f(x)}}_{\text{function space}}$$

- Partition function *Z[J]* is generator of correlation functions like $\langle f(x_1) f(x_2) f(x_3) f(x_4) \rangle$
- Connected correlators from $G_c^{(n)}(x_1, \ldots, x_n) \equiv \left(\frac{\delta}{\delta J(x_1)} \cdots \frac{\delta}{\delta J(x_n)} \ln Z[J] \right) \Big|_{J=0}$
- In infinite width limit, only $G_c^{(2)}(x_1, x_2)$ is nonzero ➔ Gaussian free field theory
- Strategy: Taylor expand about initialization with large (but finite) width

$$f(x, \boldsymbol{\theta}^*) = f(x, \boldsymbol{\theta}) + (\boldsymbol{\theta}^* - \boldsymbol{\theta}) \frac{df}{d\boldsymbol{\theta}} + \frac{1}{2} (\boldsymbol{\theta}^* - \boldsymbol{\theta})^2 \frac{d^2 f}{d\boldsymbol{\theta}^2} + \cdots$$

Claim: only need up to third derivatives, nearly Gaussian, expansion controlled by $r = L/n$

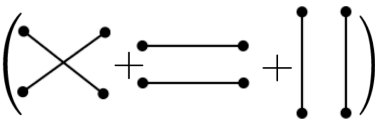
A Theory of Deep Learning II

- Preactivations z are i.i.d., have mean zero, and a variance → expand around CLT
- Moments of preactivations → can be derived with Wick's theorem and expressed diagrammatically as in QFT

Look at correlators of z's:

$$z_i^{(l)} = \sum_{j=1}^{n_{l-1}} W_{ij}^{(l)} \sigma_j^{(l-1)} + b_i^{(l)}$$

$\langle z^2 \rangle =$

$\langle z^4 \rangle =$ $\frac{1}{n}$ $+ \left(1 - \frac{1}{n}\right)$ 

If two-point correlators determine n-point correlators → Gaussian Free Field Theory

$$\frac{\langle y_i y_j \rangle}{n} = \frac{\sigma^2}{n}\delta_{ij} \implies \frac{\langle (y_1 + \cdots + y_n)^4 \rangle}{n^2} = \frac{1}{n}\langle y_1^4 \rangle + 3\left(1 - \frac{1}{n}\right)\langle y_1^2 \rangle \langle y_2^2 \rangle$$

A Theory of Deep Learning III

Roberts and Yaida (2021)

- Infinite width **n** limit can be considered a "free field theory"
- All correlations of interest → included in an action S(z), each with data-dependent couplings
- Couplings evolve according to an RG flow with each layer
- Finding fixed points of covariance prevents gradient issues during training, enables tractable ANNFT

$$p(z^{(l)}) \equiv e^{-S(z^{(l)})}$$

$$S(z^{(l)}) = \frac{1}{2}(G^{(l)})^{-1} z_i^{(l)} z_i^{(l)} - \frac{1}{8}\frac{V^{(l)}}{n} z_i^{(l)} z_i^{(l)} z_j^{(l)} z_j^{(l)} + O(\frac{1}{n^2})$$

Gaussian term

Quartic perturbation, goes like 1/n

Suppressed higher-order terms

$$G^{(l+1)} = C_b + C_W \int_{-\infty}^{\infty} dz^{(l)} \sigma(z^{(l)})^2 e^{\frac{-(z^{(l)})^2}{2G^{(l)}}}$$

initialization variances

33.4. Criticality analysis

Variance

- ReLU has a fixed point, and can be tuned to criticality
- Away from criticality, the variance either explodes or vanishes
- Training stabilized by controlling couplings

$$(C_b, C_W)_{critical} = (0, 2)$$ initialization widths

$$G^{(l+1)} = C_b + C_W \int_{-\infty}^{\infty} dz^{(l)} \sigma(z^{(l)})^2 e^{\frac{-(z^{(l)})^2}{2G^{(l)}}}$$

- Softplus looks like ReLU, but it has no fixed point!
- The variance is always increasing or vanishing
- Reinforces why ReLU > Softplus in practice

No Fixed Point!

$$G^{(l+1)} = C_b + C_W \int_{-\infty}^{\infty} dz^{(l)} \sigma(z^{(l)})^2 e^{\frac{-(z^{(l)})^2}{2G^{(l)}}}$$

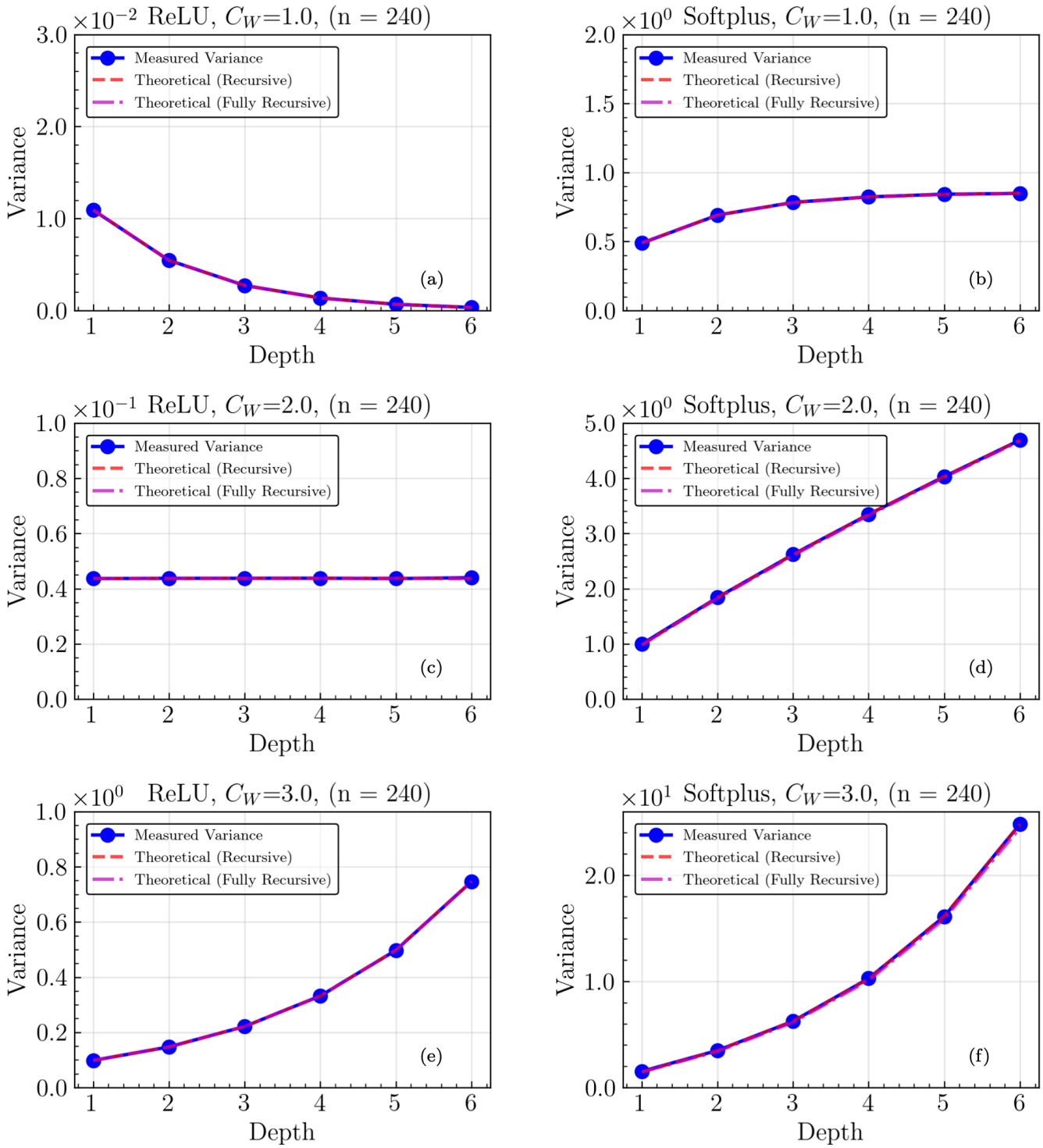
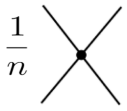

×10−2 ReLU, C_W=1.0, (n = 240)
Measured Variance
Theoretical (Recursive)
Theoretical (Fully Recursive)
Variance
Depth
(a)
×10^0 Softplus, C_W=1.0, (n = 240)
(b)
×10−1 ReLU, C_W=2.0, (n = 240)
(c)
×10^0 Softplus, C_W=2.0, (n = 240)
(d)
×10^0 ReLU, C_W=3.0, (n = 240)
(e)
×10^1 Softplus, C_W=3.0, (n = 240)
(f)

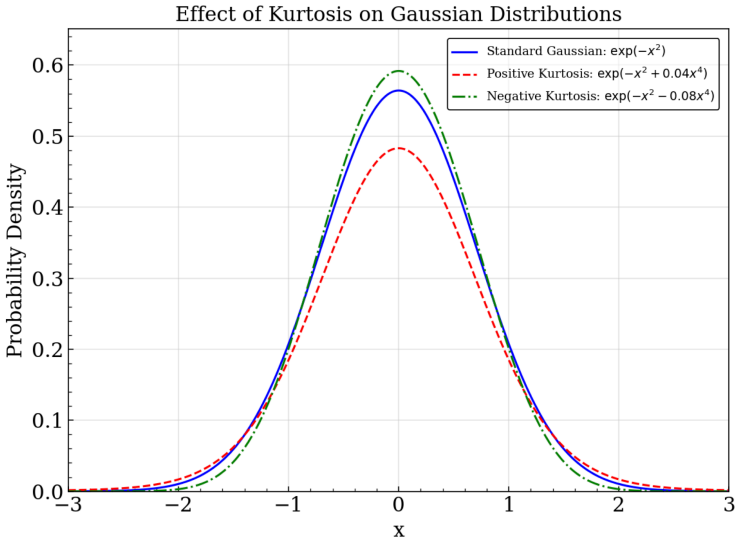
Fig. 33.8 The final layer pre-training output variance as a function of neural network depth with a fixed width of 240. Results are given for a ReLU activation function tuned (a) below ($C_W = 1.0$, $C_b = 0.0$), (c) at the critical point ($C_W = 2.0$, $C_b = 0.0$), and (e) above ($C_W = 3.0$, $C_b = 0.0$) and for a Softplus activation function with (b) lowest, (d) intermediate, and (f) highest initialization widths for the weights by using the same initialization hyperparameters as the ReLU. The measured variance is plotted in blue, a recursive calculation using empirical values of the variance is plotted as a dashed red line, and a recursive calculation using only theoretically calculated variances is plotted as a dashed pink line. When above/below the critical values of the initialization width, the variance explodes/vanishes with depth. When the ReLU network is tuned to critical initialization, the variance is fixed with depth. The Softplus activations do not have a critical point for initialization. As such, the Softplus variance either grows or asymptotes towards a constant value with depth, and does not have initialization hyperparameters that give a fixed variance with depth.

Excess kurtosis

- The kurtosis is 4th moment of a distribution; quantifies how much of the distribution lives in the tails.
- The excess kurtosis (EK) is the remaining four-point interactions after all products of two-point interactions are subtracted.
- This translates to a 4-pt vertex V in our field theory analogy; first non-Gaussian.

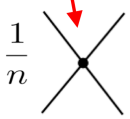
$$\frac{V^{(\ell)}}{n} = \frac{\langle z^4 \rangle - \langle z^2 \rangle^2}{n} =$$





- Like the variance, the EK also has a recursion relation with depth → exponential in general.
- When tuned to criticality, the EK becomes linear in depth and ***r***.
- Higher-order moments will have higher powers of ***r***, enabling the perturbative treatment of moments when r is small.
- This behavior only holds for critically tuned networks. Exponential behaviors spoil our theory.

$$S(z^{(l)}) = \frac{1}{2}(G^{(l)})^{-1} z_i^{(l)} z_i^{(l)} - \frac{1}{8}\frac{V^{(l)}}{n} z_i^{(l)} z_i^{(l)} z_j^{(l)} z_j^{(l)} + O(\frac{1}{n^2})$$



General EK recursion relation (complicated!)

$$V^{(\ell+1)} = \chi_{\parallel}^2(G^{(\ell)})V^{(\ell)} + C_W^2\left[\langle \sigma^4(z^{(\ell)})\rangle_{G^{(\ell)}} - \langle \sigma^2(z^{(\ell)})\rangle_{G^{(\ell)}}^2\right]$$

$$\chi_{\parallel}(G^{(\ell)}) = \frac{C_W}{G^{(\ell)}}\langle z^{(\ell)}\sigma(z^{(\ell)})\sigma(z^{(\ell)})\rangle_{G^{(\ell)}}$$

But with criticality, things simplify!

$$\frac{V^{(\ell)}}{n} \sim \nu r$$

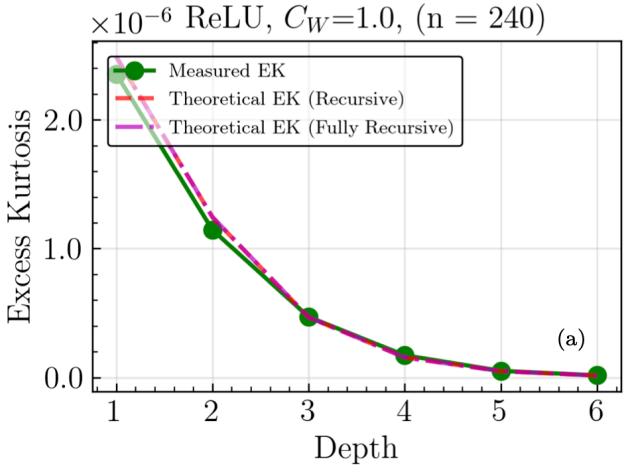
- ReLU's kurtosis scales linearly with the ratio of depth to width ***r***
- Away from criticality, the kurtosis explodes or vanishes
- Allows us to control moments order by order in ***r***

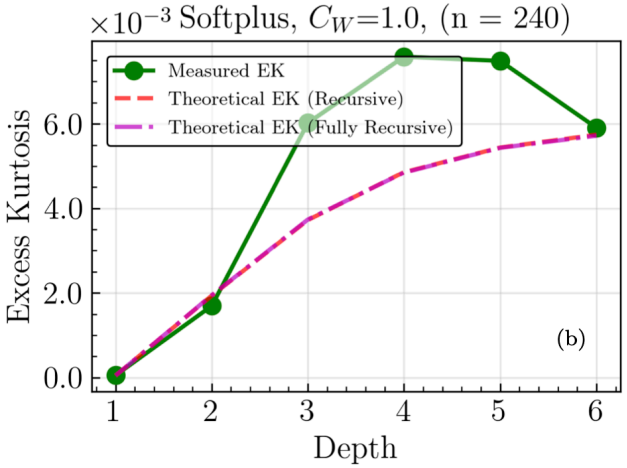
$$(C_b, C_W)_{critical} = (0, 2)$$ initialization widths

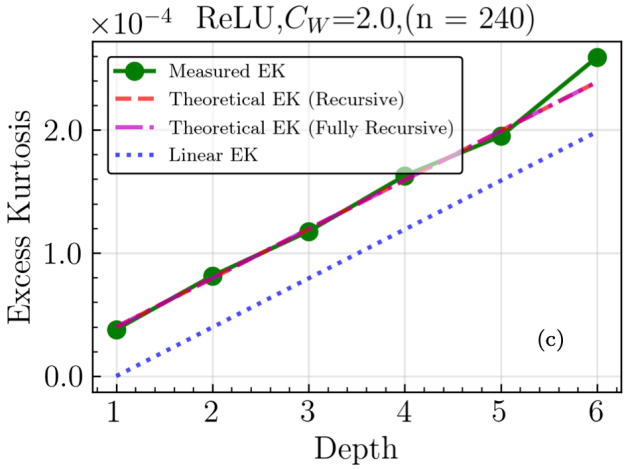
$$G^{(l+1)} = C_b + C_W \int_{-\infty}^{\infty} dz^{(l)} \sigma(z^{(l)})^2 e^{\frac{-(z^{(l)})^2}{2G^{(l)}}}$$

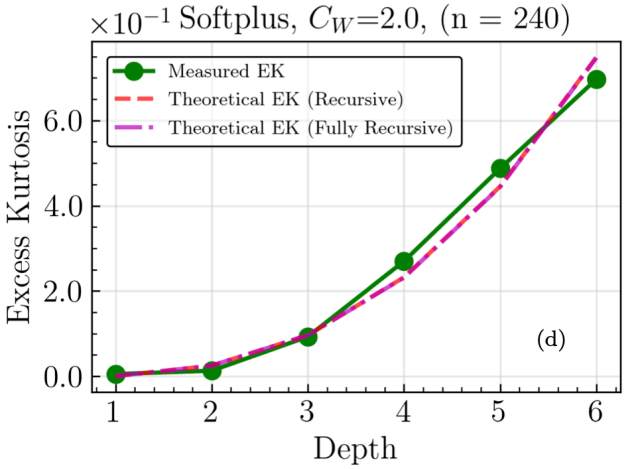
- Softplus looks like ReLU, but it has no fixed point!
- Leads to non-perturbative theory
- Reinforces why ReLU > Softplus in practice

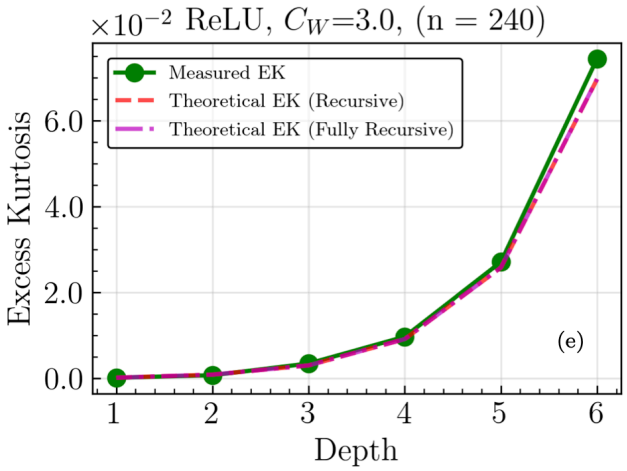
No Fixed Point!


$\times 10^{-6}$ ReLU, C_W=1.0, (n = 240)
Measured EK
Theoretical EK (Recursive)
Theoretical EK (Fully Recursive)
Excess Kurtosis
Depth
(a)


$\times 10^{-3}$ Softplus, C_W=1.0, (n = 240)
Measured EK
Theoretical EK (Recursive)
Theoretical EK (Fully Recursive)
Excess Kurtosis
Depth
(b)


$\times 10^{-4}$ ReLU,C_W=2.0,(n = 240)
Measured EK
Theoretical EK (Recursive)
Theoretical EK (Fully Recursive)
Linear EK
Excess Kurtosis
Depth
(c)


$\times 10^{-1}$ Softplus, C_W=2.0, (n = 240)
Measured EK
Theoretical EK (Recursive)
Theoretical EK (Fully Recursive)
Excess Kurtosis
Depth
(d)


$\times 10^{-2}$ ReLU, C_W=3.0, (n = 240)
Measured EK
Theoretical EK (Recursive)
Theoretical EK (Fully Recursive)
Excess Kurtosis
Depth
(e)

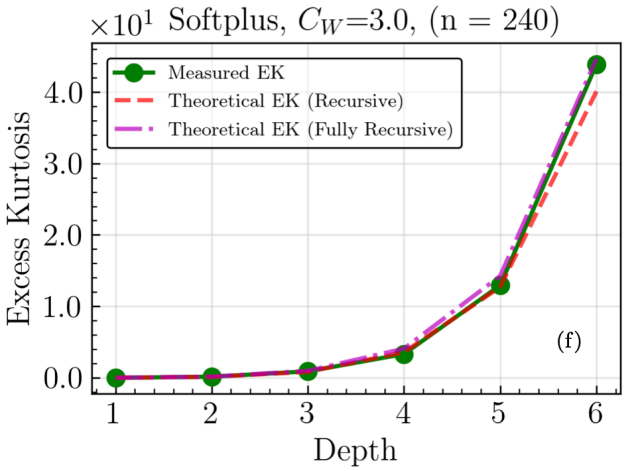
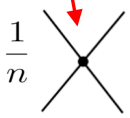

$\times 10^{1}$ Softplus, C_W=3.0, (n = 240)
Measured EK
Theoretical EK (Recursive)
Theoretical EK (Fully Recursive)
Excess Kurtosis
Depth
(f)

Fig. 33.9 The final-layer pre-training-output (unstandardized) excess kurtosis (abbreviated here as EK, and also known as the non-Gaussian 4-point correlations) as a function of neural network depth with a fixed width of 240. Results are given for a ReLU activation function tuned (a) below ($C_W = 1.0$, $C_b = 0.0$), (c) at the critical point ($C_W = 2.0$, $C_b = 0.0$) and (e) above ($C_W = 3.0$, $C_b = 0.0$), and for a Softplus activation function with (b) lowest, (d) intermediate, and (f) highest initialization widths for the weights by using the same initialization hyperparameters as ReLU. The measured EK is plotted as a green line, a recursive calculation of EK using empirical values of the variance and initial EK is plotted as a dashed red line, and a recursive calculation using only theoretical values is plotted as a dashed pink line. An additional blue line is present in (e), representing an exact calculation of the critical EK without treating $1/n$ terms as subleading. When critically tuned, ReLU networks' EK behaves linearly with depth as predicted by ANNFT, demonstrating that the higher-order moments of the distribution are controlled by an expansion in r. Non-critical distributions are seen to grow non-linearly with depth, reflecting that criticality allows for a perturbative ANNFT that lets deeper network network behavior to be analyzed.

The story so far

- Like the variance, the EK also has a recursion relation with depth → exponential in general.
- When tuned to criticality, the EK becomes linear in depth and ***r***.
- Higher-order moments will have higher powers of ***r***, enabling the perturbative treatment of moments when r is small.
- This behavior only holds for critically tuned networks. Exponential behaviors spoil our theory.

$$S(z^{(l)}) = \frac{1}{2}(G^{(l)})^{-1} z_i^{(l)} z_i^{(l)} - \frac{1}{8}\frac{V^{(l)}}{n} z_i^{(l)} z_i^{(l)} z_j^{(l)} z_j^{(l)} + O(\frac{1}{n^2})$$



General EK recursion relation (complicated!)

$$V^{(\ell+1)} = \chi_{\parallel}^2(G^{(\ell)})V^{(\ell)} + C_W^2\left[\langle\sigma^4(z^{(\ell)})\rangle_{G^{(\ell)}} - \langle\sigma^2(z^{(\ell)})\rangle_{G^{(\ell)}}^2\right]$$

$$\chi_{\parallel}(G^{(\ell)}) = \frac{C_W}{G^{(\ell)}}\langle z^{(\ell)}\sigma(z^{(\ell)})\sigma(z^{(\ell)})\rangle_{G^{(\ell)}}$$

But with criticality, things simplify!

$$\frac{V^{(\ell)}}{n} \sim \nu r$$

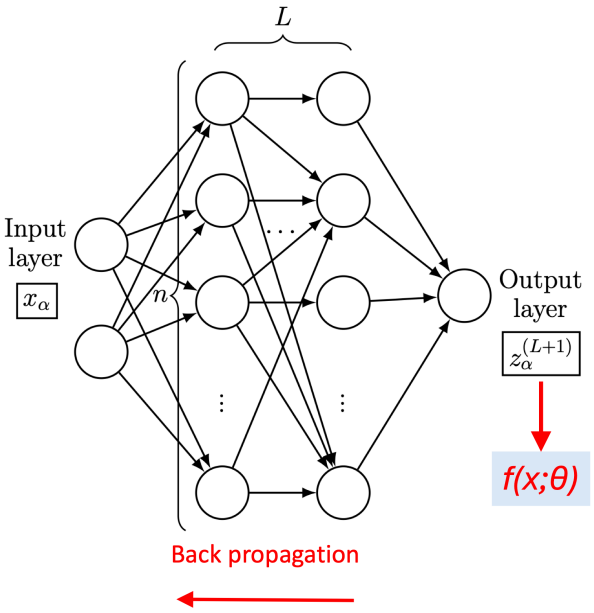
Validating ANNFT training behaviors

Anatomy of a feed-forward network

- Training: adjust weights and biases $\boldsymbol{\theta}$ to minimize loss function $\mathcal{L}$. Gradients depends on the activation function, etc.
- Poor choices of architecture and/or initialization → exploding/vanishing gradients.
- Introduces new hyperparameters: → learning rates

$$\theta_\mu(t+1) = \theta_\mu(t) - \eta \frac{d\mathcal{L}}{d\theta_\mu}\bigg|_{\theta_\mu=\theta_\mu(t)}$$

$$0 = \frac{d\mathcal{L}}{d\theta_\mu}\bigg|_{\theta=\theta^*}$$



Training and the Theory of Deep Learning

- Training has hyperparameters that can be tuned to critical values, such as the learning rates $\boldsymbol{\lambda}$.
- Neural Tangent Kernel (NTK) $\boldsymbol{H}$ → how network outputs evolve during training at large width
- NTK has recursion relations with depth, too → solve for scaling prescriptions for learning rates
- Critically scaled training allows for stable learning with introduction of a new hyperparameter.

$$p\left(z^{(l)}|Data\right) \to p\left(z^{(l)}, H^{(l)}|Data\right)$$

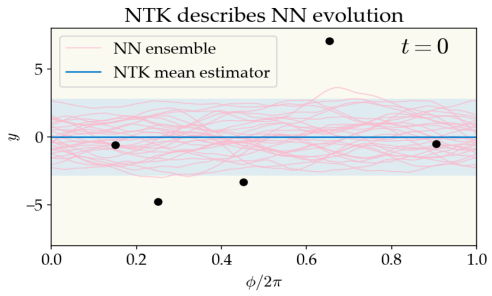
$$H_{ij}^{(l)} = \sum_{\mu\nu} \lambda_{\mu\nu} \frac{\partial z_i^{(l)}}{\partial \theta_\mu} \frac{\partial z_i^{(l)}}{\partial \theta_\nu}$$

Solve recursion relations

[Roberts and Yaida (2021)](#)

$$\lambda_W^{(\ell)} \to \frac{\lambda_W}{n_{\ell-1}} \qquad \lambda_b^{(\ell)} \to \frac{\lambda_b}{\ell}$$

(Critically scaled LRs for Tanh)



Neural Tangent Kernel Recursion

- Output evolution is determined primarily by initialization and activation function!
- Critical training results in an expansion in ***r*** for the NTK **H**.
- The NTK H determines the output gradients in each layer

New training hyperparameters

$$\widehat{H}^{(\ell+1)}_{i_1 i_2;\alpha_1\alpha_2} = \delta_{i_1 i_2}\left[\lambda_b^{(\ell+1)} + \lambda_W^{(\ell+1)}\left(\frac{1}{n_\ell}\sum_{j=1}^{n_\ell}\sigma^{(\ell)}_{j;\alpha_1}\sigma^{(\ell)}_{j;\alpha_2}\right)\right] + \sum_{j_1,j_2=1}^{n_\ell} W^{(\ell+1)}_{i_1 j_1} W^{(\ell+1)}_{i_2 j_2}\sigma'^{(\ell)}_{j_1;\alpha_1}\sigma'^{(\ell)}_{j_2;\alpha_2}\widehat{H}^{(\ell)}_{j_1 j_2;\alpha_1\alpha_2}$$

Otherwise, solely initialization and activation function

$$H^{(\ell)}_{i_1 i_2;\alpha_1\alpha_2} = H^{\{0\}(\ell)}_{i_1 i_2;\alpha_1\alpha_2} + \frac{1}{n_{\ell-1}} H^{\{1\}(\ell)}_{i_1 i_2;\alpha_1\alpha_2} + \frac{1}{n_{\ell-1}^2} H^{\{2\}(\ell)}_{i_1 i_2;\alpha_1\alpha_2} + O\left(\frac{1}{n^3}\right).$$

Mean RMS deviation between initial and final weights/biases

ANNFT assumes that the initial network weights and biases are close so that a Taylor series for the trained solution is valid.

$$f(x;\boldsymbol{\theta}^\star) = f(x;\boldsymbol{\theta}) + (\boldsymbol{\theta}^\star - \boldsymbol{\theta})\frac{df}{d\boldsymbol{\theta}} + \frac{1}{2}(\boldsymbol{\theta}^\star - \boldsymbol{\theta})^2\frac{d^2 f}{d\boldsymbol{\theta}^2} + \cdots$$

The average Frobenius norm of weights and biases was used to judge this.

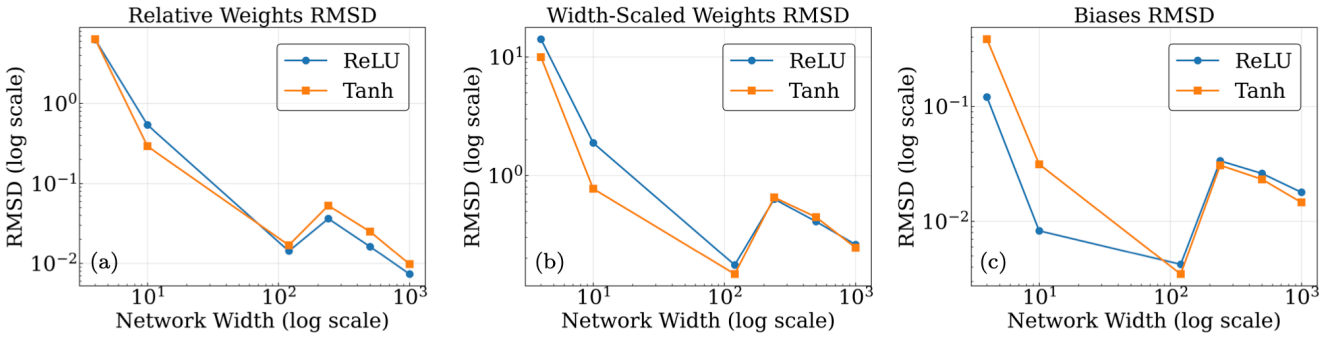


Fig. 33.10 Plots of the mean RMSD between final and initial parameters for the average network hidden layer plotted versus multiple widths (4, 10, 120, 240, 500, 1000). (a) depicts the relative RMSD for the weights (compared to the initial weights), while (b) depicts the absolute RMSD with the weight matrices rescaled by a factor of n to remove width dependence, and © has the unscaled RMSD for the biases (which are initially zero). Within the regimes useful for training, it can be seen that the changes in parameters are small, a condition required for Taylor expanding around a solution.

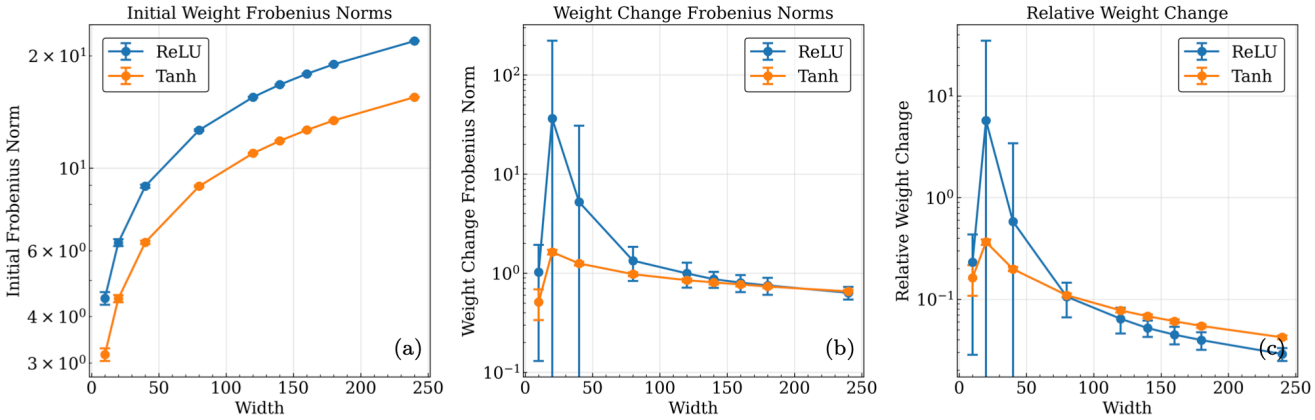


Fig. 33.11 (a) The mean hidden layer Frobenius norms of the pre-training weight matrices, (b) the difference between pre and post training matrices, and (c) the relative difference matrices versus the width of the network containing the matrices. Each norm per width is computed by averaging the mean norm for 100 network trainings of 20000 epochs, with outliers outside of the interquartile range of the distribution of each norm being excluded. The depth of each network configuration was held fixed at 4 layers ($L = 3$,$\ell_{out} = 4$). The norms give an estimate of the size of the initial parameters, and their absolute and relative difference after training, and their dependence on the width of the network.

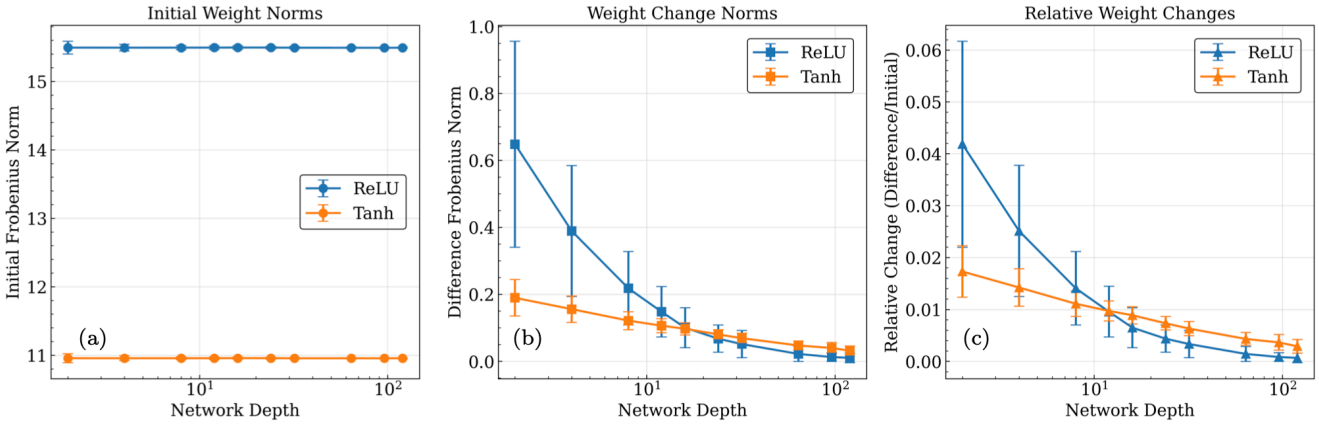


Fig. 33.12 Same as Fig. 33.11 but now with fixed width instead of depth. (a) The mean hidden layer Frobenius norms of the pre-training weight matrices, (b) the difference between pre and post training matrices, and (c) the relative difference matrices versus the depth of the network containing the matrices. Each norm per depth is computed by averaging the mean norm for 100 network trainings of 20000 epochs, with outliers outside of the interquartile range of the distribution of each norm being excluded. The width was held fixed at 120 neurons per hidden layer. The norms give an estimate of the size of the initial parameters, and their absolute and relative difference after training, and their dependence on the depth of the network.

33.6. Expansion parameter

At a large but finite width, the distribution over output functions acquires non-Gaussian components, but the near-CLT averaging enables these correlations to be treated in a controlled manner. At the same time, increasing depth ℓ_{out} introduces increasing correlations. These opposing tendencies lead to the identification of the ratio $r \equiv \ell_{out}/n$ being significant as both n and ℓ_{out} become large. r acts as an expansion parameter that suppresses higher-order, non-Gaussian correlations in the limit of small r.

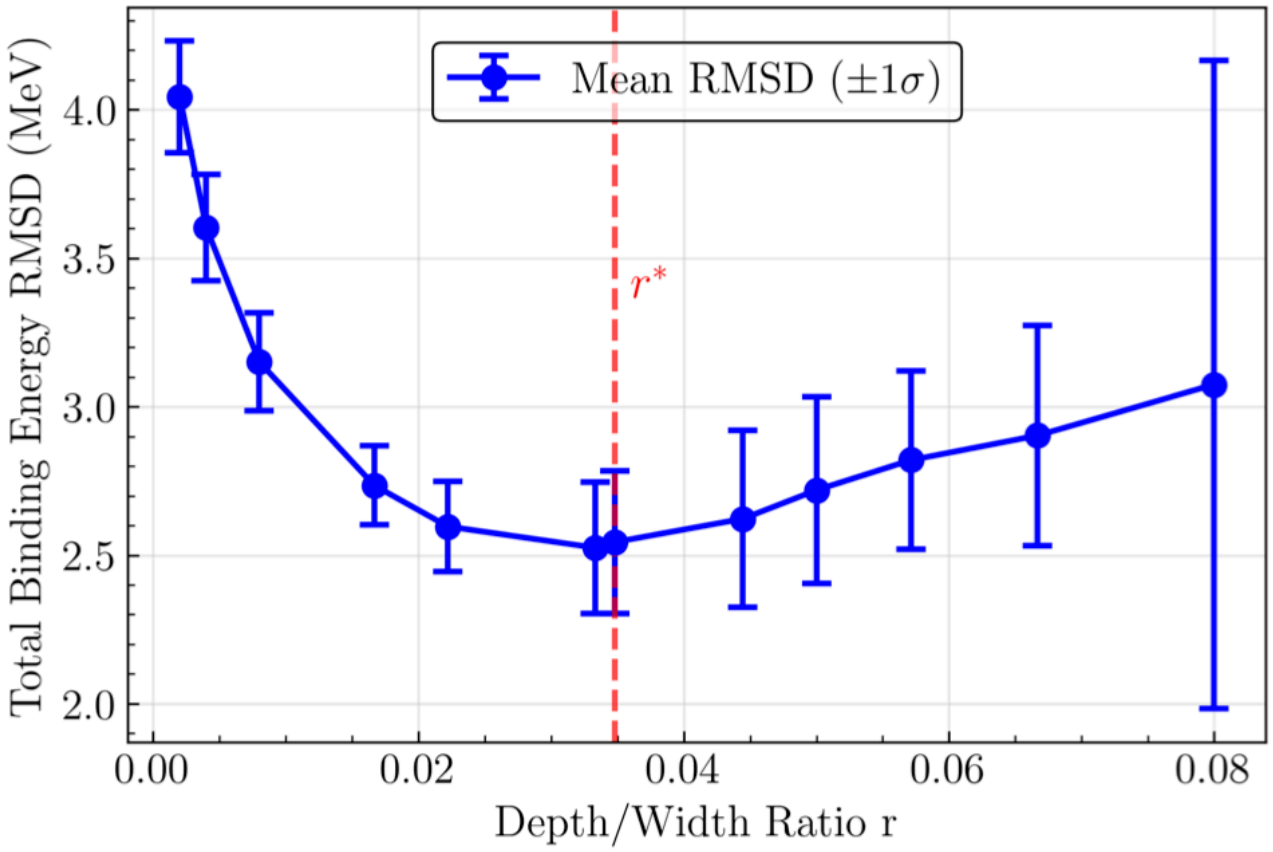


Fig. 33.13 The mean root-mean-square (rms) deviation of the binding energy data and the two-input-network outputs for 100 trainings vs. ratio of depth-to-width r, with the depth $\ell_{out} = 4$. Trainings with binding-energy RMSD≥ 30 MeV were omitted from calculation of the mean value of the binding-energy RMSD. The red horizontal line is the r^* value for this network, calculated to be $r^* = 0.034$. This labels the cutoff where the effectively deep regime begins to transition into the chaotic regime with increasing r.

As an expansion parameter, r quantifies the degree of correlation between neurons in a network. This results in three regimes describing the initialized output distribution:

- $r \to 0$, all terms in the action dependent on r vanish, and the output distribution becomes Gaussian (effectively infinite width, CLT kicks in). These networks have turned off their correlations.
- $0 < r \ll 1$, the moments are controllable, truncated, and nontrivial, as is desired in our effective theory approach. These networks are in what is known as the “effectively deep” regime.
- $r \geq 1$, the moments are strongly coupled, and every term in the r expansion contributes to the action. In this regime, the theory becomes highly non-perturbative, and an effective description becomes impossible.

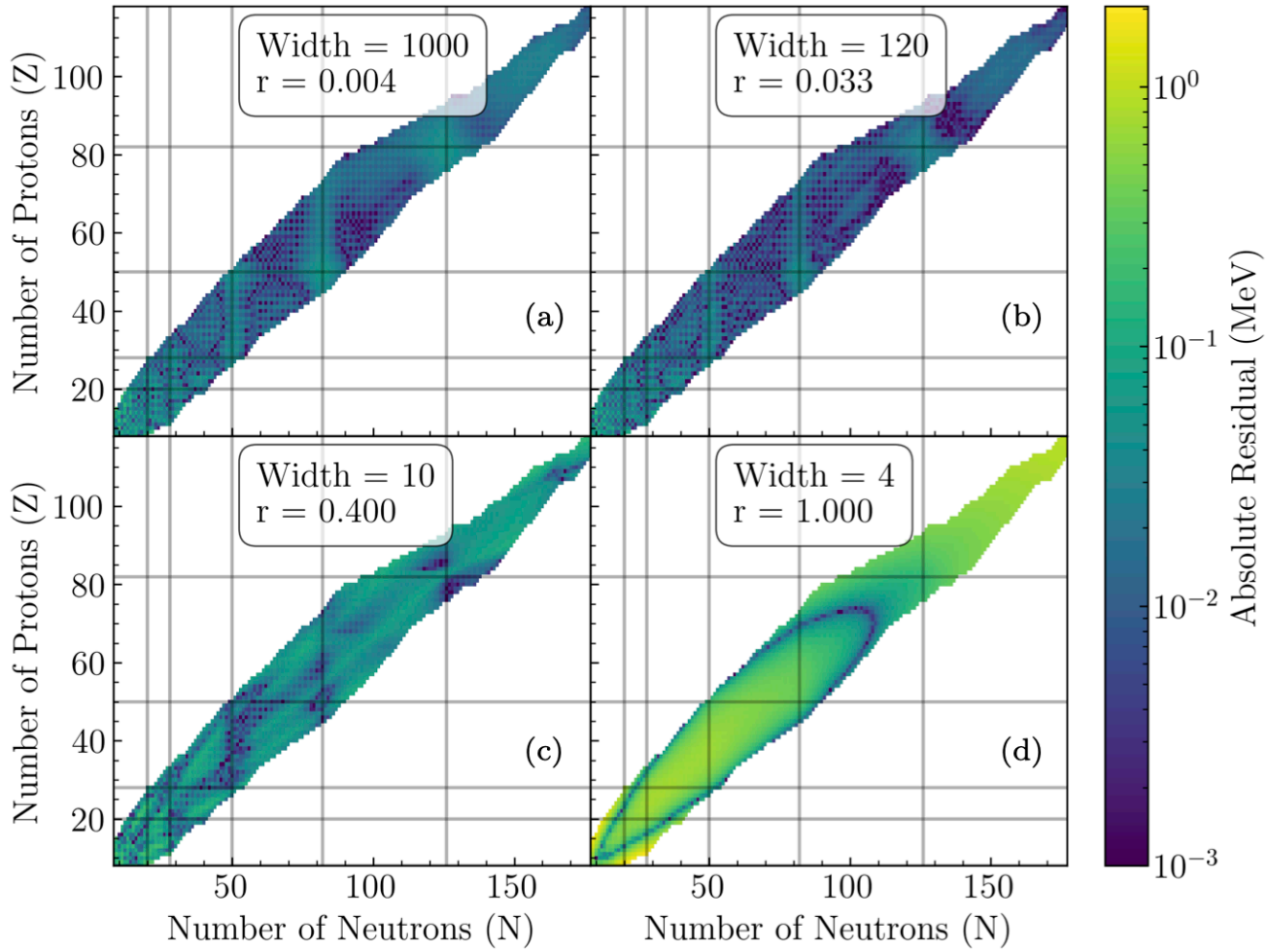


Fig. 33.14 Residual plots for a trained 2 input binding energy network with a fixed depth of 4, and widths of (a) 4, (b) 10, (c) 120, and (d) 1000 to demonstrate the different learning regimes in ANNFT.

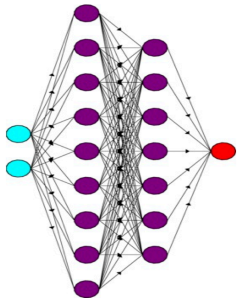
The four plots correspond to different regimes:

- $r = 0.004$: Still able to learn, but will only learn more trivially as $r \to 0$. Mean BE RMSD of 3.6 MeV.
- $r = 0.033$: The most learning-capable critical network. Mean BE RMSD of 2.52 MeV.
- $r = 0.4$: Loses feature learning to more strongly correlated neurons as $r \to 1$. Mean BE RMSD of 56 MeV (bimodal between 72 and $\sim$3 MeV).
- $r = 1.0$: Neuron correlations prevent any learning beyond the simplest loss local minima. Mean BE RMSD of 72 MeV.

33.7. Critical tuning of hyperparameters

- The ratio of depth to width ***r*** should be small for critical analysis ($0 < r < r^* \ll 1$)
- Small depth to width means we can treat ANNs as field theories with couplings that run with depth.
- Activation Functions should be chosen such that critical points exist.
- Weights and Biases should be set to criticality to avoid gradient problems during training.
- Learning rates must be critically scaled to improve performance.

Binding Energy Fitting Network

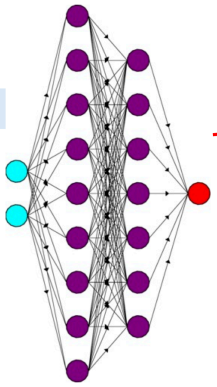


Zeng et al. (2024)

- Small depth to width! (***r*** = 3/35)
- Tanh activation in the hidden layers
- Can be tuned to criticality!

- As a test case, AME2020 binding energy data was fit by different networks
- Networks used feature different combinations of critical initialization (CI) and training (CT), and non-critical initialization (NI) and training (NT).
- Networks compared were CICT, CINT, and NINT using ReLU, and Tanh activation functions and SGD optimizers.
- Fits also performed using Adam optimizers (no critical treatment at present)

Zeng et al. (2024)



Initialization

$$p(W_{ij}) = e^{-\frac{n(W_{ij})^2}{2C_W}}$$

$$p(b_i) = e^{-\frac{(b_i)^2}{2C_b}}$$

$$(C_b, C_W)_{critical} = (0, 2)$$

Training

$$\lambda_W^{(\ell)} \to \frac{\lambda_W}{n_{\ell-1}}$$

$$\lambda_b^{(\ell)} \to \frac{\lambda_b}{\ell}$$

Depths: 1-8 Hidden Layers
Widths: 20-120 Nodes incremented by 20

Mean absolute error loss vs. epochs

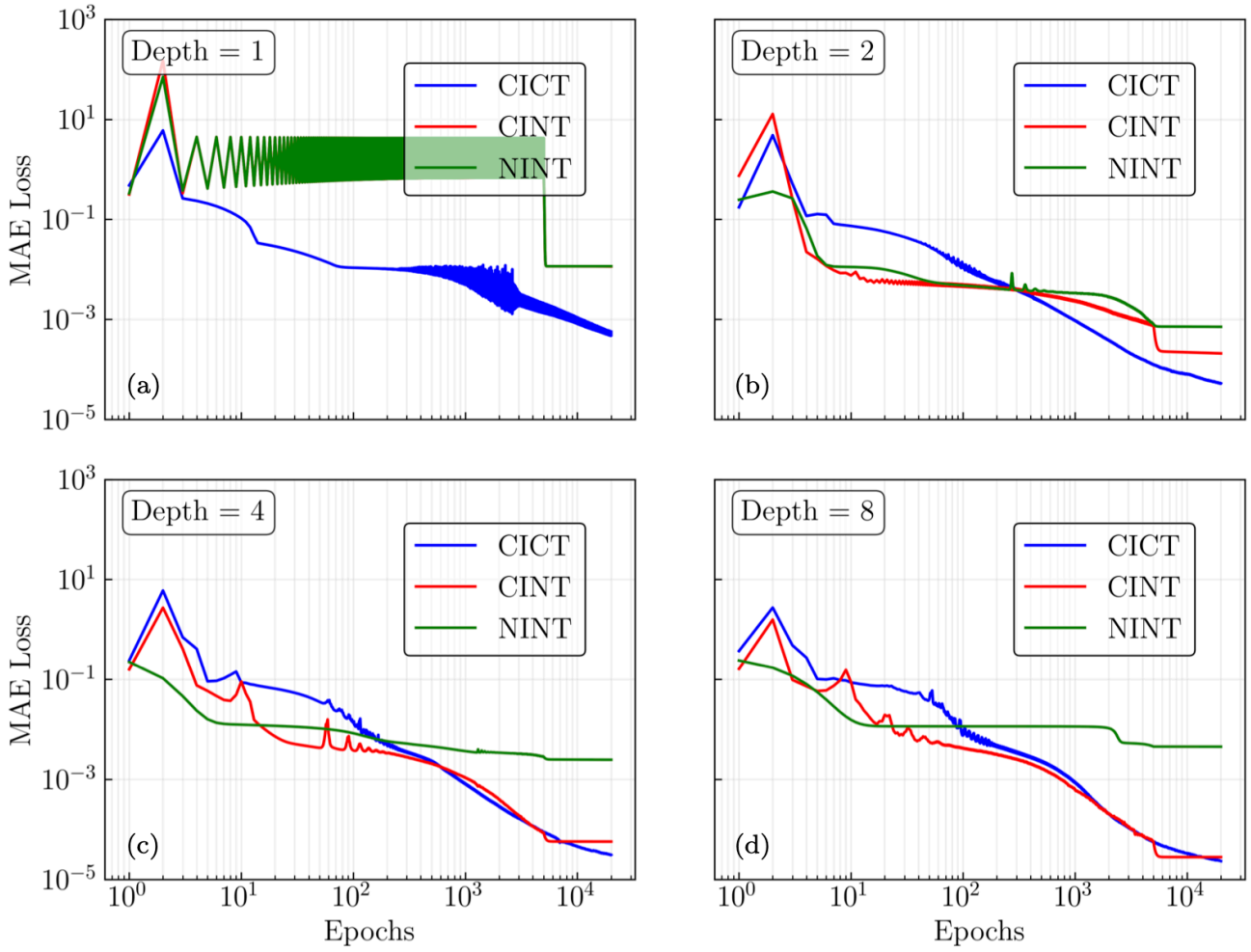


Fig. 33.15 The mean absolute error loss vs. epochs for ReLU activation functions. Hidden layer widths are at 100 neurons, and depths are 1,2,4, and 8 hidden layers. The CICT and CINT architectures outperform the NINT, and their performance improves with depth, whereas the NINT networks stagnate in performance after 2 hidden layers.

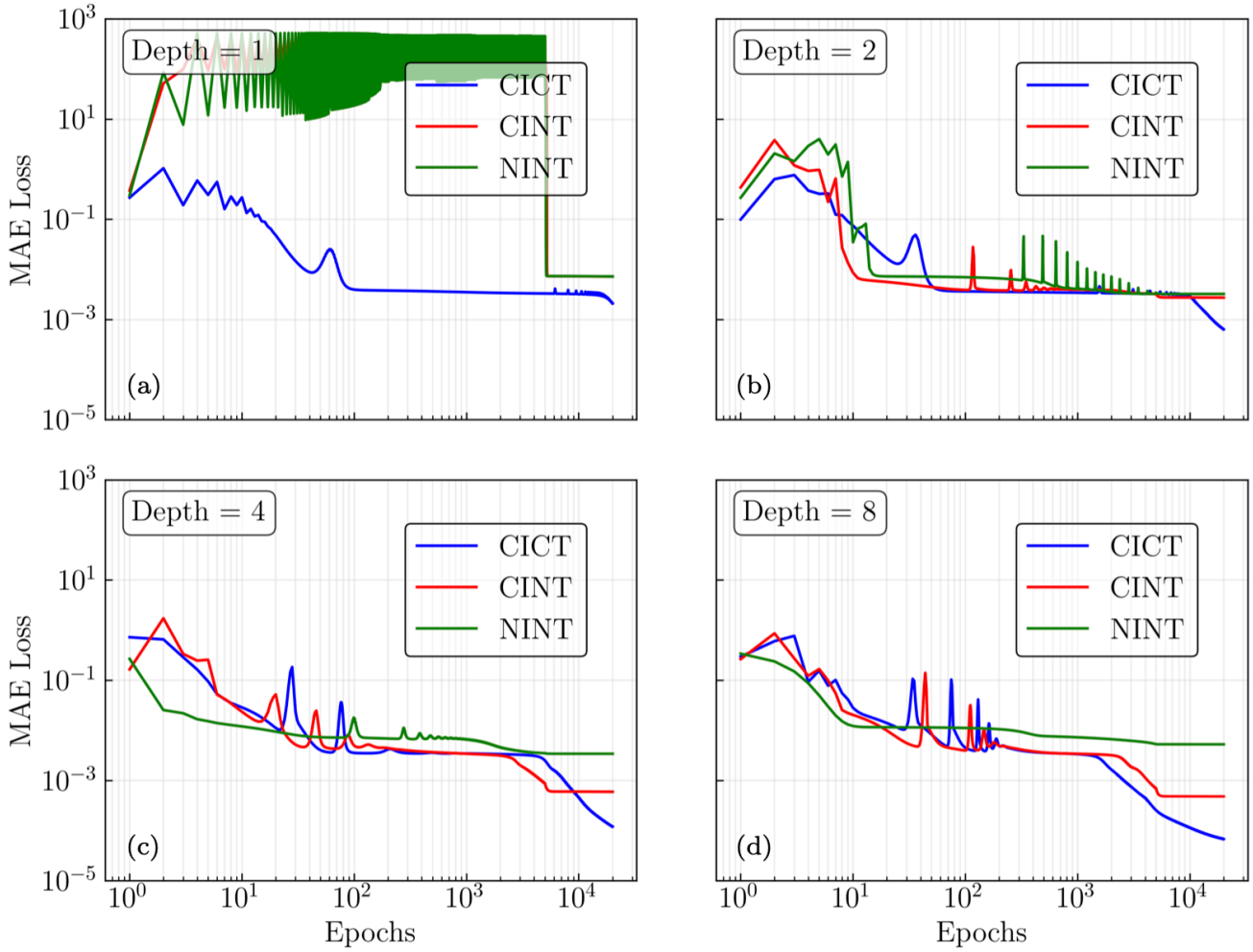


Fig. 33.16 Same as Fig. 33.15 but for Tanh activation functions.

33.8. Summary of results

Neural network criticality offers a means to understand neural network behavior, as well as providing prescriptions for architecture and training hyperparameters. Here we have explored the non-empirical approach to understanding ANNs using a field-theory-based criticality analysis (ANNFT). We used a protoypical nuclear physics ANN as a testbed, namely a two-input/one-output feed-forward network trained from the AME2020 dataset to fit nuclear binding energies.

In pre-training, we found the expected approach to Gaussian distributions for the final layer output distributions with fixed depth but increasing width. Within small fluctuations, normal distributions were seen by a width of 120 neurons for both ReLU and Softplus activations and for every other activation function we have tried. The correlations between pairs of input values with a width of 120 and increasing depths showed Gaussian process (GP) correlations with the predicted kernel at depth 1 and increasing correlations with additional hidden layers.

A comparison of measured variance and excess kurtosis (difference of the fourth moment from Gaussian expectations) from many initializations for a fixed width of 240 and increasing depths validates the theoretical predictions of criticality. In particular, for the ReLU activation function the variance grows or shrinks with depth if the initialization hyperparameters are tuned above or below the predicted critical value. At criticality the variance is flat. In contrast, the Softplus activation function does not have a critical fixed point for initialization. In accordance with

theory, the variance either grows or asymptotes to a constant value with depth. Non-zero excess kurtosis (EK) indicates non-Gaussian four-point correlations. For both activation functions, the observed behavior of the EK with depth is in good accord with theoretical expectations.

Next we considered training of the ANN. We followed the mean absolute error (MAE) loss for a large number of epochs with several different depths, using combinations of critical and non-critical initialization and training. The networks with critical initialization and critical training (CICT) reached the lowest loss in all cases both for ReLU and Tanh activation functions, with particular contrast to when non-critical hyperparameters were used for both initialization and training.

Most practitioners do not encounter these issues even though they rarely pay attention to details of initialization and training. This is because the critically tuned networks are still outperformed by adaptive optimizers. Nevertheless, the results are comparable, and differ by a much smaller factor than when untuned.

Of particular interest is the prediction by ANNFT of different training regimes, characterized by the ratio of depth to width r. The predictive analyses of ANNFT are based on the validity of a Taylor series expansion about the initialization. By considering the mean hidden-layer rms deviation and Frobenius norms of the weight matrices before and after training, we can see that the conditions of small deviations from initialization are achieved with sufficiently small r. The different learning regimes are manifested by looking at the rms deviation of the binding energy data and ANN outputs as a function of r, achieved by varying the width at fixed depth, and through heatmaps of global residuals across all N and Z for four different r values.

The results validated that ANNFT offers insight into

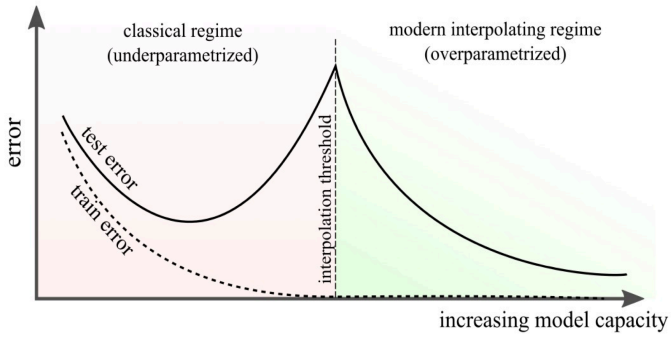
- why networks that are wider than they are deep are preferred, as they will have correlations in their output distribution attenuated by the larger width;
- the success of activation functions such as ReLU or Tanh compared to Softplus or sigmoid;
- the success of certain initialization schemes (He initialization is reproduced by ReLU's critical initialization).

We restricted the training protocol in this work to facilitate a clean analysis of criticality, but the door is open to explore heuristically successful approaches to network training in the context of ANNFT.

Overparametrization

Roberts and Yaida (2021)

- Naïve expectation: overparameterized models will overfit training data, so generalize poorly
- In practice, overparametrized neural networks are empirically successful; how to resolve?
- Effective theory perspective: measure of model complexity is *not* number of model parameters but the depth-to-width ratio *r*
- I.e., what matters is number of data-dependent couplings of the truncated (nearly Gaussian) effective theory distribution



By Dkarkada - Own work, CC BY-SA 4.0,

33.9. Appendix: field theory formalism

Training a GP in path integral form

- Given training data at n_{data} points ($\{x_i, y_i\}$) ≡ (X, Y) with gaussian noise σ_n
- Implement Bayes' theorem: $\text{pr}(A|B,I) = \frac{\text{pr}(B|A,I)\text{pr}(A|I)}{\text{pr}(B|I)} \Longrightarrow \underbrace{\text{pr}(\boldsymbol{\theta}|\mathbf{y}_{\text{exp}},I)}_{\text{posterior}} \propto \underbrace{\text{pr}(\mathbf{y}_{\text{exp}}|\boldsymbol{\theta},I)}_{\text{likelihood}} \times \underbrace{\text{pr}(\boldsymbol{\theta}|I)}_{\text{prior}}$ Bayesian updating of knowledge
- For GP with kernel *K* and prior zero mean function → $\langle \tilde{f} \rangle = K(x,X)\Big(K(X,X) + \sigma_{\text{n}}^2\Big)^{-1} Y$
- Generally: posterior for *f(x)* given data: $p\big(f(x)|\text{Data}\big) \propto p\big(\text{Data}|f(x)\big)\, p\big(f(x)\big)$
- For path integral in large width limit:
  - Prior is free field theory without sources: $S_{\text{prior}}[f] = \frac{1}{2}\int dx\, dx'\, f(x) K^{-1}(x,x') f(x')$
  - Likelihood is induced by the training points: → add source term → shifts mean $S_{\text{data}}[f] = \frac{1}{2\sigma_n^2}\sum_i \big(y_i - f(x_i)\big)^2$
- New mean (expectation value of *f*) and variance after training → minimize action for mean
- Same formula as GP updating!

33.10. 📥 Demo: Distributions of Randomly-Initialized ANNs

This notebook builds a feed-forward network using PyTorch with configurable width, depth, and activation fuction; samples many random initializations (no training); and plots a histogram of the outputs for a fixed input. A normal curve with the same empirical mean and standard deviation is overlaid.

The initialization variance for the weights in the hidden layers is scaled by 1/width. The bias is zero. Under these conditions, the output distribution (the networks constructed here will always have one output with user-selected number of inputs) will approach a Gaussian distribution in the limit of large width. This is a consequence of the central limit theorem (CLT).

ChatGPT prompts for original code

These are the prompts used to get started.

"Make me a pytorch script that takes as input a feed-forward network width and depth with one input and one output and a selected activation function. It should plot a histogram of the distribution of a large number of outputs with initialization of the weights and bias as mean zero Gaussians of specified width. There should be no training, only initialization. On the plot should also be plotted a normal distribution with the same mean and standard deviation as the output data."

"Now set this up as a jupyter notebook."

The notebook was edited to apply the scaling of the weight variance and several functions were created to consolidate the running of the model and subsequent plotting.

Things to try:

- *Vary the width for a given depth and activation function. Try widths of 1, 10, 120, for example.*
- *Compare results with different activation functions. Do they all approach the CLT at the same rate?*
- *How does the behavior of the output distribution depend on the depth?*

Code

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
```

```
def build_model(width, depth, activation, in_features=1):
    """
    Build the PyTorch model with given specifications:
      - width --- integer width of each hidden layer
      - depth --- integer depth of layers
      - activation --- activation function, from a list
    """
    layers = []
    act = activation.lower()
    for _ in range(depth):
        layers.append(nn.Linear(in_features, width))
        if   act == 'relu':       layers.append(nn.ReLU())
        elif act == 'tanh':       layers.append(nn.Tanh())
        elif act == 'sigmoid':    layers.append(nn.Sigmoid())
        elif act == 'elu':        layers.append(nn.ELU())
        elif act in ('leaky_relu','leaky-relu'):
                                   layers.append(nn.LeakyReLU())
        elif act in ('none','linear'):
                                  pass
        else:
            raise ValueError(f"Unknown activation '{activation}'")
        in_features = width

    layers.append(nn.Linear(in_features, 1))
    return nn.Sequential(*layers)

def initialize_model(model, w_std, b_std):
    """
    Initialize the ANN with given weight and bias std's.
    The means for each distribution are zero.
    """
    for m in model.modules():
        if isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, mean=0.0, std=w_std)
            nn.init.normal_(m.bias,   mean=0.0, std=b_std)
```

```
def run_model(width, depth, activation, weight_std, bias_std, in_features, input_val,
n_samples=20000):
    """
    Build and run the model given all the inputs for n_samples samples.
    Using torch.no_grad() for initialization only.
      input_val --- float input value to the ANN
    """
    outputs = np.empty(n_samples, dtype=np.float32)
    x_tensor = torch.tensor([input_val], dtype=torch.float32)

    for i in range(n_samples):
        net = build_model(width, depth, activation, in_features)
        initialize_model(net, weight_std, bias_std)
        with torch.no_grad():
            outputs[i] = net(x_tensor).item()

    # get the mean and std of the distribution of outputs
    mu, sigma = outputs.mean(), outputs.std()
    print(f"Sampled {n_samples} outputs →  mean = {mu:.4f}, std = {sigma:.4f}")

    return outputs, mu, sigma
```

```
def make_plot(width, depth, activation, outputs, mean, std,
              bins=50, range=(-.5,.5), xlims=None, ylims=None):
    """
    Plot a histogram with an overlaid normal distribution with the same mean and std.
       bins --- integer number of histogram bins
       range --- tuple of floats giving the limits of the bins
       xlim --- tuple of minimum and maximum x (default autoscale)
       ylim --- tuple of minimum and maximum y (default autoscale)
    """
    fig, ax = plt.subplots(figsize=(8,5))
    ax.hist(outputs, bins=bins, range=range, density=True, alpha=0.6, label="Network outputs")

    xs = np.linspace(mean - 4*std, mean + 4*std, 200)
    ys = 1/(std*np.sqrt(2*np.pi)) * np.exp(-0.5*((xs-mean)/std)**2)
    ax.plot(xs, ys, 'r--', lw=2,
            label=f"Normal(μ={mean:.3f}, σ={std:.3f})")

    ax.set_title(f"Depth={depth}, Width={width}, Activation={activation}")
    ax.set_xlabel("Output value")
    ax.set_ylabel("Density")
    if (xlims):
        ax.set_xlim(xlims)
    if (ylims):
        ax.set_ylim(ylims)
    ax.legend()
    fig.tight_layout()
    return fig, ax
```

Output Distribution of Randomly-Initialized feed-forward ANN (1 Input → 1

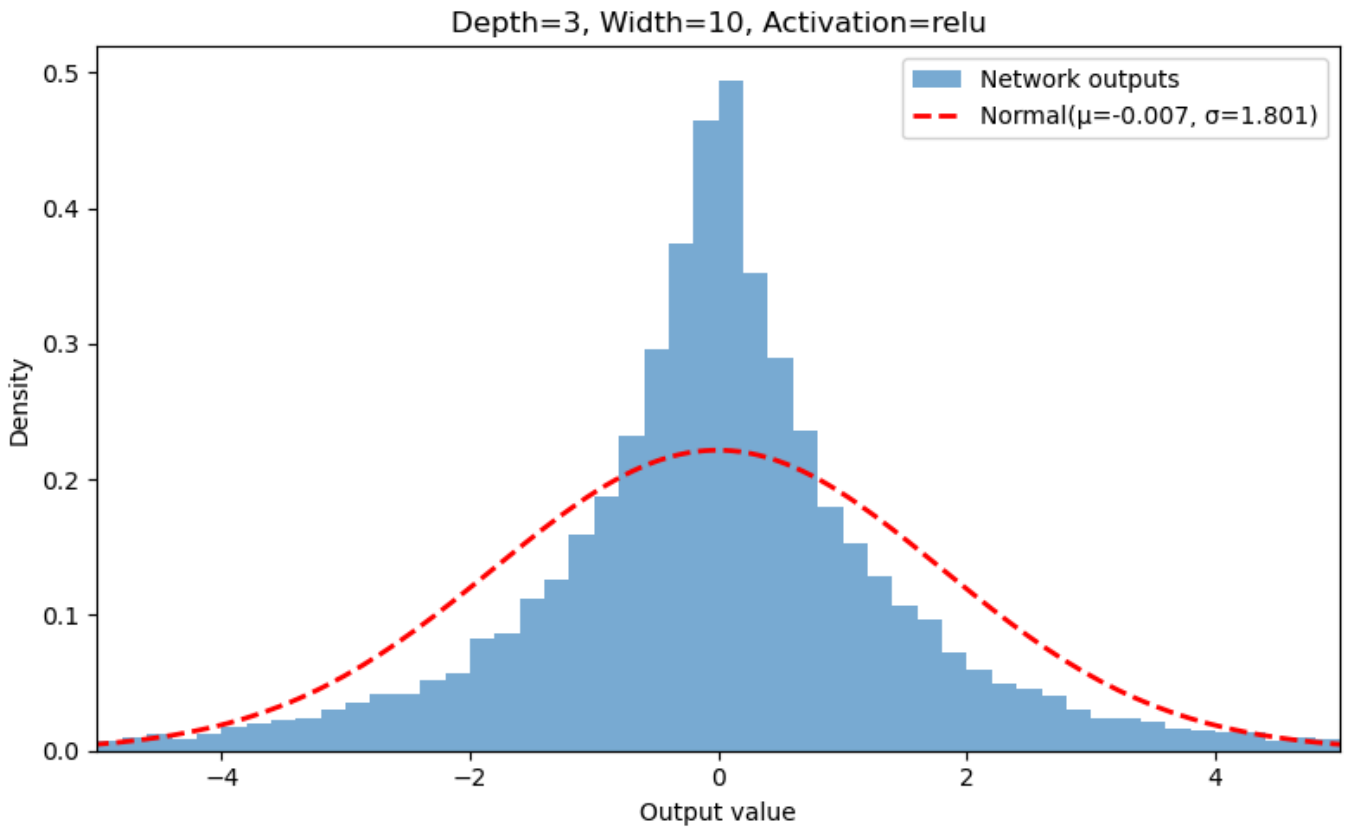
Output)

```
# ——— USER CONFIG ————————————————————————————————
width       = 10      # neurons per hidden layer
depth       = 3        # number of hidden layers
activation  = 'relu'   # relu, tanh, sigmoid, elu, leaky_relu, none
weight_std  = 2.0 / np.sqrt(width)     # stddev for weight init, variance normalized by 1/width
bias_std    = 0.0   # stddev for bias init
n_samples   = 20000    # number of random networks to sample
in_features = 1       # number of input features
input_val   = 1.      # scalar input to feed-forward
# ————————————————————————————————————————————————
```

```
my_output, my_mu, my_sigma = run_model(width, depth, activation, weight_std, bias_std,
in_features, input_val, n_samples)
```

```
Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled
only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require
Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.
Sampled 20000 outputs →  mean = -0.0069, std = 1.8009
```

```
my_fig, my_ax = make_plot(width, depth, activation, my_output, my_mu, my_sigma,
                          bins=50, range=(-5,5), xlims=(-5,5))
```



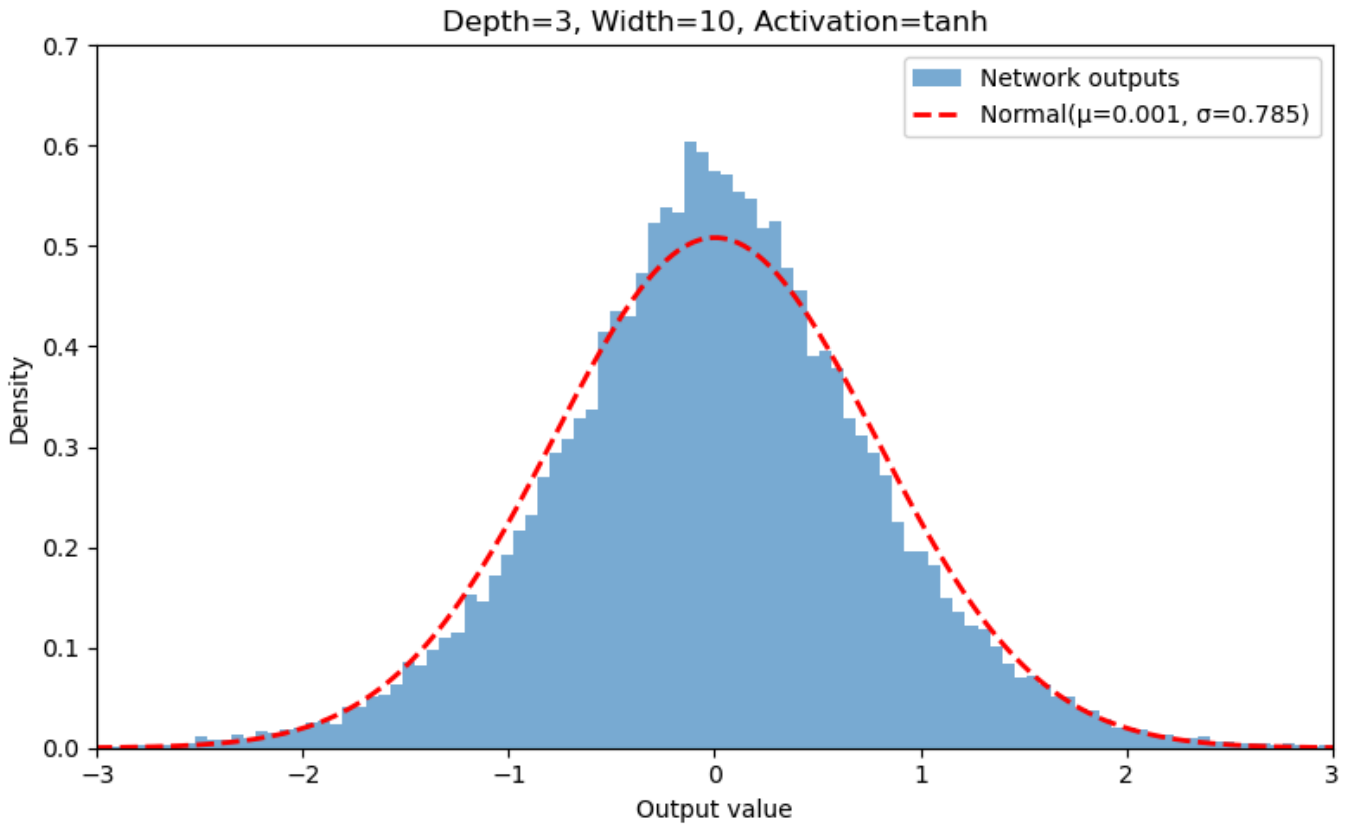
Output Distribution of Randomly-Initialized feed-forward ANN (2 Inputs → 1 Output)

```
# ─── USER CONFIG ──────────────────────────────────────────
width       = 10            # neurons per hidden layer
depth       = 3            # number of hidden layers
activation  = 'tanh'       # relu, tanh, sigmoid, elu, leaky_relu, none
weight_std  = 2.0 / np.sqrt(width)         # stddev for weight init with variance * 1/width
bias_std    = 0.0          # stddev for bias init
n_samples   = 40000        # number of random networks to sample
in_features = 2            # number of input features
input_val  = [0.13, 0.16]  # 2-dimensional input vector
# ──────────────────────────────────────────────────────────
```

```
my_output, my_mu, my_sigma = run_model(width, depth, activation, weight_std, bias_std,
in_features, input_val, n_samples)
```

```
Sampled 40000 outputs →  mean = 0.0014, std = 0.7845
```

```
my_fig, my_ax = make_plot(width, depth, activation, my_output, my_mu, my_sigma,
                          bins=101, range=(-3,3), xlims=(-3,3), ylims=(0,0.7))
```



34. Bayesian optimization

Let us first state an inconvenient fact about optimization:

> Global minimization is almost always intractable. In practice, we have to resort to local minimization:

For $f: \mathbf{R}^D \to \mathbf{R}$, with $\boldsymbol{\theta} \in \Theta \subset \mathbf{R}^D$ and possibly subject to constraints $c(\boldsymbol{\theta}) \leq 0$

Find point(s) $\boldsymbol{\theta}_*$ for which

$$f(\boldsymbol{\theta}_*) \leq f(\boldsymbol{\theta}),$$

for all $\boldsymbol{\theta} \in \Theta$ *close* to $\boldsymbol{\theta}_*$. (Here $\boldsymbol{\theta}$ are the parameters of the theoretical model.)

Nevertheless, we will often want to do the best we can toward global minimization.

Consider **expensive** objective functions, e.g.

$$f(\theta) = \chi^2(\theta) \equiv \sum_{i=1}^{N} \frac{\left[y_i^{\mathrm{exp}} - y_i^{\mathrm{th}}(\theta)\right]^2}{\sigma_i^2},$$

where $y_i^{\mathrm{th}}(\theta)$ may be computationally costly to evaluate. (The objective function is the function we want to minimize, such as a χ^2 function.) How shall we proceed? Here we consider one strategy, Bayesian optimization, which has been used in the optimization of hyperparameters of deep neural networks. It is not necessarily the best strategy (see comments at the end), but it is an option in our toolkit.

Selected references:

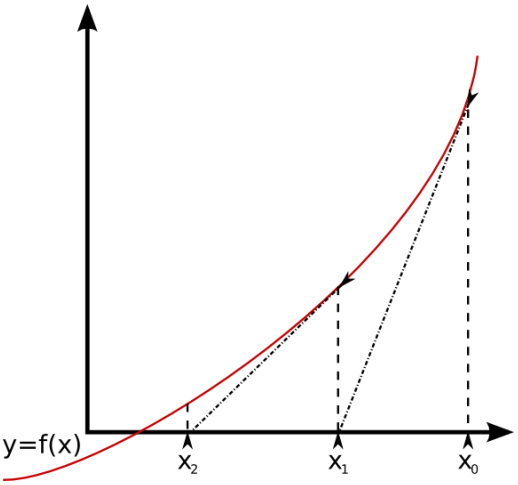
- Paper: Bayesian optimization in ab initio nuclear physics by A. Ekström, C. Forssén et al., J. Phys. G: Nucl. Part. Phys. 46, 095101 (2019).
- Book: Jonas Mockus (2012). Bayesian approach to global optimization: theory and applications. Kluwer Academic.

34.1. Overview of Bayesian optimization

> An optimization algorithm for expensive black-box functions

Bayesian optimization provides a strategy for selecting a sequence of function queries. This is an alternative to a gradient descent method, which relies on derivatives of the function to move toward a nearby local minimum.

This diagram from Wikimedia Commons illustrates the sequential moves in Newton's method for finding a root, with a sequence of x values: $x_0 \to x_1 \to x_2 \to \ldots$ that approach a nearby zero of the function.



If applied to the derivative of a function, this yields a sequential approximation to a local minimum or maximum. We seek a different way to construct this sequence.

There are two main components in the Bayesian optimization algorithm, which will each be contingent on what we call the data $\mathcal{D}$, which will be the previously evaluated pairs of θ and $f(\theta)$.

1. A prior probabilistic pdf $p(f|\mathcal{D})$ for the objective function $f(\theta)$ given some data $\mathcal{D}$. The prior is often a Gaussian process (GP) or a GP emulator. This will be updated with every iteration. The GP acts as a *surrogate* for the actual objective function, which we can evaluate at any given θ, but it is so expensive to do so that we don't know its structure a priori. The idea is that we will do Bayesian updating each iteration after confronting the surrogate model with actual data $f(\theta)$. So we refine the posterior for f, which becomes the prior for the next iteration.
2. An acquisition function $\mathcal{A}(\theta|\mathcal{D})$ given some data $\mathcal{D}$. This is a heuristic that balances *exploration* against *exploitation* and determines where to evaluate the objective function $f(\theta)$ next. Explorative means that you evaluate points in a domain of θ where the prior pdf for f has large uncertainty. This enables the escape from local minima in θ. Exploitive means that you evaluate points in a domain of θ where the prior pdf for f exhibits low mean values. (This zeroes in on a local minimum.)

Pseudo-code for BayesOpt:

1. choose initial $\theta^{(1)}, \theta^{(2)}, \ldots \theta^{(k)}$, where $k \geq 2$
2. evaluate the objective function $f(\theta)$ to obtain $y^{(i)} = f(\theta^{(i)})$ for $i = 1, \ldots, k$
3. initialize a data vector $\mathcal{D}_k = \left\{(\theta^{(i)}, y^{(i)})\right\}_{i=1}^{k}$
4. select a statistical model for $f(\theta)$ (for us some choice of GP kernel)
5. **for** $\{n = k+1, k+2, \ldots\}$
    1. select $\theta^{(n)}$ by optimizing (maximizing) the acquisition function
    2. $\theta^{(n)} = \arg\max_{\theta} \quad \mathcal{A}(\theta|\mathcal{D}_{n-1})$
    3. evaluate the objective function to obtain $y^{(n)} = f(\theta^{(n)})$
    4. augment the data vector $\mathcal{D}_n = \left\{\mathcal{D}_{n-1}, (\theta^{(n)}, y^{(n)})\right\}$
    5. update the statistical model for $f(\theta)$
6. **end for**

Some remarks:

- Use a space-filling method such as LHS (Latin hypercube sampling) or Sobol (see references) for the initial k evaluations.

- Use of a gaussian process, or a Gaussian emulator, for the statistical model of $f(\theta)$.
- The choice of aquisition function is the heart of BayesOpt. There are several possible choices; with different balance between exploration-exploitation.
  - Expected improvement
  - Lower confidence bound
- The update of the statistical model is an $\mathcal{O}(n^3)$ cost (if using a GP).
- At each iteration, *all* collected data points are used, thus we build on the full history of the optimization procedure.
- The stopping criterion might be a fixed computational budget that limits the number of function evaluations that can be made.

34.2. Acquisition functions

We will consider two different acquisition functions:

- Expected Improvement (EI)
- Lower Confidence Bound (LCB)

Note that we abbreviate the notation below and write $\mathcal{A}(\theta) \equiv \mathcal{A}(\theta|D)$.

Expected Improvement

The expected improvement acquisition function is defined by the expectation value of the rectifier $\max(0, f_{\min} - f(\theta))$, i.e. we reward any expected reduction of f in proportion to the reduction $f_{\min} - f(\theta)$. This can be evaluated analytically

$$\begin{aligned}
\mathcal{A}_{\mathrm{EI}}(\theta) = \langle \max(0, f_{\min} - f(\theta)) \rangle &= \int_{-\infty}^{\infty} \max(0, f_{\min} - f) \mathcal{N}(f(\theta)|\mu(\theta), \sigma(\theta)^2)\ df(\theta) \\
&= \int_{-\infty}^{f_{\min}} (f_{\min} - f) \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(f-\mu)^2}{2\sigma^2} \right] df \\
&= (f_{\min} - \mu)\Phi\left(\frac{f_{\min} - \mu}{\sigma} \right) + \sigma\phi\left(\frac{f_{\min} - \mu}{\sigma} \right) \\
&= \sigma\left(z\Phi(z) + \phi(z) \right),
\end{aligned}$$

where

$$\mathcal{N}(f(\theta)|\mu(\theta), \sigma(\theta)^2)$$

indicates the density function of the normal distribution, whereas the standard normal distribution and the cumulative distribution function are denoted ϕ and Φ, respectively, and we dropped the explicit dependence on θ in the third step.

In the last step we write the result in the standard normal variable $z = \frac{f_{\min} - \mu}{\sigma}$. BayesOpt will exploit regions of expected improvement when the term $z\Phi(z)$ dominates, while new, unknown regions will be explored when the

second term $\phi(z)$ dominates. For the expected improvement acquisition function, the exploration-exploitation balance is entirely determined by the set of observed data $\mathcal{D}_n$ and the $\mathcal{GP}$ kernel.

Note 1: Density function of the normal distribution: $\mathcal{N}(\theta|\mu,\sigma^2) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{1}{2\sigma^2}(\theta-\mu)^2\right)$

Note 2: Density function of the *standard* normal distribution: $\phi(z) \equiv \mathcal{N}(z|\mu=0,\sigma^2=1) = \frac{1}{\sqrt{2\pi}}\exp\left(-\frac{1}{2}z^2\right)$

Note 3: Cumulative distribution function of the standard normal: $\Phi(z) = \frac{1}{\sqrt{2\pi}}\int_{-\infty}^{z}\exp\left(-\frac{t^2}{2}\right)dt$

Lower Confidence Bound

The lower confidence-bound acquisition function introduces an additional parameter β that explicitly sets the level of exploration

$$\mathcal{A}(\theta)_{\mathrm{LCB}} = \beta\sigma(\theta) - \mu(\theta).$$

The maximum of this acquisition function will occur for the maximum of the β-enlarged confidence envelope of the $\mathcal{GP}$. We use $\beta = 2$, which is a very common setting. Larger values of β leads to even more explorative BayesOpt algorithms.

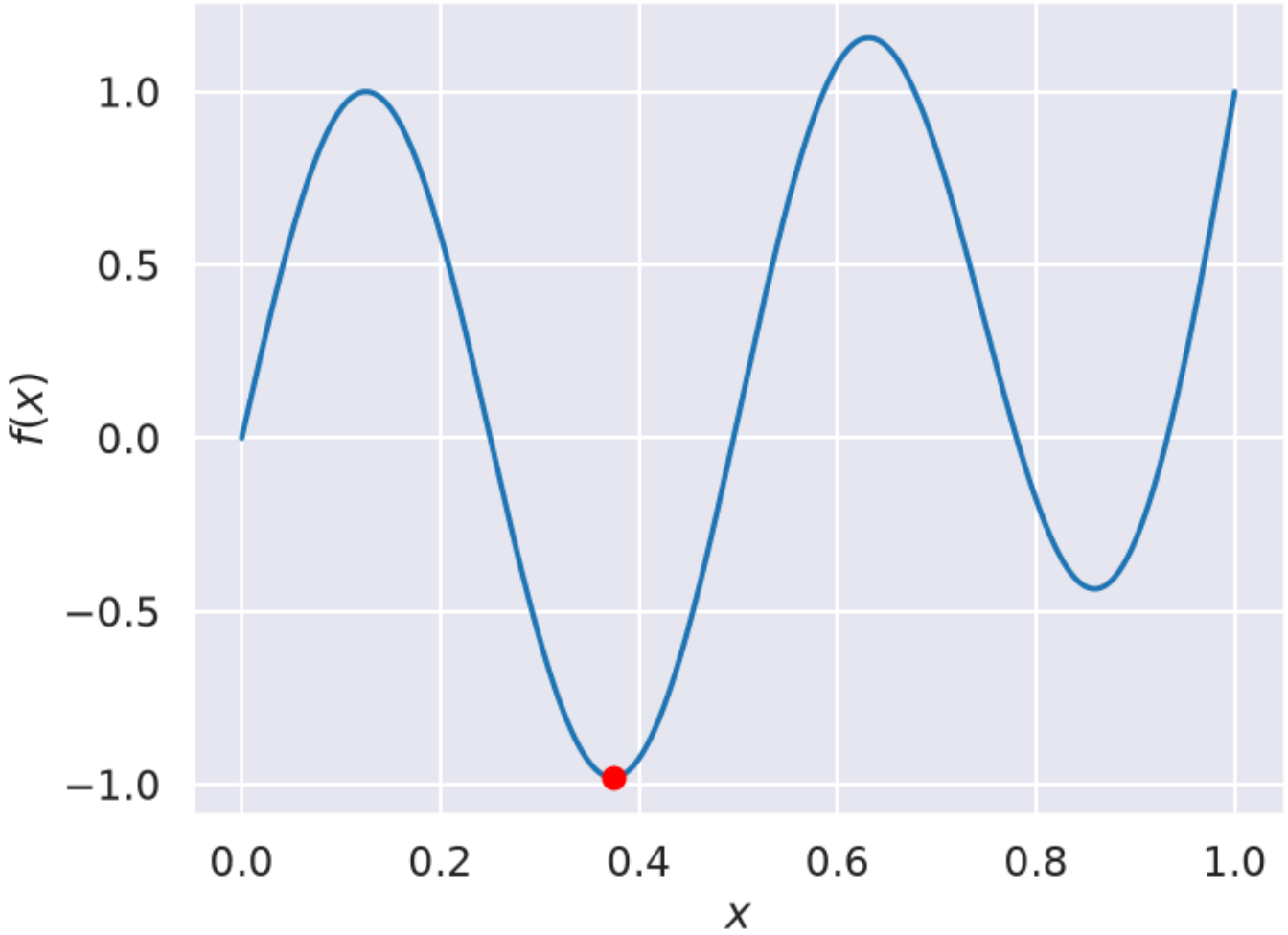
34.3. Example of Bayesian optimization

Univariate example

Here there is only one parameter: $\theta = \{x\}$.

▼ **Hide code cell source**

```
SklearnBO._update_state patched to always fill history and _convergence.
  message: Optimization terminated successfully.
  success: True
   status: 0
      fun: -0.9803640091846901
        x: [ 3.737e-01]
      nit: 5
      jac: [-7.674e-07]
 hess_inv: [[ 6.266e-03]]
     nfev: 14
     njev: 7
```



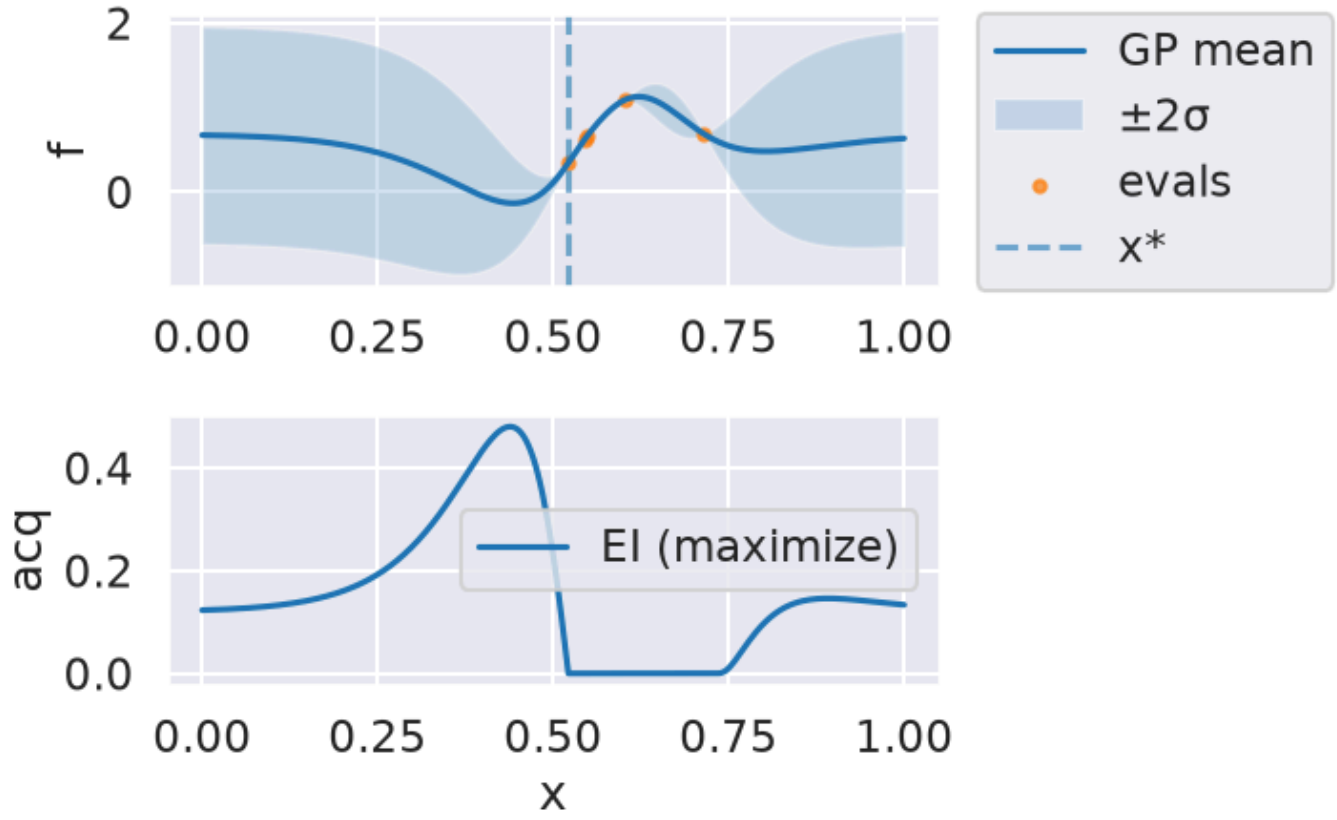
Now we can use the `myBopt.run` one step at a time (meaning we add one point per iteration), plotting the GP mean (solid black line) and 95% variance (gray line) and the acquisition function in red using `plot_acquisition`. The maximum of the acquisition function, which will be the choice for x in the next iteration, is marked by a vertical red line.

What can you tell about the GP used as a surrogate for $f(x)$?

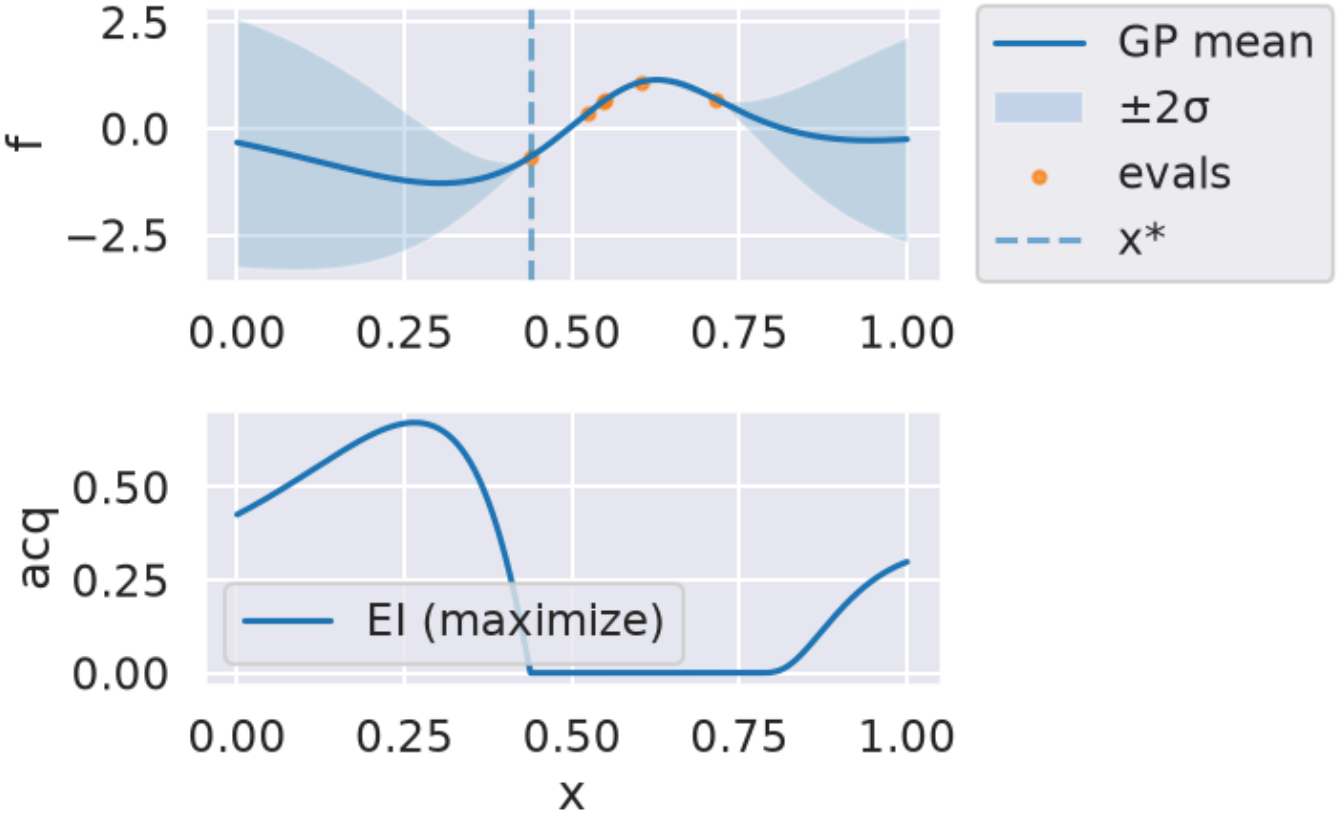
Note how the GP is refined (updated) with additional points.

▼ **Hide code cell source**

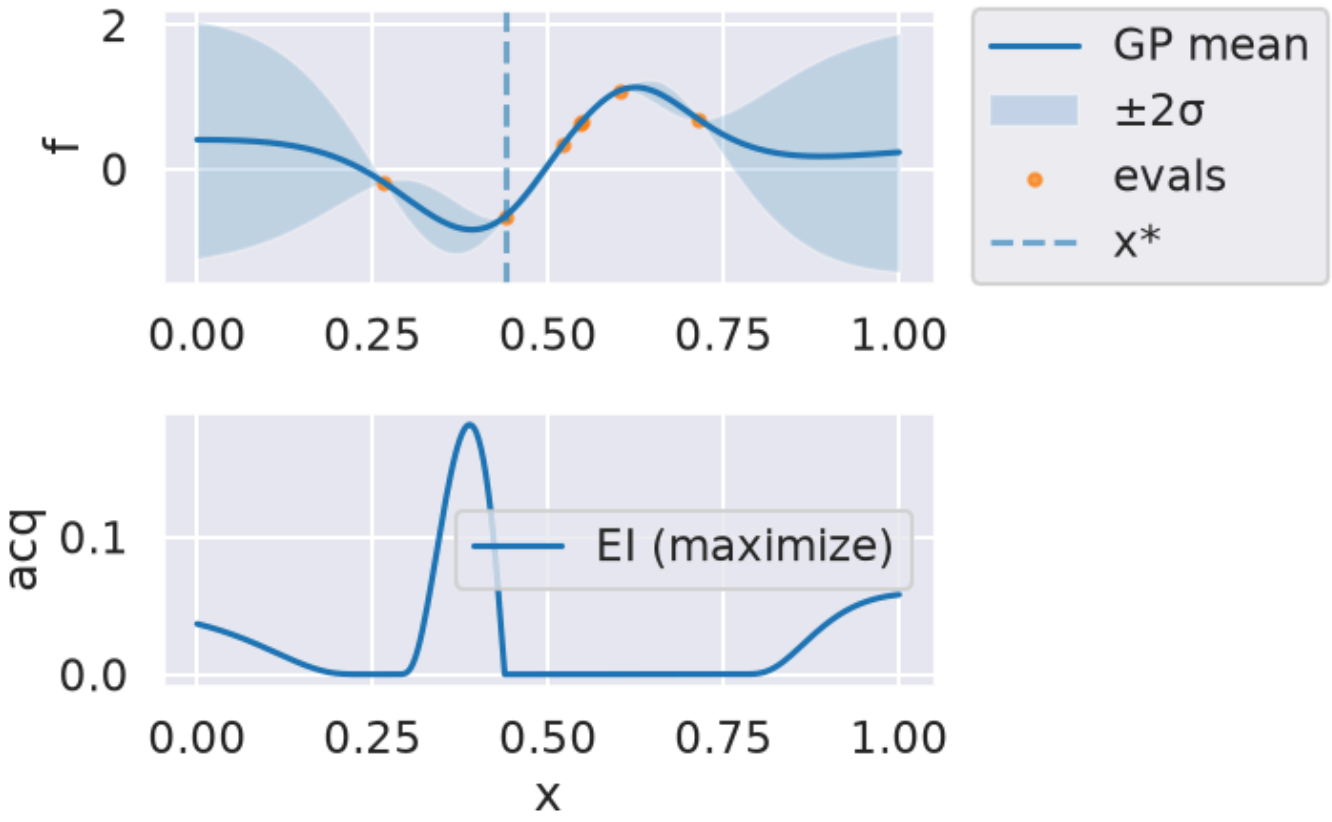
```
Starting with 4 points
Iteration  0
```


2
0
f
0.00
0.25
0.50
0.75
1.00
GP mean
±2σ
evals
x*
0.4
0.2
0.0
acq
EI (maximize)
0.00
0.25
0.50
0.75
1.00
x

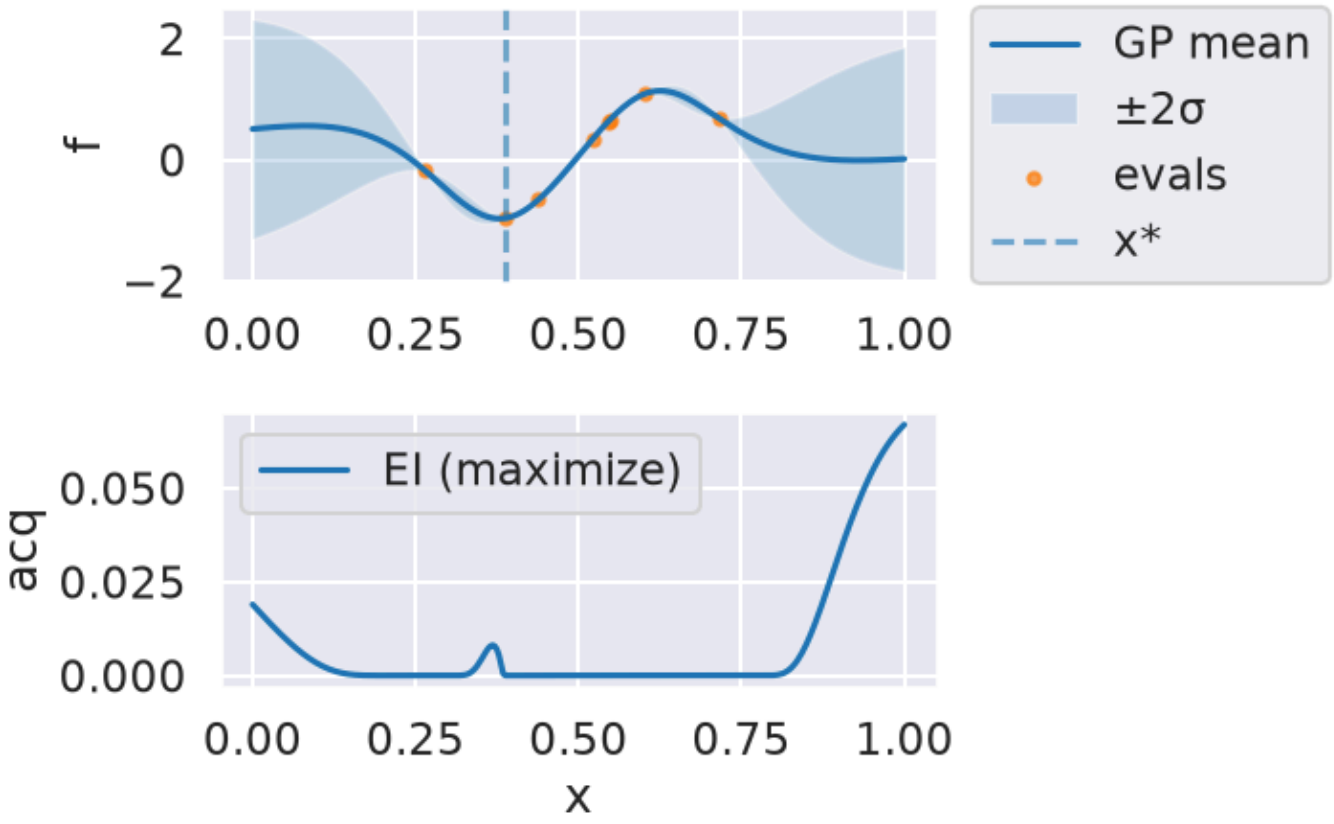
```
Iteration  1
```


2.5
0.0
−2.5
f
0.00
0.25
0.50
0.75
1.00
GP mean
±2σ
evals
x*
0.50
0.25
0.00
acq
EI (maximize)
0.00
0.25
0.50
0.75
1.00
x

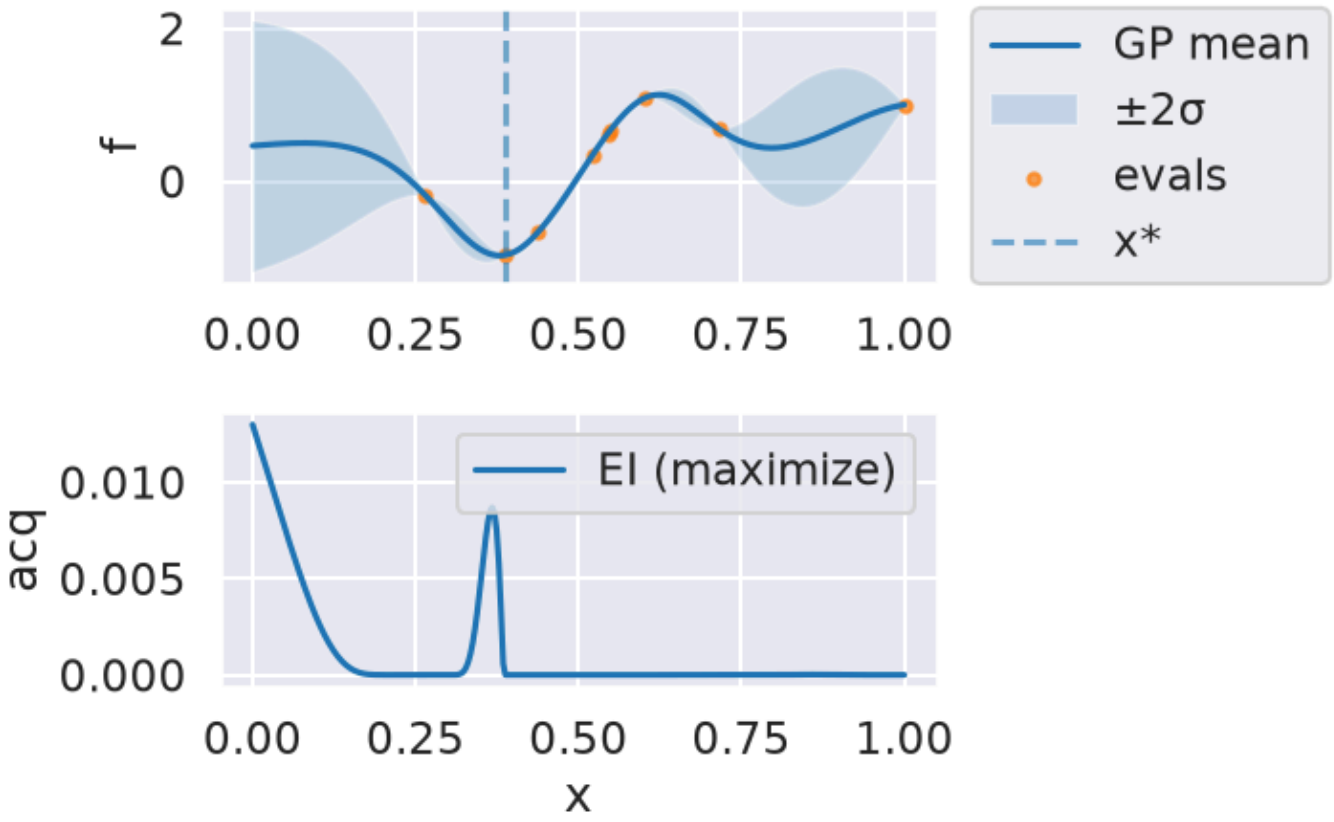
```
Iteration  2
```



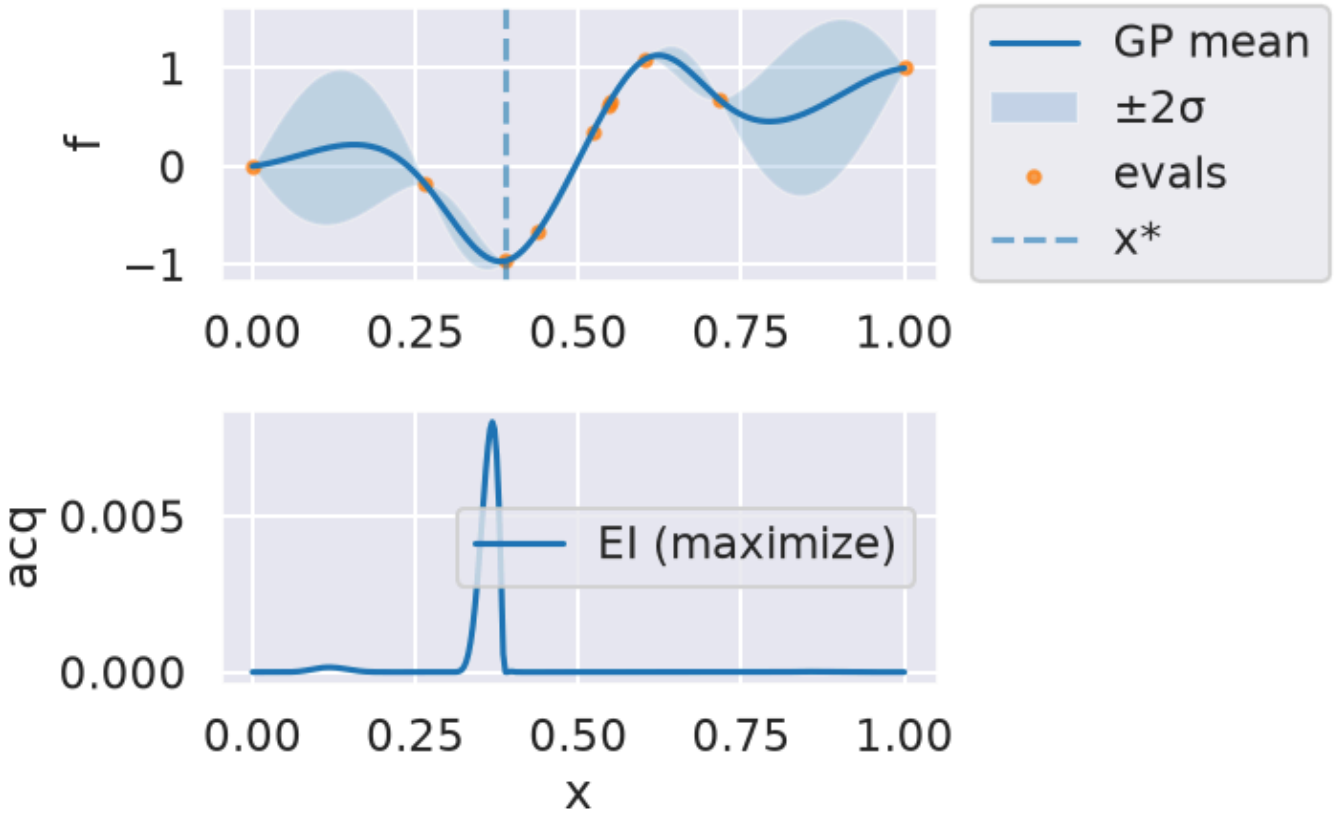

```
Iteration  3
```


GP mean
±2σ
evals
x*
f
2
0
−2
0.00
0.25
0.50
0.75
1.00
EI (maximize)
acq
0.050
0.025
0.000
x

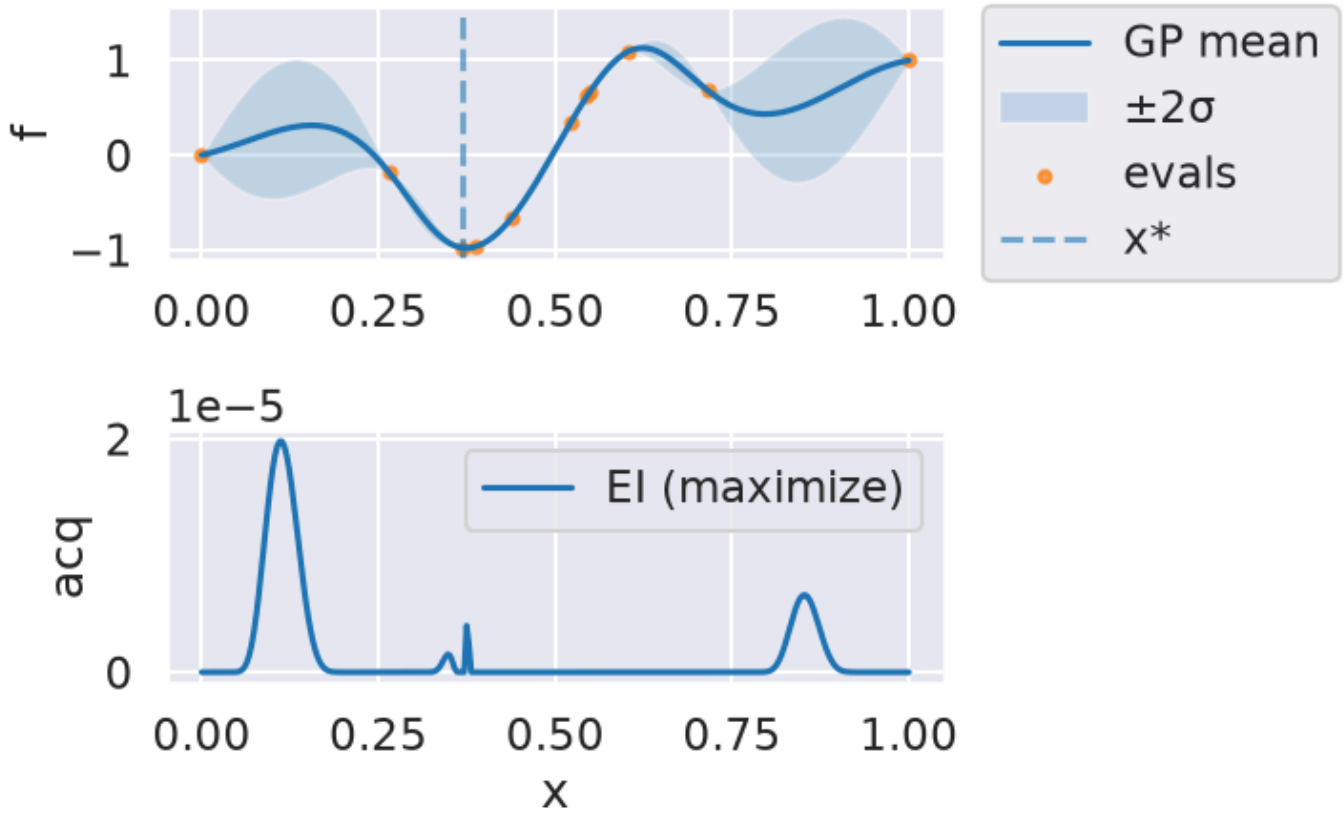
```
Iteration  4
```


GP mean
±2σ
evals
x*
f
2
0
0.00
0.25
0.50
0.75
1.00
EI (maximize)
acq
0.010
0.005
0.000
x

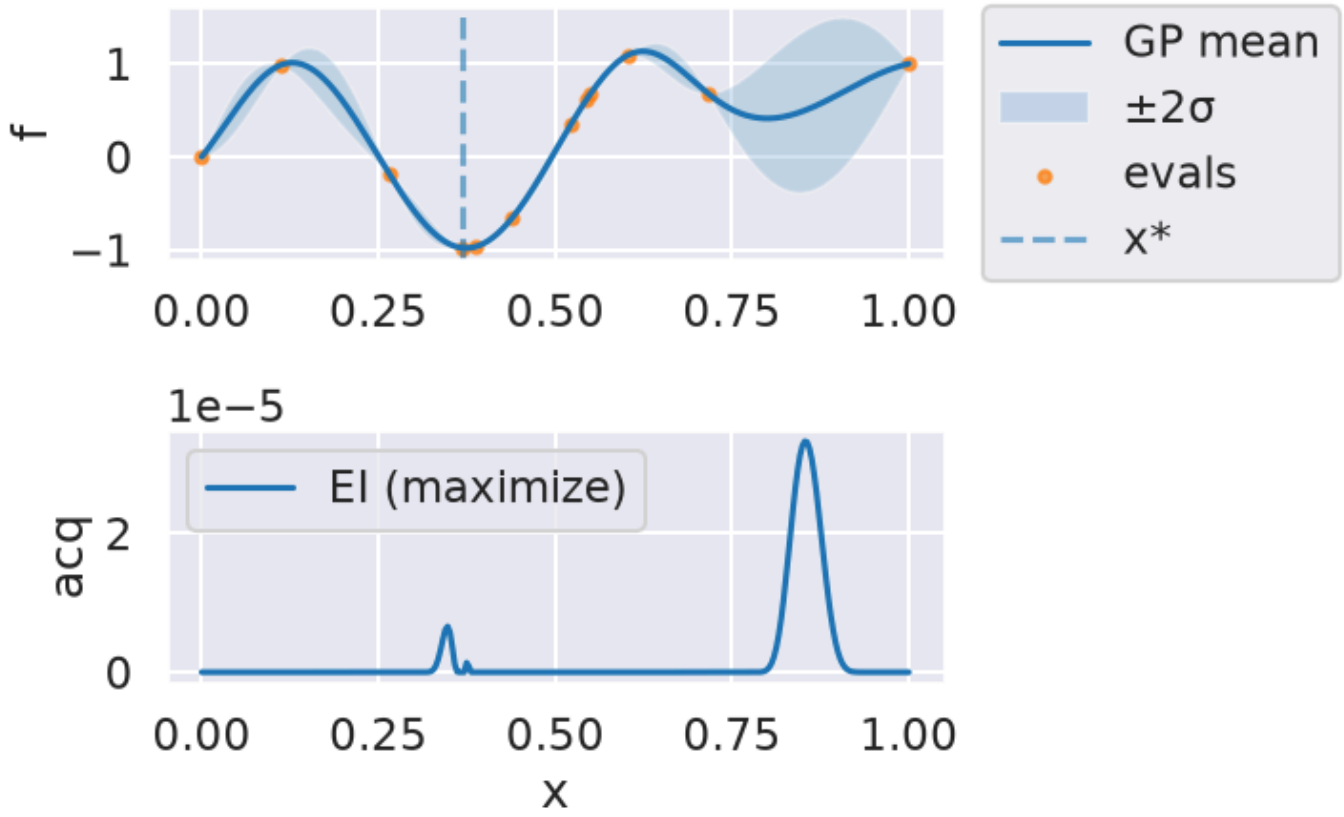
```
Iteration  5
```



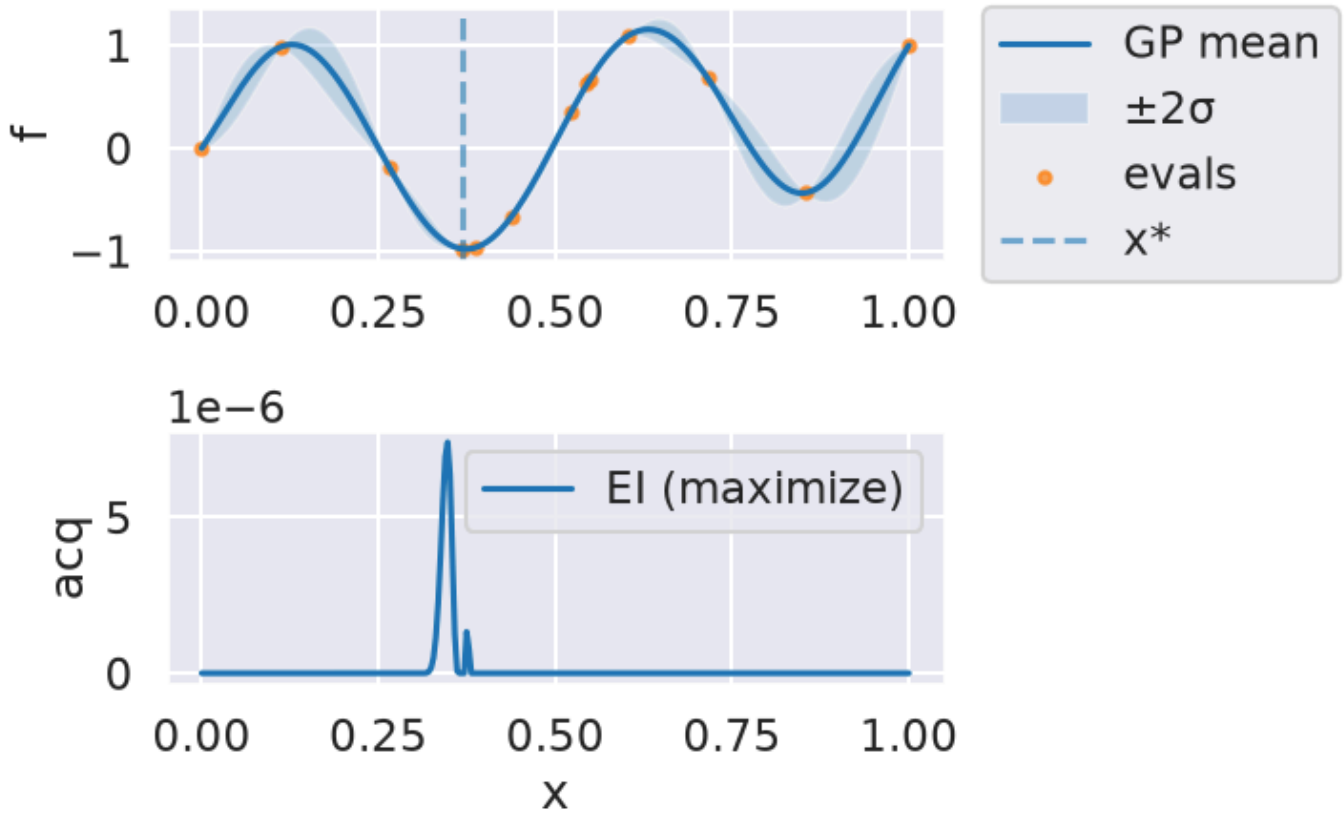

```
Iteration  6
```


GP mean
±2σ
evals
x*
f
1
0
−1
0.00
0.25
0.50
0.75
1.00
1e−5
2
0
acq
EI (maximize)
0.00
0.25
0.50
0.75
1.00
x

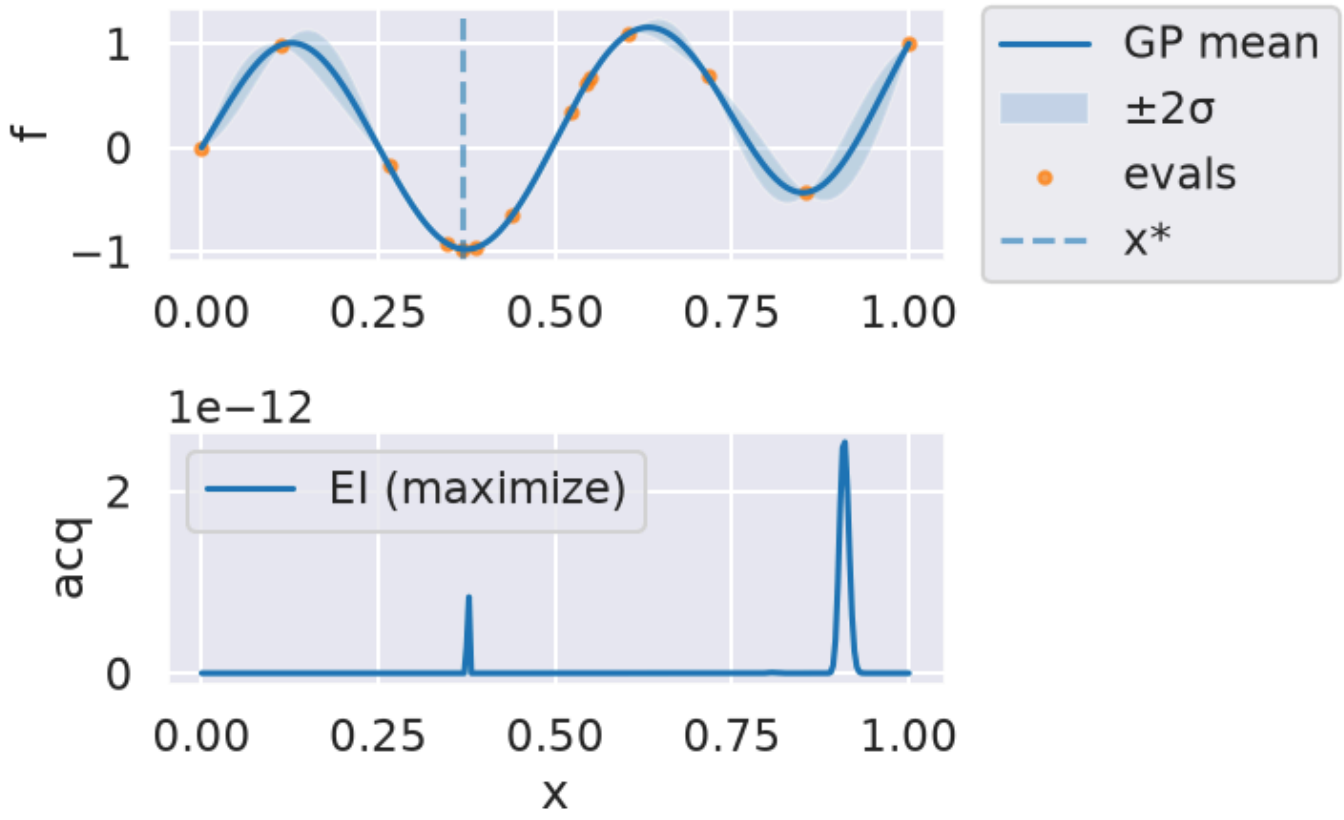
```
Iteration  7
```


GP mean
±2σ
evals
x*
f
1
0
−1
0.00
0.25
0.50
0.75
1.00
1e−5
2
0
acq
EI (maximize)
0.00
0.25
0.50
0.75
1.00
x

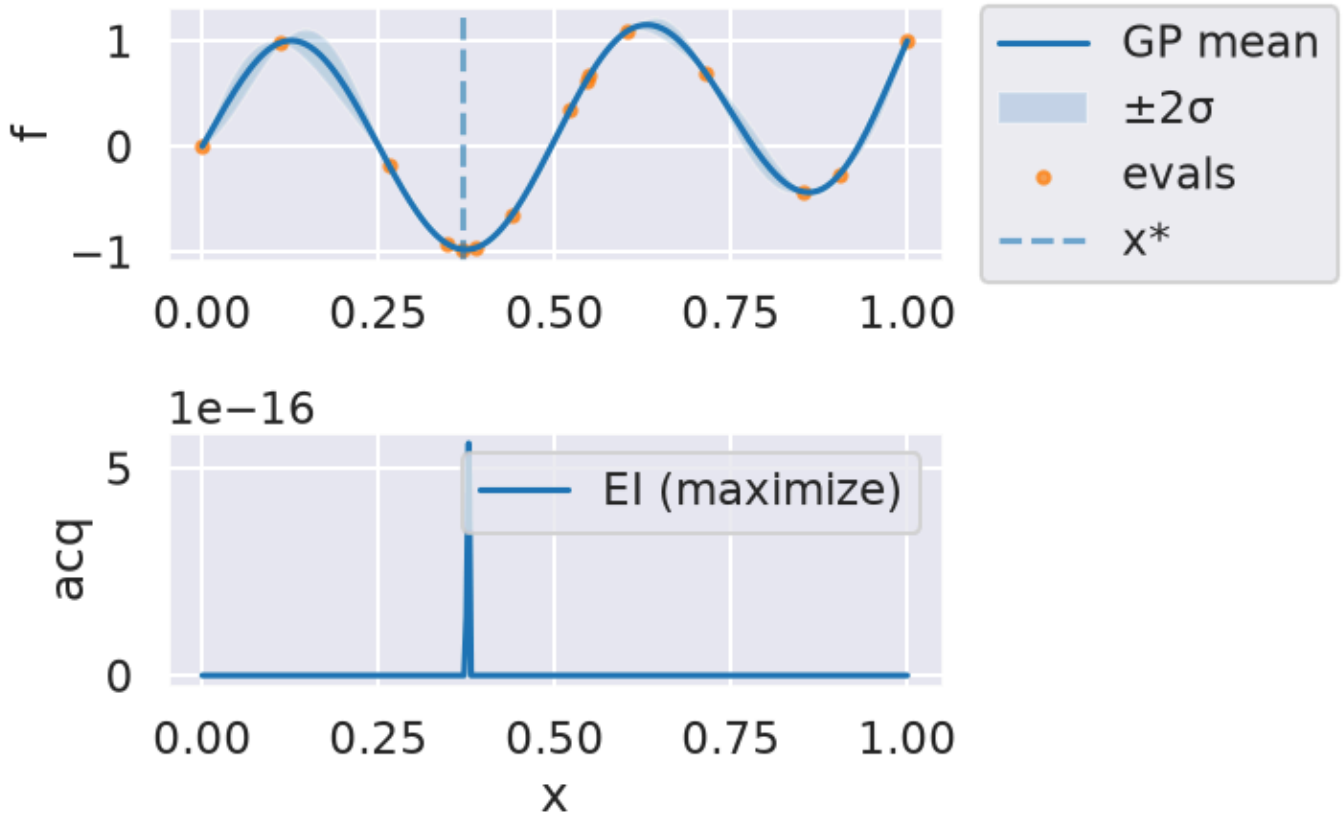
```
Iteration  8
```



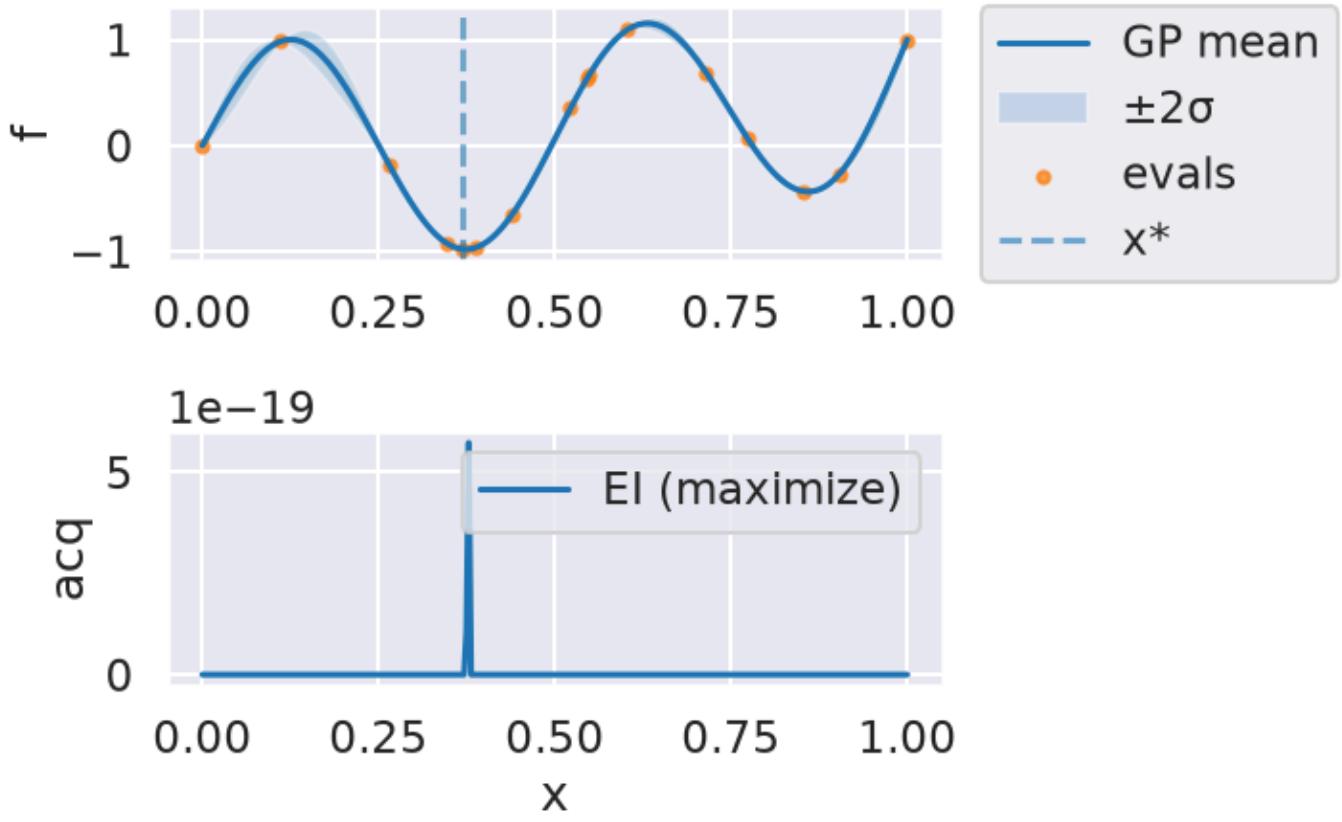

```
Iteration  9
```



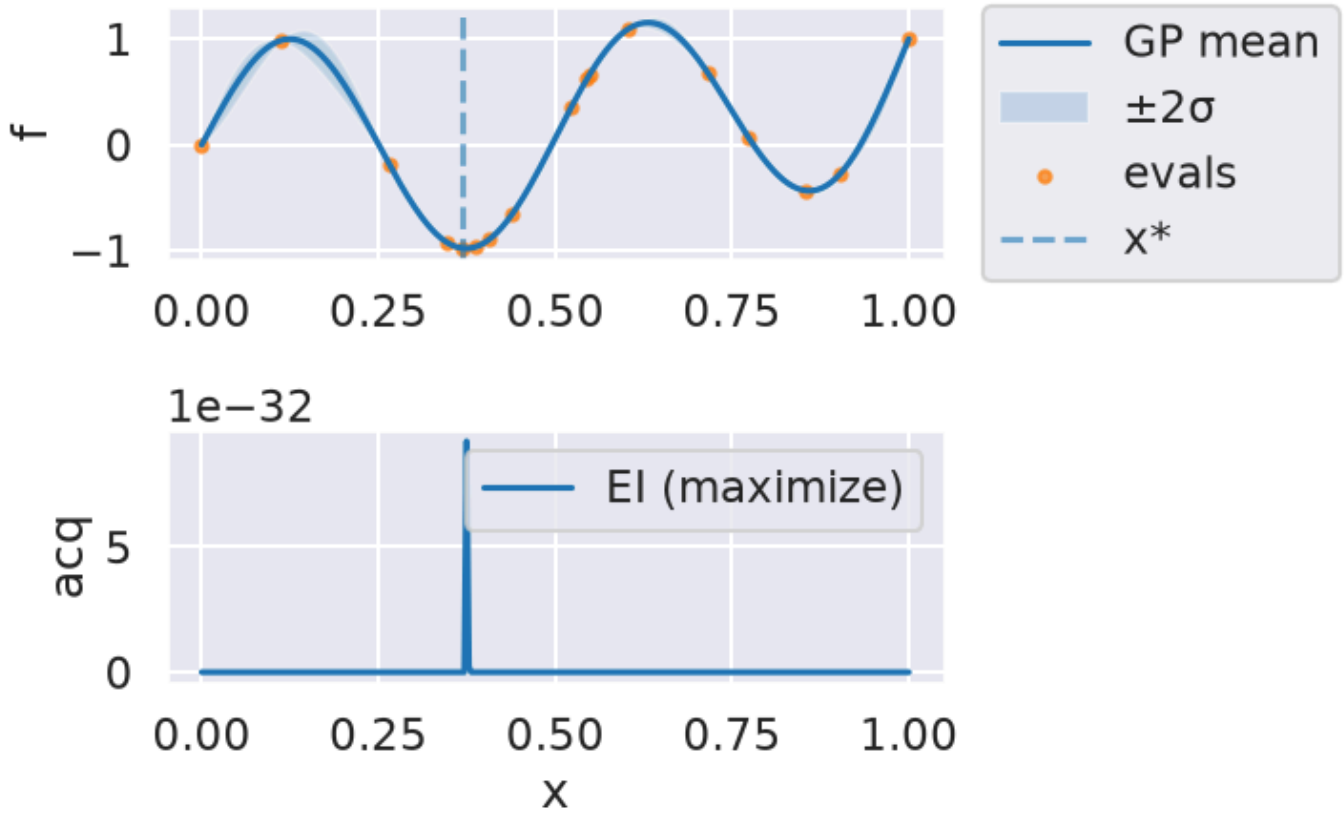

```
Iteration  10
```



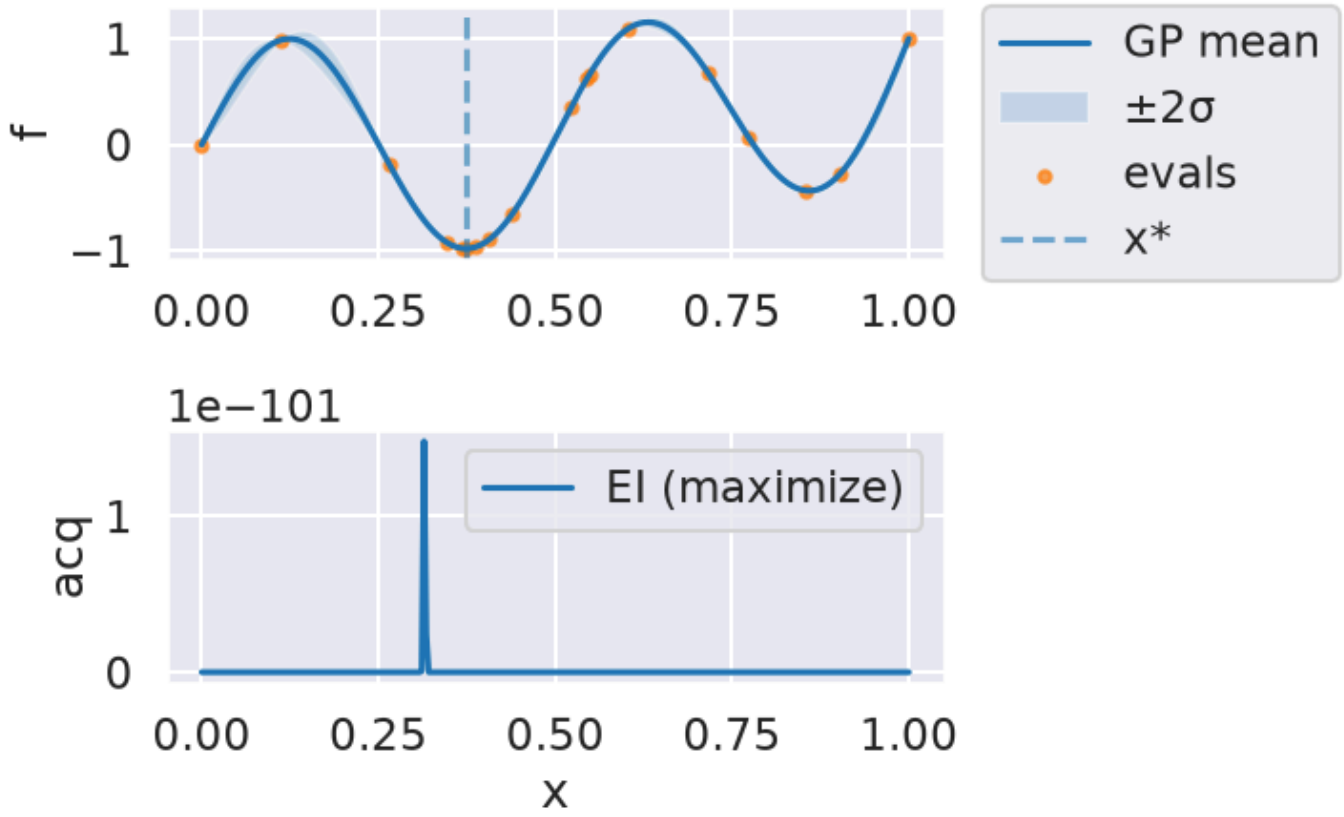
```
Iteration  11
```



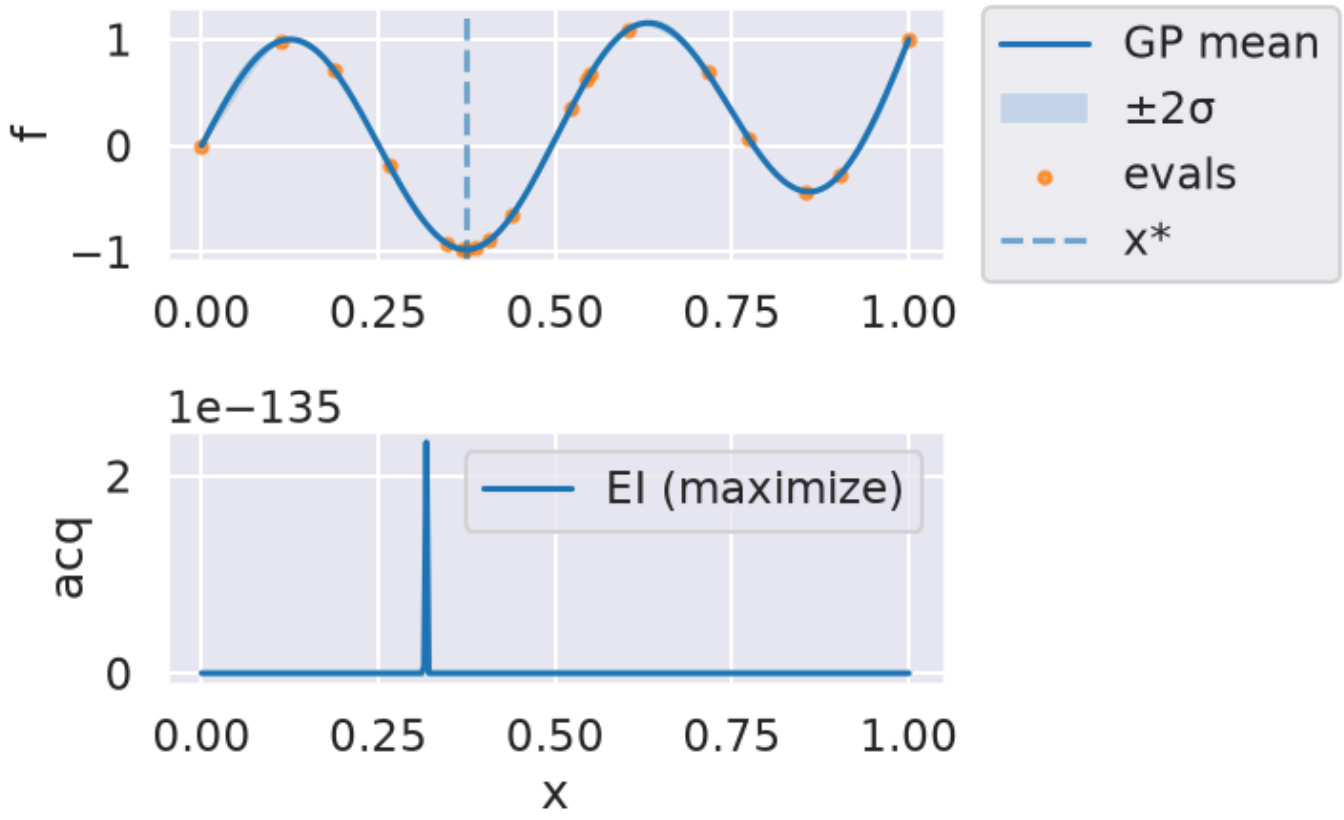

```
Iteration  12
```




```
Iteration  13
```

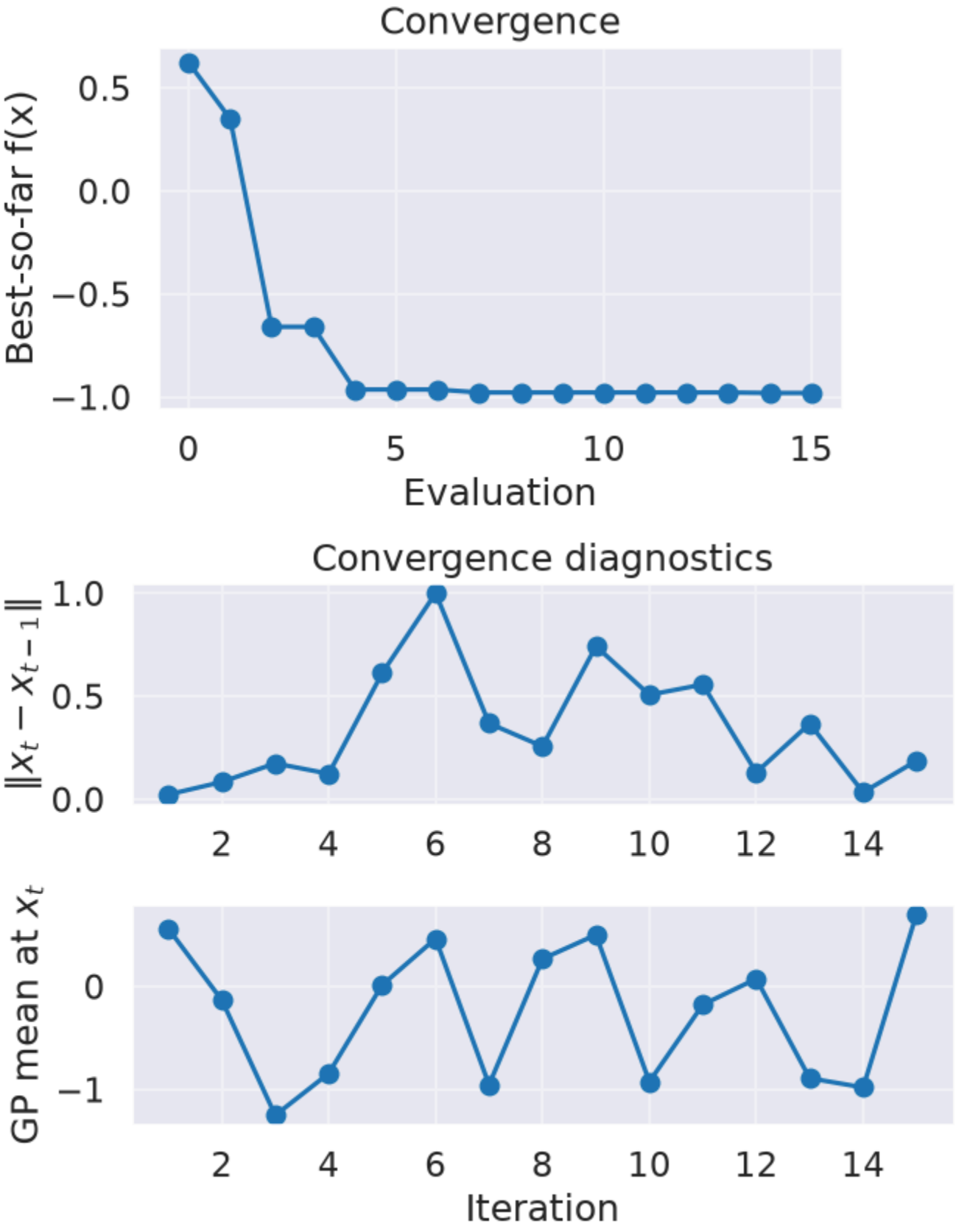


```
Iteration  14
```



▼ **Hide code cell source**

```
len(history) = 16
X shape = (19, 1)
Y shape = (19, 1)
```




```
Optimal x value = [0.37381314]
Minimized function value = -0.98036
```

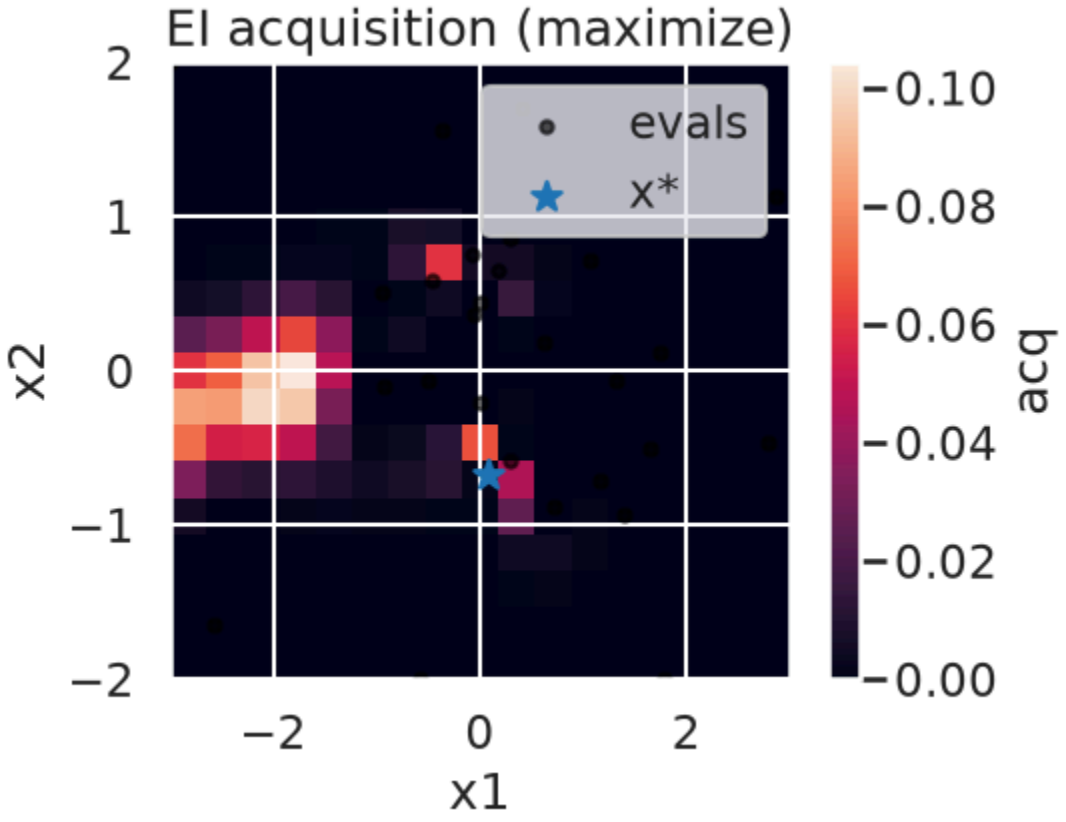
Bivariate example (two parameters)

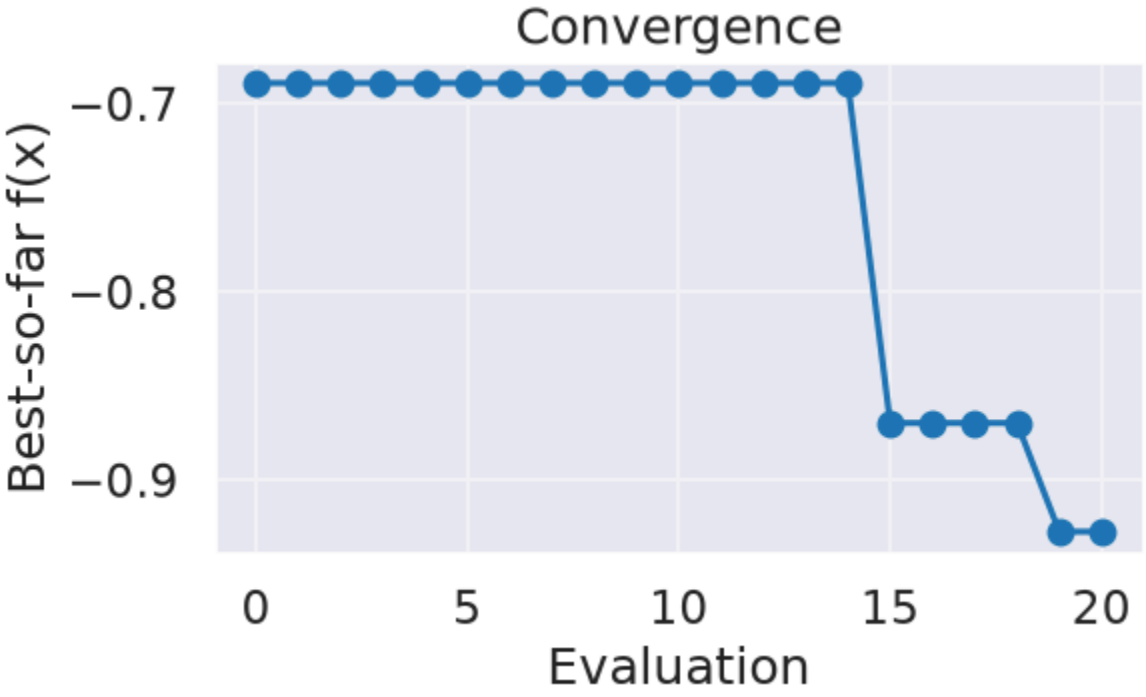
Next, we try a 2-dimensional example. In this case we minimize the Six-hump camel function

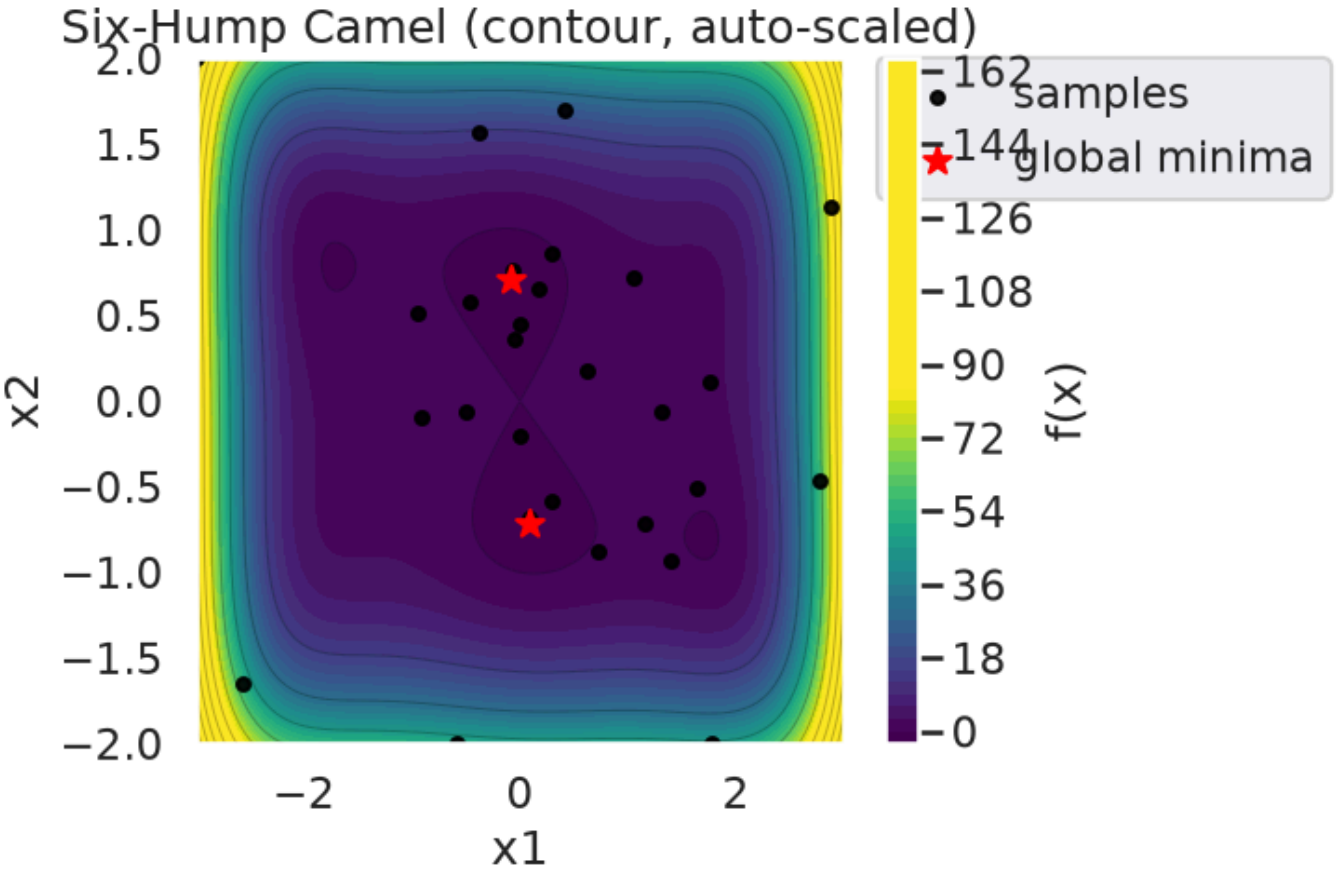
$$f(\theta_1, \theta_2) = \left(4 - 2.1\theta_1^2 + \frac{\theta_1^4}{3}\right)\theta_1^2 + \theta_1\theta_2 + \left(-4 + 4\theta_2^2\right)\theta_2^2,$$

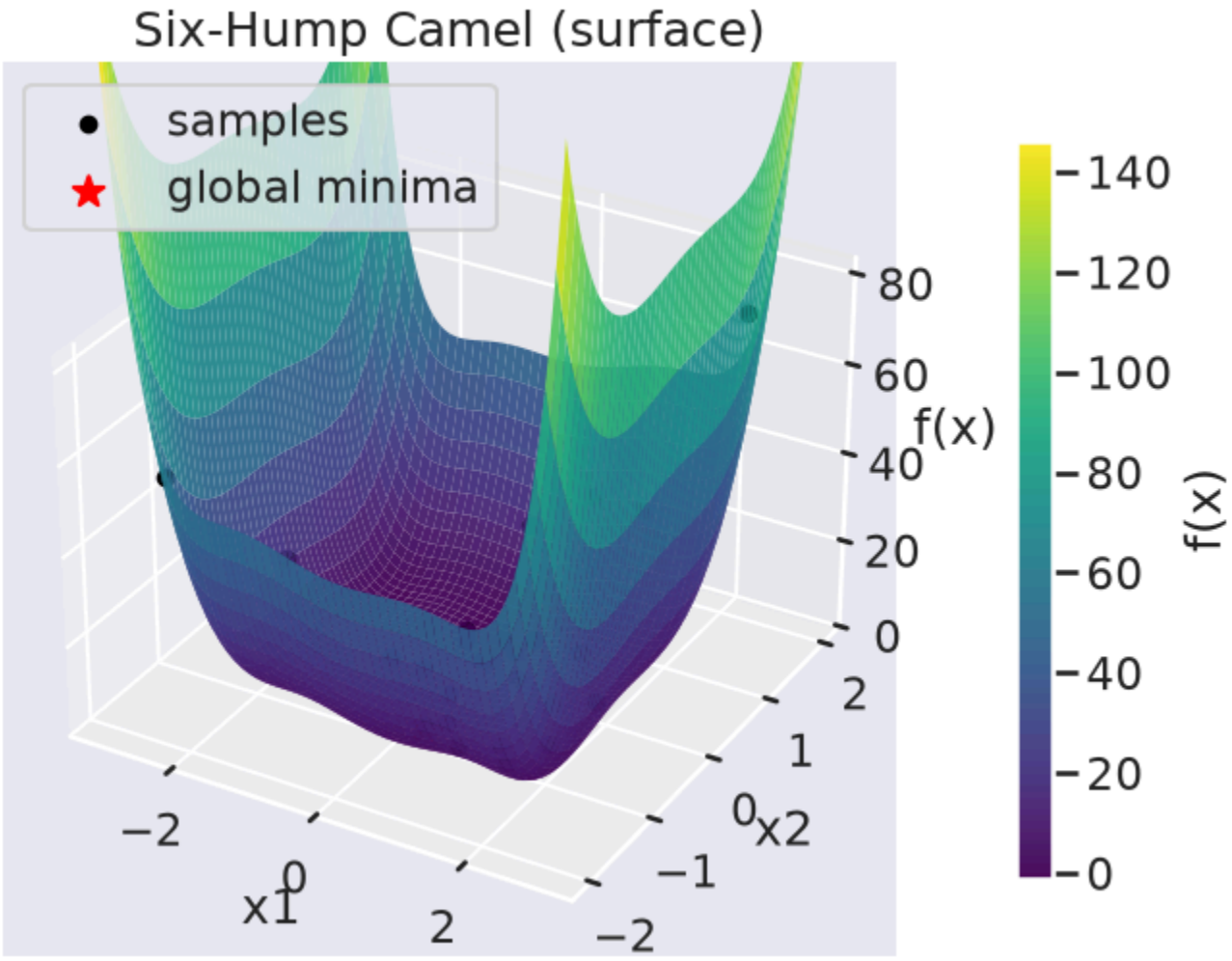
in $[-3, 3]$, $[-2, 2]$. This functions has two global minimum, at (0.0898, −0.7126) and (−0.0898, 0.7126), with function value -1.0316. The function is already pre-defined in `GPyOpt`. In this case we generate observations of the function perturbed with white noise of first of sd=0.01 and then sd=0.1.

▼ **Hide code cell source**


EI acquisition (maximize)
evals
x*
x1
x2
acq
2
1
0
−1
−2
−2
0
2
−0.10
−0.08
−0.06
−0.04
−0.02
−0.00


Convergence
Best-so-far f(x)
Evaluation
−0.7
−0.8
−0.9
0
5
10
15
20


Six-Hump Camel (contour, auto-scaled)
x2
x1
f(x)
samples
global minima
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
−2
0
2
162
144
126
108
90
72
54
36
18
0




```
X* = [ 0.08805726 -0.67863237]   f* = -0.9275626940490445
```

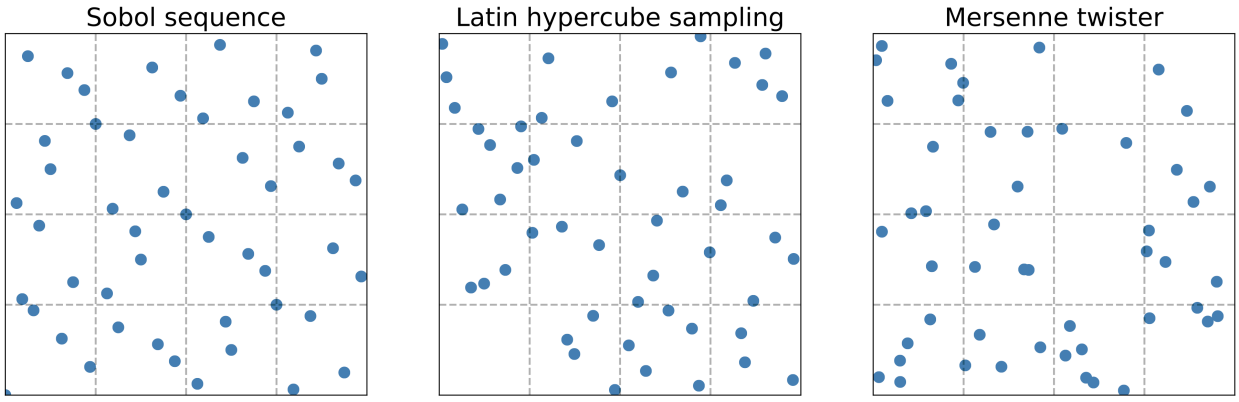
34.4. Concluding remarks

From the paper arXiv:1902.00941

- **Prior knowledge/belief is everything!** Important to tailor the acquisition function and the GP kernel to the spatial structure of the objective function. Thus, the usefulness of BayesOpt hinges on the arbitrariness and uncertainty of a priori information. Complicated by the fact that we resort to BayesOpt when little is known about the objective function in the first place, since it is computationally expensive to evaluate.
- In general, BayesOpt will never find a narrow minimum **nor be useful for extracting the exact location of any optimum**. So one might want to use it as the first stage in a hierarchical optimization scheme to identify the interesting regions of parameter space. One may also want to switch from a more explorative acquisition function in early iterations to more exploitive in later iterations.
- We find that the **acquisition function is more important** than the form of the GP-kernel.
- BayesOpt would probably benefit from a prior that captures the **large-scale structure of the objective function**.

- **High-dimensional parameter domains** are always challenging (subspace learning, dim reduction).

Space-filling sampling



- Sobol sequence sampling in Python, e.g. with sobol_seq
- Latin Hypercube Sampling in Python, e.g. with pyDOE
- Mersenne-Twister is the core random number generator in Python / numpy

35. Dimensionality reduction and emulators

In this chapter we consider various aspects of dimensionality reduction of data. A key tool is SVD, which is “singular value decomposition”, and its role in PCA, which is “principle component analysis”. These are described in Singular value decomposition (SVD) and Principal value analysis (PCA).

The challenges of simulating physical phenomena are being addressed in many subfields with a wide range of accurate *high-fidelity* methods. However, when we need to change the parameters characterizing the problem, such as Hamiltonian coupling constants, it can become computationally prohibitive to repeat high-fidelity calculations many times and challenging to reliably extrapolate. In particular, uncertainty quantification (UQ) generally requires many samples of often expensive calculations, e.g., for Bayesian calibration, sensitivity analyses, and experimental design. An alternative to expensive calculations is to replace the high-fidelity model with an *emulator*, which is an approximate computer model, in the literature sometimes referred to as a “surrogate model.”

- We start with a general discussion of emulators in Bayes goes fast: Emulator overview
- Reduced basis method (RBM) emulators are reviewed in RBM emulators.
- A demonstration example in Demo: RBM vs. HO expansion vs. GP compares calculating observables from solving a basic eigenvalue problem with a basis expansion to using RBM and GP emulators.
- The RBM discussion is continued in Building an emulator basis with a discussion of using active learning to choose the snapshot basis.
- A new type of emulator is described in Parametric Matrix Model emulators [CJHJ+25].
- For examples of multidimensional GP emulation, see surmise.

35.1. Singular value decomposition (SVD) and Principal value analysis (PCA)

PCA belongs to a general class of methods that perform dimensionality reduction on data, where data is defined broadly. This can be thought of as reducing redundancy in characterizing the data. In the context of reduced basis emulators, the “proper orthogonal decomposition” or POD approach is closely analogous to PCA. SVD is a key tool in both PCA and POD methods.

A good reference for SVD is A Singularly Valuable Decomposition: The SVD of a Matrix.

Some parts of this section were adapted from Computational-statistics-with-Python.ipynb, which is from a course taught at Duke University.

As a warm-up: a reminder and a checkpoint question.

Reminder of linear algebra using the index form

Recall that

$$(AB)^\intercal = B^\intercal A^\intercal \quad \text{and} \quad (A^\intercal)_{ji} = A_{ij}$$

$$\chi^2 = [Y - A\boldsymbol{\theta}]^\intercal \, \boldsymbol{\Sigma}^{-1} \, [Y - A\boldsymbol{\theta}] = (Y_i - A_{ij}\boldsymbol{\theta}_j)(\Sigma^{-1})_{ii'}(Y_{i'} - A_{i'j'}\boldsymbol{\theta}_{j'}),$$

where i, i' run from 1 to N and j, j' run from one to p, where the highest power term is x^{p-1}.

- Summed over i, i', j, j' because they each appear (exactly) twice.
- $(\Sigma^{-1})_{ii'} \neq (\Sigma_{i,i'})^{-1}$
- Be sure you understand the indices on the leftmost term, remembering that the matrix expression has this term transposed.
- We know χ^2 is a scalar because there are no free indices.

Checkpoint Question

Prove that the Maximum Likelihood Estimate (MLE) for χ^2 defined by

$$\chi^2 = (\boldsymbol{Y} - \boldsymbol{A\theta})^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{Y} - \boldsymbol{A\theta})$$

is

$$\boldsymbol{\theta}_{\rm MLE} = (\boldsymbol{A}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{A})^{-1} (\boldsymbol{A}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{Y}) \, .$$

Here $\boldsymbol{\theta}$ is a $m \times 1$ matrix of parameters (i.e., there are m parameters), $\boldsymbol{\Sigma}$ is the $m \times m$ covariance matrix, $\boldsymbol{Y}$ is a $N \times 1$ matrix of observations (data), and $\boldsymbol{A}$ is an $N \times m$ matrix

$$\boldsymbol{A} = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots \\ 1 & x_2 & x_2^2 & \cdots \\ \vdots & \vdots & \vdots & \cdots \\ 1 & x_N & x_N^2 & \cdots \end{pmatrix}$$

where N is the number of observations. The idea is to do this with explicit indices for vectors and matrices, using the Einstein summation convention.

Hint

A suggested approach:

- Write χ^2 in indices: $\chi^2 = (Y_i - A_{ij}\theta_j)\Sigma^{-1}_{ii'}(Y_{i'} - A_{i'j'}\theta_{j'})$, where summations over repeated indices are implied (be careful of transposes and using enough independent indices). *How do we see that χ^2 is a scalar?*
- Find $\partial\chi^2/\partial\theta_k = 0$ for all k, using $\partial\theta_j/\partial\theta_k = \delta_{jk}$. Isolate the terms with one component of $\boldsymbol{\theta}$ from those with none.
- You should get the matrix equation $(\boldsymbol{A}^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{Y}) = (\boldsymbol{A}^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{A})\boldsymbol{\theta}$. At this point you can directly solve for $\boldsymbol{\theta}$. *Why can you do this now?*

Answer

We find the MLE from $\partial\chi^2/\partial\boldsymbol{\theta}_k = 0$ for $k = 1, \ldots p$.

$$\left.\frac{\partial\chi^2}{\partial\boldsymbol{\theta}_k}\right|_{\boldsymbol{\theta}=\widehat{\boldsymbol{\theta}}} = -A_{ij}\delta_{jk}(\Sigma^{-1})_{i,i'}(Y_{i'} - A_{i'j'}\widehat{\boldsymbol{\theta}}_{j'}) + (Y_i - A_{ij}\widehat{\boldsymbol{\theta}}_j)(\Sigma^{-1})_{i,i'}(-A_{i'j'}\delta_{j'k}) = 0$$

Isolate the $\boldsymbol{\theta}$ terms on one side and show the doubled terms are equal:

$$A_{ik}(\boldsymbol{\Sigma}^{-1})_{i,i'}Y_{i'} + Y_i(\boldsymbol{\Sigma}^{-1})_{i,i'}A_{i'k} = A_{ik}(\boldsymbol{\Sigma}^{-1})_{i,i'}A_{i'j'}\widehat{\boldsymbol{\theta}}_{j'} + A_{ij}\widehat{\boldsymbol{\theta}}_j(\boldsymbol{\Sigma}^{-1})_{i,i'}A_{i'k}$$

- In the second term on the left, switch $i \leftrightarrow i'$ and use $(\boldsymbol{\Sigma}^{-1})_{i',i} = (\boldsymbol{\Sigma}^{-1})_{i,i'}$ because it is symmetric. This is then the same as the first term.
- In the first term on the right, we switch $j \leftrightarrow j'$ and use the symmetry of $\boldsymbol{\Sigma}$ again to show the two terms are the same.

Writing $A_{ik} = (A^\intercal)_{ki}$, we get

$$2(A^\intercal)_{ki}(\boldsymbol{\Sigma}^{-1})_{i,i'}Y_i = 2(A^\intercal)_{ki}(\boldsymbol{\Sigma}^{-1})_{i,i'}A_{i'j}\widehat{\boldsymbol{\theta}}_j$$

or, removing the indices,

$$(A^\intercal\boldsymbol{\Sigma}^{-1}Y) = (A^\intercal\boldsymbol{\Sigma}^{-1}A)\widehat{\boldsymbol{\theta}}$$

and then inverting (which is possible because the expression in parentheses on the right is a square, invertible matrix), we finally obtain:

$$\widehat{\boldsymbol{\theta}} = [A^\intercal \boldsymbol{\Sigma}^{-1} A]^{-1} [A^\intercal \boldsymbol{\Sigma}^{-1} Y].$$

Q.E.D.

39. The probability measure

In this chapter we define the probability measure and work with random variables in both discrete and continuous samples spaces. The concepts of expectation values and moments are introduced. There is also a brief discussion on different views on probability.

39.1. Important definitions

The set of all possible outcomes of an experiment is known as the sample space and is here denoted by S. We can think of events A as subsets of the sample space.

Whenever A and B are events that we are interested in, then we can also reasonably concern ourselves with the events ($A \cap B$), ($A \cup B$), and ($\bar{A}$) which correspond to (A and B), (A or B), and (not A), respectively.

The probability measure

Definition 39.1 (Probability measure)

A probability measure is a function $\mathbb{P}$ that assigns a number $\mathbb{P}(A) \in [0, 1]$ to every event $A \subseteq S$, satisfying

- $\mathbb{P}(S) = 1$
- $\mathbb{P}(\emptyset) = 0$
- If $A_1, A_2, \ldots A_n$ is a collection of disjoint events, such that $A_i \cap A_j = \emptyset$ for all $i \neq j$, then $\mathbb{P}\left(\cup_{i=1}^{n} A_i\right) = \sum_{i=1}^{n} \mathbb{P}(A_i)$.

In particular, we will often consider the probability for two events to be true $\mathbb{P}(A \cap B)$. For brevity, we will often use the simpler notation $\mathbb{P}(A, B)$.

Definition 39.2 (Independent events)

Two events A and B are independent if

$$\mathbb{P}(A, B) = \mathbb{P}(A)\mathbb{P}(B) \tag{39.1}$$

Definition 39.3 (Conditional probability)

Given $\mathbb{P}(A) > 0$ we define the conditional probability of B given A as

$$\mathbb{P}(B \mid A) = \frac{\mathbb{P}(A, B)}{\mathbb{P}(A)}. \tag{39.2}$$

Alternatively this can be expressed via the **product rule** of probability theory

$$\mathbb{P}(A, B) = \mathbb{P}(B \mid A)\mathbb{P}(A). \tag{39.3}$$

Given $\mathbb{P}(A) > 0$ we have that A and B are independent if and only if $\mathbb{P}(B \mid A) = \mathbb{P}(B)$.

The **total law of probability** can be obtained from the disjoint-union property of Definition 39.1 and the product rule (39.3). Consider a partition $B_1, B_2, \ldots, B_n$ of the complete state space (meaning that $B_i \cap B_j = \emptyset$ for all $i \neq j$ and $\sum_{i=1}^{n} \mathbb{P}(B_i) = 1$) such that $\mathbb{P}(B_i) > 0$ for all i. Then

$$\mathbb{P}(A) = \sum_{i=1}^{n} \mathbb{P}(A \mid B_i)\mathbb{P}(B_i) = \sum_{i=1}^{n} \mathbb{P}(A, B_i). \tag{39.4}$$

This process of summing over all possible states of an event in a joint probability to obtain the **marginal** probability of the other event is known as marginalization.

A simple example of this law would be the statement

> The total probability that it rains tomorrow is the sum of the probability that it rains tomorrow and that it rains today plus the probability that it rains tomorrow and not today.

Each of those joint probabilities can be factorized according to the product rule. For example, the probability that it rains tomorrow and that it rains today is the conditional probability of raining tomorrow given that it rains today times the probability that it rains today.

The point here is that the total probability of rain tomorrow is the sum of those two terms since the two events “it rains today” and “it does not rain today” form a complete and exhaustive partition of outcomes of the experiment “will it rain today?”.

Random variables: probability distribution and density

Let us introduce the concept of random variables and use those to introduce probability distribution and density functions.

Definition 39.4 (Random variable and distribution function)

A random (or stochastic) variable is a function $X : S \to \mathbb{R}$.

The **distribution function** P for a random variable X is the function $P : \mathbb{R} \to [0, 1]$, given by

$$P(x) = \mathbb{P}(X \leq x). \tag{39.5}$$

We can write $P_X(x)$ where it is necessary to emphasize the role of X.

Definition 39.5 (Joint probability distribution)

The joint distribution function of a vector $\boldsymbol{X}$ of random variables $\boldsymbol{X} = (X_1, X_2, \ldots, X_n)$ is the function $P : \mathbb{R}^n \to [0, 1]$ given by

$$P(\boldsymbol{x}) = \mathbb{P}(\boldsymbol{X} \leq \boldsymbol{x}), \qquad \boldsymbol{x} \in \mathbb{R}^n. \tag{39.6}$$

We can write $P_{\boldsymbol{X}}$ where it is necessary to emphasize the role of $\boldsymbol{X}$.

For random variables that are continuous it will be very useful to work with probability densities. Let us define those, starting however with the corresponding quantity (probability mass) for discrete random variables.

Definition 39.6 (Probability mass function)

The random variable X is called **discrete** if it takes values only in some countable subset $\{x_1, x_2, \ldots\}$ of $\mathbb{R}$. The function $p : \mathbb{R} \to [0, 1]$, given by

$$p(x) = \mathbb{P}(X = x), \tag{39.7}$$

is known as its **probability mass function**. Again, we can write $p_X(x)$ where it is necessary to emphasize the role of X.

The **joint probability mass function** of a random vector $\boldsymbol{X} = (X_1, X_2, \ldots, X_n)$ is the function $p : \mathbb{R}^n \to [0, 1]$ given by

$$p(x_1, x_2, \ldots, x_n) = \mathbb{P}(X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n). \tag{39.8}$$

Definition 39.7 (Probability density function)

The random variable X is called **continuous** if its distribution function can be expressed as

$$P(x) = \int_{-\infty}^{x} p(z)dz, \qquad x \in \mathbb{R}, \tag{39.9}$$

for some integrable function $p : \mathbb{R} \to [0, \infty)$ called the **probability density function** (PDF). Again, we can write $p_X(x)$ where it is necessary to emphasize the role of X.

The **joint probability density function** of a random vector $\boldsymbol{X} = (X_1, \ldots, X_n)$ of continuous variables is the function $p : \mathbb{R}^n \to [0, \infty)$ given by

$$P(x_1, \ldots, x_n) = \int_{u_1=-\infty}^{x_1} \ldots \int_{u_n=-\infty}^{x_n} p(u_1, \ldots, u_n) du_1 \ldots du_n. \tag{39.10}$$

Note that we will not differentiate in notation between probability mass and density functions as the context should make it clear whether it describes the probability density of a discrete or continuous variable. We will also refer to both as a PDF.

While discrete examples tend to be simpler, situations with continuous variables are more common in physics.

Following the above definition, there are some properties that all PDFs must have. Here we list some important ones using the simplest example of a single (continuous) random variable X

1. The first one is positivity

$$0 \leq p(x). \tag{39.11}$$

   Naturally, it would be nonsensical for any of the values of the domain to occur with a probability density less than 0.

2. Also, the PDF must be normalized. That is, all the probabilities must add up to one. The probability of *anything* to happen is always unity. For a continuous PDF this condition is

$$\int_{-\infty}^{\infty} p(x)\, dx = 1. \tag{39.12}$$

   The corresponding condition for a discrete PDF is $\sum_i p(x_i) = 1$.

3. The probability for *any* specific outcome x of a continuous variable X is zero

$$\mathbb{P}(X = x) = 0, \qquad \text{for all} \quad x \in \mathbb{R}, \tag{39.13}$$

   since probabilities will be computable from the integral measure $p(x)dx$ and $\mathbb{P}(X = x)$ would correspond to $dx \to 0$.

4. Instead it makes more sense to discuss the probability for the outcome being within a domain. E.g., for the univariate case we can quantify

$$\mathbb{P}(a \leq X \leq b) = \int_a^b p(x)dx. \tag{39.14}$$

   From which we can also note that PDFs are not dimensionless objects. We must have $[p(x)] = [x]^{-1}$ for the integral to produce a dimensionless probability.

These properties can be generalized to the multivariate case $p(x_1, x_2, \ldots)$.

For the multivariate case we also introduce the important concepts of **marginalization** and **independence** .

Property 39.1 (Marginal density functions)

Given a joint density function $p(x, y)$ of two random variables X and Y, the (marginal) probability density function of X is obtained via marginalization

$$p(x) = \int_{-\infty}^{\infty} p(x, y) dy, \tag{39.15}$$

and vice versa for $p(y)$.

Marginalization is a very powerful technique as it allows to extract probabilites for a variable of interest when dealing with multivariate problems.

Property 39.2 (Independence)

Two random variables X and Y are independent if (and only if) the joint density function factorizes

$$p(x, y) = p(x)p(y). \tag{39.16}$$

Suppose that X and Y have the joint probability density $p(x, y)$. We wish to discuss the conditional probability distribution of Y given that X takes the value x. However, we need to be careful since the event $X = x$ has zero probability. Instead, we can consider the event $x \leq X \leq x + dx$ which leads to the following definition

Definition 39.8 (Conditional probability-distribution)

The conditional distribution function of Y given $X = x$ is

$$P_{Y|X}(y|x) = \int_{-\infty}^{y} \frac{p(x, y')}{p_X(x)} dy' \tag{39.17}$$

for any x such that $p_X(x) > 0$.

The integrand is then defined as the conditional PDF

$$p_{Y|X}(y|x) = \frac{p(x, y)}{p_X(x)} = \frac{p(x, y)}{\int_{-\infty}^{\infty} p(x, y) dy}, \tag{39.18}$$

for any x such that $p_X(x) > 0$.

Note

Probability densities are usually introduced in the context of random variables (as we did here). However, from the Bayesian viewpoint, probabilities are used more generally to describe our state of knowledge. This means, for example, that we will use probability densities to quantify our knowledge of physics model parameters. Such a PDF would not make sense in an approach that requires randomness in considered variables.

Expectation values and moments

Definition 39.9 (Expectation value)

Let $h(x)$ be an arbitrary continuous function on the domain $\mathbb{R}$ of the continuous, random variable X whose PDF is $p(x)$. We define the *expectation value* of h with respect to p as follows

$$\mathbb{E}_p[h] = \int_{-\infty}^{\infty} h(x)p(x)\,dx. \tag{39.19}$$

The corresponding definition for a discrete variable X is

$$\mathbb{E}_p[h] = \sum_i h(x_i)p(x_i). \tag{39.20}$$

Note that we usually drop the index p and just write $\mathbb{E}[h]$.

A particularly useful class of expectation values are the **moments**. The n-th moment of the PDF $p(x)$ is defined as follows

$$\mathbb{E}[X^n] = \int_{-\infty}^{\infty} x^n p(x)\,dx \tag{39.21}$$

The zero-th moment $\mathbb{E}[1]$ is just the normalization condition of p. The first moment, $\mathbb{E}[X]$, is called the **mean** of p and is often denoted by the greek letter μ

$$\mathbb{E}[X] \equiv \mu \equiv \int_{-\infty}^{\infty} xp(x)dx, \tag{39.22}$$

for a continuous distribution and

$$\mathbb{E}[X] \equiv \mu \equiv \sum_i x_i p(x_i), \tag{39.23}$$

for a discrete distribution.

Qualitatively it represents the average value of the PDF and is therefore sometimes called the expectation value of $p(x)$.

Central moments: Variance and Covariance

Another special case of expectation values is the set of **central moments**, with the n-th central moment defined as

$$\mathbb{E}\left[(X-\mathbb{E}[X])^n\right] = \int_{-\infty}^{\infty} (x-\mathbb{E}[X])^n p(x)\,dx. \tag{39.24}$$

The zero-th and first central moments are both trivial; equal to 1 and 0, respectively. Instead, the second central moment is of particular interest.

Definition 39.10 (Variance)

The **variance** of a random variable X is usually denoted σ^2 or Var(X) and is defined as

$$\text{Var}\ (X) \equiv \sigma^2 \equiv \mathbb{E}\left[(X-\mathbb{E}[X])^2\right] \tag{39.25}$$

We note that

$$\begin{aligned}
\sigma^2 &= \int_{-\infty}^{\infty} (x-\mathbb{E}[X])^2 p(x) dx \\
&= \int_{-\infty}^{\infty} \left(x^2 - 2x\mathbb{E}[X] + \mathbb{E}[X]^2\right) p(x) dx \\
&= \mathbb{E}[X^2] - 2\mathbb{E}[X]\mathbb{E}[X] + \mathbb{E}[X]^2 \\
&= \mathbb{E}[X^2] - \mathbb{E}[X]^2
\end{aligned} \tag{39.26}$$

The positive square root of the variance, $\sigma = +\sqrt{\sigma^2}$ is called the **standard deviation** of p. It is the root-mean-square (RMS) value of the deviation of the PDF from its mean value, interpreted qualitatively as the "spread" of X around its mean.

When dealing with two random variables it is useful to introduce the **covariance**

Definition 39.11 (Covariance and correlation)

The **covariance** of two random variables X and Y is usually denoted σ^2_{XY} or Cov (X,Y) and is defined as

$$\text{Cov}\ (X,Y) \equiv \sigma^2_{XY} \equiv \mathbb{E}\left[(X-\mathbb{E}[X])\,(Y-\mathbb{E}[Y])\right]. \tag{39.27}$$

The **correlation coefficient** of X and Y is defined as

$$\rho_{XY} \equiv \frac{\text{Cov}\ (X,Y)}{\sqrt{\text{Var}\ (X)\text{Var}\ (Y)}}, \tag{39.28}$$

as long as the variances are non-zero.

You can show that the correlation coefficient is $-1 \leq \rho \leq 1$. In particular, the diagonal covariance is the variance and therefore $\rho_{XX} = 1$.

Two variables X and Y are called *uncorrelated* if $\text{Cov}(X,Y) = 0$. Note that the independence property of Eq. (39.16) implies that two independent variables are always uncorrelated. However, the converse is not necessarily true.

39.2. Types of probability

As we have shown, we construct a probability space by assigning a numerical probability in the range [0,1] to events (sets of outcomes) in some space.

When outcomes are the result of an uncertain but repeatable process, probabilities can always be measured to arbitrary accuracy by simply observing many repetitions of the process and calculating the frequency at which each event occurs. These **frequentist probabilities** have an appealing objective reality to them.

Discuss

How might you assign a frequentist probability to statements like:

- The electron spin is 1/2.
- The Higgs boson mass is between 124 and 126 GeV.
- The fraction of dark energy in the universe today is between 68% and 70%.
- The superconductor Hg-1223 has a critical temperature above 130K.

The answer is that you cannot (if we assume that these are universal constants), since that would require a measurable process whose outcomes had different (random) values for the corresponding universal constant. Or maybe you could if you had access to a number of multiverses with different universal constants in each.

The inevitable conclusion is that the statements we scientists are most interested in cannot be assigned frequentist probabilities.

However, if we allow probabilities to also measure our subjective "degree of belief" in a statement, then we can use the full machinery of probability theory to discuss more interesting statements. These are called **Bayesian probabilities**. Note that such probabilities are always conditional in the sense that they are statements based on given information.

Roughly speaking, the choice is between:

Frequentist statistics	**Bayesian statistics**
objective probabilities of uninteresting statements.	subjective probabilities of interesting statements.

39.3. Exercises

Exercise 39.1 (Random and colorblind)

The gene responsible for color blindness is located on the X chromosome. In other words, red-green color blindness is an X-linked recessive condition and is much more common in males (with only one X chromosome). According to the Howard Hughes medical institute about 7% of men and 0.4% of women are red-green colorblind. Furthermore, according to SCB, the Swedish population is 50.3% male and 49.7% female. What is the probability that a person selcted at random is colorblind?

Exercise 39.2 (Conditional discrete probability mass function)

The joint probability mass function of the discrete variables X, Y is

$$p(x, y) = \frac{x + y}{18}, \quad \text{for } \; x, y \in \{0, 1, 2\}.$$

- Find the conditional probability mass function $p\left(y \mid x\right)$.
- Verify that it is properly normalized.

Exercise 39.3 (Conditional probability for continuous variables)

The continuous random variables X, Y have the joint density

$$p(x, y) = e^{-x}, \quad \text{for } \; 0 < y < x < \infty.$$

Find the probability $\mathbb{P}\left(Y < 2 \mid X = 5\right)$.

Exercise 39.4 (Conditional expectation)

Assume that the continuous random variables X, Y have the joint density

$$p\left(x, y\right) = \frac{2}{xy}, \quad \text{for } \; 1 < y < x < e.$$

Find the conditional expectation $\mathbb{E}\left[Y | X = x\right]$.

Solutions to exercises

Solution to Exercise 39.1 (Random and colorblind)

Let C, M, F denote the events that a random person is colorblind, male, and female, respectively. By the law of total probability

$$\begin{aligned} \mathbb{P}(C) &= \mathbb{P}\left(C \mid M\right)\mathbb{P}(M) + \mathbb{P}\left(C \mid F\right)\mathbb{P}(F) \\ &= (0.07)(0.503) + (0.004)(0.497) = 0.037. \end{aligned}$$

Solution to Exercise 39.2 (Conditional discrete probability mass function)

The conditional probability mass function can be obtained from the ratio

$$p\left(y \mid x\right) = \frac{p(x, y)}{p(x)}.$$

Let us therefore find the marginal probability mass function

$$p(x) = \sum_{y=0}^{2} p(x,y) = \frac{x}{18} + \frac{x+1}{18} + \frac{x+2}{18} = \frac{x+1}{6}.$$

Thus we get $p\left(y \mid x\right) = \frac{(x+y)/18}{(x+1)/6} = \frac{x+y}{3(x+1)}$ for $y \in \{0,1,2\}$.

We find that this pdf (over y) is properly normalized since

$$\sum_{y=0}^{2} p\left(y \mid x\right) = \frac{x+0+x+1+x+2}{3(x+1)} = 1.$$

Solution to Exercise 39.3 (Conditional probability for continuous variables)

The desired probability is

$$\mathbb{P}\left(Y < 2 \mid X = 5\right) = \int_0^2 p_{Y|X}(y|5) dy.$$

To find the conditional density $p_{Y|X}(y|x)$ we need the marginal one

$$p\left(x\right) = \int_0^\infty p\left(x,y\right) dy = \int_0^x e^{-x} dy = x e^{-x},$$

for $x > 0$. This gives

$$p_{Y|X}(y|x) = \frac{p\left(x,y\right)}{p\left(x\right)} = \frac{e^{-x}}{x e^{-x}} = \frac{1}{x},$$

for $0 < y < x$. Note that this is a uniform distribution for y given x. Therefore

$$\mathbb{P}\left(Y < 2 \mid X = 5\right) = \int_0^2 \frac{1}{5} dy = \frac{2}{5}.$$

Solution to Exercise 39.4 (Conditional expectation)

We need the marginal density

$$p\left(x\right) = \int_1^x \frac{2}{xy} dy = \frac{2 \ln x}{x}, \quad \text{for } \; 1 < x < e,$$

to get the conditional one

$$p_{Y|X}(y|x) = \frac{2/xy}{2\ln x/x} = \frac{1}{y\ln x}, \quad \text{for } 1 < y < x.$$

The conditioned expectation is therefore

$$\mathbb{E}\left[Y|X=x\right] = \int_1^x y p_{Y|X}(y|x) dy = \int_1^x \frac{y}{y\ln x} dy = \frac{x-1}{\ln x}.$$

40. Working with probability distributions

In this chapter we list a selection of important probability distributions and explain how `scipy.stats` can be used when working with distributions. This chapter also contains definitions of relevant point estimates and credible regions.

40.1. Important distributions

Let us consider some important, univariate distributions.

The uniform distribution

The first one is the most basic PDF; namely the uniform distribution. This distribution is constant in a range $[a, b]$ and zero elsewhere. Thus, when a random variable X is uniformly distributed on $[a, b]$ we can write $X \sim \mathcal{U}([a, b])$ with

$$\mathcal{U}\left([a,b]\right) = \frac{1}{b-a}\theta(x-a)\theta(b-x). \tag{40.1}$$

For $a = 0$ and $b = 1$ we have the standard uniform distribution

$$\mathcal{U}\left([0,1]\right) = \begin{cases} 1 & x \in [0,1], \\ 0 & \text{otherwise} \end{cases} \tag{40.2}$$

Note that these functions correspond to properly normalized PDFs such that they give a total probability of one when integrated over $x \in (-\infty, \infty)$.

Gaussian distribution

The second one is the univariate Gaussian distribution (or normal distribution). A random variable $X \sim \mathcal{N}(\mu, \sigma^2)$ is normally distributed with mean value μ and standard deviation σ with

$$\mathcal{N}(\mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \tag{40.3}$$

the corresponding PDF. If $\mu = 0$ and $\sigma = 1$, it is called the **standard normal distribution**

$$\mathcal{N}(0, 1) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right). \tag{40.4}$$

We sometimes denote distributions using a notation like $\mathcal{N}(x|\mu, \sigma^2)$. This should be understood as a variable x being normally distributed with mean μ and variance σ^2.

Multivariate Gaussian distribution

The univariate Gaussian distribution can be generalized to a multivariate distribution. A multivariate random variable $\boldsymbol{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ is normally distributed with mean *vector* $\boldsymbol{\mu} \in \mathbb{R}^k$ and covariance *matrix* $\boldsymbol{\Sigma} \in \mathbb{R}^{k \times k}$ with

$$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{k/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right). \tag{40.5}$$

This distribution only exists for a positive definite covariance matrix $\boldsymbol{\Sigma}$.

40.2. Quick introduction to `scipy.stats`

The `scipy.stats` module contains a large number of probability distributions, summary and frequency statistics, correlation functions statistical tests, and more. If you google `scipy.stats`, you'll likely get the manual page as the first hit: https://docs.scipy.org/doc/scipy/reference/stats.html. Here you'll find a long list of the continuous and discrete distributions that are available for creating random variables, followed (scroll way down) by different methods (functions) to extract properties of a distribution (called Summary statistics) and to perform many other statistical tasks.

Follow the link for any of the distributions (your choice!) to find its mathematical definition, some examples of how to use it, and a list of methods. Some methods of particular interest are:

- `mean()` - Mean of the distribution.
- `median()` - Median of the distribution.
- `pdf(x)` - Value of the probability density function at x.
- `rvs(size=numpts)` - generate numpts random values of the pdf.
- `interval(alpha)` - Endpoints of the range that contains the fraction alpha of the probability mass.

Code example

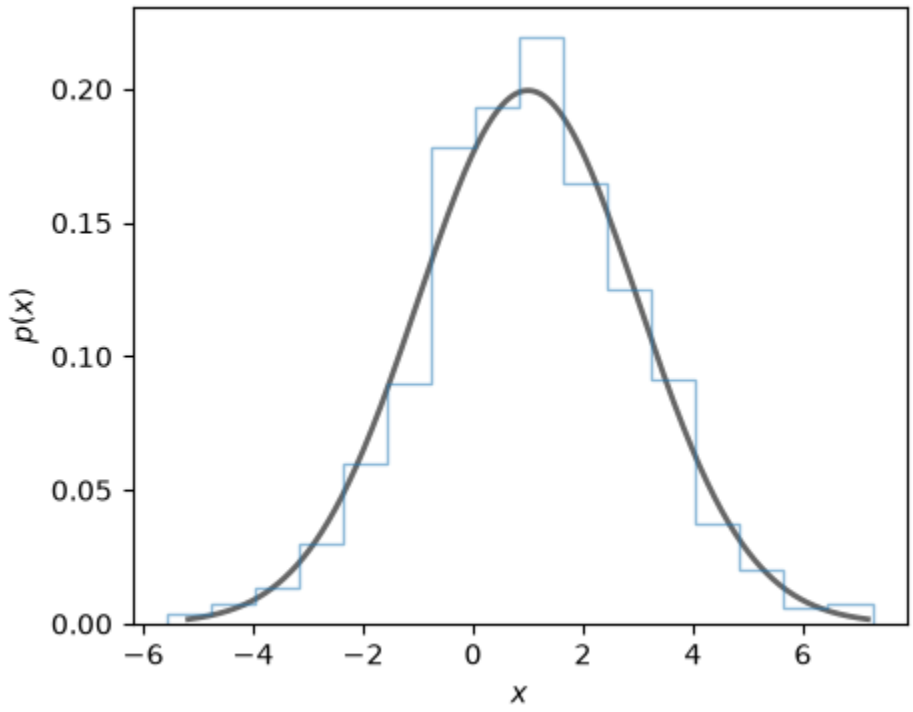
```
import scipy.stats as stats

# Define a normally distributed random variable with mean=1.0 and standard deviation = 2.0
my_norm_rv = stats.norm(loc=1.0, scale=2.0)

# Extract and print the mean and variance
print(f'The mean is {my_norm_rv.mean():3.1f}')
print(f'The variance is {my_norm_rv.var():3.1f}')

# The 68% credible interval (approximately one sigma)
(min68,max68) = my_norm_rv.interval(0.68)
print(f'The 68% credible interval is [{min68:4.2f},{max68:4.2f}]')

# Draw five random samples. Note that the last output will be printed.
my_norm_rv.rvs(size=5)
```

```
The mean is 1.0
The variance is 4.0
The 68% credible interval is [-0.99,2.99]
```

```
array([ 4.68203217,  1.69960996,  0.28828839,  0.42924771, -0.38768784])
```

Create a plot to compare the line shape of the PDF with a histogram of a large number of samples from the PDF.

```
import matplotlib.pyplot as plt
import numpy as np
fig, ax = plt.subplots(nrows=1, ncols=1, **{"figsize":(5,4)})
x = np.linspace(my_norm_rv.ppf(0.001),
                my_norm_rv.ppf(0.999), 1000)
ax.plot(x, my_norm_rv.pdf(x),
       'k-', lw=2, alpha=0.6, label='pdf')
r = my_norm_rv.rvs(size=1000)
ax.hist(r, density=True, histtype='step', bins=16, alpha=0.5)
ax.set_xlabel(r'$x$')
ax.set_ylabel(r'$p(x)$');
```



(Pseudo) random number generator

Random numbers generated by a computer are, in fact, not random at all. They are deterministically determined from a sequence of numbers that look random. The method to generate this sequence is known as a *pseudo random number generator* (PRNG), but will not be described here.

The `scipy.stats` module uses `numpy` functionality to generate random numbers and sample the distribution for a random variable. In fact, many of the standard distributions are built into `numpy` and if all you need is fast sampling of random numbers then you can often use `numpy` directly without the need for `scipy.stats`.

The lack of true randomness can be turned into a feature since it allows for experimentation, reproducibility and unit testing of code that uses stochastic modelling. The sequence of random numbers is determined by: (i) the choice of PRNG algorithm, and (ii) a seed to start the generation. Therefore, we can generate the same random numbers in two different runs by specifying the same PRNG and seed. In `numpy` the recommended procedure for random number sampling is to create a `Generator` instance with `default_rng` and call the relevant methods as attributes to this instance. In the code below, we illustrate this procedure to collect five samples from a standard normal distribution using both `numpy` and `scipy.stats`. You should get the same output if you run this code locally.

```
import numpy as np
import scipy.stats as stats
# seed number
seed=42
# Generator instance initialized with the seed
rng = np.random.default_rng(seed)
print(f'numpy samples from N(0,1) with {rng} and seed={seed}: ')
# Use numpy to collect samples from a standard normal
np_random_numbers = rng.normal(loc=0, scale=1, size=5)
print(np_random_numbers)

# The same with scipy.stats
# Create the random variable
my_norm_rv = stats.norm(loc=0.0, scale=1.0)
# sample, using the same PRNG and seed as before
rng = np.random.default_rng(seed)
print(f'scipy.stats samples from N(0,1) with {rng} and seed={seed}: ')
stats_random_numbers = my_norm_rv.rvs(size=5,random_state=rng)
print(stats_random_numbers)
```

```
numpy samples from N(0,1) with Generator(PCG64) and seed=42:
[ 0.30471708 -1.03998411  0.7504512   0.94056472 -1.95103519]
scipy.stats samples from N(0,1) with Generator(PCG64) and seed=42:
[ 0.30471708 -1.03998411  0.7504512   0.94056472 -1.95103519]
```

40.3. Point estimates and credible regions

We will use PDFs to quantify the strength of our inference processes. However, one might be in a situation where it is desirable to summarize the information contained in a PDF in a single (or a few) numbers. This is not always easy, but some common choices are listed here.

Mean, median, mode

The values of the **mode**, **mean**, and **median** can all be used as point estimates for a "typical" value of X. The mode is the position of the peak of the PDF (the most probable value), the mean was defined in Eq. (39.22), while the median is the value $\mu_{1/2}$ for which $P(\mu_{1/2}) = 0.5$. For some PDFs, these metrics will all be the same as exemplified in the first and last panel of Fig. 40.1. For others, such as exemplified in the middle panel, they will not.

Let us consider three example PDFs that illustrate some of the features of the different point estimates and the problems that might occur.

▼ **Hide code cell source**

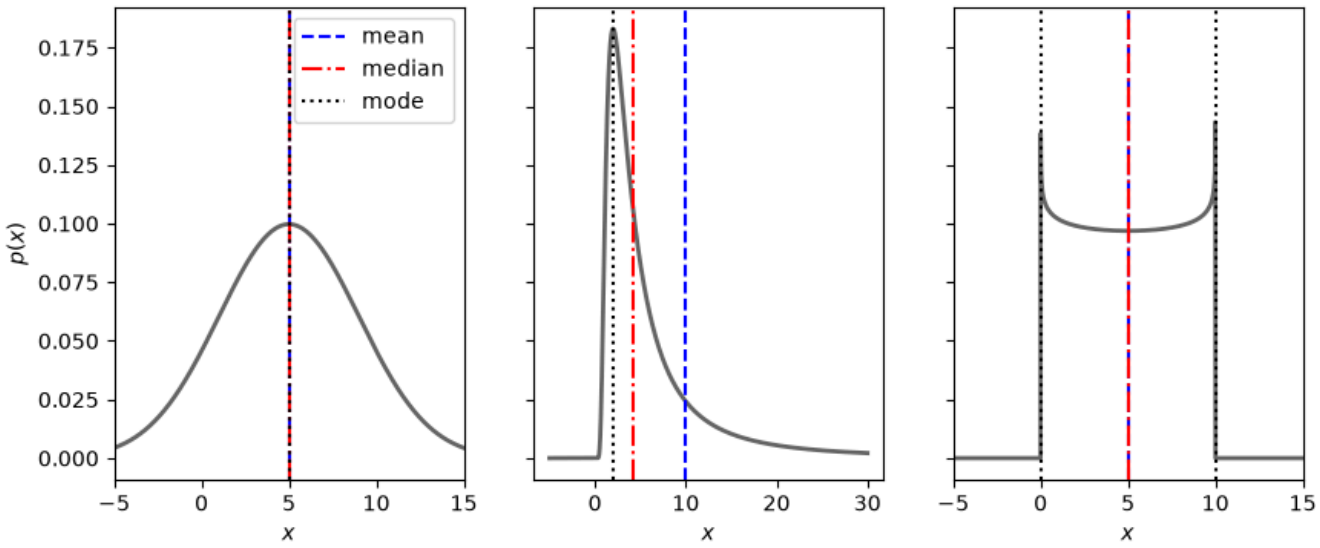


Fig. 40.1 The mean, median and mode(s) for some example PDFs. For some PDFs, several or all of these metrics are the same. For others they are not. The position of the mean is largely affected by long tails as illustrated in the middle panel. The PDF in the right panel has two modes (it is bimodal). Although no shuch example is shown here, there are PDFs for which the mean is not defined.

Discuss

Which point estimate do you consider most representative for the different PDFs?

Credible regions

The integration of the PDF over some domain translates into a probability. It is therefore possible to identify regions $\mathbb{D}_P$ for which the integrated probability equals some desired value P, i.e.,

$$P = \int_{\mathbb{D}_P} p(x_1, x_2, \ldots) dx_1 dx_2 \ldots \tag{40.6}$$

This allows to make statements such as: “There is a 50% probability that the parameters are found within the domain $\mathbb{D}_{0.5}$”. However, the identification of such a domain is not unique. Two popular choices are

1. **Highest-density regions** (HDR)
   The HDR is the smallest possible domain that gives the desired probability mass. That is

$$p(\boldsymbol{x}) \geq p(\boldsymbol{y}), \quad \text{when} \quad \boldsymbol{x} \in \mathbb{D}_P \text{ and } \boldsymbol{y} \notin \mathbb{D}_P. \tag{40.7}$$

2. **Equal-tailed interval** (ETI)
   For a univariate PDF we can define an interval $[a, b]$ such that $\int_a^b p(x)dx = P$ and the probability mass on either side (the tails) are equal. The end points of this ETI fulfil

$$\int_{-\infty}^{a} p(x)dx = \int_{b}^{\infty} p(x)dx = \frac{1-P}{2}. \tag{40.8}$$

Discuss

How would you describe a multimodal PDF using these metrics?

Let us again consider the three example PDFs from above.

▼ Hide code cell source

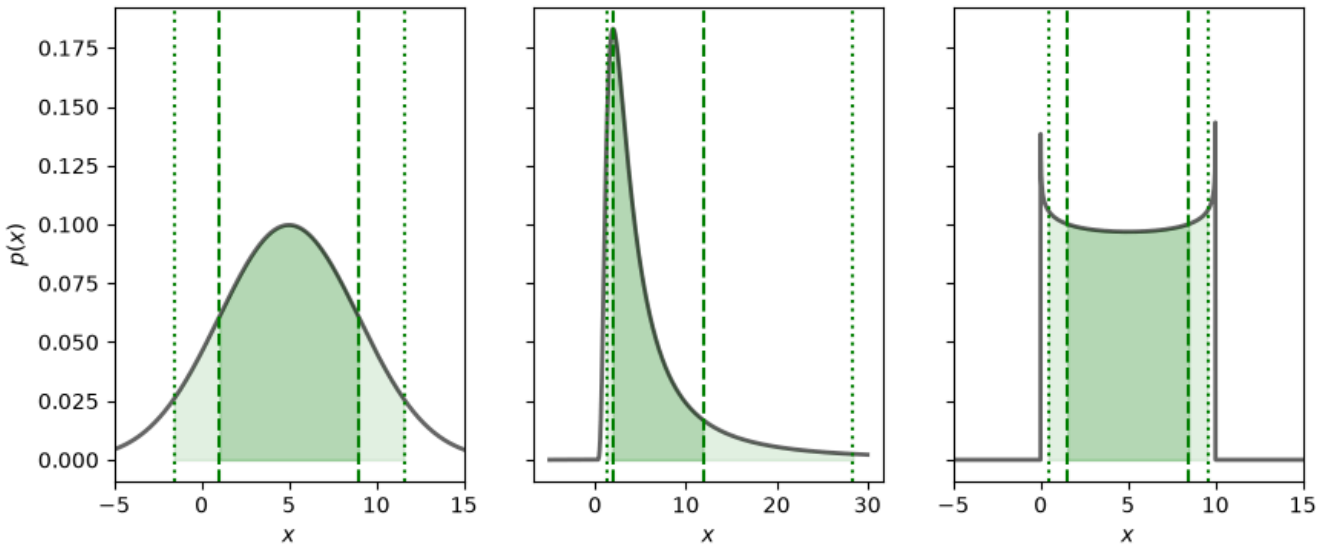


Fig. 40.2 The 68/90 percent credible regions of some example PDFs are shown in dark/light shading. These are all equal-tailed intervals.

Let us also look at a multivariate PDF. The example below is a bivariate normal distribution with non-zero off-diagonal covariance. It is represented by a so called **corner plot** of a large number of samples. The bivariate distribution is shown in the lower left panel, while the two marginal ones are shown on the diagonal.

```
mean=[0.5,1]
cov = np.array([[1,-1],[-1,4]])
my_multinorm_rv=stats.multivariate_normal(mean=mean,cov=cov)
x1x2=my_multinorm_rv.rvs(size=100000)

# We use the prettyplease package from
# https://github.com/svisak/prettyplease
import sys
import os

import prettyplease as pp

fig_x1x2 = pp.corner(x1x2, bins=50, labels=[r'$x_1$',r'$x_2$'],
                          quantiles=[0.16, 0.84], levels=(0.68,0.9),linewidth=1.0,
                          plot_estimates=True, colors='green', n_uncertainty_digits=2,
                          title_loc='center', figsize=(5,5))
glue("bivariate_fig", fig_x1x2, display=False)
```

▼ **Hide code cell output**

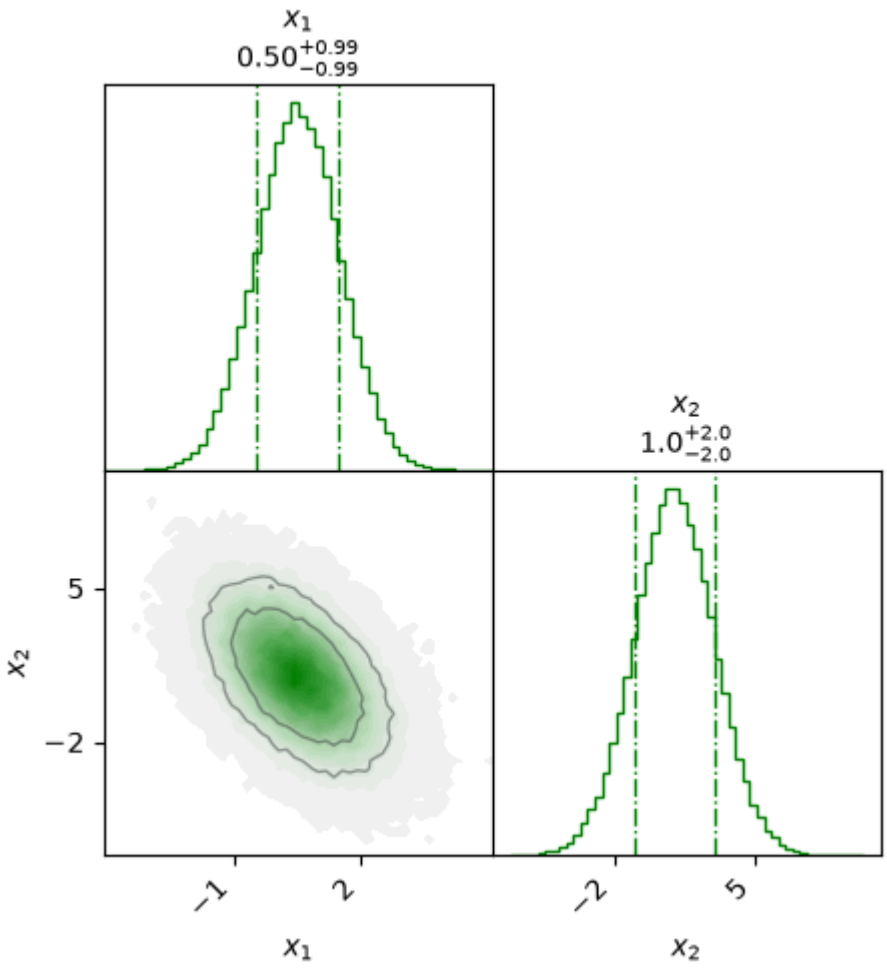


Fig. 40.3 A corner plot of a bivariate normal PDF. The 68% and 90% credible regions are indicated by level curves in the lower left panel. Note the anti-correlation between the two variables (the correlation coefficient is $\rho_{12} = -0.5$). The marginal distributions for x_1 and x_2 are shown in the diagonal panels with the dashed lines indicating the corresponding 68% credible intervals. Note that the marginal PDFs are univariate normal distributions.

40.4. Exercises

Exercise 40.1 (Scipy.stats)

Use `scipy.stats` to

- Find and print the mean and the variance;
- Find and print the 68% credible region (equal-tail interval);
- Draw and print 5 random samples;
- Plot the pdf (including at least 90% of the probability mass);

for

1. A student-t distribution with $\nu = 2$ degrees-of-freedom;
2. A student-t distribution with $\nu = 100$ degrees-of-freedom;
3. A standard normal distribution;

in all cases with the mode at 0.0.

Exercise 40.2 (Bivariate pdf)

Consider the following (multimodal) bivariate pdf

$$p\left(x,y\right) = A_1 \exp\left(-\frac{\left(x-x_1\right)^2 + \left(y-y_1\right)^2}{2\sigma_1^2}\right) + A_2 \exp\left(-\frac{\left(x-x_2\right)^2 + \left(y-y_2\right)^2}{2\sigma_2^2}\right),$$

with $A_1 = 4.82033$, $x_1 = 0.5$, $y_1 = 0.5$, $\sigma_1 = 0.2$, and $A_2 = 4.43181$, $x_2 = 0.65$, $y_2 = 0.75$, $\sigma_2 = 0.04$.

Consider the domain $x, y \in [0, 1]$ and use relevant python modules / methods to

- Plot contour levels of this pdf (useful methods: `np.meshgrid` and `plt.contour`);
- Make a three-dimensional plot of the pdf (useful methods: `plt.subplots(subplot_kw={"projection": "3d"})` and `ax.plot_surface`);
- Compute and plot the marginal pdf $p\left(y\right)$ (useful method: `scipy.integrate.quad`)

Solutions to exercises

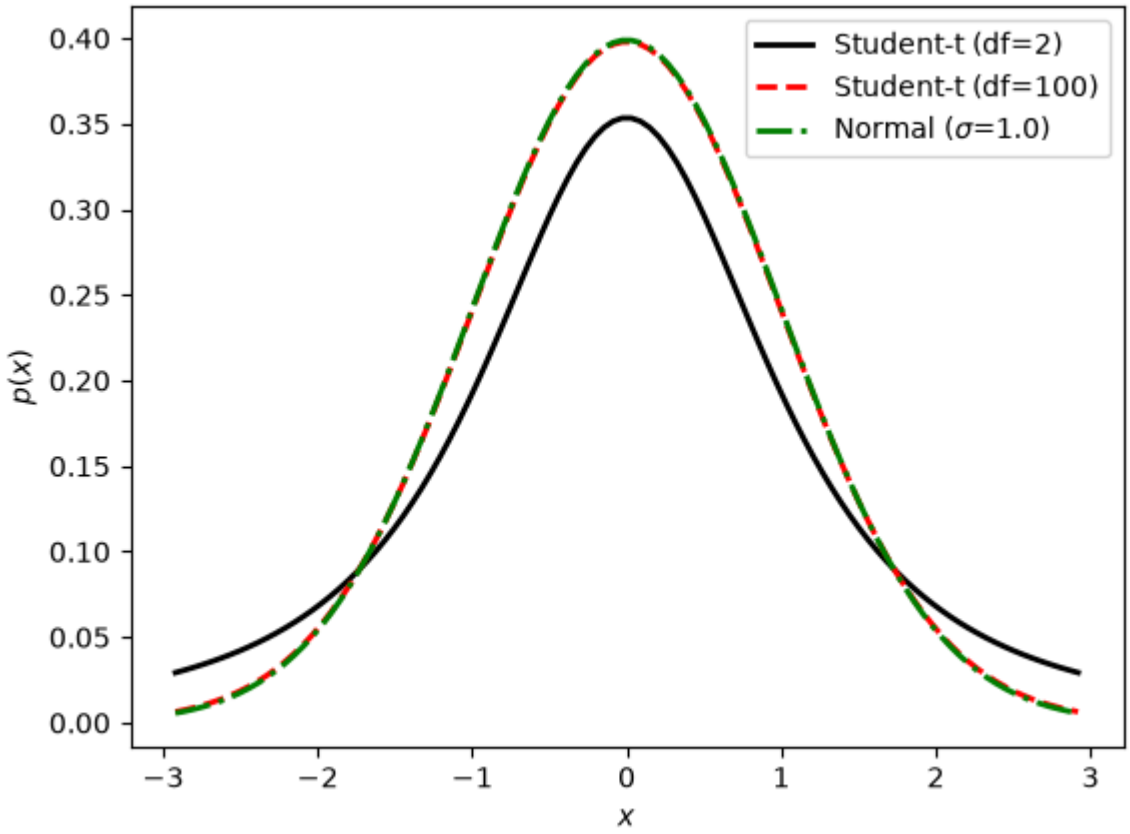
Solution to Exercise 40.1 (Scipy.stats)

See the code example in the hidden code block below.

▼ Hide code cell content

```
=========
 Student-t distribution df=2
=========
The mean is 0.0
The variance is inf
The 68% credible interval is [-1.31,1.31]
```

```
[-1.78436554  0.59767735  0.89047243 -4.92119453  0.29376337]
=========
 Student-t distribution df=100
=========
The mean is 0.0
The variance is 1.0
The 68% credible interval is [-1.00,1.00]
[-0.80582279  0.40550074  0.70822704  1.6366919   1.61871482]
=========
 Normal distribution (sigma = 1.0)
=========
The mean is 0.0
The variance is 1.0
The 68% credible interval is [-0.99,0.99]
[-0.57696116 -0.55719544  0.40572179  2.38025617  0.86552433]
```

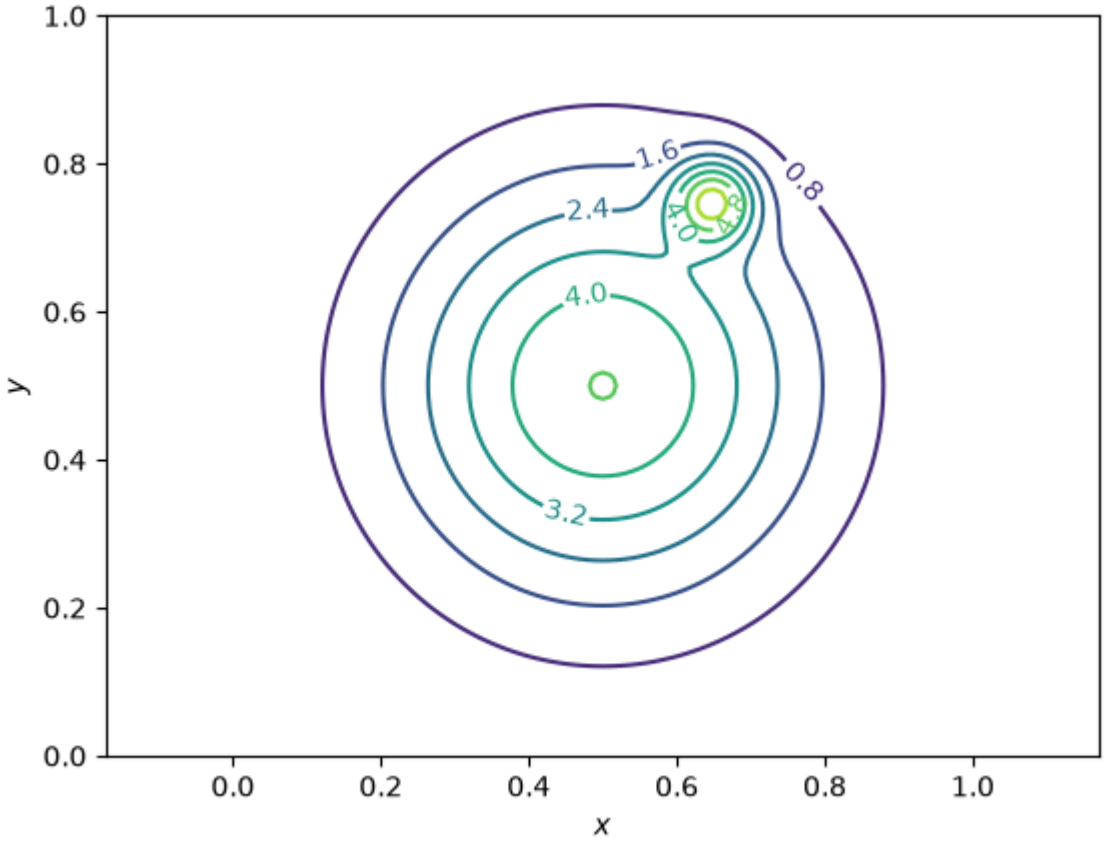


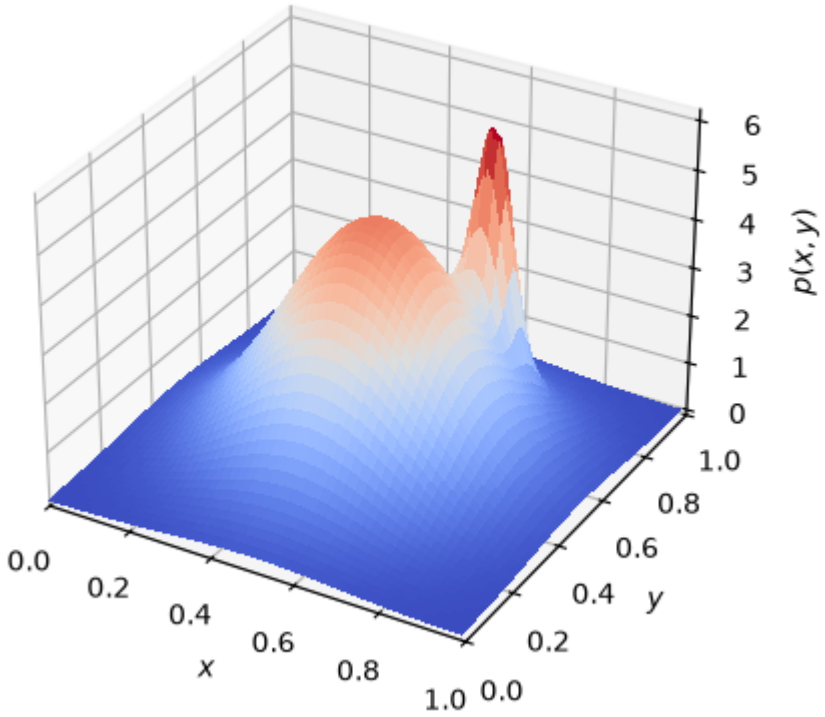
Solution to Exercise 40.2 (Bivariate pdf)

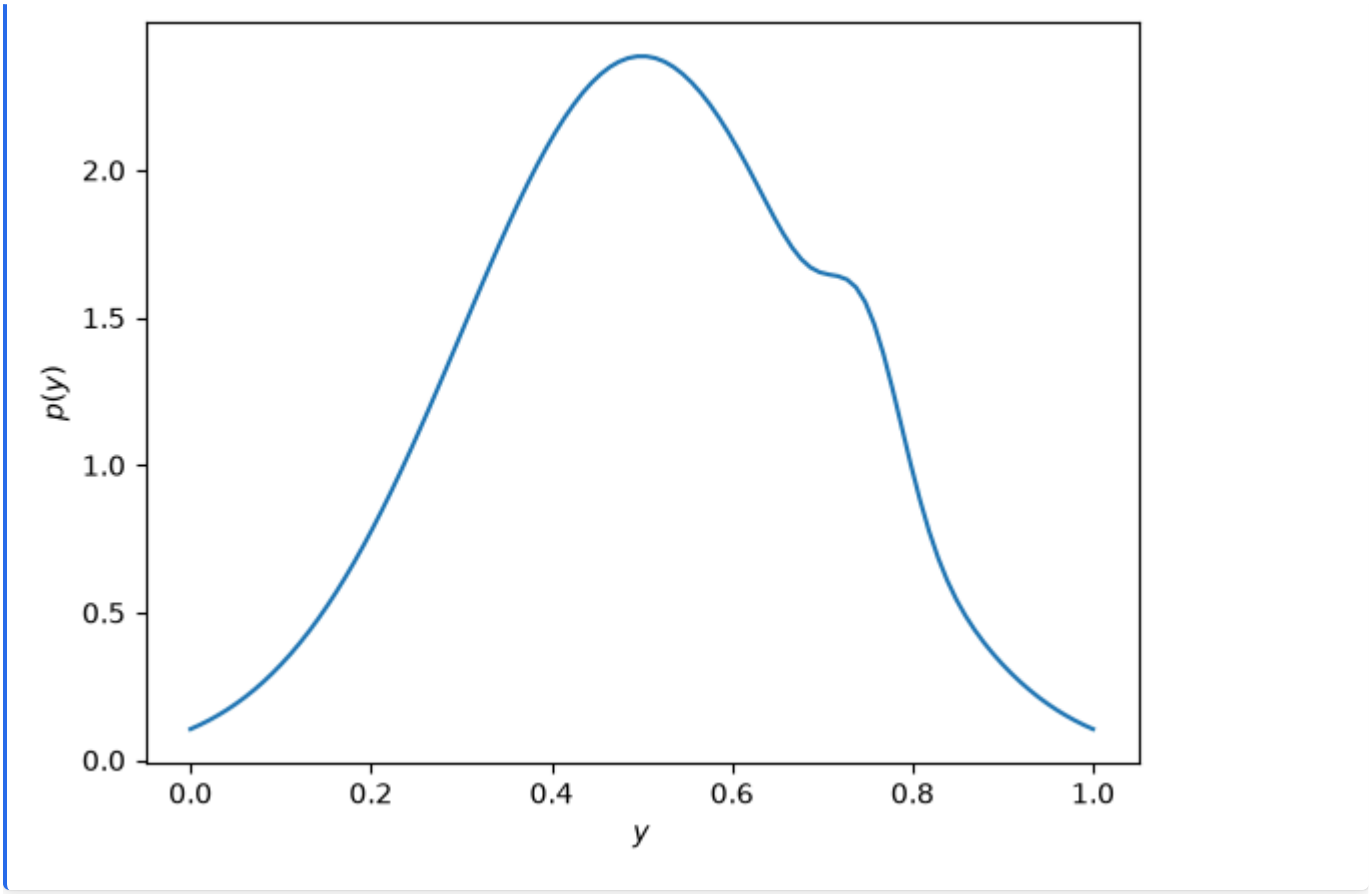
See the code example in the hidden code block below.

▼ **Hide code cell content**

```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.
```


1.0
0.8
0.6
0.4
0.2
0.0
y
0.0
0.2
0.4
0.6
0.8
1.0
x
1.6
0.8
2.4
4.0
3.2


6
5
4
3
2
1
0
p(x, y)
0.0
0.2
0.4
0.6
0.8
1.0
x
0.0
0.2
0.4
0.6
0.8
1.0
y



41. Overview of Appendix B: Scientific modeling

The material here supplements the Bayesian methods for scientific modeling introduced in Part I with a more traditional viewpoint.

- Overview of modeling address scientific models in general and regression analysis in particular. Comparisons are made between parametric and non-parametric models and between linear and non-linear models.
- Linear models gives a complete treatment of conventional (as opposed to Bayesian) linear regression.
- Mathematical optimization describes optimization schemes, which are of particular interest for minimizing loss functions of neural networks. In particular, gradient descent is covered in some detail, including stochastic and adaptive algorithms.

42. Overview of modeling

> “All models are wrong, but some are useful.”

—George Box, in *Robustness in Statistics* (1979)

42.1. Notation

Quantity	General notation
Dataset	**data**
Dependent or response variables (output)	$\boldsymbol{y}$
Independent or predictor variables (input)	$\boldsymbol{x}$
Model	$M\left(\boldsymbol{\theta}\,;\,\boldsymbol{x}\right)$
Model parameters	$\boldsymbol{\theta}$
Parameter optimum	$\boldsymbol{\theta}^*$

42.2. Models in science

In general, modeling deals with the description of **dependent** variable(s) $\boldsymbol{y}$ as a function of some **independent** variable(s) $\boldsymbol{x}$. The first variable is also often called the **response**, or the **outcome** variable while the second one can be called the **predictor** variable, or the **explanatory** variable. Both dependent and independent variables can be of various types: real-valued, integers or categorical, defined on infinite or discrete domains, etc. Note also that each of these might be a vector of variables, meaning that there could be more than one dependent variable and more than one independent variable. We therefore denote these variables as vectors using a bold font. The general act of finding a relationship between dependent and independent variables is known as *regression analysis* and usually involves a set of collected data.

Definition 42.1 (Regression analysis)

Regression analysis is the process of estimating a relationship between one or more dependent variables and one or more independent variables.

Merriam-Webster: *"A functional relationship between two or more correlated variables that is often empirically determined from data and is used especially to predict values of one variable when given values of the others.*

History: The earliest form of regression was the method of least squares, which was published by Legendre in 1805 and by Gauss in 1809. Legendre and Gauss both applied the method to the problem of using astronomical observations for determining the orbits of bodies (mostly comets) about the Sun.

The term "regression" was coined by Francis Galton in the 19th century to describe a biological phenomenon. The phenomenon was that the heights of descendants of tall ancestors tend to regress down towards a normal average (a phenomenon also known as regression toward the mean).

For simplicity in this chapter we limit ourselves to the case where both input (x) and output (y) are univariate (one input and one output) and real-valued such that

$$y \approx M\left(\boldsymbol{\theta}\,;\,x\right). \tag{42.1}$$

As indicated in this relation, a model will typically include some model parameters ($\boldsymbol{\theta}$). These might already be known, or they might need to be inferred from a set of model calibration data **data** . A large fraction of our discussions will revolve around the task of determining model parameters. We might refer to this endeavour as *model calibration* or *parameter estimation*. It is an example of a *scientific inference* problem.

Unfortunately, model calibration is a challenging task that is often performed with a lack of scientific rigour. In future chapters we will explore both the *optimization approach*—which is very common and is absolutely dominating in the construction of machine-learning models—and the scientifically more relevant process of *statistical inference*. For the latter we will in particular use *Bayesian methods*. The Bayesian perspective is very useful as it allows a more formal definition of the process of learning that we can apply also in general machine-learning contexts.

Definition 42.2 (Statistical inference)

Statistical inference is the quantitative process used for drawing conclusions about the nature of some system on the basis of data and models subject to random variation. Probability is the quantitative metric used to measure the strength of statistical inference.

Merriam-Webster: *"The act of passing from one proposition, statement, or judgment considered as true to another whose truth is believed to follow from that of the former."* (this definition is for the root term "inference")

Note that Eq. (42.1) indicates an approximate relationship. Scientific modeling usually relies on a number of approximations. The model should therefore not be expected to provide an exact representation of reality. The missing piece can be referred to as the *model discrepancy*. In addition, reality is observed via experiments that are associated with *experimental errors*. The proper way of handling these uncertainties is via random variables and associated probability distributions. We address this approach in the **Bayesian inference** part, in particular starting from Data, models, and predictions.

We will mainly consider *deterministic models* that uniquely map inputs to outputs. Despite the fact that statistical inference relies on stochastic modelling of error terms, the form of the model $M\left(\boldsymbol{\theta}\,;\,x\right)$ is still a deterministic one. However, some processes are better described by *stochastic models*. We will make some acquaintance with this kind of modeling in the **Stochastic processes** part.

42.3. Parametric versus non-parametric models

Most scientific models contain parameters and are therefore known as *parametric models*. In physics these parameters might be constrained by theoretical hypotheses and by previous observations of related systems. Moreover, the parameters values might be interesting by themselves which broadens the scope of modeling from just describing a relationship to actually extracting physics knowledge.

We should not underestimate the power of physics insights when creating a model. Such insights help in making informed decisions on relevant modeling approximations, which in turn helps in quantifying the size of model discrepancies. In fact, one could claim that true predictive power rests in the ability to make reliable statements on the precision (uncertainty) of a prediction.

It is also possible to learn a relationship without having detailed modeling insights. For this purpose one can consider very general models characterized by large flexibility (allowing to model different relationships) and set up a learning algorithm that adjusts the model to fit a specific purpose. This approach could in general be labeled as machine learning and has proven to be very powerful in many different contexts. Very often, the learning process is performed with a large amount of labeled training data but it might also be possible to achieve without such "supervision". The machine-learning models and learning algorithms usually involve various kinds of parameters,

but these are typically not meaningful in themselves. Strictly speaking, *non-parametric* models are those for which the effective number of parameters grows with the amount of training data (examples are k-nearest neighbors and Gaussian processes). The term is sometimes used more loosely for very flexible models whose many parameters lack an interpretation. We will encounter this approach in the **Machine learning** part of these lecture notes.

42.4. Linear versus non-linear models

In **linear modeling** the dependence on the model parameters $\boldsymbol{\theta}$ is **linear**. As we will see in Section 43: Linear models this implies that we can utilize rather straightforward linear algebra methods to perform linear regression analysis.

Linear models are not always applicable. When the parameter dependence is more complicated we will have to use the much broader family of **non-linear modeling**. In general it will be more computationally demanding to deal with non-linear regression analysis.

Example 42.1 (Linear models)

This is an example of a linear model

$$M\left(\boldsymbol{\theta};\, x\right) = \theta_0 + \theta_1 x + \theta_2 x^2.$$

Note that the parameters $\boldsymbol{\theta}$ enter linearly although the dependence on x (which is the independent variable) is quadratic.

Here is a second example that corresponds to a truncated trigonometric series

$$M\left(\boldsymbol{\theta};\, x\right) = A_0 + \sum_{n=1}^{N} \left[A_n \sin(nx) + B_n \cos(nx)\right],$$

where the model parameters $\boldsymbol{\theta} = \{A_0, A_1, \ldots, A_N, B_1, \ldots, B_N\}$ again enter linearly.

Example 42.2 (Non-linear models)

This is an example of a non-linear model

$$M\left(\boldsymbol{\theta};\, x\right) = \theta_0 + \theta_1 \exp(-\theta_2 x),$$

with three parameters.

42.5. Regression analysis: optimization versus inference

Assume that we have access to N_d instances of data corresponding to measured responses $\{y_1, y_2, \ldots y_{N_d}\}$ and the corresponding values for the independent variable $\{x_1, x_2, \ldots x_{N_d}\}$. Let us define a *cost function* $C(\boldsymbol{\theta})$ that quantifies how well our model $M\left(\boldsymbol{\theta};\, x\right)$ reproduces the training data,

$$C(\boldsymbol{\theta}) = \sum_{i=1}^{N_d} \frac{(y_i - M(\boldsymbol{\theta}; x_i))^2}{\sigma_i^2}, \tag{42.2}$$

where we have introduced some scaling parameters $\{\sigma_i\}_{i=1}^{N_d}$ to produce a dimensionless value. Let us stress that the choice of cost function is by no means unique and we will offer both pragmatic and statistical perspectives on this later on.

The most common approach to regression analysis is to *optimize* the model parameters. The goal is then to find

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta} \in \mathbb{R}^p}{\arg\min}\, C(\boldsymbol{\theta}). \tag{42.3}$$

We note that this task is an optimization problem that can become challenging when the model is non-linear and the parameter dimension p is large. We will discuss gradient-descent-based optimization methods in Section 44: Mathematical optimization.

The optimization approach to regression will provide limited information on the model precision. It is also prone to overfitting and other issues of high-dimensional parameter volumes. In the **Bayesian inference** part we will therefore formulate regression as an inductive inference problem, with rigorous handling of uncertainties. See in particular Section 8.3: BLR-I: Background.

42.6. Exercises

Exercise 42.1 (Independent and dependent)

Consider a free-falling body with mass m. Neglecting a drag force it is straight-forward to use Newton's equation to derive an expression for the distance d that the body has fallen during a time t when starting from rest at $t = 0$. Identify the dependent and the independent variable in this relation. What is the model parameter(s)?

Exercise 42.2 (Linear or non-linear)

Consider the relation between fall time t and velocity v for a free-falling body of mass m (starting from rest) that experiences a drag force that is modeled as bv

$$v = v_T \left(1 - e^{-\frac{b}{m}t}\right),$$

where v_T is the terminal velocity.

- What are the model parameters?
- Is this a linear or a non-linear model?
- How would the relation look like if the drag force was neglected? Would that be a linear or a non-linear model?

Exercise 42.3 (Linear or non-linear; more examples)

Are these models linear or non-linear?

1. $M(\boldsymbol{\theta}; x) = \theta_0 + (\theta_1 x)^2$
2. $M(\boldsymbol{\theta}; x) = e^{\theta_0 - \theta_1 x/2}$
3. $M(\boldsymbol{\theta}; x) = \theta_0 e^{-x/2}$
4. $M(\boldsymbol{\theta}; x) = \theta_0 e^{-x/2} + \theta_1 \sin(x^2\pi)$
5. $M(\boldsymbol{\theta}; x) = (\theta_0 + \theta_1 x)^2$
6. $M(\boldsymbol{\theta}; x) = (\theta_0 + \theta_1 x)^2 + \theta_2 x$

Exercise 42.4 (Model discrepancy)

Consider again the relation between fall time t and velocity v for a free-falling body of mass m (starting from rest) that experiences a drag force that is modeled as bv

$$v = v_T \left(1 - e^{-\frac{b}{m}t}\right),$$

where v_T is the terminal velocity. Discuss possible model discrepancies. Are they expected to be large or small effects?

Solutions to exercises

Solution to Exercise 42.1 (Independent and dependent)

The relation is $d = gt^2/2$. Here we are describing how the distance traveled depends on the time of the free fall. Therefore d is the dependent variable and t is the independent one. There is a single model parameter g that we could infer from observational data.

Solution to Exercise 42.2 (Linear or non-linear)

- The model parameters are v_T and b/m. Alternatively, since $v_T = mg/b$, we could consider g and b/m as the model parameters. It would not be correct to claim that we have three model parameters since b and m only appear in a ratio.
- This is a non-linear model since b/m appears in an exponential.
- The corresponding relation without drag force is $v = gt$. That is a linear model.

Solution to Exercise 42.3 (Linear or non-linear; more examples)

1. Linear (we can consider θ_1^2 as a parameter).
2. Non-linear.
3. Linear (there is no parameter-dependence in the exponential function).
4. Linear.
5. Non-linear. It would be tempting to consider θ_0^2, θ_1^2, and $2\theta_0\theta_1$ as three independent parameters in which case it would be a linear model. But these are not independent and we would need to keep the quadratic parameter dependence.
6. Linear if we consider θ_0^2, θ_1^2, and $2\theta_0\theta_1 + \theta_2$ as parameters.

Solution to Exercise 42.4 (Model discrepancy)

First, we are assuming that the gravitational force is constant for the duration of the fall. This approximation corresponds to setting $GM/(R+h)^2 \approx GM/R^2 = g$ where G is the gravitational constant, $M(R)$ is the earth mass(radius), and we neglect the small and varying height h. Since $GM/(R+h)^2 \approx g(1-2h/R)$, the relative error that is made here will be of order h/R which is really small.

More importantly, the linear drag force is a simplification. We could add a term that is quadratic in the velocity. The error made by not including this term will grow with velocity.

Finally, Newton's equations of motion have turned out to be an approximation of the general theory of relativity. Again, the modeling error will be negligible for "normal" masses and velocities.

43. Linear models

> "As always in life, people want a simple answer; … and it's always wrong."
>
> —Susan Greenfield

In this chapter we will discuss models M that are linear in their parameters $\boldsymbol{\theta}$. Although linear models are simple, they are sometimes useful for analyzing real-world data.

43.1. Definition of linear models

In **linear modeling** the dependence on the model parameters $\boldsymbol{\theta}$ is **linear**, and this fact will make it possible to express that regression analysis as a linear algebra problem, and as we will show it will be possible to find an analytical expression for the optimal set of model parameters. Note that we will mostly operate with models depending on more than one parameter. Hence, we denote the parameters ($\boldsymbol{\theta}$) using a bold symbol. In this chapter we will, however, consider models (M) that relate a single dependent variable (y) with a single independent one (x).

The linear parameter dependence implies that our model separates into a sum of parameters times basis functions. Assuming N_p different basis functions we have

$$M\left(\boldsymbol{\theta}\,;\,x\right) = \sum_{j=0}^{N_p-1} \theta_j f_j(x). \tag{43.1}$$

Note that there is no $\boldsymbol{\theta}$-dependence in the basis functions $f_j(x)$.

From a machine-learning perspective the different basis functions are known as **features**.

Example 43.1 (Polynomial basis functions)

A common linear model corresponds to the use of polynomial basis functions $f_j(x) = x^j$. A polynomial model of degree $N_p - 1$ would then be written

$$M(\boldsymbol{\theta}; x) = \sum_{j=0}^{N_p-1} \theta_j x^j. \tag{43.2}$$

Note that the $j = 0$ basis function is $f_0(x) = x^0 = 1$ such that the θ_0 parameter becomes the intercept.

Example 43.2 (Liquid-drop model for nuclear binding energies)

The liquid drop model is useful for a phenomenological description of nuclear binding energies (BE) as a function of the mass number A and the number of protons Z, neutrons N

$$\text{BE}\ (A, N, Z) = a_0 + a_1 A + a_2 A^{2/3} + a_3 Z^2 A^{-1/3} + a_4 (N - Z)^2 A^{-1}. \tag{43.3}$$

We have five features: the intercept (constant term, bias), the A dependent volume term, the $A^{2/3}$ surface term and the Coulomb $Z^2 A^{-1/3}$ and asymmetry $(N - Z)^2 A^{-1}$ terms. Although the features are somewhat complicated functions of the independent variables A, N, Z, we note that the $p = 5$ regression parameters $\boldsymbol{\theta} = (a_0, a_1, a_2, a_3, a_4)$ enter linearly.

43.2. Regression analysis with linear models

When performing a regression analysis with a linear model, i.e., doing linear regression, we have access to a set of data $\mathcal{D}$ for the dependent variable, i.e.,

$$\textbf{data}\ = [y_1, y_2, \ldots, y_{N_d}]^T. \tag{43.4}$$

For each datum there is an independent variable x_i, and our model for each datum y_i is

$$M_i \equiv M(\boldsymbol{\theta}; x_i) = \sum_{j=0}^{N_p-1} \theta_j f_j(x_i). \tag{43.5}$$

We can collect the basis function evaluated at each independent variable x_i in a matrix $\boldsymbol{X}$ of dimension $N_d \times N_p$

$$\boldsymbol{X} = \begin{bmatrix} f_0(x_1) & \ldots & f_{N_p-1}(x_1) \\ f_0(x_2) & \ldots & f_{N_p-1}(x_2) \\ \vdots & \ddots & \vdots \\ f_0(x_{N_d}) & \ldots & f_{N_p-1}(x_{N_d}) \end{bmatrix} \tag{43.6}$$

This matrix will be referred to as a **design matrix**.

Example 43.3 (The design matrix for polynomial models)

The design matrix for a linear model with polynomial basis functions becomes

$$\boldsymbol{X} = \begin{bmatrix} 1 & x_1^1 & x_1^2 & \dots & \dots & x_1^{p-1} \\ 1 & x_2^1 & x_2^2 & \dots & \dots & x_2^{p-1} \\ 1 & x_3^1 & x_3^2 & \dots & \dots & x_3^{p-1} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 1 & x_{N_d}^1 & x_{N_d}^2 & \dots & \dots & x_{N_d}^{p-1} \end{bmatrix}, \tag{43.7}$$

where we are considering a polynomial of degree $p-1$ which implies a model with p features (including the intercept). It is also known within linear algebra as a Vandermonde matrix.

Next, we introduce a column vector for the parameters

$$\boldsymbol{\theta} = [\theta_0, \theta_1, \theta_2, \dots, \theta_{N_p-1}]^T, \tag{43.8}$$

and we arrive at the matrix equation

$$\textbf{data} \ = \boldsymbol{X\theta} + \boldsymbol{\epsilon}. \tag{43.9}$$

The last term $\boldsymbol{\epsilon}$ is a column vector of so-called **residuals**. This term expresses the part of the dependent variable, for which we have data, that we cannot describe using a linear model. Formally, we can therefore write $\epsilon_i = y_i - M_i$ and define the vector as

$$\boldsymbol{\epsilon} = [\epsilon_1, \epsilon_2, \epsilon_3, \dots, \epsilon_{N_d}]^T. \tag{43.10}$$

It is important to realize that our model M provides an approximate description of the data. Indeed, *all models are wrong* and in a realistic setting we have no guarantee that the data is generated by a linear process. Of course, based on physics insight, or other assumptions, there might exist very good reasons for using a linear model to explain the data.

The normal equation

A regression analysis often aims at finding the model parameters $\boldsymbol{\theta}$ of a model M such that the vector of errors $\boldsymbol{\epsilon}$ is minimized in the sense of its Euclidean norm (or 2-norm). You might ask the very relevant question why this particular goal is desirable. We will return to this consideration in BLR-I: Background. Nevertheless, in order to find the "optimal" set of parameters $\boldsymbol{\theta}^*$ we seek to minimize

$$C(\boldsymbol{\theta}) \equiv \sum_{i=1}^{N_d} \epsilon_i^2 = \sum_{i=1}^{N_d} \left(y_i - M_i\right)^2 = \Big\{ \left(\textbf{data} \ - \boldsymbol{X\theta}\right)^T \left(\textbf{data} \ - \boldsymbol{X\theta}\right) \Big\}. \tag{43.11}$$

The solution to this optimization problem turns out to be a solution of the normal equation and is known as ordinary least-squares or ordinary linear regression.

Theorem 43.1 (Ordinary least squares (the normal equation))

The ordinary least-squares method corresponds to finding the optimal parameter vector $\boldsymbol{\theta}^*$ that minimizes the Euclidean norm of the residual vector $\boldsymbol{\epsilon} = \textbf{data} - \boldsymbol{X\theta}$, where **data** is a column vector of observations and $\boldsymbol{X}$ is the design matrix (43.6).

Finding this optimum turns out to correspond to solving the **normal equation**

$$\boldsymbol{X}^T \textbf{data} = \boldsymbol{X}^T \boldsymbol{X\theta}^*. \tag{43.12}$$

Given that the **normal matrix** $\boldsymbol{X}^T\boldsymbol{X}$ is invertible (which is the case if $\boldsymbol{X}$ has full column rank), the solution to the normal equation is given by

$$\boldsymbol{\theta}^* = \left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T \textbf{data}\ . \tag{43.13}$$

Proof. We will derive the normal equation in four steps:

1. *Cost function.* The cost function is the squared Euclidean norm of the residual vector. Expanding the product gives

$$C(\boldsymbol{\theta}) = (\textbf{data} - \boldsymbol{X\theta})^T (\textbf{data} - \boldsymbol{X\theta}) = \textbf{data}^T \textbf{data} - 2\boldsymbol{\theta}^T\boldsymbol{X}^T \textbf{data} + \boldsymbol{\theta}^T\boldsymbol{X}^T\boldsymbol{X\theta},$$

   where we used that $\textbf{data}^T \boldsymbol{X\theta} = \boldsymbol{\theta}^T\boldsymbol{X}^T \textbf{data}$ since it is a scalar. This is a quadratic function of the parameters.

2. *Gradient.* With the matrix elements $X_{ij} = f_j(x_i)$ of the design matrix, the cost function reads $C(\boldsymbol{\theta}) = \sum_{i=1}^{N_d}\left(y_i - \sum_{k=0}^{N_p-1} X_{ik}\theta_k\right)^2$. Its partial derivative with respect to θ_j is

$$\frac{\partial C(\boldsymbol{\theta})}{\partial \theta_j} = -2\sum_{i=1}^{N_d} X_{ij}\left(y_i - \sum_{k=0}^{N_p-1} X_{ik}\theta_k\right) = -2\left[\boldsymbol{X}^T(\textbf{data} - \boldsymbol{X\theta})\right]_j, \tag{43.14}$$

   where the last step follows since the j:th element of $\boldsymbol{X}^T\boldsymbol{v}$ is $\sum_i X_{ij}v_i$ for any vector $\boldsymbol{v}$. Collecting all elements gives the gradient vector

$$\boldsymbol{\nabla}_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) = -2\boldsymbol{X}^T(\textbf{data} - \boldsymbol{X\theta}). \tag{43.15}$$

3. *Normal equation.* A minimum of C must be a stationary point, $\boldsymbol{\nabla}_{\boldsymbol{\theta}} C(\boldsymbol{\theta}^*) = 0$, which gives

$$\boldsymbol{X}^T \textbf{data} = \boldsymbol{X}^T\boldsymbol{X\theta}^*.$$

4. *Minimum.* The second derivatives, $\partial^2 C/\partial\theta_j\partial\theta_k = 2\left(\boldsymbol{X}^T\boldsymbol{X}\right)_{jk}$, form a constant Hessian matrix $2\boldsymbol{X}^T\boldsymbol{X}$. It is positive semi-definite since $\boldsymbol{v}^T\boldsymbol{X}^T\boldsymbol{Xv} = |\boldsymbol{Xv}|_2^2 \geq 0$ for any vector $\boldsymbol{v}$. The cost function is therefore convex, and any solution of the normal equation is a global minimum. If $\boldsymbol{X}$ has full column rank, then

$\boldsymbol{X}\boldsymbol{v} \neq 0$ for all $\boldsymbol{v} \neq 0$. The normal matrix $\boldsymbol{X}^T\boldsymbol{X}$ is then positive definite, and therefore invertible, and the minimum is unique

$$\boldsymbol{\theta}^* = \left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\textbf{data}\ .$$

We note also that since our design matrix is defined as $\boldsymbol{X} \in \mathbb{R}^{N_d \times N_p}$, the product $\boldsymbol{X}^T\boldsymbol{X} \in \mathbb{R}^{N_p \times N_p}$. If $\boldsymbol{X}$ has full column rank, i.e., N_p linearly independent columns (which requires $N_d \geq N_p$), the product $\left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T$ is called the pseudo-inverse of the design matrix $\boldsymbol{X}$. It generalizes the usual matrix inverse to non-square matrices, and for a square, full-rank matrix it is equal to the usual inverse. The more general Moore-Penrose pseudo-inverse is defined also for matrices that are not full rank.

Checkpoint question

Here we have been minimizing the sum of squared residuals, see Eq. (43.11). An optimization metric that is less dependent on the number of data is the **mean-squared error**. The cost function is then

$$C_{\text{MSE}}(\boldsymbol{\theta}) \equiv \frac{1}{N_d}\Big\{(\textbf{data}\ - \boldsymbol{X}\boldsymbol{\theta})^T(\textbf{data}\ - \boldsymbol{X}\boldsymbol{\theta})\Big\}.$$

If, and how, would this choice of cost function modify:

- The optimum $\boldsymbol{\theta}^*$?
- The expression for the gradient vector $\boldsymbol{\nabla}_{\boldsymbol{\theta}} C_{\text{MSE}}(\boldsymbol{\theta})$?

Answers

The only difference is a factor $1/N_d$ which does not affect the position of the minimum. The gradient, however, will inherit this factor from the cost fundtion.

The regression residuals $\boldsymbol{\epsilon}^* = \textbf{data}\ - \boldsymbol{X}\boldsymbol{\theta}^*$ can be used to obtain an estimator s^2 of the variance of the residuals

$$s^2 = \frac{(\boldsymbol{\epsilon}^*)^T\boldsymbol{\epsilon}^*}{N_d - N_p},$$

where N_p is the number of parameters in the model and N_d is the number of data.

43.3. Ordinary linear regression: warmup

To warm up, and get acquainted with the notation and formalism, let us work out a small example. Assume that we have the situation where we have collected two datapoints $\textbf{data}\ = [y_1, y_2]^T = [-3, 3]^T$ for the predictor values $[x_1, x_2]^T = [-2, 1]^T$.

This data could have come from any process, even a non-linear one. But this is artificial data that I generated by evaluating the function $y = 1 + 2x$ at $x = x_1 = -2$ and $x = x_2 = 1$. Clearly, the data-generating process is very simple and corresponds to a linear model $y = \theta_0 + \theta_1 x$ with $[\theta_0, \theta_1] = [1, 2]$. This is the kind of information

we *never* have in reality. Indeed, we are always uncertain about the process that maps input to output, and as such our model M will always be wrong. We are also uncertain about the parameters $\boldsymbol{\theta}$ of our model. These are some of the fundamental reasons for why it can be useful to operate with a Bayesian approach where we can assign probabilities to any quantity and statement. In this example, however, we will continue with the standard (frequentist) approach based on finding the parameters that minimize the squared errors (i.e., the norm of the residual vector).

We will now assume a linear model with polynomial basis up to order one to model the data, i.e.,

$$M(\boldsymbol{\theta}; x) = \theta_0 + \theta_1 x,$$

which we can express in terms of a design matrix $\boldsymbol{X}$ and (unknown) parameter vector $\boldsymbol{\theta}$ as $M = \boldsymbol{X}\boldsymbol{\theta}$.

In the present case the two unknowns $\boldsymbol{\theta} = [\theta_0, \theta_1]^T$ can be fit to the two datapoints **data** $= [-3, 3]^T$ using pen and paper.

Exercise 43.1

In the example above you have two data points and two unknowns, which means you can easily solve for the model parameters using a conventional matrix inverse. Do the numerical calculation to make sure you have setup the problem correctly.

Exercise 43.2

Evaluate the normal equations for the design matrix $\boldsymbol{X}$ and data vector **data** in the example above.

Exercise 43.3

Evaluate the sample variance s^2 for the example above. Do you think the result makes sense?

Solutions to exercises

Here are answers and solutions to selected exercises.

Solution to Exercise 43.1

We have the following design matrix

$$\boldsymbol{X} = \begin{bmatrix} 1 & -2 \\ 1 & 1 \end{bmatrix},$$

which in the present case yields the parameter values

$$\boldsymbol{\theta}^* = \boldsymbol{X}^{-1}\textbf{data} = [1, 2]^T.$$

Solution to Exercise 43.3

For the warmup case we have fitted a straight line through two data points, which is always possible, and we cannot determine the sample variance. This will be even more clear when we come to BLR-I: Background.

43.4. Ordinary linear regression in practice

We often have situation where we have much more than just two datapoints, and they rarely fall exactly on a straight line. Let's use python to generate some more realistic, yet artificial, data. Using the function below you can generate data from some linear process with random variables for the underlying parameters. We call this a data-generating process.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def data_generating_process_reality(model_type, rng=np.random.default_rng(), **kwargs):
  if model_type == 'polynomial':
      true_params = rng.uniform(low=-5.0, high=5, size=(kwargs['poldeg']+1,))
      #polynomial model
      def process(params, xdata):
          ydata = np.polynomial.polynomial.polyval(xdata,params)
          return ydata

  # use this to define a non-polynomial (possibly non-linear) data-generating process
  elif model_type == 'nonlinear':
      true_params = None
      def process(params, xdata):
          ydata = (0.5 + np.tan(np.pi*xdata))**2
          return ydata
  else:
      print(f'Unknown Model')

  # return function for the true process the values for the true parameters
  # and the name of the model_type
  return process, true_params, model_type
```

Next, we make some measurements of this process, and that typically entails some measurement errors. We will here assume that independently and identically distributed (i.i.d.) measurement errors e_i that all follow a normal distribution with zero mean and variance σ_e^2. In a statistical notation we write $e_i \sim \mathcal{N}(0, \sigma_e^2)$. By default, we set $\sigma_e = 0.5$.

```
def data_generating_process_measurement(process, params, xdata,
                           sigma_error=0.5, rng=np.random.default_rng()):

  ydata = process(params, xdata)

  #  sigma_error: measurement error.
  error = rng.normal(0,sigma_error,len(xdata)).reshape(-1,1)

  return ydata+error, sigma_error*np.ones(len(xdata)).reshape(-1,)
```

Let us setup the data-generating process, in this case a linear process of polynomial degree 1, and decide how many measurements we make ($N_d = 10$). All relevant output is stored in pandas dataframes.

```
#the number of data points to collect
# -----
Nd = 10
# -----

# predictor values
xmin = -1 ; xmax = +1
Xmeasurement = np.linspace(xmin,xmax,Nd).reshape(-1,1)
# store it in a pandas dataframe
pd_Xmeasurement = pd.DataFrame(Xmeasurement, columns=['x'])

# Define the data-generating process.
# Begin with a polynomial (poldeg=1) model_type
# in a second run of this notebook you can play with other linear models
reality, true_params, model_type =
data_generating_process_reality(model_type='polynomial',poldeg=1)

print(f'model type      : {model_type}')
print(f'true parameters : {true_params}')
print(f'Nd = {Nd}')

# Collect measured data
# -----
sigma_e = 0.5
# -----
Ydata, Yerror =
data_generating_process_measurement(reality,true_params,Xmeasurement,sigma_error=sigma_e)
# store the data in a pandas dataframe
pd_D=pd.DataFrame(Ydata,columns=['data'])
#
pd_D['x'] = Xmeasurement
pd_D['e'] = Yerror

# We will also produce a denser grid for predictions with our model and comparison with the
true process. This is useful for plotting

xreality = np.linspace(xmin,xmax,200).reshape(-1,1)
pd_R = pd.DataFrame(reality(true_params,xreality), columns=['data'])
pd_R['x'] = xreality
```

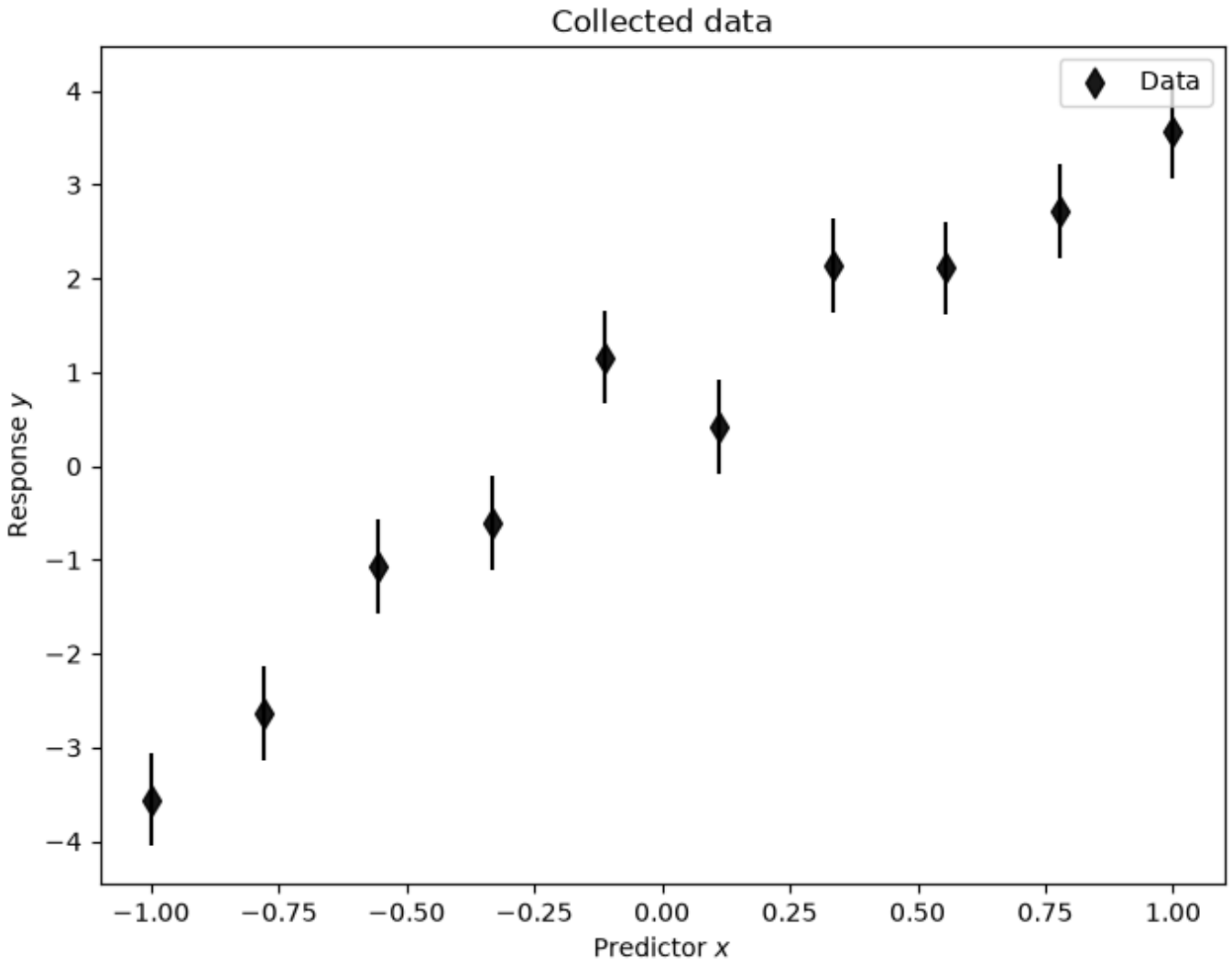
```
model type      : polynomial
true parameters : [0.44492175 3.36080377]
Nd = 10
```

Create some analysis tool to inspect the data, and later on the model.

```
# helper function to plot data, reality, and model (pd_M)
def plot_data(pd_D, pd_R, pd_M, with_errorbars = True):
    fig, ax = plt.subplots(1,1,figsize=(8,6))
    ax.scatter(pd_D['x'],pd_D['data'],label=r'Data',color='black',zorder=1,
alpha=0.9,s=70,marker="d");
    if with_errorbars:
        ax.errorbar(pd_D['x'],pd_D['data'], pd_D['e'],fmt='o', ms=0, color='black');
    if pd_R is not None:
        ax.plot(pd_R['x'], pd_R['data'],color='red', linestyle='--
',lw=3,label='Reality',zorder=10)
    if pd_M is not None:
        ax.plot(pd_M['x'], pd_M['data'],color='blue', linestyle='--
',lw=3,label='Model',zorder=11)
    ax.legend();
    ax.set_title('Collected data');
    ax.set_xlabel(r'Predictor $x$');
    ax.set_ylabel(r'Response $y$');
    return fig,ax
```

Let's have a look at the data. We set the last two arguments to `None` for visualizing only the data.

```
plot_data(pd_D, None, None);
```



Linear regression proceeds via the design matrix. We will analyze this data using a linear polynomial model of order 1. The following code will allow you to setup the corresponding design matrix $\boldsymbol{X}$ for any polynomial order (referred to as poldeg below)

```
def setup_polynomial_design_matrix(data_frame, poldeg, drop_constant=False, verbose=True):
    if verbose:
        print('setting up design matrix:')
        print('  len(data):', len(data_frame.index))

        # for polynomial models: x^0, x^1, x^2, ..., x^p
        # use numpy increasing vandermonde matrix
        print('  model poldeg:',poldeg)

    predictors = np.vander(data_frame['x'].to_numpy(), poldeg+1, increasing = True)
    if drop_constant:
        predictors = np.delete(predictors, 0, 1)
        if verbose:
            print('  dropping constant term')
    pd_design_matrix = pd.DataFrame(predictors)

    return pd_design_matrix
```

So, let's setup the design matrix for a model with polynomial basis functions. Note that there are N_p parameters in a polynomial function of order $N_p - 1$

$$M(\boldsymbol{\theta}; x) = \theta_0 + \theta_1 x.$$

```
Np=2
pd_X = setup_polynomial_design_matrix(pd_Xmeasurement,poldeg=Np-1)
```

```
setting up design matrix:
  len(data): 10
  model poldeg: 1
```

We can now perform linear regression, or ordinary least squares (OLS), as

```
#ols estimator for physical parameter theta
D = pd_D['data'].to_numpy()
X = pd_X.to_numpy()

ols_cov   = np.linalg.inv(np.matmul(X.T,X))
ols_xTd   = np.matmul(X.T,D)
ols_theta = np.matmul(ols_cov,ols_xTd)

print(f'Ndata = {Nd}')
print(f'theta_ols \t{ols_theta}')
print(f'theta_true \t{true_params}\n')
```

```
Ndata = 10
theta_ols       [0.4240423  3.40692722]
theta_true      [0.44492175 3.36080377]
```

To evaluate the (fitted) model we setup a design matrix that spans dense values across the relevant range of predictors.

```
pd_Xreality = setup_polynomial_design_matrix(pd_R,poldeg=Np-1)
```

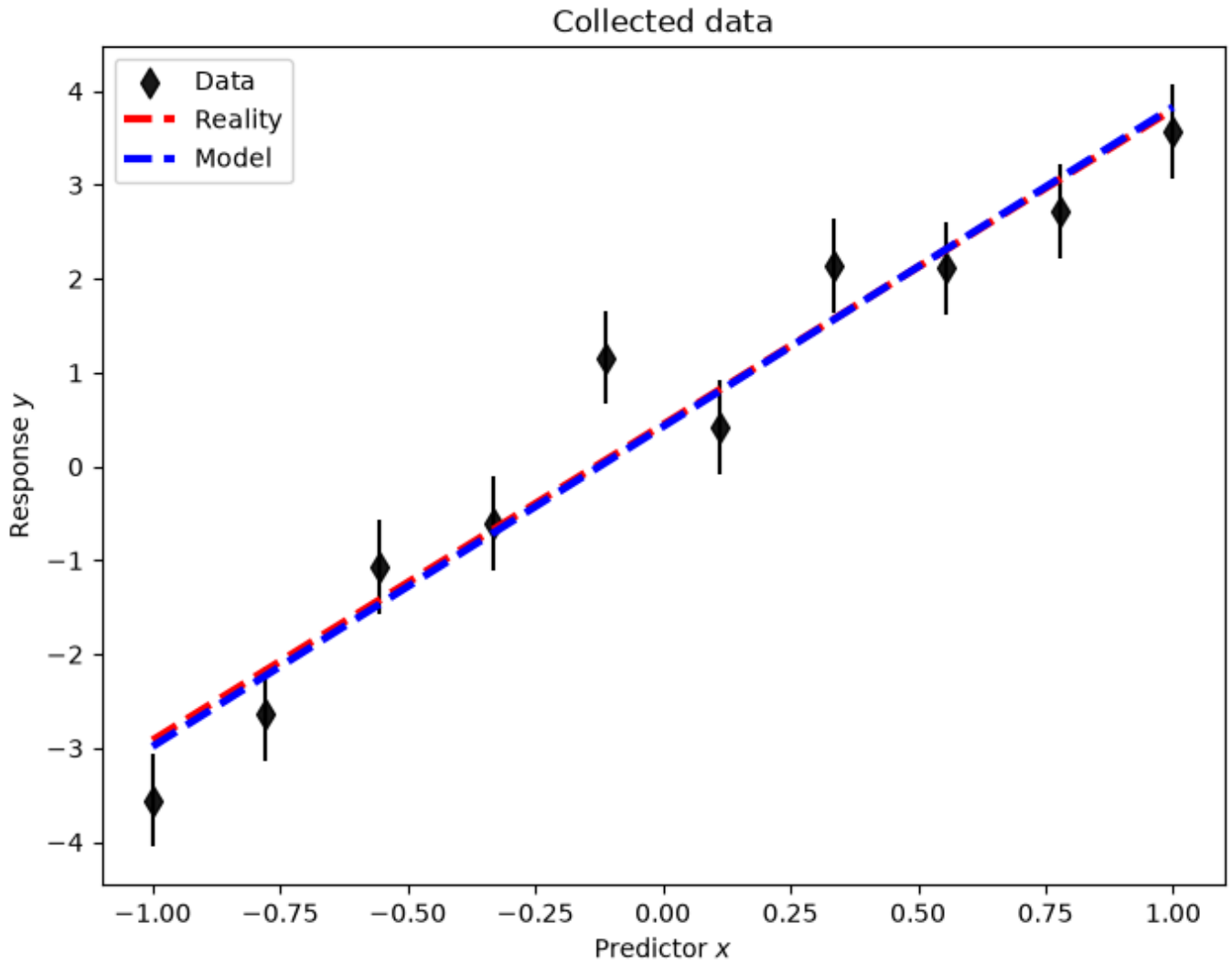
```
setting up design matrix:
  len(data): 200
  model poldeg: 1
```

and then we dot this with the fitted (ols) parameter values

```
Xreality = pd_Xreality.to_numpy()
pd_M_ols = pd.DataFrame(np.matmul(Xreality,ols_theta),columns=['data'])
pd_M_ols['x'] = xreality
```

A plot (which now includes the data-generating process 'reality') demonstrates the quality of the inference.

```
plot_data(pd_D, pd_R, pd_M_ols);
```



To conclude, we also compute the sample variance s^2

```
ols_D   = np.matmul(X,ols_theta)
ols_eps = (ols_D - D)
ols_s2 = (np.dot(ols_eps,ols_eps.T)/(Nd-Np))

print(f's^2       \t{ols_s2:.3f}')
print(f'sigma_e^2 \t{sigma_e**2:.3f}')
```

```
s^2             0.330
sigma_e^2       0.250
```

As seen, the extracted variance is in some agreement with the true one.

Using the code above, you should now try to do the following exercises.

Exercise 43.4

Keep working with the simple polynomial model $M = \theta_0 + \theta_1 x$

Reduce the number of data to 2, i.e., set Nd=2. Do you reproduce the result from the simple example in the previous section?

Increase the number of data to 1000. Do the OLS values of the model parameters and the sample variance approach the (true) parameters of the data-generating process? Is this to be expected?

Exercise 43.5

Set the data-generating process to be a 3rd-order polynomial and set limits of the predictor variable to [-3,3]. Analyze the data using a 2nd-order polynomial model.

Explore the limit of $N_d \to \infty$ by setting $N_d = 500$ or so. Will the OLS values of the model parameters and the sample variance approach the (true) values for some of the parameters?

44. Mathematical optimization

> "With four parameters I can fit an elephant, and with five I can make him wiggle his trunk."

—John von Neumann

(see https://en.wikipedia.org/wiki/Von_Neumann's_elephant for the historical context of this quote.)

Mathematical optimization is a large area of research in applied mathematics. There are many applications in science and technology. Recently, there is a particular focus on this area due to its importance in machine learning and artificial intelligence, and as a potential candidate application for quantum computing algorithms.

In a broader context, optimization problems involve maximizing or minimizing a real-valued function by systematically choosing input values from within an allowed set and computing the value of the function.

Discrete or continuous optimization

Optimization problems can be divided into two categories, depending on whether the variables are continuous or discrete:

- An optimization problem is known as discrete if it involves finding an argument from a countable set such as an integer, permutation or graph.
- A problem is known as a continuous optimization if arguments from a continuous set must be found.

We will mainly be concerned with continuous optimization problems in scientific modeling for which the input variables $\boldsymbol{\theta}$ are known as model parameters and the allowed set V is some subset of an Euclidean space $\mathbb{R}^p$.

Mathematically, we want to consider the following *minimization* problem

Definition 44.1 (Global minimization)

Given a function $C : V \to \mathbb{R}$, where V is a search space that possibly involves various constraints, we seek the element $\boldsymbol{\theta}^* \in V$ such that $C(\boldsymbol{\theta}^*) \leq C(\boldsymbol{\theta}),\ \forall \boldsymbol{\theta} \in V$.

We will often use the shorthand notation

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta} \in V}{\operatorname{argmin}}\, C(\boldsymbol{\theta}) \tag{44.1}$$

to indicate the solution of a minimization problem.

The identification of $\boldsymbol{\theta}^* \in V$ such that $C(\boldsymbol{\theta}^*) \geq C(\boldsymbol{\theta}),\ \forall \boldsymbol{\theta} \in V$, is known as *maximization*. However, since the following is valid,

$$C(\boldsymbol{\theta}^*) \leq C(\boldsymbol{\theta}) \Leftrightarrow -C(\boldsymbol{\theta}^*) \geq -C(\boldsymbol{\theta}), \tag{44.2}$$

a maximization problem can be recast as a minimization one and we will restrict ourselves to consider only minimization problems.

The function C has various names in different applications. It can be called a cost function, objective function or loss function (minimization), a utility function or fitness function (maximization), or, in certain fields, an energy function or energy functional. A feasible solution that minimizes (or maximizes, if that is the goal) the objective function is in general called an optimal solution.

Definition 44.2 (Local minimization)

A solution $\boldsymbol{\theta}^* \in V$ that fulfills

$$C(\boldsymbol{\theta}^*) \leq C(\boldsymbol{\theta}),\ \forall \boldsymbol{\theta} \in V,\ \text{where}\quad \|\boldsymbol{\theta} - \boldsymbol{\theta}^*\| \leq \delta,$$

for some $\delta > 0$ is known as a *local minimum*.

Generally, unless the cost function is convex in a minimization problem, there may be several local minima within V. In a convex problem, if there is a local minimum that is interior (not on the edge of the set of feasible elements), it is also the global minimum, but a nonconvex problem may have more than one local minimum not all of which need be global minima.

With the linear regression model we could find the best fit parameters by solving the normal equation. Although we could encounter problems associated with inverting a matrix, we do in principle have a closed-form expression for the model parameters. In general, however, the problem of optimizing the model parameters is a very difficult one.

It is important to understand that the majority of available optimizers are not capable of making a distinction between locally optimal and globally optimal solutions. They will therefore, erroneously, treat the former as actual solutions to the global optimization problem.

44.1. Gradient-descent optimization

Gradient descent is probably the most popular class of algorithms to perform optimization. It is certainly the most common way to optimize neural networks. Although there are different flavours of gradient descent, as will be discussed, the general feature is an iterative search for a (locally) optimal solution using the gradient of the cost function. Basically, the parameters are tuned in the opposite direction of the gradient of the objective function, thus aiming to follow the direction of the slope downhill until we reach a valley. The evaluation of this gradient at every iteration is often the major computational bottleneck.

Algorithm 44.1 (Gradient descent optimization)

1. Start from a *random initialization* of the parameter vector $\boldsymbol{\theta}_0$.
2. At iteration n:
   1. Evaluate the gradient of the cost function at the current position $\boldsymbol{\theta}_n$: $\boldsymbol{\nabla} C_n \equiv \boldsymbol{\nabla}_{\boldsymbol{\theta}} C(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_n}$.
   2. Choose a *learning rate* η_n. This could be the same at all iterations ($\eta_n = \eta$), or it could be given by a learning schedule that typically describes some decreasing function that leads to smaller and smaller steps.
   3. Move in the direction of the negative gradient: $\boldsymbol{\theta}_{n+1} = \boldsymbol{\theta}_n - \eta_n \boldsymbol{\nabla} C_n$.
3. Continue for a fixed number of iterations, or until the gradient vector $\boldsymbol{\nabla} C_n$ is smaller than some chosen precision δ.

Gradient descent is a general optimization algorithm. However, there are several important issues that should be considered before using it.

Challenges for gradient descent

1. It requires the computation of partial derivatives of the cost function. This might be straight-forward for the linear regression method, see Eq. (43.15), but can be very difficult for other models. Numerical differentiation can be computationally costly. Instead, *automatic differentiation* has become an important tool and there are software libraries for different programming languages. See, e.g., JAX for Python, which is well worth exploring.
2. Most cost functions—in particular in many dimensions—correspond to very *complicated surfaces with several local minima*. In those cases, gradient descent will likely not find the global minimum.
3. Choosing a proper learning rate can be difficult. A learning rate that is too small leads to painfully slow convergence, while a learning rate that is too large can hinder convergence and cause the parameter updates to fluctuate around the minimum.
4. Standard gradient-descent has particular difficulties to navigate ravines and saddle points for which the gradient is large in some directions and small in others.

For the remainder of this chapter we will consider gradient descent methods for the minimization of data-dependent cost functions, for example representing a sum of squared residuals between model predictions and observed data such as Eq. (43.11). Note that we are interested in general models, not just restricted to linear ones, for which the computational cost for evaluating the cost function and its gradient could be significant (and scale with the number of data that enter the cost function). For situations where data is abundant there are variations of gradient descent that use only fractions of the data set for computation of the gradient.

44.2. Batch, stochastic and mini-batch gradient descent

The use of the full data set in the cost function for every parameter update would correspond to *batch gradient descent* (BGD). The gradient of the cost function then has the following generic form

$$\boldsymbol{\nabla} C_n \equiv \boldsymbol{\nabla}_{\boldsymbol{\theta}} C(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_n} = \sum_{i=1}^{N_d} \boldsymbol{\nabla}_{\boldsymbol{\theta}} C^{(i)}(\boldsymbol{\theta})\Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_n}, \tag{44.3}$$

where the sum runs over the data set (of length N_d) and $C^{(i)}(\boldsymbol{\theta})$ is the cost function evaluated for a single data instance $\textbf{data}_i = (\boldsymbol{x}_i, \boldsymbol{y}_i)$.

A typical Python implementation of batch gradient descent would look something like the following:

```
# Pseudo-code for batch gradient descent
for i in range(N_epochs):
  params_gradient = evaluate_gradient(cost_function, data, params)
  params = params - learning_rate * params_gradient
```

for a fixed number of epochs `N_epochs` and a function `evaluate_gradient` that returns the gradient vector of the cost function (that depends on the data `data` and the model parameters `params`) with respect to the parameters. The update step depends on the `learning_rate` hyperparameter.

Depending on the size of the data set, batch gradient descent can be rather inefficient since all data must be processed to perform a single parameter update. Instead, one often employs *stochastic gradient descent* (SGD) for which parameter updates are performed for every data instance $\textbf{data}_i = (\boldsymbol{x}_i, \boldsymbol{y}_i)$ one at a time. Note that the total number of iterations for SGD is $N_{\text{epochs}} \times N_d$.

SGD performs frequent and fast updates of the parameters. The updates are often characterized by a high variance that can cause the cost function to fluctuate heavily during the iterations. This ultimately complicates convergence to the exact minimum, as SGD will keep overshooting. However, employing a learning rate that slowly decreases with iterations can make SGD more efficient. A typical Python implementation would look something like the following:

```
# Pseudo-code for stochastic gradient descent
for i in range(N_epochs):
  np.random.shuffle(data)
  learning_rate = decreasing_learning_rate(i)
  for data_instance in data:
    params_gradient = evaluate_gradient(cost_function, data_instance, params)
    params = params - learning_rate * params_gradient
```

where you should note the loop over all data instances for each epoch. To improve convergence and generalization to other data it is important that we shuffle the training data at every epoch.

Finally, *mini-batch gradient descent* (MBGD) combines the best of the previous algorithms and performs an update for every mini-batch of data

$$\boldsymbol{\theta} \mapsto \boldsymbol{\theta} - \eta \boldsymbol{\nabla} C_{iN_{\text{mb}}:(i+1)N_{\text{mb}}}(\boldsymbol{\theta}), \tag{44.4}$$

where N_{mb} is the mini-batch size. This way one can make use of highly optimized matrix operations for the reasonably sized mini-batch evaluations, and one usually finds much reduced variance of the parameter updates which can lead to more stable convergence. Mini-batch gradient descent is typically the algorithm of choice when training a neural network. The total number of parameter updates for MBGD is $N_{\mathrm{epochs}} \times N_d/N_{mb}$.

```
# Pseudo-code for mini-batch gradient descent
for i in range(N_epochs):
  np.random.shuffle(data)
  learning_rate = decreasing_learning_rate(i)
  for data_batch in get_batches(data, batch_size=50):
    params_gradient = evaluate_gradient(cost_function, data_batch, params)
    params = params - learning_rate * params_gradient
```

Be aware that the terms stochastic or batch gradient descent can both be used to denote mini-batch gradient descent.

44.3. Adaptive gradient descent algorithms

As outlined above, there are several convergence challenges for the standard gradient-descent methods. These are in general connected with the difficulty of navigating complicated cost function surfaces using only pointwise information on the gradient. The use of the history of past updates can help to adapt the learning schedule and has been shown to significantly increase the efficiency of gradient descent. Two ideas are central. With *momentum*, a fraction γ of the past update vector is added to the current one, $\boldsymbol{v}_n = \gamma \boldsymbol{v}_{n-1} + \eta \boldsymbol{\nabla} C_n$ and $\boldsymbol{\theta}_{n+1} = \boldsymbol{\theta}_n - \boldsymbol{v}_n$, which dampens oscillations and speeds up progress along directions with a persistent gradient. With *adaptive learning rates*, the step size for each parameter is adjusted based on the history of past (squared) gradients.

Some examples of commonly employed, adaptive methods are Adagrad [DHS11], Adadelta [Zeiler12], RMSprop, and Adam [KingmaBa14].

Adagrad

Adagrad [DHS11] is a simple algorithm that uses the history of past updates to adapt the learning rate to the different parameters. Let us introduce a shorthand notation $j_{n,i}$ for the partial derivative of the cost function with respect to parameter θ_i at the position $\boldsymbol{\theta}_n$ corresponding to iteration n

$$j_{n,i} \equiv \left. \frac{\partial C}{\partial \theta_i} \right|_{\boldsymbol{\theta}=\boldsymbol{\theta}_n}. \tag{44.5}$$

We also introduce a diagonal matrix $J_n^2 \in \mathbb{R}^{N_p \times N_p}$ where the ith diagonal element

$$J_{n,ii}^2 \equiv \sum_{k=0}^{n} j_{k,i}^2 \tag{44.6}$$

is the sum of the squares of the gradients with respect to θ_i up to iteration n. In its update rule, at iteration n, Adagrad modifies the general learning rate η_n for every parameter θ_i based on the past gradients that have been computed as described by $J_{n,ii}^2$. The parameter θ_i at the next iteration $n+1$ is then given by

$$\theta_{n+1,i} = \theta_{n,i} - \frac{\eta}{\sqrt{J^2_{n,ii} + \varepsilon}} j_{n,i}, \tag{44.7}$$

with ε a small, positive number to avoid division by zero.

Using Eq. (44.7) the learning schedule does not have to be tuned by the user as it is adapted by the history of past gradients. This is the main advantage of this algorithm. At the same time, its main weakness is the accumulation of the squared gradients in the denominator which implies that the learning rate eventually becomes infinitesimally small, at which point the convergence halts.

RMSprop

RMSprop is an extension of Adagrad that seeks to reduce its aggressive, monotonically decreasing learning rate. Instead of storing the sum of past squared gradients, the denominator is defined as a decaying average of past squared gradients. Labeling the decaying average at iteration n by $\bar{J}^2{}_{n,ii}$, we introduce a decay variable γ and define

$$\bar{J}^2_{n,ii} = \gamma \bar{J}^2_{n-1,ii} + (1-\gamma) j^2_{n,i}. \tag{44.8}$$

The inventor of RMSprop, Geoff Hinton, suggests setting the decay variable $\gamma = 0.9$, while a good default value for the learning rate η is 0.001. The update rule in RMSprop is then

$$\theta_{n+1,i} = \theta_{n,i} - \frac{\eta}{\sqrt{\bar{J}^2_{n,ii} + \varepsilon}} j_{n,i}. \tag{44.9}$$

Adam

Adaptive Moment Estimation (Adam) [KingmaBa14] also computes adaptive learning rates for each parameter. However, in addition to storing an exponentially decaying average of past squared gradients like RMSprop, Adam also keeps an exponentially decaying average of past gradients similar to a momentum. Describing this latter quantity by the diagonal matrix $\bar{M}_n$ (at iteration n) we have

$$\begin{aligned} \bar{M}_{n,ii} &= \gamma_1 \bar{M}_{n-1,ii} + (1-\gamma_1) j_{n,i}, \\ \bar{J}^2_{n,ii} &= \gamma_2 \bar{J}^2_{n-1,ii} + (1-\gamma_2) j^2_{n,i}. \end{aligned} \tag{44.10}$$

Both of these quantities are initialized with zero vectors ($\bar{M}_{0,ii} = 0$ and $\bar{J}^2_{0,ii} = 0$ for all i) which tends to introduce a bias for early time steps. This bias can be large if γ_1 and γ_2 are close to one. It was therefore proposed to use bias-corrected first and second moment estimates

$$\hat{M}_{n,ii} = \frac{\bar{M}_{n,ii}}{1-(\gamma_1)^n},$$
$$\hat{J}^2_{n,ii} = \frac{\bar{J}^2_{n,ii}}{1-(\gamma_2)^n}. \tag{44.11}$$

The update rule then becomes

$$\theta_{n+1,i} = \theta_{n,i} - \frac{\eta}{\sqrt{\hat{J}^2_{n,ii}+\varepsilon}}\hat{M}_{n,ii}. \tag{44.12}$$

Default, recommended values are $\gamma_1 = 0.9$, $\gamma_2 = 0.999$, and $\varepsilon = 10^{-8}$.

Adam has become the method of choice in many machine-learning implementations. An explanation and comparison of different gradient-descent algorithms is presented in a blog post by Sebastian Ruder.

45. Overview of Appendix C: Getting started

Appendix C has various materials to set you up for Learning from Data. You may have the option to run the Jupyter notebooks on a cloud server, but eventually you will likely want to set up an environment on your own machine.

- See Setting up for interactive use of this Jupyter Book for instructions how to use git for cloning the repository with all the book material and for guides to setting up a Python environment on your computer.
- If Python and/or Jupyter notebooks are new to you (or if you need a refresher), you should first see Jupyter notebooks and Python and then move to the execises and demos in Guides on Jupyter notebooks and Python.

46. Setting up for interactive use of this Jupyter Book

The following guides are available to help you get set up:

- Cloning the git repository of this Jupyter Book for local, interactive use of learning material, see Using git for cloning the book repository.
- Recommended installation of a Python environment, see Setting up your Python environment.

46.1. Using git for cloning the book repository

Prerequisites: Install git

If you do not have git installed, download and install it from the official site before proceeding:

- **All platforms:** https://git-scm.com/downloads
- **macOS (alternative):** git is included with the Xcode Command Line Tools. Run `xcode-select --install` in a terminal to install.

- **Linux:** Install via your package manager, e.g. `sudo apt install git` (Debian/Ubuntu) or `sudo dnf install git` (Fedora).

Open a terminal and verify that git is available by running:

```
git --version
```

Cloning the repository

Open a terminal, navigate to the directory where you want to store the book material, and run:

```
git clone https://github.com/NuclearTalent/LFD_for_Physicists.git
```

This will create a folder called `LFD_for_Physicists` in your current directory containing the repository with all files.

Move into the repository folder:

```
cd LFD_for_Physicists
```

The book material, including demo and exercise notebooks, is in the `content` subdirectory organized by parts and chapters in the book. You will need a working python environment to run the notebooks interactively on your local computer. See Setting up your Python environment.

Keeping your local copy up to date

As the book material is updated, you can pull the latest changes by running the following command from inside the repository folder:

```
git pull
```

Recommended git guides

There are plenty of tutorials and guides how to use git. Two recommended ones are:

- The Git Handbook (a GitHub Guide),
- Pro Git (an extensive online book).

46.2. Setting up your Python environment

Last revised: 22-Apr-2026

Overview

To run the Jupyter notebooks you will need a Python installation together with a number of scientific computing libraries (NumPy, SciPy, Matplotlib, and others). We manage these using **conda**, a package and environment manager that works on Windows, macOS, and Linux.

We recommend installing conda via **Miniforge**, a minimal, community-maintained distribution that is free to use without licensing restrictions and comes pre-configured with the conda-forge package channel. This replaces Anaconda's defaults channel, which includes licensing terms that are problematic for many users.

---

1. Installing Miniforge3 (fresh installation)

If you do not have conda installed, follow the instructions for your operating system below.

macOS

Open a terminal and run one of the following commands depending on your hardware:

Apple Silicon (M1/M2/M3 — most Macs from 2020 onwards):

```
curl -L https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-MacOSX-
arm64.sh -o Miniforge3.sh
bash Miniforge3.sh
```

Intel Mac:

```
curl -L https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-MacOSX-
x86_64.sh -o Miniforge3.sh
bash Miniforge3.sh
```

Follow the prompts, accept the license, and allow the installer to initialise conda in your shell. When the installation is complete, close and reopen your terminal.

Linux

Open a terminal and run:

```
curl -L https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-
x86_64.sh -o Miniforge3.sh
bash Miniforge3.sh
```

Follow the prompts as for macOS above.

Windows

Download the Windows installer from the Miniforge Latest Release page, selecting the Windows x86_64 options. This will download `Miniforge3-Windows-x86_64.exe`.

Run the installer and follow the graphical prompts. We recommend:

- Installing for the current user only (does not require administrator privileges)
- Ideally you will accept the default installation path. This may not be possible if the path contains special characters (including spaces and accents). You should get a warning if this is the case, which will require setting up another installation path.
- You have the option to **"Add Miniforge to my PATH environment variable"** if you intend to use conda from a standard Command Prompt or PowerShell window. However, this has been found to cause problems for many users and the official recommendation is to not do this. Instead, you will use the **Miniforge Prompt** that is added to your Start menu.

Verify the installation

Once installed, open the **Miniforge Prompt** (Windows) or a new terminal (macOS/Linux) and verify the installation:

```
conda --version
```

You should see version 24.x or later. Miniforge3 should be configured with the `conda-forge` channel by default, in which case no further channel configuration is needed. However, to be sure, continue with the next step.

---

2. Updating an existing conda installation (even if just installed)

Follow these steps to ensure your conda installation is compatible with current best practices.

Step 1 — Update conda to version 23.10.0 or later

Conda 23.10.0 introduced `libmamba` as the default dependency solver, which is significantly faster than the old solver. Check your current version and update if needed:

```
conda --version
conda update -n base conda
```

Verify the version is 23.10.0 or higher before proceeding.

Step 2 — Switch from the `defaults` channel to `conda-forge`

The Anaconda `defaults` channel has licensing restrictions that are problematic for many academic users. You can verify your channel configuration with:

```
conda config --show channels
```

The output should list only `conda-forge`.

If the `defaults` channel is used in your installation, then you should replace it with `conda-forge`:

```
conda config --remove channels defaults
conda config --add channels conda-forge
conda config --set channel_priority strict
```

The `strict` channel priority setting is strongly recommended when using `conda-forge` exclusively — it prevents packages from different channels from being mixed, which can cause subtle dependency conflicts.

---

3. Creating the course environment

The Python packages needed for this course are specified in the `environment.yml` file in the repository. Navigate to the `LFD_for_Physicists` directory you cloned from GitHub (see cloning instructions) and create the environment:

```
conda env create -f binder/environment.yml
```

The environment creation will take a few minutes. When it completes successfully you will see a message with instructions for activating the environment.

Activating and deactivating the environment

```
conda activate 2026-env-lfd       # activate
conda deactivate              # deactivate when done
```

On Windows, run these commands in the **Miniforge Prompt**.

---

4. Launching JupyterLab

We recommend **JupyterLab** as the interface for working with the course notebooks. Once the environment is activated, launch it with:

```
cd content
jupyter lab
```

This will open JupyterLab in your browser. Navigate to the notebook you want to open using the file browser on the left.

> **Note**
>
> If you prefer the classic single-document notebook interface, Jupyter Notebook v7 is also supported. Start it with `jupyter notebook`. The old classic Notebook interface (nbclassic) is no longer maintained and should not be used.

Verifying the correct kernel

When a notebook is open, check that the kernel shown in the upper right corner of JupyterLab reads `Python [conda env:2026-env-lfd]`. If it shows a different kernel, click on it and select the correct environment from the list.

---

5. Updating the course environment

The `environment.yml` file is updated from time to time as new packages are added. After pulling the latest changes from the repository, update your environment with:

```
conda deactivate
conda env update -f binder/environment.yml --prune
conda activate 2026-env-lfd
```

The `--prune` flag removes packages that are no longer listed in the environment file, keeping the environment clean.

47. Jupyter notebooks and Python

The exercises and demos in this book correspond to Jupyter notebooks that you can run yourself (rather than just looking at the static version shown in the book). You are highly encouraged to download the source files, as described in Using git for cloning the book repository, such that you can perform the exercises interactively.

A Jupyter notebook is a powerful tool for interactively developing and presenting data and computational science projects. Jupyter notebooks integrate markdown text notes, mathematical equations, interactive code cells and their output into a single document. They can be displayed on a web browser running on a computer, on a tablet (e.g., IPad), or even on your smartphone.

This integrated interface promotes fast, iterative development since each output of your code will be displayed right away. That's why notebooks have become very popular in data science and for explorative computational projects.

For serious code development projects, however, the Jupyter notebook format does have some serious drawbacks, such as lack of version control, and should be avoided.

47.1. What you should know about Python and using Jupyter notebooks

1. Know how to switch between `Code` and `Markdown` cells and how to run them (`shift-Return` or `Run` button)
2. Basics of Python evaluation and printing

    - string concatenation
    - exponentiation with `**`
    - use of fstrings
3. Importing numpy and basic functions (sqrt, exp, sin, . . .)
    - numpy arrays using `arange(min, max, step)`
4. Getting help via Google (or alternative): StackOverflow and manuals
5. Defining functions in Python
    - `def my_function(x):` ⟵ semicolon and then indented lines
    - position vs. keyword argument and defaults
    - `shift + tab + tab` to reveal definitions
6. Plotting with Matplotlib
    - Standard sequence for plot: data ⟶ make figure ⟶ add subplots ⟶ make plot
    - dressing up and saving a plot
    - names for the figure and axis objects are our choice
7. Numpy linear algebra
    - defining a matrix and finding its shape or particular elements
    - matrix (or vector) operations: multiplying them with @, transpose, trace, inverse, eigensolution

47.2. Good online cheat-sheets:

- Python Crash Course
- datacamp cheat-sheets
- Python debugging flowchart

48. Guides on Jupyter notebooks and Python

The following exercises and demos are available in this section:

- 📥 Exercise: Jupyter Notebooks and Python
- 📥 Exercise: Python lists and iterations
- 📥 Exercise: Linear algebra operations with NumPy
- 📥 Demonstration: Reading Data and fitting
- 📥 Demo: Making a simple widget-based UI

48.1. 📥 Exercise: Jupyter Notebooks and Python

This exercise offers a tour of the use of Python in Jupyter notebooks to get started doing data analysis. You'll learn more features and details as you proceed.

The Jupyter notebook Help menu. You can find valuable documentation under the Jupyter notebook Help menu. The "User Interface Tour" and "Keyboard Shortcuts" are useful places to start, but there are also many other links to documentation there.

Code and Markdown cells

The notebook consists of *cells*, of which two types are relevant for us:

- Markdown cells: These have headings, text, and mathematical formulas in LaTeX using a simple form of HTML called markdown.
- Code cells: These have Python code (or other languages, but we'll stick to Python).

Either type of cell can be selected with your cursor and will be highlighted in color when active. You evaluate an active cell with shift-return (as with Mathematica) or by pressing `Run` on the toolbar. Some notes:

- When a new cell is inserted, by default it is a Code cell and will have `[]:` in front. You can type Python expressions or entire programs in a cell. How you break up code between cells is your choice and you can always put Markdown cells in between. When you evaluate a cell it gets the evaluation number, e.g., `[5]:`.
- The output of an evaluated cell will be presented in an **Output cell** that is also preceeded by, e.g., `[5]:` where `5` is the evaluation number. This cell cannot be edited. It will be updated if the cell is evaluated again.
- On the menu bar is a pulldown menu that lets you change back and forth between Code and Markdown cells. Once you evaluate a Markdown cell, it gets formatted (and has a blue border). To edit the Markdown cell, double click in it.

Try double-clicking on this cell and then shift-return. You will see that a bullet list is created just with an asterisk and a space at the beginning of lines (without the space you get *italics* and with two asterisks you get **bold**).

Double click on the title header at the start of this exercise Notebook and you'll see it starts with a single #. Headings of subsections are made using ## or ###.

The Markdown language See this Markdown cheatsheet for a quick tour of the Markdown language (including how to add links!).

Now try turning the next (empty) cell to a Markdown cell and type: `Einstein says $E=mc^2$` **and then evaluate it.** This is LaTeX ! (If you forget to convert to Markdown and get `SyntaxError: invalid syntax`, just select the cell and convert to Markdown with the menu.)

The Jupyter Notebook menu and shortcuts

The menus enable you to rename your notebook file (always ending in `.ipynb`) or `Save and Checkpoint` to save the changes to your notebook. You can insert and delete cells (use the up and down arrows in the toolbar to easily move cells). You will often use the `Kernel` menu to `Restart` the notebook (and possibly clear output). The buttons below the menu allow quick access to several useful commands.

The help menu gives you direct access to important documentation. You'll be able to learn about all the Jupyter Notebook shortcuts to speed up your workflow. You also get convenient links to the Notebook and Markdown help pages, and to the documentation of some of the most important Python libraries including `Numpy`, `Scipy`, `Matplotlib`, and `Pandas`!

As you get more proficient working with notebooks, you will certainly start using the shortcut keys from the command mode of cells. A cell that is marked in blue implies that you are in command mode. You can start editing the cell by hitting `Enter` (or by clicking inside it). You can exit from edit mode into command mode by hitting `Esc`. A list of shortcut keys can be seen when opening the command palette by clicking on the keyboard button.

Useful Notebook shortcuts. In addition to the `Esc` and `Enter` keys, the use of `A` and `B` to insert new cells *above* or *below* the current one, or the use of `Y` and `M` to switch between *code* and *markdown* cell types, are some particularly useful shortcuts.

Python expressions and strings

We can use the Jupyter notebook as a super calculator much like Mathematica and Matlab. **Try some basic operations, modifying and evaluating the following cells, noting that exponentiation is with `**` and not `^`.**

```
 1 + 1  # Everything after a number sign / pound sign / hashtag)
        # in a code cell is a comment
```

```
3.2 * 4.713
```

Note that if we want a floating point number (which will be the same as a `double` in C++), we *always* include a decimal point (even when we don't have to) while a number without a decimal point is an integer.

```
3.**2
```

We can define integer, floating point, and string variables, perform operations on them, and print them. Note that we don't have to predefine the type of a variable and we can use underscores in the names (unlike Mathematica). **Evaluate the following cells and then try your own versions.**

```
x = 5.
print(x)
x   # If the last line of a cell returns a value, it is printed.
```

```
y = 3.*x**2 - 2.*x + 7.
print('y = ', y)           # Strings delimited by ' 's
```

There are several ways to print strings that includes variables from your code. We recommend using the relatively newly added `fstring`. See, e.g., this blog for examples.

```
print(f'y = {y:.0f}')      # Just a preview: more on format later
print(f'y = {y:.2f}')      # ...showing more decimals
```

The `fstring` will be used predominantly in this course, but you might also encounter older formatting syntax.

```
print('x = %.2f  y = %.2f' %(x,y))
print('x = {0:.2f}  y = {1:.2f}'.format(x, y))
print(f'x = {x:.2f}  y = {y:.2f}')
```

```
first_name = 'Emilia'     # Strings delimited by ' 's
last_name = 'Student'
full_name = first_name + ' ' + last_name  # you can concatenate strings
print(full_name)
# or
print(f'{first_name} {last_name}')
```

Mathematical functions with `numpy`

Ok, how about square roots and trigonometric functions and …

(Note: the next two cells will give error messages — keep reading to see how to fix them.)

```
sqrt(2)
```

```
sin(pi)
```

We need to `import` these functions through the NumPy library. There are other choices, but NumPy works with the arrays that we often will use.

Import libraries into their own namespace. *Never* use `from numpy import *` as it will eventually lead to conflicts in your namespace of functions. Instead you should import libraries into their own namespace through `import numpy as np`. Here `np` is just a abbreviation for numpy (which we can choose to be anything, but `np` is conventional).

```
import numpy as np
```

```
print(np.cos(0.))
```

Now functions and constants like `np.sqrt` and `np.pi` will work. Go back and fix the square root and sine.

Debugging aside …

Suppose you try to import and it fails

go ahead and evaluate the cell:

```
import numpie
```

When you get a `ModuleNotFoundError`, the first thing to check is whether you have misspelled the name. Try using Google, e.g., search for “python numpie”. In this case (and in most others), Google will suggest the correct name (here it is numpy). If the name does exist, check whether it sounds like the package you wanted.

If you have the correct spelling, check whether you have installed the relevant package. If you installed Python with `conda`, then use `conda list`, e.g., `conda list numpy` in a Terminal window (on a Mac or Linux box) or in an

Anaconda Prompt window (on a Windows PC).

`numpy` arrays

The NumPy library is extremely useful and its linear algebra features will be covered in more depth in a separate exercise Notebook. However, here we already introduce `numpy` *arrays*, which will be featured extensively. They are similar to Python `list`s but much more powerful for scientific computations. Here, as an example, we construct them with `np.arange(min, max, step)` to get an array from `min` to `max` in steps of `step`. Examples:

```
t_pts = np.arange(0., 10., .1)
t_pts
```

If we give a `numpy` array to a function, each term in the list is evaluated with that function:

```
x = np.arange(1., 5., 1.)
print(x)
print(x**2)
print(np.sqrt(x))
```

We can pick out elements of the list. Why does the last one fail?

```
print(x[0])
print(x[3])
print(x[4])
```

Zero-based numbering. Note the use of zero-based numbering in Python.

Getting help

You will often need help identifying the appropriate Python (or NumPy or SciPy or …) command or you will need an example of how to do something or you may get an error message you can't figure out. In all of these cases, Google (or equivalent) is your friend. Always include "python" in the search string (or "numpy" or "matplotlib" or …) to avoid getting results for a different language. You will usually get an online manual as one of the first responses if you ask about a function; these usually have examples if you scroll down. Otherwise, responses to *Stack Overflow* queries are your best bet to find a useful answer.

Functions

There are many Python language features that we will use eventually. We will definitely need to be able to create functions. Note that these are usually referred to as *methods* in Python to stress that they don't need to correspond to mathematical operations that return some output that is computed given some input. Sometimes a *method* just performs some tasks such as creating and saving a figure. We will use both names (functions and methods) in these notes.

When defining functions (methods), we note the role of *indentation* in Python as opposed to {}s or ()s used in other programming languages. We'll always indent four spaces (never tabs!). We know a function definition is complete

when the indentation stops. The Jupyter Notebook also works as some sort of IDE (integrated development environment) as it will assist you with proper indentation and formating.

Shift-Tab help text. To read the manual (docstring) of a Python function, put your cursor on the function name and hit `Shift+Tab` or `Shift+Tab+Tab`. **Go back and try it on `np.arange`.**

```
# Use "def" to create new functions.
#  Note the colon and indentation (4 spaces).
def my_function(x):
    """This function squares the input.  Always include a brief description
        at the top between three starting and three ending quotes.  We will
        talk more about proper documentation later.
        Try shift+Tab+Tab after you have evaluated this function.
    """
    return x**2

print(my_function(5.))

# We can pass an array to the function and it is evaluated term-by-term.
x_pts = np.arange(1.,10.,1.)
print(my_function(x_pts))
```

```
# Two variables, with a default for the second
def add(x, y=4.):
    """Add two numbers."""
    print("x is {} and y is {}".format(x, y))
    return x + y  # Return values with a return statement

# Calling functions with parameters
print('The sum is ', add(5, 6))  # => prints out "x is 5 and y is 6" and returns 11

# Another way to call functions is with keyword arguments
add(y=6, x=5)  # Keyword arguments can arrive in any order.
```

How do you explain the following result?

```
add(2)
```

Debugging aside …

There are two bugs in the following function. **Note the line where an error is first reported and fix the bugs sequentially (so you see the different error messages).**

```
def hello_function()
    msg = "hello, world!"
    print(msg)
     return msg
```

Plotting with Matplotlib

Matplotlib is the plotting library we'll use. We'll follow convention and abbreviate the module we need as `plt`.

The `%matplotlib inline` is a Jupyter Notebook built-in *magic* command that produces inline plots, i.e. the plot is shown in the Output cell.

```
%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt
```

Procedure we'll use to make the skeleton plot:

1. Generate some data to plot in the form of arrays.
2. Create a figure;
3. add one or more subplots;
4. make a plot and display it.

```
t_pts = np.arange(0., 10., .1)     # step 1.
x_pts = np.sin(t_pts)  # More often this would be from a function
                       #  *we* write.

my_fig = plt.figure()              # step 2.
my_ax = my_fig.add_subplot(1,1,1)  # step 3: rows=1, cols=1, 1st subplot
my_ax.plot(t_pts, x_pts)           # step 4: plot x vs. t
```

NOTE: When making just a single plot, you will more usually see steps 2 to 4 compressed into `plt.plot(t_pts, np.sin(t_pts))`. Don't do this. It saves a couple of lines but restricts your ability to easily extend the plot, which is what we want to make easy.

We can always go back and dress up the plot:

```
my_fig = plt.figure()
my_ax = my_fig.add_subplot(1,1,1)  # nrows=1, ncols=1, first plot
my_ax.plot(t_pts, x_pts, color='blue', linestyle='--', label='sine')

my_ax.set_xlabel('t')
my_ax.set_ylabel(r'$\sin(t)$')  # here $s to get LaTeX and r to render it
my_ax.set_title('Sine wave')

# here we'll put the function in the call to plot!
my_ax.plot(t_pts, np.cos(t_pts), label='cosine')  # just label the plot

my_ax.legend();  # turn on legend
```

Now make two subplots:

```
y_pts = np.exp(t_pts)         # another function for a separate plot

fig = plt.figure(figsize=(10,5))  # allow more room for two subplots

# call the first axis ax1
ax1 = fig.add_subplot(1,2,1)  # one row, two columns, first plot
ax1.plot(t_pts, x_pts, color='blue', linestyle='--', label='sine')
ax1.plot(t_pts, np.cos(t_pts), label='cosine')  # just label the plot
ax1.legend()

ax2 = fig.add_subplot(1,2,2)  # one row, two columns, second plot
ax2.plot(t_pts, np.exp(t_pts), label='exponential')
ax2.legend();
```

Saving a figure

Saving a figure to disk is as simple as calling `savefig` with the name of the file (or a file object). The available image formats depend on the graphics backend you use.

Let us save the figure (named ‘fig’) from the previous cell

```
fig.savefig("sine_and_exp.png")
# and a transparent version:
fig.savefig("sine_and_exp_transparent.png", transparent=True)
```

Further examples with matplotlib. The matplotlib gallery is a good resource for learning by working examples.

Advanced feature: Widgets for graphical exploration

A widget is an object such as a slider or a check box or a pulldown menu. We can use them to make it easy to explore different parameter values in a problem we’re solving, which is useful for building intuition. They act on the argument of a function.

The set of widgets we’ll use here (there are others!) is from `ipywidgets`; we’ll conventionally import the module as `import ipywidgets as widgets` and we’ll also often use `display` from `Ipython.display`.

```
import numpy as np
import matplotlib.pyplot as plt

import ipywidgets as widgets
from IPython.display import display

%matplotlib inline
```

The simplest form is to use `interact`, which we pass a function name and the variables with ranges. By default this makes a *slider*, which takes on integer or floating point values depending on whether you put decimal points in the range. **Try it! Then modify the function and try again.**

```
# We can do this to any function
def test_f(x=5.):
    """Test function that prints the passed value and its square.
       Note that there is no return value in this case."""
    print ('x = ', x, ' and  x^2 = ', x**2)

widgets.interact(test_f, x=(0.,10.));
```

```
# Explicit declaration of the widget (here FloatSlider) and details
def test_f(x=5.):
    """Test function that prints the passed value and its square.
       Note that there is no return value in this case."""
    print ('x = ', x, ' and  x^2 = ', x**2)

widgets.interact(test_f,
                 x = widgets.FloatSlider(min=-10,max=30,step=1,value=10));
```

Here's an example with some bells and whistles for a plot. **Try making changes!**

```
def plot_it(freq=1., color='blue', lw=2, grid=True, xlabel='x',
            function='sin'):
    """ Make a simple plot of a trig function but allow the plot style
        to be changed as well as the function and frequency."""
    t = np.linspace(-1., +1., 1000)  # linspace(min, max, total #)

    fig = plt.figure(figsize=(8,6))
    ax = fig.add_subplot(1,1,1)

    if function=='sin':
        ax.plot(t, np.sin(2*np.pi*freq*t), lw=lw, color=color)
    elif function=='cos':
        ax.plot(t, np.cos(2*np.pi*freq*t), lw=lw, color=color)
    elif function=='tan':
        ax.plot(t, np.tan(2*np.pi*freq*t), lw=lw, color=color)

    ax.grid(grid)
    ax.set_xlabel(xlabel)

widgets.interact(plot_it,
                 freq=(0.1, 2.), color=['blue', 'red', 'green'],
                 lw=(1, 10), xlabel=['x', 't', 'dog'],
                 function=['sin', 'cos', 'tan'])
```

48.2. 📥 Exercise: Python lists and iterations

In this notebook we explore the use of lists and iterations.

Speed comparisons

We start, however, with a word of caution. The use of lists and for-loops can be painfully slow. Here we show some examples. We use the *magic command* `%timeit` to perform the timing measurement. It runs the command(s)

multiple times and presents the mean execution time and its standard deviation. This provides a more reliable metric than the output from a single time measurement.

While `%timeit` measures the command(s) on the same line, the double `%%` implies that the whole cell will be measured.

Creating a list, range, numpy array

```
n = 1_000 # Note that we can use the '_' in integer representations
```

Creating a list via iteration is notoriously slow.

```
%%timeit
x_list = []
for i in range(n):
    x_list.append(i)
```

We must be careful that the variable `x_list` is only used inside the `timeit` call and is not retained.

```
x_list
```

We create a new copy without timing.

```
x_list = []
for i in range(n):
    x_list.append(i)
```

The creation of a numpy array is much faster.

```
import numpy as np
```

```
%timeit np.arange(n)    # Timing only of this line
x_arange = np.arange(n) # Repeating the command for later use of the variable
```

Note that the built-in function `range` is rather fast. This is because it is an `iterator` rather than a list or array.

```
%timeit range(n)
x_range = range(n)
```

```
type(x_list)
```

```
type(x_range)
```

```
type(x_arange)
```

Iteration versus array operation

Let's iterate over the list and sum the squares of the elements.

```
%%timeit
sum_of_squares = 0
for x in x_list:
    sum_of_squares += x**2
```

Using a `range` rather than a `list` does not improve the speed.

```
%%timeit
sum_of_squares = 0
for x in x_range:
    sum_of_squares += x**2
```

Whereas the vectorized evaluation of an `np.array` operation is superior.

```
%%timeit
sum_of_squares = np.sum(x_arange**2)
```

As will be shown in the following, lists are still very useful objects in python, and iteration can be used for many tasks.

Iterating through a list of parameters to draw multiple lines on a plot

Suppose we have a function of x that also depends on a parameter (call it r). We want to plot the function vs. x for multiple values of r, either on the same plot or on separate plots. We can do this with a lot of cutting-and-pasting, but how can we do it based on a list of r values, which we can easily modify?

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn; seaborn.set() # for plot formatting
```

```
def sine_map(r, x):
    """Sine map function: f_r(x) = r sin(pi x)
    """
    return r * np.sin(np.pi * x)
```

Suppose the r values initially of interest are 0.3, 0.5, 0.8, and 0.9. First the multiple copy approach:

```
x_pts = np.linspace(0,1, num=101, endpoint=True)

fig = plt.figure()
ax = fig.add_subplot(1,1,1)
ax.set_aspect(1)

ax.plot(x_pts, x_pts, color='black')  # black y=x line

ax.plot(x_pts, sine_map(0.3, x_pts), label='$r = 0.3$')
ax.plot(x_pts, sine_map(0.5, x_pts), label='$r = 0.5$')
ax.plot(x_pts, sine_map(0.8, x_pts), label='$r = 0.8$')
ax.plot(x_pts, sine_map(0.9, x_pts), label='$r = 0.9$')

ax.legend()
ax.set_xlabel(r'$x$')
ax.set_ylabel(r'$f(x)$')
ax.set_title('sine map')

fig.tight_layout()
```

This certainly works, but making changes is awkward and prone to error because we have to find where to change (or add another) r but we might not remember to change it correctly everywhere.

With minor changes we have a much better implementation (try modifying the list of r values):

```
r_list = [0.3, 0.5, 0.8, 0.9]    # this could also be a numpy array

x_pts = np.linspace(0,1, num=101, endpoint=True)

fig = plt.figure()
ax = fig.add_subplot(1,1,1)
ax.set_aspect(1)

ax.plot(x_pts, x_pts, color='black')  # black y=x line

# Step through the list.  r is a dummy variable.
#  Note the use of an f-string and LaTeX by putting rf in front of the label.
for r in r_list:
    ax.plot(x_pts, sine_map(r, x_pts), label=rf'$r = {r:.1f}$')

ax.legend()
ax.set_xlabel(r'$x$')
ax.set_ylabel(r'$f(x)$')
ax.set_title('sine map')

fig.tight_layout()
```

Now suppose we want each the different r values to be plotted on separate graphs? We could make multiple copies of the single plot. Instead, lets make a function to do any single plot and call it for each r in the list.

```
r_list = [0.3, 0.5, 0.8, 0.9]    # this could also be a numpy array

def plot_sine_map(r):
    x_pts = np.linspace(0,1, num=101, endpoint=True)

    fig = plt.figure()
    ax = fig.add_subplot(1,1,1)
    ax.set_aspect(1)

    ax.plot(x_pts, x_pts, color='black')  # black y=x line

#  Note the use of an f-string and LaTeX by putting rf in front of the label.
    ax.plot(x_pts, sine_map(r, x_pts), label=rf'$r = {r:.1f}$')

    ax.legend()
    ax.set_xlabel(r'$x$')
    ax.set_ylabel(r'$f(x)$')
    ax.set_title(rf'sine map for $r = {r:.1f}$')

    fig.tight_layout()


# Step through the list.  r is a dummy variable.
for r in r_list:
    plot_sine_map(r)
```

What if instead of distinct plots we wanted subplots of the same figure? Then create the figure and subplot axes outside of the function and have the function return the modified axis object.

```
r_list = [0.3, 0.5, 0.8, 0.9]    # this could also be a numpy array

def plot_sine_map(r, ax_passed):
    x_pts = np.linspace(0,1, num=101, endpoint=True)

    ax_passed.set_aspect(1)

    ax_passed.plot(x_pts, x_pts, color='black')  # black y=x line

#  Note the use of an f-string and LaTeX by putting rf in front of the label.
    ax_passed.plot(x_pts, sine_map(r, x_pts), label=rf'$r = {r:.1f}$')

    ax_passed.legend()
    ax_passed.set_xlabel(r'$x$')
    ax_passed.set_ylabel(r'$f(x)$')
    ax_passed.set_title(rf'sine map for $r = {r:.1f}$')

    return ax_passed

fig = plt.figure(figsize=(8, 8))

# Step through the list.  r is a dummy variable.
rows = 2
cols = 2
for index, r in enumerate(r_list):
    ax = fig.add_subplot(rows, cols, index+1)
    ax = plot_sine_map(r, ax)

fig.tight_layout()
```

Aside: List comprehensions

In the plotting examples we used for loops to iterate through parameters or plots because it is familiar to anyone who has done programming. In Python, however, it is often preferred to use a different construction called a list comprehension. Here is a quick comparison of using for loop and using a list comprehension, followed by some representative further examples of list comprehensions. You can find much more information and other examples in the online Python documentation and other sources. (The examples here are from [https://hackernoon.com/list-comprehension-in-python-8895a785550b].)

```
numbers = [1, 2, 3, 4]
squares = []

for n in numbers:
  squares.append(n**2)

print(squares)  # Output: [1, 4, 9, 16]
```

```
numbers = [1, 2, 3, 4]
squares = [n**2 for n in numbers]

print(squares)  # Output: [1, 4, 9, 16]
```

Connecting back to the speed measurements that we made arlier, the construction of lists using list comprehension can be quite fast.

```
%timeit [n_i for n_i in range(n)]
```

List comprehension is often used in *pythonic* one-liners! (not always easy to read)

```
# Find common numbers from two lists using list comprehension
list_a = [1, 2, 3, 4]
list_b = [2, 3, 4, 5]

common_num = [a for a in list_a for b in list_b if a == b]

print(common_num) # Output: [2, 3, 4]
```

```
# Return numbers from the list which are not equal as a tuple:
list_a = [1, 2, 3]
list_b = [2, 7]

different_num = [(a, b) for a in list_a for b in list_b if a != b]

print(different_num) # Output: [(1, 2), (1, 7), (2, 7), (3, 2), (3, 7)]
```

```python
# Iterate over strings
list_a = ["Hello", "World", "In", "Python"]

small_list_a = [str.lower() for str in list_a]

print(small_list_a) # Output: ['hello', 'world', 'in', 'python']
```

```python
# Making a list of lists
list_a = [1, 2, 3]

square_cube_list = [ [a**2, a**3] for a in list_a]

print(square_cube_list) # Output: [[1, 1], [4, 8], [9, 27]]
```

```python
# Using an if statement to make a list of unequal pairs of numbers
[(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
# Output: [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

48.3. 📥 Exercise: Linear algebra operations with NumPy

NumPy is the library that gives Python its ability to work with numerical data at speed. This exercise will demonstrate how to use NumPy for common tasks such as the creation and manipulation of arrays, linear algebra operations, and statistical operations. Some of the content is inspired by one (out of many) cheat sheets found on github.

The structure of this note:

1. N-dimensional arrays
2. Array shape manipulations
3. Linear algebra and numerical operations
4. Array manipulations routines (select and split)
5. Statistical operations

```python
import numpy as np
```

N dimensional arrays

What are arrays?

Arrays are a data structure for storing elements of the same type. Each item stored in an array is called an element. Each location of an element in an array has a numerical index which is used to identify the element. Arrays (and the indices) can be of more than one dimension.

1D vs 2D Array

1D array (i.e., single dimensional array) stores a list of variables of the same data type. It is possible to access each variable using the index.

2D array (i.e, multi-dimensional array) stores data in a format consisting of rows and columns. Each index will then be two numbers.

Arrays vs Lists

- Arrays use less memory than lists
- Arrays have significantly more functionality
- Arrays require data to be homogeneous; lists do not
- Arithmetic on arrays operates like matrix multiplication
- NumPy is used to work with arrays. The array object in NumPy is called `ndarray`.

Create Arrays

```
# Create 1 dimensional array (vector)
np.array([1,2,3]) # Create vector as a row
```

```
np.array([[1],[2],[3]]) #Create vector as a column. This is now a two-dimensional array.
```

```
np.array([(1,2,3),(4,5,6)]) # Two-dimensional array mith multiple rows and columns.
```

Zero-based indexing

Note that elements in `ndarrays` are accessed with zero-based indexing, meaning that the numbering starts with zero.

Row-major order

Note that elements in `ndarrays` are stored in memory using *row-major order*.

In computing, row-major order and column-major order are methods for storing multidimensional arrays in linear storage such as random access memory.

The difference between the orders lies in which elements of an array are contiguous in memory. In row-major order, the consecutive elements of a row reside next to each other, whereas the same holds true for consecutive elements of a column in column-major order.

Data layout is critical for correctly passing arrays between programs written in different programming languages. It is also important for performance when traversing an array because modern CPUs process sequential data more efficiently than nonsequential data.

Creating a Sparse Matrix

A sparse matrix is a matrix in which most of the elements are zero. Sparse matrix data formats only store nonzero elements and assume all other values will be zero, leading to significant computational savings.

Let' say, we want to create a NumPy array with two nonzero values, then converted it into a sparse matrix. If we view the sparse matrix, we can see that only the nonzero values are stored:

```
from scipy import sparse
matrix_large = np.array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                         [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
                         [3, 0, 0, 0, 0, 0, 0, 0, 0, 0]])
# Create compressed sparse row (CSR) matrix
matrix_large_sparse = sparse.csr_matrix(matrix_large)
print(matrix_large_sparse)
```

In the example above, (1, 1) and (2, 0) represent the indices of the non-zero values 1 and 3, respectively. For example, the element 1 is in the second row and second column (remember that numbering starts at zero!).

Create special `ndarray`

Here are some examples of `numpy` methods that can be used to create special arrays.

```
# Create  1d array of zeros, length 3
np.zeros(3)
```

```
# 2x3 array of zeros
np.zeros((2,3))
```

```
# Create array of ones, 3x4 (3 rows, 4 columns)
np.ones((3,4))
```

```
#  5x5 array of 0 with 1 on diagonal (Identity matrix)
np.eye(5)
```

```
# Array of 6 evenly divided values from 0 to 100
np.linspace(0,100,6)
```

```
# Array of values from 0 to less than 10 with step 3
np.arange(0,10,3)
```

```
# 2x3 array with all values 5
np.full((2,3),5)
```

```
# 2x3 array of random floats between 0–1
np.random.rand(2,3)
```

```
# 2x3 array of random floats between 0–100
np.random.rand(2,3)*100
```

```
# 2x3 array with random ints between 0–4
np.random.randint(5,size=(2,3))
```

Remember that useful `shift-Tab-Tab` shortcut in Jupyter notebooks to quickly find the docstring of a function. You can try it on any of the functions shown above.

You can also use the `np.info` function.

```
np.info(np.eye) # View documentation for np.eye
```

Array shape manipulations

Shape, size, length and data type

It will be valuable to check the shape and size of an array both for further calculations and simply as a sanity check after some operation.

NumPy arrays have an attribute called shape that returns a tuple with each index having the number of corresponding elements.

```
arr = np.array([(1,2,3),(4,5,6)])
arr.shape # Returns dimensions of arr (rows,columns)
```

In the example above, (2, 3) means that the array has 2 dimensions, and each dimension has 3 elements.

```
arr.size # Return number of elements in arr
```

```
len(arr) # Number of array dimension
```

```
arr.dtype # Return type of elements in arr
```

```
arr.dtype.name # Name of data type
```

```
arr_int = arr.astype(float) # Convert an array to a different type
arr_int.dtype
```

Reshape

It is important to know how to reshape the NumPy arrays so that our data meets the expectation of specific Python libraries. For example, Scikit- learn requires a one-dimensional array of output variables to be shaped like a two-dimensional array with one column and outcomes for each row.

Other algorithms might require input to be specified as a three-dimensional array. One example is the Long Short-Term Memory recurrent neural network in Keras that expects a three-dimensional input of samples, timesteps, and features.

`np.reshape()` allows us to restructure an array so that we maintain the same data but it is organized as a different number of rows and columns.

Note: The shape of the original and new matrix contains the same number of elements (i.e, same size).

```
# Reshaping
arr1 = np.arange(1, 11)  # numbers 1 to 10
print(f'arr1 = {arr1}\n')
print(f'The shape of array `arr1` is: {arr1.shape}\n') # Prints a tuple for the one dimension.

arr1_2d = arr1.reshape(2, 5)
print(f'Reshaping the array into:\n{arr1_2d}')
print(f'The shape is now: {arr1_2d.shape}\n')
```

The `reshape` method has a useful option of specifying `-1` as one of the shape elements. This will then automatically become the required number to maintain the array size.

```
arr1.reshape(2, 5)
arr1.reshape(-1, 5)  # same as above: arr1.reshape(2, 5)
arr1.reshape(2, -1)  # same as above: arr1.reshape(2, 5)
```

Transposing a matrix (rows <=> columns) or flattening into a one-dimensional array.

```
# Transpose
arr1_2d.T
```

```
# Flattening a Matrix
arr1_2d.flatten()
```

Note that the default of flatten is to use the row-major order. Can you find the argument to flatten with column-major instead?

```
# Resize arr1_2d to 3 rows, 4 columns
np.resize(arr1_2d, (3,4))
```

Array to/from List conversion

Arrays need to be declared whereas lists do not need declaration because they are a part of Python's syntax. This is the reason lists are more often used than arrays. But in case of performing some arithmetic function to our list, we should go with arrays instead.

It is possible to convert between lists and arrays (although list to array conversion requires the object to be of a single type).

```
arr = np.array([(1,2,3),(4,5,6)])
print(arr)
type(arr)
```

```
arr_to_list = arr.tolist() # Convert arr to a Python list
print(arr_to_list)
type(arr_to_list)
```

```
arr_from_list = np.array(arr_to_list) # Convert list to array
print(arr_from_list)
type(arr_from_list)
```

The following operation will not work with a list, but does work with an array.

```
arr_to_list/4
```

```
arr_from_list/4
```

Linear algebra and numerical operations

Let's explore some of the numerical operations that can be performed with matrices. We will often use methods from the `np.linalg` module. Note that these NumPy linear algebra functions rely on BLAS and LAPACK to provide efficient low level implementations of standard linear algebra algorithms.

Trace

The trace is the sum of all the diagonal elements of a square matrix.

```
arr = np.array([[2, 0, 0], [0, 2, 0], [0, 0, 2]])
np.trace(arr)
```

Inverse

```
matrix = np.array([[1, 2],
                   [3, 4]])
# Calculate inverse of matrix
np.linalg.inv(matrix)
```

Determinant

```
# Determinant
matrix = np.array([[1, 2, 3],
                   [2, 4, 6],
                   [3, 8, 9]])
# Return determinant of matrix
np.linalg.det(matrix)
```

Rank

```
# Rank of a matrix
matrix = np.array([[1, 1, 3],
                   [1, 2, 4],
                   [1, 3, 0]])
# Return matrix rank
np.linalg.matrix_rank(matrix)
```

Question: Why is the rank of the 3x3 matrix below not equal to 3?

```
# Rank of a matrix
matrix = np.array([[1, 7, 3],
                   [1, 9, 4],
                   [1, 1, 0]])
# Return matrix rank
np.linalg.matrix_rank(matrix)
```

Eigenvalues and eigenvectors

```
# Eigenvalues and Eigenvectors
matrix = np.array([[0, 1, 2],
                   [3, 4, 5],
                   [6, 7, 8]])
# Calculate eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(matrix)
print(f'Eigenvalues: {eigenvalues}')
print(f'Eigenvectors: {eigenvectors}')
```

Operating on matrices with special properties

> Note that there are methods that are targeted to work on matrices with certain properties, such as `np.linalg.eigh` for complex Hermitian (conjugate symmetric) or a real symmetric matrix. Be careful, however, as it can produce incorrect results if applied to an object that does not have the required property.

Scalar operations

When we add, subtract, multiply or divide a matrix by a number, this is called the scalar operation. During scalar operations, the scalar value is applied to each element in the array, therefore, the function returns a new matrix with the same number of rows and columns.

```
# Scalar Operations
new_arr = np.arange(1,10)

# Add 1 to each array element
np.add(new_arr,1)

np.subtract(arr,2) # Subtract 2 from each array element
np.multiply(arr,3) # Multiply each array element by 3
np.divide(arr,4) # Divide each array element by 4 (returns np.nan for division by zero)
np.power(arr,5) # Raise each array element to the 5th power
```

Matrix Operations

A matrix can only be added to (or subtracted from) another matrix if the two matrices have the same dimensions, that is, they must have the same number of rows and columns.

When multiplying matrices, we take rows of the first matrix and multiply them by the corresponding columns of the second matrix.

```
arr1 = np.array([[1,2],[3,4]])
arr2 = np.ones_like(arr1)*2
print('arr1 =\n',arr1)
print('arr2 =\n',arr2)
```

```
print('Elementwise add arr2 to arr1:\n',np.add(arr1,arr2))
print('Elementwise subtract arr2 from arr1:\n',np.subtract(arr1,arr2))
print('Elementwise multiply arr1 by arr2:\n',np.multiply(arr1,arr2))
print('Elementwise divide arr1 by arr2:\n',np.divide(arr1,arr2))
print('Elementwise raise arr1 to the power of arr2:\n',np.power(arr1,arr2))
print('Return True if the arrays have the same elements and shape:',np.array_equal(arr1,arr2))
```

Other math operations

```
np.sqrt(arr) # Square root of each element in the array
np.sin(arr) # Sine of each element in the array
np.log(arr) # Natural log of each element in the array
np.abs(arr) # Absolute value of each element in the array
np.ceil(arr) # Rounds up to the nearest int
np.floor(arr) # Rounds down to the nearest int
np.round(arr) # Rounds to the nearest int
```

Dot product versus elementwise multiplication

The dot product of two matrices can be computed with the `np.dot` function. Note that it is not the same as the arithmetic `*` operation that performs elementwise multiplication (Hadamard product).

```
A = np.array([[1,2],[3,4]])
print('A =\n',A)
print(r'The dot product A \cdot A:')
AA = np.dot(A,A)
print(AA)
print(r'Element-wise product A * A:')
print(A*A)
```

The dot product of a matrix by its inverse returns the identity matrix (with small floating point errors). **Verify that this is true.**

> **Further examples with numpy**
>
> The NumPy tutorial is a good resource.

Array Manipulation Routines

Adding/removing elements

The `np.append()` function is used to append values to the end of a given array.

```
# Adding/removing Elements
np.append ([0, 1, 2], [[3, 4, 5], [6, 7, 8]])
np.append([[0, 1, 2], [3, 4, 5]],[[6, 7, 8]], axis=0)
```

The axis along which values are appended. If the axis is not given, both array and values are flattened before use.

`np.insert()'` is used to insert the element before the given index of the array.

```
np.insert(arr,2,10) # Inserts 10 into arr before index 2
```

While `np.delete()` can be used to delete any row and column from the `ndarray`.

```
arr = np.arange(12).reshape(3, 4)
print('arr =\n',arr)
np.delete(arr,2,axis=0) # Deletes row (axis=0) on index 2 of arr
np.delete(arr,3,axis=1) # Deletes column (axis=1) on index 3 of arr
```

Sorting

The `np.sort()` function can be used to sort the list in both ascending and descending order.

```
# Sort
oned_arr = np.array([3,8,5,1])
np.sort(oned_arr)
arr = np.array([[5, 4, 6, 8],
                [1, 2, 4, 8],
                [1, 5, 2, 4]])
# sort each column of arr
np.sort(arr, axis=0)

# sort each row of X
np.sort(arr, axis=1)
```

Concatenate arrays

Concatenating means putting contents of two or more arrays in a single array. In NumPy, we join arrays by axes. We pass a sequence of arrays that we want to join to the `np.concatenate()` function, along with the axis. If the axis is not explicitly passed, it is taken as 0.

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
# Adds arr2 as rows to the end of arr1
np.concatenate((arr1, arr2), axis=0)
```

```
arr1 = np.array([[1, 2, 3],[4, 5, 6]])
arr2 = np.array([[7, 8, 9],[10, 11, 12]])
# Adds arr2 as columns to end of arr1
np.concatenate((arr1,arr2),axis=1)
```

Splitting arrays

We use `np.array_split()` for splitting arrays. We pass it the array we want to split and the number of splits.

```
# Splitting NumPy Arrays
# Splits arr into 4 sub-arrays
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 4)
print(newarr)
```

or `np.hsplit()` to split horizontally.

```
arr = np.array([1, 2, 3, 4, 5, 6])
# Splits arr horizontally on the 2nd index
newarr = np.hsplit(arr, 2)
print(newarr)
```

Select element(s)

NumPy offers a wide variety of methods for selecting and slicing elements or groups of elements in arrays.

```
user_name = np.array(['Katie','Bob','Scott','Liz','Sam'])
articles = np.array([100, 38, 91, 7, 25])
```

```
user_name[4] # Returns the element at index 4
```

```
articles[3] = 17 # Assigns array element on index 1 the value 4
```

```
user_name[0:3] # Returns the elements at indices 0,1,2
```

```
user_name[:2] # Returns the elements at indices 0,1
```

```
articles<50 # Returns an array with boolean values
```

```
articles[articles < 50] # Returns the element values
```

```
# Returns the user_name that read more than 50 articles but less than 100 articles
user_name[(articles < 100 ) & (articles >50)]
```

```
# Multi-dimensinal arrays
arr = np.arange(36).reshape(6,6)
print(arr)
```

Exercise: **Try to perform the following selection or assignments of elements**:

- Returns the 2D array element on index [2][5]
- Assigns array element on index [1][3] the value 10
- Returns rows 0,1,2
- Returns the elements on rows 0,1,2 at column 4
- Returns returns rows 0,1
- Returns the elements at index 1 on all rows
- Returns all elements that are divisible by 3.

Solutions are found below

```
arr[2,5] # Returns the 2D array element on index [2][5]
```

```
arr[1,3]=10 # Assigns array element on index [1][3] the value 10
print(arr)
```

```
arr[0:3] # Returns rows 0,1,2
```

```
arr[0:3,4] # Returns the elements on rows 0,1,2 at column 4
```

```
arr[:2] # Returns returns rows 0,1
```

```
arr[:,1] # Returns the elements at index 1 on all rows
```

```
arr[np.mod(arr,3)==0] # returns all elements that are divisible by 3.
```

Statistical Operations

Find the Maximum and Minimum Values

Often we want to know the maximum and minimum value in an array or subset of an array. This can be accomplished with the `np.max` and `np.min` methods. Using the axis parameter we can also apply the operation along a certain axis.

```
articles = np.array([[10, 23, 17],
                     [41, 54, 65],
                     [71, 18, 89]])
# Return maximum element
np.max(articles)
```

```
np.max(articles, axis=0) # Find maximum element in each column
```

```
np.max(articles, axis=1) # Find maximum element in each row
```

Average, Variance, and Standard Deviation

We can easily get descriptive statistics about the whole matrix or do calculations along a single axis.

```
# 4x6 array of random floats between 0–100
arr=np.random.rand(4,6)*100
print(arr)
```

```
np.mean(arr) # Returns the mean of all elements
```

```
arr.sum() # Returns the sum of all elements
```

```
np.mean(arr,axis=0) # Returns the mean along a specific axis
```

```
arr.cumsum(axis=1) #Cumulative sum of the elements
```

```
np.var(arr) # Returns the variance
```

```
np.std(arr,axis=1) # Returns the standard deviation along specific axis
```

```
np.corrcoef(arr) # Returns correlation coefficient of array
```

48.4. 📥 Demonstration: Reading Data and fitting

Try it yourself with the Jupyter notebook

Note that demonstration sections such as this one correspond to a Jupyter Notebook that you can run yourself (rather than just looking at the static version shown here). You are highly encouraged to download the source files of the lecture notes such that you have local access to these notebooks.

Our first data set is going to be a classic from nuclear physics, namely all available data on binding energies. Don't be intimidated if you are not familiar with nuclear physics. It serves simply as an example here of a data set.

We will show some of the strengths of packages like **Scikit-Learn** in fitting nuclear binding energies to specific functions using linear regression. First, however, we need to meet the **Pandas**.

```
# For showing plots inline
%matplotlib inline
```

Meet the Pandas

Pandas is a very useful Python package for reading and organizing data. It is an open source library providing high-performance, easy-to-use data structures and data analysis tools for Python.

pandas stands for panel data, a term borrowed from econometrics and is an efficient library for data analysis with an emphasis on tabular data. **pandas** has two major classes, the **DataFrame** class with two-dimensional data objects such as tabular data organized in columns, and the class **Series** with a focus on one-dimensional data objects. Both classes allow you to index data easily as we will see in the examples below. **pandas** allows you also to perform mathematical operations on the data, spanning from simple reshaping of vectors and matrices to statistical operations.

The following simple example shows how we can make tables of our data. Here we define a data set which includes names, place of birth and date of birth, and displays the data in an easy to read way.

```
import pandas as pd
from IPython.display import display
data = {'First Name': ["Frodo", "Bilbo", "Aragorn II", "Samwise"],
        'Last Name': ["Baggins", "Baggins","Elessar","Gamgee"],
        'Place of birth': ["Shire", "Shire", "Eriador", "Shire"],
        'Date of Birth T.A.': [2968, 2890, 2931, 2980]
        }
data_pandas = pd.DataFrame(data)
display(data_pandas)
```

	First Name	Last Name	Place of birth	Date of Birth T.A.
0	Frodo	Baggins	Shire	2968
1	Bilbo	Baggins	Shire	2890
2	Aragorn II	Elessar	Eriador	2931
3	Samwise	Gamgee	Shire	2980

In the above example we have imported **pandas** with the shorthand **pd**, the latter has become the standard way to import **pandas**. We then make a dictionary with various variables and reorganize the lists of values into a **DataFrame** and then print out a neat table with specific column labels as *Name*, *place of birth* and *date of birth*. Displaying these results, we see that the indices are given by the default numbers from zero to three. **pandas** is extremely flexible and we can easily change the above indices by defining a new type of indexing as

```
data_pandas = pd.DataFrame(data,index=['Frodo','Bilbo','Aragorn','Sam'])
display(data_pandas)
```

	First Name	Last Name	Place of birth	Date of Birth T.A.
Frodo	Frodo	Baggins	Shire	2968
Bilbo	Bilbo	Baggins	Shire	2890
Aragorn	Aragorn II	Elessar	Eriador	2931
Sam	Samwise	Gamgee	Shire	2980

Thereafter we display the content of the row which begins with the index **Aragorn**

```
display(data_pandas.loc['Aragorn'])
```

```
First Name            Aragorn II
Last Name                Elessar
Place of birth           Eriador
Date of Birth T.A.          2931
Name: Aragorn, dtype: object
```

We can easily append data to this DataFrame, for example using `concat` (note that the function `append` was deprecated in pandas 2.0)

```
new_hobbit = {'First Name': ["Peregrin"],
              'Last Name': ["Took"],
              'Place of birth': ["Shire"],
              'Date of Birth T.A.': [2990]
              }
data_pandas=pd.concat([data_pandas,pd.DataFrame(new_hobbit, index=['Pippin'])])
display(data_pandas)
```

	First Name	Last Name	Place of birth	Date of Birth T.A.
Frodo	Frodo	Baggins	Shire	2968
Bilbo	Bilbo	Baggins	Shire	2890
Aragorn	Aragorn II	Elessar	Eriador	2931
Sam	Samwise	Gamgee	Shire	2980
Pippin	Peregrin	Took	Shire	2990

Here are other examples where we use the **DataFrame** functionality to handle arrays, now with more interesting features for us, namely numbers. We set up a matrix of dimensionality 10×5 and compute the mean value and standard deviation of each column. Similarly, we can perform mathematical operations like squaring the matrix elements and many other operations.

```
import numpy as np
import pandas as pd
from IPython.display import display
np.random.seed(100)
# setting up a 10 x 5 matrix
rows = 10
cols = 5
a = np.random.randn(rows,cols)
df = pd.DataFrame(a)
display(df)
print(df.mean())
print(df.std())
display(df**2)
```

	0	1	2	3	4
0	-1.749765	0.342680	1.153036	-0.252436	0.981321
1	0.514219	0.221180	-1.070043	-0.189496	0.255001
2	-0.458027	0.435163	-0.583595	0.816847	0.672721
3	-0.104411	-0.531280	1.029733	-0.438136	-1.118318
4	1.618982	1.541605	-0.251879	-0.842436	0.184519
5	0.937082	0.731000	1.361556	-0.326238	0.055676
6	0.222400	-1.443217	-0.756352	0.816454	0.750445
7	-0.455947	1.189622	-1.690617	-1.356399	-1.232435
8	-0.544439	-0.668172	0.007315	-0.612939	1.299748
9	-1.733096	-0.983310	0.357508	-1.613579	1.470714

```
0   -0.175300
1    0.083527
2   -0.044334
3   -0.399836
4    0.331939
dtype: float64
0    1.069584
1    0.965548
2    1.018232
3    0.793167
4    0.918992
dtype: float64
```

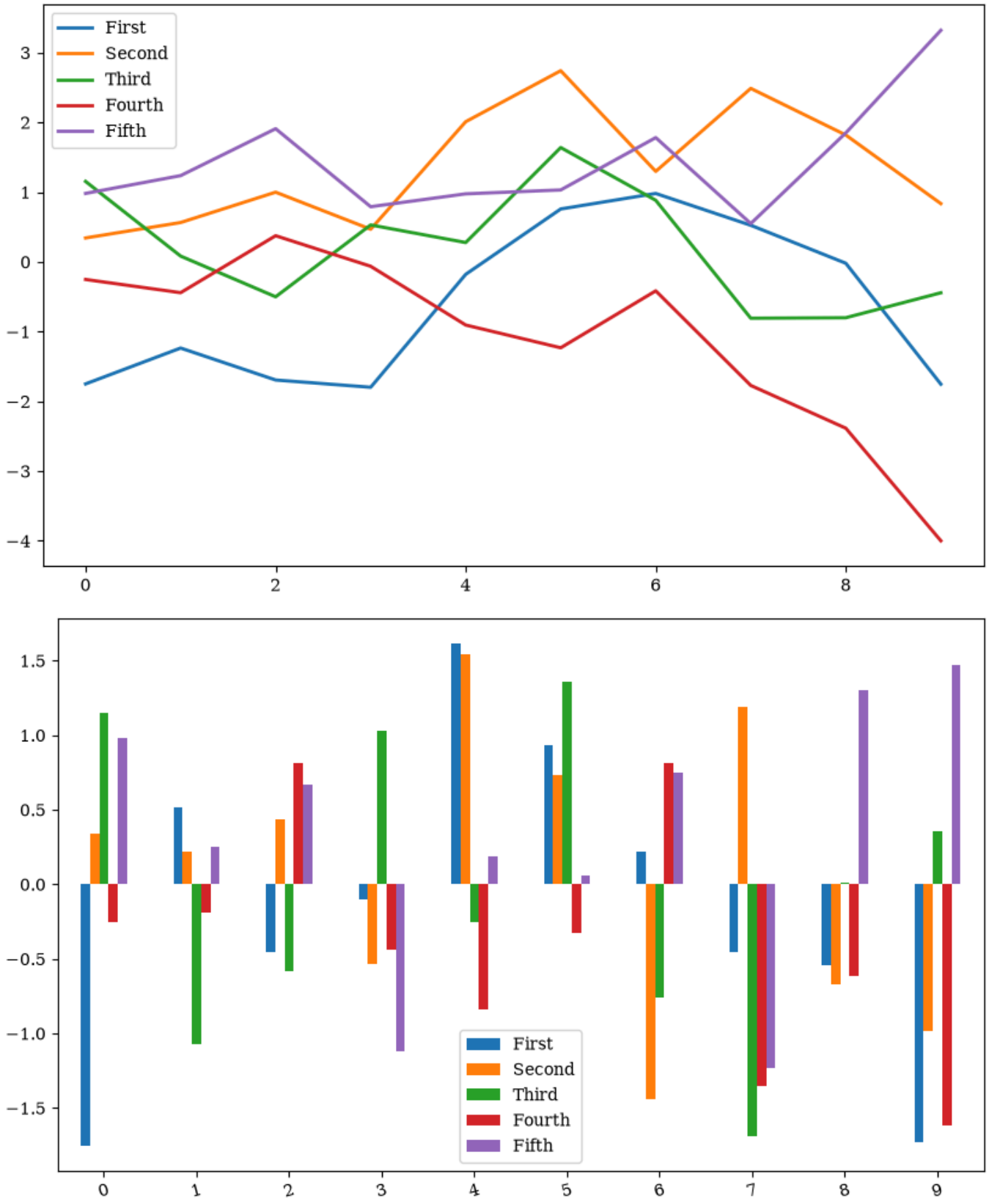
	0	1	2	3	4
0	3.061679	0.117430	1.329492	0.063724	0.962990
1	0.264421	0.048920	1.144993	0.035909	0.065026
2	0.209789	0.189367	0.340583	0.667239	0.452553
3	0.010902	0.282259	1.060349	0.191963	1.250636
4	2.621102	2.376547	0.063443	0.709698	0.034047
5	0.878123	0.534362	1.853835	0.106431	0.003100
6	0.049462	2.082875	0.572069	0.666597	0.563167
7	0.207888	1.415201	2.858185	1.839818	1.518895
8	0.296414	0.446453	0.000054	0.375694	1.689345
9	3.003620	0.966899	0.127812	2.603636	2.162999

Thereafter we can select specific columns only and plot final results

```
df.columns = ['First', 'Second', 'Third', 'Fourth', 'Fifth']
df.index = np.arange(10)

display(df)
print(df['Second'].mean() )
print(df.info())
print(df.describe())

from pylab import plt, mpl
mpl.rcParams['font.family'] = 'serif'

df.cumsum().plot(lw=2.0, figsize=(10,6));

df.plot.bar(figsize=(10,6), rot=15);
```

	First	Second	Third	Fourth	Fifth
0	-1.749765	0.342680	1.153036	-0.252436	0.981321
1	0.514219	0.221180	-1.070043	-0.189496	0.255001
2	-0.458027	0.435163	-0.583595	0.816847	0.672721
3	-0.104411	-0.531280	1.029733	-0.438136	-1.118318
4	1.618982	1.541605	-0.251879	-0.842436	0.184519
5	0.937082	0.731000	1.361556	-0.326238	0.055676
6	0.222400	-1.443217	-0.756352	0.816454	0.750445
7	-0.455947	1.189622	-1.690617	-1.356399	-1.232435
8	-0.544439	-0.668172	0.007315	-0.612939	1.299748
9	-1.733096	-0.983310	0.357508	-1.613579	1.470714

```
0.08352721390288316
<class 'pandas.DataFrame'>
Index: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   First   10 non-null     float64
 1   Second  10 non-null     float64
 2   Third   10 non-null     float64
 3   Fourth  10 non-null     float64
 4   Fifth   10 non-null     float64
dtypes: float64(5)
memory usage: 480.0 bytes
None
           First     Second      Third     Fourth      Fifth
count  10.000000  10.000000  10.000000  10.000000  10.000000
mean   -0.175300   0.083527  -0.044334  -0.399836   0.331939
std     1.069584   0.965548   1.018232   0.793167   0.918992
min    -1.749765  -1.443217  -1.690617  -1.613579  -1.232435
25%    -0.522836  -0.633949  -0.713163  -0.785061   0.087887
50%    -0.280179   0.281930  -0.122282  -0.382187   0.463861
75%     0.441264   0.657041   0.861676  -0.205231   0.923602
max     1.618982   1.541605   1.361556   0.816847   1.470714
```



We can produce a 4×4 matrix

```
b = np.arange(16).reshape((4,4))
print(b)
df1 = pd.DataFrame(b)
print(df1)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
    0   1   2   3
0   0   1   2   3
1   4   5   6   7
2   8   9  10  11
3  12  13  14  15
```

and many other operations.

The **Series** class is another important class included in **pandas**. You can view it as a specialization of **DataFrame** but where we have just a single column of data. It shares many of the same features as **DataFrame**. As with **DataFrame**, most operations are vectorized, achieving thereby a high performance when dealing with computations of arrays, in particular labeled arrays. As we will see below it leads also to a very concice code close to the mathematical operations we may be interested in. For multidimensional arrays, we recommend strongly xarray. **xarray** has much of the same flexibility as **pandas**, but allows for the extension to higher dimensions than two.

To our real data: nuclear binding energies. Brief reminder on masses and binding energies

Let us now dive into nuclear physics and remind ourselves briefly about some basic features about binding energies. A basic quantity which can be measured for the ground states of nuclei is the atomic mass $M(N, Z)$ of the neutral atom with atomic mass number A and charge Z. The number of neutrons is N. There are indeed several sophisticated experiments worldwide which allow us to measure this quantity to high precision (parts per million even).

Atomic masses are usually tabulated in terms of the mass excess defined by

$$\Delta M(N, Z) = M(N, Z) - uA,$$

where u is the Atomic Mass Unit

$$u = M(^{12}\mathrm{C})/12 = 931.4940954(57)\ \mathrm{MeV}/c^2.$$

The nucleon masses are

$$m_p = 1.00727646693(9)u,$$

and

$$m_n = 939.56536(8)\,\mathrm{MeV}/c^2 = 1.0086649156(6)u.$$

In the 2016 mass evaluation of by W.J.Huang, G.Audi, M.Wang, F.G.Kondev, S.Naimi and X.Xu [WAK+17] there are data on masses and decays of 3437 nuclei.

The nuclear binding energy is defined as the energy required to break up a given nucleus into its constituent parts of N neutrons and Z protons. In terms of the atomic masses $M(N,Z)$ the binding energy is defined by

$$BE(N,Z) = ZM_H c^2 + Nm_n c^2 - M(N,Z)c^2,$$

where M_H is the mass of the hydrogen atom and m_n is the mass of the neutron. In terms of the mass excess the binding energy is given by

$$BE(N,Z) = Z\Delta_H c^2 + N\Delta_n c^2 - \Delta(N,Z)c^2,$$

where $\Delta_H c^2 = 7.2890$ MeV and $\Delta_n c^2 = 8.0713$ MeV.

A popular and physically intuitive model which can be used to parametrize the experimental binding energies as function of A, is the so-called **liquid drop model**. The ansatz is based on the following expression

$$BE(N,Z) = a_1 A - a_2 A^{2/3} - a_3 \frac{Z(Z-1)}{A^{1/3}} - a_4 \frac{(N-Z)^2}{A},$$

where A stands for the number of nucleons and the a_is are parameters which are determined by a fit to the experimental data.

To arrive at the above expression we have assumed that we can make the following assumptions:

- There is a volume term $a_1 A$ proportional to the number of nucleons. When an assembly of nucleons of the same size is packed together into the smallest volume, each interior nucleon has a certain number of other nucleons in contact with it. This contribution is proportional to the volume. Note that the nuclear radius is empirically proportional to $A^{1/3}$.
- There is a surface energy term $a_2 A^{2/3}$. The assumption here is that a nucleon at the surface of a nucleus interacts with fewer other nucleons than one in the interior of the nucleus and hence its binding energy is less. This surface energy term takes that into account and is therefore negative and is proportional to the surface area.
- There is a Coulomb energy term $a_3 \frac{Z(Z-1)}{A^{1/3}}$. The electric repulsion between each pair of protons in a nucleus yields less binding.
- There is an asymmetry term $a_4 \frac{(N-Z)^2}{A}$. This term is associated with the Pauli exclusion principle and reflects the fact that the proton-neutron interaction is more attractive on the average than the neutron-neutron and proton-proton interactions.

We could also add a so-called pairing term, which is a correction term that arises from the tendency of proton pairs and neutron pairs to occur. An even number of particles is more stable than an odd number.

Import modules

We import also various modules that we will find useful in order to present various Machine Learning methods. Here we focus mainly on the functionality of **scikit-learn**.

```
%matplotlib inline

# Common imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn.linear_model as skl
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import os
```

```
# Nicer plots
#import seaborn as sns
#sns.set('talk')
```

Organizing our data

Let us start with reading and organizing our data. We start with the compilation of masses and binding energies from 2016. After having downloaded this file to our own computer, we are now ready to read the file and start structuring our data.

We start with preparing folders for storing our calculations and the data file over masses and binding energies.

```
# Where to fetch data files
DATA_ID = "../../assets/"

def data_path(dat_id):
    return os.path.join(DATA_ID, dat_id)
```

Our next step is to read the data on experimental binding energies and reorganize them as functions of the mass number A, the number of protons Z and neutrons N using **pandas**. Before we do this it is always useful (unless you have a binary file or other types of compressed data) to actually open the file and simply take a look at it!

Note: In the cell below, `infile` is an iterator and after you show the first 42 lines, the iterator will point to the 43rd line. In order to avoid reading the file from a non-expected start point, we put the iteration and print commands within a `with` block which means that `infile` is only defined inside this block and will then be forgotten. Thanks to (@Daniel Berg} for finding this bug.

```
with open(data_path("MassEval2016.dat"),'r') as infile:
    head = [next(infile) for x in np.arange(42)]
    print("".join(head))
```

```
1    a0boogfu                                A T O M I C   M A S S   A D J U S T M E N T
0                                                   DATE  1 Mar 2017 TIME 17:26
0        ********************                          A=   0 TO 295
```

```
         * file : mass16.txt *
         ********************

   This is one file out of a series of 3 files published in:
       "The Ame2016 atomic mass evaluation (I)"   by W.J.Huang, G.Audi, M.Wang, F.G.Kondev,
S.Naimi and X.Xu
           Chinese Physics C41 030002, March 2017.
       "The Ame2016 atomic mass evaluation (II)"  by M.Wang, G.Audi, F.G.Kondev, W.J.Huang,
S.Naimi and X.Xu
           Chinese Physics C41 030003, March 2017.
                      for files : mass16.txt  : atomic masses
                                  rct1-16.txt : react and sep energies,  part 1
                                  rct2-16.txt : react and sep energies,  part 2
   A fourth file  is the "Rounded" version of the atomic mass table (the first file)
                                  mass16round.txt : atomic masses "Rounded" version

   All files are 3436 lines long with 124 character per line.
       Headers are 39 lines long.
   Values in files 1, 2 and 3 are unrounded copy of the published ones
   Values in file  4          are exact     copy of the published ones

   col 1     :  Fortran character control: 1 = page feed  0 = line feed
   format    :
a1,i3,i5,i5,i5,1x,a3,a4,1x,f13.5,f11.5,f11.3,f9.3,1x,a2,f11.3,f9.3,1x,i3,1x,f12.5,f11.5
               cc NZ  N  Z  A    el  o     mass  unc binding unc     B  beta  unc
atomic_mass   unc
   Warnings  :  this format is identical to the ones used in Ame2003 and Ame2012
                      in particular "Mass Excess" and "Atomic Mass" values are given now, when
necessary,
                      with 5 digits after decimal point.
                decimal point is replaced by # for (non-experimental) estimated values.
                * in place of value : not calculable

....+....1....+....2....+....3....+....4....+....5....+....6....+....7....+....8....+....9....+...1


                                                                MASS LIST
                                                              for analysis

1N-Z    N    Z   A  EL    O     MASS EXCESS       BINDING ENERGY/A        BETA-DECAY ENERGY
ATOMIC MASS
                                    (keV)                (keV)                    (keV)
(micro-u)
0  1    1    0    1  n         8071.31713    0.00046      0.0      0.0   B-    782.347    0.000
1 008664.91582    0.00049
  -1    0    1    1 H          7288.97061    0.00009      0.0      0.0   B-      *
1 007825.03224    0.00009
0  0    1    1    2 H         13135.72176    0.00011   1112.283    0.000 B-      *
2 014101.77811    0.00012
```

In particular, the program that outputs the final nuclear masses is written in Fortran with a specific format. It means that we need to figure out the format and which columns contain the data we are interested in. Pandas comes with a function that reads formatted output. After having admired the file, we are now ready to start massaging it with **pandas**.

- The file begins with some basic format information in the header (that is 39 lines long).
- The data we are interested in are in columns 2, 3, 4 and 11, giving us the number of neutrons, protons, mass numbers and binding energies, respectively.
- We add also for the sake of completeness the element name.
- The data are in fixed-width formatted lines and we will convert them into the **pandas** DataFrame structure.

- The liquid drop model does not work well for the lightest elements so we skip the first few lines of data in the file (we could as well have made this cut after actually reading the data).
- The data is for the binding energy per nucleon (not the total binding energy as in the formula). We will need to be careful to compare the same quantities.

```
# Open the file again and read the experimental data with Pandas
with open(data_path("MassEval2016.dat"),'r') as infile:
    Masses = pd.read_fwf(infile, usecols=(2,3,4,6,11),
              names=('N', 'Z', 'A', 'Element', 'Ebinding'),
              widths=(1,3,5,5,5,1,3,4,1,13,11,11,9,1,2,11,9,1,3,1,12,11,1),
              header=64,
              index_col=False)

# Extrapolated values are indicated by '#' in place of the decimal place, so
# the Ebinding column won't be numeric. Coerce to float and drop these entries.
Masses['Ebinding'] = pd.to_numeric(Masses['Ebinding'], errors='coerce')
Masses = Masses.dropna()
# Convert from keV to MeV.
Masses['Ebinding'] /= 1000


# Group the DataFrame by nucleon number, A.
# Using idxmax avoids MultiIndex from groupby-apply.
idx = Masses.groupby("A")["Ebinding"].idxmax()
Masses = Masses.loc[idx].sort_values("A").reset_index(drop=True)

# Find the rows of the grouped DataFrame with the maximum binding energy.
#Masses = best.apply(lambda t: t[t.Ebinding==t.Ebinding.max()])
```

We have now read in the data, grouped them according to the variables we are interested in. We see how easy it is to reorganize the data using **pandas**. If we were to do these operations in C/C++ or Fortran, we would have had to write various functions/subroutines which perform the above reorganizations for us. Having reorganized the data, we can now start to make some simple fits using both the functionalities in **numpy** and **Scikit-Learn** afterwards.

Now we define five variables which contain the number of nucleons A, the number of protons Z and the number of neutrons N, the element name and finally the energies themselves.

```
Masses
```

	N	Z	A	Element	Ebinding
0	6	4	10	Be	6.497630
1	6	5	11	B	6.927732
2	6	6	12	C	7.680144
3	7	6	13	C	7.469849
4	8	6	14	C	7.520319
...	...	...	...	...	...
253	156	108	264	Hs	7.298375
254	157	108	265	Hs	7.296247
255	158	108	266	Hs	7.298273
256	159	110	269	Ds	7.250154
257	160	110	270	Ds	7.253775

258 rows × 5 columns

```
A = Masses['A']
Z = Masses['Z']
N = Masses['N']
Element = Masses['Element']
TotalEnergies = Masses['Ebinding'] * Masses['A']
print(Masses)
```

```
       N    Z    A Element  Ebinding
0      6    4   10      Be  6.497630
1      6    5   11       B  6.927732
2      6    6   12       C  7.680144
3      7    6   13       C  7.469849
4      8    6   14       C  7.520319
..   ...  ...  ...     ...       ...
253  156  108  264      Hs  7.298375
254  157  108  265      Hs  7.296247
255  158  108  266      Hs  7.298273
256  159  110  269      Ds  7.250154
257  160  110  270      Ds  7.253775

[258 rows x 5 columns]
```

The next step, and we will define this mathematically later, is to set up the so-called **design matrix**. We will throughout call this matrix $\boldsymbol{X}$. It has dimensionality $n \times p$, where n is the number of data points and p are the so-called predictors or features. In our case they are given by the functions of A (and N, Z) that are included in the liquid drop model and that we wish to include in the fit.

We will make the fit with respect to the total binding energy. One could argue that it would be better to fit to the binding energy per nucleon. Can you argue why that is?

Note that we also include a constant term (an intercept) since this is expected by the LinearRegression method that we will be using.

```
# Now we set up the design matrix X
X = np.zeros((len(A),5))
X[:,0] = np.ones_like(A)
X[:,1] = A
X[:,2] = A**(2.0/3.0)
X[:,3] = Z*(Z-1) * A**(-1.0/3.0)
X[:,4] = (N-Z)**2 * A**(-1.0)
```

With **scikitlearn** we are now ready to use linear regression and fit our data. Note that we have included an intercept column into our design matrix, which corresponds to a constant predictor term in our model. It is very common to have such a term in a linear regression fit and we include it here although our model actually does not have such a predictor. In fact, the built-in linear regression function that we will use does usually add such an offset automatically and we need to explicitly turn it off using the argument `fit_intercept=False` since we already have it in our design matrix.

```
clf = skl.LinearRegression(fit_intercept=False).fit(X, TotalEnergies)
fity = clf.predict(X)
```

Pretty simple! Now we can print measures of how our fit is doing, the coefficients from the fits and plot the final fit together with our data.

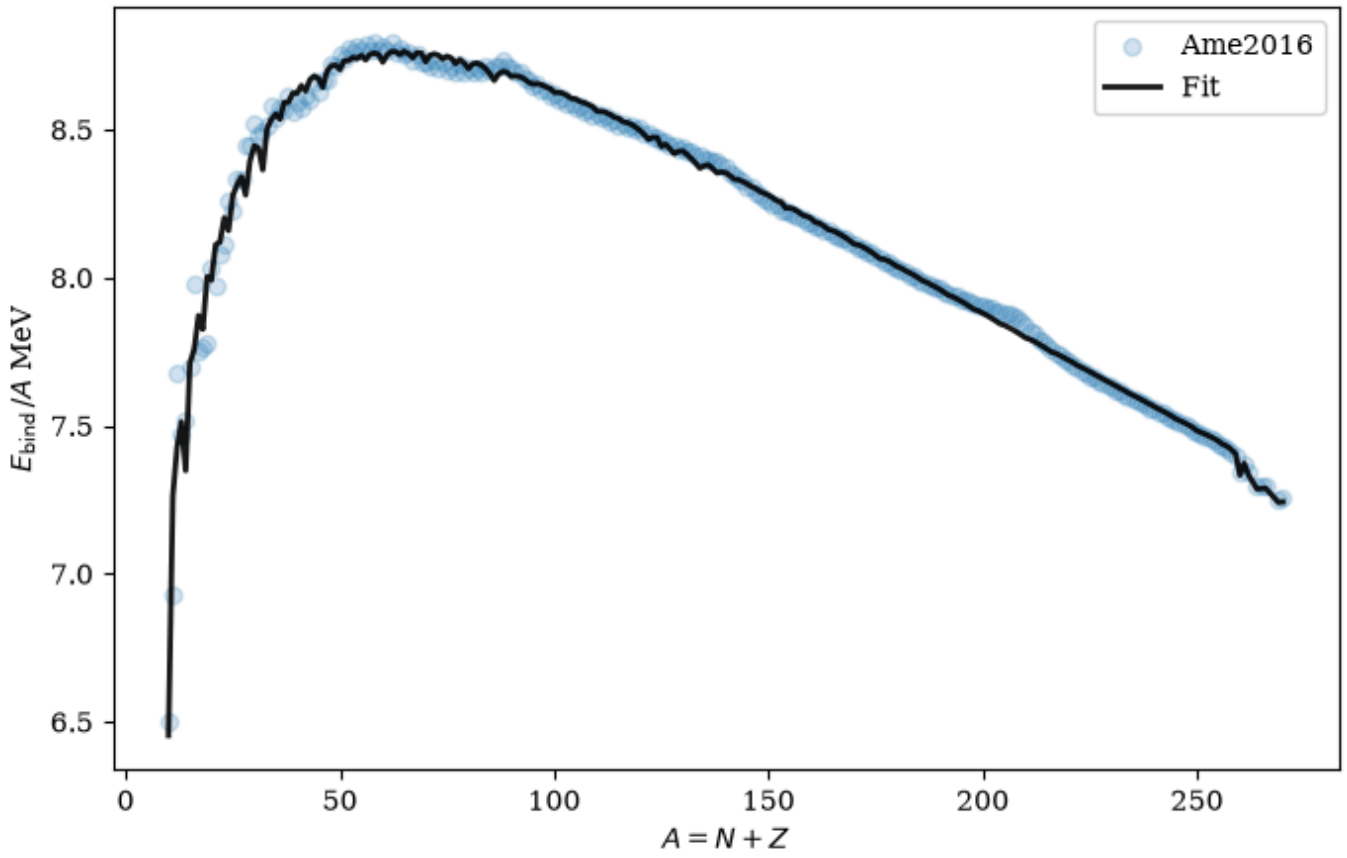
```
# The mean squared error
print(f"Mean squared error (for energy per nucleon): {mean_squared_error(TotalEnergies/A,
fity/A):.3f}")
# Explained variance score: 1 is perfect prediction
print(f'Variance score (for energy per nucleon): {r2_score(TotalEnergies/A, fity/A):.3f}')
# Mean absolute error
print(f'Mean absolute error (for energy per nucleon): {mean_absolute_error(TotalEnergies/A,
fity/A):.3f}')
```

```
Mean squared error (for energy per nucleon): 0.002
Variance score (for energy per nucleon): 0.990
Mean absolute error (for energy per nucleon): 0.025
```

```
print(clf.coef_)
# Or nicer
print('***')
print('intercept  volume    surface    coulomb     asymmetry')
print("     ".join(["%.3f"%coef for coef in clf.coef_]))
```

```
[ 19.06802716  17.20433313 -24.05418781  -0.78693299 -26.3490481 ]
***
intercept  volume    surface    coulomb     asymmetry
19.068     17.204     -24.054     -0.787     -26.349
```

```
# Generate a plot comparing the experimental with the fitted values values.
Masses['Eapprox']  = fity/A
fig, ax = plt.subplots(figsize=(8,5))
ax.set_xlabel(r'$A = N + Z$')
ax.set_ylabel(r'$E_\mathrm{bind}\,/A$ MeV')
ax.scatter(Masses['A'], Masses['Ebinding'], alpha=0.2, marker='o',
           label='Ame2016')
ax.plot(Masses['A'], Masses['Eapprox'], alpha=0.9, lw=2, c='k',
           label='Fit')
ax.legend();
```



A first summary

The aim behind these introductory words was to present to you various Python libraries and their functionalities, in particular libraries like **numpy**, **pandas**, and **matplotlib** and other that make our life much easier in handling various data sets and visualizing data.

Furthermore, **Scikit-Learn** allows us with few lines of code to implement popular Machine Learning algorithms for supervised learning.

A physics remark that concerns the liquid drop model for nuclear binding energies is that we fail to reproduce some slow variations in the target data. In fact, we know that these variations are due to shell effects, with local maxima appearing in binding energies as different shells and subshells are filled.

48.5. 📥 Demo: Making a simple widget-based UI

This is a guide-by-example to making simple user interfaces (UIs) for Jupyter notebook using the widgets provided by ipywidgets. In particular we use `ipywidgets.interactive_output`. There are both simpler and more elaborate ways to use ipywidgets; this is a happy medium.

Google “ipywidgets interact” for links to more information about widget types and other options for widgets.

v1 by Dick Furnstahl (furnstahl.1@osu.edu)

```
import numpy as np
import matplotlib.pyplot as plt
```

First pass: only minimal controls

1. Import ipywidgets and Ipython.display

```
# For convenience, we'll import explicitly the widgets we might use
import ipywidgets as widgets
from ipywidgets import HBox, VBox, Layout, Tab, Label, Checkbox
from ipywidgets import FloatSlider, IntSlider, Play, Dropdown, HTMLMath

from IPython.display import display
```

2. Create a function with all inputs to generate the output figure(s).

```
# This is a proxy for whatever calculations we need to do for the figure.
def my_function(t, A=1., omega=1., phi=0.):
    """A sine function with adjustable amplitude, frequency, and phase.
    """
    return A * np.sin(omega * t + phi)


def update_plot(omega=1.0):
    """
    Update the plot, which includes a call to a function.
    Note that the entire plot is redrawn here.
    """
    t_min = 0.
    t_max = 2. * np.pi
    num_t = 200
    t_pts = np.linspace(t_min, t_max, num_t)

    fig = plt.figure()
    ax = fig.add_subplot(1, 1, 1)
    line, = ax.plot(t_pts, my_function(t_pts, omega=omega),
                    lw=2, color='blue')
    ax.set_xlabel(r'$t$')
    ax.set_ylabel(r'$f(t)$')
```

3. Make a widget for each value you want to control.

```
# We append _w to widget objects by convention.
omega_w = FloatSlider(value=1.0, min=0.1, max=10., step=0.1,
                      description=r'$\omega$ :')
```

4. Make any explicit callback functions and add .observe methods.

We won't need this on the first pass.

5. Set up the interactive_output function.

```
plot_out = widgets.interactive_output(update_plot,
                                      dict(
                                      omega=omega_w,
                                      )
                                     )
```

6. Make the layout of the widgets.

```
UI_box = VBox([omega_w, plot_out])
```

7. Release the Kraken!

```
display(UI_box)
```

Try moving the slider. Note how the figure updates while you are still moving the slider – we'll suppress this in the second pass with `continuous_update = False`.

Second pass: More elaborate controls and options

1. Import ipywidgets and Ipython.display

```
# Import explicitly the widgets we might use (add more as needed!)
import ipywidgets as widgets
from ipywidgets import HBox, VBox, Layout, Tab, Label, Checkbox
from ipywidgets import FloatSlider, IntSlider, Play, Dropdown, HTMLMath

from IPython.display import display, Math
```

2. Create a function with all inputs that makes the output figure(s).

```
# This is a proxy for whatever calculations we need to do for the figure.
def my_function(t, A=1., omega=1., phi=0.):
    """Just a sine function with adjustable amplitude, frequency, and phase
    """
    return A * np.sin(omega * t + phi)


def update_plot(omega=1.0, A=1., phi=0.,
                t_min=0., t_max=2.*np.pi, num_t=200,
                lw=2, color='blue'
               ):
    """
    Update the plot, which includes a call to a function.
    Note that the entire plot is redrawn here.
    Now all the parameters controlling the plot are inputs to the function.
    """
    t_pts = np.linspace(t_min, t_max, num_t)

    fig = plt.figure()
    ax = fig.add_subplot(1, 1, 1)
    line, = ax.plot(t_pts, my_function(t_pts, omega=omega, A=A, phi=phi),
                    lw=lw, color=color)
    ax.set_xlabel(r'$t$')
    ax.set_ylabel(r'$f(t)$')
```

3. Make a widget for each value you want to control.

```
slider_border = Layout(border='solid 1.0px')

def float_widget(value, min, max, step, description, format):
    """Makes a FloatSlider with the passed parameters and continuous_update
       set to False."""
    return FloatSlider(value=value, min=min, max=max, step=step,
                       disabled=False,
                       description=description, continuous_update=False,
                       orientation='horizontal', layout=slider_border,
                       readout=True, readout_format=format)


omega_w = float_widget(value=1.0, min=0.1, max=10., step=0.1,
                       description=r'$\omega$ :', format='.1f')

A_w = float_widget(value=1.0, min=0.1, max=10., step=0.1,
                       description=r'$A$ :', format='.1f')

phi_w = float_widget(value=0.0, min=-10., max=10., step=0.1,
                       description=r'$\phi$ :', format='.1f')

t_min_w = float_widget(value=0.0, min=0.0, max=10., step=0.1,
                       description=r'$t_{\rm min}$ :', format='.1f')

t_max_w = float_widget(value=20.0, min=5., max=50., step=0.1,
                       description=r'$t_{\rm max}$ :', format='.1f')

num_t_w = IntSlider(value=200, min=50, max=500, step=10,
                    continuous_update=False,
                    description=r'num $t$ :', layout=slider_border)
```

4. Make any explicit callback functions and add .observe methods.

```
# Make sure that t_max is at least t_min + 1
def update_t_max(*args):
    if t_max_w.value < t_min_w.value:
        t_max_w.value = t_min_w.value + 1.
t_max_w.observe(update_t_max, 'value')
t_min_w.observe(update_t_max, 'value')
```

5. Set up the interactive_output function.

```
plot_out = widgets.interactive_output(update_plot,
                                      dict(
                                      omega=omega_w,
                                      A=A_w,
                                      phi=phi_w,
                                      t_min=t_min_w,
                                      t_max=t_max_w,
                                      num_t=num_t_w,
                                      )
                                     )
```

6. Make the layout of the widgets.

```
hbox_1 = HBox([t_min_w, t_max_w, num_t_w])
hbox_2 = HBox([omega_w, A_w, phi_w])
UI_box = VBox([hbox_1, hbox_2, plot_out])
```

7. Release the Kraken!

```
display(UI_box)
```

Third pass: a more elaborate user interface with tabs

1. Import ipywidgets and Ipython.display

```
# Import explicitly the widgets we might use (add more as needed!)
import ipywidgets as widgets
from ipywidgets import HBox, VBox, Layout, Tab, Label, Checkbox, Button
from ipywidgets import FloatSlider, IntSlider, Play, Dropdown, HTMLMath

from IPython.display import display
```

2. Create a function with all inputs that makes the output figure(s).

```
# This is a proxy for whatever calculations we need to do for the figure.
def my_function(t, A=1., omega=1., phi=0.):
    """Just a sine function with adjustable amplitude, frequency, and phase
    """
    return A * np.sin(omega * t + phi)


def update_plot(omega=1.0, A=1., phi=0.,
                t_min=0., t_max=2.*np.pi, num_t=200,
                lw=2, color='blue',
                font_size=18
               ):
    """
    Update the plot, which includes a call to a function.
    Note that the entire plot is redrawn here.
    Now all the parameters controlling the plot are inputs to the function.
    """
    t_pts = np.linspace(t_min, t_max, num_t)

    # Update the common font size
    plt.rcParams.update({'font.size': font_size})

    fig = plt.figure()
    ax = fig.add_subplot(1, 1, 1)
    line, = ax.plot(t_pts, my_function(t_pts, omega=omega, A=A, phi=phi),
                    lw=lw, color=color)
    ax.set_xlabel(r'$t$')
    ax.set_ylabel(r'$f(t)$')
```

3. Make a widget for each value you want to control.

```
slider_border = Layout(border='solid 1.0px')

def float_widget(value, min, max, step, description, format):
    """Makes a FloatSlider with the passed parameters and continuous_update
       set to False."""
    return FloatSlider(value=value, min=min, max=max, step=step,
                       disabled=False,
                       description=description, continuous_update=False,
                       orientation='horizontal', layout=slider_border,
                       readout=True, readout_format=format)


omega_w = float_widget(value=1.0, min=0.1, max=10., step=0.1,
                       description=r'$\omega$ :', format='.1f')

A_w = float_widget(value=1.0, min=0.1, max=10., step=0.1,
                       description=r'$A$ :', format='.1f')

phi_w = float_widget(value=0.0, min=-10., max=10., step=0.1,
                       description=r'$\phi$ :', format='.1f')

t_min_w = float_widget(value=0.0, min=0.0, max=10., step=0.1,
                       description=r'$t_{\rm min}$ :', format='.1f')

t_max_w = float_widget(value=20.0, min=5., max=50., step=0.1,
                       description=r'$t_{\rm max}$ :', format='.1f')

num_t_w = IntSlider(value=200, min=50, max=500, step=10,
                    continuous_update=False,
                    description=r'num $t$ :', layout=slider_border)

################### begin: text ##################
initial_text = \
    r"""
    The graph shows a sine wave specified by an amplitude, frequency,
    and phase.  All are adjustable.
    """
initial_text_w = HTMLMath(value=initial_text)

times_overview_text = \
    r"""
    Adjust the sliders to set the range in time $t$ that is plotted
    and the number of points plotted.
    """
times_overview_text_w = HTMLMath(value=times_overview_text)
################### end: text ##################


# Widgets for the set-up parameters
font_size_w = Dropdown(options=['12', '16', '18', '20', '24'], value='18',
                       description='Font size:',disabled=False,
                       continuous_update=False,layout=Layout(width='140px'))

################### begin: text for help tabs ##################
# In HTML (could move this to an external file!)
overview_text = \
   r"""<p>Here we give an example of how to display a function with
           controls.</p>
          <ul>
            <li>Parameters tab: find out how to specify the function.
            <li>Times tab: find out what the controls do.
```

```
            <li>Setup tab: change how the plots are made.
          </ul>
    """

parameters_text = \
    r"""
    We are plotting this function:
    $$
    A \sin(\omega t + \phi)
    $$
    """

times_text = \
    r"""
    You get to decide the range of time $t$!
    """

setup_text = \
    r"""
    For now, just an adjustment to the font size used for the plot.
    """
#################### end: text for help tabs ##################


# Widgets for the help section, which are HTMLMath boxes in a Tab widget
help_overview_w = HTMLMath(value=overview_text)
help_parameters_w = HTMLMath(value=parameters_text)
help_times_w = HTMLMath(value=times_text)
help_setup_w = HTMLMath(value=setup_text)
```

4. Make any explicit callback functions and add .observe methods.

```
# Make sure that t_max is at least t_min + 1
def update_t_max(*args):
    if t_max_w.value < t_min_w.value:
        t_max_w.value = t_min_w.value + 1.
t_max_w.observe(update_t_max, 'value')
t_min_w.observe(update_t_max, 'value')
```

5. Set up the interactive_output function.

```
plot_out = widgets.interactive_output(update_plot,
                                      dict(
                                      omega=omega_w,
                                      A=A_w,
                                      phi=phi_w,
                                      t_min=t_min_w,
                                      t_max=t_max_w,
                                      num_t=num_t_w,
                                      font_size=font_size_w,
                                      )
                                     )
```

6. Make the layout of the widgets.

```
# Organize the help as Tabs (which will itself be under the Help Tab)
help_max_height = '500px'
help_tab = Tab(children=[help_overview_w, help_parameters_w, help_times_w,
                         help_setup_w],
               layout=Layout(width='95%', max_height=help_max_height))
help_tab.set_title(0, 'Overview')
help_tab.set_title(1, 'Parameters')
help_tab.set_title(2, 'Times')
help_tab.set_title(3, 'Set-up')

# Boxes of widgets that will go into the top-level tabs
textbox0 = HBox([initial_text_w])
hbox0 = HBox([omega_w, A_w, phi_w])
textbox1 = HBox([times_overview_text_w])
hbox1a = HBox([t_min_w, t_max_w])
hbox1b = HBox([num_t_w])
hbox2 = HBox([font_size_w]) # font size
hbox3 = HBox([help_tab])

# We'll set up Tabs to organize the controls.  The Tab contents are declared
#  as tab0, tab1, ... (could also make this a list) and the overall Tab
#  is called tab (so its children are tab0, tab1, ...).
tab_height = '70px'  # Fixed minimum height for all tabs.
tab0 = VBox([textbox0, hbox0], layout=Layout(min_height=tab_height))
tab1 = VBox([textbox1, hbox1a, hbox1b], layout=Layout(min_height=tab_height))
tab2 = VBox([hbox2], layout=Layout(min_height=tab_height))
tab3 = VBox([hbox3], layout=Layout(min_height=tab_height))

tab = Tab(children=[tab0, tab1, tab2, tab3])
tab.set_title(0, 'Parameters')
tab.set_title(1, 'Time limits')
tab.set_title(2, 'Set-up')
tab.set_title(3, 'Help')

UI_box = VBox([tab, plot_out])
```

7. Release the Kraken!

```
display(UI_box)
```

Overview of TALENT mini-projects

The exercises in this section are multi-part but are not full-blown projects, hence the name “mini-projects”. They originated with the TALENT (“Training in Advanced Low-Energy Nuclear Theory”) course given by the authors in 2019.

The contents are:

- Mini-project I: Parameter estimation for a toy model of an EFT. The goal is to reproduce Bayesian parameter estimation results for a (published) toy model ofeffective field theories, which takes the form of Taylor series of a specified function.

- Mini-project IIIa: Bayesian Optimization explores using Bayesian optimization to minimize functions of one variable and more variables while Mini-project IIIb: Bayesian Neural Networks explores a basic classification taks with a Bayesian neural network.

Mini-project I: Parameter estimation for a toy model of an EFT

The overall project goal is to reproduce various results in this paper: *Bayesian parameter estimation for effective field theories*. It's a long paper, so don't try to read all of it! (At least not now.) We'll guide you to the relevant parts.

The paper uses toy models for effective field theories, namely Taylor series of some specified functions, to present guidelines for parameter estimation. This will also be a check of whether you can follow (or give you practice on) Bayesian statistics discussions in the physics literature.

You'll find summaries in section II that touch on topics we have discussed and will discuss. The function

$$g(x) = \left(\frac{1}{2} + \tan\left(\frac{\pi}{2}x\right)\right)^2$$

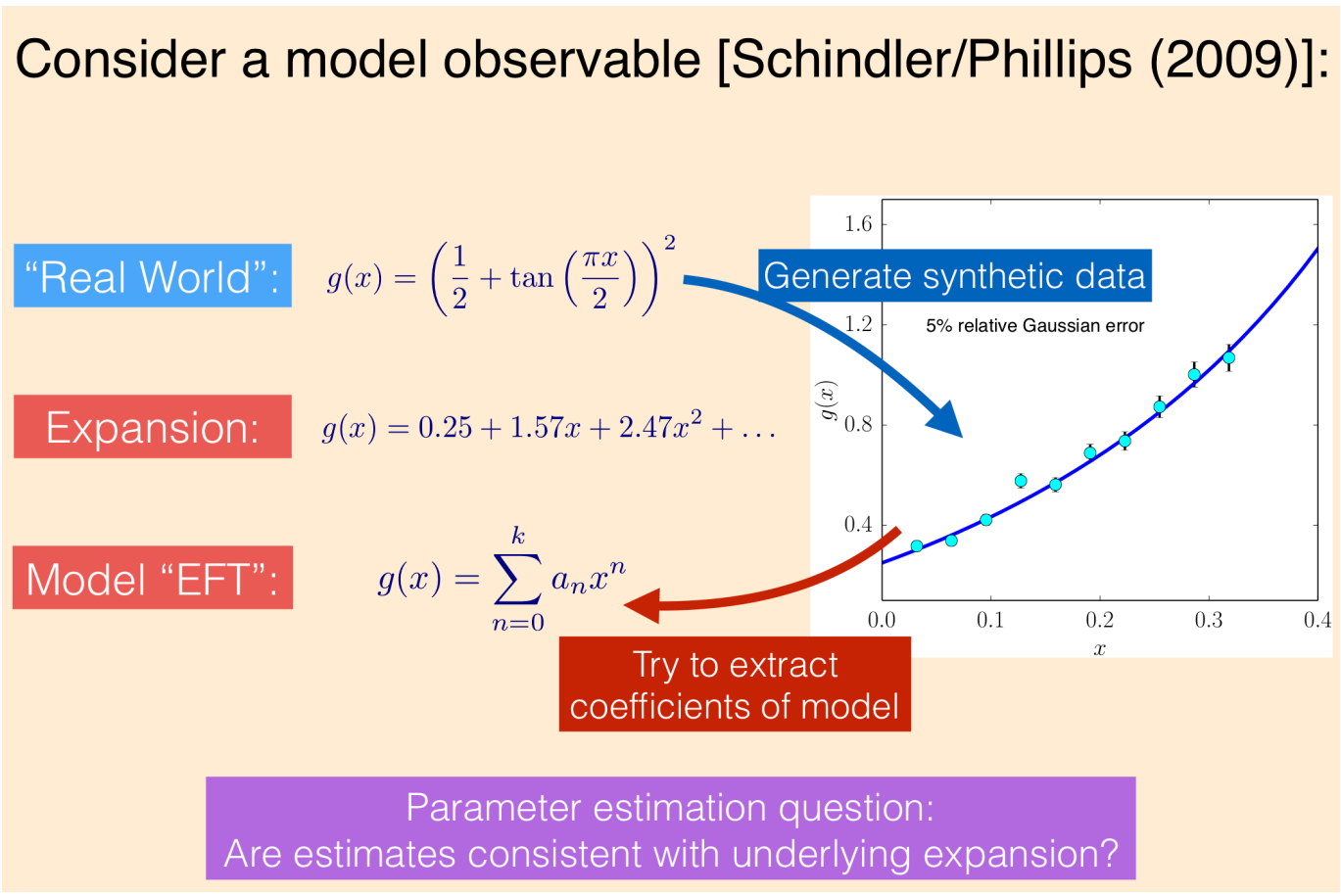
represents the true, underlying theory. It has a Taylor expansion

$$g(x) = 0.25 + 1.57x + 2.47x^2 + 1.29x^3 + \cdots$$

Our model for an EFT for this "theory" is

$$g_{\text{th}}(x) \equiv \sum_{i=0}^{k} a_i x^i$$

and your general task is to fit 1, 2, 3, … of the constants a_i, and analyze the results. **Your primary goal is to reproduce and interpret Table III on page 12 of the arXiv preprint. A secondary goal is to reproduce and interpret Figure 1 of the same paper.** You should use the emcee sampler and corner to make plots.



Learning goals:

- Apply and extend the Bayesian parameter estimation ideas and techniques from the course.
- Explore the impact of control features: dependence on how much data is used and how precise it is; apply an *informative* prior.
- Learn about some diagnostics for Bayesian parameter estimation.
- Try out sampling on a controlled problem.

Suggestions for how to proceed:

- Follow the lead of the notebooks Gaussian noise and Fitting a straight line II.
- Define a function for the exact result plus noise, noting from the arXiv paper what type of noise is added and where the points are located (i.e., what values of x).
- Define functions for the two choices of prior and for the likelihood. Also a function for the posterior.
- Call emcee to sample the posteriors.
- Use corner to create plots. You can read the answers for the tables from the corner plots (it is also possible to extract the numbers directly: see the example at the bottom of this page from the emcee documentation).
- Don't try to do too much in your code at first (start with the lowest order in Table III).
- Fill in the rest of Table III. Characterize and explain the different results from the two priors (also: which one is equivalent to standard least-squares fitting?).

- Generate figures for the lowest orders analogous to Figure 1 and then reproduce Figure 1. Interpret the projected posteriors (e.g., what coefficients are very correlated and how does this relate to overfitting?) and identify where the “prior is returned”.

Comments and suggestions

- To reproduce precisely (within fluctuations from the sampling) the numbers in Table III, you’ll need to use the same data set used in the table. This is available from the arXiv as D1_c_5.dat. These values are those plotted in the figure at the top of this notebook.
- The 5% error is a *relative* error, meaning it is 0.05 times the data at that point. This means if you generate a Gaussian random number `err` distributed with standard deviation 0.05, the value of sigma for the log likelihood is `sigma[i] = data[i] * err` (use the data, not the theory at `i`).
- The `show_titles=True` option to corner will show central results and one-σ error limits on the projected posterior plots.
- The `quantiles=[0.16, 0.5, 0.84]`option to corner adds the dashed vertical lines to the marginal posteriors on the diagonal. You can obviously change the quantiles if you want another credibility region.
- The Python command `np.percentile(samples, [16, 50, 84],axis=0)` might be useful to extract numerical values for the credibility region and the mean from a python array `samples` of shape (nsamples,ndimensions).
- The example on Fitting a Model to Data from the emcee documentation may be useful to supplement the examples in the other notebooks.
- Defining Python functions to do the sampling and plotting will make it much more efficient (and easier) to generate the numbers in the table or the particular plots for figure 1.

Bonus: additional subtasks

Do one or more of these to get a plus.

- Reproduce Figures 3 and 4, showing the predictions with error bands for the two priors compared to the true function. You can use Matplotlib’s `fill_between(x, y-error, y+error)` to make bands. (Use the `alpha` keyword for `fill_between`, e.g., `alpha=0.5`, to make the bands more transparent.)
- Reproduce Figure 5 (alternative prior and “returning the prior”),
- Reproduce Figure 6 (posterior for $\overline{a}$), and 7 (“relaxing to least squares”).
- Reproduce Figure 9 (sensitivity to choice of $x_{\rm max}$)
- Repeat analysis with same function but different data precision and/or quantity (number of data points).
- Repeat analysis with a different function from the paper or invent your own function and analyze.

Mini-project IIIa: Bayesian Optimization

Import modules

```
%matplotlib inline
import numpy as np
import scipy as sp
from scipy.stats import multivariate_normal

import matplotlib.pyplot as plt
import seaborn as sns; sns.set_style("darkgrid"); sns.set_context("talk")

import GPy
import GPyOpt   # This will do the Bayesian optimization
```

1. A univariate example with GPyOpt

Try to minimize the function

$$\sin(3\theta) + \theta^2 - 0.7\theta$$

on the interval $\theta \in [-1, 2]$.

a. **Plot the true function**

```
# Insert code here
#
```

b. Find the minimum using `scipy.optimize.minimize`. Plot in the figure with the true function. Repeat with a few different seeds for the starting point.

Do you always get the same minimum?

```
# Insert code here
#
```

c. Use Bayesian Optimization with GPyOpt (following the example in the lecture notebook).

Plot the statistical model and the acquisition function for the first ten iterations. Also plot the final summary of the BayesOpt convergence sequence.

```
# Insert code here
#
```

d. Change the acquisition function to 'LCB'. Make sure to plot the iterations.

How do the acquisition functions compare when it comes to exploration-exploitation?

```
# Insert code here
#
```

e. **Repeat with noise added to the true function when generating data.**

- Assuming that we have an input parameter vector `X`, and that we have defined `noise = 0.2`. Then we can create some noise with normal distribution using

```
noise * np.random.randn(*X.shape)
```

- Redefine your “true” function so that it returns results with such noise and repeat the `GPyOpt` implementation (see the 2-parameter example in the lecture notebook).
- It is important that your GP expects a noisy function. You must set `exact_feval = False` in `GPyOpt.methods.BayesianOptimization`.
- Plot several samples from the “noisy” true function (using e.g. `alpha=0.1` to make them semi-transparent). Also plot the true function without noise.
- Perform the Bayesian optimization. Study the convergence, but also the statistical model. **How is it different compared to the statistical model in the example without noise?**

f. **Build the statistical model in BayesOpt with a different kernel.**

- Try in particular with the `Matern32` kernel. Do you remember what it looks like?
- Define a GPy kernel with your initial guess variance and lengthscale

```
GPkernel = GPy.kern.Matern32(input_dim=1, variance=1.0, lengthscale=1.0)
```

- Include this kernel as an input argument to `GPyOpt.methods.BayesianOptimization`

```
optimizer = BayesianOptimization(f=fNoise,
                                 model_type='GP',
                                 kernel=GPkernel,
                                 ...
```

```
# Insert code here
#
#
```

Questions to answer:

- Can you decide if any of these kernels work better for this problem then the other?

- What is the observable difference between the posterior function in this experiment compared to the previous one with the default `RBF` kernel?

- How would you decide which kernel to use for your problem?**

2. Build your own BayesOpt algorithm (optional or for your project)

Now try to repeat the above, but **assemble your own BayesOpt algorithm** using functions from `numpy`, `scipy`, and `GPy` (for building the statistical model).

Recall the pseudo-code for BayesOpt

1. initial $\theta^{(1)}, \theta^{(2)}, \dots \theta^{(k)}$, where $k \geq 2$
2. evaluate the objective function $f(\theta)$ to obtain $y^{(i)} = f(\theta^{(i)})$ for $i = 1, \dots, k$
3. initialize a data vector $\mathcal{D}_k = \left\{(\theta^{(i)}, y^{(i)})\right\}_{i=1}^{k}$
4. select a statistical model for $f(\theta)$
5. **for** $\{n = k+1, k+2, \dots\}$
    1. select $\theta^{(n)}$ by optimizing the acquisition function: $\theta^{(n)} = \arg\max_{\theta} \quad \mathcal{A}(\theta | \mathcal{D}_{n-1})$
    2. evaluate the objective function to obtain $y^{(n)} = f(\theta^{(n)})$
    3. augment the data vector $\mathcal{D}_n = \left\{\mathcal{D}_{n-1}, (\theta^{(n)}, y^{(n)})\right\}$
    4. update the statistical model for $f(\theta)$
6. **end for**

Sub-tasks:

- You have to implement all steps in the above pseudo-code.
- For the statistical model you can use `GPy`, following the examples from last week's lecture and exercise. Remember that the model has to be updated at step 5D.
- Implement the LCB acquisition function for use in step 5A. The maximum of $\mathcal{A}(\theta)$ can be found using `scipy.minimize` (note that you want the maximum…). It is a good idea to try several different starting points. See example code below, or implement your own algorithm if you prefer bug checking your own code. As an alternative to LCB, if you have time, you can also try the implementation of Expected Improvement in the code snippet below. However, this code might have to be cleansed of bugs.

Example code for a function that proposes the next sampling point by computing the location of the acquisition function maximum. Optimization is restarted `n_restarts` times to avoid local optima.

```
from scipy.optimize import minimize

def propose_location(acquisition, X_sample, Y_sample, gpr, bounds, n_restarts=25):
    '''
    Proposes the next sampling point by optimizing the acquisition function.
    Args:
    acquisition: Acquisition function.
    X_sample: Sample locations (n x d).
    Y_sample: Sample values (n x 1).
    gpr: A GaussianProcessRegressor fitted to samples.
    Returns: Location of the acquisition function maximum.
    '''
    dim = X_sample.shape[1]
    min_val = 1
    min_x = None

    def min_obj(X):
        # Minimization objective is the negative acquisition function
        return -acquisition(X.reshape(-1, dim), X_sample, Y_sample, gpr)

    # Find the best optimum by starting from n_restart different random points.
    for x0 in np.random.uniform(bounds[:, 0], bounds[:, 1], size=(n_restarts, dim)):
        res = minimize(min_obj, x0=x0, bounds=bounds, method='L-BFGS-B')
        if res.fun < min_val:
            min_val = res.fun[0]
            min_x = res.x

    return min_x.reshape(-1, 1)
```

Example code for the Expected Improvement acquisition function.

```
from scipy.stats import norm

def expected_improvement(X, X_sample, Y_sample, gpr, xi=0.01):
    '''
    Computes the EI at points X based on existing samples,
    X_sample and Y_sample, using a Gaussian process surrogate model.
    Args:
    X: Points at which EI shall be computed (m x d).
    X_sample: Sample locations (n x d).
    Y_sample: Sample values (n x 1).
    m: A GP model from GPy fitted to samples.
    xi: Exploitation-exploration trade-off parameter.

    Returns: Expected improvements at points X.
    '''
    (mu, sigma) = gpr.predict(X)
    (mu_sample, sigma_sample) = gpr.predict(X_sample)

    sigma = sigma.reshape(-1, X_sample.shape[1])

    # Needed for noise-based model,
    # otherwise use np.max(Y_sample).
    mu_sample_opt = np.max(mu_sample)

    with np.errstate(divide='warn'):
        imp = mu - mu_sample_opt - xi
        Z = imp / sigma
        ei = imp * norm.cdf(Z) + sigma * norm.pdf(Z)
        ei[sigma == 0.0] = 0.0

    return ei
```

```
# Insert code here
#
#
```

3. Test on bivariate example **(Do this for a plus)**

Use your own BayesOpt implementation, or the GPy one, to find the minimum of the following objective function:

```
def langermann(x):
    """
    Langermann test objective function.
    Args:
    x: Two-dimensional point; format: [[x0, x1]]
    Returns: Function value
    """

    x=np.asarray(x[0]) # for compatibility with GPyOpt

    a = [3,5,2,1,7]
    b = [5,2,1,4,9]
    c = [1,2,5,2,3]

    return -sum(c*np.exp(-(1/np.pi)*((x[0]-a)**2 \
                               + (x[1]-b)**2))*np.cos(np.pi*((x[0]-a)**2 \
                                + (x[1]-b)**2)))
```

Be sure to investigate different choices for the acquisition function and for the covariance function of your statistical model. In particular, be sure to compare the `RBF` and `Matern32` kernels.

```
# Insert code here
#
#
```

4. Multivariate test examples (optional)

In case you have time, try one of the challenging multivariate test functions that are presented in the Appendix of Bayesian optimization in ab initio nuclear physics. arXiv:1902.00941.

```
# Insert code here
#
#
```

Mini-project IIIb: Bayesian Neural Networks

Answer questions and do tasks written in boldface.

Based on the demo-Bayesian_neural_networks_tif285.ipynb notebook by Christian Forssén, which in turn was based on:

Variational Inference: Bayesian Neural Networks

© 2016-2018 by Thomas Wiecki, updated by Maxim Kochurov.

Original blog post: https://twiecki.github.io/blog/2016/06/01/bayesian-deep-learning/ and updated version: https://docs.pymc.io/notebooks/bayesian_neural_network_advi.html.

This mini-project relies on the following additional packages:

- `theano`: Theano is a Python library that allows you to define, optimize, and evaluate mathematical expressions involving multi-dimensional arrays efficiently, featuring tight integration with NumPy, transparent use of a GPU, efficient symbolic differentiation, speed and stability optimizations and dynamic C code generation.
- `pymc3`: PyMC3 is a library for probabilistic programming in python. It allows you to write down models using an intuitive syntax to describe a data generating process.
- `arviz`: ArviZ (pronounced “AR-vees”) is a Python package for exploratory analysis of Bayesian models. Includes functions for posterior analysis, model checking, comparison and diagnostics.

This should work with conda environments using the original environment.yml conda file on Macs and Linux machines or using the environment_windows.yml conda file on Windows 10.

Generating data

First, lets generate some toy data – a simple binary classification problem that's not linearly separable. Here we use `make_moons` from scikit-learn to generate the full set of data (see https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_moons.html) and `train_test_split` to divide it into training and test sets (see https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html).

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set(); sns.set_context("talk")

from warnings import filterwarnings
filterwarnings('ignore')

import sklearn
from sklearn import datasets
from sklearn.preprocessing import scale
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_moons, make_circles
```

```
# Import theano and pymc3

# suppress some theano compilation errors with MacOSX clang compiler
import theano
# CF added (https://stackoverflow.com/questions/51238578/error-non-constant-expression-cannot-be-narrowed-from-type-npy-intp-to-int)
theano.config.gcc.cxxflags = "-Wno-c++11-narrowing"  # Is this is really necessary?
floatX = theano.config.floatX
import pymc3 as pm
import theano.tensor as T
```

```
# Create combined training and test data.  For other examples, see
#  https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_moons.html
X, Y = make_moons(noise=0.2, random_state=2, n_samples=1000)
X = scale(X)
X = X.astype(floatX)
Y = Y.astype(floatX)

# Split into training and test data. test_size is proportion in test data.
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=.5,
                                                    random_state=0)
```

To do: add code here to plot the training and test data in the empty two plots below.

```
fig, ax = plt.subplots(nrows=1, ncols=3, figsize=(18,6))

ax[0].scatter(X[Y==0, 0], X[Y==0, 1], label='Class 0')
ax[0].scatter(X[Y==1, 0], X[Y==1, 1], color='r', label='Class 1')
sns.despine(); ax[0].legend()
ax[0].set(xlabel='X1', ylabel='X2', title='Toy binary classification data set')

ax[1].set(xlabel='X1', ylabel='X2', title='Training data set')

ax[2].set(xlabel='X1', ylabel='X2', title='Test data set')


fig.tight_layout()
```

Model specification

A neural network is quite simple. The basic unit is a perceptron, which is nothing more than logistic regression. We use many of these in parallel and then stack them up to get hidden layers. Here we start with 2 hidden layers with 5 neurons each which is sufficient for such a simple problem.

```
def construct_nn(ann_input, ann_output):
    n_hidden = 5   # this is the number of neurons, not hidden layers

    # Initialize random weights between each layer
    init_1 = np.random.randn(X.shape[1], n_hidden).astype(floatX)
    init_2 = np.random.randn(n_hidden, n_hidden).astype(floatX)
    init_out = np.random.randn(n_hidden).astype(floatX)

    with pm.Model() as neural_network:
        # Trick: Turn inputs and outputs into shared variables using the data
        # container pm.Data.  It's still the same thing, but we can later
        # change the values of the shared variable (to switch in the test-data
        # later) and pymc3 will just use the new data. Like a pointer we can redirect.
        # For more info, see:
http://deeplearning.net/software/theano/library/compile/shared.html
        ann_input = pm.Data('ann_input', X_train)
        ann_output = pm.Data('ann_output', Y_train)

        # Weights from input to 1st hidden layer
        weights_in_1 = pm.Normal('w_in_1', 0, sigma=1,
                                 shape=(X.shape[1], n_hidden),
                                 testval=init_1)

        # Weights from 1st to 2nd layer
        weights_1_2 = pm.Normal('w_1_2', 0, sigma=1,
                                shape=(n_hidden, n_hidden),
                                testval=init_2)

        # Weights from 2nd hidden layer to output
        weights_2_out = pm.Normal('w_2_out', 0, sigma=1,
                                  shape=(n_hidden,),
                                  testval=init_out)

        # Build neural-network using tanh activation function
        act_1 = pm.math.tanh( pm.math.dot(ann_input, weights_in_1) )
        act_2 = pm.math.tanh( pm.math.dot(act_1, weights_1_2) )
        act_out = pm.math.sigmoid( pm.math.dot(act_2, weights_2_out) )

        # Binary classification -> Bernoulli likelihood
        out = pm.Bernoulli('out',
                           act_out,
                           observed=ann_output,
                           total_size=Y_train.shape[0] # IMPORTANT for minibatches
                          )
    return neural_network

neural_network = construct_nn(X_train, Y_train)
```

The `Normal` priors help regularize the weights. Usually we would add a constant `b` to the inputs (come back to do this later).

Variational Inference: Scaling model complexity

To find a posterior for the weights, we will use ADVI variational inference algorithm in PyMC3. This is much faster than an MCMC sampler and will scale better.

Note, however, that this is a mean-field approximation so we **ignore correlations** in the posterior.

```
from pymc3.theanof import set_tt_rng
np.random.seed(42)
set_tt_rng(42)
```

```
%%time

# Run the ADVI inference for n iterations.
with neural_network:
    inference = pm.ADVI()
    approx = pm.fit(n=30000, method=inference)
```

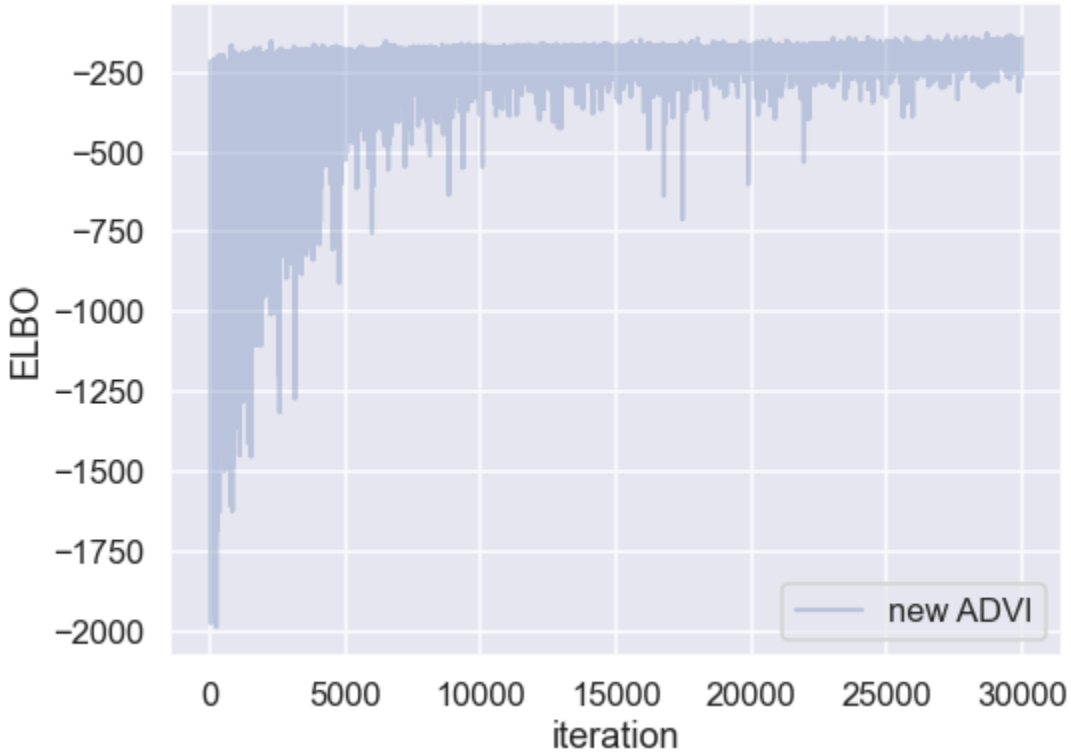
100.00% [30000/30000 00:08<00:00 Average Loss = 180.98]

```
Finished [100%]: Average Loss = 180.95
```

```
CPU times: user 33.9 s, sys: 1.11 s, total: 35.1 s
Wall time: 15.1 s
```

Plotting the objective function (ELBO) shows us how the optimization of the fit improves over time.

```
fig, ax = plt.subplots(figsize=(8,6))

ax.plot(-inference.hist, label='new ADVI', alpha=.3)
ax.legend()
ax.set_ylabel('ELBO')
ax.set_xlabel('iteration')
plt.tight_layout;
```



Now that we have trained our model, lets predict on the hold-out (i.e., test) set using a posterior predictive check (PPC).

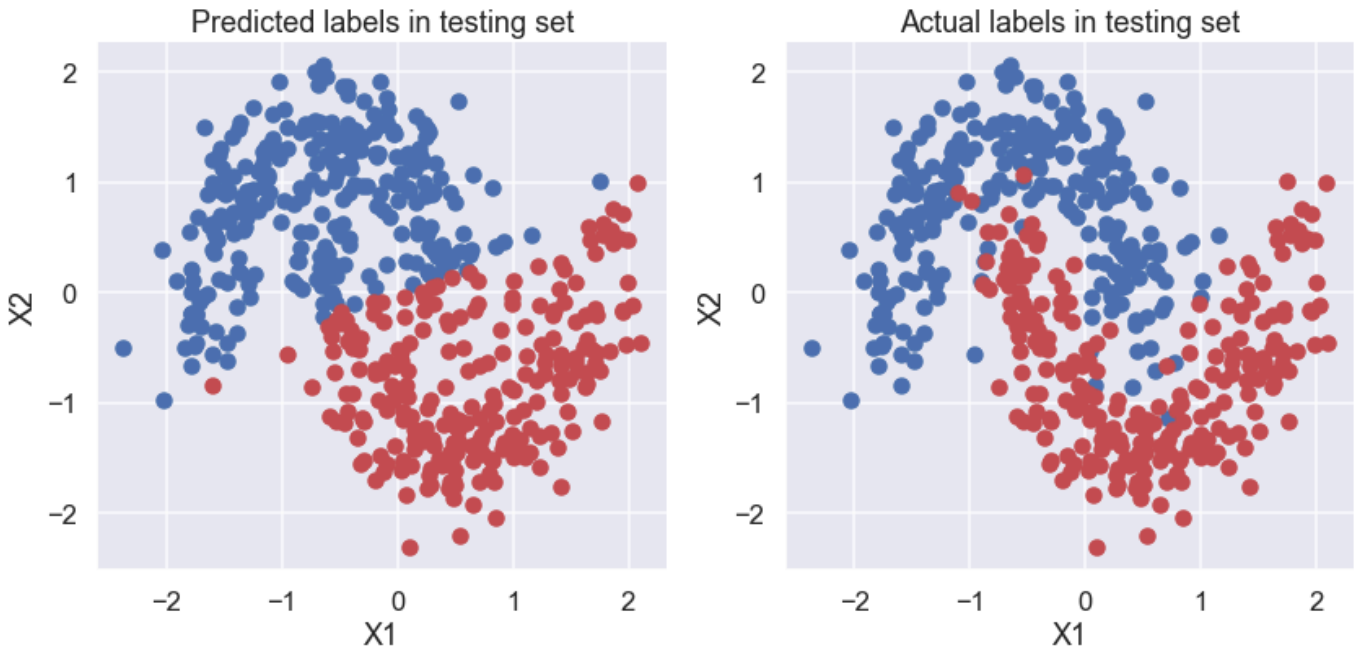
1. We can use `sample_posterior_predictive() <../api/inference.rst>`__ to generate new data (in this case class predictions) from the posterior (sampled from the variational estimation).
2. It is better to get the node directly and build theano graph using our approximation (approx.sample_node) , we get a lot of speed up. We'll use this method below.

```
# We can get predicted probability from model
neural_network.out.distribution.p
```

```
sigmoid.0
```

```
# create symbolic input in theano
x = T.matrix('X')
# symbolic number of samples is supported, we build vectorized posterior on the fly
n = T.iscalar('n')
# Do not forget test_values or set theano.config.compute_test_value = 'off'
x.tag.test_value = np.empty_like(X_train[:10])
n.tag.test_value = 100
_sample_proba = approx.sample_node(neural_network.out.distribution.p,
                                   size=n,
                                   more_replacements={neural_network['ann_input']: x})
# It is time to compile the function
# No updates are needed for Approximation random generator
# Efficient vectorized form of sampling is used
sample_proba = theano.function([x, n], _sample_proba)

# Let's go ahead and generate predictions (using method 2 from above):
pred = sample_proba(X_test, 500).mean(0) > 0.5
```

```
# Make plots of predicted and actual labels
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(12,6))
ax[0].scatter(X_test[pred==0, 0], X_test[pred==0, 1])
ax[0].scatter(X_test[pred==1, 0], X_test[pred==1, 1], color='r')
sns.despine()
ax[0].set(title='Predicted labels in testing set', xlabel='X1', ylabel='X2');

ax[1].scatter(X_test[Y_test==0, 0], X_test[Y_test==0, 1])
ax[1].scatter(X_test[Y_test==1, 0], X_test[Y_test==1, 1], color='r')
sns.despine()
ax[1].set(title='Actual labels in testing set', xlabel='X1', ylabel='X2');


fig.tight_layout()
```




```
# accuracy of predictions
print(f'Accuracy = {(Y_test == pred).mean() * 100}')
```

```
Accuracy = 87.8%
```

Let’s look at what the classifier has learned

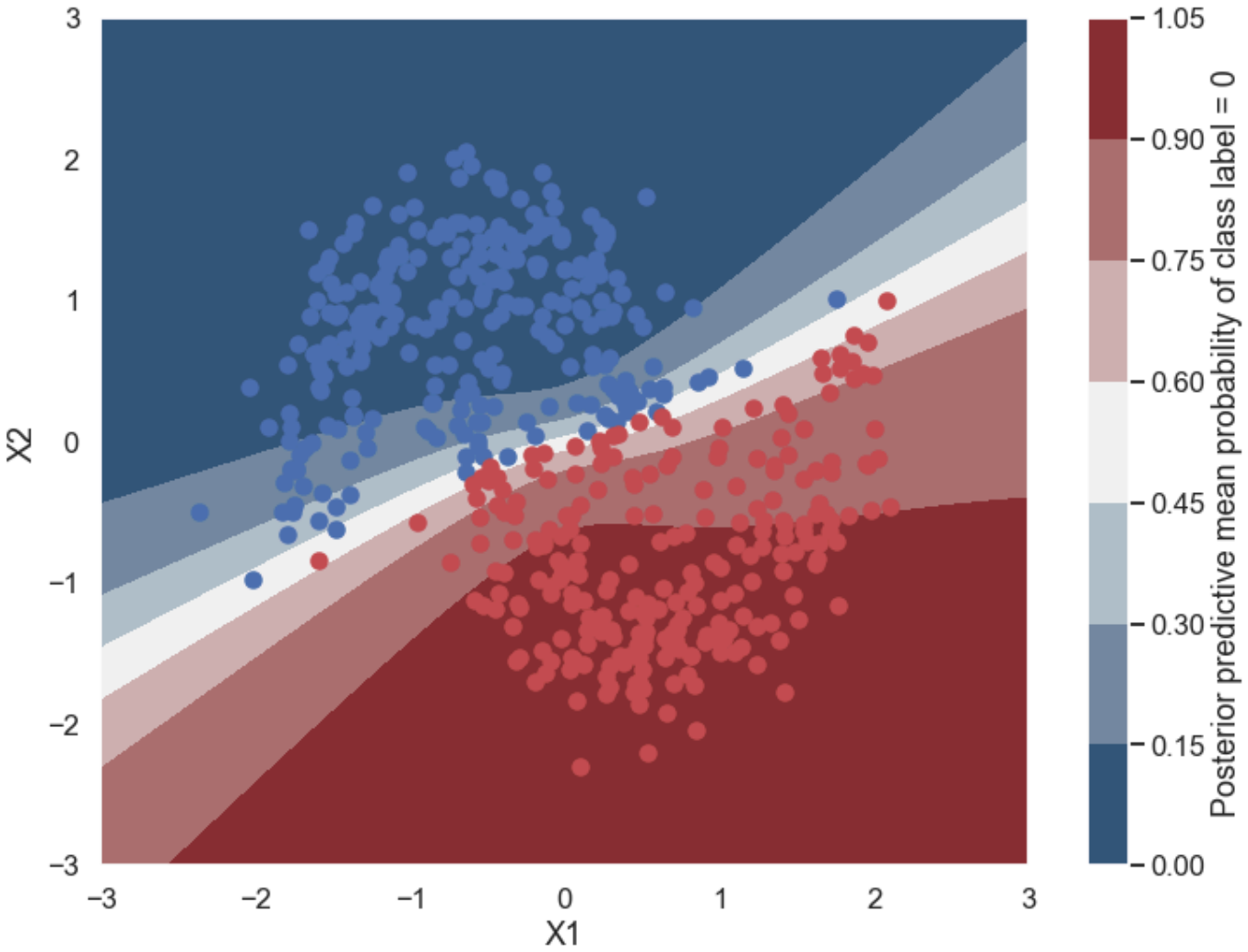
For this, we evaluate the class probability predictions on a grid over the whole input space.

```
grid = pm.floatX(np.mgrid[-3:3:100j,-3:3:100j])
grid_2d = grid.reshape(2, -1).T
dummy_out = np.ones(grid.shape[1], dtype=np.int8)
```

```
ppc = sample_proba(grid_2d ,500)
```

Probability surface

```
cmap = sns.diverging_palette(250, 12, s=85, l=25, as_cmap=True)
fig, ax = plt.subplots(figsize=(12, 9))
contour = ax.contourf(grid[0], grid[1], ppc.mean(axis=0).reshape(100, 100), cmap=cmap)
ax.scatter(X_test[pred==0, 0], X_test[pred==0, 1])
ax.scatter(X_test[pred==1, 0], X_test[pred==1, 1], color='r')
cbar = plt.colorbar(contour, ax=ax)
_ = ax.set(xlim=(-3, 3), ylim=(-3, 3), xlabel='X1', ylabel='X2');
cbar.ax.set_ylabel('Posterior predictive mean probability of class label = 0');
```

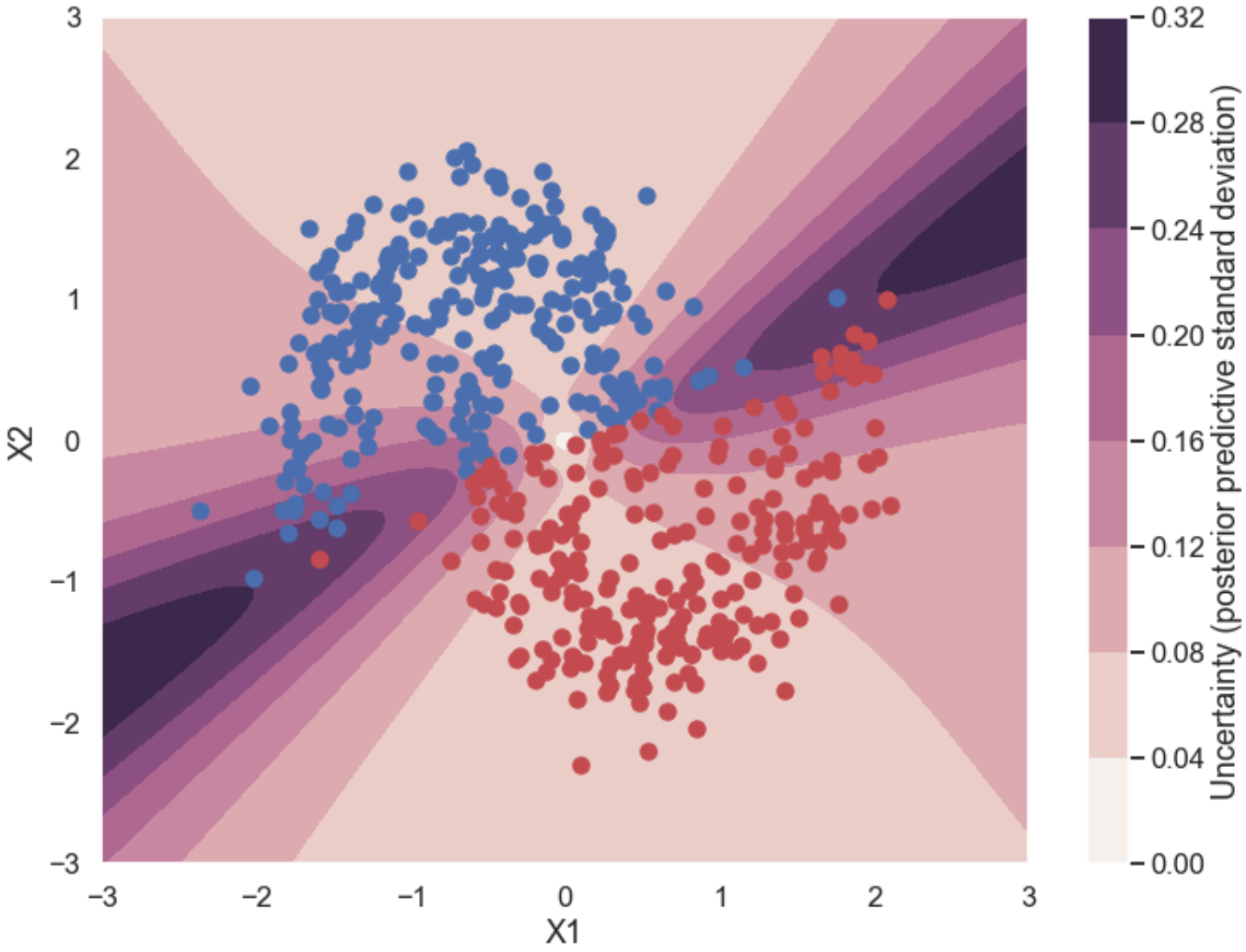


What is being plotted in this graph? How does it compare to what you would get from a non-Bayesian neural network?

Uncertainty in predicted value

What is being plotted below? Why is this different from a non-Bayesian neural network?

```
cmap = sns.cubehelix_palette(light=1, as_cmap=True)
fig, ax = plt.subplots(figsize=(12, 9))
contour = ax.contourf(grid[0], grid[1], ppc.std(axis=0).reshape(100, 100), cmap=cmap)
ax.scatter(X_test[pred==0, 0], X_test[pred==0, 1])
ax.scatter(X_test[pred==1, 0], X_test[pred==1, 1], color='r')
cbar = plt.colorbar(contour, ax=ax)
_ = ax.set(xlim=(-3, 3), ylim=(-3, 3), xlabel='X1', ylabel='X2');
cbar.ax.set_ylabel('Uncertainty (posterior predictive standard deviation)');
```



To do

For each of the parameter choices listed below, do the following:

a. Predict from the training and test data plots how well the Bayesian neural network should do (e.g., compared to the other parameter choices).

b. Record the Average Loss. Does the optimization appear to be converged?

c. Record the accuracy. Is the value consistent with your expectations from a.?

d. What does the "uncertainty in predicted value" figure tell you about where the uncertainty in picking a label (red or blue) is greatest for this case? Consider both regions where there is a lot of training data and where there is no training data. Why does this make sense given your expectations from a.?

Parameter choices

1. `n_samples = 1000`, `noise = 0.2`, `test_size = 0.5`, iterations `n = 30000`

```
a.

b.

c.

d.
```

2. `n_samples = 1000`, `noise = 0.2`, `test_size = 0.5`, iterations `n = 60000`

```
a.

b.

c.

d.
```

3. `n_samples = 1000`, `noise = 0.05`, `test_size = 0.5`, iterations `n = 60000`

```
a.

b.

c.

d.
```

4. `n_samples = 1000`, `noise = 0.5`, `test_size = 0.5`, iterations `n = 60000`

```
a.

b.

c.

d.
```

5. `n_samples = 100`, `noise = 0.2`, `test_size = 0.5`, iterations `n = 60000`

```
a.

b.

c.

d.
```

Copy below the code for the `construct_nn` function that adds a third hidden layer to the neural network. Test it on one of the parameter choices above, and describe how the results b., c., and d. compare for two and three hidden layers.

```
### Copy code for construct_nn function with 3rd hidden layer
```

For a plus: copy below the modified code for the `construct_nn` function that adds a bias to each set of weights. Test it on one of the parameter choices above, and describe how the results b., c., and d. compare with and without the biases. **

```
### Copy code for construct_nn function with biases
```